

Fusion Compiler Application Options and Attributes

Version X-2025.06-SP3, March 2026



Contents

1. Fusion Compiler Application Options	87
3	87
3dic.ai_flow.use_exhaustive_search	87
3dic.check_esd.main_runset	88
3dic.check_esd.run_dir	88
3dic.check_esd.runset	89
3dic.check_esd.spice_library	90
3dic.check_esd.user_defined_options	90
3dic.common.die_heights	91
3dic.common.redhawk_sc_et_path	92
3dic.emir.database	93
3dic.emir.engine	93
3dic.rail.expanded_pg_pattern	94
3dic.route.bump_order_report_max_count	95
3dic.route.detour_mode	96
3dic.route.enable_even_distribution_with_merge	97
3dic.route.existing_shape_report_max_count	98
3dic.route.merge_subchannels	98
3dic.route.route_90degree_only	99
3dic.route.stub_alignment_type	100
3dic.route.via_count_report_max_net_count	100
3dic.route.via_track_location	101
3dic.route.via_track_pitch	102
3dic.route.wirelength_variation_report_max_net_count	102
3dic.thermal.air_speed	103
3dic.thermal.ball_material	103
3dic.thermal.boundary_type	104
3dic.thermal.bump_via_data_model	104
3dic.thermal.c4_material	105
3dic.thermal.engine	106
3dic.thermal.htc	106
3dic.thermal.htc_components	107
3dic.thermal.iteration	108

Contents

3dic.thermal.package_config_file	108
3dic.thermal.package_file	109
3dic.thermal.pcb_config_file	110
3dic.thermal.pcb_file	110
3dic.thermal.probe_num_cols	111
3dic.thermal.probe_num_rows	112
3dic.thermal.ubump_material	112
a	113
abstract.allow_all_level_abstract	113
abstract.allow_fault_in	114
abstract.annotate_power	114
abstract.check_constraints_consistency	115
abstract.enable_improvements_for_budgeting	117
abstract.enable_signal_em_analysis	117
abstract.high_fanout_limit	118
abstract.include_aggressor_nets	119
abstract.keep_constrained_objects	119
abstract.keep_min_pulse_width_violation_pins	120
abstract.latch_levels	120
abstract.min_pulse_width_violation_slack_limit	121
c	121
ccd.adjust_io_clock_latency	121
ccd.ccd_opt_voltage_drop_enable_ml_predictor	122
ccd.dps.analyze_high_power_instances	122
ccd.dps.analyze_high_power_instances_autodetect	123
ccd.dps.analyze_high_power_instances_autodetect_threshold	124
ccd.dps.auto_detect_cts_options	124
ccd.dps.auto_target_constraint_parameters	125
ccd.dps.cspartition_latency_range	129
ccd.dps.effective_r_aware	129
ccd.dps.flow_break_on_dps_opt_failure	130
ccd.dps.flow_reporting	130
ccd.dps.focus_power_scenarios	131
ccd.dps.focus_supply_nets	132
ccd.dps.hotspot_aware	132
ccd.dps.optimize_auto_targets	133
ccd.dps.optimize_hold_tradeoff_level	136
ccd.dps.optimize_power_target_mode	137

Contents

ccd.dps.optimize_power_use_focus_supply_nets	137
ccd.dps.optimize_setup_tradeoff_level	138
ccd.dps.partition_auto_max	138
ccd.dps.partition_auto_min	139
ccd.dps.partition_auto_sinks	139
ccd.dps.partition_base_count	140
ccd.dps.plot_power_density_grid	141
ccd.dps.powershape_clocks	141
ccd.dps.powershape_exclude_clocks	142
ccd.dps.powershape_high_power_instances	143
ccd.dps.powershape_high_power_instances_autodetect_threshold	143
ccd.dps.powershape_separate_instances	144
ccd.dps.stepped_psg_window_size	144
ccd.dps.use_case_type	145
ccd.dps.use_cases	146
ccd.enable_clock_drc_improvements	149
ccd.enable_multi_vector_ccd_tns_register_sizing	149
ccd.enable_register_sizing	150
ccd.enable_top_wns_optimization	150
ccd.fmax_optimization_effort	151
ccd.hold_control_effort	151
ccd.ignore_ports_for_boundary_identification	152
ccd.ignore_scan_reset_for_boundary_identification	153
ccd.legal_aware_buffering	153
ccd.macro_seed_offset_file	154
ccd.macro_seed_offset_strategy	155
ccd.max_postpone	156
ccd.max_prepone	157
ccd.optimize_boundary_timing	158
ccd.optimize_boundary_timing_upstream	159
ccd.post_route_buffer_removal	160
ccd.reference_corner	160
ccd.respect_cts_fixed_balance_pins	160
ccd.skew_opt_default_balance_point_macros	162
ccd.skip_path_groups	162
ccd.targeted_ccd_end_points_file	163
ccd.targeted_ccd_path_groups	163
ccd.targeted_ccd_threshold_for_nontargeted_path_groups	164
ccd.targeted_ccd_wns_optimization_effort	164

Contents

ccd.timing_effort	165
ccd.voltage_drop_population_threshold	165
ccd.voltage_drop_voltage_threshold	166
chf.create_stdcell_fillers.max_leakage_vt_order_violations	166
chipfinishing.1cpp_channel_trans_plus_filler_lib_cells	167
chipfinishing.balanced_filler_lib_cells	167
chipfinishing.balanced_pad_lib_cells	168
chipfinishing.default_transition_z_width	169
chipfinishing.enable_advanced_legalizer_postfixing	169
chipfinishing.enable_al_tap_insertion	170
chipfinishing.enable_area_optimization	170
chipfinishing.enable_auto_orient_base_on_tap_coverage_table	170
chipfinishing.enable_column_based_tap_insertion	171
chipfinishing.enable_disjointed_site_rows	171
chipfinishing.enable_even_uniform_row_pattern	172
chipfinishing.inbound_complete_fill_gap_cpp	172
chipfinishing.inbound_filler_lib_cells	173
chipfinishing.inbound_max_lib_cells	173
chipfinishing.inbound_plus_filler_lib_cells	174
chipfinishing.tap_cover_power_domain_lib_cells	174
chipfinishing.trans_balanced_half_height_lib_cells	176
chipfinishing.trans_balanced_lib_cells	177
chipfinishing.trans_balanced_to_max_lib_cells	177
chipfinishing.trans_balanced_to_plus_lib_cells	178
chipfinishing.trans_max_filler_lib_cells	178
chipfinishing.trans_plus_filler_lib_cells	179
chipfinishing.vt_od_rule_setting_check_gives_warning_only	180
clock_opt.final_opto.final_clock_routing	180
clock_opt.flow.enable_ccd	180
clock_opt.flow.enable_clock_power_recovery	181
clock_opt.flow.enable_global_route_opt	182
clock_opt.flow.enable_irap	183
clock_opt.flow.enable_irdrivenopt	184
clock_opt.flow.enable_multibit_debanking	184
clock_opt.flow.enable_multibit_rewiring	185
clock_opt.flow.enable_power	185
clock_opt.flow.enable_voltage_drop_aware	186
clock_opt.flow.extra_size_only	186
clock_opt.flow.optimize_layers	187

Contents

clock_opt.flow.optimize_layers_critical_range	187
clock_opt.flow.optimize_layers_overlap_bins	188
clock_opt.flow.optimize_ndr	188
clock_opt.flow.optimize_ndr_critical_range	188
clock_opt.flow.optimize_ndr_max_nets	189
clock_opt.flow.repeatability	189
clock_opt.hold.effort	190
clock_opt.place.congestion_effort	191
clock_opt.place.effort	191
compile.auto_floorplan.buffering_aware_timing_driven	191
compile.auto_floorplan.congestion	192
compile.auto_floorplan.fix_macro_violation	193
compile.auto_floorplan.initialize	194
compile.auto_floorplan.keep_bounds	195
compile.auto_floorplan.keep_placement_blockages	195
compile.auto_floorplan.optimize_small_design	196
compile.auto_floorplan.pin_placement_strategy	197
compile.auto_floorplan.place_hard_macros	198
compile.auto_floorplan.place_hard_macros_enhanced	199
compile.auto_floorplan.place_ios	200
compile.auto_floorplan.place_pins	201
compile.auto_floorplan.shape_voltage_areas	202
compile.auto_floorplan.small_design_threshold	203
compile.auto_floorplan.timing_driven	203
compile.check.site_def	204
compile.check.site_def_verbose	204
compile.clockgate.cg_cell_naming_style	205
compile.clockgate.clock_gate_name_prefix	207
compile.clockgate.enable_activity_driven_level_expansion	208
compile.clockgate.enable_dft_wrapper_cell_support	209
compile.clockgate.enable_level_expansion	210
compile.clockgate.fanin_sequential	212
compile.clockgate.fanin_sequential_name_prefix	215
compile.clockgate.inherit_timing_exceptions	215
compile.clockgate.insertion_strategy	217
compile.clockgate.max_level_allow_ungating	217
compile.clockgate.max_number_of_levels	218
compile.clockgate.physically_aware_estimate_timing	219
compile.clockgate.self_gate_name_prefix	220

Contents

compile.clockgate.self_gating	221
compile.clockgate.sequential_delay_register_name_prefix	222
compile.clockgate.substring_list_for_dft_wrapper_dont_rename_icgs	223
compile.clockgate.use_clock_latency	224
compile.clockgate.use_enhanced_clock_gate_naming_style	225
compile.datapath.module_name_prefix	226
compile.datapath.physically_aware	227
compile.datapath.ungroup	227
compile.early_place.effort	228
compile.final_place.effort	228
compile.flow.allow_inverter_on_clock	229
compile.flow.areaResynthesis	229
compile.flow.autoungroup	230
compile.flow.boundary_optimization	230
compile.flow.clock_aware_placement	231
compile.flow.conservative_constant_propagation	232
compile.flow.constant_and_unloaded_propagation_prevention_with_no_boundary_opt	232
compile.flow.constant_and_unloaded_propagation_with_no_boundary_opt	233
compile.flow.enable_ccd	234
compile.flow.enable_ccd_fmax_flow	235
compile.flow.enable_checkpoint_compression	235
compile.flow.enable_complex_register_mapping	236
compile.flow.enable_dpa_ccd_power	236
compile.flow.enable_dpa_ccd_timing	237
compile.flow.enable_dpa_ccd_timing_with_delay_potential	237
compile.flow.enable_dps	237
compile.flow.enable_fast_cus_in_final_opto	238
compile.flow.enable_hierarchical_checkpoints	238
compile.flow.enable_high_effort_constant_propagation	239
compile.flow.enable_incremental_multibit_banking	240
compile.flow.enable_irap	240
compile.flow.enable_multibit	241
compile.flow.enable_multibit_debanking	241
compile.flow.enable_multibit_pm_cells	242
compile.flow.enable_multibit_rewiring	242
compile.flow.enable_multisource_clock_trees	243
compile.flow.enable_mv_merge_clone	244
compile.flow.enable_physical_multibit_banking	245
compile.flow.enable_power	245

Contents

compile.flow.enable_rtl_multibit_banking	246
compile.flow.enable_rtl_multibit_debanking	247
compile.flow.enable_second_pass_multibit_banking	247
compile.flow.enable_toggle_aware_rtl_multibit_reordering	248
compile.flow.enable_unloaded_dt_cell_fanin_cleanup	248
compile.flow.enable_via_ladder_insertion	249
compile.flow.enhanced_delay_opto_for_pg	249
compile.flow.high_effort_area	250
compile.flow.high_effort_timing	250
compile.flow.hold_area_budgeting	251
compile.flow.layer_aware_optimization	251
compile.flow.legalize_unmapped_fixed	252
compile.flow.merge_clock_gates	253
compile.flow.optimize_icgs	253
compile.flow.optimize_icgs_critical_range	254
compile.flow.optimize_icgs_use_timing_driven_splitting	255
compile.flow.optimize_layers	255
compile.flow.optimize_layers_critical_range	256
compile.flow.optimize_layers_overlap_bins	256
compile.flow.optimize_ndr	257
compile.flow.optimize_ndr_critical_range	257
compile.flow.optimize_ndr_max_nets	258
compile.flow.print_messages_for_redundant_registers	259
compile.flow.propagate_constants_through_dont_touch_cells	259
compile.flow.propagate_constants_through_registers	260
compile.flow.propagate_constants_through_size_only_cells	260
compile.flow.skew_macros_effort	261
compile.flow.tie_all_unused_hier_inputs_to_constants	262
compile.flow.tie_unused_hier_inputs_to_constants	262
compile.flow.tieoff_undriven_top_output_ports	263
compile.flow.trial_clock_tree	263
compile.flow.user_instance_name_prefix	264
compile.incr_initial_opto.placement_congestion	265
compile.initial_drc.global_route_based	265
compile.initial_place.buffering_aware	266
compile.initial_place.buffering_aware_placement_effort	267
compile.initial_place.effort	267
compile.log.tbc_report	268
compile.log.timestamp	268

Contents

compile.multibit.prefer_multibit_libcells_with_singlebit_equivalent	269
compile.multibit.support_wrapper_cells	269
compile.optimization.enable_hierarchical_inverter	270
compile.optimization.infer_mux_reencode_supports	271
compile.physical.save_and_restore_ndr_auto_mode	271
compile.physical.save_and_restore_ndr_during_ungroup	272
compile.retiming.optimization_priority	272
compile.retiming.simu_mismatch_prevention	273
compile.seqmap.allow_async_set_reset_priority_inversion_if_unmapped . . .	274
compile.seqmap.allow_output_inversion_if_unmapped	274
compile.seqmap.allow_tieoffs_for_latches	275
compile.seqmap.allow_tieoffs_for_registers	276
compile.seqmap.disable_async_logic_push_out_for_latches	277
compile.seqmap.disable_qn_usage	277
compile.seqmap.dw_sync_support	278
compile.seqmap.enable_output_inversion	279
compile.seqmap.enable_register_merging	279
compile.seqmap.enable_register_merging_across_bus	280
compile.seqmap.exact_map	281
compile.seqmap.gating_async_set_reset	281
compile.seqmap.honor_sync_set_reset	282
compile.seqmap.identify_shift_registers	283
compile.seqmap.ignore_const_values_on_scan_pins_during_constant_propagation	284
compile.seqmap.max_shift_register_length	284
compile.seqmap.min_shift_register_length	285
compile.seqmap.no_inv_push_for_user_size_only	286
compile.seqmap.prefer_registers_with_multibit_equivalent	286
compile.seqmap.preserve_rtl_clock_phase_strictly	287
compile.seqmap.preserve_rtl_clock_phase_strictly_for_retention	288
compile.seqmap.print_cross_probing_info_for_removed_registers	288
compile.seqmap.register_replication_max_fanout_constraint_mode	289
compile.seqmap.register_replication_naming_style	290
compile.seqmap.register_replication_placement_effort	290
compile.seqmap.register_replication_placement_location	291
compile.seqmap.register_replication_simu_mismatch_prevention	292
compile.seqmap.remove_constant_registers	293
compile.seqmap.remove_unloaded_registers	294
compile.seqmap.replicate_registers_without_asynchronous_logic	294
compile.seqmap.replicate_size_only_registers	295

Contents

compile.seqmap.scan	296
compile.seqmap.swap_ignoring_user_size_only	296
constraints_compare_full_behavior	297
create_trunk.drc.enable	298
cstrmgr.allow_gobble_for_dont_touch_network	298
cts.balance_groups.balance_by_source_latency	299
cts.balance_groups.honor_source_latency	300
cts.buffering.enable_arnoldi_analysis	300
cts.buffering.levelize_buffering	301
cts.buffering.reduce_clock_level_effort	301
cts.common.advanced_lp_solvers	302
cts.common.auto_exception_disable_self_arc	303
cts.common.auto_exception_macro_balance_point	303
cts.common.auto_exception_output_ports_balance_point	304
cts.common.default_max_transition	304
cts.common.derive_icg_and_register_references	305
cts.common.directory_for_cts_generated_files	306
cts.common.enable_auto_exceptions	306
cts.common.enable_auto_skew_target_for_local_skew	307
cts.common.enable_dirty_design_mode	308
cts.common.enable_low_power	308
cts.common.ignore_drc_only_scenario	309
cts.common.max_fanout	310
cts.common.max_net_length	310
cts.common.port_punch_prefix_original_port_name	311
cts.common.power_aware_pruning	311
cts.common.preserve_bus_ports	312
cts.common.preserve_hierarchical_ports	312
cts.common.skip_cts_clocks	313
cts.common.unlimit_delay_insertion_stages	313
cts.common.user_instance_name_prefix	314
cts.common.verbose	314
cts.compile.align_clustering_and_buffering	315
cts.compile.balanced_topology_enhancements	315
cts.compile.cg_timing_aware	316
cts.compile.enable_cell_relocation	316
cts.compile.enable_global_route	317
cts.compile.enable_local_skew	317
cts.compile.fix_clock_tree_sinks	318

Contents

cts.compile.global_route_aware_buffering	319
cts.compile.latency_enhancements	319
cts.compile.path_based_criticality	320
cts.compile.power_opt_mode	320
cts.compile.power_opt_wirelength_effort	321
cts.compile.primary_corner	321
cts.compile.remove_existing_clock_trees	322
cts.compile.report_latency_bottleneck	322
cts.compile.report_latency_bottleneck_threshold	323
cts.compile.size_pre_existing_cell_to_cts_references	324
cts.compile.timing_driven_sink_transition	324
cts.compile.topological_ndr	325
cts.flow.enable_unified_latency_estimation	325
cts.icg.equivalence_stop_at_user_size_only	326
cts.icg.latency_driven_cloning	327
cts.icg.latency_driven_collapsing	327
cts.icg.merge_cross_level	328
cts.icg.timing_aware	328
cts.multisource.cell_em_aware	329
cts.multisource.cto_toggle_rate_fanout_threshold	329
cts.multisource.cto_toggle_rate_scale	330
cts.multisource.enable_activity_aware_relocation	331
cts.multisource.enable_clock_driver_snapping	331
cts.multisource.enable_cto_honor_toggle_rate	332
cts.multisource.enable_full_mv_support	332
cts.multisource.enable_global_stem_path_skew_balancing	333
cts.multisource.enable_levelized_buffering	334
cts.multisource.enable_mlpf_flow	334
cts.multisource.enable_multi_corner_support	335
cts.multisource.enable_mv_aware_tap_assignment	336
cts.multisource.enable_pin_accessibility_for_global_clock_trees	336
cts.multisource.enable_subtree_synthesis_aware_ccd	337
cts.multisource.enable_via_ladder_insertion_on_tap_output_pins	338
cts.multisource.flexible_htrie_synthesis	338
cts.multisource.global_tree_improvements	338
cts.multisource.htree_cell_keepout_factor	339
cts.multisource.htree_explore_all_repeater_solutions	340
cts.multisource.htree_latency_only	340
cts.multisource.htree_legalize_design	341

Contents

cts.multisource.htree_pin_connection_effort_level	341
cts.multisource.htree_single_repeater_at_center	341
cts.multisource.htree_single_repeater_at_node	342
cts.multisource.ignore_drc_on_subtree_driver	342
cts.multisource.integrated_subtree_synthesis_early_ccd_support	344
cts.multisource.max_net_length_only_htree_synthesis	345
cts.multisource.power_aware_flexible_htree_synthesis	345
cts.multisource.subtree_merge_cell_name_prefix	346
cts.multisource.subtree_merge_cell_name_suffix	347
cts.multisource.subtree_merge_concatenate_length_threshold	348
cts.multisource.subtree_routing_mode	349
cts.multisource.subtree_split_cell_name_prefix	350
cts.multisource.subtree_split_cell_name_suffix	351
cts.multisource.subtree_split_net_name_prefix	352
cts.multisource.subtree_split_net_name_suffix	352
cts.multisource.subtree_synthesis_enable_power_optimization	353
cts.multisource.subtree_synthesis_timing_aware_flow	354
cts.multisource.tap_assignment_allow_cascaded_taps	355
cts.multisource.tap_assignment_improvements	356
cts.multisource.tap_assignment_print_debug_report	356
cts.multisource.tap_assignment_reassign_ignore_sinks	357
cts.multisource.tap_selection	357
cts.multisource.tap_synthesis_route_based_estimation	358
cts.multisource.verbose	359
cts.optimize.delay_insertion_enhancements	360
cts.optimize.enable_area_recovery_in_local_skew	360
cts.optimize.enable_balance_point_optimization	360
cts.optimize.enable_congestion_aware_ndr_promotion	361
cts.optimize.enable_cto_power_optimization	362
cts.optimize.enable_improvement_mode	362
cts.optimize.enable_layer_optimization_for_power	363
cts.optimize.enable_local_skew	363
cts.optimize.enable_nworst_skew_optimization	364
cts.optimize.enable_solver_augmented_skew_optimization	365
cts.optimize.improvement_mode_version	365
cts.optimize.multi_corner_effort	366
cts.optimize.print_qor_summary_table	366
cts.placement.cell_spacing_rule_style	367
cts.report.transition_histogram_datapoints	367

Contents

cts.report.verbose_paths_routing_length_threshold	368
cts.routing.fishbone_bias_threshold	369
cts.routing.fishbone_bias_window	370
cts.routing.fishbone_horizontal_bias_spacing	371
cts.routing.fishbone_max_sub_tie_distance	372
cts.routing.fishbone_max_tie_distance	373
cts.routing.fishbone_vertical_bias_spacing	374
cts.routing.global_route_topology	375
custom.indesign.custom_compiler_cache	375
custom.indesign.custom_compiler_path	376
custom.indesign.opto_compiler_cache	377
custom.indesign.opto_compiler_path	377
custom.route.balance_mode	378
custom.route.buffer_distance	378
custom.route.buffer_insertion_aware_routing	379
custom.route.bus_corner_type	380
custom.route.bus_intra_shield_placement	380
custom.route.bus_pin_trunk_offset	381
custom.route.bus_split_even_bits	382
custom.route.bus_split_ignore_width	382
custom.route.bus_tap_off_enable	383
custom.route.bus_tap_off_shielding	383
custom.route.diffpair_twist_jumper_enable	384
custom.route.diffpair_twist_jumper_interval	384
custom.route.diffpair_twist_jumper_offset	385
custom.route.diffpair_twist_jumper_style	385
custom.route.distance_to_net_pin	386
custom.route.export_path_segment	386
custom.route.honor_shield_rule_for_shield	387
custom.route.hybrid_flow	387
custom.route.ignore_shielding_gap_on_via	388
custom.route.layer_grid_mode	388
custom.route.match_box	389
custom.route.match_rc_routing_style	390
custom.route.net_max_layer_mode	390
custom.route.net_max_layer_mode_soft_cost	391
custom.route.net_min_layer_mode	392
custom.route.net_min_layer_mode_soft_cost	392
custom.route.routing_area	393

Contents

custom.route.shield_connect_mesh_overlap	394
custom.route.shield_connect_route	394
custom.route.shield_connect_to_supply	395
custom.route.shield_min_open_loop	395
custom.route.shield_min_shield_seg	396
custom.route.shield_min_signal_seg	396
custom.route.shield_net	397
custom.route.shield_second_net	397
custom.route.shield_stripe_fat_wire	398
custom.route.single_loop_match	399
custom.route.single_loop_match_max_spacing	400
custom.route.single_loop_match_min_spacing	400
custom.route.single_loop_match_offset_layer	401
custom.route.skip_connect_pin_type	401
d	402
da.check_netlist.allow_connection_class_violation	402
da.check_netlist.allow_multiply_driven_nets_by_inputs_and_outputs	403
da.check_netlist.check_for_wire_loop	403
da.check_netlist.report_all_shorted_power_ground_nets	404
da.report_design.enable_front_end_sequential_cell_counting	405
design.allow_inst_specific_user_attributes	406
design.allow_multiple_scale_factors	406
design.allow_multiple_subset_names	407
design.auto_spare_cell	407
design.bump_region_pseudo_bump_count_limit	408
design.bus_delimiters	409
design.bus_types_expanded_in_query	409
design.change_names.dc_compatible_flatten_multidimensional_bus	411
design.change_names.dc_compatible_negative_index_shift	411
design.change_names.dc_consistent	412
design.change_names.include_cell_bus	413
design.change_names.shift_negative_index	413
design.compact_pg	414
design.create_in_logical_only_mode	414
design.disable_auto_link	415
design.eco_freeze_silicon_mode	416
design.eco_restrictions	416
design.edit_read_only_libs	417

Contents

design.enable_attribute_support	418
design.enable_cmd_result_cache	418
design.enable_hierarchy_edit_restriction	419
design.enable_icc_region_query	420
design.enable_lib_cell_editing	421
design.enable_non_capturing_regexp	422
design.enable_recovery_save	422
design.enable_rule_based_query	423
design.enable_ts_block_management	423
design.enable_via_ladder_protection	424
design.enforce_terminal_name_uniqueness	424
design.high_fanout_net_threshold	425
design.is_3dic_mode	426
design.keep_terminals_check_duplicates	426
design.memory_optimization	426
design.morph_on_save_as	427
design.name_expansion_delimiters	428
design.on_disk_operation	428
design.preserve_reference_editability	429
design.purpose_number_aware_check_duplicates	430
design.recovery_save_dir_path	430
design.remove_net_shapes	431
design.report_editability_source	431
design.report_variable_row_utilization	432
design.save_session_ref_libs	433
design.session_scale_factor	433
design.set_group_port_name	434
design.set_reparent_port_name	435
design.shape_use_aware_check_duplicates	436
design.skip_read_only	437
design.skip_uneditable_subblocks	437
design.sub_blocks_for_read	438
design.ts_disable_compile_command_restriction	438
design.uniquify_naming_style	439
design.uniquify_skip_empty_modules	440
design.verbose_io	440
design.verbose_uniquify	441
dft.bit_blast	442
dft.dft_drc_use_master_cell_name_for_lib_cell	444

Contents

dft.dft_seq_sido_port_optimize	445
dft.disable_toggle_dw_cell	445
dft.enable_bit_blast_in_spf	446
dft.enable_distributed_codec	446
dft.enable_new_spec_persistency	447
dft.insertion_post_conditioning	447
dft.insertion_post_logic_opto	447
dft.pulse_all_clocks	448
dft.put_negedge_head_pipeline_flops	449
dft.scan.adaptive_pse_reclustering	449
dft.scan.add_lockup_between_equivalent_clocks	450
dft.scan.amount_of_begin_repeater	451
dft.scan.amount_of_end_repeater	451
dft.scan.arch_sort_modes_for_architecting	452
dft.scan.avoid_stitching_scan_cells_with_drc_from_scan_groups	453
dft.scan.cg_trace_te_repeater	453
dft.scan.check_ctl_internal_clock_for_lockups	454
dft.scan.connect_clock_gate_test_enables	454
dft.scan.enable_general_multimode_support	455
dft.scan.enhanced_scandef_start_stop_selection	457
dft.scan.enhanced_scandef_start_stop_selection_max_runs	457
dft.scan.force_buffers_at_logical_chain_endpoints	458
dft.scan.occ_spc_chain_arch_flat_flow	459
dft.scan.occ_spc_chain_arch_flow	459
dft.scan.occ_streaming_end_of_longest_chain	460
dft.scan.place_mm_mux_at_load	460
dft.scan.prefer_preserve_shift_registers	461
dft.scan.pse_cluster_box_x_size	461
dft.scan.pse_cluster_box_y_size	463
dft.scan.repeater_begin_prefix	464
dft.scan.repeater_end_prefix	464
dft.scan.scan_chain_report	465
dft.scan.scandef_analyze_scangroups	466
dft.scan.scandef_based_opto	466
dft.scan.scandef_based_repartition	467
dft.scan.scandef_marker_cell_for_multiloops	467
dft.scan.scandef_stop_before_lockups	467
dft.scan.scandef_use_marker_cells	468
dft.scan.set_false_path_on_lockups	468

Contents

dft.scan.skip_scandef_segment_at_logical_chain_endpoints	469
dft.scan.test_keep_connected_scan_en	469
dft.scan.update_preexisting_icg_se_pin_connections	470
dft.scan.use_core_clock_for_resync	471
dft.scan.use_resync_clock_for_repeater	471
dft.scan.use_root_clock_for_ssg_lockup	472
dft.shell_rtl_edit	472
dft.sms_allow_sub_server_shadow_server_architecture	473
dft.sms_allow_sub_server_sub_server_architecture	473
dft.sms_disable_memory_grouping_checks	474
dft.sms_dwsms_buf_mux_inst_pattern	474
dft.sms_enable_memory_scan_enable_connection_through_sms_wrapper . .	474
dft.sms_new_directory_structure	475
dft.sms_skip_release_compatibility_checks	475
dft.test_allow_separate_mixing_in_wrapper_chains	476
dft.test_andgate_p_ref_for_dft	476
dft.test_bsd_default_bidir_delay	477
dft.test_bsd_default_delay	478
dft.test_bsd_default_strobe	478
dft.test_bsd_default_strobe_width	479
dft.test_bsd_enable_wrp_of	479
dft.test_bsd_make_private_instructions_public	480
dft.test_core_wrap_add_ded_wrp_in_new_module	481
dft.test_core_wrap_analysis_via_clk_pin	481
dft.test_core_wrap_check_cgs_for_ioregs	482
dft.test_core_wrap_check_non_scan_for_wrapper_analysis	482
dft.test_core_wrap_dft_signal_per_partition	483
dft.test_core_wrap_exclude_soft_core_pin_wrap	484
dft.test_core_wrap_map_hold_mux_equiv_gates	484
dft.test_core_wrap_report_file	485
dft.test_core_wrap_retain_dw_mux_structure	485
dft.test_core_wrap_sync_ctl_segment_length	486
dft.test_core_wrp_upf_dw_mv_report	487
dft.test_ctl_internal_block_with_associated_port_internal_clk_pin	488
dft.test_default_delay	488
dft.test_default_period	489
dft.test_default_strobe	490
dft.test_dft_log_file	490
dft.test_disable_toggle_for_reset_shared_wrp	491

Contents

dft.test_drc_integrity_checks	491
dft.test_icg_n_ref_for_dft	492
dft.test_icg_p_ref_for_dft	492
dft.test_power_aware_dw_insertion_near_multi_input_cells	493
dft.test_power_aware_dw_insertion_near_port	494
dft.test_set_test_point_report_file_name	494
dft.test_soc_core_wrap_allow_multibit_ioregs	495
dft.test_soc_low_power_extest_support	495
dft.test_ultra_put_load_ser_lul	496
dft.test_update_spf_for_non_dft_port	496
dft.test_write_one_hot_decoding_in_constraints	497
dft.test_wrp_add_ded_cg_branch	498
dft.test_wrp_add_ded_ctl_branch	498
dft.test_wrp_dw_location_connected_net_check	499
dft.test_wrp_gate_cells_connect_te	499
dft.test_wrp_gate_scan_cell_only	500
dft.test_wrp_ignore_feedthrough_path	500
dft.test_wrp_new_hold_mux_arch_support	501
dft.test_wrp_new_power_domain_aware_dw_insertion_flow	501
dft.test_wrp_power_domain_aware_dw_wrp_clk	502
dft.test_wrp_remove_violated_wrp_cell_restr	502
dft.test_wrp_remove_wrapper_control_decoding_logic	503
dft.test_wrp_shell_netlist_generation_only_flow	503
dft.test_wrp_soft_core_abutted_flow	504
dft.tp_allow_adv_clock_for_tpe	504
dft.tp_allow_dw_collapsing	505
dft.tp_allow_logical_driver	505
dft.tp_analyze_false_path	506
dft.tp_apply_scan_wrap_for_memory_without_test_model	507
dft.tp_block_mcp_fp_at_source	507
dft.tp_ctl_aware_analysis	508
dft.tp_do_not_generate_force_no_scan_constraint	508
dft.tp_enable_aux_models	509
dft.tp_enable_clock_gating	510
dft.tp_enable_clock_tapping_points	510
dft.tp_enable_mv_checks	511
dft.tp_enable_partition_search	513
dft.tp_enable_partition_specifications	513
dft.tp_enable_physiically_aware_sharing	514

Contents

dft.tp_enable_power_aware_clock_selection	515
dft.tp_enable_saving_power_for_observe_tp	515
dft.tp_enable_simulation_libraries	516
dft.tp_improve_clustering_to_reduce_outliers	516
dft.tp_instance_name_prefix	517
dft.tp_move_to_upper_hier_location	518
dft.tp_no_scan_wrap_constr_gen_for_adv	518
dft.tp_no_udtp_constraint_generation_for_advisor	519
dft.tp_preserve_tp_gate	520
dft.tp_set_analysis_directory	520
dft.tp_set_cluster_effort	521
dft.tp_set_dtc_name	522
dft.tp_set_report_file_name	522
dft.tp_show_report_in_log	523
dft.tp_support_cell_based_dft_partition_for_udtp	524
dft.tp_treat_usage_clock_gating_as_controllable_for_analysis	524
dft.use_merged_pdl	525
dft.xlbist_test_mode_from_1500	525
dpx.clock_opt.distribution_effort_level	526
dpx.clock_opt.high_capacity_workers	526
dpx.enable	527
dpx.place_opt.distribution_effort_level	527
dpx.place_opt.high_capacity_workers	528
dpx.route_opt.distribution_effort_level	528
dpx.route_opt.high_capacity_workers	528
e	529
eco.add_buffer.honor_dont_touch	529
eco.add_buffer_on_route.advanced_detect_layer	530
eco.add_buffer_on_route.min_distance_buffer_to_load	531
eco.freeze_silicon.enable_multi_threads	531
eco.freeze_silicon.not_spare_cell_aware	532
eco.freeze_silicon.pfs_map_spare_cells_only	533
eco.functional.diff_ignore_tie_changes	534
eco.functional.diff_preprocess_for_verilog	534
eco.placement.optimized_legalization	535
eco.post_silicon_debug.bs_via_check_window_size	535
eco.post_silicon_debug.bs_via_percent_goal	536
eco.post_silicon_debug.fib_logic_cell_prefix	536

Contents

eco.post_silicon_debug.fib_tie_cell_prefix	537
eco.post_silicon_debug.fs_fib_check_window_size	537
eco.post_silicon_debug.fs_fib_min_placeable_area_percent	538
eco.post_silicon_debug.fs_fib_percent_goal	539
eco.post_silicon_debug.power_elevation_check_window_size	539
eco.post_silicon_debug.power_elevation_max_route_length	540
eco.post_silicon_debug.power_elevation_min_placeable_area_percent	540
eco.post_silicon_debug.power_elevation_percent_goal	541
eco.post_silicon_debug.signal_elevation_max_route_length	542
eco.remove_buffer.keep_rtl_net	542
eco.size_cell.honor_dont_touch	543
eco.size_cell.honor_dont_use	544
ecolm.freeze_silicon.psc_overwrite_auto_derived_first_track_offset	544
ecolm.freeze_silicon.psc_track_mask	545
ecolm.freeze_silicon.psc_track_offset	546
em.net_delta_temperature	547
em.net_delta_temperature_layer	547
em.net_duration_condition_for_peak	548
em.net_global_rms_relaxation_factor	549
em.net_lifetime_years	549
em.net_metal_line_number	550
em.net_min_duty_ratio	551
em.net_use_waveform_duration	552
em.net_violation_rule_types	552
em.new_temperature_nonlinear_interp	553
est_delay.ml_delay_opto_dir	554
extract.abut_as_routed	554
extract.auto_region_adjustment	554
extract.connect_open	555
extract.enable_coupling_cap	555
extract.enable_ml_vr_estimation	556
extract.enable_via_alignment	556
extract.exclusive_use_layers	557
extract.fusion_starrc_vmf	557
extract.fusion_without_starrc_config	558
extract.high_fanout_threshold	558
extract.memory_reduction	559
extract.ml_gr_level	560
extract.ml_vr_level	560

Contents

extract.rde_effort_level	561
extract.read_frame_view	562
extract.read_zero_spacing_blockage	562
extract.starrc_mode	563
extract.starrc_path	563
extract.two_pin_net_instance_port	564
extract.via_alignment_contraction_ratio_limit	565
extract.via_alignment_expansion_ratio_limit	565
extract.vr_pattern_must_join_over_pin_layer	566
extract.zero_spacing_blockage_ratio	566
f	566
file.3dblox.complete_collaterals	566
file.3dblox.drm_interface_name	567
file.3dblox.general_writer	568
file.3dblox.lvs	568
file.3dblox.oa_include_files	569
file.3dblox.set_3dbf_file	570
file.3dblox.set_apr_tech_file	570
file.3dblox.set_connection_lvs_map	571
file.3dblox.set_def_file	571
file.3dblox.set_drc_deck	571
file.3dblox.set_gds_file	572
file.3dblox.set_gds_mapping_file	572
file.3dblox.set_info_tech_file	573
file.3dblox.set_ipf_file	573
file.3dblox.set_ircx_tech_file	574
file.3dblox.set_lef_file	574
file.3dblox.set_liberty_file	575
file.3dblox.set_lvs_deck	575
file.3dblox.set_rc_tech_file	575
file.3dblox.set_root_cell	576
file.3dblox.set_spice_model	576
file.3dblox.set_thermal_name	577
file.3dblox.set_thermal_tech_file	577
file.3dblox.set_upf_file	578
file.3dblox.set_verilog_file	578
file.3dblox.write_hier_bumps	579
file.def.append_shape_and_terminal	579

Contents

file.def.check_mask_constraints	580
file.def.check_mask_constraints_exclude_layers	581
file.def.convert_rects_to_paths	581
file.def.custom_via_as_user_route	582
file.def.enable_user_attribute_properties	583
file.def.exclude_cut_metal	584
file.def.fill_purpose_map	584
file.def.ignore_uncolored_shapes	586
file.def.ignore_uncolored_shapes_exclude_layers	586
file.def.maintain_via_ladders	587
file.def.maskshift_consistency_check	587
file.def.non_default_width_wiring_to_net	588
file.def.pg_via_arrays	589
file.def.relax_via_ladder_restriction	589
file.def.rule_based_name_matching	590
file.def.scan_cells_in_netlist_order	590
file.def.set_pg_mask_fixed	591
file.def.skip_combinational_cell_check	592
file.def.skip_connection_of_incomplete_net	592
file.def.skip_net_connection	593
file.def.split_via_matrix	593
file.def.support_property_definitions	594
file.def.support_via_matrix	595
file.def.support_voltage_area	595
file.def.write_matching_net_names	596
file.def.wrong_way_wiring_to_special_net	597
file.gds.allow_incompatible_units	597
file.gds.check_mask_constraints	598
file.gds.check_mask_constraints_exclude_layers	598
file.gds.contact_prefix	599
file.gds.create_custom_via	600
file.gds.enable_half_metal_generation	601
file.gds.enable_multiple_rules_per_layer	601
file.gds.enable_special_snapping	602
file.gds.exclude_layers	603
file.gds.ignore_uncolored_shapes	604
file.gds.ignore_uncolored_shapes_exclude_layers	605
file.gds.include_sub_cell_in_block	605
file.gds.max_cell_name_length	606

Contents

file.gds.net_text_on_layers	607
file.gds.output_first_same_name_cell	607
file.gds.pg_via_arrays	608
file.gds.port_type_map	609
file.gds.prefix_for_fill	610
file.gds.read_enable_multiple_rules_per_layer	610
file.gds.rotate_pin_text_by_access_direction	611
file.gds.shape_use_for_pg_mesh	612
file.gds.split_via_matrix	613
file.gds.text_all_pins	613
file.gds.text_layer_map	614
file.gds.trace_copy_blockage_color_from_overlapped_terminal	615
file.gds.trace_copy_overlap_shape_from_sub_cell	616
file.gds.trace_merge_overlap_mask	617
file.gds.trace_terminal_length	617
file.gds.trace_terminal_length_4x_width	619
file.gds.trace_terminal_length_boundary_only	619
file.gds.trace_terminal_length_exclude_list	620
file.gds.trace_terminal_length_layer_pairs	621
file.gds.trace_terminal_length_layers	622
file.gds.trace_terminal_text	622
file.gds.trace_terminal_type	624
file.gds.trace_unmapped_text	624
file.gds.use_only_mapped_text	625
file.gds.write_metal_below_cut_to_top	626
file.lef.allow_empty_pin	627
file.lef.auto_rename_conflict_sites	628
file.lef.derive_forbidden_abutment_placement_rule	629
file.lef.derive_forbidden_placement_rule	629
file.lef.derive_must_join_for_pg	630
file.lef.derive_must_join_for_signal	631
file.lef.non_real_cut_obs_mode	632
file.lef.output_non_real_blockages	633
file.lef.output_non_real_blockages_for_macro	633
file.lef.write_parasitic_layers	634
file.merge.disable_merge_buffer_refill	635
file.oasis.allow_incompatible_units	635
file.oasis.check_mask_constraints	636
file.oasis.check_mask_constraints_exclude_layers	637

Contents

file.oasis.contact_prefix	637
file.oasis.create_custom_via	638
file.oasis.enable_half_metal_generation	639
file.oasis.enable_multiple_rules_per_layer	639
file.oasis.enable_special_snapping	640
file.oasis.exclude_layers	641
file.oasis.ignore_uncolored_shapes	642
file.oasis.ignore_uncolored_shapes_exclude_layers	643
file.oasis.include_sub_cell_in_block	643
file.oasis.max_cell_name_length	644
file.oasis.net_text_on_layers	645
file.oasis.output_first_same_name_cell	646
file.oasis.pg_via_arrays	647
file.oasis.port_type_map	647
file.oasis.prefix_for_fill	648
file.oasis.read_enable_multiple_rules_per_layer	649
file.oasis.rotate_pin_text_by_access_direction	650
file.oasis.shape_use_for_pg_mesh	650
file.oasis.split_via_matrix	651
file.oasis.text_all_pins	652
file.oasis.text_layer_map	652
file.oasis.trace_copy_blockage_color_from_overlapped_terminal	653
file.oasis.trace_copy_overlap_shape_from_sub_cell	654
file.oasis.trace_merge_overlap_mask	655
file.oasis.trace_terminal_length	656
file.oasis.trace_terminal_length_4x_width	657
file.oasis.trace_terminal_length_boundary_only	658
file.oasis.trace_terminal_length_exclude_list	658
file.oasis.trace_terminal_length_layer_pairs	659
file.oasis.trace_terminal_length_layers	660
file.oasis.trace_terminal_text	661
file.oasis.trace_terminal_type	662
file.oasis.trace_unmapped_text	663
file.oasis.use_only_mapped_text	664
file.oasis.write_metal_below_cut_to_top	665
file.ssf.current_version	665
file.ssf.supported_versions	666
file.tech.library_developer_mode	667
file.tlup.max_preservation_size	667

Contents

file.verilog.allow_multiple_top	668
file.verilog.allow_same_block	669
file.verilog.default_user_label	669
file.verilog.enable_vpp	670
file.verilog.handle_complex_bus_port	671
file.verilog.ieee_1735_decryption	672
file.verilog.keep_constant_assigns	672
file.verilog.keep_unconnected_cells	673
file.verilog.keep_unconnected_nets	674
file.verilog.read_soc_outline	674
file.verilog.reconstruct_multi_dimension_bus	675
file.verilog.write_bus_port_style	676
file.verilog.write_non_pg_net_assigns	677
file.verilog.write_pg_net_assigns	678
file.verilog.write_unconnected_pg_ports	679
finfet.ignore_grid.std_cell	679
flip_chip.route.allow_rotated_via	680
flip_chip.route.bump_via_landing_mode	680
flip_chip.route.connect_edge_center	681
flip_chip.route.connect_pad_without_endcap	681
flip_chip.route.connect_via_center	682
flip_chip.route.create_soft_rule_shield	683
flip_chip.route.design_style	683
flip_chip.route.detour_cost	684
flip_chip.route.extend_vias_on_pin	684
flip_chip.route.layer_bump_spacings	685
flip_chip.route.layer_routing_angles	686
flip_chip.route.layer_tie_shield_widths	686
flip_chip.route.point_to_point_connection_file_name	687
flip_chip.route.route_to_ring_nets	688
flip_chip.route.route_to_stripe_nets	689
flip_chip.route.routing_style	689
flip_chip.route.secondary_routing_layer_cost	690
flip_chip.route.secondary_shielding_net	690
flip_chip.route.shielding_net	691
flow.common.advanced_technology	691
flow.common.effort	692
flow.common.io_priority	692
flow.common.node_specific_ppa	693

Contents

flow.common.wire_optimization_features	693
flow.common.wire_optimization_strategy	694
flow.data_robustness.enable_ezi	694
flow.data_robustness.enable_timer_improvements	695
flow.data_robustness.version	695
flow.runtime.version	697
flow.set_qor_strategy.version	698
flow.set_tech.version	698
formality.svf.integrate_in_ndm	698
g	699
gui.annotation_symbol_dir	699
gui.auto_link_blocks	700
gui.auto_open_design_windows	700
gui.batch_x_display	701
gui.block_mark_type	702
gui.bump_comparison_location_tolerance	703
gui.command_form_close_dialog_after_run_or_script	704
gui.command_form_log_options_with_default_values	704
gui.command_form_show_result_dialog_after_run	705
gui.command_form_use_object_names_for_proc_options	706
gui.custom_setup_files	707
gui.design_rule_package_dir	708
gui.embedded_app_options_help_page	708
gui.enable_custom_setup_files	708
gui.file_editor_command	709
gui.filter_default_condition	710
gui.follow_mouse_mode	711
gui.graphics_system	711
gui.icv_home_dir	712
gui.layer_solid_fill_pattern	713
gui.max_cores	714
gui.mouse_tool_auto_raise	714
gui.mouse_tool_config	715
gui.name_based_hier_delimiter	715
gui.num_3dic_layers	715
gui.persistent_ruler	716
gui.selection_stack_depth	716
gui.selection_undo	717

Contents

gui.tk_mode	717
gui.url_browser_command	719
gui.user_icon_directories	719
gui.verdi_indesign.vcs_home	720
gui.verdi_indesign.verdi_home	721
gui.write_image_data_file	721
gui_build_query_data_table	722
gui_custom_setup_files	722
gui_default_window_type	723
gui_disable_custom_setup	724
gui_online_browser	724
gui_selection_stack_depth	725
gui_selection_undo	725
gui_suppress_auto_layout	726
h	726
hdlin.analyze_prioritize_command_line_defines	726
hdlin.elaborate.black_box	727
hdlin.elaborate.black_box_all_except	728
hdlin.elaborate.enable_ff_sync_set_reset	729
hdlin.elaborate.enable_ff_sync_set_reset_all_except	730
hdlin.elaborate.failsafe_fsm	730
hdlin.elaborate.ff_infer_async_set_reset	730
hdlin.elaborate.ff_infer_sync_set_reset	731
hdlin.elaborate.infer_complex_set_reset	731
hdlin.elaborate.infer_local_sync_enable_only	732
hdlin.elaborate.infer_mux_size_only	732
hdlin.elaborate.inout_preserve_rtl_assignment_direction	734
hdlin.elaborate.keep_signal_name	734
hdlin.elaborate.latch_infer_async_set_reset	739
hdlin.elaborate.max_port_width_allowed	740
hdlin.elaborate.maximum_while_loop_iterations	740
hdlin.elaborate.netlist_unloaded_signals	741
hdlin.elaborate.optimize_buffers	741
hdlin.elaborate.preserve_sequential	742
hdlin.elaborate.svconfig_liblist_verbose_mode	745
hdlin.flow.ieee_1735_decryption	746
hdlin.hdl_library.default_dirname	746
hdlin.hdl_library.default_name	747

Contents

hdlin.hdl_library.intermediate_file_method	748
hdlin.hdl_library.setup	748
hdlin.logfile_format	749
hdlin.naming.cell_prefix	749
hdlin.naming.hierarchical	750
hdlin.naming.module_name_limit	750
hdlin.naming.net_prefix	751
hdlin.naming.register_suffix_on_field	751
hdlin.naming.shorten_long_module_name	752
hdlin.naming.shorten_top_module_name	753
hdlin.naming.strict_template_naming_style	753
hdlin.naming.structs_records	754
hdlin.naming.sverilog_union_member	755
hdlin.naming.template_naming_style	755
hdlin.naming.template_parameter_style	756
hdlin.naming.template_separator_style	757
hdlin.naming.upf_compatible	758
hdlin.naming.verilog_generate_separator_style	759
hdlin.naming.verilog_uppercase_names	760
hdlin.naming.vhdl_generate	760
hdlin.naming.vhdl_generate_separator	761
hdlin.naming.vhdl_preserve_case	762
hdlin.report.analyze_verbose	762
hdlin.report.check_no_latch	763
hdlin.report.design_hierarchy	763
hdlin.report.design_hierarchy_filename	764
hdlin.report.level	764
hdlin.report.module_param_id_length_threshold	766
hdlin.report.non_tristate_multiple_drivers	767
hdlin.report.port_dimension_mismatch_error	768
hdlin.report.sequential_pruning	769
hdlin.report.vcs	770
hdlin.report.vcs_filename	770
hdlin.report_info	770
hdlin.report_mem	771
hdlin.report_time	772
hdlin.safety.always_fsm_complete	772
hdlin.set_top_module.uncommit_sub_blocks	772
hdlin.sverilog.allow_unpacked_array_concat_on_port	773

Contents

hdlin.sverilog.blackbox_modules	774
hdlin.sverilog.enable_persistent_macros	775
hdlin.sverilog.enable_persistent_sv_interfaces	776
hdlin.sverilog.enable_rtl_attributes	776
hdlin.sverilog.enforce_standalone_generate_blocks	777
hdlin.sverilog.interface_only_modules	778
hdlin.sverilog.packages	778
hdlin.sverilog.persistent_macros_filename	780
hdlin.sverilog.persistent_sv_interfaces_filename	781
hdlin.sverilog.standard	781
hdlin.sverilog.tic_tic_discards_whitespace	782
hdlin.sverilog.use_search_path_for_include	783
hdlin.sverilog.write_token_file	783
hdlin.verilog.hierarchy_separator_style	784
hdlin.verilog.module_architecture_name_splitting	785
hdlin.verilog.standard	786
hdlin.verilog.v2005_replication_semantics	786
hdlin.vhdl.93_style_concatenation	787
hdlin.vhdl.enable_configurations	788
hdlin.vhdl.hierarchy_separator_style	788
hdlin.vhdl.mixed_language_instantiation	790
hdlin.vhdl.standard	790
hdlin.vhdl.subprogram_default_values	791
hdlin.vhdl.syntax_extensions	791
hdlin.xrefs.inject_xref_info	792
i	792
in_gui_session	792
l	793
lib.configuration.assembly_scripts	793
lib.configuration.auto_reassemble_clib	794
lib.configuration.auto_reassemble_on_implicit_open	794
lib.configuration.cdpl_host	795
lib.configuration.central_output_dir	796
lib.configuration.create_missing_macro_frame	796
lib.configuration.default_flow_setup	797
lib.configuration.display_lm_messages	798
lib.configuration.enable	798
lib.configuration.force_timing_view_update	799

Contents

lib.configuration.icc_shell_exec	800
lib.configuration.lef_site_mapping	800
lib.configuration.local_output_dir	801
lib.configuration.process_label_mapping	802
lib.configuration.remove_local_output_dir	802
lib.configuration.update_timing_view_for_pt_delay	803
lib.logic_model.auto_remove_incompatible_timing_designs	803
lib.logic_model.auto_remove_timing_only_designs	804
lib.logic_model.preserve_nldm_pin_caps	805
lib.logic_model.relax_pin_equal_opposite_attr_check	805
lib.logic_model.require_same_opt_attrs	806
lib.logic_model.resolve_voltage_range_differences	807
lib.logic_model.use_db_rail_names	807
lib.setting.compress_design_lib	808
lib.setting.compress_lib_package	809
lib.setting.compression_format	810
lib.setting.enable_db_search_optimization	810
lib.setting.enable_multithreaded_open	811
lib.setting.enable_process_number_scaling	811
lib.setting.enable_via_region_override	812
lib.setting.fusion_lib_fast_integrity_check	813
lib.setting.get_command_scope	813
lib.setting.handle_noncolored_shapes	814
lib.setting.icc_shell_exec	815
lib.setting.max_wait_time	815
lib.setting.milkyway_exec	816
lib.setting.on_disk_operation	816
lib.setting.show_internal_pins	816
lib.setting.use_tech_scale_factor	817
lib.workspace.allow_append_layout_views	818
lib.workspace.allow_commit_workspace_overwrite	819
lib.workspace.allow_loading_layout_view_only_lib	819
lib.workspace.allow_missing_related_pg_pins	820
lib.workspace.create_workspace_tf_verbose	821
lib.workspace.enable_secondary_pg_marking	821
lib.workspace.exclude_design_filters	822
lib.workspace.explore_create_aggregate	823
lib.workspace.fast_exploration	824
lib.workspace.group_libs_fix_cell_shadowing	824

Contents

lib.workspace.group_libs_macro_grouping_strategy	825
lib.workspace.group_libs_naming_strategies	826
lib.workspace.group_libs_physical_only_name	827
lib.workspace.include_design_filters	829
lib.workspace.keep_all_physical_cells	830
lib.workspace.library_developer_mode	831
lib.workspace.remove_frame_bus_properties	831
lib.workspace.save_design_views	832
lib.workspace.save_layout_views	832
libra.complib.match_full_cell_name	833
libra.complib.match_lexical_cell_height	834
libra.complib.match_lexical_cell_tile_width	834
libra.complib.match_lexical_circuit_clockgate	835
libra.complib.match_lexical_circuit_device_sizing_variant	836
libra.complib.match_lexical_circuit_eco	836
libra.complib.match_lexical_circuit_layout_density_variant	837
libra.complib.match_lexical_circuit_layout_variant_1	838
libra.complib.match_lexical_circuit_layout_variant_2	838
libra.complib.match_lexical_circuit_multibit	839
libra.complib.match_lexical_circuit_topology_variant	840
libra.complib.match_lexical_device_channel_length	841
libra.complib.match_lexical_device_threshold_voltage	841
libra.complib.match_lexical_device_type	842
libra.complib.match_lexical_electrical_delay_opt_variant	843
libra.complib.match_lexical_electrical_hold_opt_variant	843
libra.complib.match_lexical_electrical_property_variant	844
libra.complib.match_lexical_electrical_setup_opt_variant	845
libra.complib.match_lexical_electrical_slew_opt_variant	846
libra.complib.match_lexical_electrical_symmetry_variant	846
libra.complib.match_lexical_electrical_timing_arc_variant	847
libra.complib.match_lexical_userdef_property_1	848
libra.complib.match_lexical_userdef_property_2	848
libra.complib.match_lexical_userdef_property_3	849
libra.complib.match_lexical_well_substrate_bias_architecture	850
libra.family.lexical_cell_height	851
libra.family.lexical_cell_prefix_name	851
libra.family.lexical_cell_tile_width	852
libra.family.lexical_circuit_clockgate	853
libra.family.lexical_circuit_device_sizing_variant	853

Contents

libra.family.lexical_circuit_eco	854
libra.family.lexical_circuit_layout_density_variant	855
libra.family.lexical_circuit_layout_variant_1	855
libra.family.lexical_circuit_layout_variant_2	856
libra.family.lexical_circuit_multibit	857
libra.family.lexical_circuit_topology_variant	857
libra.family.lexical_device_channel_length	858
libra.family.lexical_device_threshold_voltage	859
libra.family.lexical_device_type	859
libra.family.lexical_drive1_name	860
libra.family.lexical_drive2_name	861
libra.family.lexical_electrical_delay_opt_variant	861
libra.family.lexical_electrical_hold_opt_variant	862
libra.family.lexical_electrical_property_variant	863
libra.family.lexical_electrical_setup_opt_variant	863
libra.family.lexical_electrical_slew_opt_variant	864
libra.family.lexical_electrical_symmetry_variant	865
libra.family.lexical_electrical_timing_arc_variant	865
libra.family.lexical_library_prefix_name	866
libra.family.lexical_userdef_property_1	867
libra.family.lexical_userdef_property_2	867
libra.family.lexical_userdef_property_3	868
libra.family.lexical_well_substrate_bias_architecture	869
libra.lex_classification.keep_partial_results	870
libra.redundancies.across_families	870
libra.redundancies.area_costing	871
libra.redundancies.average_leakage_costing	872
libra.redundancies.dynamic_power_costing	872
libra.redundancies.ignore_physical_variants	873
libra.redundancies.timing_costing	874
link.allow_blasted_bus_reconstruction	874
link.allow_period_to_match_underscore	875
link.allow_port_array_interface_hybrid_matching	876
link.allow_square_bracket_to_match_underscore	877
link.bit_blast_naming_style	878
link.bit_blasted_bus_linking	879
link.do_bus_width_mismatch	880
link.error_on_net_port_width_mismatch	880
link.force_case	881

Contents

link.interface_variable_delimiter	882
link.prefer_frame	883
link.reference_limit	884
link.require_physical	884
link.truncate_bus_pin_width	885
link.undo_enabled	886
link.use_session_ref_libs	886
link.user_units_from_first_library	887
link.verbose	888
m	888
mpn.buffer_top_level_constant_port_nets	888
mpn.buffer_unloaded_constant_port_nets	889
mpn.buffer_unloaded_multiple_port_nets	889
multibit.banking.across_equivalent_icg	890
multibit.banking.bank_consecutive_cells	890
multibit.banking.enable_clock_pin_level_check	891
multibit.banking.enable_strict_scan_check	892
multibit.banking.enable_subblock_optimization	892
multibit.banking.enable_tns_degradation_estimation	893
multibit.banking.ignore_scan_cstr_list	893
multibit.banking.lcs_consider_multi_si_cells	895
multibit.banking.maximum_allowable_distance	895
multibit.banking.maximum_allowable_name_length	896
multibit.common.exclude_size_only_cells	896
multibit.common.ignore_cell_with_sdc	897
multibit.common.ignore_cstr_list	897
multibit.debanking.avoid_new_high_fanout_net	898
multibit.debanking.check_high_fanout_net	899
multibit.debanking.effort	899
multibit.debanking.honor_tie_cell_fanout_limit	900
multibit.naming.common_prefix_naming_mode	901
multibit.naming.common_prefix_naming_style	902
multibit.naming.compact_hierarchical_name	903
multibit.naming.element_separator_style	903
multibit.naming.expanded_name_style	904
multibit.naming.instance_name_max_length	906
multibit.naming.multiple_name_separator_style	906
multibit.naming.name_prefix	907

Contents

multibit.naming.range_separator_style	907
multibit.naming.rtl_banking_concatenate_single_bit_names	908
mv.cells.ao_cells_without_primary_pg_pin	909
mv.cells.auto_va_set_voltage_file_name	909
mv.cells.check_ls_ground_supply_netgroup	910
mv.cells.check_ls_power_supply_netgroup	910
mv.cells.connect_iso_control_directly	911
mv.cells.connect_power_switch_incremental_optimization_max_iterations	911
mv.cells.connect_power_switch_incremental_optimization_max_number_of_cells	912
mv.cells.connect_power_switch_min_long_distance_value	912
mv.cells.connect_power_switch_optimize_ack_out_distance	913
mv.cells.deterministic_iso_name_format	913
mv.cells.enable_connect_power_switch_incremental_optimization	914
mv.cells.enable_connect_power_switch_incremental_optimization_fast_approximation	915
mv.cells.enable_iso_cell_bias_rail_order_check	915
mv.cells.enable_multibit_pm_cells	916
mv.cells.enable_tap_cell_virtual_bias_pin_check	916
mv.cells.infer_complex_retention_cells	917
mv.cells.insert_clamp_in_zpr_hierarchy	919
mv.cells.isolate_constant_net	920
mv.cells.isolate_diode_load	920
mv.cells.ls_association_derive_missing_csn	920
mv.cells.mv_allow_buf_on_mv_boundary_nets	921
mv.cells.mv_allow_domain_primary_relative_ao	922
mv.cells.mv_use_domain_primary_for_macro_iso_pin	922
mv.cells.mv_use_domain_primary_for_zpr_clamp_input	923
mv.cells.no_resolved_iso_strategy_attr_on_dangling_path	923
mv.cells.rename_isolation_cell_with_formal_name	924
mv.cells.report_pvt_exact	924
mv.cells.report_pvt_verbose	924
mv.cells.smart_derive_iso_strategy_on_new_control_ports	925
mv.cells.support_backup_ground_only_retention	925
mv.cells.use_name_based_iso_matching	926
mv.cells.use_name_based_ls_matching	926
mv.cells.use_name_based_psw_matching	927
mv.hierarchical.afs_create_reference_supply	927
mv.hierarchical.afs_write_driver_receiver_supply	928
mv.hierarchical.allow_voltage_violation	928
mv.hierarchical.check_interface_objs_of_block_abstract	929

Contents

mv.hierarchical.etm_upf_skip_missing_object_suppress_warning	929
mv.hierarchical.ignore_mib_checking_error	930
mv.hierarchical.include_pg_in_budget_shell	930
mv.hierarchical.no_srsn_for_new_ports	931
mv.hierarchical.propagate_driver_supply_for_empty_design	932
mv.hierarchical.propagate_driver_supply_on_feedthrough	932
mv.hierarchical.propagate_driver_supply_use_same_driver_receiver	933
mv.hierarchical.propagate_driver_supply_use_soft_macro_domain_primary	933
mv.hierarchical.skip_pst_in_blocks	934
mv.hierarchical.split_constraints_include_only_existing_supplies_in_block	934
mv.hierarchical.split_constraints_write_driver_receiver_supply	935
mv.hierarchical.split_cstr_create_pst_for_outside_supply	935
mv.hierarchical.split_cstr_write_simple_object_name_for_user_attribute	936
mv.hierarchical.upf_port_pswout_dir_out	936
mv.incomplete_upf.ao_infer_csn_rule	937
mv.incomplete_upf.enable	937
mv.incomplete_upf.iso_infer_csn_rule	938
mv.incomplete_upf.ls_infer_csn_rule	938
mv.incomplete_upf.missing_va	939
mv.incomplete_upf.psw_infer_csn_rule	939
mv.incomplete_upf.ret_infer_csn_rule	940
mv.pg.connect_pg_net_auto_mode_include_tie	940
mv.pg.continue_infer_pg_with_conflict	941
mv.pg.create_floating_pg	941
mv.pg.default_ground_supply_net_name	942
mv.pg.default_ground_supply_port_name	942
mv.pg.default_power_supply_net_name	942
mv.pg.default_power_supply_port_name	943
mv.pg.honor_soft_macro	943
mv.pg.minimum_horizontal_secondary_pg_placement_va_region_size	944
mv.pg.minimum_vertical_secondary_pg_placement_va_region_size	944
mv.pg.preserve_physical_only_pg	945
mv.pg.resolve_pg_top_port_deletion	945
mv.pg.secondary_pg_placement_lateral_margin_only	946
mv.upf.allow_els_on_logic_zero	946
mv.upf.allow_is_isolated_output_check	947
mv.upf.allow_iso_on_dont_touch_networks	947
mv.upf.allow_ls_on_dont_touch_networks	948
mv.upf.allow_ls_on_ideal_networks	948

Contents

mv.upf.allow_ls_on_leaf_pins	949
mv.upf.allow_ls_on_logic_one	949
mv.upf.allow_ls_on_logic_zero	950
mv.upf.allow_negative_voltage	950
mv.upf.allow_non_bias_domain_in_bias_scope	951
mv.upf.allow_non_bias_ls_els_in_bias_domains	951
mv.upf.allow_non_bias_macros	952
mv.upf.allow_rbm_in_upf_prime_flow	952
mv.upf.allow_retn_clamp_on_dont_touch_networks	953
mv.upf.ao_legalize_gtech_buf	953
mv.upf.auto_infer_supply_connections	954
mv.upf.auto_ls_clock_nets	955
mv.upf.auto_resolve_pg_net	955
mv.upf.bias_insulation_checks_rigidity_level	956
mv.upf.connect_missing_backup_to_domain_primary	956
mv.upf.connect_supply_net_ignore_nonexistent_objects	957
mv.upf.create_switch_place_holder_cells	958
mv.upf.dft_reuse_existing_iso_cell	958
mv.upf.disable_is_analog_support	959
mv.upf.disable_upf_loading_on_non_upf_design	959
mv.upf.enable_amvf_html_report	960
mv.upf.enable_amvf_report	960
mv.upf.enable_golden_upf	961
mv.upf.enable_insulation_checks_for_ls	962
mv.upf.enable_logic_expr_new_processing	962
mv.upf.enable_missing_voltage_area	963
mv.upf.enable_nwell_only_support	964
mv.upf.enable_pg_pin_reconnection	964
mv.upf.enable_reordering_building_pst_table	965
mv.upf.generate_iso_with_pst_equivalence	965
mv.upf.generate_iso_with_same_signal	965
mv.upf.gupf_report_missing_objects	966
mv.upf.identify_clock_path	967
mv.upf.ignore_dangling_net_on_heterogenous_path	967
mv.upf.ignore_ls_main_rail	968
mv.upf.ignore_ls_main_rail_ground_supply	968
mv.upf.input_enforce_simple_names	969
mv.upf.input_supply_equal_ao_with_driver	969
mv.upf.insert_ls_on_user_cstr_only	970

Contents

mv.upf.iso_control_inv_prefix	970
mv.upf.iso_map_exclude_zpr_clamp_lib_cells	971
mv.upf.keep_map_isolation_library_cell_order	971
mv.upf.keep_map_level_shifter_library_cell_order	972
mv.upf.load_upf_continue_on_error	972
mv.upf.ls_auto_insertion_sink_domain_support	973
mv.upf.map_relax_voltage_range_check	973
mv.upf.max_supply_count_without_driver	974
mv.upf.multi_pd_shared_voltage_area_flow	974
mv.upf.multi_pd_single_va_strict_check	975
mv.upf.name_map_version	975
mv.upf.no_buffering_after_retention_clamp_cell	976
mv.upf.no_tie_off_upf_ctrl_signal	976
mv.upf.nor_iso_macro_allow_enable_supply_check	977
mv.upf.object_namespace_checks	977
mv.upf.power_model_library	978
mv.upf.power_model_search_path	979
mv.upf.power_model_verbose	979
mv.upf.preserve_hierarchical_pins_with_upf_constraints	980
mv.upf.preserve_logic_in_boolean_expr	981
mv.upf.preserve_power_domain_root_cells	981
mv.upf.prevent_uniquification_in_load_commit_upf	982
mv.upf.report_pst_internal_states_names	982
mv.upf.save_upf_blackbox_preserve_user_supply_attributes	984
mv.upf.save_upf_include_supply_exceptions	985
mv.upf.save_upf_include_supply_exceptions_for_iso_retn	985
mv.upf.save_upf_line_indent	986
mv.upf.save_upf_line_width	986
mv.upf.save_upf_update_derived_bias	987
mv.upf.save_upf_use_relative_path_for_top_scope	987
mv.upf.set_els_cell_naming_rule	988
mv.upf.skip_all_iso_insertion	988
mv.upf.skip_all_ls_insertion	989
mv.upf.skip_all_retention_insertion	989
mv.upf.skip_ao_check_for_els_input	990
mv.upf.skip_dft_iso_when_no_violation	991
mv.upf.skip_ls_on_block_crossing_nets	991
mv.upf.skip_retention_on_dft_register	991
mv.upf.skip_single_rail_for_non_primary_iso_supply	992

Contents

mv.upf.system_pst_building_time_limit	993
mv.upf.unmap_iso_with_els_failure	993
mv.upf.upf_add_power_state_21_syntax	994
mv.upf.upf_add_power_state_ignore_logic_expression_error	995
mv.upf.upf_allow_non_constant_net_reconnect	995
mv.upf.upf_enable_state_propagation_in_add_power_state	996
mv.upf.upf_skip_diff_supply_for_gmvc	996
mv.upf.upf_use_insulated_cells_in_leq	997
mv.upf.use_driver_receiver_for_io_supplies	997
mv.upf.use_preswitched_supply_for_macro_pins	998
mv.upf.virtual_supply_available_when_connected_to_regular_pg	998
mv.upf.write_create_power_domain_with_scope	999
mv.upf.write_crosstool_no_upf_update_command	1000
mv.upf.write_crosstool_wrappers	1001
mv.upf.write_csn_for_dual_rail_cells_in_ao_domain	1002
mv.upf.zpr_clamp_cell_prefix	1002
o	1003
opt.area.effort	1003
opt.buffering.check_for_skinny_blockages	1003
opt.buffering.enable_advanced_buffering	1004
opt.buffering.enable_delay_tree_rebuffering_depth	1005
opt.buffering.enable_flow_rebuffering	1006
opt.buffering.enable_hierarchy_mapping	1006
opt.buffering.enable_ldrc_tree_rebuffering_depth	1007
opt.buffering.enable_mv_rebuffering	1007
opt.buffering.enable_rebuffering	1008
opt.buffering.exclusive_hard_bound_buffering_mode	1009
opt.buffering.filter_end_cap_cells	1009
opt.buffering.filter_filler_cells	1010
opt.buffering.filter_physical_only_lib_cells	1010
opt.buffering.filter_well_tap_cells	1010
opt.buffering.hard_bound_buffering_mode	1011
opt.buffering.high_fanout_synthesis_cost	1011
opt.buffering.high_fanout_synthesis_delay_effort	1012
opt.buffering.improved_ootb_topology_generation	1012
opt.buffering.min_skinny_blockage_size	1013
opt.buffering.skinny_blockage_scope	1014
opt.buffering.two_pass_hfs	1014

Contents

opt.buffering.vr_dualrail_cost	1015
opt.clock_latency_estimation.report_skipped_pins	1016
opt.common.advanced_logic_restructuring_mode	1016
opt.common.advanced_logic_restructuring_wirelength_costing	1017
opt.common.allow_incorrect_pvt	1018
opt.common.allow_physical_feedthrough	1018
opt.common.auto_hold_effort	1019
opt.common.auto_site_def	1019
opt.common.bmap_use_partial_routing_blockages	1020
opt.common.buffer_area_effort	1021
opt.common.buffering_for_advanced_technology	1021
opt.common.do_physical_checks	1022
opt.common.drc_mode_buffering	1022
opt.common.enable_multibit_combinational_cells	1023
opt.common.enable_multibit_feature	1024
opt.common.enable_rde	1025
opt.common.enable_seq_sizing_in_ldrc	1025
opt.common.enable_seq_sizing_in_ldrc_with_max_cap_focus	1026
opt.common.enable_sizing_with_balance_point	1026
opt.common.enable_target_library_subset_opt	1027
opt.common.enable_verilog_assign_check	1027
opt.common.enable_via_ladder_insertion	1028
opt.common.estimate_clock_gate_latency	1028
opt.common.hold_effort	1029
opt.common.insert_clk_data_isolation_cells	1030
opt.common.ir_drop_threshold	1030
opt.common.ir_drop_threshold_clock	1031
opt.common.ir_drop_threshold_data	1031
opt.common.max_fanout	1032
opt.common.max_net_length	1032
opt.common.multibit_flow_strategy	1033
opt.common.optimize_ndr_user_max_layers	1033
opt.common.optimize_ndr_user_min_layers	1034
opt.common.optimize_ndr_user_rule_names	1034
opt.common.optimize_power_ndr_user_max_layers	1035
opt.common.optimize_power_ndr_user_min_layers	1035
opt.common.optimize_power_ndr_user_rule_names	1036
opt.common.pdr_aware_hold	1036
opt.common.power_integrity	1036

Contents

opt.common.power_integrity_analysis_engine	1037
opt.common.power_integrity_effort	1038
opt.common.power_integrity_indesign_post_tcl_proc	1038
opt.common.power_integrity_indesign_pre_tcl_proc	1039
opt.common.power_integrity_redhawk_script_file	1039
opt.common.prefer_exact_pvt	1040
opt.common.preserve_pin_names	1040
opt.common.pvt_setting	1041
opt.common.select_inverter	1042
opt.common.seq_enable_hold	1043
opt.common.setup_aware_hold_fixing	1043
opt.common.setup_tradeoff_threshold_for_hold	1044
opt.common.small_region_area_recovery	1044
opt.common.track_no_new_cells	1045
opt.common.use_ref_libs_only	1046
opt.common.user_instance_name_prefix	1046
opt.dft.clock_aware_scan_reorder	1047
opt.dft.clock_aware_scan_repartition_reorder	1048
opt.dft.do_repartition	1049
opt.dft.enhanced_exception_handling	1050
opt.dft.enhanced_scan_exclusive_repeater_handling	1050
opt.dft.fix_lockup_latch_clock	1051
opt.dft.fix_repeater_flop_clock	1051
opt.dft.fix_start_stop	1052
opt.dft.frozen_hier_preservation	1052
opt.dft.hier_preservation	1053
opt.dft.ignore_hier_exceptions	1053
opt.dft.ignore_inversions	1054
opt.dft.low_impact_optimization	1054
opt.dft.ng_hold_aware	1055
opt.dft.optimize_dft_detailed_report	1055
opt.dft.optimize_scan_chain	1056
opt.dft.post_fix_long_nets	1056
opt.dft.post_fix_polarity	1057
opt.dft.timing_friendly_reorder	1057
opt.dft.toggle_rate_aware	1058
opt.dft.use_ng_engines	1059
opt.dynamic_regions.enable	1059
opt.dynamic_regions.num_regions	1060

Contents

opt.mcmm.dominant_scenarios	1060
opt.mcmm.use_synthetic_corners	1061
opt.mux.select_op_heavy_module_robustness_mode	1061
opt.port.eliminate_verilog_assign	1062
opt.power.effort	1063
opt.power.leakage_type	1064
opt.power.mode	1065
opt.preroute_synthesis.architecture_booster_transform	1066
opt.tie_cell.add_to_highest_hierarchy	1066
opt.tie_cell.check_frontend_restrictions	1067
opt.tie_cell.enable_boundary_restriction_check	1067
opt.tie_cell.ignore_non_user_dont_touch	1068
opt.tie_cell.max_fanout	1068
opt.tie_cell.preserve_restricted_hier_nets	1069
opt.tie_cell.support_auto_vt_classification	1070
opt.timing.effort	1070
p	1071
package.common.angle_tolerance	1071
package.common.d2d_region_layer	1071
package.common.die_region_layer	1072
package.common.enable_obstacle_aware_concave_hull	1073
package.common.fob_region_layer	1074
package.common.length_tolerance	1074
package.common.min_floating_island_size	1075
package.common.pg_nets_layers	1076
package.common.remove_floating_shapes	1077
package.mfg.degas_hole_clearance	1078
package.pdn.void_sizing_strategy	1079
package.route.allow_violation_on_failure	1080
package.route.d2d_shielding_rules	1080
package.route.d2s_bridge_style_max_search_distance	1082
package.route.d2s_pg_via_sharing_rules	1082
package.route.d2s_via_escaping_angles	1084
package.route.d2s_via_escaping_counts	1085
package.route.d2s_via_insertion_counts	1086
package.route.d2s_via_insertion_patterns	1089
package.route.ndr_vias	1091
package.route.pg_plane_spacing_model	1093

Contents

pin_check.analyze.analyze_lib_cell_placement_cmd	1094
pin_check.analyze.analyze_lib_pin_check_routability	1094
pin_check.analyze.composite_cell_ratio	1095
pin_check.analyze.composite_cells	1096
pin_check.analyze.composite_core_height	1097
pin_check.analyze.composite_core_width	1098
pin_check.analyze.composite_inst_count	1099
pin_check.analyze.composite_pin_dist	1100
pin_check.analyze.report_cell_pin_access_even_odd_place	1101
pin_check.analyze.report_cell_pin_access_exclude_lib_pins	1101
pin_check.analyze.report_cell_pin_access_max_pin_dist	1102
pin_check.analyze.report_cell_pin_access_mode	1103
pin_check.analyze.report_cell_pin_access_pin_extension_by_layer_name	1104
pin_check.common.check_routability	1105
pin_check.common.congested_netlist	1105
pin_check.common.grd	1106
pin_check.common.grd_command	1107
pin_check.common.max_cores	1107
pin_check.common.num_cells_per_run	1108
pin_check.common.num_jobs	1109
pin_check.common.output_verilog	1110
pin_check.common.parallel_run_setup_file	1111
pin_check.common.pg_output_tcl	1111
pin_check.common.pg_script_file	1112
pin_check.common.random_netlist	1113
pin_check.common.route_pg	1114
pin_check.common.seed	1114
pin_check.common.tech_node	1115
pin_check.input.cells	1116
pin_check.input.checking_multiplier	1116
pin_check.input.exclude_cells	1117
pin_check.input.exclude_for_checking	1118
pin_check.input.filter_cells_by_pin_count	1119
pin_check.input.include_for_checking	1120
pin_check.place.allow_empty_rows	1121
pin_check.place.auto_reuse	1121
pin_check.place.cell_spacing_rule_file	1122
pin_check.place.check_legality	1123
pin_check.place.core_margin	1123

Contents

pin_check.place.design_under_test_multiplier	1124
pin_check.place.fill_empty_rows_with_blockages	1125
pin_check.place.fix_variant_cell_placement	1126
pin_check.place.flip_first_row	1127
pin_check.place.group_spacing_by_site_width	1127
pin_check.place.init_utilization	1128
pin_check.place.left_offset_by_site_width	1129
pin_check.place.legalize_placement	1130
pin_check.place.macro_spacing	1130
pin_check.place.move_core_to_origin	1131
pin_check.place.preplace_option_file	1132
pin_check.place.remove_duplicate_cell_pairs	1133
pin_check.place.reuse_design	1133
pin_check.place.right_offset_by_site_width	1134
pin_check.place.row_pattern	1135
pin_check.place.use_site_row	1135
pin_check.place.vertical_core_margin	1136
pin_check.report.derive_cell_spacing_constraints	1137
pin_check.report.icv_home_dir	1138
pin_check.report.signoff_check_drc_options	1139
pin_check.report.signoff_check_drc_stages	1140
pin_check.report.signoff_options_file	1140
pin_check.route.allow_metal1_routing	1141
pin_check.route.effort	1142
pin_check.route.incremental_search_repair	1143
pin_check.route.max_detail_route_iterations	1143
pin_check.route.post_route_tcl	1144
pin_check.route.reroute_drc_cells_iterations	1145
pin_check.route.reroute_drc_cells_only	1146
pin_check.route.route_option_file	1146
pin_check.route.signal_net_only	1147
place.coarse.advanced_node_congestion_driven_max_util	1148
place.coarse.auto_density_control	1149
place.coarse.auto_timing_control	1150
place.coarse.balance_registers	1150
place.coarse.channel_detect_mode	1151
place.coarse.clock_gate_latency_aware	1152
place.coarse.cong_restruct	1153
place.coarse.cong_restruct_depth_aware	1153

Contents

place.coarse.cong_restruct_effort	1154
place.coarse.cong_restruct_iterations	1155
place.coarse.cong_restruct_strategy	1156
place.coarse.congestion_analysis_effort	1157
place.coarse.congestion_driven_max_util	1157
place.coarse.congestion_expansion_direction	1158
place.coarse.congestion_layer_aware	1159
place.coarse.continue_on_missing_scandef	1159
place.coarse.detect_detours	1160
place.coarse.dft_enable_at_speed_modeling	1161
place.coarse.enable_advanced_mv_cell_placement	1162
place.coarse.enable_dft_modeling	1162
place.coarse.enable_direct_congestion_mode	1163
place.coarse.enable_enhanced_soft_blockages	1164
place.coarse.enable_spare_cell_placement	1165
place.coarse.enhanced_low_power_effort	1166
place.coarse.enhanced_low_power_version	1167
place.coarse.extreme_power_mode	1167
place.coarse.fix_cells_on_soft_blockages	1168
place.coarse.fix_hard_macros	1169
place.coarse.fix_ios	1170
place.coarse.fix_soft_macros	1170
place.coarse.handle_early_data	1171
place.coarse.icg_auto_bound	1172
place.coarse.icg_auto_bound_fanout_limit	1172
place.coarse.increased_cell_expansion	1173
place.coarse.ir_enable_er	1174
place.coarse.ir_enable_ir	1174
place.coarse.ir_enable_ol	1175
place.coarse.ir_enable_pd	1176
place.coarse.ir_verbose	1176
place.coarse.irdp_cts_spacing	1177
place.coarse.max_density	1178
place.coarse.max_pins_per_square_micron	1179
place.coarse.ndr_area_aware	1179
place.coarse.perturbation_effort	1180
place.coarse.pin_cost_aware	1181
place.coarse.pin_density_aware	1182
place.coarse.power_density_effective_resistance_layer	1182

Contents

place.coarse.power_density_fast	1183
place.coarse.power_density_verbose	1183
place.coarse.seq_array_icg_aware	1184
place.coarse.spread_repeater_paths	1185
place.coarse.target_routing_density	1185
place.coarse.tns_driven_placement	1186
place.coarse.use_tall_blockages	1187
place.coarse.utilization_warning_threshold	1188
place.coarse.wide_cell_use_model	1188
place.common.pnet_aware_density	1189
place.common.pnet_aware_layers	1190
place.common.pnet_aware_min_width	1190
place.common.use_placement_model	1191
place.diode.enable_backside_layer_antenna_fix	1191
place.floorplan.density_aware_hard_movebounds	1192
place.floorplan.sliver_size	1192
place.legalize.access_check_enclosure_waiving_distance	1193
place.legalize.access_check_extra_layer	1193
place.legalize.advanced_ignore_horizontal_keepout_margin	1194
place.legalize.advanced_ignore_vertical_keepout_margin	1195
place.legalize.advanced_layer_access_check	1195
place.legalize.advanced_legalizer_auto_flow	1198
place.legalize.advanced_legalizer_hierarchical	1199
place.legalize.advanced_legalizer_verbose_flow	1200
place.legalize.advanced_libpin_access_check	1200
place.legalize.align_pin_only	1201
place.legalize.allow_tiecells_on_buffer_only_blockages	1202
place.legalize.allow_touch_track_for_access_check	1203
place.legalize.allow_unaligned_blockage	1203
place.legalize.always_continue	1204
place.legalize.auto_adjust_drc_rules_for_short_pin	1205
place.legalize.auto_disable_end_to_end_rules	1205
place.legalize.avoid_backside_pins_under_preroute_layers	1206
place.legalize.avoid_backside_pins_under_preroute_libpins	1206
place.legalize.avoid_backside_pins_under_preroute_libpins_under_net	1207
place.legalize.avoid_backside_pins_under_preroute_on_no_via_region_pin	1207
place.legalize.avoid_backside_pins_under_preroute_width_threshold	1208
place.legalize.avoid_pins_under_preroute_layers	1209
place.legalize.avoid_pins_under_preroute_libpins	1209

Contents

place.legalize.avoid_pins_under_preroute_libpins_under_net	1210
place.legalize.avoid_pins_under_preroute_on_no_via_region_pin	1211
place.legalize.avoid_pins_under_preroute_width_threshold	1211
place.legalize.backside_prerouted_net_check_exclude_lib_cells	1212
place.legalize.check_maximum_number_of_arrayed_cuts	1212
place.legalize.check_metal_dpt_odd_cycle_on_layers	1213
place.legalize.check_vt_min_area_for_multi_height_cells	1214
place.legalize.color_shift_layers	1214
place.legalize.consider_pmj_vl_user_routes	1215
place.legalize.constrain_shape_search_extent	1215
place.legalize.cross_row_vt_constraints	1216
place.legalize.disable_advanced_access_check_on_libcells	1217
place.legalize.disable_advanced_access_check_on_libpins	1218
place.legalize.disable_advanced_legalizer_rules_on_instcells	1219
place.legalize.disable_advanced_legalizer_rules_on_libcells	1220
place.legalize.disable_backside_drc_rules_on_libcells	1220
place.legalize.disable_drc_rules	1221
place.legalize.disable_drc_rules_in_access_check	1222
place.legalize.disable_drc_rules_in_multi_cell_check	1224
place.legalize.disable_drc_rules_on_libcells	1225
place.legalize.disable_ignore_unextended_access_shape	1226
place.legalize.drc_verbose_level	1226
place.legalize.enable_advanced_legalizer	1227
place.legalize.enable_advanced_legalizer_rules	1227
place.legalize.enable_advanced_legalizer_sanity_checker	1228
place.legalize.enable_advanced_prerouted_net_check	1228
place.legalize.enable_advanced_track_capacity_check	1229
place.legalize.enable_allowable_orient	1230
place.legalize.enable_backside_drc_rules_on_libcells	1230
place.legalize.enable_backside_pg_drc	1231
place.legalize.enable_build_cell_runtime_enhancement	1231
place.legalize.enable_category_blockages	1231
place.legalize.enable_cldr	1232
place.legalize.enable_clo_cpode_spacing_check	1233
place.legalize.enable_color_aware_placement	1233
place.legalize.enable_color_shifting	1234
place.legalize.enable_cut_metal_color_alignment	1235
place.legalize.enable_cutmetal_color_shifting	1236
place.legalize.enable_drc_rules	1237

Contents

place.legalize.enable_drc_rules_in_access_check	1238
place.legalize.enable_drc_rules_in_multi_cell_check	1239
place.legalize.enable_drc_rules_on_libcells	1241
place.legalize.enable_early_data_support	1242
place.legalize.enable_end_on_pref_grid_center	1242
place.legalize.enable_fast_via_ladder_mode	1243
place.legalize.enable_gui_drc_error_status_support	1243
place.legalize.enable_has_rectangle_only_rule	1244
place.legalize.enable_high_local_density_handling	1244
place.legalize.enable_incr_legalization_after_clo	1245
place.legalize.enable_legal_index_rules	1246
place.legalize.enable_movable_rectilinear_cells	1246
place.legalize.enable_multi_cell_access_check	1247
place.legalize.enable_multi_cell_check_on_libcell	1247
place.legalize.enable_multi_cell_pin_access_check	1248
place.legalize.enable_multi_cell_pnet_check	1248
place.legalize.enable_multi_cell_track_capacity_check	1249
place.legalize.enable_ndr_pin_check	1250
place.legalize.enable_non_default_via_def_access_check	1250
place.legalize.enable_non_preferred_direction_span_check	1251
place.legalize.enable_npldrc_pin_color_alignment_check	1251
place.legalize.enable_odS17_checking	1252
place.legalize.enable_odS17_fixing	1252
place.legalize.enable_pin_color_alignment_check	1253
place.legalize.enable_pin_via_region_check	1254
place.legalize.enable_prerouted_net_check	1255
place.legalize.enable_rectangular_od_shapes	1255
place.legalize.enable_safety_register_groups_dual_taps	1255
place.legalize.enable_same_net_pnet_check	1256
place.legalize.enable_sav_rule	1256
place.legalize.enable_threaded_advanced_legalizer	1257
place.legalize.enable_track_capacity_check	1257
place.legalize.enable_unspecified_boundary_rule_check	1258
place.legalize.enable_variant_aware	1259
place.legalize.enable_vertical_abutment_rules	1260
place.legalize.enable_via_dpt_odd_cycle	1260
place.legalize.enable_via_ladder_checks	1261
place.legalize.extend_preroute_shape_search_space	1261
place.legalize.filler1_continuous_vertical_cell_edge_for_exception_cells . . .	1262

Contents

place.legalize.filler_lib_cells	1263
place.legalize.force_clo_always_legal	1264
place.legalize.half_height_sitedef_name	1264
place.legalize.half_row_offset_cell_legalization	1265
place.legalize.half_row_offset_cell_sitedef_name	1266
place.legalize.high_local_density_threshold	1266
place.legalize.honor_router_pin_connect_option	1267
place.legalize.ignore_pin_color_alignment_width_threshold_on_layers	1267
place.legalize.ignore_site_rows	1268
place.legalize.ignore_unaligned_shapes_on_user_fixed_cells	1269
place.legalize.include_user_specified_via_ladder	1270
place.legalize.large_displacement_threshold	1270
place.legalize.legalize_only_selected_cells	1271
place.legalize.legalizer_search_and_repair	1271
place.legalize.libcell_based_color_shifting	1272
place.legalize.limit_legality_checks	1273
place.legalize.max_legality_failures	1273
place.legalize.min_max_prerouted_net_check_layers	1274
place.legalize.ndr_backside_pin_check_exclude_libpins	1275
place.legalize.ndr_pin_check_exclude_libpins	1275
place.legalize.num_tracks_for_access_check	1276
place.legalize.optimize_orientations	1276
place.legalize.optimize_pin_access	1277
place.legalize.optimize_pin_access_access_points	1277
place.legalize.optimize_pin_access_allow_row_swap	1278
place.legalize.optimize_pin_access_cells_verbose	1279
place.legalize.optimize_pin_access_drc_variants	1279
place.legalize.optimize_pin_access_ignore_lowest_routing_layer	1280
place.legalize.optimize_pin_access_incremental	1281
place.legalize.optimize_pin_access_same_row_variants	1281
place.legalize.optimize_pin_access_strategies	1282
place.legalize.optimize_pin_access_using_cell_spacing	1283
place.legalize.optimize_pin_access_verbose	1283
place.legalize.output_violated_info	1284
place.legalize.output_violated_info_file	1284
place.legalize.pg_blockage_specs	1285
place.legalize.pin_alignment_for_clock_mesh_layer	1286
place.legalize.pin_color_alignment_layers	1286
place.legalize.pin_color_alignment_width_threshold	1287

Contents

place.legalize.pin_color_alignment_width_threshold_on_layers	1288
place.legalize.post_route_feedback_threshold_for_pin_access	1289
place.legalize.preclump	1290
place.legalize.preclump_threshold_sites	1291
place.legalize.preroute_shape_merge_distance	1291
place.legalize.prerouted_net_check_exclude_lib_cells	1292
place.legalize.reduce_conservatism_in_eol_check	1293
place.legalize.report_displacement_limit	1293
place.legalize.report_libcell_displacement_limit	1294
place.legalize.return_tcl_errors	1295
place.legalize.revert_redundant_min_area_patching	1295
place.legalize.safety_enables_advanced_legalizer	1296
place.legalize.specify_clo_window_size_on_libcells	1296
place.legalize.standard_cell_sitedef_name	1298
place.legalize.stream_effort	1298
place.legalize.stream_effort_limit	1299
place.legalize.stream_place	1299
place.legalize.support_min_area_constraint	1300
place.legalize.support_off_track_via_region	1300
place.legalize.tap_cover_drop_edges	1301
place.legalize.use_eol_spacing_for_access_check	1302
place.legalize.use_nll_query_cm	1303
place.legalize.user_rule_check_layers	1303
place.legalize.user_rule_pattern_file	1304
place.legalize.via0_alignment_grid_name	1304
place.legalize.via_ladder_auto_stagger	1304
place.rp.allow_non_rp_cells	1305
place.rp.allow_non_rp_cells_on_blockages	1306
place.rp.disable_rp_legalization	1307
place.rp.disable_rp_two_pass_flow	1307
place.rp.enable_horizontal_cell_shifting	1308
place.rp.force_user_orients_only	1309
place.rp.hard_allow_non_rp_cells	1309
place.rp.hierarchical_place_around_fix_cells	1310
place.rules.min_od_filler_size	1311
place.rules.min_tpo_filler_size	1311
place.rules.min_vt_filler_size	1311
place.rules.vertical_abutment_mode	1312
place.utilization.multi_row_penalty	1312

Contents

place_opt.final_place.effort	1313
place_opt.flow.clock_aware_placement	1313
place_opt.flow.cts_icg_resynthesis	1314
place_opt.flow.dcg_spg_mode	1315
place_opt.flow.do_path_opt	1315
place_opt.flow.do_spg	1315
place_opt.flow.enable_ccd	1316
place_opt.flow.enable_ccd_fmax_flow	1317
place_opt.flow.enable_dpa_ccd_power	1317
place_opt.flow.enable_dpa_ccd_timing	1318
place_opt.flow.enable_dpa_ccd_timing_with_delay_potential	1318
place_opt.flow.enable_dps	1319
place_opt.flow.enable_fast_cus_in_final_opto	1319
place_opt.flow.enable_incremental_multibit_banking	1319
place_opt.flow.enable_irap	1320
place_opt.flow.enable_multibit_banking	1320
place_opt.flow.enable_multibit_debanking	1321
place_opt.flow.enable_multibit_rewiring	1321
place_opt.flow.enable_multisource_clock_trees	1322
place_opt.flow.enable_power	1323
place_opt.flow.enable_self_gating	1323
place_opt.flow.estimate_clock_gate_latency	1324
place_opt.flow.hold_area_budgeting	1325
place_opt.flow.merge_clock_gates	1325
place_opt.flow.optimize_icgs	1326
place_opt.flow.optimize_icgs_critical_range	1327
place_opt.flow.optimize_icgs_use_timing_driven_splitting	1327
place_opt.flow.optimize_layers	1328
place_opt.flow.optimize_layers_critical_range	1328
place_opt.flow.optimize_layers_overlap_bins	1329
place_opt.flow.optimize_ndr	1329
place_opt.flow.optimize_ndr_critical_range	1330
place_opt.flow.optimize_ndr_max_nets	1330
place_opt.flow.remove_dr_shapes	1330
place_opt.flow.repeatability	1331
place_opt.flow.trial_clock_tree	1331
place_opt.initial_drc.global_route_based	1332
place_opt.initial_place.buffering_aware	1333
place_opt.initial_place.effort	1333

Contents

place_opt.initial_place.two_pass	1334
place_opt.place.congestion_effort	1334
plan.3dic.bump_alignment_threshold	1334
plan.3dic.bump_model	1335
plan.3dic.chip_enclosure	1336
plan.3dic.chip_horizontal_misalignment	1337
plan.3dic.chip_horizontal_spacing	1337
plan.3dic.chip_vertical_misalignment	1338
plan.3dic.chip_vertical_spacing	1339
plan.3dic.error_physical_contact	1339
plan.3dic.forbidden_spacing_of_connected_bump	1340
plan.3dic.forbidden_spacing_of_unconnected_bump	1341
plan.3dic.ignore_logic_net	1341
plan.3dic.max_bump_cluster_size	1342
plan.3dic.min_bump_shape_spacing	1343
plan.3dic.min_bump_zone_street_keepout_margin	1344
plan.3dic.min_chip_enclosure	1344
plan.3dic.min_chip_spacing	1345
plan.3dic.min_mim_pad_pad_via_probe_cell_spacing	1345
plan.3dic.min_mim_pad_pad_via_rv_spacing	1346
plan.3dic.min_mim_top_pad_via_pad_layer_spacing	1347
plan.3dic.min_mim_top_pad_via_plate_hole_spacing	1347
plan.3dic.min_mim_top_plate_hole_bottom_layer_width	1348
plan.3dic.min_mim_top_plate_hole_middle_1_layer_width	1348
plan.3dic.min_mim_top_plate_hole_top_layer_width	1349
plan.3dic.min_pad_pad_via_pad_layer_spacing	1349
plan.3dic.min_pad_pad_via_rv_spacing	1350
plan.3dic.min_probe_bumps_quad_spacing	1350
plan.3dic.min_routing_blockage_spacing	1351
plan.3dic.min_routing_shape_spacing	1352
plan.3dic.min_stitching_zone_spacing	1353
plan.3dic.min_top_pad_via_pad_layer_spacing	1353
plan.3dic.min_top_pad_via_rv_spacing	1354
plan.3dic.pattern_marker_cell_margin_bottom	1354
plan.3dic.pattern_marker_cell_margin_left	1355
plan.3dic.pattern_marker_cell_margin_right	1355
plan.3dic.pattern_marker_cell_margin_top	1356
plan.3dic.pattern_marker_lib_cell_names	1356
plan.3dic.probe_num_max	1357

Contents

plan.3dic.probe_num_min_discrete_power_ground	1357
plan.3dic.qprobe_min_orthogonal_pitch_sprobe	1358
plan.3dic.qprobe_min_pitch	1359
plan.3dic.qprobe_num_min	1359
plan.3dic.qprobe_pitch_ubump	1360
plan.3dic.sprobe_min_pitch_non_dummy_ubump_other	1360
plan.3dic.sprobe_min_pitch_non_dummy_ubump_square	1361
plan.3dic.sprobe_min_pitch_other	1361
plan.3dic.sprobe_min_pitch_square	1362
plan.3dic.sprobe_min_pitch_true_dummy_region	1362
plan.3dic.sprobe_min_pitch_true_dummy_ubump	1363
plan.auto_group.balance_area	1363
plan.auto_group.enable_feedthrough_aware_in_distributed	1364
plan.auto_group.enable_macro_balancing	1364
plan.auto_group.enable_weighted_result_grading	1365
plan.auto_group.execution_strategy	1365
plan.auto_group.flow_config_file	1366
plan.auto_group.generate_csv	1366
plan.auto_group.generate_stats_effort_level	1367
plan.auto_group.grade_results	1367
plan.auto_group.group_ncells_max	1368
plan.auto_group.group_npins_max	1369
plan.auto_group.groups_count_max	1369
plan.auto_group.macro_lib_cells	1370
plan.auto_group.multiple_results	1371
plan.auto_group.partition_name_prefix	1371
plan.auto_group.partition_only_cluster	1371
plan.auto_group.partition_size_increase_percentage_for_result_optimization	1372
plan.auto_group.partition_type	1372
plan.auto_group.physical_aware	1373
plan.auto_group.print_inter_partition_feedthroughs	1373
plan.auto_group.print_inter_partition_wirelength	1374
plan.auto_group.prioritize_factors	1374
plan.auto_group.remove_apd_file	1375
plan.auto_group.result_directory	1375
plan.auto_group.results_grading_criteria	1375
plan.auto_group.threshold_macro_count	1376
plan.budget.add_missing_feedthrough_clocks	1376
plan.budget.all_design_subblocks	1377

Contents

plan.budget.all_full_budgets	1378
plan.budget.allow_disconnected_gclocks	1380
plan.budget.allow_top_only_exceptions	1380
plan.budget.bbt_fixed_delay	1381
plan.budget.calculate_hold_budgets	1382
plan.budget.clip_unoptimized_values	1383
plan.budget.compute_rise_fall_latencies	1384
plan.budget.delay_margin_percent	1385
plan.budget.enable_data_check	1385
plan.budget.enable_tcm_flow	1386
plan.budget.estimate_timing_mode	1386
plan.budget.extend_paths	1387
plan.budget.hold_buffer_margin	1388
plan.budget.include_all_clock_inputs	1389
plan.budget.include_feedthrough_clocks	1390
plan.budget.minimize_virtual_clocks	1391
plan.budget.minimum_unfixed_segment_delay	1391
plan.budget.pessimistic_driving_cells	1392
plan.budget.remove_unused_virtual_clocks	1393
plan.budget.segment_fixed_delay_threshold	1394
plan.budget.split_latencies	1394
plan.budget.time_with_margins	1395
plan.budget.use_ccd_latency	1396
plan.budget.write_ocv_files	1397
plan.budget.write_via_variation_files	1397
plan.busplan.auto_bus_creation_min_distance	1398
plan.busplan.auto_bus_creation_min_width	1398
plan.busplan.channels_include_hm	1399
plan.busplan.channels_include_placement_blockage	1400
plan.busplan.load_all_HMs	1400
plan.busplan.max_fanout	1401
plan.busplan.movebound_aspect_ratio	1402
plan.busplan.movebound_utilization	1402
plan.busplan.routing_corridor_pitch	1403
plan.busplan.routing_corridor_pitch_multiplier	1404
plan.busplan.use_soft_movebounds	1404
plan.busplan.use_timing	1405
plan.busplan.write_bundles	1406
plan.busplan.write_max_layer	1407

Contents

plan.busplan.write_min_layer	1407
plan.busplan.write_routing_corridors	1408
plan.clock_trunk.block_fanout_limit	1409
plan.clock_trunk.default_driving_cell	1409
plan.clock_trunk.enable_mph_constraints	1411
plan.clock_trunk.enable_new_flow	1411
plan.clock_trunk.endpoint_limit	1412
plan.clock_trunk.fanout_limit	1413
plan.clock_trunk.flow_control	1413
plan.clock_trunk.report_truncation_limit	1414
plan.clock_trunk.seed_clock_pin_placement	1414
plan.clock_trunk.topo_constr_pin_place_script_name	1415
plan.commit.inner_keepout_margin	1415
plan.commit.inner_keepout_margin_size	1416
plan.commit.outer_keepout_margin	1417
plan.commit.outer_keepout_margin_size	1418
plan.dff.keep_path_detail	1418
plan.distributed_run.block_host	1419
plan.distributed_run.block_priority	1420
plan.distributed_run.enable_rbs_improvements	1421
plan.distributed_run.gui_wait_on_fail_timeout	1422
plan.distributed_run.heartbeat_interval_time	1423
plan.distributed_run.lib_cell_purpose_list	1424
plan.distributed_run.main_is_idle	1424
plan.distributed_run.priority_dependencies	1425
plan.distributed_run.priority_size	1426
plan.distributed_run.reuse_rbs_workers	1427
plan.distributed_run.set_pvt_configuration	1428
plan.distributed_run.watchdog_timer_delay	1429
plan.distributed_run.watchdog_timer_enabled	1430
plan.distributed_run.watchdog_timer_interval	1431
plan.estimate_timing.abstract_add_feedthrough_buffer	1432
plan.estimate_timing.abstract_feedthrough_buffer_side	1433
plan.estimate_timing.abstract_feedthrough_logical	1433
plan.estimate_timing.abstract_feedthrough_user_buffer	1434
plan.estimate_timing.enable_fast_ml	1434
plan.estimate_timing.enable_ml_cell_sizing	1435
plan.estimate_timing.enable_multi_threading	1435
plan.estimate_timing.honor_routes	1436

Contents

plan.estimate_timing.maximum_cell_delay	1437
plan.estimate_timing.maximum_net_delay	1437
plan.estimate_timing.ml_model_dir	1438
plan.estimate_timing.optimize_layers	1439
plan.estimate_timing.optimize_layers_count	1439
plan.estimate_timing.use_ml_model	1440
plan.explore.default_threshold	1440
plan.explore.place_empty_modules	1441
plan.floorplan.core_area_accommodate_macro	1441
plan.floorplan.enable_adjust_core_area_for_hybrid_design	1442
plan.floorplan_rule.derive_keepout_behavior	1442
plan.floorplan_rule.fill_in_partial_row	1443
plan.floorplan_rule.pre_opt_switch	1444
plan.floorplan_rule.remove_existing_blockages	1445
plan.flow.design_view_only	1445
plan.flow.initialize_floorplan_fix_core_area_from_lowestleft	1446
plan.flow.override_work_dir	1447
plan.flow.segment_rule	1447
plan.flow.target_site_def	1448
plan.flow.track_pattern_support_prf_based_initialize_floorplan_for_site_def	1449
plan.interconnection.enforce_repeater_hiers	1449
plan.interconnection.exclusive	1450
plan.interconnection.keep_polarity_within_block	1451
plan.macro.align_pins_on_parallel_edges	1451
plan.macro.apply_spacing_rules_to_boundary	1452
plan.macro.auto_buffer_channels	1453
plan.macro.auto_macro_array_max_height	1454
plan.macro.auto_macro_array_max_num_cols	1455
plan.macro.auto_macro_array_max_num_rows	1455
plan.macro.auto_macro_array_max_width	1456
plan.macro.auto_macro_array_minimize_channels	1457
plan.macro.auto_macro_array_size	1458
plan.macro.auto_pin_blockages_min_pins	1458
plan.macro.auto_pin_blockages_width	1459
plan.macro.buffer_channel_height	1460
plan.macro.buffer_channel_width	1461
plan.macro.buffer_rule_name	1461
plan.macro.congestion_iters	1462
plan.macro.conservative_auto_pin_blockage_size	1463

Contents

plan.macro.create_auto_pin_blockages	1464
plan.macro.create_temporary_pg_grid	1464
plan.macro.cross_block_connectivity_planning	1465
plan.macro.estimate_pg	1466
plan.macro.floorplan_finishing	1467
plan.macro.grid_error_behavior	1469
plan.macro.grouping_by_hierarchy	1470
plan.macro.hierarchy_area_threshold	1470
plan.macro.immediately_buffer_ios	1471
plan.macro.integrated	1472
plan.macro.integrated_incremental	1473
plan.macro.legalize_effort	1474
plan.macro.macro_place_only	1475
plan.macro.macros_on_edge	1475
plan.macro.mark_auto_pin_blockages_as_pin_protect	1476
plan.macro.max_buffer_stack_height	1477
plan.macro.max_buffer_stack_width	1478
plan.macro.max_group_area_percentage	1478
plan.macro.min_macro_keepout	1479
plan.macro.ml_apply_final_opto	1480
plan.macro.ml_apply_upto_initial_opto	1480
plan.macro.ml_create_auto_pin_blockages	1481
plan.macro.ml_data_driven	1482
plan.macro.ml_data_with_spacing_rule	1482
plan.macro.ml_detail_summary	1483
plan.macro.ml_discard_congested_results	1483
plan.macro.ml_distribute_evenly	1484
plan.macro.ml_ffm_power_optimization	1485
plan.macro.ml_honor_voltage_areas	1485
plan.macro.ml_keep_optimization_setting	1486
plan.macro.ml_mode_weight	1486
plan.macro.ml_place_existing_floorplan	1487
plan.macro.ml_place_opt_setting	1488
plan.macro.ml_use_advanced_concav_blockage	1488
plan.macro.ml_use_concav_blockage	1489
plan.macro.ml_use_existing_floorplan	1489
plan.macro.pin_shape_track_match_threshold	1490
plan.macro.remove_auto_pin_blockages	1491
plan.macro.remove_temporary_pg_grid	1492

Contents

plan.macro.spacing_rule_heights	1492
plan.macro.spacing_rule_widths	1494
plan.macro.style	1495
plan.mtcmos.allow_legalizer_pin_color_alignment	1496
plan.mtcmos.allow_partial_power_strap_overlap	1497
plan.mtcmos.allow_power_switch_outside_core	1497
plan.mtcmos.honor_min_vt_spacing_rule	1498
plan.mtcmos.honor_pin_color_alignment	1498
plan.mtcmos.ignore_all_blockages	1499
plan.mtcmos.mark_supply_net_for_filler_cells	1499
plan.mtcmos.ring_pattern_cell_orientation	1500
plan.mtcmos.snap_after_placement	1501
plan.mtcmos.snap_to_legal_index	1502
plan.mtcmos.snap_to_siterow_orient	1502
plan.mtcmos.snap_to_via0_grid	1503
plan.mtcmos.treat_keepout_as_blockage	1504
plan.mtcmos.treat_voltage_area_as_blockage	1504
plan.outline.dense_module_depth	1505
plan.outline.glue_cell_count	1506
plan.outline.keep_port_depth	1507
plan.outline.large_threshold	1507
plan.outline.message_limit	1508
plan.outline.partition	1509
plan.outline.target_block_size	1510
plan.outline.target_cell_count	1510
plan.path_analysis.enable_timing	1511
plan.pgcheck.check_non_routing_layer_pins	1512
plan.pgcheck.check_rdl_routing	1512
plan.pgcheck.check_signal_ports	1513
plan.pgcheck.fast_checking	1513
plan.pgcheck.ignore_floating_on_layers	1513
plan.pgcheck.ignore_unfixed_std_cells	1514
plan.pgcheck.max_checking_level	1514
plan.pgcheck.return_isolated_object_only	1515
plan.pgcheck.treat_terminal_as_voltage_source	1515
plan.pgcheck.use_pin_as_bridge	1516
plan.pgcheck.via_site_threshold	1516
plan.pgroute.always_align_track_color_layers	1517
plan.pgroute.assign_drc_color_mode	1518

Contents

plan.pgroute.auto_connect_pg_net	1519
plan.pgroute.color_metal_cut_internally_for_drc	1519
plan.pgroute.connect_user_route_shapes	1520
plan.pgroute.create_via_cellalign_maxpin_only	1521
plan.pgroute.decide_rail_width_from_routing_area	1521
plan.pgroute.default_tag	1522
plan.pgroute.derive_cut_net_from_pin	1523
plan.pgroute.derive_pg_mask_engine_version	1523
plan.pgroute.disable_floating_removal	1524
plan.pgroute.disable_stapling_via_fixing	1524
plan.pgroute.disable_strategy_reevaluation	1525
plan.pgroute.disable_trimming	1526
plan.pgroute.disable_via_creation	1526
plan.pgroute.discard_stackvia_with_drc	1527
plan.pgroute.drc_check_fast_mode	1527
plan.pgroute.enable_rdl_45_shapes	1528
plan.pgroute.fix_via_drc_multiple_viadev	1529
plan.pgroute.high_capacity_mode	1530
plan.pgroute.hmpin_connection_target_layers	1531
plan.pgroute.honor_cover_cell	1531
plan.pgroute.honor_ndr_same_net_spacing	1532
plan.pgroute.honor_ndr_spacing_drc	1532
plan.pgroute.honor_signal_route_drc	1533
plan.pgroute.honor_std_cell_drc	1534
plan.pgroute.load_frame_view	1534
plan.pgroute.max_extension_distance	1535
plan.pgroute.max_undo_steps	1535
plan.pgroute.maximize_total_cut_area	1536
plan.pgroute.maximum_cell_gap_for_alignment_strap	1537
plan.pgroute.merge_shapes_for_via_creation	1538
plan.pgroute.merge_shapes_in_pad_cell	1538
plan.pgroute.minimum_layer_length	1539
plan.pgroute.no_patch_fix_rectangle_only_rule	1540
plan.pgroute.optimize_track_alignment	1540
plan.pgroute.overlap_route_boundary	1541
plan.pgroute.overlap_route_boundary_exclude_design_boundary	1541
plan.pgroute.siterow_name	1542
plan.pgroute.skip_balance_tree	1542
plan.pgroute.snap_stdcell_rail	1542

Contents

plan.pgroute.stapling_via_fast_mode	1543
plan.pgroute.std_rail_from_refname	1544
plan.pgroute.treat_cell_type_as_macro	1544
plan.pgroute.treat_fat_blockage_as_fat_metal	1545
plan.pgroute.treat_fixed_stdcell_as_macro	1546
plan.pgroute.treat_multiple_pins_as_one_target	1546
plan.pgroute.trim_by_blockage	1547
plan.pgroute.use_fallback_spacing_rules	1547
plan.pgroute.use_shape_pattern	1548
plan.pgroute.use_via_matrix	1549
plan.pgroute.verbose	1549
plan.pgroute.via_array_size_control	1550
plan.pgroute.via_site_threshold	1550
plan.pins.align_to_buffer_on_feedthrough	1551
plan.pins.allow_feedthroughs_for_internal_channel_nets	1552
plan.pins.allow_mixed_feedthroughs	1553
plan.pins.allow_pins_in_narrow_macro_channels	1554
plan.pins.allow_routing_over_readonly_blocks	1554
plan.pins.black_box_design_style	1555
plan.pins.buffer_cell_module_name	1556
plan.pins.create_double_pattern_long_pin	1556
plan.pins.detour_supernet_mode	1557
plan.pins.enable_color_span_rules	1558
plan.pins.enable_feedthrough_timestamps	1558
plan.pins.enable_span_rules	1559
plan.pins.exclude_clocks_from_feedthroughs	1560
plan.pins.exclude_inout_nets_from_feedthroughs	1560
plan.pins.fast_route	1561
plan.pins.force_net_port_name_match	1562
plan.pins.ignore_pin_spacing_constraint_to_pg	1563
plan.pins.incremental	1563
plan.pins.layer_range	1564
plan.pins.maximize_black_box_pin_grouping	1565
plan.pins.minimize_feedthroughs	1566
plan.pins.new_cell_name_tag	1567
plan.pins.new_port_name_tag	1567
plan.pins.new_port_name_tag_style	1568
plan.pins.new_split_nets_use_block_names	1569
plan.pins.path_based_pin_placement	1570

Contents

plan.pins.pin_range	1571
plan.pins.reserved_channel_threshold	1571
plan.pins.retain_bus_name	1573
plan.pins.strict_alignment	1574
plan.pins.synthesize_abutted_pins	1575
plan.pins.synthesize_abutted_pins_name_tag	1576
plan.pins.synthesize_abutted_pins_name_tag_style	1577
plan.pins.treat_pg_as_shield	1578
plan.place.auto_create_blockage_channel_heights	1579
plan.place.auto_create_blockage_channel_widths	1580
plan.place.auto_create_blockages	1581
plan.place.auto_create_blockages_enable_default_va	1583
plan.place.auto_create_hard_keepout	1584
plan.place.auto_generate_blockages	1584
plan.place.auto_generate_blockages_pad_rectilinear	1585
plan.place.auto_generate_blockages_smooth_rectilinear	1586
plan.place.auto_generate_hard_blockage_channel_width	1587
plan.place.auto_generate_partial_blockage_horizontal_channel_width	1587
plan.place.auto_generate_partial_blockage_vertical_channel_width	1588
plan.place.auto_generate_soft_blockage_channel_width	1588
plan.place.auto_max_density	1588
plan.place.congestion_driven_mode	1589
plan.place.default_keepout	1590
plan.place.hierarchy_by_name	1591
plan.place.max_utilization_for_auto_blockages	1592
plan.place.poly_rule	1592
plan.place.poly_rule_vertical_edge_expanding	1593
plan.place.timing_driven_mode	1593
plan.place.trace_mode	1594
plan.pna.ignore_objects_below_layer	1595
plan.pna.power_switch_resistance	1595
plan.shadow.exclude_non_hierarchical_objects	1596
plan.shadow.exclude_non_shadow_buffers	1597
plan.shaping.import_tcl_shaping_constraints	1597
plan.shaping.new_mib_abutted_flow	1598
plan.shaping.report_import_constraints	1598
plan.shaping.snap_to_boundary_grid	1599
power.annotate_repeater_tree	1600
power.built_in_name_mapping_mode	1600

Contents

power.clock_network	1601
power.clock_network_include_clock_gating_network	1602
power.clock_network_include_clock_sink_pin_power	1603
power.default_static_probability	1604
power.default_toggle_rate	1604
power.default_toggle_rate_reference_clock	1605
power.enable_activity_persistency	1606
power.enable_ignore_create_generated_clock	1607
power.enable_nldm_cap	1608
power.enhanced_sdpd_power_calculation	1608
power.get_power_clock_scaling_single_objects_only	1609
power.get_switching_activity_format	1610
power.handle_clock_network_power_as_ideal	1611
power.idpp_use_user_map_only	1611
power.ignore_scan_out_activity	1612
power.ignore_unconnected_driver	1612
power.include_hard_macro_input_pin_annotation	1613
power.include_ideal_network_swcap	1614
power.include_memory_input_pin_annotation	1614
power.include_primary_input_swcap	1615
power.indesign_primepower_filter_x	1615
power.indesign_primepower_use_capp	1616
power.input_pin_probability_scaling	1616
power.internal_mode	1617
power.leakage_mode	1617
power.power_annotation_persistency	1618
power.propagation_effort	1619
power.ptpx_saif_map_get_all_driver_mappings	1620
power.ptpx_saif_map_include_register_output_pin	1620
power.report_user_power_groups	1621
power.saif_glitch_handling	1622
power.saif_map_disable_rtl_name	1622
power.scale_dynamic_power_at_power_off	1623
power.scale_leakage_power_at_power_off	1623
power.swcap_mode	1624
power.swcap_power_capacitance	1625
power.table_extrapolation	1625
power.timer_implied_lower_than_propagated	1626
power.use_c1cn_pin_capacitance	1626

Contents

power.use_ccs_rcv_cap	1627
power.use_enhanced_multidrive_net_driver_type	1628
power.use_generated_clock_scaling_factor	1628
preroute.common.mv_isolation	1629
q	1630
query_objects_format	1630
r	1631
rail.3dic_generate_collaterals_distributed	1631
rail.3dic_generate_collaterals_incremental	1632
rail.3dic_generate_unique_die_only	1632
rail.allow_redhawk_license_checkout	1633
rail.analysis_type	1633
rail.apl_files	1634
rail.cell_subckt_files	1635
rail.custom_script_variables	1636
rail.database	1637
rail.def_files	1637
rail.display_eco_shapes	1638
rail.dump_icc2_results_style	1639
rail.effective_resistance_instance_file	1640
rail.em_only_tech_file	1641
rail.em_temperature	1642
rail.enable_3dic	1642
rail.enable_early_emir	1643
rail.enable_eff_vd_flow	1643
rail.enable_hold_analysis	1644
rail.enable_missing_cells_report	1644
rail.enable_new_rail_scenario	1645
rail.enable_node_voltage	1645
rail.enable_parallel_run_rail_scenario	1646
rail.enable_rail_probes_collision_test	1647
rail.enable_save	1647
rail.enable_scheduler_window	1648
rail.enable_setup_analysis	1648
rail.enable_write_spef_bg	1648
rail.extra_gsr_option_file	1649
rail.frequency	1650
rail.generate_file	1650

Contents

rail.generate_file_type	1651
rail.generate_file_variables	1652
rail.gpd_dir	1652
rail.ignore_filler_cells	1653
rail.instance_power_file	1654
rail.instance_voltage_max_lines	1654
rail.ipf_leakage_data_precedence	1655
rail.layer_voltage_max_lines	1656
rail.lef_files	1656
rail.lib_files	1657
rail.macro_models	1658
rail.nets	1659
rail.node_voltage_max_lines	1659
rail.node_voltage_sort_fields	1660
rail.pad_files	1660
rail.product	1661
rail.rail_probes_epsilon	1661
rail.redhawk_path	1662
rail.redhawk_sc_simulation_cycles	1663
rail.result_file	1663
rail.scenario_name	1664
rail.search_distance_x	1665
rail.search_distance_y	1665
rail.sigma_av_drop_threshold	1666
rail.sigma_dvd_top_n_victims	1666
rail.spef_files	1667
rail.sta_file	1668
rail.switch_model_files	1668
rail.switching_activity	1669
rail.tech_file	1670
rail.technology	1671
rail.temperature	1672
rail.toggle_rate	1673
rail.write_def_cell_as_via	1673
rail.write_def_flatten_lib_cells	1674
refine_opt.flow.clock_aware_placement	1675
refine_opt.flow.do_path_opt	1676
refine_opt.flow.exclusive	1676
refine_opt.flow.optimize_layers	1677

Contents

refine_opt.flow.optimize_layers_critical_range	1677
refine_opt.flow.optimize_layers_overlap_bins	1677
refine_opt.flow.optimize_ndr	1678
refine_opt.flow.optimize_ndr_critical_range	1678
refine_opt.flow.optimize_ndr_max_nets	1679
refine_opt.flow.optimize_rp_groups	1679
refine_opt.flow.repeatability	1680
refine_opt.hold.effort	1680
refine_opt.path_opt.fixed_cell_disturbance	1680
refine_opt.place.congestion_effort	1681
refine_opt.place.effort	1681
report.safety.enable_physical_checks_in_rtl	1682
route.auto_via_ladder.allow_neighbor_ripup_in_dense_area	1682
route.auto_via_ladder.allow_patching	1683
route.auto_via_ladder.allow_samenet_intersection	1684
route.auto_via_ladder.allow_via_array_as_single_cut	1685
route.auto_via_ladder.apply_app_options_to_insert_via_ladders_command	1687
route.auto_via_ladder.auto_stagger	1688
route.auto_via_ladder.auto_stagger_for_em_via_ladder_max_number_stagger_tracks_per_level	1689
route.auto_via_ladder.auto_stagger_max_number_stagger_tracks_per_level	1690
route.auto_via_ladder.bias_top_layer_to_samenet_connection	1691
route.auto_via_ladder.check_drcs_on_existing_via_ladders	1693
route.auto_via_ladder.check_drcs_on_existing_via_ladders_in_route_eco_only	1694
route.auto_via_ladder.clean	1696
route.auto_via_ladder.connect_within_metal	1697
route.auto_via_ladder.connect_within_metal_for_em_via_ladder	1698
route.auto_via_ladder.connect_within_metal_for_via_ladder	1699
route.auto_via_ladder.connect_within_pin_mode	1701
route.auto_via_ladder.default_width_ndr_promotable_on_nonreserved_tracks	1702
route.auto_via_ladder.enable_em_to_performance_via_ladder_update	1703
route.auto_via_ladder.generate_center_track_on_off_grid_pattern_must_join_shapes	1705
route.auto_via_ladder.generate_track_width_by_layer_name	1706
route.auto_via_ladder.ignore_min_max_layer_constraints	1709
route.auto_via_ladder.ignore_routing_shape_drcs	1710
route.auto_via_ladder.misalign_adjacent_pattern_must_join	1711
route.auto_via_ladder.ndr_mode	1712
route.auto_via_ladder.pattern_must_join_max_number_stagger_tracks	1713
route.auto_via_ladder.pattern_must_join_over_pin_layer	1714
route.auto_via_ladder.pin_access_aware	1715

Contents

route.auto_via_ladder.relax_pin_layer_metal_spacing_rules	1716
route.auto_via_ladder.report_all_via_ladder_violations	1717
route.auto_via_ladder.report_all_via_ladders	1718
route.auto_via_ladder.report_via_ladder_in_area_with_drc_threshold	1720
route.auto_via_ladder.shift_vias_on_transition_layers	1721
route.auto_via_ladder.strictly_honor_cut_table	1722
route.auto_via_ladder.top_layer_to_samenet_min_nonzero_distance	1723
route.auto_via_ladder.update_during_route	1725
route.auto_via_ladder.update_on_route_eco_nets_only	1726
route.auto_via_ladder.update_on_route_group_nets_only	1726
route.auto_via_ladder.update_pattern_must_join_during_route	1727
route.auto_via_ladder.user_debug	1728
route.auto_via_ladder.verbose	1729
route.common.advance_node_timing_driven_mode	1730
route.common.allow_pg_as_shield	1731
route.common.check_shield	1732
route.common.clock_net_max_layer_mode	1733
route.common.clock_net_min_layer_mode	1734
route.common.clock_net_min_max_layer_distance_threshold	1735
route.common.clock_topology	1736
route.common.color_based_dpt_flow	1737
route.common.comb_distance	1738
route.common.concurrent_redundant_via_effort_level	1738
route.common.concurrent_redundant_via_mode	1739
route.common.connect_floating_shapes	1740
route.common.connect_within_pins_by_layer_name	1740
route.common.create_nets_for_floating_pins_pattern_must_join_via_ladder	1741
route.common.crosstalk_driven	1742
route.common.eco_fix_drc_in_changed_area_only	1743
route.common.eco_route_concurrent_redundant_via_effort_level	1744
route.common.eco_route_concurrent_redundant_via_mode	1744
route.common.eco_route_fix_existing_drc	1745
route.common.enable_auto_dirty_data_handling	1746
route.common.enable_explicit_cut_metal_generation	1746
route.common.enable_multi_thread	1747
route.common.exclude_routing_constraints_for_short_nets_threshold	1747
route.common.extra_nonpreferred_direction_wire_cost_multiplier_by_layer_name	1748
route.common.extra_preferred_direction_wire_cost_multiplier_by_layer_name	1749
route.common.extra_via_cost_multiplier_by_layer_name	1750

Contents

route.common.extra_via_off_grid_cost_multiplier_by_layer_name	1751
route.common.filter_redundant_via_mapping	1752
route.common.focus_scenario	1753
route.common.forbid_new_metal_by_layer_name	1753
route.common.freeze_layer_by_layer_name	1754
route.common.freeze_via_to_frozen_layer_by_layer_name	1755
route.common.global_backside_max_layer_mode	1756
route.common.global_backside_min_layer_mode	1757
route.common.global_max_layer_mode	1759
route.common.global_min_layer_mode	1760
route.common.high_resistance_flow	1761
route.common.ignore_var_spacing_to_blockage	1761
route.common.ignore_var_spacing_to_pg	1762
route.common.ignore_var_spacing_to_shield	1762
route.common.mark_clock_nets_minor_change	1763
route.common.min_edge_offset_for_macro_pin_connection_by_layer_name	1763
route.common.min_edge_offset_for_top_level_pin_connection_by_layer_name	1764
route.common.min_max_layer_distance_threshold	1765
route.common.min_shield_length_by_layer_name	1766
route.common.ndr_by_delta_voltage	1767
route.common.net_max_layer_mode	1768
route.common.net_max_layer_mode_soft_cost	1769
route.common.net_min_layer_mode	1770
route.common.net_min_layer_mode_soft_cost	1771
route.common.number_of_secondary_pg_pin_connections	1771
route.common.number_of_vias_over_global_backside_max_layer	1772
route.common.number_of_vias_over_global_max_layer	1773
route.common.number_of_vias_over_net_max_layer	1773
route.common.number_of_vias_under_global_backside_min_layer	1774
route.common.number_of_vias_under_global_min_layer	1775
route.common.number_of_vias_under_net_min_layer	1775
route.common.pg_shield_distance_threshold	1776
route.common.post_detail_route_fix_soft_violations	1777
route.common.post_detail_route_redundant_via_insertion	1777
route.common.post_eco_route_fix_soft_violations	1778
route.common.post_group_route_fix_soft_violations	1778
route.common.post_incremental_detail_route_fix_soft_violations	1779
route.common.post_route_timing_driven_mode	1779
route.common.rc_driven_setup_effort_level	1780

Contents

route.common.redundant_via_exclude_weight_group_by_layer_name	1780
route.common.redundant_via_include_weight_group_by_layer_name	1781
route.common.relax_soft_spacing_outside_min_max_layer	1783
route.common.reroute_clock_shapes	1783
route.common.reroute_user_shapes	1784
route.common.reshield_modified_nets	1784
route.common.rotate_default_vias	1785
route.common.route_soft_rule_effort_level	1786
route.common.route_top_boundary_mode	1787
route.common.routing_rule_effort_level	1787
route.common.separate_tie_off_from_secondary_pg	1789
route.common.shielding_nets	1790
route.common.single_connection_to_pins	1791
route.common.soft_rule_weight_to_effort_level_map	1791
route.common.support_staple_vias_during_routing	1793
route.common.threshold_noise_ratio	1794
route.common.tie_off_mode	1794
route.common.track_auto_fill	1795
route.common.treat_all_cover_cells_as_lib_cells_for_routing	1795
route.common.treat_via_array_as_big_via	1796
route.common.verbose_level	1796
route.common.via_array_mode	1797
route.common.via_on_grid_by_layer_name	1798
route.common.wide_macro_pin_as_fat_wire	1799
route.common.wire_on_grid_by_layer_name	1799
route.common.write_instance_via_color	1800
route.detail.allow_default_rule_nets_via_ladder_lower_layer_connection . . .	1801
route.detail.allow_ndr_nets_via_ladder_lower_layer_connection	1802
route.detail.antenna	1803
route.detail.antenna_fixing_preference	1803
route.detail.antenna_on_iteration	1805
route.detail.antenna_verbose_level	1805
route.detail.check_antenna_for_open_nets	1806
route.detail.check_antenna_on_diode_pins	1807
route.detail.check_antenna_on_pg	1807
route.detail.check_patchable_drc_from_fixed_shapes	1808
route.detail.check_pin_min_area_min_length	1809
route.detail.check_port_min_area_min_length	1809
route.detail.check_routes_frozen_mode_include_shape_with_attribute_locket	1810

Contents

route.detail.continue_after_large_design_rule_value_error	1811
route.detail.default_diode_protection	1811
route.detail.default_gate_size	1812
route.detail.default_nwell_size	1813
route.detail.default_port_external_antenna_area	1814
route.detail.default_port_external_gate_size	1814
route.detail.default_port_external_nwell_size	1815
route.detail.delete_redundant_diodes_during_routing	1816
route.detail.detail_route_special_design_rule_fixing_stage	1817
route.detail.diagonal_min_width	1818
route.detail.diode_insertion_mode	1819
route.detail.diode_libcell_names	1819
route.detail.diode_preference	1820
route.detail.drc_convergence_effort_level	1821
route.detail.eco_max_number_of_iterations	1822
route.detail.eco_route_special_design_rule_fixing_stage	1823
route.detail.eco_route_use_soft_spacing_for_timing_optimization	1824
route.detail.elapsed_time_limit	1824
route.detail.enable_fixing_selective_design_rule_violations_on_fixed_shapes	1825
route.detail.enable_ndr_tapering_on_voltage_rule	1826
route.detail.enable_separate_pgate_ngate_antenna_ratio_check	1827
route.detail.fat_metal_forbidden_pitch_effort_level	1828
route.detail.force_end_on_preferred_grid	1829
route.detail.force_max_number_iterations	1829
route.detail.generate_extra_off_grid_pin_tracks	1830
route.detail.generate_off_grid_feed_through_tracks	1831
route.detail.group_route_special_design_rule_fixing_stage	1831
route.detail.hop_layers_to_fix_antenna	1832
route.detail.ignore_contained_metal_for_needs_fat_contact_overlap_check	1833
route.detail.ignore_drc	1833
route.detail.ignore_self_fatness_for_fat_via_check	1835
route.detail.incremental_detail_route_special_design_rule_fixing_stage	1835
route.detail.insert_diodes_during_routing	1836
route.detail.insert_redundant_vias_layer_order_low_to_high	1837
route.detail.macro_pin_antenna_mode	1837
route.detail.max_antenna_fixing_pin_count_threshold	1838
route.detail.max_antenna_pin_count	1839
route.detail.merge_gates_for_antenna	1840
route.detail.min_max_layer_effort_level	1841

Contents

route.detail.ndr_layer_based_taper_distance_threshold	1841
route.detail.ndr_spacing_length_threshold_factor	1842
route.detail.optimize_partition_size_for_drc	1842
route.detail.optimize_tie_off_effort_level	1843
route.detail.optimize_wire_via_effort_level	1844
route.detail.pin_taper_mode	1845
route.detail.port_antenna_mode	1846
route.detail.post_process_special_design_rule_fixing_stage	1847
route.detail.repair_shorts_over_macros_effort_level	1847
route.detail.repair_shorts_over_macros_verbose	1848
route.detail.report_antenna_for_must_join_port_root	1849
route.detail.report_drc_totals_by_layer	1850
route.detail.report_drc_types_by_layer	1850
route.detail.report_hier_antenna_for_must_join_port_root	1851
route.detail.report_ignore_drc	1852
route.detail.report_soft_drc_separately	1854
route.detail.reuse_filler_locations_for_diodes	1855
route.detail.save_after_iterations	1856
route.detail.save_cell_prefix	1857
route.detail.set_new_diodes_of_clock_nets_as_application_fixed	1857
route.detail.shift_diode_locations_for_port_protection	1858
route.detail.skip_antenna_analysis_for_ground_nets	1858
route.detail.skip_antenna_analysis_for_nets	1859
route.detail.skip_antenna_analysis_for_tie_low_pins	1860
route.detail.skip_antenna_fixing_for_nets	1860
route.detail.timing_driven	1861
route.detail.top_layer_antenna_fix_threshold	1862
route.detail.topology_eco_effort_level	1863
route.detail.use_default_width_for_min_area_min_len_stub	1863
route.detail.use_lower_hierarchy_for_port_diodes	1864
route.detail.use_wide_wire_effort_level	1865
route.detail.use_wide_wire_to_input_pin	1866
route.detail.use_wide_wire_to_macro_pin	1866
route.detail.use_wide_wire_to_output_pin	1867
route.detail.use_wide_wire_to_pad_pin	1867
route.detail.use_wide_wire_to_port	1868
route.detail.user_defined_partition	1869
route.detail.var_spacing_to_same_net	1870
route.detail.via_ladder_upper_layer_via_connection_mode	1870

Contents

route.global.advance_node_timing_driven_effort	1871
route.global.avoid_blocked_unbufferable_macro_area_in_opt	1872
route.global.coarse_grid_refinement	1872
route.global.connect_pins_outside_routing_corridor	1873
route.global.crosstalk_driven	1874
route.global.custom_track_modeling_enhancement	1874
route.global.deterministic	1875
route.global.diode_insertion_dist_below_io_pin_layer	1876
route.global.double_pattern_utilization_by_layer_name	1876
route.global.eco_honor_target_dly	1877
route.global.effort_level	1878
route.global.enable_buffer_aware	1879
route.global.enable_clock_clustering	1880
route.global.enable_diode_insertion_for_io_pin	1880
route.global.enable_gr_graph_lock	1881
route.global.enhancement_on_non_preferred_direction_area	1882
route.global.exclude_blocked_gcells_from_congestion_report	1883
route.global.export_soft_congestion_maps	1883
route.global.extra_blocked_layer_utilization_reduction	1884
route.global.force_full_effort	1885
route.global.force_rerun_after_global_route_opt	1886
route.global.insert_gr_via_ladders	1887
route.global.interactive_multithread_mode	1887
route.global.macro_area_iterations	1888
route.global.macro_area_track_utilization	1888
route.global.macro_boundary_track_utilization	1889
route.global.macro_boundary_width	1890
route.global.macro_corner_track_utilization	1891
route.global.max_buffer_distance	1891
route.global.net_type_based_blockage_boundary_gcell_enhancement	1892
route.global.power_aware_routing	1893
route.global.preroute_connection_enhancement	1894
route.global.support_excl_range2	1894
route.global.timing_driven	1895
route.global.timing_driven_effort_level	1896
route.global.voltage_area_corner_track_utilization	1896
route.track.allow_wire_outside_gcell	1897
route.track.crosstalk_driven	1898
route.track.timing_driven	1898

Contents

route_opt.flow.enable_cbuf_density_fill_checks	1899
route_opt.flow.enable_cbuf_small_area_pass	1899
route_opt.flow.enable_ccd	1900
route_opt.flow.enable_ccd_clock_drc_fixing	1900
route_opt.flow.enable_clock_power_recovery	1901
route_opt.flow.enable_irdrivenopt	1902
route_opt.flow.enable_ldrc_driver_side_buffer_filters	1902
route_opt.flow.enable_ml_opto	1902
route_opt.flow.enable_multi_vector_ccd_pba_tns	1903
route_opt.flow.enable_multi_vector_ccd_pba_wns	1903
route_opt.flow.enable_multibit_debanking	1904
route_opt.flow.enable_new_pba_optimization	1904
route_opt.flow.enable_noise_closure	1904
route_opt.flow.enable_power	1905
route_opt.flow.enable_targeted_ccd_wns_optimization	1905
route_opt.flow.enable_voltage_drop_opt_ccd	1906
route_opt.flow.size_only_mode	1906
rtl_opt.clockgate.fanin_sequential	1907
rtl_opt.clockgate.self_gating	1910
rtl_opt.commit.enable_respective_lib	1912
rtl_opt.conditioning.disable_boundary_optimization_and_auto_ungrouping	1912
rtl_opt.conditioning.enable_constraint_name_tracking	1913
rtl_opt.conditioning.enable_initial_block_budgets	1914
rtl_opt.conditioning.feedthrough_percent_block_budgets	1914
rtl_opt.conditioning.ignore_repeater_for_logic_depth_budgets	1915
rtl_opt.conditioning.initial_percent_block_budgets	1915
rtl_opt.estimation.enable_dtdp	1916
rtl_opt.estimation.placement_congestion_effort	1916
rtl_opt.estimation.tieopt	1917
rtl_opt.explore_design.run_baseline	1917
rtl_opt.floorplan.enable_feedthroughs	1917
rtl_opt.floorplan.skip_stages	1918
rtl_opt.flow.enable_app_options_mapping	1919
rtl_opt.flow.enable_cts	1919
rtl_opt.flow.fully_abutted_style_floorplan	1919
rtl_opt.place.run_floorplan_mode_global_route	1920
rtlfp.flow.default_tip_flipflop	1920
rtlfp.flow.tip_rtl_mode	1921
s	1921

Contents

sdc_version	1921
selection_logging_no_core_action	1922
selection_logging_too_many_objects_action	1922
sh_allow_tcl_with_set_app_var	1923
sh_allow_tcl_with_set_app_var_no_message_list	1923
sh_arch	1924
sh_auto_complete_display_count	1924
sh_command_abbrev_mode	1924
sh_command_abbrev_options	1925
sh_command_log_file	1926
sh_continue_on_error	1926
sh_deprecated_is_error	1927
sh_dev_null	1928
sh_disabled_is_error	1928
sh_enable_page_mode	1928
sh_enable_stdout_redirect	1929
sh_help_shows_group_overview	1929
sh_language	1930
sh_new_variable_message	1930
sh_new_variable_message_in_proc	1931
sh_new_variable_message_in_script	1932
sh_obsolete_is_error	1934
sh_output_log_file	1935
sh_product_version	1935
sh_script_stop_severity	1936
sh_source_emits_line_numbers	1937
sh_source_logging	1938
sh_source_uses_search_path	1938
sh_tcllib_app_dirname	1939
sh_user_man_path	1939
shell.common.app_option_persistency	1940
shell.common.auto_sdp	1940
shell.common.auto_sdp_commands	1941
shell.common.auto_sdp_crte_timeperiod	1941
shell.common.auto_sdp_delete	1941
shell.common.auto_sdp_stack_trace_frequency	1942
shell.common.build_arch	1942
shell.common.check_error_list	1943
shell.common.collection_result_display_limit	1943

Contents

shell.common.copyright_year	1944
shell.common.enable_deterministic_mode	1945
shell.common.enable_line_editing	1945
shell.common.enable_quiet_on_filter	1946
shell.common.enable_summary_error	1947
shell.common.hyper_scale	1948
shell.common.implicit_find_mode	1948
shell.common.line_editing_mode	1949
shell.common.man_path	1950
shell.common.message_style	1950
shell.common.product_abbreviation	1951
shell.common.product_base_build_date	1952
shell.common.product_build_date	1952
shell.common.product_name	1953
shell.common.product_version	1953
shell.common.rename_program_name	1954
shell.common.report_default_significant_digits	1954
shell.common.sh_language	1955
shell.common.shell_abbreviation	1955
shell.common.shell_mode	1956
shell.common.shell_name	1957
shell.common.single_line_messages	1957
shell.common.store_messages	1958
shell.common.tmp_dir_path	1958
shell.common.track_subprocess_memory	1959
shell.dc_compatibility.black_box_is_leaf_cell	1959
shell.dc_compatibility.enable_regex_behavior	1960
shell.dc_compatibility.remove_libcell_attribute	1961
shell.dc_compatibility.return_tcl_errors	1961
shell.synthesis.logic_level_report_group_format	1962
shell.synthesis.logic_level_report_summary_format	1964
shell.undo.enabled	1965
shell.undo.max_levels	1966
shell.undo.max_memory	1967
signoff.antenna.contact_layer	1967
signoff.antenna.contact_layers_between_m0_diffusion	1968
signoff.antenna.custom_runset_file	1969
signoff.antenna.diffusion_layer	1970
signoff.antenna.enabled	1971

Contents

signoff.antenna.exclude_non_mask_layers	1971
signoff.antenna.extract_via_antenna_property	1972
signoff.antenna.gate_class1_layers	1973
signoff.antenna.gate_class2_layers	1973
signoff.antenna.gate_class3_layers	1974
signoff.antenna.hierarchical_gate_class_include_file	1975
signoff.antenna.m0_layers_for_diffusion_connection	1976
signoff.antenna.m0_layers_for_poly_connection	1977
signoff.antenna.max_gate_class_name	1977
signoff.antenna.poly_layer	1978
signoff.antenna.report_diodes	1979
signoff.antenna.top_cell_pin_only	1980
signoff.antenna.treat_source_drain_as_diodes	1981
signoff.antenna.user_defined_options	1981
signoff.antenna.v0_layers_between_m1_m0	1982
signoff.calculate_hier_antenna_property.extract_via_antenna_prop	1983
signoff.calculate_hier_antenna_property.user_defined_options	1984
signoff.check_design.generate_runset_only	1985
signoff.check_design.max_errors_per_rule	1985
signoff.check_design.run_dir	1986
signoff.check_design.runset	1987
signoff.check_design.user_defined_options	1988
signoff.check_drc.auto_eco_threshold_value	1988
signoff.check_drc.enable_icv_explorer_mode	1989
signoff.check_drc.exclude_cells	1990
signoff.check_drc.excluded_cell_types	1991
signoff.check_drc.fill_view_data	1991
signoff.check_drc.ignore_blockages_in_cells	1992
signoff.check_drc.ignore_child_cell_errors	1993
signoff.check_drc.max_errors_per_rule	1994
signoff.check_drc.merge_base_error_name	1995
signoff.check_drc.merge_incremental_error_data	1996
signoff.check_drc.read_design_views	1997
signoff.check_drc.read_layout_views	1998
signoff.check_drc.run_dir	1999
signoff.check_drc.runset	2000
signoff.check_drc.user_defined_options	2000
signoff.check_drc_live.check_only_visible_layers	2001
signoff.check_drc_live.default_layers	2002

Contents

signoff.check_drc_live.exclude_command_class	2003
signoff.check_drc_live.halo	2004
signoff.check_drc_live.hierarchy_level	2005
signoff.check_drc_live.igraph	2005
signoff.check_drc_live.keep_enclosing_results	2006
signoff.check_drc_live.run_dir	2007
signoff.check_drc_live.runset	2008
signoff.color_macro.enabled	2008
signoff.color_macro.ignore_fill_view_time_stamp	2009
signoff.color_macro.input_gds	2010
signoff.color_macro.layer_map	2010
signoff.color_macro.macro_name	2011
signoff.color_macro.output_gds	2012
signoff.color_macro.preferred_routing_direction	2013
signoff.color_macro.rule_type	2014
signoff.color_macro.run_dir	2014
signoff.color_macro.technology_file	2015
signoff.color_macro.track_color_preference	2016
signoff.color_macro.track_offset	2016
signoff.create_metal_fill.apply_nondefault_rules	2017
signoff.create_metal_fill.auto_eco_threshold_value	2018
signoff.create_metal_fill.base_layer_runset	2019
signoff.create_metal_fill.fill_over_net_on_adjacent_layer	2019
signoff.create_metal_fill.fill_shielded_clock	2020
signoff.create_metal_fill.fix_density_errors	2021
signoff.create_metal_fill.flat	2022
signoff.create_metal_fill.max_density_threshold	2023
signoff.create_metal_fill.read_design_views	2024
signoff.create_metal_fill.read_layout_views	2025
signoff.create_metal_fill.run_dir	2026
signoff.create_metal_fill.runset	2027
signoff.create_metal_fill.space_to_clock_nets	2028
signoff.create_metal_fill.space_to_clock_nets_on_adjacent_layer	2029
signoff.create_metal_fill.space_to_nets	2030
signoff.create_metal_fill.space_to_nets_on_adjacent_layer	2031
signoff.create_metal_fill.tcd_fill	2032
signoff.create_metal_fill.user_defined_options	2033
signoff.create_pg_augmentation.adjacent_layer_timing_protection_mode . .	2034
signoff.create_pg_augmentation.enable_auto_set_width	2035

Contents

signoff.create_pg_augmentation.ground_net_name	2036
signoff.create_pg_augmentation.parameter_file_check_mode	2037
signoff.create_pg_augmentation.power_net_name	2038
signoff.create_pg_augmentation.run_dir	2038
signoff.create_pg_augmentation.timing_driven	2039
signoff.create_pg_augmentation.user_defined_options	2040
signoff.fix_drc.advanced_guidance_for_rules	2041
signoff.fix_drc.check_drc	2042
signoff.fix_drc.config_file	2042
signoff.fix_drc.custom_guidance	2043
signoff.fix_drc.dpt_rule_information	2044
signoff.fix_drc.fix_detail_route_drc	2045
signoff.fix_drc.init_drc_error_db	2046
signoff.fix_drc.last_run_full_chip	2047
signoff.fix_drc.max_detail_route_iterations	2048
signoff.fix_drc.max_errors_per_rule	2049
signoff.fix_drc.run_dir	2049
signoff.fix_drc.target_clock_nets	2050
signoff.fix_drc.user_defined_options	2051
signoff.fix_isolated_via.avoid_net_types	2052
signoff.fix_isolated_via.isolated_via_max_range	2053
signoff.fix_isolated_via.run_dir	2054
signoff.fix_isolated_via.user_defined_options	2054
signoff.physical.layer_map_file	2055
signoff.physical.merge_exclude_libraries	2056
signoff.physical.merge_layer_map_file	2057
signoff.physical.merge_stream_files	2058
signoff.physical.rename_cell_files	2059
signoff.report_metal_density.create_heat_maps	2060
signoff.report_metal_density.density_window	2061
signoff.report_metal_density.density_window_step	2061
signoff.report_metal_density.fill_view_data	2062
signoff.report_metal_density.gradient_window_size	2063
signoff.report_metal_density.min_density	2064
signoff.report_metal_density.run_dir	2065
signoff.report_metal_density.user_defined_options	2066
sim.fine_sim_path	2067
sim.hspice_path	2067
sim.wave_view_path	2068

Contents

stream.merge.multi_thread	2068
synopsys_program_name	2069
synopsys_root	2070
t	2070
tbcs.record.enable	2070
techMap.mapCikInsts	2071
thermal.adaptive_grid_resolution	2072
thermal.boundary_temperature	2072
thermal.carrier_order	2073
thermal.common.database	2074
thermal.common.temperature_unit	2074
thermal.compact_model_file	2075
thermal.component_dimension	2075
thermal.component_material	2076
thermal.component_thickness	2077
thermal.database	2077
thermal.engine	2078
thermal.grid_resolution	2078
thermal.heat_sink_order	2079
thermal.heat_transfer_rate	2080
thermal.hier_block_metal_profile	2080
thermal.hier_block_power_profile	2081
thermal.material_conductivity	2081
thermal.material_density	2082
thermal.material_heat_capacity	2083
thermal.metal_density	2083
thermal.metal_profile	2084
thermal.percentage_of_max_threshold	2084
thermal.power_layer	2085
thermal.power_profile	2086
thermal.tech_file	2086
thermal.threshold	2087
thermal.use_layout	2087
time.advanced_multi_input_switching_tied_input_mode	2088
time.all_clocks_propagated	2088
time.allow_port_latency_for_feedthrough	2089
time.ambiguous_checks_sequential_infer_pt_compatibility	2090
time.analyze_subcircuit_use_pulse_waveform	2091

Contents

time.analyze_subcircuit_use_real_pg_supply_name	2092
time.analyze_subcircuit_use_specified_voltage	2093
time.aocvm_analysis_mode	2093
time.aocvm_enable_analysis	2094
time.aocvm_enable_clock_network_only	2095
time.aocvm_enable_single_path_metrics	2095
time.awp_compatibility_mode	2096
time.case_analysis_propagate_through_icg	2096
time.case_analysis_sequential_propagation	2097
time.clock_gating_propagate_enable	2098
time.clock_gating_user_setting_only	2099
time.clock_marking	2100
time.clock_reconvergence_pessimism	2101
time.convert_constraint_from_bc_wc	2102
time.create_clock_no_input_delay	2103
time.crpr_different_transition_derate	2103
time.crpr_different_transition_variation_derate	2104
time.crpr_remove_clock_to_data_crp	2105
time.delay_calc_enhanced_ccsn_waveform_analysis	2106
time.delay_calc_waveform_analysis_constraint_arcs_compatibility	2107
time.delay_calc_waveform_analysis_mode	2107
time.delay_calculation_style	2108
time.deprioritize_io_path_groups	2109
time.disable_case_analysis	2110
time.disable_case_analysis_ti_hi_lo	2111
time.disable_clock_gating_checks	2112
time.disable_cond_default_arcs	2113
time.disable_flr_for_same_disable_object	2114
time.disable_internal_inout_net_arcs	2114
time.disable_recovery_removal_checks	2115
time.early_launch_at_latch_loop_breaker	2116
time.edge_specific_source_latency	2117
time.enable_auto_mux_clock_exclusivity	2118
time.enable_ccs_rcv_cap	2118
time.enable_clock_propagation_through_preset_clear	2119
time.enable_clock_propagation_through_three_state_enable_pins	2120
time.enable_clock_to_data_analysis	2120
time.enable_constraint_variation	2121
time.enable_cumulative_incremental_derate	2122

Contents

time.enable_incr_update_for_clock_balance_points	2122
time.enable_io_path_groups	2122
time.enable_low_accuracy_reports	2123
time.enable_ml_delay_calculation	2124
time.enable_multi_input_switching_analysis	2125
time.enable_multi_input_switching_timing_window_filter	2125
time.enable_netlist_constant_transition	2126
time.enable_new_exceptions_report	2127
time.enable_non_sequential_checks	2127
time.enable_preset_clear_arcs	2128
time.enable_propagation_for_noise	2128
time.enable_si_timing_windows	2129
time.enable_slew_variation	2130
time.enable_sps_delcalc	2130
time.enable_static_noise	2131
time.enable_thermal_analysis	2132
time.enable_via_variation	2133
time.estnet_blockage_aware	2134
time.filter_power_and_ground_terms	2134
time.frequency_based_max_cap	2135
time.gclock_source_network_num_master_registers	2136
time.high_fanout_net_pin_capacitance	2137
time.high_fanout_net_threshold	2138
time.ideal_clock_zero_default_transition	2139
time.max_exclusive_mux_levels	2139
time.max_parallel_computations	2140
time.max_uncompressed_net_drivers	2140
time.multi_input_switching_analysis_mode	2141
time.multi_input_switching_precedence	2142
time.nonlinear_scaling	2142
time.ocvm_enable_distance_analysis	2143
time.ocvm_precedence_compatibility	2144
time.open_block_exclude_file_line_info	2144
time.pba_derate_only_mode	2145
time.pba_exhaustive_endpoint_path_limit	2146
time.pba_optimization_mode	2147
time.pocvm_corner_sigma	2148
time.pocvm_enable_analysis	2148
time.pocvm_enable_constraint_variation	2149

Contents

time.pocvm_enable_delay_slew_correlation	2150
time.pocvm_enable_extended_moments	2150
time.pocvm_max_transition_clock_sigma	2151
time.pocvm_max_transition_sigma	2151
time.pocvm_precedence	2152
time.port_input_threshold_fall	2152
time.port_input_threshold_rise	2154
time.port_output_threshold_fall	2155
time.port_output_threshold_rise	2156
time.port_slew_derate_from_library	2158
time.port_slew_lower_threshold_fall	2159
time.port_slew_lower_threshold_rise	2160
time.port_slew_upper_threshold_fall	2162
time.port_slew_upper_threshold_rise	2163
time.propagate_interclock_uncertainty	2164
time.rc_receiver_modeling_effort	2165
time.remove_clock_reconvergence_pessimism	2166
time.reparent_enable_inherit_constraints	2167
time.report_through_paths_as_non_io	2168
time.report_timing_column_order	2168
time.report_unconstrained_paths	2169
time.report_with_clock_path_edrc	2170
time.rh_file_use_clock_arrival	2171
time.save_block_exclude_file_line_info	2172
time.save_mode_based_cts_exceptions	2172
time.scaling_calculation_compatibility	2173
time.scenario_auto_name_separator	2174
time.si_enable_analysis	2174
time.si_filter_per_aggr_noise_peak_ratio	2175
time.si_ignore_input_data_arrival	2176
time.si_xtalk_composite_aggr_mode	2176
time.si_xtalk_composite_aggr_noise_peak_ratio	2177
time.si_xtalk_composite_aggr_quantile_high_pct	2177
time.snapshot_storage_location	2178
time.special_path_group_precedence	2179
time.through_path_max_segments	2180
time.timing_report_always_use_valid_start_end_points	2180
time.ungroup_allow_new_buffer	2181
time.ungroup_enable_inherit_constraints	2182

Contents

time.use_lib_cell_generated_clock_name	2182
time.use_pt_delay	2183
time.use_pt_si	2184
time.use_slew_variation_in_constraint_arcs	2184
time.use_special_default_path_groups	2185
time.write_script_use_mode_based_cts_exceptions	2186
time.xtalk_use_small_xcap_filter	2187
top_level.concurrent_top_block_eco_routing_for_tho	2187
top_level.continue_flow_on_check_hier_design_errors	2188
top_level.enable_mib_optimize_subblocks	2188
top_level.tho_block_to_top_map_auto_clock	2189
v	2189
vl.flow.enable_convergence_flow	2189
vl.opt.enable_em_via_ladder	2190
vl.opt.enable_em_via_ladder_on_clock	2190
vl.opt.shrink_performance_vl	2190
w	2191
wildcards	2191
2. Fusion Compiler Attributes	2195
a	2195
alignment_marker_rule_attributes	2195
anchor_attributes	2200
annotation_point_attributes	2203
annotation_shape_attributes	2205
b	2207
block_attributes	2207
block_pin_constraint_attributes	2230
bond_pad_def_attributes	2233
bound_attributes	2234
bound_shape_attributes	2239
bounding_box_attributes	2240
budget_attributes	2242
budget_clock_attributes	2252
budget_path_type_attributes	2253
budget_pin_attributes	2255
budget_pin_constraint_attributes	2258

Contents

budget_pin_data_attributes	2260
budget_segment_attributes	2263
bump_bundle_attributes	2266
bump_bundle_pattern_attributes	2268
bump_region_attributes	2269
bump_region_pattern_attributes	2274
bundle_attributes	2276
busplan_bus_attributes	2278
busplan_element_attributes	2281
c	2284
cell_array_pattern_attributes	2284
cell_attributes	2285
cell_bus_attributes	2333
clock_attributes	2335
clock_balance_group_attributes	2338
clock_group_attributes	2338
clock_group_group_attributes	2340
clock_path_attributes	2340
constraint_group_attributes	2344
core_area_attributes	2350
corner_attributes	2352
curved_poly_rect_attributes	2353
d	2354
density_rule_attributes	2354
design_attributes	2356
design_checks_attributes	2372
design_rule_attributes	2374
dff_connection_object_attributes	2376
drc_error_attributes	2377
drc_error_data_attributes	2380
drc_error_type_attributes	2382
e	2384
early_data_check_record_attributes	2384
edit_group_attributes	2385
ems_check_attributes	2389
ems_database_attributes	2390
ems_group_attributes	2391
ems_message_attributes	2393

Contents

ems_rule_attributes	2393
exception_attributes	2394
exception_group_attributes	2398
f	2399
failsafe_fsm_group_attributes	2399
failsafe_fsm_rule_attributes	2401
fill_cell_attributes	2403
g	2405
geo_mask_attributes	2405
grid_attributes	2406
group_attributes	2407
gui_annotation_attributes	2409
gui_object_attributes	2411
h	2412
hier_tech_layer_attributes	2412
i	2417
input_delay_attributes	2417
io_guide_attributes	2419
io_ring_attributes	2422
k	2425
keepout_margin_attributes	2425
l	2427
layer_attributes	2427
lib_attributes	2432
lib_cell_attributes	2438
lib_pin_attributes	2463
lib_timing_arc_attributes	2482
m	2486
manufacturing_shape_attributes	2486
matching_type_attributes	2488
material_attributes	2489
metal_area_attributes	2491
metal_area_hole_attributes	2493
mismatch_object_attributes	2495
mismatch_type_attributes	2496
mode_attributes	2498
module_attributes	2499

Contents

n	2501
net_attributes	2501
net_bus_attributes	2514
net_estimation_rule_attributes	2515
o	2518
outline_attributes	2518
output_delay_attributes	2520
overlap_blockage_attributes	2521
p	2523
parasitic_tech_attributes	2523
path_group_attributes	2524
pg_region_attributes	2525
physical_constraint_attributes	2527
pin_attributes	2529
pin_blockage_attributes	2563
pin_bus_attributes	2566
pin_constraint_attributes	2567
pin_guide_attributes	2570
placement_affinity_attributes	2573
placement_attraction_attributes	2574
placement_blockage_attributes	2576
poly_rect_attributes	2579
port_attributes	2581
port_bus_attributes	2618
power_domain_attributes	2619
power_strategy_attributes	2620
pr_rule_attributes	2621
process_layer_attributes	2622
pseudo_bump_attributes	2624
pseudo_tsv_attributes	2628
r	2631
routing_blockage_attributes	2631
routing_corridor_attributes	2636
routing_corridor_shape_attributes	2638
routing_guide_attributes	2641
routing_rule_attributes	2644
rp_blockage_attributes	2648
rp_group_attributes	2650

Contents

rtl_cell_attributes	2654
rtl_cell_construct_attributes	2660
rtl_line_attributes	2666
rtl_module_attributes	2669
rtl_module_construct_attributes	2674
s	2680
safety_core_group_attributes	2680
safety_core_rule_attributes	2682
safety_error_code_group_attributes	2684
safety_error_code_rule_attributes	2686
safety_register_group_attributes	2687
safety_register_rule_attributes	2689
scenario_attributes	2691
shape_attributes	2693
shape_pattern_attributes	2701
shaping_blockage_attributes	2710
shaping_channel_attributes	2712
shaping_constraint_attributes	2714
site_array_attributes	2717
site_def_attributes	2720
site_row_attributes	2722
skew_group_attributes	2725
stub_chain_attributes	2727
supernet_attributes	2729
supply_net_attributes	2730
supply_port_attributes	2732
supply_set_attributes	2734
t	2736
tech_attributes	2736
tech_purpose_attributes	2739
terminal_attributes	2740
timing_arc_attributes	2744
timing_exception_attributes	2747
timing_path_attributes	2750
timing_point_attributes	2759
topological_constraint_attributes	2761
topology_edge_attributes	2762
topology_group_attributes	2766

Contents

topology_node_attributes	2767
topology_plan_attributes	2772
topology_repeater_attributes	2776
track_attributes	2777
u	2782
utilization_config_attributes	2782
v	2784
variation_attributes	2784
via_attributes	2785
via_def_attributes	2792
via_ladder_attributes	2797
via_matrix_attributes	2799
via_region_attributes	2808
via_rule_attributes	2810
voltage_area_attributes	2816
voltage_area_rule_attributes	2821
voltage_area_shape_attributes	2823

1

Fusion Compiler Application Options

This document describes the application options supported by the Fusion Compiler tool.

3

3dic.ai_flow.use_exhaustive_search

Specifies whether to use exhaustive search mode for parameter space exploration

Data Types

bool

Default false

Description

This application option controls the exhaustive search mode for the AI flow.

When set to *true*, the tool performs exhaustive parameter sweeping, evaluating all possible combinations of parameter values. This mode:

- Provides complete coverage of the design space
- Is useful for full design space characterization and sensitivity analysis
- Can be computationally expensive with many parameters or large value ranges
- Guarantees that all parameter combinations are evaluated

Set this option before running *run_3d_ai* command.

See Also

- [report_app_options](#)
- [set_app_options](#)

3dic.check_esd.main_runset

Specifies main runsets to use for programmable electrical rules check (PERC). Option for the *check_3d_esd* command.

Data Types

string pair list

Default null

Description

Specifies the main runset of each die block reference while the tool invokes IC Validator to perform programmable electrical rules checking.

Examples

The following example specifies the PERC main runset *esd_topo_runset1.rs* and *esd_topo_runset2.rs* for the *check_3d_esd* command.

```
prompt> set_app_options -name 3dic.check_esd.main_runset -value  
        { {logic_die esd_topo_runset1.rs} {mem_die esd_topo_runset2.rs} }  
prompt> check_3d_esd
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)

3dic.check_esd.run_dir

Specifies the path of the working directory. Option for the *check_3d_esd* command.

Data Types

string

Default *check_3d_esd*

Description

Specifies the path of the working directory. If you do not specify this option, the tool creates a directory called *check_3d_esd* under the current working directory. The path can be either relative or absolute.

Examples

The following example runs IC Validator under the working directory named *my_esd_run*.

```
prompt> set_app_options -name 3dic.check_esd.run_dir -value "my_esd_run"
prompt> check_3d_esd
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)

3dic.check_esd.runset

Specifies foundry runsets to use for programmable electrical rules check (PERC). Option for the *check_3d_esd* command.

Data Types

string pair list

Default null

Description

Specifies the foundry runset of each die block reference while the tool invokes IC Validator to perform programmable electrical rules checking.

Examples

The following example specifies the PERC runset *n5_esd_runset* and *n3_esd_runset* for the *check_3d_esd* command.

```
prompt> set_app_options -name 3dic.check_esd.runset -value { {bot_die
n5_esd_runset} {top_die n3_esd_runset} }
prompt> check_3d_esd
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)

3dic.check_esd.spice_library

Specifies directories containing spice library files to be included for the circuits checked by programmable electrical rules check (PERC). Option for the *check_3d_esd* command.

Data Types

string pair list

Default null

Description

Specifies the directories containing spice library files of each die block reference while the tool invokes icv_nettran to transform Verilog to spice format. Note all files in the specified directories are treated as spice files, no additional checks performed.

Examples

The following example specifies the spice directories *std_cells* and *mem_cells* for the *check_3d_esd* command.

```
prompt> set_app_options -name 3dic.check_esd.spice_library -value  
      { {bot_die {std_cells mem_cells} } }  
prompt> check_3d_esd
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)

3dic.check_esd.user_defined_options

Specifies strings as part of IC Validator command line options. Option for the *check_3d_esd* command.

Data Types

string

Default null

Description

This application option specifies additional options for the IC Validator command line. The string that you specify is added to the command line used to invoke programmable electrical rules checking in IC Validator.

By default, no user-defined options are included.

Examples

The following example specifies the user-defined options named -turbo.

```
prompt> set_app_options -name 3dic.check_esd.user_defined_options -value  
"-turbo"  
prompt> check_3d_esd
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)

3dic.common.die_heights

Specifies the heights of the dies.

Data Types

string pair list

Default Empty list

Description

Specifies the heights of the dies in a string pair list. Each pair is compounded by a full library design name and its height value in the unit of um. Each height can have a single value representing the height of the die or multiple values representing the multilevel height; the multilevel height denotes the heights of the die levels from the bottom to the top orderly. Moreover, if one uses the multilevel height, the height of the die is the sum of the values of the multilevel height.

Examples

The following example shows how to specify the heights of the dies:

```
prompt> set_app_options -name 3dic.common.die_heights -value { \\  
{ si_interposer.ndm:si_interposer 100 } \\
```

```
{ leon3mp_chip.ndm:leon3mp_chip 20 } \\
{ mem.ndm:mem 20 } }
```

The following example shows how to specify the multilevel height of the die:

```
prompt> set_app_options -name 3dic.common.die_heights -value { \\
{ si_interposer.ndm:si_interposer { 50 50 } } \\
{ leon3mp_chip.ndm:leon3mp_chip 20 } \\
{ mem.ndm:mem 20 } }
```

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

3dic.common.redhawk_sc_et_path

Specifies the path to the RedHawk-SC Electrothermal executable.

Data Types

string

Default The default value is the tool specific environment variable *REDHAWK_SC_ET_PATH* after installation.

Description

Specifies the path to the RedHawk-SC Electrothermal executable. If this option has no value, the tool will search your env(PATH) directory. If an executable path is still not found, then the command will error out.

Examples

The following example shows how to set the path to the RedHawk-SC Electrothermal executable.

```
prompt> set_app_options -name 3dic.common.redhawk_sc_et_path -value
{ path }
```

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

3dic.emir.database

Specifies where to save the EMIR analysis results.

Data Types

string

Default ./EMIR_DATABASE

Description

Specifies where to save the EMIR analysis results.

Examples

The following example shows how to define the location of the EMIR database.

```
prompt> set_app_options -name 3dic.emir.database -value EMIR_DB
```

See Also

- [analyze_pg](#)
- [get_app_option_value](#)
- [query_pg](#)
- [report_app_options](#)
- [report_pg](#)
- [set_app_options](#)

3dic.emir.engine

Specifies the engine for EMIR simulation.

Data Types

fusion | *built-in* string

Default The default is "fusion".

Description

Specifies the engine for EMIR simulation. It can be "fusion" or "built-in". The former denotes RedHawk-SC engine, and the later represents builtin engine.

Examples

The following example shows how to specify the EMIR engine.

```
prompt> set_app_options -name 3dic.emir.engine -value built-in
```

See Also

- [analyze_pg](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

3dic.rail.expanded_pg_pattern

Specifies the expanded PG pattern for *create_3d_template_objects*.

Data Types

String

Description

Specifies the expanded PG pattern in the following format:

```
{
  {
    {net VDD}
    {
      {layer M3}
      {left_margin 1}
      {bottom_margin 2}
      {horizontal_pitch 50}
      {horizontal_fragment 10}
      {vertical_pitch 60}
      {vertical_separation 10}
      {vertical_bundle 2}
      {width 1.5}
      {transpose false}
      {array_size 2 3}
    }
    {
      {layer M4}
      ...
    }
  }
  {
    {net VSS}
    ...
  }
}
```

For each net and each layer, user specifies these necessary parameters for PG mesh creation. For metal layer, user specifies values of margin, pitch, fragment length, separation distance, bundle size, width, etc. For via layer, user specifies via array size. Tool will create mesh only on specified metal and via layers, except that, when none of via

layer is specified, tool will create default via between adjacent metal layers. Also, tool will create mesh only for specified nets.

bottom/left_margin

Distance between the bottom/left margin and the closest stripe's center line.

bottom_margin is the distance between the bottom margin and the stripe's center line.

left_margin is the distance between the left margin and the stripe's left boundary.

horizontal/vertical_pitch

Distance between the center points of each stripe bundle.

horizontal_fragment

Length of a stripe.

vertical_separation

Distance of the space between the adjacent stripes in a bundle.

vertical_bundle

The number of stripes in a bundle. Each stripe within the bundle is horizontal and parallel to the others, and they are positioned vertically.

width

Width of a stripe.

transpose

Switch the horizontal and vertical behaviors. Default is false.

array_size

For via array size if the layer is a via layer. Not-applicable on metal layer.

3dic.route.bump_order_report_max_count

Specifies the maximum allowed number of bump order warnings.

Data Types

integer

Default 20

Description

The bump order check is in `route_3d_channels -check_only` mode. By default, you can only see 20 warning messages (3DIC-397) at most. Each warning message indicates a nonsequential bump signal assignment.

Examples

The following example shows how to specify the maximum warning count of bump order check.

```
prompt> set_app_options -name 3dic.route.bump_order_report_max_count  
-value 100
```

See Also

- [route_3d_channels](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)
- [3DIC-396](#)
- [3DIC-397](#)

3dic.route.detour_mode

Specifies spacing amount when detour routing is required to avoid routing obstacles.

Data Types

String (pitch | half_pitch | min_spacing)

Default pitch

Description

This application option specified how much spacing is reserved between routing obstacles and routing during interpose routing. The routing obstacles are vias and preroute shapes.

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

3dic.route.enable_even_distribution_with_merge

Specifies if the three-dimensional channel router distributes nets to subchannels evenly by merging subchannels.

Data Types

Boolean *true* | *false*

Default The default is "false", which does not enable the automatic even distribution.

Description

Enable the tool to evenly distribute three-dimensional channel nets to subchannels by merging subchannels. If the feature is enabled, the tool automatically figures out the maximum number of nets that are going to be routed in each subchannel, which is the ceiling of net count divided by subchannel count. If a subchannel has nets more than the threshold, the tool distributes the nets by merging the needed subchannels so that the number of nets does not exceed the restriction in each subchannel. This merge occurs after the merge caused by cross-subchannel nets. Also, this feature is only supported in UCle and D2D mode. For more details about merging subchannels, check 3dic.route.merge_subchannels.

Examples

The following example shows how to turn on the even distribution feature.

```
prompt> set_app_options -name  
3dic.route.enable_even_distribution_with_merge -value true
```

With the application option enabled, if a three-dimensional channel net distribution from subchannel 0 to 9 is 13,19,17,15,14,14,15,17,19,13, the following subchannels are be merged:

{0, 1}, {2, 3}, {6, 7}, {8, 9}

There are information messages like this:

Information: Merging subchannels 0 and 1 due to routing resources. (3DIC-851)

See Also

- [route_3d_channels](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)
- [3DIC-850](#)

- [3DIC-851](#)
- [3DIC-852](#)

3dic.route.existing_shape_report_max_count

Specifies the maximum number of existing shape count.

Data Types

integer

Default 20

Description

The existing shape check is in `route_3d_channels -check_only` mode. By default, you can only see 20 existing shapes at most.

Examples

The following example shows how to specify the maximum shape count of existing shape check.

```
prompt> set_app_options -name 3dic.route.existing_shape_report_max_count  
-value 100
```

See Also

- [route_3d_channels](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

3dic.route.merge_subchannels

Specifies the subchannel merge mode for three-dimensional channel routing.

Data Types

string *auto* | *none*

Default The default is "auto", which enables the automatic subchannel merge.

Description

When the feature is enabled, the tool automatically merges the needed subchannels if a net is crossing subchannels. Then, the net is no longer regarded as a cross-subchannel net. In HBM and UCle mode, two subchannels can only be merged when they are adjacent and those have been merged cannot be merged again with others, meaning that there might be merge conflicts. In D2D mode, a subchannel merge can be extended to multiple subchannels so that there are no merge conflicts no matter which kind of cross-subchannels nets exist. Also, it is allowed to merge multiple subchannels in pattern file in D2D mode.

Examples

The following example shows how to specify the subchannel merge mode.

```
prompt> set_app_options -name 3dic.route.merge_subchannels -value none
```

See Also

- [route_3d_channels](#)
- [report_3d_channels](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)
- [3DIC-394](#)
- [3DIC-834](#)
- [3DIC-835](#)

3dic.route.route_90degree_only

Allows only 90-degree routing for route_3d_channels

Data Types

Boolean true | false Boolean

Default false

Description

Specifies the routing angle for route_3d_channels. By default, route_3d_channels supports both 90-degree and 45-degree routing. When this option is enabled, route_3d_channels will route signal and shielding paths using only 90-degree shapes.

This option is supported only in UCIE and D2D modes.

See Also

- [route_3d_channels](#)
- [report_3d_channels](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

3dic.route.stub_alignment_type

3dic.route.stub_alignment_type allows users to specify how routing stubs are aligned at bump-array boundaries. Valid options are: inline, staggered_before, and staggered_after.

Data Types

string

Default staggered_before

Description

This application option specifies one of three stub alignment types: inline, stagger_before, or stagger_after. The default value is stagger_before

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

3dic.route.via_count_report_max_net_count

Specifies the maximum number of nets reported in via count report.

Data Types

integer

Default 20

Description

The via count report is in `report_3d_channels`. By default, you can only see 20 nets for each via count at most. You can decide the number of nets reported.

Examples

The following example shows how to specify the maximum net count of via report.

```
prompt> set_app_options -name 3dic.route.via_count_report_max_net_count
        -value 100
```

See Also

- [report_3d_channels](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

3dic.route.via_track_location

Specifies which bump array side has designated via track and where the designated via track is located.

Data Types

list of pairs of {string string} ({start side} {start center} {end side} {end center})

Default empty

Description

When this application option is specified, the designated via track feature is enabled. This application option specifies pairs of which bump region has designated via track and its via track location within a subchannel. When side designated via track is specified, either or both bump regions can have designated via tracks. When center designated via track is specified, exactly one bump region can have designated via track. Only HBM channel supports centered via track. The default is off, and `detour_mode` is used.

See Also

- [get_app_option_value](#)
- [report_app_options](#)

3dic.route.via_track_pitch

Specifies via_track routing pitch.

Data Types

String (full | half)

Default empty

Description

This application option specified the pitch between designated via track and next signal routing track in HBM mode. the default is empty and the tool decides the pitch.

See Also

- [get_app_option_value](#)
- [report_app_options](#)

3dic.route.wirelength_variation_report_max_net_count

Specifies the maximum number of nets reported in wire length variation report.

Data Types

integer

Default 20

Description

The wire length variation report is in report_3d_channels. By default, you can only see the detailed wire length of 20 nets at most. You can decide the number of nets reported.

Examples

The following example shows how to specify the maximum net count of wire length report.

```
prompt> set_app_options -name  
3dic.route.wirelength_variation_report_max_net_count -value 100
```

See Also

- [report_3d_channels](#)
- [get_app_option_value](#)

- [report_app_options](#)
- [set_app_options](#)

3dic.thermal.air_speed

Specifies the air speed used in thermal simulation.

Data Types

non-negative floating point

Default 1.0

Description

Specifies the air speed used in thermal simulation with a non-negative floating point. It will be used in thermal simulation when *3dic.thermal.boundary_type* is set to be JA.

Examples

The following example shows how to define the value of the air speed.

```
prompt> set_app_options -name 3dic.thermal.air_speed -value 5.43
```

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

3dic.thermal.ball_material

Specifies user-defined material of BGA ball.

Data Types

string pair list

Default { { tref 25 } { ts 25 } { k 48 } \\
{ density 8 } { cp 167e-3 } { c 7000 } }

Description

Specifies user-defined material of BGA ball in a string pair list. Each pair is compounded by thermal property name and its value.

Examples

The following example shows how to specify user-defined material of BGA ball.

```
prompt> set_app_options -name 3dic.thermal.ball_material -value { \\  
{ tref 20 } { ts 20 } { k 40 } \\  
{ density 9 } { c 6000 } }
```

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

3dic.thermal.boundary_type

Specifies the boundary type used in thermal simulation.

Data Types

enumerated

Default JA

Description

Specifies the boundary type used in thermal simulation with the enumerated values of *JA*, *JB*, *JC*, and *User*.

For built-in thermal engine, the boundary type is always *User*.

Examples

The following example shows how to define the boundary type.

```
prompt> set_app_options -name 3dic.thermal.boundary_type -value JB
```

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

3dic.thermal.bump_via_data_model

Specifies the data model of bump, ball, via, and tsv used in thermal simulation.

Data Types

enumerated

Default `Line`**Description**

Specifies the data model used in thermal simulation with the enumerated values of *Line* and *Solid*.

Examples

The following example shows how to define the data model.

```
prompt> set_app_options -name 3dic.thermal.bump_via_data_model -value
Solid
```

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

3dic.thermal.c4_material

Specifies user-defined material of C4 bump.

Data Types

string pair list

```
Default    { { tref 25 } { ts 25 } { k 48 } \\
             { density 8 } { cp 167e-3 } { c 7000 } }
```

Description

Specifies user-defined material of C4 bump in a string pair list. Each pair is compounded by thermal property name and its value.

Examples

The following example shows how to specify user-defined material of C4 bump.

```
prompt> set_app_options -name 3dic.thermal.c4_material -value { \\
{ tref 20 } { ts 20 } { k 40 } \\
{ density 9 } { c 6000 } }
```

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

3dic.thermal.engine

Specifies the engine for thermal simulation.

Data Types

fusion | *built-in* string

Default The default is "fusion", which is the RedHawk-SC Electrothermal engine.

Description

Specifies the engine for thermal simulation. It can be "fusion" or "built-in".

Examples

The following example shows how to specify the thermal engine.

```
prompt> set_app_options -name 3dic.thermal.engine -value built-in
```

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

3dic.thermal.htc

Specifies heat transfer coefficient which works only for 'User' boundary type.

Data Types

string pair list

Default { {package {{ top 50000 }}} {pcb {{ bottom 5 }}} }

Description

Specifies heat transfer coefficient of package or pcb in a string pair list. For package, you can define one or both top and bottom surface. For pcb, you can define any or multiple of

top, bottom and side surface, The value is valid only if *3dic.thermal.boundary* is set to be *User*. Also, if *3dic.thermal.htc_components* is set, the value will be ignored.

Examples

The following example shows how to specify heat transfer coefficient.

```
prompt> set_app_options -name 3dic.thermal.htc -value { {top 50000} }
prompt> set_app_options -name 3dic.thermal.htc -value { {package {{top
120} {side 5}} } {pcb {{top 130} {side 10} {bottom 5}}} }
prompt> set_app_options -name 3dic.thermal.htc -value { {package {{top
2500}}} {pcb {{bottom 5}}} }
```

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

3dic.thermal.htc_components

Specifies component-wise user-defined heat transfer coefficient or fixed-temperature value which works only for 'User' boundary type.

Data Types

string pair list

Default Empty list

Description

Specified component-wise user-defined heat transfer coefficient or fixed-temperature value in a string pair list, which is applied on the top surface and/or bottom surface. Each pair is compounded by a thermal component name and its user-defined boundary conditions. The valid thermal component name is *heatsink*, *molding*, *package*, *ball*, *pcb*, or *\$inst_name*. Note that each setting takes effect if the specified component is enabled in the thermal analysis flow. You can define the condition for one surface, top or bottom, or define it for both top and bottom. The boundary condition value of each thermal component supports both HTC value in the unit of W/m²-C and fixed temperature in the unit of Celsius, represented by a floating value followed by *H*, or *C*, respectively. This app option takes effect only if *3dic.thermal.boundary* is set to be *User*.

Examples

The following example shows how to specify component-wise user-defined heat transfer coefficient and fixed-temperature value.

```
prompt> set_app_options -name 3dic.thermal.htc_components -value { \\  
{ heatsink { { top 80C      } } } \\  
{ molding  { { top 1000H    } { bottom 1000H } } } \\  
{ package  { { top 24000H   } } } \\  
{ ball     { { top 1000H    } { bottom 100C } } } \\  
{ pcb      { { bottom 2000H } } } \\  
{ si_interposer_inst { { top 2000H    } { bottom 90C } } } \\  
{ logic_die_inst     { { bottom 100H } } } }
```

3dic.thermal.iteration

Specifies the iteration of thermal simulation.

Data Types

positive integer

Default 50

Description

Specifies the max iteration of thermal simulation per time step.

Examples

The following example shows how to specify the iteration.

```
prompt> set_app_options -name 3dic.thermal.iteration -value 20
```

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

3dic.thermal.package_config_file

Specifies the layout file for package setup.

Data Types

string

Default Empty string

Description

Specifies the layout file for package setup, and its value is a path of file with extension name `conf`. This app option will be adopted and be a MUST if `3dic.thermal.package_file` is specified with a `gds` file.

For the built-in thermal engine, the app option is not supported.

Examples

The following example shows how to specify the layout file for package setup.

```
prompt> set_app_options -name 3dic.thermal.package_config_file  
-value ./files/pkg_cfg_file.conf
```

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

3dic.thermal.package_file

Specifies the layout file for package setup.

Data Types

string

Default Empty string

Description

Specifies the layout file for package setup, and its value is a path of *directory*, *file without extension name*, or *file of the following extension names: xfl, kdb, gds, mcm, sip, xtr, pre, tsv_via, gbr, dxf, asc, or tgz*. This app option takes priority over `3dic.thermal.package_spec`.

For the built-in thermal engine, the app option is not supported.

Examples

The following example shows how to specify the layout file for package setup.

```
prompt> set_app_options -name 3dic.thermal.package_file  
-value ./files/pkg_file.tsv_via  
prompt> set_app_options -name 3dic.thermal.package_file  
-value ./files/pkg_file.tgz
```

```
prompt> set_app_options -name 3dic.thermal.package_file  
-value ./pkg_files
```

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

3dic.thermal.pcb_config_file

Specifies the layout file for pcb setup.

Data Types

string

Default Empty string

Description

Specifies the layout file for pcb setup, and its value is a path of file with extension name conf. This app option will be adopted and be a MUST if *3dic.thermal.pcb_file* is specified with a gds file.

For the built-in thermal engine, the app option is not supported.

Examples

The following example shows how to specify the layout file for pcb setup.

```
prompt> set_app_options -name 3dic.thermal.pcb_config_file  
-value ./files/pcb_cfg_file.conf
```

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

3dic.thermal.pcb_file

Specifies the layout file for pcb setup.

Data Types

string

Default Empty string**Description**

Specifies the layout file for pcb setup, and its value is a path of *directory*, *file without extension name*, or *file of the following extension names: xfl, kdb, gds, mcm, sip, xtr, pre, tsv_via, gbr, dxf, asc, or tgz*. This app option takes priority over *3dic.thermal.pcb_spec*.

For the built-in thermal engine, the app option is not supported.

Examples

The following example shows how to specify the layout file for pcb setup.

```
prompt> set_app_options -name 3dic.thermal.pcb_file
         -value ./files/pcb_file.gbr
prompt> set_app_options -name 3dic.thermal.pcb_file
         -value ./files/pcb_file.sip
prompt> set_app_options -name 3dic.thermal.pcb_file -value ./pcb_files
```

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

3dic.thermal.probe_num_cols

Specifies the horizontal resolution of the pre-collection of the temperature probing in transient analysis.

Data Types

positive integer

Default 10**Description**

Specifies the horizontal resolution of the pre-collection of the temperature probing in transient analysis. When the value is N , there will be $N+1$ probes in a row.

Examples

The following example specifies the horizontal resolution of 6 probes in a row:

```
prompt> set_app_options -name 3dic.thermal.probe_num_cols -value 5
```

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

3dic.thermal.probe_num_rows

Specifies the vertical resolution of the pre-collection of the temperature probing in transient analysis.

Data Types

positive integer

Default 10

Description

Specifies the vertical resolution of the pre-collection of the temperature probing in transient analysis. When the value is N , there will be $N+1$ probes in a column.

Examples

The following example specifies the vertical resolution of 6 probes in a column:

```
prompt> set_app_options -name 3dic.thermal.probe_num_rows -value 5
```

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

3dic.thermal.ubump_material

Specifies user-defined material of micro-bump.

a

Data Types

string pair list

```
Default  { { tref 25 } { ts 25 } { k 400 } \\
           { density 8933e-3 } { cp 385e-3 } { c 5800 } }
```

Description

Specifies user-defined material of micro-bump in a string pair list. Each pair is compounded by thermal property name and its value.

Examples

The following example shows how to specify user-defined material of mirco-bump.

```
prompt> set_app_options -name 3dic.thermal.ubump_material -value { \\
{ tref 20 } { ts 20 } { k 350 } \\
{ density 4573e-3 } { c 6000 } }
```

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

a

abstract.allow_all_level_abstract

Allows creating abstract for a block even if its child blocks are linked with abstracts

Data Types

String

Default auto

Description

In a multi-level physical hierarchy design, when the app-option value is *auto*, if the target use of the abstract is implementation, abstracts are allowed to be created even if its child blocks are linked with abstracts. and if the target use of the abstract is planning, abstracts are allowed to be created for a middle level block only if none of its descendant blocks can be abstract view.

a

User can use this app option to change the above behavior in each stage by setting this app option to true or false as needed.

abstract.allow_fault_in

Adds automatically the missing pins to the abstract boundary

Data Types

Boolean

Default false

Description

This application option is used to support automatic missing pin additions to the abstract boundary when necessary. You must set this application option as true to enable the support. Set this application option before opening the top design.

See Also

- [get_app_option_value](#)
- [set_app_options](#)

abstract.annotate_power

When true, the `create_abstract` command creates abstracts that include power dissipation information of the content.

Data Types

Boolean

Default false

Description

This app option is used to instruct the abstraction engine to annotate power dissipation values that represent the power dissipation of the content of the block. Power dissipation values are not stored for the cells in the abstract view (typically boundary cells) as these values can be recomputed. For these cells, the top-level power calculation re-calculates the power based on actual activity and transition information. For other cells, e.g. cells that are abstracted away, power data is stored in the abstract in a condensed format. A power abstract facilitates the generation of an accurate top-level power report including the power of the sub blocks, even if these have abstract views.

a

When the app options is set to true, both activity and transition data will be updated if these are out of date. Note that this may add significant additional runtime to the `create_abstract` command.

To have accurate power abstraction data, it is important that the activities at the primary inputs of the abstracted blocks are meaningful. Preferably these are derived from a top-level activity propagation or from an activity annotation. Annotation of the activity at the primary inputs of the block is a responsibility of the user.

The user needs to set this app-option before running the `create_abstract` command.

See Also

- [get_app_option_value](#)
- [set_app_options](#)
- [create_abstract](#)
- [report_power](#)

abstract.check_constraints_consistency

When running `check_hier_design`, include a check that compares top-level and block-level sdc constraints.

Data Types

Boolean

Default false

Description

The timing constraints present during timing abstract creation can influence what gets included in the abstract of the block. If the block-level constraints are different then the constraints present at top, the timing information of the top-level design might not match what we would see in a flattened view.

The command `check_hier_design` can run a high-level constraint comparison and report constraint differences between top and block level. This app option can be used to turn this comparison on.

The report generated will separately list out mismatches in each active scenario according to the scenario, mode and corner respectively.

a

The following constraints are checked:

- *set_false_path*
- *set_multicycle_path*
- *set_min_delay/set_max_delay*
- *set_sense -type data*
- *create_clock*

Clocks defined on block ports are considered a match if the block pin is part of a clock network at the top-level.

- *create_generated_clock*
- *set_case_analysis*
- *set_disable_timing*
- *set_clock_balance_points*
- *set_clock_gating_check*
- *set_cell_mode*
- *set_latch_loop_breaker*
- *set_clock_latency*
- *set_annotated_delay*
- *set_annotated_transition*
- *set_data_check*
- *set_input_delay*

Block ports are not checked, since budgeting can generate *set_input_delay* constraints on the ports as part of the regular hierarchical flow.

- *set_output_delay*

Block ports are not checked, since budgeting can generate *set_output_delay* constraints on the ports as part of the regular hierarchical flow.

- *set_max_time_borrow*
- *set_timing_derate*
- *set_voltage*
- *set_temperature*

a

- *set_process_label*
- *set_process_number*
- *set_aocvm_coefficient*

abstract.enable_improvements_for_budgeting

To enable compression improvements of the abstracts which are created for the purpose of use in budgeting.

Data Types

Boolean

Default false

Description

In the design planning flow, ahead of budgeting, while you perform `estimate_timing` on the hierarchical design, the sub-blocks can be represented using abstracts to provide runtime and capacity benefits. In this specific flow where abstracts are created and used, enable this application option to provide further runtime benefits to this flow by improving the compression of the abstracts created. You need to set this application option before creation of abstracts with estimated corner.

See Also

- [get_app_option_value](#)
- [set_app_options](#)
- [compute_budget_constraints](#)
- [estimate_timing](#)
- [create_abstract](#)

abstract.enable_signal_em_analysis

To enable signal em analysis in abstract.

Data Types

Boolean

Default false

a

Description

This app-option is used to support Signal EM Analysis in block-abstracts. User must set this as true to enable signal EM analysis on nets belonging to the block-abstract in the block-abstract netlist. User need to set this app-option before create_abstract command.

See Also

- [get_app_option_value](#)
- [set_app_options](#)

abstract.high_fanout_limit

Set threshold for high fanout when generating "full" timing abstracts.

Data Types

integer

Default 600

Description

The -timing_level option of the create_abstract command may be used to create a "full" abstract. Full abstracts contain nearly all of the boundary (donut) logic that can be found in the original design. But some of the boundary logic is filtered out in order to keep the abstract size reasonable. In particular, logic connected only to high-fanout networks is filtered. Usually, the filtered logic includes clock networks, scan enable, and reset. For these signals, only the essential part of the logic is kept, instead of all of the logic.

The abstract.high_fanout_limit option allows you to control how much logic is filtered. If you set the value to 1000, then only high fanout networks with more than 1000 fanouts will be filtered. If you set the value to 5000, then only high fanout networks with more than 5000 fanouts will be filtered.

You can set the value to 0 if you want to turn off filtering completely. Usually this will cause every register in your design to be included in the abstract.

If you set the value very low, you will get an abstract which is very similar to a "compact" abstract.

See Also

- [get_app_option_value](#)
- [report_app_options](#)

a

- [set_app_options](#)
- [create_abstract](#)

abstract.include_aggressor_nets

To include aggressors in the block-abstract netlist.

Data Types

Boolean

Default true

Description

This app-option is used to support Signal Integrity(SI) flow with block-abstracts. User must set this as true to include the aggressors of the nets belonging to the block-abstract in the block-abstract netlist. This will help improve timing correlation with SI. User need to set this app-option before create_abstract command.

See Also

- [get_app_option_value](#)
- [set_app_options](#)

abstract.keep_constrained_objects

When true, the create_abstract -placement command keeps all objects that have timing constraints defined on them.

Data Types

Boolean

Default true

Description

When this option is true, the placement abstract creation checks if the design has timing constraints. If timing constraints are found objects that have timing constraints defined on them are retained in the placement abstract.

If this app option is false, no extra constrained objects are retained.

This app option has no impact on timing abstract.

a

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)
- [create_abstract](#)

abstract.keep_min_pulse_width_violation_pins

When true, the `create_abstract` command will include fan-in cells/nets to the clock sink pins, that are in the clock path of boundary clock sources and have Min Pulse Width violations, into the block abstract.

Data Types

Boolean

Default `false`

Description

When this option is true, the timing abstract creation checks if Min Pulse Width (MPW) on clock sink pins, that are in the clock path of boundary clock sources, are below a given slack limit. If such pins are found, all fan-in cells/nets to them are included in the block's abstract. By default, MPW minimum slack limit is assumed to be 0.0. To change the MPW slack limit for this app-option, another app option: `abstract.min_pulse_width_violation_slack_limit` can be used.

This app option has no impact on placement abstracts.

See Also

- [create_abstract](#)
- [report_min_pulse_width](#)

abstract.latch_levels

Controls the number of latch levels included in the timing abstract

Data Types

Integer

Default `3`

c

Description

This app-option is used to control the number of latch levels that will be included in the timing abstract. Using a higher value can increase the accuracy of timing operations at the top-level, at the expense of additional run-time.

See Also

- [get_app_option_value](#)
- [set_app_options](#)

abstract.min_pulse_width_violation_slack_limit

This app option sets the slack limit to be considered for `abstract.keep_min_pulse_width_violation_pins` app-option.

Data Types

FLOAT

Default 0.0

Description

The value set with this app option is used to filter the clock sink pins obtained, when `abstract.keep_min_pulse_width_violation_pins` is enabled. Fan-in cells/nets of only the pins with Min Pulse Width (MPW) slack lower than the value set with this app option, are included into the abstract.

See Also

- [create_abstract](#)

c

ccd.adjust_io_clock_latency

Controls whether concurrent clock and data optimization (CCD) adjusts I/O clock latency to optimize timing.

Data Types

Boolean

Default true

c

Description

If CCD is enabled in `clock_opt build_clock`, it will adjust the I/O clock latencies (like virtual clock latencies) to improve the I/O timing and all subsequent `compute_clock_latency` will also use CCD to adjust the I/O latencies. This application option can be used to enable/disable this feature.

This feature is on-by-default when CCD is enabled in `clock_opt build_clock`. However the feature will automatically get disabled in the following cases

- If `ccd.optimize_boundary_timing` is set as false
- If I/O paths are included in the path groups specified in `ccd.skip_path_groups`
- If I/O clocks are specified using `set_latency_adjustment_options -exclude_clocks`, then CCD will not adjust their latencies

See Also

- [set_app_options](#)
- [compute_clock_latency](#)
- [set_latency_adjustment_options](#)

ccd.ccd_opt_voltage_drop_enable_ml_predictor

Enable ML predictor for IRD-CCD after IRD-OPT.

Data Types

boolean

Default false

Description

For ML route_opt, ML predictor is utilized to substitute Red Hawk (RH) fusion analysis after IRD-OPT. Then IRD-CCD is performed according to the results from the ML predictor. Use the `ccd.ccd_opt_voltage_drop_enable_ml_predictor` app option to enable the ML predictor.

See Also

- [set_app_options](#)

ccd.dps.analyze_high_power_instances

List of clock sinks to be controlled by special power mode settings in the DPS optimization.

c

Data Types*list***Default** ""**Description**

DPS optimization adjusts clock latency to improve dynamic power integrity.

In a use-case, controlling the behavior of high-power instances, such as large RAMs, is important. To control which instances are recognized by a use-case, the application option *ccd.dps.analyze_high_power_instances* can specify a list of clock sinks that are to be controlled by special power mode settings in use-cases.

The manually specified high-power instances is an addition to any auto detected high-power instances.

Examples

To set the list of high-power instances:

```
prompt> set_app_options \\  
        -name ccd.dps.analyze_high_power_instances \\  
        -value {module1/FF1/CP module2/RAM2/CP}
```

See Also

- [set_app_options](#)
- [place_opt](#)

ccd.dps.analyze_high_power_instances_autodetect

Controls whether Dynamic Power Shaping (DPS) automatically detect high-power instances used by special power mode setting in use-cases.

Data Types

Boolean

Default true**Description**

DPS optimization adjusts clock latency to improve dynamic power integrity.

In a use-case, controlling the behavior of high-power instances, such as large RAMs, is important. By default, DPS auto detects such high-power instances.

c

To disable auto detection of high power instances, set the `ccd.dps.analyze_high_power_instances_autodetect` application option to *false*.

See Also

- [set_app_options](#)
- [place_opt](#)

ccd.dps.analyze_high_power_instances_autodetect_threshold

Defines the threshold in Dynamic Power Shaping (DPS) for auto detection of high-power instances used by special power mode setting in use-cases.

Data Types

float

Default 0.1

Description

DPS optimization adjusts clock latency to improve dynamic power integrity.

In a use-case, controlling the behavior of high-power instances, such as large RAMs, is important. By default, DPS auto detects such high-power instances.

The application option `ccd.dps.analyze_high_power_instances_autodetect_threshold` defines the threshold for high power sinks in the design. The default of 0.1, means that all clock sinks that potentially consume 0.1 percent or more of the total possible dynamic power, are labeled as a high power sink.

See Also

- [set_app_options](#)
- [place_opt](#)

ccd.dps.auto_detect_cts_options

Auto-detect CTS parameters (supply, buffers, fanout, max transition, skew) for the clock tree estimation used in the Dynamic Power Shaping (DPS) optimization.

Data Types

boolean

Default true

c

Description

DPS optimization adjusts clock latency to improve dynamic power integrity.

DPS performs a clock tree estimation continuously during the optimization, in order to keep the final clock tree size to a minimum. For such estimation, it is required to have specific values for application options `ccd.dps.clock_tree_buffers` and `ccd.dps.clock_tree_supply_net`. Because they have no default values, the user must specify them manually. Other parameters for the clock tree estimation have functional defaults, but can nevertheless be derived with more accuracy from a trial CTS if available.

Enabling this application option `ccd.dps.auto_detect_cts_options` can provide useful values:

Clock tree buffers

Will be set to the preferred buffers by CTS (if trial CTS enabled) or a subset of the library cells with CTS purpose and with the highest strength.

Clock tree supply

Will be the supply connecting to highest number of clock sinks.

Clock tree buffer max transition

Can use internal default if nothing specified in `cts.common.default_max_transition`.

Clock tree max sink transition

Can use internal default if nothing specified in `cts.common.default_max_transition`.

Clock tree max fanout

Can use internal default if nothing specified in `cts.common.max_fanout`.

Clock tree target skew

Will fallback to the clock setup uncertainty if not specified. This is used for power signature smoothing.

See Also

- [set_app_options](#)
- [place_opt](#)

`ccd.dps.auto_target_constraint_parameters`

Specifies the list of parameters controlling the adaptive auto target aliases (generated auto targets) used in the Dynamic Power Shaping (DPS) optimization.

c

Data Types

list of list of strings

Default {}

Description

DPS optimization adjusts clock latency to improve dynamic power integrity.

The default auto target specified using the application option `ccd.dps.auto_targets` contains the two adaptive aliases named `generate_ws_auto_slack_offset` and `generate_constraint_tns`. The behavior of these aliases is controlled by changing the defaults using the application option `ccd.dps.auto_target_constraint_parameters`. The defaults are changed by applying one or more of the following name/value pairs described below.

The option is equipped with a syntax checker that verifies the validity of the keys and values. Note that, as the validation is linked solely to the option, every value whose dependencies are associated with other options is waived by the checker.

setup_aggression_factor

Specifies in percentage the number of failing end-points to use when calculating the `slack_offset` for setup worst slack targets.

Default: 50%

setup_slack_offset_minimum

The minimum value of the `slack_offset` for setup worst slack targets. (waived by syntax check)

Default: 20ps

setup_slack_offset_multiplication_factor

A factor that is multiplied on the setup `slack_offset` calculated from the `setup_aggression_factor`.

Default: 1.0

hold_aggression_factor

Specifies in percentage the number of failing end-points to use when calculating the `slack_offset` for hold worst slack targets.

Default: 95%

c

hold_slack_offset_minimum

The minimum value of the slack_offset for hold worst slack targets. (waived by syntax check)

Default: 200ps

hold_slack_offset_multiplication_factor

A factor that is multiplied on the hold slack_offset calculated from the hold_aggression_factor.

Default: 1.0

setup_tns_overhead

The percentage degradation of setup TNS to allow.

Default: 0%

setup_tns_floor

The minimum value to allow for degradation of setup TNS per endpoint. The purpose is to ensure that some headroom is allowed even when baseline value is very low (and thus the percentage overhead is very small). (waived by syntax check)

Default: 0.1ps

setup_nfe_overhead

The allowed overhead on setup NFE targets. A value of "-" means no setup NFE targets are created. Default: "-"

setup_nfe_floor_factor

A factor to multiply with the number of endpoints to get the minimum setup NFE required value for NFE constraints (only used if setup_nfe_overhead is not "-"). Default: 0.1%

hold_tns_overhead

The percentage degradation of hold TNS to allow.

Default: 10%

hold_tns_floor

The minimum value to allow for degradation of hold TNS per endpoint. The purpose is to ensure that some headroom is allowed even when baseline value is very low (and thus the percentage overhead is very small). (waived by syntax check)

Default: 0.1ps

c

hold_nfe_overhead

The allowed overhead on hold NFE targets. A value of "-" means no hold NFE targets are created.

Default: "-"

hold_nfe_floor_factor

A factor to multiply with the number of endpoints to get the minimum hold NFE required value for NFE constraints (only used if hold_nfe_overhead is not "-").

Default: 0.1%

constraint_scenarios

Scenarios to include in the TNS and NFE constraint sections of the Auto Targets. (waived by syntax check)

Default: ""

constraint_path_groups

Path groups to include in the TNS and NFE constraint sections of the Auto Targets.

Default: "all" "in2reg" "reg2out" "reg2reg"

tns_aggregation_mode

The aggregation mode to use for TNS constraint targets. "scenario" or "none". If "scenario", the TNS is aggregated for all scenarios in the TNS constraint targets.

Default: "none"

nfe_aggregation_mode

The aggregation mode to use for NFE constraint targets. "scenario" or "none". If "scenario", the NFE is aggregated for all scenarios in the NFE constraint targets.

Default: "none"

Examples

The following example changes the setup_aggression_factor and tns_aggregation_mode:

```
prompt> set_app_options \\  
-name ccd.dps.auto_target_constraint_parameters \\  
-value {{setup_aggression_factor 80%} {tns_aggregation_mode scenario}}
```

c

See Also

- [set_app_options](#)
- [place_opt](#)

ccd.dps.cspartition_latency_range

Specifies the maximum range of partition latencies used in the Dynamic Power Shaping (DPS) optimization.

Data Types*list***Default** {-200ps 200ps}

Pair of values (implicit/explicit user unit)

Description

DPS optimization adjusts clock latencies to improve dynamic power integrity.

During a DPS optimization, a clock latency offset is determined for each cspartition. The allowed range for the offsets can be controlled by this application option. If either the minimum or maximum value (or both) is set to nan, the value will be taken from the application options *ccd.max_prepone* and *ccd.max_postpone* respectively. If none of these specify a valid value the default will be to use a range of +/- half a clock cycle.

Examples

The following example changes the latency range to -120ps to 120ps:

```
prompt> set_app_options \\  
        -name ccd.dps.cspartition_latency_range \\  
        -value {-120ps 120ps}
```

See Also

- [set_app_options](#)
- [place_opt](#)

ccd.dps.effective_r_aware

Controls whether Dynamic Power Shaping (DPS) utilizes any available effective resistance information to guide its optimization.

Data Types

Boolean

Default true

Description

When this value is set to true, DPS optimization utilizes any available effective resistance information to guide its optimization, increasing the focus on reducing dynamic current demand in areas with high effective resistance.

To disable use of effective resistance, set the *ccd.dps.effective_r_aware* application option to *false*.

See Also

- [set_app_options](#)
- [place_opt](#)

ccd.dps.flow_break_on_dps_opt_failure

Sets the flow to break, if DPS optimization fails.

Data Types

boolean

Default false

Description

When false (default), the *place_opt* flow continues even if DPS optimization fails. If set to true the flow will break if DPS optimization fails.

See Also

- [set_app_options](#)
- [place_opt](#)

ccd.dps.flow_reporting

List of wanted reports during Dynamic Power Shaping (DPS) flow (power, power_density, solution, partitions, use_case).

c

Data Types

list_of_string

Default {}

Description

DPS may generate several reports after its power analysis stage and optimization stage. The user can specify which reports that he or she would like to see through application option *ccd.dps.flow_reporting*. The values (reports) for this option can be one or more of the following: power, power_density, solution, partitions, use_case or *. Specifying wildcards * will produce all reports.

See Also

- [set_app_options](#)
- [place_opt](#)

ccd.dps.focus_power_scenarios

Specifies the scenarios for power analysis and power optimization targets in Dynamic Power Shaping (DPS).

Data Types

string

Default ""

Description

This option allows the user to select the scenarios for DPS power analysis and optimization. The default value is "" which corresponds to all scenarios.

If the user specifies a value for the *ccd.dps.use_cases* app option, the *focus_power_scenarios* app option will be ignored and the power analysis will be performed according to the use case defined by the user. If the user specifies a scenario for the power optimization target in *ccd.dps.optimize_auto_targets*, the *focus_power_scenarios* app option will be ignored and the optimization for the power target will be performed solely on the scenario defined in *ccd.dps.optimize_auto_targets*. Note that, a scenario specified by the user in the power target has to have a use case defined.

c

See Also

- [set_app_options](#)
- [place_opt](#)

ccd.dps.focus_supply_nets

Sets the supply nets to focus on when optimizing with Dynamic Power Shaping (DPS).

Data Types

string

Default <empty>

Description

This app option can be used to specify a list of supply nets (space separated) which the Dynamic Power Shaping algorithm should focus on (i.e. DPS will only look at the dynamic power response of this subset of supply nets). This is used for Density Weighted Partitioning and optionally also for the power target in the DPS optimizer (when `ccd.dps.optimize_power_use_focus_supply_nets` is true). This app option can be set to "" to automatically select the major supply nets in the design with a minimum supply connection coverage decided by the app option `ccd.dps.focus_supply_nets_auto_coverage`.

See Also

- [set_app_options](#)
- [place_opt](#)

ccd.dps.hotspot_aware

Enables hotspot awareness in Dynamic Power Shaping (DPS) optimizations.

Data Types

bool

Default false

Description

This app option is a "mega switch" which will enable the following features in DPS: * Density weighted partitioning (`ccd.dps.partition_density_weighted`) * Focus supply nets in auto targets (`ccd.dps.optimize_power_use_focus_supply_nets`)

c

* SPSG Filtering (ccd.dps.optimize_filter_spsgs) * Protect hotspot skew (ccd.dps.optimize_protect_hotspot_skew)

The app option for each of these features will be enabled when the hotspot_aware app option is enabled. When setting ccd.dps.hotspot_aware to false, the four feature app options will be set back to default (off or any user default).

See Also

- [set_app_options](#)
- [place_opt](#)

ccd.dps.optimize_auto_targets

Specifies the optimization targets used in the Dynamic Power Shaping (DPS) optimization.

Data Types

string

Default "generate_ws_auto_slack_offsets\\n reduce_peak_power_stepped_psgs\\n generate_constrain_tns\\n reduce_clock_tree"

Description

DPS optimization adjusts clock latency to improve dynamic power integrity. \ An optimization is driven by a set of optimization targets, directing the optimization process. \ The specification is carried out using the Auto Target concept. You can set high-level, \ easily readable optimization requirements, and then let DPS automatically carry out the optimization accordingly.

Targets can be defined specifically by setting values for target options or using aliases of more advanced underlying targets.

The option is equipped with a syntax checker that verifies the validity of the keys and values. Note that, as the validation is linked solely to the option, every value whose dependencies are associated with other options is waived by the checker.

BASIC AUTO TARGETS

Targets like the default Auto Targets are aliases of more basic Auto Targets. \ While an Auto Target alias has recommended values for the target arguments and options, \ a basic Auto Target relies on default values and has the following format:

```
\<target_type\>:\<option1\>:\<option2\>:\<option3\>...
```

In this format, the \<target_type\> specifies the basic type of the target (*power*, *timing*, *voltage*, and so on), \ while all following options give detailed values for the optimization

c

target. \ An Auto Target option might either be of the form `\<property\>=\<value\>`, \ to set a property to a given value, or simply `\<switch\>`, if the option is a switch.

An example is given here:

```
power:treat_master_clocks_individually:standoff=5%:target_value=100
```

In the example, the target type is *power*. This is followed by a set of options separated by ':'. \ The *treat_master_clocks_individually* switch is set, \ and the *standoff* is set to 5%. Finally, the *target_value* is set to 100, \ which can be translated to a wanted power reduction of 100%. \ In general, Auto Target properties follow the properties available in the underlying targets of the same type.

In addition to properties and switches corresponding to target properties, \ there is a set of special keys available to Auto Targets: *required_value*, *target_value* and *standoff*.

required_value

The *required_value* is a value of the target that must always be maintained, \ even during the optimization of other targets in the Auto Targets clause. \ It is a hard requirement, for the full optimization. \

target_value

The *target_value* is a value of the target which is desirable to achieve. \ This is an aim of the optimization. \

standoff

The *standoff* is used when choosing the final solution. \ During the optimization, the solution space is mapped out, \ resulting in a number of solutions with different trade-offs between targets. \ Among those, the final solution is picked based on the standoff value and the order of targets. \ Beginning with the first target, the value that best achieves the target value is determined. \ A subset of solutions is selected by stepping back from the best value by the given standoff. \ The standoff can be given either as an absolute value or as a percentage. In case it is given as a \ percentage, this percentage will be applied to the available range of values among the available solutions \ when evaluating each target. \ The subset is then used to find the solution that best achieves the target value of the second target \ and its standoff is used to further shrink the number of solutions. \ Subsequently, all targets and standoffs are processed. \ If the final solution subset contains more than one solution, the most balanced solution is picked. \ The standoff value thus allows some headroom within which the following targets might work, to achieve their goals.

substandoff

The *substandoff* is similar to the *standoff* except that it operates on the create subtargets, e.g. specific \ targets for each scenario, use case and power supply. For each of the subtargets in order from first to last \ it will select the solutions with values in the range

c

from the best and within the standoff range. \ The standoff can be given either as an absolute value or as a percentage. In case it is given as a \ percentage, this percentage will be applied to the available range of values among the available solutions \ when evaluating each subtarget.

AUTO TARGET ALIASES

Auto Target Aliases are seemingly simple targets, that expand into basic Auto Targets. An example of an alias is the `reduce_clock_tree` target, which is a synonym for the basic Auto Target: `clock_tree:required_value=10:target_value=-100:standoff=2`

So in principle, Auto Target Aliases are a short-form notation of more detailed Auto Targets.

dont_make_setup_timing_worse

`timing_setup:accept_initial_slack:required_value=0ps:target_value=-`

dont_make_hold_timing_worse

`timing_hold:accept_initial_slack:required_value=0ps:target_value=0`

allow_worse_setup_timing

`timing_setup:required_value=-:target_value=-`

allow_worse_hold_timing

`timing_hold:required_value=-:target_value=-`

fix_setup_timing

`timing_setup:timing_mode=total_negative_slack:target_value=0ps:standoff=10%`

fix_hold_timing

`timing_hold:timing_mode=total_negative_slack:target_value=0ps:standoff=10%`

fix_setup_nfe

`timing_setup:timing_mode=number_failing_endpoints:target_value=0:standoff=10%`

fix_hold_nfe

`timing_hold:timing_mode=number_failing_endpoints:target_value=0:standoff=10%`

reduce_peak_power_stepped_psgs

`power:power_mode=contiguous:target_value=100:\ stepped_psgs:
treat_master_clocks_individually:standoff=10%`

c

reduce_peak_power

power:power_mode=contiguous:target_value=100:\
treat_master_clocks_individually:standoff=10%

reduce_clock_tree

clock_tree:required_value=10:target_value=-100:standoff=2

no_implicit_targets

timing_setup:required_value=-:target_value=-\
timing_hold:required_value=-:target_value=-\ clock_tree:required_value=-:target_value=-

generate_ws_auto_slack_offsets

This alias creates targets which ensure that the bulk of the setup and hold timing \ is not worsened while allowing some flexibility by calculating slack offsets based on the aggression factors.

generate_constrain_tns

Creates targets which attempts to improve TNS or NFE for all scenarios and path groups but constrain them to the baseline with a margin according to the overhead and floor values.

See Also

- [set_app_options](#)
- [place_opt](#)

ccd.dps.optimize_hold_tradeoff_level

Sets the tradeoff level for hold timing for Dynamic Power Shaping (DPS).

Data Types

string

Default 'medium'

Description

The hold timing constraints for DPS optimization can be controlled with this app option. It adjusts the amount of overhead allowed in the TNS targets as well as the elasticity of the worst slack constraints. This will allow for more or less flexibility in the timing constraints and will affect the potential for dynamic power optimizations. With little flexibility, the timing is kept within strict bounds, but the potential for reductions in dynamic power may be small. Larger flexibility, on the other hand, may improve the power reductions but it might

c

cost in terms of degraded timing after `place_opt`. In many cases the timing degradations can be recovered in later stages of the flow. Valid values are 'low', 'medium' and 'high'.

See Also

- [set_app_options](#)
- [place_opt](#)

ccd.dps.optimize_power_target_mode

Sets the mode of the power target 'reduce_dynamic_power' for Dynamic Power Shaping (DPS).

Data Types

string

Default 'stepped_psgs'

Description

The `reduce_dynamic_power` target in DPS can operate in two modes. It can either attempt to reduce the global dynamic peak current or it can attempt to reduce the local peak current across regions (windows) of the chip (to reduce the IR drop).

See Also

- [set_app_options](#)
- [place_opt](#)

ccd.dps.optimize_power_use_focus_supply_nets

Enables the use of focus supply nets in the power target for Dynamic Power Shaping (DPS).

Data Types

bool

Default false

Description

When this option is enabled the power target for DPS will use only the focus supply nets (not all supply nets). This saves optimization time and ensures that the optimization favors improvements in the focus supply nets see (`ccd.dps.focus_supply_nets`).

c

This feature is automatically turned on by the `ccd.dps.hotspot_aware` app option.

See Also

- [set_app_options](#)
- [place_opt](#)

ccd.dps.optimize_setup_tradeoff_level

Sets the tradeoff level for setup timing for Dynamic Power Shaping (DPS).

Data Types

string

Default 'medium'

Description

The setup timing constraints for DPS optimization can be controlled with this app option. It adjusts the amount of overhead allowed in the TNS targets as well as the elasticity of the worst slack constraints. This will allow for more or less flexibility in the timing constraints and will affect the potential for dynamic power optimizations. With little flexibility, the timing is kept within strict bounds, but the potential for reductions in dynamic power may be small. Larger flexibility, on the other hand, may improve the power reductions but it might cost in terms of degraded timing after `place_opt`. In many cases the timing degradations can be recovered in later stages of the flow. Valid values are 'low', 'medium' and 'high'.

See Also

- [set_app_options](#)
- [place_opt](#)

ccd.dps.partition_auto_max

Specifies the maximum number of partitions to allow when calculating partition count automatically.

Data Types

integer

Default 300

Description

DPS optimization adjusts clock latency to improve dynamic power integrity.

c

The application option `ccd.dps.partition_auto_max` is used during the partitioning as the maximum number of partitions to allow as the result of calculating a partition base count. The final number of clock sink partitions might be larger, depending on a number of factors, such as; number of high-power instances, number of multiclocked cells, and so on.

See Also

- [set_app_options](#)
- [place_opt](#)

ccd.dps.partition_auto_min

Specifies the minimum number of partitions to allow when calculating partition count automatically.

Data Types

integer

Default 80

Description

DPS optimization adjusts clock latency to improve dynamic power integrity.

The application option `ccd.dps.partition_auto_min` is used during the partitioning as the minimum number of partitions to allow as the result of calculating a partition base count. The final number of clock sink partitions might be larger, depending on a number of factors, such as; number of high-power instances, number of multiclocked cells, and so on.

See Also

- [set_app_options](#)
- [place_opt](#)

ccd.dps.partition_auto_sinks

Specifies the desired average sink count per partition when calculating partition count automatically.

Data Types

integer

c

Default 800**Description**

DPS optimization adjusts clock latency to improve dynamic power integrity.

The application option *ccd.dps.partition_auto_sinks* is used during the partitioning as a desired average number of sinks to get in each clock sink partition when calculating a partition base count. The total number of sinks is divided by this number to get an initial partition base count. The final number of clock sink partitions might be larger than this calculated value, depending on a number of factors, such as; number of high-power instances, number of multiclocked cells, and so on.

See Also

- [set_app_options](#)
- [place_opt](#)

ccd.dps.partition_base_count

Specifies the desired number of clock-sink-pin partitions for the DPS optimization.

Data Types

integer

Default 0**Description**

DPS optimization adjusts clock latency to improve dynamic power integrity.

The application option *ccd.dps.partition_base_count* is used during the partitioning as a guiding number of final cspartitions. The final number of clock sink partitions might be larger, depending on a number of factors, such as; number of high-power instances, number of multiclocked cells, and so on.

A value of zero means the partition count is automatically calculated based on other parameters

See Also

- [set_app_options](#)
- [place_opt](#)

ccd.dps.plot_power_density_grid

Specifies the resolution of the power density grid used in the Dynamic Power Shape (DPS) optimization.

Data Types

a pair of strings

Default {}

Description

DPS optimization adjusts clock latency to improve dynamic power integrity. One may want to see the result of current demand per area (which is called power density in DPS) across the design. The application *ccd.dps.plot_power_density_grid* helps the user specifies the resolution of the view. If not specified by the user, the power density grid aligns with the stepped window size..

Examples

The following example will create a grid of 50rows x 50columns across the design:

```
prompt> set_app_options -name ccd.dps.plot_power_density_grid -value {50 50}
```

The following example will create a grid of rectangles across the design, the size of each rectangle is 50um(W) x 40um(H):

```
prompt> set_app_options -name ccd.dps.plot_power_density_grid -value {50um 40um}
```

See Also

- [set_app_options](#)
- [place_opt](#)

ccd.dps.powershape_clocks

List of clocks that will be included in the DPS optimization.

Data Types

list_of_strings

Default *

c

Description

DPS optimization adjusts clock latency to improve dynamic power integrity.

By default, *ccd.dps.powershape_clocks* enables partitioning of all clock domains. If some clocks domains should remain untouched, e.g. critical clock domains with almost no logic (power contribution), these can be defined using the *ccd.dps.powershape_exclude_clocks* application option. A clock sink is powershaped if, at least, one of its clocks is defined in the *ccd.dps.powershape_clocks* application option and none of its clocks are defined in the *ccd.dps.powershape_exclude_clocks* application option.

The *ccd.dps.powershape_clocks* list is a positive list, meaning that only the clocks in the list are used as basis for the partitioning.

See Also

- [set_app_options](#)
- [place_opt](#)

ccd.dps.powershape_exclude_clocks

List of clocks that will be excluded from the DPS optimization.

Data Types

list_of_strings

Default *

Description

DPS optimization adjusts clock latency to improve dynamic power integrity.

If some clocks domains should remain untouched and excluded from the DPS optimization, e.g. critical clock domains with almost no logic (power contribution), these can be defined using the *ccd.dps.powershape_exclude_clocks* application option. A clock sink is powershaped if, at least, one of its clocks is defined in the *ccd.dps.powershape_clocks* application option and none of its clocks are defined in the *ccd.dps.powershape_exclude_clocks* application option.

See Also

- [set_app_options](#)
- [place_opt](#)

ccd.dps.powershape_high_power_instances

List of cells that will be partitioned separately in the DPS optimization.

Data Types

list_of_strings

Default ""

Description

DPS optimization adjusts clock latency to improve dynamic power integrity.

By default, DPS partitions the design with a balanced amount of dynamic peak current demand in all partitions. But single high-power instances, such as large RAMs, may 'un-balance' this power balancing. To avoid this, DPS by default auto-detect high-power instances.

It is also possible to manually define a list of high-power instances by using the *ccd.dps.powershape_high_power_instances* application option. The manually specified high-power instances is an addition to any auto detected high-power instances.

See Also

- [set_app_options](#)
- [place_opt](#)

ccd.dps.powershape_high_power_instances_autodetect_thresh old

Defines the threshold for auto detection of high power instances by Dynamic Power Shaping (DPS).

Data Types

float

Default 1.0

Description

DPS optimization adjusts clock latency to improve dynamic power integrity.

By default, DPS partitions the design with a balanced amount of dynamic peak current demand in all partitions. But single high-power instances, such as large RAMs, might 'un-balance' this power balancing. To avoid this, DPS by default auto-detect high-power instances.

c

The application option *ccd.dps.powershape_high_power_instances_autodetect_threshold* defines the threshold for high power sinks in the design. The default of 1.0, means that all clock sinks that potentially consume 1.0 percent or more of the total possible dynamic power, are labeled as a high power sink.

See Also

- [set_app_options](#)
- [place_opt](#)

ccd.dps.powershape_separate_instances

Specifies list of cells that will be partitioned separately in the Dynamic Power Shaping (DPS) optimization.

Data Types

list_of_strings

Default ""

Description

DPS optimization adjusts clock latency to improve dynamic power integrity.

By using the application option *ccd.dps.powershape_separate_instances* it is possible to specify a list of instances that should be placed in individual partitions. This could be instances where timing is difficult (setup- or hold-time violations), such as RAM macros.

See Also

- [set_app_options](#)
- [place_opt](#)

ccd.dps.stepped_psg_window_size

Specifies the stepped window size used in the Dynamic Power Shaping (DPS) optimization.

Data Types

Pair

Default {100 100}

c

Description

DPS optimization adjusts clock latency to improve dynamic power integrity.

During a DPS optimization, it automatically generates a set of PSG windows by stepping a window of a given size across the design, with a given step size. This allows for an optimization to focus on reducing peak power locally, which is particularly useful when aiming to reduce dynamic voltage drop in flip-chip designs, in which the on-chip voltage drop has a localized nature.

Examples

The following example changes the window size to 200um x 200um:

```
prompt> set_app_options \\  
        -name ccd.dps.stepped_psg_window_size \\  
        -value {200 200}
```

See Also

- [set_app_options](#)
- [place_opt](#)

ccd.dps.use_case_type

Specified the use case type for default use cases in Dynamic Power Shaping (DPS) power analysis.

Data Types

string

Default "autovector"

Description

This option allows the user to change the default use case type used for DPS power analysis between "autovector" and "activity".

For the "autovector" type, two use cases will be generated per scenario: One with `high_power_instances_mode` high and one with `high_power_instances_mode` low. This results in the following use case definition:

```
{name RAMs_active scenario * high_power_instances_mode high type autovector}  
{name RAMs_idle scenario * high_power_instances_mode low type autovector}}
```

c

For the "activity" type, one use case will be generated per scenario with the activity type. This gives the following use case definition:

```
{{name default_use_case scenario * type activity}}
```

If the user specifies a value for the `ccd.dps.use_cases` app option, the `use_case_type` app option will be ignored.

See Also

- [set_app_options](#)
- [place_opt](#)

ccd.dps.use_cases

Specifies list of parameters defining use-cases for DPS optimization.

Data Types

list of list of strings

Default <empty>

Description

This option specifies the circuit activation used for the DPS optimization. The DPS optimization relies on short logic simulations, which generates the current signatures used for the optimization process.

The simulation is vectorless, and honors the circuit timing. The simulation should provide a realistic worst case view of the peak current draws.

Each use case is linked to a number of scenarios, the optimization is run for all use cases and all specified scenarios. In order to limit run time overhead of the DPS optimization it is suggested to reduce the number of use cases, and the number of scenarios for each use case.

When selecting scenarios for the use cases, favor those which have many clock sinks, high power, and which are expected to represent a critical activation of the circuit.

The activation of high power cells can be controlled by setting a 'power mode' separately from the regular sequential cell activity. This helps ensuring that the simulation provides a valid view of the power signatures. High power cells can be detected automatically or specified explicitly using the application options: `ccd.dps.analyze_high_power_instances`, `ccd.dps.analyze_high_power_instances_autodetect` and `ccd.dps.analyze_high_power_instances_autodetect_threshold`.

c

The option is equipped with a syntax checker that verifies the validity of the keys and values. Note that, as the validation is linked solely to the option, every value whose dependencies are associated with other options is waived by the checker.

USE CASE PROPERTIES

The properties of the use cases is specified as keys alternating with values. The following properties are defined:

name

(Required) Specifies the name of the use case. Can be used to refer to the use case when setting up optimization targets and for reporting.

scenario

Specifies the scenario(s) that the use-case belongs to. Wildcards '*' and '?' are accepted. Multiple names and wildcards may be separated by spaces. (waived by syntax check)

Default: "", results to all scenarios.

vectorless_activity

Specifies an activity factor between 0 and 1. The activity factor determines the probability of outputs of sequential cells toggling in each clock-cycle. Setting this too high may result in too pessimistic modeling of the transient current.

Default: 0.2

vectorless_port_activity

Specifies an activity factor between 0 and 1. The activity factor determines the probability of input ports toggling in each clock-cycle. Inputs ports will only toggle if there is an input delay specified for the port. Setting this too high may result in too pessimistic modeling of the transient current.

Default: 0.2

high_power_instances_mode

High power instances are sinks which have a high power consumption. The value specifies the power-mode for events on the clock sinks, forcing the cells into a given pin state during the autovector power analysis:

high

Cells are always set to use highest possible dynamic power peak.

low

Cells are always set to use lowest possible dynamic power peak.

c

zero

Cells are not using any power.

simulated

Cell power is determined according to actual input pin states.

statistical

Randomly use either highest or lowest possible dynamic power peak. Probability for each state is fixed at 50%.

Default: ""

length

Specifies the simulation time. The value accepts time unit, if no time unit is given the user unit is used. The simulation time should generally correspond to 4-5 clock cycles of the clock with the most sinks. If the length is not set then it is adaptively computed based on the clock whose faster clocks cumulatively cover at least 90% of the sinks. (waived by syntax check)

Default: "", result to adaptive length based on clock period

active_clocks

Specifies which clocks are used during power analysis, and hence considered during power optimization. (waived by syntax check)

Default: "", results to all clocks.

Examples

When use cases string is left empty, the default use cases are controlled by the *ccd.dps.use_case_type* app option. The default value will create two use-cases, named RAMs_active and RAMs_idle. One with high power mode; the other with low power mode. They are active for all scenarios.

The below setting will create only a single use case for which the length is increased to 15ns and which is enabled for all scenarios:

```
{{name RAMs_active scenario * high_power_instances_mode high length 15ns}}
```

In order to reduce the runtime, the number of scenarios used for power optimization can be reduced. Below only the scenarios scen1 and scen2 are referenced in the use cases.

```
{{name RAMs_active scenario "scen1 scen2" high_power_instances_mode high length 10ns}}
```

```
{name RAMs_idle scenario "scen1 scen2" high_power_instances_mode low length 10ns}}
```


See Also

- [set_app_options](#)
- [place_opt](#)

ccd.enable_clock_drc_improvements

Enable clock drc fixing enhancements in CCD.

Data Types

Boolean

Default false

Description

Enable clock drc fixing enhancements in CCD. When this app option is enabled, we can expect better clock max transition results. Please note that enabling this feature may increase runtime.

See Also

- [set_app_options](#)

ccd.enable_multi_vector_ccd_tns_register_sizing

Enable multi vector CCD register sizing for timing in hyper_route_opt.

Data Types

boolean

Default false

Description

Enable register sizing for better timing in hyper_route_opt CCD. Please note that enabling this feature may increase runtime.

See Also

- [set_app_options](#)
- [hyper_route_opt](#)

ccd.enable_register_sizing

Enable CCD register sizing for timing in clock_opt final_opto, route_opt, and hyper_route_opt.

Data Types

boolean

Default false

Description

Enable register sizing for better timing in clock_opt final_opto, route_opt, and hyper_route_opt CCD. Please note that enabling this feature may increase runtime.

See Also

- [set_app_options](#)
- [clock_opt](#)
- [route_opt](#)
- [hyper_route_opt](#)

ccd.enable_top_wns_optimization

Controls whether concurrent clock and data optimization (CCD) further optimize top WNS endpoints.

Data Types

Boolean

Default false

Description

The *ccd.enable_top_wns_optimization* application option controls whether CCD performs further optimization on top WNS endpoints. If this option is turned on, CCD will further optimize the TNS of top 300 WNS endpoints.

This option only affects CCD in compile_fusion, place_opt and clock_opt build_clock steps.

See Also

- [set_app_options](#)

ccd.fmax_optimization_effort

Effort level to focus on wns optimization inside CCD engine.

Data Types

string

Default *low*

Description

CCD optimization adjusts clock latency to improve timing and power. User can control CCD engine to spend different effort on WNS optimization. Higher wns optimization effort may sacrifice TNS and power to achieve better WNS result. This app option can be used during clock_opt final_opto step and route_opt with CCD enabled. It supports *low*, *medium* and *high*, and *low* is the default setting.

See Also

- [set_app_options](#)

ccd.hold_control_effort

set the effort level to control hold degradation during ccd setup optimization

Data Types

String

Default *low*

Description

This application option specifies the effort level for hold during CCD optimization. Higher effort level will result in better hold timing during CCD optimization but setup timing improvement during CCD can reduce.

Valid values are

- *none*
- *low*
- *medium*
- *high*
- *ultra*

c

This setting will impact CCD optimization in all stages provided hold scenarios are active during the CCD optimizations.

See Also

- [set_app_options](#)

ccd.ignore_ports_for_boundary_identification

Specifies the ports to exclude during boundary identification. This application option is honored only when the *ccd.optimize_boundary_timing* application option is *false*.

Data Types

string

Default {}

Description

The *ccd.ignore_ports_for_boundary_identification* application option specifies the ports to exclude when concurrent clock and data optimization (CCD) performs boundary identification. Typically you would specify high-fanout ports, such as scan enable and reset ports, to prevent large numbers of flip-flops from being identified as boundary flip-flops.

The tool performs boundary identification only when the *ccd.optimize_boundary_timing* application option is *false*. If the *ccd.optimize_boundary_timing* application option is *true*, the tool ignores the setting of the *ccd.ignore_ports_for_boundary_identification* application option.

Examples

The following examples ignore the dddi[25] and clk1 ports during boundary identification.

```
prompt> set_app_options \\  
        -name ccd.ignore_ports_for_boundary_identification \\  
        -value "dddi[25] clk1"  
ccd.ignore_ports_for_boundary_identification {dddi[25] clk1}
```

```
prompt> set_app_options \\  
        -name ccd.ignore_ports_for_boundary_identification \\  
        -value [list dddi[25] clk1]  
ccd.ignore_ports_for_boundary_identification {{dddi[25]}} clk1}
```

The following example ignores the input ports during boundary identification.

```
prompt> set_app_options \\  
        -name ccd.ignore_ports_for_boundary_identification \\  
        -value [get_object_name [get_ports -filter "direction==in"]]
```

c

```
ccd.ignore_ports_for_boundary_identification {rst clk1 clk2 clk3 clk4
{sel[2]} {sel[1]} {sel[0]} {di[63]} {di[62]} {di[61]} {di[60]} ... }
```

See Also

- [set_app_options](#)

ccd.ignore_scan_reset_for_boundary_identification

Specifies to exclude scan/reset connectivity during boundary identification. This application option is honored only when the *ccd.optimize_boundary_timing* application option is *false*.

Data Types

boolean

Default `true`

Description

The *ccd.ignore_scan_reset_for_boundary_identification* application option is used to ignore the connectivities on scan/reset pins when concurrent clock and data optimization (CCD) performs boundary flop identification.

The tool performs boundary flop identification only when the *ccd.optimize_boundary_timing* application option is *false*. If the *ccd.optimize_boundary_timing* application option is *true*, the tool ignores the setting of the *ccd.ignore_scan_reset_for_boundary_identification* application option.

See Also

- [set_app_options](#)

ccd.legal_aware_buffering

Enable legal-aware buffering in during Concurrent Clock and Data (CCD) optimization.

Data Types

bool

Default `false`

Description

This application option helps to find legal locations for multiple buffers insertion in dense area during CCD optimization. Note that this application option is effective only when Concurrent Legalization and Optimization (CLO) is enabled.

See Also

- [set_app_options](#)

ccd.macro_seed_offset_file

Specifies the name of an (optional) file where seed offsets (as computed by the strategy selected in *ccd.macro_seed_offset_strategy*) are written.

Data Types

string

Default (no value)

Description

To allow customization of macro seed offsets, the *ccd.macro_seed_offset_file* application option can be set to a valid file name.

During execution of the flow core commands (*compile_fusion* and *place_opt*), and if a strategy other than "none" is selected in the *ccd.macro_seed_offset_strategy* application option, a file with this name is created.

The generated file contains, for every macro instance, the necessary *set_clock_tree_balance_points* and *set_clock_latency* constraints (in the form of a Tcl script) that were generated by the selected *ccd.macro_seed_offset_strategy*.

This file can then be edited with custom values for either balance points or clock latencies, and then sourced before executing any flow command in a ulterior run. The customized values are used instead of the ones generated by the selected strategy.

For designs with multiple active scenarios, the generated file contains values for one scenario only. The values for other scenarios are automatically derived using the *copy_useful_skew* command.

If an absolute path is not specified, the file is saved to the current directory. If the file already exists, it is not overwritten.

Examples

In order to save the seed offsets computed by the "model" strategy to a file:

```
prompt> set_app_options -name ccd.macro_seed_offset_strategy -value model
prompt> set_app_options -name ccd.macro_seed_offset_file -value
      seed_offsets.tcl
prompt> place_opt
```

After the *place_opt* command finishes, the *seed_offsets.tcl* file can be viewed and edited with customized offsets for macros.

c

To use the customized offsets for a second run, the edited file must be sourced before starting any of the flow commands.

```
prompt> source seed_offsets.tcl
prompt> place_opt
```

The customized clock balance points and clock latencies are always used, irrespective of the value of the `ccd.macro_seed_offset_strategy` option.

See Also

- [set_app_options](#)
- [set_clock_balance_points](#)
- [set_clock_latency](#)
- [copy_useful_skew](#)

ccd.macro_seed_offset_strategy

Controls the initial useful skew applied to macros during *compile_fusion* and *place_opt*.

Data Types

string

Default "none"

Description

The *ccd.macro_seed_offset_strategy* application option controls the initial (seed) clock latency offsets and clock tree balance points which are applied to macros having internal clock tree models.

When activated, seed offsets override or mask the internal clock tree models of macros, depending on the strategy selected. Seed offsets are applied automatically during *compile_fusion* and *place_opt*, allowing a more consistent timing picture between pre-CTS and post-CTS.

If a macro clock tree pin has a preexisting clock tree balance point before starting any flow core command, the preexisting clock balance points are always honored and not modified, irrespective of the strategy selected.

Seed offsets are generated even if CCD is disabled. When CCD is enabled, the seed macro offsets are individually optimized through the flow; the clock balance points at the end of the flow command might not necessarily retain the initial values of the seed offsets. To customize the seed offsets, please enable application option *ccd.macro_seed_offset_file*.

c

Valid values for this application option are:

"none"

With this strategy, no seed offsets are applied during *compile_fusion* or *place_opt*. The clock balance points and clock latencies of macro clock pins are not affected. During *clock_opt*, CTS uses the internal clock tree delay of macros as a guideline for clock tree construction. This is the default.

"zero"

A clock tree balance point with zero delay is applied on all macro clock pins with an internal clock tree model, ignoring the internal clock delay of the macro. The ideal clock latencies for the macros remain unchanged. This strategy is equivalent to the (obsolete) *ccd.skew_opt_default_balance_point_macros* option. It is the recommended strategy if the internal clock tree model should be ignored for building the clock tree.

"model"

For all macros with an internal clock tree model, both the ideal clock latency and the clock tree balance point delay of the macro clock pins are updated with a prepone compensating the macro's internal clock tree delay. The internal clock tree delay is used in full even if it exceeds other useful skew limits such as *ccd.max_prepone*. This strategy is recommended whenever the internal clock tree models are to be used for building the clock tree.

See Also

- [set_app_options](#)
- [set_clock_latency](#)
- [set_clock_balance_points](#)

ccd.max_postpone

Specifies the maximum postpone value adjusted from concurrent clock and data (CCD) optimization.

Data Types

string

Default auto

c

Description

CCD optimization adjusts clock latency to improve timing and power. This option controls all CCD steps throughout the flow including `compile_fusion`, `place_opt`, `clock_opt`, and `route_opt`.

To limit the scope of the adjustment from these optimizations throughout the flow, use the `ccd.max_prepone` application option to limit the latency reduction and the `ccd.max_postpone` application option for latency increase. Specify a positive value in user input units for these application options.

The pre- and postpone limits will apply to the reference corner in CCD which is per default automatically selected. For scenarios in other corners the limits will scale according to the corner scaling factors. Please use the `ccd.reference_corner` app option for selecting a specific reference corner that the range limits should apply to.

By default, there is a 200ps postpone limit in `compile_fusion` and `place_opt`, and there is no limit in `clock_opt` and `route_opt`.

Example: Below setting will ensure that the max postpone allowed during CCD is 0.1ns (design units is 1ns).

```
set_app_options -name ccd.max_postpone -value 0.1
```

See Also

- [set_app_options](#)

ccd.max_prepone

Specifies the maximum prepone value adjusted from concurrent clock and data (CCD) optimization.

Data Types

string

Default auto

Description

CCD optimization adjusts clock latency to improve timing and power. This option controls all CCD steps throughout the flow including `compile_fusion`, `place_opt`, `clock_opt`, and `route_opt`.

To limit the scope of the adjustment from these optimizations throughout the flow, use the `ccd.max_prepone` application option to limit the latency reduction and the `ccd.max_postpone` application option for latency increase. Specify a positive value in user input units for these application options.

c

The pre- and postpone limits will apply to the reference corner in CCD which is per default automatically selected. For scenarios in other corners the limits will scale according to the corner scaling factors. Please use the `ccd.reference_corner` app option for selecting a specific reference corner that the range limits should apply to.

By default, there is a 300ps prepone limit in `compile_fusion` and `place_opt`, and there is no limit in `clock_opt` and `route_opt`.

Example: Below setting will ensure that the max prepone allowed during CCD is 0.1ns (design units is 1ns).

```
set_app_options -name ccd.max_prepone -value 0.1
```

See Also

- [set_app_options](#)

ccd.optimize_boundary_timing

Controls whether concurrent clock and data optimization (CCD) optimizes boundary timing paths.

Data Types

Boolean

Default true

Description

The `ccd.optimize_boundary_timing` application option controls whether CCD performs optimization on boundary timing paths.

By default(*true*), CCD optimizes boundary timing paths. In this case, all flip-flops in the block are available for CCD. The tool does not perform boundary identification and ignores the setting of the `ccd.ignore_ports_for_boundary_identification` and `ccd.ignore_scan_reset_for_boundary_identification` application options.

To disable optimization on boundary timing paths, such as I/O paths, set the `ccd.optimize_boundary_timing` application option to *false*. When this option is *false*,

- Only internal flip-flops are available for CCD

The tool performs boundary identification to differentiate between the boundary and internal flip-flops. If your design contains ports that connect to a large number of flip-flops, such as scan enable and reset ports, you might want to exclude these ports from the boundary identification; otherwise, most of the flip-flops are considered boundary flip-flops and the CCD scope is very limited. To specify the ports to exclude, set the `ccd.ignore_ports_for_boundary_identification` application option; If you specify

c

ccd.ignore_scan_reset_for_boundary_identification application option, all of the scan/reset ports will not to be considered.

- The tool does not adjust the latencies of the boundary flip-flops
- The tool can optimize the clock tree fanin to the boundary flip-flops if the clock path is shared with an internal flip-flop

To disable optimization on the clock tree fanin to the boundary flip-flops, set the *ccd.optimize_boundary_timing_upstream* application option to *false*.

Note that setting both the *ccd.optimize_boundary_timing* and *ccd.optimize_boundary_timing_upstream* application options to *false* prevents CCD from optimizing most of the clock tree.

See Also

- [set_app_options](#)

ccd.optimize_boundary_timing_upstream

Controls whether concurrent clock and data optimization (CCD) can optimize the clock tree fanin to the boundary flip-flops. This application option is honored only when the *ccd.optimize_boundary_timing* application option is *false*.

Data Types

Boolean

Default true

Description

The *ccd.optimize_boundary_timing_upstream* application option controls whether CCD can optimize the clock tree fanin to the boundary flip-flops.

By default (*true*), CCD can optimize the clock tree fanin to the boundary flip-flops if it is shared with an internal flip-flop.

To disable optimization on the clock tree fanin to the boundary flip-flops, set the *ccd.optimize_boundary_timing_upstream* application option to *false*.

Note that setting both the *ccd.optimize_boundary_timing* and *ccd.optimize_boundary_timing_upstream* application options to *false* prevents CCD from optimizing most of the clock tree.

See Also

- [set_app_options](#)

ccd.post_route_buffer_removal

Specify if concurrent clock and data (CCD) performs buffer removal during post route optimization.

Data Types

boolean

Default false

Description

This option specifies if CCD will perform buffer or inverter pair removal during post route optimization. By default, it is false. Buffer or inverter pair removal is performed by CCD in pre route by default.

See Also

- [set_app_options](#)

ccd.reference_corner

Specifies the corner which maximum prepone and postpone limits for concurrent clock and data (CCD) optimization apply to.

Data Types

string

Default <empty>

Description

When limits for prepone and postpone are being set, their values apply to a reference corner and appropriate scaling is applied for the limits for other corners.

The reference corner is by default automatically selected by the tool. Use the *ccd.reference_corner* app option to explicitly specify which corner the prepone/postpone limits.

See Also

- [set_app_options](#)

ccd.respect_cts_fixed_balance_pins

Used to control CCD engine not to optimize user specified sinks pins

c

Data Types*string***Default** "false"**Description**

This application option is used to exclude the sink pins with attribute *cts_fixed_balance_pin* during CCD optimization and the tool will not adjust the latencies of these pins.

The valid values are

- *false* CCD is free to adjust the latencies of the sink pins and does not honor the *cts_fixed_balance_pin* attribute on a pin
- *true* CCD will not adjust the latencies of the sink pins with attribute *cts_fixed_balance_pin*. But the latencies to the sink pins with attribute *cts_fixed_balance_pin* can still change because the clock cells in the clock path to these sink pins can still be optimized by CCD when skewing other sink pins
- *upstream* CCD will not adjust the latencies of the sink pins with attribute *cts_fixed_balance_pin*. Tool will not do any change to the clock cells in the clock path to these sinks

User can change the values at any stage depending on where they want the skewing to be disallowed/allowed.

Examples

The following example controls CCD engine to avoid to adjust latency for sinks pins with attribute *cts_fixed_balance_pin*:

```
prompt> set_app_options -name ccd.respect_cts_fixed_balance_pins -value true
```

To prevent optimization on upstream clock cells to the *cts_fixed_balance_pin* sink:

```
prompt> set_app_options -name ccd.respect_cts_fixed_balance_pins -value upstream
```

The flop clock pins that need not be skewed by CCD should have the following setting:

```
set ff_cp [get_pins <flop clock pins>]
set_attribute $ff_cp cts_fixed_balance_pin true
```

c

See Also

- [set_attribute](#)
- [get_attribute](#)
- [set_app_options](#)

ccd.skew_opt_default_balance_point_macros

Adds automatically zero balance point on macro clock pins.

Data Types

Boolean

Default false

Description

The *ccd.skew_opt_default_balance_point_macros* application option adds automatically zero balance point on macro clock pins. This practice is advised in order to get consistent results independent of internal clock latency in macros.

See Also

- [set_app_options](#)

ccd.skip_path_groups

Skip the path groups for concurrent clock and data (CCD) optimization.

Data Types

string

Default empty string

Description

This option provides a list of path groups and CCD will skip these path groups during critical endpoint selection for optimization. This app option does not skip optimization on data path on the specified path groups. This app option affects all concurrent clock and data optimizations in all stages.

See Also

- [set_app_options](#)

ccd.targeted_ccd_end_points_file

Specify a file which contains the end points for targeted concurrent clock and data (CCD) optimization.

Data Types

string

Default empty string

Description

Using this option, user can specify a list of end points through a file for CCD to focus on for WNS optimization. This is honored by CCD in compile_fusion, place_opt, clock_opt build_clock and also by targeted route_opt CCD . At compile_fusion, place_opt and clock_opt build_clock stage, CCD will focus on the paths to these targeted endpoints along with design WNS and TNS.

In route_opt stage, targeted CCD will consider only specified critical end points during critical endpoint selection for optimization. The other end points which are not specified are skipped during targeted CCD in route_opt.

See Also

- [set_app_options](#)

ccd.targeted_ccd_path_groups

Specify the path groups for targeted concurrent clock and data (CCD) optimization.

Data Types

string

Default empty string

Description

Using this option, user can specify a list of path groups for CCD to focus on for WNS optimization. This is honored only by CCD in compile_fusion. It is not recommended to use this with targeted endpoint flow and needs to be reset before TEP route_opt.

See Also

- [set_app_options](#)

ccd.targeted_ccd_threshold_for_nontargeted_path_groups

Set wns threshold of nontargeted path groups for targeted concurrent clock and data (CCD) optimization.

Data Types

string

Default 0.0

Description

This option allows wns degradation of nontargeted path groups up to the specified threshold. Allowable values is negative floating value.

When initial WNS value of nontargeted path groups is already lower than (that is more negative than) this threshold value, then targeted CCD cannot degrade it further.

Example: If the starting WNS value for non-targeted path groups is -0.05 and the user set this app option to -0.08, targeted CCD can degrade the WNS of non-targeted path groups till it becomes -0.08.

If the starting WNS value for non-targeted path group is -0.05 and the user set this app option to -0.03, targeted CCD cannot degrade the WNS of the non-targeted path group from -0.05.

See Also

- [set_app_options](#)

ccd.targeted_ccd_wns_optimization_effort

Select optimization effort to improve targeted wns for targeted concurrent clock and data (CCD) optimization.

Data Types

string

Default high

Description

This option selects optimization effort for targeted CCD optimization. Allowable options are "high", "medium" and "low". Default effort is "high".

Higher effort means better timing QoR improvement for user specified path groups and endpoints but might degrade timing for non-specified path groups and endpoints.

c

Low effort will minimize the timing improvements while maintaining the QoR of the non-specified path groups and endpoints.

See Also

- [set_app_options](#)

ccd.timing_effort

Effort level to focus on timing optimization inside CCD engine.

Data Types

string

Default medium

Description

CCD optimization adjusts clock latency to improve timing and power. User can control CCD engine to spend different effort on timing optimization. Higher effort may sacrifice power to achieve better timing result. It supports *ultra_low*, *low*, *medium* and *high*, and *medium* is the default setting. This app option affects all concurrent clock and data optimizations in clock_opt and route_opt stages.

See Also

- [set_app_options](#)

ccd.voltage_drop_population_threshold

Set voltage drop population ratio for IR Driven concurrent clock and data (CCD) optimization.

Data Types

float

Description

This option allows experts to specify threshold ratio for picking the cells with the highest voltage drop, relative to the number of cells.

This can determine how many cells that IR Driven CCD can try to consider.

Allowable values is non-negative floating value from 0.0 to 1.0 inclusive.

The tool automatically determines the cells to fix by default.

c

Example: If user set this app option to 0.01, IR Driven CCD considers top 1% cells with worst voltage drop as cells violating voltage drop requirement and considers them in the optimization

See Also

- [set_app_options](#)

ccd.voltage_drop_voltage_threshold

Set voltage drop voltage threshold for IR Driven concurrent clock and data (CCD) optimization.

Data Types

float

Description

This option allows experts to specify threshold ratio for picking the cells with that much voltage drop or higher, relative to the supply voltage.

This can determine how many cells that IR Driven CCD can try to consider.

Allowable values is non-negative floating value from 0.0 to 1.0 inclusive.

The tool automatically determines the cells to fix by default.

Example: If user set this app option to 0.01, IR Driven CCD considers cells with voltage drop higher than 1% of VDD as cells violating voltage drop requirement and considers them in the optimization

See Also

- [set_app_options](#)

chf.create_stdcell_fillers.max_leakage_vt_order_violations

Used to control the max number of leakage vt order violations.

Data Types

integer

Default 100

c

Description

When using option `-leakage_vt_check` for command `create_stdcell_fillers`, this app option can be used to control the max number of violations reported. By default it will report at most 100 violations.

See Also

- [create_stdcell_fillers](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

chipfinishing.1cpp_channel_trans_plus_filler_lib_cells

1-CPP channel Trans-Plus filler libcell names

Data Types

String set of libcell names

Default Empty

Description

Specified the special trans-plus filler libcell names for used in 1-CPP channels. Wildcard syntax is supported.

Examples

The following example shows the format of using this application option.

```
prompt> set_app_options -name
chipfinishing.1cpp_channel_trans_plus_filler_lib_cells -value
{ FILLTRANP2N0* FILLTRANP0N2* }
```

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

chipfinishing.balanced_filler_lib_cells

Balanced-NPN filler libcell names

c

Data Types*String set of libcell names***Default** Empty**Description**

Specified the filler libcell names for inbound logic cells. Wildcard syntax is supported.

Examples

The following example shows the format of using this application option.

```
prompt> set_app_options -name chipfinishing.inbound_filler_lib_cells  
-value { FILLP6N0* FILLP0N6* }
```

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

chipfinishing.balanced_pad_lib_cells

Balanced-NPN filler libcell names

Data Types*String set of libcell names***Default** Empty**Description**

Specified the filler libcell names for inbound logic cells. Wildcard syntax is supported.

Examples

The following example shows the format of using this application option.

```
prompt> set_app_options -name chipfinishing.inbound_filler_lib_cells  
-value { FILLP6N0* FILLP0N6* }
```

c

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

chipfinishing.default_transition_z_width

Sets default z width

Data Types

float

Default 0.0

Description

Specifies the default z width value according to the cell height of the design. The unit is in micron. The default is 0.0. If value is 0.0, then it is considered that default z width has not been set.

Examples

The following example shows the format of using this application option.

```
prompt> set_app_options -name chipfinishing.default_transition_z_width  
-value {0.032}
```

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

chipfinishing.enable_advanced_legalizer_postfixing

Specifies whether advanced legalizer is enabled for post-fixing in hybrid-row flow. The post-fixing includes (1) replace single-row height fill wall cell with double-row height fill wall cell, (2) replace boundary cell with pTap boundary cell near Ncorner cell when row number meets the criterion, (3) replace 2X tall-row pTap cell with 2X short-row pTap cell if available.

Data Types

boolean

c

Default false**Description**

Specifies whether advanced legalizer is enabled for post-fixing in hybrid-row flow. The post-fixing includes (1) replace single-row height fill wall cell with double-row height fill wall cell, (2) replace boundary cell with pTap boundary cell near Ncorner cell when row number meets the criterion, (3) replace 2X tall-row pTap cell with 2X short-row pTap cell if available.

chipfinishing.enable_al_tap_insertion

Specifies whether the new N3 tap insertion engine is enabled.

Data Types

boolean

Default False**Description**

Specifies whether the new N3 tap insertion engine is enabled.

~ ~

chipfinishing.enable_area_optimization

Specifies whether to replace boundary cell with PTap boundary cell near NCorner cell.

Data Types

boolean

Default fals**Description**

Specifies whether to replace boundary cell with PTap boundary cell near NCorner cell.

chipfinishing.enable_auto_orient_base_on_tap_coverage_table

In N3 design, enable this option when "left_boundary" and "right_boundary" share same libCell, "p_left_tap" and "p_right_tap" share same libCell, "n_left_tap" and "n_right_tap" share same libCell. The tap-coverage tables of each libCell must be defined in PRF. Tool will decide cell orientation based on tap-coverage table.

c

Data Types

Boolean

Default False

Description

In N3 design, enable this option when "left_boundary" and "right_boundary" share same libCell, "p_left_tap" and "p_right_tap" share same libCell, "n_left_tap" and "n_right_tap" share same libCell. The tap-coverage tables of each libCell must be defined in PRF. Tool will decide cell orientation based on tap-coverage table.

chipfinishing.enable_column_based_tap_insertion

Enhancement of N3/N2 tap insertion. compile_tap_boundary_wall_cells would put most of tap cells column-by-column. This application option improves the runtime of compile_tap_boundary_wall_cells.

Data Types

boolean

Default false

Description

Enhancement of N3/N2 tap insertion. compile_tap_boundary_wall_cells would put most of tap cells column-by-column. This application option improves the runtime of compile_tap_boundary_wall_cells.

chipfinishing.enable_disjointed_site_rows

Enables disjointed site process.

Data Types

Boolean

Default True

Description

To disable disjoint site process.

Examples

The following example disable disjoint site process.

c

```
prompt> set_app_options\\  
        -name chipfinishing.enable_disjointed_site_rows -value false
```

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

chipfinishing.enable_even_uniform_row_pattern

Specifies whether the input is uniform-row and even-row design in specific technology node.

Data Types

boolean

Default false

Description

Specifies whether the input is uniform-row and even-row design in specific technology node.

chipfinishing.inbound_complete_fill_gap_cpp

Number of gaps, in cpp, to be completely filled between two Inbound-Max logic cells

Data Types

Integer value to indicate cpp

Default 0

Description

Specified the gap size in cpp to be completely filled between two Inbound-Max logic cells.

Examples

The following example shows the format of using this application option.

```
prompt> set_app_options -name chipfinishing.inbound_complete_fill_gap_cpp  
        -value { 6 }
```


c

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

chipfinishing.inbound_filler_lib_cells

Inbound filler libcell names

Data Types

String set of libcell names

Default Empty

Description

Specified the filler libcell names for inbound logic cells. Wildcard syntax is supported.

Examples

The following example shows the format of using this application option.

```
prompt> set_app_options -name chipfinishing.inbound_filler_lib_cells  
-value { FILLP6N0* FILLP0N6* }
```

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

chipfinishing.inbound_max_lib_cells

Inbound-Max libcell names

Data Types

String set of libcell names

Default Empty

Description

Specified the libcell names for Inbound-Max logic cells. Wildcard syntax is supported.

c

Examples

The following example shows the format of using this application option.

```
prompt> set_app_options -name chipfinishing.inbound_max_lib_cells -value  
{ *Z6* }
```

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

chipfinishing.inbound_plus_filler_lib_cells

Inbound-Plus filler libcell names

Data Types

String set of libcell names

Default Empty

Description

Specified the filler libcell names for inbound-plus logic cells. Wildcard syntax is supported.

Examples

The following example shows the format of using this application option.

```
prompt> set_app_options -name chipfinishing.inbound_plus_filler_lib_cells  
-value { FILLP6N0* FILLP0N6* }
```

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

chipfinishing.tap_cover_power_domain_lib_cells

Settings of different power domain lib cells for different voltage areas.

Data Types

String set of lib cells

c

Default Empty**Description**

Specifies different power domain lib cells for different voltage areas. When this application option is used, the `-va_mapping` must be used in `set_tap_boundary_wall_cell_rules`.

Examples

The following example shows the format of using this application option. Using `-va_mapping {VA1 VPP_SET VA2 VBB_SET DEFAULT_VA DEFAULT}` in `set_tap_boundary_wall_cell_rules` can replace NW tap cells with VPP cells in VA1 and replace PW tap cells with VBB cells in VA2. This usage also specified there is no replacement inside the `DEFAULT_VA`.

```
prompt> set_app_options\\
-name chipfinishing.tap_cover_power_domain_lib_cells -value {
{ DEFAULT "
  TAPCELL_PW
  TAPCELL_NW
  TAPCELL_PWWALL
  TAPCELL_NWWALL
  BOUNDARY_PROWPWTAPWALL
  BOUNDARY_NROWNWTAPWALL
  BOUNDARY_LEFTPWTPAP
  BOUNDARY_LEFTTNWTAP
  BOUNDARY_RIGHTPWTPAP
  BOUNDARY_RIGHTTNWTAP
  BOUNDARY_PROWPWTAP
  BOUNDARY_NROWNWTAP
  BOUNDARY_PCORNERPWTPAP
  BOUNDARY_NCORNERNWTAP
  BOUNDARY_PINCORNERPWTPAP
  BOUNDARY_PINCORNERNWTAP
  BOUNDARY_NINCORNERPWTPAP
  BOUNDARY_NINCORNERNWTAP " }
{ VPP_SET "
  TAPCELL_PW
  TAPCELL_VPP
  TAPCELL_PWWALL
  TAPCELL_VPPWALL
  BOUNDARY_PROWPWTAPWALL
  BOUNDARY_NROWVPPTAPWALL
  BOUNDARY_LEFTPWTPAP
  BOUNDARY_LEFTVPPTAP
  BOUNDARY_RIGHTPWTPAP
  BOUNDARY_RIGHTVPPTAP
  BOUNDARY_PROWPWTAP
  BOUNDARY_NROWVPPTAP
  BOUNDARY_PCORNERPWTPAP
  BOUNDARY_NCORNERVPPTAP
  BOUNDARY_PINCORNERPWTPAP
```

c

```

BOUNDARY_PINCORNervPPTAP
BOUNDARY_NINCORNerpWTAP
BOUNDARY_NINCORNervPPTAP " }
{ VBB_SET "
  TAPCELL_VBB
  TAPCELL_NW
  TAPCELL_VBBWALL
  TAPCELL_NWWALL
  BOUNDARY_PROVVBbtAVBBall
  BOUNDARY_NROWNwtAVBBall
  BOUNDARY_LEFTVBBtAP
  BOUNDARY_LEFTNwtAP
  BOUNDARY_RIGHTVBBtAP
  BOUNDARY_RIGHTNwtAP
  BOUNDARY_PROVVBbtAP
  BOUNDARY_NROWNwtAP
  BOUNDARY_PCORNervBBtAP
  BOUNDARY_NCORNERNwtAP
  BOUNDARY_PINCORNervBBtAP
  BOUNDARY_PINCORNERNwtAP
  BOUNDARY_NINCORNervBBtAP
  BOUNDARY_NINCORNERNwtAP " } }

```

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)
- [set_tap_boundary_wall_cell_rules](#)

chipfinishing.trans_balanced_half_height_lib_cells

Trans-Plus filler libcell names

Data Types

String set of libcell names

Default Empty

Description

Specified the trans-plus fillerlibcell names. Wildcard syntax is supported.

Examples

The following example shows the format of using this application option.

c

```
prompt> set_app_options -name chipfinishing.trans_plus_filler_lib_cells  
-value { FILLTRANP2N0* FILLTRANP0N2* }
```

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

chipfinishing.trans_balanced_lib_cells

Trans-Plus filler libcell names

Data Types

String set of libcell names

Default Empty

Description

Specified the trans-plus fillerlibcell names. Wildcard syntax is supported.

Examples

The following example shows the format of using this application option.

```
prompt> set_app_options -name chipfinishing.trans_plus_filler_lib_cells  
-value { FILLTRANP2N0* FILLTRANP0N2* }
```

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

chipfinishing.trans_balanced_to_max_lib_cells

Trans-Plus filler libcell names

Data Types

String set of libcell names

Default Empty

c

Description

Specified the trans-plus fillerlibcell names. Wildcard syntax is supported.

Examples

The following example shows the format of using this application option.

```
prompt> set_app_options -name chipfinishing.trans_plus_filler_lib_cells  
-value { FILLTRANP2N0* FILLTRANP0N2* }
```

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

chipfinishing.trans_balanced_to_plus_lib_cells

Trans-Plus filler libcell names

Data Types

String set of libcell names

Default Empty

Description

Specified the trans-plus fillerlibcell names. Wildcard syntax is supported.

Examples

The following example shows the format of using this application option.

```
prompt> set_app_options -name chipfinishing.trans_plus_filler_lib_cells  
-value { FILLTRANP2N0* FILLTRANP0N2* }
```

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

chipfinishing.trans_max_filler_lib_cells

Trans-plus filler libcell names

c

Data Types*String set of libcell names***Default** Empty**Description**

Specified the trans-plus filler libcell names. Wildcard syntax is supported.

Examples

The following example shows the format of using this application option.

```
prompt> set_app_options -name chipfinishing.trans_max_filler_lib_cells  
-value { FILLTRANP6N0* FILLTRANP0N6* }
```

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

chipfinishing.trans_plus_filler_lib_cells

Trans-Plus filler libcell names

Data Types*String set of libcell names***Default** Empty**Description**

Specified the trans-plus fillerlibcell names. Wildcard syntax is supported.

Examples

The following example shows the format of using this application option.

```
prompt> set_app_options -name chipfinishing.trans_plus_filler_lib_cells  
-value { FILLTRANP2N0* FILLTRANP0N2* }
```

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

chipfinishing.vt_od_rule_setting_check_gives_warning_only

Give warning only when vt od rule setting check

Data Types

Boolean

Default False

Description

Give warning only when vt od rule setting check

clock_opt.final_opto.final_clock_routing

Data Types

Boolean

Default true

Description

Runs clock routing at the end to patch any open clock routes, when set to true in the final_opto step of the *clock_opt*. No additional route_group -all_clock_nets is required. This application option only works on versions earlier than Q-2019.12-SP3 release. This feature is deprecated from version Q-2019.12-SP3 release and later, and this application option is no-op.

See Also

- [clock_opt](#)
- [route_group](#)

clock_opt.flow.enable_ccd

Enables concurrent clock and data (CCD) optimization during the *clock_opt* command.

c

Data Types

Boolean

Default `true`

Description

This application option controls whether the `clock_opt` command performs concurrent clock and data (CCD) optimization.

When this option is set to `true`, the `clock_opt` command performs CCD optimization. Setting this application option to `true` also sets the following application options to `true` during the execution of the `clock_opt` command:

- `cts.compile.enable_local_skew`

This application option enables local skew optimization during clock tree synthesis.

- `cts.optimize.enable_local_skew`

This application option enables local skew optimization during clock tree optimization.

See Also

- [set_app_options](#)
- [clock_opt](#)

`clock_opt.flow.enable_clock_power_recovery`

Enable power or area recovery from clock cells and registers during `clock_opt`, applies to both CCD and non-CCD flow.

Data Types

string

Default `auto`

Description

This option turns on power recovery or area recovery during CCD in `clock_opt final_opto`. When power recovery mode is run, CCD will work on improving the power depending upon the scenario settings for power. If only dynamic power scenarios are enabled, CCD will work on improving dynamic power alone. If both leakage and dynamic power scenarios are enabled, CCD will work on improving total power. When area recovery mode is run, CCD will work on improving the area.

c

During power or area recovery, CCD performs clock cell sizing, clock repeater removal, clock repeater sizing and flip-flop sizing.

The valid values and the respective behavior are as follows:

- *auto* (default value)

Power recovery CCD will be run for designs with scenarios enabled for dynamic power if *clock_opt.flow.enable_ccd* is true. There will not be any power recovery CCD run if *clock_opt.flow.enable_ccd* is set as false or if scenarios are not enabled for dynamic power.

- *power*

Power recovery CCD will be run for designs with scenarios enabled for dynamic power even if *clock_opt.flow.enable_ccd* is set as false. If there are no scenarios enabled for dynamic power, power recovery will not happen. Power recovery CCD is performed twice in the flow while power mode enabled.

- *area*

Area recovery CCD will be run instead of power recovery CCD. This will be run irrespective of whether *clock_opt.flow.enable_ccd* is set as true or false. Note that area mode is no longer supported in GRE flow.

- *none*

This will disable power recovery or area recovery CCD irrespective of whether *clock_opt.flow.enable_ccd* is set as true or false.

See Also

- [set_app_options](#)
- [clock_opt](#)

clock_opt.flow.enable_global_route_opt

Enables the *global_route_opt* stage during the *clock_opt* flow whenever it is not explicitly included as part of the standard *clock_opt* flow control using the *-from* and *-to* options.

Data Types

Boolean

Default false

c

Description

The `global_route_opt` stage is conditionally executed as the final stage of the `clock_opt` flow.

This application option controls whether the `global_route_opt` stage is executed when you run the `clock_opt` or `clock_opt -from` command without explicitly specifying the `global_route_opt` stage.

If you run the `clock_opt` command with the `-from` or `-to` stage as `global_route_opt`, the tool honors this specification and ignores the application option setting.

Examples

When you run either of the following commands, the `clock_opt` command executes the `global_route_opt` stage only when this application option is `true`.

```
prompt> clock_opt -to final_opto
```

or

```
prompt> clock_opt -from final_opto
```

When you run either of the following commands, the `clock_opt` command always executes the `global_route_opt` stage, regardless of the setting of this application option.

```
prompt> clock_opt -to global_route_opt
```

or

```
prompt> clock_opt -from global_route_opt
```

See Also

- [set_app_options](#)
- [clock_opt](#)

clock_opt.flow.enable_irap

Enables IR aware placement (IRAP) during `clock_opt`.

Data Types

Boolean

Default false

c

Description

When this option is set to *true*, IR aware placement optimization (IRAP) will be enabled during *clock_opt*. This will reduce the IR drop by reducing power density in high IR drop spots.

See Also

- [clock_opt](#)

clock_opt.flow.enable_irdrivenopt

Enables IR driven optimization during *clock_opt*.

Data Types

Boolean

Default false

Description

When this option is set to *true*, IR driven optimization will be enabled during *clock_opt*. This will reduce the IR drop by sizing datapath cells in high IR drop hotspots.

See Also

- [clock_opt](#)

clock_opt.flow.enable_multibit_debanking

Enable debanking of multibit sequential cells to improve total negative slack during *clock_opt* final_opto stage.

Data Types

Boolean

Default false

Description

This app option allows users to enable debanking of multibit sequential cells to improve total negative slack. The debanking is done to lower width multibit and/or single bit sequential cells. The tool also modifies the netlist and does scan chain related changes. The debanking is being done during *final_opto* stage of command *clock_opt*, and only *GRE based* *clock_opt* is supported.

See Also

- [clock_opt](#)

clock_opt.flow.enable_multibit_rewiring

Enable rewiring to swap the nets on bits of multibit cells to improve usage of mixed drive strength and mixed VT cells.

Data Types

Boolean

Default false

Description

This app option allows users to enable rewiring among bits of multibit cells to increase the number of mixed drive strength cells and mixed VT cells. Such kind of cells have few bits with higher drive strength and other with lower drive strength. Usage of such cells leads to better total power because of lower power consumption on lower drive strength bits. The target of swapping bits is to drive critical nets or bits having higher load with high drive strength or higher VT bits, reducing need of going to a multibit cell with all higher drive strength bits. The rewiring is done both during timing optimization and during power/area optimization. The benefits of this feature will be seen mostly in the designs having mixed drive strength or mixed VT multibit lib cells in the library.

See Also

- [clock_opt](#)

clock_opt.flow.enable_power

Data Types

boolean

Default true

Description

Specifies whether the *clock_opt* command enables power optimization. By default, power type is determined based on scenario settings as shown in the table below. If this app option is set to false, power optimization will be disabled regardless of scenario settings.

Leakage-scenario Dynamic-scenario -> FlowType

False False -> Area

c

True False -> Leakage

False True -> Total

True True -> Total

See Also

- [clock_opt](#)

clock_opt.flow.enable_voltage_drop_aware

Controls whether cts/cto uses RedHawk grid resistance data to take voltage drop metric into consideration.

Data Types

Boolean

Default false

Description

When this option is *true*, clock tree synthesis and clock tree optimization will use buffer location adjustment to reduce power grid resistance for buffers. By default this option is *false*.

See Also

- [synthesize_clock_trees](#)

clock_opt.flow.extra_size_only

Enables extra size only iteration at the end of clock_opt.

Data Types

String

Default false

Description

When this option is set to *true*, do one more opto iteration with sizing cells (equal or smaller) at the end of *clock_opt*. This will improve QoR without changing cell location. When this option is set to *footprint*, allows resizing only to those library cells that have the same *cell_footprint* attribute as the original cell. You must ensure that the library cells have the correct *cell_footprint* attributes.

c

See Also

- [clock_opt](#)

clock_opt.flow.optimize_layers

Enable layer optimization in `clock_opt`

Data Types

String

Default `false`

Description

This option has been deprecated; this option will be removed in a subsequent release.

However, the `clock_opt` command always performs layer optimization that is tightly integrated with buffering, which ensures that critical signals are promoted to low-RC layers and buffered to take advantage of those layers.

See Also

- [set_app_options](#)

clock_opt.flow.optimize_layers_critical_range

Set critical range for layer optimization when `clock_opt.flow.optimize_layers` is set to true.

Data Types

Float

Default `0.0001`

Description

It accepts values from $\{0 \sim 1.0\}$. For a given value y , it optimizes nets that have slacks between $\{y \cdot \text{WNS}, \text{WNS}\}$, where WNS is the worst negative slack. Set it to a lower value to fix paths that are in less critical range. It may cause a lot more nets to be promoted and use much more routing resource. So, lower the critical range only if necessary.

See Also

- [set_app_options](#)

clock_opt.flow.optimize_layers_overlap_bins

Enable overlap bins when *clock_opt.flow.optimize_layers* is set to true or auto.

Data Types

Boolean

Default false

Description

When the switch is enabled, layer bins can be overlapped. For example: {M10, M9}, {M9, M8}, {M8, M7}, {M7, M6}, {M6, M5} When it is disabled, layer bins do not overlap. For example: {M10, M9}, {M8, M7}, {M6, M5}

The feature might help improve QoR when routing resources is unbalanced. For example, if M10 routing capacity is low, but M9 is high. Enable overlap bin: Nets can be promoted to {M9, M8} to improve QoR. Disable overlap bin: Nets promote to {M8, M7}

See Also

- [set_app_options](#)

clock_opt.flow.optimize_ndr

Enable NDR optimization in clock_opt

Data Types

Boolean

Default false

Description

clock_opt flows will run automatic non-default-routing rule optimization that assigns long timing critical nets to NDR to improve timing.

See Also

- [set_app_options](#)

clock_opt.flow.optimize_ndr_critical_range

Set critical range for Non-default routing (NDR) based optimization when *clock_opt.flow.optimize_ndr* is set to true.

c

Data Types

*Float***Default** 0.5

Description

It accepts values from $\{0 \sim 1.0\}$. For a given value y , it optimizes nets that have slacks between $\{y \cdot \text{WNS}, \text{WNS}\}$, where WNS is the worst negative slack. Set it to a lower value to fix paths that are in less critical range. It may cause a lot more nets to have NDRs and use much more routing resource. So, lower the critical range only if necessary.

See Also

- [set_app_options](#)

clock_opt.flow.optimize_ndr_max_nets

Set maximum number of nets to be routed with Non-default routing (NDR) rule when *clock_opt.flow.optimize_ndr* is set to true.

Data Types

*integer***Default** 2000

Description

During NDR-based optimization, the number of maximum nets to be assigned NDRs is given by this value. This number limits the number of potential candidates that can benefit from applying NDRs, and must be a non-negative integer.

See Also

- [set_app_options](#)

clock_opt.flow.repeatability

clock_opt trades off QoR repeatability for runtime speedup.

Data Types

*Boolean***Default** false

Description

If set to true, the QoR at the end of `clock_opt` becomes repeatable at the expense of non-negligible runtime overhead. Unlike `place_opt` and `refine_opt`, the runtime overhead for repeatable `clock_opt` (in multicore environment) is substantial. The runtime overhead is mainly contributed by clock routing steps, which can be only repeatable while running in a slower mode.

See Also

- [clock_opt](#)
- [set_app_options](#)

clock_opt.hold.effort

Controls the optimization effort for hold fixing in version O-2018.06 and earlier versions.

Note:

This application option has no effect starting with version O-2018.06-SP1.

Data Types

string

Default medium

Description

In version O-2018.06 and earlier versions, this application option controls the hold fixing effort in *clock_opt* flows. Starting with version O-2018.06-SP1, this application option has no effect.

The hold fixing effort can be: *none*, *low*, *medium*, or *high*. Setting this application option to *none* disables hold fixing.

Setting this application option to *high* might result in a slight degradation in setup total negative slack (TNS).

See Also

- [clock_opt](#)
- [set_app_options](#)

clock_opt.place.congestion_effort

Data Types

string

Default medium

Description

Specifies the effort level for the congestion alleviation in *clock_opt*. Allowed values are *none*, *medium*, or *high*. Expect a significant increase in runtime for high effort. If set to *none*, *clock_opt* will not run in a congestion mode.

See Also

- [clock_opt](#)
- [create_placement](#)

clock_opt.place.effort

Data Types

string

Default medium

Description

Specifies the CPU effort level for coarse placements invoked in *clock_opt*. Allowed values are *medium* or *high*.

See Also

- [clock_opt](#)
- [create_placement](#)

compile.auto_floorplan.buffering_aware_timing_driven

Enables a timing-driven placement using an approximate timing model that estimates the effects of buffering long nets and high-fanout nets .

Data Types

bool

Default false

c

Description

This application option controls buffering aware timing driven macro placement when integrated macro placement is enabled. Intended for use as an initial placement step that gives a better starting point for later timing optimizations.

Use application option *plan.macro.style freeform* to enable integrated macro placement. Note that the integrated macro placement only supports the freeform macro placement style, see also the *plan.macro.style* application option. If the style is set differently, the integrated macro placement will be disabled.

Valid values are

- *false* (default). Macro placement is not buffering aware.
- *true*. Macro placement is buffering aware timing driven.

See Also

- [compile_fusion](#)
- [set_app_options](#)
- [get_app_option_value](#)
- [report_app_options](#)

compile.auto_floorplan.congestion

Enables a congestion-driven placement mode.

Data Types

bool

Default false

Description

This application option enables congestion driven placement mode.

Use application option *plan.place.congestion_driven_mode* to adjust the behavior.

Valid values are

- *false* (default). Macro placement is not congestion driven.
- *true*. Macro placement is congestion driven.

See Also

- [compile_fusion](#)
- [set_app_options](#)
- [get_app_option_value](#)
- [report_app_options](#)

compile.auto_floorplan.fix_macro_violation

Controls whether auto floorplanning checks and fixes floorplan violations on hard macros.

Data Types

string

Default auto

Description

The *compile.auto_floorplan.fix_macro_violation* application option controls whether auto floorplanning checks and fixes floorplan violations on hard macros. Use command *report_floorplan_rules* to report existing floorplan rules.

Valid values are

- *auto* (default)
Auto floorplanning checks and fixes floorplan violations on hard macros placed by Auto floorplan. Macros placed by Auto floorplan are guided by app option *compile.auto_floorplan.place_hard_macros*.
- *unfixed*
Auto floorplanning checks and fixes floorplan violations on placed and unplaced hard macros. The placement status is determined from the DEF files or Tcl floorplan.
- *all*
Auto floorplanning checks and fixes floorplan violations on all hard macros regardless of their placement status.
- *none*
Auto floorplanning does not check and fix floorplan violation on any macros.

To determine the current value of this application option, use the *get_app_option_value -name compile.auto_floorplan.fix_macro_violation* command.

See Also

- [compile_fusion](#)
- [report_floorplan_rules](#)

compile.auto_floorplan.initialize

Controls whether auto floorplanning creates the core and boundary.

Data Types

string

Default auto

Description

This application option controls whether auto floorplanning creates the core and boundary.

Valid values are

- *auto* (default)

Auto floorplanning creates the missing information for the floorplan. The existing core and boundary are preserved. Auto floorplan constraints are consumed to create the floorplan. Existing user floorplan information, generated either by `read_def` or design planning UI commands, are considered golden and will over-ride auto floorplan constraints.

- *true*

Auto floorplanning creates the core and boundary. The existing core and boundary are deleted and re-created. Auto floorplan constraints are consumed to create the floorplan.

- *false*

Auto floorplanning does not create the core or boundary, and uses only the existing information. The existing core and boundary are preserved.

To determine the current value of this application option, use the *get_app_option_value -name compile.auto_floorplan.initialize* command.

For a list of all compile application options and their current values, use the *report_app_options compile.** command.

c

See Also

- [compile_fusion](#)
- [set_auto_floorplan_constraints](#)

compile.auto_floorplan.keep_bounds

Controls whether auto floorplanning keeps existing bounds.

Data Types

string

Default auto

Description

This application option controls whether auto floorplanning keeps existing bounds.

Valid values are

- *auto* (default)

Auto floorplanning removes existing bounds if both the *compile.auto_floorplan.place_hard_macros* and *compile.auto_floorplan.shape_voltage_areas* application options are set to *all*.

- *true*

Auto floorplanning keeps all existing bounds.

- *false*

Auto floorplanning removes all existing bounds.

To determine the current value of this application option, use the *get_app_option_value -name compile.auto_floorplan.keep_bounds* command.

For a list of all compile application options and their current values, use the *report_app_options compile.** command.

See Also

- [compile_fusion](#)

compile.auto_floorplan.keep_placement_blockages

Controls whether auto floorplanning keeps existing placement blockages.

c

Data Types

string

Default auto**Description**

This application option controls whether auto floorplanning keeps existing placement blockages.

Valid values are

- *auto* (default)

Auto floorplanning removes existing placement blockages if:

- *compile.auto_floorplan.place_hard_macros* is set to *all*
- *compile.auto_floorplan.shape_voltage_areas* is set to *all*
- A floorplan creation or re-creation is triggered.

- *true*

Auto floorplanning keeps all existing placement blockages.

- *false*

Auto floorplanning removes all existing placement blockages.

If the placement blockages must be retained through floorplan creation or re-creation, *compile.auto_floorplan.keep_placement_blockages* has to be *true*.

To determine the current value of this application option, use the *get_app_option_value -name compile.auto_floorplan.keep_placement_blockages* command. For a list of all compile application options and their current values, use the *report_app_options compile.** command.

See Also

- [compile_fusion](#)

compile.auto_floorplan.optimize_small_design

Specify auto floorplan to allocate more spaces for a small design

Data Types

Boolean

Default false

c

Description

When this application option is *true*, the auto floorplanning functionality in the *compile_fusion* command will allocate more spaces for a small design if:

- the design has macros; or
- the *set_isolate_ports* command is executed to insert buffers to isolate I/O ports.

A small design is defined as one whose cell count is at most the value specified by the application option *compile.auto_floorplan.small_design_threshold*.

See Also

- [compile_fusion](#)
- [set_isolate_ports](#)

compile.auto_floorplan.pin_placement_strategy

Controls the pin placement strategy invoked by Auto Floorplan.

Data Types

string

Default auto

Description

This application option controls which pin placement strategy is used when pin placement is done by Auto Floorplan.

Valid values are

- *global_route*
Pin placement is done using Global Route in all stages of Auto Floorplan.
- *flyline*
Auto Floorplan invokes pin placement using a flyline-based strategy in all stages.
- *fast_route*
Auto floorplan invokes a fast route based pin placement, equivalent to using *plan.pins.fast_route* app option.
- *auto* (default)
Auto Floorplan invokes pin placement using a flyline-based strategy in all logic_opto stages and in initial place. Refine Pin Placement is done with a fast route strategy.

c

To determine the current value of this application option, use the command `get_app_option_value -name compile.auto_floorplan.pin_placement_strategy`.

For a list of all compile application options and their current values, use the `report_app_options compile.*` command.

See Also

- [compile_fusion](#)

compile.auto_floorplan.place_hard_macros

Controls whether auto floorplanning places hard macros.

Data Types

string

Default unplaced

Description

The `compile.auto_floorplan.place_hard_macros` application option controls whether auto floorplanning places hard macros.

Note that if the `place.coarse.fix_hard_macros` application option is set to `true`, auto floorplanning does not place hard macros, regardless of the setting of the `compile.auto_floorplan.place_hard_macros` application option.

Valid values are

- *unplaced* (default)
Auto floorplanning places hard macros that are not placed. Existing placed or fixed hard macros are preserved. The placement status is determined from the DEF files or Tcl floorplan.
- *unfixed*
Auto floorplanning places hard macros that are either not placed or not fixed. Existing fixed hard macros are preserved. The placement status is determined from the DEF files or Tcl floorplan.
- *all*
Auto floorplanning places all hard macros regardless of their placement status.
- *none* Auto floorplanning does not place any macros. Existing hard macros are preserved.

c

To determine the current value of this application option, use the *get_app_option_value -name compile.auto_floorplan.place_hard_macros* command.

For a list of all compile application options and their current values, use the *report_app_options compile.** command.

See Also

- [compile_fusion](#)

compile.auto_floorplan.place_hard_macros_enhanced

Controls whether auto floorplanning places hard macros using the enhanced or integrated macro placement flow.

Data Types

string

Default false

Description

This application option controls whether integrated macro placement is enabled.

Valid values are

- *false* (default)
Macro placement will only happen as part of the auto floorplanning steps in logic_opto and will be considered fixed for the rest of the flow.
- *true*
The macro placement will be updated as part of the standard cell placement steps throughout the compile flow, up to and including final_place. At the initial_place stage, the placement is done from scratch, and any pre-existing unfixed placement is ignored.
- *incremental*
The macro placement will be updated as part of the standard cell placement steps throughout the compile flow, up to and including final_place. At the initial_place stage, the placement is done incrementally, starting from the pre-existing placement.

Note that the integrated macro placement only supports the freeform macro placement style, see also the *plan_macro.style* application option. If the style is set differently, the integrated macro placement will be disabled.

Also note that, if the *compile.auto_floorplan.place_hard_macros* application option set to *none*, or the *place.coarse.fix_hard_macros* application option is set to *true*, auto

c

floorplanning does not place hard macros, regardless of the setting for this application option.

Also note that if the integrated macro placement is enabled (so the `compile.auto_floorplan.place_hard_macros` application option is not set to `none`), we always place all the unfixed macros, as if the `compile.auto_floorplan.place_hard_macros` application option is set to `unfixed`.

See Also

- [compile_fusion](#)
- [set_app_options](#)
- [get_app_option_value](#)
- [report_app_options](#)

compile.auto_floorplan.place_ios

Controls whether auto floorplanning places I/Os.

Data Types

string

Default `unplaced`

Description

This application option controls whether auto floorplanning places I/Os.

Valid values are

- `unplaced` (default)

Auto floorplanning places the I/Os that are not placed. Existing placed or fixed I/Os are preserved. The placement status is determined from the DEF files or Tcl floorplan.

- `unfixed`

Auto floorplanning places the I/Os that are either not placed or not fixed. Existing fixed I/Os are preserved. The placement status is determined from the DEF files or Tcl floorplan.

- `all`

Auto floorplanning places all I/Os.

c

- *none*

Auto floorplanning does not place any I/Os. Existing I/Os are preserved.

To determine the current setting of this application option, use the *get_app_option_value -name compile.auto_floorplan.place_ios* command.

For a list of all compile application options and their current settings, use the *report_app_options compile.** command.

See Also

- [compile_fusion](#)

compile.auto_floorplan.place_pins

Controls whether auto floorplanning places pins.

Data Types

string

Default unplaced

Description

This application option controls whether auto floorplanning places pins.

Valid values are

- *unplaced* (default)

Auto floorplanning places pins that are not placed. Existing placed or fixed pins are preserved. The placement status is determined from the DEF files or Tcl floorplan.

- *unfixed*

Auto floorplanning places pins that are either not placed or not fixed. Existing fixed pins are preserved. The placement status is determined from the DEF files or Tcl floorplan.

- *all*

Auto floorplanning places all pins.

- *none*

Auto floorplanning does not place any pins. Existing pins are preserved.

To determine the current value of this application option, use the *get_app_option_value -name compile.auto_floorplan.place_pins* command.

c

For a list of all compile application options and their current values, use the `report_app_options compile.*` command.

See Also

- [compile_fusion](#)

compile.auto_floorplan.shape_voltage_areas

Controls whether auto floorplanning shapes voltage areas.

Data Types

string

Default unshaped

Description

This application option controls whether auto floorplanning shapes voltage areas.

Valid values are

- *unshaped* (default)

Auto floorplanning shapes voltage areas that are not shaped. Existing shaped or fixed voltage areas are preserved. The placement status is determined from the DEF files or Tcl floorplan.

- *unfixed*

Auto floorplanning shapes voltage areas that are not shaped or fixed. Existing voltage areas are preserved. The placement status is determined from the DEF files or Tcl floorplan.

- *all*

Auto floorplanning shapes all voltage areas.

- *none*

Auto floorplanning does not shape any voltage areas. Existing voltage areas are preserved.

By default, the `compile_fusion` command does not create voltage areas for power domains without an associated voltage area. Such power domains are associated to `DEFAULT_VA`. To change this, set the `mv.upf.enable_missing_voltage_area` application option to `true`. When this option is set to `true` before running the `commit_upf` command, the `compile_fusion` command creates one voltage area per power domain without an associated voltage area, and auto floorplanning attempts to shape these voltage areas.

c

To determine the current value of this application option, use the *get_app_option_value -name compile.auto_floorplan.shape_voltage_areas* command.

For a list of all compile application options and their current values, use the *report_app_options compile.** command.

See Also

- [compile_fusion](#)

compile.auto_floorplan.small_design_threshold

Specify the maximum cell count of a small design for auto floorplan

Data Types

Integer

Default 2000

Description

This application option specifies the maximum cell count of a small design. This is used together with the application option *compile.auto_floorplan.optimize_small_design* so that the auto floorplanning functionality in the *compile_fusion* command can allocate more spaces for a small design.

See Also

- [compile_fusion](#)
- [set_isolate_ports](#)

compile.auto_floorplan.timing_driven

Enables a timing-driven macro placement mode.

Data Types

bool

Default false

Description

This application option controls timing driven macro placement mode.

Use application option *plan.place.timing_driven_mode* to adjust the behaviour of macro placement.

c

Valid values are

- *false* (default). Macro placement is not timing driven.
- *true*. Macro placement is timing driven.

See Also

- [compile_fusion](#)
- [set_app_options](#)
- [get_app_option_value](#)
- [report_app_options](#)

compile.check.site_def

Check site def legality

Data Types

integer

Default 1

Description

When this application option is set to 0, tool will not check site def legality of the design.

When this option is set to 1, tool will check if site def used in design matches site def specified in floorplan and user settings. Warning messages will be printed.

See Also

- [report_site_defs](#)
- [set_auto_floorplan_constraints](#)

compile.check.site_def_verbose

Check site def legality with verbose messages

Data Types

boolean

Default false

c

Description

When this application option is set to true, tool will check if site def used in design matches site def specified in floorplan and user settings. Verbose messages about each violating library cell and mapped cell will be printed.

See Also

- [report_site_defs](#)
- [set_auto_floorplan_constraints](#)

compile.clockgate.cg_cell_naming_style

Specifies custom naming style for clock gates.

Data Types

string

Default "%r"

Description

The name of a clock gate has two fields: * clock gate specific string (clock_gate, self_gate, and others) * user defined string which by default is the name of the register that is gated the most

This application option allows the customization of the user-defined names and modifies the clock-gate-specific name by appending the snps_ prefix. For instance, the name of a regular clock gate that begins with the clock_gate string is updated to snps_clock_gate. However, the snps_ prefix is appended only if the default clock-gate-specific strings are not modified. The specified naming style is used both to name newly inserted clock gates and to identify clock gates inserted by the tool.

```
set_app_options -name compile.clockgate.cg_cell_naming_style -value
"%r_midfix_%c_suffix"
    %c - Clock net name (If the clock net name contains reserved
strings, then all
        strings beginning with the reserved strings are removed.)
    %r - Most common name among the gated registers
    midfix - User-defined string, but cannot contain reserved strings
    suffix - User-defined string, but cannot contain reserved strings
    reserved midfix/suffix - snps, ml, clock_gate, self_gate,
seq_clock_gate, snps_clock_gate, snps_self_gate,
        snps_seq_clock_gate, and the current regular
clock gate and self-gates prefixes.
```

c

The parameters can be specified in any order as long as they are separated by underscore (_). To determine the current value of this application option, use the `get_app_option_value -name compile.clockgate.cg_cell_naming_style` command.

For the naming style to take effect, the application option `compile.clockgate.use_enhanced_clock_gate_naming_style` must be set to `true`.

Examples

The following examples show how to specify the naming style and the expected clock gate name.

Assume two distinct regular clock gates are used to gate the `out_reg` and `out_reg_1` registers.

The names of the clock gates based on the naming style are given as follows:

```
prompt> set_app_options -name
        compile.clockgate.use_enhanced_clock_gate_naming_style -value true
prompt> set_app_options -name compile.clockgate.cg_cell_naming_style
        -value "%r"
snps_clock_gate_out_reg and snps_clock_gate_out_reg_1.
```

```
prompt> set_app_options -name
        compile.clockgate.use_enhanced_clock_gate_naming_style -value true
prompt> set_app_options -name compile.clockgate.cg_cell_naming_style
        -value "%r_vendorXYZ"
snps_clock_gate_out_reg_vendorXYZ and
snps_clock_gate_out_reg_1_vendorXYZ.
```

```
prompt> set_app_options -name
        compile.clockgate.use_enhanced_clock_gate_naming_style -value true
prompt> set_app_options -name compile.clockgate.cg_cell_naming_style
        -value "%c_%r"
```

Assuming the clock net name is `clk`, the names of the clock gates are as follows:

`snps_clock_gate_clk_out_reg` and `snps_clock_gate_clk_out_reg_1`.

```
prompt> set_app_options -name
        compile.clockgate.use_enhanced_clock_gate_naming_style -value true
prompt> set_app_options -name compile.clockgate.cg_cell_naming_style
        -value "%r"
snps_clock_gate_out_reg and snps_clock_gate_out_reg_1.
```

```
prompt> set_app_options -name
        compile.clockgate.use_enhanced_clock_gate_naming_style -value true
prompt> set_app_options -name compile.clockgate.cg_cell_naming_style
        -value "snps"
```

The string `snps` is a reserved prefix, thus results in an error.

c

```
prompt> set_app_options -name
compile.clockgate.use_enhanced_clock_gate_naming_style -value true
prompt> set_app_options -name compile.clockgate.cg_cell_naming_style
-value "%c_midfix_%r_vendorXYZ"
```

If the clock net name is `clk_snps_clock_gate`, the updated clock net name is `clk` (every string starting from a reserved prefix is removed). The clock gate names are the following:
`snps_clock_gate_clk_midfix_out_reg_vendorXYZ` and
`snps_clock_gate_clk_midfix_out_reg_1_vendorXYZ`.

```
prompt> set_app_options -name
compile.clockgate.use_enhanced_clock_gate_naming_style -value true
prompt> set_app_options -name compile.clockgate.cg_cell_naming_style
-value "%r_vendorXYZ"
prompt> set_app_options -name compile.clockgate.clock_gate_name_prefix
-value "clkgate"
```

Since the default clock gate prefix has been changed, the clock gate names are as follows:
`clkgate_out_reg_vendorXYZ` and `clkgate_out_reg_1_vendorXYZ`.

```
prompt> set_app_options -name
compile.clockgate.use_enhanced_clock_gate_naming_style -value true
prompt> set_app_options -name compile.clockgate.cg_cell_naming_style
-value "%r_vendorXYZ"
prompt> set_app_options -name compile.clockgate.self_gate_name_prefix
-value "clkgate"
```

The newly created self-gate is `clkgate_out_reg_vendorXYZ`.

For a list of all the compile application options and their current values, use the `report_app_options compile.*` command.

See Also

- [compile_fusion](#)
- [report_clock_gating](#)
- [get_clock_gates](#)

compile.clockgate.clock_gate_name_prefix

Specifies the prefix to use for tool-inserted clock gates.

Data Types

string

c

Default "clock_gate"**Description**

This application option specifies the prefix to use for tool-inserted clock gates. The specified prefix is used both to name newly inserted clock gates and to identify clock gates inserted by the tool. Its value depends on the application option *compile.clockgate.use_enhanced_clock_gate_naming_style*. If this application option is true, then the prefix used is *snps_clock_gate*. Otherwise, *clock_gate* will be used.

To determine the current value of this application option, use the *get_app_option_value -name compile.clockgate.clock_gate_name_prefix* command. Please note that even if the prefix value reported is *snps_clock_gate*, this value takes effect only if the application option *compile.clockgate.use_enhanced_clock_gate_naming_style* is set to true.

For a list of all compile application options and their current values, use the *report_app_options compile.** command.

See Also

- [compile_fusion](#)
- [report_clock_gating](#)
- [get_clock_gates](#)

compile.clockgate.enable_activity_driven_level_expansion

Controls whether *compile_fusion* executes clock-gate level expansion in activity-driven mode.

Data Types

Boolean

Default true**Description**

This application option controls whether *compile_fusion* executes clock-gate level expansion in activity-driven mode.

If the application option *compile.clockgate.enable_level_expansion* is *true* (which it is not by default), activity-driven clock gating level expansion is on by default. If *compile.clockgate.enable_level_expansion* is *false*, this application option is ignored.

In the activity-driven mode, *Fusion Compiler* considers the switching activities on a clock gate when it performs level expansion.

c

Activity-driven optimization is accurate only if switching activities has been annotated on the design. Use the *read_saif* command to read in switching activities before executing *compile_fusion*. The activity-driven mode is not recommended if the SAIF is not available.

To determine the current value of this application option, use the *get_app_option_value -name compile.clockgate.enable_activity_driven_level_expansion* command.

For a list of all compile application options and their current values, use the *report_app_options compile.** command.

See Also

- [compile_fusion](#)
- [report_clock_gating](#)
- [get_app_option_value](#)
- [read_saif](#)

compile.clockgate.enable_dft_wrapper_cell_support

Controls whether clock gate splitting support for DFT wrapper cell is enabled.

Data Types

Boolean

Default false

Description

This application option controls the splitting of clock gates into two where one drives internal cells and the other drives DFT wrapper cells during compile fusion. However, splitting does not take effect if the minimum bitwidth is not honored.

```
set_app_options -name compile.clockgate.enable_dft_wrapper_cell_support
               -value enableDftSupport
               %enableDftSupport - a true or false Boolean value
```

To determine the current value of this application option, use the *get_app_option_value -name compile.enable_dft_wrapper_cell_support* command.

Examples

The following example shows the expected clock gate number based on the application option value:

Consider a design with ten clock gates, each driving five internal and five DFT wrapper cells.

c

Then, the number of clock gates after splitting (minimum bit with = 3 and maxfanout =16) is provided as follows:

```
prompt> set_app_options -name
        compile.clockgate.enable_dft_wrapper_cell_support -value false
prompt> compile_fusion -from logic_opto -to logic_opto
prompt> get_clock_gates -gating_wrapper_cells_only
Since the application option is disabled, the number of clock gates
driving only DFT wrapper cells is zero.
```

```
prompt> set_app_options -name
        compile.clockgate.enable_dft_wrapper_cell_support -value true
prompt> compile_fusion -from logic_place -to logic_opto
prompt> get_clock_gates -gating_wrapper_cells_only
With the application option enabled, the number of clock gates driving
only DFT wrapper cells is ten.
```

For a list of all the compile application options and their current values, use the *report_app_options compile.** command.

See Also

- [compile_fusion](#)
- [report_clock_gating](#)
- [get_clock_gates](#)

compile.clockgate.enable_level_expansion

Controls whether the *compile_fusion* command executes clock gating multilevel expansion

Data Types

Boolean

Default false

Description

The objective of the clock gating level expansion is to insert more than one level of clock gate by factoring out the enable signals among a group of clock gates, considering all clock gates including pre-existing and tool-inserted.

To trigger the clock gating level expansion, you should set *compile.clockgate.max_number_of_levels* to a value different than 0 and the variable *compile.clockgate.enable_level_expansion* to true.

To insert a new clock gating level, the group of clock gates to be expanded must have at least 2 clock gate instances with a common enable condition term.

The new inserted clock gates will be named according to the following rules:

- **clock_gate_ml_+ register_base_name:** If there is at least one tool-inserted clock gate in the fanout of the new inserted clock gate, the new inserted clock gate is considered as tool-inserted.
- **clock_gate_name_from_fanout_ + ml_pe:** If all the clock gates in the fanout are pre-existing the new inserted clock gate will be a pre-existing clock gate too with the postfix **ml_pe**.

To prevent the multilevel clock gating expansion in specific clock gates instances, use **set_clock_gate_transformations -expand_levels false** command option. For more information read the **set_clock_gate_transformations** man page.

Special considerations:

- Clock gating level expansion and clock gating level collapsing are not opposite operations. This means that if an expansion operation inserts a new clock gate and later the same clock gate is collapsed, the result might not be the same as before the expansion operation.
- The clock gating expansion operation always tries to insert new levels of clock gates if a common factor exists among the group.
- The maximum number of levels is honored by all clock gating insertion, including self-gating insertion. This means that if self-gating insertion is enabled

(by setting the **compile.clockgate.self_gating** application option to true),

When **-interaction insert** is used, and the number of clock gating level is above

or met with max clock gate level setting due to level expansion, self-gating cells will not be inserted,

If **interaction_with_clock_gating** option is set to collapse, tool will collapse tool inserted clock gates.

- Using the **set_clock_latency** command does not prevent the clock gate cell from being expanded.
- You cannot use multilevel clock gate expansion if you set the **compile.clockgate.inherit_timing_exceptions** application option to true.

By default, the Fusion Compiler tool analyzes the switching activity on a clock gate to optimize level expansion. To disable this feature, set the **compile.clockgate.enable_acitivity_driven_level_expansion** application option to *false*.

This application option is deprecated and is scheduled for removal in a future release. Please use **set_clock_gating_tree_options** instead.

See Also

- [set_clock_gate_transformations](#)
- [set_self_gating_options](#)
- [compile_fusion](#)
- [get_clock_gates](#)
- [report_clock_gating](#)
- [set_clock_gating_tree_options](#)

compile.clockgate.fanin_sequential

Control whether the *compile_fusion* command executes fanin-based sequential clock gating optimizations.

Data Types

Boolean

Default false

Description

This application option controls whether the *compile_fusion* command executes fanin-based sequential clock gating optimizations.

When the value of this application option is *true* and the *FC-New-Tech-Power* license is available, fanin-based sequential clock gating insertion is performed in the execution of *compile_fusion* command.

Fanin-based sequential clock gating is the process of extracting and propagating enable conditions from upstream sequential elements in the data path for the purpose of enabling clock gating insertion for additional registers. A register can be gated during a given clock cycle if all the registers in the transitive fanin did not change state in the previous clock cycle.

To create a delayed version of the enable signal extracted from the transitive fanin registers, the tool creates a new register and uses the output pin of this register as the driver of the enable pin for the new clock gate. This new register is referred to as the "delay register".

If the transitive fanin registers do not share exactly the same enable signal, the tool searches for, or synthesizes, a logic cover of all the enables, and uses that as the enable signal to be delayed.

c

If the register to be clock gated has an enable signal already, either as a synchronous enable pin, as a feedback loop to the data pin, or as the enable of a regular clock gate, a special circuitry is added at the input pins of the delay register to guarantee the logical correctness of the transformation.

Given that new logic is created to implement fanin-based sequential clock gating, a small increase in combinational and sequential area is expected.

Fanin-based sequential clock gating insertion occurs at two different stages of the *compile_fusion* flow. During *initial_map* stage, the analysis is limited to the local hierarchy of the registers. During *logic_opto* stage, the tool performs analysis across hierarchical boundaries.

By default, the name of fanin-based sequential clock gates inserted by the tool begins with the prefix "seq_clock_gate_fi_", but it can be setup by the use of the app option `compile.clockgate.fanin_sequential_name_prefix`. The name of the delay register inserted by the tool as part of the transformation begins with the prefix "seqcg_delay_reg_", but it can be setup by the use of the app option `compile.clockgate.sequential_delay_register_name_prefix`. In both cases, the remaining part of the name follows the standard clock-gating naming conventions.

CONFIGURATION OPTIONS

Fanin-based sequential clock gates inserted by the tool honor the settings in the *set_clock_gate_style* command, which applies to all types of clock gates. However, following configuration commands are specific to fanin-based sequential clock gating:

Use the *set_fanin_sequential_clock_gating_options* command to specify structural options for fanin-based sequential clock gating, such as the minimum bitwidth and maximum fanout of a bank. And use the *report_fanin_sequential_clock_gating_options* command to report these settings.

Use the *set_fanin_sequential_clock_gating_objects* command to control which objects should be included or excluded from fanin-based sequential clock gating insertion. And use the *report_fanin_sequential_clock_gating_objects* command to report these settings.

REQUIRED CONDITIONS

Regardless of settings made with the *set_fanin_sequential_clock_gating_options* and *set_fanin_sequential_clock_gating_objects* commands, the following conditions prevent fanin-based sequential clock gating insertion for a given register:

- The register under analysis contains objects other than edge-triggered flip-flops in its transitive fanin.
- The register under analysis, or any registers in its transitive fanin, have asynchronous state changes.
- The enable extracted from registers in the transitive fanin depends on their output pins.

c

- The registers in the transitive fanin are controlled by a different base clock, or triggered by the opposite clock edge.
- At least one of the registers in the transitive fanin is always-enabled.
- Either the next state function of the register under analysis, or the registers in its transitive fanin, depending on logic that contains a `dont_care` condition in the RTL.
- The register under analysis is multibit and its slices don't share the same enable signal.
- The tools finds that transformation is not likely to provide a power reduction benefit given the switching activity conditions.

REPORTING

When fanin-based sequential clock gating is enabled, the *report_clock_gating* command automatically displays an additional summary table showing the gating statistics.

Use *report_clock_gating -ungated* command to report the reasons why fanin-based sequential clock gating was not inserted for each ungated register.

Use *get_clock_gates* command to get a collection with all types of clock gates, including fanin-based sequential clock gates. And use *get_clock_gates -type fanin_seqcg* command to get a collection with fanin-based sequential clock gates only.

FORMAL VERIFICATION SUPPORT

The use of the *Formality* tool is required for synthesis verification when fanin-based sequential clock is enabled. Third party formal verification tools are not supported. When fanin-based sequential clock gating is enabled, there is limited support for the ECO flow. Please refer to *Formality* documentation for further details.

See Also

- [compile_fusion](#)
- [get_clock_gates](#)
- [report_clock_gating](#)
- [report_fanin_sequential_clock_gating_objects](#)
- [report_fanin_sequential_clock_gating_options](#)
- [set_clock_gate_style](#)
- [set_fanin_sequential_clock_gating_objects](#)
- [set_fanin_sequential_clock_gating_options](#)

compile.clockgate.fanin_sequential_name_prefix

Specifies the prefix for tool-inserted fanin sequential clock gates.

Data Types

string

Default "seq_clock_gate_fi"

Description

This application option specifies the prefix to use for tool-inserted fanin sequential clock gates. The specified prefix is used both to name newly inserted fanin sequential clock gates and to identify fanin sequential clock gates inserted by the tool. The default value depends on the application option *compile.clockgate.use_enhanced_clock_gate_naming_style*. If this application option is true, then the default sequential fanin clock gate prefix is *snps_seq_clock_gate_fi*. Otherwise, *seq_clock_gate_fi* will be used.

To determine the current value of this application option, use the *get_app_option_value -name compile.clockgate.fanin_sequential_name_prefix* command.

Consider that the tool does not allow the use of the same naming prefix for different type of tool-inserted clock gates (regular, self-gate and fanin sequential).

For a list of all compile application options and their current values, use the *report_app_options compile.** command.

See Also

- [compile_fusion](#)
- [report_clock_gating](#)
- [get_clock_gates](#)

compile.clockgate.inherit_timing_exceptions

Specifies that during the *compile_fusion* command, timing exceptions defined on registers must be automatically inferred to the enable pin of the clock gate that is gating the registers.

Data Types

Boolean

Default false

Description

If this application option is set to *true* before running the *compile_fusion* command, the clock gate enable pin inherits the timing exceptions from the gated registers for all defined scenarios, if possible.

The following timing exceptions are considered:

- Multicycle paths
- Minimum delay
- Maximum delay
- False paths
- Path margins

The following conditions apply:

- Only cell-level exceptions are inherited, which means that all synchronous pins of a register must have the same set of timing exceptions before they can be inferred onto a clock gate.
- If a set of registers with the same enable conditions has different timing exceptions, they must be split into smaller sets of registers with equivalent timing exceptions before clock-gate insertion occurs. This splitting might create banks that fall below the minimum bitwidth specified for clock gating. If so, these banks might remain ungated. All synchronous input pins are considered to determine compatibility between two registers.
- Timing exceptions on the enable pin of the clock gate might prevent clock gate removal.
- The set of timing exceptions are copied only when the register has timing exceptions and no other timing constraints.
- This feature does not support the inheritance of timing exceptions in self-gating nor sequential clock gating insertion. Registers affected by timing violations can be excluded from clock gating insertion using *set_self_gating_objects* or *set_fanin_sequential_clock_gating_objects* commands with the *-exclude object_list* option.

Note: Any sub or intermediate steps of the *compile_fusion* command are not permitted.

To guarantee the correct functionality, verify that a clock exists.

To determine the current value of this application option, use the *get_app_option_value -name compile.clockgate.inherit_timing_exceptions* command.

c

For a list of all compile application options and their current values, use the *report_app_options compile.** command.

Multicorner-Multimode Support

The copy is performed for all defined scenarios with its corresponding timing exception settings.

See Also

- [compile_fusion](#)
- [report_exceptions](#)
- [report_clock_gating](#)

compile.clockgate.insertion_strategy

Controls the clock gating heuristics' aggressiveness used during *compile_fusion*.

Data Types

int

Default 0

Description

Set application option *compile.clockgate.insertion_strategy* to 1 to enable the use of aggressive clock gating heuristics in the enable signal selection, in an attempt to improve dynamic power in the design.

These heuristics may have an impact in area and/or timing.

See Also

- [compile_fusion](#)

compile.clockgate.max_level_allow_ungating

Controls whether clock gate removal by transferring the enable logic to the registers can be used to honor *compile.clockgate.max_number_of_levels*.

Data Types

Boolean

Default false

c

Description

By default, *compile_fusion* honors the maximum clock gate level specified by *compile.clockgate.max_number_of_levels* by collapsing clock gates to reduce the number of levels.

When this application option is *true*, the *compile_fusion* command can perform, in addition to the clock gate collapsing, clock gate removal by transferring the enable logic to the registers to achieve the specified clock gate level.

This application option is deprecated and is scheduled for removal in a future release. Please use *set_clock_gating_tree_options* instead.

See Also

- [compile_fusion](#)
- [set_clock_gating_tree_options](#)

compile.clockgate.max_number_of_levels

Specifies the maximum number of clock gate levels allowed in the design.

Data Types

integer

Default 0

Description

The *compile_fusion* command uses the value of this application option as a constraint on the number of cascading clock gates.

A value of 0 means that the number of levels is unlimited.

A clock gate's level is the number of clock gates in its fanin until the clock port, plus 1. If combinational cells exist between the clock gates, the longer path to a clock port is computed.

The *compile_fusion* command attempts to honor this constraint by collapsing clock gates to reduce the number of levels.

However, to achieve the specified number of levels, it might be necessary to remove some clock gates. By default, the tool does not remove clock gates to honor this constraint. To allow this, set the *compile.clockgate.max_level_allow_ungating* application option to *true*.

To determine the current value of this application option, use the *get_app_option_value -name compile.clockgate.max_number_of_levels* command.

c

For a list of all compile application options and their current values, use the `report_app_options compile.*` command.

This application option is deprecated and is scheduled for removal in a future release. Please use `set_clock_gating_tree_options` instead.

See Also

- [compile_fusion](#)
- [report_clock_gating](#)
- [get_clock_gates](#)

compile.clockgate.physically_aware_estimate_timing

Controls whether the `compile_fusion` command executes clock gating in physically-aware mode with timing estimates for the clock tree.

Data Types

Boolean

Default true

Description

This application option controls whether the `compile_fusion` command executes clock gating in physically-aware mode with timing estimates for the clock tree.

This application option has an effect only if the `compile.clockgate.physically_aware` application option is `true` and the application `opt.common.estimate_clock_gate_latency` is `false`.

If both the `compile.clockgate.physically_aware` and `compile.clockgate.physically_aware_estimate_timing` application options are `true`, the `compile_fusion` command executes clock gating optimizations in physically-aware mode and uses estimates for the delay of the clock tree which is not yet propagated.

This will be one of the following:

- The annotation of clock latencies to clock gate and register clock pins based on the estimated latency of the clock tree after clock tree synthesis. This happens if the value of the `compile.clockgate.use_clock_latency` application option is `true`, which is the default.
- The annotation of path margins on the enable pins of clock gates proportional to the difference in estimated latencies from the clock tree source to the clock gate and the

c

clock tree source to the clock tree leaves in the transitive fanout of the clock gate. This happens if the value of the *compile.clockgate.use_clock_latency* application option is *false*.

If the *set_clock_gate_latency* command has been applied to the design, physically-aware clock gating annotates latencies only to those clock gates for which *set_clock_gate_latency* has not specified a latency value.

To determine the current value of this application option, use the *get_app_option_value -name compile.clockgate.physically_aware_estimate_timing* command.

For a list of all compile application options and their current values, use the *report_app_options compile.** command.

See Also

- [compile_fusion](#)
- [report_clock_gating](#)
- [set_clock_gate_latency](#)

compile.clockgate.self_gate_name_prefix

Specifies the prefix for tool-inserted self-gates.

Data Types

string

Default "self_gate"

Description

This application option specifies the prefix to use for tool-inserted self-gates. The specified prefix is used both to name newly inserted self-gates and to identify self-gates inserted by the tool. Its value depends on the application option *compile.clockgate.use_enhanced_clock_gate_naming_style*. If this application option is true, then the prefix used is *snps_self_gate*. Otherwise, *self_gate* will be used.

To determine the current value of this application option, use the *get_app_option_value -name compile.clockgate.self_gate_name_prefix* command. Please note that even if the prefix value reported is *snps_self_gate*, this value takes effect only if the application option *compile.clockgate.use_enhanced_clock_gate_naming_style* is set to true.

For a list of all compile application options and their current values, use the *report_app_options compile.** command.

See Also

- [compile_fusion](#)
- [report_clock_gating](#)
- [get_clock_gates](#)

compile.clockgate.self_gating

Controls whether the *compile_fusion* command executes self-gating optimizations.

Data Types

Boolean

Default false

Description

This application option controls whether the *compile_fusion* command executes self-gating optimizations.

When the value of this application option is *true*, self-gating optimization is performed in the execution of *compile_fusion* command, at the *initial_opto* stage.

Given that new logic is created to implement self-gating, a small increase in combinational and sequential area is expected.

The name prefix of inserted self-gates can be controlled by using application option *compile.clockgate.self_gate_name_prefix*. The remaining part of the name follows the standard clock-gating naming conventions.

CONFIGURATION OPTIONS

Self-gates inserted by the tool honor the settings in the *set_clock_gate_style* command, which applies to all types of clock gates. However, following configuration commands are specific to self-gating:

Use the *set_self_gating_objects* command to control which objects should be included, excluded or forced from self-gating insertion, as well as the type of comparator logic (XOR, NAND, OR, AUTO) to use on them. And use the *report_self_gating_objects* command to report these settings.

Use the *set_self_gating_options* command to specify structural options for self-gating, that are instance-specific at the hierarchical level, such as the minimum and maximum bitwidth of a bank, or how the feature interacts with the presence of tool-inserted clock gates. And use the *report_self_gating_options* command to report these settings.

Use the `set_global_self_gating_options` command to control options for self-gating behavior that are applicable throughout the entire design. And use the `report_global_self_gating_options` command to report these settings.

REPORTING

When self-gating is enabled, the `report_clock_gating` command automatically displays an additional summary table showing the gating statistics.

Use `report_clock_gating -ungated` command to report the reasons why self-gating was not inserted for each ungated register.

Use `get_clock_gates` command to get a collection with all types of clock gates, including self-gates. And use `get_clock_gates -type self_gate` command to get a collection with self-gates only.

NOTES

Inheritance of timing exceptions is not supported in self-gating. Timing violations introduced in the self-gate enable pin can be avoided by excluding affected registers using the command `set_self_gating_objects -exclude object_list`

See Also

- [compile_fusion](#)
- [get_clock_gates](#)
- [report_clock_gating](#)
- [report_global_self_gating_options](#)
- [report_self_gating_objects](#)
- [report_self_gating_options](#)
- [set_global_self_gating_options](#)
- [set_self_gating_objects](#)
- [set_self_gating_options](#)

compile.clockgate.sequential_delay_register_name_prefix

Specifies the prefix for tool-inserted fanin sequential registers.

Data Types

string

Default "seqcg_delay_reg"

c

Description

This application option specifies the prefix to use for tool-inserted fanin sequential registers. The specified prefix is used in sequential cg step, when a new register is created (is tool-inserted) as a delay register. The default value depends on the application option *compile.clockgate.use_enhanced_clock_gate_naming_style*. If this application option is true, then the default sequential delay register prefix is *snps_seqcg_delay_reg*. Otherwise, *seqcg_delay_reg* will be used.

To determine the current value of this application option, use the *get_app_option_value -name compile.clockgate.sequential_delay_register_name_prefix* command.

For a list of all compile application options and their current values, use the *report_app_options compile.** command.

See Also

- [compile_fusion](#)
- [report_clock_gating](#)
- [get_clock_gates](#)

compile.clockgate.substring_list_for_dft_wrapper_dont_rename_icgs

Specifies the pattern that prevents renaming of clock gates driving DFT wrapper cells.

Data Types

string

Default "Empty string"

Description

This application option accepts a list of strings as its value and restrict the renaming of clock gates driving DFT wrapper cells that matches the substring/pattern during *compile_fusion*.

```
set_app_options -name
  compile.clockgate.substring_list_for_dft_wrapper_dont_rename_icgs -value
  listOfStrings
    %listOfString - a space-separated list of strings
```

To determine the current value of this application option, use the *get_app_option_value -name compile.clockgate.substring_list_for_dft_wrapper_dont_rename_icgs* command.

c

Examples

The following examples show how to specify the pattern and the expected clock gate names:

Consider a design with two clock gates named `clock_gate_reg0` and `clkgt_icg0` that are driving DFT wrapper cells.

To modify the names of the clock gates, use the following example:

```
prompt> set_app_options -name
        compile.clockgate.substring_list_for_dft_wrapper_dont_rename_icgs -value
        ""
```

```
prompt> compile_fusion -to logic_opto
```

```
prompt> get_clock_gates -gating_wrapper_cells_only
```

A clock gate driving a DFT wrapper cell is prefixed with the word `dft_wrapper_`. Thus, the name of the clock gates are `dft_wrapper_clock_gate_reg0` and `dft_wrapper_clkgt_icg0`.

```
prompt> set_app_options -name
        compile.clockgate.substring_list_for_dft_wrapper_dont_rename_icgs -value
        {clock_gate clkgt}
```

```
prompt> compile_fusion -to logic_opto
```

```
prompt> get_clock_gates -gating_wrapper_cells_only
```

Since the names of the clock gates match one of the substrings, the names of the clock gates (`clock_gate_reg0` and `clkgt_icg0`) remain intact.

For a list of all the compile application options and their current values, use the `report_app_options compile.*` command.

See Also

- [compile_fusion](#)
- [report_clock_gating](#)
- [get_clock_gates](#)

compile.clockgate.use_clock_latency

Controls whether the `compile_fusion` command uses clock latencies or path margins for timing annotation in physically-aware clock gating.

Data Types

Boolean

Default true

c

Description

This application option controls whether the *compile_fusion* command uses clock latencies or path margins for timing annotation in physically-aware clock gating.

This only has an effect if both the *compile.clockgate.physically_aware* and *compile.clockgate.physically_aware_estimate_timing* application options are *true* and the application option *opt.common.estimate_clock_gate_latency* is *false*. If this application option is *true*, the clock tree timing estimates are annotated as latencies. Otherwise, they are annotated as path margins on the enable pins of the clock gates.

See Also

- [compile_fusion](#)
- [report_clock_gating](#)
- [set_clock_gate_latency](#)

compile.clockgate.use_enhanced_clock_gate_naming_style

Control whether the *compile_fusion* command uses enhanced naming style for clock gating cells.

Data Types

Boolean

Default `false`

Description

If this application option is set to *true* before running the *compile_fusion* command, the clock gates follow the naming style defined by the application option *compile.clockgate.cg_cell_naming_style*. Consistent setting of this application option must be ensured throughout the flow.

To determine the current value of this application option, use the *get_app_option_value -name compile.use_enhanced_clock_gate_naming_style* command.

For a list of all compile application options and their current values, use the *report_app_options compile.** command.

c

See Also

- [compile_fusion](#)
- [report_clock_gating](#)
- [get_clock_gates](#)

compile.datapath.module_name_prefix

Controls the module name prefix of DesignWare and datapath blocks generated during the *compile_fusion* command.

Data Types

String

Default ""

Description

When this application option is set to a non-empty string, the flow always prefix DesignWare module name with it. For datapath module such as *'DP_OP_**', prefix is not allowed, instead the string will be encoded to the last numeric part of its name.

The characters that can be used in *compile.datapath.module_name_prefix* include lower and upper case alphabet 'a' to 'z', digit '0' to '9', and underscore '_'.

For a list of all compile application options and their current values, use the *report_app_options compile.** command.

Examples

The following example will prefix *"BLK_"* to all generated DesignWare modules, such as *"BLK_DW01_add_J23_H0_D0"*. Datapath module name will be changed differently like *"DP_OP_12_2351_88351_J12_H0_D135883516"*, i.e. the prefix string was encoded to last numeric part in the name. It's deterministic but may not be the same for different modules.

```
prompt> set_app_options -name compile.datapath.module_name_prefix -value
"BLK_"
```

See Also

- [compile_fusion](#)

compile.datapath.physically_aware

Controls the physical reselection of DesignWare and datapath blocks during the *compile_fusion* command.

Data Types

Boolean

Default false

Description

When this application option is set to *true*, the flow enables physical reselection of DesignWare and datapath during the physical optimization phase of the *compile_fusion* command.

Set the *compile.datapath.physically_aware* application option to *true* if you want the *compile_fusion* command to do physical reselection on DesignWare and datapaths. Doing so might degrade area and timing QoR. Certain DesignWare and datapath instances will have ungrouping restrictions before entering physical optimization phase.

To determine the current value of this application option, use the *get_app_option_value -name compile.datapath.physically_aware* command.

For a list of all compile application options and their current values, use the *report_app_options compile.** command.

See Also

- [compile_fusion](#)

compile.datapath.ungroup

Controls the ungrouping of DesignWare and datapath blocks during the *compile_fusion* command.

Data Types

Boolean

Default true

Description

When this application option is set to *false*, the flow disables DesignWare and datapath ungrouping during the timing optimization phase of the *compile_fusion* command.

c

Set the *compile.datapath.ungroup* application option to *false* if you do not want the *compile_fusion* command to ungroup DesignWare and datapaths during timing optimization. Doing so might degrade area and timing QoR.

To determine the current value of this application option, use the *get_app_option_value -name compile.datapath.ungroup* command.

For a list of all compile application options and their current values, use the *report_app_options compile.** command.

See Also

- [compile_fusion](#)

compile.early_place.effort

Specifies the CPU effort level for the early placement performed by the *compile_fusion* command.

Data Types

string

Default medium

Description

This application option specifies the CPU effort level for the early placement performed by the *compile_fusion* command. Valid values are *low*, *medium*, and *high*.

See Also

- [compile_fusion](#)
- [create_placement](#)

compile.final_place.effort

Controls the effort level of the final_place stage of the *compile_fusion* command.

Data Types

string

Default medium

c

Description

Specifies the CPU effort level for direct timing-driven placement during the `final_place` stage of the `compile_fusion` command.

Valid values are *low*, *medium*, or *high*.

See Also

- [compile_fusion](#)
- [set_app_options](#)

compile.flow.allow_inverter_on_clock

Allow inverter on clock pin of the sequential cells.

Data Types

Boolean

Default `true`

Description

This app option allows users to control if inverter on the latch enable and clock nets are allowed. If this variable is set to false, the compile will not insert inveter on the latch enable or clock nets.

See Also

- [compile](#)

compile.flow.areaResynthesis

Enables resynthesis for improved area in the `compile_fusion` command.

Data Types

Boolean

Default `"false"`

Description

When you enable `compile.flow.area.Resynthesis`, the `compile_fusion` command runs additional optimization steps in `logic_opto` to improve the total area of the design, possibly at the cost of additional runtime. This app option modifies only the `logic_opto` stage. Use

c

this app option only on designs where ungrouping is widely applied on DesignWare or user hierarchies.

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

compile.flow.autoungroup

Controls whether automatic ungrouping is enabled during the *compile_fusion* command.

Data Types

Boolean

Default true

Description

This application option controls whether automatic ungrouping is performed by the *compile_fusion* command.

By default, automatic ungrouping is enabled during the *compile_fusion* command.

When this application option is set to *false*, all hierarchies are preserved unless otherwise specified.

See Also

- [compile_fusion](#)
- [set_app_options](#)
- [set_ungroup](#)

compile.flow.boundary_optimization

Controls whether cross-boundary optimization is enabled during the *compile_fusion* command.

Data Types

Boolean

Default true

c

Description

This application option controls whether the *compile_fusion* command performs hierarchical boundary optimization.

By default, boundary optimization is enabled during the *compile_fusion* command.

By default, setting this option to *false* does not disable constant propagation across design hierarchies. To disable constant propagation when this option is set to *false*, set the *compile.flow.constant_and_unloaded_propagation_with_no_boundary_opt* application option to *false*.

See Also

- [compile_fusion](#)
- [set_app_options](#)

compile.flow.clock_aware_placement

Controls whether the *compile_fusion* command performs clock-aware placement.

Data Types

Boolean

Default false

Description

This application option controls whether the *compile_fusion* command performs clock-aware placement.

By default, the *compile_fusion* command does not perform clock-aware placement.

To enable clock-aware placement, set this application option to *true*.

This application option corresponds to the *place_opt.flow.clock_aware_placement* application option for the *place_opt* command.

See Also

- [compile_fusion](#)
- [set_app_options](#)

compile.flow.conservative_constant_propagation

This application option for *compile_fusion* command has been deprecated and is now obsolete starting Y-2026.03 release.

Data Types

Boolean

Default false

Description

This application option used to enable conservative mode of constant propagation engine inside *compile_fusion*. Constant propagation done in conservative mode results in discovery of fewer number of constant registers. As a result, less number of constant registers will be removed.

This application switch was provided for backward compatibility only. Enabling conservative constant propagation reduces removal of constant registers and increases design area.

With this application option obsoleted, any attempt to set it will trigger an error. User is recommended to remove usage in their scripts.

See Also

- [compile_fusion](#)

compile.flow.constant_and_unloaded_propagation_prevention_with_no_boundary_opt

When constant and unloaded propagations are prevented, this application option provides further control on the 'prevention' direction.

Data Types

integer

Default 3

Description

Application option *compile.flow.constant_and_unloaded_propagation_with_no_boundary_opt* or *set_boundary_optimization* can be used to disable constant and unloaded propagations, based on different values this app-option provides further control on the 'prevention' direction.

c

When this application option value is 3 (the default), the tool prevents propagation in both directions. That is, parent to child and child to parent hierarchies.

When this application option value is 2, the tool prevents propagation only from parent to child hierarchies and allows propagation in the other direction.

When this application option value is 1, the tool prevents propagation only from child to parent hierarchies and allows propagation in the other direction.

This application option should be used only when constant and unloaded propagation is disabled. It has no impact on propagation of equal and opposite information and phase inversion.

See Also

- [compile_fusion](#)
- [set_app_options](#)
- [set_ungroup](#)

compile.flow.constant_and_unloaded_propagation_with_no_boundary_opt

Specifies whether to propagate constants and unloaded logic across hierarchies when boundary optimization is disabled.

Data Types

Boolean

Default true

Description

When this application option is *true* (the default), the tool propagates constants across module hierarchies and simplifies the logic even when boundary optimization is disabled.

When the application option is set to *false*, the tool does not propagate constants and unloaded logic across hierarchical boundaries.

This application option has no effect on other optimizations, such as the propagation of equal and opposite information and phase inversion.

c

See Also

- [compile_fusion](#)
- [set_app_options](#)
- [set_ungroup](#)

compile.flow.enable_ccd

Enables concurrent clock and data optimization (CCD) during the *compile_fusion* command.

Data Types

Boolean

Default true

Description

With this application option is true (the default), concurrent clock and data optimization (CCD) is enabled during the *compile_fusion* command. The data path optimizations are interleaved with useful skew computations during the *initial_opto* stage of the *compile_fusion* command. The useful skew computation incrementally updates the existing user-specified offsets (both balance points and clock latencies). If you provide initial skew offsets, you should match the settings of the *set_clock_balance_points* and *set_clock_latency* commands.

For example, if you specify an ideal latency for a certain clock sink to be -200ps when all other clock sinks have 0ps (the default), that clock sink pin has a 200ps faster clock arrival time. It should have +200ps as clock balance points to instruct clock tree synthesis to implement 200ps faster arrival time for the pin. In this way, there is better correlation between ideal and propagated clocks. The magnitude of skew offsets added by compile CCD is bounded by the *compile.flow.enable_ccd_useful_skew_max_postpone* and the *compile.flow.enable_ccd_useful_skew_max_prepone* application options.

To disable CCD during the *compile_fusion* command, set this application option to *false*.

The general recommendation is to align the CCD setting with that used for the *clock_opt* command.

See Also

- [compile_fusion](#)

compile.flow.enable_ccd_fmax_flow

Data Types

boolean

Default false

Description

Specifies whether the *compile_fusion* command enables the CCD Fmax flow. When this application option is set to true, CCD in *compile_fusion* does a skewing aware of the data path optimization potential, puts more focus to optimize using useful skew optimization the violations which cannot be optimized efficiently by data optimization. It also reduces the useful skew overhead on the clock tree in terms of latency or clock area/power by coming up with the offsets that are only really needed to improve the overall timing. If this application option is set to false, CCD Fmax flow is disabled.

See Also

- [compile_fusion](#)

compile.flow.enable_checkpoint_compression

Enable or disable compression of checkpoint files in SVF.

Data Types

boolean

Default false

Description

This application option enables or disables compression of checkpoint files in SVF.

See Also

- [set_svf](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

compile.flow.enable_complex_register_mapping

Maximize logic absorption and map to complex registers during sequential mapping.

Data Types

Boolean

Default false

Description

This application option controls whether the *compile_fusion* command is decomposing combinational input logic driving a register and prioritize logic absorption and use of complex registers during sequential mapping. It affects the *initial_map* stage of the *compile_fusion* command.

See Also

- [compile_fusion](#)

compile.flow.enable_dpa_ccd_power

Enables Data Path Aware (DPA) Concurrent Clock Data (CCD) optimization for power during *compile_fusion*.

Data Types

boolean

Default false

Description

The *place_opt.flow.enable_dpa_ccd_power* application option controls whether the Data Path Aware (DPA) Concurrent Clock Data (CCD) flow for power optimization is enabled during *compile_fusion*. If *true*, DPA CCD for power is enabled.

During DPA CCD for power, useful skew is also used to improve the power of data paths. Using this flow increases the number of clock tree sinks which are skewed, but this allows data path optimization during *compile_fusion* to provide further power reduction that would not be possible otherwise.

See Also

- [compile_fusion](#)

compile.flow.enable_dpa_ccd_timing

Data Types

boolean

Default false

Description

Specifies whether the *compile_fusion* command enables the DPA CCD timing feature. If this app option is set to true, data-path optimization over-optimizes all positive paths in order to provide extra borrowable slack for paths that are limiting the frequency. If this app option is set to false, DPA CCD timing will be disabled.

See Also

- [compile_fusion](#)

compile.flow.enable_dpa_ccd_timing_with_delay_potential

Data Types

boolean

Default false

Description

Specifies whether the *compile_fusion* command enables the DPA CCD timing feature with the delay potential prediction. If this app option is set to true, a delay potential predictor is used to identify the paths with positive slack that can further be optimized. Data-path over-optimizes only the positive paths with delay potential, as they can be easily closed, to provide extra borrowable slack for paths that are limiting the frequency. If this app option is set to false, DPA CCD timing with delay potential will be disabled.

See Also

- [compile_fusion](#)

compile.flow.enable_dps

Enables Dynamic Power Shaping (DPS) during compile.

Data Types

Boolean

c

Default false**Description**

When this option is set to *true*, Dynamic Power Shaping optimization (DPS) will be enabled during *compile*. This will reduce transient power by utilizing usefull skew. The skew offsets are forwarded to clock_opt stage in the form of clock balance points, for finalizing clock trees in clock_opt.

See Also

- [compile](#)

compile.flow.enable_fast_cus_in_final_opto**Data Types***boolean***Default** false**Description**

This application option enables several recipe changes in compile_fusion final_opto flow, which improve the elapsed time. The changes increase the incrementality and interleaving of skew optimization with datapath optimization. The resulting flow trajectory might be different, but timing and power are expected to be on par by the end of the routing optimization stage.

See Also

- [compile_fusion](#)

compile.flow.enable_hierarchical_checkpoints

Enables generation of verification checkpoints for hierarchical block(s) linked to the current physical block within compile

Data Types

Boolean

Default false

c

Description

This application option controls writing verification checkpoints for hierarchical block(s) linked to the current physical block. By default, verification checkpoints are generated for current physical block.

When this application option is set to *true*, verification checkpoints for hierarchical block(s) are also included, only if hierarchical block(s) is(are) in the same NDM lib and view type is design.

See Also

- [set_verification_checkpoints](#)
- [set_app_options](#)

compile.flow.enable_high_effort_constant_propagation

Controls whether the *compile_fusion* command is allowed to remove constant registers whose initial state cannot be determined.

Data Types

Boolean

Default *true*

Description

This application option controls whether the *compile_fusion* command is allowed to remove constant registers with unknown initial reset state.

Removing such registers from the design can result in mismatch in simulation between RTL and synthesized netlist.

By default *compile_fusion* assumes a constant initialization state. Based on the analysis done using assumed initial state, registers can be found to be constant and removed. In some cases removal of such registers may not be desired.

To prevent removal of registers, set *set_app_options -name compile.flow.enable_high_effort_constant_propagation -value false*

To determine the current value of this application option, use the *get_app_option_value -name compile.flow.enable_high_effort_constant_propagation* command.

For a list of all compile application options and their current values, use the *report_app_options compile.** command.

See Also

- [compile_fusion](#)

compile.flow.enable_incremental_multibit_banking

Controls banking of sequential cells to multibit sequential cells during the `final_opto` stage of the *compile_fusion* command.

Data Types

Boolean

Default false

Description

This application option controls banking of single bit and multibit sequential cells to multibit sequential cells during the `final_opto` stage of the *compile_fusion* command. The tool also modifies the netlist and does scan chain related changes.

Note that the *compile.flow.enable_multibit* application option must also be set to *true* to enable the incremental banking.

See Also

- [compile_fusion](#)

compile.flow.enable_irap

Enables IR aware placement (IRAP) during compile.

Data Types

Boolean

Default false

Description

When this option is set to *true*, IR aware placement optimization (IRAP) will be enabled during *compile*. This will reduce the IR drop by reducing power density in high IR drop spots.

See Also

- [compile](#)

compile.flow.enable_multibit

Controls the mapping to multibit library cells for the *compile_fusion* command.

Data Types

Boolean

Default false

Description

When this application option is set to *true*, the tool performs mapping using available multibit library cells during the *compile_fusion* command.

If you do not want the *compile_fusion* command to use multibit library cells, set the *compile.flow.enable_multibit* application option to *false*.

To determine the current value of this application option, use the *get_app_option_value -name compile.flow.enable_multibit* command.

For a list of all compile application options and their current values, use the *report_app_options compile.** command.

See Also

- [compile_fusion](#)

compile.flow.enable_multibit_debanking

Enables debanking of multibit sequential cells to improve total negative slack.

Data Types

Boolean

Default true

Description

This application option controls whether the *compile_fusion* command performs debanking of multibit sequential cells to improve total negative slack.

The debanking is done to lower width multibit or single-bit sequential cells. The tool also modifies the netlist and does scan chain related changes. The debanking is done during the *initial_opto* and *final_opto* stages of the *compile_fusion* command.

To enable debanking, the *compile.flow.enable_multibit* application option must be set to *true*.

See Also

- [compile_fusion](#)

compile.flow.enable_multibit_pm_cells

Controls the mapping to multibit library cells for the power management cells (ISO/LS/ELS) *compile_fusion* command.

Data Types

Boolean

Default false

Description

When this application option is set to *true*, the tool performs mapping using available multibit power management library cells (ISO/LS/ELS) during the *compile_fusion* command.

If you do not want the *compile_fusion* command to use multibit library cells, set the *compile.flow.enable_multibit_pm_cells* application option to *false*.

To determine the current value of this application option, use the *get_app_option_value -name compile.flow.enable_multibit_pm_cells* command.

For a list of all compile application options and their current values, use the *report_app_options compile.** command.

This application option is obsolete. Option *mv.cells.enable_multibit_pm_cells* is added to replace it.

See Also

- [compile_fusion](#)

compile.flow.enable_multibit_rewiring

Enable rewiring to swap the nets on bits of multibit cells to improve usage of mixed drive strength and mixed VT cells.

Data Types

Boolean

Default false

c

Description

This app option allows users to enable rewiring among bits of multibit cells to increase the number of mixed drive strength cells and mixed VT cells. Such kind of cells have few bits with higher drive strength and other with lower drive strength. Usage of such cells leads to better total power because of lower power consumption on lower drive strength bits. The target of swapping bits is to drive critical nets or bits having higher load with high drive strength or higher VT bits, reducing need of going to a multibit cell with all higher drive strength bits. The rewiring is done both during timing optimization and during power/area optimization. The benefits of this feature will be seen mostly in the designs having mixed drive strength or mixed VT multibit lib cells in the library.

See Also

- [compile](#)

compile.flow.enable_multisource_clock_trees

Enables integrated tap assignment and structural multisource clock tree synthesis in the *compile_fusion* command.

Data Types

Boolean

Default false

Description

When true, enables integrated multisource clock tree synthesis within *compile_fusion* internally. MSCTS option sets are picked up from *set_multisource_clock_tap_options* for Regular MSCTS and *set_multisource_clock_subtree_options* for Structural MSCTS. While using this feature, care must be taken that all required multisource CTS related setup such as global clock tree (Htree) creation, clock mesh creation, annotation on first level net and pin(s), tap/subtree driver insertion, definition of options for tap assignment and structural subtree synthesis, removing constraints like *user_dont_touch*, *fixed*, etc. to allow cell cloning and required CTS related settings are provided before *compile_fusion*. Integrated Regular MSCTS: With integrated tap assignment, *compile_fusion* can see the clock structure changes done as part of tap assignment especially ICG clones, and perform improved placement and optimization. Integrated tap assignment can be performed in ideal or propagated (trial clock tree and *optimize_icgs*) modes. All features honoured by *synthesize_multisource_clock_taps* and *compile_fusion* are supported in integrated regular MSCTS. In this flow, there is no requirement to run standalone *synthesize_multisource_clock_taps*. The database after *compile_fusion* with integrated tap assignment can be taken directly to *clock_opt build_clock*. Integrated Structural MSCTS: With integrated subtree synthesis, *compile_fusion* can see the clock structure changes done as part of structural subtree synthesis, especially ICG clones,

c

and perform improved optimization. Integrated structural subtree synthesis can be performed only in ideal mode. All steps of `synthesize_multisource_clock_subtrees` and features honoured by `compile_fusion` are supported in integrated structural MSCTS. In integrated SMSCTS in `compile_fusion` flow with CCD, offsets derived by CCD during `final_opto` can be implemented by a subsequent round of subtree optimization after `compile_fusion`. This can be performed in an integrated manner by `clock_opt build_clock` command by setting the app_option `clock_opt.flow.enable_multisource_clock_trees` to true. Alternatively, this incremental round of subtree synthesis can be done by a call to `synthesize_multisource_clock_subtrees`, which will operate incrementally and automatically start from optimize step.

See Also

- [compile_fusion](#)
- [synthesize_multisource_clock_taps](#)

compile.flow.enable_mv_merge_clone

Enables the merge/clone of multi-voltage cells during `compile_fusion` command.

Data Types

boolean

Default false

Description

When this application option is true, the tool performs the merge/clone of isolation cells, level shifters and enable level shifters.

This feature explores the merging/cloning of a set of multi-voltage cells to improve metrics such as timing and area. The multi-voltage cells can be merged if:

- They are all the same type, e.g, all isolation cells.
- Belong to the same hierarchy.
- Have the same local driver(s).
- Associated with the same strategy. For level shifters not associated to any level shifter strategy (auto inserted by the tool), they are also able to be merged if match other 3 conditions.

c

See Also

- [compile_fusion](#)
- [set_app_options](#)

compile.flow.enable_physical_multibit_banking

Controls multibit packing that considers placement during the *compile_fusion* command.

Data Types

Boolean

Default `true`

Description

When this application option is *true* (the default) and the *compile.flow.enable_multibit* application option is set to *true* (which enables multibit library cells mapping), the tool automatically performs in-place multibit packing of compatible registers that are in physical proximity.

The packing honors PVT compatibility, scan chain integrity, exceptions, and constraints. The packed registers belong to the same hierarchy. To preserve timing QoR, only uncritical registers are considered for multibit packing.

To determine the current value of this application option, use the *get_app_option_value -name compile.flow.enable_physical_multibit_banking* command.

For a list of all compile application options and their current values, use the *report_app_options compile.** command.

See Also

- [compile_fusion](#)
- [set_multibit_options](#)

compile.flow.enable_power**Data Types**

boolean

Default `false`

c

Description

Specifies whether the *compile_fusion* command enables power optimization. If this app option is set to true, power optimization is enabled and power type is determined based on scenario settings as shown in the table below. If this app option is set to false, power optimization will be disabled regardless of scenario settings.

Leakage-scenario Dynamic-scenario -> FlowType

False False -> Area

True False -> Leakage

False True -> Total

True True -> Total

See Also

- [compile_fusion](#)

compile.flow.enable_rtl_multibit_banking

Controls multibit packing when mapping sequential cells during compile.

Data Types

Boolean

Default true

Description

When this application option is set to *true* and the *compile.flow.enable_multibit* application option is set to *true* (which enables multibit library cell mapping), the tool automatically performs in-place multibit packing of compatible registers during sequential mapping.

To determine the current value of this application option, use the *get_app_option_value -name compile.flow.enable_rtl_multibit_banking* command.

For a list of all compile application options and their current values, use the *report_app_options compile.** command.

See Also

- [compile_fusion](#)
- [report_multibit](#)
- [set_multibit_options](#)

compile.flow.enable_rtl_multibit_debanking

Controls the multibit unpacking that considers placement during compile.

Data Types

Boolean

Default true

Description

When this application option is set to *true*, which enables multibit library cell mapping, the tool automatically performs multibit unpacking of critical registers. The unpacking honors scan chain integrity, exceptions, and constraints. To minimize degradation in power QoR, only critical registers are considered for multibit unpacking.

To determine the current value of this application option, use the *get_app_option_value -name compile.flow.enable_rtl_multibit_debanking* command.

For a list of all compile application options and their current values, use the *report_app_options compile.** command.

See Also

- [compile_fusion](#)
- [report_multibit](#)
- [set_multibit_options](#)

compile.flow.enable_second_pass_multibit_banking

Controls banking of sequential cells to multibit sequential cells during the final_place stage of the *compile_fusion* command.

Data Types

Boolean

Default false

Description

This application option controls banking of single bit and multibit sequential cells to multibit sequential cells during the final_place stage of the *compile_fusion* command. The tool also modifies the netlist and does scan chain related changes.

Note that the *compile.flow.enable_multibit* application option must also be set to *true* to enable banking.

See Also

- [compile_fusion](#)

compile.flow.enable_toggle_aware_rtl_multibit_reordering

Enable the toggle-rate aware reorder of bits in the mapped multibit cells

Data Types

Boolean

Default false

Description

This application option, when set to *true*, enables the toggle-rate aware reorder of bits in mapped cells across a bus. This feature aims to minimize power by assigning the bit with the lowest toggle activity to the scan chain tail. Since the scan chain tail has long scan connectivity, such an assignment lowers the net capacitance. The reordering is performed across the mapped single-bit and multibit cells of a bus.

The reordering is only performed if the multibit cells in the bus all have internal serial scan-chain. The reordering is skipped if the multibit cells in the bus are not scannable, have parallel scan, or have all single-bit cells. The names of the multibit and the single-bit cells are changed to reflect the new reordering.

See Also

- [compile_fusion](#)

compile.flow.enable_unloaded_dt_cell_fanin_cleanup

Enables deletion of fanin of unloaded cells with only user_dont_touch

Data Types

Boolean

Default false

Description

When this application option is *true*, the tool will delete fanin of unloaded cells with only user_dont_touch.

c

See Also

- [compile_fusion](#)
- [set_app_options](#)

compile.flow.enable_via_ladder_insertion

Controls via ladder insertion in the *compile_fusion* command.

Data Types

Boolean

Default false

Description

This application option controls via ladder insertion in the *compile_fusion* command.

By default, the *compile_fusion* command does not perform via ladder insertion.

When you set this application option to *true*, performance via ladders are inserted during the last stage of the *compile_fusion* command.

This application option will be removed in a future release. Use the *opt.common.enable_via_ladder_insertion* application option instead.

See Also

- [compile_fusion](#)
- [set_app_options](#)
- [set_via_ladder_constraints](#)
- [set_via_ladder_rules](#)
- [set_via_ladder_spacing](#)

compile.flow.enhanced_delay_opto_for_pg

Enables enhanced timing optimizations in the *compile_fusion* command.

Data Types

Integer (0 or 1)

Default 0

c

Description

When you enable the `compile.flow.enhanced_delay_opto_for_pg` application option, the `compile_fusion` command runs an enhanced strategy to improve the delay of the design, possibly at the cost of additional runtime, area and power.

Use this application option only on designs that require path groups to improve timing optimization. This strategy performs additional runtime intensive passes to achieve better timing optimization.

See Also

- [compile_fusion](#)
- [set_app_options](#)

compile.flow.high_effort_area

Enables high-effort area optimizations in the `compile_fusion` command.

Data Types

Boolean

Default `false`

Description

When you enable the `compile.flow.high_effort_area` application option, the `compile_fusion` command runs a strategy intended to improve the total area of the design, possibly at the cost of additional runtime. Use this application option only on designs that require further area optimization. This strategy performs additional runtime intense passes to achieve better area optimization.

See Also

- [compile_fusion](#)
- [set_app_options](#)

compile.flow.high_effort_timing

Enables high-effort timing optimizations in the `compile_fusion` command.

Data Types

integer (0 or 1)

Default `0`

c

Description

When you enable the *compile.flow.high_effort_timing* application option, the *compile_fusion* command runs a strategy intended to improve the delay of the design, possibly at the cost of additional runtime.

Use this application option only on designs that require further timing optimization. This strategy performs additional runtime intensive passes to achieve better timing optimization.

This application option should only be set to a value of 0 or 1. For backward compatibility, values greater than 1 are accepted and treated the same as a value of 1. In the future, the application option type will change to Boolean and values greater than 1 will not be accepted. You should update your scripts to use 0 or 1.

See Also

- [compile_fusion](#)
- [set_app_options](#)

compile.flow.hold_area_budgeting

Enable the enhanced Hold Area Budgeting feature in the *compile_fusion* final_place stage

Data Types

string

Default false

Description

This application option enables the enhanced Hold Area Budgeting feature during *compile_fusion*. This feature models and reserves the amount of space required for hold fixing during clock_opt in the *compile_fusion* stage itself, thus creating a more convergent flow. Valid values are "false" and "enhanced".

See Also

- [compile_fusion](#)

compile.flow.layer_aware_optimization

Enables layer-aware optimization in the *compile_fusion* command.

c

Data Types

Boolean

Default true

Description

This application option enables layer-aware optimization in the *compile_fusion* command.

By default, the *compile_fusion* command performs layer-aware optimization, which does dynamic layer assignment and assigns nets with large delay to the top metal (less resistive) layers to improve timing.

See Also

- [compile_fusion](#)
- [set_app_options](#)

compile.flow.legalize_unmapped_fixed

Specifies whether *compile_fusion* will convert all unmapped cells *physical_status* from fixed to unplaced or *legalize_only*.

Data Types

Boolean

Default "false"

Description

Cells with physical status are subjected replacement restrictions in *compile_fusion* and cells cannot remain unmapped by end of *initial_map*. *Compile_fusion* will have to remove fixed'ness on the unmapped cells or convert them into another status sothat they get cells mapped but not replaced. If this option is *false*, default value, fixed unmapped cells status will be converted to unplaced. This will enable optimization engines to map the cells and optimize, if required. If this option is *true*, fixed status will be converted to *legalize_only*. *Legalize_only* cells are mapped and subjected to replacement restrictions, but can not be removed.

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

compile.flow.merge_clock_gates

Enable ICG merging optimization during final_place step of compile_fusion.

Data Types

Boolean

Default false

Description

When this app_option is set to *true*, ICG merging (*merge_clock_gates*) will be run internally inside *compile_fusion* final_place stage. Here clock gate merging happens as a first step of final_place.

See Also

- [compile_fusion](#)
- [merge_clock_gates](#)

compile.flow.optimize_icgs

Controls integrated clock gating (ICG) optimization in the *compile_fusion* command.

Data Types

Boolean

Default false

Description

This application option controls integrated clock gating (ICG) optimization in the *compile_fusion* command.

By default, the *compile_fusion* command does not perform ICG optimization.

When you set this application option to *true*, the *compile_fusion* command runs automatic ICG optimization that performs trial clock synthesis, timing-aware ICG splitting for critical enable paths, and clock-aware placement.

Trial clock tree synthesis is performed regardless of the value of the *compile.flow.trial_clock_tree* application option, to identify critical ICG enable paths and all the *compile_fusion* optimization steps.

After the *compile_fusion* command completes, the timing QoR cannot be directly compared to conventional ideal-clock-based compile QoR. The trial clock tree is

c

automatically removed and resynthesized by the *clock_opt* or *synthesize_clock_trees* command.

ICG oriented clock-aware placement is automatically used, which further helps ICG enable path timing improvement.

To control the aggressiveness of ICG splitting, use the *compile.flow.optimize_icgs_critical_range* application option.

To maximize the benefit from the well-correlated trial clock trees, apply as many clock-related settings as possible before running the *compile_fusion* command, such as clock nondefault routing rules or post-clock uncertainties.

Top-level design support is limited. With read-only subblocks, ICG optimization is limited to the current block, and the *compile_fusion* command immediately exits with an error message if it detects modifiable subblocks.

See Also

- [compile_fusion](#)
- [set_app_options](#)

compile.flow.optimize_icgs_critical_range

Specifies the critical range for integrated clock gating (ICG) optimization in compile.

Data Types

float

Default 0.75

Description

This application option specifies the critical range for ICG optimization in compile. You must specify a value between 1.0 and 0.0.

When set to X , only ICGs whose EN violations are within $\{EN_WNS, EN_WNS*(1-X)\}$ are considered for splitting. The larger the value, the more aggressive ICG splitting is expected by including more ICGs for splitting consideration.

This application has an effect only when the *compile.flow.optimize_icgs* application option is set to *true*.

See Also

- [compile_fusion](#)
- [set_app_options](#)

compile.flow.optimize_icgs_use_timing_driven_splitting

Enable timing-driven ICG optimization in compile

Data Types

Boolean

Default true

Description

This is a sub option for *compile.flow.optimize_icgs*. This option will only take effect when *compile.flow.optimize_icgs* is enabled in the compile flow. When set to true, more runtime efficient timing-driven ICG splitting will be used in the ICG optimization flow in compile.

See Also

- [set_app_options](#)
- [compile](#)

compile.flow.optimize_layers

Controls the legacy layer optimization technology in the *compile_fusion* command.

Data Types

Boolean

Default false

Description

This option has been deprecated; this option will be removed in the subsequent release.

This application option controls the legacy layer optimization technology in the *compile_fusion* command.

By default, the legacy layer optimization technology is disabled.

However, the *compile_fusion* command always performs layer optimization that is tightly integrated with buffering, which ensures that critical signals are promoted to low-RC layers and buffered to take advantage of those layers.

See Also

- [compile_fusion](#)
- [set_app_options](#)

compile.flow.optimize_layers_critical_range

Specifies the critical range for layer optimization.

Data Types

float

Default 0.0001

Description

This application option specifies the critical range for layer optimization. You must specify a value between 0.0 and 1.0.

For a given value y , the *compile_fusion* command optimizes nets that have slack between $\{y \cdot \text{WNS}, \text{WNS}\}$, where WNS is the worst negative slack.

To fix paths that are in a less critical range, set this application option to a lower value. However, this can cause many more nets to be promoted and use much more routing resources. Therefore, lower the critical range only if necessary.

This application option has an effect only when the *compile.flow.optimize_layers* application option is set to *true*.

See Also

- [compile_fusion](#)
- [set_app_options](#)

compile.flow.optimize_layers_overlap_bins

Controls overlap bins for layer optimization.

Data Types

Boolean

Default false

Description

This application option controls overlap bins for layer optimization.

By default, layer bins do not overlap. For example, {M10, M9}, {M8, M7}, {M6, M5}.

When this application option is set to *true*, layer bins can overlap. For example, {M10, M9}, {M9, M8}, {M8, M7}, {M7, M6}, {M6, M5}.

c

The feature might help improve QoR when routing resources are unbalanced. For example, assume that the M10 routing capacity is low, but M9 is high. When layer bins can overlap, nets can be promoted to {M9, M8} to improve QoR. When layer bins cannot overlap, nets are promoted to {M8, M7}.

This application option has an effect only when the *compile.flow.optimize_layers* application option is set to *true*.

See Also

- [compile_fusion](#)
- [set_app_options](#)

compile.flow.optimize_ndr

Controls nondefault routing rule (NDR) optimization in the *compile_fusion* command.

Data Types

Boolean

Default false

Description

This application option controls NDR optimization in the *compile_fusion* command.

By default, the *compile_fusion* command does not perform NDR optimization.

When this application option is set to *true*, the *compile_fusion* command performs automatic NDR optimization that assigns long timing critical nets to NDRs to improve timing.

See Also

- [compile_fusion](#)
- [set_app_options](#)

compile.flow.optimize_ndr_critical_range

Specifies the critical range for nondefault routing rule (NDR) optimization.

Data Types

float

Default 0.05

c

Description

This application option specifies the critical range for NDR optimization. Specify a value between 0.0 and 1.0.

For a given value y , the *compile_fusion* command optimizes nets that have slack between $\{y \cdot \text{WNS}, \text{WNS}\}$, where WNS is the worst negative slack.

To fix paths that are in a less critical range, set this application option to a lower value. However, this might cause many more nets to have NDRs and use more routing resources. Therefore, lower the critical range only if necessary.

This application option has an effect only when the *compile.flow.optimize_ndr* application option is set to *true*.

See Also

- [compile_fusion](#)
- [set_app_options](#)

compile.flow.optimize_ndr_max_nets

Specifies the maximum number of nets to assign to nondefault routing rules (NDRs) during NDR optimization.

Data Types

integer

Default 2000

Description

This application option specifies the maximum number of nets to assign to nondefault routing rules (NDRs) during NDR optimization. This number limits the number of potential candidates that can benefit from applying NDRs; it must be a non-negative integer.

This application option has an effect only when the *compile.flow.optimize_ndr* application option is set to *true*.

See Also

- [compile_fusion](#)
- [set_app_options](#)

compile.flow.print_messages_for_redundant_registers

Enables printing of messages for redundant registers in the *compile_fusion* command.

Data Types

Boolean

Default false

Description

This application option enables the printing of messages for redundant registers in the *compile_fusion* command.

When the app option is set to *true*, the tool prints the name of register pins which have become constant or unloaded due to redundancy removal optimisation and could be optimized as constant/unloaded registers in later steps.

See Also

- [CGRR-0001](#)
- [CGRR-0002](#)

compile.flow.propagate_constants_through_dont_touch_cells

Controls whether the *compile_fusion* command enables constant propagation through cells with a *dont_touch* constraint.

Data Types

Boolean

Default false

Description

This application option controls whether the *compile_fusion* command enables constant propagation through cells with a *dont_touch* constraint.

When this application option is *true*, the *compile_fusion* command enables the propagation of constant values through cells with a *dont_touch* constraint, which mean the nets and cells that follow those constant cells can be disconnected or removed.

When this application option is *false* (the default), the propagation stops at those *dont_touch* constant cells.

To determine the current value of this application option, use the *get_app_option_value -name compile.flow.propagate_constants_through_dont_touch_cells* command.

For a list of all compile application options and their current values, use the *report_app_options compile.** command.

See Also

- [compile_fusion](#)

compile.flow.propagate_constants_through_registers

Controls whether the *compile_fusion* command enables constant propagation through registers.

Data Types

Boolean

Default true

Description

This application option controls whether the *compile_fusion* command enables constant propagation through registers (clock gating cells included, latches and retention cells excluded).

When this application option is *true* (the default), the *compile_fusion* command enables the propagation of constant values through all registers.

When this application option is *false*, the *compile_fusion* command disables the propagation of constant values through all registers.

To set the value of this application option, use the *set_app_options -name compile.flow.propagate_constants_through_registers -value false* command.

To get the current value of this application option, use the *set_app_option_value -name compile.flow.propagate_constants_through_registers* command.

For a list of all compile application options and their current values, use the *report_app_options compile.** command.

See Also

- [compile_fusion](#)

compile.flow.propagate_constants_through_size_only_cells

Controls whether the *compile_fusion* command enables constant propagation through cells with a *size_only* constraint.

c

Data Types

Boolean

Default false

Description

This application option controls whether the *compile_fusion* command enables constant propagation through cells with a *size_only* constraint.

When this application option is *true*, the *compile_fusion* command enables the propagation of constant values through cells with a *size_only* constraint, which mean the nets and cells that follow those constant cells can be disconnected or removed. But the cells with a *size_only* constraint will not be removed (unless overridden by *set_size_only -enable_constant_gate_removal* command).

When this application option is *false* (the default), the propagation stops at those *size_only* constant cells.

To determine the current value of this application option, use the *get_app_option_value -name compile.flow.propagate_constants_through_size_only_cells* command.

For a list of all compile application options and their current values, use the *report_app_options compile.** command.

See Also

- [compile_fusion](#)
- [set_size_only](#)

compile.flow.skew_macros_effort

Specifies the effort level for *set_skew_macros* commands

Data Types

string

Default medium

Description

This option specifies the effort level for *set_skew_macros* commands. This can influence various decisions during skewing of macros. For example, the amount of skew could be the lowest with *low*, moderate with *medium* and highest with *high*.

See Also

- [set_skew_macros](#)

compile.flow.tie_all_unused_hier_inputs_to_constants

Specifies whether to connect unused hierarchical input pins to logic constants after each stage in `compile_fusion`

Data Types

Boolean

Default `false`

Description

When this application option is *true* (the default is *false*), the tool will connect unused hierarchical input pins to the default logic constant.

When the application option is set to *false*, the tool does not connect unused hierarchical input pins to the default logic constant.

This application option controls the constant hook up across all stages in `compile_fusion`.

See Also

- [compile_fusion](#)
- [set_app_options](#)

compile.flow.tie_unused_hier_inputs_to_constants

Specifies whether to connect unused hierarchical input pins to logic constants after each stage in `compile_fusion`

Data Types

Boolean

Default `false`

Description

When this application option is *true* (the default is *false*), the tool will connect unused hierarchical input pins to the default logic constant.

When the application option is set to *false*, the tool does not connect unused hierarchical input pins to the default logic constant.

c

This application option controls the constant hook up to the `initial_opto` stage in `compile_fusion`.

See Also

- [compile_fusion](#)
- [set_app_options](#)

compile.flow.tieoff_undriven_top_output_ports

Controls whether undriven top level (primary) ports will be tied to `tie_low` during during the `compile_fusion` command.

Data Types

Boolean

Default `false`

Description

This application option controls whether undriven top level (primary) ports will be tied to `tie_low` during during the `compile_fusion` command.

By default, such ports are not tied to `tie_low` during the `compile_fusion` command.

When this application option is set to `true`, such ports will be tied to `tie_low`.

See Also

- [compile_fusion](#)
- [set_app_options](#)

compile.flow.trial_clock_tree

Enables early clock tree synthesis in the `compile_fusion` command.

Data Types

string

Default `false`

c

Description

This application option controls whether the *compile_fusion* command performs early clock tree synthesis, which is useful for low power placement enabled compile and the ICG optimization flow. Valid values are *false* (the default) and *true*.

When this application option is set to *true*, the *compile_fusion* command performs clock tree synthesis and uses the propagated clocks throughout the compile flow. Therefore, the compile QoR is not directly comparable to conventional ideal clock based compile flows. The *compile_fusion* command performs placement and optimization based on the clock tree structure as well as the propagated clocks from the trial clock trees.

The trial clock trees are automatically removed and resynthesized by the *clock_opt* or *synthesize_clock_trees* command, which builds the finalized clock trees.

To maximize the benefit from well-correlated trial clock trees, apply as many clock-related settings as possible before running the *compile_fusion* command, such as clock nondefault routing rules or post-clock uncertainties.

See Also

- [compile_fusion](#)
- [report_app_options](#)
- [set_app_options](#)

compile.flow.user_instance_name_prefix

The value of option *compile.flow.user_instance_name_prefix* is used as prefix of names of new cells created during *compile_fusion*.

Data Types

String

Default ""

Description

Users can specify a string to be prepended to new cell names during *compile_fusion* by setting app option value of *compile.flow.user_instance_name_prefix*. By default, all newly created cells during *compile_fusion* may have different or no prefix depending upon the optimization during which cell is generated. This option provides the same prefix to all cells irrespective of the optimization, value of this option will be added as prefix after applying other optimization specific applicable prefixes.

If newly created cell's hierarchy gets ungrouped then this value will be inside full name. For in-compile DFT flow, newly created cells will not be honoring this app option but will those

c

cells can be identifiable as they will be under U_DFT_TOP_IP_* cell. UPF created cells may not follow this app option as they are driven by UPF specific naming policies.

Here is example of usage:

```
set_app_option -name compile.flow.user_instance_name_prefix -value  
"FCFE_"  
compile_fusion
```

See Also

- [compile_fusion](#)
- [set_app_options](#)

compile.incr_initial_opto.placement_congestion

Enables Congestion Timing Driven Placement in incremental initial_opto.

Data Types

boolean

Default false

Description

When you enable the *compile.incr_initial_opto.placement_congestion* application option, the incremental *compile_fusion* command runs congestion placement intended to improve congestion of the design, possibly at the cost of additional runtime.

Use this application option only on designs that require further congestion optimization. This strategy performs additional runtime intensive passes to achieve better congestion values.

This application option only applies on incremental *compile_fusion* calls.

See Also

- [compile_fusion](#)
- [set_app_options](#)

compile.initial_drc.global_route_based

Controls whether the *compile_fusion* command runs global-route-based buffering when performing high fanout synthesis in the initial_drc stage.

c

Data Types

integer

Default false

Description

This application option controls whether the *compile_fusion* command runs global-route-based buffering when performing high fanout synthesis in the *initial_drc* stage.

By default, global-route-based buffering is not enabled.

When this application option is set to *true*, the *compile_fusion* command works with globally routed nets to determine the topology for buffering the trees during high fanout net synthesis in the *initial_drc* stage. After buffering, all global routes are deleted, and the modified topologies anchor the buffers and guide downstream optimization.

This application option affects buffering only during the *initial_drc* stage.

See Also

- [compile_fusion](#)

compile.initial_place.buffering_aware

Controls whether the *compile_fusion* command runs buffering-aware timing-driven placement for the initial placement step.

Data Types

Boolean

Default true

Description

This application option controls whether the *compile_fusion* command runs buffering-aware timing-driven placement for the initial placement step.

By default, the *compile_fusion* command runs buffering-aware timing-driven placement to generate a better initial placement for later timing optimization steps in the *compile_fusion* command. Buffering-aware timing-driven placement performs timing-driven placement using a timing model that estimates the effect of later buffering of long nets and high-fanout nets. The netlist is not changed.

To disable buffering-aware timing-driven placement for the initial placement step, set this application option to *false*.

See Also

- [compile_fusion](#)
- [set_app_options](#)

compile.initial_place.buffering_aware_placement_effort

Specifies the CPU effort level for the buffering aware initial placement performed by the *compile_fusion* command.

Data Types

string

Default medium

Description

This application option specifies the CPU effort level for the buffering aware timing-driven placement in initial placement step performed by the *compile_fusion* command. Valid values are *low*, *medium*, and *high*.

To enable buffering aware timing-driven placement, set the application *compile.initial_place.buffering_aware* to *true*.

See Also

- [compile_fusion](#)
- [set_app_options](#)

compile.initial_place.effort

Specifies the CPU effort level for the initial placement performed by the *compile_fusion* command.

Data Types

string

Default medium

Description

This application option specifies the CPU effort level for the initial placement performed by the *compile_fusion* command. Valid values are *low*, *medium*, and *high*.

c

See Also

- [compile_fusion](#)
- [create_placement](#)

compile.log.tbcs_report

Reports Tbc files read in and evaluated in the application.

Data Types

boolean

Default true

Description

The application option value specifies if the commands *compile_fusion* and *rtl_opt* should evaluate the command *report_tbcs* at their startup.

Set the value to *true* if you want to report Tbcs in the output log.

Set the value to *false* or reset the app option if you dont want to see Tbcs reported in the output log of *compile_fusion* and *rtl_opt* commands.

See Also

- [compile_fusion](#)
- [set_app_options](#)
- [report_tbcs](#)

compile.log.timestamp

Specifies the timestamp shown with each compile progress percentage.

Data Types

string

Default elapsed_time

Description

This application option specifies the type of timestamp to use when displaying the compile progress percentage. Valid values are *current_time*, *elapsed_time*, and *none*.

c

See Also

- [compile_fusion](#)
- [set_app_options](#)

compile.multibit.prefer_multibit_libcells_with_singlebit_equivalent

Directs the *compile_fusion* command to map sequential multibit elements to be mapped to multibit library cells that have singlebit equivalent.

Data Types

Boolean

Default false

Description

This application option directs the *compile_fusion* command to map sequential elements to multibit library cells that have singlebit equivalent.

When this application option is *false* (the default), the *compile_fusion* command maps the sequential elements in the design to sequential multibit library cells without regard to whether the multibit library cells has a singlebit equivalent or not.

When this application option is set to *true*, the *compile_fusion* command will prefer to map the sequential elements in the design to sequential multibit library cells that have singlebit equivalent. If the tool fails to maps to such sequential multibit library cells, it falls back to default behavior.

To determine the current value of this application option, use the *get_app_option_value -name compile.multibit.prefer_multibit_libcells_with_singlebit_equivalent* command.

For a list of all compile application options and their current values, use the *report_app_options compile.** command.

See Also

- [compile_fusion](#)

compile.multibit.support_wrapper_cells

Implements debanking of multibit cells to ensure compatibility of DFT core wrapper cell types during *compile_fusion* command.

c

Data Types

Boolean

Default false

Description

compile_fusion must ensure wrapper cells that belong to the same type are banked together. *insert_dft* will produce erroneous results if wrapper and non-wrapper cells are found to be banked. During *compile_fusion*, if multibit cells are found to have slices belonging to incompatible types, they must be debanked such that debanked slices belong to same type. Set value of this app-option to true to ensure wrapper cell type compatibility during multibit optimizations done by *fusion_compiler*.

See Also

- [compile_fusion](#)

compile.optimization.enable_hierarchical_inverter

Controls whether inverters can be moved across hierarchical boundaries during boundary optimization.

Data Types

Boolean

Default true

Description

This application option affects the behavior of the boundary optimization feature in the *compile_fusion* command.

When this application option is *true* (the default), boundary optimization attempts to push inverters across hierarchies to improve the optimization cost of the design.

When this application option is *false*, boundary optimization does not move inverters across hierarchical boundaries even if doing so could improve the design.

See Also

- [compile_fusion](#)
- [set_boundary_optimization](#)

compile.optimization.infer_mux_reencode_supports

Controls the structure of the mux tree generated for SELECT_OPs that are marked with the *infer_mux_override* pragma or *map_to_mux* attribute.

Data Types

Boolean

Default false

Description

This application option affects the synthesis of SELECT_OPs that have been marked with the *infer_mux_override* pragma or *map_to_mux* attribute.

When this application option is *false* (the default), the mux tree generated for the SELECT_OP uses the select signals inferred during the elaboration of the RTL.

When this application option is *true*, the select signals of the SELECT_OP are re-encoded to a smaller set when possible. By reducing the number of select signals controlling the mux tree, the number of the mux gates generated for the SELECT_OP is reduced, thereby reducing the area of the mux tree corresponding to the SELECT_OP.

See Also

- [compile_fusion](#)

compile.physical.save_and_restore_ndr_auto_mode

Controls whether the *compile_fusion* command saves and restores non default routing rules during *initial_map* and *logic_opto* stages.

Data Types

Boolean

Default true

Description

This application option controls whether the *compile_fusion* command saves and restores non default routing rules during *initial_map* and *logic_opto* stages.

See Also

- [create_routing_rule](#)
- [remove_routing_rules](#)

- [report_routing_rules](#)
- [set_routing_rule](#)

compile.physical.save_and_restore_ndr_during_ungroup

Controls the saves and restores of non default routing rules during *ungroup*.

Data Types

Boolean

Default true

Description

This application option controls the saves and restores of non default routing rules during *ungroup*.

See Also

- [create_routing_rule](#)
- [remove_routing_rules](#)
- [report_routing_rules](#)
- [set_routing_rule](#)

compile.retiming.optimization_priority

Specifies the priority metric to use during retiming.

Data Types

string

Default auto

Description

This application option specifies the priority metric to use during retiming. Valid values are

- *auto* (default)
The tool determines the best tradeoff between register count and timing.
- *timing*
Timing is high priority while minimizing register count.

c

- *register_count*

Register count reduction is prioritized over timing. Timing violations are assumed to be recovered by other optimizations.

See Also

- [get_app_option_value](#)
- [set_app_options](#)
- [compile_fusion](#)
- [set_optimize_registers](#)

compile.retiming.simu_mismatch_prevention

Prevents forward retiming of some critical registers which can lead to simulation mismatch.

Data Types

bool

Default false

Description

In some special cases, retiming of some no-reset registers can cause an endpoint to take up a non-deterministic state at power-up, which was initially having a deterministic state, eventually leading to simulation failure. This application option controls whether to prevent forward retiming of such registers. Such situation arises only in some special cases, when there is an interaction of reset and noreset registers and after retiming the no-reset registers sit at some specific position. Hence, the app-option is false by default.

When this application option is true, forward retiming may be prevented for some no-reset registers which can potentially cause simulation failure at some outputs. If any such register is prevented, it is reported with *RTM-1025* message. If still some endpoint becomes non-deterministic after retiming it is reported by a *RTM-1026* message.

When this application option is false, no register is prevented from retiming for simulation mismatch reasons.

Examples

The following command sets simulation mismatch prevention to true.

```
prompt> set_app_options \ -name compile.retiming.simu_mismatch_prevention -value true
```

c

See Also

- [compile_fusion](#)
- [get_app_option_value](#)
- [set_app_options](#)

compile.seqmap.allow_async_set_reset_priority_inversion_if_unmapped

Controls whether the *compile_fusion* command allows sequential elements to have the priority of their asynchronous set and reset pins inverted when it is necessary to map them to sequential library cells.

Data Types

Boolean

Default true

Description

This application option controls whether the *compile_fusion* command allows sequential elements to have the priority of their asynchronous set and reset pins inverted when it is necessary to map them to sequential library cells.

When this application option is *true* (the default), the *compile_fusion* command allows the mapping of the sequential elements in the design to sequential library cells with inverted priority between their asynchronous set and reset pins when cells with the priority inversion are unavailable.

To determine the current value of this application option, use the *get_app_option_value -name compile.seqmap.allow_async_set_reset_priority_inversion_if_unmapped* command.

For a list of all compile application options and their current values, use the *report_app_options compile.** command.

See Also

- [compile_fusion](#)

compile.seqmap.allow_output_inversion_if_unmapped

Controls whether the *compile_fusion* command allows sequential elements to have their output phase inverted when necessary to map them to sequential library cells.

c

Data Types

Boolean

Default true

Description

This application option controls whether the *compile_fusion* command allows sequential elements to have their output phase inverted when necessary to map them to sequential library cells.

When this application option is *true* (the default), the *compile_fusion* command allows the mapping of sequential elements in the design to sequential library cells whose output phase is inverted, when cells without output phase inversion are unavailable.

When using sequential output inversion, you must use the SVF file to verify the functionality of the design using Formality.

To determine the current value of this application option, use the *get_app_option_value -name compile.seqmap.allow_output_inversion_if_unmapped* command.

For a list of all compile application options and their current values, use the *report_app_options compile.** command.

See Also

- [compile_fusion](#)

compile.seqmap.allow_tieoffs_for_latches

Controls whether the *compile_fusion* command allows sequential elements be mapped to latch cells in the library with some ports not used.

Data Types

Boolean

Default true

Description

This application option controls whether the *compile_fusion* command allows sequential elements be mapped to latch cells in the library with some ports not used.

When this application option is *true* (the default), the *compile_fusion* command allows the sequential elements in the design to be mapped to library latch cells whose ports are not fully used. The dangling ports are tied off to ensure correct functionality. This can help improve QoR. In some libraries, library cells with fewer ports might have a larger area

c

than those with more ports. This gives the tool the flexibility to choose cells that match the binary decision diagrams (bdds) and result in the best QoR.

When this application option is *false*, the *compile_fusion* command only allows SEQGENs to be mapped to library latch cells with an exact port match. This feature might be required by some users.

To determine the current value of this application option, use the *get_app_option_value -name compile.seqmap.allow_tieoffs_for_latches* command.

For a list of all compile application options and their current values, use the *report_app_options compile.** command.

See Also

- [compile_fusion](#)

compile.seqmap.allow_tieoffs_for_registers

Controls whether the *compile_fusion* command allows sequential elements to be mapped to register cells in the library with some ports not used.

Data Types

Boolean

Default true

Description

This application option controls whether the *compile_fusion* command allows sequential elements to be mapped to register cells in the library with some ports not used.

When this application option is *true* (the default), the *compile_fusion* command allows the sequential elements in the design to be mapped to library register cells whose ports are not fully used. The dangling ports are tied off to ensure the correct functionality. This can help improve QoR. In some libraries, library cells with fewer ports could have a larger area than those with more ports. This gives the tool the flexibility to choose cells that match the binary decision diagrams (bdds) and result in the best QoR.

When this application option is *false*, the *compile_fusion* command only allows sequential elements be mapped to library register cells with an exact port match. This feature might be required by some users.

To determine the current value of this application option, use the *get_app_option_value -name compile.seqmap.allow_tieoffs_for_registers* command.

For a list of all compile application options and their current values, use the *report_app_options compile.** command.

See Also

- [compile_fusion](#)

compile.seqmap.disable_async_logic_push_out_for_latches

Don't allow logic push out for latch SEQGEN

Data Types

Boolean

Default false

Description

This application option controls whether the *compile_fusion* command will allow set/reset logic push out for latch SEQGEN instance. When this command is set to *true*, it means logic push out is not allowed. When this command is set to *false*, set/reset logic will be pushed out if library doesn't have set/reset type latch. Note this app option doesn't apply to pre-instantiated latch instance. Pre-instantiated latch by default doesn't allow logic push out.

To determine the current value of this application option, use the *get_app_option_value -name compile.seqmap.disable_async_logic_push_out_for_latches* command.

For a list of all compile application options and their current values, use the *report_app_options compile.** command.

See Also

- [compile_fusion](#)

compile.seqmap.disable_qn_usage

Controls whether the *compile_fusion* command allows the usage of QN pin connections of sequential elements.

Data Types

Boolean

Default false

Description

This application option controls whether the *compile_fusion* command allows the usage of QN pin connections of sequential elements.

c

When this application option is *true*, the *compile_fusion* command will not use QN pin for any connection for all sequential elements. This is useful when there is a constraint on the Q pin of some registers and the user does not want the sequential mapper to replace those Q pins.

When this application option is *false*(the default), the *compile_fusion* command allows sequential elements to use QN pin connections to improve QoR.

This option and *compile.seqmap.enable_output_inversion* are two different features. This option only restricts QN usage. Output inversion can happen even if this option set to *true*. Output inversion is related to output phase.

Pre-instantiated QN cells with dont-touch or size-only will still keep the QN pins even if this option set to *false*.

To determine the current value of this application option, use the *get_app_option_value -name compile.seqmap.disable_qn_usage* command.

For a list of all compile application options and their current values, use the *report_app_options compile.** command.

See Also

- [compile_fusion](#)

compile.seqmap.dw_sync_support

Enable support for complex DW synchronizer cell mapping in *Fusion Compiler*.

Data Types

Boolean

Default true

Description

This application option controls the feature for enabling DW sync mapping. When this command is set to *true*, it will allow tool to group DW single stage synchronizers to complex multi-stage DW_sync cells(2-4 stages). When this command is set to *false*, it has no impact on compile flow.

To determine the current value of this application option, use the *get_app_option_value -name compile.seqmap.dw_sync_support* command.

For a list of all compile application options and their current values, use the *report_app_options compile.** command.

See Also

- [compile_fusion](#)

compile.seqmap.enable_output_inversion

Controls whether the *compile_fusion* command allows sequential elements to have their output phase inverted.

Data Types

Boolean

Default true

Description

This application option controls whether the *compile_fusion* command allows sequential elements to have their output phase inverted.

When this application option is *true* (the default), the *compile_fusion* command allows the mapping of the sequential elements in the design to sequential library cells whose output phase is inverted. This can help improve QoR. It is also useful when mapping sequential cells to a target library whose sequential cells have only one type of asynchronous inputs (either set or reset). In this case, the only way to match a sequential cell that uses the missing asynchronous input is to use a library cell with the other type of asynchronous input and to invert the output of that cell.

When using sequential output inversion, you must use the SVF file to verify the functionality of the design using Formality.

To determine the current value of this application option, use the *get_app_option_value -name compile.seqmap.enable_output_inversion* command.

For a list of all compile application options and their current values, use the *report_app_options compile.** command.

See Also

- [compile_fusion](#)

compile.seqmap.enable_register_merging

Controls whether the *compile_fusion* command tries to identify and merge equal and opposite registers.

c

Data Types

Boolean

Default true

Description

This application option controls whether the *compile_fusion* command tries to identify and merge equal and opposite registers.

When this application option is *true* (the default), the *compile_fusion* command tries to identify and merge registers in the current design that are equal or opposite, which improves the area of the design.

Register merging can also be controlled on a design-by-design or cell-by-cell basis by using the *set_register_merging* command. *set_register_merging* can be used to selectively disable register merging on a few hierarchies/registers. However, when *compile.seqmap.enable_register_merging* app option is set to false, register merging is completely disabled and *set_register_merging* command can not be used to selectively enable it on a few hierarchies/registers.

To determine the current value of this application option, use the *get_app_option_value -name compile.seqmap.enable_register_merging* command.

For a list of all compile application options and their current values, use the *report_app_options compile.** command.

See Also

- [compile_fusion](#)
- [set_register_merging](#)

compile.seqmap.enable_register_merging_across_bus

Controls whether the *compile_fusion* command should merge registers across bus or not.

Data Types

Boolean

Default true

Description

This application option controls whether the *compile_fusion* command should merge registers across bus or not.

c

When this application option is *false*, the *compile_fusion* command does not merge registers that belong to different buses. Also, it does not merge multi-bit registers with single-bit registers.

See Also

- [compile_fusion](#)
- [set_register_merging](#)

compile.seqmap.exact_map

Provide a similar solution for *Fusion Compiler* to mimic the *exact_map* behavior in *Design Compiler*

Data Types

Boolean

Default false

Description

This application option controls whether the *compile_fusion* command will map cell as it is. When this command is set to *true*, it specifies that sequential cells are mapped exactly as indicated in the HDL code. It does not mean the QN pin won't be used in the mapped sequential element. But it would disable sequential output inversion. When this command is set to *false*, it has no impact on compile flow.

To determine the current value of this application option, use the *get_app_option_value -name compile.seqmap.exact_map* command.

For a list of all compile application options and their current values, use the *report_app_options compile.** command.

See Also

- [compile_fusion](#)

compile.seqmap.gating_async_set_reset

App option to turn on/off the feature for gating scan chain cells.

Data Types

Boolean

Default true

c

Description

This application option controls the feature to gate scan chain cell asynchronous set/reset ports. The app option is set to *false* at the moment, but will be made OBD in the future.

The step name is *fc/initial_map/gatingAsyncSetReset* and it is triggered in *initial_place* and incremental compile stage. For now we only support in-compile DFT flow. Typical compile steps would be: *compile_fusion -from initial_map -to logical_opto insert_dft compile_fusion -from initial_place -to initial_opto*

Note that if user directly run *compile_fusion -from initial_map -to initial_opto* The step would be skipped.

User need to set gating control port in script via command *set_register_async_gating_pin -port xxx* to enable the feature.

User can use the following command to check gating status *report_async_gated_registers*

When this app option is set to *false*, it'll disable the flow step.

To determine the current value of this application option, use the *get_app_option_value -name compile.seqmap.gating_async_set_reset* command.

For a list of all compile application options and their current values, use the *report_app_options compile.** command.

See Also

- [set_register_async_gating_pin](#)
- [report_async_gated_registers](#)

compile.seqmap.honor_sync_set_reset

Helps to turn on or off the feature for the feature to honor defined synchronous set or reset signal of sequential cells at *initial_map* stage. In the case of pushing out synchronous set or reset from the sequential cells, the tool would preserve pushed out synchronous set or reset logic from being optimized.

Data Types

Boolean

Default *false*

Description

This application option controls the feature to honor the synchronous set/reset in *initial_map*. And to preserve pushed out synchronous set or reset logic if the tool needs to

c

push out synchronous set or reset from the sequential cells. The app option is set to *false* by default.

When this application option is set to *false*, it disables the feature. When this application option is set to *true*, initial_map seqmap would honor synchronous set/reset signal on sequential cells. In the case the tool needs to push out synchronous set or reset, restriction *pushed_out_set_reset_logic* will be applied on the corresponding pushed out synchronous set or reset logic input, which is persistent. As a result, these signals are preserved and are not optimized by downstream engines, which prevents x-propagation to synchronous set or reset.

To determine the current value of this application option, use the *get_app_option_value -name compile.seqmap.honor_sync_set_reset* command.

For a list of all compile application options and their current values, use the *report_app_options compile.** command.

compile.seqmap.identify_shift_registers

Controls the identification of shift registers during the *compiler_fusion* command.

Data Types

Boolean

Default true

Description

When this application option is set to *true* (the default), the tool automatically identifies shift registers in the design during the *compile_fusion* command.

When all the shift registers are identified, only the first register is mapped to a scan cell and the remaining registers are mapped to nonscan cells. This can save a significant amount of area for designs that contain many identified shift registers.

After these shift registers are identified, DFT synthesis also recognizes the identified shift registers as shift-register scan segments. However, DFT breaks these scan segments, if necessary, to respect the test setup requirements, such as maximum chain length.

Set the *compile.seqmap.identify_shift_registers* application option to *false* if you do not want the *compile_fusion* command to identify shift registers.

To determine the current value of this application option, use the *get_app_option_value -name compile.seqmap.identify_shift_registers* command.

For a list of all compile application options and their current values, use the *report_app_options compile.** command.

See Also

- [compile_fusion](#)

compile.seqmap.ignore_const_values_on_scan_pins_during_constant_propagation

Controls how the *compile_fusion* command treats registers with scan pins connected to constant nets.

Data Types

Boolean

Default false

Description

By default, constant propagation engine of *compile_fusion* does not perform constant propagation through registers whose scan enable pin is driven by active constant. As a result, such registers are not removed if found to be constant.

In some cases, user can find this to be overly strict. To relax the restriction, user can set *compile.seqmap.ignore_const_values_on_scan_pins_during_constant_propagation* to false. When set to false, *compile_fusion* will consider register for constant propagation irrespective of the constant value on scan enable pin.

Note that *compile_fusion* will not propagate constants through registers whose scan control pins are connected to signal drivers. This application option takes effect on only those registers whose scan enable pin is connected to constant signals.

To determine the current value of this application option, use the *get_app_option_value -name compile.seqmap.ignore_const_values_on_scan_pins_during_constant_propagation* command.

For a list of all compile application options and their current values, use the *report_app_options compile.** command.

See Also

- [compile_fusion](#)

compile.seqmap.max_shift_register_length

Controls the maximum length of shift registers during the *compile_fusion* command.

c

Data Types

Integer

Default -1

Description

The default setting of this application option is *-1* which means the maximum length of all shift registers is unlimited.

The value of this application option can be set to any integer that is equal or larger than 2 and it specifies the maximum length of all shift registers.

To set the current value of this application option, use the *set_app_option -name compile.seqmap.max_shift_register_length -value 4* command.

To determine the current value of this application option, use the *get_app_option_value -name compile.seqmap.max_shift_register_length* command.

For a list of all compile application options and their current values, use the *report_app_options compile.** command.

See Also

- [compile_fusion](#)

compile.seqmap.min_shift_register_length

Controls the minimum length of shift registers during the *compiler_fusion* command.

Data Types

Integer

Default 2

Description

The default setting of this application option is 2 which means the minimum length of all the shift registers is 2.

The value of this application option can be set to any integer that is equal or larger than 2 and it specifies the minimum length of all shift registers.

To set the current value of this application option to 4, use the *set_app_option -name compile.seqmap.min_shift_register_length -value 4* command.

To determine the current value of this application option, use the *get_app_option_value -name compile.seqmap.min_shift_register_length* command.

c

For a list of all compile application options and their current values, use the *report_app_options compile.** command.

See Also

- [compile_fusion](#)

compile.seqmap.no_inv_push_for_user_size_only

Swap cell with no phase inv for those with user size_only attribute.

Data Types

Boolean

Default true

Description

This application option controls whether the *compile_fusion* command will swap out pre-instantiated user size_only cell with no phase inv. When this command is set to *true*, it means swapping won't allow inversion on input/output side. When this command is set to *false*, it means swapping will allow inversion on input/output side. This switch will depend on *compile.seqmap.swap_ignoring_user_size_only*. It's working only when the above switch is true.

To determine the current value of this application option, use the *get_app_option_value -name compile.seqmap.no_inv_push_for_user_size_only* command.

For a list of all compile application options and their current values, use the *report_app_options compile.** command.

See Also

- [compile_fusion](#)

compile.seqmap.prefer_registers_with_multibit_equivalent

Directs the *compile_fusion* command to prioritize sequential singlebit elements to be mapped to singlebit library cells that have at least one multibit equivalent.

Data Types

Boolean

Default false

c

Description

This application option directs the *compile_fusion* command to prioritize sequential singlebit elements to be mapped to singlebit library cells that have at least one multibit equivalent.

When this application option is *false* (the default), the *compile_fusion* command maps the sequential singlebit elements in the design to singlebit library cells without regard to whether they have a multibit equivalent or not.

When this application option is set to *true*, the *compile_fusion* command will prefer to map the sequential singlebit elements in the design to singlebit library cells that have at least one multibit equivalent. If the tool fails to map to such singlebit library cells, it falls back to default behavior.

To determine the current value of this application option, use the *get_app_option_value -name compile.seqmap.prefer_registers_with_multibit_equivalent* command.

For a list of all compile application options and their current values, use the *report_app_options compile.** command.

See Also

- [compile_fusion](#)

compile.seqmap.preserve_rtl_clock_phase_strictly

Controls whether the *compile* command allows sequential elements (SEQGENs) to have their clock phase inverted when it is necessary to map them to sequential library cells.

Data Types

Boolean

Default false

Description

This variable is meant for compliance to certain RTL coding policies. It is sometimes desirable to have latch-enable and clock nets free from any additional logic, such as buffers and inverters, at least during the initial stages of compilation. This typically requires some care in coding the initial RTL. When the value of this variable is *true*, the *compile* command will take additional actions when mapping level-sensitive latches and flip-flops, to insure that no inverters appear on latch enable or clock lines.

To determine the current value of this application option, use the *get_app_option_value -name compile.seqmap.preserve_clock_phase_strictly* command. For a list of all compile application options and their current values, use the *report_app_options compile.** command.

c

See Also

- [compile](#)

compile.seqmap.preserve_rtl_clock_phase_strictly_for_retention

Controls whether the *compile* command allows retention type sequential elements (RETGENs) to have their clock phase inverted when it is necessary to map them to sequential library cells.

Data Types

Boolean

Default false

Description

This variable is meant for compliance to certain RTL coding policies. It is sometimes desirable to have clock nets on retention cells free from any additional logic, such as buffers and inverters, at least during the initial stages of compilation. This typically requires some care in coding the initial RTL. When the value of this variable is *true*, the *compile* command will take additional actions when mapping retention cells, to insure that no inverters appear on clock lines of retention cells.

To determine the current value of this application option, use the *get_app_option_value -name compile.seqmap.preserve_clock_phase_strictly_for_retention* command. For a list of all compile application options and their current values, use the *report_app_options compile.** command.

See Also

- [compile](#)

compile.seqmap.print_cross_probing_info_for_removed_registers

Controls whether to print the cross probing info for the removed constant/unloaded/merged registers.

Data Types

Boolean

Default false

c

Description

This application option controls whether to print the cross probing info for the removed registers log messages and in the reporting of *report_transformed_registers* command.

See Also

- [set_app_options](#)
- [report_transformed_registers](#)

compile.seqmap.register_replication_max_fanout_constraint_mode

Enables manual replication engine to create additional copies dictated by max fan-out constraint to achieve better QOR.

Data Types

string

Default strict

Description

When -max_fanout option of a set_register_replication is used, Fusion Compiler calculates number of copies by taking a division of number of load pins and the max_fanout constraint. In some cases, the load pins are very heavily clustered in a geometric area. As a result, one large group of pins that violates the max_fanout constraint is created. When pins from one large cluster are removed to meet max_fanout and distributed across other clusters, replicated register can end-up driving loads that are far apart. This results in heavy buffering of nets, impacting QOR.

To overcome this issue, this app-option is provided. When set to *relaxed*, the clusters that violate max_fanout constraints are sub-divided until each one meets max_fanout constraint. As a side outcome, Fusion Compiler creates more number of copies. However, a dedicated copy is available to drive a cluster with closely placed pins. This improves QOR.

Valid values of this option are: *strict*(default), and *relaxed*.

See Also

- [set_register_replication](#)

compile.seqmap.register_replication_naming_style

Allows user to define naming of manually replicated registers during *compile_fusion*.

Data Types

string

Default "%s_rep%d"

Description

This application option controls convention to name manually replicated registers by *compile_fusion* command. By default, new registers are named by using their base name and post fixing them by `_rep` followed by unique number. For example, two copies of register with name "reg" will be named "reg_rep1" and "reg_rep2". You can override default naming style by setting this app-option. For example, setting *compile.seqmap.register_replication_naming_style* to "%s_replicated%d" will create copies with name `reg_replicated1` and `reg_replicated2`. Please note that, naming style must have %s and %d identifiers.

To determine the current value of this application option, use the *get_app_option_value -name compile.seqmap.register_replication_naming_style* command.

For a list of all compile application options and their current values, use the *report_app_options compile.** command.

See Also

- [compile_fusion](#)
- [set_register_replication](#)

compile.seqmap.register_replication_placement_effort

Specifies the effort level for placement of registers and combinatorial logic replicated during replication transformations.

Data Types

string

Default low

Description

This application option specifies the effort spent in placement of registers and combinatorial logic generated during replication transforms. Valid values are: *high*, and *low*(default).

c

low: This is a default placement effort. At low effort, replication engine places registers and combinatorial logic replicated for a single `set_register_replication` directive. If there are multiple `set_register_replication` directives, then placement is done multiple times, one for each `set_register_replication` directive.

high: At high effort, replication engine collects all replicated registers and combinatorial logic and considers their placement simultaneously. This placement strategy is global in scope and considers locations of logic gates duplicated in all `set_register_replication` directives. The use of *high* effort is more useful when replication engine is called late in the compile flow, in features such as multi-source clock tree synthesis (MSCTS). MSCTS engine replicates clock gates along with registers driving enable signals. The placement of registers driving enable signals need to be done with high effort to get a better QOR. MSCTS is one such engine where Use of *high* effort is recommended.

To provide better appreciation of *high* effort, consider a pipeline of registers, R3, R2, R1, R0. Lets consider R3, R2, R1, R0 is replicated as Rx3, Rx2, Rx1, Rx0 using `set_register_replication` commands. At *low* effort, when R0 is replicated, (R0, Rx0) will be immediately placed assuming their driver register, R1 as "unmovable" cell. A process will continue. However, in *high* effort mode, placement is done after replication of all registers. Placement of R3, R2, R1, R0, Rx3, Rx2, Rx1, Rx0 will be done simultaneously. This global view results in a better placement and translates to better QOR.

See Also

- [set_register_replication](#)

compile.seqmap.register_replication_placement_location

Adjusts the placement of replicated logic in the center of their loads pins.

Data Types

string

Default `center_of_loads_and_drivers`

Description

This application option can be used to place replicated logic in the center of load pins. This option is useful when replicated logic drives large number of loads and logic needs to be placed in the center of load pins. Valid values are: *center_of_loads_and_drivers(default)*, and *center_of_loads*.

center_of_loads_and_drivers: This is a default placement location. The replicated logic is placed in the center of load pins as well as driver pins. Replication engine chooses a location half way to centers of driver pins and load pins. Such location is optimal and ensures slack on D side is not compromised due to placement.

c

center_of_loads: When this option value is specified, The replicated logic is placed in the center of load pins ignoring locations of driver pins. This option is useful when replicated logic drives large number of fan-outs and placing replicated logic in the center of load pins gives better QOR. Please use this setting carefully as D side slack can degrade.

See Also

- [set_register_replication](#)

compile.seqmap.register_replication_simu_mismatch_prevention

Prevents register replication of some critical registers which can lead to simulation mismatch.

Data Types

bool

Default false

Description

In some special cases, replication of some no-reset registers can cause an endpoint to take up a non-deterministic state at power-up, which was initially having a deterministic state, eventually leading to simulation failure. This application option controls whether to prevent register replication of such registers. Such situation arises only in some special cases, when there is an interaction of reset and noreset registers and after replication the no-reset registers sit at some specific position. Hence, the app-option is false by default.

When this application option is true, register replication will be prevented for some no-reset registers if it will cause any output from having a deterministic state at power-up to having a non-deterministic state. If any such register is prevented, it is reported with *SQM-4052* message.

When this application option is false, no register is prevented from replication for simulation mismatch reasons.

Examples

The following command sets simulation mismatch prevention to true.

```
prompt> set_app_options \ -name  
compile.seqmap.register_replication_simu_mismatch_prevention -value true
```


c

See Also

- [compile_fusion](#)
- [get_app_option_value](#)
- [set_app_options](#)

compile.seqmap.remove_constant_registers

Controls whether the *compile_fusion* command allows sequential elements to be removed when they can be implemented as a constant.

Data Types

Boolean

Default true

Description

This application option controls whether the *compile_fusion* command allows sequential elements to be removed when they can be implemented as a constant.

When this application option is *true* (the default), the *compile_fusion* command allows the removal sequential elements in the design when the tool determines that the cell's output is always a constant. This can improve the area of the design by saving the area of the cell itself, and allowing simplification of the logic in the fanout of the cell. A register can be implemented using constant 0, constant 1, or X.

The number of registers removed is reported using the SQM-3100 message. To report the complete set of registers removed as constants, use the *report_transformed_registers* command.

When using constant register removal, you must use the SVF file to verify the functionality of the design using Formality.

To determine the current value of this application option, use the *get_app_option_value -name compile.seqmap.remove_constant_registers* command.

For a list of all compile application options and their current values, use the *report_app_options compile.** command.

See Also

- [compile_fusion](#)
- [report_transformed_registers](#)

compile.seqmap.remove_unloaded_registers

Controls whether the *compile_fusion* command allows unloaded sequential elements to be removed.

Data Types

Boolean

Default true

Description

This application option controls whether the *compile_fusion* command allows unloaded sequential elements to be removed.

When this application option is *true* (the default), the *compile_fusion* command allows the removal sequential elements in the design when the tool determines that the cell's output is not required to implement the design. This can improve the area of the design.

Registers that are used in the input design can become unloaded during optimizations on the surrounding logic in the design.

The number of registers removed is reported using the OPT-3101 message. To report the complete set of registers removed as unloaded, use the *report_transformed_registers* command.

To determine the current value of this application option, use the *get_app_option_value -name compile.seqmap.remove_unloaded_registers* command.

For a list of all compile application options and their current values, use the *report_app_options compile.** command.

See Also

- [compile_fusion](#)
- [report_transformed_registers](#)

compile.seqmap.replicate_registers_without_asynchronous_logic

Controls whether the *compile_fusion* command allows replication of registers without asynchronous pins.

Data Types

Boolean

Default true

c

Description

This application option controls whether the *compile_fusion* command is allowed to replicate registers without asynchronous set or reset pins. By default *compile_fusion* will allow replication of registers without asynchronous set or reset pins. If set to false, *compile_fusion* will not replicate registers unless it has either asynchronous set or reset pin.

To determine the current value of this application option, use the *get_app_option_value -name compile.seqmap.replicate_registers_without_asynchronous_logic* command.

For a list of all compile application options and their current values, use the *report_app_options compile.** command.

See Also

- [compile_fusion](#)
- [set_register_replication](#)

compile.seqmap.replicate_size_only_registers

Controls whether the *compile_fusion* command allows replication of size only registers.

Data Types

Boolean

Default false

Description

This application option controls whether the *compile_fusion* command is allowed to replicate registers that are marked size only. By default *compile_fusion* will not allow replication of size only registers. If set to true, *compile_fusion* will replicate size only registers.

The order in which this app-option, *set_register_replication* and *set_size_only* commands are specified decides the outcome of replication.

If this app-option is set to true after *set_register_replication* command, *set_register_replication* command will reject replication of size only registers.

If this app-option is set to false (which is a default), but register is marked size only after *set_register_replication*, then *compile_fusion* will reject replication of the register.

To determine the current value of this application option, use the *get_app_option_value -name compile.seqmap.replicate_size_only_registers* command.

c

For a list of all compile application options and their current values, use the *report_app_options compile.** command.

See Also

- [compile_fusion](#)
- [set_register_replication](#)
- [set_size_only](#)

compile.seqmap.scan

Controls whether the *compile_fusion* command considers the impact of scan insertion on mission-mode constraints during optimization.

Data Types

Boolean

Default true

Description

This application option controls whether the *compile_fusion* command considers the impact of scan insertion on mission-mode constraints during optimization.

When this application option is *true* (the default), the *compile_fusion* command replaces sequential elements with equivalent scan-enabled elements if the logic library supports scan elements. The command might degenerate sequential cells to find a matching scan cell. Some scan-replaced sequential cells might be converted to nonscan cells later in the test synthesis process because of test design rule violations or explicit user specifications.

To determine the current value of this application option, use the *get_app_option_value -name compile.seqmap.scan* command.

For a list of all compile application options and their current values, use the *report_app_options compile.** command.

See Also

- [compile_fusion](#)

compile.seqmap.swap_ignoring_user_size_only

Ignore user *size_only* attribute during swapping

c

Data Types

Boolean

Default true

Description

This application option controls whether the *compile_fusion* command will swap out pre-instantiated cell with user size_only attribute. When this command is set to *true*, it means swapping is allowed on instance with user size_only attribute. When this command is set to *false*, pre-instantiated instances with user size_only attribute will not be swapped out.

To determine the current value of this application option, use the *get_app_option_value -name compile.seqmap.swap_ignoring_user_size_only* command.

For a list of all compile application options and their current values, use the *report_app_options compile.** command.

See Also

- [compile_fusion](#)

constraints_compare_full_behavior

Specifies whether case analysis and disabled timing arcs are compared using the behavioral checker in the *compare_constraints* command, in addition to the default behavior.

Data Types

Boolean

Default false

Description

This variable is available only if you invoke the *rtl_shell*.

When *false* (the default), *compare_constraints* command compares case analysis and disabled timing arcs at the definition points. If the definition points are unmapped using the name mapping file (or *define_name_maps* command), it will throw violations for missing or mismatching case / disabled arcs. Clocks and exceptions are compared using the behavioral checker.

When the *constraints_compare_full_behavior* variable is *true*, even if the case analysis and disabled arcs that have non-mapped definition points, but show the same behavior at timing path endpoint both scenarios / designs, will not be shown as violations. Basically,

c

case/disable arcs that are not defined at the same definition points, but have the same impact at the timing path endpoints, will not be flagged as violations.

To determine the current value of this variable, use *printvar constraints_compare_full_behavior*.

create_trunk.drc.enable

In the create_trunk_* family of commands use eDRC engine to calculate safe placement spacing for generated geometry.

Data Types

Boolean

Default False

Description

In the create_trunk_* family of commands (create_trunk_pin_to_pin, create_trunk_topology, etc.) use eDRC engine to calculate safe spacing for generated geometry.

When this app option is enabled, create_trunk_* commands consider more drc rules to check at the expense of more heavy computations.

This app option is supposed to be used together with -avoid_obstructions of create_trunk_* command (see e.g., create_trunk_pin_to_pin). When option -avoid_obstructions is set to false, app option create_trunk.drc.enable does not have any effect.

See Also

- [create_trunk_pin_to_pin](#)
- [create_trunk_topology](#)
- [create_trunk](#)
- [create_trunk_pin_to_trunk](#)
- [create_trunk_shared_track](#)

cstrmgr.allow_gobble_for_dont_touch_network

Determines whether or not unloaded or constant objects on the dont_touch network can be gobbled.

c

Data Types

boolean

Default *true*

Description

When the *set_dont_touch_network* is used to restrict a network, the objects on the network are restricted for limited optimizations. By default, if such objects are unloaded or constant, they could be deleted. Setting the application option value to *false* would prevent such objects from being deleted.

To determine the current value of the application option, use *get_app_option_value -name cstrmgr.allow_no_gobble_for_dont_touch_network*.

See Also

- [set_dont_touch_network](#)

cts.balance_groups.balance_by_source_latency

This option controls whether or not the *balance_clock_groups* program will achieve the purpose by adjusting source latency of clocks.

Data Types

Boolean

Default *false*

Description

When *true*, the *balance_clock_groups* command will not insert buffers or inverters. Instead, it will set source latency of clocks to achieve the balance.

When this application option is *false*, it will insert buffers or inverters to meet the delay for balancing.

See Also

- [balance_clock_groups](#)
- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)

cts.balance_groups.honor_source_latency

Controls whether the *balance_clock_groups* program will honor source latency of clocks.

Data Types

Boolean

Default false

Description

When *true*, the *balance_clock_groups* command will honor the source latency of clocks. Therefore, a clock delay will include its source latency and network latency.

When this application option is *false*, the *balance_clock_groups* command will only consider network latency.

See Also

- [balance_clock_groups](#)
- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)

cts.buffering.enable_arnoldi_analysis

This application option controls the *Gate-By-Gate Clock Tree Synthesis* and *DRC Fixing Beyond Exceptions* steps use the Arnoldi delay model instead of the Elmore delay model to synthesize the clock tree.

Data Types

Boolean

Default false

Description

When this option is set to true, the Gate-By-Gate Clock Tree Synthesis and DRC Fixing Beyond Exceptions steps will use Arnoldi delay model to synthesize the clock tree. This improves the accuracy of delay and transition estimation for the clock tree synthesis.

The max transition violation is expected to be reduced after Gate-By-Gate Clock Tree Synthesis when the app option is enabled.

Allowed values are "true" and "false".

See Also

- [synthesize_clock_trees](#)
- [clock_opt](#)

cts.buffering.levelize_buffering

This application option controls which buffering algorithm is used during gate-by-gate synthesis.

Data Types

Integer

Default 0

Description

This option will change the algorithm used for buffering the clock tree during gate-by-gate synthesis.

By default, tool synthesizes the clock tree bottom up based on an approach similar to data buffering.

When this option is set to a value larger than 0, the algorithm tries to buffer in a levelized fashion by trying to add a repeater as high as possible behind intermediate points added to structure the sinks. By doing so the buffering algorithm should add less buffer levels and therefore reduce power, area and number of repeaters while maintaining latency and skew.

Allowed values are 0 (disable) or 1 (enable).

See Also

- [synthesize_clock_trees](#)
- [get_app_option_value](#)
- [set_app_options](#)

cts.buffering.reduce_clock_level_effort

This application option specifies the effort of reducing the clock repeater level during gate-by-gate synthesis.

c

Data Types

Enum

Default "off"

Description

This option will change the effort of reducing the clock repeater level during gate-by-gate synthesis.

By default, tool synthesize the clock tree without considering the repeater levels.

When this option is set, high fanout clock tree synthesis will estimate the level criticality and try to reduce the clock levels.

Allowed values are "off", "high". Set the values to "high" for enabling the feature.

See Also

- [synthesize_clock_trees](#)
- [set_clock_tree_options](#)

cts.common.advanced_lp_solvers

Controls the type of LP solver used in different parts of CTS. This currently includes MOO CCD and ICG relocation engines.

Data Types

String

Default default

Description

Allowed values are "default", "best_cpu" and "best_gpu".

When the option is set to "default", the default LP solver library is used. When the option is set to "best_cpu", the best non-GPU solver is used for each application. When the option is set to "best_gpu", then a GPU solver is used in ICG relocation. Values of the option other than the ones described will result in using the default LP solver everywhere.

See Also

- [synthesize_clock_trees](#)

cts.common.auto_exception_disable_self_arc

Enable auto exception to disable self arc on pins with through arc.

Data Types

Boolean

Default false

Description

When *cts.common.auto_exception_disable_self_arc* application option is *true*, auto exception step of CTS disables self arc on pins with through arc. These pins with self arc disabled are then considered as through pins only.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)
- [synthesize_clock_trees](#)
- [set_disable_timing](#)

cts.common.auto_exception_macro_balance_point

Controls whether the *synthesize_clock_trees* command sets clock tree balance points for macros with large internal skew when the *cts.common.enable_auto_exceptions* application option is *true*.

Data Types

Boolean

Default true

Description

Controls whether the *synthesize_clock_trees* command sets balance points for macros with large internal skew when the *cts.common.enable_auto_exceptions* application option is *true*. This balance point is created with an offset equal to the largest latency inside the macro.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)
- [synthesize_clock_trees](#)

cts.common.auto_exception_output_ports_balance_point

Enable auto exception to set output ports on clock network as balance points.

Data Types

Boolean

Default false

Description

When *cts.common.auto_exception_output_ports_balance_point* application option is *true*, auto exception step of CTS sets output ports on clock network as balance point so that these output ports will be considered as sinks and will be automatically balanced.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)
- [synthesize_clock_trees](#)

cts.common.default_max_transition

Max transition constraint value.

Data Types

float

Default 0.5

c

Description

Transition constraint to use when CTS doesn't find any lib pin transition constraint, clock specific transition constraint or global transition constraint.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)
- [synthesize_clock_trees](#)

cts.common.derive_icg_and_register_references

Controls whether the clock initialization phase of the *synthesize_clock_trees* and *clock_opt* commands sets the *included_purposes* attribute to *cts* for integrated clock gating cells (ICGs) and registers.

Data Types

Boolean

Default false

Description

When this application option is set to *true*, the clock initialization phase of the *synthesize_clock_trees* and *clock_opt* commands sets the *included_purposes* attribute to *cts* for ICGs and registers whose references have an included purpose of *optimization*.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)
- [set_lib_cell_purpose](#)
- [synthesize_clock_trees](#)

cts.common.directory_for_cts_generated_files

Set a directory for new files created by clock tree synthesis tools.

Data Types

string

Default ""

Description

This app option sets a directory for new files created by clock tree synthesis tools. The CTS programs will automatically create this directory if it does not exist. By default CTS program uses the current directory to store new files.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)
- [synthesize_clock_trees](#)

cts.common.enable_auto_exceptions

Controls whether the auto_exceptions feature is enabled for the clock tree synthesis tools.

Data Types

Boolean

Default true

Description

This option is deprecated in R-2020.09 and scheduled for removal in a future release.

This option controls whether the auto_exceptions feature is enabled for the clock synthesis tools. The auto_exceptions feature will derive exceptions such as ignore point based on analysis covering scenarios, such as structurally impossible-to-balance, and internal pins in macro cells.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)
- [synthesize_clock_trees](#)

cts.common.enable_auto_skew_target_for_local_skew

Controls whether the `auto_skew_target` feature is enabled when local skew optimization is turned on for clock tree optimization.

Data Types

Boolean

Default `true`

Description

This option controls whether the `auto_skew_target` feature is enabled when local skew optimization is also enabled (i.e. when `cts.optimize.enable_local_skew` is set to `true`).

When this option is enabled along with local skew optimization, the tool will analyze the timing information as well as the clock frequencies to arrive at a global skew target such that the overall clock area is minimized without impacting the timing metrics at the end of `synthesize_clock_trees` command.

If the local skew optimization is disabled, then this feature is also disabled.

Note that when this feature is enabled, user specified `target_skew`, if any, will be overridden.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)
- [synthesize_clock_trees](#)

cts.common.enable_dirty_design_mode

Controls whether the dirty_design_mode feature is enabled for the clock tree synthesis tools.

Data Types

Boolean

Default false

Description

This option controls whether the dirty_design_mode feature is enabled for the clock synthesis tools. When the dirty_design_mode is enabled, the CTS could override the user settings, such as max transitions, max capacitances, cell spacing rules, and clock routing rules.

Once set to true, the following constraint may be ignored or set:

1. Removes max_netlength constraint if it is too tight (less than 50um).
2. Ignores clock cell spacing rules if the spacing requirement setting is too high (x spacing is more than 3 times of site height, y spacing is more than 20 times of site height).
3. Ignores NDR if width and spacing requirement settings are too high (width + space requirement is more than 10 times of default width + default spacing).
4. Ignores max_transitions or max_capacitance if they are too small (compared against internally calibrated values).
5. Treats dont_touch flag on nets as soft constraint if there is a large transition or net delay violation on the nets.
6. Treats dont_touch_network as soft constraint if it is causing large skew.
7. Sets auto_exceptions on ETM internal sinks as ignore to prevent impossible-to-fix large skew.
8. Disables legalizer's physical design check (PDC) rules.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)
- [synthesize_clock_trees](#)

cts.common.enable_low_power

Controls whether the low-power improvement techniques are enabled during clock tree synthesis.

c

Data Types

Boolean

Default true

Description

This option controls whether the low-power improvement techniques are enabled during clock tree synthesis.

By default (*true*), clustering and re-clustering are enabled during clock tree synthesis to reduce the wire length and improve the clock tree power.

To disable these low-power improvement techniques, set this option to *false*.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)
- [synthesize_clock_trees](#)

cts.common.ignore_drc_only_scenario

This setting controls the clock tree synthesis tool (*synthesize_clock_trees* command) to work on a DRC only scenario.

Data Types

Boolean

Default false

Description

By default the clock tree synthesis tool will work on a scenario if one of the following 4 analysis types is turned on, which is setup, hold, max_transition, or max_capacitance. If this option is set to true, the clock synthesis tools will skip a DRC only scenario, which is a scenario with only max_transistion or man_capacitance on.

See Also

- [synthesize_clock_trees](#)

cts.common.max_fanout

Fanout constraint for clock tree synthesis

Data Types

integer

Default 1000000

Description

This option sets a fanout constraint for clock tree synthesis in the *synthesize_clock_trees* command. Clock tree synthesis will ensure that clock tree cells do not drive more than max_fanout cells.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)
- [synthesize_clock_trees](#)

cts.common.max_net_length

Sets the maximum net length constraint for the clock tree synthesis tools.

Data Types

float

Default 1.0e+30

Description

This option sets the maximum net length constraint for the clock synthesis tools. Here the net length is measured from the net driver pin to its farthest load pin.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)

c

- [set_app_options](#)
- [synthesize_clock_trees](#)

cts.common.port_punch_prefix_original_port_name

Controls whether original port name will be prefixed with the newly punched port. Also stores the mapping of original to punched port in SVF file.

Data Types

Boolean

Default false

Description

If CTS/abuf punches port during CTS/CTO then original port name will be added as prefix along with cts/abuf in the newly added port. Also SVF file will be dumped at the end of CTS containing mapping of original to punched port.

See Also

- [synthesize_clock_trees](#)

cts.common.power_aware_pruning

Enable pruning of power inefficient libcells during CTS/CTO

Data Types

Boolean

Default false

Description

This option enables the pruning of power inefficient libcells during the clock tree synthesis. The pruning algorithm sorts the libcells based on their sum of internal and leakage power and removes the power inefficient ones.

See Also

- [get_app_option_value](#)
- [get_app_options](#)

- [report_app_options](#)
- [set_app_options](#)

cts.common.preserve_bus_ports

Controls whether clock tree synthesis and optimization will preserve hierarchical bus ports.

Data Types

Boolean

Default false

Description

When this option is turned on, clock tree synthesis and optimization will preserve hierarchical bus ports.

See Also

- [synthesize_clock_trees](#)

cts.common.preserve_hierarchical_ports

Controls whether original hierarchical ports preservation is enabled for the clock tree synthesis tools.

Data Types

Boolean

Default false

Description

This option controls whether hierarchical port are preserved from deletion during CTS/CTO.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)
- [synthesize_clock_trees](#)

cts.common.skip_cts_clocks

set clocks to skip in clock tree synthesis tools.

Data Types

String

Default empty string

Description

This application option specifies clocks to skip in clock tree synthesis tools. When skipped clocks do not want to be updated latency in flow, use the following option to exclude them for *compute_clock_latency* / *update_clock_latency* *set_latency_adjustment_options* *-exclude_clocks* [*get_clocks* *skipped_clocks*]

See Also

- [synthesize_clock_trees](#)
- [clock_opt](#)
- [compute_clock_latency](#)

cts.common.unlimit_delay_insertion_stages

Unlimit expected repeater levels for CTS delay insertion.

Data Types

Boolean

Default false

Description

This option removes the limit on the number of expected repeater levels required to achieve a target delay with repeater chain insertion within CTS engines.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)

cts.common.user_instance_name_prefix

Set a prefix for new cell names created by clock tree synthesis tools.

Data Types

string

Default ""

Description

This app option sets a prefix for new cell names created by clock tree synthesis tools. The default value of the prefix is a NULL string.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)
- [synthesize_clock_trees](#)

cts.common.verbose

Controls the verbosity level of clock tree synthesis tools.

Data Types

integer

Default 1

Description

This option sets the verbosity level for the clock synthesis log file. The verbosity level can be set to 0 or 1, where 0 is no debugging information and 1 is more debugging information.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)

- [set_app_options](#)
- [synthesize_clock_trees](#)

cts.compile.align_clustering_and_buffering

Improves the correlation between the clustering and buffering engines during high fanout clock tree synthesis in build_clock.

Data Types

Boolean

Default true

Description

When this option is set to *true*, the clustering engine inside high fanout clock tree synthesis during clock_opt build_clock will correlate better with the buffering engine.

See Also

- [synthesize_clock_trees](#)

cts.compile.balanced_topology_enhancements

Enables enhancements to the balanced topology generation engine during high fanout clock tree synthesis in build_clock.

Data Types

Boolean

Default false

Description

When this option is set to *true*, the balanced topology generation engine inside high fanout clock tree synthesis during clock_opt build_clock will be improved to find a better trade-off between clock wirelength and skew.

See Also

- [synthesize_clock_trees](#)

cts.compile.cg_timing_aware

Enable timing awareness with respect to critical clockgates during clock tree synthesis in `build_clock`.

Data Types

Boolean

Default `false`

Description

When this option is set to *true*, high fanout clock tree synthesis during `clock_opt build_clock` will be ICG timing aware and build faster subtrees for the timing critical clockgates. This assumes that the ICG enable timing information has been captured earlier in the flow.

See Also

- [synthesize_clock_trees](#)

cts.compile.enable_cell_relocation

Specifies the mechanism for relocating the existing cells in the clock tree prior to clock tree synthesis.

Data Types

enumerated

Default `"all"`

Description

This option controls how existing clock tree cells are relocated before clock tree synthesis. Possible values are *all*, *leaf_only*, *no_data_connection*, *none* and *auto*. *all* is the default.

The default value, *all*, relocates all of the cells in the clock tree, for which relocation is allowed, in order to achieve the best latency. The *leaf_only* argument relocates clock tree cells, for which relocation is allowed, to the center of the bounding box of its fanout, only if none of the sinks in the fanout has downstream phase delay. The *no_data_connection* argument allows relocation of clock cells that do not have a data connection like clock repeaters. It disallows moving cells with data connections like clock gates. The *none* argument keeps the incoming clock tree intact and prevents any relocation of the existing clock tree cells.

The *auto* argument automatically decides on one of the values *no_data_connection* or *all* based on the fact whether latency was considered during pre-CTS

c

placement. Pre-CTS latency aware placement is controlled by app option `place.coarse.clock_gate_latency_aware`. If latency was considered during pre-CTS placement, then only cells without a data connection will be relocated.

Application options are set using the `set_app_options` command, and specified globally.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)

cts.compile.enable_global_route

Specifies the mechanism for using global router in the initial stage of `synthesize_clock_trees` command.

Data Types

Boolean

Default `false`

Description

When this option is *true*, global router will be used during the initial stage of the `synthesize_clock_trees` command. By default this option is *false* and virtual router is used during initial synthesis.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)
- [synthesize_clock_trees](#)

cts.compile.enable_local_skew

Controls whether local skew is enabled during clock tree synthesis.

c

Data Types

Boolean

Default false

Description

When this option is *true*, clock tree synthesis will construct local skew friendly clock trees by considering timing interactions between the clock tree sinks. By default this option is *false* and clock tree synthesis considers only the traditional metrics like latency, skew and area. In *clock_opt*, this option will be turned on automatically when *clock_opt.flow.enable_ccd* is *true*.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)
- [synthesize_clock_trees](#)

cts.compile.fix_clock_tree_sinks

This application option controls whether clock tree sinks will be marked as fixed after *synthesize_clock_trees* command.

Data Types

Boolean

Default false

Description

When this options is *true*, clock tree sinks will be marked as fixed after *synthesize_clock_trees* command.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)

c

- [set_app_options](#)
- [synthesize_clock_trees](#)

cts.compile.global_route_aware_buffering

Specifies the mechanism for using global router aware buffering in the initial stage of the *synthesize_clock_trees* command.

Data Types

Boolean

Default false

Description

When this application option is set to *true*, global router is used for routing topology during the initial stage of the *synthesize_clock_trees* command. By default, this application option is set to *false* and virtual router is used during initial synthesis.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)
- [synthesize_clock_trees](#)

cts.compile.latency_enhancements

Enables clock latency related enhancements in the high fanout clock tree synthesis in *build_clock*.

Data Types

Integer

Default 0

Description

When this option is set to , the high fanout clock tree synthesis engine during *clock_opt build_clock* will try to improve latency by enabling critical load isolation, improved latency estimation, reduction in topology balancing detours and higher weightage for latency.

c

When this option is set to , in addition to the features stated above, the tool will also enable pruning of CUS offsets from latency bottleneck paths.

See Also

- [synthesize_clock_trees](#)

cts.compile.path_based_criticality

It is used to control slow-fast buffer chain for Gate-By-Gate synthesis with-in a driver.

Data Types

bool

Default false

Description

Within a driver, Each sink has different criticality based on its downstream delay and driver to sink delay. To save area/power use slow-fast buffer chain based on criticality of sink. Common path of critical and non-critical sink will be using fast chain. Option is off by default.

cts.compile.power_opt_mode

Specifies the mechanism for clock power improvement during clock tree synthesis.

Data Types

enumerated

Default "none"

Description

This option controls which low-power improvement techniques are enabled during clock tree synthesis. Possible values are *gate_relocation*, *low_power_targets*, *all* and *none*. *none* is the default.

When the app_option is set to *gate_relocation*, activity aware gate relocation is enabled at the end of gate by gate clock tree synthesis. When the app_option is set to *low_power_targets*, clock tree synthesis relaxes some internal constraints and builds a clock tree with less area and power. When the app_option is set to *all*, both the above features are turned on. By default, none of the above features is enabled during clock tree synthesis.

c

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)
- [synthesize_clock_trees](#)

cts.compile.power_opt_wirelength_effort

Controls the trade-off between clock power improvement and clock wirelength degradation during activity driven clock gate relocation during the *synthesize_clock_trees* command.

Data Types

enumerated

Default "low"

Description

When this option is set to *high* or *medium*, the tool will have tighter control over wirelength degradation while doing activity driven clock gate relocation during the *synthesize_clock_trees* command.

See Also

- [set_app_options](#)
- [synthesize_clock_trees](#)

cts.compile.primary_corner

This application option specifies the primary corner used in the *synthesize_clock_trees* command.

Data Types

String

Default ""

Description

The *synthesize_clock_trees* command will pick a primary corner in the compile stage. It inserts buffers to balance clock delays in all modes on this corner. By default the primary

c

corner is selected with the worst delay corner. This option will force the tool to use the specified corner as primary corner.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)
- [synthesize_clock_trees](#)

cts.compile.remove_existing_clock_trees

Controls the `synthesize_clock_trees` command to remove existing buffers or inverters in the clock trees.

Data Types

Boolean

Default `true`

Description

When this option is *true*, the `synthesize_clock_trees` command will remove existing buffers and inverters in the clock trees before synthesis. Usually this will result in better clock latencies and skews.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)
- [synthesize_clock_trees](#)

cts.compile.report_latency_bottleneck

Controls whether the `synthesize_clock_trees` command reports the Latency Bottleneck Analysis CTS Step.

c

Data Types

Boolean

Default false

Description

Controls whether the *synthesize_clock_trees* command reports the Latency Bottleneck Analysis CTS Step. This feature is on by default and a separate CTS STEP called 'Latency Bottleneck Analysis' gets printed after Gate-by-Gate synthesis which reports latency critical path for every clock, skew group and mode for the primary corner.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)
- [synthesize_clock_trees](#)

cts.compile.report_latency_bottleneck_threshold

Controls threshold value of latency jump reported at the latency bottleneck reporting stage of CTS

Data Types

Float

Default 0.5

Description

In CTS, latency bottleneck is reported at the end. For latency jump between driver and loads threshold value can be controlled through this app option. Default threshold value is 0.5ns

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)

c

- [set_app_options](#)
- [synthesize_clock_trees](#)

cts.compile.size_pre_existing_cell_to_cts_references

Controls the sizing of pre-existing cells on the clock network always with references from CTS reference list during `synthesize_clock_trees` command.

Data Types

Boolean

Default `false`

Description

When this option is true, if the reference of an pre-existing instance on the clock network is not in CTS purpose list, then the pre-existing instance will be re-sized to the equivalent one from the CTS reference list during the `synthesize_clock_trees` command. By default this option is false and pre-existing cells may not be to sized to the reference which are in CTS Reference list.

See Also

- [set_app_options](#)
- [synthesize_clock_trees](#)

cts.compile.timing_driven_sink_transition

Set tight transition constraint on timing critical sink pins before Gate-By-Gate synthesis. The constraints persist through `synthesize_clock_trees` flow and are reverted to original values before postlude.

Data Types

`bool`

Default `false`

Description

Timing critical sink CK pins are constrained with tight transition constraint. This is done before buffering and reverted before the command `aynthesize_clock_trees` exits. Option is off by default.

cts.compile.topological_ndr

Enables application of level-based nondefault routing rules (NDR) on clock nets in topological order.

Data Types

Boolean

Default false

Description

When this application option is set to *true*, the level based nondefault routing rules (NDR) applied on the clock nets are in topological order from clock root to clock sinks. A clock net driving any pin, which is not a clock sink, is applied with the specified internal NDR rule.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)
- [synthesize_clock_trees](#)
- [clock_opt](#)

cts.flow.enable_unified_latency_estimation

Controls whether the unified latency estimation features are enabled for the place_opt, compile_fusion and clock tree synthesis tools.

Data Types

integer

Default 0

c

Description

This application option controls how the unified latency estimation features are enabled in the physical implementation flow. The unified latency estimation features can combine two different subfeatures in the flow, namely:

1. forcing a unique clock delay estimation engine through the physical implementation flow.
2. balance point offset pruning at certain steps in the `compile_fusion`, `place_opt` and `synthesize_clock_trees` commands.

The option can take four string values:

0:

disabled

3:

both (1) balance point pruning and (2) unique clock delay estimation engine through the physical implementation flow are enabled.

1:

only (1) is enabled.

2:

only (2) is enabled.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)
- [synthesize_clock_trees](#)
- [compile_fusion](#)
- [place_opt](#)

cts.icg.equivalence_stop_at_user_size_only

This app option is a control to prevent merging of ICGs which have user size only set on combinational logic in the logic cone driving the ICG enable pin .

c

Data Types

Boolean

Default false

Description

ICG equivalence is established by analyzing the logic cones driving the enable pins of the clock gates. If the enable functions are logically equivalent, the ICGs can be merged. However, the equivalence checker does not stop at combination cells in the enable paths which have user size only. With this app option *true*, this equivalence analysis will stop at combinational cells that are marked as size only by the user and the ICGs will not be merged. In the presence of user size only, this app option will result in lesser ICG merging opportunities. This app option will impact atomic command *merge_clock_gates* and ICG merging called during *place_opt* and *clock_opt* flow.

See Also

- [synthesize_clock_trees](#)

cts.icg.latency_driven_cloning

Enables cloning of pre-existing gates in the clock tree to improve insertion delay in the *synthesize_clock_trees* command.

Data Types

Boolean

Default false

Description

When this option is *true*, while synthesizing the clock tree, existing gates may be cloned if they are in the latency critical path in order to improve the clock insertion delay. The feature kicks in after tap assignment in regular multisource clock tree synthesis flow and then again at the beginning of clock tree synthesis in *clock_opt* *build_clock* step.

See Also

- [set_app_options](#)
- [synthesize_clock_trees](#)

cts.icg.latency_driven_collapsing

Enables collapsing of clock gates during *final_place* to reduce path delay of clock trees.

Data Types

Boolean

Default false

Description

When this option is *true*, multi-level clock gates with large path delay will be collapsed during final_place stage. In clock trees with many multi-level clock gates this should reduce overall latency metrics.

To exclude specific clock gates from collapsing see *set_clock_gate_transformations*.

See Also

- [set_app_options](#)
- [report_clock_gating](#)
- [set_clock_gate_transformations](#)

cts.icg.merge_cross_level

Enable cross level ICG merging.

Data Types

Boolean

Default true

Description

When this option is set *true*, *merge_clock_gates* command will merge ICGs of different levels in clock tree. Level only counts ICGs, not clock buffer or inverter.

See Also

- [get_app_options](#)
- [merge_clock_gates](#)

cts.icg.timing_aware

Enable timing aware ICG relocation in build_clock.

Data Types

Boolean

c

Default false**Description**

When this option is set *true*, power driven ICG relocation and latency driven ICG relocation will be timing aware. This assumes that the ICG enable timing information has been captured earlier in the flow. The tool will ensure that the ICG enable timing is not made worse by the relocation move.

cts.multisource.cell_em_aware

Enables Cell EM aware optimization within Multisource Clock Tree Synthesis Engine

Data Types

Boolean

Default true**Description**

This application option controls Cell EM optimization for Multisource Clock Tree Synthesis Htree flow and Structural Multisource Clock Tree Synthesis Flow. By default, cellem optimization is not enabled. Turning this option ON enables MSCTS commands to calculate max toggle rate and optimize the design to meet Cell EM constraint.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)

cts.multisource.cto_toggle_rate_fanout_threshold

Control the maximum fanout limit to trigger toggle-rate aware optimization during CTO buffering in integrated-SMSCTS flow

Data Types

Integer

Default 50

c

Description

This `app_option` control the maximum limit of fanout of the net to trigger toggle-rate awareness on CTO buffering during integrated SMSCTS flow.

By Default, the option value is 50

If net fanout is equal or less than the limit, then CTO buffering will favor buffering at the nets which has lower toggle-rate, which are usually at the output of clock-gating cells. Otherwise, if the net fanout is more than the limit, then CTO buffering will not observe toggle-rate between input and output of the clock-gating cells. This allow user to control the buffering trade off between area (internal power) and dynamic power.

The scale factor to trade-off the insertion at lower toggle-rate net is controlled with `cts.multisource.cto_toggle_rate_scale`.

See Also

- [set_app_options](#)

cts.multisource.cto_toggle_rate_scale

Control the maximum fanout limit to trigger toggle-rate aware optimization during CTO buffering in integrated-SMSCTS flow

Data Types

Float

Default 3.0

Description

This `app_option` control the scaling factor of the toggle-rate at output of clock-gating cells, relative to toggle-rate at input of clock-gating cells. The nets with scaled-output-toggle-rate smaller than input-toggle-rate will be considered for buffering at output net of the clock-gating cells.

By Default, the option value is .0 This means only when the output toggle-rate is smaller than 3x than input toggle-rate of a clock-gating cell, then CTO will buffer at the output of the clock-gating cell instead of the input.

The scale factor to trade-off the insertion at lower toggle-rate net is controlled with `cts.multisource.cto_toggle_rate_scale`.

See Also

- [set_app_options](#)

cts.multisource.enable_activity_aware_relocation

Enables activity aware relocation (SAIF) during subtree synthesis

Data Types

Boolean

Default false

Description

Setting this app_option to *true* enables activity aware relocation during subtree synthesis which helps in reducing the total clock tree dynamic power.

By Default, the option is off i.e. value *false* To enable activity aware relocation during synthesize_multisource_clock_subtrees, set the value to *true*

cts.multisource.enable_clock_driver_snapping

Enables snapping of tap drivers and mesh drivers to clock mesh in *create_clock_drivers* commands.

Data Types

Boolean

Default false

Description

This option when set to *true* will enable the snapping of tap drivers and mesh drivers in *create_clock_drivers* command.

Using this option, clock drivers can be placed very close to the clock mesh routes thereby improving their pin connectivity. The clock mesh should have been created before the buffer insertion is done using *create_clock_drivers* command.

See Also

- [create_clock_drivers](#)
- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)

cts.multisource.enable_cto_honor_toggle_rate

Enables toggle-rate aware optimization during CTO buffering in integrated-SMSCTS flow

Data Types

Boolean

Default `false`

Description

Setting this app_option to *true* enables CTO buffering to observe toggle-rate specified on candidate nets for buffering, and will prefer buffer insertion on nets with low-toggle rate over nets with high toggle-rate.

By Default, the option is off i.e. value *false*. To enable toggle-rate aware during CTO buffering in integrated-SMSCTS flow, set the value to *true*.

The CTO buffering is performed near end of integrated SMSCTS flow, which is invoked using *clock_opt -from build_clock -to build_clock*. The purpose of CTO buffering is to address short-path violations by performing buffering on the short-path nets. By default, the buffering will prefer inserting buffer at most common point to save area. However, the most common point near root may have higher toggle-rate. By enabling toggle-aware CTO, the buffering will prefer insertion at nets with lower toggle-rate to save dynamic-power. Please note that this may incur higher area, which may also increase internal-power.

The scale factor to trade-off the insertion at lower toggle-rate net is controlled with *cts.multisource.cto_toggle_rate_scale*.

To control the fanout limit of when CTO toggle-rate will be applied, please refer to *cts.multisource.cto_toggle_rate_fanout_threshold*.

See Also

- [set_app_options](#)

cts.multisource.enable_full_mv_support

Enables code to fully support multi voltage (MV) constraints and requirements in MS-CTS commands *create_clock_drivers*, *synthesize_multisource_global_clock_trees*, and *synthesize_multisource_clock_taps*.

Data Types

Boolean

Default `false`

c

Description

This option when set to *true* will enable special code in MS-CTS commands to support designs with MV setup defined through UPF. It should guarantee that a MV clean input will be MV clean at the end of MS-CTS commands. A clean MV setup can be checked with command *check_mv_design*. When set to *false* (the default) the commands may not or only partially be able to consider MV related constraints and requirements. This can lead to invalid designs with various kinds of MV and UPF related warnings and errors.

See Also

- [create_clock_drivers](#)
- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)
- [synthesize_multisource_clock_taps](#)
- [synthesize_multisource_global_clock_trees](#)

cts.multisource.enable_global_stem_path_skew_balancing

Enables skew balancing during global stem path building.

Data Types

Boolean

Default *false*

Description

Setting this app_option to *true* directs global tree synthesis to balance the skew among taps of different sections.

By Default, the option is off i.e. value *false* To enable skew balancing among taps of different sections, set the value to *true*

See Also

- [set_regular_multisource_clock_tree_options](#)
- [synthesize_regular_multisource_clock_trees](#)
- [set_app_options](#)

cts.multisource.enable_levelized_buffering

Controls whether to enable levelized buffering during clock tree synthesis.

Data Types

Boolean

Default false

Description

This option enables or disables the use of levelized buffering when building global clock trees. When enabled, this feature organizes buffers in distinct levels throughout the clock tree, which can improve timing predictability and reduce skew.

See Also

- [set_regular_multisource_clock_tree_options](#)
- [synthesize_regular_multisource_clock_trees](#)
- [set_irregular_multisource_clock_tree_options](#)
- [synthesize_irregular_multisource_clock_trees](#)
- [set_app_options](#)

cts.multisource.enable_mlph_flow

Enables Multi-Level Physical Hierarchy Support in *create_clock_drivers* and *synthesize_multisource_global_clock_trees* commands.

Data Types

Boolean

Default false

Description

This option when set to *true* will enable the Multi-Level Physical Hierarchy Support in *create_clock_drivers* command as well as *synthesize_multisource_global_clock_trees* command. Using this flow, clock drivers can be inserted inside different physical hierarchy blocks from top level. The H-tree routing can also be done from the top level. The physical hierarchies in which the buffers are to be inserted and the H-tree routing has to be done must have a fixed placement status and must be editable.

c

See Also

- [create_clock_drivers](#)
- [synthesize_multisource_global_clock_trees](#)
- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)

cts.multisource.enable_multi_corner_support

Enables multi-corner support during subtree synthesis

Data Types

Boolean

Default `false`

Description

Setting this app_option to *true* directs subtree synthesis to optimize clock QoR targets for all the active corners for the given clock mode.

By Default, the option is off i.e. value *false* To enable multi-corner support during subtree optimization set the value to *true*

Structural multisource subtree synthesis by default performs optimizations on the primary corner of the given clock mode to meet DRC and improve max/min latency and global skew. When multi-corner support is enabled, DRC and latency optimization happens across all the active corners.

if true, incoming balance points *existing on only one active scenario* associated with subtree mode *are scaled across other active scenarios* as well. This includes user set and tool derived balance offsets.

See Also

- [set_app_options](#)

cts.multisource.enable_mv_aware_tap_assignment

Data Types

Boolean

Default false

Description

This option enables the multi-voltage aware tap assignment feature in the CTS (Clock Tree Synthesis) process. When set to true, it allows the tap assignment to be aware of multi-voltage designs, enabling the cloning and reassignment of intermediate cells across different voltage domains.

User must make sure MV compatible equivalent cells are available in the design for optimal tap assignment.

See Also

- [set_multisource_clock_tap_options](#)
- [remove_multisource_clock_tap_options](#)
- [synthesize_multisource_clock_taps](#)

cts.multisource.enable_pin_accessibility_for_global_clock_trees

Enable code to improve the accessibility of pins of newly created cells by the commands *create_clock_drivers* and *synthesize_multisource_global_clock_trees*.

Data Types

Boolean

Default true

Description

The router used during global clock tree synthesize will primarily use the highest two layers allowed for the net to be synthesized. To be able to connect from these wiring shape in higher metal layers down to the pins additional caution have to be taken when placing new cells. This will significantly impact the available space where new cells can be placed. This option when set to *true* will temporarily add structures to improve the accessibility of newly inserted cells at the cost of limited placement area. When set to *false* new cells are placed at ideal locations but with the possible impact of routing detours and larger amount of wiring on lower layers impacting skew and insertion delay. By default, it is set to *true*.

c

See Also

- [create_clock_drivers](#)
- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)
- [synthesize_multisource_global_clock_trees](#)

cts.multisource.enable_subtree_synthesis_aware_ccd

Enables improved correlation between the structural multisource clock tree synthesis (SMSCTS) and concurrent clock and data (CCD) during the `place_opt` and `build_clock` stage of the `clock_opt` command.

Data Types

Boolean

Default `false`

Description

Setting this application option to *true* enables SMSCTS and CCD flow during `place_opt` and `build_clock` stage of the `clock_opt` command.

By default, the option is set to *false*. To enable SMSCTS and CCD, set the value to *true* during the `place_opt` and `build_clock` stage of the `clock_opt` command.

By default, CCD engine is not clock gate cloning aware. This can limit the CCD engine from computing timing friendly and clock latency implementation friendly useful skew profile. With this application option set to *true*, CCD engine comes up with an improved useful skew profile resulting in improved clock and timing QoR.

See Also

- [compile_fusion](#)
- [clock_opt](#)
- [synthesize_multisource_clock_subtrees](#)
- [set_app_options](#)

cts.multisource.enable_via_ladder_insertion_on_tap_output_pins

Enable via ladder insertion on tap output pins.

Data Types

Boolean

Default false

Description

With this app option turned on, via ladders will be inserted on tap output pins during *synthesize_regular_multisource_clock_trees* command.

See Also

- [synthesize_regular_multisource_clock_trees](#)

cts.multisource.flexible_htree_synthesis

Data Types

true | false

Default false

Description

Enables latency aware flexible tap synthesis and tap assignment during the *synthesize_regular_multisource_clock_trees* and *synthesize_multisource_clock_taps* commands. During tap synthesis, the tap locations chosen will be symmetric and will try to optimize the worst latency of the clock tree. During tap assignment, sinks will be reassigned from taps with higher latency to neighbouring taps with lower latency.

See Also

- [synthesize_regular_multisource_clock_trees](#)
- [synthesize_multisource_clock_taps](#)

cts.multisource.global_tree_improvements

Data Types

String

Default none

c

Description

Enables additional optimization and routing features during global tree synthesis

The `cts.multisource.global_tree_improvements` is an off-by-default app_option that enables features designed to optimize global tree synthesis and routing. While some global tree improvements are enabled by default and do not require any special settings, others are not yet enabled on-by-default, and this app_option allows users to enable those enhancements. The improvements enabled by this command are expected to be turned on- by-default in a future release. The features within this app_option are available in revisions that can be activated by setting the app_option value to a revision tag in the format GTI_YYYYMMDD. Setting the app_option value to 'latest' will automatically adjust it to the most recent revision tag. The default value of this app_option is 'none', by which no off-by-default enhancements will be enabled. This app_option must be set before executing the global tree synthesis command, as it applies exclusively to global tree synthesis.

Global Tree Improvements Information : Revision : GTI_20250401 Additional Enhancements : Htree Routing Improvements, Optimized Clock Driver Orientation

Examples

The following example sets the global tree improvements to GTI_20250401

```
prompt> set_app_options -name cts.multisource.global_tree_improvements
        -value GTI_20250401
```

See Also

- [set_regular_multisource_clock_tree_options](#)
- [synthesize_regular_multisource_clock_trees](#)

cts.multisource.htree_cell_keepout_factor**Data Types**

string

Default ""

Description

By setting the application option `cts.multisource.htree_cell_keepout_factor` to {x y} halo is created around Htree repeaters and tap cells.

c

Examples

The example below shows the use:

```
prompt> set_app_options -name cts.multisource.htree_cell_keepout_factor  
-value {2 3}
```

cts.multisource.htree_explore_all_repeater_solutions

Generate all possible solutions with all repeaters.

Data Types

Boolean

Default false

Description

By enabling this option, the tool will generate all possible solutions incorporating all repeaters and select the optimal solution that ensures the best delay solution.

Runtime degradation will be expected when more number of lib cells are specified in Htree lib cell collection as more estimations are now computed.

cts.multisource.htree_latency_only

Data Types

Boolean

Default false

Description

By enabling this option, the tool will construct a minimum latency H-tree using the resource specified honoring the DRC constraints.

The minimum latency H-tree is constructed considering latency as the only cost ignoring area and pin capacitance. Because of this neutral or minor area or power degradation can be expected.

See Also

- [synthesize_regular_multisource_clock_trees](#)
- [synthesize_multisource_clock_taps](#)

cts.multisource.htree_legalize_design

Data Types

True | False

Default `true`

Description

Controls cell legalization during htree synthesis, when it is set to false, only non-htree cells overlapping with htree cells are legalized, otherwise, enforces full cell legalization, which is the default behaviour.

See Also

- [synthesize_regular_multisource_clock_trees](#)

cts.multisource.htree_pin_connection_effort_level

Specifies the effort level used to relax hard constraint for pin connection.

Data Types

`string`

Default `high`

Description

This application option specifies the effort level used to allow larger value for guide shape length for pin connection using Z-route. The higher the effort, the tighter constraint for Zroute to finish pin connection.

Valid values are *low*, *medium*, and *high*.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [synthesize_regular_multisource_clock_trees](#)

cts.multisource.htree_single_repeater_at_center

Enables to choose single repeater at center of Htree.

c

Data Types*Boolean***Default** false**Description**

With this app option turned on, single repeater will be inserted at the center of the Htree.

cts.multisource.htree_single_repeater_at_node

Enables to choose single repeater at each node of Htree.

Data Types*Boolean***Default** false**Description**

By default, H-trees are constructed with considering logical Design Rule Checking (DRC) and latency metrics and the number of levels in the H-tree and the repeaters per level are reliant on these metrics.

During H-tree Synthesis, all viable DRC compliant solutions for building the H-tree to the tap drivers are computed and the one with the minimal latency is selected. Consequently, the H-tree could be synthesized with levels having two repeaters at one Steiner node, placed adjacently, each driving loads in separate directions of the downstream tree. This strategy bifurcates the load at a node, potentially leading to H-trees with fewer levels. However, solutions having two repeaters at the same node might not be acceptable for some users due to routing issues to the reference cell's pins or concerns about placing two H-tree repeaters closely. To address such concerns, the application option, cts.multisource.htree_single_repeater_at_node, can be set to true to constrain H-tree synthesis to select only those solutions with a singular repeater at every node. However, the presence of a single repeater at every node is assured only when a DRC clean solution of this nature exists. Imposing a solution with a singular repeater at each node might increase the number of levels in the H-tree to achieve DRC cleanliness, which may consequently influence the H-tree latency.

cts.multisource.ignore_drc_on_subtree_driver

Specifies DRC constraints that should be ignored for MSCTS driver object outputs

Data Types*string*

c

Default "none"**Description**

During multisource subtree synthesis, max transition, max capacitance, and max fanout constraints are considered for all pins in the clock subtrees. The driver objects as specified using the `set_multisource_clock_subtree_options -driver_objects` command typically have very relaxed capacitance constraints and maybe no max-fanout constraint. In order to ignore one or both of these constraints for the driver objects you can use this app option. This app option can take the values "none" (the default, meaning that all constraints on driver objects are considered), "max_fanout", and "max_capacitance".

This app option affects only the behavior of the `synthesize_multisource_clock_subtrees` command.

Examples

The following example excludes max capacitance checking on the driver objects.

```
prompt> set_app_options \\  
        -name cts.multisource.ignore_drc_on_subtree_driver \\  
        -value "max_capacitance"
```

The following example excludes max fanout checking on the driver objects.

```
prompt> set_app_options \\  
        -name cts.multisource.ignore_drc_on_subtree_driver -value "max_fanout"
```

The following example excludes both, max fanout and max_capacitance, checking on the driver objects.

```
prompt> set_app_options \\  
        -name cts.multisource.ignore_drc_on_subtree_driver \\  
        -value "max_fanout max_capacitance"
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)
- [synthesize_multisource_clock_subtrees](#)
- [set_max_capacitance](#)

cts.multisource.integrated_subtree_synthesis_early_ccd_support

Enables the support to use Concurrent Clock and Data optimization (CCD) in the *compile_fusion/place_opt* initial_opto stage on clocks synthesized using integrated Structural Multisource Clock Tree Synthesis (iSMSCTS)

Data Types

Boolean

Default false

Description

The app_option *cts.multisource.integrated_subtree_synthesis_early_support* enables the Early CCD iSMSCTS support for integrated Structural Multisource Clock Tree Synthesis in *compile_fusion/place_opt* commands to be used with *compile_fusion/place_opt* initial_opto CCD.

By default, the app_option *cts.multisource.integrated_subtree_synthesis_early_support* is set to *false*. To enable the Early CCD iSMSCTS support, this app_option must be set to *true* prior to the *compile_fusion/place_opt* *initial_opto* stage. When enabled, this flow allows the CCD in the initial_opto stage to generate SMSCTS clock structure-aware, level friendly useful skew offsets to the timing endpoints and introduces a new step in the integrated structural multisource clock tree synthesis (iSMSCTS) process during the *compile_fusion/place_opt* final_place stage, referred to as Buffer Adjust. This step takes place after the pre-process step and before the merge step of subtree synthesis, prior to the placement functions in the final_place stage. The Buffer Adjust stage performs addition and/or removal of repeaters to modify the clock subtree to each sink, ensuring that the number of levels aligns with the initial_opto CCD offsets. By default, CCD resolution is used as the buffer delay value, which helps determine how many buffer levels need to be added for each timing endpoint. This buffer delay can be modified through the app-option *cts.multisource.subtree_buf_delay_for_cus_prepone_postpone*. Following the Buffer Adjust and subsequent merge steps, the flow will execute DRC fixing step to address any issues within the subtrees before proceeding to the placement steps. After the placement functions are completed, the second round of iSMSCTS is performed, which includes DRC fixing and offset guidance handshake with *compile_fusion/place_opt* final_opto step as usual.

This app_option pertains to the integrated SMSCTS flow and does not impact standalone structural MSCTS flow or other clock methodologies.

Ensure that *CCD* is enabled in *compile_fusion/place_opt* by setting the *compile.flow.enable_ccd/place_opt.flow.enable_ccd* app_option, which is true by default.

To enable this feature, the app_option must be set to true before executing the *compile_fusion/place_opt* initial_opto command. If the app_option is enabled before the

c

final_place stage but not before the initial_opto stage, a warning will be triggered during the iSMCTS steps in the final_place stage.

See Also

- [compile_fusion](#)
- [clock_opt](#)

cts.multisource.max_net_length_only_htree_synthesis

Enable htree construction considering only the max net length constraint while ignoring other DRC checks.

Data Types

Boolean

Default false

Description

This option controls the consideration of DRC constraints. Possible values are *true* and *false* (the default).

When this option is set to *true* it enables htree construction considering only the max net length constraint while ignoring other DRC checks.

Setting this option to *false* (DEFAULT) will enable htree construction considering all the DRC checks.

cts.multisource.power_aware_flexible_htree_synthesis**Data Types**

Boolean

Default false

Description

Enables power awareness along latency during flexible tap synthesis to identify the tap configuration which is optimal for both clock latency and power.

See Also

- [synthesize_regular_multisource_clock_trees](#)
- [set_regular_multisource_clock_tree_options](#)

cts.multisource.subtree_merge_cell_name_prefix

Specifies the prefix to use in the cell name of the new merged cell created during clock logic merging in subtree synthesis.

Data Types

string

Default ""

Description

When merging of clock logic occurs during subtree synthesis, the new cell name created by merging is called <mergedName>, where the convention for <mergedName> is controlled by the *cts.multisource.subtree_merge_concatenate_length_threshold* app option.

If this *subtree_merge_cell_name_prefix* app option is used, then the merged cell name will be called <prefix>_<mergedName> instead, where <prefix> is defined by this app option.

If this app option is not specified, the default behavior is that the merged cell name has no user defined prefix.

See also the *cts.multisource.subtree_merge_cell_name_suffix* app option for defining a merged cell name suffix. If a prefix and suffix are both defined, then the merged cell name convention is <prefix>_<mergedName>_<suffix>.

The *cts.multisource.subtree_split_** app options also exist for controlling naming conventions of cells that are split during multisource subtree synthesis.

This app option affects only the behavior of the *synthesize_multisource_clock_subtrees* command.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)
- [synthesize_multisource_clock_subtrees](#)

cts.multisource.subtree_merge_cell_name_suffix

Specifies the suffix to use in the cell name of the new merged cell created during clock logic merging in subtree synthesis.

Data Types

string

Default ""

Description

When merging of clock logic occurs during subtree synthesis, the new cell name created by merging is called <mergedName>, where the convention for <mergedName> is controlled by the *cts.multisource.subtree_merge_concatenate_length_threshold* app option.

If this *subtree_merge_cell_name_suffix* app option is used, then the merged cell name will be called <mergedName>_<suffix> instead, where <suffix> is defined by this app option.

If this app option is not specified, the default behavior is that the merged cell name has no user defined suffix.

See also the *cts.multisource.subtree_merge_cell_name_prefix* app option for defining a merged cell name prefix. If a prefix and suffix are both defined, then the merged cell name convention is <prefix>_<mergedName>_<suffix>.

The *cts.multisource.subtree_split_** app options also exist for controlling naming conventions of cells that are split during multisource subtree synthesis.

This app option affects only the behavior of the *synthesize_multisource_clock_subtrees* command.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)
- [synthesize_multisource_clock_subtrees](#)

cts.multisource.subtree_merge_concatenate_length_threshold

Specifies the name of the new cell created by merging of clock logic during subtree synthesis.

Data Types

Integer

Default 0

Description

When merging of clock logic occurs during subtree synthesis, the new cell name created by merging is called <mergedName>, where the convention for <mergedName> is controlled by this app option.

By default, the merged cell name is arbitrarily inherited from the name of one of the non-merged cells. For example, if cell1, cell2, and cell3 are merged during multisource subtree synthesis, then the merged cell name might be cell2.

Setting this app option to a positive integral value enables name concatenation, where the merged cell name is derived by concatenating the names of the unmerged cells. There is some identification of common parts of the cell names avoid duplicating the common part during the name concatenation.

For example, the original cells may have been named cell_123 and cell_456. The concatenated name for the merged cell could be cell_123_456, or cell_456_123.

Name concatenation only occurs if the total length of the cell name does not exceed the value set by this app option. If the name length does exceed the concatenation threshold, then the default behavior is used of just choosing one of the original cell's names for the merged cell name.

Consider the same example again where the concatenated name is cell_123_456, which is 12 characters long. If this app option is set to 12 or greater, then the merged cell name of cell_123_456 will be used. But if this app option is set to a value less than 12, then the merged cell name will be either cell_123 or cell_456.

See also the *cts.multisource.subtree_merge_cell_name_prefix* app option for defining a merged cell name prefix. If a prefix is defined, then the merged cell name convention is <prefix>_<mergedName>.

See also the *cts.multisource.subtree_merge_cell_name_suffix* app option for defining a merged cell name suffix. If a suffix is defined, then the merged cell name convention is <mergedName>_<suffix>.

If both a prefix and suffix are set with those app options, then the merged cell name is <prefix>_<mergedName>_<suffix>.

c

Note that this app option only considers <mergedName> when checking if the name exceeds the concatenation length threshold, not the <prefix> and/or <suffix> parts of the name.

The *cts.multisource.subtree_split_** app options also exist for controlling naming conventions of cells that are split during multisource subtree synthesis.

This app option affects only the behavior of the *synthesize_multisource_clock_subtrees* command.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)
- [synthesize_multisource_clock_subtrees](#)

cts.multisource.subtree_routing_mode

Specifies the routing mode to route the clock nets during multisource clock tree synthesis.

Data Types

enumerated

Default global

Description

When multisource CTS subtree synthesis runs, the clock nets can be routed using one of three methods, as configured by this app option.

Possible options are *global*, *fishbone*, and *none*.

The default routing mode is *global*, which uses global routing on clock nets. This app option can also be set to *fishbone*, which enables fishbone routing on all clock nets for the subtrees. The other available setting for this app option is *none*, which uses virtual routing for all clock nets on the subtrees.

When fishbone routing is used, the clock nets will be a mix of detailed routes and global routes. A fishbone routing structure involves a central fishbone trunk off of the net driver, which extends the full length of the cluster of net loads. Fishbone routing will only be used on a net if the fishbone trunk is at least 5 gcells in length. Off of this trunk, several fishbone fingers may extend. The trunk and fingers are routed on the two highest allowed routing

c

layers. The trunk and the fingers are protected detailed route shapes with shape_use of *stripe*.

The net loads are connected to the fishbone fingers by short comb routes, which are global routed. The comb routes can use all allowed routing layers for the net.

When fishbone routing is used, there are several app options available to control the fishbone routing behavior for CTS commands. See the *cts.routing.fishbone_** app options for more details.

This app option affects only the behavior of the *synthesize_multisource_clock_subtrees* command.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)
- [synthesize_multisource_clock_subtrees](#)

cts.multisource.subtree_split_cell_name_prefix

Specifies the prefix to use in the cell name of the new split cell created during clock logic splitting in subtree synthesis.

Data Types

string

Default ""

Description

During multisource subtree synthesis, clock cells may be split in order to meet optimization goals. This app option configures a prefix for the names of cells created by splitting.

When splitting, the naming convention for newly created cells is `<originalCellName>_<index>`, where `<index>` is incremented for each newly created cell. By default, no prefix is attached to this cell name.

This app option defines a prefix for the split cell names. When this app option is set to a non-null value, the newly created cells are named `<prefix>_<originalCellName>_<index>`, where `<prefix>` is defined by this app option.

c

This app option affects only the behavior of the *synthesize_multisource_clock_subtrees* command.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)
- [synthesize_multisource_clock_subtrees](#)

cts.multisource.subtree_split_cell_name_suffix

Specifies the suffix to use in the cell name of the new split cell created during clock logic splitting in subtree synthesis.

Data Types

string

Default ""

Description

During multisource subtree synthesis, clock cells may be split in order to meet optimization goals. This app option configures a suffix for the names of cells created by splitting.

When splitting, the naming convention for newly created cells is `<originalCellName>_<index>`, where `<index>` is incremented for each newly created cell. By default, no suffix is attached to this cell name.

This app option defines a suffix for the split cell names. When this app option is set to a non-null value, the newly created cells are named `<originalCellName>_<index>_<suffix>`, where `<suffix>` is defined by this app option.

This app option affects only the behavior of the *synthesize_multisource_clock_subtrees* command.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)

c

- [set_app_options](#)
- [synthesize_multisource_clock_subtrees](#)

cts.multisource.subtree_split_net_name_prefix

Specifies the prefix to use in the net name of the new net created during clock logic splitting in subtree synthesis.

Data Types

string

Default ""

Description

During multisource subtree synthesis, clock cells may be split in order to meet optimization goals. Every time a cell is split, a new net must be created which is driven by that split cell. This app option configures a prefix for the names of the new nets created by splitting.

When splitting, the naming convention for newly created nets is `<originalNetName>_<index>`, where `<index>` is incremented for each newly created cell/net. By default, no prefix is attached to this net name.

This app option defines a prefix for the split net names. When this app option is set to a non-null value, the newly created nets are named `<prefix>_<originalNetName>_<index>`, where `<prefix>` is defined by this app option.

This app option affects only the behavior of the `synthesize_multisource_clock_subtrees` command.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)
- [synthesize_multisource_clock_subtrees](#)

cts.multisource.subtree_split_net_name_suffix

Specifies the suffix to use in the net name of the new net created during clock logic splitting in subtree synthesis.

c

Data Types

string

Default ""**Description**

During multisource subtree synthesis, clock cells may be split in order to meet optimization goals. Every time a cell is split, a new net must be created which is driven by that split cell. This app option configures a suffix for the names of the new nets created by splitting.

When splitting, the naming convention for newly created nets is `<originalNetName>_<index>`, where `<index>` is incremented for each newly created cell/net. By default, no suffix is attached to this net name.

This app option defines a suffix for the split net names. When this app option is set to a non-null value, the newly created nets are named `<originalNetName>_<index>_<suffix>`, where `<suffix>` is defined by this app option.

This app option affects only the behavior of the `synthesize_multisource_clock_subtrees` command.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)
- [synthesize_multisource_clock_subtrees](#)

cts.multisource.subtree_synthesis_enable_power_optimization

Enables power aware optimization during subtree synthesis

Data Types

Boolean

Default false**Description**

Setting this app_option to *true* directs subtree synthesis to achieve clock QoR targets while reducing total clock tree power.

c

By Default, the option is off i.e. value *false* To enable power optimization set the value to *true*

Structural multisource subtree synthesis by default performs optimizations on clock cells to meet DRC and improve max/min latency and global skew. When power aware optimization is enabled, DRC and latency optimization tries to improve total clock power along with similar clock QoR i.e. DRCs, latency, skew and clock cell area.

See Also

- [set_app_options](#)

cts.multisource.subtree_synthesis_timing_aware_flow

This app_option enables Slack Aware SMSCTS within integrated Structural Multisource CTS (iSMSCTS) in clock_opt build_clock command

Data Types

Boolean

Default false

Description

The cts.multisource.subtree_synthesis_timing_aware_flow application option enables slack awareness for structural subtree synthesis when running the integrated flow with the clock_opt build_clock command. This feature uses timing slack information collected after the compile_fusion or place_opt stage to reduce area on clock paths that do not impact critical timing. During this process, the tool assigns each sink in the structural subtree the worst-case timing slack (Q-slack) from all timing paths launched from that sink. Sinks with positive slack are then identified. For these non-timing-critical sinks, the tool relaxes latency optimization, which may result in exceeding user-defined network delay targets. This approach helps reduce clock area and power consumption while preserving timing neutrality. By default, this application option is set to false. It is applicable only in the integrated SMSCTS flow and does not affect standalone structural MSCTS flows. To enable slack awareness, set this application option to true before the final placement stage of compile_fusion or place_opt, and keep it enabled through the clock_opt build_clock stage.

See Also

- [compile_fusion](#)
- [clock_opt](#)

cts.multisource.tap_assignment_allow_cascaded_taps

Data Types

Boolean

Default false

Description

Controls the cascaded tap assignment feature that supports tap assignment for two levels of tap drivers.

By setting the application option *cts.multisource.tap_assignment_allow_cascaded_taps* to true, the tool automatically identifies the cascaded paths where the tap option for one set is defined below the other set. With this feature, tap assignment assigns the fanout, and the sinks of both upstream and downstream sets independently. Loads of 2nd-level tap drivers are distributed across 2nd-level taps and the sinks under the 1st-level taps are reassigned across 1st level taps.

Tap options for each level must be defined separately in the same mode using the *-name* switch option along with *set_multisource_clock_tap_options* command.

Examples

The example below shows the necessary setup before integrated or standalone tap assignment commands for the cascaded tap assignment feature to kick in:

```
prompt> set_app_options -list
{cts.multisource.tap_assignment_allow_cascaded_taps true}

prompt> set_multisource_clock_tap_options \
-name cascaded_level_1_settings \
-clock CLK \
-driver_objects < > \
-num_taps < >

prompt> set_multisource_clock_tap_options \
-name cascaded_level_2_settings \
-clock CLK \
-driver_objects < > \
-num_taps < >
```

See Also

- [set_multisource_clock_tap_options](#)
- [remove_multisource_clock_tap_options](#)
- [synthesize_multisource_clock_taps](#)

cts.multisource.tap_assignment_improvements

Data Types

String

Default none

Description

Enables additional optimization during tap assignment. The `cts.multisource.tap_assignment_improvements` is an off-by-default app_option that enables features designed to optimize tap assignment. While some tap assignment improvements are enabled by default and do not require any special settings, others are not yet enabled on-by-default, and this app_option allows users to enable those enhancements. The improvements enabled by this command are expected to be turned on-by-default in a future release. The features within this app_option are available in revisions that can be activated by setting the app_option value to a revision tag in the format `TAI_YYYYMMDD`. Setting the app_option value to 'latest' will automatically adjust it to the most recent revision tag. The default value of this app_option is 'none', by which no off-by-default enhancements will be enabled. This app_option must be set before executing the tap assignment command, as it applies exclusively to tap assignment.

Tap Assignment Improvements Information : Revision : `TAI_20251101`, latest Additional Enhancements : Gate Relocation, Latency-Aware Location Estimation, Limited Cluster Size.

Examples

The following example sets the tap assignment improvements to `TAI_20251101`

```
prompt> set_app_options -name cts.multisource.tap_assignment_improvements  
-value TAI_20251101
```

See Also

- [set_multisource_clock_tap_options](#)
- [remove_multisource_clock_tap_options](#)
- [synthesize_multisource_clock_taps](#)

cts.multisource.tap_assignment_print_debug_report

Data Types

Boolean

Default false

c

Description

Enables tap assignment debug report to be printed at the end of tap assignment.

Summary report prints latency bottleneck, farthest connected and region outlier sinks for all the taps, with the reason of assignment. Also prints current manhattan distance from the tap driver and nearest neighboring tap with its distance.

See Also

- [set_multisource_clock_tap_options](#)
- [remove_multisource_clock_tap_options](#)
- [synthesize_multisource_clock_taps](#)

cts.multisource.tap_assignment_reassign_ignore_sinks**Data Types**

Boolean

Default false

Description

By enabling this option, the tool will prevent cloning of ICG to drive only ignore sinks during tap assignment.

Avoiding of such unnecessary cloning during tap assignment helps to prevent CUS timing degradations.

See Also

- [set_multisource_clock_tap_options](#)
- [remove_multisource_clock_tap_options](#)
- [synthesize_multisource_clock_taps](#)

cts.multisource.tap_selection**Data Types**

enumerated

Default user

c

Description

This `app_option` defines the tap configuration mode for flexible htree synthesis in `fBsynthesize_regular_multisource_clock_trees` command.

Possible values are *user* and *auto*.

The default value for this `app_option` is *user*, which means the flexible htree will be constructed with the tap configuration specified by the user through the `-tap_boxes` option in `set_regular_multisource_clock_tree_options` command.

When the `app_option` is set to *auto*, the tool will ignore the user specified configuration, and instead automatically select the optimal tap configuration.

See Also

- [synthesize_regular_multisource_clock_trees](#)
- [set_regular_multisource_clock_tree_options](#)

cts.multisource.tap_synthesis_route_based_estimation

By default, tap assignment considers the placeable standard cell area when computing the path length between a tap cell and a clock sink. Setting this option to *true* let's tap assignment perform buffer-aware global routing based distance computation.

Data Types

Boolean

Default `true`

Description

This option controls how multisource tap assignment performs the estimation of distance between tap cells and clock tree sinks. Possible values are *true* (the default) and *false*. Sinks are assigned to tap cells based on the closest of such distance.

When this option is set to *false* tap assignment computes the distance between a clock sink and a tap cell by estimating a path in the placeable standard cell area, i.e. placement blockages or macros might introduce detouring. Setting this option to its default value of *true* will enable a buffer-aware global routing based estimation. Small placement blockages or macros won't trigger detouring in case tap assignment determines that CTS can route over those blockages honoring clock DRC constraints. NDRs and min or max layer constraints on clock nets are considered while performing the routing based estimation.

This application option is set using the `set_app_options` command, and specified per design.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)
- [synthesize_multisource_clock_taps](#)

cts.multisource.verbose

Controls the verbosity level of logfile messages for all multisource synthesis commands.

Data Types

Integer

Default 0

Description

All multisource CTS synthesis commands generate some logfile output. By default, this logfile output is limited, giving some information about progress and results of the command, as well as information about any problems encountered during synthesis.

The verbosity of this logfile output can be increased by setting this app option to 1. This will give more detailed logfile output that may be helpful in debugging a more complex problem with multisource CTS synthesis. For most users, the default setting of 0 should be sufficient.

This app option controls verbosity of logfile messages for all multisource synthesis commands.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)
- [synthesize_multisource_clock_subtrees](#)

cts.optimize.delay_insertion_enhancements

Specifies whether to enable the enhanced features in delay insertion engine which will be used in clock tree optimization stage.

Data Types

boolean

Default false

Description

By default, this option is *false*. When this option is set to *true*, user can expect better result for *Target latency delay insertion* step and *Inter-clock delay balancing* step while other steps in clock tree optimization stage may also be slightly affected. Compared with the result of setting this option to *false*, user may find the repeater count slightly increased after setting it to *true*.

See Also

- [synthesize_clock_trees](#)
- [clock_opt](#)

cts.optimize.enable_area_recovery_in_local_skew

Controls whether local skew for area recovery is enabled during clock tree optimization.

Data Types

Boolean

Default true

Description

When this option is *true*, clock tree optimization will perform area recovery as part of the local skew optimization. By default this option is *true*.

See Also

- [synthesize_clock_trees](#)

cts.optimize.enable_balance_point_optimization

Turn on the MTCTO *User balance point optimization* feature in the Solver-Augmented Skew Optimization flow.

c

Must be enabled with the Solver-Augmented Skew Optimization flow.

Data Types

Boolean

Default `false`

Description

The current MTCTO global latency and skew optimization engine targets to optimize the global skew by minimizing the delay difference between the longest path and shortest path. If the optimization is stuck in any of paths, engine would bail out and rest of the paths of user instructed balance points stay under optimized.

When this feature is enabled by setting the application option to *true*, MTCTO skew optimization engine will target on implementing user balance points in the Solver-Augmented Skew Optimization flow. Besides, to preserve the implemented user balance points, the area recovery engine also skips removing the clock path cells of user balance points.

Please note that this feature must be enabled with the Solver-Augmented Skew Optimization feature, which is controlled by *cts.optimize.enable_solver_augmented_skew_optimization*.

See Also

- [synthesize_clock_trees](#)
- [set_clock_balance_points](#)

cts.optimize.enable_congestion_aware_ndr_promotion

Controls whether SI prevention based on congestion aware and NDR promotion is enabled during clock tree optimization.

Data Types

Boolean

Default `false`

Description

When this option is *true*, clock tree optimization will apply congestion aware NDR promotion for SI prevention. By default, this option is *false* and clock tree optimization will apply no change to routing rules.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)
- [synthesize_clock_trees](#)

cts.optimize.enable_cto_power_optimization

Controls whether to turn on power optimization in clock tree optimization.

Data Types

Boolean

Default `false`

Description

This option turns on or off power optimization in clock tree optimization.

See Also

- [synthesize_clock_trees](#)

cts.optimize.enable_improvement_mode

Controls which optimization improvement features are enabled during clock tree optimization.

Data Types

String

Default `none`

Description

When this option is not *none*, new clock tree optimization improvement features will be enabled. Current available modes include *runtime*, *hpc*, *skew*.

runtime

Improve clock tree optimization runtime with potentially small QOR perturbations.

c

hpc

Include all *runtime* improvement features, and features for better latency and R2R TNS, which are needed for high performance computing.

skew

Include all *runtime* and *hpc* improvement features, and features for better global skew.

Examples

To enable a new revisions, use the *cts.optimize.improvement_mode_version* option to specify the revision tag. For example:

```
prompt> set_app_options -as_user_default -name
cts.optimize.improvement_mode_version -value EIM_20230330
prompt> set_app_options -name cts.optimize.enable_improvement_mode -value
skew
```

Please note that use the *-as_user_default* argument to ensure the tag is registered globally, and specify the revision tag before setting the *cts.optimize.enable_improvement_mode* option.

See Also

- [synthesize_clock_trees](#)

cts.optimize.enable_layer_optimization_for_power

Controls whether layer optimization is enabled during clock tree optimization.

Data Types

Boolean

Default *true*

Description

When this option is *true*, clock tree optimization will perform layer optimization based on re-assigning clock routing rules to clock nets. By default this option is *true*.

See Also

- [synthesize_clock_trees](#)

cts.optimize.enable_local_skew

Controls whether local skew optimization is enabled during clock tree optimization.

c

Data Types

Boolean

Default `false`

Description

When this option is *true*, clock tree optimization will minimize local skew of the clock trees. By default, this option is *false* and clock tree optimization targets global skew only. In *clock_opt*, this option will be turned on automatically when *clock_opt.flow.enable_ccd* is *true*.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)
- [synthesize_clock_trees](#)

cts.optimize.enable_nworst_skew_optimization

Controls whether MTCTO optimize the skew of nworst paths instead of only the shortest path.

Data Types

Boolean

Default `false`

Description

When this option is *true*, MTCTO will optimize the skew of nworst paths. Up to 2% of the total endpoints will be visited for skew optimization. By default, this option is *false* and MTCTO optimize the skew of the shortest path.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)

c

- [set_app_options](#)
- [synthesize_clock_trees](#)

cts.optimize.enable_solver_augmented_skew_optimization

Turn on the MTCTO *Solver-Augmented* skew optimization feature.

Data Types

Boolean

Default false

Description

With this feature turned on by setting this application option to true, MTCTO skew optimization will use *Linear-Programming Solver* to minimize clock skew, which will benefit complex clock structures with clocks from all scenarios largely overlapping with each other. The clock structures will be analyzed and abstracted, and a model suitable for linear-programming will be constructed. The solver will solve the linear-programming model and generate solutions with budgeted delays on the clock tree, target to minimize skews of all clocks across all scenarios. The budgeted solutions will finally be implemented by clock tree optimization engine.

See Also

- [synthesize_clock_trees](#)

cts.optimize.improvement_mode_version

Specifies the revision for *cts.optimize.enable_improvement_mode*.

Data Types

String

Default EIM_20230313

Description

Use this option to select which revision to use for *cts.optimize.enable_improvement_mode*. New revisions are typically released at the beginning of a calendar quarter. Currently available revisions include:

```
EIM_20250401
EIM_20250101
EIM_20241001
EIM_20240701
```

c

EIM_20240401
EIM_20240101
EIM_20230930

See Also

- [synthesize_clock_trees](#)

cts.optimize.multi_corner_effort

Controls the level of effort for optimizing across different corners during clock tree global skew optimization.

Data Types

String

Default low

Description

During clock tree optimization, the skew balancing requirements on different corners may create conflicting conditions that involves tradeoffs. For example, the skew in the setup corner may not be reduced further unless the latency of the hold corner, of the same clock tree, is increased.

When this option is set to *medium* or *high*, the skew optimization engine will choose the solutions that balance the clock metrics for all corners.

Valid values are *low*, *medium*, and *high*.

See Also

- [synthesize_clock_trees](#)

cts.optimize.print_qor_summary_table

Controls whether qor summary tables are printed in clock tree optimization.

Data Types

Boolean

Default false

Description

When this option is *true*, clock tree optimization will print qor summary tables. By default, this option is *false* and clock tree optimization will not print such tables.

c

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)
- [synthesize_clock_trees](#)

cts.placement.cell_spacing_rule_style

This application option implies the style of the spacing rules used between the cells in the clock tree for which such rules are defined.

Data Types

enumerated

Default maximum

Description

Possible values are *cumulative* and *maximum*.

When *cumulative* is specified and spacing rules are present valid distance between 2 cells affected by the rules changes from default - (*maximum*) between x1 and x2 and (*maximum*) between y1 and y2 to the sum of x1 and x2 and sum of y1 and y2.

Application options are set using the *set_app_options* command, and specified globally.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)
- [set_clock_cell_spacing](#)

cts.report.transition_histogram_datapoints

Controls how the transition histograms are populated for the *-histogram_type transition* option of the *report_clock_qor* command.

c

Data Types

Enumerated

Default "loads"

Description

When *loads*, only net loads are populated in the histograms. When *drivers*, only net drivers are populated. When *all*, all clock pins or ports are considered as datapoints for the transition histograms.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [report_clock_qor](#)
- [set_app_options](#)

cts.report.verbose_paths_routing_length_threshold

This option gives user control to filter the output of the routing layer field to only show the layers which have length over a certain threshold in *-show_verbose_path* report of the *report_clock_qor* command.

Data Types

float

Default 100000.00

Description

This option controls for the routing layer information in *show_verbose_paths* reports. The default value is 10um. If a certain routing layer has length below the threshold, it wouldn't be printed. So for example with threshold of 0 we may see {M1 2.0} {M2 15.4} {M3 300.8} and with threshold of 20um we would instead just see {M3 300.8}.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)

c

- [report_clock_qor](#)
- [set_app_options](#)

cts.routing.fishbone_bias_threshold

Data Types

float

Default 0

Description

This app option controls fishbone routing behavior for all CTS synthesis commands that have fishbone routing enabled. It has no effect if fishbone routing is not used during CTS.

Fishbone routing topologies can be biased towards power or ground nets so that those nets act as shields, to improve signal integrity on the fishbone route. Biasing, if enabled, is performed on the fishbone trunks only, not the fishbone fingers nor the comb routes connecting the net loads to the fingers.

This app option specifies a minimum trunk length on a fishbone route before biasing of the trunk placement is attempted. It has no effect unless fishbone trunk biasing is enabled through either or both of the *fishbone_horizontal_bias_spacing* and *fishbone_vertical_bias_spacing* app options.

The default value of 0 means that biasing will be attempted on all fishbone trunks. By setting this app option to a positive value, it prevents biasing attempts on trunks shorter than the specified length threshold. The threshold should be specified in length units as configured by the *set_user_units -type length -input* command, which is micrometers (um) by default.

See also the *fishbone_vertical_bias_spacing* app option to enable biasing on vertical fishbone trunks.

See also the *fishbone_horizontal_bias_spacing* app option to enable biasing on horizontal fishbone trunks.

See also the *fishbone_bias_window* app option to specify a maximum displacement of a trunk from its preferred location in order to achieve biasing.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)

c

- [set_app_options](#)
- [synthesize_multisource_clock_subtrees](#)

cts.routing.fishbone_bias_window

Data Types

float

Default 0

Description

This app option controls fishbone routing behavior for all CTS synthesis commands that have fishbone routing enabled. It has no effect if fishbone routing is not used during CTS.

Fishbone routing topologies can be biased towards power or ground nets so that those nets act as shields, to improve signal integrity on the fishbone route. Biasing, if enabled, is performed on the fishbone trunks only, not the fishbone fingers nor the comb routes connecting the net loads to the fingers.

This app option specifies a maximum displacement from the preferred location of the fishbone trunk, in order to achieve biasing against a power or ground net. It has no effect unless fishbone trunk biasing is enabled through either or both of the *fishbone_horizontal_bias_spacing* and *fishbone_vertical_bias_spacing* app options.

Each fishbone trunk has a preferred location, based on the locations of the net driver and the net loads. Typically the fishbone trunk must be shifted away from its preferred location in order to achieve biasing against the nearest power or ground net. The default value of 0 on this app option means that there is no limit to how far the fishbone trunk may be displaced to achieve biasing. Setting this app option to a positive value places a maximum displacement limit on the fishbone trunks to achieve biasing. If the required displacement exceeds this threshold for a given fishbone trunk, then it will not be biased. The threshold should be specified in length units as configured by the *set_user_units -type length -input* command, which is micrometers (um) by default.

See also the *fishbone_vertical_bias_spacing* app option to enable biasing on vertical fishbone trunks.

See also the *fishbone_horizontal_bias_spacing* app option to enable biasing on horizontal fishbone trunks.

See also the *fishbone_bias_threshold* app option to specify a minimum trunk length before attempting biasing.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)
- [synthesize_multisource_clock_subtrees](#)

cts.routing.fishbone_horizontal_bias_spacing**Data Types**

float

Default 0**Description**

This app option controls fishbone routing behavior for all CTS synthesis commands that have fishbone routing enabled. It has no effect if fishbone routing is not used during CTS.

Fishbone routing topologies can be biased towards power or ground nets so that those nets act as shields, to improve signal integrity on the fishbone route. Biasing, if enabled, is performed on the fishbone trunks only, not the fishbone fingers nor the comb routes connecting the net loads to the fingers. This app option controls biasing for horizontal fishbone trunks.

The default value for this app option is 0, meaning that no attempt is made to bias the placement of horizontal fishbone trunks towards power or ground nets. If this app option is set to a positive value, then biasing of the horizontal fishbone trunks is enabled, and this app option indicates the desired spacing between the fishbone trunk and the power or ground net. The spacing should be specified in length units as configured by the *set_user_units -type length -input* command, which is micrometers (um) by default.

See also the *fishbone_vertical_bias_spacing* app option to enable biasing on vertical fishbone trunks.

See also the *fishbone_bias_threshold* app option to specify a minimum trunk length before attempting biasing.

See also the *fishbone_bias_window* app option to specify a maximum displacement of a trunk from its preferred location in order to achieve biasing.

c

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)
- [synthesize_multisource_clock_subtrees](#)

cts.routing.fishbone_max_sub_tie_distance**Data Types**

float

Default 0**Description**

This app option controls fishbone routing behavior for all CTS synthesis commands that have fishbone routing enabled. It has no effect if fishbone routing is not used during CTS.

A fishbone routing structure involves a central fishbone trunk off of the net driver, which extends the full length of the cluster of net loads. Off of this trunk, several fishbone fingers may extend. The net loads are connected to the fishbone fingers by short comb routes. Comb routes are not shared: each comb routes connects only one net load to its fishbone finger.

Optionally, another level of fishbone routing can be introduced, called sub-fingers. These connect to the fingers, and then the comb routes can instead connect the net loads to the sub-fingers. This can reduce wirelength, especially in the case that a large fishbone_max_tie_distance is used. Multiple comb routes can connect to a single sub-finger, though each comb route still connects to only one net load.

The fishbone trunk and fingers are routed on the two highest layers configured for the net. If sub-fingers are used, they are routed on the layer directly below the finger.

By default, sub-fingers are not created. But by setting this app option to a value greater than 0, sub-fingers are created. This app option controls the maximum distance from a fishbone sub-finger to all of its connected net loads. In other words, it defines a window around the sub-finger (orthogonal to the routing direction of the sub-finger) in which net loads can be connected to that sub-finger. For example, if the max sub-tie distance is configured to 20um on a vertical sub-finger, then the farthest away net loads might be 13um to the west and 7um to the east of the sub-finger.

c

This behavior is identical to that of the *fishbone_max_tie_distance* app option, but in this case the connection window is defined for sub-fingers rather than fishbone fingers.

The max sub-tie distance should be specified in length units as configured by the *set_user_units -type length -input* command, which is micrometers (um) by default.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)
- [synthesize_multisource_clock_subtrees](#)

cts.routing.fishbone_max_tie_distance

Data Types

float

Default 0

Description

This app option controls fishbone routing behavior for all CTS synthesis commands that have fishbone routing enabled. It has no effect if fishbone routing is not used during CTS.

A fishbone routing structure involves a central fishbone trunk off of the net driver, which extends the full length of the cluster of net loads. Off of this trunk, several fishbone fingers may extend. The net loads are connected to the fishbone fingers by short comb routes.

This app option controls the maximum distance from a fishbone finger to all of its connected net loads. In other words, it defines a window around the finger (orthogonal to the routing direction of the finger) in which net loads can be connected to that finger. For example, if the max tie distance is configured to 20um on a vertical finger, then the farthest away net loads might be 13um to the west and 7um to the east of the finger.

In a sense, this app option controls the frequency with which fishbone fingers are created along the fishbone trunk. The smaller the max tie distance, the greater the number of fishbone fingers that will be created to connect to the net loads.

The default value of this app option is 0. With this default, a max tie distance equal to 20 gcells distance is automatically derived. When specifying a non-default value for this app option, the max tie distance should be specified in length units as configured by the *set_user_units -type length -input* command, which is micrometers (um) by default.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)
- [synthesize_multisource_clock_subtrees](#)

cts.routing.fishbone_vertical_bias_spacing**Data Types**

float

Default 0

Description

This app option controls fishbone routing behavior for all CTS synthesis commands that have fishbone routing enabled. It has no effect if fishbone routing is not used during CTS.

Fishbone routing topologies can be biased towards power or ground nets so that those nets act as shields, to improve signal integrity on the fishbone route. Biasing, if enabled, is performed on the fishbone trunks only, not the fishbone fingers nor the comb routes connecting the net loads to the fingers. This app option controls biasing for vertical fishbone trunks.

The default value for this app option is 0, meaning that no attempt is made to bias the placement of vertical fishbone trunks towards power or ground nets. If this app option is set to a positive value, then biasing of the vertical fishbone trunks is enabled, and this app option indicates the desired spacing between the fishbone trunk and the power or ground net. The spacing should be specified in length units as configured by the *set_user_units -type length -input* command, which is micrometers (um) by default.

See also the *fishbone_horizontal_bias_spacing* app option to enable biasing on horizontal fishbone trunks.

See also the *fishbone_bias_threshold* app option to specify a minimum trunk length before attempting biasing.

See also the *fishbone_bias_window* app option to specify a maximum displacement of a trunk from its preferred location in order to achieve biasing.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)
- [synthesize_multisource_clock_subtrees](#)

cts.routing.global_route_topology

This application option selects the type of topology used by global route in clock tree synthesis and optimization.

Data Types

string (*automatic atree or steiner*)

Default atree

Description

When *atree*, A-tree topology is used. When *steiner*, Steiner topology is used. When *automatic* (the default in CCD mode), atree or steiner topology is selected automatically depending on the circuit. In non CCD mode, the tool uses atree mode (automatic is not available), but steiner may be specified.

Application options are set using the *set_app_options* command.

custom.indesign.custom_compiler_cache

Option for the *run_custom_compiler* command. Specifies the cache directory in which to create OA design data.

Data Types

string

Description

Specifies the cache directory in which to create OA design data by Custom Compiler when the *run_custom_compiler* command is run. If you do not set this option, then the tool will use your current working directory path and create a 'codesign' directory there.

Examples

The following example shows how to set the cache directory path for the OA design data.

c

```
prompt> set_app_options -list {custom.indesign.custom_compiler_cache  
    {./codesign}}
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)
- [run_custom_compiler](#)

custom.indesign.custom_compiler_path

Option for the *run_custom_compiler* command. Specifies the executable path for Custom Compiler.

Data Types

string

Default None

Description

Specifies the executable path for Custom Compiler. If you do not set this option, the tool will search your env(PATH) directory. If an executable path is still not found, then the *run_custom_compiler* command will error out.

Examples

The following example shows how to set the executable path for Custom Compiler.

```
prompt> set_app_options -list {custom.indesign.custom_compiler_exec  
    {/global/customcompiler_2020.03-SP1/bin}}
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)
- [run_custom_compiler](#)

custom.indesign.opto_compiler_cache

Option for the *run_opto_compiler* command. Specifies the cache directory in which to create OA design data.

Data Types

string

Description

Specifies the cache directory in which to create OA design data by Opto Compiler when the *run_opto_compiler* command is run. If you do not set this option, then the tool will use your current working directory path and create a 'codesign' directory there.

Examples

The following example shows how to set the cache directory path for the OA design data.

```
prompt> set_app_options -list {custom.indesign.opto_compiler_cache  
    {./codesign}}
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)
- [run_opto_compiler](#)

custom.indesign.opto_compiler_path

Option for the *run_opto_compiler* command. Specifies the executable path for OptoCompiler.

Data Types

string

Default None

Description

Specifies the executable path for OptoCompiler. If you do not set this option, the tool will search your env(PATH) directory. If an executable path is still not found, then the *run_opto_compiler* command will error out.

Examples

The following example shows how to set the executable path for Custom Compiler.

```
prompt> set_app_options -list {custom.indesign.opto_compiler_exec  
  {/  
global/apps/optocompiler_2024.09-SP1/photonicssolutions/W-2024.09-SP1/bin}}
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)
- [run_opto_compiler](#)

custom.route.balance_mode

This option controls whether to enable or disable balance_mode.

Data Types

Boolean

Default false

Description

This option controls whether to enable or disable net balance mode routing. By default, this is disabled. The user can override this by setting this option to true.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [route_custom](#)

custom.route.buffer_distance

This option specifies the buffer distance for custom router buffer insertion aware routing mode.

c

Data Types

*float***Default** -1.0

Description

This option specifies the buffer distance for custom router buffer insertion aware routing mode. This app option takes effect when the app option "custom.route.buffer_insertion_aware_routing" is set as true to turn on custom router buffer insertion aware routing mode. The default value is -1.0, which means there is no buffer distance constraints. route_custom command will route within bufferable region as much as possible. When set this value, route_custom will try to route through bufferable region no farther than this distance.

Examples

```
prompt> set_app_options -name custom.route.buffer_insertion_aware_routing
-value true
prompt> set_app_options -name custom.route.buffer_distance -value 100.0
prompt> route_custom -nets $nets
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [route_custom](#)

custom.route.buffer_insertion_aware_routing

This option controls whether to enable or disable custom router buffer insertion aware routing mode.

Data Types

*Boolean***Default** false

Description

This option controls whether to enable or disable custom router buffer insertion aware routing mode. When it is enabled, route_custom will try to create routing paths to pass through bufferable voltage areas of the net. By default, this option is disabled.

c

Examples

```
prompt> set_app_options -name custom.route.buffer_insertion_aware_routing  
-value true
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [route_custom](#)

custom.route.bus_corner_type

This option specifies the bus corner type.

Data Types

Enum

Default "auto"

Description

This option controls the bus corner type.

"auto", the default, indicates the router can determine the best corner type to use.

"river" indicates the router will use a river style corner pattern.

"cross" indicates the router will use a crossing style corner pattern.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [route_custom](#)

custom.route.bus_intra_shield_placement

This option specifies the shield placement between bus bit nets.

Data Types

Enum

Default "outside"

Description

This option specifies the shield placement between bus bit nets.

"outside" is the default and results in shielding only around the outside bit nets of the bus.

"interleave" results in a single shield wire placed between each bit net of the bus.

"double_interleave" results in two shield wires placed between each bit net of the bus.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [route_custom](#)

custom.route.bus_pin_trunk_offset

This option specifies an offset value used for placing auto generated bus virtual pins.

Data Types

Float

Default 0.0

Description

This option allows the user to specify an offset value used for placing auto generated bus virtual pins.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [route_custom](#)

custom.route.bus_split_even_bits

This option controls whether auto bus splitting will leave an even number of nets between pre-routes (power stripes).

Data Types

Boolean

Default false

Description

This option controls whether auto bus splitting will leave an even number of nets between pre-routes (power stripes). The default is false.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [route_custom](#)

custom.route.bus_split_ignore_width

This option specifies a width such that any pre-routes whose width is less than this value are ignored in auto bus split mode.

Data Types

Float

Default 0.0

Description

This option specifies a width such that any pre-routes whose width is less than this value are ignored in auto bus split mode.

See Also

- [get_app_option_value](#)
- [get_app_options](#)

- [report_app_options](#)
- [route_custom](#)

custom.route.bus_tap_off_enable

This option controls whether to enable or disable bus tap-off routing.

Data Types

Boolean

Default true

Description

This option controls whether to enable or disable bus tap-off routing. By default, this is enabled. The user can override this by setting this option to false.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [route_custom](#)

custom.route.bus_tap_off_shielding

This option controls whether to enable shielding in bus tap-off routing.

Data Types

Boolean

Default true

Description

This option controls whether the route can add shielding to the bus tap-off routing. By default, shielding is added.

See Also

- [get_app_option_value](#)
- [get_app_options](#)

- [report_app_options](#)
- [route_custom](#)

custom.route.diffpair_twist_jumper_enable

This option controls whether to enable twist jumpers on differential pairs.

Data Types

Boolean

Default false

Description

This option controls whether to enable twist jumpers on differential pairs. If true, then the router will create twisted jumpers.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [route_custom](#)

custom.route.diffpair_twist_jumper_interval

This option controls the differential pair twisted jumper interval.

Data Types

Float

Default 0.0

Description

This option allows you to specify the differential pair twisted jumper interval. The value is in microns and the default is 0.0

See Also

- [get_app_option_value](#)
- [get_app_options](#)

c

- [report_app_options](#)
- [route_custom](#)

custom.route.diffpair_twist_jumper_offset

This option specifies the differential pair twisted jumper offset.

Data Types

Float

Default 0.0

Description

This option allows you to specify the differential pair twisted jumper offset. The value is in microns and the default is 0.0

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [route_custom](#)

custom.route.diffpair_twist_jumper_style

This option specifies the differential pair jumper style.

Data Types

Enum

Default "orthogonal"

Description

This option specifies the differential pair jumper style. "orthogonal", the default, will make 90-degree angles when adding jumper segments. "diagonal" means the router will make 45-degree angles when adding jumper segments.

See Also

- [get_app_option_value](#)
- [get_app_options](#)

c

- [report_app_options](#)
- [route_custom](#)

custom.route.distance_to_net_pin

This option controls the maximum distance from a pin the routing can stop.

Data Types

List of net and distance pairs

Default {} (Empty list)

Description

This option controls the maximum distance from a pin the routing can stop. Specify the value as a list of pairs where the first item is the net name and the second item is the maximum distance for that net.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [route_custom](#)

custom.route.export_path_segment

This option forces the custom router to create each route segment as a two-point path.

Data Types

Boolean

Default false

Description

If true, this option forces the custom router to export all routing segments as two-point paths instead of combining points into a single path. Some flow operations need paths created this way, especially DDR flow using `add_buffer_on_route`. The default is false.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [route_custom](#)
- [set_app_options](#)

custom.route.honor_shield_rule_for_shield

This option controls whether honor shield rule only and ignore NDR spacing.

Data Types

Boolean

Default false

Description

This option controls whether honor shield rule only and ignore NDR spacing. By default, this is disabled. The user can override this by setting this option to true.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [route_custom](#)
- [create_net_shielding](#)

custom.route.hybrid_flow

Use this application option to enable or disable the hybrid flow routing.

Data Types

Boolean

Default false

c

Description

You can use the `custom.route.hybrid_flow` application option to enable or disable the hybrid flow routing. It is set to `false` by default. Setting it to `true` enables the hybrid flow routing.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [route_custom](#)

custom.route.ignore_shielding_gap_on_via

This option controls whether to ignore shielding gap on overlapped via.

Data Types

Boolean

Default `false`

Description

This option controls whether to ignore shielding gap specified in `create_net_shielding` on overlapped via. By default, when creating shield vias that overlap with shields, shielding gap is considered. If switched on, such via shapes will be created regardless of shielding gap.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [create_net_shielding](#)
- [route_custom](#)

custom.route.layer_grid_mode

This option controls the grid mode on a per layer basis.

Data Types

A list of string pairs of layer name and value.

Default {} (Empty list)

Description

This option controls the grid mode on a per layer basis.

If you specify an empty list, then the router will follow a strict grid on every layer.

If you specify a list where the first entry contains the layer name ALL, then the router will apply the second entry (the grid mode) to all layers.

Otherwise, you should specify a list of pairs where the layer name is the first entry and the mode is the second entry.

Valid modes are "off", "on" or a numeric cost from 0-20.

"off" means the layer will be routed without any grid constraints.

"on" means the layer will be routed strictly on grid.

A numeric cost influences how much the router will allow itself to route off the grid.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [route_custom](#)

custom.route.match_box

This option specifies the bounding box of wire match routing.

Data Types

BBox

Default { {0 0} {0 0} }

Description

This option specifies the bounding box of wire match routing. If the value is anything other than the default of { {0 0} {0 0} } then the router will limit match routing to the area inside the box. This may result in connections that fail to meet the match constraint.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [route_custom](#)
- [set_app_options](#)

custom.route.match_rc_routing_style

This option specifies the RC style for match routing.

Data Types

Enum

Default "endrun"

Description

This option specifies the RC style for match routing.

Each option specifies a different pattern or approach to adding additional length to meet the match constraints.

"endrun" is the default. "endrun_multilayer" "lasso"

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [route_custom](#)

custom.route.net_max_layer_mode

Data Types

enumerated

Default hard

c

Description

This option controls the net maximum layer mode. By default, the router cannot route above the maximum layer, even if some pin connections require it. You can set the mode to one of the following values to give the router some flexibility:

- *hard* (the default)

Routing cannot occur above the maximum layer, resulting in more unrouted nets.

- *soft*

The router adjusts the distance it uses automatically.

- *allow_pin_connection*

The router can route outside of the minimum and maximum layer range, within its escape distance window, to reach a pin. To adjust the escape distance window, set the *custom.route.distance_to_net_pin* application option.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [route_custom](#)

custom.route.net_max_layer_mode_soft_cost

Data Types

Enum

Default "medium"

Description

This option is used in conjunction with `custom.route.net_max_layer_mode` to control how much cost the router will assign to shapes above the max layer when soft mode is chosen.

The legal values are "low", "medium" (the default), and "high". High will make it the most expensive and should significantly reduce the amount of routing above the max.

See Also

- [get_app_option_value](#)
- [get_app_options](#)

c

- [report_app_options](#)
- [route_custom](#)

custom.route.net_min_layer_mode

Controls whether any routing is allowed below the minimum layer.

Data Types

enumerated

Default `soft`

Description

This option sets the net minimum layer mode. By default, the router cannot route below the minimum layer, even if some pin connections require it. You can set the mode to one of the following values to give the router some flexibility:

- *hard*
Routing cannot occur below the minimum layer, resulting in more unrouted nets.
- *soft* (the default)
The router adjusts the distance it uses automatically.
- *allow_pin_conection*
The router can route outside of the minimum and maximum layer range, within its escape distance window, to reach a pin. To adjust the escape distance window, set the *custom.route.distance_to_net_pin* application option.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [route_custom](#)

custom.route.net_min_layer_mode_soft_cost

Data Types

Enum

c

Default "medium"**Description**

This option is used in conjunction with `custom.route.net_min_layer_mode` to control how much cost the router will assign to shapes below the min layer when soft mode is chosen.

The legal values are "low", "medium" (the default), and "high". High will make it the most expensive and should significantly reduce the amount of routing below the min.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [route_custom](#)

custom.route.routing_area

This option specifies the bounding box to limit the routing area.

Data Types*BBox***Default** { {0 0} {0 0} }**Description**

This option specifies a bounding box to limit the routing area.

If the default { {0 0} {0 0} } is set, then the router will use the top cell PR boundary as the routing area.

If any other box is set, then the router will only be able to route connections fully contained inside the routing area. Connections fully or partially outside the routing area will be skipped. To improve efficiency, the router will not process all data outside the routing area.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [route_custom](#)

custom.route.shield_connect_mesh_overlap

This option controls whether to enable creating multiple connections from shields to power mesh.

Data Types

Boolean

Default false

Description

This option controls whether to enable creating multiple connections from shields to power mesh. By default only a single connection is created.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [route_custom](#)

custom.route.shield_connect_route

This option controls whether to connect shield connections to supply using auto router engine.

Data Types

Boolean

Default false

Description

This option controls whether to connect shield connections to supply using auto router engine. By default the router uses a simple algorithm to make the connections.

See Also

- [get_app_option_value](#)
- [get_app_options](#)

c

- [report_app_options](#)
- [route_custom](#)

custom.route.shield_connect_to_supply

This option controls whether shield shapes need to connect to supply.

Data Types

Boolean

Default false

Description

This option controls whether shield shapes need to connect to supply. By default, the route is allowed to create shield that does not connect to supply if it has trouble making the supply connection. If true, then it will not create shield that cannot connect to the supply.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [route_custom](#)

custom.route.shield_min_open_loop

This option specifies the minimum total length of a open shield loop

Data Types

String

Default none

Description

This option specifies the minimum total length of a open shield loop in same layer. The shield shape created needs to be longer than shield_min_open_loop or it will not be used. By default there is no minimum total length.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [route_custom](#)

custom.route.shield_min_shield_seg

This option specifies the minimum length of a shield line.

Data Types

Float

Default 0.0

Description

This option specifies the minimum length of a shield line. The shield shape created needs to be longer than `shield_min_shield_seg` or it will not be used. By default there is no minimum segment length.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [route_custom](#)

custom.route.shield_min_signal_seg

Specifies the minimum length of a signal segment for shielding to be created.

Data Types

Float

Default 0.0

Description

This option specifies the minimum length of a signal segment for shielding to be created. If the signal shape is less than `shield_min_signal_seg`, then it will be left unshielded.

c

By default, there is no minimum segment length and the router attempts to shield every segment of the signal net.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [route_custom](#)
- [set_app_options](#)

custom.route.shield_net

This option allows you to specify shield net name (when it is not specified in shield constraint).

Data Types

String

Default none

Description

This option allows you to specify the second shield net name (if it is not specified in shield constraint). There is no default. If no value is specified, then a shield net is only used if it was specified in the shield constraint. If this value is not specified and the constraint does not specify a shield net, the router will not create any shielding.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [route_custom](#)

custom.route.shield_second_net

This option allows you to specify the second shield net name (if it is not specified in shield constraint).

c

Data Types*String***Default** none**Description**

This option allows you to specify the second shield net name (if it is not specified in shield constraint). There is no default. If no value is specified, then a second shield net is only used if it was specified in the shield constraint.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [route_custom](#)

custom.route.shield_stripe_fat_wire

This option allows user to specify the striping constraint for fat tandem shielding paths.

Data Types*String***Default** none**Syntax**

collection *custom.route.shield_stripe_fat_wire*

```
[-layer layer_settings]
[-maxWidth]
[-honorGap]
[-nocut]
[-clear]
```

Data Types

layer_settings list

Arguments

-layer layer_settings

The *layer_settings* is a list that contains groups *{layer width gap}* information.

c

The *layer* can be specified as ***, which means all layer will use the width and gap setting in this group. If a group with *** and a group for a specific layer coexist, the group for the specific layer will be prioritized.

`-maxWidth`

If set, stripe the shields by maximum width.

`-honorGap`

If set, honor the gap setting specified in `create_net_shielding` constraint.

`-nocut`

If set, fat shielding shapes will not be cut.

`-clear`

Clear stripe constraints.

Description

This option allows user to specify the striping constraint for fat tandem shielding paths. By default, shielder will not stripe fat tandem shielding paths. Note that only one of "-layer", "-maxWidth", and "-honorGap" will be honored for this app option.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [create_net_shielding](#)
- [route_custom](#)

custom.route.single_loop_match

Data Types

Boolean

Default false

Description

This option controls whether the router will use a post-route single loop process to adjust the length of nets to meet the matching constraint. Set to true if this process is desired. The router uses a multi-layer one-sided loop approach to add length.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [route_custom](#)

custom.route.single_loop_match_max_spacing

Data Types

Float

Default 0

Description

This option controls the maximum spacing the router will use when creating a single loop shape for matched length constraints. Spacing is calculated between the centerlines of the loop shapes.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [route_custom](#)

custom.route.single_loop_match_min_spacing

Data Types

Float

Default 0

Description

This option controls the minimum spacing the router will use when creating a single loop shape for matched length constraints. Spacing is calculated between the centerlines of the loop shapes.

c

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [route_custom](#)

custom.route.single_loop_match_offset_layer**Data Types***Boolean***Default** false**Description**

If true, the router will try to use other layers (which have the same direction) when creating the loop shape, in addition to trying to add length on the same layer. The router will need to add four vias if it attempts to use offset layers to create the additional length.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [route_custom](#)

custom.route.skip_connect_pin_type**Data Types***Enum***Default** "none"**Description**

This option controls which pins can have the routing stop close to the pin as opposed to touching the pin.

"none", the default, means no pins will stop close to the pin.

"auto" indicates the router should use pin analysis algorithms to choose the pins.

d

"all" indicates the router will stop short of any pin.

"stdcell" indicates the router will stop short of a pin which is part of a standard cell.

"macro" indicates the router will stop short of a pin which is part of a macro cell.

"io" indicates the router will stop short of a pin which is part of an io cell.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [route_custom](#)

d

da.check_netlist.allow_connection_class_violation

Controls whether the tool allow the net have connection class violation

Data Types

bool

Default false

Description

The tool will suppresses connection class warning messages if the value is set to true. This switch is useful while working on GTECH designs and netlists on which connection class violations are expected. By default, these messages are reported.

Examples

The following example specifies the *allow_connection_class_violation* for the *check_netlist* command.

```
prompt> set_app_options -name  
da.check_netlist.allow_connection_class_violation -value true
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)

- [report_app_options](#)
- [set_app_options](#)
- [check_netlist](#)

da.check_netlist.allow_multiply_driven_nets_by_inputs_and_outputs

Controls whether the tool checks for input ports that connect to multiply driven nets whose drivers include an output pin.

Data Types

bool

Default false

Description

This option provides a control to skip the checking of input ports that connect to multiply driven nets whose drivers include an output pin.

Examples

The following example specifies the *allow_multiply_driven_nets_by_inputs_and_outputs* for the *check_netlist* command.

```
prompt> set_app_options \\  
        -name  
        da.check_netlist_allow_multiply_driven_nets_by_inputs_and_outputs \\  
        -value true
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)
- [check_netlist](#)

da.check_netlist.check_for_wire_loop

Controls whether the tool checks for wire loops that have a timing loop with no cells in it.

Data Types

bool

Default true

Description

This option provides a control to skip the checking for a wire loop.

Examples

The following example specifies the *check_for_wire_loop* for the *check_netlist* command.

```
prompt> set_app_options -name da.check_netlist.check_for_wire_loop -value  
true
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)
- [check_netlist](#)

da.check_netlist.report_all_shorted_power_ground_nets

Reports each net segment for the shorted power ground nets when set to true, otherwise only one net will be reported.

Data Types

bool

Default false

Description

This option provides a control whether it should report warnings for all detected shorted power and ground nets in check netlist command.

Examples

The following example specifies the *report_all_shorted_power_ground_nets* for the *check_netlist* command.


```
prompt> set_app_options \\  
-name  
da.check_netlist.check_netlist.report_all_shorted_power_ground_nets \\  
-value true
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)
- [check_netlist](#)

da.report_design.enable_front_end_sequential_cell_counting

Enable the sequential cell counting in front end.

Data Types

bool

Default true

Description

This option determines whether it should count and update information for sequential cells in report design output.

Examples

The following example specifies the *enable_front_end_sequential_cell_counting* for the *report_design* command.

```
prompt> set_app_options \\  
-name da.report_design.enable_front_end_sequential_cell_counting \\  
-value false
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)

- [set_app_options](#)
- [report_design](#)

design.allow_inst_specific_user_attributes

Controls whether instance specific user attributes are enabled on MIB netlist objects like inst, pin, port and net.

Data Types

Boolean

Default false

Description

By default (*false*), instance specific user attributes are disabled in MIB netlist objects.

When this option is *true*, instance specific user attributes are enabled in MIB netlist objects like inst, pin, port and net.

For example,

```
set_app_options -name design.allow_inst_specific_user_attributes -value  
true
```

See Also

- [set_app_options](#)
- [get_app_option_value](#)
- [get_app_options](#)

design.allow_multiple_scale_factors

Enables opening of libraries with different scale factors

Data Types

Boolean

Default false

Description

When this option is *true*, and *design.session_scale_factor* is set then libraries with different scale factors can be opened.

The *design.session_scale_factor* must be a multiple of scale factors of all libraries the user intends to open.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)

design.allow_multiple_subset_names

Enables support of multiple subset_names for library cells.

Data Types

Boolean

Default false

Description

When this option is *false*, only one subset_name is allowed for a library cell. That means one library cell can only be included in one libcell_subset family. Also, after the library cells are grouped into a family, they are not available for use by cells without *set_libcell_subset* constraint during optimization.

When this option is *true*, then multiple subset_names are allowed for libcell design. That means one libcell can be included in multiple libcell_subset families. Even when the library cells are grouped into a libcell_subset families, they are still available for use by all cells with or without *set_libcell_subset* constraint during optimization. The libcell_subset constraint only restricts library cell selection for cells with libcell_subset constraint.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)

design.auto_spare_cell

Turns on or off automatic detection of spare cells in the design.

Data Types

boolean

Default true

Description

This app_option specifies that spare cells are detected by the tool automatically in addition to cells marked spare by the user. A cell, not explicitly marked as spare, is considered a spare cell, if it meets the following criteria 1) Is not a physical only cell 2) All input pins are floating or tied 3) All output pins are floating

Clock pins on the cell may or may not be connected

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

design.bump_region_pseudo_bump_count_limit

Sets the maximum allowable pseudo_bump count per bump_region.

Data Types

Unsigned integer

Default 50000000

Description

This option sets a limit on the number of allowable pseudo_bumps per bump_region. This prevents an overflow of system resources in case the bump_region or its referenced bump_region_pattern is misconfigured.

See Also

- [create_bump_region](#)
- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)

design.bus_delimiters

Specifies the bus delimiters to be used in port, pin, and net names.

Data Types

String

Default `[]`

Description

This app_option specifies the bus delimiters to be used in port, pin, and net names to denote bus subscripts. Valid values are `[]`, `{}`, `()`, and `<>`. However the bus delimiters may not be the same as the name expansion delimiters.

These delimiters may be used in the *get_ports*, *get_pins*, and *get_nets* commands, as well as any command that creates a collection from port, pin, or net names.

All bus port, pin, and net names are displayed with these delimiters.

See Also

- [get_ports](#)
- [get_pins](#)
- [get_nets](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

design.bus_types_expanded_in_query

Enable cell and port bus expansion in queries.

Data Types

list_of string

Default `cell port`

Description

When this app option is set to *cell*, the *get_cells* and *get_pins* commands search for `cell_bus` objects matching the cell name in the given query pattern and include their contents in the returned collection.

d

When this app option is set to *port*, the *get_ports* and *get_pins* commands search for *port_bus* objects matching the port name in the given query pattern and include their contents in the returned collection.

Examples

The following example creates cell bus named *cellBus* from cell 1 and 2 and then *get_cells* with different pattern and app option value. It also creates port bus names *portBus* from port 1 and 2 and then *get_ports* with different pattern and app option value.

```
prompt> create_cell_bus -reference RISC_CORE cellBus[1:2]
{cellBus}
prompt> create_port_bus portBus[1:2] -create_ports
{portBus}
prompt> get_app_option_value -name design.bus_types_expanded_in_query
cell port
prompt> get_cells cellBus
{{cellBus[1]} {cellBus[2]}}
```



```
prompt> get_pins cellBus/A1
{{cellBus[1]/A1} {cellBus[2]/A1}}
```



```
prompt> get_ports portBus
{{portBus[1]} {portBus[2]}}
```



```
prompt> set_app_options -name design.bus_types_expanded_in_query -value
""
design.bus_types_expanded_in_query port
prompt> get_cells cellBus
Warning: No cell objects matched 'cellBus' (SEL-004)
```



```
prompt> get_pins cellBus/A1
Warning: No pin objects matched 'cellBus/A1' (SEL-004)
```



```
prompt> get_ports portBus
Warning: No port objects matched 'portBus' (SEL-004)
```

See Also

- [get_cells](#)
- [get_pins](#)
- [get_ports](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

design.change_names.dc_compatible_flatten_multidimensional_bus

This option specifies whether multi-dimensional bus should be flatten compatible to dc or not.

Data Types

boolean

Default false

Description

This app_option specifies that whether the bus should be flatten compatible to dc or not. If set to true, multi-dimensional bus indexes will be flatten as per dc.

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

design.change_names.dc_compatible_negative_index_shift

This option shifts negative index to positive during change_names as per DC methodology.

Data Types

boolean

Default false

Description

This app option is set false by default. If this app option is set to true during change_names -

If bus has one of the index aligned to 0, and the other one is negative, the bus indexes will be converted to positive keeping one of the index aligned to 1.

For example - If there is a PortBus A[-2:0], If the app option design.change_names.dc_compatible_negative_index_shift is set to true, then during the execution of change_names it will be converted to A[1:3]

If both the indices are negative it will be shifted to positive keeping one of the index aligned to zero.

For example - If there is a PortBus B[-5:-3], If the app option `design.change_names.dc_compatible_negative_index_shift` is set to true. During the execution of `change_names` it will be converted to A[0:2]

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

design.change_names.dc_consistent

This option changes names of netlist objects during `change_names` as per DC methodology.

Data Types

boolean

Default false

Description

This app option is set false by default. If this app option is set to true during `change_names` -

If `collapse_name_space` rule or `case_insensitive` rule is applied then '%d' will be used as postfix instead of '_%d'.

If `first_restricted` rule is applied then 'p/u/n' will be used as prefix instead of 'P/U/N' for 'pins/cells/nets' respectively.

Negative index bus will be shifted even if app option '`design.change_names.dc_compatible_negative_index_shift`' is set to false.

No word limit in a line while generating name change file through '`change_names -log changes`'.

`report_name_rules` will report the prefix as 'p/c/n' instead of 'P/C/N'.

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

design.change_names.include_cell_bus

This option specifies whether change_names operation to be applied on cell buses or not.

Data Types

boolean

Default false

Description

This app_option specifies that whether the change_names operations to be applied on cell buses or not.

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

design.change_names.shift_negative_index

This option shifts negative index to positive during the execution of change_names.

Data Types

boolean

Default false

Description

This app option is set to false by default. If this app option is set to true during change_names -

It shift negative index to positive keeping one of the index (starting or ending) as zero as per bit order.

For example - If there is a PortBus A[-2:0], If the app option `design.change_names.shift_negative_index` is set to `true` during the execution of `change_names` it will be shifted to A[0:2]

Similarly B[-3:-5] will be shifted to B[2:0].

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

design.compact_pg

Controls all the individual application options for Compact PG.

Data Types

Boolean

Default `false`

Description

By default (*false*), Compact PG application options are not enabled.

When this option is *true*, all the controls of all different functional areas for Compact PG are enabled.

For example, `set_app_option -name design.compact_pg -value true`.

See Also

- [close_blocks](#)
- [open_block](#)
- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)

design.create_in_logical_only_mode

Tell `create_block` to create logical-only designs.

Data Types

Boolean

Default false

Description

When this application option is set to *true*, all new designs created from `create_block` will have "logical_only" flag set to true:

```
set_app_option -name design.create_in_logical_only_mode create_block 1 get_attribute [get_blocks 1] logical_only true
```

To determine the value of this application option, use the *get_app_option_value - name design.create_in_logical_only_mode* command

See Also

- [set_query_rules](#)

design.disable_auto_link

Disables auto-linking of blocks when commands are executed.

Data Types

Boolean

Default false

Description

By default (*false*), auto-linking of blocks takes place.

When this app option is set to true, auto-linking is disabled. Commands which can work with partial linked design will return objects from top-design. However, the output may be different when physical hierarchy is present. Commands which require completely linked block to proceed will raise an error.

This application option is linked with `gui.auto_link_blocks`, which means changing the value of one option will update the other's value correspondingly. (Note that these two application options are opposite. When `design.disable_auto_link=true`, means `gui.auto_link_blocks=false` applied).

See Also

- [get_app_option_value](#)
- [get_app_options](#)

- [report_app_options](#)
 - [set_app_options](#)
-

design.eco_freeze_silicon_mode

Specifies if the design is in freeze silicon mode.

Data Types

boolean

Default false

Description

This app_option tells the tool that the design is in freeze silicon mode.

This is a special ECO mode for late stages of the design. Most of the commands are disabled in this mode and only a limited number of commands which are *freeze silicon aware* or *freeze silicon tolerant* are available.

These commands keep minimize the changes in design and manage the allowed changes to silicon.

See Also

- [check_freeze_silicon](#)
 - [connect_freeze_silicon_tie_cells](#)
 - [map_freeze_silicon](#)
 - [place_freeze_silicon](#)
 - [get_app_option_value](#)
 - [report_app_options](#)
 - [set_app_options](#)
-

design.eco_restrictions

Prevents unsafe netlist changes when set to true.

Data Types

boolean

Default false

Description

This `app_option` controls prevention of unsafe netlist changes. It is meant to be used later in the design flow to enable the tool to check if a netlist change is safe before going thru with it.

See Also

- [create_cell](#)
- [create_net](#)
- [create_pin](#)
- [remove_cell](#)
- [remove_net](#)
- [connect_net](#)
- [disconnect_net](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

design.edit_read_only_libs

Controls whether the `open_block` command changes a read-only library to edit mode when you open a block in the library in edit mode.

Data Types

Boolean

Default `true`

Description

This option controls how the `open_block` command treats a read-only library when opening a block in the library.

By default (`true`), the `open_block` command changes a read-only library to edit mode when you open a block in the library in edit mode.

When this option is set to `false`, the behavior depends on whether you explicitly open the block in edit mode by using the `-edit` option with the `open_block` command.

- If you specify the *-edit* option, the *open_block* command fails with a DES-029 error.
- If you do not specify the *-edit* option, the *open_block* command opens the block in read-only mode.

See Also

- [open_block](#)
- [reopen_block](#)
- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)

design.enable_attribute_support

Enables or disables the support for attribute copy in ECO regions in the Formality AutoEco Targeted Synthesis Flow.

Data Types

boolean

Default `false`

Description

This *app_option*, when set to *true*, enables the support for attribute copy in ECO regions in the Formality AutoEco Targeted Synthesis Flow.

When this *app_option* is *false*, attribute copy is disabled for ECO regions in the Formality AutoEco Targeted Synthesis flow.

To determine the value of this application option, use the *get_app_option_value - name design.enable_attribute_support* command.

See Also

- [set_fm_eco_mode](#)

design.enable_cmd_result_cache

Determines whether the result of *get_pins* command is cached or not.

Data Types

Boolean

Default true

Description

When *true* (the default), result of `get_pins` command will be cached and the cached result will be returned if the same command is invoked later in the same session. Please note that caching is not supported if collection is passed in pattern or in `-of_objects` option or if the `-filter` option contains multiple attributes or attributes other than "name" or "full_name". Cache is invalidated if the design is modified or certain app-options are set/unset. The caching strategy used is least recently used(lru) and the cache size is fixed at 10 per design. When *false*, caching will not be used and result will be computed every time `get_pins` command is invoked.

See Also

- [get_app_option_value](#)
- [get_app_options](#)

design.enable_hierarchy_edit_restriction

Enforces the hierarchical edit control of blocks.

Data Types

boolean

Default false

Description

This app_option, when set to *true*, enforces the hierarchical edit control of blocks.

When this app_option is *false*, hierarchical edit control of blocks is not enforced.

Example: - Editability of block r4000 is set to false, prompt> report_editability
-blocks r4000.design ***** Block Is_editable
----- r4000 false

app option is set to false prompt> report_app_options
design.enable_hierarchy_edit_restriction Name Type Value User-value User-
default System-default Scope Status Source -----

design.enable_hierarchy_edit_restriction bool false -- -- false global normal --

```

-----
-----

If we try removing a cell in design r4000, cell will be removed. prompt> remove_cells U298
-design r4000 1

Now if we set app option to true, editability will be honored and cell won't be
removed. prompt> report_app_options design.enable_hierarchy_edit_restriction
***** Name Type Value User-value User-default System-
default Scope Status Source -----
----- design.enable_hierarchy_edit_restriction
bool true true true false global normal

prompt> remove_cells U298 -design r4000 Error: 1 cell are in un-editable hierarchy and
hence the command is not executed, few of which are U298. (NDMUI-868)

```

See Also

- [set_editability](#)
- [get_editability](#)
- [report_editability](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

design.enable_icc_region_query

Controls whether IC Compiler region query behavior is enabled in the *get_objects_by_location* command.

Data Types

Boolean

Default false

Description

This application option controls whether IC Compiler region query behavior is enabled in the *get_objects_by_location* command.

This setting changes the behavior of the *-within* and *-intersect* options of the *get_objects_by_location* command.

When set to *true*, the IC Compiler meaning of the *-within* and *-intersect* options is enabled.

- *-within* includes the objects that are strictly inside the queried region and does not include objects that touch the queried region from the inside.
- *-intersect* includes the objects that overlap the queried region, including the objects that touch the queried region from the inside or outside.

See Also

- [get_objects_by_location](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

design.enable_lib_cell_editing

Enables mutable lib-cell attributes editability in ICC2 shell.

Data Types

String

Default none

Description

This app option controls whether mutable lib-cell attributes can be modified within the ICC2 shell. When the app option value is "none", these attributes cannot be modified. When the app option is "mutable", these attributes can be modified. These attribute modifications are not persistent and cannot be saved in ICC2 shell.

In general, there are three types of lib-cell attributes. The first type cannot be changed after the lib-cell library is built. The second type cannot be changed outside of the Library Manager because the checks, which make sure the value is changed properly, only exist within the Library Manager. The third type (i.e., the mutable lib-cell attributes) can be changed from inside the ICC2 shell, but only when the app option `design.enable_lib_cell_editing` is set to "mutable".

These mutable lib-cell attribute modifications are not persistent, because lib-cell libraries are read-only in the ICC2 shell. In order to make these modifications persistent, you would need to return to the Library Manager and use the edit flow to change these attributes. Or you could return to the original library setup flow and change these attributes before checking and committing the workspace.

See Also

- [ATTR-25](#)

design.enable_non_capturing_regexp

Enable non-capturing group in the regexp pattern.

Data Types

boolean

Default `true`

Description

When this app_option is set to *false*, the colon (:) in the regular expression pattern is treated as a delimiter, splitting the pattern into the library's source_file_name and the object name pattern.

If the app_option is set to *true*, the (? : ...) pattern is recognized as a non-capturing group, and the colon (:) is only interpreted as part of that group, not as a delimiter.

This app_option is supported by the commands `get_lib_cells` and `get_lib_pins`.

See Also

- [get_lib_cells](#)
- [get_lib_pins](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

design.enable_recovery_save

Controls whether the recovery data is to be saved or not.

Data Types

Boolean

Default `false`

Description

When *true*, recovery data will be saved. When *false* (the default), recovery data will not be saved. Recovery save will not be done if the triggering command is `save_lib`, `save_block` or `write_lib_package`.

See Also

- [get_app_option_value](#)
- [get_app_options](#)

design.enable_rule_based_query

Enables or disables rule-based matching in the `get_cells`, `get_pins`, `get_ports`, and `get_nets` commands.

Data Types

Boolean

Default `false`

Description

When this application option is set to *true*, rule-based matching is enabled. However, you must run the `set_query_rules` command first. If you set this application option to *true* without running the `set_query_rules` command first, the default query rules are used.

When this application option is set to *true*, the runtime for the query might be slower. Disable rule-based matching when it is no longer needed.

To determine the value of this application option, use the `get_app_option_value - name design.enable_rule_based_query` command.

See Also

- [set_query_rules](#)

design.enable_ts_block_management

Enables automatically loading the output block after `compile_ts_eco`

Data Types

Boolean

Default `true`

Description

The *design.enable_ts_block_management* application option allows that after executing *compile_ts_eco* (new UI flow of FMECO target synthesis), the *current_block* in the tool corresponds to the newly generated ECO patch.

If this option is disabled, the current block will be the ONET block, which is usually read-only.

design.enable_via_ladder_protection

Restrict editing of via ladder member.

Data Types

Boolean

Default false

Description

By default (*false*).

When this option is *true*, via ladder member can not be edited using following commands.

`remove_object remove_vias remove_shapes set_attribute set_via_def add_to_edit_group`

For example,

```
set_app_options -name design.enable_via_ladder_protection -value true
```

See Also

- [set_app_options](#)
- [get_app_option_value](#)
- [get_app_options](#)

design.enforce_terminal_name_uniqueness

Specifies whether to disallow setting a terminal name to the same name as any existing terminal in the block.

Data Types

boolean

Default false

Description

This `app_option` causes the tool to not allow giving any terminal the same name as any existing terminal in the terminal's block, therefore enforcing terminal name uniqueness across the entire block.

When this `app_option` is set to *true*, it will be an error to set a new or existing terminal's name to that of any other terminal in the block.

When this `app_option` is set to *false*, it will be an error to set a new or existing terminal's name only to that of any other terminal belonging to the same port; that is, terminal name uniqueness is only enforced at the port level.

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

design.high_fanout_net_threshold

Specifies a default threshold value used to determine if a net is a high-fanout net for command `all_high_transitive_fanout`.

Data Types

Integer

Default "1000"

Description

This `app_option` specifies a default threshold value used to determine if a net is a high-fanout net for command `all_high_transitive_fanout`. The `app_option` is effective when the `'-threshold numThreshold'` option of `all_high_transitive_fanout` isn't specified or the `numThreshold` is less than 1.

The `app_option` may be effective only in the *all_high_transitive_fanout* command.

See Also

- [all_high_transitive_fanout](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

design.is_3dic_mode

Enables 3DIC mode for the session.

Data Types

Boolean

Default false

Description

When this option is *true*, the tool is started in 3DIC mode, which enables scaling for 3DIC blocks. If *design.session_scale_factor* is also set, then libraries with different scale factors can be opened. Also, if any library has a valid *half_node_scale_factor* attribute, the designs from that library are scaled accordingly.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)

design.keep_terminals_check_duplicates

Checks if a duplicate shape has a terminal associated with it and if it has, `check_duplicates -remove` will not remove it.

Data Types

Boolean

Default false

Description

When set to *true*, `check_duplicates -remove` command will not remove those duplicate shapes which have a terminal associated to them.

design.memory_optimization

Controls all the individual application options for general memory optimization.

Data Types

Boolean

Default false

Description

By default (*false*), Memory Optimization application options are not enabled.

When this option is *true*, all the controls of all different functional areas for General Memory Optimization are enabled.

For example, *set_app_option -name design.memory_optimization -value true*.

See Also

- [close_blocks](#)
- [open_block](#)
- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)

design.morph_on_save_as

Controls whether the *save_block* command renames the block before saving it when you use the *-as* option.

Data Types

Boolean

Default false

Description

By default (*false*), the *save_block -as* command does not rename the block before saving it.

When this option is set to *true*, the command first renames the block and then saves it.

See Also

- [save_block](#)
- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)

design.name_expansion_delimiters

Specifies the name expansion delimiters to be used in port, pin, and net names.

Data Types

string

Default "()"

Description

This application option specifies the name expansion delimiters to be used in port, pin, and net names to denote name expansion subscripts. Valid values are "[]", "{ }", "()", and "<>". However, the name expansion delimiters might not be the same as the bus delimiters.

These delimiters might be used in the *get_ports*, *get_pins*, and *get_nets* commands, as well as any commands that creates a collection from port, pin, or net names.

See Also

- [get_ports](#)
- [get_pins](#)
- [get_nets](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

design.on_disk_operation

This option specifies whether to save the block or not during *copy_block* command.

Data Types

boolean

Default false

Description

This app_option specifies that whether the block copied should be saved or not during copy_block command. Saving the block to disk happens only when the source block is copied to the specified destination.

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

design.preserve_reference_editability

Enable rebind to retain editability when the reference changes either the library name or the block name.

Data Types

Boolean

Default false

Description

By default (*false*), the commands that changes the reference of a design block wouldn't retain the editability and make the the new reference uneditable (read_only) if the new reference has either different library name or different block name.

When this app option set to true, the editability propagates from the old reference to the new reference for change of library name or block name for the commands: *change_abstract -lib*, *link_block -rebind -force* and *set_reference*.

See Also

- [change_abstract](#)
- [link_block](#)
- [set_reference](#)
- [get_app_option_value](#)

- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)

design.purpose_number_aware_check_duplicates

Used to consider purpose number on the shapes while reporting and removing duplicate shapes.

Data Types

Boolean

Default `false`

Description

When set to *true*, `check_duplicates` command will consider purpose number along with geometry while reporting and removing duplicate shapes.

Two geometrically equivalent shapes on same layer are not considered duplicates if their purpose numbers are different when this app option is set to *true*.

design.recovery_save_dir_path

Sets the location for saving recovery data.

Data Types

string

Default Current working directory.

Description

This application option sets the location to which the recover data will be saved.

To determine the current value of this application option, use `get_app_option_value -name design.recovery_save_dir_path`.

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

design.remove_net_shapes

Forces the `remove_nets` and `remove_ports` commands to remove shapes owned by the deleted nets and ports.

Data Types

boolean

Default `false`

Description

This `app_option`, when set to *true*, specifies that the `remove_nets` and `remove_ports` commands should always remove the shapes owned by the deleted nets and ports.

When this `app_option` is *false*, the `remove_nets` and `remove_ports` commands do not remove the shapes, unless the command option `-remove_shapes` is specified.

See Also

- [remove_nets](#)
- [remove_ports](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

design.report_editability_source

Enable `report_editability` to print an additional column "Edit_lock_source" reporting the source of reference owned edit-lock if any.

Data Types

Boolean

Default `false`

Description

By default (*false*), `report_editability` reports the boolean "Is_editable" column for the design blocks. Blocks specified "editable" in the hierarchical context would return false when its reference lib or design is edit-locked. It might confuse the end-user when changing the context specific hierarchical editability does not impact the reported boolean value in presence of any edit-lock owned by its reference lib or design.

When this app option set to true, the reporting command prints an additional column "Edit_lock_source" reporting the source of reference owned edit-lock if any. The reported value in this column has value of either "lib" or "block" to help user identify the owner of the edit-lock.

See Also

- [report_editability](#)
- [set_editability](#)
- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)

design.report_variable_row_utilization

Enables report utilization in variable row design mode.

Data Types

Boolean

Default true

Description

This application option, when set to *true*, specifies that the report_utilization reports utilization in variable row design mode. It means that if design is having variable or hybrid site-row setup then utilization is reported as it is with the -hybrid option in the report_utilization command.

When this application option is *false*, then the report_utilization command reports utilization as it is without the -hybrid option.

See Also

- [report_utilization](#)
- [create_utilization_configuration](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

design.save_session_ref_libs

Specifies that the `save_lib` command should propagate and save the session scoped reference library list to the library being saved.

Data Types

Boolean

Default `false`

Description

If this application option is set to true and the session scoped reference library list is set, `save_lib` will propagate the session scoped reference library list to the library and then save the library. This affects only the library data on disk and does not affect the library currently opened in memory, so the effects will not be seen until after the library is closed and reopened.

The session scoped reference library list is set through the application option, `link.use_session_ref_libs`. If the session scoped reference library list is not set, this application option has no effect.

If this application option is set to false, `save_lib` will not propagate the session scoped reference library list to the library. The library will retain its local reference library list on disk.

To determine the value of this application option, use the `get_app_option_value - name design.save_session_ref_libs` command.

See Also

- [save_lib](#)

design.session_scale_factor

Defines the scale factor for all libraries in the session.

Data Types

integer

Default `10000`

Description

This application option only takes effect if the application option `design.is_3dic_mode` is set to true; otherwise, its value is ignored. The value for this application option must be at

least the least common multiple or any multiple of the scale factors of all the libraries that you intend to open in the session.

The value for this application option can only be changed when no libraries are open.

Example 1: If your reference library has a scale factor of 2000 and your design has a scale factor of 4000, the *design.session_scale_factor* application option should be set to at least 4000. It can be even 8000 or 12000.

Example 2: If the reference library has a scale factor of 4000 and the design has a scale factor of 10000, the *design.session_scale_factor* application option should be set to at least 20000. It cannot be 10000.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)

design.set_group_port_name

Configures the naming scheme for the new pins created by the `group_cells` command.

Data Types

String

Default ""

Description

This application option configures the naming scheme for the new pins created by the `group_cells` command.

Supported values are combinations of the following keywords:

#netname

Name of the original net connected to the corresponding original pin.

#si

Driver instance name. If the driver is not an instance (top-level port), then *#si* value is "".

#sp

Driver pin name or port name.

#ri

Receiver instance name. If the receiver is not an instance (top-level port), then **#ri** value is "". If multiple instances are driven by the same driver, the first instance in alphabetical order is selected (even if this instance is not part of the new group).

#rp

Receiver pin name or port name. If multiple ports are driven by the same driver, the first port in alphabetical order is selected (even if the corresponding instance is not part of the new group).

Supported combinations are ([prefix] is an optional string): [prefix]#netname [prefix]#sp #si_#sp [prefix]#rp #ri_#rp

To determine the current value of this application option,
use `get_app_option_value -name design.set_group_port_name`.

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)
- [group_cells](#)

design.set_reparent_port_name

Configures the naming scheme for the new pins created by the `reparent_cells` command.

Data Types

String

Default ""

Description

This application option configures the naming scheme for the new pins created by the `reparent_cells` command.

Supported values are combinations of the following keywords:

#netname

Name of the original net connected to the corresponding original pin.

#si

Driver instance name. If the driver is not an instance (top-level port), then `#si` value is "".

`#sp`

Driver pin name or port name.

`#ri`

Receiver instance name. If the receiver is not an instance (top-level port), then `#ri` value is "". If multiple instances are driven by the same driver, the first instance in alphabetical order is selected (even if this instance is not part of the new parent).

`#rp`

Receiver pin name or port name. If multiple ports are driven by the same driver, the first port in alphabetical order is selected (even if the corresponding instance is not part of the new parent).

Supported combinations are ([`prefix`] is an optional string): [`prefix`]`#netname` [`prefix`]`#sp` `#si` `#sp` [`prefix`]`#rp` `#ri` `#rp`

When this application option has a non-default value, the `-port_prefix` option of the `reparent_cells` command is ignored.

To determine the current value of this application option, use below

`get_app_option_value -name design.set_reparent_port_name`.

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)
- [reparent_cells](#)

design.shape_use_aware_check_duplicates

Enables removal of duplicate shapes and vias based on its shape use.

Data Types

Boolean

Default `false`

Description

When set to *true*, `check_duplicates` command will consider shape use along with geometry while reporting and removing duplicate shapes and vias.

Two geometrically equivalent shapes and vias on same layer are not considered duplicates if their shape use are different when this app option is set to true.

design.skip_read_only

Controls how the `save_block -hier -label` command treat sub-blocks in read-only library.

Data Types

Boolean

Default false

Description

By default (*false*), the `save_block-hier -label` command save the entire block hierarchy into the specified label. It errors out if any of the sub-block libraries are read-only.

When this app option set to true, blocks in writable libraries are saved while blocks in read-only libraries are skipped.

See Also

- [save_block](#)
- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)

design.skip_uneditable_subblocks

Controls whether to skip saving uneditable subblocks with a new label, during "`save_block -hierarchical -label`"

Data Types

Boolean

Default false

Description

This app option when set to *false*, "save_block -hierarchical -label" will save uneditable subblock as well with a new label. This app option when set to *true*, "save_block -hierarchical -label" will skip saving uneditable subblock with new label.

design.sub_blocks_for_read

Controls whether the *open_block* and *link_block* command opens the sub-blocks in read-only mode.

Data Types

Boolean

Default false

Description

By default (*false*), the *open_block* and *link_block* command opens the sub-blocks in the same open-mode as its library.

When this option is set to *true*, the sub-blocks are always open in read-only mode even when the library is in edit mode or with the *open_block -ref_libs_for_edit* option.

See Also

- [open_block](#)
- [link_block](#)
- [lib_attributes](#) Description of the application defined lib attributes.
- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)

design.ts_disable_compile_command_restriction

Disables restriction on compile commands when fmeco mode is enabled with *-enable_user_initialization*.

Data Types

Boolean

Default false

Description

The *design.ts_disable_compile_command_restriction* application option allows execution of *compile_fusion* and *compile_logical* commands, even if *set_fm_eco_mode* had been set with the *-enable_user_initialization* option.

This option is meant as a workaround in case there are issues with the error *FAE-1019*, please always use *compile_ts_eco* command on this flow.

See Also

- [FAE-1019](#)

design.uniquify_naming_style

Specifies the naming style string to be used by "uniquify" and "uniquify_block" commands while naming new reference copy created during uniquification process. It is a block scoped app option.

Data Types

String

Default "%s_%d"

Description

This app option provides naming style string for commands "uniquify" and "uniquify_block" for naming new reference copy during uniquification process. Default value for app option is "%s_%d", where "%s" denotes name of existing reference and "%d" denotes incrementally generated integer. Value provided for this app option must contain single instance of substrings "%s" and "%d", and these two substrings should be present in that order. To include "%" in actual string use "%%".

Examples

Following is an example of valid input value:

```
prompt> set_app_options {design.uniquify_naming_style pre_%s_mid_%d_suf}  
-global  
prompt> set_app_options {design.uniquify_naming_style %s_%d_%%s} -global
```

Following are few examples of invalid input values:

```
prompt> set_app_options {design.uniquify_naming_style pre_%d} -global  
prompt> set_app_options {design.uniquify_naming_style %s_suf} -global  
prompt> set_app_options {design.uniquify_naming_style %d_%s} -global  
prompt> set_app_options {design.uniquify_naming_style %s_%d_%s} -global
```

See Also

- [uniquify](#)
- [uniquify_block](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

design.uniquify_skip_empty_modules

Skips empty module(s) uniquification during "uniquify" command execution. It is a global scoped app option.

Data Types

Boolean

Default false

Description

This app option prevents uniquification of empty modules during "uniquify" command. Default value of this app option is false, and when this app option is set to true empty modules are skipped from uniquification.

Examples

Example usage: prompt> set_app_options -name design.uniquify_skip_empty_modules -value true

See Also

- [uniquify](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

design.verbose_io

Controls whether each design I/O operation issues an information message.

Data Types

Boolean

Default false

Description

By default (*false*), design I/O operations do not issue an information message.

When this option is *true*, the following design I/O operations issue an information message: reading a design from a library, saving a design to a library, closing a design, and changing the open mode of a design.

For example,

```
Information: Opening design myLib:myDesign.design, mode edit. (NDM-065)
```

See Also

- [close_blocks](#)
- [open_block](#)
- [remove_blocks](#)
- [reopen_block](#)
- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)

design.verbose_uniquify

Controls whether each module uniquification issues an information message.

Data Types

Boolean

Default false

Description

By default (*false*), module unification does not issue an information message.

When this option is *true*, whenever a module is uniquified and a new module is generated, an information message is issued.

For example,

Information: In block 'A' uniquifying module 'm0_0' from 'm0' for cell 'inst1'. (NDM-055)

See Also

- [uniquify](#)
- [uniquify_block](#)

dft.bit_blast

The app option converts a bus signal from vector representation to multiple single bit scalar signals in files related to dft drc.

Data Types

boolean

Default false

Description

The app option modifies the bus signal into multiple single bit scalar signals in files generated by command dft_drc. Every bit signal will have a backslash '\\' as first character for escaping of the name. Following DRC generated files will be impacted by app option. File Naming Convention Verilog library file Ex:comboLib_xxx.xxx.v Protocol file Ex:design_name__xxx.xxx.spf Design netlist Ex:design_name_xxx.xxx.v CTL verilog Models Ex:ctl_name_xxx.xxx.v

Examples

```
prompt> set_app_options -name dft.bit_blast -value true
```

```
DRC netlist With bitBlast
module des_unit ( clk_st, rst_st, i_cs, \\i_addr[7], \\i_addr[6],
\\i_addr[5] , \\i_addr[4] , \\i_addr[3] , \\i_addr[2] , \\i_addr[1] ,
\\i_addr[0] , Y) ;
input  clk_st ;
input  rst_st ;
input  i_cs ;
input  \\i_addr[7] ;
input  \\i_addr[6] ;
input  \\i_addr[5] ;
input  \\i_addr[4] ;
input  \\i_addr[3] ;
input  \\i_addr[2] ;
input  \\i_addr[1] ;
input  \\i_addr[0] ;
output Y;
```

d

```

end module

DRC protocol file with bitBlast
STIL 1.0 {
    Design 2005;
}

Header {
    Title "Minimal STIL for design `des_unit`";
    Date "Mon Oct 11 16:52:10 2021";
    Source "Fusion Compiler S-2021.06-SP3-BETA";
}

Signals {
    "clk_st" In;
    "rst_st" In;
    "i_cs" In;
    "\\i_addr[7]" In;
    "\\i_addr[6]" In;
    "\\i_addr[5]" In;
    "\\i_addr[4]" In;
    "\\i_addr[3]" In;
    "\\i_addr[2]" In;
    "\\i_addr[1]" In;
    "\\i_addr[0]" In;
    "\\o_dout[7]" Out;
    "\\o_dout[6]" Out;
    "\\o_dout[5]" Out;
    "\\o_dout[4]" Out;
    "\\o_dout[3]" Out;
    "\\o_dout[2]" Out;
    "\\o_dout[1]" Out;
    "\\o_dout[0]" Out;
}

DRC design netlist without bitBlast
module des_unit (clk_st, rst_st, i_cs, i_addr, Y ) ;
input  clk_st ;
input  rst_st ;
input  i_cs ;
input  [7:0] i_addr ;
output Y;
end module

DRC protocol file without bitBlast
STIL 1.0 {
    Design 2005;
}
Header {
    Title "Minimal STIL for design `des_unit`";
    Date "Mon Oct 11 19:06:09 2021";
    Source "Fusion Compiler S-2021.06-SP3-BETA";
}

```

```

Signals {
  "clk_st" In;
  "rst_st" In;
  "i_cs" In;
  "i_addr[7]" In;
  "i_addr[6]" In;
  "i_addr[5]" In;
  "i_addr[4]" In;
  "i_addr[3]" In;
  "i_addr[2]" In;
  "i_addr[1]" In;
  "i_addr[0]" In;
  "o_dout[7]" Out;
  "o_dout[6]" Out;
  "o_dout[5]" Out;
  "o_dout[4]" Out;
  "o_dout[3]" Out;
  "o_dout[2]" Out;
  "o_dout[1]" Out;
  "o_dout[0]" Out;
}

```

See Also

- [set_dft_drc_configuration](#)
- [dft_drc](#)

dft.dft_drc_use_master_cell_name_for_lib_cell

Writes only master cell name for every lib cell.

Data Types

boolean

Default false

Description

There can be several variant of a master cell. Every variant will have a different cell name. This application enables the tool to write only the master cell name for every variant of lib cell. The app option only affect the cell names in netlist written for tetramax during dft_drc.

Examples

```

prompt> set_app_options -name
dft.dft_drc_use_master_cell_name_for_lib_cell -value true

```


See Also

- [dft_drc](#)

dft.dft_seq_sido_port_optimize

Optimizes port punching on fabric output in case of shared SI flow. Also reuse pipeline of same location and depth for cores/codec.

Data Types

Boolean

Default False

Description

This app option optimizes the port punching on the fabric output in scenarios where a shared scan-in (SI) flow is utilized. Additionally, it reuses the pipeline of the same location and depth for cores and codecs, enhancing efficiency and reducing redundancy. This feature is only supported for seq codec with split pipeline flow.

To check the current value of this application option, use the *get_app_option_value -name dft.dft_seq_sido_port_optimize* command.

dft.disable_toggle_dw_cell

Data Types

boolean

Default false

Description

Enabling this app option will create wrapper IP without toggle logic. Toggle logic of dedicated wrapper cell will be disabled when enabled this app option.

Examples

```
prompt> set_app_options -name dft.disable_toggle_dw_cell -value true
```

See Also

- [insert_dft](#)
- [preview_dft](#)

dft.enable_bit_blast_in_spf

Data Types

boolean

Default false

Description

Enabling this app option will add space before index bracket and after bus name. "\gbif_pdx_if[0]_plevel_gray [1]". example: \myspecial[0]_bus [0] for port \myspecial[0]_bus[0].

Examples

```
prompt> set_app_options -name dft.enable_bit_blast_in_spf -value true
```

See Also

- [write_test_model](#)

dft.enable_distributed_codec

Enables distributed codec feature in compression IP. This feature is supported for DFTMax, DFTMax Ultra and SEQ.

Data Types

boolean

Default false

Description

Using this app option user can enable distributed codec feature to achieve Best-In Class PPA for DFT Compression IP. This will enable the following features: - Splitting of DFTMax/DFTMax Ultra and SEQ IP Codec IP into multiple sub-units. - DFT sub-codecs hierarchy will be kept. - Tool will perform scan chains clustering. - Tool will place compressors and decompressors considering scan chain clustering.

Examples

```
prompt> set_app_options -name dft.enable_distributed_codec -value true
```

See Also

- [report_app_options](#)
- [set_app_options](#)

dft.enable_new_spec_persistency

Enables Test data persistency for forward compatibility.

Data Types

boolean

Default false

Description

This application option enables forward compatibility for Test data persistency so that user can load the data which is saved with newer version of the tool in the previous version of the tool.

Examples

```
prompt> set_app_options -name dft.enable_new_spec_persistency -value true
```

See Also

- [write_test_model](#)

dft.insertion_post_conditioning

Enables full in-compile DFT insertion support in RTLA

Data Types

boolean

Default true

Description

Enabling this app option will allow user to provide all DFT commands and will enable full DFT insertion support in RTLA. This would also disable the light weight DFT insertion flow.

See Also

- [preview_dft](#)
- [insert_dft](#)

dft.insertion_post_logic_opto

Enables early DFT insertion after `compile_fusion -logic_opto` within the brakpoint flow.

Data Types

boolean

Default false

Description

This application option enables the use of `insert_dft` in breakpoint flow, after a *compile_fusion -to logic_opto* was already performed.

Examples

A typical use mode will consist on this sequence of commands:

```
prompt> set_app_options -name dft.insertion_post_logic_opto -value true

prompt> compile_fusion -to logic_opto

prompt> <-- DFT SETUP HERE-->

prompt> preview_dft

prompt> insert_dft

prompt> compile_fusion -from initial_place -to initial_opto
```

See Also

- [preview_dft](#)
- [insert_dft](#)
- [compile_fusion](#)

dft.pulse_all_clocks

Enables pulsing of all dft clocks in the flow.

Data Types

boolean

Default true

Description

This application option enables pulsing of all dft clocks in the flow. If user wanted to pulse all dft clock without considering whether those are connected or dangling, *dft.pulse_all_clocks* should be set to true.

Examples

```
prompt> set_app_options -name dft.pulse_all_clocks -value true
```

See Also

- [write_test_protocol](#)

dft.put_negedge_head_pipeline_flops

Enables addition of negative edge triggered head pipeline flops and serial dftmax ultra regs.

Data Types

Boolean

Default False

Description

This app option changes the clock polarity of head pipeline scan data registers and serial registers of DFTMAX Ultra to trailing edged. By default FC will add posedge head pipeline flops.

To check the current value of this application option, use the *get_app_option_value -name dft.put_negedge_head_pipeline_flops* command.

See Also

- [insert_dft](#)
- [set_dft_signal](#)
- [set_scan_compression_configuration](#)

dft.scan.adaptive_pse_reclustering

Enables the adaptive PSE reclustering feature.

Data Types

Boolean

Default true

Description

This application option controls if the adaptive PSE reclustering feature is enabled. If this application option is set to true, tool will use the adaptive algorithm which is able to create and destroy PSE logic when PSE reclustering takes place.

The adaptive algorithm honors the PSE cluster bounding box maximum size as well as maximum number of loads per PSE group setup for a given NDM block, provided that block meets the following requirements:

- The PSE setup is defined before running *insert_dft*, and
- *insert_dft* runs.

For the PSE cluster bounding box maximum size, the application options *dft.scan.pse_cluster_box_x_size* and *dft.scan.pse_cluster_box_y_size* must be set to values greater than zero.

For the maximum number of elements per PSE group, the command *set_scan_configuration*, option *-pipeline_fanout_limit* must be defined.

If PSE reclustering is done on an imported netlist (that is, ASCII hand off) and not on an NDM block in which *insert_dft* ran, then the adaptive algorithm will be disabled.

This feature is automatically deactivated if PSE register stitching into scan chains is enabled via *set_pipeline_scan_enable_configuration -scan_pse_registers true*.

See Also

- [insert_dft](#)
- [set_pipeline_scan_enable_configuration](#)

dft.scan.add_lockup_between_equivalent_clocks

Enable lockup latch insertion between scan flops clocked by equivalent clocks.

Data Types

Boolean

Default false

Description

Enable lockup latch insertion between scan flops clocked by equivalent clocks. The equivalent clocks are defined by command *set_dft_equivalent_signals*.

Examples

The following example sets the user specification:

Enable the `dft.scan.add_lockup_between_equivalent_clocks` application option:

```
prompt> set_app_options -name  
dft.scan.add_lockup_between_equivalent_clocks -value true
```

See Also

- [set_dft_equivalent_signals](#)

dft.scan.amount_of_begin_repeater

Determines the number of repeater flops to be added at the beginning of the scan chains.

Data Types

Positive integer

Default -1

Description

This application option defines the number of repeater flip-flops to be added at the beginning of the scan chains during Scan Synthesis. Beginning repeater cells will be inserted only if either this or *dft.scan.amount_of_repeater* application option were specified. This application option will take precedence over *dft.scan.amount_of_repeater* if both were specified.

To determine the current value of this application option, use the *get_app_option_value -name dft.scan.amount_of_begin_repeater* command.

See Also

- [insert_dft](#)
- [set_scan_configuration](#)

dft.scan.amount_of_end_repeater

Determines the number of repeater flops to be added at the end of the the scan chains.

Data Types

Positive integer

Default -1

Description

This application option defines the number of repeater flip-flops to be added at the end of the scan chains during Scan Synthesis. End repeater cells will be inserted only if either this or *dft.scan.amount_of_repeater*s application option were specified. This application option will take precedence over *dft.scan.amount_of_repeater*s if both were specified.

To determine the current value of this application option, use the *get_app_option_value -name dft.scan.amount_of_end_repeater*s command.

See Also

- [insert_dft](#)
- [set_scan_configuration](#)

dft.scan.arch_sort_modes_for_architecting

In the flow where each test mode is scan architected independently enables presorting modes according to chain count in descending order.

Data Types

Boolean

Default false

Description

In the flow where each test mode is scan architected independently enables presorting modes according to chain count in descending order. Thus, first are architected modes with most number of chains. Usually such modes have shorter scan chains. Consequently, number of reconfiguration muxes is reduced.

The sorting of modes is redone for every partition. In case if *max_length* specification is used, *chain_count* is estimated for the purpose of the sorting.

This app option requires to set *dft.scan.enable_general_multi_mode_support* to true.

Examples

The following example sets the user specification:

Enable the *dft.scan.arch_sort_modes_for_architecting* application option:

```
prompt> set_app_options -name dft.scan.arch_sort_modes_for_architecting  
-value true
```

dft.scan.avoid_stitching_scan_cells_with_drc_from_scan_groups

Controls if stitching is avoided for cells with DFT DRC violations that are also inside a scan group definition.

Data Types

Boolean

Default false

Description

The default behavior of the tool is to stitch cells that are inside a scan group definition, even if they have DFT DRC violations.

This application option, when enabled, ensures that stitching is avoided for cells with DFT DRC violations which are also part of a scan group.

Examples

```
prompt> set_app_options -name  
dft.scan.avoid_stitching_scan_cells_with_drc_from_scan_groups -value  
true
```

See Also

- [dft_drc](#)
- [set_scan_group](#)

dft.scan.cg_trace_te_repeater

Allows for stitching of clock gate TE pins that are tied through repeaters and combinational logic.

Data Types

Boolean

Default false

Description

Clock gate TE pin stitching during insert_dft requires the TE pin to either be unconnected, or tied to logic 0 or logic 1. If repeaters, combinational logic cells, or MV cells are found connected to the clock gate TE pin, then the tool does not perform the stitching of the clock gate TE pin.

d

This application option, when set to true, enhances the clock gate TE pin stitching behavior so that TE pin stitching is not avoided when repeaters, combinational cells, and MV cells are present in the pre existing interconnection between the clock gate's TE pin and the tie-off point. The combinational or MV cell are allowed if one of their inputs is tied-off to logic 1 or logic 0.

This application option interacts with `dft_drc`, so it should be defined before `dft_drc` is called.

Examples

```
prompt> set_app_options -name dft.scan.cg_trace_te_repeaters -value true
```

See Also

- [insert_dft](#)
- [set_dft_signal](#)
- [dft_drc](#)

dft.scan.check_ctl_internal_clock_for_lockups

Enable checking CTL internal clocks for lockup latches calculation.

Data Types

Boolean

Default false

Description

Enable checking CTL internal clocks for lockup latches calculation.

Examples

The following example sets the user specification:

Enable the `dft.scan.check_ctl_internal_clock_for_lockups` application option:

```
prompt> set_app_options -name  
dft.scan.check_ctl_internal_clock_for_lockups -value true
```

dft.scan.connect_clock_gate_test_enables

Marks a design is ICG TE pin stitched

Data Types

Boolean

Default false

Description

Signs that the design has its ICG cell's TestEnable (TE) pins connected to the corresponding ScanEnable (SE), TestMode (TM) or equivalent DFT test signals, according to DFT specification. This procedure is called ICG TE stitching.

The value of this option is false if ICG TE stitching has not been performed yet.

The ICG TE stitching is performed during the execution of *insert_dft*. Once this state has been completed, this option is internally set to true, and it can be evaluated using command *get_app_option_value*.

If a design is imported to Fusion Compiler reading its Verilog netlist, user can sign that the design is ICG TE stitched by setting this Application Option to true using command *set_app_options*. Any further call to commands like *insert_dft* will skip its internal ICG TE stitching stage, because the ICG TE stitching is assumed to be ready.

Once this option is set to true value, because of internal ICG TE stitching or by explicit user setup in the UI, the option becomes read-only and cannot be set back to false using the UI.

Examples

The following example sets the state of the design as ICG TE stitching after loading a Verilog netlist:

```
prompt> read_verilog user_design.v
prompt> link
prompt> set_app_options -name dft.scan.connect_clock_gate_test_enables
        -value true
```

See Also

- [insert_dft](#)

dft.scan.enable_general_multimode_support

Enables scan architecting of each mode independently. When it is disabled scan chains are architected only for the shortest-chain mode and then these short chains are concatenated to create the other test modes.

Data Types

Boolean

Default true

Description

When this option is enabled, scan architecting processes each mode independently to meet its constraints. This allows user to apply distinct settings for each mode, such as clock / voltage / power mixing. Also, enabling this application option improves scan chain balancing.

When this option is disabled, scan chains are first architected for the shortest-chain mode and then concatenated to create the other test modes. In this case, clock-mixing, scan-cell exclusion and scan chain balancing cannot be performed for the long chain modes.

Examples

The following example sets the user specification:

Enable the `dft.scan.enable_general_multimode_support` application option:

```
prompt> set_app_options -name dft.scan.enable_general_multimode_support
        -value true
```

Define the test modes:

```
prompt> define_test_mode SComp_mode    -usage scan_compression
prompt> define_test_mode uncompress    -usage scan
prompt> define_test_mode scandump      -usage scan
prompt> reset_test_mode
prompt> set_scan_compression_configuration -inputs 16 -outputs 16
        -max_length 400 \\\
        -test_mode SComp_mode
prompt> set_scan_configuration -clock_mixing mix_clocks \\\
        -add_test_retiming_flops begin_and_end \\\
        -insert_terminal_lockup true \\\
        -test_mode SComp_mode
```

Scan Configuration:

```
prompt> set_scan_configuration -exclude_elements [get_cell -hier    \\\
        -filter "filter_criteria"] -test_mode {scandump}
prompt> set_scan_configuration -chain_count 16 -test_mode uncompress \\\
        -clock_mixing no_mix
prompt> set_scan_configuration -chain_count 1  -test_mode scandump \\\
        -clock_mixing mix_clocks
prompt> set_scan_configuration -clock_mixing no_mix -test_mode SComp_mode
```

See Also

- [define_test_mode](#)
- [get_cells](#)

- [reset_test_mode](#)
- [set_scan_configuration](#)
- [set_scan_compression_configuration](#)
- [set_wrapper_configuration](#)

dft.scan.enhanced_scandef_start_stop_selection

Enables physical based post processing of scan architecting results.

Data Types

Boolean

Default false

Description

Enables post processing of scan architecting results via physical based reordering of logical chain elements to help reduce wire length for interconnections between scandef chains and access pins.

The number of optimization passes used in the reordering process can be controlled using the `dft.scan.enhanced_scandef_start_stop_selection_max_runs` application option.

Examples

The following example sets the user specification:

```
Enable the dft.scan.enhanced_scandef_start_stop_selection application option:
```

```
prompt> set_app_options -name  
dft.scan.enhanced_scandef_start_stop_selection -value true
```

See Also

- [insert_dft](#)
- [optimize_dft](#)

dft.scan.enhanced_scandef_start_stop_selection_max_runs

Controls the number of optimization passes used during enhanced scandef start/stop selection post processing of scan architecting results

Data Types

int

Default 3

Description

Enhanced start/stop selection post processing of scan architecting results uses an iterative process. The number of iterations used by the algorithm is controlled by the `dft.scan.enhanced_scandef_start_stop_max_runs` application option. The specified number of iterations should be greater than zero. If the specified value is zero or negative, the iterative process defaults to 1 optimization pass.

This application option requires to set `dft.scan.enhanced_scandef_start_stop_selection` to true.

Examples

The following example sets the user specification:

```
Specify the value of the
dft.scan.enhanced_scandef_start_stop_selection_max_runs
application option:

prompt> set_app_options -name
dft.scan.enhanced_scandef_start_stop_selection_max_runs -value 1
```

See Also

- [insert_dft](#)
- [optimize_dft](#)

dft.scan.force_buffers_at_logical_chain_endpoints

Forces to insert a buffer at the logical scan chains endpoints.

Data Types

Boolean

Default false

Description

When this app option is enabled the tool will insert a buffer at the begin and end of each logical scan chain. The buffers inserted by this feature may be used as START or STOP of their respective ScanDEF chains during ScanDEF generation, similar to marker buffers feature.

See Also

- [insert_dft](#)

dft.scan.occ_spc_chain_arch_flat_flow

Enable OCC / SPC chain architecting with scan cells engine.

Data Types

Boolean

Default false

Description

Enable OCC / SPC chain architecting with scan cells engine. Thus, it allows different per mode set_scan_path specifications for OCC / SPC chains.

This app option requires to set dft.scan.enable_general_multi_mode_support to true.

Examples

The following example sets the user specification:

Enable the dft.scan.occ_spc_chain_arch_flat_flow application option:

```
prompt> set_app_options -name dft.scan.occ_spc_chain_arch_flat_flow  
          -value true
```

dft.scan.occ_spc_chain_arch_flow

Enable OCC / SPC chain architecting with scan cells engine.

Data Types

Boolean

Default false

Description

Enable OCC / SPC chain architecting with scan cells engine. Thus, it allows different per mode set_scan_path specifications for OCC / SPC chains.

This app option requires to set dft.scan.enable_general_multi_mode_support to true.

Examples

The following example sets the user specification:

Enable the `dft.scan.occ_spc_chain_arch_flow` application option:

```
prompt> set_app_options -name dft.scan.occ_spc_chain_arch_flow -value  
true
```

dft.scan.occ_streaming_end_of_longest_chain

Enable OCC chain stitching into longest compressed chain in DFTMax Ultra compression mode.

Data Types

Boolean

Default false

Description

Enable OCC chain stitching into longest compressed chain in DFTMax Ultra compression mode. This support is required incase of single input is given for DFTMax Ultra Compression configuration.

This app option requires to set `dft.scan.enable_general_multi_mode_support` to true.

Examples

The following example sets the user specification:

Enable the `dft.scan.occ_streaming_end_of_longest_chain` application option:

```
prompt> set_app_options -name dft.scan.occ_streaming_end_of_longest_chain  
-value true
```

dft.scan.place_mm_mux_at_load

Enables placing multimode reconfiguration mux at load. The default position (with disabled app option) is within DFT IP hierarchy.

Data Types

Boolean

Default false

Description

When this option is enabled, reconfiguration muxes are created at the same hierarchy of load scan flop of corresponding scan chain (depending which mode is architected first). Reconfiguration muxes will be placed nearby their scan load.

When this option is disabled reconfiguration muxes are created inside the DFT IP hierarchy.

Examples

The following example sets the user specification:

Enable the `dft.scan.place_mm_mux_at_load` application option:

```
prompt> set_app_options -name dft.scan.place_mm_mux_at_load -value true
```

dft.scan.prefer_preserve_shift_registers

Disables shift register splitting for scan architecting purposes.

Data Types

Boolean

Default false

Description

When this application option is enabled, the tool will not split shift registers during scan architecting. This might produce unbalanced scan chains.

See Also

- [set_scan_configuration](#)
-

dft.scan.pse_cluster_box_x_size

Determines the max width allowed for the X-axis of a pipeline scan enable cluster in the X-axis.

Data Types

Positive float

Default 0.0

Description

This application option defines the maximum width allowed to a Pipeline Scan Enable (PSE) cluster in the X-axis direction. This option is used only during SCAN Synthesis commands *preview_dft* and *insert_dft*.

For this application option to be considered, both *dft.scan.pse_cluster_box_x_size* and *dft.scan.pse_cluster_box_y_size* must have a setup value greater than zero. The default value of zero in any of them disables the PSE cluster bounding box analysis for PSE cluster assignment and creation.

During SCAN Synthesis PSE cluster analysis and assignment stages, when any SCAN element compatible with the PSE logic of a PSE cluster is processed, its physical location is compared to the current cluster's bounding box size: in case that the current group accepts the element but its X-axis bounding box size is bigger than the defined *dft.scan.pse_cluster_box_x_size* value, then the SCAN element is then discarded for being added to the current PSE cluster being processed.

The value of this application option represents multiples of the length input user units defined for the design. Default value for length user units is 1.00um.

This application option can interact with the command *set_scan_configuration*, option *-pipeline_fanout_limit*: both can limit the amount of elements being assigned by SCAN Synthesis commands to a single PSE cluster. The final PSE cluster size then will be limited when the *-pipeline_fanout_limit* of elements is reached for the group, or when there is no new SCAN elements available to be added to the PSE cluster that does not exceed the bounding box size defined by the application options *dft.scan.pse_cluster_box_x_size* and *dft.scan.pse_cluster_box_y_size*, whatever it happens first.

This application option affects PSE reclustering during *optimize_dft*, where the bounding box size will be honored if the value of the *dft.scan.adaptive_pse_reclustering* application option is true.

To determine the current value of this application option, use the *get_app_option_value -name dft.scan.pse_cluster_box_x_size* command.

See Also

- [insert_dft](#)
- [preview_dft](#)
- [report_user_units](#)
- [set_scan_configuration](#)
- [optimize_dft](#)

dft.scan.pse_cluster_box_y_size

Determines the max width allowed for the Y-axis of a pipeline scan enable cluster.

Data Types

Positive float

Default 0.0

Description

This application option defines the maximum width allowed to a Pipeline Scan Enable (PSE) cluster in the Y-axis direction. This option is used only during SCAN Synthesis commands *preview_dft* and *insert_dft*.

For this application option to be considered, both *dft.scan.pse_cluster_box_x_size* and *dft.scan.pse_cluster_box_y_size* must have a setup value greater than zero. The default value of zero in any of them disables the PSE cluster bounding box analysis for PSE cluster assignment and creation.

During SCAN Synthesis PSE cluster analysis and assignment stages, when any SCAN element compatible with the PSE logic of a PSE cluster is processed, its physical location is compared to the current cluster's bounding box size: in case that the current group accepts the element but its Y-axis bounding box size is bigger than the defined *dft.scan.pse_cluster_box_y_size* value, then the SCAN element is then discarded for being added to the current PSE cluster being processed.

The value of this application option represents multiples of the length input user units defined for the design. Default value for length user units is 1.00um.

This application option can interact with the command *set_scan_configuration*, option *-pipeline_fanout_limit*: both can limit the amount of elements being assigned by SCAN Synthesis commands to a single PSE cluster. The final PSE cluster size then will be limited when the *-pipeline_fanout_limit* of elements is reached for the group, or when there is no new SCAN elements available to be added to the PSE cluster that does not exceed the bounding box size defined by the application options *dft.scan.pse_cluster_box_x_size* and *dft.scan.pse_cluster_box_y_size*, whatever it happens first.

This application option affects PSE reclustering during *optimize_dft*, where the bounding box size will be honored if the value of the *dft.scan.adaptive_pse_reclustering* application option is true.

To determine the current value of this application option, use the *get_app_option_value -name dft.scan.pse_cluster_box_y_size* command.

See Also

- [insert_dft](#)
- [preview_dft](#)
- [report_user_units](#)
- [set_scan_configuration](#)
- [optimize_dft](#)

dft.scan.repeater_begin_prefix

Determines the naming style prefix used for scan inserted beginning repeater flops.

Data Types

String

Default DFT_BGN_RPTR_

Description

The prefix is to be used for all repeater flip-flops added at the beginning of the scan chains during Scan Synthesis. The prefix 'DFT_BGN_RPTR_' will be used if this application option was not specified.

Use the *dft.scan.amount_of_begin_repeater* or *dft.scan.amount_of_repeater* application options to define the number of repeaters cells to be added at the beginning of the scan chains.

To check the current value of this application option, use the *get_app_option_value -name dft.scan.repeater_begin_prefix* command.

See Also

- [insert_dft](#)
- [set_scan_configuration](#)

dft.scan.repeater_end_prefix

Determines the naming style prefix used for scan inserted end repeater cells.

Data Types

String

Default DFT_END_RPTR_

Description

The prefix is to be used for all repeater flip-flops added at the end of the scan chains during Scan Synthesis. The prefix 'DFT_END_RPTR_' will be used if this application option was not specified.

Use the *dft.scan.amount_of_end_repeater*s or *dft.scan.amount_of_repeater*s application options to define the number of repeaters cells to be added at the end of the scan chains.

To check the current value of this application option, use the *get_app_option_value -name dft.scan.repeater_end_prefix* command.

See Also

- [insert_dft](#)
- [set_scan_configuration](#)

dft.scan.scan_chain_report

Defines the name of the internally generated SCAN DFT report.

Data Types

string

Default scan_dft.rpt

Description

This application option defines the name of internally generated SCAN DFT report. By default, its name is *scan_dft.rpt*. If the name contains a folder, it must exist before use.

Examples

The following example creates a FOLDER if it does not exist and defines the name of the internally generated report to CUSTOM_SCAN.rpt on that folder.

```
prompt> sh mkdir -p FOLDER
prompt> set_app_options -name dft.scan.scan_chain_report -value
      "FOLDER/CUSTOM_SCAN.rpt"
```

See Also

- [insert_dft](#)
- [preview_dft](#)

dft.scan.scandef_analyze_scangroups

Enables ScanDEF generation of virtual scan groups (defined with *set_scan_group -serial_routed true*).

Data Types

Boolean

Default false

Description

When this app option is enabled the tool will consider virtual scan groups which are defined with *set_scan_groups -serial_routed true*, for ScanDEF generation. To do this the tool will trace between the ScanDataIn and ScanDataOut access pins, and then will generate a ScanDEF ordered segment with the internal elements found during the tracing.

The user can specify the leaf cells included in the pre-stitched group by using the *-include_elements* option of *set_scan_group* command. The tool will use this information to guide the internal tracing, improving the performance by avoiding potential wrong paths, and ensuring the correct elements are included in the ScanDEF ordered segment if there are multiple paths connecting both data access pins.

See Also

- [insert_dft](#)
- [set_scan_group](#)

dft.scan.scandef_based_opto

Enables ScanDEF-based optimization in incremental compile.

Data Types

Boolean

Default false

Description

When this option is enabled, compile incremental performs scan chain optimizations based on ScanDEF data, which by default includes both reordering and repartitioning.

dft.scan.scandef_based_repartition

Controls whether to apply repartitioning optimization when ScanDEF-based optimization is enabled.

Data Types

Boolean

Default true

Description

When this option is enabled, ScanDEF-based optimization during compile incremental performs repartitioning optimization. This option can be used to explicitly disable repartitioning without disabling other ScanDEF-based optimizations.

dft.scan.scandef_marker_cell_for_multiloads

Extends marker buffer support for multi-fanout SCANDEF segments.

Data Types

Boolean

Default true

Description

When both *dft.scan.scandef_use_marker_cells* and this application option are enabled, the tool will insert marker buffers as START/STOP points for stub elements with multiple loads.

See Also

- [insert_dft](#)
- [write_scan_def](#)

dft.scan.scandef_stop_before_lockups

If a stub chain ends with a lockup latch, sets the STOP point to the last chain cell's SI or hookup pin instead.

Data Types

Boolean

Default false

Description

When this application option is enabled, the tool will ensure no STOP point will be set on a lockup latch. Instead, the stub chain's STOP point will be the last cell before its last lockup latch. This avoids the possibility of back-to-back lockup latches post-reordering.

Also, it should be noted that CTL segments with terminal lockup latches will be included at the SCANDEF file.

See Also

- [insert_dft](#)
- [write_scan_def](#)

dft.scan.scandef_use_marker_cells

Enables the use of dedicated buffers/inverters at the SCANDEF chain ends to be used as START/STOP marks, to avoid promoting SCANDEF components for this purpose.

Data Types

Boolean

Default false

Description

When this application option is enabled, the tool will insert marker buffers at the beginning or end of a stub chain if a stub element is to be promoted as START/STOP. If the tool identifies a valid START/STOP element at the stub chain's end, it will be used regardless of this application option's value.

See Also

- [insert_dft](#)
- [write_scan_def](#)

dft.scan.set_false_path_on_lockups

Defines whether scan synthesis should set a false path constraint on the input pin of every lockup latch it automatically inserts.

Data Types

Boolean

Default false

Description

When this application option is *true*, scan synthesis automatically applies a *set_false_path -through* constraint on the input pin of every lockup latch it inserts in scan chains.

See Also

- [set_dft_signal](#)
- [set_scan_configuration](#)
- [set_false_path](#)

dft.scan.skip_scandef_segment_at_logical_chain_endpoints

Forces to promote the ScanDEF segments at the logical scan chain endpoints as START and STOP of their respective ScanDEF chains.

Data Types

Boolean

Default false

Description

When this app option is enabled the tool will skip the ScanDEF segment at the begin and end of each logical scan chain and will promote them as START or STOP of their respective ScanDEF chains. This feature will prevent that non-flops are used as START or STOP of logical scan chains. If the START or STOP of a logical scan chain is already going to be a sequential flop, the ScanDEF segment at the respective endpoint will not be skipped.

See Also

- [insert_dft](#)
- [write_scan_def](#)

dft.scan.test_keep_connected_scan_en

Controls whether existing connections to scan-enable pins are preserved during DFT insertion.

Data Types

Boolean

Default false

Description

During DFT insertion, the tool identifies the scan-enable pins of scan cells and scan cores that should be connected to the global scan-enable signal. These are known as scan-enable target pins.

By default, if a scan-enable target pin already has a connection, the tool will disconnect it to make the connection to a scan-enable signal. When this application option is set to *true*, the tool will skip that target pin in order to preserve its existing connection.

See Also

- [insert_dft](#)
- [set_dft_signal](#)

dft.scan.update_preexisting_icg_se_pin_connections

Allows for pre connected clock gate TE pins to be reconnected during insert_dft.

Data Types

Boolean

Default false

Description

TE pins of clock gates which are pre connected to ports/pins with DFT signals of type ScanEnable/TestMode and usage clock_gating/wrp_clock_gating are not reconnected during DFT insertion. This might lead to unexpected results if the usage of the DFT signal is not aligned with the actual fanout of the clock gate, for example, a clock gate whose TE pin is driven by a "wrp_clock_gating" usage DFT signal, while not driving any wrapper cells.

This application option, when enabled, ensures that the tool reconnects the TE pins of these clock gates to the correct usage DFT signal, namely clock_gating, or wrp_clock_gating, depending on the actual fanout of the clock gate.

Examples

```
prompt> set_app_options -name  
dft.scan.update_preexisting_icg_se_pin_connections -value true
```

See Also

- [insert_dft](#)
- [set_dft_signal](#)

dft.scan.use_core_clock_for_resync

Enables selecting an available CTL feedthrough clock to drive a resynchronization flip-flop when inserting resynchronization flip-flops for CTLs.

Data Types

Boolean

Default false

Description

During *insert_dft*, resynchronization flip-flops can be inserted for CTLs. The clock that is selected for the resynchronization flip-flop is typically the clock of the closest flip-flop.

When this application option is enabled, the clock that is used to drive the resynchronization flip-flop is the feedthrough output clock of the CTL block, if it exists.

Examples

```
prompt> set_app_options -name dft.scan.use_core_clock_for_resync -value  
true
```

See Also

- [insert_dft](#)
- [set_dft_signal](#)

dft.scan.use_resync_clock_for_repeater

Enables clock selection for repeater flip-flop to select the adjacent resync flip-flop's clock net.

Data Types

Boolean

Default false

Description

During *insert_dft*, repeater flip-flops can be inserted. The clock that is selected for the repeater flip-flop is typically the clock of the closest flip-flop.

When this application option is enabled, the clock that is used to drive the repeater flip-flop is the clock of an adjacent resync flip-flop, if available.

Examples

```
prompt> set_app_options -name dft.scan.use_resync_clock_for_repeater  
-value true
```

See Also

- [insert_dft](#)
- [set_dft_signal](#)

dft.scan.use_root_clock_for_ssg_lockup

Defines to which clock lockup latch inserted after virtual scan group is connected.

Data Types

Boolean

Default false

Description

When this appoption is disabled (default value) lockup latch inserted after virtual scan group is connected to the closest cell's clock. When this appoption is enabled and virtual scan group has -clock specification, inserted lockup latch uses this clock.

See Also

- [set_scan_group](#)

dft.shell_rtl_edit

The app option allows the use of RTL editing commands in TMM shell.

Data Types

boolean

Default True

Description

This app_option enables the usage of TMM RTL editing capabilities in customer flows. The RTL editing commands can leverage support of FC native filter and query. The following RTL editing commands are supported within this app_option :- add_connection delete_connection copy_connection add_instance uniquify_instance add_port load_rtl create_design set_rtl_parameter set_port_naming_style

d

After execution of each command the user may check the modifications using query commands.(As shown in examples below) Once all the desired RTL modification commands are done executing, the user may run the `modify_rtl` command to generate the desired output RTL. Since this `app_option` is on by default, so there is no need to specifically turn it on before using the above editing commands.

Examples

```
prompt> set_app_options -name dft.shell_rtl_edit -value true
prompt> add_port -cell i2/i23 -direction in {new_port1 new_port2}
1
prompt> get_pins i2/i23/new_port*
{i2/i23/new_port1, i2/i23/new_port2}
prompt> modify_rtl -output_directory RTL_outdir
```

dft.sms_allow_sub_server_shadow_server_architecture

To allow sub-server(core) -> shadow-server(subsystem) arch.

Data Types

Bool

Default 0

Description

The sub-server(core) -> shadow-server(subsystem) architecture is not allowed by default due to some known limitations. User can still proceed further by setting this app option.

To check the current value of this application option, use the `get_app_option_value -name dft.sms_allow_sub_server_shadow_server_architecture` command.

dft.sms_allow_sub_server_sub_server_architecture

To allow sub-server(core) -> sub-server(subsystem) arch

Data Types

Bool

Default 0

Description

The sub-server(core) -> sub-server(subsystem) architecture is not allowed by default due to some known limitations. User can still proceed further by setting this app option.

To check the current value of this application option, use the `get_app_option_value -name dft.sms_allow_sub_server_sub_server_architecture` command.

dft.sms_disable_memory_grouping_checks

Disable memory grouping checks. Checks based on grouping CSV to ensure the memories are connected to correct processor and wrappers.

Data Types

Bool

Default 0

Description

TMM generate some additional constraints for "validate_dft -check connectivity" from memory grouping CSV file to make sure processors, wrappers and memories are connected as specified in memory grouping CSV file. Users can disable generation of these constraints by using this app option in case of any incorrect violations or run time issues during validate_dft command.

To check the current value of this application option, use the *get_app_option_value -name dft.sms_disable_memory_grouping_checks* command.

dft.sms_dwsms_buf_mux_inst_pattern

Change the instance pattern to search mux instance in SMS wrapper which needs to be connected to scan enable signal.

Data Types

String

Default ""

Description

TMM connects some MUX instances inside SMS wrapper to Scan Enable signal. The MUX instances are tagged based on some fixed instance path pattern. User can use this app option to modify instance path pattern to search MUX instances.

To check the current value of this application option, use the *get_app_option_value -name dft.sms_dwsms_buf_mux_inst_pattern* command.

dft.sms_enable_memory_scan_enable_connection_through_sms_wrapper

Enable scan enable connection to mux instance within SMS wrapper.

Data Types

Bool

Default 0

Description

This app option enables connection of MUX instances inside SMS wrapper to Scan Enable signal.

To check the current value of this application option, use the *get_app_option_value -name dft.scan.test_mode_bar_port_naming_style* command.

dft.sms_new_directory_structure

Enable new directory flow which created different directories for different steps like plan_dft, generate_dft_dft, connetc_dft, validate_dft etc.

Data Types

Bool

Default 0

Description

TMM by default create all the internal files in current working directory. This app option enables TMM to create separate directories for each main command like plan_dft, generate_dft, connect_dft, validate_dft etc. and create files generated in those commands inside these directories.

To check the current value of this application option, use the *get_app_option_value -name dft.sms_new_directory_structure* command.

dft.sms_skip_release_compatibility_checks

Skips release compatibilty checks. Convert errors to warnings

Data Types

Bool

Default 0

Description

TMM use different tools like SpyGlass, EMBEDIT, COMPILERS etc. which needs to have a specific version to work properly with TMM. In case user still want to use different version which is not recommended users can set this app option and bypass the version

related errors. The prefix 'test_te_bar' will be used if this application option was not specified.

To check the current value of this application option, use the *get_app_option_value -name dft.sms_skip_release_compatibility_checks* command.

dft.test_allow_separate_mixing_in_wrapper_chains

Allows to have separate mixing settings for wrapper chains during scan chain architecting planning.

Data Types

Boolean

Default false

Description

Allows to have separate mixing settings (clocks, power, voltage) for wrapper chains during scan chain architecting planning. By default, wrapper chains use the same mixing settings as scan chains.

See Also

- [insert_dft](#)
- [set_scan_configuration](#)
- [set_wrapper_configuration](#)

dft.test_andgate_p_ref_for_dft

Data Types

string

Default ""

Description

With this option user can give an "AND" gate from a lib cell . Tool will use this AND gate while generating DFT IP rather than using behavioral AND gate.

Examples

```
prompt> set_app_options -name dft.test_andgate_p_ref_for_dft -value  
AND2X1_HVT
```


See Also

- [report_app_options](#)
- [set_app_options](#)
- [insert_dft](#)
- [preview_dft](#)

dft.test_bsd_default_bidir_delay

Defines the default switching time of bidirectional ports in a tester cycle for BSD applications.

Data Types

positive nonzero integer

Default 0.0 in Design Compiler non-topographical mode, Design Compiler topographical mode, Design Compiler NXT non-topographical mode, and Design Compiler NXT topographical mode

Variable is not supported in DC Explorer.

Description

This variable defines the default switching time of bidirectional ports in a tester cycle. The value is a positive real number in nanoseconds. This value is used when a test protocol is generated for ATPG.

For ATPG, `test_bsd_default_bidir_delay` defines the following for the design under test :
> The default time at which values are applied (driven) to the bidirectional ports in input mode during the parallel measure cycle.
> The time at which bidirectional ports are released (undriven) during the capture cycle.
> This variable affects only BSD applications.
The value of `test_bsd_default_bidir_delay` must be less than the output strobe time and greater than the capture clock active edge value.

The value of this variable affects the outcome of the `write_bsdI` command and the test program produced by the `write_testbench` commands. The `write_bsdI` command uses the value of this variable to check a design against the design rules of a bsd methodology.

Changing the value of `test_bsd_default_bidir_delay` modifies the content of the inferred or created test protocol generated by `write_bsdI`. To generate the test program, `write_testbench` uses the value of `test_bsd_default_bidir_delay` in the test protocol attached to the vector database (.vdb) file.

dft.test_bsd_default_delay

Defines the default time in a tester cycle to apply values to input ports for BSD applications.

Data Types

positive nonzero integer

Default 0.0

Description

This variable defines the default time in a tester cycle to apply values to input ports. The value is a positive real number in nanoseconds. This variable is used to generate a protocol file for ATPG.

For ATPG, *dft.test_bsd_default_delay* defines for the design under test the default time at which values are applied (driven) to the primary inputs for BSD applications. The value of *dft.test_bsd_default_delay* must be less than the output strobe time, and less than the capture clock edge value.

The value of this variable affects the outcome of the *generate_dft* command and the test program produced by the *write_testbench* command.

To generate the test program, *write_testbench* uses the value of *dft.test_bsd_default_delay* in the test protocol attached to the vector database (.vdb) file.

dft.test_bsd_default_strobe

Defines the default strobe time in a test cycle for output ports and bidirectional ports in output mode for BSD applications.

Data Types

positive nonzero integer

Default 95.0

Description

This variable defines the default strobe time in a test cycle for output ports and bidirectional ports in output mode for BSD applications. The value is a positive real number in nanoseconds. This value is used when a test protocol is generated for ATPG.

For ATPG, *dft.test_bsd_default_strobe* defines for the design under test the default time at which values are strobed on the primary output ports and bidirectional ports in output mode. The value of this variable must be less than or equal to the clock period value.

The value of this variable affects the outcome of the *generate_dft* command and the test program produced by the *write_testbench* command. The *generate_dft* command uses the value of this variable to check a design against the design rules of a bsd methodology.

Changing the value of *dft.test_bsd_default_strobe* modifies the content of the inferred or created test protocol generated by *generate_dft*. To generate the test program, *write_testbench* uses the value of *dft.test_bsd_default_strobe* in the test protocol attached to the vector database (.vdb) file.

dft.test_bsd_default_strobe_width

Defines the default strobe pulse width, which is the default time that specifies how long after invocation the strobe pulse needs to be held active for BSD applications.

Data Types

positive nonzero integer

Default 0.0

Description

This variable defines the default strobe pulse width, which is the default time that specifies how long after invocation the strobe pulse needs to be held active for BSD applications.

Changing the value of this variable modifies the content of the inferred or created test protocol generated by the *generate_dft* command. The *write_testbench* command uses the value of this variable (as stated in the test protocol attached to the design) to generate the test program for vector formats that support the notion of strobe width. For those formats that do not support it, the strobe width value specified is ignored.

The value of the *dft.test_bsd_default_strobe_width* variable is a real number value whose units are assumed to be nanoseconds, and the value must be a positive number. The sum of this value and the strobe time value must be less than or equal to the clock period value.

dft.test_bsd_enable_wrp_of

Defines the test mode which is used to define the wrp_of mode

Data Types

User must set the app option *dft.test_bsd_enable_wrp_of* with string value which is same as name of wrap of test mode.

Default 0.0

Description

This variable defines the wrp_of test mode for the BSD IP.

dft.test_bsd_make_private_instructions_public

Specifies whether private instructions should be included in BSDL and pattern files.

Data Types

Boolean

Default false

Description

This variable controls whether private instructions are included during BSDL and pattern file generation. Private instructions are created using the *-private* option of the *set_bsd_instruction* command.

When this variable is set to *false* (the default), the *write_bsd* and *write_testbench* commands ignore private instructions and their associated test data registers. This behavior is useful for creating BSDL and pattern files for external customers.

When this variable is set to *true*, the *write_bsd* and *write_testbench* commands include private instructions. The *write_bsd* command includes private instructions and any associated test data registers in the REGISTER_ACCESS section of the BSDL output file, and the *write_testbench* command includes private instructions in test pattern generation. This behavior is useful for creating BSDL and pattern files for internal testing.

To determine the current value of this variable, type *get_app_option_value -name dft.test_bsd_make_private_instructions_public*.

To see the current value of this variable, type

```
prompt> get_app_option_value -name dft.test_bsd_make_private_instructions_public
```

Examples

The following example generates BSDL and pattern files for external use:

```
prompt> set_app_options -name  
dft.test_bsd_make_private_instructions_public -value false  
  
prompt> write_testbench -o bsd_rtl_patterns_dcglke -test_mode  
all_dft_protocol -type {all}  
  
prompt> write_bsd -output des.bsd
```

The following example generates BSDL and pattern files for internal testing:

```
prompt> set_app_options -name  
dft.test_bsd_make_private_instructions_public -value true  
  
prompt> write_testbench -o bsd_rtl_patterns_dcglke -test_mode  
all_dft_protocol -type {all}  
  
prompt> write_bsd -output des_INTERNAL.bsd
```

To determine the current value of this variable, type *get_app_option_value -name dft.test_bsd_make_private_instructions_public*. For a list of *bsd* variables and their current values, type *get_app_options bsd*.

dft.test_core_wrap_add_ded_wrp_in_new_module

Data Types

boolean

Default false

Description

Enable support for grouping all wrapper cells inside newly created hierarchy in dedicated wrapper flow without UPF.

Examples

```
prompt> set_app_options -name  
dft.test_core_wrap_add_ded_wrp_in_new_module -value true
```

See Also

- [set_wrapper_configuration](#)
- [insert_dft](#)
- [preview_dft](#)

dft.test_core_wrap_analysis_via_clk_pin

Continue wrapper fanin analysis via clock pin of wrapper cells

Data Types

Boolean

Default False

Description

Enabling this app option makes sure that fanin analysis continues via the clock pin of shared wrapper cells.

To check the current value of this application option, use the *get_app_option_value -name dft.test_core_wrap_analysis_via_clk_pin* command.

See Also

- [set_wrapper_configuration](#)

dft.test_core_wrap_check_cgs_for_ioregs

Data Types

boolean

Default true

Description

Enabling this app option will process clock gating cells during wrapper analysis. Disabling this app option will ignore clock gating cell found during wrapper analysis.

Examples

```
prompt> set_app_options -name dft.test_core_wrap_check_cgs_for_ioregs  
-value true
```

See Also

- [report_app_options](#)
- [set_app_options](#)
- [insert_dft](#)
- [preview_dft](#)

dft.test_core_wrap_check_non_scan_for_wrapper_analysis

check non scan elements during fanin/fanout traversal of a port

Data Types

Boolean

Default False

Description

When this app option is enabled, tool adds a dedicated wrapper cell on the port when it encounters any of the below type of cells during wrapper analysis of a specific port:

OPTIONS

- * Non-scan cell (cells with scan_element false or dft_drc violated cells)
- * Scan excluded cell
- * ETM without CTL
- * BAM without CTL
- * Non-wrapper scan cell*

*non-wrapper scan cells are the potential shared wrapper cell candidates which are forced to be stitched in to non-wrapper scan chains by the user using set_wrapper_configuration -scan_cells{}

See Also

- [set_wrapper_configuration](#)

dft.test_core_wrap_dft_signal_per_partition

Enables support to use partition specific control signals for wrapper clock masking.

Data Types

Boolean

Default True

Description

Enabling this app option provides support to use partition specific control signals for wrapper clock masking.

To check the current value of this application option, use the *get_app_option_value -name dft.test_core_wrap_dft_signal_per_partition* command.

See Also

- [set_dft_signal](#)
- [set_wrapper_configuration](#)

dft.test_core_wrap_exclude_soft_core_pin_wrap

Enables support to exclude soft core boundary pins from dedicated wrapper insertion during fanout traversal for certain dft signals.

Data Types

Boolean

Default False

Description

Enabling this app option makes sure that soft core boundary pins present in the fanout cone of dft signal, of types: MasterClock, Reset, Constant, ScanEnable and TestMode are excluded from dedicated wrapper insertion.

To check the current value of this application option, use the *get_app_option_value -name dft.test_core_wrap_exclude_soft_core_pin_wrap* command.

See Also

- [set_dft_signal](#)
- [set_wrapper_configuration](#)

dft.test_core_wrap_map_hold_mux_equiv_gates

Data Types

boolean

Default false

Description

Enables tool to preserve the XOR and MUX structure of hold mux logic with toggle feature.

To enable hold mux for wrapper insertion below specification required.

set_wrapper_configuration -hold_mux enable

Examples

```
prompt> set_app_options -name dft.test_core_wrap_map_hold_mux_equiv_gates  
-value true
```

```
SH SEE ALSO  
set_wrapper_configuration(2)
```


dft.test_core_wrap_report_file

Enabling this app option generates a report for wrapper analysis performed in design.

Data Types

String (filename)

Default compile_wrp_report

Description

Enables user to write out wrapper analysis report generated during wrapper analysis. This report contains list of all ports wrapped by tool. This report also gives information regarding wrapper cells, clock used in wrapper connection, type of port(input/output), reason for wrapping by dedicated wrapper or shared wrapper. Example:

```
***** Test wrapper plan report Design : top
***** Reuse Threshold : 4 Wrapper Length : 5 Wrapper
Control Safe Wrapper Index Port Function Cell Type Cell Impl Value Clock Cell Name -----
----- 4 y output WC_S1 - INP 0 clk_st dd_a/f_reg
3 g output WC_S1 - INP 0 clk_st g_reg 2 e output WC_S1 - INP 0 clk_st dd_a/e_reg 1
f output WC_S1 - INP 0 clk_st dd_b/e_reg 0 b input WC_S1 - INP 0 clk_st dd_b/f_reg 0
b output WC_S1 - INP 0 clk_st dd_b/f_reg(*)i 0 x output WC_S1 - INP 0 clk_st dd_b/
f_reg(*)
```

Number of ports not wrapped : 72

MY_TM_1 MY_TM_2 SE WCLK a

To check the current value of this application option, use the *get_app_option_value -name dft.test_core_wrap_report_file* command.

See Also

- [insert_dft](#)
- [set_wrapper_configuration](#)
- [preview_dft](#)

dft.test_core_wrap_retain_dw_mux_structure

Preserves the mux-structure of the hold-logic and safe-state logic added inside the dedicated wrapper cell hierarchy till the end of insert_dft step.

Data Types

Boolean

Default False**Description**

Enabling this app option makes sure that the mux-structure is preserved for the hold-logic and safe-state logic(if safe-state is enabled) added inside the dedicated wrapper cell hierarchy by the end of `insert_dft`. By-default, the incremental LBO step run within `insert_dft` is free to map these logic to any type of gate like AND/OR/complex AOI gates. This mux-structure is retained only till `insert_dft` and further optimization steps by the subsequent `compile_fusion` commands are free to remap/optimize these gates in order to obtain maximum synthesis QoR. User can choose to add synthesis constraints(like `set_size_only`) to the muxes at the end of `insert_dft` to preserve them. The mux responsible for holding the data during capture cycle has a prefix '`wrp_hold_mux`' and the mux added for `safe_state` feature has a prefix '`safe_state_mux`'.

dft.test_core_wrap_sync_ctl_segment_length

Specifies the maximum length of CTL-modeled synchronizer register that can be included in wrapper chains.

Data Types

Integer

Default 10000**Description**

This variable specifies the maximum segment length of synchronizer registers that can be included in wrapper chains as shared wrapper cells during maximized reuse core wrapping flow. If a synchronizer register associated with a port exceeds this segment length, a dedicated wrapper cell is added to that port.

A synchronizer register is defined as a library cell register with a CTL model containing a single scan chain from scan data input to scan data output. Multibit registers are not affected by this.

CTL-modeled cells other than synchronizer registers always result in dedicated wrapper cells for their associated ports.

This functionality does not apply to synchronizer registers modeled using Liberty library constructs.

To check the current value of this application option, use the `get_app_option_value -name dft.test_core_wrap_sync_ctl_segment_length` command.

See Also

- [set_wrapper_configuration](#)

dft.test_core_wrp_upf_dw_mv_report

Enabling this app option generates UPF debug report in power aware dedicated wrapper insertion flow with file name given by user as app option value

Data Types

String (filename)

Default ""

Description

Enables user to debug the location of dedicated wrapper insertion in power aware wrapper analysis. A file would be written named "filename.rpt" in current working directory. Where filename is the string given by user through app option. To enable the power aware dedicated wrapper insertion. Below specification is required. set_wrapper_configuration -add_wrapper_cells_to_power_domains

Example:

```
***** Wrapper MV debug report Design : top
***** Type Port Driver Net Load UPF Reason ---- ---- ---- ---
---- --- Input a a dd_b/ctmi_7/A Illegal Net a is less on than both driver a and load
dd_b/ctmi_7/A Input a a dd_b/a dd_b/ctmi_7/A Legal Input a a dd_a/ctmi_6/A Illegal Net
a is less on than both driver a and load dd_a/ctmi_6/A Input a a dd_a/f_reg/EN Illegal
Net a is less on than both driver a and load dd_a/f_reg/EN Input a a dd_a/a dd_a/ctmi_6/
A Legal Input a a dd_a/a dd_a/f_reg/EN Legal Input a a a ctmi_5/A Legal Input a a a dd_b/
phfnr_buf_13/A Illegal Net a is less on than both driver a and load dd_b/phfnr_buf_13/
A Input a a dd_b/a dd_b/phfnr_buf_13/A Legal Input a a a dd_a/phfnr_buf_12/A Illegal
Net a is less on than both driver a and load dd_a/phfnr_buf_12/A Input a a dd_a/a dd_a/
phfnr_buf_12/A Legal
```

To set the app option `set_app_options -list { dft.test_core_wrp_upf_dw_mv_report mv_violation_debug_report}` command. App option will write file named mv_violation_debug_report.rpt

See Also

- [insert_dft](#)
- [set_wrapper_configuration](#)
- [preview_dft](#)

dft.test_ctl_internal_block_with_associated_port_internal_clk_pin

Support to add associated clock port in Internal block of CTL in case clocks are defined as associated_clocks

Data Types

Boolean

Default False

Description

Enabling this app option will write top level clock port of associated_internal_clocks as capture/launch clock in SI/SO internal block of the CTL. So that It can be intergrated by the top level. `set_dft_signal -view exist -type ScanMasterClock -port clk_st -associated_internal_clock mux_a/MUX_OUT -timing {45 55}` With App option as false

```
SI_18" { CaptureClock "mux_a/MUX_OUT" { LeadingEdge; } DataType ScanDataIn
{ ScanDataType Internal; } } "SO_18" { LaunchClock "mux_a/MUX_OUT" { LeadingEdge; }
DataType ScanDataOut { ScanDataType Internal; } }
```

With App option as true

```
SI_18" { CaptureClock "clk_st" { LeadingEdge; } DataType
ScanDataIn { ScanDataType Internal; } } "SO_18" { LaunchClock "clk_st" { LeadingEdge; }
DataType ScanDataOut { ScanDataType Internal; } }
```

To check the current value of this application option, use the `get_app_option_value -name dft.test_ctl_internal_block_with_associated_port_internal_clk_pin` command.

dft.test_default_delay

Defines the default time in a tester cycle to apply values to input/bidi ports.

Data Types

positive decimal

Default 0.0

Description

This application option defines the default time in a tester cycle to apply values to input or bidirectional ports. The value is a positive real number in nanoseconds. The default value is 0.0. This application option is used to generate a protocol file for ATPG.

For ATPG, `dft.test_default_delay` defines for the design under test the default time at which values are applied (driven) to the primary inputs or bidirectionals. The value of `dft.test_default_delay` must be less than the output strobe time, and less than the capture clock edge value.

The value of this application option affects the outcome of the *dft_drc* command and the test program produced by the *write_test* command. The *dft_drc* command uses the value of this application option to check a design against the design rules of a scan test methodology.

Changing the value of *test_default_delay* modifies the content of the inferred or created test protocol generated by *dft_drc* or *create_test_protocol*.

See Also

- [dft_drc](#)
- [write_test_protocol](#)

dft.test_default_period

Defines the length of a test vector cycle.

Data Types

positive integer

Default 100.0

Description

This application option defines the time duration of a test vector cycle. This value translates directly to the speed of application of the test vectors on automated test equipment (ATE). For example, a period of 100 nanoseconds describes ATE running at 10 MHz.

This application option controls the test clock period of the test protocol created by the *create_test_protocol* command. Test clocks defined with the *set_dft_signal* command must be defined using this period, except for free-running reference clocks defined with *set_dft_signal -type refclock*, which can be defined with any period value.

The value of the *dft.test_default_period* variable must be an integer whose units are assumed to be nanoseconds. The value must be a positive integer.

See Also

- [dft_drc](#)
- [set_dft_signal](#)
- [write_test_protocol](#)

dft.test_default_strobe

Defines the default strobe time in a test cycle for output ports and bidirectional ports in output mode.

Data Types

positive nonzero decimal

Default 40.0

Description

This application option defines the default strobe time in a test cycle for output ports and bidirectional ports in output mode. The value is a positive real number in nanoseconds. This value is used when a test protocol is generated for ATPG.

For ATPG, *dft.test_default_strobe* defines, for the design under test, the default time at which values are strobed on the primary output ports and bidirectional ports in output mode. The value of this application option must be less than or equal to the clock period value.

The value of this application option affects the outcome of the *dft_drc* command and the test program produced by the *write_test* command. The *dft_drc* command uses the value of this application option to check a design against the design rules.

Changing the value of *dft.test_default_strobe* modifies the content of the inferred or created test protocol generated by *dft_drc*.

See Also

- [dft_drc](#)
- [write_test_protocol](#)

dft.test_dft_log_file

Data Types

String (filename)

Default ""

Description

App option stores the complete logs of wrapper analysis in filename given by user as app option value.

```
prompt> set_app_options -list { dft.test_dft_log_file compile_dft_log }
```

See Also

- [insert_dft](#)
- [set_wrapper_configuration](#)

dft.test_disable_toggle_for_reset_shared_wrp

Shared wrapper cells identified during fanin traversal of reset pin will not have toggle logic.

Data Types

Boolean

Default False

Description

During wrapper analysis when hold_mux is enabled and signal wrp_td_test is specified, tool adds hold mux and toggle XOR logic for all shared wrapper cells. However with this app option enabled, toggle logic is not added for any shared wrapper cells which are identified in fanin of reset pin.

To check the current value of this application option, use the *get_app_option_value -name dft.test_disable_toggle_for_reset_shared_wrp* command.

See Also

- [set_wrapper_configuration](#)

dft.test_drc_integrity_checks

Enable integrity checks in DFT-DRC.

Data Types

boolean

Default true

Description

This application option enables the Integrity Checks mentioned Below

- 1) Error msgs for violation in read_netlist
- 2) Crosscheck ports against module ports
- 3) Crosscheck ports against ports on top module
- 4) Error msgs for ports of type power and ground found in design
- 5) Error msg for pins with dft specifications but doesn't exist on design (pins sometimes get modified in FC)

Examples

```
prompt> set_app_options -name dft.test_drc_integrity_checks -value true
```

See Also

- [set_dft_drc_configuration](#)

dft.test_icg_n_ref_for_dft

Defines the integrated clock-gating cell to be used for implementing return-to-one clock gating in the DFT logic.

Data Types

string

Default ""

Description

When DFT Compiler inserts clock-gating cells as a part of the DFT logic, this variable constrains the insertion process to use the specified library cell to gate any clock signal that has a return-to-one (RTO) waveform. The name must match an available cell in the target libraries, and this cell must be an integrated clock-gating cell.

Specify the cell name without the library name. If the library name is included, the specification is ignored without warning or error.

The cell specified by this variable is used only for the initial logic construction. After insertion, subsequent design optimizations can resize the cell (if allowed by the library cell attributes).

This variable applies to DFT logic inserted by TMM for BSD.

By default, when this variable is not set, TestMAX will gate clock signals using regular combinational gates.

dft.test_icg_p_ref_for_dft

Defines the integrated clock-gating cell to be used for implementing return-to-zero clock gating in the DFT logic.

Data Types

string

Default ""

d

Description

When DFT Compiler inserts clock-gating cells as a part of the DFT logic, this variable constrains the insertion process to use the specified library cell to gate any clock signal that has a return-to-zero (RTZ) waveform. The name must match an available cell in the target libraries, and this cell must be an integrated clock-gating cell.

Specify the cell name without the library name. If the library name is included, the specification is ignored without warning or error.

The cell specified by this variable is used only for the initial logic construction. After insertion, subsequent design optimizations can resize the cell (if allowed by the library cell attributes).

This variable applies to DFT logic inserted by TMM for BSD.

By default, when this variable is not set, TestMAX will gate clock signals using regular combinational gates.

dft.test_power_aware_dw_insertion_near_multi_input_cells

Prefer nets post buffer/inverter/level-shifter/isolation cells away from port for inserting dedicated wrapper cell in power aware dedicated wrapper cell flow

Data Types

Boolean

Default False

Description

Enabling this app option makes tool prefer nets near ports after skipping buffer/inverter/level-shifter/isolation cells. Tool will try to minimize number of dedicated wrapper cell even after skipping above mentioned cells

This app option takes effect only when `set_wrapper_configuration -add_wrapper_cells_to_power_domains enable` configuration is set and `dft.test_power_aware_dw_insertion_near_port` is false

To check the current value of this application option, use the `get_app_option_value -name dft.test_power_aware_dw_insertion_near_multi_input_cells` command.

See Also

- [insert_dft](#)
- [set_wrapper_configuration](#)

dft.test_power_aware_dw_insertion_near_port

Prefer nets near port for dedicated wrapper cell insertion in power aware dedicated wrapper cell flow

Data Types

Boolean

Default True

Description

Enabling this app option makes tool prefer nets near ports for adding dedicated wrapper cell insertion. Disabling the app option makes tool prefer nets away from ports for adding dedicated wrapper cell insertion.

This app option takes effect only when `set_wrapper_configuration -add_wrapper_cells_to_power_domains enable` configuration is set

To check the current value of this application option, use the `get_app_option_value -name dft.test_power_aware_dw_insertion_near_port` command.

See Also

- [insert_dft](#)
- [set_wrapper_configuration](#)

dft.test_set_test_point_report_file_name

Sets the path of test point report file within working directory.

Data Types

string

Default ""

Description

This application option enables test point report file to be generated at specified location during `preview_dft` and `insert_dft`. This option is succeeded by application option `dft.tp_set_report_file_name`

Examples

A typical use mode will consist on this sequence of commands:

```
prompt> set_app_options -name dft.test_set_test_point_report_file_name  
-value test_point_reports/design.test_point.rpt  
  
prompt> <-- TEST POINT AND DFT SETUP HERE-->  
  
prompt> run_test_point_analysis  
  
prompt> insert_dft
```

See Also

- [set_testability_configuration](#)
- [run_test_point_analysis](#)

dft.test_soc_core_wrap_allow_multibit_ioregs

Data Types

boolean

Default true

Description

App option controls whether multibit scan cells found during wrapper analysis can be reused as shared wrapper cell. Enabling app option will allow tool to use multi-bit cell as shared wrapper cell. Disabling the app option will add dedicated wrapper cell to the port for which MB cell is found during wrapper analysis.

Examples

```
prompt> set_app_options -name  
dft.test_soc_core_wrap_allow_multibit_ioregs -value true
```

See Also

- [report_app_options](#)
- [set_app_options](#)

dft.test_soc_low_power_extest_support

Controls the low-power extest feature to minimize power consumption during extest mode

Data Types

boolean

Default true

Description

To minimize power consumption in EXTEST mode, all core scan chain cells are loaded with logic 0. This reduces toggle activity inside the block during outward-facing (EXTEST) modes. To accomplish this, additional logic is inserted that keeps core chains shifting constant data. When disabled the AND gate at the beginning of core chains and the OR gates to control scan-enable connections to make sure they shift on corresponding test modes for all chains are not added

dft.test_ultra_put_load_ser_lul

Enables feature of adding lockup latch in the Ultra Decompressor IP. This feature is supported only for DFTMax Ultra.

Data Types

boolean

Default false

Description

Using this app option user can enable feature of adding LockUp Latch in the Ultra Decompressor IP. This will put lockup latch output of the Ultra Load Compressor if it is missing.

Examples

```
prompt> set_app_options -name dft.test_ultra_put_load_ser_lul -value  
true
```

See Also

- [report_app_options](#)
- [set_app_options](#)

dft.test_update_spf_for_non_dft_port

This feature helps to update non-dft ports created post 'create_test_protocol' in the STIL protocol file.

Data Types

boolean

Default false

Description

If any non-dft ports are created after create_test_protocol. Then to update the changes in spf you need to make this option as true.

Examples

```
prompt> set_app_options -name dft.test_update_spf_for_non_dft_port -value
true
```

See Also

- [report_app_options](#)
- [set_app_options](#)

dft.test_write_one_hot_decoding_in_constraints

App option to dump one hot encoding is TMM output constraints file in integration flow.
set_dft_configuration -mode_decoding_style one_hot

Data Types

boolean

Default false

Description

In general multimode flow: In RTL, TMM will be building Test Modes decoding logic inside codec / fabric IP. In general OCC/SPC flow with Fabric IP present, one-hot TM decoding pin on Fabric IP will be used, like: define_test_mode sub0_sub1_scan -usage scan -encoding { U_DFTFabricIP/sub0_sub1_scan_tm 1 U_DFTFabricIP/sub0_sub1_sccomp_tm 0 U_DFTFabricIP/sub2_scan_tm 0 U_DFTFabricIP/sub2_sccomp_tm 0 U_DFTFabricIP/sub3_scan_tm 0 U_DFTFabricIP/sub3_sccomp_tm 0 U_DFTFabricIP/sub4_core1_scan_tm 0 U_DFTFabricIP/sub4_core1_sccomp_tm 0 U_DFTFabricIP/sub4_scan_tm 0 U_DFTFabricIP/sub4_sccomp_tm 0 U_DFTFabricIP/top_scan_tm 0 U_DFTFabricIP/top_sccomp_tm 0 U_DFT_TOP_IP_1/top_scan 0 U_DFT_TOP_IP_1/top_sccomp 0 U_DFT_TOP_IP_2/top_scan 0 U_DFT_TOP_IP_2/top_sccomp 0 } -target { top } Since these decoded Test Modes signals generated in RTL are already one-hot encoding, user could leverage this, to avoid DC/FC to generate redundant and unnecessary Test Modes decoding logic again in design, which will also cause DC long run time if there is a lot of modes defined.

Above requirement could be achieved by below option (both for DC and FC):
set_dft_configuration -mode_decoding_style one_hot This command is written in TMM generated constraints file if this app option is specified. set_app_options -name dft.test_write_one_hot_decoding_in_constraints -value true option does not work if new general occ-spc stitching flow is not enabled.

dft.test_wrp_add_ded_cg_branch

During wrapper analysis, the tool adds a dedicated wrapper cell to ports/pins associated with the functional enable of clock-gating cells.

Data Types

Boolean

Default False

Description

Enabling this app option makes sure that any clock-gating cell which is found during wrapper analysis of port and is connected through functional enable pin, will have a dedicated wrapper cell

To check the current value of this application option, use the *get_app_option_value -name dft.test_wrp_add_ded_cg_branch* command.

See Also

- [set_wrapper_configuration](#)

dft.test_wrp_add_ded_ctl_branch

During wrapper analysis, the tool adds a dedicated wrapper cell to ports/pins associated with the pins of CTL cells.

Data Types

Boolean

Default False

Description

Enabling this app option makes sure that any CTL cell pin which is found during wrapper analysis of port will have a dedicated wrapper cell

To check the current value of this application option, use the *get_app_option_value -name dft.test_wrp_add_ded_ctl_branch* command.

See Also

- [set_wrapper_configuration](#)

dft.test_wrp_dw_location_connected_net_check

Controls whether only nets connected to port in specific location will be used for UPF aware dedicated wrapper cell insertion flow

Data Types

boolean

Default false

Description

This app option controls whether only nets connected to port in the provided location will be used in UPF aware dedicated wrapper cell insertion flow.

See Also

- [set_boundary_cell](#)
- [set_dft_location](#)

dft.test_wrp_gate_cells_connect_te

Add clock masking logic to Test Enable (TE) pins of clock gating cells

Data Types

Boolean

Default True

Description

When clock masking is enabled via -gate_cells of set_wrapper_configuration command, Test Enable (TE) pin of clock-gating will be connected to the appropriate clock masking signal. If this app option is false, the Test Enable (TE) pin will not be gated.

To check the current value of this application option, use the *get_app_option_value -name dft.test_wrp_gate_cells_connect_te* command.

See Also

- [set_wrapper_configuration](#)

dft.test_wrp_gate_scan_cell_only

Add clock masking logic to clock gating cells driving only scan cells

Data Types

Boolean

Default False

Description

When clock masking is enabled via `-gate_cells` of `set_wrapper_configuration` command, clock-gating cells driving wrapper cells or a combination of wrapper and scan cells will not be gated by their appropriate clock masking signals.

To check the current value of this application option, use the `get_app_option_value -name dft.test_wrp_gate_scan_cell_only` command.

See Also

- [set_wrapper_configuration](#)

dft.test_wrp_ignore_feedthrough_path

During dedicated wrapper insertion, do not wrap feedthrough paths for input and output ports

Data Types

Boolean

Default False

Description

Enabling this app option will tool ignore feedthrough paths between ports for UPF aware dedicated wrapper cell insertion

To check the current value of this application option, use the `get_app_option_value -name dft.test_wrp_ignore_feedthrough_path` command.

See Also

- [set_wrapper_configuration](#)

dft.test_wrp_new_hold_mux_arch_support

Enable the support for new hold mux architecture

Data Types

Boolean

Default False

Description

Enabling this app option makes the hold mux logic insertion before scan stitching and Scan Enable connections are made by SCAN engine.

To check the current value of this application option, use the *get_app_option_value -name dft.test_wrp_new_hold_mux_arch_support* command.

See Also

- [insert_dft](#)
- [set_dft_signal](#)
- [set_wrapper_configuration](#)

dft.test_wrp_new_power_domain_aware_dw_insertion_flow

New Power Domain aware Dedicated Wrapper insertion flow

Data Types

Boolean

Default True

Description

During wrapper analysis when dedicated wrapper is to be added on a port, tool will try to find legal or fixable locations for dedicated wrapper cells depending where the user has specified (closer to core or closer to port). This will work when *-add_wrapper_cells_to_power_domains enable* is specified in *set_wrapper_configuration*

To check the current value of this application option, use the *get_app_option_value -name dft.test_wrp_new_power_domain_aware_dw_insertion_flow* command.

See Also

- [set_wrapper_configuration](#)

dft.test_wrp_power_domain_aware_dw_wrp_clk

Enable Power Domain aware DW clock selection from defined wrp clocks

Data Types

Boolean

Default False

Description

When this app option is enabled, tool picks a power aware wrp_clock for that dedicated wrapper cell, amongst the multiple wrp_clock(s) defined by the user. User must set *-use_system_clock_for_dedicated_wrp_cells disable* in set_wrapper_configuration for this feature to work

To check the current value of this application option, use the *get_app_option_value -name dft.test_wrp_power_domain_aware_dw_wrp_clk* command.

See Also

- [set_dft_signal](#)
- [set_wrapper_configuration](#)

dft.test_wrp_remove_violated_wrp_cell_restr

Remove wrapper restrictions and dcAttrs from violated wrapper cells.

Data Types

Boolean

Default True

Description

Enabling this app option makes sure that the wrapper restrictions and dcAttrs gets removed from violated wrapper cells. The app option is only applicable for DFT precompile flow.

To check the current value of this application option, use the *get_app_option_value -name dft.test_wrp_remove_violated_wrp_cell_restr* command.

See Also

- [set_wrapper_configuration](#)

dft.test_wrp_remove_wrapper_control_decoding_logic

Ensures the wrapper control logic is not gated by test mode encoding logic

Data Types

Boolean

Default False

Description

Enabling this app option makes sure that wrapper control logic is not gated by test mode encoding logic

To check the current value of this application option, use the *get_app_option_value -name dft.test_wrp_remove_wrapper_control_decoding_logic* command.

See Also

- [insert_dft](#)
- [set_wrapper_configuration](#)

dft.test_wrp_shell_netlist_generation_only_flow

Enable shell netlist flow to generate shell netlist using verilog writer rather than removing non extest logic from the design

Data Types

Boolean

Default False

Description

Enabling this app option will enable flow to generate shell netlist using verilog writer which generates shell netlist excluding non extest logic in the design. Without enabling this app option tool will modify the current design to remove non extest logic. The shell netlist generated will be in current directory with top module name with suffix `_extest_shell.v`. Enabling this flow should reduce the runtime for generating shell netlist

To check the current value of this application option, use the *get_app_option_value -name dft.test_wrp_shell_netlist_generation_only_flow* command.

See Also

- [insert_dft](#)
- [create_dft_netlist](#)

dft.test_wrp_soft_core_abutted_flow

Enables processing only partitions defined for soft core. This app option stops processing top level

Data Types

Boolean

Default False

Description

Enabling this app option makes sure that top level processing is skipped and partition level soft core processing is done. This app option is particularly for enabling wrapper insertion for soft cores and not processing in top.

To check the current value of this application option, use the *get_app_option_value -name dft.test_wrp_soft_core_abutted_flow* command.

See Also

- [insert_dft](#)
- [set_dft_signal](#)
- [set_wrapper_configuration](#)

dft.tp_allow_adv_clock_for_tpe

Enables TestMAX Advisor analysis for test points specified using *set_test_point_element* command.

Data Types

boolean

Default false

Description

Enables TestMAX Advisor based clock selection for test points specified using *set_test_point_element* command.

Examples

```
prompt> set_app_options -name dft.tp_allow_adv_clock_for_tpe -value true
```

See Also

- [set_test_point_element](#)
- [run_test_point_analysis](#)

dft.tp_allow_dw_collapsing

Enables test points inside designware instances.

Data Types

boolean

Default false

Description

This application option enables test points inside designware instances, It is applicable to user defined test points *set_test_point_element* and TestMAX Advisor based test points using *set_testability_configuration*.

Examples

```
prompt> set_app_options -name dft.tp_allow_dw_collapsing -value true
```

See Also

- [set_testability_configuration](#)
- [run_test_point_analysis](#)

dft.tp_allow_logical_driver

Allows test points to be inserted at hierarchical pins.

Data Types

boolean

Default true

Description

Allows TestMAX Advisor based test points to be inserted at hierarchical pins. By default test points are inserted at pins of leaf cells.

Examples

```
prompt> set_app_options -name dft.tp_allow_logical_driver -value false
```

See Also

- [set_testability_configuration](#)
- [run_test_point_analysis](#)

dft.tp_analyze_false_path

analyze false path constraints in test points

Data Types

boolean

Default false

Description

This application option analyzes false path constraints in test points during *run_test_point_analysis*.

Target should be `multicycle_paths`

Examples

A typical use mode will consist on this sequence of commands:

```
prompt> set_app_options -name dft.tp_analyze_false_path -value true
```

```
prompt> set_testability_configuartion -target {multicycle_paths}  
-clock_signal clk -control_signal TM
```

```
prompt> preview_dft
```

```
prompt> run_test_point_analysis
```

```
prompt> insert_dft
```

See Also

- [set_testability_configuration](#)
- [run_test_point_analysis](#)

dft.tp_apply_scan_wrap_for_memory_without_test_model

scan_wrap constraints of memory cells are not generated for advisor when CTL is not provided.

Data Types

boolean

Default false

Description

To enable generation of scan_wrap constraints on memory cells for advisor when CTL is not provided. There are two valid values (multiple values are not supported):

false: scan_wrap constraints of memory cells for advisor will not be generated.

true: scan_wrap constraints of memory cells for advisor will be generated.

Examples

In the following example, the test point will be generated:

```
prompt> set_app_options -name  
dft.tp_apply_scan_wrap_for_memory_without_test_model -value true
```

See Also

- [run_test_point_analysis](#)

dft.tp_block_mcp_fp_at_source

Enables new spyglass feature for multicyclepath and false test points

Data Types

bool

Default false

Description

Enables new spyglass feature for multicyclepath and false test points of multicycle_paths target

Examples

```
prompt> set_app_options -name dft.tp_block_mcp_fp_at_source -value true  
prompt> set_app_options -name dft.tp_block_mcp_fp_at_source -value false
```

See Also

- [run_test_point_analysis](#)
- [preview_dft](#)
- [insert_dft](#)

dft.tp_ctl_aware_analysis

Enables CTL model aware test point analysis

Data Types

boolean

Default false

Description

Enables CTL model aware test point analysis Improves test point analysis on CTL module blocks

Examples

```
prompt> set_app_options -name dft.tp_ctl_aware_analysis -value true
```

See Also

- [set_testability_configuration](#)
- [run_test_point_analysis](#)

dft.tp_do_not_generate_force_no_scan_constraint

Disables the force_no_scan SGDC constraint generation for TestMAX Advisor during run_test_point_analysis.

Data Types

boolean

Default false

Description

This application option disables the force_no_scan SGDC constraint generation for TestMAX Advisor during run_test_point_analysis. This option can be used, if the number of the specified set_test_point elements is quite big and it TestMAX Advisor takes more time to parse the SGDC during test point analysis. If the objects for

d

`set_test_point_element` are the result of any wildcard patterns, the same pattern can be supplied directly to TestMAX Advisor to achieve better runtime.

Examples

```
prompt> echo "\n" \
  "current_design my_design_top\n" \
  "define_macro -macro NON_SCAN_FLOPS -type { FLIP_FLOP_OUT } -filter_in
  {"*_NON_SCAN_FLOPS_*" "*pattern_A*" } -ignore_boundary_for_wildcard
  "\n" \
  "define_macro -macro NSF_REGS -type { NON_SCAN_FLOPS } -filter_in
  {"*_pattern3*" } -ignore_boundary_for_wildcard\n" \
  "force_no_scan -type NSF_REGS " > [pwd]/udtp_for_adv.sgdc
prompt> echo "read_file -type sgdc [pwd]/udtp_for_adv.sgdc" >
  sg_option.tcl
prompt> set_testability_configuration -sg_command_file
  [pwd]/sg_option.tcl
prompt> set_app_options -name
  dft.tp_do_not_generate_force_no_scan_constraint -value true
```

See Also

- [set_test_point_element](#)
- [set_testability_configuration](#)
- [run_test_point_analysis](#)

dft.tp_enable_aux_models

Enables auxillary verilog models for test point analysis.

Data Types

boolean

Default true

Description

This application option enables the use of auxillary models available in test point analysis during *run_test_point_analysis*.

Examples

A typical use mode will consist on this sequence of commands:

```
prompt> set_app_options -name dft.tp_enable_aux_models -value false

prompt> <-- TEST POINT AND DFT SETUP HERE-->

prompt> run_test_point_analysis
```

```
prompt> preview_dft
```

```
prompt> insert_dft
```

See Also

- [set_testability_configuration](#)
- [run_test_point_analysis](#)

dft.tp_enable_clock_gating

Enables clock-gating logic (can also specify its type) to be added for test point data controllers.

Data Types

string

Default false

Description

Enables clock-gating logic (can also specify its type) to be added for test point data controllers.

Examples

```
prompt> set_app_options -name dft.tp_enable_clock_gating -value true
prompt> set_app_options -name dft.tp_enable_clock_gating -value
        clock_gating_cell
prompt> set_app_options -name dft.tp_enable_clock_gating -value and_gate
```

See Also

- [set_testability_configuration](#)

dft.tp_enable_clock_tapping_points

Enables local clock connections from tapping points of the nearest clock path for test point registers.

Data Types

bool

Default false

Description

Enables local clock connections from tapping points of the nearest clock path for test point registers.

The option `dft.tp_enable_clock_tapping_points` doesn't work with `dft.tp_enable_clock_gating` as the aim of enabling tapping point for test points is to reuse the existing clock-paths in the design.

When user-specified clock signal is present, tool will search for clock tap points only when the tool generated(from TestMAX Advisor) clock is equal to user-specified clock signal.

Tool will only search for tap points on test points whose root clock is found.

Examples

```
prompt> set_app_options -name dft.tp_enable_clock_tapping_points -value true
prompt> set_app_options -name dft.tp_enable_clock_tapping_points -value false
```

See Also

- [run_test_point_analysis](#)
- [preview_dft](#)
- [insert_dft](#)

dft.tp_enable_mv_checks

Enables power aware test point insertion.

Data Types

string

Default clock loation_not_fixable

Description

To enable the power aware test point insertion. There are eight valid values (multiple values are supported):

"location": Test point will not be inserted if it introduces any MV violation.

"location_not_fixable": Test point will not be inserted if it introduces Not fixable MV violation.

"clock": Test point will not be inserted if tool could not find an MV safe clock.

"user_clock" : Test point will not be inserted if its user specified clock introduces an MV violation.

"control": Test point will not be inserted if tool could not find an MV safe control signal.

"user_control": Test point will not be inserted if its user specified control signal introduces an MV violation.

"all": Test point will not be inserted if its location, clock or control signal introduces an MV violation.

"none": To disable the power aware test point insertion.

Reasons to reject test points:

Net is dont touch.

Net is IO pad Net.

Net is between domain boundary and iso/ls/repeater cells.

Net is UPF signal net.

Net is in default VA which doesn't allow tpi cell.

Net is in hierarchy which doesn't allow new cell.

The below cases are fixable if there is a boundary between driver and load

Net is less on than both driver and load.

Driver is less on than net.

Driver has voltage violation than net.

Load is more on than net.

Load has voltage violation than net.

Examples

In the following example, the test point will not be inserted if its clock or control signal introduces an MV violation:

```
prompt> set_app_options -name dft.tp_enable_mv_checks -value "clock  
control"
```

See Also

- [set_testability_configuration](#)
- [run_test_point_analysis](#)

dft.tp_enable_partition_search

Enables partition search for a test point in TPI flow.

Data Types

boolean

Default false

Description

This application option enables the search for a test point in partitions in test point analysis.

Examples

A typical use mode will consist on this sequence of commands:

```
prompt> set_app_options -name dft.tp_enable_partition_search -value false
prompt> <-- TEST POINT AND DFT SETUP HERE-->
prompt> run_test_point_analysis
prompt> preview_dft
prompt> insert_dft
```

See Also

- [set_testability_configuration](#)
- [run_test_point_analysis](#)

dft.tp_enable_partition_specifications

Enables partition specifications in test point analysis.

Data Types

boolean

Default false

Description

This application option enables the use of partition specifications in test point analysis.

Examples

A typical use mode will consist on this sequence of commands:

```
prompt> set_app_options -name dft.tp_enable_partition_specifications  
-value false  
  
prompt> <-- TEST POINT AND DFT SETUP HERE-->  
  
prompt> run_test_point_analysis  
  
prompt> preview_dft  
  
prompt> insert_dft
```

See Also

- [set_testability_configuration](#)
- [run_test_point_analysis](#)

dft.tp_enable_physically_aware_sharing

Enables physically aware sharing for test points insertion.

Data Types

boolean

Default true

Description

Enables physically aware sharing for test point insertion. It is applicable to TestMAX Advisor based test points and user defined test point specified using *set_test_point_element*. For user defined test points, test points can only be shared within the same *set_test_point_element* command.

Examples

```
prompt> set_app_options -name dft.dft.tp_enable_physically_aware_sharing  
-value false
```

See Also

- [set_testability_configuration](#)
- [run_test_point_analysis](#)

dft.tp_enable_power_aware_clock_selection

Enables power aware test point clock selection.

Data Types

boolean

Default false

Description

This application option is use to enable power aware clock selection for test points.

Examples

```
prompt> set_app_options -name dft.tp_enable_power_aware_clock_selection  
-value true
```

See Also

- [set_testability_configuration](#)
- [run_test_point_analysis](#)

dft.tp_enable_saving_power_for_observe_tp

Enables power saving for observe xor logic by adding extra and gates with TestMode signals with the data_in to avoid toggling. The option does not work when set_testability_configuration -test_point_without_control_signal

Data Types

boolean

Default false

Description

Enables power saving for observe xor logic by adding extra and gates with TestMode signals with the data_in

Examples

```
prompt> set_app_options -name dft.tp_enable_saving_power_for_observe_tp  
-value true  
prompt> set_app_options -name dft.tp_enable_saving_power_for_observe_tp  
-value false
```

See Also

- [set_testability_configuration](#)

dft.tp_enable_simulation_libraries

Enables simulation libraries to be used for test point analysis.

Data Types

boolean

Default true

Description

This application option enables simulation libraries to be used for test point analysis during *run_test_point_analysis*.

Examples

A typical use mode will consist on this sequence of commands:

```
prompt> set_app_options -name dft.tp_enable_simulation_libraries -value  
false
```

```
prompt> <-- TEST POINT AND DFT SETUP HERE-->
```

```
prompt> run_test_point_analysis
```

```
prompt> preview_dft
```

```
prompt> insert_dft
```

See Also

- [set_testability_configuration](#)
- [run_test_point_analysis](#)

dft.tp_improve_clustering_to_reduce_outliers

reduces potential outliers by adding new TP registers by 10%.

Data Types

boolean

Default true

Description

reduces potential outliers by adding new TP registers by 10%.

Examples

```
prompt> set_app_options -name  
dft.dft.tp_improve_clustering_to_reduce_outliers -value false
```

See Also

- [set_testability_configuration](#)
- [run_test_point_analysis](#)

dft.tp_instance_name_prefix

Appends the user specified prefix to test point inserted logic gate name and adds abbreviated test point type to suffix.

Data Types

string

Default "_unspecified_"

Description

To append user specified prefix to test point inserted logic gate name and adds abbreviated test point type to suffix along with unique number. The value should be legal verilog identifier(except escape characters).

Examples

In the following example, the prefix will be appended to the test point type control_01:

```
prompt> set_app_options -name dft.tp_instance_name_prefix -value  
"test_point"  
prompt> The test point logic gate name would be : test_point_c01_17
```

See Also

- [set_testability_configuration](#)
- [preview_dft](#)
- [insert_dft](#)

dft.tp_move_to_upper_hier_location

Enables test points to be moved to upper hierarchy violation.

Data Types

boolean

Default false

Description

Enables test points to be moved to higher hierarchy to reduce port punching, isolation and level shifter cells. It may not always be possible to find a functional equivalent of original test point location, so not all test points can be relocated. Movement is limited accross single input and single output gates only.

Examples

```
prompt> set_app_options -name dft.tp_move_to_upper_hier_location -value  
true
```

See Also

- [set_testability_configuration](#)
- [run_test_point_analysis](#)

dft.tp_no_scan_wrap_constr_gen_for_adv

scan_wrap constraints of memory cells are not generated for advisor when CTL is not provided.

Data Types

string

Default true

Description

To enable generation of scan_wrap constraints of memory cells for advisor when CTL is not provided. There are true valid values (multiple values are not supported):

"false": scan_wrap constraints of memory cells for advisor will be generated.

"true": scan_wrap constraints of memory cells for advisor will not be generated.

Examples

In the following example, the test point will not be generated:

```
prompt> set_app_options -name dft.tp_no_scan_wrap_constr_gen_for_adv
        -value "true"
```

See Also

- [run_test_point_analysis](#)

dft.tp_no_udtp_constraint_generation_for_advisor

Disables the SGDC constraint generation for TestMAX Advisor during run_test_point_analysis.

Data Types

boolean

Default false

Description

This application option disables the SGDC constraint generation for TestMAX Advisor during run_test_point_analysis. This option can be used, if the number of the specified set_test_point elements is quite big and it TestMAX Advisor takes more time to parse the SGDC during test point analysis. If the objects for set_test_point_element are the result of any wildcard patterns, the same pattern can be supplied directly to TestMAX Advisor to achieve better runtime.

Examples

```
prompt> echo "\n" \
        "current_design my_design_top\n" \
        "define_macro -macro MCP_REG_PINS -type { FLIP_FLOP_OUT } -filter_in
        {"*pattern_A*" "*pattern_B*"} -ignore_boundary_for_wildcard
        -filter_out {"*clk_gate*"}\n" \
        "define_macro -macro F0_TP -type { MCP_REG_PINS } -filter_in
        {"*pattern3*"} -ignore_boundary_for_wildcard\n" \
        "test_point -type F0_TP -tp_type force-0 " > [pwd]/udtp_for_adv.sgdc
prompt> echo "read_file -type sgdc [pwd]/udtp_for_adv.sgdc" >
        sg_option.tcl
prompt> set_testability_configuration -sg_command_file
        [pwd]/sg_option.tcl
prompt> set_app_options -name
        dft.tp_no_udtp_constraint_generation_for_advisor -value true
```

See Also

- [set_test_point_element](#)
- [set_testability_configuration](#)
- [run_test_point_analysis](#)

dft.tp_preserve_tp_gate

Enables test point inserted gate to be retained through compile.

Data Types

boolean

Default false

Description

This application option add no gobble restriction on test point logic gates.

Examples

In the following example, the test point gate will not be merged with functional logic:

```
prompt> set_app_options -name dft.tp_preserve_tp_gate -value true
```

See Also

- [set_testability_configuration](#)
- [run_test_point_analysis](#)

dft.tp_set_analysis_directory

Sets the name of directory for test point analysis.

Data Types

string

Default ""

Description

This application option enables user specified directory name for test point analysis during *run_test_point_analysis*.

Examples

A typical use mode will consist on this sequence of commands:

```
prompt> set_app_options -name dft.tp_set_analysis_directory -value  
early_version_1  
  
prompt> <-- TEST POINT AND DFT SETUP HERE-->  
  
prompt> run_test_point_analysis  
  
prompt> insert_dft
```

See Also

- [set_testability_configuration](#)
- [run_test_point_analysis](#)

dft.tp_set_cluster_effort

sets the effort value for physical clustering

Data Types

string

Default medium

Description

To set the effort value for physical clustering There are four valid values (multiple values are not supported):

"lowest", "low" , "medium", "high"

Examples

In the following example, the the clustering will not look for physical aware support when lowest is specified :

```
prompt> set_app_options -name dft.tp_set_cluster_effort -value "lowest"
```

See Also

- [preview_dft](#)
- [insert_dft](#)

dft.tp_set_dtc_name

Specify names for test point data controllers.

Data Types

string

Default ""

Description

This application option enable customization of names for test point inserted data controllers. Supported data controller types are `auto_self_gating`, `control`, `observe`, `force`, `mcp` and `self_gating`. Only non-escape legal verilog identifiers can be specified as names

Examples

```
prompt> set_app_options -name dft.tp_set_dtc_name -value {{control  
    tp_ctrl} {observe tp_obs}}  
prompt> set_app_options -name dft.tp_set_dtc_name -value {{control  
    tp_ctrl} {observe ao_obs} {force force_tp} {mcp mcp_loopback}  
    {self_gating sg} {auto_self_gating asg_tp}}
```

See Also

- [set_testability_configuration](#)
- [run_test_point_analysis](#)

dft.tp_set_report_file_name

Sets the pathname for test point report file.

Data Types

string

Default ""

Description

This application option enables test point report file name and location to be customized. during *preview_dft* and *insert_dft*.

Examples

A typical use mode will consist on this sequence of commands:

```
prompt> set_app_options -name dft.tp_set_report_file_name  
    -value /custom/full/path/custom_report.rpt
```

```
prompt> set_app_options -name dft.tp_set_report_file_name -value  
        custom_report_name.rpt  
  
prompt> <-- TEST POINT AND DFT SETUP HERE-->  
  
prompt> run_test_point_analysis  
  
prompt> insert_dft
```

See Also

- [set_testability_configuration](#)
- [run_test_point_analysis](#)

dft.tp_show_report_in_log

Displays the test point report in log

Data Types

string

Default summary

Description

To display test point report in log. There are three valid values (multiple values are not supported):

"none": Test point report will not be displayed.

"full": Complete Test point report will be displayed.

"summary": Only the summary in Test point will be displayed.

Examples

In the following example, the test point report will not be displayed in log if its none:

```
prompt> set_app_options -name dft.tp_show_report_in_log -value "none"
```

See Also

- [set_testability_configuration](#)
- [preview_dft](#)
- [insert_dft](#)

dft.tp_support_cell_based_dft_partition_for_udtp

Enables supports cell based partition with user defined test points

Data Types

boolean

Default false

Description

Enables supports cell based partition with user defined test points when the app option is specified, the set_test_point_element options "-dft_partition" and "-clock_signal" are mandatory Supported test point types are control_0, control_1, observe, and force_01

Examples

```
prompt> set_app_options -name
dft.tp_support_cell_based_dft_partition_for_udtp -value true
prompt> set_test_point_element {<ports_list>} -clock_signal TP_CLK
-control_signal TP_TM -test_points_per_source_or_sink 4 -dft_partition
partition_pl
```

See Also

- [set_testability_configuration](#)

dft.tp_treat_usage_clock_gating_as_controllable_for_analysis

Enables user to treat usage clock gating type signal as controllable in advisor.

Data Types

bool

Default false

Description

This application option allows the user to control the controllability of usage clock gating in advisor.

Examples

```
prompt> set_app_options -name
dft.tp_treat_usage_clock_gating_as_controllable_for_analysis -value true
prompt> set_app_options -name
dft.tp_treat_usage_clock_gating_as_controllable_for_analysis -value
false
```


See Also

- [run_test_point_analysis](#)
- [preview_dft](#)
- [insert_dft](#)

dft.use_merged_pdl

Allow the use of merged pdl for YA SPF.

Data Types

boolean

Default false

Description

This App Option is used if you want to use merged pdl instead of mode specific pdl for the generation of YA based SPF.

Examples

A typical use mode will consist on this sequence of commands:

```
prompt> set_app_options -as_user_default -name dft.use_merged_pdl -value  
true
```

```
prompt> write_test_protocol -test_mode top_scan -o top_scan.spf -target  
{{u1 ./modA_unit_merged.pdl} {top ./top_merged.pdl}}
```

dft.xlbist_test_mode_from_1500

DFT XLBIST Test Mode signal comes from DFT 1500 register.

Data Types

boolean

Default false

Description

This application option enables to have the *Test Mode control from the UI*.

This App Option is used if there are other modes to be set in XLBIST/SEQ.

Examples

A typical use mode will consist on this sequence of commands:

```
prompt> set_app_options -name dft.xlbist_test_mode_from_1500 -value true  
prompt> generate_dft
```

dpx.clock_opt.distribution_effort_level

Specify effort to make clock_opt distributed.

Data Types

Boolean

Default ultra

Description

This application option changes distribution effort level for DPX clock_opt. Runtime is expected to decrease gradually when value of this app option is increased from "low" to "extreme".

See Also

- [start_dpx_workers](#)

dpx.clock_opt.high_capacity_workers

Reduce memory used by every DPX worker in clock_opt.

Data Types

Boolean

Default ultra

Description

This application option changes memory footprint of DPX workers: "basic": small memory reduction "advanced": medium memory reduction "ultra": high memory reduction

See Also

- [start_dpx_workers](#)

dpx.enable

Enable distributed multi-processing feature.

Data Types

Boolean

Default false

Description

This application option enables/disables distributed multi-processing feature.

In order to use distributed multi-processing feature, please first specify *dpx_host_options* using *set_host_options* command: e.g., use 4 workers from grid,

```
prompt> set_host_options -name dpx_host_options -num_process 4  
        -submit_command "qsub -l memfree=10G"
```

Then enable distributed multi-processing feature by:

```
prompt> set_app_options -list {dpx.enable true}
```

Then start workers:

```
prompt> start_dpx_workers
```

The *route_opt* followed will use distributed multi-processing feature.

See Also

- [start_dpx_workers](#)

dpx.place_opt.distribution_effort_level

Specify effort to make place_opt distributed.

Data Types

Boolean

Default ultra

Description

This application option changes distribution effort level for DPX place_opt. Runtime is expected to decrease gradually when value of this app option is increased from "low" to "extreme".

See Also

- [start_dpx_workers](#)

dpdx.place_opt.high_capacity_workers

Reduce memory used by every DPX worker in place_opt/compile_fusion.

Data Types

Boolean

Default basic

Description

This application option changes memory footprint of DPX workers: "basic": small memory reduction "advanced": medium memory reduction "ultra": high memory reduction

See Also

- [start_dpx_workers](#)

dpdx.route_opt.distribution_effort_level

Specify effort to make route_opt/hyper_route_opt distributed.

Data Types

Boolean

Default ultra

Description

This application option changes distribution effort level for DPX route_opt/hyper_route_opt. Runtime is expected to decrease gradually when value of this app option is increased from "low" to "extreme".

See Also

- [start_dpx_workers](#)

dpdx.route_opt.high_capacity_workers

Reduce memory used by every DPX worker in route_opt/hyper_route_opt.

e

Data Types

*Enum***Default** ultra

Description

This application option changes memory footprint of DPX workers: "basic": small memory reduction "advanced": medium memory reduction "ultra": high memory reduction

See Also

- [start_dpx_workers](#)

e

eco.add_buffer.honor_dont_touch

Option for the *add_buffer* command and the *insert_buffer* command. Skip nets with *dont_touch* attribute.

Data Types

*bool***Default** false

Description

When this option is set to true, nets with *dont_touch* attribute are skipped.

Examples

The following example shows how to enable to skip nets with *dont_touch* attribute.

```
prompt> set_app_options -name eco.add_buffer.honor_dont_touch -value true
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)

- [add_buffer](#)
- [insert_buffer](#)

eco.add_buffer_on_route.advanced_detect_layer

Option for the *add_buffer_on_route* command. When *-detect_layer* is turned on, add buffer even if the specified location has same distance to the route shapes on different layers.

Data Types

bool

Default false

Description

When you use location-based *add_buffer_on_route* with option *-location* or *-user_specified_buffers*, and turn on option *-detect_layer* or specify layer 0 to let the command decide which layer the buffer should be added onto, it is possible that the command error out with UIED-333 when it finds multiple route shapes have same distance to the input location and these shapes are on different layers. In this situation, you can assign a particular layer in format *-location {x y layer_name}*. If you don't care which layer to choose, you can turn on this app option. The command will choose the lowest found layers automatically. It will also ensure the buffer is not added onto a dangling route if this layer is chosen. Otherwise, the upper layer will be chosen.

Examples

The following example shows how to turn on this app option.

```
prompt> set_app_options -name  
eco.add_buffer_on_route.advanced_detect_layer -value true
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)
- [add_buffer_on_route](#)

eco.add_buffer_on_route.min_distance_buffer_to_load

Option for the *add_buffer_on_route* command. Specifies the minimum distance from the last buffer to a load.

Data Types

float

Default Automatically setting as 1% of *-repeater_distance* of *add_buffer_on_route*

Description

When *add_buffer_on_route* tries to add a buffer whose route distance (path travelled by the net) to load pin is less than the value specified by this application option, the buffer will not be inserted. It is an absolute value and its unit is micron. If the buffer to add is an inverter and the number of inverter is odd, the inverter will not be inserted. If the buffer to add is an inverter and the number of inverter is even, it still will be added and will be moved away from the load until the distance is bigger than the application option value. This is to make sure inverter is always paired.

Examples

The following example specifies the *min_distance_buffer_to_load* for the *add_buffer_on_route* command.

```
prompt> set_app_options -name  
eco.add_buffer_on_route.min_distance_buffer_to_load -value 1
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)
- [add_buffer_on_route](#)

eco.freeze_silicon.enable_multi_threads

enable the multi-threading feature in advanced place_freeze_silicon

Data Types

bool

Default true

Description

This option provides a control to enable the multi-threading feature in advanced PSC mapping.

Examples

The following example shows how to disable multi-threading feature in freeze-silicon flow.

```
prompt> set_app_options -name eco.freeze_silicon.enable_multi_threads  
-value false
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)
- [place_freeze_silicon](#)

eco.freeze_silicon.not_spare_cell_aware

Disable the spare cell aware feature for *add_buffer_on_route* and *size_cell* commands in freeze-silicon flow

Data Types

bool

Default false

Description

This option provides a control to disable the spare cell aware feature for *add_buffer_on_route* and *size_cell* commands in freeze-silicon mode

Examples

The following example shows how to disable the spare cell aware feature in freeze-silicon flow.

```
prompt> set_app_options -name eco.freeze_silicon.not_spare_cell_aware  
-value true
```


See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)
- [add_buffer_on_route](#)
- [size_cell](#)

eco.freeze_silicon.pfs_map_spare_cells_only

Option for the *place_freeze_silicon* command. Skip the coarse placement operation and map spare cells only.

Data Types

bool

Default false

Description

This option provides a control to skip the coarse placement operation and map spare cells only.

Examples

The following example specifies the *pfs_map_spare_cells_only* for the *place_freeze_silicon* command.

```
prompt> set_app_options -name eco.freeze_silicon.pfs_map_spare_cells_only  
          -value true
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)
- [place_freeze_silicon](#)

eco.functional.diff_ignore_tie_changes

Option for the *eco_netlist* command. Ignore the unexpected netlist difference related to tie nets.

Data Types

bool

Default false

Description

read_verilog will generate unexpected tie net changes caused by tie net assignment, this option provides a control to ignore these unexpected difference.

Examples

The following example specifies the *diff_ignore_tie_changes* for the *eco_netlist* command.

```
prompt> set_app_options -name eco.functional.diff_ignore_tie_changes  
         -value true
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)
- [eco_netlist](#)

eco.functional.diff_preprocess_for_verilog

Option for the *eco_netlist* command. Ignore the unexpected netlist difference caused by verilog syntax.

Data Types

bool

Default true

Description

Due to the verilog syntax, *write_verilog* will generate some netlist difference from the original design. *eco_netlist* will detect these difference when using the outputted verilog file

e

as golden. Since these difference are not user intention, this option provides a control to ignore these unexpected difference.

Examples

The following example specifies the *diff_preprocess_for_verilog* for the *eco_netlist* command.

```
prompt> set_app_options -name eco.functional.diff_preprocess_for_verilog
-value true
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)
- [eco_netlist](#)

eco.placement.optimized_legalization

Specifies whether place_eco_cells -legalize_mode minimum_physical_impact should stops at free site only step when any eco cell cannot find a free sitelocation within -maximum_displacement_threshold.

Data Types

boolean

Default false

Description

Setting it to false means command will end when there is a eco cell that cannot find a legal free site location within -max_displacement_threshold. If set to true, tool will continue to the second step (allow_move_other_cells) to find a legal location for such eco cells.

eco.post_silicon_debug.bs_via_check_window_size

Used in check_backside_navigation command. Specifies the window size for checking.

Data Types

pair of floats

Default {45 45}

Description

Example:

```
prompt> set_app_options -name  
eco.post_silicon_debug.bs_via_check_window_size \\  
-value {50 50}
```

See Also

- [get_app_option_value](#)
- [set_app_options](#)
- [check_backside_navigation](#)

eco.post_silicon_debug.bs_via_percent_goal

Used in `check_backside_navigation` command. Specifies the expected coverage. Can be overridden by `-percent_goal` command option.

Data Types

float

Default 80

Description

Example:

```
prompt> set_app_options -name eco.post_silicon_debug.bs_via_percent_goal  
\ \  
-value 95
```

See Also

- [get_app_option_value](#)
- [set_app_options](#)
- [check_backside_navigation](#)

eco.post_silicon_debug.fib_logic_cell_prefix

Used in `create_frontside_fibs` command for FIB logic cell prefix.

Data Types

string

Default fib_logic_

Description

Example:

```
prompt> set_app_options -name  
eco.post_silicon_debug.fib_logic_cell_prefix \\  
-value "fib_cell_"
```

See Also

- [get_app_option_value](#)
- [set_app_options](#)
- [create_frontside_fibs](#)

eco.post_silicon_debug.fib_tie_cell_prefix

Used in create_frontside_fibs command for FIB tie cell prefix.

Data Types

string

Default fib_tie_lo_

Description

Example:

```
prompt> set_app_options -name  
eco.post_silicon_debug.fib_logic_cell_prefix \\  
-value "fib_tie_"
```

See Also

- [get_app_option_value](#)
- [set_app_options](#)
- [create_frontside_fibs](#)

eco.post_silicon_debug.fs_fib_check_window_size

Used in check_frontside_fibs command. Specifies the window size for checking.

Data Types

pair of floats

Default {50 50}

Description

Example:

```
prompt> set_app_options -name
eco.post_silicon_debug.fs_fib_check_window_size\\
-value {100 100}
```

See Also

- [get_app_option_value](#)
- [set_app_options](#)
- [check_frontside_fibs](#)

eco.post_silicon_debug.fs_fib_min_placeable_area_percent

Used in check_frontside_fibs command. Specifies the expected coverage.

Data Types

float

Default 60

Description

Can be overridden by -percent_goal command option.

Example:

```
prompt> set_app_options -name
eco.post_silicon_debug.fs_fib_min_placeable_area_percent \\
-value 75
```

See Also

- [get_app_option_value](#)
- [set_app_options](#)
- [check_frontside_fibs](#)

eco.post_silicon_debug.fs_fib_percent_goal

Used in check_frontside_fibs command. Specifies the expected coverage. Can be overridden by -percent_goal command option.

Data Types

float

Default 80

Description

Example:

```
prompt> set_app_options -name eco.post_silicon_debug.fs_fib_percent_goal  
\\  
-value 95
```

See Also

- [get_app_option_value](#)
- [set_app_options](#)
- [check_frontside_fibs](#)

eco.post_silicon_debug.power_elevation_check_window_size

Used in check_pg_elevation command. Specifies the window size for checking.

Data Types

pair of floats

Default {200 200}

Description

Example:

```
prompt> set_app_options -name  
eco.post_silicon_debug.power_elevation_check_window_size\\  
-value {100 100}
```

See Also

- [get_app_option_value](#)
- [set_app_options](#)
- [check_pg_elevation](#)

eco.post_silicon_debug.power_elevation_max_route_length

Used in check_pg_elevation command. Specifies the maximum route length from tie cell pin to the port.

Data Types

float

Default 5

Description

If the route length exceeds this number (in um), checker reports error.

Example:

```
prompt> set_app_options -name  
eco.post_silicon_debug.power_elevation_max_route_length \\  
-value 8
```

See Also

- [get_app_option_value](#)
- [set_app_options](#)
- [check_pg_elevation](#)

eco.post_silicon_debug.power_elevation_min_placeable_area_percent

Used in check_pg_elevation command. Specifies the minimum placeable percentage area of a checking window.

Data Types

float

Default 60

Description

If the percentage of placeable area is less than the specified number, the checking window is considered invalid.

Example:

```
prompt> set_app_options -name  
eco.post_silicon_debug.power_elevation_min_placeable_area_percent \\  
-value 75
```

See Also

- [get_app_option_value](#)
- [set_app_options](#)
- [check_pg_elevation](#)

eco.post_silicon_debug.power_elevation_percent_goal

Used in check_pg_elevation command. Specifies the expected coverage.

Data Types

float

Default 80

Description

Can be overridden by -percent_goal command option.

Example:

```
prompt> set_app_options -name  
eco.post_silicon_debug.power_elevation_percent_goal \\  
-value 95
```

See Also

- [get_app_option_value](#)
- [set_app_options](#)
- [check_pg_elevation](#)

eco.post_silicon_debug.signal_elevation_max_route_length

Used in `check_signal_elevation` command. Specifies the maximum route length from signal elevation buffer output pin to the port.

Data Types

float

Default 5

Description

If the route length exceeds this number (in um), checker reports error.

Example:

```
prompt> set_app_options -name
eco.post_silicon_debug.signal_elevation_max_route_length \
-value 8
```

See Also

- [get_app_option_value](#)
- [set_app_options](#)
- [check_signal_elevation](#)

eco.remove_buffer.keep_rtl_net

Option for the `remove_buffer` command. Specifies whether need to keep RTL net when a buffer is removed.

Data Types

bool

Default false

Description

After the buffer is removed, the command will ensure the net connected with ports has same name as one of the ports. By default, the naming priority is 1. Driver port 2. Any of the load ports. When this app option is turned on, the priority will change to 1. Driver port 2. Any of the none-feedthrough load ports 3. Any of feedthrough load ports. Specially, if the net connects to two load ports, one is RTL port, and the other is feedthrough port. The net name will be changed to be same as RTL port.

Examples

The following example specifies the *keep_rtl_net* for the *remove_buffer* command.

```
prompt> set_app_options -name eco.remove_buffer.keep_rtl_net -value true
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)
- [remove_buffer](#)

eco.size_cell.honor_dont_touch

Option for the *size_cell* command. Skip cells with dont_touch attribute.

Data Types

bool

Default false

Description

When this option is set to true, cells with dont_touch attribute are skipped.

Examples

The following example shows how to enable to skip cells with dont_touch attribute.

```
prompt> set_app_options -name eco.size_cell.honor_dont_touch -value true
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)
- [size_cell](#)

eco.size_cell.honor_dont_use

Option for the *size_cell* command. Check if the new library cell has true value in dont_use attribute.

Data Types

bool

Default false

Description

When this option is set to true, *size_cell* checks if the new library cell has true value in dont_use attribute and errors out if the library cell has dont_use attribute set to true.

Examples

The following example shows how to enable to check the value of dont_use attribute of new library cell.

```
prompt> set_app_options -name eco.size_cell.honor_dont_use -value true
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)
- [size_cell](#)

ecolm.freeze_silicon.psc_overwrite_auto_derived_first_track_off set

Option for the *derive_programmable_spare_cell_mapping_rules* command. Overwrite auto-derived first track offset value with user input value.

Data Types

bool

Default false

e

Description

This option provides a control to overwrite auto-derived first track offset value with user input value. First track offset is the distance between the core area left boundary and the first track in the area.

Examples

The following example specifies the *psc_overwrite_auto_derived_first_track_offset* for the *derive_programmable_spare_cell_mapping_rules* command.

```
prompt> set_app_options -name  
        ecolm.freeze_silicon.psc_overwrite_auto_derived_first_track_offset  
        -value true
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)
- [derive_programmable_spare_cell_mapping_rules](#)

ecolm.freeze_silicon.psc_track_mask

Option for the *derive_programmable_spare_cell_mapping_rules* command. Input first track mask value by user.

Data Types

string

Description

This option provides a control to let user input first track mask value for the PG layer. First track mask is the mask of the first track in the core area.

Examples

The following example specifies the *psc_track_mask* for the *derive_programmable_spare_cell_mapping_rules* command.

```
prompt> set_app_options -name ecolm.freeze_silicon.psc_track_mask -value  
        mask_one
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)
- [derive_programmable_spare_cell_mapping_rules](#)

ecolm.freeze_silicon.psc_track_offset

Option for the *derive_programmable_spare_cell_mapping_rules* command. Input first track offset value by user.

Data Types

float

Default -1

Description

This option provides a control to let user input first track offset value for the PG layer. First track offset is the distance between the core area left boundary and the first track in the area.

Examples

The following example specifies the *psc_track_offset* for the *derive_programmable_spare_cell_mapping_rules* command.

```
prompt> set_app_options -name ecolm.freeze_silicon.psc_track_offset  
-value 0
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)
- [derive_programmable_spare_cell_mapping_rules](#)

em.net_delta_temperature

Define the delta temperature used in signal EM RMS limit lookup.

Data Types

float

Default 5.0

Description

This application option define delta temperature used in signal EM RMS limit lookup. If the signal EM RMS constraints are defined on delta temperature, the setting value is used in the limit lookup. Higher delta temperature allows higher RMS current. By default, the value is 5.0.

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design.

Use *get_app_option_value -design \$design -name em.net_delta_temperature* to get the current value of this application option on a specific design.

Use *get_app_option_value -name em.net_delta_temperature* to retrieve the current value. Use *set_app_options -as_user_default {em.net_delta_temperature value}* to set the new value. Use *report_app_options* to show the value of multiple application options.

See Also

- [get_app_option_value](#)
- [set_app_options](#)

em.net_delta_temperature_layer

Define the layer and delta temperature pairs used in signal EM RMS limit lookup.

Data Types

string

Default ""

Description

This application option define the pairs of the layers and the delta temperature used in signal EM RMS limit lookup. If the signal EM RMS constraints are defined on delta temperature, the setting value is used in the limit lookup. Higher delta temperature allows higher RMS current.

e

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design.

Use *get_app_option_value -design \$design -name em.net_delta_temperature_layer* to get the current value of this application option on a specific design.

Use *get_app_option_value -name em.net_delta_temperature_layer* to retrieve the current value. Use *set_app_options -as_user_default {em.net_delta_temperature_layer value}* to set the new value. Use *report_app_options* to show the value of multiple application options.

Examples

The following example shows how to specify user-defined delta temperature for layers.

```
prompt> set_app_options -name em.net_delta_temperature_layer -value { M0
10 M1 20}
```

See Also

- [get_app_option_value](#)
- [set_app_options](#)

em.net_duration_condition_for_peak

Define the duration value for signal EM PEAK current checking.

Data Types

float

Default 0.0

Description

This application option define the duration value for signal EM PEAK current checking. The PEAK current is defined as the maximum current value of current waveform. The PEAK_DC current is defined as PEAK current multiplied by square root of duty ratio. By default, PEAK_DC current is used to check against with limit. When the option is set, PEAK current is used if the computed duration is not less than the condition, otherwise PEAK_DC current is used.

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design.

Use *get_app_option_value -design \$design -name em.net_duration_condition_for_peak* to get the current value of this application option on a specific design.

e

Use `get_app_option_value -name em.net_duration_condition_for_peak` to retrieve the current value. Use `set_app_options -as_user_default {em.net_duration_condition_for_peak value}` to set the new value. Use `report_app_options` to show the value of multiple application options.

See Also

- [get_app_option_value](#)
- [set_app_options](#)

em.net_global_rms_relaxation_factor

Define the global relaxation factor on signal EM RMS limit.

Data Types

float

Default 1.0

Description

This application option define the global relaxation factor on signal EM RMS limit. The RMS current limit from constraints file is multiplied by the relaxation factor as the real limit. By default, there is no relaxation.

Application options are set using the `set_app_options` command, and they may be specified globally or with respect to a design.

Use `get_app_option_value -design $design -name em.net_global_rms_relaxation_factor` to get the current value of this application option on a specific design.

Use `get_app_option_value -name em.net_global_rms_relaxation_factor` to retrieve the current value. Use `set_app_options -as_user_default {em.net_global_rms_relaxation_factor value}` to set the new value. Use `report_app_options` to show the value of multiple application options.

See Also

- [get_app_option_value](#)
- [set_app_options](#)

em.net_lifetime_years

Define the net lifetime years used in signal EM flow.

e

Data Types*float***Default** 10.0**Description**

This application option define the net lifetime years which used in the signal EM flow. The Formula is shown below: $EM\ limit = DC_limit / sidefile_lifetime_years * net_lifetime_years$.

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design.

Use *get_app_option_value -design \$design -name em.net_lifetime_years* to get the current value of this application option on a specific design.

Use *get_app_option_value -name em.net_lifetime_years* to retrieve the net lifetime years.
Use *set_app_options -as_user_default {em.net_lifetime_years value}* to set the new value.
Use *report_app_options* to show the value of multiple application options.

See Also

- [get_app_option_value](#)
- [set_app_options](#)

em.net_metal_line_number

Specifies the metal line number for signal EM analysis.

Data Types*Integer***Default** 0**Description**

This application option specifies the metal line number for signal EM analysis.

The signal EM RMS constraints are defined for the worst case assuming that a signal current passes through all routing wires with minimum spacing at the same time. If some of the routing wires are not active with current passing through or the signal routing does not have the minimum spacing, the constraints should be relaxed.

The RMS constraint relaxation factor table is provided through signal EM constraints file. The metal line number defined by this application option is used to look up the factor table and to get the relaxation factor.

e

The metal line number can also be defined in signal EM constraints file. This application option will overwrite the number from signal EM constraints file. If the metal line number is not defined in signal EM constraints file or by the application option, no relaxation is applied by default.

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design. To determine the current value of this application option on a specific design, use *get_app_option_value -design \$design -name em.net_metal_line_number*. Use *report_app_options* to show the value of multiple application options.

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

em.net_min_duty_ratio

Define the minimum duty ratio value used in signal EM PEAK_DC current calculation.

Data Types

float

Default 0.0

Description

This application option define the minimum duty ratio value used in signal EM PEAK_DC current calculation. Duty ratio is defined as duration divided by period. If the computed duty ratio value is not less than the minimum value, the computed value is used. If the computed duty ratio value is smaller than the minimum value, the minimum value is used. By default, the computed duty ratio is used.

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design.

Use *get_app_option_value -design \$design -name em.net_min_duty_ratio* to get the current value of this application option on a specific design.

Use *get_app_option_value -name em.net_min_duty_ratio* to retrieve the current value. Use *set_app_options -as_user_default {em.net_min_duty_ratio value}* to set the new value. Use *report_app_options* to show the value of multiple application options.

See Also

- [get_app_option_value](#)
- [set_app_options](#)

em.net_use_waveform_duration

Define the duration used in signal EM PEAK_DC current calculation.

Data Types

Boolean

Default false

Description

This application option controls which duration is used in signal EM PEAK_DC current calculation. There are two duration definition, current integral based or current waveform based. By default, the current integral based duration is used and it is defined as the integral of current in one period divided by the current peak value. When set to "true", the current waveform based duration is used and it is defined as the time range of half peak points on the current waveform.

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design.

Use *get_app_option_value -design \$design -name em.net_use_waveform_duration* to get the current value of this application option on a specific design.

Use *get_app_option_value -name em.net_use_waveform_duration* to retrieve the current value. Use *set_app_options -as_user_default {em.net_use_waveform_duration true|false}* to set the new value. Use *report_app_options* to show the value of multiple application options.

See Also

- [get_app_option_value](#)
- [set_app_options](#)

em.net_violation_rule_types

Specify the types of signal EM rules used in signal EM analysis violation checking.

Data Types

string

e

Default auto**Description**

This application option specifies the types of signal EM rules used in signal EM analysis violation checking. There are three types of rules supported in signal EM analysis, avg/rms/peak. Any combination of rule types could be specified. If it's set as auto, signal EM analysis checks the current for which constraints are given. Auto is the default behavior. Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design.

Use *get_app_option_value -design \$design -name em.net_violation_rule_types* to get the current value of this application option on a specific design.

Use *get_app_option_value -name em.net_violation_rule_types* to retrieve the current value. Use *set_app_options -as_user_default {em.net_violation_rule_types value}* to set the new value. Use *report_app_options* to show the value of multiple application options.

See Also

- [get_app_option_value](#)
- [set_app_options](#)

em.new_temperature_nonlinear_interp

Choose the new nonlinear formula used in signal EM average current calculation.

Data Types

Boolean

Default false**Description**

Choose the new nonlinear formula used in signal EM average current calculation.

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design.

Use *get_app_option_value -design \$design -name em.new_temperature_nonlinear_interp* to get the current value of this application option on a specific design.

Use *get_app_option_value -name em.new_temperature_nonlinear_interp* to retrieve the current value. Use *set_app_options -as_user_default {em.new_temperature_nonlinear_interp true|false}* to set the new value. Use *report_app_options* to show the value of multiple application options.

See Also

- [get_app_option_value](#)
- [set_app_options](#)

est_delay.ml_delay_opto_dir

Data Types

string

Description

Specifies ML data and model directory when using `stimate_delay` command or ML Timing-Driven Pre-Route Optimization feature.

extract.abut_as_routed

Flag to treat net with abutted pins as routed in extraction.

Data Types

Boolean

Default False

Description

When `extract.abut_as_routed` is true, extractor will check if there are abutted pins in the net without routing shape. If there're abutted pins, the net without routing shape will be treated as routed in extraction.

To detect abutted pins in net without routing shape requires lots of running time, so its default is set to false to save running time in CTS/CTO stage while majority of nets are unrouted.

See Also

- [write_parasitics](#)

extract.auto_region_adjustment

Data Types

double

Default 100.0

Description

This app option sets the padding amount of the bounding box that is automatically generated based on the nets specified by the user in `extract_channel_parasitics`. The default value is 50 microns. The padding value cannot be negative.

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

extract.connect_open

Forces extraction to create dummy connections for open nets.

Data Types

Boolean

Default `true`

Description

Controls whether extraction creates dummy connections for open nets.

By default (*true*), extraction creates dummy connections for open nets.

When you set this option to *false*, extraction keeps the nets as open.

This option works for clock tree optimization, global routing, and detail routing extraction when it uses real geometry to build the resistance network. The original open resistance network must have internal nodes for the dummy connections; otherwise, the nets remain open.

See Also

- [write_parasitics](#)

extract.enable_coupling_cap

Enable coupling capacitance calculation in detail-route extraction.

Data Types

Boolean

Default false

Description

When this option is set to its default of *false*, coupling capacitance extraction is not performed.

When you set this option to *true*, coupling capacitance will be extracted in detail route extraction.

GR/CTO/VR extraction will not handle with coupling capacitance even this option is on.

Option time.si_enable_analysis ON will set this option to true when running update_timing.

See Also

- [write_parasitics](#)

extract.enable_ml_vr_estimation

Flag to apply machine learning extractor in the flow for the unrouted nets.

Data Types

Boolean

Default False

Description

When extract.enable_ml_vr_estimation is true, extractor would use the pre-trained machine learning models to estimate RC for the unrouted nets to achieve the better RC correlation. It is applied to designs which technolog node is 5nm and below.

extract.enable_via_alignment

Data Types

boolean

Default false

Description

This app option must be turned on to enable via alignment techniques so that it is easier for HFSS to mesh. The app option can be used in conjugation with extract.via_alignment_expansion_ratio_limit and extract.via_alignment_contraction_ratio_limit to expand/contract vias. The via alignment technique performs the following: a) Via expansion or contraction to provide better

alignment with upper and lower metal layer. b) Removal of floating vias, that is, vias which are only on one layer shape. c) Via merging for two vias, which have the same upper and lower metal shapes.

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

extract.exclusive_use_layers

Specifies layer names exclusively used for clock nets only in the flow. No signal nets are routed on these layers.

Default ""

Description

When design has layers exclusively used for clock nets and no signal nets are routed on these layers, please specify these layer names with this option. It would help flow achieve the better correlation and convergence for these clock nets between preroute and postroute stages.

See Also

- [get_app_option_value](#)
- [set_app_options](#)

extract.fusion_starrc_vmf

Controls virtual metal fill handling in fusion extraction.

Data Types

String

Default basic

Description

This application option controls how virtual metal fill is handled in fusion extraction for routed blocks.

The meaningful values are: 1. By default (*basic*), ICC2 virtual metal fill file is used

1. When set to (*advanced*), StarRC full virtual metal fill file is used

See Also

- [set_extraction_options](#)

extract.fusion_without_starrc_config

To trigger fusion extraction without using starrc_config file

Data Types

Integer

Default 2

Description

When extract.fusion_without_starrc_config is set to 1, detail route extractor's behavior will be: If StarRC config is set, fusion extraction will use the setting in config else if TLU+ file from read_parasitic_tech is comboTluplusNxtgrd, fusion extraction will be kicked in and use nxtgrd in the combo file. else fusion extraction will issue warning about non-Nxtgrd in TLU+ file and back to native extractor.

When it is set to 0, detail route extractor will try to read starrc config first and trigger fusion extraction if the setting is complete. Otherwise it will fall back to native extractor without checking if the TLU+ file is comboTluplusNxtgrd.

For default value 2, extractor will check tech_node of design. If it's N5 and below, fusion extractor's behavior is same as value 1; if it's N6 & above, it has same behavior as value 0.

See Also

- [update_timing](#)
- [write_parasitics](#)

extract.high_fanout_threshold

Threshold for high fanout net extraction.

Data Types

int

Default 1000

Description

When a net's number of pin is bigger than this threshold, its extraction will be skipped.
when it's less than this threshold, extractraction will be performed on it.

See Also

- [write_parasitics](#)

extract.memory_reduction

Specifies memory reduction effort level in fusion extraction.

Data Types

string

Default none

Description

This application option specifies the memory reduction effort level. It only affects fusion detail-route extraction.

Valid values are

- *none*
Default setting.
- *low*
Low effort memory reduction setting, which can cause minor runtime degradation.
- *medium*
Medium effort memory reduction setting, which will dump temporary files to disk to reduce memory peak.
- *high*
High effort memory reduction setting, which will adjust number of running thread and dump compressed temporary files to disk to reduce memory peak.
- *ultra*
Ultra effort memory reduction setting, which will adjust number of running thread and dump uncompressed temporary files to disk to reduce memory peak(and no extra memory usage by child process to decompress files).

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

extract.ml_gr_level

Specifies MLGR effort level.

Data Types

string

Default normal

Description

This application option specifies the MLGR effort level. ultra level is higher effort to have more pessimism parasitics than normal level(default level).

Valid values are

- *normal*(default)
Default GRML setting.
- *ultra*
ultra level is higher effort to have more pessimism parasitics than normal level.

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

extract.ml_vr_level

Specifies MLVR effort level.

Data Types

string

Default normal

Description

This application option specifies the MLVR effort level. *fast* level is lower effort to have more optimism parasitics than normal level(default level). Valid values are *ultra* level is higher effort to have more pessimism parasitics than normal level(default level).

- *normal*(default)

Default VRML setting.

- *fast*

fast level is lower effort to have more optimism parasitics than normal level.

- *ultra*

ultra level is higher effort to have more pessimism parasitics than normal level.

See Also

- [get_app_option_value](#)

extract.rde_effort_level

Specifies RDE effort level.

Data Types

string

Default medium

Description

This application option specifies the RDE effort level. Fresh RDE capturing is needed when effort level is changed.

Valid values are

- *low*

Default RDE setting.

- *medium* (default)

Sensitivity reduction in the flow. It would give better flow convergence.

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

extract.read_frame_view

To read cell instance shape from frame view. If frame view does not exist, read cell instance shape from design view.

Data Types

Boolean

Default true

Description

Controls how to read cell instance shape.

By default (*true*), to read from frame view. If frame view does not exist, read from design view.

When you set this option to *false*, to read from design view first. If design view does not exist, read from frame view.

This option should match StarRC command: NDM_DESIGN_VIEW in detail-routed extraction.

See Also

- [write_parasitics](#)

extract.read_zero_spacing_blockage

Flag to read zero spacing blockage

Data Types

Boolean

Default False

Description

If `extract.read_zero_spacing_blockage` is set to true, extractor will read in zero spacing blockage's shape as background geometry for extraction.

See Also

- [write_parasitics](#)

extract.starrc_mode

Controls StarRC feature for detail routed block's extraction.

Data Types

String

Default fusion_adv

Description

This application option controls how StarRC is used in extraction for routed blocks.

The meaningful values are: 1. By default (*fusion_adv*), Fusion performs extraction

1. When set to *in_design*, StarRC In-Design performs extraction
2. When set to *none*, the implementation tool performs extraction.

StarRC In-Design invokes full StarRC run. For Fusion compiler StarRC In-Design extraction, you need to specify StarRC installation PATH as it needs to call the StarRC binary.

Fusion extraction does not invoke full StarRC run, instead Fusion extraction does its only preprocessing including reading the design and partition, etc. It uses StarRC single partition extraction engine to run extraction. Then Fusion extraction does its own postprocessing, including stitching parasitic data across different partitions.

See Also

- [set_starrc_in_design](#)
- [report_starrc_in_design](#)

extract.starrc_path

Specifies the path of StarRC image StarXtract.

Data Types

string

Default none

Description

This application option specifies the image used by StarRC-in-design. It only affects StarRC-in-design detail-route extraction.

Valid values are

- *StarRC image from config*
Default setting.
- *\$path to a valid StarRC image*
Path to a valid StarRC image used by user.

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

extract.two_pin_net_instance_port

To treat pin shapes of two pin net as conductive or superconductive

Data Types

String

Default superconductive

Description

This app_option only works with fusion extraction. When `extract.two_pin_net_instance_port` is set to conductive, 2-pin net's pin shapes will be marked for resistance extraction. This can be used to match StarRC behavior.

With its default value false, 2-pin net's pin shapes are marked as superconductive and skipped in resistance extraction to save memory usage and run time as it creates dummy/dangling branch for most cases.

See Also

- [write_parasitics](#)

extract.via_alignment_contraction_ratio_limit

Data Types

double

Default 1.0

Description

This app option sets the limit for via contraction so that the vias align with the metals/bumps/vias the via is intersecting with on its upper and lower metal layer. This is done to make it easier for HFSS to mesh the EDB. The default value is 1.0 (i.e. no contraction). The via is not modified unless the result of the upper and lower layer via intersections results in a rectangle or a regular octagon.

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

extract.via_alignment_expansion_ratio_limit

Data Types

double

Default 1.0

Description

This app option sets the limit for via expansion so that the vias align with the metals/bumps/vias the via is intersecting with on its upper and lower metal layer. This is done to make it easier for HFSS to mesh the EDB. The default value is 1.0 (i.e. no expansion). The via is not modified unless the result of the upper and lower layer via intersections results in a rectangle or a regular octagon.

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

f

extract.vr_pattern_must_join_over_pin_layer

Specifies number of levels of pattern must join structure. The default value is 1, and the maximum is 2.

Data Types

Integer

Default 1

Description

If the value set to 1, via estimation for a pattern must join would be from pin shapes which has is_pattern_must_join attribute. If the value set to 2, via estimation for pattern must join would be from the top of the original 1-level pattern must join structure. The via shape at level-2 would have is_pattern_must_join attribute. And router would connect it through its upper via enclosure.

extract.zero_spacing_blockage_ratio

Density ratio to apply to zero spacing blockages

Data Types

float

Default

1.

Description

To specify density ratio applied to zero spacing blockage. Its value should be between 0 & 1. It only works when extract.read_zero_spacing_blockage is set to true.

See Also

- [write_parasitics](#)

f

file.3dblox.complete_collaterals

Controls whether write_3dblox dumps out regular files to output directory instead of symbolic link.

f

Data Types

Boolean

Default false**Description**

When the value is true, *write_3dblox* will write out regular files to collaterals under output directory. When the value is false, *write_3dblox* will write out symbolic link files to collaterals under output directory.

See Also

- [write_3dblox](#)

file.3dblox.drm_interface_name

Settings of *DRM_interface_name* for the specified block based on the layer and the connected block.

Data Types

List

Default {}**Description**

Enable the configuration of *DRM_interface_name* for designs without bump regions. Specify the bump region's layer name and the connected block name as a key, and the DIN (DRM interface name) as the value in a 3-value group. The *write_3dblox* command uses this setting to include DIN in the generated 3DBlox files.

Format: {{LayerName1 connectedBlockName1 DIN1} {LayerName2 connectedBlockName2 DIN2 }} LayerName: the bump region layer of the specified block. connectedBlockName: the die block connected to this interface. DIN: the DRM interface name assigned to this interface, determined by the first two keys.

Generally it is used together with *file.3dblox.lvs* in 3DBlox ICV flow.

See Also

- [read_3dblox](#)
- [write_3dblox](#)

f

file.3dblox.general_writer

Controls whether the write_3dblox dumps out data from general NDM 3dic database or from a database created by read_3dblox.

Data Types

Boolean

Default false

Description

When the value is true, it is a general writer for write_3dblox to write from a general NDM 3dic database.

When the value is false, write_3dblox uses data structures created by read_3dblox in the database like bump regions and topology plans.

See Also

- [read_3dblox](#)
 - [write_3dblox](#)
-

file.3dblox.lvs

Settings of LVS for the 3dic system based on die blocks.

Data Types

List

Default {}

Description

Enable the configuration of LVS for 3dic designs with no topology connections. Specify the connected die names and the side (optional) as a key, and the lvs as the value in a 4-value group. The write_3dblox command uses this setting to include lvs in the generated 3DBlox files.

Format: {{block1 block2 side lvs} {block3 block4 side lvs} } First two values: the name of the die blocks that build the connection. Specially, to specify connection to outside world, specify "~" for the second block name. The third value: the side of the connections. This is for the scenario when one die is embedded to another die. Specify "TOP" when the connection is on the upper side and on an internal_ext region. Specify "BOT" when the connection is on the lower side and on an internal_ext region. Specify "~" for non-embedded situation. One example: { {RDL ~ ~ lvs_file1} --> 1, RDL to outside world {RDL

f

LSI1 BOT lvs_file2} \ --> 2, LSI1 is embedded to RDL, specify lower side {RDL LSI2_SI ~ lvs_file3} \ --> 3 {LSI2_SI LSI2_RDL ~ lvs_file4} \ --> 4 {RDL LSI1 TOP lvs_file5} \ --> 5, LSI1 is embedded to RDL, specify upper side }

Generally it is used together with file.3dblox.drm_interface_name in 3DBlox ICV flow.

See Also

- [read_3dblox](#)
- [write_3dblox](#)

file.3dblox.oa_include_files

Provides additional inputs required for read_3dblox coupe mode.

Data Types

list of string pairs

Default empty list

Description

This application option provides additional inputs needed in 3Dblox coupe flow. Set it before run read_3dblox -mode coupe. Format: {<master name> /path/to/side_file1.yaml} {<master name> /path/to/side_file2.yaml} <master name>: the top design name or chiplet name /path/to/side_file1.yaml: side file path, if it is relative path, the base path is current working directory

Side file is in YAML style. Each keyword description: 3DIC_verilog_file: indicates the verilog file path of the design. This is optional. 3DIC_photonic_die: indicates whether the design is a photonic die or not. This is optional. 3DIC_APR_tech_file: indicates the technology file path for the design. This is required. 3DIC_NDM_ref_libs: indicates the reference libraries path referenced by the design. This is required. CC_include_libdefs: indicates the additional lib.defs file path needed for CC setup of the design. This is optional.

Available content of top design side file: 3DIC_verilog_file: [path/to/verilog.vg] #optional
3DIC_photonic_die: true #note: required and must be true for top design

Available content of chiplet side file: 3DIC_APR_tech_file: [/path/to/fusion.tf] #required
3DIC_NDM_ref_libs: [/path/to/bump.ndm, /path/to/pad.ndm, ...] #required
3DIC_photonic_die: true | false #optional
CC_include_libdefs: [a/b/lib.defs, c/d/e/lib.defs] #optional

See Also

- [read_3dblox](#)

file.3dblox.set_3dbf_file

This option is to provide a list of strings containing the associated 3dbf files with path relative to the output directory.

Data Types

list of string

Default empty list

Description

This option provides a list of 3dbf file path for current block. Once this application option is set, the list of strings will be output to the ChipletDef section of the corresponding 3dbv file or the TopDef section of corresponding 3dbv file. It is a design scoped application option.

See Also

- [write_3dblox](#)

file.3dblox.set_apr_tech_file

This option is to provide a list of strings containing the associated APR tech files with path relative to the output directory.

Data Types

list of string

Default empty list

Description

This option provides a list of APR tech files path for current block. Once this application option is set, the list of strings will be output to the ChipletDef section of the corresponding 3dbv file. It is a design scoped application option.

See Also

- [write_3dblox](#)

f

file.3dblox.set_connection_lvs_map

This option is to provide a list of string pairs denoting the LVS deck for the connection of current block.

Data Types

list of string pairs

Default empty list

Description

This option provides a list of string pairs representing connection versus LVS deck for current block. The format of string pairs is {connName Interface_LVS_deck}. Once this application option is set, the Interface_LVS_deck will be output to the Connection section of the corresponding 3dbx file. It is a design scoped application option.

See Also

- [write_3dblox](#)

file.3dblox.set_def_file

This option is to provide a list of strings containing the associated def files with path relative to the output directory.

Data Types

list of string

Default empty list

Description

This option provides a list of def files path for current block. Once this application option is set, the list of strings will be output to the ChipletInst section of the corresponding 3dbx file. It is a design scoped application option.

See Also

- [write_3dblox](#)

file.3dblox.set_drc_deck

This option is to provide a list of strings containing the associated DRC decks with path relative to the output directory.

Data Types

list of string

Default empty list

Description

This option provides a list of DRC deck path for current block. Once this application option is set, the list of strings will be output to the ChipletDef section of the corresponding 3dbv file or the TopDef section of corresponding 3dbv file. It is a design scoped application option.

See Also

- [write_3dblox](#)

file.3dblox.set_gds_file

This option is to provide a list of strings containing the associated gds files with path relative to the output directory.

Data Types

list of string

Default empty list

Description

This option provides a list of gds file path for current block. Once this application option is set, the list of strings will be output to the ChipletInst section of corresponding 3dbx file. It is a design scoped application option.

See Also

- [write_3dblox](#)

file.3dblox.set_gds_mapping_file

This option is to provide a list of strings containing the associated GDS mapping files with path relative to the output directory.

Data Types

list of string

Default empty list

f

Description

This option provides a list of GDS mapping file path for current block. Once this application option is set, the list of strings will be output to the ChipletDef section of the corresponding 3dbv file. It is a design scoped application option.

See Also

- [write_3dblox](#)

file.3dblox.set_info_tech_file

This option is to provide a list of strings containing the associated InFO tech files with path relative to the output directory.

Data Types

list of string

Default empty list

Description

This option provides a list of InFO tech file path for current block. Once this application option is set, the list of strings will be output to the ChipletDef section of the corresponding 3dbv file. It is a design scoped application option.

See Also

- [write_3dblox](#)

file.3dblox.set_ipf_file

This option is to provide a list of strings containing the associated ipf files with path relative to the output directory.

Data Types

list of string

Default empty list

Description

This option provides a list of ipf file path for current block. Once this application option is set, the list of strings will be output to the ChipletInst section of the corresponding 3dbx file. It is a design scoped application option.

See Also

- [write_3dblox](#)

file.3dblox.set_ircx_tech_file

This option is to provide a list of strings containing the associated iRCX tech files with path relative to the output directory.

Data Types

list of string

Default empty list

Description

This option provides a list of iRCX tech file path for current block. Once this application option is set, the list of strings will be output to the ChipletDef section of the corresponding 3dbv file. It is a design scoped application option.

See Also

- [write_3dblox](#)

file.3dblox.set_lef_file

This option is to provide a list of strings containing the associated LEF files with path relative to the output directory.

Data Types

list of string

Default empty list

Description

This option provides a list of LEF file path for current block. Once this application option is set, the list of strings will be output to the ChipletDef section of the corresponding 3dbv file. It is a design scoped application option.

See Also

- [write_3dblox](#)

file.3dblox.set_liberty_file

This option is to provide a list of strings containing the associated liberty files with path relative to the output directory.

Data Types

list of string

Default empty list

Description

This option provides a list of liberty file path for current block. Once this application option is set, the list of strings will be output to the ChipletDef section of the corresponding 3dbv file. It is a design scoped application option.

See Also

- [write_3dblox](#)

file.3dblox.set_lvs_deck

This option is to provide a list of strings containing the associated LVS decks with path relative to the output directory.

Data Types

list of string

Default empty list

Description

This option provides a list of LVS deck path for current block. Once this application option is set, the list of strings will be output to the ChipletDef section of the corresponding 3dbv file. It is a design scoped application option.

See Also

- [write_3dblox](#)

file.3dblox.set_rc_tech_file

This option is to provide a list of strings containing the associated RC tech files with path relative to the output directory.

Data Types

list of string

Default empty list

Description

This option provides a list of RC tech file path for current block. Once this application option is set, the list of strings will be output to the ChipletDef section of the corresponding 3dbv file. It is a design scoped application option.

See Also

- [write_3dblox](#)

file.3dblox.set_root_cell

Specifies the name of root cell that connects to the external environment.

Data Types

string

Default empty string

Description

This option specifies the root cell for current 3DIC stack. In the 3dBlox system, a root cell serves as the entry point to the external environment and is used to establish the virtual connection in the 3dBlox file. Manually set this app option when current design lacks a top-level signal port connection, which is common during the early stages of design planning. The write_3dblox command detects the root cell automatically. If the user-specified root cell differs from the tool-detected root cell, error message is displayed.

See Also

- [write_3dblox](#)

file.3dblox.set_spice_model

This option is to provide a list of strings containing the associated SPICE model files with path relative to the output directory.

Data Types

list of string

Default empty list

Description

This option provides a list of SPICE model file path for current block. Once this application option is set, the list of strings will be output to the ChipletDef section of the corresponding 3dbv file. It is a design scoped application option.

See Also

- [write_3dblox](#)

file.3dblox.set_thermal_name

This option is to provide a list of strings containing the associated thermal names of current block.

Data Types

list of string

Default empty list

Description

This option provides a list of thermal names for current block. Once this application option is set, the list of strings will be output to the ChipletDef section of the corresponding 3dbv file. It is a design scoped application option.

See Also

- [write_3dblox](#)

file.3dblox.set_thermal_tech_file

This option is to provide a list of strings containing the associated thermal tech files with path relative to the output directory.

Data Types

list of string

Default empty list

Description

This option provides a list of thermal tech file path for current block. Once this application option is set, the list of strings will be output to the TopDef section of the corresponding 3dbv file. It is a design scoped application option.

See Also

- [write_3dblox](#)

file.3dblox.set_upf_file

This option is to provide a list of strings containing the associated upf files with path relative to the output directory.

Data Types

list of string

Default empty list

Description

This option provides a list of upf file path for current block. Once this application option is set, the list of strings will be output to the Design section of the corresponding TopDesign 3dbx file or the ChipletInst section of corresponding 3dbx file. It is a design scoped application option.

See Also

- [write_3dblox](#)

file.3dblox.set_verilog_file

This option is to provide a list of strings containing the associated verilog files with path relative to the output directory.

Data Types

list of string

Default empty list

Description

This option provides a list of verilog file path for current block. Once this application option is set, the list of strings will be output to the Design section of the corresponding TopDesign 3dbx file or the ChipletInst section of corresponding 3dbx file. It is a design scoped application option.

See Also

- [write_3dblox](#)

f

file.3dblox.write_hier_bumps

Controls whether write_3dblox writes out hierarchical bumps into bmap file.

Data Types

Boolean

Default false

Description

When the value is true, *write_3dblox* will write all bumps in all hierarchy levels with the bump region to bmap file, and write test probes to pmap file if exists. When the value is false, *write_3dblox* will write all bumps in die top level to bmap file, and write test probes to pmap file if exists.

See Also

- [write_3dblox](#)

file.def.append_shape_and_terminal

Controls whether the *read_def* command appends net wiring shape and terminal.

Data Types

Boolean

Default true

Description

This application option controls whether the *read_def* command appends net wiring shape and terminal. This applies to the nets and pins defined in the DEF NETS, SPECIALNETS and PINS sections.

When this application option is *false*, *read_def* will remove all the net wiring shapes and terminals on the nets and pins defined in the DEF NETS, SPECIALNETS and PINS sections before creating the new net wiring shapes and terminals.

When this application option is *true*, *read_def* will append net wiring shape and terminal.

See Also

- [read_def](#)

file.def.check_mask_constraints

Used by "write_def" and "read_def" command to validate mask constraints of shapes/tracks in the block.

Data Types

string

Default "none"

Description

The valid values are "none", "loose" or "strict". This option is to control if shapes/tracks (including the shapes of vias, via matrix and shape patterns) violating the mask constraints are written out by write_def or read into database by read_def.

When the option is "none", there is no check during read_def/write_def.

When the option is "loose", all the mask constraints violating tracks and shapes will issue a warning during both read_def and write_def.

When the option is "strict", all the mask constraints violating tracks and shapes will issue an error.

The following table summarizes the read_def behavior:

	Object Type	Layer type	
"loose" mode	"strict" mode		
Colored	Uni-pattern	Warning	Error tracks layer
	Uncolored	Multi-pattern	
Warning	Error tracks layer		
Colored	Uni-pattern	Warning + shape	Error shapes layer converted
to colorless			
Uncolored	Multi-pattern	Warning + shape	Error shapes layer dropped

The following table summarizes the write_def behavior:

	Object Type	Layer type	
"loose" mode	"strict" mode		
Colored	Uni-pattern	Warning	Error tracks layer
	Uncolored	Multi-pattern	
Warning	Error tracks layer		
Colored	Uni-pattern	Warning + shape	Warning + shape shapes
layer written as	written as color-	colored shape.	less shape.
	Uncolored		
Multi-pattern	Warning + shape	Error shapes layer	dropped

f

If a via or shape_pattern has any mask constraint violating shape, the whole via or shape_pattern will be treated as above.

See Also

- [write_def](#)
- [read_def](#)
- [check_shapes](#)

file.def.check_mask_constraints_exclude_layers

Used by "*write_def*" command to exclude layers while using file.def.check_mask_constraints option.

Data Types

list

Default empty

Description

This option is to specify excluded layers when file.def.check_mask_constraints option is used.

When the list of excluded layers provided, all the shapes/tracks of those layers, will be excluded from checking.

When the list of excluded layers not provided, all shapes/tracks will be checked.

See Also

- [write_def](#)
- [check_shapes](#)

file.def.convert_rects_to_paths

Controls whether the *read_def* command translates the DEF NETS section RECT definitions into rectangle or path shapes.

Data Types

Boolean

Default false

f

Description

This application option controls whether the *read_def* command translates the DEF NETS section RECT definitions into rectangle or path shapes. This does not affect the RECT definitions in the DEF SPECIALNETS section.

If the application option is *false*, the *read_def* command translates them into rectangle shapes.

If the application option is *true*, the *read_def* command converts them into path shapes. The path orientation will follow the layer preferred routing direction if it is set. The path will have half-width square endcaps if possible. If the path is too short to have half-width endcaps, it will have flush endcaps. If the RECT cannot be created as a path, it will be created as a rectangle.

See Also

- [read_def](#)

file.def.custom_via_as_user_route

Controls whether the *read_def* command uses *user_route* as default *shape_use* for all via ladders and custom vias if DEF file not explicitly specify *shape_use*.

Data Types

Boolean

Default *false*

Description

This application option controls whether the *read_def* command uses *user_route* as default *shape_use* for all via ladders and custom vias if DEF file not explicitly specify *shape_use*.

When this application option is *false*, the *read_def* command uses default *shape_use* based on the net section (regular or special net section), net type and routing restriction status(routed, fixed, or cover)for all via ladders and custom vias if DEF file not explicitly specify *shape_use*.

When this application option is *true*, the *read_def* command uses *user_route* as default *shape_use* for all via ladders and custom vias if DEF file not explicitly specify *shape_use*.

See Also

- [read_def](#)

file.def.enable_user_attribute_properties

Controls whether the *read_def* and *write_def* commands preserve the DEF design, component, region, special net, regular net, nondefault rule, and row property definitions and values as user attributes.

Data Types

Boolean

Default false

Description

This application option controls whether the *read_def* and *write_def* commands preserve the DEF property definitions and values as user attributes. DEF design, component, region, special net, regular net, nondefault rule, and row properties are supported by this feature. DEF component pin and group properties are not supported by this feature.

During *read_def*, if the application option is true, user attributes are created in the block. These user attributes are accessible in the TCL UI. DEF DESIGN properties are created as block user attributes. DEF COMPONENT properties are created as cell user attributes. DEF REGION properties are created as move bound user attributes. DEF SPECIALNET and NET properties are merged together and created as net user attributes. DEF NONDEFAULTRULE properties are created as routing rule user attributes. DEF ROW properties are created as site row user attributes. DEF properties with INTEGER, REAL, and STRING data types respectively become user attributes with int, double, and string data types. All created user attributes are persistent, however the range min and max values are not persistent.

During *write_def*, if the application option is true, the above mentioned objects' user attributes are output as DEF properties. Note that net user attribute definitions are output as DEF SPECIALNET and NET property definitions. Also user attributes with float or double data types are output as DEF properties with REAL data types. There is no distinction between float and double data types in the DEF syntax.

This app option overrides the related app option, *file.def.support_property_definitions*, when reading and writing the object types mentioned above. When both app options are set to true, DEF DESIGN, COMPONENT, REGION, SPECIALNET, NET, NONDEFAULTRULE, and ROW properties are translated to and from user attributes which are accessible through the TCL UI. However the DEF GROUP properties are translated to and from an internal storage location which are not accessible through the TCL UI.

f

See Also

- [read_def](#)
- [write_def](#)

file.def.exclude_cut_metal

Controls whether the *write_def* command will exclude cut_metal and metal under cut metal shapes from output.

Data Types

Boolean

Default false

Description

This application option controls whether the *write_def* command excludes cut metal shapes on cut metal layers and metal shapes under cut metal shapes in the output DEF file or not. When set to "true", it is similar to specifying *write_def -exclude { cut_metal }* and has higher priority over specifying cut_metal option in *-include* option. When set to false (default), cut metals are included or excluded depending on *-include* or *-exclude* option.

See Also

- [read_def](#)
- [write_def](#)

file.def.fill_purpose_map

Maps fill shape use values in a DEF file to *purpose_number* attribute values in the design library.

Data Types

list_of_pairs

Default null

Description

Controls how the *read_def* and *write_def* commands map between fill shape use values in the DEF file and *purpose_number* attribute values in the design library.

DEF net fills are defined in the SPECIALNETS section. Each shape use is indicated by the "+ SHAPE DRCFILL|FILLWIRE|FILLWIREOPC" syntax.

f

DEF floating fills are defined in the FILLS section. If the shape is defined with the "+ OPC" syntax, the shape use is FILLWIREOPC. Otherwise, the shape use is FILLWIRE.

The syntax for this application option is

```
{ { DEF_shape_use purpose_number } ... }
```

The DEF shape use must be one of these values:

- DRCFILL
- FILLWIRE
- FILLWIREOPC

The *purpose_number* attribute must be an integer or one of the following system purpose names:

- active_fill
- special_router_extension

If this application option is set, the tool performs mapping as per the map provided. If DRCFILL is mapped to the *active_fill* system purpose, then shapes with the *special_router_extension* system purpose will get excluded from output (unless it is mapped to another DEF shape use) and vice versa.

By default, if this application option is not set, DEF DRCFILL shapes are mapped to and from shapes with the *special_router_extension* system purpose and shapes with the *active_fill* system purpose are skipped. All other fills are mapped to and from default purpose number 0.

Examples

The following example maps DEF DRCFILL to the *active_fill* system purpose.

```
prompt> set_app_options -name file.def.fill_purpose_map \\  
-value {{DRCFILL active_fill}}
```

The following example maps DEF DRCFILL to the **special_router_extension** system purpose and maps DEF FILLWIRE and FILLWIREOPC to purpose number 50.

```
prompt> set_app_options -name file.def.fill_purpose_map \\  
-value {{DRCFILL special_router_extension}  
        {FILLWIRE 50}  
        {FILLWIREOPC 50}}
```

See Also

- [read_def](#)
- [write_def](#)

f

file.def.ignore_uncolored_shapes

Used by "*write_def*" command to ignore uncolored shapes in the block.

Data Types

Boolean

Default false

Description

This option is to control if uncolored shapes which includes the shapes of via, via matrix and shape pattern are written out.

When the option is on, all the uncolored shapes will be ignored, if a via has any uncolored shape, the via will be ignored.

When the option is off, uncolored shapes are also written out.

See Also

- [write_def](#)
- [get_app_option_value](#)
- [set_app_options](#)

file.def.ignore_uncolored_shapes_exclude_layers

Used by "*write_def*" command to exclude layers while using file.def.ignore_uncolored_shapes option.

Data Types

list

Default empty

Description

This option is to specify exclude layers when file.def.ignore_uncolored_shapes option is true.

When the list of exclude layers provided, all the uncolored shapes of those layers, will not be ignored.

When the list of exclude layers not provided, all uncolored shapes will be ignored.

f

See Also

- [write_def](#)
- [read_def](#)

file.def.maintain_via_ladders

Controls whether the *write_def* and *read_def* commands maintain via ladders and Pattern Must Join (PMJ) ladders through DEF by writing and reading additional information in DEF.

Data Types

Boolean

Default false

Description

This application option controls whether the *write_def* and *read_def* commands maintain via ladders and Pattern Must Join (PMJ) ladders through DEF. When set to *true*, the *write_def* command writes additional information about the via ladders and PMJ ladders to the NETS, SPECIALNETS, and VIAS sections. The *read_def* command uses this information to reconstruct the ladders.

When set to *false*, the via ladders and PMJ ladders are written out and read from DEF as custom vias. The *write_def* command will not write out additional information about the via ladders and PMJ ladders. The *read_def* command will not reconstruct the ladders. The geometries of the via ladders and PMJ ladders are maintained, but the *is_via_ladder* and *is_pattern_must_join* attributes on the shapes and vias are not maintained.

See Also

- [write_def](#)

file.def.maskshift_consistency_check

Controls whether the *read_def* command checks the mask shift consistency.

Data Types

Boolean

Default false

Description

This application option controls whether the *read_def* command checks the mask shift consistency between cell and its reference block.

f

When this application option is *false*, the *read_def* command does not check the consistency.

When this application option is *true*, the *read_def* command checks the consistency. If any inconsistency is found, *read_def* will error out and give error message DEFR-067.

Take the following DEF file as an example, there is a cell U210 which references NAND3X1_LVT, the *read_def* command will error out when the reference block meets any of the following conditions. 1. The attribute "is_mask_shiftable" is false. 2. The attribute "mask_shift_layers" does not exist or the values does not contain M7 or M6.

```
DESIGN TEST ; COMPONENTMASKSHIFT M7 M6 M3 M2 M1 ; COMPONENTS 1 ; -
U210 NAND3X1_LVT + UNPLACED + MASKSHIFT 11000 ; END COMPONENTS END
DESIGN
```

See Also

- [read_def](#)

file.def.non_default_width_wiring_to_net

Controls whether the *write_def* command writes nets with nondefault width to the DEF NETS section or SPECIALNETS section.

Data Types

Boolean

Default false

Description

Controls whether the *write_def* command writes nets with nondefault width to the DEF NETS section or SPECIALNETS section.

By default (*false*), the *write_def* command writes nets with nondefault width to the SPECIALNETS section.

If you set the application option to *true*, the *write_def* command creates a nondefault routing rule named SNPS_DefGenNDR_<n> for each nondefault width, and then writes the nets with nondefault width to the DEF NETS section with a NONDEFAULTRULE statement for single nondefault width and TAPERRULE statements for multiple nondefault widths, which makes the output DEF file less dependent on the user-specified nondefault routing rules.

The *read_def* command creates a nondefault routing rule in the design cell if it finds a definition in the NONDEFAULTRULES section, unless its prefix is SNPS_DefGenNDR_<n>.

See Also

- [read_def](#)
- [write_def](#)
- [create_routing_rule](#)

file.def.pg_via_arrays

Controls whether the *write_def* command writes all regular spaced vias on power and ground nets in compact via array format using DO-STEP loop.

Data Types

Boolean

Default true

Description

This application option controls whether the *write_def* command writes all regular spaced vias on power and ground nets in compact via array format using DO-STEP loop. This applies only to vias defined on power and ground nets in the DEF SPECIALNETS sections.

When this application option is *false*, the power and ground vias are written out individually even if a regular spacing pattern exists.

When this application option is *true*, the power and ground vias are written out in compact via array format using DO-STEP loop, if a regular spacing pattern exists.

See Also

- [write_def](#)

file.def.relax_via_ladder_restriction

Controls whether the *read_def* command set the *physical_status* of via ladder(including PMJ) shapes which have FIXED DEF routing status to unrestricted.

Data Types

Boolean

Default false

f

Description

This application option controls whether the *read_def* command set the *physical_status* of via ladder(including PMJ) shapes which have FIXED DEF routing status to unrestricted.

By default, *read_def* command set the *physical_status* of via ladder(including PMJ) shapes which have FIXED DEF routing status to fixed. When this application option is *true*, the *read_def* command will set the *physical_status* of via ladder(including PMJ) shapes which have FIXED DEF routing status to unrestricted instead of fixed.

See Also

- [read_def](#)

file.def.rule_based_name_matching

Controls whether the *read_def* command uses rule-based name matching to find the DEF COMPONENTS and PINS (cells and ports) in the existing design.

Data Types

Boolean

Default *true*

Description

When annotating DEF attributes onto an existing cell or port in the design, the *read_def* command needs to find the cell or port by name. A simple string match is performed if this application option is *false*. Rule-based name matching is performed if this application option is *true*. To set the rules, use the *set_query_rules* command; otherwise, the tool uses the default query rules.

See Also

- [read_def](#)
- [set_query_rules](#)

file.def.scan_cells_in_netlist_order

Controls whether the *write_def* command outputs floating scan cells in netlist order.

Data Types

Boolean

Default *false*

f

Description

This application option controls whether the *write_def* command outputs floating scan cells in netlist order. This applies to scan chain defined in the DEF SCANCHAINS section.

When this application option is *false*, all the floating scan cells are written out in the order of the original scandef.

When this application option is *true*, all the floating scan cells are written out in netlist order.

See Also

- [write_def](#)

file.def.set_pg_mask_fixed

Controls whether the *read_def* command sets all power and ground shape and vias masks to fixed.

Data Types

Boolean

Default false

Description

This application option controls whether the *read_def* command sets all power and ground shape and via masks to fixed. This applies to shapes and vias defined on power and ground ports and nets in the DEF PINS, NETS, and SPECIALNETS sections.

When this application option is *false*, the power and ground shape and vias masks are not marked as fixed. The shape *is_mask_fixed* attribute is set to false. The via *is_lower_mask_fixed*, *is_cut_mask_fixed*, and *is_upper_mask_fixed* attributes are set to false.

When this application option is *true*, the power and ground shape and vias masks are marked as fixed. The shape *is_mask_fixed* attribute is set to true, if the shape has a mask constraint. The via *is_lower_mask_fixed*, *is_cut_mask_fixed*, and *is_upper_mask_fixed* attributes are set to true, if the respective mask constraint exists.

See Also

- [read_def](#)

f

file.def.skip_combinational_cell_check

Controls whether the *read_def* command skips scan chain bit length checking on combinational cells.

Data Types

Boolean

Default false

Description

This application option controls whether the *read_def* command scan chain bit length checking on combinational cells. This affects how the DEF SCANCHAINS section is translated into the database.

When this application option is *false*, the *read_def* command checks the bit lengths specified for combinational cells in the DEF SCANCHAINS section. Combinational cells are expected to have zero bit lengths. For each combinational cell with non-zero bit length, an error is printed and the cell's bit length is set to 0.

When this application option is *true*, the *read_def* command does not check the bit lengths specified for combinational cells in the DEF SCANCHAINS section. The combinational cell's bit length is set to the DEF specified value. If the bit length is not specified, it is set to 0.

See Also

- [read_def](#)

file.def.skip_connection_of_incomplete_net

Controls whether the *write_def* command skips the net connections when outputting the net shape or via to NETS and/or SPECIALNETS section.

Data Types

Boolean

Default true

Description

This app option, if set to false, indicates that the *write_def* command will output the net connections.

This app option, if set to true (default), indicates that the *write_def* command will skip the net connections if and only if user specifies net shape or via as the input of option -objects.

See Also

- [get_app_option_value](#)
- [write_def](#)
- [report_app_options](#)
- [set_app_options](#)

file.def.skip_net_connection

Controls whether the `write_def` command skips the net connections based on net type when outputting the NETS and/or SPECIALNETS section.

Data Types

list of net types

Default empty

Description

This app option, when set to empty(default), indicates that the `write_def` command will output the all net connections.

The allowable values of net types are: power ground signal

This app option, if set to a list of net types, indicates that the `write_def` command will skip the net connections of those types.

For example, prompt > `set_app_options -name file.def.skip_net_connection -value { power ground }` prompt > `write_def design.def` This will result in `write_def` skipping the power and ground net connections in the `design.def` file.

See Also

- [get_app_option_value](#)
- [write_def](#)
- [report_app_options](#)
- [set_app_options](#)

file.def.split_via_matrix

Controls whether the `write_def` command splits non-fully-populated via matrix to multiple DEF via arrays and vias.

Data Types

Boolean

Default true

Description

The application option is used to reduce DEF file size.

When it is true, `write_def` will split non-fully-populated via matrix to multiple DEF via arrays and vias if and only if: 1. Its orientation is R0. 2. Each base via of it has the same coloring.

Otherwise, the non-fully-populated via matrix is written as individual vias.

See Also

- [write_def](#)

file.def.support_property_definitions

Controls whether the `read_def` and `write_def` commands preserve the DEF property definitions and values.

Data Types

Boolean

Default false

Description

This application option controls whether the `read_def` and `write_def` commands preserve the DEF property definitions and values. Component, design, group, net, nondefault rule, region, row, and special net properties are supported. Component pin properties are not supported.

During `read_def`, if the application option is true, DEF properties are created in the block. Note that these properties are not visible or accessible in the user interface. They are only preserved in the database, so that they can be output by the `write_def` command.

During `write_def`, if the application option is true, DEF properties are written to the output file. Note that object type specific property definitions are written if and only if the corresponding object type is included in the output. For example, if cells are included, then component property definitions are written out. But if cells are excluded, component property definitions are not written out.

f

See Also

- [read_def](#)
- [write_def](#)

file.def.support_via_matrix

Controls whether the *read_def* command creates a via matrix instead of individual vias for a DEF via array.

Data Types

Boolean

Default false

Description

This application option controls whether the *read_def* command creates a via matrix instead of individual vias for a DEF via array.

When this application option is *true*, the *read_def* command creates a DEF via array as a via matrix in the block for the following cases:

- The DEF via array is defined with "(x y) [MASK viaMaskNum] viaName [orient] DO numx BY numY STEP stepX stepY".
- The DEF via array is individual vias defined with "(x y) [MASK viaMaskNum] viaName [orient]" and the `#SNPS_VIA_MATRIX ...` pragma.

Otherwise, a DEF via array is created as individual vias.

See Also

- [read_def](#)

file.def.support_voltage_area

Controls whether the *read_def* command creates a voltage area instead of a move bound for a DEF region with FENCE type, and whether *write_def* writes voltage areas to the DEF file.

Data Types

Boolean

Default false

f

Description

This application option controls whether the *read_def* command creates a voltage area instead of a move bound for a DEF region with FENCE type. Similarly, *write_def* outputs voltage areas as DEF FENCE regions when this option is set to *true*.

By default, *read_def* command creates DEF region with FENCE type as exclusive move bound. When this application option is *true*, *read_def* will create voltage area instead of move bound. Note : When the voltageArea property exists under the PROPERTYDEFINITIONS section, then any FENCE regions that have the voltageArea property (with value 1) are treated as voltage areas, and all other FENCE regions are treated as move bounds. Even if this app option is set to true.

See Also

- [read_def](#)

file.def.write_matching_net_names

Controls whether the *write_def* command writes unconnected ports with net names matching the port names to the DEF PINS section.

Data Types

Boolean

Default false

Description

The *write_def* command writes unconnected ports with dummy net connections, because the DEF PINS syntax requires each port to be connected to a net by a "+ NET name" statement. This application option controls the names used for the dummy nets in the generated DEF.

By default, the *write_def* command names each dummy net as "_dummysnet" plus an integer to make the net name unique in the generated DEF.

If you set the application option to *true*, the *write_def* command names each dummy net with the same name as the port. However if a net already exists with that name, an integer is appended to the end to make the net name unique in the generated DEF.

See Also

- [read_def](#)
- [write_def](#)

file.def.wrong_way_wiring_to_special_net

Controls whether the *write_def* command writes wiring with wrong-way default widths to the DEF SPECIALNETS section instead of the NETS section.

Data Types

Boolean

Default false

Description

This application option controls whether the *write_def* command writes wiring with wrong-way default widths to the DEF SPECIALNETS section instead of the NETS section.

This application option has effect only when writing DEF 5.8 and newer. DEF 5.7 does not support wrong-way default widths, so this application option is a no-op in that case. In DEF 5.7, wrong-way default width wires are output always to the SPECIALNETS section.

When this application option is *false* (and writing DEF 5.8 or newer), the *write_def* command writes wrong-way default width wires to the DEF NETS section. This is in compliance with the DEF 5.8 wrong-way wiring syntax.

When this application option is *true* (and writing DEF 5.8 or newer), the *write_def* command writes wrong-way default width wires to the DEF SPECIALNETS section.

See Also

- [read_def](#)
- [write_def](#)

file.gds.allow_incompatible_units

Specifies whether to allow incompatible units to be specified to the *write_gds* command.

Data Types

Boolean

Default false

Description

By default, *write_gds* will error out when incompatible units is specified. However, if *file.gds.allow_incompatible_units* is set to true, *write_gds* will give a warning and then continue instead of error out.

f

See Also

- [get_app_option_value](#)
- [write_gds](#)
- [report_app_options](#)
- [set_app_options](#)

file.gds.check_mask_constraints

Used by "*write_gds*" command to check mask constraints for shapes in the block.

Data Types

string

Default "none"

Description

The option is to check if there are mismatches for layers and metal, via, fill shapes or tracks. There are 3 check severities. When you set the value to *none*, there will be no check. This is the default severity. When you set the value to *loose*, if the shape is uncolored but the layer is double pattern layer, the shape will be ignored; if the shape is colored but the layer is uncolored layer, the shape will still be outputted. A warning message will be shown for each of the situation. When you set the value to *strict*, either of the mismatch will stop the command and error out, with an error message outputted. This option is an enhancement for app option *file.gds.ignore_uncolored_shapes*, which will finally be deprecated. We recommend you to use this option instead.

See Also

- [write_gds](#)
- [get_app_option_value](#)
- [set_app_options](#)

file.gds.check_mask_constraints_exclude_layers

Used by "*write_gds*" command to list the layers you don't want to check mask constraints for shapes in the block.

Data Types

list

f

Default null**Description**

This option is used together with *file.gds.check_mask_constraints*. You can specify the layers you don't want to check by this option.

```
prompt> set_app_options -name "file.gds.check_mask_constraints" -value
strict
prompt> set_app_options -name
"file.gds.check_mask_constraints_exclude_layers" -value {M2 M3}
```

See Also

- [write_gds](#)
- [get_app_option_value](#)
- [set_app_options](#)

file.gds.contact_prefix

Used by "*write_gds*" command to prefix the contact via names to be written as Structures into the GDSII file.

Data Types*string***Default** \$\$**Description**

This option set the prefix to be used in the name when the contact via name is to be written as Structures into the GDSII file. This option is design-scoped.

The default value for the prefix to be used is \$\$.

Examples

The following example defines the contact prefix to be used for naming the contact via Structures in the output GDSII file for the current design.

```
prompt> set_app_options -name "file.gds.contact_prefix" -value "PREFIX_"
```

See Also

- [write_gds](#)
- [get_app_option_value](#)

- [report_app_options](#)
- [set_app_options](#)

file.gds.create_custom_via

Used by "*read_gds*" and "*trace_connectivity*" commands in *icc2_lm_shell* to enable or disable creation of custom vias instances from the cut shapes read from the input GDSII file.

Data Types

Boolean

Default false

Description

This option enables or disables creating custom vias from cut geometries by the *read_gds* and *trace_connectivity* commands. The default behavior is to retain the cut geometries in the input GDSII file as cut shapes. This option can be used to retain the input cut geometries as cut shapes only or generate custom vias. This option will not be required to be enabled in most general cases, so this option should be set only in most rare cases.

The default value is false and hence the geometries read from the via layer of the input GDSII file are created as cut shapes which are normal geometries in the VIA layer. So by default, they are not created as custom via instances. You must set this option before running the *read_gds* or *trace_connectivity* commands.

Examples

The following example enables the creation of custom via from the cut shapes during GDSII import and the connectivity tracing and convert them to custom via.

```
prompt> set_app_options -name "file.gds.create_custom_via true" -value  
true
```

See Also

- [read_gds](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

file.gds.enable_half_metal_generation

Used by the *write_gds* command to enable half metal generation when command option "-connect_below_cut_metal" is specified.

Data Types

Boolean

Default false

Description

Setting this application option to true to make *write_gds* generate the metal below the cut metal with half width of the cut metal when there is only one metal shape (including non-zero spacing routing blockage) abutting the cut metal.

By default (*false*), *write_gds* generates the metal below cut metal only when there are two metal shapes abutting the cut metal.

You must set this option before running the *write_gds* command.

Examples

The following example is to make *write_gds* command to enable half metal generation.

```
prompt> set_app_options -name file.gds.enable_half_metal_generation \\  
        -value true
```

See Also

- [write_gds](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

file.gds.enable_multiple_rules_per_layer

Used by "*write_gds*" command to enable layout data to be written as several records into GDSII file, corresponding to the matched layer mapping rules.

Data Types

boolean

Default false

f

Description

By default, a layout shape or text is output into GDSII file corresponding to the last matched layer mapping rule. For example, a layer shape will be output as a rectangle record on mapped GDSII layer and data type.

When the option is on, a layout shape or text is output into GDSII file corresponding to all matched layer mapping rules. For example, there are two matched layer mapping rules for a layer shape, the layer shape is output as two rectangle records on different mapped GDSII layer and data type.

Examples

The following example describes the behavior when the option is on. The layer mapping rules as below: 17:0:*:* 17:0 17:0:area_fill:* 19:0 17:0:routing_blockage:* 117:0 17:*:*:* 217:0 17:0:*:* 317:0

A polygon for wiring on layer 17, and its purpose number is 0, the matched layer mapping rules are following ones:

17:0:*:*	17:0
17:*:*:*	217:0
17:0:*:*	317:0

When the option is on, there are three polygons are output for the polygon, corresponding to above three GDSII layer and data type pairs.

The example to set the app option as below:

```
prompt> set_app_options -name "file.gds.enable_multiple_rules_per_layer"
-value true
```

See Also

- [write_gds](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

file.gds.enable_special_snapping

Used by the *write_gds* command to enable special snapping for path.

Data Types

Boolean

Default false

Description

Setting this application option to true to make `write_gds` use special snapping rule to snap path.

By default (*false*), `write_gds` uses NDM snapping API to snap path.

You must set this option before running the `write_gds` command.

Examples

The following example enables the special snapping.

```
prompt> set_app_options -name file.gds.enable_special_snapping \\  
        -value true
```

See Also

- [write_gds](#)
- [write_oasis](#)
- [merge_stream](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

file.gds.exclude_layers

Specifies the rules for mapping between the input layers and the layers which are to be used for excluding geometries from the specified input layers.

Data Types

list

Default null

Description

This option specifies the mapping between input layers and other layers which are to be used for exclusion. The geometries in the other layers are subtracted from same colored geometries of the input layer during the `read_gds` and `trace_connectivity` commands.

The tool uses the geometries in a specific color of the mapped layers to subtract same colored geometries from the input layers. Multiple mapping between the same input layer and exclude layers can be specified. The exclusions works by subtracting geometries of the same color using relationship specified in the mapping.

f

You must set this option before running the *read_gds* or *trace_connectivity* commands.

You can specify the mapping for one or more input layers and excluded layers and color. The syntax used to specify each mapping is

```
{ {metal_layer1:purpose1 exclude_layer1[:exclude_purpose1][:mask_name]}
  {metal_layer2:purpose2 exclude_layer2[:exclude_purpose2][:mask_name]}
  ... }
```

You specify the layer names and purpose to be excluded from input layer names and purpose by using the information in the technology file.

Examples

The following example defines the exclude layers mapping for the M1 and M2 metal layers.

In the example below, mask_one of M1 will be subtracted by the mask_one geometries of EM1. And if layer M2 has two mask, mask_one geometries of layer M2 will be subtracted by the mask_one geometries of EM2. mask_two of EM2 will remove geometries occupied by the mask_two of M2. Layers EM1 and EM2 will not be modified. Only input layers M1 and M2 will change.

```
prompt> set_app_options -name "file.gds.exclude_layers" -value { {M1
  EM1:mask_one} {M2 EM2} }
```

See Also

- [read_gds](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

file.gds.ignore_uncolored_shapes

Used by "*write_gds*" command to ignore uncolored shapes in the block.

Data Types

Boolean

Default false

Description

This option is to control if uncolored shapes which includes the shapes of via, via matrix and shape pattern are written out.

f

When the option is on, all the uncolored shapes will be ignored.

When the option is off, uncolored shapes are also written out.

See Also

- [write_gds](#)
- [get_app_option_value](#)
- [set_app_options](#)

file.gds.ignore_uncolored_shapes_exclude_layers

Used by "*write_gds*" command to exclude layers while using *file.gds.ignore_uncolored_shapes* option.

Data Types

list

Default empty

Description

This option is to specify exclude layers when *file.gds.ignore_uncolored_shapes* option is true.

When the list of exclude layers provided, all the uncolored shapes of those layers, will not be ignored.

When the list of exclude layers not provided, all uncolored shapes will be ignored.

See Also

- [write_gds](#)
- [get_app_option_value](#)
- [set_app_options](#)

file.gds.include_sub_cell_in_block

Used by "*read_gds*" command to include sub cells in the block.

Data Types

Boolean

Default false

f

Description

This option is to control how to load the sub cells when user specifies block mapping file.

Take the following mapping as an example. *CORE*:add macro *:ignore

When the option is on, all the blocks in *CORE* block's hierarchy will be loaded, other blocks will be ignored.

When the option is off, only the block which name matches *CORE* will be loaded.

See Also

- [read_gds](#)
- [get_app_option_value](#)
- [set_app_options](#)

file.gds.max_cell_name_length

Used by "write_gds" command to limit maximum cell name length.

Data Types

int

Default 0

Description

This options specifies the name length limit for cells. If the cell name exceeds the length limit, the write_gds command truncates the cell name and automatically resolves any naming conflicts. The default maximum length is 32.

This option has higher priority than write_gds option -long_names.

You must set this option before running the *write_gds* command.

Examples

The following example sets the maximum cell name length to 40.

```
prompt> set_app_options -name file.gds.max_cell_name_length -value 40
```

See Also

- [write_gds](#)
- [get_app_option_value](#)

- [report_app_options](#)
- [set_app_options](#)

file.gds.net_text_on_layers

Used by "*write_gds*" command to output net text on specific layers.

Data Types

list

Default null

Description

This option is used together with command option "*-output_net_text*". You can specify text layers so that only net text on these layers will be output. It also respects to "*-net_text_on*" option.

```
prompt> set_app_options -name "file.gds.net_text_on_layers" -value {M2  
M3}
```

See Also

- [write_gds](#)
- [get_app_option_value](#)
- [set_app_options](#)

file.gds.output_first_same_name_cell

Controls whether the *write_gds* command writes out only the first encountered cell when there are multiple cells with the same name.

Data Types

Boolean

Default false

Description

This application option controls whether the *write_gds* command writes only the first encountered cell when there are multiple cells with the same name.

By default (*false*), the *write_gds* command renames the cells with the same name and writes them as different structures in the output GDSII file.

f

When *true*, the *write_gds* command writes out only the first encountered cell and ignores additional cells with the same name.

You must set this option before running the *write_gds* command.

Examples

The following example enables streaming out only the first cell when multiple cells have the same name. The default action is to rename the cells and write all the multiple cells as different structures.

```
prompt> set_app_options -as_user_default \\  
        -name file.gds.output_first_same_name_cell -value true
```

See Also

- [write_gds](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

file.gds.pg_via_arrays

Used by the *write_gds* command to compact individual PG vias into via arrays when generating GDSII layout files.

Data Types

Boolean

Default false

Description

This application option controls whether the *write_gds* command compacts PG vias into via arrays in the layout files, which reduces the GDSII file size.

You must set this option before running the *write_gds* command.

See Also

- [write_gds](#)
- [write_oasis](#)
- [get_app_option_value](#)

f

- [report_app_options](#)
- [set_app_options](#)

file.gds.port_type_map

Used by the "read_gds" and "trace_connectivity" commands in *icc2_lm_shell* to identify port types correctly

Data Types

list_of_pairs

Default null

Description

This option is used to specify the mapping between port type and port name. This map is used in commands like *read_gds* and *trace_connectivity* to correctly identify the port type. This option shall be set before calling *read_gds* or *trace_connectivity* commands. The first element in the value pair is the port type and the second element in the pair is the name of the port name to be matched. One port type can be used with multiple port names.

The following are the port types supported now.

- *power*
- *ground*
- *pwell*
- *nwell*

Examples

The following example shows how the application option is to be set

```
prompt> set_app_options {file.gds.port_type_map {{power VDD} {ground  
VSS}}}
```

See Also

- [read_gds](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

file.gds.prefix_for_fill

Used by "*write_gds*" command to prefix the fill cell names with the owner design name that is to be written as Structures into the GDSII file.

Data Types

Boolean

Default `true`

Description

This option enables or disables using the design name as prefix to the fill cell name that is to be written as Structures into the GDSII file. This option is design-scoped.

The default value is true which means the design name is used as a prefix to the fill cell name.

Examples

The following example disables using the design name as prefix for the fill cells to be written as Structures in the output GDSII file for the current design.

```
prompt> set_app_options -name "file.gds.prefix_for_fill" -value false
```

See Also

- [write_gds](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

file.gds.read_enable_multiple_rules_per_layer

Used by "*read_gds*" command to enable the shape records in the GDSII file to be read as several shapes into database, corresponding to the matched multiple layer mapping rules.

Data Types

boolean

Default `false`

Description

By default, a shape or text record in the GDSII file is created according to the last matched layer mapping rule.

f

When the option is on, a shape or text record in the GDSII file is created corresponding to all matched layer mapping rules. For example, there are two matched layer mapping rules for a shape record, the shape record is created as two rectangles in the database on different mapped layer and data type.

Examples

The following example describes the behavior when the option is on. Assuming there is a shape record on layer 531 in the GDSII file, two shapes with mask_one mask_constraint attribute will be created on layer 37 and 31 respectively in the database.

The layer mapping rules as below: 37:::mask_one 531" 31:::mask_one 531"

The example to set the app option as below:

```
prompt> set_app_options -name
"file.gds.read_enable_multiple_rules_per_layer" -value true
```

See Also

- [read_gds](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

file.gds.rotate_pin_text_by_access_direction

Used by "*write_gds*" command to enable or disable controlling of the rotation of the TEXT object when the pin object is written into the GDSII file.

Data Types

Boolean

Default false

Description

This option enables or disables controlling of the value of the ANGLE token in the TEXT object of a pin during the *write_gds* flow. If this app-option is enabled, then the value of the ANGLE token will be dependent on the access-direction flags on the pin object. If the access-direction flags has left or right direction set then a value of R0 will be used as the TEXT object's orientation. In other cases a value of R90 will be used as the TEXT object's orientation. This option is design-scoped.

By default this option is disabled and the TEXT objects are always written with an orientation value of R0.

f

Examples

The following example defines that the rotation of the TEXT object of the pin should be controlled by the access direction flags set on the pin.

```
prompt> set_app_options -as_user_default  
{file.gds.rotate_pin_text_by_access_direction true}
```

See Also

- [write_gds](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

file.gds.shape_use_for_pg_mesh

This option provides list of shape use for pg_mesh use type which is specified in layer mapping rules.

Data Types

list

Default "stripe ring"

Description

This option is to control pg_mesh collection, when pg_mesh use type is specified in layer mapping file.

When this option is not used, by default shapes of stripe and ring shape_use are written out. When this option is used, shapes corresponding to the specified shape_use in the value are written out. Available value: "", or any combination of "stripe, ring, lib_cell_pin_connect, macro_pin_connect".

Examples

The following example specifies shape_use for pg_mesh use type.

```
prompt> set_app_options -name file.gds.shape_use_for_pg_mesh \  
-value {stripe ring lib_cell_pin_connect macro_pin_connect}
```


See Also

- [write_gds](#)
- [get_app_option_value](#)
- [set_app_options](#)

file.gds.split_via_matrix

Controls whether the *write_gds* command splits non-fully-populated via matrix to multiple AREF and SREF records.

Data Types

Boolean

Default true

Description

The application option is used to reduce GDS file size.

When it is true, *write_gds* will split non-fully-populated via matrix to multiple AREF and SREF records if and only if its mask pattern is uniform.

Otherwise, the non-fully-populated via matrix is written as multiple SREF records.

See Also

- [write_gds](#)

file.gds.text_all_pins

Used by "*write_gds*" command to write the text for all pins of the port into the GDSII file.

Data Types

bool

Default false

Description

This option set the value such that text for each pins of the port are written into the GDSII file. The default behavior is to write a single text for all pins of the port.

f

Examples

The following example specifies that the text for each pins of the port are to be written into the output GDSII file.

```
prompt> set_app_options -name "file.gds.text_all_pins" -value true
```

See Also

- [write_gds](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

file.gds.text_layer_map

Specifies the mapping between metal layers and text layers that is used for tracing ports.

Data Types

list

Default null

Description

This option specifies the mapping between metal layers and text layers that is used for tracing ports by the *read_gds* and *trace_connectivity* commands.

The tool uses the text labels in a GDSII file to associate metal shapes with a net or pin and then traces the connectivity of the labeled metal shapes to identify additional metal shapes associated with that net or pin. If the text for a port is not on the same layer as the metal shape, you must set this option to specify the layer mapping so the tool can correctly identify the ports. You must set this option before running the *read_gds* or *trace_connectivity* commands.

You can specify the mapping for one or more metal layers. The syntax used to specify each mapping is

```
{metal_layer {text_layer1[:data_type1] text_layer2[:data_type2] ...}}
```

You must specify the layer names and data type numbers by using the information in the technology file.

Examples

The following example defines the text layer mapping for the M1 and M2 metal layers.

f

```
prompt> set_app_options { file.gds.text_layer_map { {M1 M1PIN} {M2
M2PIN} } }
```

See Also

- [read_gds](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

file.gds.trace_copy_blockage_color_from_overlapped_terminal

Specifies layers in which a routing blockage of no_mask will be changed to a new mask_constraint as the overlapped terminal.

Data Types

list

Default null

Description

This option indicates "*read_gds*" and "*trace_connectivity*" commands to change a routing blockage of no_mask of the specified layers with another mask from an overlapping terminal, by setting its is_mask_fixed to true and its mask type to user_set. If multiple terminals are overlapped, the mask is got from a randomly terminal of another mask.

You must set this option before running the *read_gds* or *trace_connectivity* commands.

You can specify one or more metal layers.

The syntax used to specify each mapping is

```
{layer1[:data_type1] layer2[:data_type2] ...}
```

You specify the layer names and data type numbers by using the information in the technology file.

Examples

The following example specify layers M3 and M5 in which layers routing blockages of no_mask to get another mask from an overlapping terminal. The is_mask_fixed are set to true and mask types are set to use_set for the routing blockages.

```
prompt> set_app_option -name
file.gds.trace_copy_blockage_color_from_overlapped_terminal -value {M3
M5}
```

f

See Also

- [read_gds](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

file.gds.trace_copy_overlap_shape_from_sub_cell

Used by "*read_gds*" and "*trace_connectivity*" commands in *icc2_lm_shell* to indicate shapes in sub-level blocks are to be duplicated to the top block.

Data Types

bool

Default false

Description

This option indicates whether a shape in a sub-level block is to be duplicated to the top block when the shape is overlapped by terminals.

When the app option "file.gds.exclude_layers" is specified and the shape in the sub-level block is divided into several parts by the shape on the excluding layer in the physical hierarchy, only the part overlapping the top terminal will be duplicated to the top block.

When the non-pg terminal goes through an instance referencing a sub-level block no matter in horizon or vertical, no shape will be duplicated from the sub-level block for the instance.

A via is excluded from the terminals.

You must set this option before running the *read_gds* or *trace_connectivity* commands.

Note: This option can work for abutting, and it can't work in "top level". 'read_gds -block_map' will bring a sub-level block to top according to map file which makes the option cannot bring a pin in the sub-level block to top block as a terminal.

Examples

The following example enables to duplicate an overlapped shape during GDSII import and the connectivity tracing .

```
prompt> set_app_options -name  
file.gds.trace_copy_overlap_shape_from_sub_cell -value true
```

f

See Also

- [read_gds](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

file.gds.trace_merge_overlap_mask

Used by "*read_gds*" and "*trace_connectivity*" commands to merge the colorless via metal enclosure into overlapped color metal terminal. The default valuse is false.

Data Types

boolean

Default false

Description

When the option is on, a colorless shape is covered by a color shape, the colorless shape need to be set to the color of the colored shape. By default, this app option is false. Long run time would be expected when this app option is true.

You must set this option before running the *read_gds* or *trace_connectivity* commands.

Examples

The following example enables this feature.

```
prompt> set_app_options -name file.gds.trace_merge_overlap_mask -value  
true
```

See Also

- [read_gds](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

file.gds.trace_terminal_length

Used by "*read_gds*" and "*trace_connectivity*" commands \to indicate a long pin interacting with the block boundary side should be split into a terminal with the specified length

f

and a shape. When the option value is 0, the split doesn't happen. However, if you use `file.gds.trace_terminal_length_layer_pairs` to set a layer specific terminal length, that will be used for splitting instead of this global one.

Data Types

float

Default 0

Description

This option indicates whether a long pin interacting with the block boundary side should be split into a pin with the specified length and a shape. When the option value is 0, the split doesn't happen. When the qualified pin is more than 0, the long pin is split into a new pin and a new shape. If the pin length is less than the app option, the long pin won't be split either. The new pins and shapes inherit all other attributes from the original pin. The new pin covers the portion of the specified length including the original pin on the die boundary or out of the die boundary if it exists.

The option value should be specified in length units as configured by the `set_user_units -type length -input` command, which is micrometers (um) by default.

This option has only effect on design view.

A via is excluded from the long pins and they cannot be split.

You must set this option before running the `read_gds` or `trace_connectivity` commands.

Examples

The following example enables the long pin split during GDSII import and the connectivity tracing. the portion within the 1 user unit from the boundary of the cell is the length of the new pin, while the other portion belongs to a new generated shape.

```
prompt> set_app_options -name file.gds.trace_terminal_length -value 1
```

See Also

- [read_gds](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

f

file.gds.trace_terminal_length_4x_width

Used by "*read_gds*" and "*trace_connectivity*" commands to indicate that a long signal pin of the specified layers interacting with one of the block boundary side should be split into a terminal with the length equal to 4 times of its width.

Data Types

list of string

Default empty list

Description

This option indicates which long signal pin of the specified layer interacting with one of the block boundary side should be split into a pin with the length equal to 4 times its width

You must set this option before running the *read_gds* or *trace_connectivity* commands.

Examples

The following example, the long signal pin will be split during GDSII import and the connectivity tracing, with terminal length of 4 times its width in layer M1 and layer M2 respectively.

```
prompt> set_app_options -name file.gds.trace_terminal_length_4x_width  
-value {M1 M2}
```

See Also

- [read_gds](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

file.gds.trace_terminal_length_boundary_only

Used by "*read_gds*" and "*trace_connectivity*" commands in *lm_shell* to indicate a terminal of a port touches the top block boundary side should be kept but other terminals belong only to the same port should become a shape in a top design.

Data Types

bool

Default false

f

Description

This option indicates a terminal of a port touching the top block boundary side should be kept but other terminals belong only to the same port should become a shape in a top design.

This option has only effect on design view.

You must set this option before running the *read_gds* or *trace_connectivity* commands.

Examples

The following example enables the terminals become a shape during GDSII import and the connectivity tracing. The terminals belong a port in a top design. At least one terminal to this port touches the top design.

```
prompt> set_app_options -name  
file.gds.trace_terminal_length_boundary_only -value true
```

See Also

- [read_gds](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

file.gds.trace_terminal_length_exclude_list

This option is to provide list of ports which need to be excluded from getting trimmed. This option is used along with file.gds.trace_terminal_length* options.

Data Types

list of string

Default empty list

Description

This option provides list of ports which need to be excluded from trimming. This option can be use along with file.gds.trace_terminal_length* options.

You must set this option before running the *read_gds* or *trace_connectivity* commands.

Examples

The following example, the ports with name "Port_1" and "Port_2" will be excluded from splitting during GDSII import and the connectivity tracing.

f

```
prompt> set_app_options -name file.gds.trace_terminal_length_4x_width  
-value {M1 M2}  
prompt> set_app_options -name file.gds.trace_terminal_length_exclude_list  
-value {Ports_1 Ports_2}
```

See Also

- [read_gds](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

file.gds.trace_terminal_length_layer_pairs

Used by "*read_gds*" and "*trace_connectivity*" commands to indicate that a long pin of the specified layers interacting with the block boundary side should be split into a terminal with the specified length for that layer and a shape.

Data Types

list of {string float} pairs

Default empty list

Description

This option indicates which layer of a long pin of the specified type interacting with the block boundary side should be split into a pin with the specified length for its layer and a shape.

This option has only effect on design view.

You must set this option before running the *read_gds* or *trace_connectivity* commands.

Examples

The following example, the long PG pin will be split during GDSII import and the connectivity tracing, with terminal length of 1.5 and 1.8 in layer M1 and layer M2 respectively.

```
prompt> set_app_options -name file.gds.trace_terminal_length_layer_pairs  
{{M1 1.5} {M2 1.8}}
```

See Also

- [read_gds](#)
- [get_app_option_value](#)

f

- [report_app_options](#)
- [set_app_options](#)

file.gds.trace_terminal_length_layers

Used by "*read_gds*" and "*trace_connectivity*" commands to indicate a long pin of the specified layers interacting with the block boundary side should be split into a terminal with a specified length and a shape. When the option value is null, valid long pins of all layers are to be split. However, if you use *file.gds.trace_terminal_length_layer_pairs* to set a layer specific terminal length, long pins in those layers will be split as well.

Data Types

list

Default null

Description

This option indicates which layer of a long pin of the specified type interacting with the block boundary side should be split into a pin with a specified length and a shape. When the option value is null, the split happens to all layers.

This option has only effect on design view.

You must set this option before running the *read_gds* or *trace_connectivity* commands.

Examples

The following example enables of M1 and M2:35 layers the long PG pin split during GDSII import and the connectivity tracing.

```
prompt> set_app_options -name file.gds.trace_terminal_length_layers {M1  
M2:35}
```

See Also

- [read_gds](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

file.gds.trace_terminal_text

Specifies the mapping between metal layers and text layers that is used for tracing ports.

f

Data Types

list

Default null

Description

This option indicates "*read_gds*" and "*trace_connectivity*" commands to extract specified shapes as terminals. These shapes should be covered by the specified text and shapes in layers specified by the option. The later shapes must be reached by connections in all layers through a cut with the former shapes no matter what the traced type is.

You must set this option before running the *read_gds* or *trace_connectivity* commands.

When "*read_gds -trace_all_layers_for_port_names*" and this option both turn on. "*read_gds -trace_all_layers_for_port_names*" has the priority over the option. The specified port_names are traced in all-layer styles instead of the way specified by the option.

You can specify the mapping for one or more metal layers. The syntax used to specify each mapping is

```
{text {layer1[:data_type1] layer2[:data_type2] ...}}
```

You specify the layer names and data type numbers by using the information in the technology file.

Examples

The following example enables shapes overlapped by text 'A', and 'B' to be extracted as terminals; For text 'A', shapes in layers M3, V4, M5 connected in all layers with the shape overlapped by text 'A' can be extracted as terminals; For text 'B', shapes in layers M3 connected with the shape overlapped by text 'B' can be extracted as terminals;

```
prompt> set_app_option -name file.gds.trace_terminal_text -value {{A {M3  
V4 M5}} {B {M3}}}
```

See Also

- [read_gds](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

file.gds.trace_terminal_type

Used by "*read_gds*" and "*trace_connectivity*" commands in *icc2_lm_shell* to indicate a long pin of the specified type interacting with the block boundary side should be split into a terminal with a specified length and a shape. When the option value is default, PG and signal terminals are to be split.

Data Types

string

Default default

Description

This option indicates whether a long pin of the specified type interacting with the block boundary side should be split into a pin with a specified length and a shape. When the option value is default, the split happens to both PG and signal terminals. When the option is case-insensitive PG or signal, a qualified PG or signal terminal to be split.

This option has only effect on design view.

A via is excluded from the long pins and they cannot be split.

You must set this option before running the *read_gds* or *trace_connectivity* commands.

Examples

The following example enables the long PG pin split during GDSII import and the connectivity tracing.

```
prompt> set_app_options -name file.gds.trace_terminal_type PG
```

See Also

- [read_gds](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

file.gds.trace_unmapped_text

Enables or disables tracing on all routing layers for the texts not mapped to any routing layers when used by the "*read_gds*" and "*trace_connectivity*" commands in *icc2_lm_shell*.

f

Data Types

Boolean

Default false

Description

This option enables or disables tracing on all routing layers for text labels not mapped to a routing layer by the *read_gds* and *trace_connectivity* commands. The default behavior is to ignore use of such text labels for identification of pins. This option must be enabled in most rare cases, so this option must be disabled in most cases.

The tool uses the text labels in a GDSII file to associate metal shapes with a net or pin and then traces the connectivity of the labeled metal shapes to identify additional metal shapes associated with that net or pin. If the text for a port is not on the same layer as the metal shape, you must set the option *file.gds.text_layer_map* to specify the layer mapping so the tool can correctly identify the ports. In some rare cases only, you might need to set this option to make the tool identify pin geometries on all routing layers for such unmapped text labels. So, this option must be used to disable the setting to trace for unmapped texts. You must set this option before running the *read_gds* or *trace_connectivity* commands.

Examples

The following example defines that tracing must be enabled for all texts not mapped to the routing layers.

```
prompt> set_app_options { file.gds.trace_unmapped_text true}
```

See Also

- [read_gds](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

file.gds.use_only_mapped_text

Controls whether port tracing uses only the text specified by the *file.gds.text_layer_map*.

Data Types

Boolean

Default false

f

Description

This option controls whether the *read_gds* and *trace_connectivity* commands use only the text specified by the *file.gds.text_layer_map* application option when tracing ports.

The tool uses the text labels in a GDSII file to associate metal shapes with a net or pin and then traces the connectivity of the labeled metal shapes to identify additional metal shapes associated with that net or pin.

To use only the text mapped by the *file.gds.text_layer_map* application option for identifying the ports, set this option to *true* before running the *read_gds* or *trace_connectivity* command.

Examples

The following example defines the text layer mapping for the M1 and M2 metal layers and uses only that text for port tracing.

```
prompt> set_app_options -name file.gds.text_layer_map \\  
-value { {M1 M1PIN} {M2 M2PIN} }  
prompt> set_app_options -name file.gds.use_only_mapped_text \\  
-value true
```

See Also

- [read_gds](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

file.gds.write_metal_below_cut_to_top

Used by the *write_gds* command to write the metal below the cut metal to the parent cell when the *design_type* of the cell which owns the cut metal is not module and command option "-connect_below_cut_metal" is specified.

Data Types

Boolean

Default false

Description

Setting this application option to true to make *write_gds* write the metal below the cut metal to the parent cell instead of the cell which owns the cut metal when the *design_type* of the cell which owns the cut metal is not module.

f

By default (*false*), `write_gds` writes the metal below cut metal to the cell which owns the cut metal.

You must set this option before running the `write_gds` command.

Examples

The following example is to make `write_gds` command to turn on this app option.

```
prompt> set_app_options -name file.gds.write_metal_below_cut_to_top \\  
-value true
```

See Also

- [write_gds](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

file.lef.allow_empty_pin

Specifies whether to allow no PORT specified for PIN statement in the LEF file.

Data Types

Boolean

Default false

Description

A PIN statement without PORT specified is not a valid LEF syntax according to the LEF manual. By default, the `read_lef` command will output LEFR-056 error message and abort if the PIN is of signal type; output LEFR-057 warning message and continue if the PIN is not of signal type.

This app option, if set to true, indicates that the `read_lef` command will not output LEFR-056 and LEFR-057 message when input LEF file contains a PIN statement without PORT specified. The PIN will be created without any geometry.

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

f

- [LEFR-056](#)
- [LEFR-057](#)

file.lef.auto_rename_conflict_sites

Specifies whether to automatically rename a conflict site definition.

Data Types

Boolean

Default false

Description

A conflict site definition means a same name site definition exists with different size. The conflict site definition is usually an error condition (*LEFR-018*) and causes *read_lef* to fail. However, if *file.lef.auto_rename_conflict_sites* is set to true, an attempt to rename the site definition is done. The renamed site definition has a name such as *<lefFileName>.<siteName>*. Here, *<lefFileName>* is the base name of the LEF file that is reading; *<siteName>* is the conflict site definition name. If *<lefFileName>.<siteName>* also exists, then *<lefFileName>.<siteName>_0*, *<lefFileName>.<siteName>_1*,... is tried. When a proper site definition name is found, the conflict site definition is created using this new name and *LEFR-026* is reported. After reading all data in the LEF file, if the renamed site definition has never been referred by any macros, then the site definition is removed, and message *LEFR-038* is reported. Since there is possible that renamed site definition is removed eventually, *LEFR-039* is reported when *file.lef.auto_rename_conflict_sites* is set to true.

See Also

- [LEFR-018](#)
- [LEFR-026](#)
- [LEFR-038](#)
- [LEFR-039](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

file.lef.derive_forbidden_abutment_placement_rule

Controls whether the *read_lef* command converts PRF forbidden abutment rules from LEF file.

Data Types

Boolean

Default false

Description

This application option controls whether the *read_lef* command convert the LEF58_CELLEDGESPACINGTABLE and LEF58_EDGETYPE properties from LEF to PRF forbidden abutment rules.

This application option will work when application option 'file.lef.derive_forbidden_placement_rule' is set to true.

When this application option is *false*, the *read_lef* command convert the TOP/BOTTOM syntax to PRF forbidden spacing rules.

When this application option is *true*, the *read_lef* command convert the TOP/BOTTOM syntax to PRF forbidden abutment rules.

Take the following LEF file as an example, when define LEF58_CELLEDGESPACINGTABLE in PROPERTYDEFINITIONS section and LEF58_EDGETYPE property in MACRO, the *read_lef* command will convert them to PRF rules. 1. Covert CELLEDGESPACINGTABLE to PRF forbidden_vertical_abutment_pairs rules. 2. Covert LEF58_EDGETYPE in MACRO to cell's vertical_abutment_labels_top / vertical_abutment_labels_bottom.

```
PROPERTYDEFINITIONS LIBRARY LEF58_CELLEDGESPACINGTABLE STRING "
CELLEDGESPACINGTABLE EDGETYPE GROUPC GROUPD EXACT 0.2 ; " ; MACRO
LEF58_EDGETYPE STRING ; END PROPERTYDEFINITIONS MACRO TEST SIZE
32.000 BY 252.000 ; PROPERTY LEF58_EDGETYPE " EDGETYPE TOP GROUPA
RANGE 0 2.16 ; EDGETYPE BOTTOM GROUPA RANGE 0 2.16 ; EDGETYPE LEFT
GROUPA RANGE 0 2.16 ; EDGETYPE RIGHT GROUPA RANGE 0 2.16 ; END TEST
```

See Also

- [write_lef](#)

file.lef.derive_forbidden_placement_rule

Controls whether the *read_lef* command converts PRF rules from LEF file.

f

Data Types

Boolean

Default false**Description**

This application option controls whether the *read_lef* command convert the LEF58_CELLEDGESPACINGTABLE and LEF58_EDGETYPE properties from LEF to PRF rules.

When this application option is *false*, the *read_lef* command does not convert the syntax.

When this application option is *true*, the *read_lef* command convert the syntax.

Take the following LEF file as an example, when define LEF58_CELLEDGESPACINGTABLE in PROPERTYDEFINITIONS section and LEF58_EDGETYPE property in MACRO, the *read_lef* command will convert them to PRF rules. 1. Covert CELLEDGESPACINGTABLE to PRF forbidden_vertical_spacing rules. 2. Covert LEF58_EDGETYPE in MACRO to cell's vertical_spacing_labels.

```
PROPERTYDEFINITIONS LIBRARY LEF58_CELLEDGESPACINGTABLE STRING "
CELLEDGESPACINGTABLE EDGETYPE GROUPC GROUPD EXACT 0.2 ; " ; MACRO
LEF58_EDGETYPE STRING ; END PROPERTYDEFINITIONS MACRO TEST SIZE
32.000 BY 252.000 ; PROPERTY LEF58_EDGETYPE " EDGETYPE TOP GROUPA
RANGE 0 2.16 ; EDGETYPE BOTTOM GROUPA RANGE 0 2.16 ; EDGETYPE LEFT
GROUPA RANGE 0 2.16 ; EDGETYPE RIGHT GROUPA RANGE 0 2.16 ; END TEST
```

See Also

- [write_lef](#)

file.lef.derive_must_join_for_pg

Specifies how to derive the PROPERTY LEF58_MUSTJOINALLPORTS for PG pins in the LEF file.

Data Types*Enum***Default** false**Description**

For LEF PIN with PROPERTY LEF58_MUSTJOINALLPORTS and USE POWER/GROUND, the tool will set different attributes based on the setting of this option.

The valid values for this option are

f

- *false* (the default)

The tool does not derive LEF58_MUSTJOINALLPORTS for PG pin.

- *pattern_must_join*

The tool set *pattern_must_join* attribute to true for PG pin with LEF58_MUSTJOINALLPORTS.

- *must_join_group*

The tool set terminal's *must_join_group* attribute to same value for PG pin with LEF58_MUSTJOINALLPORTS.

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

file.lef.derive_must_join_for_signal

Specifies how to derive the PROPERTY LEF58_MUSTJOINALLPORTS for signal pins in the LEF file.

Data Types

list

Default fBpattern_must_join

Description

For LEF PIN with PROPERTY LEF58_MUSTJOINALLPORTS and USE SIGNAL, the tool will set different attributes based on the setting of this option.

The valid values for this option are

- *pattern_must_join* (the default)

The tool set *pattern_must_join* attribute to true for signal pin with LEF58_MUSTJOINALLPORTS.

- *false*

The tool does not derive LEF58_MUSTJOINALLPORTS for signal pin.

f

- *must_join_group*

The tool set terminal's *must_join_group* attribute to same value for signal pin with LEF58_MUSTJOINALLPORTS.

Usage: `set_app_options -name file.lef.derive_must_join_for_signal -value { inputPinVal outputPinVal }` The input signal pins will honor inputPinVal which is one of { `pattern_must_join` | `must_join_group` | `false` } The output signal pins will honor outputPinVal setting which is one of { `pattern_must_join` | `must_join_group` | `false` } For the other signal pins(for example: inout pins), keep the default value(`pattern_must_join`).

For example: To set input pin as `must_join_group` and output pin as `pattern_must_join`:
`set_app_options -name file.lef.derive_must_join_for_signal -value { must_join_group pattern_must_join }`

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

file.lef.non_real_cut_obs_mode

Specifies whether to distinguish non-real cut routing blockages from regular cut routing blockages of lef cut obstructions.

Data Types

Boolean

Default `true`

Description

When this app option is true, the `read_lef` command will set a lef cut obstruction as zero-spacing if it does not have `SPACING` and `DESIGNRULEWIDTH` syntax and meets any of the following conditions:

- is not rectangular
- the width and height attributes does not match any of the following values
 - the minimum width of the layer
 - the default width of the layer

f

- the values in cutHeightTbl and cutWidthTbl of the layer
- the values of cut size of any simple via definitions of the layer

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

file.lef.output_non_real_blockages

Specifies whether to output non-real blockages from write_lef.

Data Types

Boolean

Default false

Description

This app option, if set to true, indicates that the *write_lef* command will output non-real blockages. Following types of blockages are non-real blockages:

- the is_zero_spacing attribute is true
- converted from IC Compiler no-signal route guides or system layer blockages

This app option, if set to false (default), indicates that the *write_lef* command will not output non-real blockages.

See Also

- [get_app_option_value](#)
- [write_lef](#)
- [report_app_options](#)
- [set_app_options](#)

file.lef.output_non_real_blockages_for_macro**Data Types**

Boolean

f

Default true**Description**

This app option, if set to true (default), indicates that the *write_lef* command will output Macro non-real blockages. Following types of blockages are non-real blockages:

- the *is_zero_spacing* attribute is true
- converted from IC Compiler no-signal route guides or system layer blockages

This app option, if set to false, indicates that the *write_lef* command will not output macro non-real blockages.

See Also

- [get_app_option_value](#)
- [write_lef](#)
- [report_app_options](#)
- [set_app_options](#)

file.lef.write_parasitic_layers

Specifies whether to output layers with property 'isParasiticOnlyLayer' or 'isParasiticOnlyConductorLayer' in *write_lef*.

Data Types*Boolean***Default** false**Description**

This app option, if set to true, indicates that the *write_lef* command will output layers with property 'isParasiticOnlyLayer' or 'isParasiticOnlyConductorLayer'. This app option, if set to false (default), indicates that the *write_lef* command will not output those kind of layers.

See Also

- [get_app_option_value](#)
- [write_lef](#)
- [report_app_options](#)
- [set_app_options](#)

f

file.merge.disable_merge_buffer_refill

Used by the *merge_stream* command to control whether the merge buffer refill mechanism is enabled when merging GDSII and OASIS layout files.

Data Types

Boolean

Default false

Description

This application option disables the merge buffer refill mechanism when merging GDSII and OASIS layout files.

By default (*false*), the merge buffer refill mechanism is enabled, which can improve the *merge_stream* runtime when merging GDSII and OASIS layout files.

You must set this option before running the *merge_stream* command.

Examples

The following example disables the merge buffer refill mechanism.

```
prompt> set_app_options -name file.merge.disable_merge_buffer_refill \\  
        -value true
```

See Also

- [write_gds](#)
- [write_oasis](#)
- [merge_stream](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

file.oasis.allow_incompatible_units

Specifies whether to allow incompatible units to be specified to the *write_oasis* command.

Data Types

Boolean

Default false

f

Description

By default, `write_oasis` will error out when incompatible units is specified. However, if `file.oasis.allow_incompatible_units` is set to true, `write_oasis` will give a warning and then continue instead of error out.

See Also

- [get_app_option_value](#)
- [write_oasis](#)
- [report_app_options](#)
- [set_app_options](#)

file.oasis.check_mask_constraints

Used by "`write_oasis`" command to check mask constraints for shapes in the block.

Data Types

string

Default "none"

Description

The option is to check if there are mismatches for layers and metal, via, fill shapes or tracks. There are 3 check severities. When you set the value to *none*, there will be no check. This is the default severity. When you set the value to *loose*, if the shape is uncolored but the layer is double pattern layer, the shape will be ignored; if the shape is colored but the layer is uncolored layer, the shape will still be outputted. A warning message will be shown for each of the situation. When you set the value to *strict*, either of the mismatch will stop the command and error out, with an error message outputted. This option is an enhancement for app option `file.oasis.ignore_uncolored_shapes`, which will finally be deprecated. We recommend you to use this option instead.

See Also

- [write_oasis](#)
- [get_app_option_value](#)
- [set_app_options](#)

f

file.oasis.check_mask_constraints_exclude_layers

Used by "*write_oasis*" command to list the layers you don't want to check mask constraints for shapes in the block.

Data Types

list

Default null

Description

This option is used together with *file.oasis.check_mask_constraints*. You can specify the layers you don't want to check by this option.

```
prompt> set_app_options -name "file.oasis.check_mask_constraints" -value
strict
prompt> set_app_options -name
"file.oasis.check_mask_constraints_exclude_layers" -value {M2 M3}
```

See Also

- [write_oasis](#)
- [get_app_option_value](#)
- [set_app_options](#)

file.oasis.contact_prefix

Used by "*write_oasis*" command to prefix the contact via names to be written as Structures into the OASIS file.

Data Types

string

Default \$\$

Description

This option set the prefix to be used in the name when the contact via name is to be written as Structures into the OASIS file. This option is design-scoped.

The default value for the prefix to be used is \$\$.

Examples

The following example defines the contact prefix to be used for naming the contact via Structures in the output OASIS file.

f

```
prompt> set_app_options -as_user_default -name  
"file.oasis.contact_prefix" -value "PREFIX_"
```

See Also

- [write_oasis](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

file.oasis.create_custom_via

Used by "*read_oasis*" commands in *icc2_lm_shell* to enable or disable creation of custom vias from the cut shapes read from the input OASIS file.

Data Types

Boolean

Default false

Description

This option enables or disables creating custom vias from cut geometries by the *read_oasis* command. The default behavior is to retain the cut geometries in the input OASIS file as cut shapes. This option can be used to retain the input cut geometries as cut shapes only or generate custom vias. This option will not be required to be enabled in most general cases, so this option should be set only in most rare cases.

The default value is false and hence generates cut shapes from cut layer geometries. You must set this option before running the *read_oasis* command.

Examples

The following example enables the creation of custom via from the cut shapes during OASIS import and the connectivity tracing and convert them to custom via.

```
prompt> set_app_options -name "file.oasis.create_custom_via" -value true
```

See Also

- [read_oasis](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

f

file.oasis.enable_half_metal_generation

Used by the *write_oasis* command to enable half metal generation when command option "-connect_below_cut_metal" is specified.

Data Types

Boolean

Default false

Description

Setting this application option to true to make *write_oasis* generate the metal below the cut metal with half width of the cut metal when there is only one metal shape (including non-zero spacing routing blockage) abutting the cut metal.

By default (*false*), *write_oasis* generates the metal below cut metal only when there are two metal shapes abutting the cut metal.

You must set this option before running the *write_oasis* command.

Examples

The following example is to make *write_oasis* command to enable half metal generation.

```
prompt> set_app_options -name file.oasis.enable_half_metal_generation \\  
        -value true
```

See Also

- [write_oasis](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

file.oasis.enable_multiple_rules_per_layer

Used by "*write_oasis*" command to enable layout data to be written as several records into OASIS file, corresponding to the matched layer mapping rules.

Data Types

bool

Default false

f

Description

By default, a layout shape or text is output into OASIS file corresponding to the last matched layer mapping rule. For example, a layer shape will be output as a rectangle record on mapped OASIS layer and data type.

When the option is on, a layout shape or text is output into OASIS file corresponding to all matched layer mapping rules. For example, there are two matched layer mapping rules for a layer shape, the layer shape is output as two rectangle records on different mapped OASIS layer and data type.

Examples

The following example describes the behavior when the option is on. The layer mapping rules as below: 17:0:*:* 17:0 17:0:area_fill:* 19:0 17:0:routing_blockage:* 117:0 17:*:*:* 217:0 17:0:*:* 317:0

A polygon for wiring on layer 17, and its purpose number is 0, the matched layer mapping rules are following ones:

17:0:*:*	17:0
17:*:*:*	217:0
17:0:*:*	317:0

When the option is on, there are three polygons are output for the polygon, corresponding to above three layer and data type pairs.

The example to set the app option as below:

```
prompt> set_app_options -name
"file.oasis.enable_multiple_rules_per_layer" -value true
```

See Also

- [write_oasis](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

file.oasis.enable_special_snapping

Used by the *write_oasis* command to enable special snapping for path.

Data Types

Boolean

Default false

f

Description

Setting this application option to true to make `write_oasis` use special snapping rule to snap path.

By default (*false*), `write_oasis` uses NDM snapping API to snap path.

You must set this option before running the `write_oasis` command.

Examples

The following example enables the special snapping.

```
prompt> set_app_options -name file.oasis.enable_special_snapping \\  
        -value true
```

See Also

- [write_oasis](#)
- [merge_stream](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

file.oasis.exclude_layers

Specifies the rules for mapping between the input layers and the layers which are to be used for excluding geometries from the specified input layers.

Data Types

list

Default null

Description

This option specifies the mapping between input layers and other layers which are to be used for exclusion. The geometries in the other layers are subtracted from same colored geometries of the input layer during the `read_oasis` command.

The tool uses the geometries in a specific color of the mapped layers to subtract same colored geometries from the input layers. Multiple mapping between the same input layer and exclude layers can be specified. The exclusions works by subtracting geometries of the same color using relationship specified in the mapping.

You must set this option before running the `read_oasis` command.

f

You can specify the mapping for one or more input layers and excluded layers and color. The syntax used to specify each mapping is

```
{ {metal_layer1:purpose1 exclude_layer1[:exclude_purpose1][:mask_name]}
  {metal_layer2:purpose2 exclude_layer2[:exclude_purpose2][:mask_name]}
... }
```

You specify the layer names and purpose to be excluded from input layer names and purpose by using the information in the technology file.

Examples

The following example defines the exclude layers mapping for the M1 and M2 metal layers.

In the example below, mask_one of M1 will be subtracted by the mask_one geometries of EM1. And if layer M2 has two mask, mask_one geometries of layer M2 will be subtracted by the mask_one geometries of EM2. mask_two of EM2 will remove geometries occupied by the mask_two of M2. Layers EM1 and EM2 will not be modified. Only input layers M1 and M2 will change.

```
prompt> set_app_options -name "file.oasis.exclude_layers" -value { {M1
  EM1:mask_one} {M2 EM2} }
```

See Also

- [read_oasis](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

file.oasis.ignore_uncolored_shapes

Used by "write_oasis" command to ignore uncolored shapes in the block.

Data Types

Boolean

Default false

Description

This option is to control if uncolored shapes which includes the shapes of via, via matrix and shape pattern are written out.

When the option is on, all the uncolored shapes will be ignored.

f

When the option is off, uncolored shapes are also written out.

See Also

- [write_oasis](#)
- [get_app_option_value](#)
- [set_app_options](#)

file.oasis.ignore_uncolored_shapes_exclude_layers

Used by "*write_oasis*" command to exclude layers while using file.oasis.ignore_uncolored_shapes option.

Data Types

list

Default empty

Description

This option is to specify exclude layers when file.oasis.ignore_uncolored_shapes option is true.

When the list of exclude layers provided, all the uncolored shapes of those layers, will not be ignored.

When the list of exclude layers not provided, all uncolored shapes will be ignored.

See Also

- [write_oasis](#)
- [get_app_option_value](#)
- [set_app_options](#)

file.oasis.include_sub_cell_in_block

Specifies the rules for mapping between the input layers and the layers which are to be used for excluding geometries from the specified input layers.

Data Types

list

Default null

f

Description

This option specifies the mapping between input layers and other layers which are to be used for exclusion. The geometries in the other layers are subtracted from same colored geometries of the input layer during the *read_gds* and *trace_connectivity* commands.

The tool uses the geometries in a specific color of the mapped layers to subtract same colored geometries from the input layers. Multiple mapping between the same input layer and exclude layers can be specified. The exclusions works by subtracting geometries of the same color using relationship specified in the mapping.

You must set this option before running the *read_gds* or *trace_connectivity* commands.

You can specify the mapping for one or more input layers and excluded layers and color. The syntax used to specify each mapping is

```
{ {metal_layer1:purpose1 exclude_layer1[:exclude_purpose1][:mask_name]}
  {metal_layer2:purpose2 exclude_layer2[:exclude_purpose2][:mask_name]}
  ... }
```

You specify the layer names and purpose to be excluded from input layer names and purpose by using the information in the technology file.

Examples

The following example defines the exclude layers mapping for the M1 and M2 metal layers.

In the example below, mask_one of M1 will be subtracted by the mask_one geometries of EM1. And if layer M2 has two mask, mask_one geometries of layer M2 will be subtracted by the mask_one geometries of EM2. mask_two of EM2 will remove geometries occupied by the mask_two of M2. Layers EM1 and EM2 will not be modified. Only input layers M1 and M2 will change.

```
prompt> set_app_options -name "file.gds.exclude_layers" -value { {M1
  EM1:mask_one} {M2 EM2} }
```

See Also

- [read_gds](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

file.oasis.max_cell_name_length

Used by "write_oasis" command to limit maximum cell name length.

Data Types

int

Default 0

Description

This options specifies the name length limit for cells. If the cell name exceeds the length limit, the `write_oasis` command truncates the cell name and automatically resolves any naming conflicts. By default, there is no length limit.

You must set this option before running the `write_oasis` command.

Examples

The following example sets the maximum cell name length to 40.

```
prompt> set_app_options -name file.oasis.max_cell_name_length -value 40
```

See Also

- [write_oasis](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

file.oasis.net_text_on_layers

Used by "`write_oasis`" command to output net text on specific layers.

Data Types

list

Default null

Description

This option is used together with command option "`-output_net_text`". You can specify text layers so that only net text on these layers will be output. It also respects to "`-net_text_on`" option.

```
prompt> set_app_options -name "file.oasis.net_text_on_layers" -value {M2  
M3}
```

f

See Also

- [write_oasis](#)
- [get_app_option_value](#)
- [set_app_options](#)

file.oasis.output_first_same_name_cell

Controls whether the *write_oasis* command writes out only the first encountered cell when there are multiple cells with the same name.

Data Types

Boolean

Default false

Description

This application option controls whether the *write_oasis* command writes only the first encountered cell when there are multiple cells with the same name.

By default (*false*), the *write_oasis* command renames the cells with the same name and writes them as different structures in the output GDSII file.

When *true*, the *write_oasis* command writes out only the first encountered cell and ignores additional cells with the same name.

You must set this option before running the *write_oasis* command.

Examples

The following example enables streaming out only the first cell in case multiple cells have same name. The default action is to rename and write all the multiple cells as different cells.

```
prompt> set_app_options -as_user_default \\  
        -name file.oasis.output_first_same_name_cell -value true
```

See Also

- [write_oasis](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

file.oasis.pg_via_arrays

Used by the *write_oasis* command to compact individual PG vias into via arrays when generating OASIS layout files.

Data Types

Boolean

Default false

Description

This application controls whether the *write_oasis* command compacts PG vias into via arrays in the layout files, which reduces the OASIS file size.

You must set this option before running the *write_oasis* command.

See Also

- [write_gds](#)
- [write_oasis](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

file.oasis.port_type_map

Used by the "read_oasis" command in *icc2_lm_shell* to identify port types correctly

Data Types

list_of_pairs

Default null

Description

This option is used to specify the mapping between port type and port name. This map is used in command *read_oasis* to correctly identify the port type. This option shall be set before calling *read_oasis* command. The first element in the value pair is the port type and the second element in the pair is the name of the port name to be matched. One port type can be used with multiple port names.

The following are the port types supported now.

f

- *power*
- *ground*
- *pwell*
- *nwell*

Examples

The following example shows how the application option is to be set

```
prompt> set_app_options {file.oasis.port_type_map {{power VDD} {ground  
VSS}}}
```

See Also

- [read_oasis](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

file.oasis.prefix_for_fill

Used by "*write_oasis*" command to prefix the fill cell names with the owner design name that is to be written as Structures into the OASIS file.

Data Types

Boolean

Default true

Description

This option enables or disables using the design name as prefix to the fill cell name that is to be written as Structures into the OASIS file. This option is design-scoped.

The default value is true which means the design name is used as a prefix to the fill cell name.

Examples

The following example disables using the design name as prefix for the fill cells to be written as Structures in the output OASIS file for the current design.

```
prompt> set_app_options -name "file.oasis.prefix_for_fill" -value false
```

f

See Also

- [write_oasis](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

file.oasis.read_enable_multiple_rules_per_layer

Used by "*read_oasis*" command to enable the shape records in the OASIS file to be read as several shapes into database, corresponding to the matched multiple layer mapping rules.

Data Types

boolean

Default false

Description

By default, a shape or text record in the OASIS file is created according to the last matched layer mapping rule.

When the option is on, a shape or text record in the OASIS file is created corresponding to all matched layer mapping rules. For example, there are two matched layer mapping rules for a shape record, the shape record is created as two rectangles in the database on different mapped layer and data type.

Examples

The following example describes the behavior when the option is on. Assuming there is a shape record on layer 531 in the OASIS file, two shapes with mask_one mask_constraint attribute will be created on layer 37 and 31 respectively in the database.

The layer mapping rules as below: 37:::mask_one 531" 31:::mask_one 531"

The example to set the app option as below:

```
prompt> set_app_options -name
"file.oasis.read_enable_multiple_rules_per_layer" -value true
```

See Also

- [read_oasis](#)
- [get_app_option_value](#)

f

- [report_app_options](#)
- [set_app_options](#)

file.oasis.rotate_pin_text_by_access_direction

Used by "*write_oasis*" command to enable or disable controlling of the rotation of the TEXT object when the pin object is written into the OASIS file.

Data Types

Boolean

Default false

Description

This option enables or disables controlling of the value of the ANGLE token in the TEXT object of a pin during the *write_oasis* flow. If this app-option is enabled, then the value of the ANGLE token will be dependent on the access-direction flags on the pin object. If the access-direction flags has left or right direction set then a value of R0 will be used as the TEXT object's orientation. In other cases a value of R90 will be used as the TEXT object's orientation. This option is design-scoped.

By default this option is disabled and the TEXT objects are always written with an orientation value of R0.

Examples

The following example defines that the rotation of the TEXT object of the pin should be controlled by the access direction flags set on the pin.

```
prompt> set_app_options -as_user_default  
{file.oasis.rotate_pin_text_by_access_direction true}
```

See Also

- [write_oasis](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

file.oasis.shape_use_for_pg_mesh

This option provides list of shape use for pg_mesh use type which is specified in layer mapping rules.

f

Data Types

list

Default "stripe ring"**Description**

This option is to control pg_mesh collection, when pg_mesh use type is specified in layer mapping file.

When this option is not used, by default shapes of stripe and ring shape_use are written out. When this option is used, shapes corresponding to the specified shape_use in the value are written out. Available value: "", or any combination of "stripe, ring, lib_cell_pin_connect, macro_pin_connect".

Examples

The following example specifies shape_use for pg_mesh use type.

```
prompt> set_app_options -name file.oasis.shape_use_for_pg_mesh \\  
-value {stripe ring lib_cell_pin_connect macro_pin_connect}
```

See Also

- [write_oasis](#)
- [get_app_option_value](#)
- [set_app_options](#)

file.oasis.split_via_matrix

Controls whether the *write_oasis* command splits non-fully-populated via matrix to multiple PLACEMENT records with and without repetition.

Data Types

Boolean

Default true**Description**

The application option is used to reduce OASIS file size.

When it is true, write_oasis will split non-fully-populated via matrix to multiple PLACEMENT records with and without repetition if and only if its mask pattern is uniform.

Otherwise, the non-fully-populated via matrix is written as multiple PLACEMENT records without repetition.

See Also

- [write_oasis](#)

file.oasis.text_all_pins

Used by "*write_oasis*" command to write the text for all pins of the port into the OASIS file.

Data Types

bool

Default false

Description

This option set the value such that text for each pins of the port are written into the OASIS file. The default behavior is to write a single text for all pins of the port.

Examples

The following example specifies that the text for each pins of the port are to be written into the output OASIS file.

```
prompt> set_app_options -name "file.oasis.text_all_pins" -value true
```

See Also

- [write_oasis](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

file.oasis.text_layer_map

Specifies the mapping between metal layers and text layers that is used for tracing ports.

Data Types

list

Default null

Description

This option specifies the mapping between metal layers and text layers that is used for tracing ports by the *read_oasis* command.

f

The tool uses the text labels in a OASIS file to associate metal shapes with a net or pin and then traces the connectivity of the labeled metal shapes to identify additional metal shapes associated with that net or pin. If the text for a port is not on the same layer as the metal shape, you must set this option to specify the layer mapping so the tool can correctly identify the ports. You must set this option before running the `read_oasis` command.

You can specify the mapping for one or more metal layers. The syntax used to specify each mapping is

```
{metal_layer {text_layer1[:data_type1] text_layer2[:data_type2] ...}}
```

You specify the layer names and data type numbers by using the information in the technology file.

Examples

The following example defines the text layer mapping for the M1 and M2 metal layers.

```
prompt> set_app_options { file.oasis.text_layer_map { {M1 M1PIN} {M2  
M2PIN} } }
```

See Also

- [read_oasis](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

file.oasis.trace_copy_blockage_color_from_overlapped_terminal

Specifies layers in which a routing blockage of no_mask will be changed to a new mask_constraint as the overlapped terminal.

Data Types

list

Default null

Description

This option indicates "`read_oasis`" commands to change a routing blockage of no_mask of the specified layers with another mask from an overlapping terminal, by setting its is_mask_fixed to true and its mask type to user_set. If multiple terminals are overlapped, the mask is got from a randomly terminal of another mask.

You must set this option before running the `read_oasis`.

f

You can specify one or more metal layers.

The syntax used to specify each mapping is

```
{layer1[:data_type1] layer2[:data_type2] ...}
```

You specify the layer names and data type numbers by using the information in the technology file.

Examples

The following example specify layers M3 and M5 in which layers routing blockages of no_mask to get another mask from an overlapping terminal. The is_mask_fixed are set to true and mask types are set to use_set for the routing blockages.

```
prompt> set_app_option -name
file.oasis.trace_copy_blockage_color_from_overlapped_terminal -value {M3
M5}
```

See Also

- [read_gds](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

file.oasis.trace_copy_overlap_shape_from_sub_cell

Used by "*read_oasis*" and "*trace_connectivity*" commands in *icc2_lm_shell* to indicate shapes in sub-level blocks are to be duplicated to the top block.

Data Types

bool

Default false

Description

This option indicates whether a shape in a sub-level block is to be duplicated to the top block when the shape is overlapped by terminals.

When the app option "file.oasis.exclude_layers" is specified and the shape in the sub-level block is divided into several parts by the shape on the excluding layer in the physical hierarchy, only the part overlapping the top terminal will be duplicated to the top block.

f

When the non-pg terminal goes through an instance referencing a sub-level block no matter in horizon or vertical, no shape will be duplicated from the sub-level block for the instance.

A via is excluded from the terminals.

You must set this option before running the *read_gds* or *trace_connectivity* commands.

Note: This option can work for abutting, and it can't work in "top level". 'read_gds -block_map' will bring a sub-level block to top according to map file which makes the option cannot bring a pin in the sub-level block to top block as a terminal.

Examples

The following example enables to duplicate an overlapped shape during GDSII import and the connectivity tracing.

```
prompt> set_app_options -name file.oasis.trace_duplicate_overlap_shape  
-value true
```

See Also

- [read_gds](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

file.oasis.trace_merge_overlap_mask

Used by "*read_oasis*" and "*trace_connectivity*" commands to merge the colorless via metal enclosure into overlapped color metal terminal. The default valuse is false.

Data Types

boolean

Default false

Description

When the option is on, a colorless shape is covered by a color shape, the colorless shape need to be set to the color of the colored shape. By default, this app option is false. Long run time would be expected when this app option is true.

You must set this option before running the *read_oasis* or *trace_connectivity* commands.

f

Examples

The following example enables this feature.

```
prompt> set_app_options -name file.oasis.trace_merge_overlap_mask -value  
true
```

See Also

- [read_oasis](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

file.oasis.trace_terminal_length

Used by "*read_oasis*" command in to indicate a long pin interacting with the block boundary side should be split into a terminal with the specified length and a shape. When the option value is 0, the split doesn't happen. However, if you use *file.oasis.trace_terminal_length_layer_pairs* to set a layer specific terminal length, that will be used for splitting instead of this global one.

Data Types

float

Default 0

Description

This option indicates whether a long pin interacting with the block boundary side should be split into a pin with the specified length and a shape. When the option value is 0, the split doesn't happen. When the qualified pin is more than 0, the long pin is split into a new pin and a new shape. If the pin length is less than the app option, the long pin won't be split either. The new pins and shapes inherit all other attributes from the original pin. The new pin covers the portion of the specified length including the original pin on the die boundary or out of the die boundary if it exists.

The option value should be specified in length units as configured by the *set_user_units -type length -input* command, which is micrometers (um) by default.

This option has only effect on design view.

A via is excluded from the long pins and they cannot be split.

You must set this option before running the *read_oasis* or *trace_connectivity* commands.

f

Examples

The following example enables the long pin split during OASIS import and the connectivity tracing. the portion within the 1 user unit from the boundary of the cell is the length of the new pin, while the other portion belongs to a new generated shape.

```
prompt> set_app_options -name file.oasis.trace_terminal_length -value 1
```

See Also

- [read_oasis](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

file.oasis.trace_terminal_length_4x_width

Used by "*read_oasis*" and "*trace_connectivity*" commands to indicate that a long signal pin of the specified layers interacting with one of the block boundary side should be split into a terminal with the length equal to 4 times of its width.

Data Types

list of string

Default empty list

Description

This option indicates which long signal pin of the specified layer interacting with one of the block boundary side should be split into a pin with the length equal to 4 times its width

You must set this option before running the *read_oasis* or *trace_connectivity* commands.

Examples

The following example, the long signal pin will be split during oasisII import and the connectivity tracing, with terminal length of 4 times its width in layer M1 and layer M2 respectively.

```
prompt> set_app_options -name file.oasis.trace_terminal_length_4x_width  
-value {M1 M2}
```

See Also

- [read_oasis](#)
- [get_app_option_value](#)

f

- [report_app_options](#)
- [set_app_options](#)

file.oasis.trace_terminal_length_boundary_only

Used by "*read_oasis*" command in *lm_shell* to indicate a terminal of a port touches the top block boundary side should be kept but other terminals belong only to the same port should become a shape in a top design.

Data Types

bool

Default false

Description

This option indicates a terminal of a port touching the top block boundary side should be kept but other terminals belong only to the same port should become a shape in a top design.

This option has only effect on design view.

You must set this option before running the *read_oasis* command.

Examples

The following example enables the terminals become a shape during OASIS import and the connectivity tracing. The terminals belong to a port in a top design. At least one terminal to this port touches the top design.

```
prompt> set_app_options -name  
file.oasis.trace_terminal_length_boundary_only -value true
```

See Also

- [read_oasis](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

file.oasis.trace_terminal_length_exclude_list

This option is to provide list of ports which need to be excluded from getting trimmed. This option is used along with file.oasis.trace_terminal_length* options.

f

Data Types*list of string***Default** empty list**Description**

This option provides list of ports which need to be excluded from trimming. This option can be use along with `file.oasis.trace_terminal_length*` options.

You must set this option before running the `read_oasis` or `trace_connectivity` commands.

Examples

The following example, the ports named "Ports_1" and "Ports_2" will be excluded from splitting during oasis import and the connectivity tracing.

```
prompt> set_app_options -name file.oasis.trace_terminal_length_4x_width
        -value {M1 M2}
prompt> set_app_options -name
        file.oasis.trace_terminal_length_exclude_list -value {Ports_1 Ports_2}
```

See Also

- [read_oasis](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

file.oasis.trace_terminal_length_layer_pairs

Used by "`read_oasis`" command to indicate that a long pin of the specified layers interacting with the block boundary side should be split into a terminal with the specified length for that layer and a shape.

Data Types*list of {string float} pairs***Default** empty list**Description**

This option indicates which layer of a long pin of the specified type interacting with the block boundary side should be split into a pin with the specified length for it's layer and a shape.

f

This option has only effect on design view.

You must set this option before running the *read_oasis* command.

Examples

The following example, the long PG pin will be split during OASIS import and the connectivity tracing, with terminal length of 1.5 and 1.8 in layer M1 and layer M2 respectively.

```
prompt> set_app_options -name
file.oasis.trace_terminal_length_layer_pairs {{M1 1.5} {M2 1.8}}
```

See Also

- [read_oasis](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

file.oasis.trace_terminal_length_layers

Used by "*read_oasis*" command to indicate a long pin of the specified layers interacting with the block boundary side should be split into a terminal with a specified length and a shape. When the option value is null, valid long pins of all layers are to be split. However, if you use *file.oasis.trace_terminal_length_layer_pairs* to set a layer specific terminal length, long pins in those layers will be split as well.

Data Types

list

Default null

Description

This option indicates which layer of a long pin of the specified type interacting with the block boundary side should be split into a pin with a specified length and a shape. When the option value is null, the split happens to all layers.

This option has only effect on design view.

You must set this option before running the *read_oasis* command.

Examples

The following example enables of M1 and M2:35 layers the long PG pin split during GDSII import and the connectivity tracing.

f

```
prompt> set_app_options -name file.oasis.trace_terminal_length_layers {M1
M2:35}
```

See Also

- [read_oasis](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

file.oasis.trace_terminal_text

Specifies the mapping between metal layers and text layers that is used for tracing ports.

Data Types

list

Default null

Description

This option indicates *"read_oasis"* command to extract specified shapes as terminals. These shapes should be covered by the specified text and shapes in layers specified by the option. The later shapes must be reached by connections in all layers through a cut with the former shapes no matter what the traced type is.

You must set this option before running the *read_oasis* command.

When *"read_oasis -trace_all_layers_for_port_names"* and this option both turn on. *"read_oasis -trace_all_layers_for_port_names"* has the priority over the option. The specified port_names are traced in all-layer styles instead of the way specified by the option.

You can specify the mapping for one or more metal layers. The syntax used to specify each mapping is

```
{text {layer1[:data_type1] layer2[:data_type2] ...}}
```

You specify the layer names and data type numbers by using the information in the technology file.

Examples

The following example enables shapes overlapped by text 'A', and 'B' to be extracted as terminals; For text 'A', shapes in layers M3, V4, M5 connected in all layers with the shape

f

overlapped by text 'A' can be extracted as terminals; For text 'B', shapes in layers M3 connected with the shape overlapped by text 'B' can be extracted as terminals;

```
prompt> set_app_option -name file.oasis.trace_terminal_text -value {{A
      {M3 V4 M5}} {B {M3}}}
```

See Also

- [read_oasis](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

file.oasis.trace_terminal_type

Used by "*read_oasis*" and "*trace_connectivity*" commands in *icc2_lm_shell* to indicate a long pin of the specified type interacting with the block boundary side should be split into a terminal with a specified length and a shape. When the option value is default, PG and signal terminals are to be split.

Data Types

string

Default default

Description

This option indicates whether a long pin of the specified type interacting with the block boundary side should be split into a pin with a specified length and a shape. When the option value is default, the split happens to both PG and signal terminaps. When the option value is case-insensitive PG or signal, a qualified PG or signal terminal to be split.

This option has only effect on design view.

A via is excluded from the long pins and they cannot be split.

You must set this option before running the *read_oasis* or *trace_connectivity* commands.

Examples

The following example enables the long PG pin split during GDSII import and the connectivity tracing.

```
prompt> set_app_options -name file.oasis.trace_terminal_type PG
```

f

See Also

- [read_gds](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

file.oasis.trace_unmapped_text

Used by "*read_oasis*" command in *icc2_lm_shell* to enable or disable tracing on all routing layers for the texts not mapped to any routing layers.

Data Types

Boolean

Default false

Description

This option enables or disables tracing on all routing layers for text labels not mapped to a routing layer by the *read_oasis* and command. The default behavior is to ignore use of such text labels for identification of pins. This option will be required to be enabled in most rare cases, so this option should be disabled in most cases.

The tool uses the text labels in a OASIS file to associate metal shapes with a net or pin and then traces the connectivity of the labeled metal shapes to identify additional metal shapes associated with that net or pin. If the text for a port is not on the same layer as the metal shape, you must set the option *file.oasis.text_layer_map* to specify the layer mapping so the tool can correctly identify the ports. In some rare cases only, you may need to set this option to make the tool identify pin geometries on all routing layers for such unmapped text labels. So normally this option should be used to disable the setting to trace for unmapped texts. You must set this option before running the *read_oasis* command.

Examples

The following example defines that tracing should be enabled for all texts not mapped to the routing layers.

```
prompt> set_app_options { file.oasis.trace_unmapped_text true}
```

See Also

- [read_oasis](#)
- [get_app_option_value](#)

f

- [report_app_options](#)
- [set_app_options](#)

file.oasis.use_only_mapped_text

Controls whether port tracing uses only the text specified by the *file.oasis.text_layer_map*.

Data Types

Boolean

Default false

Description

This option controls whether the *read_oasis* and *trace_connectivity* commands use only the text specified by the *file.oasis.text_layer_map* application option when tracing ports.

The tool uses the text labels in a OASIS file to associate metal shapes with a net or pin and then traces the connectivity of the labeled metal shapes to identify additional metal shapes associated with that net or pin.

To use only the text mapped by the *file.oasis.text_layer_map* application option for identifying the ports, set this option to *true* before running the *read_oasis* or *trace_connectivity* command.

Examples

The following example defines the text layer mapping for the M1 and M2 metal layers and uses only that text for port tracing.

```
prompt> set_app_options -name file.oasis.text_layer_map \\  
-value { {M1 M1PIN} {M2 M2PIN} }  
prompt> set_app_options -name file.oasis.use_only_mapped_text \\  
-value true
```

See Also

- [read_oasis](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

file.oasis.write_metal_below_cut_to_top

Used by the *write_oasis* command to write all the metal below the cut metal to the parent cell when the *design_type* of the cell which owns the cut metal is not module and command option "-connect_below_cut_metal" is specified.

Data Types

Boolean

Default false

Description

Setting this application option to true to make *write_oasis* write the metal below the cut metal to the parent cell instead of the cell which owns the cut metal when the *design_type* of the cell which owns the cut metal is not module.

By default (*false*), *write_oasis* writes the metal below cut metal to the cell which owns the cut metal.

You must set this option before running the *write_oasis* command.

Examples

The following example is to make *write_oasis* command to turn on this app option.

```
prompt> set_app_options -name file.oasis.write_metal_below_cut_to_top \\  
-value true
```

See Also

- [write_oasis](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

file.ssf.current_version

Provides the current version supported by Safety Specification Format (SSF) reader and writer.

Data Types

string

Default Not applicable

f

Description

This application option denotes the current version of SSF file reader and writer in the tool. It is a read-only application option.

To determine the current value of this application option, use the *get_app_option_value -name file.ssf.current_version* command. For a list of all SSF related application options and their current values, use the *report_app_options file.ssf.** command.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [load_ssf](#)
- [save_ssf](#)

file.ssf.supported_versions

Provides all the versions supported by the Safety Specification Format (SSF) reader and writer.

Data Types

string

Default Not applicable

Description

This application option lists all the SSF file versions supported by the SSF file reader and writer in the tool. It is a read-only application option.

To determine the current value of this application option, use the *get_app_option_value -name file.ssf.supported_versions* command. For a list of all SSF related application options and their current values, use the *report_app_options file.ssf.** command.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)

- [load_ssf](#)
- [save_ssf](#)

file.tech.library_developer_mode

Enable the developer mode option within tech file checker when create_workspace or read_tech_file is used.

Data Types

Boolean

Default *false*

Description

This application option is used to enable developer mode tech file check. By default, the option is false; you need set it to true to enable developer mode. If false, the checks will be done using the default, or enduser mode. The difference is in both the behavior of overall loading of tech file, and message severity. In developer mode, issues can be reported as Error and fail to load the techfile, while in enduser mode, the same issue can be reported as just a Warning. Only dual_mode checks are currently available in both modes.

For example: `icc2_lm_shell> set_app_options -name file.tech.library_developer_mode -value true`

See Also

- [read_tech_file](#)
- [create_lib](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

file.tlup.max_preservation_size

Specifies the maximum TLUPlus file size, in MB, to preserve.

Data Types

integer

Default 20

f

Description

Normally, for purpose of extraction validation/debugging, original TLUPlus file will be preserved as an attachment in database. But this will increase database size. This application option limits the size of TLUPlus file to preserve. The default value is 20MB. TLUPlus file won't be preserved if it exceeds size allowed.

To determine the current value of this application option, use the *get_app_option_value -name file.tlup.max_preservation_size* command.

See Also

- [read_parasitic_tech](#)

file.verilog.allow_multiple_top

Controls whether the Verilog reader should select one top module when multiple candidates are possible.

Data Types

Boolean

Default true

Description

This application option, if set to true, causes the Verilog reader to pick one when multiple candidates for top module exist. If set to false, the Verilog reader requires a single identifiable candidate to succeed.

To determine the current value of this application option, use the *get_app_option_value -name file.verilog.allow_multiple_top* command. For a list of all Verilog application options and their current values, use the *report_app_options file.verilog.** command.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [read_verilog](#)
- [report_app_options](#)

f

file.verilog.allow_same_block

Controls whether the Verilog reader should target the current block if block is unlinked or block is an empty linked block that without netlist data.

Data Types

Boolean

Default true

Description

This application option, if set to true, causes the Verilog reader to target the current block if block is unlinked or block is an empty linked block that without netlist data.

To determine the current value of this application option, use the *get_app_option_value -name file.verilog.allow_same_block* command. For a list of all Verilog application options and their current values, use the *report_app_options file.verilog.** command.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [read_verilog](#)
- [report_app_options](#)

file.verilog.default_user_label

Specifies the user label in which to create the block.

Data Types

string

Default ""

Description

This application option specifies the user label in which to create the block.

When this option is set to a non-empty string, the Verilog reader creates the block in the specified user label. When set to an empty string, the Verilog reader creates the block in the default, unnamed label.

f

To determine the current value of this application option, use the *get_app_option_value -name file.verilog.default_user_label* command. For a list of all verilog application options and their current values, use the *report_app_options file.verilog.** command.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [read_verilog_outline](#)
- [read_verilog](#)
- [report_app_options](#)

file.verilog.enable_vpp

Enables or disables preprocessing of Verilog files by the Verilog preprocessor.

Data Types

Boolean

Default false

Description

This application option enables or disables preprocessing of Verilog files by the Verilog preprocessor. It is used only by the native Verilog reader.

When *true*, before the Verilog reader reads a Verilog file, the Verilog preprocessor scans for and expands the Verilog preprocessor directives ``define`, ``undef`, ``include`, ``ifdef`, ``else`, and ``endif`. Intermediate files from the preprocessor are created in the directory specified by the *icc2_tmp_dir* variable. Also, the ``include` directive uses the *search_path* to find files.

Very few structural Verilog files use preprocessor directives. Set this application option to *true* only if your Verilog file contains directives that require the preprocessor. Without the preprocessor, the native Verilog reader does not recognize these directives.

To determine the current value of this application option, use the *get_app_option_value -name file.verilog.enable_vpp* command. For a list of all verilog application options and their current values, use the *report_app_options file.verilog.** command.

See Also

- [get_app_option_value](#)
- [get_app_options](#)

f

- [read_verilog_outline](#)
- [read_verilog](#)
- [report_app_options](#)

file.verilog.handle_complex_bus_port

Controls how the write_verilog command outputs complex HDL type bus ports.

Data Types

string

Default bit_blast

Description

This application option controls how the write_verilog command outputs complex HDL type bus ports. Complex bus ports could be multidimensional or indexed by non-numerical values. The write_verilog command bit-blasts them by default.

There are three possible values:

1) off

Complex bus ports are bit-blasted. If other ports in the bus are non-complex, they remain bused.

2) bit_blast

All ports in the complex bus port's bus are bit-blasted. If other ports in the bus are non-complex, they are bit-blasted as well.

3) aggregate

All ports in the complex bus port's bus are output using the aggregated port expression format. If other ports in the bus are non-complex, they are output using the aggregated format as well.

The option will be obsoleted in a future release. Please use "file.verilog.write_bus_port_style" instead.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)

- [set_app_options](#)
- [write_verilog](#)

file.verilog.ieee_1735_decryption

Controls whether the verilog reader can decrypt an IEEE 1735 standard encrypted verilog file .

Data Types

Boolean

Default false

Description

This application option is used only by the verilog reader.

When this application option is set to *true*, the *read_verilog* command can decrypt IEEE 1735 standard encrypted verilog file.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [read_verilog](#)
- [report_app_options](#)

file.verilog.keep_constant_assigns

Controls whether the Verilog reader keeps constant assigns as separate nets in the design.

Data Types

Boolean

Default true

Description

This application option controls whether the Verilog reader creates a distinct tie net for each Verilog constant assign. A constant assign is a Verilog assign statement with a constant value on the right hand side (e.g., assign n1 = 1'b0).

f

When *false*, the Verilog reader creates one tie low net and net names assigned to 1'b0 become synonyms of this tie low net in the module. Likewise the Verilog reader creates one tie high net and net names assigned to 1'b1 become synonyms of this tie high net in the module.

When *true*, the Verilog reader creates a distinct tie net for each net name assigned to a constant (i.e., 1'b0 or 1'b1). If there are M nets assigned to 1'b0, there will be M tie low nets. Likewise if there are N nets assigned to 1'b1, there will be N tie high nets.

To determine the current value of this application option, use the *get_app_option_value -name file.verilog.keep_constant_assigns* command. For a list of all verilog application options and their current values, use the *report_app_options file.verilog.** command.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [read_verilog_outline](#)
- [read_verilog](#)
- [report_app_options](#)

file.verilog.keep_unconnected_cells

Controls whether the Verilog reader keeps unconnected cells in the netlist.

Data Types

Boolean

Default true

Description

This application option controls whether the Verilog reader keeps unconnected cells in the netlist. It is used only by the native Verilog reader.

When *false*, if no instances of a certain reference have connections, those cell instances are discarded. This saves memory by not representing cells that have no impact on the timing of the design, such as metal fill cells. When *true*, such cells are preserved.

To determine the current value of this application option, use the *get_app_option_value -name file.verilog.keep_unconnected_cells* command. For a list of all verilog application options and their current values, use the *report_app_options file.verilog.** command.

f

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [read_verilog_outline](#)
- [read_verilog](#)
- [report_app_options](#)

file.verilog.keep_unconnected_nets

Controls whether the Verilog reader keeps unconnected nets in the netlist.

Data Types

Boolean

Default true

Description

This application option controls whether the Verilog reader keeps unconnected nets in the netlist. It is used only by the native Verilog reader.

When *true* (the default), unconnected nets are preserved. When *false*, unconnected nets are discarded.

To determine the current value of this application option, use the *get_app_option_value -name file.verilog.keep_unconnected_nets* command. For a list of all verilog application options and their current values, use the *report_app_options file.verilog.** command.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [read_verilog_outline](#)
- [read_verilog](#)
- [report_app_options](#)

file.verilog.read_soc_outline

Controls whether the Verilog reader *read_verilog* will use *read_verilog_shell* command to create a soc floorplan outline view.

f

Data Types

Boolean

Default false

Description

This application option, if set to true, causes the Verilog reader to invoke the `read_verilog_shell` command to create an soc outline view.

To determine the current value of this application option, use the `get_app_option_value -name file.verilog.read_soc_outline` command. For a list of all Verilog application options and their current values, use the `report_app_options file.verilog.*` command.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [read_verilog](#)
- [set_dp_strategy](#)

file.verilog.reconstruct_multi_dimension_bus

Controls whether the Verilog reader preserves multidimensional bus ports.

Data Types

Boolean

Default false

Description

This application option controls whether the Verilog reader preserves multidimensional bus ports.

When the application option is *false*, multidimensional bus ports are bit-blasted into one-dimensional bus ports. When set to *true*, the multidimensional bus ports are preserved and supported by the `read_verilog` command.

To determine the current value of this application option, use the `get_app_option_value -name file.verilog.reconstruct_multi_dimension_bus` command. For a list of all verilog application options and their current values, use the `report_app_options file.verilog.*` command.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_port_buses](#)
- [report_net_buses](#)
- [read_verilog](#)
- [report_app_options](#)

file.verilog.write_bus_port_style

Controls how the write_verilog command outputs complex HDL type bus ports.

Data Types

string

Default bit_blast

Description

This application option controls how the write_verilog command outputs complex HDL type bus ports. Complex bus ports could be multidimensional or indexed by non-numerical values. The write_verilog command bit-blasts them by default.

There are three possible values:

1) off

Complex bus ports are bit-blasted. If other ports in the bus are non-complex, they remain bused.

2) bit_blast

All ports in the complex bus port's bus are bit-blasted. If other ports in the bus are non-complex, they are bit-blasted as well.

3) aggregate

All ports in the complex bus port's bus are output using the aggregated port expression format. If other ports in the bus are non-complex, they are output using the aggregated format as well.

f

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)
- [write_verilog](#)

file.verilog.write_non_pg_net_assigns

Controls whether the Verilog writer outputs extra assign statements for renamed signal, tie, and clock nets, so that their original names are retained in the Verilog output.

Data Types

Boolean

Default false

Description

This application option, if set to true, causes the Verilog writer to output extra assign statements for renamed signal, tie, and clock nets, so that their original names are retained in the Verilog output. If set to false, the extra assign statements are omitted and the original net names are not retained in the Verilog netlist.

This application option affects each net that is named differently from its port. In the Verilog language, the port and pins of a net are connected together by an implicit net. This net is implicitly named to match one of its ports. So when writing Verilog, a net name is implicitly renamed to its connected port's name, and thus the original net name is not retained in the Verilog output. The netlist is valid. However, to retain the original net name, an assign statement is necessary to assign the original net name to the actual net name in the Verilog output.

To determine the current value of this application option, use the *get_app_option_value -name file.verilog.write_non_pg_net_assigns* command. For a list of all verilog application options and their current values, use the *report_app_options file.verilog.** command.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [read_verilog_outline](#)

f

- [read_verilog](#)
- [report_app_options](#)

file.verilog.write_pg_net_assigns

Controls whether the Verilog writer outputs extra assign statements for renamed power and ground nets, so that their original names are retained in the Verilog output.

Data Types

Boolean

Default true

Description

This application option, if set to true, causes the Verilog writer to output extra assign statements for renamed power and ground nets, so that their original names are retained in the Verilog output. If set to false, the extra assign statements are omitted and the original net names are not retained in the Verilog netlist.

This application option affects each net that is named differently from its port. In the Verilog language, the port and pins of a net are connected together by an implicit net. This net is implicitly named to match one of its ports. When writing Verilog, a net name is implicitly renamed to its connected port's name, and thus the original net name is not retained in the Verilog output. The netlist is valid. However, to retain the original net name, an assign statement is necessary to assign the original net name to the actual net name in the Verilog output.

To determine the current value of this application option, use the *get_app_option_value -name file.verilog.write_pg_net_assigns* command. For a list of all verilog application options and their current values, use the *report_app_options file.verilog.** command.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [read_verilog_outline](#)
- [read_verilog](#)
- [report_app_options](#)

file.verilog.write_unconnected_pg_ports

Controls whether the write_verilog outputs unconnected PG ports.

Data Types

Boolean

Default false

Description

When the value is true, unconnected PG ports are outputted when -include pg_objects and unconnected_port or -exclude without unconnected_port used here.

When the value is false, unconnected PG ports are not outputted regardless the usage of -include and -exclude options.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [read_verilog_outline](#)
- [read_verilog](#)
- [report_app_options](#)

finfet.ignore_grid.std_cell

Controls whether various commands honor the FinFET grid for standard cells, site definitions, site rows, site arrays, and the core area in the whole block or certain steps of the flow.

Data Types

Boolean

Default false

Description

By default (*false*), the commands honors the FinFET grid for standard cells, site definitions, site rows, site arrays, and the core area.

When set to *true*, commands such as *create_site_array*, *create_site_row*, and *initialize_floorplan* ignore the FinFET grid for standard cells, site definitions, site rows, site arrays, and the core area even if the FinFET grid is defined in the technology file.

See Also

- [check_finfet_grid](#)
- [initialize_floorplan](#)
- [create_site_row](#)
- [create_site_array](#)
- [report_app_options](#)
- [set_app_options](#)

flip_chip.route.allow_rotated_via

Option for the RDL router commands. Specifies how to create NDR vias.

Data Types

Boolean

Default true

Description

When this option is true, the RDL router can create rotated NDR vias. When this option is false, RDL router create the specified NDR vis only.

See Also

- [route_rdl_flip_chip](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

flip_chip.route.bump_via_landing_mode

Option for the RDL router commands. Specifies routing from bump to nearest user_route via.

Data Types

Boolean

Default false

f

Description

When this option is false, RDL router performs normal pin-to-bump or bump-to-bump routing. When this option is true, RDL router connects each bump to the nearest user_route via. If there is no same net via on the routing layer, the bump is left unconnected. Note that RDL router removes unfixed floating shapes before routing, thus the shape_type of these pre placed vias need to be set as "user_route" to be kept until the routing stage.

See Also

- [route_rdl_flip_chip](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

flip_chip.route.connect_edge_center

Option for the RDL router commands. Specifies how to connect to Through-Silicon Vias (TSVs) or bump cells.

Data Types

Boolean

Default false

Description

When this option is false, the RDL router might not connect to TSVs or bumps at the center of an edge. When this option is true, RDL router is forced to connect to TSVs or bumps at the center of edge.

See Also

- [route_rdl_flip_chip](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

flip_chip.route.connect_pad_without_endcap

Option for the RDL router commands. Specifies how to connect to pads or vias.

f

Data Types

Boolean

Default false

Description

When this option is true, the RDL router uses a path with flush endcap to connect to pads or vias. When this option is false, RDL router uses paths with half_width / octagon endcap to connect to pads or vias depending on the layer routing angle.

See Also

- [route_rdl_flip_chip](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

flip_chip.route.connect_via_center

Specifies how the RDL router connects RDL routes to vias.

Data Types

Boolean

Default false

Description

When this option is true, the RDL router connects the centerline of an RDL route to the center of the connected via. By default, this option is false and the RDL router is not required to connect to the via center.

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [route_rdl_flip_chip](#)
- [set_app_options](#)

f

flip_chip.route.create_soft_rule_shield

Option for the RDL router commands. Specifies how to create shielding shapes.

Data Types

Boolean

Default false

Description

When this application option is set to *false* , RDL router creates shielding shapes according to hard NDR rules. When this application option is set to *true* , RDL router not only creates shielding shapes with hard NDR rules but also creates shielding shapes with soft shield width and spacing.

See Also

- [create_rdl_shields](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [route_rdl_flip_chip](#)
- [set_app_options](#)

flip_chip.route.design_style

Specifies the design style.

Data Types

String (flip_chip_peripheral_io | 3dic_single_die | 3dic_interposer)

Default flip_chip_peripheral_io

Description

Specifies the design style before using RDL commands. For ordinary RDL design, set *flip_chip_peripheral_io*. For 3DIC stacked die, set *3dic_single_die*. For interposer, set *3dic_interposer*.

See Also

- [route_rdl_flip_chip](#)
- [route_3d_rdl](#)

f

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

flip_chip.route.detour_cost

Specifies the detour cost level for the auto via insertion flow.

Data Types

String (*low* | *high*)

Default low

Description

Specifies the detour cost level for the auto via insertion flow. If the detour cost is *low*, the RDL router avoids congestion by implementing detours on the same layer rather than changing layers. If the detour cost is *high*, the RDL router uses different layers instead of implementing detours.

See Also

- [route_rdl_flip_chip](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

flip_chip.route.extend_vias_on_pin

Option for the RDL router commands. Specifies how to create vias and connect with pin.

Data Types

String (*false* | *true* | *full_pin*)

Default false

Description

The new option is to increase the number of vias to connect with the target pin. There are three modes. When this option is *true*, the RDL router will extend original stack vias by a wire. The width of extended wire is the same as the original Net setting. And then the RDL router will create as many as possible vias under the extended wire to connect with

f

target pin. When this option is *full_pin*, the RDL router will extend original stack vias by a wire. The extended wire will try to cover the target pin. The width of extended wire will not follow the original Net setting. And then the RDL router will create as many as possible vias under the extended wire to connect with target pin. When this option is *false*, the RDL router will not change original behavior. This is default setting. For more efficiently creating vias and ease of use, the new vias will not follow the NDR via setting.

See Also

- [route_rdl_flip_chip](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

flip_chip.route.layer_bump_spacings

Option for the RDL router commands. Specifies the bump-to-route spacing rules for each given metal layer.

Data Types

list

Default null

Description

This option specifies the bump-to-route spacing rules for each given metal layer. You can specify spacing values that are larger than the net nondefault routing rule (NDR) spacing. The list format is:

```
{layer_name1 spacing1
    layer_name2 spacing2
    layer_name3 spacing3 ...}
```

You must specify the layer name by using the layer name from technology file. The units are in microns.

By default, the RDL router uses the minimum spacing specified in the technology file.

See Also

- [route_rdl_flip_chip](#)
- [get_app_option_value](#)

f

- [report_app_options](#)
- [set_app_options](#)

flip_chip.route.layer_routing_angles

Option for the RDL router commands. Specifies the routing angle for each given metal layer.

Data Types

list

Default null

Description

Specifies the routing angle for each given metal layer. The list format is:

```
{layer_name1 routing_angle1
  layer_name2 routing_angle2 ...}
```

You must specify the layer name by using the layer name from technology file. The supported routing angle settings are:

```
45_degree
  90_degree
```

By default, the RDL router uses 45_degree for routing.

See Also

- [route_rdl_flip_chip](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

flip_chip.route.layer_tie_shield_widths

Specifies the width of shields to tie for each given metal layer. Option for the RDL router commands.

Data Types

list

Default null

f

Description

Specifies the width of shields to tie for each given metal layer. The list format is:

```
{layer_name1 width1
  layer_name2 width2 ...}
```

You must specify the layer name by using the layer name from technology file.

By default, the RDL router uses NDR width of shielding net.

See Also

- [create_rdl_shields](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [route_rdl_flip_chip](#)
- [set_app_options](#)

flip_chip.route.point_to_point_connection_file_name

Option for the RDL router commands. Specifies the file name of point-to-point connection data for point-to-point routing.

Data Types

string

Default Empty

Description

The point-to-point connection file format is as follow:

```
#netName #layer #point1_x #point1_y #point2_x #point2_y
```

- **#netName**: net name, which is the net for routing
- **#layer**: layer name, which is the target routing layer for the point-to-point connection
- **#point1_x #point1_y**: first connection point
- **#point2_x #point2_y**: second connection point

Example:

```
Net1 AP 10.5 15.0 20.5 20.0
```

f

means to inform router route net Net1 from point (10.5, 15.0) to point (20.5, 20.0) on the AP layer.

The points need to follow following rules:

- The points need to fall inside same net shapes, such as wire, via, or pin shape.
 - For wire shapes, the point needs to be exactly the end_point of the wire.
 - For pin or via shape, router connects to the pin/via but not guarantee to be exactly the point specified in the file.
- The points need to be on min grid.
- The points need to be on a DRC violation free coordinate, which means when routing a unit wire on this point, there is no DRC violation with neighboring shapes.

See Also

- [route_rdl_flip_chip](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

flip_chip.route.route_to_ring_nets

Option for the RDL router commands. Specifies the ring nets for RDL routing.

Data Types

List

Default null

Description

For each ring net, the RDL router connects its floating bumps to the specified ring net. The list format is:

```
{net_name1 net_name2 ...}
```

See Also

- [route_rdl_flip_chip](#)
- [get_app_option_value](#)

- [report_app_options](#)
- [set_app_options](#)

flip_chip.route.route_to_stripe_nets

Option for the RDL router commands. Specifies the stripe nets for RDL routing.

Data Types

List

Default null

Description

For each stripe net, the RDL router connects its floating bumps to the specified stripe net. The list format is:

```
{net_name1 net_name2 ...}
```

See Also

- [route_rdl_flip_chip](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

flip_chip.route.routing_style

Specifies how to connect RDL nets from driver pads to bumps.

Data Types

String (star | spanning_tree)

Default star

Description

Specifies how to connect RDL nets from driver pads to bumps. The *star* option directs the RDL router to create a separate connection between each driver pad and bump. The *spanning_tree* option directs the RDL router to connect all routing targets (drivers and bump cells) on the same net into a spanning tree.

f

See Also

- [route_rdl_flip_chip](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

flip_chip.route.secondary_routing_layer_cost

Option for the RDL router commands. Specifies the secondary layer cost for the auto via insertion flow.

Data Types

String (low | medium | high)

Default medium

Description

Specifies the secondary layer cost for the auto via insertion flow. You can use this option to trade off between via number and secondary layer utilization rate. Use the *high* cost level to implement a low utilization rate for the secondary layer; use the *low* cost level to implement a low number of vias. By default, the secondary routing layer cost is *medium*.

See Also

- [route_rdl_flip_chip](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

flip_chip.route.secondary_shielding_net

Specifies the secondary shielding net for RDL shielding. This option is used by `create_rdl_shields` command.

Data Types

String

Default ""

Description

Specifies the secondary shielding net for RDL shielding. This option is used to achieve better shield ratio in `create_rdl_shields` command.

See Also

- [create_rdl_shields](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

flip_chip.route.shielding_net

Specifies the shielding net for RDL routing. This option is used by the RDL router commands.

Data Types

String

Default ""

Description

Specifies the shielding net for RDL routing. Since the RDL router uses only one shielding net for each run, only one shielding net is allowed in this option. If this option is not set, RDL router uses the ground net with maximum number of ports as the shielding net.

See Also

- [create_rdl_shields](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [route_rdl_flip_chip](#)
- [set_app_options](#)

flow.common.advanced_technology

Data Types

string

f

Default ""**Description**

This application option controls the enablement of select tool features that are targeted to become on by default in a future release.

Supported values are shown below. "AT-20230902" "*latest*"

When the app option is set to "*latest*", select tool features that are part of the most recent supported value are turned on.

flow.common.effort

Enables select tool features that are not on by default.

Data Types

string

Default -1**Description**

This application option controls the enablement of select tool features that are targeted to become on by default in a future release.

When the app option is set to (12345), select tool features that are not on by default are turned on.

flow.common.io_priority

Controls the effort for IO path group from various Fusion Compiler engines, including optimization, placer, routing, CTS ...

Data Types

string

Default high**Description**

This app option controls the optimization effort for IO paths. It provides a tradeoff between R2R timing paths and IO timing paths. When the app option is set to (*low*), the tool does not optimize IO paths, no cell sizing or buffering or layer promotion on the IO paths. When the app option is set to (*medium*), R2R paths are prioritized over IO paths by the different engines. The tool does limited optimization on IO paths with consideration of LDRC &

f

power efficient, and skip IO paths in CTS, CCD. When the app option value is (*high*), IO paths and R2R paths have equal priority. The tool optimizes both IO & R2R timing paths.

flow.common.node_specific_ppa

Data Types

Boolean

Default true

Description

Specifies if node specific PPA flow should be enabled.

See Also

- [set_app_options](#)

flow.common.wire_optimization_features

Controls Wire Optimization features.

Data Types

list of string

Default ""

Description

This application option controls Wire Optimization settings for the optimization and routing engines.

It is recommended to use *flow.common.wire_optimization_strategy* to turn on main Wire Opt settings and then use this application option to enable additional features.

When (*auto_ndr*) is added to the list of the application option, settings for Auto NDR are used.

When (*vlo*) is added to the list of the application option, settings for Via Ladder Optimization are used.

When (*power*) is added to the list of the application option, additional switching power optimization in *clock_opt* will be run.

See Also

- [enable_wireopt_improvements](#)

flow.common.wire_optimization_strategy

Controls Wire Optimization strategies.

Data Types

integer

Default 0

Description

This application option controls Wire Optimization settings for the optimization and routing engines with different versions. The latest supported version is 3.

Use *flow.common.wire_optimization_features* to enable additional features for the Wire Optimization, such as Via Ladder Optimization.

Versions

- 1 Legacy Wire Optimization, prone to over-promote nets due to insufficient length and congestion control mechanisms
- 2 Improved Wire Optimization featuring adaptive layer binning and length control
- 3 Enhanced congestion control for Virtual Route based optimization and improved Global Route robustness to incoming layer promotion

See Also

- [enable_wireopt_improvements](#)

flow.data_robustness.enable_ezi

Enable EZI improvements when EDR is enabled.

Data Types

Boolean

Default false

Description

This application option enables EZI (Enable Zroute Improvements) when the Enable Data Robustness (EDR) feature is active. When set to true, the tool applies enhanced algorithms and optimizations specific to EZI processing that improve the robustness and quality of results.

f

This option should be set before invoking the *enable_data_robustness* command to ensure the EZI improvements are properly applied during EDR feature processing.

See Also

- [get_app_option_value](#)
- [set_app_options](#)
- [report_app_options](#)

flow.data_robustness.enable_timer_improvements

Enable timer improvements when EDR is enabled.

Data Types

Boolean

Default false

Description

This application option enables timer improvements when the Enable Data Robustness (EDR) feature is active. When set to true, the tool applies enhanced optimizations that improve the robustness of timing.

This option should be set before invoking the *enable_data_robustness* command to ensure the timer improvements are properly applied during EDR feature processing.

See Also

- [get_app_option_value](#)
- [set_app_options](#)
- [report_app_options](#)

flow.data_robustness.version

Specifies the version of EDR features to be applied.

Data Types

string

Default latest

f

Description

This application option specifies which version of Enable Data Robustness (EDR) features should be applied when the *enable_data_robustness* command is invoked without the *-version* command line option.

Valid values are:

- *latest* (default)

Automatically selects the most recent EDR version that is appropriate for the current product build date. This ensures compatibility and stability while providing the latest validated improvements. It is recommended to always use the latest version.

- *1, 2, 3, ...* (fewer than 8 digits)

Specifies a particular EDR version by its version number.

- *v1, v2, v3, ...*

Specifies a particular EDR version by its version identifier with 'v' prefix. This is equivalent to the bare integer format above.

- *YYYYMMDD* or *YYYY-MM-DD* (exactly 8 digits)

Specifies a date in YYYYMMDD or YYYY-MM-DD format. The tool will apply all EDR features that were released on or before this date. This provides fine-grained control over which features are included based on their release dates.

- *EDR-YYYYMMDD* or *EDR-YYYY-MM-DD*

Specifies a date with the EDR- prefix in YYYYMMDD or YYYY-MM-DD format. This is equivalent to the date formats above and is provided for convenience.

Command Line Precedence

If the *-version* option is specified on the *enable_data_robustness* command line, it takes precedence over this application option. In this case, if the application option is set to a value other than 'latest', a warning (EDR-1004) will be issued.

Invalid Values

If an invalid value is specified and the *enable_data_robustness* command is invoked without a command line *-version* option, a warning (EDR-1005) will be issued and the tool will default to 'latest'.

To determine the current value of this application option, use the *get_app_option_value -name flow.data_robustness.version* command.

For a list of all flow.data_robustness application options and their current values, use the *report_app_options flow.data_robustness.** command.

f

To see available EDR versions, use the *enable_data_robustness -list_versions* command.

Examples

Set the version to latest (automatic selection):

```
prompt> set_app_options -name flow.data_robustness.version -value latest
```

Set the version to a specific version number (bare integer):

```
prompt> set_app_options -name flow.data_robustness.version -value 2
```

Set the version to a specific version identifier (with 'v' prefix):

```
prompt> set_app_options -name flow.data_robustness.version -value v2
```

Set the version to a specific date:

```
prompt> set_app_options -name flow.data_robustness.version -value  
2025-09-15
```

Set the version using the EDR- prefix format:

```
prompt> set_app_options -name flow.data_robustness.version -value  
EDR-2025-09-15
```

See Also

- [get_app_option_value](#)
- [set_app_options](#)
- [report_app_options](#)

flow.runtime.version

Data Types

string

Default ERI-20220712

Description

Specifies which version *enable_runtime_improvements* should apply.
enable_runtime_improvements -list_versions reports all versions supported.

See Also

- [enable_runtime_improvements](#)

f

flow.set_qor_strategy.version

Data Types

*string***Default** empty string

Description

Specifies which version *set_qor_strategy* should apply. Valid string is SQS-YYYYMMDD. Use *set_qor_strategy -list_versions* to show supported versions.

flow.set_tech.version

Data Types

*string***Default** empty string

Description

Specifies which version *set_technology* should apply. Valid string is ST-YYYYMMDD.

See Also

- [set_technology](#)
-

formality.svf.integrate_in_ndm

Enables or disables SVF in NDM feature.

Data Types

*boolean***Default** false

Description

This application option enables or disables save the SVF in NDM. The recommended flow is as follows:

```
set_app_options -name formality.svf.integrate_in_ndm -value true //
  turns on the feature
set_svf precomp.svf //
  create the first svf file
analyze -f sverilog design.sv
elaborate top
```

g

```

set_top_module top
set_svf_initial_map.svf //
  create the second svf file
compile_fusion -from initial_map -to initial_map
save_block -as initial_map

```

The guidance's generated: precomp.svf and initial_map.svf are saved into block 'initial_map', its recommended to put differential names for each svf, otherwise the tool will save it as numbered files.

To list the svf files use the command:
`extract_svf -list_svf`

To generate the svf files from NDM use the command:
`extract_svf -generate -files { precomp.svf initial_map.svf }`

NOTE: The switches "-off" "-append" from svf command are not supported when you are using the SVF in NDM feature, using them could lead issues in the flow.

See Also

- [extract_svf](#)
- [set_svf](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

g

gui.annotation_symbol_dir

Specifies the directory for SVG files used in annotation_shapes of type symbol.

Data Types

string

Default ""

Description

This option specifies the search path for SVG files used for annotation_shapes of the type symbol. If specified, the file name needs to be provided only; the full path should be used, otherwise.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)

gui.auto_link_blocks

Controls whether the tool automatically links the current block.

Data Types

Boolean

Default true

Description

This application option controls whether the tool automatically links the current block.

When set to *true* (the default) and the GUI is active, the tool automatically links the current block when the current block changes. The link occurs when the application is idle (for example, when the current command or script finishes).

This application option has been Obsoleted. Use `design.disable_auto_link` instead. (Note that these two application options are opposite. When `design.disable_auto_link=true`, means `gui.auto_link_blocks=false`).

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)

gui.auto_open_design_windows

Controls whether the GUI automatically opens a new design window when you change the current block.

Data Types

Boolean

Default true

Description

By default (*true*), the GUI automatically opens a new design window when you change the current block.

The new window will be a Layout window for a new physical block.

In fusion shell front end or single shell mode the new window will be a Hierarchy window for a new logical block.

To prevent the GUI from opening a new design window when you change the current block, set this option to *false*.

See Also

- [current_block](#)
- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)

gui.batch_x_display

Controls the X display output of the GUI in batch mode for support of offscreen mode.

Data Types

String

Default default (no batch mode), offscreen (batch mode)

Description

This application option controls the default value for how the GUI outputs to the X display in batch mode (-batch application option). Valid values are "default", and "offscreen".

If the "default" value is used then the application defaults the display to the normal X display (defined by the DISPLAY environment variable).

If the "offscreen" value is used then the application defaults the display to an offscreen image and does not use/or need access to the X display.

When the gui is started (using the gui_start command) in batch mode this option is used to determine the display mode, unless explicitly overridden by gui_start command options.

Offscreen mode is designed to be used when running batch scripts which need access to the gui to generate screen shots of the design. In this mode gui dialogs will not block (will

run as if OK was automatically selected) and Tk is disabled (this package requires an X display).

See Also

- [gui_start](#)
- [get_app_option_value](#)
- [set_app_options](#)
- [report_app_options](#)

gui.block_mark_type

Specifies the block mark type that will be used in "block" timing_path attributes.

Data Types

string

Default *logical_full_name*

Description

Specifies the block mark type that will be used in "block" timing_path attributes. These attributes will then be used to categorize timing_path's in the Path Analyzer in the GUI.

The "block" timing_path attributes supported are:

all_blocks: An ordered Tcl list of the "coarse grain" elements that a path travels through, including elements such as the full names of hard or soft macros, "_Top_" (represents the top block), and "_Port_" (represents a design port).

start_block: Its value is the first (start) element name that appears in the Tcl list value of the "all_blocks" attribute.

end_block: Its value is the last (end) element name that appears in the Tcl list value of the "all_blocks" attribute.

start_end_blocks: Its value contains only the first (start) and the last (end) element names from the list value of the "all_blocks" attribute (in Tcl list format).

through_blocks: Similar to "all_blocks", except that the first (start) and the last (end) element names are omitted from the Tcl list value.

The *gui.block_mark_type* can take on the following values to customize the block marks for the "block" attributes of a given timing_path:

logical_name: Logical hierarchy

g

physical_name: Physical hierarchy

boundary_name: Boundary hierarchy as defined by color_by_hierarchy feature

rtl_module_name: RTL module hierarchy

logical_full_name: Full name logical hierarchy

physical_full_name: Full name physical hierarchy

boundary_full_name: Full name Boundary hierarchy

rtl_module_full_name: Full name RTL module hierarchy

Examples

The following example shows how to set the app option to customize the "block" timing_path attributes to return full name logical hierarchy data.

```
prompt> set_app_options -list {gui.block_mark_type {logical_full_name}}
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)

gui.bump_comparison_location_tolerance

Set tolerance for location-based comparison of the bumps or pseudo bumps in design I/O CSV files.

Data Types

Double

Default 0.0

Description

This option allows the user to specify a tolerance for comparing the x and y coordinates of bumps or pseudo bumps during a bump ECO process using the "compare_design_io" command. If the Euclidean distance between two bumps is within the specified tolerance, they are considered to be at the same location. When two bumps are treated as being at the same location due to the tolerance, and all other fields match, they will not be included in the diff table even if a small movement has occurred. Consequently, such movements

will not be reflected when reading back the diff CSV file using the 'read_design_io -eco' command.

See Also

- [compare_design_io](#)

gui.command_form_close_dialog_after_run_or_script

Controls whether a shell command form invoked from command search closes itself after the command is successfully executed or scripted.

Data Types

Boolean

Default false

Description

When you invoke the command search from the menu bar search icon and select a shell command during the search, the GUI creates a form for entering the command parameters.

This option controls whether the command form automatically closes after the command is successfully executed or added to the script editor.

By default (*false*), the command form remains open so that you can execute the same command multiple times, possibly after changing some of the parameters.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)

gui.command_form_log_options_with_default_values

Controls whether a shell command form invoked from command search appends optional command options with default values to the command string for execution and logging.

Data Types

Boolean

Default false

Description

When you invoke the command search from the menu bar search icon and select a shell command during the search, the GUI creates a command form. The command form allows the user to input values for command options, if any. By default (*false*), when users do not enter a value explicitly for an optional option, the option string is omitted when the command is invoked. Setting this application option to *true* forces the command form to append optional options with default values to the command string for execution and logging.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)

gui.command_form_show_result_dialog_after_run

Controls whether a shell command form invoked from command search displays the results from the command execution in a message dialog.

Data Types

Boolean

Default false

Description

When you invoke the command search from the menu bar search icon and select a shell command during the search, the GUI creates a form for entering the command parameters.

This option controls whether the tool displays the results of a successfully executed command in a message dialog.

Command results can often be uninteresting, such as the name of a resulting collection, so having the results displayed in a dialog which must be dismissed is unnecessary. By default (*false*), the tool outputs the command results to the terminal but does not display them in a message dialog. If you set this option to *true*, the tool displays the results in a message dialog.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)

gui.command_form_use_object_names_for_proc_options

Controls the value format for Tcl proc options declared as design objects.

Data Types

Boolean

Default false

Description

Many shell commands and Tcl procs contain options for inputting one or more design objects, such as "cell" or "net". The value types for Tcl proc options are declared using the *define_proc_attributes* command. GUI command forms for these shell commands and Tcl procs use the object chooser control to allow the user to easily choose and enter one or more objects as option values.

Using the object chooser, the user may enter objects of heterogeneous types. By default, the object chooser resolves user input to a collection of objects. Using a collection of objects leaves no ambiguity about the object type for each collection member.

In some past product releases, the object chooser resolved user input to a list of object names instead of a collection of objects. User scripts exist with Tcl procs that expect a list of names for option values and do not operate properly when called with a collection of objects. This application option provides a backward compatible mode for these user scripts. Setting this option to *true* causes the object chooser to resolve user input to a list of object names in place of a collection when the GUI form is used for a Tcl proc.

This application option is provided as a temporary workaround until user scripts can be updated to properly handle the collection input for option values. Since the list of object names leaves ambiguity about the associated object types, it is error prone. In addition, the command form also allows the end user to input a command or a variable reference as an option input value. In these cases, the possibility of the option value resolving to a collection still exists regardless of this application option setting.

See Also

- [define_proc_attributes](#)
- [gui_show_command_form](#)
- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)

gui.custom_setup_files

Specifies a list of Tcl files to source when the GUI starts.

Data Types

list

Default ""

Description

This option specifies a list of Tcl files to source when the GUI starts.

If the file name is fully qualified (starts with `./` or `/`), the tool loads it from the specified directory. If the file name is not fully qualified, the tool searches the following directories for the file:

- 1) `<root>/admin/setup`, where `<root>` is the installation root of the executable
- 2) `~/.synopsys_icc2_gui`
- 3) `.`

The tool does not issue a warning or error message if the file is not found in any of the search directories.

Note that the custom setup files are loaded only if the *gui.enable_custom_setup_files* application option is *true*, which is the default.

See Also

- [gui_start](#)
- [get_app_option_value](#)
- [get_app_options](#)

- [report_app_options](#)
- [set_app_options](#)

gui.design_rule_package_dir

Set a datafile where gui_design_rule_assistant can load data from.

Data Types

string

Default ""

Description

The database file which stores the rule information in form of sqlite3. If this option is missing. User should user -dbfile to specify which database file to load.

gui.embedded_app_options_help_page

Show the help page in the application options dialog

Data Types

Boolean

Default false

Description

When this option is enabled, the help page of the selected application option shows in the application options dialog.

gui.enable_custom_setup_files

Controls whether custom setup files are loaded.

Data Types

Boolean

Default true

Description

By default (*true*), if you have specified custom GUI setup files with the *gui.custom_setup_files* application option, the tool loads these files when it opens the GUI.

If you set this option to *false*, the tool does not load the custom setup files.

See Also

- [gui_start](#)
- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)

gui.file_editor_command

Controls the command used to edit a file

Data Types

String

Default ""

Description

The application option allows the user to specify a custom command to be used to edit a file in an external editor using the `gui_show_file_in_editor` command.

If the option is set to a non-empty string then this string will be used as the command to run when editing the external file. The command string can include the following special tokens:

`\%filename` which will be replaced by the name of the file being edited. If `\%filename` is not found in the command string then the filename will just be appended to the end of the command string.

`\%line` which will be replaced by the line number in the file to start editing at. If `\%line` is not found in the command string then the line number will not be added.

Examples

```
# edit file in xterm using vim
set_app_options -list {gui.file_editor_command {xterm -e "vim %filename
+%line"}}

# edit file in xterm using emacs
set_app_options -list {gui.file_editor_command {xterm -e "emacs -nw
+%line %filename"}}

# edit file using gvim
set_app_options -list {gui.file_editor_command {gvim %filename +%line}}
```

g

```
# edit file using xemacs
set_app_options -list {gui.file_editor_command {xemacs +%line %filename}}
```

NOTES

Any specified commands will be searched for in the user's current path. If the executable cannot be found then use a fully qualified path.

See Also

- [gui_show_file_in_editor](#)
- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)

gui.filter_default_condition

Default condition of "Quick Match" filter in "Set pattern matching and options" in GUI filters.

Data Types

String

Default Starts_with

Description

This application option controls the default filter "Quick Match" condition in "Set pattern matching and options" of GUI Filters on a global scale. When user enters the filter expression, filtered by quick match condition, the matched objects will show in the list. Notice the app option only controls the default condition type. Once user changes the condition manually, the app option does not apply further.

There are 4 types of value used for setting the Quick Match to the according condition:

- If the "Equals" value is used then Quick Match is set to Equals. Objects with name the same as filter expression are matched.
- If the "Starts_with" value is used then Quick Match is set to Starts with. Objects with name starts with the filter expression will be matched.
- If the "Ends_with" value is used then Quick Match is set to Ends with. Objects with name ends with the filter expression are matched.
- If the "Contains" value is used then Quick Match is set to Contains. Objects with name contains the filter expression are matched.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)

gui.follow_mouse_mode

Enables auto activation of views when the mouse hovers over them.

Data Types

Boolean

Default false

Description

This control enables auto activation of inactive views when the user hovers the mouse over them instead of requiring the user to click the mouse on the inactive view first.

Hover means that the user moves the mouse over the view and then does not move the mouse for a short period of time.

The control is designed to help users who:

- prefer the standard window manager follow mouse option where windows are activated, and optionally raised, when the mouse moves over the window.
- want to avoid accidental clicks in view when trying to activate and already active view.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)

gui.graphics_system

Controls the graphics system used by the GUI.

Data Types

String

Default auto

Description

This application option controls how the GUI draws to the X-Server. Valid values are "auto", "native" and "raster". This application option becomes read-only after the GUI has been started for the first time, since it cannot be changed after the GUI has been started. After the GUI has started the app option will reflect the chosen graphics system (either native or raster) as RENDER X-Server extension. For X-Servers missing this capability, text and lines may not be anti-aliased and will look jagged, and support for transparencies will be missing. In general this graphics system requires less network bandwidth than raster.

Complex views such as the layout view will also have their own separate controls that impact the performance of those views, so you should look at their specific view settings for additional details.

The "raster" graphics system draws all display images locally and then sends those images to the X-Server for display. This system will give a nice appearance on any X-Server, even if the RENDER extension is not present. The raster graphics system requires more network bandwidth than the native system. In cases where there is a local X-Server running on the same machine as the application it can provide better performance than native.

See Also

- [get_app_option_value](#)
- [set_app_options](#)
- [report_app_options](#)

gui.icv_home_dir

Specifies the ICV home directory if ICV is installed.

Data Types

string

Default ""

Description

This option specifies the IC Validator (ICV) home directory if it is installed. This option defaults to the ICV_HOME_DIR environment variable. The tool uses this option to source ICV tcl files to setup ICV menu items. If not specified, ICV menu items will not be available from the GUI menus.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)

gui.layer_solid_fill_pattern

Controls the default display of solid-filled layers.

Data Types

string

Default Dense4Pattern

Description

The technology defines the appearance of layers in the GUI. Some technology files define layers with a solid fill pattern. Using a solid fill pattern causes some usability issues because valuable information is not visible in the layout window. For example, you cannot see net names for shapes on solid-filled layers.

This application option controls how these solid fill patterns are mapped. By default, the GUI maps the solid pattern to a pattern that appears more transparent. This allows net names to be visible for example. Set this option to *SolidPattern* to disable this mapping.

To see the set of available fill patterns, use the `get_app_option_value -details -name gui.layer_solid_fill_pattern` command.

See Also

- [gui_start](#)
- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)

gui.max_cores

Controls the maximum number of CPU cores used by the GUI.

Data Types

int

Default 0

Description

This application option controls how many CPU cores the GUI can use in order to improve interactive response time. The default setting of 0 means that the GUI is allowed to use the available number of CPU cores on the system, up to 16 cores. A value of 1 means that only one core will be used.

This is a global application option; therefore, you can add the setting of this option to your application setup file to override the default setting.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [set_app_options](#)
- [report_app_options](#)

gui.mouse_tool_auto_raise

Set auto raise for mouse tool palette

Data Types

Boolean

Default false

Description

This option allows the user to choose whether the GUI mouse tool palette will automatically be raised above other palettes when opened.

See Also

- [gui_mouse_tool](#)

gui.mouse_tool_config

Set style for GUI mouse tool UI

Data Types

String

Default toolbar

Description

This option allows the user to chose between a toolbar or a palette for the GUI mouse tool UI.

See Also

- [gui_mouse_tool](#)

gui.name_based_hier_delimiter

Specifies the hierarchy delimiter used by the Name Based Hierarchy Browser.

Data Types

String

Default /

Description

This application option specifies the hierarchy delimiter used by the Name Based Hierarchy Browser when processing a logically flat design. You can change this value and use a different delimiter to extract the hierarchy from a design. Only one character is allowed. The effective scope is block. The default value is "/".

See Also

- [get_app_option_value](#)
- [set_app_options](#)

gui.num_3dic_layers

Controls the number of backside and front side metal layers displayed in a 3dic design.

Data Types

int

Default 3

Description

This option controls how many front and back side metal layers are displayed in the layout. By default is it set to zero, which means the layers are unlimited.

When it is set to non zero value and layout view is opened for a 3dic design, the layout view limits the visible layers displayed in each chip to those layers on the "outside" of the chip. We define the outside as the top most metal layers, and the bottom most back metal layers.

This is a global app_option, so you can add the setting of this app_option to your application setup file to override the default setting.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)

gui.persistent_ruler

Select the type of ruler the ruler tool creates.

Data Types

Boolean

Default true

Description

With this option enabled, the ruler tool will create a persistent ruler, however, if it is disabled, the ruler tool will create a gui ruler.

See Also

- [create_ruler](#)

gui.selection_stack_depth

Specifies the maximum depth of the selection stack

Data Types

Integer

Default 10

Description

When you push your selection onto the selection stack, the tool checks for the maximum depth and any extra items on the selection stack beyond the stack depth, are discarded.

This check is to enforce a limit of memory and resources needed to maintain the selection stack where each selection can contain a large number of object references.

If the value is set to a value less than 1 then the maximum depth is ignored so items are never discarded.

See Also

- [gui_selection_stack](#)

gui.selection_undo

Enable/Disable single level undo for change_selection

Data Types

Boolean

Default true

Description

When this option is enabled, and the user changes the current selection, the previous selection is saved so the user can undo the change and restore the previous selection.

The user can use the command 'change_selection -undo' to undo the selection change.

See Also

- [change_selection](#)

gui.tk_mode

Controls how Tk is initialized and used by the GUI.

Data Types

String

Default yes

Description

This application option controls how the GUI initializes Tk on `gui_start` when not running in offscreen mode.

Valid values are "on", "off" and "auto_load".

If the "on" value is used then Tk is loaded on the first `gui_start` call.

If the "off" value is used then Tk is not loaded on `gui_start` and is disabled.

If the "auto_load" value is used then Tk is not loaded on `gui_start` but will be automatically loaded when the uses a command that requires Tk or it is explicitly loaded using using "package require Tk"

Standard GUI commands that load Tk are:

- `gui_create_tk_palette_type`
- `gui_create_tk_view_type`
- `gui_create_task_page -type tk`

TK ISSUES

Loading Tk causes issues with saving checkpoints, using "save_checkpoint" command, and when closing the GUI display, using "gui_stop -close" command, because the Tk third party library does not shutdown the X display correctly.

If Tk is enabled "save_checkpoint" is not allowed (will fail with error message).

See Also

- [gui_start](#)
- [get_app_option_value](#)
- [set_app_options](#)
- [report_app_options](#)
- [gui_create_tk_palette_type](#)
- [gui_create_tk_view_type](#)
- [gui_create_task_page](#)
- [gui_stop](#)

gui.url_browser_command

Controls the command used to display a URL in a web browser

Data Types

String

Default ""

Description

The application option allows the user to specify a custom command to be used to display a URL in an external web browser using the `gui_show_url_in_browser` command.

If the option is set to a non-empty string then this string will be used as the command to run when displaying the URL. The command string can include the special characters '%url' which will be replaced by the URL being displayed, if the characters '%url' are not found in the command string then the URL will just be appended to the end of the command string.

See Also

- [gui_show_url_in_browser](#)
- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)

gui.user_icon_directories

Specifies a list of directories to search for icon files to use in various GUI commands

Data Types

list

Default ""

Description

This option specifies a list of directories to search for user specified icon files.

When the application requests an icon of a specified name, the specified user icon directories are searched first followed by the installation's icon file directories.

The application looks for a filename `<icon_name>.svg` which is an SVG file defining the icon.

The user supplied SVG icons can be used in the GUI for user define interface items e.g. menu buttons on toolbars.

If the application is running in dark mode, and a 'dark' subdirectory of a specified directory contains the file, then that file will be used instead.

See Also

- [gui_start](#)
- [gui_create_toolbar_item](#)
- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)

gui.verdi_indesign.vcs_home

Specifies the home directory of the VCS (Simulator) home. Option for the *gui_start_verdi* command.

Data Types

string

Description

Specifies the home directory of the VCS (Simulator) home. If set, the tool uses this information when the *gui_start_verdi* command is run; else, the tool uses the value of the *VCS_HOME* environment variable.

Examples

The following example shows how to set the path for the VCS (Simulator) home.

```
prompt> set_app_options -list {gui.verdi_indesign.vcs_home  
    {/global/apps/vcs_2019.06-SP1}}
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)

gui.verdi_indesign.verdi_home

Specifies the home directory of the Verdi executable. Option for the *gui_start_verdi* command.

Data Types

string

Description

Specifies the home directory of the Verdi executable. If set, the tool uses this information when the *gui_start_verdi* command is run; else, the tool uses the value of the *VERDI_HOME* environment variable.

Examples

The following example shows how to set the path for the Verdi executable.

```
prompt> set_app_options -list {gui.verdi_indesign.verdi_home  
    {/global/apps/verdi_2019.06-SP1}}
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)

gui.write_image_data_file

Enable gui write image to data file.

Data Types

Boolean

Default true

Description

Specifies if image writing to data file is enabled. This option shows whether write image to a data file is enabled, however, if it is disabled, the option to write image to a data file is disabled.

See Also

- [create_ruler](#)

gui_build_query_data_table

Controls whether the tool initializes data for fast data query after the GUI starts.

Data Types

Boolean

Default The default value for *gui_build_query_data_table* is true.

Description

This variable controls whether IC Compiler initializes query data during GUI startup. By default, the *gui_build_query_data_table* variable is true and IC Compiler initializes query data, which might require some time for a large design. You can set the variable to false to prevent the tool from initializing query data.

Examples

The following example prevents the tool from initializing query data.

```
prompt> set gui_build_query_data_table false
```

See Also

- [printvar](#)

gui_custom_setup_files

Variable for specifying GUI customization files to be loaded during GUI startup.

Data Types

Tcl list of fully-qualified file names

Default \$synopsys/admin/setup/.synopsys_<app>_gui.tcl

Description

This variable specifies a set of files that should be sourced when the GUI starts up. You can add this variable setting to the application setup file to provide GUI customizations to individuals, or share customizations among a group of users. A GUI customization file typically contains commands to specify hotkeys, menus, or toolbars which implement specific functions to support a custom environment or flow.

The GUI initializes and searches the list of files in order. Each file in the list is sourced. Finally, your `.synopsys_<app>_gui.tcl` file is loaded to complete the customizations.

You can disable the loading of the customizations by setting the `gui_disable_custom_setup` variable.

See Also

- [printvar](#)
- [gui_disable_custom_setup](#) Variable for disabling gui customizations

gui_default_window_type

,IP `gui_default_window_type` Read-only variable specifying the default window type for GUI customization commands.

Data Types

Boolean

Default `false`

Description

This read-only variable contains the name of the window type that is used as the default when a menu, hotkey, or toolbar customization is specified without specifying a window type explicitly.

See Also

- [gui_create_menu](#)
- [gui_create_toolbar](#)
- [gui_create_toolbar_item](#)
- [gui_delete_menu](#)
- [gui_delete_toolbar](#)
- [gui_delete_toolbar_item](#)
- [gui_report_hotkeys](#)
- [gui_set_hotkey](#)
- [printvar](#)

gui_disable_custom_setup

Variable for disabling gui customizations

Data Types

Boolean

Default false

Description

This variable specifies whether GUI customization loading is disabled during GUI startup. Set the variable to true to prevent GUI customizations from loading. This includes customizations specified in the *gui_custom_setup_files* variable as well as your .synopsys_<app>_gui.tcl file.

See Also

- [printvar](#)
- [gui_custom_setup_files](#) Variable for specifying GUI customization files to be loaded during GUI startup.

gui_online_browser

Specifies the name of the browser used to invoke the online help system from the help menu of the product.

Data Types

string

Default netscape

Group

gui

Description

This string variable holds the value of the default browser which is used to invoke online help system from Help menu of the application. The value the variable should be either *netscape*, *mozilla* or *firefox*.

If you specify a browser other than those listed above, the application defaults to the netscape browser.

Use the following command to determine the current value of the variable:

```
prompt> printvar gui_online_browser
```

See Also

- [gui_custom_setup_files](#) Variable for specifying GUI customization files to be loaded during GUI startup.

gui_selection_stack_depth

Specifies the maximum depth of the selection stack.

Data Types

Integer

Default 10

Group

gui

Description

When the user pushes a selection onto the selection stack the max depth is checked and any extra items, beyond the stack depth, are discarded.

This check is to enforce a limit of memory and resources needed to maintain the selection stack where each selection can contain a large number of object references.

If the value is set to a value less than 1 then the maximum depth is ignored so items are never discarded.

```
prompt> printvar gui_selection_stack_depth
```

See Also

- [gui_selection_stack](#)

gui_selection_undo

Enable/Disable single level undo for change_selection

Data Types

Boolean

Default true

Description

When this option is enabled, and the user changes the current selection, the previous selection is saved so the user can undo the change and restore the previous selection.

The user can use the command 'change_selection -undo' to undo the selection change.

See Also

- [change_selection](#)

gui_suppress_auto_layout

Variable for disabling automatic opening of the Layout window.

Data Types

Boolean

Default false

Description

Specifies whether the Layout window is prevented from opening when loading a new design.

See Also

- [printvar](#)

h

hdlin.analyze_prioritize_command_line_defines

Enables prioritization of command-line define macros, disabling their redefinition.

Data Types

boolean

Default TRUE

Description

The *hdlin.analyze_prioritize_command_line_defines* application option allows the user to prioritize command-line defines, disabling their redefinition from RTL code. If disabled,

higher-precedence defines, for example, the ones defined in the command line or -vcs files can be redefined. For example:

```
`define VALB 4
module test (output [3:0] b);
    assign b=`VALB;
endmodule

// Pass defines from analyze command
analyze -define "VALB=7" -format sverilog test.sv
elaborate test

write -format verilog -hier -output net.v

Netlist:
assign b[3] = 1'b0;
assign b[2] = 1'b1;
assign b[1] = 1'b1;
assign b[0] = 1'b1;
```

See Also

- [analyze](#)

hdlin.elaborate.black_box

Specify one or more modules names criterias, including wildcard matching, to have the corresponding module body ignored during design elaboration.

Data Types

string

Default ""

Description

The *hdlin.elaborate.black_box* application options allows the user to specify one or more modules names criterias, including wildcard matching, to have the corresponding body ignored during elaboration.

The modules names criterias are specified by setting the application option *hdlin.elaborate.black_box* to a list of modules, e.g.

```
fc_shell> set_app_options -name hdlin.elaborate.black_box -value "mod1
mod2 ..."
```

Here mod1, mod2 are names of modules as coded in the RTL.

The warning message (ELAB-747) is generated during the *elaborate*. An example of the message is shown below:

```
Warning: ./test.sv:521: The body of module 'mod1' is being discarded,  
because the module name meets hdlin.elaborate.black_box criteria.  
(ELAB-747)
```

The valid module names are those coded in the RTL and design names, e.g. those reported by *get_modules*, giving priority to the design names.

If a modules RTL name meets `hdlin.elaborate.black_box` criteria and its design name meets `hdlin.elaborate.black_box_all_except` criteria, its body will not be ignored.

If a modules design name meets `hdlin.elaborate.black_box` criteria and its RTL name meets `hdlin.elaborate.black_box_all_except` criteria, its body will be ignored.

No messages are generated for specified names that do not meet valid module names.

See Also

- [elaborate](#)
- [get_modules](#)

hdlin.elaborate.black_box_all_except

Specify one or more modules names criterias, including wildcard matching, to not ignore the body of during design elaboration.

Data Types

string

Default ""

Description

The *hdlin.elaborate.black_box_all_except* application option allows the user to specify one or more modules names criterias, including wildcard matching, to not ignore the corresponding body of during elaboration.

The modules names criterias are specified by setting the application option *hdlin.elaborate.black_box_all_except* to a list of modules, e.g.

```
fc_shell> set_app_options -name hdlin.elaborate.black_box_all_except  
-value "top mod1 ..."
```

Here top, mod1, mod2 are names of modules as coded in the RTL.

The warning message (ELAB-747) is generated during the *elaborate* command. An example of the message is shown below:

```
Warning: ./test.sv:521: The body of module 'mod2' is  
being discarded, because the module name doesn't meet  
hdlin.elaborate.black_box_all_except criteria. (ELAB-747)
```

The valid module names are those coded in the RTL and design names, e.g. those reported by *get_modules*, giving priority to the design names.

If a modules RTL name meets hdlin.elaborate.black_box criteria and its design name meets hdlin.elaborate.black_box_all_except criteria, its body will not be ignored.

If a modules design name meets hdlin.elaborate.black_box criteria and its RTL name meets hdlin.elaborate.black_box_all_except criteria, its body will be ignored.

No messages are generated for specified names that do not match valid module names.

See Also

- [elaborate](#)
- [get_modules](#)

hdlin.elaborate.enable_ff_sync_set_reset

Enables inferring of synchronous set/reset signals on registers for the listed modules

Data Types

string

Default ""

Group

hdl_variables

Description

This application option controls inference of synchronous set and reset signals on flip-flops on the listed modules. This application option is only applicable when hdlin.elaborate.ff_infer_sync_reset is set to false. This option has no impact on the inference of synchronous set/reset signals on the always blocks that have "sync_set_reset_local" or "sync_set_reset_local_all" pragmas.

hdlin.elaborate.enable_ff_sync_set_reset_all_except

Enables inferring of Synchronous Set/Reset signals on FlipFlops for all modules that are not listed in this app option

Data Types

string

Default ""

Group

hdl_variables

Description

This application option controls the inference of synchronous set and reset signals on the flip-flops. The inference of synchronous set and reset is disabled for modules listed in this application option, and enabled for all other modules. This application options is only applicable when hdlin.elaborate.fd_infer_sync_set_reset is set to false. This option has no impact on inferene of synchronous set/reset signals on the always blocks that have "sync_set_reset_local" or "sync_set_reset_local_all" pragams.

hdlin.elaborate.failsafe_fsm

Enable Failsafe Finite State Machine support

Data Types

boolean

Default TRUE

Description

When this application option is set true, Failsafe Finite State Machine will be enabled. Its default value is true.

hdlin.elaborate.ff_infer_async_set_reset

Controls whether the tool checks and reports asynchronous set and reset conditions of flip-flops.

Data Types

Boolean

Default true

Description

This application option, when set to *true* instructs the tool to check and report asynchronous set and reset conditions of flip-flops. Setting this application option to *false* disables this behavior.

Set this application option to *false* if you do not use asynchronous set or reset conditions, if you do not want asynchronous set or reset devices inferred, or if you want your design to be input faster.

The *async_set_reset* pragma can be used to overwrite this application option. See the *HDL Compiler for Verilog User Guide* or the *HDL Compiler for VHDL User Guide* for more details about the pragma.

To determine the current setting of this application option, use the *report_app_options hdlin.elaborate.ff_infer_async_set_reset* command. For a list of HDL application options and their current settings, use *report_app_options hdlin.**.

hdlin.elaborate.ff_infer_sync_set_reset

Controls whether every constant 0 loaded on a flip-flop under the clock event is used for synchronous reset, and every constant 1 loaded on a flip-flop under the clock event is used for synchronous set.

Data Types

Boolean

Default false

Description

This application option controls whether every constant 0 loaded on a flip-flop under the clock event is used for synchronous reset, and every constant 1 loaded on a flip-flop under the clock event is used for synchronous set. Setting the value to *true* for a design subsequently analyzed, results in this behavior.

For more details, see the *HDL Compiler for Verilog User Guide* or the *HDL Compiler for VHDL User Guide*.

To determine the current setting of this application option, use the *report_app_options hdlin.elaborate.ff_infer_sync_set_reset* command. For a list of HDL application options and their current settings, use *report_app_options hdlin.**.

hdlin.elaborate.infer_complex_set_reset

Controls whether the two commands *analyze* and *elaborate* attempt to infer complex set/reset logic for sequential cells.

Data Types

Boolean

Default false

Description

Controls whether the two commands *analyze* and *elaborate* attempt to infer complex set/reset logic for sequential cells.

By default Presto HDL Compiler will not attempt to infer set/reset logic that it estimates to be 'complex', for example, driven by some other instance or with many gates between the input and the sequential cell. And the complex set/reset logic will stay on the data line of the sequential cell.

For quality of result it's usually desirable for the set/reset logic to be fast and simple, but when this is not so, setting the variable to true tells Presto HDL Compiler to consider some additional, more complex possibilities, although potentially at the cost of increased runtime and memory usage.

The variable works for both Verilog and VHDL input files.

See Also

- [analyze](#)
- [elaborate](#)

hdlin.elaborate.infer_local_sync_enable_only

Controls which controls are fed into synch_enable pins and which are fed into next_state pins.

Data Types

Boolean

Default false

Description

When the value of this app_option is *true* and a particular coding style has been used, HDL Compiler produces a netlist in which the control signals local to sequential blocks feed into synch_enable pins, while external ones feed into next_state pins.

hdlin.elaborate.infer_mux_size_only

Controls the behavior of the *infer_mux* pragma in the Fusion Compiler tool.

Data Types

integer

Default 0

Description

To ensure that SELECT_OP WVGTECH cells are mapped to MUX technology cells, you must apply a *map_to_mux* attribute to the cells to prevent logic decomposition in later optimization steps. Beginning with the P-2019.03-SP5 release, you can control which SELECT_OP cells receive the *map_to_mux* attribute by using the *hdlin.elaborate.infer_mux_size_only* application option. The following options are valid for *hdlin.elaborate.infer_mux_size_only*:

0

Specifies that the *infer_mux* pragma is a no-op in the Fusion Compiler tool.

1

Specifies that all SELECT_OP cells with MUX functionality directly inferred by RTL with an *infer_mux* pragma and that are on set and reset signals receive the *map_to_mux* attribute ensuring mapping to a MUX technology cell.

2

Specifies that all SELECT_OP cells with MUX functionality directly inferred by RTL with an *infer_mux* pragma receive the *map_to_mux* attribute ensuring mapping to a MUX technology cell.

3

Specifies that all SELECT_OP cells with MUX functionality directly or indirectly inferred and that are on set and reset signals receive the *map_to_mux* attribute ensuring mapping to a MUX technology cell.

4

Specifies that all SELECT_OP cells with MUX functionality directly or indirectly inferred receive the *map_to_mux* attribute ensuring mapping to a MUX technology cell.

To determine the current setting of this application option, use the *report_app_options hdlin.elaborate.infer_mux_size_only* command. For a list of HDL application options and their current settings, use *report_app_options hdlin.**.

Note that all SELECT_OP cells inferred using the *infer_mux_override* directive receive the *map_to_mux* attribute.

See Also

- [set_size_only](#)

hdlin.elaborate.inout_preserve_rtl_assignment_direction

Controls if the registers and instances in a design are named with their `inout_rtl_assignment_direction` path attached or not. Support for keeping the driving net when: - out port is assigned to an out port - inout port is assigned to an inout port

Data Types

Boolean

Default false

Description

This application option controls if registers and instances are named in the `inout_rtl_assignment_direction` way or not. For example,

```
4 module top (input a,b,c,output x,y,w);
5   and an (y,a,b);
6   assign x = y;
7   buf buff (w,y);
8 endmodule
```

If `hdlin.elaborate.inout_preserve_rtl_assignment_direction` is set to *true*: - the verilog netlist generated by line 6 will be 'assign x = y ;' - the verilog netlist generated by line 7 will be 'assign w = y ;'

To determine the current setting of this application option, use the `report_app_options hdlin.elaborate.inout_preserve_rtl_assignment_direction` command. For a list of HDL application options and their current settings, use `report_app_options hdlin.*`.

hdlin.elaborate.keep_signal_name

Determines whether HDL Compiler attempts to keep a signal name.

Data Types

string

Default all_driving

Description

The `hdlin.elaborate.keep_signal_name` application option is available only in HDL Compiler Verilog and HDL Compiler VHDL. For this discussion, these tools will

be referred to as Presto. The original HDL and VHDL compilers do not support *hdlin.elaborate.keep_signal_name*.

The value of *hdlin.elaborate.keep_signal_name* determines whether the tool attempts to keep a signal name. This includes preserving cells between the signal and the input or output port. The *hdlin.elaborate.keep_signal_name* application option provides Presto with guideline information for keeping a signal name, but it is not intended to be a 100% guarantee for keeping a signal. In some Presto optimizations, such as removing redundant code, the signal name may be optimized away, even if the signal is associated with the *keep_signal_name* attribute or directive. The allowed values are *all*, *none*, *user*, *user_driving*, and *all_driving*, as described below. The *user* and *user_driving* values require the *keep_signal_name* directive.

none

Presto does not attempt to keep any signals. This value overrides the *keep_signal_name* directive.

all

Presto attempts to preserve a signal if the signal is not removed by optimizations. Dangling and driving nets are considered. This value does not guarantee that a signal is kept. The *all* value generally preserves fewer signals than the *user* value. For example, if there are 3 sequential nets named *N1*, *N2*, and *N3*, the *all* value may keep only *N1*. Setting *hdlin.elaborate.keep_signal_name* to the value of *user* and setting the *keep_signal_name* directive on *N1*, *N2*, and *N3* generally keeps *N1*, *N2*, *N3*.

This value may cause the *check_design* command to issue LINT-2 and LINT-3 warnings:

```
Warning: In design '...', net '...' has no drivers.  
Logic 0 assumed. (LINT-3)
```

```
Warning: In design '...', net '...' driven by pin '  
(no pin) ' has no loads. (LINT-2)
```

all_driving

Presto attempts to preserve a signal if the signal is not removed by optimizations and the signal is in an output path. Only driving nets are considered. This value does not guarantee that a signal is kept. For signals that are not in an input path (do not have drivers), Presto connects the input to ground, assuming logic 0 (ground) for the driver.

user

This value works with the *keep_signal_name* directive. Presto attempts to preserve a signal if the signal is not removed by optimizations, and that signal is labeled with the *keep_signal_name* directive. Dangling and driving nets are

considered. Although not guaranteed, Presto usually keeps the specified signal for this configuration.

user_driving

This value works with the *keep_signal_name* directive. Presto attempts to preserve a signal if the signal is not removed by optimizations, the signal is in an output path, and the signal is labeled with the *keep_signal_name* directive. Only driving nets are considered. Although not guaranteed, Presto usually keeps the specified signal for this configuration. For signals that are not in an input path (do not have drivers), Presto connects the input to ground, assuming logic 0 (ground) for the driver.

Examples

In Example 1, you use the *hdlin.elaborate.keep_signal_name* application option and the *keep_signal_name* directive to control what signals Presto attempts to preserve.

Example 1 - Verilog

```
module test (a, b, T, A, c);
input  [3:0]  a;
input  [7:0]  b;
input T;
input A;
output reg [7:0] c;
wire d, e;
//synopsys async_set_reset "A"
//synopsys keep_signal_name "e"

assign e = ( a[3] & ~a[2] & a[1] & ~a[0] );
assign d = ( a[3] & ~a[2] & a[1] & ~a[0] );

always @(T or b or A or d)
if (A)
    c = 8'h0;
    else if (T & d)
        c = b;
endmodule
```

The internal wire *e* is not kept if the line *//synopsys keep_signal_name "e"* is removed, and *hdlin.elaborate.keep_signal_name* is either *none*, *user*, *user_driving*, or *all_driving*. However, *e* is kept if *hdlin.elaborate.keep_signal_name* is *all*. Also, *e* is kept if *hdlin.elaborate.keep_signal_name* is *user* and *//synopsys keep_signal_name* is attached to *"e"*.

In Example 2, by default Presto attempts to preserve the *test1* and *test2* signals because they are in output paths. Note that the *test2* signal is not in an input path; Presto does not try to keep the *test3* signal because it is not in an output path. Presto optimizes away the *syn1* and *syn2* signals.

Example 2 - Verilog

```
module test12 (in1, in2, in3, in4, out1,out2 );
    input  [3:0]   in1;
    input  [7:0]   in2;
    input in3;
    input in4;
    output reg  [7:0] out1,out2;

    wire test1,test2, test3, syn1,syn2;

    //synopsys async_set_reset "in4"

    assign test1 = ( in1[3] & ~in1[2] & in1[1] & ~in1[0] );
    //test1 signal is in an input and output path
    assign test2 = syn1+syn2;
    //test2 signal is in a output path, but it is not in an input path
    assign test3 = in1 + in2;
    //test3 signal is in an input path, but it is not in an output path
    always @(in3 or in2 or in4 or test1)
    out2 = test2+out1;
    always @(in3 or in2 or in4 or test1)
        if (in4)
            out1 = 8'h0;
        else
            if (in3 & test1)
                out1 = in2;
endmodule
```

One way to keep the test3 signal is to set *hdlin.elaborate.keep_signal_name* to user and place the *keep_signal_name* directive on test3, as shown in Example 3.

Example 3 - Verilog

```
module test12 (in1, in2, in3, in4, out1,out2 );
    input  [3:0]   in1;
    input  [7:0]   in2;
    input in3;
    input in4;
    output reg  [7:0] out1,out2;

    wire test1,test2, test3, syn1,syn2;
    //synopsys keep_signal_name "test3"

    //synopsys async_set_reset "in4"

    assign test1 = ( in1[3] & ~in1[2] & in1[1] & ~in1[0] );
    //test1 signal is in an input and output path
    assign test2 = syn1+syn2;
    //test2 signal is in a output path, but it is not in an input path
    assign test3 = in1 + in2;
    //test3 signal is in an input path, but it is not in an output path
    always @(in3 or in2 or in4 or test1)
```

```
out2 = test2+out1;
always @(in3 or in2 or in4 or test1)
  if (in4)
    out1 = 8'h0;
  else
    if (in3 & test1)
      out1 = in2;
endmodule
```

Table 1 shows how the application option settings affect the preservation of signal test3, with and without the directive applied to it. Asterisks (*) indicate that Presto does not attempt to keep the signal and might remove it.

Table 1 Application Options and Directive Matrix for Signal test3

hdlin.elaborate.keep_signal_name =		all	none	user	user_driving
all_driving					

keep_signal_name is not set on test3	attempts to keep	*	*	*	*
keep_signal_name is set on test3	attempts to keep	*	attempts to keep	*	*

Table 2 shows how the application option settings affect the preservation of signal test2, with and without the directive applied to it. Asterisks (*) indicate that Presto does not attempt to keep the signal and might remove it.

Table 2 Application Options and Directive Matrix for Signal test2

hdlin.elaborate.keep_signal_name =		all	none	user	user_driving
all_driving					

keep_signal_name is attempts not set on test2	attempts to keep	*	*	*	to keep
keep_signal_name is attempts set on test2	attempts to keep	*	attempts to keep	attempts to keep	attempts to keep

Table 3 shows how the application option settings affect the preservation of signal test1, with and without the directive applied to it. Asterisks (*) indicate that Presto does not attempt to keep the signal and might remove it.

Table 3 Application Options and Directive Matrix for Signal test1

hdlin.elaborate.keep_signal_name =		all	none	user	user_driving
all_driving					

keep_signal_name is	attempts	*	*	*	
attempts					
not set on test1	to keep				to keep
keep_signal_name is	attempts	*	attempts	attempts	
attempts					
set on test1	to keep		to keep	to keep	to keep

To determine the current setting of this application option, use the following command:

```
prompt> report_app_options hdlin.elaborate.keep_signal_name
```

Use the following command to get a list of HDL application options and their current values in DB (dcsh) mode :

```
prompt> report_app_options hdlin.*
```

Use the following command to get a list of HDL application option and their current values in XG and DB (Tcl) mode:

```
prompt> report_app_options
```

hdlin.elaborate.latch_infer_async_set_reset

Uses every constant 0 loaded on a latch for asynchronous reset and uses every constant 1 loaded on a latch for asynchronous set for a design subsequently analyzed.

Data Types

Boolean

Default false

Description

This application option, when set to *true*, instructs the tool to use every constant 0 loaded on a latch for asynchronous reset, and to use every constant 1 loaded on a latch for asynchronous set, for a design subsequently analyzed.

The default value is *false*.

For more details, see the *HDL Compiler for Verilog User Guide* or the *HDL Compiler for VHDL User Guide*.

To determine the current setting of this application option, use the *report_app_options hdlin.elaborate.latch_infer_async_set_reset* command. For a list of HDL application options and their current settings, use *report_app_options hdlin.**.

hdlin.elaborate.max_port_width_allowed

Setting the value of maximum port width allowed.

Data Types

Integer

Default 1048576

Description

The *hdlin.elaborate.max_port_width_allowed* This application option changes the threshold value for allowed maximum ports size; it has no effect on functionality of the design Setting large numbers may result in longer runtimes and more memory use. Bypassing the error ELAB-1012 is possible with this option; for further information, see the error's man page.

hdlin.elaborate.maximum_while_loop_iterations

Places an upper bound on the number of times a loop is unrolled to prevent potential infinite loops.

Data Types

string

Default 4096

Description

This application option places an upper bound on the number of times a loop is unrolled to prevent potential infinite loops. Loop unrolling occurs until the loop terminates. If you know that your loop will execute more times than the limit allows and that your loop will terminate at some point, increase the value of this application option.

The default of this application option is 1024.

To determine the current setting of this application option, use the *report_app_options hdlin.elaborate.maximum_while_loop_iterations* command. For a list of HDL application options and their current settings, use the *report_app_options hdlin.** command.

hdlin.elaborate.netlist_unloaded_signals

Controls whether Presto optimizes away the unloaded signals before netlisting.

Data Types

Boolean

Default false

Description

This variable controls whether Presto optimizes away unloaded signals before netlisting.

When set to *true*, Presto netlists unloaded signals. The purpose is for Presto to report pruned registers accurately when these redundant netlist is removed later.

When set to *false* (the default), Presto optimizes away unloaded signals before netlisting. So Presto will not create the redundant logic first and then remove. The early optimization may cause missing report pruned registers.

hdlin.elaborate.optimize_buffers

Determines whether Presto attempts to optimize buffers.

Data Types

boolean

Default true

Description

The value of *hdlin.elaborate.optimize_buffers* determines whether Presto attempts to optimize buffer.

If the buffer is not optimized, UPF flow will insert an isolation cell to address any power domain consistency issues between the hierarchical boundaries, however a simple wire does not trigger the insertion of isolation cell. Formality, will not immediately optimize the logic to a buffer, so FM too will insert an ISO cell.

Examples

```
module test (input A, output logic Y);  
  reg Y ;  
  always @(*) begin  
    Y = (A==1'b1) ? 1'b1 : 1'b0 ;  
  end  
endmodule
```

When the value of this switch is false, buffer will be preserved, such as.

```
module test ( A , Y ) ;
input  A ;
output Y ;

WVGTECH_BUF B_0 ( .I ( A ) , .Z ( Y ) ) ;
endmodule
```

When the value of this switch is true, buffer will be optimized, such as.

```
module test ( A , Y ) ;
input  A ;
output Y ;
assign Y = A ;
endmodule
```

hdlin.elaborate.preserve_sequential

Controls whether the *elaborate* and *read* commands retain unloaded sequential cells in the design.

Data Types

string

Default none

Description

This application option instructs the *elaborate* command or the *read* command to retain unloaded sequential cells in the design. The default is *none*. The following values are allowed:

none or false

 No unloaded sequential cells are preserved.

all or true

 All unloaded sequential cells are preserved, excluding unloaded sequential cells that are used solely as loop variables.

all+loop_variables or true+loop_variables

 All unloaded sequential cells are preserved, including unloaded sequential cells that are used solely as loop variables.

ff

 Only flip-flop cells are preserved, excluding unloaded sequential cells that are used solely as loop variables.

`ff+loop_variables`

Only flip-flop cells are preserved, including unloaded sequential cells that are used solely as loop variables.

`latch`

Only unloaded latch cells are preserved, excluding unloaded sequential cells that are used solely as loop variables.

`latch+loop_variables`

Only unloaded latch cells are preserved, including unloaded sequential cells that are used solely as loop variables.

When elaborating an RTL-level design description, HDL Compiler (Presto) infers a sequential cell (a latch or flip-flop) for any Verilog *reg* or VHDL *variable* objects that are conditionally assigned or are assigned under a clock edge. However, if the design never reads the values of these objects or never uses the values to compute any output of the design, the HDL Compiler (Presto) tool, by default, does not retain sequential cells for these objects. These sequential cells have no loads or there is no path from them to the outputs, and so the tool does not need them to implement the design.

You might want to preserve unnecessary sequential cells, however, to improve observability of the design. The `hdlin.elaborate.preserve_sequential` application option controls, which unloaded sequential cells remain in the design.

The `+loop_variables` option affects which sequential cells are inferred when the value of `hdlin.elaborate.preserve_sequential` is not set as *false* or *none*, and the `+loop_variables` option is not specified, the tool treats index variables as special and no sequential cells are inferred.

If you do not want variables treated in this special way, specify `+loop_variables` for the variable values.

In the following example module, the bits of *fail* would ordinarily not exist as sequential cells after elaboration of the design. However, note the following:

- If you set the value of `hdlin.elaborate.preserve_sequential` as *all*, *true*, or *ff*, the tool infers sequential cells for *fail*, and since the `+loop_variables` option is not specified, the tool treats *i* in the example as special, because *i* is used only as a loop variable. Therefore, the tool does not infer sequential cells for *i*.
- If you set the value of `hdlin.elaborate.preserve_sequential` as *all+loop_variables*, *true+loop_variables*, or *ff+loop_variables*, the tool infers sequential cells for *fail*, and since the `+loop_variables` option is specified, the tool treats *i* the same way as it treats all other variables and assigns *i* to sequential cells.
- If you set the value of `hdlin.elaborate.preserve_sequential` as *none* (the default), *false*, or *latch*, the tool does not assign either *fail* or *i* to sequential cells in the design.

Example 1:

```
module test (clk,rst,error);
  parameter N = 8;
  input clk,rst,error;
  reg [N-1:0] fail;
  integer i, j;

  always @(posedge clk or posedge rst)
    if (rst)
      fail <= {N-1{1'b0}};
    else
      for ( i = 0; i < N; i = i + 1 )
        fail[i] <= fail[i] | error;
endmodule // test
```

Table 1 shows how the application option settings affect the sequential cell inferring for *fail* and *i* in Example 1.

Asterisks (*) indicate that Presto infers the sequential cells.

Table 1 Variables and Sequential Cell Inferring Matrix for module test

-----+-----+-----+-----+-----				
hdlin.elaborate.preserve_sequential				all/true/ff
none/false/latch				
-----+-----+-----+-----+-----				
option "+loop_variables" is specified		true		false true
false				
-----+-----+-----+-----+-----				
reg [7:0] fail		*		*
reg [31:0] i		*		
-----+-----+-----+-----+-----				

Important: The process of elaboration might preserve unloaded sequential cells, but the cells are deleted by the *compile_fusion* command (by default). To preserve the cells throughout the compile process, set the value of the *compile.seqmap.remove_unloaded_registers* application option as *false*.

If there are other variables in your design that cause sequential elements to be inferred, but that you do not want in the final design, specify the preservation of sequential cells on a variable-by-variable basis, using the Verilog *preserve_sequential* pragma in a declaration.

For more information on the *preserve_sequential* pragma, see the *HDL Compiler for Verilog User Guide* or the *HDL Compiler for VHDL User Guide*.

To determine the current setting of this application option, use the *report_app_options hdlin.elaborate.preserve_sequential* command. For a list of HDL application options and their current settings, use *report_app_options hdlin.**.

See Also

- [elaborate](#)

hdlin.elaborate.svconfig_liblist_verbose_mode

Queries information of Verilog and SystemVerilog configuration about the HDL libraries when elaborating instances.

Data Types

integer

Default 0

Group

hdl_variables

Description

The *hdlin.elaborate.svconfig_liblist_verbose_mode* app option lets you query information from Verilog or SystemVerilog configuration file about the HDL libraries that are used when elaborating an instance. To get the information about the HDL libraries used to elaborate an instance with an *instance* clause specified with a *liblist* expansion clause, set this variable to 1. To also get information about HDL libraries used to elaborate an instance not specified with an *instance* clause specified with a *liblist* expansion clause, set the variable to 2. The default is 0.

The informational messages can be found in the log by looking for the messages codes (ELAB-962) and (ELAB-963). For example,

```
Information: Using 'BUF INV MULT' HDL liblist for 'mod_1' instantiated  
from 'top'. (ELAB-962)  
Information: Using 'BUF INV MULT' HDL liblist from 'top' for instantiated  
'mod_2'. (ELAB-963)
```

See Also

- [elaborate](#)

hdlin.flow.ieee_1735_decryption

Data Types

Boolean

Default true

Description

When this application option is set to *true*, the *analyze* command supports decryption of Verilog and SystemVerilog encrypted envelopes in the style of IEEE Std 1735-2014. IEEE encryption std 1735-2014 for VHDL is enabled by default and cannot be controlled by this setting.

See Also

- [analyze](#)

hdlin.hdl_library.default_dirname

Specifies the HDL Compiler template library location.

Data Types

String

Default ./HDL_LIBRARIES

Description

Specifies the default directory for HDL Compiler template libraries.

Examples

For example:

```
analyze -hdl_library top_lib -format verilog top.v
elaborate -hdl_library top_lib top
set_top_module top
```

"top_lib" is a template library.

If "hdlin.hdl_library.default_dirname" is not reset from its default value, "top_lib" is mapped to "./HDL_LIBRARIES/TOP_LIB".

If "hdlin.hdl_library.default_dirname" is reset, for example, by:

```
set_app_options -name hdlin.hdl_library.default_dirname -value /u/tmp
```

then "top_lib" is mapped to "/u/tmp/TOP_LIB".

See Also

- [analyze](#)
- [define_hdl_library](#)
- [elaborate](#)
- [report_hdl_libraries](#)
- [set_top_module](#)

hdlin.hdl_library.default_name

Specifies the HDL Compiler template library name.

Data Types

String

Default "WORK"

Description

Specifies the default name for HDL Compiler template library.

Examples

For example:

```
analyze -hdl_library top_lib -format verilog top.v
elaborate -hdl_library top_lib top
set_top_module top
```

"top_lib" is a template library. If option "-hdl_library" is not specified, template library will be the value of hdlin.hdl_library.default_name.

```
set_app_options -name hdlin.hdl_library.default_name -value myLib
```

then "myLib" is the template library.

See Also

- [analyze](#)
- [define_hdl_library](#)
- [elaborate](#)

hdlin.hdl_library.intermediate_file_method

Specifies how HDL Compiler stores information about analyzed RTL files.

Data Types

string

Default legacy

Description

This application option controls how the *analyze* command stores the data from analyzed RTL files, which is subsequently used by the *elaborate* command.

The *hdlin.hdl_library.intermediate_file_method* application option can be set as follows:

- *legacy* - The data is stored as multiple files (named after module, interfaces or packages names) in the directory defined by the *define_hdl_library* command.
- *compact* - The data is stored in a single file (named "default.hdllib") in the directory defined by the *define_design_lib* command.
- *none* - The data is stored in memory instead of on disk.

For the values of *legacy* and *compact*, the data remains persistent and available for use by elaboration in subsequent sessions.

For the value of *none*, all RTL analysis and elaboration must be performed in a single session. However, runtime can improve significantly (versus storing files on disk) if the filesystem performance is poor.

This application option applies only to Verilog and SystemVerilog RTL analysis; it is not yet implemented for VHDL RTL analysis.

See Also

- [analyze](#)
- [elaborate](#)
- [define_hdl_library](#)

hdlin.hdl_library.setup

User specifies the setup file containing library mapping

Data Types

string

Default ""

Group

hdl_variables

Description

User specifies the setup file containing library mapping

hdlin.logfile_format

Configures the format of the log file.

Data Types

string

Default 202409

Description

Configures the log file to a particular format. Each format is represented by the tool version that introduced substantial changes to the reports and information messages. Valid values for *hdlin.logfile_format* are *202409* and *pre202409*.

hdlin.naming.cell_prefix

Sets the internal cell name prefix for Presto HDL Compiler.

Data Types

string

Default C

Description

This application option sets the internal cell name prefix for HDL Compiler. Use the *hdlin.naming.cell_prefix* application option to specify the cell name prefix. If you do not specify the application option, HDL Compiler generates the cell name with a "C" prefix.

The cell name prefix is only applied to cells whose names are not determined by another HDL Compiler cell naming convention.

The default value for the *hdlin.naming.cell_prefix* application option is *C*.

To determine the current setting of this application option, use the *report_app_options hdelin_prest_cell_name_prefix* command. For a list of HDL application options and their current settings, use *report_app_options hdlin.**.

hdlin.naming.hierarchical

Controls if the registers and instances in a design are named with their hierarchical path attached or not.

Data Types

Boolean

Default false

Description

This application option controls if registers and instances are named in the hierarchical way or not. For example,

```
module test (clk, rst, datain, qout);
    input clk, rst;
    input [1:0] datain;
    output reg [1:0] qout;
    genvar i;
    generate
        for (i=0; i < 2; i = i + 1 ) begin : mygenblk
            always @(posedge clk or negedge rst)
                begin : myblk
                    reg myreg;
                    if( ! rst ) begin
                        qout[i] <= '0;
                        myreg <= '0;
                    end else begin
                        qout[i] <= myreg;
                        myreg <= datain[i];
                    end
                end
            end
        endgenerate
    endmodule
```

If *hdlin.naming.hierarchical* is set to *true*, the registers inferred from the code on line 6 will be 'mygenblk[0].myblk.myreg_reg' and 'mygenblk[1].myblk.myreg_reg'.

To determine the current setting of this application option, use the *report_app_options hdlin.naming.hierarchical* command. For a list of HDL application options and their current settings, use *report_app_options hdlin.**.

hdlin.naming.module_name_limit

Specifies the length threshold for compressing elaborated module names when the *hdlin.naming.shorten_long_module_name* application option is set to *true*.

Data Types

integer

Default 2048

Description

This application option specifies the length threshold for whether the Presto HDL Compiler compresses names for elaborated modules when the *hdlin.naming.shorten_long_module_name* application option is set to *true*.

When *hdlin.naming.shorten_long_module_name* is *true*, if the name of an elaborated module is longer than the value of *hdlin.naming.module_name_limit* (default 2048) and at least one parameter has no default value, then a compressed name is used that is easier for downstream tools to handle. The initial part of the name is the module name, but the tail of the name regarding the parameters names and values is replaced with numbers, such as from "submod_SIZE256_WIDTH248" to "submod_h_249".

To determine the current setting of this application option, use the *report_app_options hdlin.naming.module_name_limit* command. For a list of HDL application options and their current settings, use *report_app_options hdlin.**.

hdlin.naming.net_prefix

Sets the internal net name prefix for Presto HDL Compiler.

Data Types

string

Default N

Description

This application option sets the internal net name prefix. The net name generated by Synopsys DB always has the prefix "n". The net name generated by Presto has the prefix "N" by default. Use this application option to set the Presto net name prefix to avoid inconsistent net names. The default value for the *hdlin.naming.net_prefix* application option is *N*.

To determine the current setting of this application option, use the *report_app_options hdlin.naming.net_prefix* command. For a list of HDL application options and their current settings, use *report_app_options hdlin.**.

hdlin.naming.register_suffix_on_field

Enables suffix ("_reg"/"_tri"/"_tri_enable_reg") appending on the field of structured cells.

Data Types

boolean

Default FALSE

Description

The *hdlin.naming.register_suffix_on_field* application option allows the user to add the register suffix on the basename of the structure or as the last field.

Some examples of the change using this application option are:

FALSE	TRUE
out1_reg	out1_reg
out2_reg[1]	out2[1]_reg
out3_tri[0][somefield]	out3[0][somefield]_tri
out3_tri_enable_reg[0][somefield][1]	
out3[0][somefield]_tri_enable_reg[1]	

See Also

- [elaborate](#)

hdlin.naming.shorten_long_module_name

Controls whether the Presto HDL Compiler compresses long names for elaborated modules.

Data Types

Boolean

Default true

Description

This application option controls whether the Presto HDL Compiler compresses long names for elaborated modules.

When the value of this application option is *true* (the default), if the name of an elaborated module is longer than the value of *hdlin.naming.module_name_limit* (default 2048) and at least one parameter has no default value, then a compressed name is used that is easier for downstream tools to handle. The initial part of the name is the module name, but the tail of the name regarding the parameters names and values is replaced with numbers, such as from "submod_SIZE256_WIDTH248" to "submod_h_249".

When the value is *false*, the names of elaborated modules are uncompressed, even if they exceed the length specified by *hdlin.naming.module_name_limit*.

To determine the current setting of this application option, use the *report_app_options hdlin.naming.module_name_limit* command. For a list of HDL application options and their current settings, use *report_app_options hdlin.**.

hdlin.naming.shorten_top_module_name

Controls whether the Presto HDL Compiler compresses long names for elaborated top level modules.

Data Types

Boolean

Default false

Description

This application option controls whether the Presto HDL Compiler compresses long names for elaborated top level modules.

When the value of this application option, as well as *hdlin.naming.shorten_long_module_names*, is *true*, if the name of an elaborated top level module is longer than the value of *hdlin.naming.module_name_limit* (default 256), then a compressed name is used that is easier for downstream tools to handle. The initial part of the name is still recognizable, but the tail of the name is replaced with numbers.

When the value is *false* (the default), the names of elaborated modules are uncompressed, even if they exceed the length specified by *hdlin.naming.module_name_limit*.

To determine the current setting of this application option, use the *report_app_options hdlin.naming.module_name_limit* command. For a list of HDL application options and their current settings, use *report_app_options hdlin.**.

hdlin.naming.strict_template_naming_style

Force user defined template_naming/parameter_styles for numeric parameters.

Data Types

Boolean

Default false

Description

Force user defined template_naming/parameter_styles for numeric parameters. This is particularly useful for integer parameters expressed in binary, octal or hex formats.

Examples

Following example is valid if:

```

hdlin.naming.template_naming_style = "%s_%p" (the default value)
module shift_reg #(parameter WIDTH = 8, DEPTH = 8, NONINT = "abc")
(
    input clk, rst, en,
    input [WIDTH-1:0] shift_in,
    output [WIDTH-1:0] shift_out
);
endmodule

# Module instantiation -> output module name
shift_reg #(.WIDTH(7), .DEPTH(8)) ex1
    -> shift_reg_WIDTH7_DEPTH8
shift_reg #(.DEPTH(4'b0100), .NONINT("cde")) ex2
    -> shift_reg_DEPTH4_NONINTcde
shift_reg #(.WIDTH(4'h0001), .DEPTH(9)) ex3
    -> shift_reg_WIDTH1_DEPTH9
shift_reg #(.WIDTH(4'o0001), .DEPTH(10), .NONINT("hij")) ex4
    -> shift_reg_WIDTH1_DEPTH10_NONINThij

```

hdlin.naming.structs_records

Defines the parts of the net names that the tool generates corresponding to the fields in VHDL records or in SystemVerilog structs.

Data Types

string

Default ""

Description

This application option defines the parts of the net names that the tool generates corresponding to the fields in VHDL records and SystemVerilog structs. By default, the *hdlin.naming.structs_records* is derived from the *bus_naming_style* and *bus_dimension_separator_style*. But when these are "%s[%d]" and "]" or are "%s<%d>" and "><", then it is possible to specify an independent *hdlin.naming.structs_records*, such as "%s<%s>", "%s[%s]", or "%s.%s", in which the first %s stands for the name up to the field and the second %s stands for the field. The *hdlin.naming.structs_records* must be of the form "%sX%s" or "%sX%sY", where X and Y are (possibly identical) non-whitespace characters.

hdlin.naming.sverilog_union_member

Controls the naming styles for elements associated with the union data type in SystemVerilog.

Data Types

Boolean

Default false

Description

By default, the tool uses simple names for elements inferred from unions in SystemVerilog. Setting this application option to *true* enables the tool to use the name of the first union member as a reference for the port, net, and cell names associated with the union data type. For example,

```
1 typedef union packed {
2     byte      field1;
3     logic [7:0] field2;
4 } packet;
5
6 module test (input packet p1, output packet p2, input clk);
7     always_ff @ (posedge clk)
8         p2.field1 = p1.field2;
9 endmodule
```

When you set this application option to *false*, the following names are inferred:

Ports and nets: p1[7], p1[6], ..., p1[0]
Cells: p2_reg[7], p2_reg[6], ..., p2_reg[0]

When you set this application option to *true*, the following names are inferred:

Ports and nets: p1[field1][7], p1[field1][6], ..., p1[field1][0]
Cells: p2_reg[field1][7], p2_reg[field1][6], ..., p2_reg[field1][0]

When you use the *read_saif* command to read in a SAIF file generated from a FSDB, setting this application option to *true* might increase the acceptance of the annotated switching activity.

hdlin.naming.template_naming_style

Generates automatically a unique name when a module is built.

Data Types

string

Default %s_%p

Description

This application option automatically generates a unique name when a module is built. This application option is one of three string application options that determine the naming conventions for parameterized modules (templates) built into a design through the *elaborate* command or automatically instantiated from an HDL file. The unique name automatically generated uses the module name, parameter names, and parameter values.

The *hdlin.naming.template_naming_style* application option determines what character or characters appear between the design name and the parameter name. The string value must contain %s, which represents the name of the original design, and %p, which represents the name and value of the parameter or parameters. You can optionally include any ASCII character or characters, or none, between %s and %p. For example, for a design named *DesignName* that has a parameter *parm1*, the default %s_%p causes the name *DesignName_parm1* to be generated.

Further, %s\$p, %s_*_p, and %s%p would generate respectively the names *DesignName\$parm1*, *DesignName_*_parm1*, and *DesignNameparm1*.

If a design has a noninteger parameter (or if *hdlin.naming.template_naming_style* = ""), the following definitions are locked down for these application options:

```
hdlin.naming.template_naming_style = %s_%p
hdlin.naming.template_parameter_style = %d
hdlin.naming.template_separator_style = _
```

See Also

- [elaborate](#)

hdlin.naming.template_parameter_style

Generates automatically a unique name when a module is built.

Data Types

string

Default %s%d

Description

This application option automatically generates a unique name when a module is built. This application option is one of three string application options that determine the naming conventions for parameterized modules (templates) built into a design through the *elaborate* command or automatically instantiated from an HDL file. The unique name automatically generated uses the module name, parameter names, and parameter values.

The *hdlin.naming.template_parameter_style* application option determines what character or characters appear between the parameter name and its value.

The string must contain %s, which represents the name of the parameter, and %d, which represents the value of the parameter. You can optionally include any ASCII character or characters, or none, between %s and %d. For example, for a parameter named *parm* that has a value of 1, the default %s%d causes the name *parm1* to be generated.

The following are additional examples:

```
hdlin.naming.template_naming_style = "%s$p"
hdlin.naming.template_parameter_style = %s_%d    results
in DesignName$parm_1

hdlin.naming.template_naming_style = "%s_%p"
hdlin.naming.template_parameter_style = %s@%d    results
in DesignName_parm@1
```

If a design has a noninteger parameter (or if *hdlin.naming.template_naming_style* = ""), the following definitions are locked down for these application options:

```
hdlin.naming.template_naming_style = %s_%p
hdlin.naming.template_parameter_style = %d
hdlin.naming.template_separator_style = _
```

See Also

- [elaborate](#)

hdlin.naming.template_separator_style

Generates automatically a unique name when a module is built.

Data Types

string

Default _

Description

This application option automatically generates a unique name when a module is built. This application option is one of three string application options that determine the naming conventions for parameterized modules (templates) built into a design through the *elaborate* command or automatically instantiated from an HDL file. The unique name automatically generated uses the module name, parameter names, and parameter values.

The *hdlin.naming.template_separator_style* application option determines what character or characters appear between parameter names for templates that have more than one

parameter. You can designate any ASCII character or characters, or none. The default value is an underscore (`_`).

For example, for a design called *DesignName* that has parameters named *parm1*, *parm2*, and *parm3*, if `hdlin.naming.template_naming_style = "%s_%p"` (the default value), and `hdlin.naming.template_separator_style = "_"`, the name *DesignName_parm1_parm2_parm3* is generated.

The following are additional examples:

```
hdlin.naming.template_naming_style = "%s$p"
hdlin.naming.template_separator_style = "_"      results
in DesignName$parm1_parm2_parm3
hdlin.naming.template_naming_style = "%s#p"
hdlin.naming.template_separator_style = "/"      results
in DesignName#parm1/parm2/parm3
```

If a design has a noninteger parameter (or if `hdlin.naming.template_naming_style = ""`), the following definitions are locked down for these application options:

```
hdlin.naming.template_naming_style = %s_%p
hdlin.naming.template_parameter_style = %d
hdlin.naming.template_separator_style = _
```

See Also

- [elaborate](#)

hdlin.naming.upf_compatible

Controls HDL naming style settings to make it easier to apply the same UPF file across multiple tools at the RTL level.

Data Types

Boolean

Default false

Description

This application option controls HDL naming style settings to make it easier to apply the same UPF file across multiple tools at the RTL level.

Setting this application option to *true* has the following effects:

- Sets the *hdlin.naming.hierarchical* application option to *true*
- Sets the *hdlin.naming.structs_records* application option to *"%s.%s"*
- Sets the *hdlin.naming.vhdl_preserve_case* application option to *true*

These settings typically make the naming style conform more closely to the language standard naming conventions.

To determine the current setting of this application option, use the *report_app_options hdlin.naming.upf_compatible* command.

For a list of HDL application options and their current settings, use *report_app_options hdlin.**.

hdlin.naming.verilog_generate_separator_style

Specifies the separator style for cells instantiated within generated blocks for Verilog.

Data Types

string

Default "."

Group

hdl_variables

Description

This variable specifies the separator style for cells instantiated within generated blocks for Verilog.

For example,

```
module CLA_multiplier (a,b,clk,product);
    parameter a_width = 8, b_width = 8;
    localparam product_width = a_width+b_width;
    input [a_width-1:0] a;
    input [b_width-1:0] b;
    input clk;
    output reg [product_width-1:0] product;
endmodule
```

```
module top (a,b,clk,product);
    parameter a_width = 8, b_width = 8, reset=0;
    localparam product_width = a_width+b_width;
    input [a_width-1:0] a;
```

```

input [b_width-1:0] b;
input clk;
output [product_width-1:0] product;

generate
    if(!reset) begin: reg_enable
        if((a_width < 8) || (b_width < 8)) begin: mult_one_logic
            CLA_multiplier #(a_width,b_width) u1(a, b, clk,
product);
                end
            else begin: mult_two_logic
                CLA_multiplier #(a_width,b_width) u2(a, b, clk,
product);
            end
        end
    endgenerate
endmodule

```

By default, the generated instance's name will be "reg_enable.mult_two_logic.u2".

If `hdlin.naming.verilog_generate_separator_style` is set to "_",

then generated instance's name will be "reg_enable_mult_two_logic_u2".

hdlin.naming.verilog_uppercase_names

Controls whether identifiers in the Verilog source code are converted to uppercase letters or left in their original case.

Data Types

Boolean

Default false

Description

This application option controls whether identifiers in the Verilog source code are converted to uppercase letters or left in their original case.

The default setting of this application option is *false*, which means that all names are left the way they are.

Setting the application option to *true* causes conversion of all identifiers in the Verilog code (variables, ports, and so on) to uppercase letters.

hdlin.naming.vhdl_generate

Specifies the naming style for *generated* design instances in VHDL designs.

Data Types

string

Default %s_%d

Description

This application option specifies the naming style for *generated* design instances. For example,

```
gen: for i in 0 to 1 generate
    U: CELL port map(a(i), z(i));
end generate gen;
```

If *hdlin.naming.vhdl_generate* is set to %s_%d (the default), the example results in instances named U_0 and U_1.

To determine the current setting of this application option, use the *report_app_options hdlin.naming.vhdl_generate* command. For a list of HDL application options and their current settings, use *report_app_options hdlin.**.

hdlin.naming.vhdl_generate_separator

Specifies the separator string for instances generated in multiple-nested loops.

Data Types

string

Default _

Description

This application option specifies the separator string for instances generated in multiple-nested loops. For example;

```
L1: for I in 3 downto 0 generate
    L2: for J in I downto 0 generate
        U: MY_AND port map ( A, B, Z);
    end generate;
end generate;
```

If *hdlin.naming.vhdl_generate_separator* is set to __ (the default is _), the example results in instances named U_3__3, U_3__2, U_3__1, U_3__0, U_2__2, U_2__1, U_2__0, U_1__1, U_1__0, and U_0__0.

To determine the current setting of this application option, use the *report_app_options hdlin.naming.vhdl_generate_separator* command. For a list of HDL application options and their current settings, use *report_app_options hdlin.**.

The value of this application option affects only designs read in VHDL format.

hdlin.naming.vhdl_preserve_case

Controls if VHDL record fields preserve its case after HDLC.

Data Types

Boolean

Default false

Description

This application option only affects to the case setting of the field naming on VHDL record type names, it does not impact other VHDL names. By default the field switch to uppercase after HDLC.

When *hdlin.naming.vhdl_preserve_case* is set to *true* the fields will preserve its original case after HDLC. For example, take the following record:

```
type myType is record
  Reg_oNe      : std_logic;
end record;
```

If an output register is defined as myType it will be named as Out1_reg[REG_ONE] by default. With *hdlin.naming.vhdl_preserve_case* set to *true* it will be named as Out1_reg[Reg_oNe].

hdlin.report.analyze_verbose

Queries information about preprocessing of Verilog and SystemVerilog conditional compilation compiler directives and macro expansions.

Data Types

integer

Default 0

Description

The *hdlin.report.analyze_verbose* application option lets you query information about preprocessing of the Verilog or SystemVerilog RTL, including macro expansions and conditional compilation compiler directives. You use this information to debug design issues, especially for designs with a large number of macros. To query the preprocessing information about conditional compilation compiler directives such as ``ifdef`, set this application option to 1. To query about these and additionally macro expansions, set the application option to 2. The default is 0.

The informational messages can be found in the log by looking for the message code (VER-7). For example,

```
Information: ./example.v:6: Analyzing `ifdef then clause because MYRTL is
defined. (VER-7)
Information: ./example.v:7: Macro |`MYMACRO| expanded to |1'b0|. (VER-7)
Information: ./example.v:8: Skipping `else clause. (VER-7)
```

See Also

- [analyze](#)

hdlin.report.check_no_latch

Controls whether a warning message is issued if a latch is inferred from a design.

Data Types

Boolean

Default false

Description

This application option controls whether a warning message is issued if a latch is inferred from a design. When set to *true*, the warning message is issued. This is useful for verifying that a combinational design does not contain memory components. The default value is *false*.

To determine the current setting of this application option, use the *report_app_options hdlin.report.check_no_latch* command. For a list of HDL application options and their current settings, use *report_app_options hdlin.**.

hdlin.report.design_hierarchy

Generate a design hierarchy report

Data Types

string

Default ""

Group

hdl_variables

Description

This option is used to generate a design report in json format. It needs to be set to “json” for the thing to work. This will generate a report on the design hierarchy, which includes the instances and its reference modules, including the library from which the module was elaborated, the file name, the line number etc.

hdlin.report.design_hierarchy_filename

filename prefix to write design hierarchy report

Data Types

string

Default "design_report"

Group

hdl_variables

Description

This option is used to generate a design report in json format and write it to the file. It gives file name to write to. It needs to be set to “json” for the thing to work. This will generate a report on the design hierarchy, which includes the instances and its reference modules, including the library from which the module was elaborated, the file name, the line number etc.

hdlin.report.level

Determines which information Presto HDL Compiler prints in the report.

Data Types

string

Default basic

Description

This application option controls the amount of output information to be included in the Presto elaboration report.

The following information is under control:

- *floating_net_to_ground* prints the report for floating net connects to ground. This application option is best used in conjunction with the *set hdlin.elaborate.keep_signal_name user* command and is not guaranteed to report all nets. Use the *check_design* command for detecting unconnected pins and ports.
- *fsm* prints the report for inferred state variables.
- *inferred_modules* prints the report for inferred sequential elements.
- *mux_op* prints the report for MUX_OPs.
- *syn_cell* prints the report for synthetic cells.
- *tri_state* prints the report for inferred tristate elements.

The *hdlin.report.level* application option can be set to 4 base settings: *none*, *basic*, *comprehensive*, and *verbose*, as shown in the following table:

Table 1 Base Settings

Information included in report	none	basic	comprehensive	verbose
<i>floating_net_to_ground</i>	false	false	true	true
<i>fsm</i>	false	false	true	true
<i>inferred_modules</i>	false	true	true	verbose
<i>mux_op</i>	false	true	true	true
<i>syn_cell</i>	false	false	true	true
<i>tri_state</i>	false	true	true	true

In addition to the base settings above, you can also modify the base settings to have fine grain control of individual reports by either adding (+) or subtracting (-) specific report(s) from the base setting with the following keywords:

```
floating_net_to_ground
fsm
syn_cell
mux_op
inferred_modules
tri_state
```

Examples

The following example uses *comprehensive-fsm*:

```
set hdlin.report.level comprehensive-fsm
```

The generated report shows the following settings:

```
floating_net_to_ground  true
fsm                     false
inferred_modules        true
mux_op                  true
syn_cell                true
tri_state               true
```

The following example uses *verbose-mux_op-tri_state*:

```
set hdlin.report.level verbose-mux_op-tri_state
```

The generated report shows the following settings:

```
floating_net_to_ground  true
fsm                     true
inferred_modules        verbose
mux_op                  false
syn_cell                true
tri_state               false
```

The following example shows two commands that generate equivalent reports:

```
set hdlin.report.level basic+floating_net_to_ground+syn_cell+fsm
```

```
set hdlin.report.level comprehensive
```

To determine the current setting of this application option, use the *report_app_options hdlin.report.level* command. For a list of HDL application options and their current settings, use *report_app_options hdlin.**.

hdlin.report.module_param_id_length_threshold

Configures the threshold for showing an ID association for a set of non-default parameters during elaboration in log.

Data Types

string

Default 128

Description

This option configures the threshold for the string that characterizes the non-default parameters of a module instantiation during elaboration. If the parameter string length is less than this option value, the *ELAB-193* information message will show all parameters; on the other hand, if the string is greater, an ID will be created and associated with that set

of parameters. The ID is numeric, incremental (starting from 1), and will be preserved for the same session. Consider that this feature is only enabled if *hdlin.logfile_format* option value is equal or greater than 202409.

For example, consider the following parametrized module and a threshold of 5:

```
module sub #(parameter PARAM1 = 0, parameter PARAM2 = 0) ();
  initial begin
    $display("PARAM1: %d, PARAM2: %d", PARAM1, PARAM2);
  end
endmodule

module top;
  sub #(1, 2) u_sub1 ();
  sub #(.PARAM1(3), .PARAM2(4)) u_sub2 ();
endmodule
```

If *top* module is elaborated, *u_sub1* will be instantiated with arguments "1,2", being a string length of 3; whereas *u_sub2* will be instantiated with the argument string "PARAM1=3,PARAM2=4", having a size (17) that exceeds the threshold, hence, an ID is associated.

hdlin.report.non_tristate_multiple_drivers

Controls whether the HDL Compiler tool issues an error or warning message when it finds multiple drivers of a net.

Data Types

Boolean

Default true

Description

This application option controls whether the HDL Compiler tool issues an error or warning message when it finds multiple drivers of a net. The application option supports Verilog and SystemVerilog input formats.

Multiple drivers occur when a reg variable is driven by more than one always block or a wire variable is driven by more than one continuous assignment or input port and all of the drivers are not tristates.

Before the 2001.08 release, the tool does not issue an error message for multiple-driver nets. After version 2001.08, the tool issues an error message by default. You can convert this ELAB-366 error message to an ELAB-365 warning by setting this application option to *false*. However, this setting is provided primarily for backward compatibility and is not recommended. You should inspect such warnings carefully. Setting the

h

hdlin.report.non_tristate_multiple_drivers application option to *false* can result in invalid designs.

In simulation, if a reg variable is driven by more than one always block, the definition depends on which block executed most recently. In synthesis, because all blocks are concurrently executing at all times, this behavior is not possible and invalid designs can result if the drivers are shorted together. Therefore, the tool issues an error message if any bits of any variables are driven by more than one process.

The Verilog standard prohibits a multiple-driven wire variable, although some simulators permit this behavior. The tool issues an ELAB-366 error message in this situation. If you want multiple-driven wire variables, set the *hdlin.report.non_tristate_multiple_drivers* application option to *false*. Note that this application option setting might cause invalid designs.

Multiple drivers are permitted on tri nets, although a warning is issued if all the drivers are not tristate devices.

To determine the current setting of this application option, use the *report_app_options hdlin.report.non_tristate_multiple_drivers* command. For a list of all HDL application options and their current values, use the *report_app_options hdlin.** command.

hdlin.report.port_dimension_mismatch_error

Vector or memory size mismatch on port results in an error instead of a warning.

Data Types

boolean

Default TRUE

Description

The *hdlin.report.port_dimension_mismatch_error* application options allows the user to get an error instead of a warning when a vector or memory size mismatch on port is detected, e.g.

```
module m1 (a);  
    input [3:0] a;  
    wire [5:0] a;  
endmodule
```

See Also

- [analyze](#)
- [VER-151](#)
- [VER-152](#)

hdlin.report.sequential_pruning

Controls reporting about removal of tentatively inferred sequential.

Data Types

Boolean

Default FALSE

Description

The *hdlin.report.sequential_pruning* application option lets you control whether HDL Compiler should issue a message when it detects that a sequential element that has already been tentatively inferred, but that is not actually required for proper functioning of the design being elaborated, is removed.

An RTL-level design description in Verilog or VHDL may contain application options for which a sequential cell would ordinarily be inferred (either because they are assigned under a clock edge, or because they are conditionally assigned) but whose values are never read, or whose values are not used in computing any output of the design. Because their values are not used (that is, they have no loads, or no path to the outputs), by default HDL Compiler will attempt not to infer sequential cells for these application options.

But if it does infer them, it will then try to prune them before the end of elaborate.

In some circumstances designers wish to retain these sequential cells.

Unloaded sequential cells can be inferred by two methods. First, if *hdlin.elaborate.preserve_sequential* is "all", then a sequential cell will be inferred for any application option that is conditionally assigned or assigned under a clock, regardless of whether its value is used or not. Second, if *hdlin.elaborate.preserve_sequential* is not set, but some application options are marked with the "preserve_sequential" pragma, only those application options will receive sequential cells even if they have no loads.

See Also

- [elaborate](#)

hdlin.report.vcs

Generate a design hierarchy report

Data Types

string

Default ""

Group

hdl_variables

Description

This option is used to generate a design report in json format. It needs to be set to "json" for the thing to work. This will generate a report on the design hierarchy, which includes the instances and its reference modules, including the library from which the module was elaborated, the file name, the line number etc.

hdlin.report.vcs_filename

filename prefix to write design hierarchy report

Data Types

string

Default "design_report"

Group

hdl_variables

Description

This option is used to generate a design report in json format and write it to the file. It gives file name to write to. It needs to be set to "json" for the thing to work. This will generate a report on the design hierarchy, which includes the instances and its reference modules, including the library from which the module was elaborated, the file name, the line number etc.

hdlin.report_info

Enables runtime and memory usage info to be displayed when elaborating modules.

Data Types

boolean

Default TRUE

Description

The *hdlin.report_info* application option allows the user to get runtime and memory usage information when elaborating a module. The message is shown when the runtime or memory usage goes over a threshold set by *hdlin.report_time* and *hdlin.report_mem* respectively.

Example of the message:

```
Module: top, Elapsed Time: 00:35:01, CPU Time: 00:37:12, Total Mem: 5.09
GB, Mem: 3.29 GB, Time: Mon May 12 16:45:13 2023
```

See Also

- [elaborate](#)

hdlin.report_mem

Set minimum memory usage increase in MB when elaborating a module to show runtime and memory usage information.

Data Types

integer

Default 100

Description

The *hdlin.report_mem* application option allows the user to set the minimum memory usage increase in MB when elaborating a module to get runtime and memory usage information.

Example of the message:

```
set_app_option -name hdlin.report_mem -value 3000
Module: top, Elapsed Time: 00:35:01, CPU Time: 00:37:12, Total Mem: 5.09
GB, Mem: 3.29 GB, Time: Mon May 12 16:45:13 2023
```

See Also

- [elaborate](#)

hdlin.report_time

Set minimum elapsed time in minutes when elaborating a module to show runtime and memory usage information.

Data Types

integer

Default 1

Description

The *hdlin.report_time* application option allows the user to set the minimum elapsed time in minutes when elaborating a module to get runtime and memory usage information.

Example of the message:

```
set_app_option -name hdlin.report_time -value 30
Module: top, Elapsed Time: 00:35:01, CPU Time: 00:37:12, Total Mem: 5.09
GB, Mem: 3.29 GB, Time: Mon May 12 16:45:13 2023
```

See Also

- [elaborate](#)

hdlin.safety.always_fsm_complete

Set attribute:FSM_COMPLETE for all of FSMs

Data Types

boolean

Default FALSE

Description

When this application option is set true, all of FSMs will have attribute: FSM_COMPLETE. Its default value is false.

hdlin.set_top_module.uncommit_sub_blocks

Determines if sub blocks are automatically uncommitted.

Data Types

boolean

Default FALSE

Description

The `hdlin.set_top_module.uncommit_sub_blocks` determines if sub blocks are automatically uncommitted, when linked with the `set_top_module` command

hdlin.sverilog.allow_unpacked_array_concat_on_port

Controls whether concat with unpacked array operand(s) in instance port expression is allowed. Effective only for Presto HDL Compiler.

Data Types

Boolean

Default false

Group

hdl_variables

Description

Controls whether concat with unpacked array operand(s) in instance port expression is allowed, such as `b[3:2]` in `.A({a, b[3:2], c})` shown in the following sample test case:

```
module top (
    input  logic a,
    input  logic b[3:2],
    input  logic c,
    output logic d
);

sub sub(.A({a, b[3:2], c}), .B(d));

endmodule

module sub(
    input  logic A[3:0],
    output logic B
);
    ...
endmodule
```

where, `b[3:2]` in the instance port expression `{a, b[3:2], c}` is an unpacked array.

The default setting is *false*, meaning that Presto does not allow the above case, and an error (*ELAB-929*) will be issued for the above case.

When `hdlin.sverilog.allow_unpacked_array_concat_on_port` is turned on, Presto will allow it with a warning message (*ELAB-1991*) issued.

As a precaution, you should examine the RTL code to make sure the connection is as expected. For example, in the corresponding port declaration in module *sub*, the port *A* should also be declared as an unpacked array, instead of a packed array such as an integer or bit-vector.

See Also

- [ELAB-929](#)
- [ELAB-1991](#)

hdlin.sverilog.blackbox_modules

Specify one or more SystemVerilog modules to be ignored during design read.

Data Types

string

Default ""

Description

The *hdlin.sverilog.blackbox_modules* allows the user to specify one or more SystemVerilog modules to be ignored during design read.

The modules are specified by setting the application option *hdlin.sverilog.blackbox_modules* to a list of modules, e.g.

```
dc_shell> set hdlin.sverilog.blackbox_modules "mod1 mod2 ..."
```

Here mod1, mod2 are names of modules as coded in the RTL.

The warning message (VER-746) is generated during *read_sverilog* or *analyze -f sverilog* command. An example of the message is shown below:

Warning: mod1.v:2: The declaration of module 'mod1' is being ignored, because the module name is in hdlin.sverilog.blackbox_modules. (VER-746)

If the tool tries to link the modules (during *elaborate* or *link* commands), *LINK-35 message is generated*:

Warning: All references to module 'mod1' are ignored and treated as black boxes. (LINK-35)

The valid module names are those coded in the RTL. The design names, e.g. those reported by *list_designs*, *may not work with this feature*.

No messages are generated for specified names that do not match valid module names.

See Also

- [analyze](#)
- [elaborate](#)

hdlin.sverilog.enable_persistent_macros

Controls whether a macro definition encountered during a Verilog or SystemVerilog analyze command is remembered in subsequent Verilog or SystemVerilog analyze commands.

Data Types

Boolean

Default false

Group

hdl_variables

Description

By default, each Verilog and SystemVerilog analyze command is independent, except for the intermediate files that are saved into the libraries defined by `define_hdl_library`.

When the `hdlin.sverilog.enable_persistent_macros` application option is enabled, the analyze command as its first action compiles the file of macro definitions specified by the `hdlin.sverilog.persistent_macros_filename` application option (if the file exists) and then as its last action it overwrites that file with the current macro definitions (or creates the file if it doesn't exist yet).

For example, if the analyze command encounters ``undef` directives or macro redefinitions, those changes will be reflected in the file at the end of the analyze command.

Like the intermediate files that are saved into hdl libraries, the macro definitions file is protected with Synopsys encryption.

It is recommended to remove any macros definition file that might be left over from a previous session by doing the following before the first analyze command.

```
file delete -force [get_app_option_value -name  
hdlin.sverilog.persistent_macros_filename]
```

hdlin.sverilog.enable_persistent_sv_interfaces

Controls whether a System Verilog Interface declaration encountered during a SystemVerilog analyze command is remembered in subsequent SystemVerilog analyze commands.

Data Types

boolean

Default FALSE

Group

hdl_variables

Description

By default, each Verilog and SystemVerilog analyze command is independent, except for the intermediate files that are saved into the libraries defined by `define_design_lib`.

When the `hdlin_enable_persistent_sv_interfaces` application variable is enabled, the analyze command as its first action compiles the file of macro definitions specified by the `hdlin_persistent_sv_interfaces_filename` application variable (if the file exists) and then as its last action it overwrites that file with the current macro definitions (or creates the file if it doesn't exist yet).

Like the intermediate files that are saved into libraries, the macro definitions file is protected with Synopsys encryption.

It is recommended to remove any interface declaration file that might be left over from a previous session by doing the following before the first analyze command.

```
file delete -force [get_app_var hdlin_persistent_sv_interfaces_filename]
```

hdlin.sverilog.enable_rtl_attributes

Enable SystemVerilog user attributes.

Data Types

boolean

Default FALSE

Description

The `hdlin.sverilog.enable_rtl_attributes` SystemVerilog allows users to specify their own attributes on cell, port, pin, and module. For example:

```
module top (  
    (* user_attr = "true" *) input TOPIN,  
    (* user_attr = "top port" *) output TOPOUT);  
  
    (* my_attr="P" *) BOT U_BOT ((* Pin_Attr = 0  
    *) .BOTIN(TOPIN), .BOTOUT(TOPOUT));  
endmodule
```

```
prompt> report_attributes [get_ports *]  
Design      Object      Type      Attribute Name      Value  
-----  
top         TOPIN       string    user_attr           true  
top         TOPOUT      string    user_attr           top port  
  
prompt> report_attributes [get_cells *]  
Design      Object      Type      Attribute Name      Value  
-----  
top         U_BOT       string    my_attr             P
```

See Also

- [analyze](#)
- [elaborate](#)

hdlin.sverilog.enforce_standalone_generate_blocks

Enforce that standalone generate blocks will be banned in SystemVerilog and Verilog-2005

Data Types

boolean

Default TRUE

Group

hdl_variables

Description

Enforce that generate blocks have either a loop or conditional construct according to the SystemVerilog and Verilog-2005 LRMs, i.e. the standalone generate block will be banned.

Setting this variable to FALSE will reduce the severity of using a standalone generate from an error (VER-946) to a warning (VER-945).

See Also

- [VER-945](#)
- [VER-946](#)

hdlin.sverilog.interface_only_modules

Enable HDL Compiler to read SystemVerilog designs as interface only

Data Types

string

Default ""

Description

This application option lists SystemVerilog designs as interface only. HDL Compiler parses the module interface of the listed designs, skipping the module content, and creates a black box for each module.

During elaboration, HDL Compiler issues a warning message that the module content is discarded and ignored.

Here is an example:

```
prompt> set_app_options -name hdlin.sverilog.interface_only_modules -value  
{my_module1 my_module2}
```

```
prompt> analyze -f sverilog top.sv
```

Warning: ./rtl/top.sv:21: The body of module 'my_module1' is being discarded, because the module name is in hdlin.sverilog.interface_only_modules. (VER-747)

hdlin.sverilog.packages

Specifies whether and how System Verilog packages should be analyzed.

Data Types

string

Default dont_chain

Description

Specifies which semantics the *analyze* command should apply when a *-format sverilog* source file declares a package. The setting affects the analyze step of SystemVerilog package declarations or references; it has no effect during *elaborate*. The allowed values for *hdlin.sverilog.packages* are *dont_chain* (the default) and *chain*.

The default behavior is as described in Clause 26 of IEEE Std 1800-2012, the standard for SystemVerilog. Overriding the default to *chain* changes how an *import* statement treats names imported into the topmost (global) scope of a package_declaration:

- The default *dont_chain* setting enforces the IEEE standard and prevents imported names from being re-exported to clients of the package being declared.
- The *chain* setting instead re-exports names that are imported into the global scope of a package; they may all be imported by the intermediate package's clients. An imported name and its definition which are re-exported (or chained) will not collide or interfere with copies of themselves in those cases where several intermediate packages redistribute content they acquired from a common source package (provided they all acquire it from a compatible analyzed version of the same source file).

Example: By default, output *dont_chain* will get 1 and output *chain* will get 0, but if the application option is overridden to *chain*, then output *dont_chain* will get 0 and output *chain* will get 1.

```
package A; localparam N = 44; endpackage
package B; localparam N = 77; endpackage
package C; import B::N; endpackage
import A::*;
module VER934 import C::*; (output dont_chain, chain);
    assign dont_chain = (N == 44);
    assign chain = (N == 77);
endmodule
```

The analyzed result, a file named *package_identifier.pvk*, always contains a full copy of all imported content. A package's .pvk file can stand alone; it does not require its clients to access the .pvk files that supplied its imported ingredients (unlike source-level file inclusion).

The chaining issue only concerns whether imported names become explicitly visible to an intermediate package's clients as do objects explicitly declared at the outermost level of the intermediate package. Because a wildcard "import intermediate_pkg::*;" encumbers all of the exportable names found in "intermediate_pkg", the selection of *chain* or *dont_chain* can alter the outcome of name resolutions when several packages are combined in a downstream client.

The setting of *hdlin.sverilog.packages* at the time a package is analyzed is compiled into a visibility property on the imported, global names in the resulting .pvk file. This is an independent property that can be different at each level of a supply chain; it is not an

inherited property of the name itself. A VER-934 informational message always indicates how this property is being set for those names where it may eventually matter.

To determine the current setting of this application option, use the *report_app_options hdlin.sverilog.packages* command. For a list of HDL application options and their current settings, use *report_app_options hdlin.**.

See Also

- [analyze](#)
- [VER-20](#)
- [VER-21](#)
- [VER-934](#)

hdlin.sverilog.persistent_macros_filename

Controls the name of the file to remember macro definitions encountered during a Verilog or SystemVerilog analyze command in subsequent Verilog or SystemVerilog analyze commands when the *hdlin.sverilog.enable_persistent_macros* application option is enabled.

Data Types

string

Default "syn_auto_generated_macro_file.sv.e"

Group

hdl_variables

Description

By default, each Verilog and SystemVerilog analyze command is independent, except for the intermediate files that are saved into the libraries defined by *define_hdl_library*.

When the *hdlin.sverilog.enable_persistent_macros* application option is enabled, the analyze command as its first action compiles the file of macro definitions specified by the *hdlin.sverilog.persistent_macros_filename* application option (if the file exists) and then as its last action it overwrites that file with the current macro definitions (or creates the file if it doesn't exist yet).

For example, if the analyze command encounters ``undef` directives or macro redefinitions, those changes will be reflected in the file at the end of the analyze command.

Like the intermediate files that are saved into hdl libraries, the macro definitions file is protected with Synopsys encryption.

It is recommended to remove any macros definition file that might be left over from a previous session by doing the following before the first analyze command.

```
file delete -force [get_app_option_value -name  
hdlin.sverilog.persistent_macros_filename]
```

hdlin.sverilog.persistent_sv_interfaces_filename

Controls the name of the file to remember SystemVerilog interface definitions encountered during a SystemVerilog analyze command in subsequent SystemVerilog analyze commands when the `hdlin_enable_persistent_sv_interfaces` application variable is enabled.

Data Types

string

Default "syn_auto_generated_interface_file.sv"

Group

hdl_variables

Description

By default, each Verilog and SystemVerilog analyze command is independent, except for the intermediate files that are saved into the libraries defined by `define_design_lib`.

When the `hdlin_enable_persistent_sv_interfaces` application variable is enabled, the analyze command as its first action compiles the file of macro definitions specified by the `hdlin_persistent_sv_interfaces_filename` application variable (if the file exists) and then as its last action it overwrites that file with the current macro definitions (or creates the file if it doesn't exist yet).

Like the intermediate files that are saved into libraries, the macro definitions file is protected with Synopsys encryption.

It is recommended to remove any interface declaration file that might be left over from a previous session by doing the following before the first analyze command.

```
file delete -force [get_app_var hdlin_persistent_sv_interfaces_filename]
```

hdlin.sverilog.standard

Controls whether HDLC SystemVerilog enforces SystemVerilog 2005, 2009, 2012 or 2017.

Data Types

integer

Default 2017

Description

This application option controls whether HDLC SystemVerilog is to enforce SystemVerilog 2005 or SystemVerilog 2009 or SystemVerilog 2012 or SystemVerilog 2017.

- If the application option is set to 2017 (the default), SystemVerilog 2017 is enforced. There is no Verilog 2017.
- If the application option is set to 2012, SystemVerilog 2012 is enforced. There is no Verilog 2012.
- If the application option is set to 2009, SystemVerilog 2009 is enforced. There is no Verilog 2009.
- If the application option is set to 2005, SystemVerilog 2005 is enforced.

The SystemVerilog 2009 standard merged two previous standards, Verilog 2005 and SystemVerilog 2005, which defined extensions to it. Those two standards were designed to be used as one language, so merging the base Verilog language and the SystemVerilog extensions into a single standard provides all information regarding syntax and semantics in a single document. Additionally, there are many extensions beyond SystemVerilog 2005 in SystemVerilog 2009.

The SystemVerilog 2012 standard adds extensions beyond SystemVerilog 2009.

The SystemVerilog 2017 standard clarifies some details of SystemVerilog 2012 and reserves no additional keywords.

hdlin.sverilog.tic_tic_discards_whitespace

Data Types

Boolean

Default false

Description

As described in section 22.5.1 of IEEE Std 1800-2017, a `` (double backtick) in a SystemVerilog macro definition "delimits lexical tokens without introducing white space, allowing identifiers to be constructed from arguments."

Enabling this application option (default is *false*) brings the tool into alignment with simulators on a related topic that is not clarified in the standard, because any whitespace surrounding macro arguments will be removed unless that whitespace closes an escaped identifier.

For example, unless this application option is enabled, analyzing the following RTL:

```
module top(output test_bit);
  `define M(arg) wire \_``arg``_ [2];
  `M(   t@st   )
  assign test_bit = \_t@st_ [0] & \_t@st_ [1];
  assign \_t@st_ = '{default:1'b1};
  initial assert final (test_bit);
endmodule
```

results in the following message:

```
Error: ./top.sv:3: Syntax error at or near token '_'
      in macro "M"
      called from file "./top.sv" (line 3). (VER-294)
```

hdlin.sverilog.use_search_path_for_include

Looks for files in the search path if they are not found in the given path in a Verilog *include* statement.

Data Types

Boolean

Default false

Description

Extend the behavior of the *include* statement in Verilog, looking for a file in the directories in the 'search path' variable if the file is not found in the provided path. By default, if a file is given with a relative or absolute path in a include statement, it is searched for only on the provided path, failing if it is not found there.

See Also

- [analyze](#)

hdlin.sverilog.write_token_file

Specifies whether a tokens file should be written out during the analysis of SystemVerilog designs.

Data Types

Boolean

Default false

Description

When *hdlin.sverilog.write_token_file* is turned on, the lexical tokens that are sent to the parser during analysis of a SystemVerilog design (such as with an *analyze -f sverilog* or a *read_sverilog* command) will also be saved to an output file in the work directory.

When using complex macros or nested conditional compilation directives, it might help to see what the actual RTL will be after preprocessing, for example, to understand a syntax error.

When the value of *hdlin.sverilog.write_token_file* is overridden and set to *true*, an expanded version of the source files is written to an output file *tokens.n{n}.sv*, instead of being sent to the parser. In the output file, conditional compilation directives will already have been taken into account and all macro invocations will already have been expanded. The first tokens file written out in a particular work directory will be *tokens.1.sv*, the next *tokens.2.sv*, the next *tokens.3.sv*, and so on.

Although the output file is legal SystemVerilog, it is formatted in a way that is intended to be conveniently human readable, by using standard line directives to indicate the original source files and line numbers. This makes it relatively easy to track down the source of an error using the file name and line number in the error message. For example, if the error message refers to line 321 of the file "*my_file.sv*", then look for *line 321 "my_file.sv"* in the output file.

The only comments preserved are those that are sent to the parser, because they are in more than mere comments, such as synthesis pragmas and embedded scripts.

If any of the source files are encrypted, then the output file is not created.

See Also

- [analyze](#)

hdlin.verilog.hierarchy_separator_style

Specifies the separator style for cells instantiated within nested blocks for Verilog.

Data Types

string

Default "."

Group

hdl_variables

Description

This variable specifies the separator style for cells instantiated within nested blocks for Verilog.

For example,

```
module top(clk, rst, datain, qout);
    input clk, rst, datain;
    output reg qout;

    generate if ( 1 ) begin
        always @(posedge clk or negedge rst) begin : myblk
            reg myreg;
            if( ! rst ) begin
                qout = 0;
                myreg = 0;
            end
            else begin
                qout = myreg;
                myreg = datain;
            end
        end
    end
endgenerate

endmodule
```

By default, the generated instance's name will be "genblk1.myblk.myreg_reg".

If `hdlin.verilog.generate_separator_style` is set to "_",

then generated instance's name will be "genblk1_myblk_myreg_reg".

hdlin.verilog.module_architecture_name_splitting

Controls whether Presto HDL Compiler recognizes a special format of Verilog module names, which allows users to specify both a module and an implementation architecture.

Data Types

Boolean

Default false

Description

During an *analyze* or *read* command, when the *hdlin.verilog.module_architecture_name_splitting* value is *true*, and a Verilog module name contains the special separator string "__" (two underscores), Presto HDL Compiler interprets the module name as specifying both a module and a specific architecture

or implementation for that module. The portion of the original module name before the separator string becomes the new module name, and the portion after the string becomes the architecture.

For example, a module named *mod__impl* would be renamed to *mod* and marked as having architecture *impl*.

If *hdlin.verilog.module_architecture_name_splitting* is set to *false* (the default), the renaming never occurs and modules always retain their original names. If the application option is set to *true*, the "__" is used to divide the module name into module and architecture as described above.

This capability exists to allow the creation of synthetic libraries using Verilog, with multiple architectures per module.

To determine the current setting of this application option, use the *report_app_options hdlin.verilog.module_architecture_name_splitting* command. For a list of HDL application options and their current settings, use *report_app_options hdlin.**.

hdlin.verilog.standard

Controls whether Presto Verilog enforces Verilog 1995, Verilog 2001, or Verilog 2005.

Data Types

integer

Default 2005

Description

This application option controls whether Presto Verilog enforces Verilog 1995, Verilog 2001, or Verilog 2005.

- If this application option is set to the value of 2005 (the default), Verilog 2005 is enforced.
- If the application option is set to 2001, Verilog 2001 is enforced.
- If the application option is set to 1995, Verilog 1995 is enforced.

hdlin.verilog.v2005_replication_semantics

Controls the treatment of zero replication constants in Verilog-1995 and Verilog-2001.

Data Types

Boolean

Default true**Description**

Until Verilog-2005 it was non-standard for the first operand in a replication operation to be zero, but, instead of issuing an error, Fusion Compiler would implement `{0{expression}}` as `1'b0`, regardless of the size or value of the second operand. By default, the tool now implements a zero replication constant according to the rules of Verilog-2005, even if the Verilog standard is overridden back to 1995 or 2001, in which case overriding this variable to *false* restores the older behavior. Overriding this variable has no effect when the standard is 2005. Because the older style of implementing a non-standard zero replication constant is likely to mismatch with simulation, an ELAB-364 warning is issued when it happens.

For example, in Verilog-2001 by default

```
1 module test(input [1:0] in, output [3:0] out);
2   assign out = { {2{in}}, {0{in}} };
3 endmodule
```

is implemented the same as

```
1 module test(input [1:0] in, output [3:0] out);
2   assign out = {2{in}};
3 endmodule
```

but when the variable is overridden to *false* it is implemented the same as

```
1 module test(input [1:0] in, output [3:0] out);
2   assign out = { in[0], in, 1'b0 };
3 endmodule
```

and an ELAB-364 warning is issued.

hdlin.vhdl.93_style_concatenation

Controls the concatenation behavior the tool uses to conform to the VHDL '93 Standard or the VHDL '87 Standard.

Data Types

Boolean

Default true**Description**

The *hdlin.vhdl.93_style_concatenation* application option controls the concatenation behavior the Presto VHDL Compiler uses to conform to the VHDL '93 Standard or the VHDL '87 Standard. If you set the value of this application option as *true* (the default), the

tool follows the VHDL '93 Standard. If you set the value as *false*, the tool follows the VHDL '87 Standard.

hdlin.vhdl.enable_configurations

Controls the configuration support by the tool.

Data Types

Boolean

Default false

Description

The *hdlin.vhdl.enable_configurations* application option enables configuration support by the tool when set to *true*. The default value is *false*.

To specify a configuration, use the *analyze* command. To elaborate an entity with a specific configuration, use the *elaborate* command with the configuration name as the design name. For example, if file.vhdl contains the configuration named *my_configuration*, read and elaborate the design as follows:

```
prompt> set hdlin.vhdl.enable_configurations true
prompt> analyze -f vhdl file.vhdl
prompt> elaborate my_configuration
```

The *read* command ignores configurations.

Setting this application option to *true* entails more stringent link-time checks. If designs do not use configurations, it is advised not to turn on this switch.

For more information refer to the *HDL Compiler for VHDL User Guide*.

See Also

- [analyze](#)
- [elaborate](#)

hdlin.vhdl.hierarchy_separator_style

Specifies the separator style for cells instantiated within nested blocks for VHDL.

Data Types

string

Default ""

Group

hdl_variables

Description

This variable specifies the separator style for cells instantiated within nested blocks for VHDL. For example,

```
library ieee;
  use ieee.std_logic_1164.all;
  use ieee.numeric_std.all;
  use ieee.math_real.all;

  entity mult is
    generic (
      mult_share_g      : natural
    );
    Port (
      clk      : in std_logic;
      rst_n    : in std_logic;
      enable   : in std_logic;
      a        : in signed(11 downto 0);
      b        : in signed(12 downto 0);
      mult_o   : out signed(24 downto 0)
    );
  end entity mult;

  architecture str of mult is

    constant mult_share_c : natural := mult_share_g;

  begin

    g_mult_g3 : if mult_share_c = 3 generate

      signal mult_i : signed(24 downto 0);

      begin
        p_data_mult : process (clk, rst_n)
        begin
          if rst_n = '0' then
            mult_i <= (others => '0');
          elsif rising_edge(clk) then
            if enable = '1' then
              mult_i <= a * b;
            end if;
          end if;
        end process p_data_mult;

        mult_o <= mult_i;

      end generate g_mult_g3;
```

```
end architecture str;
```

By default, the instance's name will be "g_mult_g3/mult_i_reg".

If `hdlin.vhdl.generate_separator_style` is set to "_",

then generated instance's name will be "g_mult_g3_mult_i_reg".

hdlin.vhdl.mixed_language_instantiation

Controls if mixed language instantiations are to be used in VHDL.

Data Types

Boolean

Default false

Description

This application option controls if mixed language instantiations are to be used in VHDL.

When you set the *hdlin.vhdl.mixed_language_instantiation* application option to true, HDL Compiler allows VHDL direct entity instantiations of a lower-level design that is not VHDL. Furthermore, if VHDL configurations are enabled, you can configure (standalone or architecture embedded) a lower-level design that is not VHDL to be used for a VHDL component; that is, you can configure the components inside this lower-level design.

All parameters have to be mapped explicitly in the generic mappings, though one or more are set to the default. Additionally, you cannot map literals in the generic mappings; that is, literal constants, such as 11, should be replaced with local constants like `my_bit_vector_constant`.

To see the current setting of this application option, use the *report_app_options hdlin.vhdl.mixed_language_instantiation* command. For a list of HDL application options and their current settings, use the *report_app_options hdlin.** command.

hdlin.vhdl.standard

Controls whether HDL Compiler follows the VHDL 2008 standard, the 1993 standard, or the 1987 standard.

Data Types

integer

Default 2008

Description

This application option determines the standard that the HDL Compiler is to follow in your design.

- If this application option is set to 2008 (the default), the compiler uses the VHDL 2008 standard.
- If this application option is set to 1993, the compiler uses the VHDL 1993 standard.
- If this application option is set to 1987, the compiler uses the VHDL 1987 standard.

hdlin.vhdl.subprogram_default_values

Determines which value the compiler will use as the default value for application options, 'LEFT of its type or 0s.

Data Types

Boolean

Default false

Description

This application option determines which value the compiler will use as the default value for application options, 'LEFT of its type or 0s.

When this application option is set to *true*, 'LEFT of the type of application option is used as its default value. If you set this application option to *false* (the default), 0s are used.

hdlin.vhdl.syntax_extensions

Enables VHDL language features that are currently outside of the supported synthesizable subset.

Data Types

Boolean

Default false

Description

This application option enables VHDL language features that are currently outside of the supported synthesizable subset. Use of these language constructs must be accompanied by thorough verification.

i

The following features are currently enabled:

- Deferred constant definition: a constant declaration not accompanied by an assignment expression.
- Arrays of base type Boolean: use Boolean as the range type of an array.
- Impure functions: a function specification contains the reserved word `impure`.

hdlin.xrefs.inject_xref_info

Data Types

Boolean

Default `false`

Description

When this application option is enabled (*true*), the synthesis commands (`compile_fusion`, `compile_logical`, and `rtl_opt`) will incorporate cross-probing on instances and ports lacking cross-referencing information during the compile process.

See Also

- [report_cross_probing](#)

i

in_gui_session

This read-only variable is "true" when the GUI is active and "false" when the GUI is not active.

Data Types

Boolean

Default `false`

Description

This variable can be used in writing Tcl code that depends on the presence the graphical user interface (GUI). The read-only variable has the value "true" if *gui_start* has been invoked and the GUI is active. Otherwise, the variable has the value "false" (default).

See Also

- [printvar](#)
- [gui_start](#)
- [gui_stop](#)

lib.configuration.assembly_scripts

lib.configuration.assembly_scripts Specifies the mapping between assembly scripts and physical source files.

Data Types

list

Default null

Description

This application option specifies the mappings between pre-qualified assembly scripts and physical source files.

By default, library configuration uses a predefined default script to create the cell libraries.

When you specify this application option, library configuration uses the assembly script that corresponds to the specified physical source files.

Use the following format to specify this application option:

```
{{assembly_script1 physical_file1} ...}
```

For example, to perform library configuration using the 1.tcl assembly script for the 1.frame physical library, the 2.tcl assembly script for the 2.frame physical library and the default assembly script for the 3.frame physical library, use the following commands:

```
prompt> set_app_options -name lib.configuration.assembly_scripts \\  
-value {{1.tcl 1.frame} {2.tcl 2.frame}}  
prompt> create_lib -ref_libs {1.frame 2.frame 3.frame}
```

The assembly script specified in this application option must use the following variables to read and write data:

- *db_files* : The logic library (.db) files to read
- *frame_files* : The physical libraries to read

- *lef_files* : The LEF files to read
- *clib_dir*: The output directory. This variable is set to the same value as the *lib.configuration.local_output_dir* application option.
- *clib_name*: The generated library name.

See Also

- [create_lib](#)
- [set_ref_libs](#)

lib.configuration.auto_reassemble_clib

Specifies whether to automatically reassemble clib.

Data Types

Boolean

Default true

Description

This application option, if set to false, indicates to not automatically reassemble clibs during *open_lib*.

This application option, if set to true (default), indicates to automatically reassemble clibs during *open_lib*.

See Also

- [open_lib](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

lib.configuration.auto_reassemble_on_implicit_open

Specifies whether to automatically reassemble clib when open library implicitly.

Data Types

Boolean

Default false

Description

This application option, if set to true, indicates to automatically reassemble clibs during an implicit library open. Library can be implicitly opened in many block commands without using *open_lib* command explicitly. For example, *open_block* command can accept a block specification with a library prefix. If that library is not open, it will open the library implicitly.

To make the value true taking effect, the *lib.configuration.auto_reassemble_clib* app option must be true.

This application option, if set to false (default), indicates not to automatically reassemble clibs during an implicit library open.

See Also

- [create_block](#)
- [copy_block](#)
- [get_app_option_value](#)
- [move_block](#)
- [open_block](#)
- [rename_block](#)
- [report_app_options](#)
- [save_block](#)
- [set_app_options](#)

lib.configuration.cdpl_host

lib.configuration.cdpl_host Specifies the CDPL host for library configuration.

Data Types

string

Default *""*

Description

This application option specifies the CDPL host for library configuration: The Common Distributed Processing Library (CDPL) is a common framework available to all Synopsys applications needing to incorporate parallel computing. CDPL allows library configuration to be parallelized, significantly reducing runtime To run library assembly in parallel on

the same host, use the following application option setting and set the maximum desired number of processes. `set_app_options -name lib.configuration.cdpl_host -value "-hosts :8"`

Note that quotation marks are needed around the "-hosts :8" value because of the embedded space in the syntax. Users must make sure the host has enough compute resource to run the processes parallelly when setting the previous application option. By default, the application option value is "". It means the tool will not use CDPL feature for the library configuration.

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

lib.configuration.central_output_dir

lib.configuration.central_output_dir Specifies the central output directory for library configuration.

Data Types

string

Default ""

Description

This application option specifies the directory to put reusable cell libraries. When *create_lib*, *set_ref_libs* and *open_lib* commands need to build a cell library, it will first check this directory for same name library and see if it can be reused.

See Also

- [create_lib](#)
- [open_lib](#)
- [set_ref_libs](#)

lib.configuration.create_missing_macro_frame

lib.configuration.create_missing_macro_frame Specifies whether to create dummy physical cells for the db files without physical source in library configuration flow.

Data Types

Boolean

Default *false*

Description

This application option, if set to true, indicates to create dummy physical cells for the db files without physical source. This application option, if set to false (default), the tool will issue LIB-083 for ignoring the db files without physical source.

See Also

- [LIB-083](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

lib.configuration.default_flow_setup

lib.configuration.default_flow_setup Specifies the unix file path of a setup script for customize the library configuration.

Data Types

string

Default *Empty*

Description

This application option specifies the unix file path of a setup script for customize the library configuration. The *create_lib*, *set_ref_libs* and *open_lib* commands depend on this application option. The script specified by this application option will be sourced at the beginning of the library configuration. By default, this application option is empty which means no setup script will be sourced.

See Also

- [create_lib](#)
- [open_lib](#)
- [set_ref_libs](#)

lib.configuration.display_lm_messages

lib.configuration.display_lm_messages Enables the display of library manager messages when performing library configuration in the implementation tool.

Data Types

Boolean

Default false

Description

This application option enables the display of library manager messages when you run the *create_lib* or *set_ref_libs* command with the *-ref_libs* option.

By default (*false*), no library manager messages are displayed when you run the *create_lib* and *set_ref_libs* commands.

When set to *true*, the commands display the following library manager error messages: LM-*, FRAM-*, LEFR-*, LMUI-*, FILE-*, EMS-*. In addition, they display the following warning and information messages: LM-001, LM-014, LM-026, LM-028, LM-033, LM-042, LM-044, LM-045, LM-046, LM-047, LM-048, LM-054, LM-055, LM-058, LM-063, LM-064, LM-068, LM-073, LM-075, LM-080, LM-084, LM-089, LM-090, LM-098, LM-109, LM-110, LM-111, LM-112, LM-113, LM-114, LM-118, LM-121, LM-130, LM-133, LM-134, LM-135, LM-136, LM-152, LM-154, LM-158, LM-160, FRAM-001, FRAM-002, FRAM-007, FRAM-012, FRAM-013, FRAM-014, FRAM-015, FRAM-016, FRAM-018, FRAM-021, FRAM-025, FRAM-027, FRAM-028, FRAM-030, FRAM-031, FRAM-032, FRAM-033, FRAM-035, FRAM-036, FRAM-038, FRAM-039, FRAM-040, FRAM-041, FRAM-042, FRAM-045, FRAM-046, FRAM-047, FRAM-050, FRAM-053, FRAM-054, LEFR-014, LEFR-016, LEFR-026, LEFR-035, LEFR-037, LEFR-043, LEFR-054, LEFR-057, LEFR-062, LMUI-009, LMUI-036, LMUI-042, LMUI-043, LMUI-047, LMUI-048, LMUI-050, and FILE-007.

See Also

- [create_lib](#)
 - [set_ref_libs](#)
-

lib.configuration.enable

Specifies whether to enable the configuration of clibs.

Data Types

Boolean

Default `true`

Description

This application option, if set to `true` (default), indicates to enable the automatic configuration of clibs during *create_lib*, *set_ref_libs* and *open_lib*.

This application option, if set to `false`, indicates to disable the automatic configuration of clibs during *create_lib*, *set_ref_libs* and *open_lib*.

See Also

- [create_lib](#)
- [open_lib](#)
- [set_ref_libs](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

lib.configuration.force_timing_view_update

Specifies whether to update the timing views of prebuilt cell libraries.

Data Types

Boolean

Default `false`

Description

Controls whether the timing views of prebuilt cell libraries are updated during library configuration.

By default (*false*), prebuilt cell libraries are not handled by library configuration.

If *true*, the tool updates the timing views of prebuilt cell libraries during library configuration. The physical views of the prebuilt cell libraries are read in directly by the *read_ndm* command. The logic libraries (*.db*) files still come from the *link_library* application variable.

The generated cell libraries are written to the directory specified by the *lib.configuration.local_output_dir* application option. If the prebuilt cell libraries are already in this directory, library configuration fails.

See Also

- [get_app_option_value](#)
- [create_lib](#)
- [set_ref_libs](#)
- [report_app_options](#)
- [set_app_options](#)

lib.configuration.icc_shell_exec

Specifies the full path to an executable script that runs IC Compiler.

Data Types

String

Default Empty

Description

This application option specifies the name of a script file that runs IC Compiler, which will be used by the library configuration to export Milkyway data. This option only needed to be set when Milkyway libs are provided as physical source data. The file must be executable.

See Also

- [create_lib](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)
- [set_ref_libs](#)

lib.configuration.lef_site_mapping

lib.configuration.lef_site_mapping Specifies the site_def name conversion which will be passed to read_lef -convert_sites.

Data Types

list_of {string string}

Default Empty

Description

This application option specifies the `site_def` name conversion which will be passed to the `-convert_sites` option of the `read_lef` command in the library configuration. By default, this application option is empty which means the `-convert_sites` option will not be used with `read_lef` in the library configuration.

Examples

The following example maps `site_def` name *HPCORE* to *unit* when read LEF files in the library configuration.

```
prompt> set_app_options -name lib.configuration.lef_site_mapping -value  
        {{HPCORE unit}}
```

See Also

- [create_lib](#)
- [open_lib](#)
- [set_ref_libs](#)

lib.configuration.local_output_dir

lib.configuration.local_output_dir Specifies the local output directory for library configuration.

Data Types

string

Default *"CLIBs"*

Description

This application option specifies the directory to put cell libraries. The *create_lib*, *set_ref_libs* and *open_lib* commands depend on this application option. If Milkyway libs are provided as physical source data, the converted frame libs will be put under *auto_frame_libs* sub directory of the directory specified by this application option. By default, if this application option is empty, it will use the *CLIBs* directory to put auto created cell libraries.

See Also

- [create_lib](#)
- [open_lib](#)
- [set_ref_libs](#)

lib.configuration.process_label_mapping

lib.configuration.process_label_mapping Specifies the mapping between process_label and the corresponding db files list.

Data Types

list

Default null

Description

This application option specifies the mapping between process_label and corresponding db files. when users specify such mapping, the tool will attach process label to its corresponding db files. You can specify the mapping for one or more process labels. The syntax used to specify each mapping is:

```
{process_label_name {db_name1 db_name2 ...}}
```

Here is an example of set such mapping: set_app_options -name lib.configuration.process_label_mapping -value {{p1 {1.db 2.db 3.db}} {p2 {4.db 5.db}} {p3 {6.db}}} with the above mapping, the tool will attach process label 'p1' to {1.db 2.db 3.db}, p2 to {4.db 5.db}, p3 to 6.db. which means the tool will execute the below commands with the specified process label for its mapping db files list. read_db -process_label "p1" {1.db 2.db 3.db} read_db -process_label "p2" {4.db 5.db} read_db -process_label "p3" {6.db}

See Also

- [create_lib](#)
- [set_ref_libs](#)

lib.configuration.remove_local_output_dir

Specifies whether to remove the local output directory upon exit of current session.

Data Types

Boolean

Default false

Description

This application option, if set to true, indicates to remove the local output directory upon exit of current session. This application option can be set to true only when directory indicated by *lib.configuration.local_output_dir* does not exist, or does not contain any file.

This application option, if set to false (default), the local output directory will not be removed upon exit of current session.

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

lib.configuration.update_timing_view_for_pt_delay

Specifies whether to re-generate pre-1806 clibs for pt delay calculation.

Data Types

Boolean

Default false

Description

This application option, if set to true, indicates to re-generate pre-1806 clib regardless whether the clib was previously created by library configuration, or whether its source files have been changed.

This application option, if set to false (default), pre-1806 clib will be re-generated only when they are created by library configuration and their source files have been changed.

See Also

- [create_lib](#)
- [open_lib](#)
- [set_ref_libs](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

lib.logic_model.auto_remove_incompatible_timing_designs

Specifies whether to automatically remove timing designs which fail to compare to the same design in the base (first) logical library.

Data Types

Boolean

Default false

Description

When set to *true*, if *check_workspace* encounters a timing design that does not compare to the same design in the base (first) logical library for any reason (missing port, different port direction, different logic function, etc.), the offending design will be automatically removed, and no timing for that design for that library will be present in the workspace. Incompatible designs are a sign of out of sync libraries. This is an option to avoid.

See Also

- [LM-032](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

lib.logic_model.auto_remove_timing_only_designs

Specifies whether to automatically remove designs which exist in logical (DB, Liberty) libraries which do not exist in physical libraries.

Data Types

Boolean

Default false

Description

When set to *true*, designs that exist in logical libraries that do not exist in physical libraries will be removed from the workspace automatically by *check_workspace*

See Also

- [LM-035](#)
- [LM-092](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

lib.logic_model.preserve_nldm_pin_caps

Specifies whether the capacitances on the pins are converted from CCS receiver_capacitance or are taken from the logical library source files (DB, .lib).

Data Types

Boolean

Default false

Description

When there is CCS receiver_capacitance group defined on pin or timing arc in DB or .lib file, will convert the CCS receiver data to NLDM pin capacitances. If conversion happens, the original NLDM pin capacitances are replaced by the conversion result.

With this application option set as true, the original NLDM capacitances on pins from DB or .lib file will be preserved, the conversion from CCS receiver data to NLDM pin capacitances is disabled.

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

lib.logic_model.relax_pin_equal_opposite_attr_check

Relax inconsistencies check for equal/opposite attributes on lib pin across (DB) libraries.

Data Types

boolean

Default false

Description

The equal/opposite attributes on lib pin should be consistent across logical (DB) libraries, command *check_workspace* will issue error LM-060 if any inconsistency is identified and check will be failed.

However it is quite often that some ETM macro DB with equal/opposite attributes mismatched across libraries, Library Compiler does report the inconsistency for such mismatch. For user that this mismatch is not critical and would like to continue down the flow, this app option is provided to relax the check. With the app option set as true, it is

allowed that some DB with equal/opposite attributes while some DB without the equal/opposite attributes on the corresponding lib pins. It is not allowed that equal/opposite attributes are conflict across DB libraries, for example, pin A and pin B are logical equal in one DB, but pin A and pin B are logical opposite in another DB.

Even with the app option set as true, *check_workspace* will still check the equal/opposite attributes consistencies across libraries. If any inconsistency is identified, warning message LM-160 will be issued.

See Also

- [LM-060](#)
- [LM-160](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

lib.logic_model.require_same_opt_attrs

Specifies whether optimization attributes have to match between the base logical (DB) library and other DB libraries in the library manager.

Data Types

Boolean

Default false

Description

When set to *true*, a difference in the *dont_touch*, *dont_use*, or *use_for_size_only* design attributes between the base DB library and other DB libraries becomes an error which you must correct with *set_attribute*. In all cases, you are warned when there is a difference.

See Also

- [LM-080](#)
- [LM-081](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

lib.logic_model.resolve_voltage_range_differences

Specifies how to handle voltage range differences across the logical (DB) libraries.

Data Types

enum

Default `use_base_library`

Description

The `input_voltage_range` and `output_voltage_range` range constructs on Liberty cells and pins are expected to match across the DBs for a technology. If any differences are found, a warning is issued (LM-115). The default is to take the value from the base library (the first library read). There are two other possibilities.

You can force an LM-081 error if the values are different by setting this app option to *require_same*.

If you set the app option *use_outer_bounds*, the minimum of the low values and the maximum of the high values will be used. So if you have a pin with an `input_voltage_range` in the base library of 0.9, 1.0 and an value of 0.85, 0.95 in another, the result will be 0.85, 1.0.

See Also

- [LM-115](#)
- [LM-081](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

lib.logic_model.use_db_rail_names

Specifies whether voltage rail names are taken from the logical library source files (DB, .lib) or if they are automatically generated.

Data Types

Boolean

Default `true`

Description

In Liberty, the *voltage_map* construct defines the voltage rails. The voltage rail name is essentially a constant which is replaced with a voltage value. For this application, there can be only one name for a rail in the committed NDM library. So, when you combine a set of DB or .lib source libraries with this application, the voltage rail names must match across these DB libraries. You can avoid most rail naming problems by not encoding a voltage value into the name; for example use VDD, not VDD_1.0 for the 1.0 volt rail. You can also use a template approach, where you put the same voltage_map entries in all of your .lib files, whether you need them or not.

When the rails don't match, you have some choices:

- You can use the *rename_rail* command to rename the rails in one or more of the libraries which came from DB. See the man page for the *rename_rail* command for good examples.
- You can let the application choose voltage rail names for you by setting the application option *lib.logic_model.use_db_rail_names* to false. There are cases when it is very difficult to find a combination of *rename_rail* commands that will get you do a consistent set of rail names. The application can do this by spreading the voltage values out across all of the library cell pins for all libraries, then collecting unique combinations and considering each as a voltage rail. The rail names are very simple: power_1, power_2, ground_1, and so on. Note that this may result in a reduction in the number of rails from DB if there are duplicate values. Currently, during *check_workspace*, this will result in a number of errors followed by a message that rail mismatches have been fixed.

Note that this option is a last-resort fix for rail problems. It is intended only for cases when there is a loop in the rail naming scheme. This should not be used to get around combining unrelated libraries. If you are not sure which DB or .lib files go together, consider using the exploration flow to group the related libraries.

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

lib.setting.compress_design_lib

Tells save_lib/save_block to save design library in compressed format.

Data Types

boolean

Default true

VALUES

false true

Description

When this app option is set to true, all design library contents, except for attachments and side-files, will be saved in compressed format. After a design or library has been saved in compressed format, the "is_compressed" attribute on the design/library is set to true. Subsequent saves on the design/library will remain in compressed format.

Setting this app_option to false does not change the default format of already compressed a design or library back to uncompressed. The only way to revert a compressed design or library to uncompressed format is to set the "is_compressed" attribute on the library and its designs to false and then save it with save_block/save_lib.

See Also

- [save_lib](#)
- [save_block](#)
- [design_attributes](#) Description of the application defined design attributes.
- [lib_attributes](#) Description of the application defined lib attributes.

lib.setting.compress_lib_package

Specifies whether lib_package has to be compressed.

Data Types

Boolean

Default "true"

Description

This app_option specifies whether lib_package created by write_lib_package has to be compressed or not. By default, lib_package will be compressed.

Compression of lib_package can be disabled by setting the app_option to false.

See Also

- [write_lib_package](#)
- [read_lib_package](#)

lib.setting.compression_format

Specify the compression format for save_lib/save_block when lib.setting.compress_desing_lib is true.

Data Types

string

Default zstd

VALUES

gzip zstd

Description

This option specifies the compression format to use when the lib.setting.compress_design_lib app option is set to true. After a design/library has been saved in compressed format, the "compression_format" attribute on the design/library is set to the compression format that was used for save.

"gzip" has been the only supported format previously, but from S-2021.06-SP5, "zstd" is also supported for save. Design/library saved in "zstd" format cannot be opened in binaries before S-2021.06-SP5.

See Also

- [save_lib](#)
- [save_block](#)
- [design_attributes](#) Description of the application defined design attributes.
- [lib_attributes](#) Description of the application defined lib attributes.

lib.setting.enable_db_search_optimization

Enables optimization to be done while performing db search and resolving the matching between original db path name to the mapped db path name

Data Types

boolean

Default true

VALUES

false true

Description

When this app option is set to true, the command resolving the db path will use the directory structure information of original db file path to resolve to unique mapped db files before using the search path. This is necessary to resolve uniquely when multiple db files have the same base name. In such case, the search path doesnot resolve uniquely as search path has a predifined order which always resolve to a single file.

Setting this app_option to false means user intends to use only the search path while resolving the mapping between the original db file path to the mapped db file path.

Setting this app-option value to true would also enable optimization to search for the db path name by using the directory structure information of the original db files. This is normally useful when multiple db files have the same base file names and unique resolution cannot be done only using search path information.

lib.setting.enable_multithreaded_open

Controls if the ref-libs are loaded using multi-thread or not.

Data Types

Boolean

Default false

VALUES

true false

Description

By default, reference libraries are loaded sequentially. If multi-threaded loading is enabled, reference libraries are loaded using multiple threads during the execution of the open_lib and read_lib_package commands. This can significantly improve performance, especially when dealing with high number of reference libraries.

See Also

- [open_lib](#)
- [read_lib_package](#)

lib.setting.enable_process_number_scaling

Enables process number scaling during timing analysis.

Data Types

boolean

Default false

VALUES

false true

Description

When this app option is set to true, it can make `define_scaling_lib_group` allow panes with different process numbers and treat process number just like voltage and temperature for scaling.

lib.setting.enable_via_region_override

Enables creation of design-library specific override of via-regions for all reference-libraries with inconsistent tech.

Data Types

boolean

Default true

VALUES

false true

Description

`read_tech_file/set_ref_libs` might cause tech inconsistency of reference-libraries with respect to design library. When this app option is set to true, supported commands would create design library specific via-regions to override existing via-regions in lib-cell frame of reference-library.

Setting this app_option to false means user intends to use reference-library via-regions and design-library specific override via-regions would not be used anymore.

Changing value of this app-option does not trigger generation of via-region override. Change of tech-file (`read_tech_file`) or change of ref-libs (`set_ref_libs`) might cause tech inconsistency. This app-option must be set to true after tech-file or ref-libs update but before any command supporting this app-option for override via-region generation.

Setting this app-option value to true would also enable usage of any generated design-library specific override via-regions for inconsistent reference-library.

See Also

- [read_tech_file](#)
- [set_ref_libs](#)
- [derive_design_level_via_regions](#)

lib.setting.fusion_lib_fast_integrity_check

Specifies whether enable fast integrity check mode.

Data Types

bool

Default false

Description

This application option specifies whether enable fast integrity check mode.

The default value is false. The integrity check will fully check the lib in this mode.

When set the value to true, the integrity check will skip check crc and get better runtime.

Integrity check will only throw out warning for timestamp difference when this value is set to true, user should make sure the lib only been copied or moved and has not been changed.

See Also

- [open_lib](#)
- [FLIB-2](#)
- [FLIB-11](#)
- [FLIB-51](#)
- [FLIB-12](#)

lib.setting.get_command_scope

Controls how get_libs, get_lib_cells and get_lib_pins search for libraries.

Data Types

string

Default global

VALUES

global design_and_ref_libs

Description

By default, `get_libs`, `get_lib_cells` and `get_lib_pins` look at all matching libraries in memory. When `current_lib` points to a design library, and this app_option is set to "design_and_ref_libs", only the libs/lib-cells/lib-pins within the `current_lib` and its `ref_libs` will be searched.

See Also

- [open_lib](#)
- [report_ref_libs](#)
- [set_ref_libs](#)

lib.setting.handle_noncolored_shapes

Specifies the value to enable the noncolored shapes checking.

Data Types

String

Default "none"

Description

This application option can be one of the value of list {none | loose | strict}.

The following commands will honor the application option setting: `lm_shell` commands: `read_lef`, `write_lef`, `read_ndm`, `check_workspace` `icc2_shell` commands: `create_frame`, `write_lef`, `open_lib`, `create_lib`

By default, the value is *none* the tool does not check the noncolored shapes.

If the value is *loose*, the previous commands will check the noncolored shapes with loose mode, when founding noncolored shapes, these commands will issue warning message for the shapes and continue the executing. Note: `read_lef` will drop the noncolored shapes in this mode.

If the value is *strict*, the previous commands will check the noncolored shapes with strict mode, when founding noncolored shapes, the commands have the below behaviors: `read_lef`, `write_lef`, `create_frame` and `check_workspace` will issue warning message for the shapes and drops the shapes, and then continue the executing. `create_lib` and `open_lib` will issue warning message for the shapes and continue the executing.

See Also

- [write_lef](#)
- [create_lib](#)
- [open_lib](#)
- [create_frame](#)

lib.setting.icc_shell_exec

Specifies the full path of an IC Compiler shell.

Data Types

String

Default Empty

Description

This application option specifies the full path of an IC Compiler shell, which will be used by some commands to operate on Milkyway FRAMs. This application option must be specified before running those commands.

lib.setting.max_wait_time

Controls the maximum wait time before the lock retry times out.

Data Types

integer

Default 3600

Description

A library is locked during open and save. The tool waits and retries if another process already has the library locked. A lock request times out if a lock still is not available after the maximum wait time.

See Also

- [open_lib](#)
- [save_lib](#)

lib.setting.milkyway_exec

Specifies the full path of a Milkyway executable.

Data Types

String

Default Empty

Description

This application option specifies the full path of a Milkyway executable, which will be used by some commands to operate on Milkyway FRAMs. This application option must be specified before running those commands.

lib.setting.on_disk_operation

Specifies whether the new library is saved to the disk or memory during the execution of the create_lib command. Default value is false.

Data Types

boolean

Default false

VALUES

false true

Description

This application option is used to specify whether the new library is saved to the disk after it is created using the create_lib command. When this application option is set to true, the library is saved to the disk. By default, this application option is set to false, which indicates that the newly created library is saved only in the memory.

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

lib.setting.show_internal_pins

Controls the visibility of internal pins at the UI level.

Data Types

Boolean

Default true

Description

By default, the tool always returns/displays internal pins; however, the user may set this app option to false to avoid seeing internal pins in the UI.

Use the following command to determine the current value of the application option:

```
prompt> get_app_option_value -name lib.setting.show_internal_pins
```

See Also

- [get_ports](#)
- [get_lib_pins](#)
- [get_pins](#)

lib.setting.use_tech_scale_factor

Uses the scale factor from the tech file to create a new library.

Data Types

Boolean

Default False.

Description

When a new design or lib cell library is created, it needs to have a scale factor, also known as database units per user unit, for length. The user unit for length is microns and the scale factor determines the maximum precision the database can represent.

By default, NDM libraries are created with a scale factor of 10000, allowing distances to be specified with a precision of 1 Angstrom.

It can be overridden either by using this app option or with the option "-scale_factor" to commands `create_lib` and `create_workspace`. When this app option is used, the library is created with the same scale factor as the `lengthPrecision` specified in the tech file used to create the libraries.

This app option only works when options "-scale_factor" and "-ref_libs" are not specified in command `create_lib` or "-scale_factor" is not specified in command `create_workspace`.

See Also

- [create_lib](#)
- [lib_attributes](#) Description of the application defined lib attributes.
- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)

lib.workspace.allow_append_layout_views

Specifies whether allow append layout views from different physical source libraries.

Data Types

Boolean

Default false

Description

This application option is used with the *lib.workspace.save_layout_views* application option to allow append layout views from physical source libraries different than design and frame views.

When set to *true*, the tool will append the layout views from physical source libraries different than design and frame views into the cell library.

By default, the layout views of the physical source libraries different than design and frame views will not be appended.

This application option is applicable to normal, frame and edit flow.

Examples

Following example will add layout view of a cell from input.gds into result cell library regardless whether there is a design or frame view in the test.ndm.

```
prompt> set_app_options -name lib.workspace.save_layout_views -value true
prompt> set_app_options -name lib.workspace.allow_append_layout_views
         -value true
prompt> set_app_options -name lib.workspace.keep_all_physical_cells
         -value true
prompt> create_workspace -technology input.tf
prompt> read_ndm test.ndm
prompt> read_gds input.gds -trace_option none
```

```
prompt> check_workspace  
prompt> commit_workspace
```

See Also

- [read_gds](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

lib.workspace.allow_commit_workspace_overwrite

Specifies whether allow the command *commit_workspace* to overwrite existing file.

Data Types

Boolean

Default false

Description

When set to *false*, *commit_workspace* will error out if there is a file has the same name with your specified name or default name. When set to *true*, *commit_workspace* will overwrite the existing file. *commit_workspace* will overwrite the existing file when option *-force* is specified, even though this application option value is *false*.

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

lib.workspace.allow_loading_layout_view_only_lib

Specifies whether an existing reference library is allowed to update the layout views from GDSII files in edit flow.

Data Types

Boolean

Default false

Description

This application option is only applicable to the edit flow. This application option is used with the *lib.workspace.save_layout_views* application option to update the layout views from the GDSII files without changing the existing design and frame views in the reference library.

When the set to *true*, the tool will update the layout views from the GDSII files without changing the existing design and frame views in the reference library.

By default, the layout views of the existing library will not be updated.

Examples

To update test.ndm with GDSII file(input.gds).

```
prompt> set_app_options -name lib.workspace.save_layout_views -value true
prompt> set_app_options -name lib.workspace.save_design_views -value true
prompt> set_app_options -name
lib.workspace.allow_loading_layout_view_only_lib -value true
prompt> create_workspace -technology input.tf -flow edit test.ndm
prompt> read_gds input.gds -trace_option none
prompt> check_workspace
prompt> commit_workspace
```

See Also

- [read_gds](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

lib.workspace.allow_missing_related_pg_pins

Specifies whether signal pins are allowed to have no related power and ground pins.

Data Types

Boolean

Default false

Description

When set to *false*, a signal pin is not allowed to have no related power or ground pin except that if the containing cell has no power and ground pins at all or if the signal pin is an internal pin. If a signal pin is found to have no related power or ground pin, error

messages will be displayed and the `check_workspace` command will fail. When set to `true`, if the related pg pin information of a signal pin is missing, information of the signal pin and its containing lib cell will be displayed but the missing of related pg pins will not cause `check_workspace` to fail.

See Also

- [LM-088](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

lib.workspace.create_workspace_tf_verbose

In the Library Manager, customize the detail in output from `create_workspace` when used with option `-technology`.

Data Types

Boolean

Default `true`

Description

This application option is used to customize the messages output by `create_workspace` with option `-technology`. *It can be set to true or false. Detail messages like unsupported syntax, will be reported when this application option value is true.*

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

lib.workspace.enable_secondary_pg_marking

This app option specifies whether secondary power and ground pins should be marked.

Data Types

Boolean

Default `true`

Description

When set to true, secondary pg pins will be marked by the library manager. By default the option is set to true. It can be set to false with the `set_app_options` command.

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

lib.workspace.exclude_design_filters

Specifies one or more patterns to match designs to exclude from the library workspace when reading library source files in `icc2_lm_shell`.

Data Types

list

Default empty

Description

This application option is used with the `lib.workspace.include_design_filters` application option to determine which designs are loaded into the library workspace when you run commands such as `read_db`, `read_lib`, `read_lef`, `read_gds`, and `read_ndm`.

By default, all designs in a source library file are loaded into the library workspace by the read commands and none are excluded.

The comparisons are done in two steps:

- If a design name matches any *exclude* filter, the design is not loaded into the library workspace.
- If a design name matches any *include* filter (or if there are no include filters), the design is loaded into the library workspace. Note that if you specify an include filter, any design that does not match the filter is not loaded into the library workspace.

These options are useful for splitting your libraries into more or finer groups. You can accomplish the same effect by reading all the source libraries, building collections of designs to remove by using the `get_lib_cells` command and manipulating those collections, and then using the `remove_lib_cells` command. Both methods work, but using the read filters is more efficient.

If you specify any filters, the tool issues a one-line notation for each included design. For example, if you set the include filter to AN2 and used *read_db*, you might see this:

```
\&... including AN2
```

If you read an NDM file which might contain several views of the same design, you might see this:

```
\&... including AN2/frame  
\&... including AN2/design
```

Examples

To exclude designs AND2 and OR2:

```
prompt> set_app_options -name lib.workspace.exclude_design_filters \\  
-value "AND2 OR2"
```

See Also

- [read_gds](#)
- [get_lib_cells](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

lib.workspace.explore_create_aggregate

Specifies whether to generate an aggregate library in exploration flow.

Data Types

Boolean

Default false

Description

This app option affects *commit_workspace*, *process_workspaces* and *write_workspace* commands. It only has effect when there are multiple workspaces specified, either explicitly or implicitly.

When set to *true*, the *commit_workspace* and *process_workspaces* commands will create an aggregate library containing all libraries generated from specified workspaces. The *write_workspace* command will append an aggregate workspace at the end of the outputted tcl file if single file should be outputted, or create a separate tcl file containing

an aggregate workspace if multiple files should be outputted. The aggregate workspace containing all libraries of the specified workspaces.

When set to *false*, no aggregate library will be generated.

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

lib.workspace.fast_exploration

Determines how some library source files are read in the exploration flow.

Data Types

Boolean

Default `false`

Description

This application option, when true, changes the way some library source files are read so that only the minimum information needed by *group_libs* is made available. In some cases, it can be 50 times faster. With this mode, you can only use *write_workspace* after grouping; the *process_workspaces* command is disabled. So the usage model becomes *group_libs*, *write_workspace*, *remove_workspace*, and *source* the script(s) you created with *write_workspace*. In a future release, when this restriction is lifted and both the script and *process_workspaces* methods work, then the default value for this application option will be true and it will become hidden.

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

lib.workspace.group_libs_fix_cell_shadowing

Specifies whether allow the command *group_libs* to fix potential cell shadowing.

Data Types

Boolean

Default `true`

Description

When set to *false*, *group_libs* will perform grouping based on cell name signatures only, which means logical libraries with exact same set of lib cells are grouped together. However, if there are cell misaligned among set of libraries, some cells may be created in multiple reference ndm libraries, each library only contains partial of pvt panes, this will cause cell shadowing during link designs.

When set to *true* which is default, *group_libs* will fix potential cell shadowing if detects any. The cell shadowing fix will break down some logical libraries into pieces, each contains subset of lib cells of original library. Refer EXAMPLE of *LM-121* for more details.

See Also

- [LM-121](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

lib.workspace.group_libs_macro_grouping_strategy

Specifies the strategy for grouping macro cells in *group_libs* command during the exploration flow.

Data Types

Enum

Default `aggregate_single_cell`

Description

This application option specifies the strategy used for grouping macro cells in *group_libs* command during the exploration flow. Allowed values are *aggregate_single_cell* and *single_cell_per_lib*.

In this context, any input DB or Liberty source files containing a single cell (for example, a ram) is considered a macro. This option allows you to organize the macro cells in different ways depending on your needs. The basic strategy is *single_cell_per_lib*. This strategy will generate one NDM reference library for each macro cell. Each library includes all contributing PVTs for one macro cell. With this strategy, you will have as many NDM reference libraries as you have macros. For example, if you have 100 macros with 4 PVTs (so 400 DB files), in this case, we will create 100 NDM reference libraries.

By default, the tool uses the *aggregate_single_cell* strategy. This builds on the *single_cell_per_lib* strategy by taking the individual NDM reference libraries and aggregating them into a single NDM. So in the above example, you would get 1 library with 100 cells. The file is named by the workspace with *_macros* appended to it. If the workspace name is SRAM, then the file name is SRAM_macros.ndm.

Examples

To apply *single_cell_per_lib* strategy:

```
prompt> set_app_options -name
lib.workspace.group_libs_macro_grouping_strategy\
-value single_cell_per_lib
```

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

lib.workspace.group_libs_naming_strategies

Specifies the naming strategy for the library groups created by the *group_libs* command during the exploration flow.

Data Types

list

Default common_prefix_and_common_suffix

Description

This application option specifies the naming strategy used for library groups created by the *group_libs* command during the exploration flow.

This name is used as the name of the subworkspace for the library group, as well as its committed reference library.

By default, the tool uses the *common_prefix_and_common_suffix* strategy. This strategy names the library groups by using a unique combination of logic library prefix and suffix characters, except the physical-only library group, which is named <root_workspace_name>_physical_only where <root_workspace_name> is the name of the root workspace, which was specified with the *create_workspace* command.

To change the naming strategy, specify an ordered list of one or more of the following values:

- *common_prefix*

This strategy names the library groups by using a common prefix of the logic libraries in the library group.

- *common_suffix*

This strategy names the library groups by using a common suffix of the logic libraries in the library group.

- *common_prefix_and_common_suffix*

This strategy names the library groups by using both the common prefix and common suffix of the logic libraries in the library group. If the tool can find only a common prefix or a common suffix, it uses that as the library group name.

- *first_logical_lib_name*

This naming strategy names the library groups by using the name of the first logic library name in the library group.

If you specify multiple values, the tool uses the first naming strategy that generates a valid name.

Examples

To apply common prefix first followed by common suffix strategy:

```
prompt> set_app_options -name lib.workspace.group_libs_naming_strategies
\\
    -value common_prefix common_suffix
```

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

lib.workspace.group_libs_physical_only_name

Specifies the name of the physical_only library group created by the *group_libs* command in the exploration flow.

Data Types

string

Default "" (an empty string)

Description

This application option specifies the name of the `physical_only` library group created by the `group_libs` command in the exploration flow. The name of a library group is used as the name of the subworkspace for the library group as well as its committed reference library.

The default value of this application option is "", i.e. an empty string. If the value of this option is "", the `physical_only` library group created by the `group_libs` command is named `<root_workspace_name>_physical_only`, where `<root_workspace_name>` is the name of the `root_workspace` which was specified with the `create_workspace` command.

You can change the naming of the `physical_only` group by setting the `lib.workspace.group_libs_physical_only_name` application option to the name you want before running the `group_libs` command. To change the naming back to using `<root_workspace_name>_physical_only`, set the option to "" or simply reset it with the `reset_app_options` command.

Examples

To name the physical library group "physical_only":

```
prompt> set_app_options -name lib.workspace.group_libs_physical_only_name
\\
-value physical_only
```

To set the option back to its default value:

```
prompt> set_app_options -name lib.workspace.group_libs_physical_only_name
\\
-value ""
```

To name the physical library group "ABC":

```
prompt> set_app_options -name lib.workspace.group_libs_physical_only_name
\\
-value ABC
```

To reset the option to its default value:

```
prompt> reset_app_options lib.workspace.group_libs_physical_only_name
```

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

lib.workspace.include_design_filters

Specifies one or more patterns to match designs to load into the library workspace when reading library source files in `icc2_lm_shell`.

Data Types

list

Default empty

Description

This application option is used with the `lib.workspace.exclude_design_filters` application option to determine which designs are loaded into the library workspace when you run commands such as `read_db`, `read_lib`, `read_lef`, `read_gds`, and `read_ndm`.

By default, all designs in a source library file are loaded into the library workspace by the read commands and none are excluded.

The comparisons are done in two steps:

- If a design name matches any *exclude* filter, the design is not loaded into the library workspace.
- If a design name matches any *include* filter (or if there are no include filters), the design is loaded into the library workspace. Note that if you specify an include filter, any design that does not match the filter is not loaded into the library workspace.

These options are useful for splitting your libraries into more or finer groups. You can accomplish the same effect by reading all the source libraries, building collections of designs to remove by using the `get_lib_cells` command and manipulating those collections, and then using the `remove_lib_cells` command. Both methods work, but using the read filters is more efficient.

If you specify any filters, the tool issues a one-line notation for each included design. For example, if you set the include filter to AN2 and used `read_db`, you might see this:

```
\&... including AN2
```

If you read an NDM file which might contain several views of the same design, you might see this:

```
\&... including AN2/frame  
\&... including AN2/design
```

Examples

To only include designs AND2 and OR2:

```
prompt> set_app_options -name lib.workspace.include_design_filters \\  
-value "AND2 OR2"
```

See Also

- [read_gds](#)
- [get_lib_cells](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

lib.workspace.keep_all_physical_cells

Specifies whether the generated reference library includes cells from physical sources which don't have any contribution from a logical source.

Data Types

Boolean

Default false

Description

When set to *false*, The generated reference library contains the cells that exist in both the logic and physical library files. If the physical library files contain additional cells that are not in the logic library files, they are not included in the generated reference library. When set to *true*, the additional cells that are not in the logic library files, they will be built into the generated reference library, but no timing view.

This application option is not applicable to the exploration flow since exploration flow will put all physical only cells into a physical only library. In exploration flow, if really need to create a reference library that includes all physical only cells, please write out a script using *write_workspace* command and source the script instead.

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

lib.workspace.library_developer_mode

Controls whether the library preparation flow is in library developer mode.

Data Types

Boolean

Default false

Description

When you set this application option to *true*, some issues that are warnings in normal mode are escalated to errors. Library developers can use this mode to identify issues that impact the reference library quality. Library end users should leave this option as *false*.

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)
- [LEFR-066e](#)
- [LEFR-067e](#)
- [LEFR-068e](#)
- [LM-167](#)

lib.workspace.remove_frame_bus_properties

Specifies whether remove bus properties which exist in frame view but not in timing view.

Data Types

Boolean

Default false

Description

When set to *true*, bus properties which exist in frame views but not in timing views will be removed automatically by command *check_workspace*

See Also

- [LM-095](#)
- [LM-096](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

lib.workspace.save_design_views

Specifies whether design views from LEF or GDS input will be saved.

Data Types

Boolean

Default false

Description

When set to *true*, *read_gds*, *read_lef* and *check_workspace* will keep design views in the result library.

See Also

- [read_gds](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

lib.workspace.save_layout_views

Specifies whether layout views from GDS input will be saved.

Data Types

Boolean

Default false

Description

When set to *true*, *read_gds* and *check_workspace* will keep layout views in the result library.

See Also

- [read_gds](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

libra.complib.match_full_cell_name

Use the full cell name as the matching condition to create the library cell matching pairs for *compare_libraries*.

Data Types

Boolean

Default `true`

Description

This option controls how to create library cell matching pairs for comparison in the command *compare_libraries*.

When this app-option is *"true"*, the tool will try to find a library cell from the target library with the full cell name exactly matched to the reference library cell.

If a matched library cell is available, a library cell matching pair will be created for further comparison and reporting.

An user can optionally use the lexical attribute values instead of the full cell name as the matching conditions by setting this option to *"false"*. Then, configure those *libra.complib.match_lexical_** app-options to set the lexical matching conditions.

See Also

- [compare_libraries](#)
- [classify_lib_cell_attributes](#)
- [report_lib_cell_classification_attributes](#)
- [report_app_options](#)
- [set_app_options](#)

libra.complib.match_lexical_cell_height

Use the lexical attribute *lexical_cell_height* as a matching condition to create the library cell matching pairs for the command *compare_libraries*.

Data Types

Boolean

Default true

Description

This option controls how to use lexical attributes to create library cell matching pairs for comparison in the command *compare_libraries*.

When this app-option is "*true*", the value of the lexical attribute *lexical_cell_height* of the reference and the target library cells must be exactly matched, as one of the conditions when the tool is trying to find the library cell matching pairs.

This app-option is only effective when *libra.complib.match_full_cell_name* is set to "*false*".

See Also

- [compare_libraries](#)
- [classify_lib_cell_attributes](#)
- [report_lib_cell_classification_attributes](#)
- [set_app_options](#)
- [report_app_options](#)

libra.complib.match_lexical_cell_tile_width

Use the lexical attribute *lexical_cell_tile_width* as a matching condition to create the library cell matching pairs for the command *compare_libraries*.

Data Types

Boolean

Default true

Description

This option controls how to use lexical attributes to create library cell matching pairs for comparison in the command *compare_libraries*.

When this app-option is *"true"*, the value of the lexical attribute *lexical_cell_tile_width* of the reference and the target library cells must be exactly matched, as one of the conditions when the tool is trying to find the library cell matching pairs.

This app-option is only effective when *libra.complib.match_full_cell_name* is set to *"false"*.

See Also

- [compare_libraries](#)
- [classify_lib_cell_attributes](#)
- [report_lib_cell_classification_attributes](#)
- [set_app_options](#)
- [report_app_options](#)

libra.complib.match_lexical_circuit_clockgate

Use the lexical attribute *lexical_circuit_clockgate* as a matching condition to create the library cell matching pairs for the command *compare_libraries*.

Data Types

Boolean

Default `true`

Description

This option controls how to use lexical attributes to create library cell matching pairs for comparison in the command *compare_libraries*.

When this app-option is *"true"*, the value of the lexical attribute *lexical_circuit_clockgate* of the reference and the target library cells must be exactly matched, as one of the conditions when the tool is trying to find the library cell matching pairs.

This app-option is only effective when *libra.complib.match_full_cell_name* is set to *"false"*.

See Also

- [compare_libraries](#)
- [classify_lib_cell_attributes](#)
- [report_lib_cell_classification_attributes](#)
- [set_app_options](#)
- [report_app_options](#)

libra.complib.match_lexical_circuit_device_sizing_variant

Use the lexical attribute *lexical_circuit_device_sizing_variant* as a matching condition to create the library cell matching pairs for the command *compare_libraries*.

Data Types

Boolean

Default `true`

Description

This option controls how to use lexical attributes to create library cell matching pairs for comparison in the command *compare_libraries*.

When this app-option is "*true*", the value of the lexical attribute *lexical_circuit_device_sizing_variant* of the reference and the target library cells must be exactly matched, as one of the conditions when the tool is trying to find the library cell matching pairs.

This app-option is only effective when *libra.complib.match_full_cell_name* is set to "*false*".

See Also

- [compare_libraries](#)
- [classify_lib_cell_attributes](#)
- [report_lib_cell_classification_attributes](#)
- [set_app_options](#)
- [report_app_options](#)

libra.complib.match_lexical_circuit_eco

Use the lexical attribute *lexical_circuit_eco* as a matching condition to create the library cell matching pairs for the command *compare_libraries*.

Data Types

Boolean

Default `true`

Description

This option controls how to use lexical attributes to create library cell matching pairs for comparison in the command *compare_libraries*.

When this app-option is *"true"*, the value of the lexical attribute *lexical_circuit_eco* of the reference and the target library cells must be exactly matched, as one of the conditions when the tool is trying to find the library cell matching pairs.

This app-option is only effective when *libra.complib.match_full_cell_name* is set to *"false"*.

See Also

- [compare_libraries](#)
- [classify_lib_cell_attributes](#)
- [report_lib_cell_classification_attributes](#)
- [set_app_options](#)
- [report_app_options](#)

libra.complib.match_lexical_circuit_layout_density_variant

Use the lexical attribute *lexical_circuit_layout_density_variant* as a matching condition to create the library cell matching pairs for the command *compare_libraries*.

Data Types

Boolean

Default `true`

Description

This option controls how to use lexical attributes to create library cell matching pairs for comparison in the command *compare_libraries*.

When this app-option is *"true"*, the value of the lexical attribute *lexical_circuit_layout_density_variant* of the reference and the target library cells must be exactly matched, as one of the conditions when the tool is trying to find the library cell matching pairs.

This app-option is only effective when *libra.complib.match_full_cell_name* is set to *"false"*.

See Also

- [compare_libraries](#)
- [classify_lib_cell_attributes](#)
- [report_lib_cell_classification_attributes](#)

- [set_app_options](#)
- [report_app_options](#)

libra.complib.match_lexical_circuit_layout_variant_1

Use the lexical attribute *lexical_circuit_layout_variant_1* as a matching condition to create the library cell matching pairs for the command *compare_libraries*.

Data Types

Boolean

Default `true`

Description

This option controls how to use lexical attributes to create library cell matching pairs for comparison in the command *compare_libraries*.

When this app-option is *"true"*, the value of the lexical attribute *lexical_circuit_layout_variant_1* of the reference and the target library cells must be exactly matched, as one of the conditions when the tool is trying to find the library cell matching pairs.

This app-option is only effective when *libra.complib.match_full_cell_name* is set to *"false"*.

See Also

- [compare_libraries](#)
- [classify_lib_cell_attributes](#)
- [report_lib_cell_classification_attributes](#)
- [set_app_options](#)
- [report_app_options](#)

libra.complib.match_lexical_circuit_layout_variant_2

Use the lexical attribute *lexical_circuit_layout_variant_2* as a matching condition to create the library cell matching pairs for the command *compare_libraries*.

Data Types

Boolean

Default `true`

Description

This option controls how to use lexical attributes to create library cell matching pairs for comparison in the command *compare_libraries*.

When this app-option is *"true"*, the value of the lexical attribute *lexical_circuit_layout_variant_2* of the reference and the target library cells must be exactly matched, as one of the conditions when the tool is trying to find the library cell matching pairs.

This app-option is only effective when *libra.complib.match_full_cell_name* is set to *"false"*.

See Also

- [compare_libraries](#)
- [classify_lib_cell_attributes](#)
- [report_lib_cell_classification_attributes](#)
- [set_app_options](#)
- [report_app_options](#)

libra.complib.match_lexical_circuit_multibit

Use the lexical attribute *lexical_circuit_multibit* as a matching condition to create the library cell matching pairs for the command *compare_libraries*.

Data Types

Boolean

Default `true`

Description

This option controls how to use lexical attributes to create library cell matching pairs for comparison in the command *compare_libraries*.

When this app-option is *"true"*, the value of the lexical attribute *lexical_circuit_multibit* of the reference and the target library cells must be exactly matched, as one of the conditions when the tool is trying to find the library cell matching pairs.

This app-option is only effective when *libra.complib.match_full_cell_name* is set to *"false"*.

See Also

- [compare_libraries](#)
- [classify_lib_cell_attributes](#)
- [report_lib_cell_classification_attributes](#)
- [set_app_options](#)
- [report_app_options](#)

libra.complib.match_lexical_circuit_topology_variant

Use the lexical attribute *lexical_circuit_topology_variant* as a matching condition to create the library cell matching pairs for the command *compare_libraries*.

Data Types

Boolean

Default `true`

Description

This option controls how to use lexical attributes to create library cell matching pairs for comparison in the command *compare_libraries*.

When this app-option is *"true"*, the value of the lexical attribute *lexical_circuit_topology_variant* of the reference and the target library cells must be exactly matched, as one of the conditions when the tool is trying to find the library cell matching pairs.

This app-option is only effective when *libra.complib.match_full_cell_name* is set to *"false"*.

See Also

- [compare_libraries](#)
- [classify_lib_cell_attributes](#)
- [report_lib_cell_classification_attributes](#)
- [set_app_options](#)
- [report_app_options](#)

libra.complib.match_lexical_device_channel_length

Use the lexical attribute *lexical_device_channel_length* as a matching condition to create the library cell matching pairs for the command *compare_libraries*.

Data Types

Boolean

Default true

Description

This option controls how to use lexical attributes to create library cell matching pairs for comparison in the command *compare_libraries*.

When this app-option is "*true*", the value of the lexical attribute *lexical_device_channel_length* of the reference and the target library cells must be exactly matched, as one of the conditions when the tool is trying to find the library cell matching pairs.

This app-option is only effective when *libra.complib.match_full_cell_name* is set to "*false*".

See Also

- [compare_libraries](#)
- [classify_lib_cell_attributes](#)
- [report_lib_cell_classification_attributes](#)
- [set_app_options](#)
- [report_app_options](#)

libra.complib.match_lexical_device_threshold_voltage

Use the lexical attribute *lexical_device_threshold_voltage* as a matching condition to create the library cell matching pairs for the command *compare_libraries*.

Data Types

Boolean

Default true

Description

This option controls how to use lexical attributes to create library cell matching pairs for comparison in the command *compare_libraries*.

When this app-option is *"true"*, the value of the lexical attribute *lexical_device_threshold_voltage* of the reference and the target library cells must be exactly matched, as one of the conditions when the tool is trying to find the library cell matching pairs.

This app-option is only effective when *libra.complib.match_full_cell_name* is set to *"false"*.

See Also

- [compare_libraries](#)
- [classify_lib_cell_attributes](#)
- [report_lib_cell_classification_attributes](#)
- [set_app_options](#)
- [report_app_options](#)

libra.complib.match_lexical_device_type

Use the lexical attribute *lexical_device_type* as a matching condition to create the library cell matching pairs for the command *compare_libraries*.

Data Types

Boolean

Default `true`

Description

This option controls how to use lexical attributes to create library cell matching pairs for comparison in the command *compare_libraries*.

When this app-option is *"true"*, the value of the lexical attribute *lexical_device_type* of the reference and the target library cells must be exactly matched, as one of the conditions when the tool is trying to find the library cell matching pairs.

This app-option is only effective when *libra.complib.match_full_cell_name* is set to *"false"*.

See Also

- [compare_libraries](#)
- [classify_lib_cell_attributes](#)
- [report_lib_cell_classification_attributes](#)

- [set_app_options](#)
- [report_app_options](#)

libra.complib.match_lexical_electrical_delay_opt_variant

Use the lexical attribute *lexical_electrical_delay_opt_variant* as a matching condition to create the library cell matching pairs for the command *compare_libraries*.

Data Types

Boolean

Default `true`

Description

This option controls how to use lexical attributes to create library cell matching pairs for comparison in the command *compare_libraries*.

When this app-option is *"true"*, the value of the lexical attribute *lexical_electrical_delay_opt_variant* of the reference and the target library cells must be exactly matched, as one of the conditions when the tool is trying to find the library cell matching pairs.

This app-option is only effective when *libra.complib.match_full_cell_name* is set to *"false"*.

See Also

- [compare_libraries](#)
- [classify_lib_cell_attributes](#)
- [report_lib_cell_classification_attributes](#)
- [set_app_options](#)
- [report_app_options](#)

libra.complib.match_lexical_electrical_hold_opt_variant

Use the lexical attribute *lexical_electrical_hold_opt_variant* as a matching condition to create the library cell matching pairs for the command *compare_libraries*.

Data Types

Boolean

Default `true`

Description

This option controls how to use lexical attributes to create library cell matching pairs for comparison in the command *compare_libraries*.

When this app-option is *"true"*, the value of the lexical attribute *lexical_electrical_hold_opt_variant* of the reference and the target library cells must be exactly matched, as one of the conditions when the tool is trying to find the library cell matching pairs.

This app-option is only effective when *libra.complib.match_full_cell_name* is set to *"false"*.

See Also

- [compare_libraries](#)
- [classify_lib_cell_attributes](#)
- [report_lib_cell_classification_attributes](#)
- [set_app_options](#)
- [report_app_options](#)

libra.complib.match_lexical_electrical_property_variant

Use the lexical attribute *lexical_electrical_property_variant* as a matching condition to create the library cell matching pairs for the command *compare_libraries*.

Data Types

Boolean

Default *true*

Description

This option controls how to use lexical attributes to create library cell matching pairs for comparison in the command *compare_libraries*.

When this app-option is *"true"*, the value of the lexical attribute *lexical_electrical_property_variant* of the reference and the target library cells must be exactly matched, as one of the conditions when the tool is trying to find the library cell matching pairs.

This app-option is only effective when *libra.complib.match_full_cell_name* is set to *"false"*.

See Also

- [compare_libraries](#)
- [classify_lib_cell_attributes](#)
- [report_lib_cell_classification_attributes](#)
- [set_app_options](#)
- [report_app_options](#)

libra.complib.match_lexical_electrical_setup_opt_variant

Use the lexical attribute *lexical_electrical_setup_opt_variant* as a matching condition to create the library cell matching pairs for the command *compare_libraries*.

Data Types

Boolean

Default `true`

Description

This option controls how to use lexical attributes to create library cell matching pairs for comparison in the command *compare_libraries*.

When this app-option is *"true"*, the value of the lexical attribute *lexical_electrical_setup_opt_variant* of the reference and the target library cells must be exactly matched, as one of the conditions when the tool is trying to find the library cell matching pairs.

This app-option is only effective when *libra.complib.match_full_cell_name* is set to *"false"*.

See Also

- [compare_libraries](#)
- [classify_lib_cell_attributes](#)
- [report_lib_cell_classification_attributes](#)
- [set_app_options](#)
- [report_app_options](#)

libra.complib.match_lexical_electrical_slew_opt_variant

Use the lexical attribute *lexical_electrical_slew_opt_variant* as a matching condition to create the library cell matching pairs for the command *compare_libraries*.

Data Types

Boolean

Default true

Description

This option controls how to use lexical attributes to create library cell matching pairs for comparison in the command *compare_libraries*.

When this app-option is "*true*", the value of the lexical attribute *lexical_electrical_slew_opt_variant* of the reference and the target library cells must be exactly matched, as one of the conditions when the tool is trying to find the library cell matching pairs.

This app-option is only effective when *libra.complib.match_full_cell_name* is set to "*false*".

See Also

- [compare_libraries](#)
- [classify_lib_cell_attributes](#)
- [report_lib_cell_classification_attributes](#)
- [set_app_options](#)
- [report_app_options](#)

libra.complib.match_lexical_electrical_symmetry_variant

Use the lexical attribute *lexical_electrical_symmetry_variant* as a matching condition to create the library cell matching pairs for the command *compare_libraries*.

Data Types

Boolean

Default true

Description

This option controls how to use lexical attributes to create library cell matching pairs for comparison in the command *compare_libraries*.

When this app-option is *"true"*, the value of the lexical attribute *lexical_electrical_symmetry_variant* of the reference and the target library cells must be exactly matched, as one of the conditions when the tool is trying to find the library cell matching pairs.

This app-option is only effective when *libra.complib.match_full_cell_name* is set to *"false"*.

See Also

- [compare_libraries](#)
- [classify_lib_cell_attributes](#)
- [report_lib_cell_classification_attributes](#)
- [set_app_options](#)
- [report_app_options](#)

libra.complib.match_lexical_electrical_timing_arc_variant

Use the lexical attribute *lexical_electrical_timing_arc_variant* as a matching condition to create the library cell matching pairs for the command *compare_libraries*.

Data Types

Boolean

Default `true`

Description

This option controls how to use lexical attributes to create library cell matching pairs for comparison in the command *compare_libraries*.

When this app-option is *"true"*, the value of the lexical attribute *lexical_electrical_timing_arc_variant* of the reference and the target library cells must be exactly matched, as one of the conditions when the tool is trying to find the library cell matching pairs.

This app-option is only effective when *libra.complib.match_full_cell_name* is set to *"false"*.

See Also

- [compare_libraries](#)
- [classify_lib_cell_attributes](#)
- [report_lib_cell_classification_attributes](#)

- [set_app_options](#)
- [report_app_options](#)

libra.complib.match_lexical_userdef_property_1

Use the lexical attributes *lexical_userdef_property_1_name* and *lexical_userdef_property_1_value* as a matching condition to create the library cell matching pairs for the command *compare_libraries*.

Data Types

Boolean

Default `true`

Description

This option controls how to use lexical attributes to create library cell matching pairs for comparison in the command *compare_libraries*.

When this app-option is *"true"*, the values of the lexical attribute *lexical_userdef_property_1_name* and *lexical_userdef_property_1_value* of the reference and the target library cells must be exactly matched, as one of the conditions when the tool is trying to find the library cell matching pairs.

This app-option is only effective when *libra.complib.match_full_cell_name* is set to *"false"*.

See Also

- [compare_libraries](#)
- [classify_lib_cell_attributes](#)
- [report_lib_cell_classification_attributes](#)
- [report_app_options](#)
- [set_app_options](#)

libra.complib.match_lexical_userdef_property_2

Use the lexical attributes *lexical_userdef_property_2_name* and *lexical_userdef_property_2_value* as a matching condition to create the library cell matching pairs for the command *compare_libraries*.

Data Types

Boolean

Default true

Description

This option controls how to use lexical attributes to create library cell matching pairs for comparison in the command *compare_libraries*.

When this app-option is *"true"*, the values of the lexical attribute *lexical_userdef_property_2_name* and *lexical_userdef_property_2_value* of the reference and the target library cells must be exactly matched, as one of the conditions when the tool is trying to find the library cell matching pairs.

This app-option is only effective when *libra.complib.match_full_cell_name* is set to *"false"*.

See Also

- [compare_libraries](#)
- [classify_lib_cell_attributes](#)
- [report_lib_cell_classification_attributes](#)
- [set_app_options](#)
- [report_app_options](#)

libra.complib.match_lexical_userdef_property_3

Use the lexical attributes *lexical_userdef_property_3_name* and *lexical_userdef_property_3_value* as a matching condition to create the library cell matching pairs for the command *compare_libraries*.

Data Types

Boolean

Default true

Description

This option controls how to use lexical attributes to create library cell matching pairs for comparison in the command *compare_libraries*.

When this app-option is *"true"*, the values of the lexical attribute *lexical_userdef_property_3_name* and *lexical_userdef_property_3_value* of the reference and the target library cells must be exactly matched, as one of the conditions when the tool is trying to find the library cell matching pairs.

This app-option is only effective when *libra.complib.match_full_cell_name* is set to *"false"*.

See Also

- [compare_libraries](#)
- [classify_lib_cell_attributes](#)
- [report_lib_cell_classification_attributes](#)
- [set_app_options](#)
- [report_app_options](#)

libra.complib.match_lexical_well_substrate_bias_architecture

Use the lexical attribute *lexical_well_substrate_bias_architecture* as a matching condition to create the library cell matching pairs for the command *compare_libraries*.

Data Types

Boolean

Default `true`

Description

This option controls how to use lexical attributes to create library cell matching pairs for comparison in the command *compare_libraries*.

When this app-option is *"true"*, the value of the lexical attribute *lexical_well_substrate_bias_architecture* of the reference and the target library cells must be exactly matched, as one of the conditions when the tool is trying to find the library cell matching pairs.

This app-option is only effective when *libra.complib.match_full_cell_name* is set to *"false"*.

See Also

- [compare_libraries](#)
- [classify_lib_cell_attributes](#)
- [report_lib_cell_classification_attributes](#)
- [set_app_options](#)
- [report_app_options](#)

libra.family.lexical_cell_height

Use the lexical attribute *lexical_cell_height* as one of the separation conditions to group library cells into initial library cell families.

Data Types

Boolean

Default false

Description

This option controls how to use lexical attributes to group library cells into initial families with command *identify_lib_cell_families*.

When this app-option is "*true*", the value of the lexical attribute *lexical_cell_height* of the library cells will be checked. Library cells with different attribute values will be separated into different initial library cell families.

See Also

- [identify_lib_cell_families](#)
- [remove_lib_cell_family](#)
- [classify_lib_cell_attributes](#)
- [report_lib_cell_classification_attributes](#)
- [set_app_options](#)
- [report_app_options](#)

libra.family.lexical_cell_prefix_name

Use the lexical attribute *lexical_cell_prefix_name* as one of the separation conditions to group library cells into initial library cell families.

Data Types

Boolean

Default false

Description

This option controls how to use lexical attributes to group library cells into initial families with command *identify_lib_cell_families*.

When this app-option is *"true"*, the value of the lexical attribute *lexical_cell_prefix_name* of the library cells will be checked. Library cells with different attribute values will be separated into different initial library cell families.

See Also

- [identify_lib_cell_families](#)
- [remove_lib_cell_family](#)
- [classify_lib_cell_attributes](#)
- [report_lib_cell_classification_attributes](#)
- [set_app_options](#)
- [report_app_options](#)

libra.family.lexical_cell_tile_width

Use the lexical attribute *lexical_cell_tile_width* as one of the separation conditions to group library cells into initial library cell families.

Data Types

Boolean

Default false

Description

This option controls how to use lexical attributes to group library cells into initial families with command *identify_lib_cell_families*.

When this app-option is *"true"*, the value of the lexical attribute *lexical_cell_tile_width* of the library cells will be checked. Library cells with different attribute values will be separated into different initial library cell families.

See Also

- [identify_lib_cell_families](#)
- [remove_lib_cell_family](#)
- [classify_lib_cell_attributes](#)
- [report_lib_cell_classification_attributes](#)
- [set_app_options](#)
- [report_app_options](#)

libra.family.lexical_circuit_clockgate

Use the lexical attribute *lexical_circuit_clockgate* as one of the separation conditions to group library cells into initial library cell families.

Data Types

Boolean

Default false

Description

This option controls how to use lexical attributes to group library cells into initial families with command *identify_lib_cell_families*.

When this app-option is "*true*", the value of the lexical attribute *lexical_circuit_clockgate* of the library cells will be checked. Library cells with different attribute values will be separated into different initial library cell families.

See Also

- [identify_lib_cell_families](#)
- [remove_lib_cell_family](#)
- [classify_lib_cell_attributes](#)
- [report_lib_cell_classification_attributes](#)
- [set_app_options](#)
- [report_app_options](#)

libra.family.lexical_circuit_device_sizing_variant

Use the lexical attribute *lexical_circuit_device_sizing_variant* as one of the separation conditions to group library cells into initial library cell families.

Data Types

Boolean

Default false

Description

This option controls how to use lexical attributes to group library cells into initial families with command *identify_lib_cell_families*.

When this app-option is *"true"*, the value of the lexical attribute *lexical_circuit_device_sizing_variant* of the library cells will be checked. Library cells with different attribute values will be separated into different initial library cell families.

See Also

- [identify_lib_cell_families](#)
- [remove_lib_cell_family](#)
- [classify_lib_cell_attributes](#)
- [report_lib_cell_classification_attributes](#)
- [set_app_options](#)
- [report_app_options](#)

libra.family.lexical_circuit_eco

Use the lexical attribute *lexical_circuit_eco* as one of the separation conditions to group library cells into initial library cell families.

Data Types

Boolean

Default false

Description

This option controls how to use lexical attributes to group library cells into initial families with command *identify_lib_cell_families*.

When this app-option is *"true"*, the value of the lexical attribute *lexical_circuit_eco* of the library cells will be checked. Library cells with different attribute values will be separated into different initial library cell families.

See Also

- [identify_lib_cell_families](#)
- [remove_lib_cell_family](#)
- [classify_lib_cell_attributes](#)
- [report_lib_cell_classification_attributes](#)
- [set_app_options](#)
- [report_app_options](#)

libra.family.lexical_circuit_layout_density_variant

Use the lexical attribute *lexical_circuit_layout_density_variant* as one of the separation conditions to group library cells into initial library cell families.

Data Types

Boolean

Default false

Description

This option controls how to use lexical attributes to group library cells into initial families with command *identify_lib_cell_families*.

When this app-option is "*true*", the value of the lexical attribute *lexical_circuit_layout_density_variant* of the library cells will be checked. Library cells with different attribute values will be separated into different initial library cell families.

See Also

- [identify_lib_cell_families](#)
- [remove_lib_cell_family](#)
- [classify_lib_cell_attributes](#)
- [report_lib_cell_classification_attributes](#)
- [set_app_options](#)
- [report_app_options](#)

libra.family.lexical_circuit_layout_variant_1

Use the lexical attribute *lexical_circuit_layout_variant_1* as one of the separation conditions to group library cells into initial library cell families.

Data Types

Boolean

Default false

Description

This option controls how to use lexical attributes to group library cells into initial families with command *identify_lib_cell_families*.

When this app-option is *"true"*, the value of the lexical attribute *lexical_circuit_layout_variant_1* of the library cells will be checked. Library cells with different attribute values will be separated into different initial library cell families.

See Also

- [identify_lib_cell_families](#)
- [remove_lib_cell_family](#)
- [classify_lib_cell_attributes](#)
- [report_lib_cell_classification_attributes](#)
- [set_app_options](#)
- [report_app_options](#)

libra.family.lexical_circuit_layout_variant_2

Use the lexical attribute *lexical_circuit_layout_variant_2* as one of the separation conditions to group library cells into initial library cell families.

Data Types

Boolean

Default false

Description

This option controls how to use lexical attributes to group library cells into initial families with command *identify_lib_cell_families*.

When this app-option is *"true"*, the value of the lexical attribute *lexical_circuit_layout_variant_2* of the library cells will be checked. Library cells with different attribute values will be separated into different initial library cell families.

See Also

- [identify_lib_cell_families](#)
- [remove_lib_cell_family](#)
- [classify_lib_cell_attributes](#)
- [report_lib_cell_classification_attributes](#)
- [set_app_options](#)
- [report_app_options](#)

libra.family.lexical_circuit_multibit

Use the lexical attribute *lexical_circuit_multibit* as one of the separation conditions to group library cells into initial library cell families.

Data Types

Boolean

Default false

Description

This option controls how to use lexical attributes to group library cells into initial families with command *identify_lib_cell_families*.

When this app-option is "*true*", the value of the lexical attribute *lexical_circuit_multibit* of the library cells will be checked. Library cells with different attribute values will be separated into different initial library cell families.

See Also

- [identify_lib_cell_families](#)
- [remove_lib_cell_family](#)
- [classify_lib_cell_attributes](#)
- [report_lib_cell_classification_attributes](#)
- [set_app_options](#)
- [report_app_options](#)

libra.family.lexical_circuit_topology_variant

Use the lexical attribute *lexical_circuit_topology_variant* as one of the separation conditions to group library cells into initial library cell families.

Data Types

Boolean

Default false

Description

This option controls how to use lexical attributes to group library cells into initial families with command *identify_lib_cell_families*.

When this app-option is *"true"*, the value of the lexical attribute *lexical_circuit_topology_variant* of the library cells will be checked. Library cells with different attribute values will be separated into different initial library cell families.

See Also

- [identify_lib_cell_families](#)
- [remove_lib_cell_family](#)
- [classify_lib_cell_attributes](#)
- [report_lib_cell_classification_attributes](#)
- [set_app_options](#)
- [report_app_options](#)

libra.family.lexical_device_channel_length

Use the lexical attribute *lexical_device_channel_length* as one of the separation conditions to group library cells into initial library cell families.

Data Types

Boolean

Default false

Description

This option controls how to use lexical attributes to group library cells into initial families with command *identify_lib_cell_families*.

When this app-option is *"true"*, the value of the lexical attribute *lexical_device_channel_length* of the library cells will be checked. Library cells with different attribute values will be separated into different initial library cell families.

See Also

- [identify_lib_cell_families](#)
- [remove_lib_cell_family](#)
- [classify_lib_cell_attributes](#)
- [report_lib_cell_classification_attributes](#)
- [set_app_options](#)
- [report_app_options](#)

libra.family.lexical_device_threshold_voltage

Use the lexical attribute *lexical_device_threshold_voltage* as one of the separation conditions to group library cells into initial library cell families.

Data Types

Boolean

Default false

Description

This option controls how to use lexical attributes to group library cells into initial families with command *identify_lib_cell_families*.

When this app-option is "*true*", the value of the lexical attribute *lexical_device_threshold_voltage* of the library cells will be checked. Library cells with different attribute values will be separated into different initial library cell families.

See Also

- [identify_lib_cell_families](#)
- [remove_lib_cell_family](#)
- [classify_lib_cell_attributes](#)
- [report_lib_cell_classification_attributes](#)
- [set_app_options](#)
- [report_app_options](#)

libra.family.lexical_device_type

Use the lexical attribute *lexical_device_type* as one of the separation conditions to group library cells into initial library cell families.

Data Types

Boolean

Default false

Description

This option controls how to use lexical attributes to group library cells into initial families with command *identify_lib_cell_families*.

When this app-option is *"true"*, the value of the lexical attribute *lexical_device_type* of the library cells will be checked. Library cells with different attribute values will be separated into different initial library cell families.

See Also

- [identify_lib_cell_families](#)
- [remove_lib_cell_family](#)
- [classify_lib_cell_attributes](#)
- [report_lib_cell_classification_attributes](#)
- [set_app_options](#)
- [report_app_options](#)

libra.family.lexical_drive1_name

Use the lexical attribute *lexical_drive1_name* as one of the separation conditions to group library cells into initial library cell families.

Data Types

Boolean

Default false

Description

This option controls how to use lexical attributes to group library cells into initial families with command *identify_lib_cell_families*.

When this app-option is *"true"*, the value of the lexical attribute *lexical_drive1_name* of the library cells will be checked. Library cells with different attribute values will be separated into different initial library cell families.

See Also

- [identify_lib_cell_families](#)
- [remove_lib_cell_family](#)
- [classify_lib_cell_attributes](#)
- [report_lib_cell_classification_attributes](#)
- [set_app_options](#)
- [report_app_options](#)

libra.family.lexical_drive2_name

Use the lexical attribute *lexical_drive2_name* as one of the separation conditions to group library cells into initial library cell families.

Data Types

Boolean

Default false

Description

This option controls how to use lexical attributes to group library cells into initial families with command *identify_lib_cell_families*.

When this app-option is "*true*", the value of the lexical attribute *lexical_drive2_name* of the library cells will be checked. Library cells with different attribute values will be separated into different initial library cell families.

See Also

- [identify_lib_cell_families](#)
- [remove_lib_cell_family](#)
- [classify_lib_cell_attributes](#)
- [report_lib_cell_classification_attributes](#)
- [set_app_options](#)
- [report_app_options](#)

libra.family.lexical_electrical_delay_opt_variant

Use the lexical attribute *lexical_electrical_delay_opt_variant* as one of the separation conditions to group library cells into initial library cell families.

Data Types

Boolean

Default false

Description

This option controls how to use lexical attributes to group library cells into initial families with command *identify_lib_cell_families*.

When this app-option is *"true"*, the value of the lexical attribute *lexical_electrical_delay_opt_variant* of the library cells will be checked. Library cells with different attribute values will be separated into different initial library cell families.

See Also

- [identify_lib_cell_families](#)
- [remove_lib_cell_family](#)
- [classify_lib_cell_attributes](#)
- [report_lib_cell_classification_attributes](#)
- [set_app_options](#)
- [report_app_options](#)

libra.family.lexical_electrical_hold_opt_variant

Use the lexical attribute *lexical_electrical_hold_opt_variant* as one of the separation conditions to group library cells into initial library cell families.

Data Types

Boolean

Default false

Description

This option controls how to use lexical attributes to group library cells into initial families with command *identify_lib_cell_families*.

When this app-option is *"true"*, the value of the lexical attribute *lexical_electrical_hold_opt_variant* of the library cells will be checked. Library cells with different attribute values will be separated into different initial library cell families.

See Also

- [identify_lib_cell_families](#)
- [remove_lib_cell_family](#)
- [classify_lib_cell_attributes](#)
- [report_lib_cell_classification_attributes](#)
- [set_app_options](#)
- [report_app_options](#)

libra.family.lexical_electrical_property_variant

Use the lexical attribute *lexical_electrical_property_variant* as one of the separation conditions to group library cells into initial library cell families.

Data Types

Boolean

Default false

Description

This option controls how to use lexical attributes to group library cells into initial families with command *identify_lib_cell_families*.

When this app-option is "*true*", the value of the lexical attribute *lexical_electrical_property_variant* of the library cells will be checked. Library cells with different attribute values will be separated into different initial library cell families.

See Also

- [identify_lib_cell_families](#)
- [remove_lib_cell_family](#)
- [classify_lib_cell_attributes](#)
- [report_lib_cell_classification_attributes](#)
- [set_app_options](#)
- [report_app_options](#)

libra.family.lexical_electrical_setup_opt_variant

Use the lexical attribute *lexical_electrical_setup_opt_variant* as one of the separation conditions to group library cells into initial library cell families.

Data Types

Boolean

Default false

Description

This option controls how to use lexical attributes to group library cells into initial families with command *identify_lib_cell_families*.

When this app-option is *"true"*, the value of the lexical attribute *lexical_electrical_setup_opt_variant* of the library cells will be checked. Library cells with different attribute values will be separated into different initial library cell families.

See Also

- [identify_lib_cell_families](#)
- [remove_lib_cell_family](#)
- [classify_lib_cell_attributes](#)
- [report_lib_cell_classification_attributes](#)
- [set_app_options](#)
- [report_app_options](#)

libra.family.lexical_electrical_slew_opt_variant

Use the lexical attribute *lexical_electrical_slew_opt_variant* as one of the separation conditions to group library cells into initial library cell families.

Data Types

Boolean

Default false

Description

This option controls how to use lexical attributes to group library cells into initial families with command *identify_lib_cell_families*.

When this app-option is *"true"*, the value of the lexical attribute *lexical_electrical_slew_opt_variant* of the library cells will be checked. Library cells with different attribute values will be separated into different initial library cell families.

See Also

- [identify_lib_cell_families](#)
- [remove_lib_cell_family](#)
- [classify_lib_cell_attributes](#)
- [report_lib_cell_classification_attributes](#)
- [set_app_options](#)
- [report_app_options](#)

libra.family.lexical_electrical_symmetry_variant

Use the lexical attribute *lexical_electrical_symmetry_variant* as one of the separation conditions to group library cells into initial library cell families.

Data Types

Boolean

Default false

Description

This option controls how to use lexical attributes to group library cells into initial families with command *identify_lib_cell_families*.

When this app-option is "*true*", the value of the lexical attribute *lexical_electrical_symmetry_variant* of the library cells will be checked. Library cells with different attribute values will be separated into different initial library cell families.

See Also

- [identify_lib_cell_families](#)
- [remove_lib_cell_family](#)
- [classify_lib_cell_attributes](#)
- [report_lib_cell_classification_attributes](#)
- [set_app_options](#)
- [report_app_options](#)

libra.family.lexical_electrical_timing_arc_variant

Use the lexical attribute *lexical_electrical_timing_arc_variant* as one of the separation conditions to group library cells into initial library cell families.

Data Types

Boolean

Default false

Description

This option controls how to use lexical attributes to group library cells into initial families with command *identify_lib_cell_families*.

When this app-option is *"true"*, the value of the lexical attribute *lexical_electrical_timing_arc_variant* of the library cells will be checked. Library cells with different attribute values will be separated into different initial library cell families.

See Also

- [identify_lib_cell_families](#)
- [remove_lib_cell_family](#)
- [classify_lib_cell_attributes](#)
- [report_lib_cell_classification_attributes](#)
- [set_app_options](#)
- [report_app_options](#)

libra.family.lexical_library_prefix_name

Use the lexical attribute *lexical_library_prefix_name* as one of the separation conditions to group library cells into initial library cell families.

Data Types

Boolean

Default false

Description

This option controls how to use lexical attributes to group library cells into initial families with command *identify_lib_cell_families*.

When this app-option is *"true"*, the value of the lexical attribute *lexical_library_prefix_name* of the library cells will be checked. Library cells with different attribute values will be separated into different initial library cell families.

See Also

- [identify_lib_cell_families](#)
- [remove_lib_cell_family](#)
- [classify_lib_cell_attributes](#)
- [report_lib_cell_classification_attributes](#)
- [set_app_options](#)
- [report_app_options](#)

libra.family.lexical_userdef_property_1

Use the lexical attribute *lexical_userdef_property_1_name* and *lexical_userdef_property_1_value* as one of the separation conditions to group library cells into initial library cell families.

Data Types

Boolean

Default false

Description

This option controls how to use lexical attributes to group library cells into initial families with command *identify_lib_cell_families*.

When this app-option is "*true*", the value of the lexical attributes *lexical_userdef_property_1_name* and *lexical_userdef_property_1_value* of the library cells will be checked. Library cells with different attribute values will be separated into different initial library cell families.

See Also

- [identify_lib_cell_families](#)
- [remove_lib_cell_family](#)
- [classify_lib_cell_attributes](#)
- [report_lib_cell_classification_attributes](#)
- [set_app_options](#)
- [report_app_options](#)

libra.family.lexical_userdef_property_2

Use the lexical attribute *lexical_userdef_property_2_name* and *lexical_userdef_property_2_value* as one of the separation conditions to group library cells into initial library cell families.

Data Types

Boolean

Default false

Description

This option controls how to use lexical attributes to group library cells into initial families with command *identify_lib_cell_families*.

When this app-option is *"true"*, the value of the lexical attributes *lexical_userdef_property_2_name* and *lexical_userdef_property_2_value* of the library cells will be checked. Library cells with different attribute values will be separated into different initial library cell families.

See Also

- [identify_lib_cell_families](#)
- [remove_lib_cell_family](#)
- [classify_lib_cell_attributes](#)
- [report_lib_cell_classification_attributes](#)
- [set_app_options](#)
- [report_app_options](#)

libra.family.lexical_userdef_property_3

Use the lexical attribute *lexical_userdef_property_3_name* and *lexical_userdef_property_3_value* as one of the separation conditions to group library cells into initial library cell families.

Data Types

Boolean

Default `false`

Description

This option controls how to use lexical attributes to group library cells into initial families with command *identify_lib_cell_families*.

When this app-option is *"true"*, the value of the lexical attributes *lexical_userdef_property_3_name* and *lexical_userdef_property_3_value* of the library cells will be checked. Library cells with different attribute values will be separated into different initial library cell families.

See Also

- [identify_lib_cell_families](#)
- [remove_lib_cell_family](#)
- [classify_lib_cell_attributes](#)
- [report_lib_cell_classification_attributes](#)
- [set_app_options](#)
- [report_app_options](#)

libra.family.lexical_well_substrate_bias_architecture

Use the lexical attribute *lexical_well_substrate_bias_architecture* as one of the separation conditions to group library cells into initial library cell families.

Data Types

Boolean

Default `false`

Description

This option controls how to use lexical attributes to group library cells into initial families with command *identify_lib_cell_families*.

When this app-option is *"true"*, the value of the lexical attribute *lexical_well_substrate_bias_architecture* of the library cells will be checked. Library cells with different attribute values will be separated into different initial library cell families.

See Also

- [identify_lib_cell_families](#)
- [remove_lib_cell_family](#)
- [classify_lib_cell_attributes](#)
- [report_lib_cell_classification_attributes](#)
- [set_app_options](#)
- [report_app_options](#)

libra.lex_classification.keep_partial_results

To determine if the partial lexical classification results should be kept or discarded.

Data Types

Boolean

Default `true`

Description

This option controls how the lexical classification results to be processed.

After the lexical classification is processed on a library cell, the tool performs a sanity check to check if the lexical classification is successful or is possibly partially successful.

When this app-option is *"true"*, even the tool detects the lexical classification may be only partially successful, those partial results will still be stored.

When this app-option is *"false"*, the results will be stored only when the sanity check is passed.

See Also

- [classify_lib_cell_attributes](#)
- [report_lib_cell_classification_attributes](#)
- [report_lexical_unclassified_lib_cells](#)
- [set_app_options](#)
- [report_app_options](#)

libra.redundancies.across_families

Specifies that the library cell families that have the same functional properties are analyzed together during redundancy and gap analysis.

Data Types

Boolean

Default `"true"`

Description

This option controls whether the library analysis for redundancy and gap should analyze the library cell families that have the same functional properties together. Setting this

option to *"false"* will ensure that the families with the same functional properties will be analyzed separately. The total number of redundant, outlier, equivalent, and gap library cells that are found during analysis can be different based on the value of this option.

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design. Use *report_app_options* to show the value of application options. This application option is global only.

See Also

- [find_library_redundancies](#)
- [find_library_gaps](#)
- [report_app_options](#)
- [set_app_options](#)

libra.redundancies.area_costing

Specifies that the redundancy analysis should consider the area as one of the cost metrics while comparing the library cells.

Data Types

Boolean

Default "true"

Description

This option controls whether the redundancy analysis should use area as one of the cost metrics during comparison of the library cells. Setting this option to *"false"* will ensure that area cost will not be considered during redundancy analysis. The total number of redundant, outlier, and equivalent library cells that are found during analysis can be different based on the value of this option.

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design. Use *report_app_options* to show the value of application options. This application option is global only.

See Also

- [find_library_redundancies](#)
- [report_app_options](#)
- [set_app_options](#)

libra.redundancies.average_leakage_costing

Specifies that the redundancy analysis should consider the leakage as one of the cost metrics while comparing the library cells.

Data Types

Boolean

Default "true"

Description

This option controls whether the redundancy analysis should use leakage as one of the cost metrics during comparison of the library cells. Setting this option to *"false"* will ensure that leakage cost will not be considered during redundancy analysis. The total number of redundant, outlier, and equivalent library cells that are found during analysis can be different based on the value of this option.

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design. Use *report_app_options* to show the value of application options. This application option is global only.

See Also

- [find_library_redundancies](#)
- [report_app_options](#)
- [set_app_options](#)

libra.redundancies.dynamic_power_costing

Specifies that the redundancy analysis should consider the dynamic power as one of the cost metrics while comparing the library cells.

Data Types

Boolean

Default "true"

Description

This option controls whether the redundancy analysis should use dynamic power as one of the cost metrics during comparison of the library cells. Setting this option to *"false"* will ensure that dynamic power cost will not be considered during redundancy analysis. The total number of redundant, outlier, and equivalent library cells that are found during analysis can be different based on the value of this option.

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design. Use *report_app_options* to show the value of application options. This application option is global only.

See Also

- [find_library_redundancies](#)
- [report_app_options](#)
- [set_app_options](#)

libra.redundancies.ignore_physical_variants

Specifies that the physical variant library cells will be ignored during redundancy and gap analysis.

Data Types

Boolean

Default "true"

Description

This option controls the handling of physical variant library cells during redundancy and gap analysis. Setting this option to *"false"* will ensure that physical variant library cells will be included in the analysis. The total number of redundant, outlier, equivalent, and gap library cells that are found during analysis can be different based on the value of this option.

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design. Use *report_app_options* to show the value of application options. This application option is global only.

See Also

- [find_library_redundancies](#)
- [find_library_gaps](#)
- [report_app_options](#)
- [set_app_options](#)

libra.redundancies.timing_costing

Specifies that the redundancy analysis should consider the timing as one of the cost metrics while comparing the library cells.

Data Types

Boolean

Default "true"

Description

This option controls whether the redundancy analysis should use timing as one of the cost metrics during comparison of the library cells. Setting this option to *"false"* will ensure that timing cost will not be considered during redundancy analysis. The total number of redundant, outlier, and equivalent library cells that are found during analysis can be different based on the value of this option.

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design. Use *report_app_options* to show the value of application options. This application option is global only.

See Also

- [find_library_redundancies](#)
- [report_app_options](#)
- [set_app_options](#)

link.allow_blasted_bus_reconstruction

Enables the linker to reconstruct bit blasted busses.

Data Types

String

Default "off"

Description

Setting this application option allows linking to recognize bit-blasted buses by pin names. If the pin names follow a specific pattern, the tool infers a bus when encountering the individual bits.

Typically, the RTL pin names and the logic library pin names should match. Signals are defined the same way in both the RTL and the library. They are defined as buses or a

bus is defined by its individual wires. Occasionally, mismatches occur during design development; for example, when you transfer the design from one technology to another. To fix the mismatches, you can set this application option before the design is read.

To determine the value of this application option, use the *get_app_option_value -name link.allow_blasted_bus_reconstruction.3* command.

link.allow_period_to_match_underscore

Enables the linker to allow a period (.) as an alternative to an underscore (_) when doing port name matching.

Data Types

Boolean

Default false

Description

During linking, if named port mapping is used in the cell instance statement, the linker resolves port connections based on the port names. If the linker is unable to find a matching port name, and the *link.allow_period_to_match_underscore* app option is set to *true*, the linker replaces, for comparison purposes, the "." characters in the port name from the cell instance statement with the "_" character to see if there is a match. This application option has no effect when positional port mapping is used instead.

The following Verilog example shows the effect of this application option on the *link* command:

```
module top ( B , A, Y);
    input A,B;
    output Y;
    wire Y;
    mid midl (.\B.X (B), .A(A), .Y(Y));
endmodule

module mid (B_X, A, Y);
    input B_X;
    input A;
    output Y;
    assign Y = B_X & A;
endmodule
```

The following scenarios assume that you have already read in the Verilog file and are running the *link* command.

Scenario 1:

set_app_options -name link.allow_period_to_match_underscore -value false

The linker attempts to resolve the port connections but is unable to find a port with the name "B.X" in the port list of the mid design. This causes the linker to issue a LNK-008 error.

```
Error: Can't find inout port 'B.X' on reference to 'mid' in 'top'.  
(LNK-008)
```

This is expected because the instantiation of mid1 uses B.X as the port name but the actual port name is B_X.

Scenario 2:

```
set_app_options -name link.allow_period_to_match_underscore -value true
```

The linker takes "B.X" and replaces all '.' characters with the '_' character. For this comparison, the linker will compare "B_X" against "B_X". In this case, the port names match and the design links successfully.

To determine the value of this application option, use the *get_app_option_value -name link.allow_period_to_match_underscore* command.

link.allow_port_array_interface_hybrid_matching

Controls if the top level module interface port array interface is to be mapped to reference module using hybrid matching.

Data Types

Boolean

Default false

Description

The option controls if the top level module interface port array interface is to reference module using hybrid matching. In this mode, the interface name is initially matched by name matching and the individual ports in the interface array are mapped by the Port order.

By default (*false*), while doing the set_top_module doesnot map the port array interface using hybrid method.

To use the mapping the interace port array of the module to map to reference module set the application option to *true*.

See Also

- [get_app_option_value](#)
- [get_app_options](#)

- [analyze](#)
- [elaborate](#)
- [set_top_module](#)

link.allow_square_bracket_to_match_underscore

Enables the linker to allow a square bracket([]) as an alternative to an underscore (_) when doing port name matching.

Data Types

Boolean

Default false

Description

During linking, the linker resolves port connections based on port names when named port mapping is used in cell instantiation. In the default mode, the linker looks for an exact match of the port names. Setting the *link.allow_square_bracket_to_match_underscore* application option to *true*, allows the linker to also match the "[]" characters with the "_" character. This application option has no effect when positional port mapping is used. This application option should be used only for matching modport arrays.

The following SystemVerilog example shows the effect of this application option on the *link_block* command. *Note* - this example requires another related application option, *link.allow_period_to_match_underscore*, to also be set to *true*. This is because this example contains both "[]" and "." characters in the original portname.

```
interface AA;

    wire valid;
    wire ready;

    modport slave (input valid, output ready);
    modport master (output valid, input ready);

endinterface

module subblk (
    AA.master aa_master[0:3],
    AA.slave aa_slave[0:3]
);

    // subblk logic
    assign aa_master[0].valid=aa_slave[0].valid & aa_slave[1].valid ;
    assign aa_master[1].valid=aa_slave[1].valid & aa_slave[2].valid ;
    assign aa_master[2].valid=aa_slave[2].valid & aa_slave[3].valid ;
    assign aa_master[3].valid=aa_slave[3].valid & aa_slave[0].valid ;
```

|

```

endmodule

module top ( input clk_a, input reasrt_n);
  AA aa_vec[0:3] ();
  subblk U_subblk ( .aa_master
(aa_vec.master), .aa_slave(aa_vec.slave));
endmodule

```

The following scenarios assume that you have already read in the SystemVerilog file and are running the *link_block* command.

Scenario 1:

```

set_app_options \
  -name link.allow_square_bracket_to_match_underscore -value false
set_app_options \
  -name link.allow_period_to_match_underscore -value false

```

The linker attempts to resolve the port connections but is unable to find a port with the name "aa_master[0].valid" in the port list of the subblk design. This causes the linker to issue a LINK-1 error.

```

Error: Can't find inout port 'aa_master[0].valid' on reference
      to 'subblk' in 'top'. (LINK-1)

```

This is expected because the instantiation of 'subblk' uses 'aa_master[0].valid' as the port name but the actual port name is 'aa_master_0__valid'.

Scenario 2:

```

set_app_options \
  -name link.allow_square_bracket_to_match_underscore -value true
set_app_options \
  -name link.allow_period_to_match_underscore -value true

```

The linker now matches all '.', and '[' characters with the '_' character, and "aa_master[0].valid" matches "aa_master_0__valid", and the design links successfully.

See Also

- [link_block](#)

link.bit_blast_naming_style

Specifies the naming format for blasted bus bits which will be used during linking to resolve differences of bus bit naming styles between instance and its reference.

Data Types

list_of_string

Default %s(%d) %s_%d_**Description**

The linker will not bind any verilog instance to its reference when blasted bus ports across the two don't match which may be due to different naming styles of bit blasted ports. In order to enable linker to understand compatible bit blasted bus ports having different naming styles this app option is used.

This app option is a list of string where each value must contain one occurrence each of %s and %d and it must start with %s followed by the %d. Later when multi dimension bus will be supported, this format will be enhanced.

Examples

Following example will allow verilog instance with port A[0] and A[1] to match the reference with ports A_0 and A_1.

```
prompt> set_app_options -name link.bit_blast_naming_style -value %s_%d
link.bit_blast_naming_style %s_%d
```

Following example is for different references having different naming styles.

```
prompt> set_app_options -name link.bit_blast_naming_style -value {%s_%d
%s(%d)}
link.bit_blast_naming_style {%s_%d %s(%d)}
```

link.bit_blasted_bus_linking

Controls if the module port interface is to be mapped to bit blasted port interface of the reference module.

Data Types

Boolean

Default false**Description**

By default (*false*), while doing the set_top_module doesnot map the module interface to the bit blasted reference module interface.

To use the mapping the bus port of the module to map to reference bit blasted port, set the application option to *true*.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [analyze](#)
- [elaborate](#)
- [set_top_module](#)

link.do_bus_width_mismatch

Checks for width mismatch of bus ports during link or set_top_module commands.

Data Types

Boolean

Default false

Description

By default (*false*), link or set_top_module commands do not check for bus width mismatches between instance and reference if any.

To check for width mismatch of bus ports, set the application option to *true*. If instance bus port width is more than reference bus port width, then this app option will check for such cases, record and repair them based on the configuration.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [analyze](#)
- [elaborate](#)
- [set_top_module](#)

link.error_on_net_port_width_mismatch

Checks and errors if the net width is less than the port width of the module interface during initial link.

Data Types

Boolean

Default false

Description

The options detects logic error by detecting the case when the net width is less than the port width during initial link.

By default, the tool does not checks for the net width and port width mismatch during the initial linking a block.

To enable this check during initial linking, set this option to *true*.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)

link.force_case

Controls the case-sensitive or case-insensitive behavior of the *link_block* command.

Data Types

string

Default none

Description

Controls the case-sensitive or case-insensitive matching policy to be used on HDL identifiers considered during a *link_block* command. The value of this application option can be set to *none* (the default) or *case_insensitive*.

By default, the reference and the design cells to be linked are checked to ascertain the alphabet-case matching policy of the HDL input format that created that particular cell. Then, the least-sensitive matching rule is applied to determine whether the design cell resolves the reference. For example, a VHDL reference is linked case-insensitively and a Verilog reference is linked case-sensitively, while mixed language linkage candidates match case-insensitively. To override this behavior, you can set the value of the *link.force_case* app option to *case_insensitive*.

The appropriate matching policy applies to all the HDL identifiers that must be matched by the link: design template, port, and parameter names. It does not apply to the (canonical forms of) data values in any parameter override, which (for string data) are always case-sensitive. It also does not apply to library and architecture identifiers, which are always case-insensitive. When link candidates are built by elaboration, case-forcing does not change the HDL identifiers of the design or reference cells that are built.

Setting this app option to *case_insensitive* value can lead to results which are inconsistent with the default rules, since a forced rule applies throughout the linked hierarchy.

To determine the value of this application option, use the *get_app_option_value - name link.force_case* command.

See Also

- [link_block](#)

link.interface_variable_delimiter

Controls if the top level module interface port array interface is to be mapped to reference module using hybrid matching and variable delimiter solution .

Data Types

Boolean

Default false

Description

The option controls if the top level module interface port array interface is to reference module using hybrid matching and using the variable delimiter solution. In this mode, the interface name is initially matched by name matching and the individual ports in the interface array are mapped by the Port order.

By default (*false*), while doing the *set_top_module* does not map the port array interface using hybrid method.

To use the mapping the interface port array of the module to map to reference module set the application option to *true*.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [analyze](#)

- [elaborate](#)
- [set_top_module](#)

link.prefer_frame

Controls whether the tool uses a stub module in the Verilog source or a 'frozen' version of the block from the reference library list. A block is considered frozen if it has a FRAME view. This action only occurs during the initial link of newly read Verilog.

When a 'frozen' block is used, the linker uses the view switch list to decide which view of that block to link to. This can be changed with the *set_view_switch_list* command. Individual instances can be changed to reference a different view with the *change_abstract*, *change_view* or *set_reference* commands.

Data Types

Boolean

Default true

Description

By default (*true*), if a block has a frame view, the tool uses the frame view when linking a newly read Verilog netlist.

To use the Verilog module stubs instead of the frame view, set this application option to *false*.

See Also

- [change_abstract](#)
- [change_view](#)
- [get_app_option_value](#)
- [get_app_options](#)
- [get_view_switch_list](#)
- [link_block](#)
- [read_verilog](#)
- [report_app_options](#)
- [set_app_options](#)

- [set_reference](#)
- [set_view_switch_list](#)

link.reference_limit

Specifies the maximum number of detailed unresolved reference errors to display for each block when linking the design.

Data Types

integer

Default 12

Description

In many cases, many link errors have the same root cause, such as a missing leaf cell library. Subsequent error messages do not provide additional information to resolve the issue. This option allows you to control the maximum number of errors displayed for each block before the tool moves to the next design in the physical design hierarchy. After the specified limit has been reached, the tool outputs a summary of the number of unresolved references.

By default, the tool displays a maximum of 12 detailed unresolved reference errors.

To display all unresolved reference error messages, set this option to -1.

See Also

- [link_block](#)
- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)

link.require_physical

Controls whether subblocks must have physical information or not.

Data Types

Boolean

Default false

Description

To create the physical hierarchy for a block, the tool requires that all subblocks have physical information.

By default (*false*), the tool can link a block with subblocks that are missing physical information. This can happen for an improperly prepared library cell library containing some library cells that do not have a physical description. However, subsequent commands might give unexpected results with these designs.

Tool can verify that all subblocks have physical information when it links a block by setting this option to *true*.

See Also

- [link_block](#)
- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)

link.truncate_bus_pin_width

Truncates wide based instance-pin to match the size of narrow reference based port.

Data Types

Boolean

Default `false`

Description

The option controls if MSBs of the wide based instance-pin are discarded to match the size of the narrow reference based port.

By default (*false*) linker errors out when instance has more bus width than reference.

To truncate wide based instance-pin to the size of narrow reference based port set the application option to *true*.

See Also

- [get_app_option_value](#)
- [get_app_options](#)

- [set_top_module](#)
- [link_block](#)
- [set_reference](#)

link.undo_enabled

Controls whether to keep undo information so the *link_block* command can be undone with the *undo* command.

Data Types

Boolean

Default false

Description

The *link_block* command makes considerable changes when first linking a design read from Verilog. By default (*false*), these changes cannot be undone by the *undo* command.

You can enable the changes made by the *link_block* command to be undone by setting this option to *true*. However, this can cause the undo cache to grow very large. If there are problems when linking the design, you typically reread the design from Verilog, so there is less value in allowing the *link_block* command to be undone.

For more information, see the *undo* man page.

See Also

- [link_block](#)
- [redo](#)
- [undo](#)
- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)

link.use_session_ref_libs

Specifies an opened library whose reference library list should be used as the session scoped reference library list for linking.

Data Types

String

Default ""

Description

This application option takes the name of an opened library whose reference library list should be used as the session scoped reference library list. The linker will link each block using this session scoped reference library list, instead of the block library's reference library list.

This application option automatically resets to an empty string when the library, specified to this application option, is closed.

If this application option is an empty string, the session scoped reference library list is not set. The linker links each block using the block library's reference library list. This is the default linker behavior.

To determine the value of this application option, use the *get_app_option_value - name link.use_session_ref_libs* command.

See Also

- [link_block](#)

link.user_units_from_first_library

Controls whether the tool uses the units from the first library cell library in the reference library list if values are not set explicitly using the *set_user_units* command.

Data Types

Boolean

Default true

Description

By default (*true*), the tool uses the units from the first library cell library in the reference library list unless the units have been set explicitly by the user. This makes the tool consistent with other tools, such as Design Compiler and IC Compiler.

The user units are set once per session, to avoid them being reset unexpectedly.

If you set the units using *set_user_units* command, they are never changed. This is the recommended method, to avoid any dependence on order of reference libraries.

See Also

- [link_block](#)
- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)

link.verbose

Controls whether the tool outputs verbose messages when linking a block.

Data Types

Boolean

Default false

Description

Verbose messages can help you debug link issues, but for most design flows they are unnecessary.

By default, the tool does not issue verbose message when linking a block.

To enable verbose messages during linking, set this option to *true*.

See Also

- [link_block](#)
- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)

m

mpn.buffer_top_level_constant_port_nets

Enables buffering of top level constant port nets.

Data Types

boolean

Default true

Description

This app option will have an effect on Multiple Port Net (MPN) feature. When true, it enables the buffering of top level constant port nets. When false, it disables the buffering of top level constant port nets due to which there will be residual assign statements in the netlist.

See Also

- [set_fix_multiple_port_nets](#)

mpn.buffer_unloaded_constant_port_nets

Enables buffering of unloaded hierarchical constant port nets.

Data Types

boolean

Default true

Description

This app option will have an effect on Multiple Port Net (MPN) feature. When true, it enables the buffering of unloaded hierarchical constant port nets. When false, it disables the buffering due to which there will be residual assign statements but, there will also be no hierarchical constant-driven unloaded buffers in the netlist.

See Also

- [set_fix_multiple_port_nets](#)

mpn.buffer_unloaded_multiple_port_nets

Enables buffering of unloaded hierarchical non constant port nets.

Data Types

boolean

Default true

Description

This app option will have an effect on Multiple Port Net (MPN) feature. When true, it enables the buffering of unloaded hierarchical non constant port nets. When false, it disables the buffering due to which there will be residual assign statements but, there will also be no hierarchical non constant-driven unloaded buffers in the netlist.

See Also

- [set_fix_multiple_port_nets](#)

multibit.banking.across_equivalent_icg

Bank cells across equivalent ICG in integrated physical aware multibit banking.

Data Types

boolean

Default false

Description

If true, bank cells across equivalent ICGs. Equivalent ICGs are one which has same enable logic and functionality.

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

multibit.banking.bank_consecutive_cells

Bank cells in ordered scan chain list and scan cells not in scandef.

Data Types

boolean

Default true

Description

If true, bank only consecutive cells in ordered scan chain and scan cells not in scandef such that formal logic remains same. While banking consecutive cells buffers and even

number of inverters are ignored. By default cells in ordered scan chain list are not banked and scan cells not in scandef are banked randomly.

See Also

- [identify_multibit](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

multibit.banking.enable_clock_pin_level_check

Data Types

boolean

Default false

Description

When this app option is set to true, only cells with the same clock pin level can be banked together. Clock pin level means the level from the clock source to the clock sink. Please note that this app option only takes effect when across ICG banking is enabled.

When multibit.banking.across_equivalent_icg is true, reg1 and reg2 can be banked even if multibit.banking.enable_clock_pin_level_check is true or false.

```
+--- ICG -- reg1 | clock_src-----+--- ICG -- reg2
```

However in below example case, when multibit.banking.across_equivalent_icg is true, reg1 and reg2 cannot be banked if multibit.banking.enable_clock_pin_level_check is true. They can be banked when multibit.banking.across_equivalent_icg is true and multibit.banking.enable_clock_pin_level_check is false (default).

```
+--- ICG -- buffer/ICG/any_logic -- reg1 | clock_src-----+--- ICG -- reg2
```

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

multibit.banking.enable_strict_scan_check

Ignores scan flops which are not part of scandef while doing multibit banking.

Data Types

boolean

Default false

Description

If true, tool ignores scan flops (with valid SI connection) which are not part of scandef while doing multibit banking. By default such flops are considered for banking.

See Also

- [identify_multibit](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

multibit.banking.enable_subblock_optimization

Enables multibit banking or debanking of the objects inside editable subblocks from the top-level in the Transparent Hierarchical Optimization (THO) flow.

Data Types

boolean

Default false

Description

When this app-option is set to true, multibit banking and debanking shall be allowed on the objects inside editable subblocks during THO flow whenever multibit optimization step gets triggered for the design. However, if the subblock object has a SDC constraint set on it, it shall not be subject to multibit banking and debanking to avoid loss of constraint in the respective subblock.

See Also

- [identify_multibit](#)
- [report_multibit](#)

- [report_transformed_registers](#)
- [report_app_options](#)
- [set_app_options](#)

multibit.banking.enable_tns_degradation_estimation

Enable TNS degradation estimation to avoid TNS jump during physical banking by excluding registers from banking.

Data Types

boolean

Default false

Description

When this app option is set to true, banking engine estimates TNS degradation and does not bank the registers that degrade TNS significantly. Please note that when this app option is enabled, banking attempts with high TNS impact will be rejected, which may lead to lower banking ratio.

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)
- [place_opt](#)
- [compile_fusion](#)

multibit.banking.ignore_scan_cstr_list

Specify the type of exception constraints especially for the test pins of the cells which makes the cells can still be considered for banking.

Data Types

string

Default ""

Description

This application option works only when `multibit.common.ignore_cell_with_sdc` is set to true. This application option enables users to precisely specify the type of exception constraints for the test pins, allowing cells to remain eligible for banking. If we only want to ignore the sdc constraints on the test pins of a cell, we can use this app option "`multibit.banking.ignore_scan_cstr_list`". On the other hand, if we want to ignore the ones on all the pins of the cell, we should use the app option "`multibit.common.ignore_cstr_list`" instead. By default, it sets to null which means that registers with any timing exceptions on the test pins will be excluded from banking when `multibit.common.ignore_cell_with_sdc` is set to true.

Supported constraint names: *ANNOTATION*, *IDEAL_NETWORK*, *MULTICYCLE_PATH*, *PATH_GROUP*, *CLOCK_LATENCY*, *MAX_MIN_DELAY*, *FALSE_PATH*

ANNOTATION covers annotation constraints. e.g., `set_annotated_transition`, `set_annotated_delay`, `set_annotated_check`, `set_ideal_transition`, `set_ideal_latency`

IDEAL_NETWORK covers ideal network constraints. e.g., `set_ideal_network`

MULTICYCLE_PATH covers multicycle path constraints. e.g., `set_multicycle_path`

PATH_GROUP covers path group constraints. e.g., `group_path`

CLOCK_LATENCY covers clock latency constraints. e.g., `set_clock_latency`

MAX_MIN_DELAY covers maximum/minimum delay constraints. e.g., `set_max_delay`, `set_min_delay`

FALSE_PATH covers false path constraints. e.g., `set_false_path`

Example:

1. The following example allows registers that ONLY the test pins of cells with ONLY `set_annotated_transition` constraint to be included in banking:

```
set_app_options -name multibit.common.ignore_cell_with_sdc -value true
set_app_options -name multibit.banking.ignore_scan_cstr_list -value "ANNOTATION"
```

1. The following example shows that the cells will not be banked with "`multibit.common.ignore_cstr_list`", but they will be allowed to be banked after applying the second app option "`multibit.banking.ignore_scan_cstr_list`"

```
set_ideal_network reg/D reg/SI set_annotated_transition 10 reg/SI set_app_options -name
multibit.common.ignore_cstr_list -value "IDEAL_NETWORK" set_app_options -name
multibit.banking.ignore_scan_cstr_list -value "ANNOTATION"
```

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

multibit.banking.lcs_consider_multi_si_cells

Considers both internal SI and multi-SI lib-cells while choosing lib-cells for multibit cells.

Data Types

boolean

Default false

Description

If true, in banking step during *place_opt*, tool considers both internal SI and multi-SI lib-cells as candidates for banked multibit cell when both type of lib-cells are available in the library.

See Also

- [identify_multibit](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

multibit.banking.maximum_allowable_distance

The maximum allowable distance in terms of site height for flops to be clustered in the same group.

Data Types

Integer

Default 25

Description

This app option defines the maximum allowable distance in terms of site height for the flops to be clustered into the same group during *identify_multibit* and banking step in *place_opt* and *compile_fusion*. If multiple site definitions are present in a uniform

row design, use the minimum site height to represent. For hybrid row design, use the maximum site height among all site rows to represent.

See Also

- [identify_multibit](#)

multibit.banking.maximum_allowable_name_length

The maximum allowable name length of banked mbit.

Data Types

integer

Default no limit

Description

This app option defines the maximum allowable name length of banked mbit cell during *identify_multibit* and integrated banking flow.

For example, if the name length of 8 bits of the mbit banked cell exceeds the limit, the 8 bits mbit cell would not be banked. Instead, banking the cells into the lower bits of mbit cell will be tried.

See Also

- [identify_multibit](#)

multibit.common.exclude_size_only_cells

If true, excludes only user size only cells during banking and debanking flow.

Data Types

boolean

Default true

Description

If true, exclude only user size only cells during banking and debanking flow.

See Also

- [identify_multibit](#)
- [get_app_option_value](#)

- [report_app_options](#)
- [set_app_options](#)

multibit.common.ignore_cell_with_sdc

Ignore cells set with sdc constraints while banking and debanking.

Data Types

boolean

Default false

Description

Enabling this option ignores cells set with sdc constraints while banking and debanking.

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

multibit.common.ignore_cstr_list

Specify the type of exception constraints that can still be considered for banking/debanking.

Data Types

string

Default " "

Description

This application option works only when `multibit.common.ignore_cell_with_sdc` is set to true. This application option allows users to specify the type of exception constraints that can still be considered for banking/debanking. By default, it sets to null which means that registers with any timing exceptions will be excluded from banking/debanking when `multibit.common.ignore_cell_with_sdc` is set to true.

Supported constraint names: *ANNOTATION, IDEAL_NETWORK, MULTICYCLE_PATH, PATH_GROUP, CLOCK_LATENCY, LATENCY_OFFSET, MAX_MIN_DELAY, FALSE_PATH, MAX_TRANSITION*

ANNOTATION covers annotation constraints. e.g., `set_annotated_transition`, `set_annotated_delay`, `set_annotated_check`, `set_ideal_transition`, `set_ideal_latency`

IDEAL_NETWORK covers ideal network constraints. e.g., `set_ideal_network`

MULTICYCLE_PATH covers multicycle path constraints. e.g., `set_multicycle_path`

PATH_GROUP covers path group constraints. e.g., `group_path`

CLOCK_LATENCY covers clock latency constraints. e.g., `set_clock_latency`

LATENCY_OFFSET covers the clock latency constraints generated by the tool. e.g., `set_clock_latency`

MAX_MIN_DELAY covers maximum/minimum delay constraints. e.g., `set_max_delay`, `set_min_delay`

FALSE_PATH covers false path constraints. e.g., `set_false_path`

MAX_TRANSITION covers max transition constraints. e.g., `set_max_transition`

Example: The following example allows registers with ONLY `set_multicycle_path` constraint to be included in banking and debanking: `set_app_options -name multibit.common.ignore_cell_with_sdc -value true set_app_options -name multibit.common.ignore_cstr_list -value "MULTICYCLE_PATH"`

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

multibit.debanking.avoid_new_high_fanout_net

Avoid creating high fanout nets in debanking.

Data Types

boolean

Default `false`

Description

When true, debanking avoids creating high fanout nets. Tool rejects to debank, if debanking makes the fanout of connected nets exceed the limit. The fanout limit is controlled by the app option *time.high_fanout_net_threshold*. However, ideal nets and high fanout nets are unrestricted by the check.

See Also

- [split_multibit](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

multibit.debanking.check_high_fanout_net

Check high fanout nets in debanking.

Data Types

boolean

Default true

Description

When true, tool rejects to debank cells connected to high fanout nets to save runtime for copying timing constraints. The high fanout limit is controlled by the app option *time.high_fanout_net_threshold*, and the high fanout limit is $5 * \text{high_fanout_net_threshold}$. Cells connected to nets with fanout count higher than the limit will not be debanked. However, clock nets are unrestricted by the check.

See Also

- [split_multibit](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

multibit.debanking.effort

Set the effort level of debanking during compile_fusion and place_opt.

Data Types

string

Default "low"

Description

This app option can be set to "*low*", "*medium*", "*high*", and "*ultra*". The *medium*, *high*, and *ultra* effort are provided to get better timing, by debanking more multibit cells on critical paths. Please note that running debanking with *medium*, *high*, and *ultra* effort will lead to a higher number of multibit cells being debanked, which may lead to lower banking ratio and hence worse power. The *low* effort is the default mode and it debanks multibit cells only if various things being considered do not degrade like density, transition, timing, etc. By default, there is one debanking step in `initial_opto` and two debanking steps in `final_opto`. If the debanking effort is not *low*, additional debanking steps will be enabled. The *medium*, *high* and *ultra* effort enable one additional debanking step in `initial_opto` of `compile_fusion`, and one additional debanking step in `final_place` of `place_opt` and `compile_fusion`. The *ultra* effort enables an additional debanking step in `final_opto` of `place_opt` and `compile_fusion`. The *high* effort allows the additional debanking steps to debank multibit cells in higher density areas of the designs. The *ultra* effort makes the additional debanking steps to debank all critical multibit cells to single-bit cells and also allows debanking in higher density areas of the designs.

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

multibit.debanking.honor_tie_cell_fanout_limit

Honor tie cell fanout limit in debanking.

Data Types

boolean

Default `true`

Description

When true, debanking honors the tie cell fanout limit. Tool rejects to debank, if debanking makes the fanout of the tie cell exceed the limit. The tie cell fanout limit is controlled by the app option `opt.tie_cell.max_fanout`.

See Also

- [split_multibit](#)
- [get_app_option_value](#)

- [report_app_options](#)
- [set_app_options](#)

multibit.naming.common_prefix_naming_mode

The mode of how the name of the merged multibit cell is decided after the common prefix.

Data Types

string

Default "abbreviation"

Description

The accepted value is "*abbreviation*", "*exact*" and "*abbreviation_unmatched_square_bracket*". The app option *multibit.naming.common_prefix_naming_style* should be *true* to enable the feature.

If the value is "*abbreviation*", tool generates the name of the banked cell such a way that common name of constituent cells becomes the prefix followed by "mflop{bit_width_of_multibit_cell}_{random_number}". In case of no common prefix, the name of the multibit flop will be "mflop{bit_width_of_multibit_cell}_{random_number}".

If the value is "*exact*", tool generates the name of the banked cell such a way that common name of the constituent cells before "_" or "/" becomes the prefix followed by "{remaining part of cell1}_{remaining part of cell2}_...". In case of no common prefix, the name of the multibit flop will be "{cell1}_{cell2}_...".

If the value is "*abbreviation_unmatched_square_bracket*", tool generates the name of the banked cell such a way that common name of constituent cells becomes the prefix with discarding the unmatched square brackets followed by "mflop{bit_width_of_multibit_cell}_{random_number}". In case of no common prefix, the name of the multibit flop will be "mflop{bit_width_of_multibit_cell}_{random_number}".

This application option is not supported in RTL multibit banking and debanking.

Examples

Value is "*abbreviation*":

Single Bit Register Names: 1. *flopA_1_* 2. *flopB_2_* Multibit Register Name : *flop_mflop2_{number}*

Value is "*exact*":

Single Bit Register Names: 1. *A/B/CCC_DD_E3/reg1* 2. *A/B/CCC_DD_E4/reg2* Multibit Register Name : *A/B/CCC_DD_E3/reg1_E4/reg2*

Value is "*abbreviation_unmatched_square_bracket*":

Single Bit Register Names: 1. *A/B/CCC_reg[12]* 2. *A/B/CCC_ref[13]* The common prefix is *A/B/CCC_reg[1* and the unmatched square brackets will be discarded. The output common prefix will be *A/B/CCC_reg*. Multibit Register Name : *A/B/CCC_reg_mflop2_{number}*

multibit.naming.common_prefix_naming_style

Generates the name of merged multibit cell keeping the common prefix of constituent cells.

Data Types

boolean

Default *false*

Description

If the app option is made true, tool generates the name of the banked cell such a way that common name of constituent cells becomes the prefix followed by "*mflop{bit_width_of_multibit_cell}_{random_number}*" by default. In case of no common prefix, the name of the multibit flop will be "*mflop{bit_width_of_multibit_cell}_{random_number}*". The followed trailing name of merged multibit is controlled by the app option *multibit.naming.common_prefix_naming_mode*

Examples

Single Bit Register Names: 1. *flopA_1_* 2. *flopB_2* Multibit Register Name : *flop_mflop2_{number}*

Single Bit Register Names: 1. *aFlop* 2. *bFlop* Multibit Register Name : *mflop2_{number}*

See Also

- [identify_multibit](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

multibit.naming.compact_hierarchical_name

Controls whether the multibit register will have hierarchical names before each individual register or not.

Data Types

Boolean

Default false

Description

When the value of this application option is *true*, the common hierarchical name of each individual single bit register is added as a prefix to the final multibit name. Note that only the names of ungrouped hierarchies are affected by this application option.

If the value of this application option is *false*, the name of the multibit register is a concatenation of the names of all single-bit registers.

When using compact hierarchical names, you must use the SVF file to verify the functionality of the design using Formality.

To determine the current value of this application option, use the *get_app_option_value -name multibit.naming.compact_hierarchical_name* command. For a list of all multibit naming application options and their current values, use the *report_app_options multibit.naming** command.

Examples

Single Bit Register Names: 1. *b1/A_reg* 2. *b1/B_reg*

1. If hierarchy b1 is ungrouped: b1 becomes the hierarchical path of the single bit cell names and the multibit register name will be affected by this application option
Option Value : *true* Multibit Register Name : *b1/A_reg_B_reg* Option Value : *false* Multibit Register Name : *b1/A_reg_b1/B_reg*
 2. If hierarchy b1 is not ungrouped: b1 is the hierarchy name and the multibit register name will not be affected by this application option
Option Value : *true* Multibit Register Name : *b1/A_reg_B_reg* Option Value : *false* Multibit Register Name : *b1/A_reg_B_reg*
-

multibit.naming.element_separator_style

Controls the elements which separates the non contiguous bits in the name of the multibit register

Data Types

String

Default ,

Description

Use this application option to decide the character that separates the non-contiguous bits of a multibit register

When using `element_separator_style`, you must use the SVF file to verify the functionality of the design using Formality.

To determine the current value of this application option, use the `get_app_option_value -name multibit.naming.element_separator_style` command. For a list of all multibit naming application options and their current values, use the `report_app_options multibit.naming*` command.

Examples

Single Bit Register Names: 1. `A_reg[4:2]` 2. `A_reg[0]`

1. Option Value : ,, Multibit Register Name : `A_reg[4:2,,0]`

multibit.naming.expanded_name_style

Controls the expansion of the single bit register indices when forming a multibit register name. Permissible values are 0,1,2, 4 and 8.

Data Types

Integer

Default 0

Description

Below are the description for the different values to be given

0

No expansion of the bits in the multibit register name, the contiguous bits would be clubbed using string specified by `multibit.naming.range_separator_style` (default ":") and the non-contiguous indices are separated using the string specified by `multibit.naming.element_separator_style` (default ",")

1

The non contiguous indices with the register name would be expanded in the final Multibit Register name. If the multibit register is formed using multidimensional single bit registers, then the inner bits would be expanded.

2

If the Multibit Register is formed using the multidimensional single bit registers, all the bits would be expanded in the Multibit register name

4

If the indices are union fields in the single bit registers they can be expanded using option value 4.

8

If the Multibit Register is formed using the multidimensional single bit registers, all the bits would be expanded in the Multibit register name. It also uses union field name and does the expansion of all the bits

When using `expanded_name_style`, you must use the SVF file to verify the functionality of the design using Formality.

This application option is not supported in physical-aware multibit banking and debanking.

To determine the current value of this application option, use the `get_app_option_value -name multibit.naming.expanded_name_style` command. For a list of all multibit naming application options and their current values, use the `report_app_options multibit.naming*` command.

Examples

1. Single Bit Register Names: 1. `A_reg[4]` 2. `A_reg[3]` 3. `A_reg[2]` 4. `A_reg[0]`
2. Option Value : 0 Multibit Register Name : `A_reg[4:2,0]` 2. Option Value : 1 Multibit Register Name : `A_reg[4:2]_A_reg[0]` 3. Option Value : 2 Multibit Register Name : `A_reg[4]_A_reg[3]_A_reg[2]_A_reg[0]`
3. Single Bit Register Names: 1. `A_reg[2][1]` 2. `A_reg[2][0]` 3. `A_reg[1][1]` 4. `A_reg[1][0]`
4. Option Value : 0 Multibit Register Name : `A_reg[2:1][1:0]` 2. Option Value : 1 Multibit Register Name : `A_reg[2][1:0]_A_reg[1][1:0]` 3. Option Value : 2 Multibit Register Name : `A_reg[2][1]_A_reg[2][0]_A_reg[1][1]_A_reg[1][0]`
5. Single Bit Register Names: 1. `A_reg[unionField1]` 2. `A_reg[unionField2]`
6. Option Value : 0 Multibit Register Name : `A_reg[1:0]` 2. Option Value : 4 Multibit Register Name : `A_reg.unionField1_A_reg.unionField2`
7. Single Bit Register Names: 1. `A_reg[unionField][1]` 2. `A_reg[unionField][0]`

8. Option Value : 0 Multibit Register Name : *A_reg[1:0]* 2. Option Value : 1 Multibit Register Name : *A_reg[1:0]* 3. Option Value : 2 Multibit Register Name : *A_reg[1]_A_reg[0]* 4. Option Value : 4 Multibit Register Name : *A_reg.unionField[1:0]* 5. Option Value : 8 Multibit Register Name : *A_reg.unionField[1]_A_reg.unionField[0]*

multibit.naming.instance_name_max_length

The limit of the name length of newly banked multibit cells.

Data Types

integer

Default 1024

Description

If the name length of a banked cell exceeds the value of `multibit.naming.instance_name_max_length`, the name will be generated as "{prefix}_{random_number}". The prefix is set by `multibit.naming.name_prefix`. If the prefix is not specified, the prefix will be "multibit_inst".

This application option is not supported in RTL multibit banking and debanking.

Examples

Single Bit Register Names: 1. *flopA_1* 2. *flopB_2* Value of `multibit.naming.instance_name_max_length`: 8 Multibit Register Name: *multibit_inst_{number}*

See Also

- [identify_multibit](#)

multibit.naming.multiple_name_separator_style

String to be used to separate the name of the individual register names in the name of the multibit register

Data Types

String

Default _

Description

Use this application option to decide the string that separates the individual register names in the multibit register name.

When using `multiple_name_separator_style`, you must use the SVF file to verify the functionality of the design using Formality.

To determine the current value of this application option, use the `get_app_option_value -name multibit.naming.multiple_name_separator_style` command. For a list of all multibit naming application options and their current values, use the `report_app_options multibit.naming*` command.

Examples

Single Bit Register Names: 1. `A_reg` 2. `B_reg`

1. Option Value : `_SEP_` Multibit Register Name : `A_reg_SEP_B_reg`

multibit.naming.name_prefix

Prefix used before the name of the multibit register

Data Types

String

Default Empty String

Description

The prefix is to be used before the name of the multibit register name. If no prefix is specified using the app Option no prefix would be applied before the name of the multibit register.

When using `name_prefix`, you must use the SVF file to verify the functionality of the design using Formality.

To determine the current value of this application option, use the `get_app_option_value -name multibit.naming.name_prefix` command. For a list of all multibit naming application options and their current values, use the `report_app_options multibit.naming*` command.

Examples

Single Bit Register Names: 1. `u1/a_reg` 2. `u1/b_reg`

1. Option Value : `_PRE_` Multibit Register Name : `u1/_PRE_a_reg_u1/b_reg`

multibit.naming.range_separator_style

Specifies the character used to separate the range in multibit register names.

Data Types

string

Default :

Description

This application option specifies the character that separates the contiguous range in a multibit register name.

When using this application option, you must use the SVF file to verify the functionality of the design using Formality.

This application option is not supported in physical-aware multibit banking and debanking.

To determine the current value of this application option, use the *get_app_option_value -name multibit.naming.range_separator_style* command.

For a list of all multibit naming application options and their current values, use the *report_app_options multibit.naming** command.

Examples

Single Bit Register Names:

1. `A_reg[2]`
2. `A_reg[1]`

1. Option Value : :
Multibit Register Name : `A_reg[2:1]`

multibit.naming.rtl_banking_concatenate_single_bit_names

The multibit name is the concatenation of the single bit names.

Data Types

boolean

Default false

Description

In RTL banking, the default multibit name is generated based on the bused instance name. If this is set to true, the multibit name is created by concatenating the single bit names together.

When this app option is set, RTL banking will only support options *multibit.naming.compact_hierarchical_name*, *multibit.naming.multiple_name_separator_style*, and *multibit.naming.name_prefix*, and the rest multibit naming options become ineffective.

This app option will impact the names of MBIT registers created by RTL banking, RTL debanking and constant/unloaded multibit register removal.

Examples

Single bit register names: 1. *b1/out_reg[3]* 2. *b1/out_reg[2]* 3. *b1/out_reg[1]* 4. *b1/out_reg[0]*

If option value is false, the multibit register name: *b1/out_reg[3:0]*

If option value is true, the multibit register name: *b1/out_reg[3]_b1/out_reg[2]_b1/out_reg[1]_b1/out_reg[0]*

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

mv.cells.ao_cells_without_primary_pg_pin

Option to turn on support of always-on library cells with no primary power pin

Description

When this app option is set to true, tool will only use always-on buffer, inverter, and isolation library cells with no primary power pin whenever regular dual-rail library cells are required.

default

false

See Also

- [create_mv_cells](#)

mv.cells.auto_va_set_voltage_file_name

Option to specify the name of the file which contains the voltage value for the all the power and ground nets defined in UPF

Description

For all scenario, and all corners, user needs to specify the voltage value for all the power and ground nets defined in UPF in this file. This appOption is essential for `derive_disjoint_voltage_area_ml` command.

mv.cells.check_ls_ground_supply_netgroup

Guide level shifter cell insertion to select ground supply net in the same netgroup of driver supply or load supply.

Data Types

Boolean

Default false

Description

This application option controls whether or not level shifter insertion must use ground supply in the same netgroup or not.

To disallow the tool to select other ground supply in other PST equivalent netgroup set this application option to *true*.

See Also

- [check_mv_design](#)
- [compile_fusion](#)
- [create_mv_cells](#)

mv.cells.check_ls_power_supply_netgroup

Guide level shifter cell insertion to select power supply net in the same netgroup of driver supply or load supply.

Data Types

Boolean

Default false

Description

This application option controls whether or not level shifter insertion must use power supply in the same netgroup or not.

To disallow the tool to select other power supply in other PST equivalent netgroup set this application option to *true*.

See Also

- [check_mv_design](#)
- [compile_fusion](#)
- [create_mv_cells](#)

mv.cells.connect_iso_control_directly

Connect isolation cell control pin to isolation signal from strategy directly, ignore existing connections.

Description

By default the tool will connect isolation cell control pin resuing existing connections to avoid punching new ports. By setting this to true, tool will connect isolation cell control pin to isolation signal from strategy directly, ignore existing connections. This will result in punching new ports.

default

false

See Also

- [create_mv_cells](#)

mv.cells.connect_power_switch_incremental_optimization_max_iterations

This app option sets the number of incremental iterations for improvement of switch connections for connect_power_switch command with -mode shortest.

Data Types

Integer

Default 200

Description

With 'connect_power_switch -mode shortest' the tool will try to connect all switches given in the connect_power_switch, such that the total distance between the switches is minimized.

If the incremental optimization is enabled, this app options sets the maximum number of such iterations. Setting a higher value will help to further optimize the connections.

Set it to a lower value if run time is of major concern.

See Also

- [connect_power_switch](#)

mv.cells.connect_power_switch_incremental_optimization_max_number_of_closest_nodes

This app option sets the maximum value of closest nodes to store in the memory while performing the fast approximation algorithm for connect_power_switch with shortest mode.

Data Types

Integer

Default 300

Description

With 'connect_power_switch -mode shortest' the tool will try to connect all switches given in the connect_power_switch, such that the total distance between the switches is minimized.

If incremental improvement and fast approximation algorithm are enabled, setting this app option sets the number of closest nodes to save for each switch. The higher the number, the more optimal the connection and the higher memory usage.

If memory is of a great concern, set this app option to a value between 5 to 20.

See Also

- [connect_power_switch](#)

mv.cells.connect_power_switch_min_long_distance_value

This app option sets the minimum length for a net to be considered a long net, and it is used to measure the quality of the connections made with connect_power_switch in daisy and shortest modes.

Data Types

Integer

Default 1000

Description

With 'connect_power_switch -mode shortest or daisy', the tool will count the number of "long" nets.

Long nets are defined to be nets whose estimated wire length (while accounting for blockages), is higher than the value of this app option.

Use this app option to count the number of such nets, and compare the quality of connections.

See Also

- [connect_power_switch](#)

mv.cells.connect_power_switch_optimize_ack_out_distance

Specifies whether the location of ack port and its distance from power switch cells should be considered in shortest mode of connect_power_switch command.

Data Types

Boolean

Default false

Description

This app option specifies whether shortest mode of connect_power_switch command considers the location of ack port and its distance from the last power switch cell for optimizing and reporting the switch connections.

See Also

- [connect_power_switch](#)

mv.cells.deterministic_iso_name_format

Data Types

String

Default none

Description

Isolation cells can use one of two modes for uniquely generating a name: index based (default) and deterministic. Three values are available for this app option: "none" (default), "latch_only", and "all".

When this option is set to *"none"*, the default behavior is to name isolation cells with the following format:

`"<prefix>snps_<power domain>__<isolation strategy>_snps_<pin name><suffix>"`

Name collisions for any two or more cells, regardless of the value set for this app option, includes an index before the suffix that isn't deterministically ordered. So to avoid this situation as much as possible, the remaining two values of this app option can be used.

When set to *"all"*, all cells that are isolating a high-conn pin will include the name of the corresponding instance right after the pin's name. So for example, given a cell isolating a pin "a" on an instance called "m1", then a plausible name would be: `"snps_PD__ISO_snps_a_m1_UPF_ISO"`.

However, when set to *"latch_only"*, the previous format is only applied to latch isolation cells. Any other isolation cells would use the default naming format.

See Also

- [associate_mv_cells](#)
- [create_mv_cells](#)
- [commit_upf](#)

mv.cells.enable_connect_power_switch_incremental_optimization

This app option enables the incremental improvement of switch connection for `connect_power_switch` command with `-mode shortest`.

Data Types

Boolean

Default `true`

Description

With `'connect_power_switch -mode shortest'` the tool will try to connect all switches given in the `connect_power_switch`, such that the total distance between the switches is minimized. When setting this app option to `TRUE`, after finishing the first set of connections, the tool will try to further improve the connections in incremental iterations.

Setting this app option to `FALSE` will prevent the incremental optimization, and should be used if the run time is of major concern.

See Also

- [connect_power_switch](#)

mv.cells.enable_connect_power_switch_incremental_optimization_fast_approximation

This app option enables the tool to use a fast approximation algorithm for incremental improvement of switch connection for `connect_power_switch` command with `-mode shortest`.

Data Types

Boolean

Default `true`

Description

With '`connect_power_switch -mode shortest`' the tool will try to connect all switches given in the `connect_power_switch`, such that the total distance between the switches is minimized.

If incremental improvement is enabled, setting this app option to `TRUE`, will make the tool to use a fast approximation algorithm instead of the slow but more optimal algorithm.

Setting this app option to `FALSE` make the tool to use the slow, but more optimal algorithm, which takes a longer time to finish, but will produce a more optimal result.

See Also

- [connect_power_switch](#)

mv.cells.enable_iso_cell_bias_rail_order_check

Enable check for rail order violation for the related PG/bias supply pair for single rail iso cell at mapping.

Data Types

Boolean

Default `false`

Description

When this app option value is set to `true`, mapper will perform the check about rail order violation for the related PG/bias supply pair to make sure that the bias supply more AO than the primary supply net.

See Also

- [create_mv_cells](#)
- [check_mv_design](#)

mv.cells.enable_multibit_pm_cells

Controls the mapping to multibit library cells for the PM cells (ISO/LS/ELS) *compile_fusion* command.

Data Types

Boolean

Default false

Description

When this application option is set to *true*, the tool performs mapping using available multibit PM library cells (ISO/LS/ELS) during the *compile_fusion* command.

If you do not want the *compile_fusion* command to use multibit library cells, set the *mv.cells.enable_multibit_pm_cells* application option to *false*.

To determine the current value of this application option, use the *get_app_option_value -name mv.cells.enable_multibit_pm_cells* command.

For a list of all compile application options and their current values, use the *report_app_options compile.** command.

See Also

- [compile_fusion](#)

mv.cells.enable_tap_cell_virtual_bias_pin_check

Enable the check about virtual bias pin of tap cells in *check_mv_design*.

Data Types

Boolean

Default false

Description

When this app option value is set to *true*, *check_mv_design* will perform the check about virtual bias pin of tap cells to make sure that the bias supply is not shorted to the primary supply net.

See Also

- [check_mv_design](#)

mv.cells.infer_complex_retention_cells

Controls inference of complex retention cells in place of complex non-retention cells which fall under certain retention strategy.

A cell is considered complex if it does not have a valid functionality.

Data Types

Boolean

Default false

Description

When this application option is *false*, tool will not infer any complex retention cells in place of pre-instantiated complex non-retention cells.

When this application option is set to *true*, tool will infer complex retention cells in place of pre-instantiated complex non-retention cells that fall under retention strategy. This style of inference is supported both in *compile_fusion* and *create_mv_cells*.

Tool will try to infer a complex retention cell based on pin name matching. The tool performs the following steps:

1. Pin names of the non-retention cell will be matched with the pin names of the retention library cells specified in the *map_retention_cell* command.
2. The first retention library cell whose pin names exactly match (barring the retention save/restore pins) will be picked up and swapped in place of the non-retention cell.
3. If a matching retention library cell is not found, then a warning message will be printed saying that the non-retention cell cannot be converted to its retention counterpart.

With the pin name matching approach, the tool will pick up the first matching retention library cell from the *map_retention_cell* command. If there are multiple matches available in *map_retention_cell*, and if the user wants a particular retention library cell to be picked up among the matches, then users will be able to achieve this by setting a new attribute called *retention_equivalent* on the non-retention library cell. The value of this attribute will be the name of the preferred (equivalent) retention library cell.

Usage -

```
define_user_attribute -type string -classes lib_cell -name  
retention_equivalent
```

```
set_attribute -objects [get_lib_cells
library/complex_non_retention_library_cell] -name retention_equivalent
-value {complex_retention_library_cell}
```

If the retention library cell (specified as the attribute value) is invalid, then the tool will fall back to the pin name matching approach. A retention library cell will be considered invalid in the following scenarios –

1. The retention library cell is not present in any of the target libraries.
2. The retention library cell has the `dont_use` or the `dont_touch` attributes on it.
3. The retention library cell doesn't have the "retention_cell" attribute on it.
4. The retention library cell is not present as one of the library cells in the `map_retention_cell` command.
5. The non-retention library cell has at least one pin that is not present on the retention library cell.

So, if an invalid retention library cell is found as the attribute value, then the tool will not honor the *retention_equivalent* attribute, and it will fall back to the pin name matching approach.

Point # 5 means that all the pins present on the non-retention library cell must also be present on the retention library cell. But it is okay for the retention library cell to have additional pins (other than the retention save/restore pins). Examples for additional pins are scan input/output and scan enable pins. If a retention library cell having additional pins is inferred by the tool, it connects the additional pins to constant zero. If a particular pin is not supposed to be connected to constant zero, you should fix the connection after compilation (before taking the netlist further down the flow). Whenever a retention library cell having additional pins is inferred, a message is issued to let you know about such an inference. This should help you to watch out for the connections made by the tool to the additional pins. Inferring retention cells with additional pins will be helpful for users who want to replace a non-scan non-retention cell (instantiated in the RTL) with a scan retention cell. `insert_dft` can later do the required scan stitching with the scan input/scan enable pins.

Handling mismatching pin names: Users may have library cells where certain pin names of the non-retention library cell do not match the pin names of the retention library cell. In such cases, users can use the same *retention_equivalent* attribute to specify the pin mappings between the mismatching pin names.

Usage -

```
set_attribute -objects
{library/complex_non_retention_library_cell_name} -name
```

```
retention_equivalent -value {complex_retention_library_cell_name {pin1  
pin2} {pin3 pin4}}
```

In this usage, the attribute value has the retention library cell name listed first and then the pin mappings between the non-retention library cell and the retention library cell. The above usage means that –

1. “pin1” on the non-retention library cell corresponds to (or is equivalent to) “pin2” on the retention library cell.
2. “pin3” on the non-retention library cell corresponds to “pin4” on the retention library cell.

This information will help the implementation tools to transfer the net connections appropriately. If the pin mappings are invalid, then the tool will fall back to the pin name matching approach. Pin mappings will be considered invalid in the following scenarios –

1. The specified pin name does not exist on the non-retention library cell/retention library cell.
2. Other than the specified pin mappings, the non-retention library cell has at least one pin that is not present on the retention library cell.

See Also

- [set_retention](#)
- [set_retention_control](#)
- [commit_upf](#)
- [check_mv_design](#)

mv.cells.insert_clamp_in_zpr_hierarchy

Option to control whether to insert clamp cell in the same hierarchy as zpr retention cell

Description

By default the tool is allowed to insert clamp cells in hierarchy different from zpr retention cell to minimize number of clamp cells.

default

false

See Also

- [create_mv_cells](#)

mv.cells.isolate_constant_net

Option to control whether to insert isolation cells on constant net when isolation strategy has matching clamp value on mapped design.

Description

For unmapped design, tool always insert isolation cell on constant net and constant propagation will optimize isolation cells. This app option will control ISO cell insertion on constant net for mapped design only.

By default the tool will always insert isolation cells on constant net on mapped design. When set it to false, tool will not insert isolation cell when isolation strategy has matching clamp value.

default

true

See Also

- [create_mv_cells](#)

mv.cells.isolate_diode_load

Option to control whether to fix isolation violation for path with diode as load or not.

Description

By default the tool will perform isolation violation fixing for path with diode load as normal load. If set to false, isolation violation fixing will skip diode load path.

default

true

See Also

- [create_mv_cells](#)

mv.cells.ls_association_derive_missing_csn

Option to control whether to create *connect_supply_net* for level shifter main rails at association.

Description

When the value is set as true, the tool will derive and create a *connect_supply_net* for all pre-instantiated level shifter cells mail rails when associate them with a strategy, only if their main rails do not have any *connect_supply_net* set, neither by the user or tool generated.

default

false

See Also

- [commit_upf](#)
- [associate_mv_cells](#)

mv.cells.mv_allow_buf_on_mv_boundary_nets

Allow buffering the nets between mv cells and the power domain boundaries associated with the strategies of those cells

Data Types

Boolean

Default false

Description

By default, we do not allow buffering the nets between mv cells (isolation cells, level shifters and enable level shifters) and the power domain boundaries associated with their strategies.

With this appOption, mega commands (compile_fusion, place_opt, clock_opt, route_opt etc) can add buffers on the nets between the mv cells and their associated power domain boundaries to improve QoR. These buffers can be normal buffers, physical feedthrough buffers and logical feedthrough buffers. check_bufferability command will also show net is bufferable. With these buffers in the nets between the MV cells and their associated power domain boundaries, the MV cells association should remain intact.

Please note: When user sets this appOption to be true, user should still expect to see dont_touch derived on the nets between the MV cells and power domain boundaries associated with the strategies. The dont_touch restriction is relaxed and re-propogated before and after the buffering step to allow buffering to happen.

mv.cells.mv_allow_domain_primary_relative_ao

Enables relative onness support during mv buffering

Data Types

Boolean

Default false

Description

By default, the intermediate voltage area(VA)/power domain(PD) will only be allowed to use the equivalent or PST equivalent of either the driver supply or the sink supply. This can be single-rail or dual rail depending on the onness relationship between the supply available in the intermediate VA/PD and either the source or sink VAs.

With this appOption, it uses relative-onness to allow the usage of the primary supply in intermediate VA/PD that is not equivalent or PST equivalent with either source or sink, as long as the onness satisfies $\text{onness}(\text{source}) > \text{onness}(\text{intermediate}) > \text{onness}(\text{sink})$. Using this appOption, there is a chance that less dual rail cells need to be used.

Please note: This feature only looks at the current local source and sink when it decides which supply to use. It does not consider the whole path. Therefore the solution is not guaranteed to be optimum as far as dual-rail usage or even for buffering all possible.

mv.cells.mv_use_domain_primary_for_macro_iso_pin

Using domain primary to buffer the power domain where the macro is_isolated pin sits

Data Types

Boolean

Default false

Description

By default, for the net that drives the macro is_isolated pin (this excludes those is_isolated macro pins that have explicit set_related_supply_net set on them), we always use driver's supply to buffer. If the driver's supply is not the same or PST equivalent as the domain primary, then dual-rail buffers/inverters will be required.

With this appOption, for the net that drives the macro is_isolated pin, the tool will use the domain primary to buffer in the power domain where the macro is_isolated pin sits, if the domain primary is less or equal always on than the driver's supply. If the macro has its own power domain, then domain primary to buffer applies to the power domain where the logical parent of the macro belongs to. Using this appOption, there is a chance that less dual rail cells need to be used.

Please note: When user sets this appOption to be true, it is assumed that users have simulated and verified that when the isolation control signal is off, there will not be any mv violation between the domain primary and the supply of the load that the isolation is driving.

mv.cells.mv_use_domain_primary_for_zpr_clamp_input

Using domain primary to buffer the power domain where the zero pin retention clamp input pin sits

Data Types

Boolean

Default false

Description

By default, for the net that drives the zero pin retention (zpr) clamp input pin, we always use driver's supply to buffer. If the driver's supply is not the same or PST equivalent as the domain primary, then dual-rail buffers/inverters will be required.

With this appOption, for the net that drives the zpr clamp input pin, the tool will use the domain primary to buffer in the power domain where the zpr clamp input pin sits, if the domain primary is less or equal always on than the driver's supply. Using this appOption, there is a chance that less dual rail cells need to be used.

Please note: When user sets this appOption to be true, it is assumed that users have simulated and verified that when the zpr control signal is off, there will not be any mv violation between the domain primary and the supply of the load that the zpr is driving.

mv.cells.no_resolved_iso_strategy_attr_on_dangling_path

Option to control whether to add resolved_iso_strategy attribute on dangling net.

Description

By default the tool will add resolved_iso_strategy on dangling net.

default

false

See Also

- [create_mv_cells](#)

mv.cells.rename_isolation_cell_with_formal_name

set option for the Tool to rename Isolation with formal name during ISO association.

Description

By default the Tool will rename Isolation Cells using formal naming rule during association (when running *associate_mv_cells*, *create_mv_cells* or *commit_upf*). Turning this value to *false* will allow the Tool to keep original Isolation Cell name.

default

true

See Also

- [commit_upf](#)
- [create_mv_cells](#)
- [associate_mv_cells](#)

mv.cells.report_pvt_exact

Print PVT mismatch messages if cells fail exact PVT matching.

Data Types

boolean

Default false

Description

This option enables printing of PVT mismatch messages if cells fail exact PVT matching in *compile_fusion*.

By default, PVT-mismatch summary message MV-1030 will be printed if cells fail to match PVT constraints by range. When this option is set to *true*, tool will print MV-1030 if cells fail to match PVT constraints exactly.

mv.cells.report_pvt_verbose

Print verbose message for each cell which fails PVT checking.

Data Types

boolean

Default false

Description

This option enables verbose message for PVT checking in compile_fusion.

By default, only PVT-mismatch summary message MV-1030 will be issued. When this option is set to *true*, PVT-mismatch messages MV-1031 and PVT-1201 will also be printed for each individual cell which fails PVT checking.

For example:

```
Warning: PVT mismatch found in MID1/in_UPF_ISO (LIB1/iso). (MV-1031)
Warning: Can't use LIB1/ISO because it doesn't match isolation power
supply net voltage (VDD2 = 0.500V) in corner 'default'. (PVT-1201)
Warning: PVT mismatch found in MID2/in_UPF_ISO (LIB2/iso). (MV-1031)
Warning: Can't use LIB2/ISO because it doesn't match process number
(2.000) in corner 'default'. (PVT-1201)
Warning: PVT mismatch found in 2 isolation cells. (MV-1030)
```

mv.cells.smart_derive_iso_strategy_on_new_control_ports

Option to turn on smart no_isolation derivation on retention control path.

Description

When this app option is set to true, tool will only derive no_isolation strategy on punched ports of retention control path when there is no isolation violation.

default

false

See Also

- [create_mv_cells](#)

mv.cells.support_backup_ground_only_retention

support ground backup supply only retention type of retention cell(foot type).

Data Types

Boolean

Default false

Description

When this application option is set to *true*, the tool will correctly check backup power/ground for this type of retention cell in the libraries.

See Also

- [create_mv_cells](#)
- [analyze_mv_feasibility](#)

mv.cells.use_name_based_iso_matching

set option for the tool to associate ISO cells according to naming match with strategy.

Data Types

Boolean

Default true

Description

By default the tool associate ISO cells based on naming match. It will try to use topology match when naming match failed. Turning this to false will only allow the tool associate ISO cell based on topology.

See Also

- [create_mv_cells](#)
- [associate_mv_cells](#)

mv.cells.use_name_based_ls_matching

set option for the tool to associate level shifter cells according to naming match with strategy.

Data Types

Boolean

Default true

Description

By default the tool associate level shifter cells based on naming match. It will try to use topology match when naming match failed. Turning this to false will only allow the tool associate level shifter cell based on topology.

See Also

- [create_mv_cells](#)
- [associate_mv_cells](#)

mv.cells.use_name_based_psw_matching

set option for the tool to associate power switch cells according to naming match with strategy.

Data Types

Boolean

Default true

Description

By default the tool associate power switch cells based on naming match. It will try to use topology match when naming match failed. Turning this to false will only allow the tool associate power switch cell based on topology.

See Also

- [create_mv_cells](#)
- [associate_mv_cells](#)

mv.hierarchical.afs_create_reference_supply

Application option to control creation of reference supply during assign_feedthrough_supply if the supply is not available in desired domain.

Data Types

Boolean

Default true

Description

Setting this application option to *false* prevents the creation of reference supply during assign_feedthrough_supply if the supply is not available in desired domain.

mv.hierarchical.afs_write_driver_receiver_supply

Allow assign_feedthrough_supply to write out both -driver_supply and -receiver_supply with set_port_attributes.

Data Types

Boolean

Default false

Description

By default, *assign_feedthrough_supply* will write out only one of -driver_supply or -receiver_supply with set_port_attributes depending on the port. Setting this app-option to true will enable assign_feedthrough_supply to write out both -driver_supply and -receiver_supply with set_port_attributes in assign_feedthrough_supply. The same supply will be written out with both -driver_supply and -receiver_supply.

See Also

- [assign_feedthrough_supply](#)

mv.hierarchical.allow_voltage_violation

Dont give error in split_constraints if level shifter strategy is not specified on a path with voltage violation.

Data Types

Boolean

Default false

Description

Normally *split_constraints* will give an error if it finds a path crossing the block boundary which has a voltage violation but no level shifter strategy is specified for it. Setting this application option to *true* will allow split_constraints to succeed even when there is path crossing the block boundary which has voltage violation and no level shifter strategy is specified for it.

This is usually used when the path is expected to go through feedthrough blocks but the feedthroughs for the path are not yet created. Feedthroughs are usually created using the *place_pins* command.

See Also

- [split_constraints](#)
- [place_pins](#)
- [assign_feedthrough_supply](#)
- [set_level_shifter](#)

mv.hierarchical.check_interface_objs_of_block_abstract

This option controls whether `check_mv_design` will check violations related to block interface objects of block `abstract(s)`.

Data Types

Boolean

Default `false`

Description

This option controls whether `check_mv_design` will check violations related to block interface objects of block `abstract(s)`. By default, `check_mv_design` skips checking objects that are completely inside the block abstracts.

See Also

- [check_mv_design](#)

mv.hierarchical.etm_upf_skip_missing_object_suppress_warning

This option controls whether ETM UPF constraints issue warning message for missing/nonexistent objects.

Data Types

Boolean

Default `true`

Description

This option controls whether UPF constraints at the scope of an extracted timing model (ETM) instance issue warning message for missing/nonexistent objects. By default, warning messages for missing/nonexistent objects in ETM UPF constraints are skipped.

See Also

- [SEL-003](#)

mv.hierarchical.ignore_mib_checking_error

Controls if *split_constraints* should error out when Multi-Instantiated-Module's (MIM) UPF files are not equivalent.

Data Types

Boolean

Default false

Description

By default when the target blocks specified by *set_budget_options -add_blocks* are MIM, *split_constraints* checks if a common UPF can be generated for all MIM instances. When MIM's UPFs are not equivalent, *split_constraints* cannot generate a common UPF and errors out with message *UPF-130*.

When this app_option is *true*, reduce the *UPF-130* message severity from *Error* to *Warning*. It also takes the UPF from the first instance in *set_budget_options -add_blocks* as the common UPF. If multiple MIMs are specified in *set_budget_options -add_blocks*, it uses the first of each module to create the common UPF for each module. You can set the cell attribute *is_upf_mim_primary_instance* to specify which instance UPF is selected as the common UPF. Check the manpage of *cell_attributes* to know more details about the attribute.

Warning: When this app_option is true, the resulting common UPF could not be valid for other instances.

See Also

- [set_budget_options](#)
- [UPF-130](#)
- [cell_attributes](#) Description of the application defined cell attributes.
- [set_attribute](#)

mv.hierarchical.include_pg_in_budget_shell

This option controls whether budget shells include pg pin, related power/ground pin and other multi-voltage related information during budget shell generation.

Data Types

Boolean

Default true

Description

This option controls whether budget shells include pg pin, related power/ground pin and other multi-voltage related information during budget shell generation.

Budget shell is a construct generated by the *write_budgets* command. When this app option is set to true, the following multi-voltage related syntax and attributes will be derived:

- pg_pin group
- voltage_map
- related_power_pin
- related_ground_pin
- is_isolated
- input_signal_level
- input_voltage_range
- is_unconnected
- short
- is_analog

Related power and ground pin and other attributes are captured or derived based on existing netlist, and MV cells should already exist in the netlist.

See Also

- [write_budgets](#)

mv.hierarchical.no_srsn_for_new_ports

Disables feedthrough supply assignment during place_pins

Data Types

Boolean

Default false

Description

Setting this application option to *true* prevents the setting of related supplies on feedthrough pins during *place_pins*.

See Also

- [place_pins](#)

mv.hierarchical.propagate_driver_supply_for_empty_design

Data Types

Boolean

Default true

Description

This application option controls if undriven and unloaded nets are processed by *assign_feedthrough_supply* or not. If the value is true, *assign_feedthrough_supply* processes these nets and assumes domain primary supply as related supply for undriven or unloaded pins. If the value is false, those nets are not processed and are reported as skipped using MV-1052.

See Also

- [assign_feedthrough_supply](#)
- [MV-1052](#)

mv.hierarchical.propagate_driver_supply_on_feedthrough

Enables driver supply propagation on feedthroughs during *place_pins*.

Data Types

Boolean

Default false

Description

Setting this application option to *true* allows driver supply propagation during *place_pins*.

See Also

- [place_pins](#)

mv.hierarchical.propagate_driver_supply_use_same_driver_receiver

Data Types

Boolean

Default false

Description

Using the default value of this application option (*false*) makes *assign_feedthrough_supply* checks *set_port_attributes -driver_supply* for driver and *set_port_attributes -receiver_supply* for load, respectively.

When its value is *true*, *assign_feedthrough_supply* will use *set_port_attributes -receiver_supply* as driver supply if *set_port_attributes -driver_supply* is not specified (and *vice versa* for load).

See Also

- [set_app_options](#)
- [assign_feedthrough_supply](#)
- [set_port_attributes](#)

mv.hierarchical.propagate_driver_supply_use_soft_macro_domain_primary

Data Types

Boolean

Default false

Description

With default value of *false*, *assign_feedthrough_supply* only considers soft-macro boundary with explicit *set_port_attributes* as end points. When this application option is enabled, any soft-macro boundary is considered as end point. If a soft-macro port has no user specified *set_port_attributes*, tool uses domain primary supply set instead.

See Also

- [set_app_options](#)
- [assign_feedthrough_supply](#)
- [set_port_attributes](#)

mv.hierarchical.skip_pst_in_blocks

Skip PST tables in the specified blocks.

Data Types

String

Default ""

Description

Setting this application option to exclude PST table from the derived system-level PST table. It takes a list of hierarchy names. The PST table contained inside the specified hierarchy will be excluded from derived PST tables. "" means all nested hierarchy and only the PST tables defined in top-most hierarchy will be included.

See Also

- [create_pst](#)
- [commit_upf](#)

mv.hierarchical.split_constraints_include_only_existing_supplies_in_block

Specifies whether split_constraints should include block supplies only.

Data Types

Boolean

Default false

Description

Specifies whether split_constraints should include block supplies only. When the app option value is false, split_constraints includes all supplies in the block, including outside supplies that are defined in the parent scopes or scopes above the block scope. This is the default behavior. When the app option is set to true, only supplies that are defined in

the block will be included. Any supplies that are created in the parent scopes or scopes that are above the block scope will be excluded by `split_constraints`.

See Also

- [split_constraints](#)

mv.hierarchical.split_constraints_write_driver_receiver_supply

Allow `split_constraints` to write out both `-driver_supply` and `-receiver_supply` with `set_port_attributes`.

Data Types

Boolean

Default false

Description

By default, *split_constraints* will write out only one of `-driver_supply` or `-receiver_supply` with `set_port_attributes` depending on the port. Setting this app-option to true will enable `split_constraints` to write out both `-driver_supply` and `-receiver_supply` with `set_port_attributes` in `split_constraints`. The same supply will be written out with both `-driver_supply` and `-receiver_supply`.

See Also

- [split_constraints](#)

mv.hierarchical.split_cstr_create_pst_for_outside_supply

`split_constraints` to write separate PST tables for the supplies used outside of block.

Data Types

Boolean

Default false

Description

By default, `split_constraints` writes one PST tables for the block partition including all the supplies. Setting this application option to let `split_constraints` to write out separate PST tables for the supplies used outside of the block. All existing PST tables in the block design will also be preserved in block UPF.

See Also

- [split_constraints](#)
- [create_pst](#)

mv.hierarchical.split_cstr_write_simple_object_name_for_user_attribute

split_constraints to write out set_port_attribute user attribute in nested scope with simple object name.

Data Types

Boolean

Default false

Description

Setting this application option to let split_constraints to write out set_port_attribute user attribute in nested scope with simple object name. By default, split_constraints writes out user port attribute by full hierarchical name in top-most hierarchy. When this app option is set to true, split_constraints will set_scope to the scope of pin and write out user port attribute with simple port name. For pins on leaf cell without its own power scope (set_scope), split_constraints writes out user port attribute in the parent hierarchy of the leaf cell.

See Also

- [split_constraints](#)
- [set_port_attributes](#)

mv.hierarchical.upf_port_pswout_dir_out

For supply port on physical block boundary that is connected to power switch output inside the block, specify whether split_constraints should determine the supply port direction as output if the supply port is not originally scoped on the block boundary.

Data Types

Boolean

Default false

Description

For supply port on physical block boundary that is connected to power switch output inside the block, by default, `split_constraints` will determine the direction based on whether supply driver(s) is inside and/or outside the block if the supply port is not originally scoped on the block boundary. When this app option is set to true, `split_constraints` will determine the direction of such supply ports to be output.

See Also

- [split_constraints](#)

mv.incomplete_upf.ao_infer_csn_rule

Specifies rule for inferring `connect_supply_net` for always-on cells missing exceptional supply connections in "incomplete_upf" mode.

Data Types

String

Default `infer_from_primary`

Description

This app option specifies inference rule for always-on cells missing exceptional connections in "incomplete_upf" mode when option "mv.incomplete_upf.enable" is set. Allowed values are "infer_from_driver_or_load" and "infer_from_primary". The default value is "infer_from_primary". "infer_from_driver_or_load" means exceptional connection will be inferred from the driver pin's related supply or load pins' related supply. We prefer load supply over driver supply. For heterogeneous fanouts case, we will try to get the most always-on load supply; go with driver supply if we don't have an applicable load supply. "infer_from_primary" means exceptional connection will be inferred from the primary supply of the domain where the always-on cell is located.

mv.incomplete_upf.enable

Enables incomplete_upf support functionality and flow.

Data Types

Boolean

Default `false`

Description

This option enables the `incomplete_upf` support. Please note this app option needs to be set to `TRUE`, in order to access the `incomplete_upf` functionalities.

Please note: - This option has to be set before loading UPF constraints. - Once `incomplete_upf` support is turned on, it cannot be disabled without first resetting the UPF for the design. - Once `incomplete_upf` support is turned on, `save_upf` will preserve the original UPF and will not write out any changes.

mv.incomplete_upf.iso_infer_csn_rule

Specifies rule for inferring `connect_supply_net` for isolation cells missing exceptional supply connections in "incomplete_upf" mode.

Data Types

String

Default `infer_from_primary`

Description

This app option specifies inference rule for isolation cells missing exceptional connections in "incomplete_upf" mode when option "mv.incomplete_upf.enable" is set. Allowed values are "infer_from_control_signal", "infer_from_sink" and "infer_from_primary". The default value is "infer_from_primary". "infer_from_sink" means exceptional connections will be inferred from load pin's related supply of the output net. "infer_from_control_signal" means exceptional connections will be inferred from the related supply net of the driver pin of the control net connecting the enable pins of the isolation cell. "infer_from_primary" means exceptional connection will be inferred from the primary supply of the domain where the isolation cell is located.

mv.incomplete_upf.ls_infer_csn_rule

Specifies rule for inferring `connect_supply_net` for level-shifter cells missing exceptional supply connections in "incomplete_upf" mode.

Data Types

String

Default `infer_from_primary`

Description

This app option specifies inference rule for level-shifter cells missing exceptional connections in "incomplete_upf" mode when option "mv.incomplete_upf.enable" is set.

Allowed values are "infer_input_from_source_output_from_sink" and "infer_from_primary". The default value is "infer_from_primary". "infer_input_from_source_output_from_sink" means exceptional connections for input related supply will be inferred from driver pin's related supply of the input net, while the output related supply will be inferred from load pin's related supply of the output net. "infer_from_primary" means exceptional connection will be inferred from the primary supply of the domain where the level-shifter cell is located.

mv.incomplete_upf.missing_va

Specifies rule for inferring corresponding voltage area for power domains missing voltage area in "incomplete_upf" mode.

Data Types

String

Default off

Description

This app option specifies inference rule for power domains missing voltage area in "incomplete_upf" mode when option "mv.incomplete_upf.enable" is set. Allowed values are "off", "default_voltage_area" and "auto". The default value is "off". "off" means no voltage area will be inferred for the power domain missing voltage area. "default_voltage_area" means power domains missing voltage area will be associated with DEFAULT_VA. "auto" means power domains missing voltage area will be associated automatically by inheriting the voltage area from its parent hierarchy recursively until a voltage area is found.

The option will be obsoleted in future release. Please use "mv.upf.enable_missing_voltage_area" instead.

mv.incomplete_upf.psw_infer_csn_rule

Specifies rule for inferring connect_supply_net for power switch cells missing exceptional supply connections in "incomplete_upf" mode.

Data Types

String

Default infer_from_primary

Description

This app option specifies inference rule for power switch cells missing exceptional connections in "incomplete_upf" mode when option "mv.incomplete_upf.enable" is set. Allowed values are "infer_from_most_always_on_supply" and "infer_from_primary".

The default value is "infer_from_primary". "infer_from_most_always_on_supply" means exceptional connection will be inferred from the most always_on supply of the target domain where the power switch cell is located. "infer_from_primary" means exceptional connection will be inferred from the primary supply of the domain where the power switch cell is located.

mv.incomplete_upf.ret_infer_csn_rule

Specifies rule for inferring connect_supply_net for retention cells missing exceptional supply connections in "incomplete_upf" mode.

Data Types

String

Default infer_from_primary

Description

This app option specifies inference rule for retention cells missing exceptional connections in "incomplete_upf" mode when option "mv.incomplete_upf.enable" is set. Allowed values are "infer_from_most_always_on_supply" and "infer_from_primary". The default value is "infer_from_primary". "infer_from_most_always_on_supply" means exceptional connection will be inferred from the most always_on supply of the target domain where the retention cell is located. "infer_from_primary" means exceptional connection will be inferred from the primary supply of the domain where the retention cell is located.

mv.pg.connect_pg_net_auto_mode_include_tie

Specifies whether automatic mode of connect_pg_net command should include tie connections by default when -pg and -tie command options are not specified.

Data Types

enum

Default auto

Description

This option specifies whether automatic mode of connect_pg_net command should include tie connections when -pg and -tie command options are not specified.

When this option value is *true*, automatic mode of connect_pg_net command includes both PG and tie-off pins by default.

When this option value is *false*, automatic mode of connect_pg_net command includes PG pins by default.

When this option value is *auto*, automatic mode of `connect_pg_net` command includes tie-off pins by default if one of the following conditions is satisfied for the design:

- `initial_opto` stage (or beyond) of `compile_fusion` has been completed.
- The compile stage of the design cannot be determined and the design is fully mapped.

See Also

- [connect_pg_net](#)

mv.pg.continue_infer_pg_with_conflict

`infer_supply_from_pg_net` does not stop on PG and UPF conflict errors.

Data Types

Boolean

Default `false`

Description

This option controls whether `infer_supply_from_pg_net` command errors out

`infer_supply_from_pg_net` will infer UPF statement `connect_supply_net` from PG netlist. if there is an explicit user CSN or a tool-derived CSN in the UPF, and there is a conflicting connection in the PG netlist, by default there will be an error.

If set to true, the error will not be reported, UPF CSN will get precedence.

See Also

- [infer_supply_from_pg_net](#)

mv.pg.create_floating_pg

Controls whether `connect_pg_net` create floating PG nets.

Data Types

Boolean

Default `false`

Description

connect_pg_net has two rules for PG net creation : 1. no pg net created if pg type of supply net is unknown. 2. no pg net created if no pin or ports needs to be connected to this SN.

When this option is *true*, and pg type of supply net is not unknown, floating PG nets can be created by connect_pg_net.

See Also

- [connect_pg_net](#)

mv.pg.default_ground_supply_net_name

Specifies default ground supply net name.

Data Types

String

Default VSS

Description

This application option specifies the default ground supply net name for the default power domain flow.

mv.pg.default_ground_supply_port_name

Specifies default ground supply port name.

Data Types

String

Default VSS

Description

This application option specifies the default ground supply port name for the default power domain flow.

mv.pg.default_power_supply_net_name

Specifies default power supply net name.

Data Types

String

Default VDD

Description

This application option specifies the default power supply net name for the default power domain flow.

mv.pg.default_power_supply_port_name

Specifies default power supply port name.

Data Types

String

Default VDD

Description

This application option specifies the default power supply port name for the default power domain flow.

mv.pg.honor_soft_macro

Controls whether connect_pg_net honor soft macro port SPA or soft macro domain primary supply.

Data Types

Boolean

Default false

Description

By default (*false*), connect_pg_net does not honor soft macro port SPA or domain primary supply

When this option is *true*, connect_pg_net will honor soft macro port SPA supply setting, if no SPA set, soft macro domain primary supply will be honored during load supply(ies) checking.

See Also

- [connect_pg_net](#)

mv.pg.minimum_horizontal_secondary_pg_placement_va_region_size

This appOption specifies the minimum horizontal distance allowed for secondary pg voltage area regions

Data Types

Float

Default 0.0

Description

This control is to avoid preserving many small voltage area regions that get created due to straps interacting with each other or with voltage areas in small distances.

Value of 0 is default, there is no restriction on the horizontal distance of the voltage area regions. Positive non-zero float values will remove any voltage area regions with horizontal distance being smaller than the value.

Please note:

See Also

- [create_secondary_pg_placement_constraints](#)

mv.pg.minimum_vertical_secondary_pg_placement_va_region_size

This appOption specifies the minimum vertical distance allowed for secondary pg voltage area regions

Data Types

Float

Default 0.0

Description

This control is to avoid preserving many small voltage area regions that get created due to straps interacting with each other or with voltage areas in small distances.

Value of 0 is default, there is no restriction on the vertical distance of the voltage area regions. Positive non-zero float values will remove any voltage area regions with vertical distance being smaller than the value.

Please note:

See Also

- [create_secondary_pg_placement_constraints](#)

mv.pg.preserve_physical_only_pg

Controls whether `resolve_pg_nets` and `connect_pg_net` preserve PG connections to physical only cells.

Data Types

Boolean

Default `false`

Description

By default (*false*), `resolve_pg_nets` and `connect_pg_net` do not preserve PG connections to physical only cells.

When this option is *true*, `resolve_pg_nets` and `connect_pg_net` preserve PG connections to physical only cells. No PG connection to any physical only cell will be changed.

See Also

- [resolve_pg_nets](#)
- [connect_pg_net](#)

mv.pg.resolve_pg_top_port_deletion

Specifies whether `resolve_pg_nets` deletes top port(s) when resolving conflicts between UPF intent and PG netlist.

Data Types

Boolean

Default `false`

Description

This option specifies whether `resolve_pg_nets` should delete top port(s) when resolving conflicts between UPF intent and PG netlist.

See Also

- [resolve_pg_nets](#)

mv.pg.secondary_pg_placement_lateral_margin_only

secondary pg voltage area shape grow laterally with margin.

Data Types

Boolean

Default false

Description

This app option controls how the secondary pg voltage area regions (voltage area shapes) will be derived from the straps. When `create_secondary_pg_placement_constraints` is specified with `-layers` and `-margin`, the secondary PG VA region is derived based on the specified layers and margin.

By default, the route shapes of the specified layers of the straps are expanded by the margin amount both horizontally and vertically (all four directions). When this app option is true, each shape of the straps for all supplies with secondary PG placement constraints in all voltage areas will only be expanded in the lateral direction (only width is expanded and not the length). In other words, if the strap is vertical, the VA region will be derived by expanding the strap by the margin amount only horizontally; if the strap is horizontal, it will only be expanded vertically.

This can be useful when using `connect_pg_via_ladders` to allow the cells to be placed only laterally and not longitudinally.

See Also

- [connect_pg_via_ladders](#)

mv.upf.allow_els_on_logic_zero

Allows enable level shifter insertion on logic zero driver path.

Data Types

Boolean

Default false

Description

This application option controls whether level shifter insertion is allowed on logic zero driver path for enable level shifter

See Also

- [compile_fusion](#)
- [create_mv_cells](#)

mv.upf.allow_is_isolated_output_check

Controls if is_isolated attribute is to be ignored on output pins.

Data Types

Boolean

Default true

Description

This variable controls whether tool should perform the on-ness check between the macro output pin and all of its loads, when the macro pin has the "is_isolated" attribute. The default value of this variable is *true*.

When this variable is set to true (the default), tool ignores the "is_isolated" attribute on the macro output pin. And tool checks if the macro output pin is equal or more always on than all of its loads. In the event it is not, tools reports MV-013 error message. When this variable is set to false, tool skips macro output pin to load on-ness check, on macro outputs that have "is_isolated" attribute.

If the macro output pin has alive_during_partial_power_down attribute set to true then, tool will ignore isolation checks between the macro output pin and all load pins irrespective of the setting of this variable.

See Also

- [check_mv_design](#)

mv.upf.allow_iso_on_dont_touch_networks

Data Types

boolean

Default true

Description

When this option is set to true, isolation insertion engine is allowed to insert isolation cells on dont touch networks.

See Also

- [set_app_options](#)

mv.upf.allow_ls_on_dont_touch_networks

Data Types

boolean

Default true

Description

When this option is set to true, level shifter/enable level shifter insertion engine is allowed to insert LS/ELS cells on dont touch networks.

See Also

- [set_app_options](#)

mv.upf.allow_ls_on_ideal_networks

Directs automatic level shifter insertion to insert level shifters on ideal nets.

Data Types

Boolean

Default true

Description

When this app option is set to true, automatic level shifter insertion will insert level shifters on all ideal nets that need level shifters.

By default, automatic level shifter insertion will insert level shifters on ideal nets.

See Also

- [create_mv_cells](#)
- [set_ideal_network](#)

mv.upf.allow_ls_on_leaf_pins

Allows level shifter insertion on leaf pin (such as macro cell pin) boundaries.

Data Types

Boolean

Default false

Description

This application option controls whether level shifter insertion is allowed on leaf pin boundaries that are not power domain boundaries.

When a macro cell is operating at a voltage different from its surrounding logic, and you want level shifters to be inserted at the interface, the recommended flow is to define a power domain around the macro cell by specifying the macro cell as the root cell of a power domain.

If you do not define the macro cell as the root cell, level shifters are not inserted at the interface, because the interface is not a power domain boundary. By default, level shifters are inserted only at power domain boundaries.

If you cannot define a power domain around the macro cell, but still require level shifters to be inserted, use this application option to enable the nondefault behavior.

See Also

- [compile_fusion](#)
- [create_mv_cells](#)

mv.upf.allow_ls_on_logic_one

Allows level shifter insertion on logic one driver path.

Data Types

Boolean

Default true

Description

This application option controls whether level shifter insertion is allowed on logic one driver path

See Also

- [compile_fusion](#)
- [create_mv_cells](#)

mv.upf.allow_ls_on_logic_zero

Allows level shifter insertion on logic zero driver path.

Data Types

Boolean

Default false

Description

This application option controls whether level shifter insertion is allowed on logic zero driver path

See Also

- [compile_fusion](#)
- [create_mv_cells](#)

mv.upf.allow_negative_voltage

Allows the use of negative voltages for certain supplies of macros.

Data Types

Boolean

Default false

Description

This option enables the use of negative supplies for macros. By default the tool does not allow the use of negative voltages for macros. This app option is for limited use only.

See Also

- [add_port_state](#)
- [add_power_state](#)

mv.upf.allow_non_bias_domain_in_bias_scope

Allows the use of non-bias power domains in a bias enabled scope.

Data Types

Boolean

Default false

Description

This option enables the use of non-bias power domains in bias enabled scopes. By default the tool does not allow the use of non-bias power domains in a bias scope. This app-option can be set to true to allow non-bias power domains to be defined within scopes where enable_bias is set to true/derived. This app-option needs to be set prior to load_upf.

See Also

- [set_design_attributes](#)

mv.upf.allow_non_bias_ls_els_in_bias_domains

Allows the use of non-bias level shifters and enable level shifters in bias power domains.

Data Types

Boolean

Default false

Description

This option enables the use of non-bias level shifters and enable level shifters in bias power domains. By default the tool does not have this support. This app-option can be set to true to allow non-bias level shifters and enable level shifters to be used within bias domains. To mark the lib cells that would be permitted inside the bias domains one must define and set the following attribute:

```
define_user_attribute -type boolean -class lib_cell -name  
bias_cell_without_exposed_bias_pin
```

```
set_attribute -objects [get_lib_cells */*] -name bias_cell_without_exposed_bias_pin -value  
true
```

With this setting, the specified lib cells will ignore bias related checks, this may cause bad logic and verification issues and it is user responsibility to use this attribute.

See Also

- [define_user_attribute](#)

mv.upf.allow_non_bias_macros

Allows the use of non-bias macros in a bias enabled design.

Data Types

Boolean

Default true

Description

This option enables the use of non-bias macros in a bias enabled design. By default the tool does not allow the use of non-bias blocks in a bias-enable design. This app option overrides the default behavior for macros.

See Also

- [set_design_attributes](#)

mv.upf.allow_rbm_in_upf_prime_flow

Enables rule based matching in UPF Prime flow when set to true.

Data Types

Boolean

Default false

Description

Specifies whether rule based matching will be allowed while loading UPF during UPF Prime flow. This only supports the default Golden UPF query rules, so any options given with `set_query_rules` will be ignored. This option is ignored when running in Golden UPF flow.

See Also

- [set_query_rules](#)

mv.upf.allow_retn_clamp_on_dont_touch_networks

Data Types

boolean

Default true

Description

When this option is set to true, retention clamp cell insertion is allowed to insert retention clamp cells on dont touch networks.

See Also

- [set_app_options](#)

mv.upf.ao_legalize_gtech_buf

Run always-on legalization on GTECH buffers and inverters in the netlist

Data Types

Boolean

Default false

Description

This app option is used mainly to fix MV violations of pre-instantiated buffers and inverters in the netlist. When the app option is set to true, both *compile_fusion* and *create_mv_cells* will run always-on legalization on GTECH buffers and inverters in the netlist. GTECH buffers or inverters with MV violations will be fixed.

The following types of MV violations will be fixed if possible:

- Isolation violation to/from GTECH buffers/inverters. Single-rail buffers/inverters will be swapped into dual-rail or vice versa.
- Voltage violation involving GTECH buffers/inverters (if no level shifter is required). Single-rail buffers/inverters will be swapped into dual-rail or vice versa.
- Incorrect supply net connections to GTECH buffers/inverters. Correct supply nets will be connected to PG pins of buffer/inverters via *connect_supply_net* in UPF.
- Power Domain - Voltage Area mismatch if GTECH buffers/inverters are placed. Physical feedthrough or logical feedthrough needs to be enabled in *create_voltage_area_rule -allow_physical_feedthrough true* or *create_voltage_area_rule -allow_logical_feedthrough true*.

See Also

- [check_mv_design](#)
- [create_mv_cells](#)
- [create_voltage_area_rule](#)
- [compile_fusion](#)
- [connect_supply_net](#)

mv.upf.auto_infer_supply_connections

Allows inference of supply net connections in the UPF based on PG net connections in the RTLPG netlist when `commit_upf` is executed.

Data Types

Boolean

Default `true`

Description

This option controls the inference of supply net connections in the power intent based on the connections of PG nets with the same names in the netlist upon execution of `commit_upf`.

If a PG net in the netlist has a corresponding UPF supply net with the same name in the power intent, the connection will be inferred in UPF using `connect_supply_net` based on the connections in the RTLPG netlist.

If set to false, supply net connections will not be inferred.

The `check_mv_design -power_connectivity` command will provide a list of nets that do not have matching supply nets in UPF.

This option is deprecated, please consult the man page for the new command *infer_supply_from_pg_net*.

See Also

- [commit_upf](#)
- [connect_supply_net](#)
- [infer_supply_from_pg_net](#)

mv.upf.auto_ls_clock_nets

Directs automatic level shifter insertion to insert level shifters on specified clocks.

Data Types

string

Default `"**"`

Description

This app option can be set to `""`, `"**"`, or a list of clock names (delimited by spaces or commas). When set to `"**"`, automatic level shifter insertion inserts level shifters on all clock nets that need level shifters.

When this app option is set to a list of clock names, automatic level-shifter insertion inserts level shifters on the net of these clocks, if needed.

When this app option is set to `""`, level shifter insertion will not insert level shifters on the nets driven by the clocks.

Note that if a net is ideal, automatic level shifter insertion won't insert any level shifter on it, unless the app option *mv.upf.allow_ls_on_ideal_networks* is set to *true*.

See Also

- [create_clock](#)
- [set_ideal_network](#)
- [set_level_shifter](#)

mv.upf.auto_resolve_pg_net

Data Types

Boolean

Default `true`

Description

If set to false, the `commit_upf` command will not try to resolve PG nets, i.e. it will not resolve potential conflicts between power intent and pg net connections.

See Also

- [commit_upf](#)
- [connect_supply_net](#)

mv.upf.bias_insulation_checks_rigidity_level

Controls the level of severity used by the tool to enforce bias insulation requirements.

Data Types

String

Default strict

Description

Normally, tool will require the usage of split-well cells when a rail-order violation or forward/reverse biasing condition is detected between a backup supply and it local bias supply.

This variable can be used to control the level at which the usage of split-well cell is enforced by the tool. The option accepts three values: *strict*, *light* and *none*. Setting any other value will result in an error.

The tool behavior at each different level is as follows:

- *strict*: default value. Tool is expected to use split-well cells at all times when required. Some compile prelude checks might cause the command to stop when the absence of split-well cells is detected.
- *light*: Tool is expected to use split-well cells at all times when required. Lib cell availability prelude checks will no longer cause compile to stop.
- *none*: Tool is allowed to use shared-well cells in place of split-well cells. Lib cell availability prelude checks will no longer cause compile to stop. Severity of all related messaged is downgraded to 'warning'

See Also

- [set_port_attributes](#)

mv.upf.connect_missing_backup_to_domain_primary

Controls whether dangling always-on (dual-rail) backup pg pins are left dangling or automatically connected to the power domain primary supply

Data Types

Boolean

Default false

Description

By default (app option set false), backup pg pins for always-on (dual-rail buffer or inverter) cells must be connected to a supply with the UPF connect_supply_net command. Otherwise, MV-221 errors will occur:

Error: Missing supply net information on pin %s with related power pin %s and related ground pin %s of cell %s (MV-221)

These error messages indicate that optimization will not behave correctly when the backup supply connections are missing. The related supply for the data pins on these cells cannot be derived due to the missing pg connection.

When this app option is set true, pg connections for dangling backup power pins on always-on cells are automatically derived to the primary supply of the power domain of the cell. Backup power pins will automatically connect to the power domain primary power, and backup ground pins (if they exist) will connect to the power domain primary ground.

Use caution when setting this app option to true. In most use models, the backup supply of always-on cells should not be connected to the domain primary. Typically (but not always) it should be connected to a more-on supply than the domain primary. So, this app option does not act as a simple resolution to any MV-221 errors. Understand the power intent of your design, and if the backup power pins should be connected to anything but the domain primary, use connect_supply_net to connect to the appropriate supply rail.

See Also

- [get_related_supply_nets](#)
- [connect_supply_net](#)
- [MV-221](#)

mv.upf.connect_supply_net_ignore_nonexistent_objects

Allows the connect_supply_net command to ignore nonexistent objects.

Data Types

Boolean

Default true

Description

This option controls whether the `connect_supply_net` command ignores nonexistent objects. By default, the `connect_supply_net` command errors out when a nonexistent object is specified.

See Also

- [connect_supply_net](#)

mv.upf.create_switch_place_holder_cells

create place holder cells for UPF switches.

Data Types

Boolean

Default false

Description

When this app option is set to true, `commit_upf` will create placeholder cells for every `create_power_switch` command in the UPF. If the `map_power_switch` command is given for the `create_power_switch` command, we will try to find the appropriate library cell from the cells specified in the `map_power_switch` command that matches the `create_power_switch` command. If `map_power_switch` command is not given or we can not find a matching library cells, then a dummy placeholder cell will be created. With the placeholder cells created, the `create_mv_cells` or `compile_fusion` command will now be able to insert level shifters or isolation cells on the switch signals as needed.

See Also

- [create_power_switch](#)
- [map_power_switch](#)

mv.upf.dft_reuse_existing_iso_cell

With this app option, dft connection may tap the fanout of existing isolation cells to make new connection.

Data Types

Boolean

Default true

Description

This app option controls whether or not dft connection to tap output of existing isolation cells on the domain boundary port driven by the dft driver to make new connection.

When making new DFT connection, the driver may already drive domain boundary port with existing isolation cells. In this case, tool will make the connection from the isolation output port for the new DFT connection if the new load to be connected has the same supply of existing load. By setting this app option to false, tool will not consider output of such isolation cells.

mv.upf.disable_is_analog_support

This app option disables the support for is_analog attribute. The tool will not make any difference between pins with and without is_analog attribute, i.e., it will not perform special handling for analog nets.

Data Types

Boolean

Default false

Description

This app option controls whether or not the tool should support analog nets in MV. The app option is by default set to off (false), so analog support is by default on.

Analog pins and ports are either marked in the liberty file (for leaf cells), or they are marked using set_port_attributes -is_analog command. A net will be marked as analog if there is at least one pin/port with is_analog attribute connected to it.

Level-shifter, isolation cells, and repeaters will not be inserted on analog nets.

By setting this app option to true (i.e. disabling the analog support), the set_port_attribute -is_analog will be disallowed. Further, the tool will not check for analog nets during commit_upf, association and insertion of level-shifter, isolation cells, and repeaters. In other words, it will treat all nets equally.

See Also

- [set_port_attributes](#)

mv.upf.disable_upf_loading_on_non_upf_design

Prevent the loading of UPF constraints on a non-UPF design.

Data Types

Boolean

Default false

Description

This option prevents the tool from processing UPF constraints. It has no effect if used on a design where UPF constraints are already present.

See Also

- [load_upf](#)

mv.upf.enable_amvf_html_report

Enables analyze_mv_feasibility report in HTML format.

Data Types

Boolean

Default false

Description

This application option, when set to true, enables analyze_mv_feasibility report in HTML format. By default, the application option is set to false and analyze_mv_feasibility reports only in text for unmapped ISO/ELS cells inserted in the flow. With this option enabled, if there is any ISO/ELS cell cannot be mapped, a report in HTML format will also be created besides the text report, using the file name reported by the message UPF-909. In the case all cells can be mapped, HTML report file won't be created.

See Also

- [analyze_mv_feasibility](#)

mv.upf.enable_amvf_report

Enables analyze_mv_feasibility report after PM cell insertion and mapping.

Data Types

Boolean

Default true

Description

This application option, when set to true, enables `analyze_mv_feasibility` report generation after PM cell insertion and mapping when there is any LS/ELS/RETN insertion issue or ISO/LS/ELS/RETN mapping failure.

By default, the application option is true.

If the customer wants to skip the report in earlier design stage(MV not clean, PM cell insertion and mapping will fail) to avoid long report log file for each failure, turn this app option OFF,

See Also

- [analyze_mv_feasibility](#)

mv.upf.enable_golden_upf

Enables the golden UPF mode when set to true.

Data Types

Boolean

Default false

Description

This application option, when set to true, enables the golden UPF mode. By default, the application option is set to false and the golden UPF mode is disabled. In that case, the `save_upf` command writes out a full set of UPF commands that reflect the changes made by the tool to the UPF power intent of the design, as well as the UPF commands run during the tool session.

In the golden UPF mode, the same original "golden" UPF script file is used throughout the synthesis, physical implementation, and verification flow. The `save_upf` command writes out a supplemental UPF file that reflects only the changes made by the tool to the UPF power intent of the design. Downstream tools and verification tools use the golden and supplemental UPF files together to completely specify the power intent of the design. To use the golden UPF mode, set this application option to true before you execute any UPF commands. Once enabled, the golden UPF mode requires that you run UPF commands only by using the `load_upf` command. You cannot execute individual UPF commands at the shell prompt, except for UPF query commands.

In the golden UPF mode, the `save_upf` command writes out a supplemental UPF file, which contains only the UPF changes made by the tool during the session, such as tool-derived power intent and addition of power management cells. You can choose to write out a concise or verbose supplemental UPF file. A verbose file contains `connect_supply_net`

commands that connect supply nets to the power management cells; these commands are needed if the netlist written in the session does not include PG connections. If the netlist includes PG connections, you can write out a concise supplemental UPF file instead.

See Also

- [load_upf](#)
- [save_upf](#)

mv.upf.enable_insulation_checks_for_ls

Check for bias insulation requirements during LS/ELS mapping

Data Types

Boolean

Default false

Description

On Bias enabled designs, tool will perform checks on power management cells' supplies to determine if nwell/pwell insulated library cells are required during mapping. By default these checks are not performed for level_shifter or enabled_level_shifter library cells but can be enabled by setting this option to true.

See Also

- [report_mv_lib_cells](#)

mv.upf.enable_logic_expr_new_processing

Enables logic expressions in *add_power_state*, *create_power_switch*, and *set_retention* to be processed according to updated rules.

Data Types

Boolean

Default true

Description

This application option controls whether logic expressions in *add_power_state*, *create_power_switch*, and *set_retention* are processed according to syntactical checks and validation of object references in the expression.

By default (*true*), the logic expressions for these three commands will be subject to expression parsing and validation. If an error occurs, the command fails.

When set to *false*, for *create_power_switch* and *set_retention*, the tool will read in logic expressions without a strict validation of their correctness. This is to support backwards compatibility. For *add_power_state*, the tool will continue to validate the logic expression with expression parsing and validation, but any writing of output UPF will use the expression as it was given in the command.

See Also

- [add_power_state](#)
- [create_power_switch](#)
- [set_retention](#)

mv.upf.enable_missing_voltage_area

Allows the tool to automatically assign the default voltage area, DEFAULT_VA, to power domains that do not have a user-specified voltage area.

Data Types

Boolean

Default true

Description

This application option provides the tool with the flexibility to automatically assign power domains to the default voltage area, DEFAULT_VA. This application option does not change settings of power domains that already have a user-specified voltage area.

So generally, we want the flow to proceed with missing VA (inferred VA) as far as possible for all shells. Front-end commands such as *compile_fusion* also allow missing VA by default.

Back-end commands listed below allow missing VA only in the incomplete UPF flow. This means you should turn on both *mv.incomplete_upf.enable* and *mv.upf.enable_missing_voltage_area* to enable missing VA flow or you need to create voltage areas for those "missing VA" domains. Or else errors will be reported against "missing VA" case for them: 1. *place_opt/route_opt/clock_opt/refine_opt* 2. *create_buffer_tree* 3. *create_clock_drivers* 4. *synthesize_regular_multisource_clock_trees* 5. *synthesize_clock_trees* 6. *synthesize_multisource_clock_subtrees* 7. *synthesize_multisource_clock_taps* 8. *synthesize_multisource_global_clock_trees* 9. *add_buffer_on_route*

See Also

- [create_power_domain](#)
- [create_voltage_area](#)
- [set_voltage_area](#)
- [compile_fusion](#)
- [UPF-511](#)
- [UPF-242](#)

mv.upf.enable_nwell_only_support

This app option is needed for bias enabled nwell-only designs and must be set prior to loading the UPF.

Data Types

Boolean

Default false

Description

This app option must be set to *true* for bias enabled nwell-only designs. It must be set prior to loading the UPF. Failing to set this app-option can result in errors in `commit_upf` for nwell-only designs.

See Also

- [commit_upf](#)

mv.upf.enable_pg_pin_reconnection

Allows the `connect_supply_net` command to reconnect power and ground pins.

Data Types

Boolean

Default false

Description

This option controls whether or not to allow reconnection to power and ground pins. By default, the `connect_supply_net` command errors out when connecting a supply net to a PG pin that has an existing explicit supply net connection.

By setting this option to true, the `connect_supply_net` command reconnects the new supply net to the PG pin and the previous connection is discarded.

See Also

- [connect_supply_net](#)

mv.upf.enable_reordering_building_pst_table

Enable PST table building reordering algorithm

Data Types

Boolean

Default true

Description

This app option control the enabling of the PST table building reordering algorithm in the MV/UPF client to reduce runtime of building, querying for different MV/UPF clients.

mv.upf.generate_iso_with_pst_equivalence

Generate isolation strategies based on PST equivalence driver and load supplies.

Data Types

Boolean

Default false

Description

Normally, `generate_upf_strategies` use functional equivalent driver/load supplies to generate generate isolation strategies. When setting this app option to true, tool will compare driver/load supplies based on PST table.

See Also

- [generate_upf_strategies](#)

mv.upf.generate_iso_with_same_signal

Generate isolation strategies with equivalence driver and load supplies, and all matching isolation strategies have same isolation signal.

Data Types

Boolean

Default false

Description

`generate_upf_strategies` use equivalent driver/load supplies to generate isolation strategies. And there can be multiple strategies with equivalent driver/load supplies. When setting this app option to true, tool will only use strategies if all strategies have same isolation signal.

See Also

- [generate_upf_strategies](#)

mv.upf.gupf_report_missing_objects

Enables reporting of missing objects during Golden UPF reapplication.

Data Types

boolean

Default FALSE

Description

Setting this application option to true enables reporting of all missing objects when you reapply the golden UPF file to the design with the *load_upf* command.

In the golden UPF flow, the synthesis and physical implementation tools can change or remove objects in the design due to optimization, grouping, ungrouping, or expansion of wildcard names. Therefore, when you reapply the same golden UPF to the modified design, missing objects are expected as a normal part in the flow.

For this reason, by default, the tool does not report missing objects during reapplication of the golden UPF to the design, except for missing control signals identified by the *set_isolation_control*, *set_retention_control* and *create_power_switch* commands. These signals are not expected to be changed or removed by synthesis or physical implementation tools.

To enable reporting of all missing objects under these conditions, set the *mv.upf.gupf_report_missing_objects* to true. In that case, during reapplication of the golden UPF with the *load_upf* command, all missing objects are reported.

See Also

- [load_upf](#)

mv.upf.identify_clock_path

Allows the PM cell insertion/mapping engines to identify clock path. If clock path identified, PM cell insertion/mapping will insert CTS purpose (clock type) library cell.

Data Types

Boolean

Default true

Description

Setting this application option to *true* to guide the *create_mv_cells* and *compile_fusion* mapper to insert or map with CTS purpose(clock type) library cell. This may trigger timing update for up-to-date result.

If library cell purpose is not a concern for PM cell, can set it to false to avoid extra timing update.

See Also

- [create_mv_cells](#)
- [compile_fusion](#)

mv.upf.ignore_dangling_net_on_heterogenous_path

Specifies whether level shifter insertion engine should ignore dangling net on heterogenous path.

Data Types

Boolean

Default false

Description

Specifies whether level shifter insertion engine should ignore dangling net on heterogenous path, so the level shifter cell will be inserted in the right branch of the net segment with loads only.

See Also

- [create_mv_cells](#)

mv.upf.ignore_ls_main_rail

Allows level shifter insertion not to select Standard Cell Main Rail (SCMR) as the main power supply for level shifters.

Data Types

Boolean

Default false

Description

This application option controls whether or not level shifters use SCMR as their main power supply. By default, level shifters must get their main power supply from SCMR.

To allow the tool to select other power sources as the main power supply for level shifters, set this application option to *true*.

See Also

- [check_mv_design](#)
- [compile_fusion](#)
- [create_mv_cells](#)

mv.upf.ignore_ls_main_rail_ground_supply

Allows level shifter insertion not to select Standard Cell Main Rail (SCMR) as the main ground supply for level shifters.

Data Types

Boolean

Default false

Description

This application option controls whether or not level shifters use SCMR as their main power supply while ignoring the groupd supply checking. By default, level shifters must get their main power and ground supply from SCMR.

To allow the tool to select other ground source as the main ground supply for level shifters, set this application option to *true*.

See Also

- [check_mv_design](#)
- [compile_fusion](#)
- [create_mv_cells](#)

mv.upf.input_enforce_simple_names

Enforces the use of simple names for restricted commands as per the IEEE 1801 (UPF) standard.

Data Types

Boolean

Default true

Description

This option controls the use of hierarchical names for the arguments of certain restricted commands that require simple names as per the IEEE 1801 (UPF) standard.

The default value of this option is true. So, by default, the tool accepts simple names since the IEEE 1801 (UPF) standard requires them to be simple. If you specify hierarchical names for the option which expects simple names, tool will issue an error.

When you set this option to false, the tool allows you to use hierarchical names for the arguments of few selected commands.

See Also

- [load_upf](#)

mv.upf.input_supply_equal_ao_with_driver

Specifies whether isolation cell conversion of enable level shifter should use equal AOness supply net for input supply connection.

Data Types

Boolean

Default true

Description

Specifies whether isolation cell conversion of enable level shifter should use equal AOness supply net for input supply connection.

See Also

- [create_mv_cells](#)

mv.upf.insert_ls_on_user_cstr_only

Inserts level shifters only at the domain boundaries with a level shifter strategy.

Data Types

Boolean

Default false

Description

This app option controls whether level shifters can be inserted at power domain boundaries without a strategy. By default, level shifters can be inserted at power domain boundaries without a strategy. This gives more flexibility to the tool and has better QoR.

If you want to restrict the tool to insert level shifters only at power domain boundaries where the *set_level_shifter* command is specified, set the app option to *true*.

Notice that the tool will not insert level shifters at all power domain boundaries with *set_level_shifter* strategies, but only at those boundaries when level shifters are needed to fix voltage violations.

If you want a level shifter to be always inserted at a power domain boundary even if there is no voltage violation, please specify a *set_level_shifter -force_shift* command to the target power domain boundary.

See Also

- [create_mv_cells](#)
- [set_level_shifter](#)

mv.upf.iso_control_inv_prefix

Specifies the prefix to be used in inverter name inserted on iso/els control path by ISO/ELS insertion engine.

Data Types

string

Default ""

Description

This application option specifies the prefix to be used in the inverter cell name inserted on iso/els control path by ISO/ELS insertion engine.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)

mv.upf.iso_map_exclude_zpr_clamp_lib_cells

Exclude isolation lib cells specified in map_retention_clamp_cell from isolation cell mapping.

Data Types

String

Default false

Description

This option exclude isolation lib cells specified in map_rention_clamp_cell from isolation cell mapping. If there is map_isolation_cells specified with isolation strategy, this strategy will not be affected by this app option.

When there is no map_isolation_cells specified with isolation strategy, by default the tool will use all available isolation lib cells, and not be affected by this app option.

With this app option set to "true", isolation engine will not use any lib cells spcified in map_retention_clamp_cells.

With this app option set to "nor", isolation engine will not use NOR-ISO lib cells spcified in map_retention_clamp_cells.

See Also

- [map_isolation_cell](#)
- [map_retention_clamp_cell](#)

mv.upf.keep_map_isolation_library_cell_order

This app option can be set to true to keep the library cell order in the map_isolation_cell command for the isolation strategy

Data Types

Boolean

Default false

Description

When set to true, isolation insertion/mapping engine will use the library cell in the same order as specified in map_isolation command for the strategy. The order is from left to right order.

See Also

- [set_isolation](#)

mv.upf.keep_map_level_shifter_library_cell_order

This app option can be set to true to keep the library cell order in the map_level_shifter_cell command for the level shifter strategy

Data Types

Boolean

Default false

Description

When set to true, level shifter insertion/mapping engine will use the library cell in the same order as specified in map_level_shifter command for the strategy. The order is from left to right order.

See Also

- [set_level_shifter](#)

mv.upf.load_upf_continue_on_error

load_upf does not stop on errors.

Data Types

Boolean

Default true

Description

This option controls whether `load_upf` command stops reading the UPF file on errors. Similar to setting the shell variable `sh_continue_on_error` to `true`, but only applies to this particular `load_upf` command.

See Also

- [load_upf](#)

mv.upf.ls_auto_insertion_sink_domain_support

Allows the skipping of insertion of all level-shifter cells.

Data Types

Boolean

Default `false`

Description

Setting this application option to *true* to guide the *create_mv_cells* and *compile_fusion* commands to inserting level-shifter cells in sink/load domain if possible.

This application option is intended only to control the insertion of new level-shifter cells and does not apply to pre-existing level-shifter cells in the design.

See Also

- [create_mv_cells](#)

mv.upf.map_relax_voltage_range_check

Controls voltage range checking at matching multibit LS/ELS lib cell.

Data Types

Boolean

Default `false`

Description

When this application option is set to *true*, the tool will relax voltage range check at selecting LS/ELS lib cell.

To determine the current value of this application option, use the *get_app_option_value -name mv.upf.map_relax_voltage_range_check* command.

For a list of all compile application options and their current values, use the *report_app_options compile.** command.

mv.upf.max_supply_count_without_driver

Maximum limit of supply nets without legal driver in the design

Data Types

Integer

Default 7

Description

When a UPF supply net does not have a valid driver object, `commit_upf` issues a UPF-176 warning, indicating that there may be problems with the UPF constraints since all supplies should have a valid driver. By default, once seven of these warnings are issued, the tool then issues a UPF-177 error, indicating that too many supply nets without drivers have been detected.

This app option controls the threshold for issuing the UPF-177 error. If a large number of UPF-176 warnings are expected with the current UPF constraints, this app option can be used to increase the UPF-177 threshold and avoid the error. Regardless of whether the UPF-177 error message is issued or not, any supply nets which missing drivers should eventually be cleaned up to ensure proper and complete power intent.

Integer between 1 and 1000 is allowed.

What Next

UPF-176 warning messages indicates which supplies do not have valid drivers. See the UPF-176 manpage for additional information.

See Also

- [commit_upf](#)
- [connect_supply_net](#)

mv.upf.multi_pd_shared_voltage_area_flow

Specifies the `multi_pd_shared_voltage_area` flow type.

Data Types

enum

Default legacy

Description

This application option specifies the `multi_pd_shared_voltage_area` flow type.

Valid values are

- *legacy* (the default)
- *new*

mv.upf.multi_pd_single_va_strict_check

Data Types

Boolean

Default `true`

Description

With the default value (*true*) this application option allows multiple Power Domains with same primary/default supplies and available supplies to be added to a single Voltage Area.

When this application option is set to *false*, allows to add multiple Power Domains with *PST-equivalent primary/default supplies* to a single primary Voltage Area. It also allows to add multiple Power Domains with different available supplies to a single primary or gas-station type Voltage Area.

See Also

- [set_app_options](#)
- [create_voltage_area](#)
- [set_voltage_area](#)

mv.upf.name_map_version

Specifies the default version of name mapping used for applying UPF to the design.

Data Types

enum

Default `none`

Description

This application option specifies the default version of name mapping used for applying UPF to the design.

Valid values are

- *none* (the default)

Name mapping is not applied to UPF constraints.

- *v1.0*
- *v2.0* Introduces the *hierarchical_name_conversions* command. This value is supported only in back-end mode.

See Also

- [load_upf](#)

mv.upf.no_buffering_after_retention_clamp_cell

Disallows buffer insertion between retention clamp cell and retention cell.

Data Types

Boolean

Default false

Description

This application option gives users the ability to prevent the buffer insertion on the net between retention clamp cell and retention cell. By default, buffer insertion is allowed on the net.

See Also

- [create_mv_cells](#)
- [compile_fusion](#)

mv.upf.no_tie_off_upf_ctrl_signal

UPF control signals without a driver will not be tied to logic constant value.

Data Types

Boolean

Default false

Description

By default, during synthesis, tool will tie UPF control signals without a driver to logic constant values. Set this option to *true* to leave them untied.

UPF control signals correspond to pin or nets in the design that are designated in UPF power management strategies as control elements for power management cell behavior: isolation control signal, retention save/restore signal, power switch control/ack signal.

See Also

- [set_isolation](#)
- [set_retention](#)
- [create_power_switch](#)

mv.upf.nor_iso_macro_allow_enable_supply_check

Controls if on-ness checks are to be done for all enable pins of nor(or nand) isolated macro output pins with multi-input isolation enable condition.

Data Types

Boolean

Default *true*

Description

This variable controls how the tool performs on-ness checks between enable pins of a nor(or nand) isolated macro output pin. By default the value of this variable is *true*. A warning message is issued if any enable pin is less always on or unrelated to one(or more) load pin(s).

User has a way to avoid this warning by setting this variable to *false*. In this case, tool only requires at least one enable pin to be equal or more always on than all load pins.

See Also

- [check_mv_design](#)

mv.upf.object_namespace_checks

Check the UPF object name in the whole UPF name space.

Data Types

Boolean

Default true

Description

This option controls whether to check the UPF object name in the whole UPF name space. If set to false, the behavior will be rolled back to keep backward compatability.

See Also

- [load_upf](#)
- [UPF-336](#)
- [UPF-337](#)
- [UPF-338](#)

mv.upf.power_model_library

This app option specifies a list of power model UPF files tool should read.

Data Types

String

Default ""

Description

This app option specifies a list of power model UPF files in the directory specified by app option *mv.upf.power_model_search_path* that tool should read at the beginning of the *load_upf* command. Only the *define_power_model* command in these UPF files will be processed.

This app option should be set before the first *load_upf* command is executed.

Example:

```
prompt> set_app_options -as_user_default  
                        -name mv.upf.power_model_library  
                        -value "model_1.upf model_2.upf model_3.upf"
```

See Also

- [apply_power_model](#)
- [define_power_model](#)
- [report_power_model](#)

mv.upf.power_model_search_path

This app option specifies the path to search for power model UPF files.

Data Types

String

Default ""

Description

This app option specifies a list of directory paths that store power model UPF files. Tool should search for power model UPF files specified by app option *mv.upf.power_model_library* in this list of directory paths.

Power models will be read in the same order as directory paths and library files appear in these app options. Duplicated power models with the same names will be skipped.

This app option should be set before the first *load_upf* command is executed.

Example:

```
prompt> set_app_options -as_user_default
                        -name mv.upf.power_model_search_path
                        -value
                        "/usr/lib/power_model_lib ./power_model_lib"
```

See Also

- [apply_power_model](#)
- [define_power_model](#)
- [report_power_model](#)

mv.upf.power_model_verbose

This app option enables verbose mode when a power model is applied to a macro instance.

Data Types

Boolean

Default TRUE

Description

When this app option is set to TRUE, UPF commands executed during the *apply_power_model* command will be echoed to standard output stream. *UPF-410*

message will also be printed for each macro instance to which the power model is applying.

Example:

```
set_app_options -as_user_default -list {mv.upf.power_model_verbose TRUE}
set_scope /
define_power_model MODEL {
    create_power_domain PD_MACRO_TOP
    create_supply_net VDD
}
apply_power_model MODEL -elements {macro_0 macro_1}
```

```
Information: Applying power model 'MODEL' in top scope to macro_0.
(U PF-410)
set_scope macro_0
create_power_model PD_MACRO_TOP
create_supply_net VDD
set_scope /
Information: Applying power model 'MODEL' in top scope to macro_1.
(U PF-410)
set_scope macro_1
create_power_model PD_MACRO_TOP
create_supply_net VDD
set_scope /
Information: Applied power model 'MODEL' in top scope to 2 instances.
(U PF-398)
```

See Also

- [apply_power_model](#)
- [define_power_model](#)
- [report_power_model](#)

mv.upf.preserve_hierarchical_pins_with_upf_constraints

Controls whether the tool sets size_only on hierarchical pins with an isolation or level-shifter strategy defined on them.

Data Types

Boolean

Default false

Description

This application option if set to true, sets `size_only` on hierarchical pins with isolation or level-shifter strategies defined on them. This will help preserve these pins and prevent them from being optimized away.

See Also

- [compile_fusion](#)
- [create_mv_cells](#)

mv.upf.preserve_logic_in_boolean_expr

Enables preservation of referenced logic ports and nets in `add_power_state` and `set_retention` logic expressions.

Data Types

Boolean

Default `false`

Description

This application option allows for the preservation of logic ports and nets in `add_power_state` and `set_retention` logic expressions, such that logic ports cannot be ungrouped, if hierarchical, and cannot be removed. Referenced logic nets will instead force the preservation of its driving port in the same manner.

By default (*false*), tools that can generate changes in the netlist will not take any measures in preserving these ports and nets.

This application option must be set before constraints are loaded.

See Also

- [add_power_state](#)
- [set_retention](#)

mv.upf.preserve_power_domain_root_cells

This app option can be set to true to prevent empty hierarchies that are power domain root cells from being optimized away in compile.

Data Types

Boolean

Default false

Description

This app option can be set to true to prevent empty hierarchies that are power domain root cells from being optimized away in compile.

See Also

- [commit_upf](#)

mv.upf.prevent_uniquification_in_load_commit_upf

When this app-option is true, the tool will prevent uniquification of MIM's in load/commit upf commands. And if it was not prevented it will issue a warning message.

Data Types

Boolean

Default true

Description

This variable with default value true, will affects the commands `commit_upf` and `load_upf`. It will prevent the creation of nets in those commands that would trigger uniquification and if a uniquification happens for other reasons, such as name change for association purposes, it will issue a warning message that the uniquification was not prevented to inform the user.

See Also

- [load_upf](#)
- [commit_upf](#)
- [uniquify](#)

mv.upf.report_pst_internal_states_names

This variable controls which names the supply states in the command `report_pst` -derived will use.

Data Types

Boolean

Default false

Description

If set to true the derived pst will be reported using the internally generated names.

If set to false and the supply states of the derived PST are representable with user defined names by `add_power_state`, those states will be reported using the supply sets as columns names. The state names will be the names specified `add_power_state` command instead of the internally generated names.

When using these user defined names if `-voltage_type` is given, the supply set column names will also contain the supplies used in the `add_power_state` command. The supplies of the supply set will be represented as:

[p] = power

[g] = ground

[Pwell]= pwell

[Nwell] = nwell

If `-supplies` is used with `-derived`, the report will be done with the internally generated names instead. If tool generated supply states are present in the derived pst, such as `voltage_reconciliation` generated states, the pst will be reported using internal names.

EXAMPLE 1 - Derived_pst representable with add_power_states.

```
create_supply_set SST -function {ground VSS} -function {power VDD1}
create_supply_set SSM -function {ground VSS} -function {power VDD2}
create_supply_set SSB -function {ground VSS} -function {power VDD3}

add_power_state SST -state S2 {-supply_expr {power == `{FULL_ON, 0.9} && ground ==
`{FULL_ON, 0.0}}}}

add_power_state SST -state S1 {-supply_expr {power == `{FULL_ON, 0.8} && ground ==
`{OFF}}}} add_power_state SSM -state S2 {-supply_expr {power == `{FULL_ON, 0.9} &&
ground == `{FULL_ON, 0.0}}}}

add_power_state SSM -state S1 {-supply_expr {power == `{FULL_ON, 0.99} && ground
== `{OFF}}}}

add_power_state SSB -state S2 {-supply_expr {power == `{FULL_ON, 0.9} && ground ==
`{FULL_ON, 0.0}}}}

add_power_state SSB -state S1 {-supply_expr {power == `{FULL_ON, 1.0} && ground ==
`{OFF}}}}

set_app_options -name mv.upf.report_pst_internal_states_names -value true

report_pst -derived -voltage_type all
```

Supplies : VDD1 | VDD2 | VDD3 | VSS |

States

(1) <drv> : SNPS_INT_S1 [0.80] | SNPS_INT_S1 [0.99] | SNPS_INT_S1 [1.00] |
SNPS_INT_S1 [off] |

(2) <drv> : SNPS_INT_S2 [0.90] | SNPS_INT_S2 [0.90] | SNPS_INT_S2 [0.90] |
SNPS_INT_S2 [0.00] |

set_app_options -name mv.upf.report_pst_internal_states_names -value false

report_pst -derived -voltage_type all

Supplies : SST[p][g] | SSM[p][g] | SSB[p][g] |

States

(1) <drv> : S1 [0.80][off] | S1 [0.99][off] | S1 [1.00][off] |

(2) <drv> : S2 [0.90][0.00] | S2 [0.90][0.00] | S2 [0.90][0.00] |

See Also

- [add_power_state](#)
- [report_pst](#)

mv.upf.save_upf_blackbox_preserve_user_supply_attributes

Uses user-specified *-driver_supply* / *-receiver_supply* attributes instead of tool-derived values during *save_upf -for_empty_blackbox*.

Data Types

Boolean

Default false

Description

The command *save_upf -for_empty_blackbox* is used to create a UPF file for an empty black_box version of the current design. As part of this process, the tool derives *-driver_supply* and *receiver_supply* port attributes matching the related_supply UPF associates with the boundary ports.

When this option is *false* (the default), the tool will ignore any user-specified *-driver_supply* and *-receiver_supply* attributes and write tool-derived attributes to the black_box UPF file.

When this option is set to *true*, the tool will instead propagate user-specified *-driver_supply* and *-receiver_supply* attributes to the black_box UPF file, even if they do not match the tool-derived values.

See Also

- [save_upf](#)

mv.upf.save_upf_include_supply_exceptions

Specifies whether save_upf must include explicit supply net connections in the supplemental UPF section.

Data Types

Boolean

Default true

Description

Handles whether explicit supply net connections for Isolation, Level Shifter, Buffer and Retention cells are written or not during *save_upf* command. Default value is to write all of them. The main usage of this application option is to reduce the size of *supplemental UPF*.

If you only want to control how explicit supply net connections are written only for Isolation and Retention cells, refers to *mv.upf.save_upf_include_supply_exceptions_for_iso_retn*.

See Also

- [save_upf](#)

mv.upf.save_upf_include_supply_exceptions_for_iso_retn

Specifies whether save_upf should include explicit supply net connections for isolation and retention cells in the supplemental UPF section.

Data Types

Boolean

Default true

Description

Specifies whether the supplemental UPF section in the output UPF should include explicit supply net connections (*connect_supply_net* command) to the PG pins of isolation and retention cells or not. Explicit supply net connections help subsequent tools to correctly

associate isolation and retention cells to their respective UPF strategies. If this app option is set to false, `save_upf` will not include explicit supply net connections for isolation and retention cells. Subsequent tools will then rely on the UPF strategies to rederive the PG pins connections to these power management cells.

See Also

- [save_upf](#)

mv.upf.save_upf_line_indent

Specifies the number of empty characters before the continuing lines, when a command is split into multiple lines. There is no indent for a first command line.

Data Types

Integer

Default 4

Description

This is a design scoped application option.

Specifies the number of empty characters before the continuing lines, when a command is split into multiple lines.

The value can be honored by `save_upf` to output UPF file, only when `save_upf` is in split mode.

The value is non-negative. The default value is 4.

See Also

- [save_upf](#)

mv.upf.save_upf_line_width

Specifies whether the `save_upf` command writes out commands as multiple lines, and if so, the threshold for splitting lines.

If the value is 0, it means to output UPF file in nosplit mode.

Data Types

Integer

Default 80

Description

This is a design scoped application option.

Specifies whether the `save_upf` command writes out commands as multiple lines, and if so, the threshold for splitting lines.

The value can be honored by `save_upf` to output UPF file, only when `save_upf` is in split mode.

The value is non-negative. The default value is 80.

See Also

- [save_upf](#)

mv.upf.save_upf_update_derived_bias

Specifies whether `save_upf` must write out derived bias connections with `create_supply_set -update`.

Data Types

Boolean

Default false

Description

Specifies whether the supplemental UPF section in the output UPF should include derived bias connections due to `enable_bias` derived.

See Also

- [save_upf](#)

mv.upf.save_upf_use_relative_path_for_top_scope

Specifies whether `save_upf` should save `'set_scope /'` command in terms of path relative to the current scope

Data Types

Boolean

Default true

Description

When user specifies 'set_scope /' to go back to the top most scope, this app option will tell save_upf to save '/' in terms of path relative the scope at which set_scope gets executed. This is useful in bottom up design methodology, where block UPF contains 'set_scope /'. When such block UPF is loaded from the top UPF, 'set_scope /' inside the block UPF will set the scope to the top of the chip instead of top of the block. But block UPF meant to set the scope to the top of the block instead. This app option will convert '/' to required number of '../' characters to set the scope to top of the block.

See Also

- [save_upf](#)

mv.upf.set_els_cell_naming_rule

Guide enable level shifter insertion to name enable level shifter cell with naming rule.

Data Types

string

Default default

Description

This application option controls the enable level shifter naming rule, Default rule will not append _UPF_LS as suffix for enable level shifter inserted if no user suffix defined in the level shifter strategy(or no level shifter strategy defined by auto insertion). Legacy rule will append _UPF_LS as suffix for enable level shifter inserted if no user suffix defined in the level shifter strategy(or no level shifter strategy defined by auto insertion). This is usually helpful to help customer to identify the enable level shifter is inserted by the tool or manually inserted in the design.

See Also

- [check_mv_design](#)
- [compile_fusion](#)
- [create_mv_cells](#)

mv.upf.skip_all_iso_insertion

Allows the skipping of insertion of all isolation cells.

Data Types

Boolean

Default false

Description

Setting this application option to *true* prevents the *create_mv_cells* and *compile_fusion* commands from inserting isolation cells in any part of the design.

This application option is intended only to control the insertion of new isolation cells and does not apply to preexisting isolation cells in the design.

See Also

- [create_mv_cells](#)
- [compile_fusion](#)

mv.upf.skip_all_ls_insertion

Allows the skipping of insertion of all level-shifter cells.

Data Types

Boolean

Default false

Description

Setting this application option to *true* prevents the *create_mv_cells* and *compile_fusion* commands from inserting level-shifter cells in any part of the design.

This application option is intended only to control the insertion of new level-shifter cells and does not apply to preexisting level-shifter cells in the design.

See Also

- [create_mv_cells](#)
- [compile_fusion](#)

mv.upf.skip_all_retention_insertion

Prevents the insertion of any new retention cells.

Data Types

Boolean

Default false

Description

Setting this application option to *true* prevents the *create_mv_cells* and *compile_fusion* commands from inserting retention cells in any part of the design.

This application option is intended only to control the insertion of new retention cells and does not apply to preexisting retention cells in the design.

See Also

- [create_mv_cells](#)
- [compile_fusion](#)

mv.upf.skip_ao_check_for_els_input

Controls whether enable level-shifter cells inserted by the tool can use a power supply that is more always-on than the driver supply.

Data Types

Boolean

Default false

Description

An enable level-shifter is a power management cell that performs both isolation and level shifting between two power domains.

This application option controls whether enable level-shifter cells inserted by the tool can use a power supply that is more always-on than the driver supply.

By default (*false*), enable level-shifter cells inserted by the tool must use a power supply that is equally always-on compared to the supply for the driver of the inputs of the enable level-shifter.

When this application option is set to *true*, enable level-shifter cells inserted by the tool can use a power supply that is either equally always-on or more always-on than the supply for the driver of the inputs of the enable level-shifter.

See Also

- [create_mv_cells](#)

mv.upf.skip_dft_iso_when_no_violation

Data Types

boolean

Default false

Description

When this option is set to true, tool will not apply isolation strategy specified by `set_dft_isolation` when the existing path doesn't have isolation violation. Redundant isolation could still happen when both driver domain and load domain have `set_dft_isolation` specification.

See Also

- [set_app_options](#)

mv.upf.skip_ls_on_block_crossing_nets

Skips insertion of level-shifter cells on nets between physical blocks.

Data Types

Boolean

Default false

Description

This application option gives users the flexibility to prevent insertion of level-shifter cells in the `create_mv_cells` or `compile_fusion` command on two types of nets:

- Nets that connect to top-level ports
- Nets connecting to physical block ports

See Also

- [create_mv_cells](#)
- [compile_fusion](#)

mv.upf.skip_retention_on_dft_register

Skips register created by DFT engine from getting mapped to retention register.

Data Types

Boolean

Default false

Description

This application option, when set to true, skips register created by DFT engine from getting mapped retention register if they are found in a power domain having domain specific retention strategy. DFT register will be mapped to retention register when this application option is set to false.

This application option must be set before running the *create_mv_cells* or *compile_fusion* command.

See Also

- [set_retention](#)
- [insert_dft](#)
- [create_mv_cells](#)
- [compile_fusion](#)

mv.upf.skip_single_rail_for_non_primary_iso_supply

Control the Isolation cell mapper to skip using single rail library cell when dual rail library cell required but not available.

Data Types

Boolean

Default false

Description

This application option controls the Isolation cell mapper to skip using single rail library cell when dual rail library cell required but not available. If the option is set to true, the mapper will keep the Isolation cell unmapped if dual rail isolation library cell is not available, including all PVT matching checking. This happens for isolation supply does not match with domain primary power supply, dual rail isolation cell is necessary to be used.

To allow the mapper to use single rail library cell, keep this application option as *false*. Otherwise, set it to true.

See Also

- [check_mv_design](#)
- [compile_fusion](#)
- [create_mv_cells](#)

mv.upf.system_pst_building_time_limit

Set the time limit for building system-level power state table.

Data Types

Integer

Default 3600

Description

This application option specifies the time limit for building system-level power state table, which merges all the PSTs and power state table in the design for electrical violation checks.

See Also

- [commit_upf](#)
- [report_pst](#)

mv.upf.unmap_iso_with_els_failure

The default value of this app option is false. When the path has both ISO and Voltage violation, but only ISO cell can be inserted, it cannot be converted to ELS due to some reason, the ISO will be mapped as ISO cell only, leaving the path with voltage violation.

When this app options is set to true, PM cell insertion and mapping engines will leave such ISO cell as unmapped when Enable Level Shifter is needed but engine fails to convert the ISO into ELS. In `compile_fusion` flow, the flow will stop with unmapped PM cell, customer can check the report and fix the problem in the earlier stage.

Data Types

Boolean

Default false

Description

This application option controls whether or not to leave the ISO cell as unmapped or continue to map it if ELS is needed in the path but the engine cannot convert the ISO into ELS due to some reason. If it's in compile_fusion flow, the flow will stop with unmapped PM cells, customer can check the reason and fix the design issue, e.g. library cell not available, UPF constraints issue, etc.

See Also

- [analyze_mv_feasibility](#)
- [check_mv_design](#)
- [compile_fusion](#)
- [create_mv_cells](#)

mv.upf.upf_add_power_state_21_syntax

Enable the support for UPF 2.1 syntax for -state option of the add_power_state command

Data Types

Boolean

Default false

Description

This app option enables the user to use the UPF 2.1 syntax for the -state option of the add_power_state command. Before UPF 2.1, the -state option took two values, name of the state and a string describing the state. For example, the following add_power_state command uses the pre UPF 2.1 syntax for the -state option of add_power_state command.

```
add_power_state SS -state STATE1 {-supply_expr {power == {OFF}}}
```

Now the following command shows the UPF 2.1 syntax for the -state option of the add_power_state command

```
add_power_state SS -state {STATE1 -supply_expr {power == {OFF}}}
```

Notice that in 2.1 syntax, -state command only takes one string as an option and the state name is the first thing in that string.

This app option is off by default. It means by default, -state takes two values. This option must be set before any power states are created in the design. Its value can not be changed once power states have been created in the design.

This app option will be removed in future releases.

See Also

- [add_power_state](#)

mv.upf.upf_add_power_state_ignore_logic_expression_error

Controls whether the *add_power_state* UPF command skips supply state logic expression syntax checking.

Data Types

Boolean

Default false

Description

This application option controls whether the *add_power_state* command skips supply state logic expression syntax checking.

By default (*false*), some basic Boolean equation syntax checking is performed. If an error occurs, the command fails.

When set to *true*, the tool ignores logic expressions for supply states. It keeps the logic expression in the UPF, but does not process or validate it.

See Also

- [add_power_state](#)

mv.upf.upf_allow_non_constant_net_reconnect

Enable the *-reconnect* option of *connect_logic_net* command to be used on non-constant nets

Data Types

Boolean

Default true

Description

This app option enables the user to use *connect_logic_net -reconnect* on nets that are not a constant. LRM recommends limiting *-reconnect* option only for constants, however Synopsys tools support also for non constant nets with this app_option.

See Also

- [connect_logic_net](#)

mv.upf.upf_enable_state_propagation_in_add_power_state

Enables propagation of supply set power states to the supply set functions

Data Types

Boolean

Default false

Description

When this app option is set to true, the power states defined on the supply sets are propagated to their functions and the supply nets and ports that they are connected to. These power states can be used on in the create_pst commands. When this app options is set, it overrides the design attribute enable_state_propagation_in_add_power_state. This app option should be set before loading any UPF or opening a block that has UPF.

For more details on the power state propagation to supply set functions from supply sets, please see the set_design_attribute man page section on enable_state_propagation_in_add_power_state attribute.

By default, supply set power states are not propagated to the supply set functions and can not be used in the create_pst command.

This app option will be removed in future releases. Please use the design attribute enable_state_propagation_in_add_power_state instead.

See Also

- [set_design_attributes](#)
- [add_power_state](#)
- [create_supply_set](#)

mv.upf.upf_skip_diff_supply_for_gmvc

Allows the skipping of isolation strategy -diff_supply_only during generate_mv_constraints.

Data Types

Boolean

Default true

Description

Setting this application option to *true* prevents the *generate_mv_constraints* commands from updating diff_supply isolation strategies.

Setting this to false if needs generate_mv_constraints to update the diff_supply isolation strategies.

See Also

- [generate_mv_constraints](#)

mv.upf.upf_use_insulated_cells_in_leq

Enables insulated bias cells to be forcefully picked for use by LEQ even if shared-well cells are legal to be used in the given context.

Data Types

Boolean

Default false

Description

This option enabled insulated-well cells to be forcefully chosen by LEQ even if shared-well cells are legal to be used in a given context. User must have additionally set up preference of insulated cells via set_power_domain_constraints.

mv.upf.use_driver_receiver_for_io_supplies

Controls whether derived supply for top-level port from connected pad cells.

Data Types

Boolean

Default true

Description

Normally, the supply of the top level port is taken from the supply of the top level power domain. When the app option is true, and the port is connected with a pad cell, and the supply of the pad cell differs from the top level power domain, then the supply of the top level port is replaced. Instead of using the default domain supply, the supply of the pad cell is used.

See Also

- [commit_upf](#)
- [MV-140](#)
- [MV-141](#)
- [MV-142](#)
- [MV-143](#)
- [MV-144](#)

mv.upf.use_preswitched_supply_for_macro_pins

Use preswitched supply for related_supply for macro data pins related to internal pg pin without connect_supply_net.

Data Types

Boolean

Default false

Description

This option controls whether or not to use preswitched supply for related_supply for macro data pins related to internal pg pin without connect_supply_net. By default, tool will return an empty supply net for such case.

By setting this option to true, tool will return a pre-switched supply net connected to pins specified by pg_function of the the internal pg pin.

It only applies to macro data pins with input direction.

See Also

- [get_related_supply_nets](#)

mv.upf.virtual_supply_available_when_connected_to_regular_pg

Make virtual supply available anywhere when it is connected to regular PG pins.

Data Types

Boolean

Default false

Description

This app option makes virtual supply available anywhere when it is also connected to regular PG pins(primary/backup). By default, a supply is considered as virtual and not available for buffering/level shifter insertion when it is connected to internal PG pins. A virtual supply is only available in the power domain where the supply is explicitly used, which includes domain primary, default supply, available supply or connected to regular PG. With this app option, a supply connecting to internal PG pin will be made available in all power domains if it is also connected to some regular PG pin.

See Also

- [connect_supply_net](#)
- [report_power_domain](#)

mv.upf.write_create_power_domain_with_scope

Specifies whether -scope option of create_power_domain command is allowed to be written out.

Data Types

Boolean

Default true

Description

The option specifies whether -scope option of create_power_domain command is allowed to be written out. When the app option value is true, -scope option of create_power_domain command is allowed to be written out. When the app option value is false, -scope option of create_power_domain command will not be written out by the tool.

This option does not affect the user section of save_upf output as UPF constraints in user section follow what are specified in the input UPF.

See Also

- [create_power_domain](#)
- [save_upf](#)
- [split_constraints](#)

mv.upf.write_crosstool_no_upf_update_command

Specifies whether crosstool compatible UPF can include the *-update* option on power domain and strategy constraints.

Data Types

Boolean

Default false

Description

With this application option, you can prevent the exported crosstool compatible UPF from having any UPF strategies or *create_power_domain* constraints that use the *-update* option. The purpose of this is export UPF for other tools that do not yet support this newer UPF *-update* construct.

This application option has no effect unless crosstool compatible UPF export is enabled by the *mv.upf.write_crosstool_wrapper* application option.

For example, suppose the original user UPF file includes this constraint:

```
create_power_domain PD1 -elements {u1}
```

During optimization, physical feedthrough buffers are physically placed in the voltage area of PD1. This is done by creating "power domain on instance" (PDOI) constraints for those buffers in the UPF. So after the physical feedthrough buffers are added, the user UPF section is the same, but the derived UPF section includes the new PDOI buffer instances:

```
create_power_domain PD1 -elements {u1}
set_derived_upf true
if {$derived_upf} {
    create_power_domain PD1 -elements {u2/pft_buf1 u2/pft_buf2} -update
}
```

Now, suppose crosstool compatible UPF export is enabled:

```
set_app_options -name mv.upf.write_crosstool_wrappers -value true
```

When UPF is exported, PDOI buffers are replaced by wrapper hierarchies, so the crosstool UPF looks like:

```
create_power_domain PD1 -elements {u1}
if {$derived_upf} {
    create_power_domain PD1 -elements {u2/SNPS_VAO_HIER_PD1} -update
}
```

Now, suppose crosstool compatible UPF export is enabled, along with this application option to prevent *-update* strategies in the UPF:

```
set_app_options -name mv.upf.write_crosstool_wrappers -value true
set_app_options -name mv.upf.write_crosstool_no_upf_update_command \\\
  -value true
```

In this case, it is only possible to avoid creating *-update* statements by modifying the user UPF section directly. So the resulting crosstool compatible UPF looks like this, with nothing in the derived section for this power domain:

```
create_power_domain PD1 -elements {u1 u2/SNPS_VAO_HIER_PD1}
```

mv.upf.write_crosstool_wrappers

Enables wrapper creation for instances with power domain on instance (PDOI) for crosstool compatibility.

Data Types

Boolean

Default false

Description

IC Compiler II supports PDOI for improved optimization of data and clock paths. Since other tools may not support PDOI, IC Compiler II can write a compatible netlist and UPF for ASCII migration to other tools by setting mv.upf.write_crosstool_wrappers app option to true. This option enables creation of hierarchical modules for leaf instances which have PDOI. This app option is only needed when physical feedthrough buffering has been enabled (opt.common.allow_physical_feedthrough) and physical feedthrough cells were added during optimization or CTO by the tool. You can check for the existence of PDOI cells in the design by looking at the elements attribute of power domains and filtering out hierarchical cells and macros:

```
get_cells [get_attribute [get_power_domains ] elements] -f !is_hierarchical&&!  
is_hard_macro -quiet
```

This app option only impacts the output of write_verilog command. The state of this option does not impact implementation. When this app option is set to true, the write_verilog command will behave differently in two ways:

1. write_verilog will perform on-the-fly processing to place the PDOI cells into new wrapper hierarchies
2. write_verilog will also write a UPF file that matches the netlist with wrapper hierarchies

Please note, this app option does not impact the `save_upf` command, so the output of `save_upf` can be different than the UPF written from `write_verilog` command. The output of `save_upf` should not be used for ASCII migration when `mv.upf.write_crosstool_wrappers` app option is set to true.

mv.upf.write_csn_for_dual_rail_cells_in_ao_domain

Write `connect_supply_net` for dual rail cells in the AO block domain.

Data Types

Boolean

Default `false`

Description

When this app option is set to true, `save_upf` will write out `connect_supply_net` for the backup pin of every dual rail cell in the AO block domain. the `connect_supply_net` command will connect the backup pin to the primary power of the AO block domain. `connect_supply_net` will not be written out for the isolation cell's back pins. These pins are implicitly connected to the isolation supply.

See Also

- [create_power_domain](#)
- [connect_supply_net](#)

mv.upf.zpr_clamp_cell_prefix

Specifies the prefix to be used in the retention clamp cells name.

Data Types

string

Default `""`

Description

This application option specifies the prefix to be used in the retention clamp cell name.

o

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)

o

opt.area.effort

Control the optimization effort for area recovery in place_opt/clock_opt/refine_opt commands.

Data Types

String

Default low

Description

Enable higher effort area recovery in preroute flows.

The area recovery effort can be: low, medium, high or ultra. Area recovery trades off between TNS and area. Area recovery effort low recovers the least area and degrades TNS the least, while area recovery effort ultra recovers the most area and degrades TNS the most.

See Also

- [place_opt](#)
- [set_app_options](#)

opt.buffering.check_for_skinny_blockages

Enables checking if and which setting can help reducing runtime when using virtual routing engine.

Data Types

boolean

Default true

o

Description

This app option controls if a check should be done if the design floorplan contains long but thin obstacles. Those obstacles will create a large number of extra detour points impacting the runtime of the virtual routing engine. If those obstacles are found and do not overlap a lot with standard cells message OPT-505 is printed stating it will ignore them. If they overlap a lot with standard cell the message OPT-504 is printed. It will specify a recommended setting of app option *opt.buffering.min_skinny_blockage_size*. Be aware of the information noted in the man page of that app option.

opt.buffering.enable_advanced_buffering

Enables the advanced buffering engine for high fanout net synthesis

Data Types

boolean

Default true

Description

This app option is deprecated in R-2020.09-SP3. As the legacy buffering engine is going to be retired, this app option will be obsoleted in the future release.

This app option controls the behavior of the high fanout net synthesis in *place_opt*. This is the initial stage of buffering that is run in the flow, occurring during the *initial_drc* stage of *place_opt*. It also impacts behavior of any subsequent calls to the *place_opt initial_drc* stage, for example in two-pass placement flows. It also controls the behavior of the *create_buffer_trees* command for standalone buffer tree building.

When this app option is set to *true*, the advanced buffering high fanout net synthesis engine is called. When it is set to *false*, the legacy buffering engine is used. The advanced buffering engine gives general QOR improvement compared to the legacy buffering engine, as measured at the end of preroute optimization, or at the end of the full flow. Typical benefits include reduced total wirelength, power, area, and timing QOR improvement.

Most other existing buffering app options do not affect the advanced buffering engine, though they can still affect other buffering steps in the flow, like incremental buffering that occurs in the *place_opt initial_opto* and *final_opto* flow stages. This includes app options like *opt.common.buffering_for_advanced_technology* and *opt.common.drc_mode_buffering*.

By default, the virtual router is used to build the buffering topologies. But, advanced buffering also supports "GRopto" mode, where the global router is used to guide the buffer tree building. Global route based buffering incurs a higher runtime cost during the *initial_drc* stage, due to the congestion map update that is required. But, it can give QOR

o

benefit, particularly on channel-based congested designs, where the early knowledge of congestion during initial buffering can give a better starting point to subsequent *place_opt* optimization steps.

"GRopto" mode is enabled in *place_opt initial_drc* by either or both of these app options:

- *place_opt.initial_drc.global_route_based* set to 1 •
- *opt.common.use_route_aware_estimation* set to *true*

The first app option is the same "GRopto" mode app option that was also used by the legacy high-fanout buffering engine. The second app option is the one that enables "global route layer binning" (GRLB). That app option triggers a global route congestion map update in *place_opt initial_drc* anyway, so advanced buffering will take advantage of that available information to do global route based buffering topology generation.

These app options do not enable "GRopto" mode in the *create_buffer_trees* command. Instead, use the *create_buffer_trees -global_route_based* option.

See Also

- [create_buffer_trees](#)
- [place_opt](#)

opt.buffering.enable_delay_tree_rebuffering_depth

This application option sets the maximum rebuffering levels during delay optimization.

Data Types

integer

Default 0

Description

Most buffer tree optimization after *place_opt initial_drc* is incremental, working on tuning the existing buffer trees. Rebuffering fully removes buffer trees or parts of buffer trees and completely rebuilds them, keeping the solution if improved QoR is seen.

Rebuffering is done in the default flow in some limited cases, mainly to rebuild buffer chains after layer promotion.

This application option is allowed to specify a depth number that limits the maximum of rebuffering levels during delay optimization.

o

opt.buffering.enable_flow_rebuffering

This application option enables the rebuffering during logical DRC and delay optimization.

Data Types

integer

Default 0

Description

Most buffer tree optimization after place_opt initial_drc is incremental, working on tuning the existing buffer trees. Rebuffering fully removes buffer trees or parts of buffer trees and completely rebuilds them, keeping the solution if improved QOR is seen. Rebuffering optimizes for multiple costs, including setup timing, logical DRCs, wirelength, and area. If a rebuffered solution degrades QOR, it is rejected and the original solution is kept, so rebuffering should not degrade QOR.

Rebuffering is done in the default flow in some limited cases, mainly to rebuild buffer chains. Setting this app option to 1 enables more general rebuffering in the place_opt final_opto stage. The expected impact is improvement to design QOR at the cost of additional runtime.

This application option is allowed to specify a depth number that limits the maximum of rebuffering levels during logical DRC optimization.

opt.buffering.enable_hierarchy_mapping

Enables enhanced MV aware buffering engine in preroute flow.

Data Types

boolean

Default false

Description

When this app option is set to *true*, the logical & physical VA mapping technique is turned on in preroute optimization flow for MV nets. MV nets are those nets that logically or physically cross over more than one voltage areas.

This app option improves the logic DRCs fixing in typical MV designs at a slight cost of additional runtime, particularly on top level nets with multiple voltage areas, unusual voltage area shapes like U-shape/L-shape.

o

When this app option is turned on, it impacts:

- `initial_drc`, `initial_opto`, and `final_opto` stages in *place_opt*
- `final_opto` stage in *clock_opt*
- *refine_opto*

It takes effect in two-pass placement flows. In SPG flow, it takes effect from `initial_opto`.

It has been observed that TNS/Hold are slightly degraded in a few MV designs. If this happened, the subsequent TNS/Hold optimization may be needed to recover them.

See Also

- [place_opt](#)

opt.buffering.enable_ldrc_tree_rebuffering_depth

This application option sets the maximum rebuffering levels during logical DRC optimization.

Data Types

integer

Default 0

Description

Most buffer tree optimization after `place_opt` `initial_drc` is incremental, working on tuning the existing buffer trees. Rebuffering fully removes buffer trees or parts of buffer trees and completely rebuilds them, keeping the solution if improved QoR is seen.

This application option is allowed to specify a depth number that limits the maximum of rebuffering levels during logical DRC optimization.

opt.buffering.enable_mv_rebuffering

Enable MV net rebuffering in the `place_opt` `initial_opto` flow stage

Data Types

boolean

Default false

o

Description

Most buffer tree optimization after `place_opt initial_drc` is incremental, working on tuning the existing buffer trees. Rebuffering fully removes buffer trees or parts of buffer trees and completely rebuilds them, keeping the solution if improved QOR is seen. Rebuffering optimizes for multiple costs including setup timing, logical DRCs, wirelength, and area. If a rebuffed solution degrades QOR, it is rejected and the original solution is kept, so rebuffering should not degrade QOR.

Rebuffering is done in the default flow in some limited cases, mainly to rebuild buffer chains after layer promotion. Setting this app option to true enables additional rebuffering during the `place_opt initial_opto` stage on MV nets only. These are nets that cross over or buffer through more than one voltage area. Particularly on designs with many voltage areas, unusual voltage area shapes, or disjoint voltage areas, rebuffering these nets can yield QOR improvement, at the cost of additional runtime.

See also the `opt.buffering.enable_rebuffering` app option for enabling more general rebuffering in the `place_opt final_opto` stage.

opt.buffering.enable_rebuffering

Enable net rebuffering in the `place_opt final_opto` flow stage

Data Types

boolean

Default false

Description

NOTE: This app_option is now deprecated and no longer in use. Most buffer tree optimization after `place_opt initial_drc` is incremental, working on tuning the existing buffer trees. Rebuffering fully removes buffer trees or parts of buffer trees and completely rebuilds them, keeping the solution if improved QOR is seen. Rebuffering optimizes for multiple costs, including setup timing, logical DRCs, wirelength, and area. If a rebuffed solution degrades QOR, it is rejected and the original solution is kept, so rebuffering should not degrade QOR.

Rebuffering is done in the default flow in some limited cases, mainly to rebuild buffer chains after layer promotion. Setting this app option to true enables more general rebuffering in the `place_opt final_opto` stage. The expected impact is improvement to design QOR at the cost of additional runtime.

See also the `opt.buffering.enable_mv_rebuffering` app option for enabling MV net specific rebuffering in the `place_opt initial_opto` stage.

o

opt.buffering.exclusive_hard_bound_buffering_mode

Specifies the mode for buffering in exclusive hard move bounds.

Data Types

integer

Default 2

Description

This application option specifies the buffering mode used in exclusive hard move bounds. Valid values for this application option are 2 or 0.

By default (a setting of 2), the tool can add cells to an exclusive move bound based on QoR irrespective of the logical hierarchy of the exclusive bound.

When this application option is set to 0, the tool honors the exclusive hard move bound and cells that do not belong to the exclusive hard move bound are not placed inside the exclusive bound.

See Also

- [place_opt](#)

opt.buffering.filter_end_cap_cells

Enables filtering of cells of design type *end_cap* during buffer blockage map creation.

Data Types

boolean

Default false

Description

This app option controls if physical only cells of design type *end_cap* are filtered during construction of the blockage map for buffering. The app option should be enabled only for large top levels designs where millions of those physical only cells exist. The number of those cells can be verified with the command *sizeof_collection [get_cells -hier * -filter "design_type==end_cap"]*. If the number exceeds a million or more it can be considered to filter them to improve runtime of the buffering engines. The drawback of doing so is the fact that at the border of macros some blockage may miss which can lead to unexpected unbufferable routing segments.

o

opt.buffering.filter_filler_cells

Enables filtering of cells of design type *filler* during buffer blockage map creation.

Data Types

boolean

Default false

Description

This app option controls if physical only cells of design type *filler* are filtered during construction of the blockage map for buffering. The app option should be enabled only for large top levels designs where millions of those physical only cells exists. The number of those cells can be verified with the command *sizeof_collection [get_cells -hier * -filter "design_type==filler"]*. If the number exceeds a million or more it can be considered to filter them to improve runtime of the buffering engines. The drawback of doing so is the fact that at the border of macros some blockage may miss which can lead to unexpected unbufferable routing segments.

opt.buffering.filter_physical_only_lib_cells

Enables filtering of cells of design type *physical_only* during buffer blockage map creation.

Data Types

boolean

Default false

Description

This app option controls if lib cells which are marked as *is_physical_only* are filtered during construction of the blockage map for buffering. The app option should be enabled only for large top levels designs where millions of those physical only cells exists. The number of those cells can be verified with the command *sizeof_collection [get_cells -hier * -filter "is_physical_only==true"]*. If the number exceeds a million or more it can be considered to filter them to improve runtime of the blockage map construction. The drawback of doing so is the fact that at the border of macros some blockage may miss which can lead to unexpected unbufferable routing segments.

opt.buffering.filter_well_tap_cells

Enables filtering of cells of design type *well_tap* during buffer blockage map creation.

o

Data Types

boolean

Default false**Description**

This app option controls if physical only cells of design type *well_tap* are filtered during construction of the blockage map for buffering. The app option should be enabled only for large top levels designs where millions of those physical only cells exists. The number of those cells can be verified with the command *sizeof_collection [get_cells -hier * -filter "design_type==well_tap"]*. If the number exceeds a million or more it can be considered to filter them to improve runtime of the buffering engines. The drawback of doing so is the fact that at the border of macros some blockage may miss which can lead to unexpected unbufferable routing segments.

opt.buffering.hard_bound_buffering_mode

Specifies the mode for buffering in hard move bounds.

Data Types

integer

Default 0**Description**

This application option specifies the buffering mode used in hard move bounds. Valid values for this application option are 0 or 2.

By default (a setting of 0), the tool will not add new cells to a hard move bound.

When this application option is set to 2, the tool will honor a hard move bound and new cells will get a hard move bound assignment based on whether the location is inside the bound.

See Also

- [place_opt](#)

opt.buffering.high_fanout_synthesis_cost

Specified the preffered costing during high fanout tree synthesis

Data Types*string*

o

Default composite**Description**

This option allows user to specify the main objective of the costing during high fanout buffer tree insertion. Valid values for this application option are area, leakage, bias_delay, delay, composite and slack_aware.

See Also

- [place_opt](#)
- [compile_fusion](#)
- [create_buffer_trees](#)

opt.buffering.high_fanout_synthesis_delay_effort

Specified the effort level of the datapath high fanout synthesis to improve circuit delay.

Data Types*string***Default** medium**Description**

This option allows user to specify an effort with which high fanout synthesis would optimize circuit's TNS component. Valid values for this application option are high, medium, low, ultra-low. Different effort level would result in delay / area tradeoff at the end of the high fanout synthesis. Value high would correspond to the best delay and value ultra_low would correspond to the best area at the end of the step.

See Also

- [place_opt](#)

opt.buffering.improved_ootb_topology_generation

Enables out of the box improved topology construction.

Data Types

boolean

Default false

o

Description

This app option enables an enhancement to generate improved topologies. That includes less sensitivity when the locations of cell pins slightly change, as well as topologies with less number of sequences of edges over obstacles exceeding the buffer distance. In total the enhancement should improve the quality` of the topology so that a better and repeatable buffering is possible.

See Also

- [compile_fusion](#)
- [create_buffer_trees](#)
- [place_opt](#)

opt.buffering.min_skinny_blockage_size

Specifies the minimum size of an obstacle so that it is not ignored for detouring.

Data Types

float

Default 0.0

Description

This app option controls which obstacles are ignored for detouring when the virtual routing engine is used during either of the following command *place_opt*, *clock_opt* and *create_buffer_trees* to generate topologies for buffering. Any obstacle which is either thinner or shorter than the specified value is ignored. The default value does not filter obstacles. When the app option *opt.buffering.check_for_skinny_blockages* is enabled message OPT-504 is printed if those obstacles exists and what a recommended value is. Please be aware that QoR might degrade if the placement is not legalized and hence cells may overlap with those obstacles. In those situations the generated topology might run for a long distance over the ignored obstacles and hence will not be buffered.

See Also

- [set_app_options](#)
- [create_buffer_trees](#)
- [place_opt](#)
- [clock_opt](#)
- [legalize_placement](#)

o

opt.buffering.skinny_blockage_scope

Controls the scope of the search for skinny blockages.

Data Types

string

Default small

Description

This app option controls the scope when searching for skinny blockages. It is used when *opt.buffering.check_for_skinny_blockages* is enabled. It can be set to three values: *small* (the default), *medium* and *large*. When set to *small* the maximal width when searching for skinny blockages is limited to 10% of the default buffer distance. When set to *medium* the width is limited to 25% of the default buffer distance. For value *large* this limit will be 50% of the default buffer distance. Setting the value to *medium* or *large* can have an impact on runtime when a longer route will be in an area recognized as a skinny blockage. In such a situation the search window to find a legal location will be large and hence will take time to find such a legal location. Please check the man page for app option *opt.buffering.min_skinny_blockage_size* for more details. Especially be aware of message OPT-504 which will be printed with higher probability the larger the selected scope is.

opt.buffering.two_pass_hfs

Enables two pass high fanout synthesis for improved DRC convergence.

Data Types

boolean

Default false

Description

In global-route-based flow, when this app option is set to *true*, the *compile_fusion* command will run two pass high fanout net synthesis in *initial_drc* stage. In virtual-route-based flow, two pass high fanout net synthesis is enabled by default in *initial_drc* stage. It will have additional accuracy improvement with this app option set to *true*.

This app option also affects high fanout net synthesis in *initial_opto* stage, which is always virtual-route-based and two pass by default.

The expected impact is improved DRC convergence and timing correlation.

o

See Also

- [compile_fusion](#)
- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)

opt.buffering.vr_dualrail_cost

Specifies the allowed detour in percentage to avoid entering voltage areas requiring dual rail buffering during virtual routing.

Data Types

float

Default 0.0

Description

This app option controls how much the virtual routing engine can detour to avoid entering a voltage area where dual rail buffers are used to buffer the signal. If the length of the computed virtual routing result avoiding any voltage areas requiring dual rail buffers exceeds the length of the virtual routing allowing to enter those voltage areas the shorter result will be used. This app option is used during data path buffering of either of the following commands *compile_fusion*, *place_opt*, *clock_opt* and *create_buffer_trees*. The default value does not distinguish between voltage areas requiring dual rail buffers versus voltage areas requiring single rail buffers.

Examples

The following example would allow additional detour of 100% (doubling the length)

```
prompt> set_app_options -name opt.buffering.vr_dualrail_cost -value 100
```

See Also

- [set_app_options](#)
- [create_buffer_trees](#)
- [place_opt](#)
- [clock_opt](#)

o

opt.clock_latency_estimation.report_skipped_pins

Issue a message for each pin that was skipped during automatic clock gate latency estimation.

Data Types

Boolean

Default "false"

Description

This option controls whether messages should be generated when certain pins are skipped for clock gate latency estimation. Clock gate pins may be skipped for various reasons. The reason is also printed in the messages.

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design. Use *report_app_options* to show the value of application options.

See Also

- [OPT-911](#)
- [report_app_options](#)
- [set_app_options](#)

opt.common.advanced_logic_restructuring_mode

Controls whether Design Fusion logic restructuring is enabled.

Data Types

string

Default none

Description

This application option controls Design Fusion logic restructuring during the *final_opto* step of the following commands:

- *compile_fusion*
- *place_opt*
- *clock_opt*

o

This feature uses logic restructuring techniques to explore improvements in various metrics such as area, timing, and total power. It accepts the following values:

- *none*: No Design Fusion logic restructuring; the tool uses the previous restructuring algorithms instead
- *area*: Restructure for leakage-aware area improvement
- *timing*: Restructure for timing improvement
- *power*: Restructure for total power improvement, which also optimizes area
- *area_timing*: Restructure for area (leakage-aware) and timing improvement
- *timing_power*: Restructure for timing and power improvement, which also optimizes area

Note that timing-based advanced restructuring is always enabled during the *compile_fusion* and *place_opt* final_opto steps, and cannot be disabled by this application option. Timing-based restructuring is not automatically enabled in the *clock_opt* final_opto step. For example, setting this application option to *power* does the equivalent of the *timing_power* behavior in the *compile_fusion* and *place_opt* commands, and does *power* behavior in the *clock_opt* command.

See Also

- [compile_fusion](#)
- [place_opt](#)
- [clock_opt](#)
- [set_app_options](#)

opt.common.advanced_logic_restructuring_wirelength_costing

Restrict wire length increase from Design Fusion Logic Restructuring for area.

Data Types

string

Default high

Description

This app option allows users to control wirelength increase from Design Fusion Logic Restructuring.

o

If the app option is set to *medium*, we can see more area reduction as restriction on wire length increase is relaxed slightly. If the app option is set to *none*, we can see maximum area reduction as there is no restriction on wire length increase.

Designs that are not routing challenging can be tried with *medium* or *none* setting to get more benefits out of logic restructuring. Please note that this app-option is effective in area or area_timing mode of Design Fusion Logic Restructuring.

See Also

- [place_opt](#)
- [clock_opt](#)
- [set_app_options](#)

opt.common.allow_incorrect_pvt

Allow tool to choose library cells with process, voltage, or temperature (PVT) settings which do not match user settings.

Data Types

string

Default none

Description

This option allows tool to choose library cells whose PVT settings do not match the user settings in logic optimization.

When the option is set to *"pm_cells"*, logic optimization can select a level shifter, enabled level shifter, isolation, or retention library cell whose process label, process number, voltage, or temperature do not match user-specified settings. But tool will give preference to library cells whose PVT settings match the user-specified settings.

When the option is set to *"none"*, logic optimization will select a level shifter, enabled level shifter, isolation, or retention library cell whose PVT settings match user-specified settings according to app option *opt.common.pvt_checking_mode*. This is the default value.

See Also

- [set_app_options](#)

opt.common.allow_physical_feedthrough

Controls whether to allow physical feedthroughs in multivoltage designs.

o

Data Types

Boolean

Default false**Description**

This application option controls whether a multivoltage design should allow physical feedthroughs. A physical feedthrough cell is one that physically belongs to a voltage area, but logically does not. These cells are marked with power domain on instance in the UPF.

See Also

- [place_opt](#)
- [clock_opt](#)
- [route_opt](#)
- [set_app_options](#)

opt.common.auto_hold_effort**Data Types***boolean***Default** true**Description**

This app option controls the enablement of automatic hold effort adjustment. The default value is false. When this app option is set to true, the optimization flow will analyze the design and automatically detect if incoming hold timing is dirty, such as serious hold total negative slack. Accordingly, we will reduce the effort spent in fixing hold violations. This is done to avoid adding too many hold buffers which can destabilize the design wrt. runtime, area, power, congestion, or route DRC.

opt.common.auto_site_def

Specify optimization engines to automatically derive site def settings if design has no floorplan

Data Types*boolean***Default** false

o

Description

When this application option is set to false, optimization engines will only use site def settings from the actual floorplan. If no floorplan exists in the design, site def checking will be skipped.

When the option is set to true, optimization engines will first get site def settings from the actual floorplan. If no floorplan exists in the design, site def settings will be automatically derived from the followings in decreasing order of preference:

- *set_auto_floorplan_constraints -site_def*
- application option *plan.flow.target_site_def*
- application option *plan.flow.prf_based_initialize_floorplan*: design is intended for hybrid row
- application option *plan.shaping.derive_multiple_site_rows*: design is intended for hybrid row
- default site def from tech file

See Also

- [report_app_options](#)
- [report_site_defs](#)
- [set_auto_floorplan_constraints](#)
- [set_app_options](#)

opt.common.bmap_use_partial_routing_blockages

Option to make Abuf blockage map honor partial routing blockages.

Data Types

boolean

Default false

Description

By default, the buffering engine will prevent buffer routing through narrow routing channels. Turning on this app_option will force the buffering engine to route through narrow routing channels.

o

See Also

- [place_opt](#)
- [clock_opt](#)
- [set_app_options](#)

opt.common.buffer_area_effort

Sets the effort level for buffer area usage in data path optimization.

Data Types

string

Default low

Description

This option sets the effort level for reducing buffer area usage in data path optimization. Default is *low*. Effort level *medium*, *high* and *ultra* can be set for reducing buffer area usage further. QoR degradation might occur, and it does not guarantee the reduction in buffer area at each effort level.

See Also

- [place_opt](#)
- [clock_opt](#)
- [set_app_options](#)

opt.common.buffering_for_advanced_technology

Buffering for place_opt and clock_opt for advanced technologies to handle pre- and post-route mis-correlation.

Data Types

boolean

Default false

Description

This option when set to true relaxes buffering targets for designs on advanced technologies. This can reduce the impact of mis-correlation between virtual router based pre-route timing to post-route timing.

o

See Also

- [place_opt](#)
- [clock_opt](#)
- [set_app_options](#)

opt.common.do_physical_checks**Data Types***string***Default** legalize**Description**

Specifies when physical design checks are performed during optimization. Recognized values are *never*, *legalize* and *always*. The default value, *legalize*, causes physical design checks to be activated during legalization only. This should result in no violations after optimization.

Using the value of *never* usually gives the fastest runtime, but there may be power strap, pin access or other violations reported by *check_legality* after optimization.

Using the value of *always* will result in a significant increase in runtime. This may produce better results if there are a significant number of cells that are highly restricted in their placement due to pre-routed metal.

See Also

- [check_legality](#)
- [legalize_placement](#)
- [place_opt](#)

opt.common.drc_mode_buffering

Buffering mode for drc stages of *place_opt* and *clock_opt*.

Data Types*string***Default** auto

o

Description

This option sets the buffering mode for the drc stages of `place_opt` and `clock_opt`. Values of *auto*, *false* and *true* can be set. Default value is *auto*. When set to *false*, internal targets are used for buffering in the drc stages. When set to *true*, drc constraints are used for these stages. With the default *auto* setting the tool uses the true setting for advanced technology designs, and the false setting for all others.

See Also

- [place_opt](#)
- [clock_opt](#)
- [set_app_options](#)

opt.common.enable_multibit_combinational_cells

Data Types

boolean

Default *false*

Description

When this app option is true, the combinational multibit banking will be enabled in `initial_opto` and `final_opto`.

When this app option is false, the combinational multibit banking will be disabled in `initial_opto` and `final_opto`.

To determine the current value of this application option, use `get_app_option_value -name opt.common.enable_multibit_combinational_cells`.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)
- [get_attribute](#)

o

opt.common.enable_multibit_feature

Enable specific features for multibit banking and debanking.

Data Types

String

Default ""

Description

This option enables the multiple features for physical multibit banking and debanking. The available value is "true" and "false". The enhancements are "CUS", "CLO", "bottleneck", and "enhanced_debanking" when the option value is "true".

CUS

Enable useful skew calculation during debanking step. The feature is enabled in both `place_opt/compile_fusion final_place` and `final_opto`.

CLO

Enable CLO related enhancements to reduce the number of debanking rejection because of a large displacement.

bottleneck

Enable bottleneck debanking in both `place_opt/compile_fusion final_place`. The bottleneck debanking tries to debank the multibit cell with higher slack value and higher number of driving endpoints. Also, the sub-critical starting points are also debanked.

enhanced_debanking

Enable all the feature from CUS, CLO and bottleneck. In addition, additional debanking step is enabled at the end of `final_opto`, if `place_opt` effort is high or `compile_fusion` effort is high. Furthermore, if `compile.flow.enable_second_pass_multibit_banking` is enabled, the multibit banking step is disabled in the second pass of the `final_place`. Last, the debanking step is disabled in the first pass of `final_place`.

See Also

- [compile_fusion](#)
- [place_opt](#)

o

opt.common.enable_rde

Controls whether route-driven extraction (RDE) is enabled during preroute optimization to handle preroute to postroute miscorrelation.

Data Types

string

Default auto

Description

This application option enables route-driven extraction (RDE) during preroute optimization starting with the `final_opto` step of the `place_opt` command.

RDE can reduce the impact of miscorrelation between virtual-router-based preroute timing and postroute timing. The RDE flow runs global routing inside preroute optimization (starting at the `place_opt` `final_opto` stage) and the tool uses the global routing data to build an RDE extraction table. This extraction engine is used subsequently for all virtual net extraction.

By default (*auto*), the tool infers whether the design uses advanced technology nodes ($\leq 16\text{nm}$) and, if so, automatically enables the flow.

If set to *true*, the RDE flow is always triggered.

If this flow is enabled during pre-route optimization, you should continue to run all steps in pre-route optimization with the flow enabled.

See Also

- [place_opt](#)
- [clock_opt](#)
- [remove_route_aware_estimation](#)
- [set_app_options](#)

opt.common.enable_seq_sizing_in_ldrc

Data Types

boolean

Default false

o

Description

Setting this option to true enables sizing of sequential cells during DRC Optimization in `compile_fusion` and `place_opt final_opto`. This feature aims to fix DRCs in design. By default this option is not enabled, and only combinational cells are optimized.

See Also

- [set_app_options](#)

opt.common.enable_seq_sizing_in_ldrc_with_max_cap_focus**Data Types**

boolean

Default false

Description

Setting this option to true enables sizing of sequential cells during DRC Optimization in `compile_fusion` and `place_opt final_opto`. Compared to `opt.common.enable_seq_sizing_in_ldrc`, this option prioritizes max capacitance fixing over other DRC metrics. By default this option is not enabled.

See Also

- [set_app_options](#)

opt.common.enable_sizing_with_balance_point**Data Types**

Boolean

Default false

Description

The app option controls whether the useful skew optimization is done during sequential sizing for delay optimization in pre-CTS stage.

See Also

- [compile_fusion](#)
- [place_opt](#)
- [set_app_options](#)

o

opt.common.enable_target_library_subset_opt

Specifies to enable the settings of target library subsets during optimization.

Data Types

integer

Default 0

Description

Setting this application option to 1 enables the settings of target library subsets during optimization. By default, this application option is not enabled.

Enabling this application option might increase the runtime; therefore, enable this application option only if you have defined target library subsets for the design.

If a design does not have any user-defined target library subset, enabling this application option will create a default target library subset at the top level of the design. The default target library subset includes all the reference libraries as the target libraries.

See Also

- [set_target_library_subset](#)
- [report_target_library_subset](#)
- [remove_target_library_subset](#)
- [set_app_options](#)

opt.common.enable_verilog_assign_check

Data Types

boolean

Default false

Description

This application option ensures no new verilog assign during the flow.

See Also

- [route_opt](#)

o

opt.common.enable_via_ladder_insertion

Enable/disable via ladder insertion in `clock_opt` and `route_opt`.

Data Types

Boolean

Default `false`

Description

With this app option turned on, performance via ladders will be inserted during `clock_opt` and `route_opt` commands.

See Also

- [set_app_options](#)
- [set_via_ladder_constraints](#)
- [set_via_ladder_rules](#)
- [set_via_ladder_spacing](#)

opt.common.estimate_clock_gate_latency

Enable automatic clock gate latency estimation.

Data Types

Boolean

Default `"true"`

Description

This option controls whether clock gate latency estimation is automatically performed after certain steps in the flow. With automatic clock gate latency estimation, `set_clock_latency`-offset assertions will be generated that reflect the estimated clock tree latency. It will make the flow aware of the reduced timing budget for clock gate enable paths. It is an alternative to manual `set_clock_latency` annotation at clock gate input pins or to manual `set_clock_gate_latency` settings. The annotated latencies will automatically be kept in sync with the state of the clock tree in the design.

In order to have accurate estimations, it is important to setup CTS constraints like routing-rules and buffer selection before `place_opt` or `compile_fusion`. The latency estimator will consider these settings when doing the estimations.

o

Clock gate latency estimations will take place after major logical or physical clock reconstruction steps. These steps include placement, multi-source CTS and multi-bit banking. Enabling clock gate latency estimations will also enable clock gate latency aware placement if app option `place.coarse.clock_gate_latency_aware` was set to "auto".

Clock gate latency estimation will not be performed when it is called with propagated clocks. Therefore clock gate latency estimation is incompatible with pre-CTS trial tree construction because trial tree construction forces clock timing to be in propagated mode. Pre-CTS trial clock tree construction is enabled with app options `place_opt.flow.trial_clock_tree`, `compile.flow.trial_clock_tree`, `place_opt.flow.optimize_icgs` or `compile.flow.optimize_icgs`.

To inspect the applied offset latencies, please use `write_script` since offset latencies are not listed in the output of `write_sdc`.

Application options are set using the `set_app_options` command, and they may be specified globally or with respect to a design. Use `report_app_options` to show the value of application options.

See Also

- [place_opt](#)
- [write_script](#)
- [report_app_options](#)
- [set_app_options](#)

opt.common.hold_effort

Controls the optimization effort for hold fixing.

Data Types

string

Default high

Description

This app option controls the hold effort used for optimization. It provides a tradeoff between hold fixing and runtime. Default value of the app option is high. When the app option is set to medium, we will disable offroute hold fixing in onroute optimization flows (`route_opt` and `global_route` based `clock_opt`) and will also apply some density throttling to avoid overpacking hold cells in locally dense areas.

o

When the app option is set to low, in addition to the settings in medium, we will use lower thresholds to throttle density, and will also fix hold only on the worst violating scenario. We will also not fix hold on endpoints with large hold violations.

These settings are expected to provide improvements in runtime, compared with the default high setting, while hold TNS may not be fully fixed. These settings are intended to be used on customer design flows where hold constraints are not clean, or the user does not want the tool to strive for last mile hold closure.

See Also

- [set_app_options](#)

opt.common.insert_clk_data_isolation_cells

This app-option enables adding isolation cell between clock and data net.

Data Types

bool

Default TRUE

Description

This app-option enables adding isolation cell between clock and data net.

opt.common.ir_drop_threshold

Sets the target voltage drop threshold in percentage of the supply voltage.

Data Types

float

Default 8.0

Description

This option set the target voltage drop threshold in percentage of the supply voltage. If the opt.common.power_integrity_effort is set to *high* (default value), then the power integrity features that have been turned on will see this value as the max IR drop constraint.

o

See Also

- [place_opt](#)
- [compile](#)
- [clock_opt](#)

opt.common.ir_drop_threshold_clock

Sets the target voltage drop threshold in percentage of the supply voltage for clock network cells.

Data Types

float

Default 8.0

Description

This option set the target voltage drop threshold in percentage of the supply voltage for clock network cells. If the `opt.common.power_integrity_effort` is set to *high* (default value), then the power integrity features that have been turned on will see this value as the max IR drop constraint.

See Also

- [place_opt](#)
- [compile](#)
- [clock_opt](#)

opt.common.ir_drop_threshold_data

Sets the target voltage drop threshold in percentage of the supply voltage for non-clock network cells.

Data Types

float

Default 8.0

Description

This option set the target voltage drop threshold in percentage of the supply voltage for non-clock network cells. If the `opt.common.power_integrity_effort` is set to *high* (default

o

value), then the power integrity features that have been turned on will see this value as the max IR drop constraint.

See Also

- [place_opt](#)
- [compile](#)
- [clock_opt](#)

opt.common.max_fanout

Fanout constraint for data path optimization.

Data Types

int

Default 40

Description

This option sets a fanout constraint for optimization of data path cells in `place_opt` and `clock_opt`. Optimization will try to ensure that data path cells do not drive more than `max_fanout` cells. This is a soft constraint.

See Also

- [place_opt](#)
- [clock_opt](#)
- [set_app_options](#)

opt.common.max_net_length

Sets the net length constraint for data path optimization.

Data Types

float

Default 0.0

Description

This option sets a net length constraint in um for optimization of data path cells in `place_opt` and `clock_opt`. Optimization tries to ensure that data path cells do not drive

o

nets with length more than `max_net_length`. The maximum driver to fanout pin distance is considered to be the net length. Default of 0.0 means this constraint is ignored. This is a soft constraint.

See Also

- [place_opt](#)
- [clock_opt](#)
- [set_app_options](#)

opt.common.multibit_flow_strategy

Enable multibit flow to run in different strategies.

Data Types

string

Default default

Description

This app-option controls the strategy used for multibit flow during the `compile_fusion` command. The available strategies are:

default The default strategy for multibit flow.

conservative_debanking Better banking ratio without compromising on timing.

This strategy enables banking and disables debanking in the first `initial_opto` step. It disables banking and enables late enhanced debanking in any subsequent (incremental) `initial_opto` steps. Enhanced physical debanking is enabled in the `final_opto` step.

opt.common.optimize_ndr_user_max_layers

Specify the candidate user maximum layers for Non-default-rule optimization.

Data Types

string or list of strings

Default Empty

Description

The Non-default-rule optimization will loop the candidate user rules list, the rule which can improve the best timing will be assigned on that critical net under optimization. The max

o

layer provided could be a single layer for all the rules or as many number of max layers as the number of rules.

See Also

- [set_app_options](#)

opt.common.optimize_ndr_user_min_layers

Specify the candidate user minimum layers for Non-default-rule optimization.

Data Types

string or list of strings

Default Empty

Description

The Non-default-rule optimization will loop the candidate user rules list, the rule which can improve the best timing will be assigned on that critical net under optimization. The min layer provided could be a single layer for all the rules or as many number of min layers as the number of rules.

See Also

- [set_app_options](#)

opt.common.optimize_ndr_user_rule_names

Specify the candidate user rules for Non-default-rule optimization in compile.

Data Types

string

Default Empty

Description

The Non-default-rule optimization will loop the candidate user rules list, the first rule which can improve the timing will be assigned on that critical net under optimization.

See Also

- [set_app_options](#)

o

opt.common.optimize_power_ndr_user_max_layers

Specify the max layers corresponding to the Non-default-rule listed for power optimization.

Data Types

string

Default Empty

Description

The Non-default-rule optimization will loop the max layers for each of the candidate Non-default-rule user rules provided and associate each max layer with the corresponding rule in the order provided. If a single max layer is provided for a list of multiple user rules, all the rules in the list will get assigned the same max layer. If the number of max layers provided is greater than one, but less than the number of the rules provided, then the max layer specification will be ignored.

See Also

- [set_app_options](#)

opt.common.optimize_power_ndr_user_min_layers

Specify the min layers corresponding to the Non-default-rule listed for power optimization.

Data Types

string

Default Empty

Description

The Non-default-rule optimization will loop the min layers for each of the candidate Non-default-rule user rules provided and associate each min layer with the corresponding rule in the order provided. If a single min layer is provided for a list of multiple user rules, all the rules in the list will get assigned the same min layer. If the number of min layers provided is greater than one, but less than the number of the rules provided, then the min layer specification will be ignored.

See Also

- [set_app_options](#)

o

opt.common.optimize_power_ndr_user_rule_names

Specify the candidate user rules names for Non-default-rule application for power optimization.

Data Types

string

Default Empty

Description

The Non-default-rule optimization will loop the candidate user rules list, the best rule which can improve the timing will be assigned on that critical net under power optimization.

See Also

- [set_app_options](#)

opt.common.pdr_aware_hold

Sub circuit hold xpdr costing effort.

Data Types

string

Default none

Description

This option controls XPDR costing effort in hold fixing. XPDR aims at less power overhead during hold fixing. When XPDR is on, tool only allows the moves which are below certain pdr threshold.

Available options are 'low', 'medium' and 'high'.

opt.common.power_integrity

Switch on power integrity features during the flow

Data Types

Boolean

Default false

o

Description

This option when set to *true* will turn on the recommended method for power integrity during the flow. This method consist of a set of features that will reduce the overall IR drop in the design. The recommended power integrity flow consists of the following features:

- * Dynamic Power Shaping during `place_opt` or `compile`
- * IR aware placement (IRAP) during `clock_opt`.

If the option is set to *false* the features are turned on depending on the the enable app option settings. The following app options can be used for enabling the features:

- * `place_opt.flow.enable_dps`
- * `place_opt.flow.enable_irap`
- * `compile.flow.enable_dps`
- * `compile.flow.enable_irap`
- * `clock_opt.flow.enable_irap`

The settings for the power integrity method are controlled by the following app options: `opt.common.power_integrity_effort`; `opt.common.ir_drop_threshold`.

See Also

- [place_opt](#)
- [compile](#)
- [clock_opt](#)

opt.common.power_integrity_analysis_engine

Select which power analysis engine has to be used during Power Integrity (PI) flow (mega commands)

Data Types

String

Default `redhawk`

Description

When this option is set to default value *redhawk*, RedHawk internal engine is used to calculate power consumption for RedHawk Analysis.

When this option is set to *icc2*, ICC2 power engine is used to calculate power consumption for RedHawk Analysis.

o

See Also

- [place_opt](#)
- [compile](#)
- [clock_opt](#)
- [route_opt](#)

opt.common.power_integrity_effort

Sets the power integrity effort level.

Data Types

string

Default low

Description

This option determines the effort level of the power integrity features in the flow. It has two levels, the level *low* the power integrity features that will be used, will not do this at cost of timing QoR. The level *high* the power integrity features that will be used will do every effort to get to goals set by `opt.common.ir_drop_threshold`. The power integrity features that are used are controlled by `opt.common.power_integrity`.

See Also

- [place_opt](#)
- [compile](#)
- [clock_opt](#)

opt.common.power_integrity_indesign_post_tcl_proc

Sets the tcl proc to be run after inDesign RedHawk.

Data Types

string

Default ""

Description

This option sets the tcl procedure that will be run after running inDesign RedHawk. In this tcl procedure the user can set the commands that need to be executed after doing the

o

inDesign RedHawk analysis, e.g. removing stapling vias. By default, the value is empty and no extra tcl procedure will be called after inDesign RedHawk.

See Also

- [place_opt](#)
- [compile](#)
- [clock_opt](#)

opt.common.power_integrity_indesign_pre_tcl_proc

Sets the tcl proc to be run before inDesign RedHawk.

Data Types

string

Default ""

Description

This option sets the tcl procedure that will be run before running inDesign RedHawk. In this tcl procedure the user can set the commands that need to be executed before doing the inDesign RedHawk analysis, e.g. inserting stapling vias. By default, the value is empty and no extra tcl procedure will be called before inDesign RedHawk.

See Also

- [place_opt](#)
- [compile](#)
- [clock_opt](#)

opt.common.power_integrity_redhawk_script_file

Sets the redhawk script to be used for RedHawk.

Data Types

string

Default ""

o

Description

When set, this app option points to the redhawk script file to be used for RedHawk during power integrity inDesign calls. During these inDesign calls, the `analyze_rail` command is called to generate IR drop data for optimization using power integrity flow (e.g. IRDP, DPS). When RH is used to analyze the IR drop, `analyze_rail` can either create the script file to be used, or you can specify the to be used by RH-SC. By default, this app option will let `analyze_rail` create the script file. When set it will be the script file to be used for RH.

See Also

- [place_opt](#)
- [compile](#)
- [clock_opt](#)

opt.common.prefer_exact_pvt

Give preference to choosing library cells with process, voltage, and temperature (PVT) settings which match user settings exactly.

Data Types

string

Default none

Description

When the option is set to *"none"*, logic optimization will select a level shifter, enabled level shifter, isolation, or retention library cell whose PVT settings fall within user-specified settings. But tool will not give preference to a library cell whose PVT setting match exactly those of the user settings. This is the default value.

When the option is set to *"pm_cells"*, logic optimization will give preference to selecting a level shifter, enabled level shifter, isolation, or retention library cell whose process label, process number, voltage, or temperature exactly match user-specified settings.

See Also

- [set_app_options](#)

opt.common.preserve_pin_names

Controls sizing operations to only cells that are pin-name compatible.

o

Data Types

integer

Default never

Description

This option controls how the set of cells that can be used during sizing optimization is determined. Possible values are "never" and "always". The default is "never".

Assume that there are two buffer cells in the library. One buffer (buffA) has pins "A" and "O" and the other (buffX) has pins "X" and "Z". When *opt.common.preserve_pin_names* is set to "never", buffA can be replaced with buffX (and vice versa) during sizing optimization. However, when *opt.common.preserve_pin_names* is set to "always", then buffA and buffX are not interchangeable anymore.

See Also

- [place_opt](#)
- [clock_opt](#)
- [set_app_options](#)

opt.common.pvt_setting

Sets the pvt setting mode for process, voltage, and temperature (PVT) used in selecting library cells during logic optimization.

Data Types

string

Default voltage_range_match

Description

This option sets the mode used to check PVT setting of library cells in logic optimization.

When the option is set to *exact_match*, logic optimization selects a library cell only if its process label, process number, voltage, and temperature all exactly match user-specified settings.

When the option is set to *range_match*, logic optimization selects a library cell only if its process label and process number exactly match, and voltage and temperature fall within the range. In other words, the *set_voltage* and *set_temperature* settings of the supply net must fall within the max and min voltages/temperatures of all library panes.

o

When the option is set to *closest_match*, logic optimization selects a library cell with closest matching PVT. Order of preference of a library cell is as follows in decreasing order of preference:

- Library cell with Exact PVT match
- Library cell with Proc Label exact match
- Library cell with Proc Number exact match
- Library cell with Voltage exact match
- Library cell with Voltage range match
- Library cell with Temperature exact match
- Library cell with Temperature range match
- Library cell with incorrect PVT

When the option is set to *voltage_range_match*, logic optimization selects a library cell with voltage within the range. Process Label, Process Number, and Temperature are not checked in this mode. This is the default.

Examples

Example usages of app-option: `set_app_options -name opt.common.pvt_setting -value "range_match"` `set_app_options -name opt.common.pvt_setting -value "exact_match=iso"` `set_app_options -name opt.common.pvt_setting -value "range_match= 1s iso, exact_match=buf"`

See Also

- [set_app_options](#)

opt.common.select_inverter

Selects inverter directive for data path optimization.

Data Types

Boolean

Default true

Description

This option directs data path optimization to not use inverters when false. By default, inverters and buffers are both used as appropriate.

o

See Also

- [place_opt](#)
- [clock_opt](#)
- [set_app_options](#)

opt.common.seq_enable_hold**Data Types***integer***Default** 0**Description**

This option enables the flow to do sequential sizing to fix hold violations before performing sizing and buffering on the combinational logic. This helps reduce the work for hold fixing step which only works on combinational cells. The default value is 0. When this app option is set to 1, sequential hold fixing will be activated in virtual route based optimization in place_opt/clock_opt. When this app option is set to 2, sequential hold fixing will be activated in global route based optimization in clock_opt. When this app option is set to 4, sequential hold fixing will be activated in hyper_route_opt/route_opt. To enable the feature in all mega commands, a value of 7 (1+2+4) can be used.

See Also

- [route_opt](#)
- [clock_opt](#)
- [set_app_options](#)
- [get_app_option_value](#)
- [report_app_options](#)

opt.common.setup_aware_hold_fixing

Enables concurrent setup and hold optimization during hold fixing.

Data Types

Boolean

Default false

o

Description

When this app option is set to true, the hold engine considers both setup and hold timing simultaneously to achieve better overall timing closure during hold fixing.

Examples

```
set_app_options -list {opt.common.setup_aware_hold_fixing true}
```

See Also

- [set_app_options](#)
- [get_app_option_value](#)
- [report_app_options](#)

opt.common.setup_tradeoff_threshold_for_hold

Specifies a threshold(absolute value in ns) to allow setup degradation during costing when fixing hold.

Data Types

positive float

Default 0.0

Description

By setting this option, user can expect the tool to size/insert buffers on drivers where there is setup slack degrade up to the margin specified. This value applies per driver and not per endpoint.

See Also

- [get_app_option_value](#)
- [report_app_options](#)

opt.common.small_region_area_recovery

Enable area recovery flow for small regions.

Data Types

bool

Default false

o

Description

This option enable a special stage in the place_opt/clock_opt flow to target area recovery on small regions that are potentially not legalizable due to high utilization. This special stage will enable area recovery moves that will reasonably degrade timing. Slight QoR impact is expected. Should only be used in cases when design is otherwise non-legalizable due to small shapes or bounds restrictions.

See Also

- [place_opt](#)
- [clock_opt](#)
- [set_app_options](#)

opt.common.track_no_new_cells

App option to enable tracking of no_new_cell violations. Each violations will result in OPT-6003 or OPT-6006 warnings.

Data Types

boolean

Default false

Description

This option, when set to true enables tracking of new cell creation for abutted design/VA feature. It issues warning OPT-6003, when new cell is created in a hierarchy restricted through set_allow_new_cells or OPT-6006, when restricted through create_voltage_area_rule -allow_new_cells.

If this warning is issued for cells generated during user driven commands e.g. create_mv_cells or other PM cell insertion command such as create_power_switch, create_power_switch_array etc. then it is user's responsibility to fix the issue by correcting the UPF and/or any user driven PM cell insertion commands. Same holds true for physical cells inserted by user through compile_boundary_cell, create_tap_cells etc.

During debugging violations, one may encounter OPT-6006 for transitional violations e.g. cell was temporarily created in restricted VA and then immediately moved to proper VA. One can ignore such violations safely if they are not present at the end of the command. These transitional violations will not cause any real violations.

o

See Also

- [set_allow_new_cells](#)
- [create_voltage_area_rule](#)
- [set_app_options](#)
- [set_attribute](#)

opt.common.use_ref_libs_only

Specify optimization engines to choose library cells from reference libraries only

Data Types

boolean

Default true

Description

When this application option is set to false, library cells from available user libraries can be used by optimization engines.

When the option is set to true, only library cells from reference libraries specified by *set_ref_libs* can be chosen by optimization engines.

See Also

- [create_lib](#)
- [current_lib](#)
- [open_lib](#)
- [report_ref_libs](#)
- [set_ref_libs](#)

opt.common.user_instance_name_prefix

The value of option *opt.common.user_instance_name_prefix* is used as prefix of names of new cells created by *place_opt*, *clock_opt*, *route_opt*, *create_buffer_trees*, and *remove_buffer_trees*.

Data Types

String

o

Default ""**Description**

Users can specify a string to be prepended to new cell names in optimization by setting app option value of *opt.common.user_instance_name_prefix*. This applies to cell names created during data optimization. Names created during clock optimization are controlled by *cts.common.user_instance_name_prefix*.

For example, if a cell is expected to be created by *place_opt* and to have name as *buft_p_60*, then by setting *opt.common.user_instance_name_prefix* as "user_anno_" before *place_opt*, the cell's name would be *user_anno_buft_p_60* after *place_opt*.

Here is another example of usage:

```
set_app_option -name opt.common.user_instance_name_prefix -value
"place_opt_"
place_opt
set_app_option -name opt.common.user_instance_name_prefix -value
"clock_opt_"
clock_opt
set_app_option -name opt.common.user_instance_name_prefix -value
"route_opt_"
route_opt
```

See Also

- [set_app_options](#)
- [place_opt](#)
- [clock_opt](#)
- [route_opt](#)
- [create_buffer_trees](#)
- [remove_buffer_trees](#)

opt.dft.clock_aware_scan_reorder

Enable clock driver aware scan reordering only within *optimize_dft* in post-cts optimization stage to reduce hold violations along scan path.

Data Types

Boolean

Default false

o

Description

With this `app_option` setting to be true, `optimize_dft` or `dft` optimization within `clock_opt` checks the hold violation status in scan chains. If there exists hold violations along scan path, clock driver aware reordering only will be employed to reduce the hold violations along scan path within each scan chain to save the hold fixing resource on scan path. As a cost, the total scan wirelength increase may be observed. However, if there is no hold violation along scan path, `optimize_dft` engine will still optimize scan chains for total scan wirelength minimization.

This app option should be enabled in post-cts optimization stage. For the design with large number of hold violations on scan path after CTS, less hold violations and less area are expected after post-cts optimization stage with this app option enabled.

When the user expects more hold violation reduction efficiency along scan path in the cost of more scan wirelength increase, he/she can use another app option named 'opt.dft.clock_aware_scan_repartition_reorder' to enable the clock driver aware scan repartition and reordering, to perform scan reconnection across scan chains within the same partition group.

See Also

- [clock_opt](#)
- [optimize_dft](#)
- [set_app_options](#)

opt.dft.clock_aware_scan_repartition_reorder

Enable clock driver aware scan repartition and reordering within `optimize_dft` in post-cts optimization stage to reduce hold violations along scan path.

Data Types

Boolean

Default false

Description

With this `app_option` setting to be true, `optimize_dft` or `dft` optimization within `clock_opt` checks the hold violation status in scan chains. If there exists hold violations along scan path, clock driver scan repartition and reordering will be employed to reduce the hold violations along scan path to save the hold fixing resource on scan path. As a cost, the total scan wirelength increase may be observed. However, if there is no hold violation along scan path, `optimize_dft` engine will still optimize scan chains for total scan wirelength minimization.

o

This app option should be enabled in post-cts optimization stage. For the design with large number of hold violations on scan path after CTS, less hold violations and less area are expected after post-cts optimization stage with this app option enabled.

When the user expects the scan reconnection happening only within each scan chain, not across scan chains, or less total scan wirelength increase is expected for hold violation minimization along scan path, the user can use another app option named 'opt.dft.clock_aware_scan_reorder', to enable clock driver scan reordering only.

When 'opt.dft.clock_aware_scan_repartition_reorder' is set to be true, the setting of 'opt.dft.clock_aware_scan_reorder' will be redundant.

See Also

- [clock_opt](#)
- [optimize_dft](#)
- [set_app_options](#)

opt.dft.do_repartition

control whether to perform scan repartition in optimize_dft.

Data Types

Boolean

Default true

Description

By default, both scan repartition and reordering is employed in optimize_dft. Scan repartition is performed to re-allocate the scan cells within the same PARTITION label to achieve a better scan structure, and scan reordering is done for each scan chain. With this app_option setting to be FALSE, scan repartition will be disabled in optimize_dft. The impact is the optimize_dft efficiency on either scan wirelength reduction or hold violation avoidance may be decreased, but the runtime of optimize_dft will be shortened.

See Also

- [optimize_dft](#)
- [report_app_options](#)
- [set_app_options](#)

o

opt.dft.enhanced_exception_handling

Enables improvements for exception handling during optimize_dft.

Data Types

Boolean

Default false

Description

When this application option is set to TRUE, it enables improvements in the exception handling of stub chain's interconnections during optimize_dft, preventing exceptions that otherwise would remove part of the interconnections from the optimization pool and reduce DFT optimization opportunities.

See Also

- [optimize_dft](#)
- [report_app_options](#)
- [set_app_options](#)

opt.dft.enhanced_scan_exclusive_repeater_handling

Controls whether to remove scan exclusive repeaters only in modified interconnections in optimize_dft.

Data Types

Boolean

Default false

Description

With this application option set to be TRUE, scan exclusive repeaters will only be removed on scan interconnections that were changed during optimize_dft.

See Also

- [optimize_dft](#)
- [report_app_options](#)
- [set_app_options](#)

o

opt.dft.fix_lockup_latch_clock

Allows the user to reconnect the clock pin of the Lockup Latch to the clock driver of its adjacent Scan Flop.

Data Types

Boolean

Default false

Description

When this app_option is set to TRUE, it will perform the reconnection of the clock pins of the Lockup Latches that are stop of a stub chain. The clock driver to which they will be reconnected will be that of their adjacent Scan Flop. To achieve this, the polarity of the clock signal of the Lockup Latch and its adjacent Scan Flop is validated. This occurs in the final stage of optimize_dft

See Also

- [optimize_dft](#)
- [report_app_options](#)
- [set_app_options](#)

opt.dft.fix_repeater_flop_clock

Allows the user to reconnect the clock pin of the Repeater Flop to the clock driver of its adjacent Scan Flop.

Data Types

Boolean

Default false

Description

When this app_option being is set to TRUE, it will perform the reconnection of the clock pins of the Repeater Flops that are start or stop of a stub chain, including a propagation in case there is more than one Repeater Flop. The clock driver to which they will be reconnected will be that of their adjacent Scan Flop. This occurs in the final stage of optimize_dft.

o

See Also

- [optimize_dft](#)
- [report_app_options](#)
- [set_app_options](#)

opt.dft.fix_start_stop

Allow the user to bind the start and stop interconnections of SCANDEF stub chains to avoid the dft optimization of them.

Data Types

Boolean

Default false

Description

With this app_option being set to be TRUE, will not touch the interconnections of start and stop of SCANDEF chains. This is not recommended because it prevents the tool from improving the cost of these interconnections.

See Also

- [optimize_dft](#)
- [report_app_options](#)
- [set_app_options](#)

opt.dft.frozen_hier_preservation

Control whether to preserve frozen hierarchy in optimize_dft.

Data Types

Boolean

Default false

Description

With this app_option set to TRUE, frozen hierarchy preservation DFT optimization mode will be enabled, that is, all hierarchical ports will be preserved in cells with frozen ports, and no new hierarchical port will be created in those cells within optimize_dft. This feature

o

is only available with the new optimization engines. Enabling full hierarchical preservation, controlled by `opt.dft.hier_preservation`, will override this `app_option`.

See Also

- [optimize_dft](#)
- [set_freeze_ports](#)
- [report_app_options](#)
- [set_app_options](#)

opt.dft.hier_preservation

Control whether to preserve hierarchy in `optimize_dft`.

Data Types

Boolean

Default `false`

Description

With this `app_option` being setting to be `TRUE`, hierarchy preservation dft optimization mode will be enabled, that is, all the hierarchical ports will be preserved, and no new hierarchical port will be created within `optimize_dft`. In hierarchy preservaton dft optimization mode, the scan repartition step is skiped because the benefit of scan repartition will be little with the hierarhcy preservation constraint. The physical aware scan reordering method will be employed in this mode. The hierarhcy preservation dft optimization is incompatible with the clock aware dft optimization mode and timing friendly scan reordering mode.

See Also

- [optimize_dft](#)
- [report_app_options](#)
- [set_app_options](#)

opt.dft.ignore_hier_exceptions

Control whether to ignore hierchical exceptions on nets in `optimize_dft`.

Data Types

Boolean

o

Default false**Description**

With this application option set to be TRUE, hierarchical exceptions will not be considered in `optimize_dft`.

e.g. boundary optimization

See Also

- [optimize_dft](#)
- [report_app_options](#)
- [set_app_options](#)

opt.dft.ignore_inversions

Control whether to optimize scan connections with odd number of inversions on the scan exclusive path in `optimize_dft`.

Data Types

Boolean

Default false**Description**

With this application option set to be TRUE, scan paths with odd number of inversions will not be optimized in `optimize_dft`.

See Also

- [optimize_dft](#)
- [report_app_options](#)
- [set_app_options](#)

opt.dft.low_impact_optimization

Enables low timing impact DFT optimization.

Data Types

Boolean

Default false

o

Description

When this app option is set to true, it will enable all the settings and features that contribute to reducing the timing impact of DFT optimization steps.

See Also

- [optimize_dft](#)
- [set_app_options](#)

opt.dft.ng_hold_aware

Enables hold awareness for next generation engines in DFT optimization.

Data Types

Boolean

Default false

Description

With this app_option turned on, the next generation engines for optimize DFT will attempt to reduce hold violations on the scan path.

See Also

- [optimize_dft](#)
- [set_app_options](#)
- [get_app_options](#)
- [report_app_options](#)

opt.dft.optimize_dft_detailed_report

Enables a more detailed report at the end of optimize DFT.

Data Types

Boolean

Default false

Description

With this application option turned on, at the end of optimize DFT the report will contain additional information.

o

See Also

- [optimize_dft](#)
- [set_app_options](#)
- [get_app_options](#)
- [report_app_options](#)

opt.dft.optimize_scan_chain

Enables `compile_fusion`, `create_placement`, `place_opt` and `clock_opt` to call dft optimization.

Data Types

Boolean

Default `true`

Description

With this application option set to `true`, `compile_fusion`, `create_placement`, `place_opt` and `clock_opt` runs dft optimization step. This results in better scan path related metrics, which can benefit routability or timing in later flow. However, these improvements come at the cost of increased tool runtime. With this application option set to `false`, the steps with `optimize_dft` are skipped.

See Also

- [create_placement](#)
- [place_opt](#)
- [clock_opt](#)
- [set_app_options](#)

opt.dft.post_fix_long_nets

Control whether to run an optimization to reduce long nets after repartitioning and reordering to minimize long scan nets in `optimize_dft`.

Data Types

Boolean

Default `false`

o

Description

With this application option set to be TRUE, outlier nets will be considered as candidates to be optimized in order to minimize long scan nets in `optimize_dft` after repartitioning and reordering.

The trade-off is more runtime consumption and longer total scan chain wire-length.

See Also

- [optimize_dft](#)
- [report_app_options](#)
- [set_app_options](#)

opt.dft.post_fix_polarity

Control whether to run a step to fix polarity in all scan segments of all scan stub chains.

Data Types

Boolean

Default false

Description

With this application option set to true, for each scan segment of the design which has an odd number of inversions, it adds an inverter to the scan in pin of the load cell.

This is performed after repartitioning and reordering.

See Also

- [optimize_dft](#)
- [report_app_options](#)
- [set_app_options](#)

opt.dft.timing_friendly_reorder

Control whether to consider timing info within scan reordering to minimize long scan nets in `optimize_dft`.

Data Types

Boolean

o

Default true**Description**

With this app_option being setting to be TRUE, timing info will be considered into scan reordering to minimize long scan nets in optimize_dft. The trade-off is more runtime consumption.

See Also

- [optimize_dft](#)
- [report_app_options](#)
- [set_app_options](#)

opt.dft.toggle_rate_aware

Enables toggle rate aware optimize_dft to optimize the active wirelength of the scan path.

Data Types*Boolean***Default** false**Description**

When this app option is set to true, the optimize_dft next generation engines will consider the active wirelength of scan path registers as the primary metric for optimizations, so when calculating the cost of implementing new interconnections, the ones that reduce the active wirelength will be prioritized.

The active wirelength correspond to the product of the physical wirelength of the interconnection and the toggle rate of its driver, and it is expected to work as a proxy goal that contributes on reducing the dynamic power of the scan interconnections during optimize_dft.

If multiple potential interconnections share the same active wirelength cost, the engine will decide using the next cost metric, being total scan wirelength minimization the default option.

Toggle rate aware optimize_dft runs under NG infrastructure, so it must be enabled in conjunction with opt.dft.use_ng_engines app option. It also requires switching activity information being propagated across the scan path registers, either manually provided by the user or inferred by the tool using infer_switching_activity command.

o

See Also

- [optimize_dft](#)
- [set_app_options](#)
- [infer_switching_activity](#)

opt.dft.use_ng_engines

Enables next generation engines in DFT optimization.

Data Types

Boolean

Default false

Description

With this app_option turned on, DFT optimization will run using the new optimization engines.

See Also

- [set_optimize_dft_options](#)
- [optimize_dft](#)
- [set_app_options](#)

opt.dynamic_regions.enable

Enables *Dynamic Regions* framework in the *hyper_route_opt* command.

Data Types

bool

Default false

Description

Dynamic Regions is an automatic partitioning framework for reducing *hyper_route_opt* command runtime when the target host has many cores, ideally more than 100. The number of cores is specified using the *set_host_options -max_cores* command.

The framework divides the design into multiple regions during the *hyper_route_opt* command and runs optimization on the created regions in parallel, thus reducing runtime.

o

Dynamic Regions works best on large designs (ideally >5M instances) and requires around 2-3X more memory.

See Also

- [hyper_route_opt](#)
- [set_host_options](#)

opt.dynamic_regions.num_regions

Controls the number of *Dynamic Regions* in the *hyper_route_opt* command when the *opt.dynamic_regions.enable* application option is set to *true*.

Data Types

integer (range 2 - 8)

Default 4

Description

Dynamic Regions is an automatic partitioning framework for reducing *hyper_route_opt* command runtime when the target host has many cores, ideally more than 100. The number of cores is specified using the *set_host_options -max_cores* command.

The *opt.dynamic_regions.num_regions* application option allows you to control the number of regions during the *hyper_route_opt* command. More regions typically provide additional runtime savings at the expense of higher memory and slightly worse PPA in some cases.

See Also

- [hyper_route_opt](#)
- [set_host_options](#)

opt.mcmm.dominant_scenarios

Specifies the user-specified dominant scenarios for optimization.

Data Types

string

Default null

o

Description

Optimization engine might combine scenarios or select a subset of scenarios to work on to improve runtime. If specific scenarios need more optimization, their names can be passed to optimization engine through this application option.

See Also

- [set_app_options](#)

opt.mcmm.use_synthetic_corners

Enable synthetic corner technology to speed up optimization

Data Types

bool

Default false

Description

Optimization engine can combine corners that have similar characteristics into synthetic ones to reduce number of active corners and scenarios. With reduced number of corners and scenarios, optimization can converge faster without losing QoR. The timing numbers for scenarios with synthetic corners will be slightly pessimistic. In the optimization log, a QoR summary will be printed after each optimization stage. The output for a regular scenario would look like

Scenario Func.081v.125c_Cmax WNS = 1.217896, TNS = 264.570660

while the output for a synthetic scenario would look like

Scenario Func.081v.125c_Cmax+Func.081v.0c_Cmax WNS = 2.332123, TNS = 463.1123432 (pessimistic)

See Also

- [place_opt](#)
- [set_app_options](#)

opt.mux.select_op_heavy_module_robustness_mode

Controls robustness mode in the *compile_fusion* for modules having excessive SELECT_OPs.

o

Data Types

string (abort or skip or continue)

Default abort

Description

This application option controls how modules with excessive SELECT_OPs are treated during mux optimization in the initial_map stage of *compile_fusion* command.

This application option supports 3 values, *abort*, *continue* and *skip*.

abort is the default. It aborts compile_fusion on finding modules with excessive SELECT_OPs. An error message is issued with the line numbers of RTL files that contributed to excessive SELECT_OPs.

continue allows mux optimization to proceed as usual. However, a warning message is issued with the line number of RTL files that contributed to excessive SELECT_OPs. Setting this value could result in very long runtime or high memory consumption.

skip skips mux optimizations that can potentially take more runtime or memory due to handling of excessive SELECT_OPs. This value makes mux optimization to run faster and not use up high memory, but might impact area and other metrics.

See Also

- [compile_fusion](#)
- [set_app_options](#)

opt.port.eliminate_verilog_assign

Controls whether optimization eliminates and avoids assign statements in the exported Verilog netlist

Data Types

boolean

Default false

o

Description

Verilog netlists will contain assign statements when multiple ports of the same module are connected to each other inside that module with no other logic between them. This happens when either:

- A module input port directly connects to a module output port (feedthrough connection)
- Multiple module output ports are driven by the same pin inside the module

This happens because a Verilog module port is implicitly connected to a net of the same name as the port. So, when multiple ports are connected together, only one of those port's net names can be used, and the others must be 'assigned' to that same name.

By default, optimization goes to no special effort to either eliminate existing assign statements or avoid creating new assign statements in the exported Verilog netlist.

Some verification tools do not handle assign statements in the netlist, requiring optimization to eliminate them before exporting a netlist to be input to that other tool. This is done by setting the *opt.port.eliminate_verilog_assign* app option to *true*. When this is set, optimization will eliminate existing assign statements in the netlist by adding buffers between connected module ports. These buffers use a *FTB* naming convention. It will also ensure no new assign statements are introduced during optimization. Some small QOR cost may be seen when setting this app option *true*, since some buffers may be inserted in the netlist that otherwise would not have been required to meet the timing and logical DRC QOR goals.

Setting this app option to *true* is not a guarantee that every assign statement will be eliminated in the Verilog netlist. If a net is not a candidate for data buffering, or otherwise could not be buffered due to user constraints like *set_dont_touch*, or due to lib_cell or supply availability issues, then no buffer will be inserted to eliminate the Verilog assign. Use *check_bufferability* to debug and understand reasons for buffering failure.

See Also

- [check_bufferability](#)
- [place_opt](#)
- [set_dont_touch](#)
- [write_verilog](#)

opt.power.effort

Enables higher effort power optimization flows in the preroute flow.

o

Data Types

string

Default low**Description**

There are three effort levels for power optimization in the preroute flow: low, medium, high. The tool default is low. When this option is set to medium or high, the tool runs more aggressive power optimization flows. Higher effort flows will take more runtime, give better power numbers but potentially worse TNS and/or Area.

Please see `opt.power.mode` app-option for details on how to enable power optimization and specify the type of power optimization.

Note: This app-option has been deprecated. Please use `place_opt.flow.enable_power <true/false>`, `clock_opt.flow.enable_power <true/false>` or `compile.flow.enable_power <true/false>`.

See Also

- [place_opt](#)
- [clock_opt](#)
- [report_power](#)
- [set_app_options](#)

opt.power.leakage_type

Enable percentage LVT optimization flow

Data Types*String***Default** conventional**Description**

There are two modes for leakage power optimization: conventional and `percentage_lvt`. The tool default to conventional.

If the user wants to run percentage LVT optimization flow, please use: `opt.power.mode leakage opt.power.leakage_type percentage_lvt`

o

Please make sure the VT groups have been marked using `set_threshold_voltage_group_type` command. Please note that percentage LVT optimization cannot be run with total power optimization enabled.

See Also

- [place_opt](#)
- [clock_opt](#)
- [report_power](#)
- [set_app_options](#)

opt.power.mode

Enable power optimization in preroute flows

Data Types

String

Default none

Description

There are four modes for power optimization: none, leakage, dynamic and total (leakage + dynamic). The tool defaults to <none> and no power optimization flow is done. When this option is set to leakage, the tool will run leakage (no dynamic or total) power optimization flow. When this option is set to dynamic, the tool will run dynamic (no leakage or total) power optimization flow. When this option is set to total, the tool will run total (leakage + dynamic) power optimization flow. Please make sure at least 1 corner is appropriately power enabled (-leakage or -dynamic) Please use `opt.power.effort` app-option to enable higher effort power optimization flows.

Note: This app-option has been deprecated. Please use `place_opt.flow.enable_power <true/false>`, `clock_opt.flow.enable_power <true/false>` or `compile.flow.enable_power <true/false>`.

See Also

- [place_opt](#)
- [clock_opt](#)
- [report_power](#)
- [set_app_options](#)

o

opt.preroute_synthesis.architecture_booster_transform

Turn on re-synthesis of suboptimal arithmetic operators.

Data Types

boolean

Default false

Description

Setting this application option to true will allow optimization engine to re-architect suboptimal arithmetic operators during physical synthesis. It can be used to speed up long data-path chains by spending more resources.

Please note that re-architecting can induce regional area overhead it should be used only early in the optimization flow.

See Also

- [place_opt](#)
- [clock_opt](#)
- [set_app_options](#)

opt.tie_cell.add_to_highest_hierarchy

Controls tie-cell insertion to highest possible hierarchy.

Data Types

boolean

Default True

Description

During optimization, tie-cells will be inserted to drive constant pins if tie-cells were available for optimization purpose. By default, tie-cells are inserted to the highest possible hierarchy. This option can be used to change the behavior and insert tie-cell to the lowest possible hierarchy. This option will not affect the existing tie-cells unless they are already violating other constraints.

o

See Also

- [place_opt](#)
- [set_lib_cell_purpose](#)
- [set_app_options](#)

opt.tie_cell.check_frontend_restrictions

Enable tiecell insertion to check front end restrictions on constant net segments

Data Types

boolean

Default false

Description

During optimization, tie-cells will be inserted to drive constant pins if tie-cells were available for optimization purpose. This option allows to check for front-end restriction related constraints on constant net segments passing through hierarchial load pins. If this option is enabled, then tie opt will check the net for any front-end related constraints. If found, any such constraints, then tie cell will be added to a port outside the net, if present.

See Also

- [place_opt](#)
- [set_lib_cell_purpose](#)
- [set_app_options](#)

opt.tie_cell.enable_boundary_restriction_check

Enable tiecell insertion to check front end boundary optimization restrictions on constant net segments

Data Types

boolean

Default true

Description

During optimization, tie-cells will be inserted to drive constant pins if tie-cells were available for optimization purpose. This option allows to check for boundary optimization

o

restriction related constraints on constant net segments passing through hierarchical load pins. If this option is enabled, then tie_opt will check each constant net segments whether any of its hierarchical load pins is having any boundary optimization related constraints, primarily front-end restrictions. If found, any such constraints, then tie cell will not be inserted for the constant nets and its driving loads.

See Also

- [place_opt](#)
- [set_lib_cell_purpose](#)
- [set_app_options](#)

opt.tie_cell.ignore_non_user_dont_touch

App option to ignore non user dont_touch in tiecell engine.

Data Types

boolean

Default false

Description

Enabling this app option make tieopt engine to ignores any non user dont_touch like dc_generic and any other specific dont_touch reasons.

See Also

- [place_opt](#)
- [clock_opt](#)
- [set_app_options](#)

opt.tie_cell.max_fanout

Specifies the maximum fanout a tie-cell can drive.

Data Types

integer

Default 999

o

MINIMUM

1

Description

During optimization, tie-cells are inserted to drive constant pins if tie-cells are available for optimization purpose. This option limits the maximum fanout each newly inserted tie-cell can drive even if it can drive more pins without violating other constraints. If there are pre existing tie-cells in the design that violates max fanout constraint, they are removed and new tie-cells are inserted to satisfy the constraint unless pre existing tie-cells have dont_touch attribute. If opt.tie_cell.scan_pin_max_fanout is not set by you and if the estimated number of tie-cell count is greater than 50 percent of the number of sequential cells in the design, the max_fanout setting for the scan-chain constant pins are set to 999. In that case, all the other constant pins are considered for tie-cell insertion following opt.tie_cell.max_fanout).

See Also

- [place_opt](#)
- [set_lib_cell_purpose](#)
- [set_app_options](#)

opt.tie_cell.preserve_restricted_hier_nets

Enables to preserve MV restricted net segments driving constant pins.

Data Types

Boolean

Default false

Description

When enabled, this application option makes tieopt engine to consider handling some MV restricted nets by inserting tie cell before the restricted net segments. This helps to prevent the check_mv_design error for tie-off connection, and MV loads being driven by Logic 1/0 directly.

See Also

- [place_opt](#)
- [clock_opt](#)
- [set_app_options](#)

o

opt.tie_cell.support_auto_vt_classification

Enable auto leakage vt classification during tie-cell optimization

Data Types

boolean

Default true

Description

During optimization, the master list of available lib-cells gets sorted based on leakage, and least power hungry cell is selected while adding a tie-cell. If this option is disabled, then power requirement won't be considered and tie-cell needing more power may be picked up.

See Also

- [place_opt](#)
- [set_app_options](#)

opt.timing.effort

Control the optimization effort for timing (WNS/TNS) in place_opt/clock_opt/refine_opt commands.

Data Types

String

Default low

Description

Enable higher effort timing optimization in preroute flows.

The timing recovery effort can be: low (default), medium or high.

With increasing effort, the tool is expected to give better WNS/TNS at the cost of runtime, area and possibly power.

See Also

- [place_opt](#)
- [set_app_options](#)

p

p

package.common.angle_tolerance

Specify the angle tolerance for acute angle checking/fixing in package DRC.

Data Types

Double

Default 0.01 (degree)

Description

This option specifies the angle of tolerance for acute angle checking. The value is a double number. The unit is degree.

Currently, this is only effective for acute angle fixing in DRC checking/fixing (check/fix_package_drc) commands. The affected commands and flow will be extended in the future.

The angle tolerance is for users who are not satisfied with the default angle tolerance. If this app option is set to a non-negative value, the tolerance related to angle will be set accordingly. This value is for user to have the flexibility to set the angle tolerance for acute angle checking/fixing in package DRC.

Examples

The following example sets the angle tolerance to 0.1-degree.

```
prompt> set_app_options -name package.common.angle_tolerance -value 0.1
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)

package.common.d2d_region_layer

Specify the layer name of die-to-die region.

Data Types

string

Default Empty string

p

Description

This option specifies the layer name of die-to-die region. It would be used in package router and PDN features for die-to-die connections.

Examples

The following example sets die-to-die region layer to D2D.

```
prompt> set_app_options -name package.common.d2d_region_layer -value D2D
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)

package.common.die_region_layer

Specify the layer name of die region.

Data Types

string

Default Empty string

Description

This option specifies the layer name of die region. It would be used in package router for die-to-die routing.

Examples

The following example sets die region layer to DIE.

```
prompt> set_app_options -name package.common.die_region_layer -value DIE
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)

package.common.enable_obstacle_aware_concave_hull

Enable or disable obstacle-aware behavior for concave hull mode in *create_package_metal_areas* and *create_package_bounding_shape* commands.

Data Types

Boolean

Default False

Description

Specify a value to enable or disable obstacle-aware concave hull generation for *create_package_metal_areas* and *create_package_bounding_shape* commands.

If obstacle-aware mode is enabled, it would treat unselected objects as obstacles even if they are same net objects. It would avoid generating polygons touching the obstacles if possible. Not all shape types are supported.

- Command *create_package_metal_areas* supports vias and bumps.
- Command *create_package_bounding_shape* supports vias, bumps, and path types.
- Unsupported types like polygon and rectangle would be ignored during concave hull generation.

If obstacle-aware mode is disabled, concave hull generation would not consider any obstacles in the design. It would generate concave hulls with selected objects only.

Examples

The following example enables obstacle-aware concave hull generation for *create_package_bounding_shape* command.

```
prompt> set_app_options -name  
package.common.enable_obstacle_aware_concave_hull -value true  
prompt> create_package_bounding_shape -layer RDL1 -from_selection -mode  
concave_hull
```

The following example enables obstacle-aware concave hull generation for *create_package_metal_areas* command.

```
prompt> set_app_options -name  
package.common.enable_obstacle_aware_concave_hull -value true  
prompt> create_package_metal_areas -nets VSS -layers RDL1 -from_selection  
-method bounding -bounding_mode concave_hull
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)

package.common.fob_region_layer

Specify the layer name of fanout boundary region.

Data Types

string

Default Empty string

Description

This option specifies the layer name of fanout boundary region. It would be used in package router and PDN features.

Examples

The following example sets fanout boundary region layer to FOB.

```
prompt> set_app_options -name package.common.fob_region_layer -value FOB
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)

package.common.length_tolerance

Specify the length tolerance for min width rules.

Data Types

Double

Default -1.0

Description

This option specifies the length of tolerance for min width rules. The value is a double number.

p

Currently, this is only effective for min width fixing in DRC checking/fixing (check/fix_package_drc) commands. The affected commands and flow will be extended in the future.

The length tolerance is for users who are not satisfied with the technology file-defined min width tolerance value. If this app option is set to a non-negative value, the tolerance related to length will be set accordingly. Generally, users who set this value lower than the technology file-defined value are more strict on the length tolerance, which is more conservative. Furthermore, it is legal for users to set the value that is larger than the technology file-defined value but it might generate some DRC violations which might not be expected.

Examples

The following example sets the length tolerance to 0.1um.

```
prompt> set_app_options -name package.common.length_tolerance -value 0.1
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)

package.common.min_floating_island_size

Specify the minimal floating island area size threshold used by floating metal shape removal in automation commands.

Data Types

Float point number

Default -1

Description

This option specifies the minimal floating island area size threshold. The value is in double precision floating point number. The unit is in squared micrometer (um²). The default value is -1.0 (negative). When the value is -1.0 or any negative value, the command uses the default value of twice of smallest via area size at each layer. For example, when this value is -1.0, and the smallest via metal diameter is 18um at CU_PPI layer, the command uses $2 * \text{Pi} * 9.0 \times 9.0 = 508.938$ for CU_PPI layer floating islands.

When the value is 0.0, no floating shape is reported.

p

By specifying the minimal floating island area size threshold, any floating island area less than the value is removed during `create_package_shapes_for_metal_area` command.

Currently this is only effective on (1) PDN metal generation (`create_package_shapes_for_metal_area` command), and (2) `check_package_drc` command. The effected commands and flow will be extended in the future.

A floating shape is a metal shape with no center point connection to any via, pin, and bump. The floating metal shapes are usually undesirable, because they often degrades electric characteristics and increase compacitance. There is a sub-type of floating shape, which connects only one via (one center point). When this type of shapes are smaller than the minimal floating island area size threshold, they are also reported as floating.

Examples

The following example sets the minimal floating island size to 100 μm^2 at all layers in Package Compiler commands.

```
prompt> set_app_options -name package.common.min_floating_island_size
        -value 100
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)

package.common.pg_nets_layers

Specify the layer assignment of PG nets.

Data Types

string

Default Empty string

Description

This option specifies the layer assignment of PG nets. The value format is `{{pg_net_name_1 routing_layer_1 [routing_layer_2 ...]} [...]}`. It would be used in package router and PDN features. There could be multiple layer assignments for each PG net. The first layer is called primary layer, and the other layers are called secondary layers. Package router would insert vias from pin layers to the primary layer. Both primary layer and secondary layer affect router's DRC model for the PG nets. It would ignore same net spacing violations and always apply plane or line spacing depending on value of the app

p

option *package.route.pg_plane_spacing_model*. If PG nets are not specified in this option, router would automatically determine one primary layer for each unspecified PG net.

Examples

The following example sets primary layers of net VDD and VSS to RDL1 and RDL2 layers, respectively. It also sets secondary layers of net VSS to RDL1 and RDL3 layers.

```
prompt> set_app_options -name package.common.pg_nets_layers -value {{VDD
RDL1} {VSS RDL2 RDL1 RDL3}}
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)

package.common.remove_floating_shapes

Specify the behavior of floating metal shape removal in automation commands.

Data Types

Boolean

Default True

Description

This option specifies the behavior of floating metal shape removal in automation commands. The value is Boolean true or false.

Currently this is only effective on PDN metal generation (create_package_shapes_for_metal_area command). The effected commands and flow will be extended in the future.

A floating shape is a metal shape with no connection to any via, pin, and bump. The connection is center-point model. A metal shape is floating when (A) it does not touch the center point of any via, pin, or bump. Or (B) it touches the center point of only one via, pin, or bump. (B) is a sub-type of floating shape. The floating metal shapes are usually undesirable, because they often degrades electric characteristics and increase capacitance.

When their area size is smaller than the value specified in package.common.min_floating_island_size, they are treated as floating shapes.

p

Examples

The following example disables automatic floating shape removal in Package Compiler commands.

```
prompt> set_app_options -name package.common.remove_floating_shapes
        -value false
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)

package.mfg.degas_hole_clearance

Specify the distance between degas holes and via, path, and bump objects.

Data Types

string

Default Empty string

Description

The option specifies the distance between degassing holes and via, path, and bump objects. The value will be at least one single value and multiple layer-values pairs in the following format:

```
{ [default_cvalue] [ { layer_name [layer_cvalue] [{object_type
object_cvalue} ...] } ... ] }.
```

The user can provide a single value(default_cvalue) as the default value for all layers and objects. For each layer pair, the user can specify a default value(layer_cvalue) for all objects in the layer and a specific value(object_cvalue) for each object type. When the value is not specified for an object type, the default value for the layer will be used. When the value is not specified for a layer, the overall default value will be used. If the app option is not set, the degassing holes will keep zero clearance with the via, path, and bump objects.

Examples

The following examples show how to set the degas hole clearance for different layers and objects.

The example sets the default clearance to 10 user-units for all objects in all layers.

```
prompt> set_app_options -name package.mfg.degas_hole_clearance -value 10
```

p

The example sets the clearance to 10 user-units for all objects in layer1 and 20 user-units for all objects in layer2.

```
prompt> set_app_options -name package.mfg.degas_hole_clearance -value
        {{layer1 10} {layer2 20}}
```

The example sets the clearance to 10 user-units for vias and 20 user-units for paths in layer1 and 30 user-units for bumps in layer2.

```
prompt> set_app_options -name package.mfg.degas_hole_clearance -value
        {{layer1 {via 10} {path 20}} \
        {layer2 {bump 30}}}
```

The example sets the clearance to 10 user-units for vias and 20 user-units for paths in layer1 and 30 user-units for bumps in layer2, 20 user-units for other objects in layer2.

```
prompt> set_app_options -name package.mfg.degas_hole_clearance -value
        {{layer1 {via 10} {path 20}} \
        {layer2 20 {bump 30}}}
```

The example sets the clearance to 10 user-units for layer1 and 20 for other layers.

```
prompt> set_app_options -name package.mfg.degas_hole_clearance -value {20
        {layer1 10}}
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)

package.pdn.void_sizing_strategy

Specify the sizing strategy for width-based spacing rule in create_package_shapes_for_metal_area command.

Data Types

enum

Default complex_width

Description

For *complex_width*, multiple width-based spacing values are applied to the object, based on the width of each fragment of the object. For *simple_width*, a single width-based spacing value is applied to the object, based on the widest width in metal_area and shape. For *forced_narrow_width*, a single width-based spacing value is applied to the object,

p

based on the widest width in shape. The metal_area will be regarded as narrow width. For *forced_all_narrow_width*, a single width-based spacing value is applied to the object. The shape and metal_area will be regarded as narrow width. For *forced_all_wide_width*, a single width-based spacing value is applied to the object. The shape and metal_area will be regarded as wide width.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)

package.route.allow_violation_on_failure

Specify whether to leave results with violations when routing failed.

Data Types

Boolean

Default True

Description

The option controls whether to leave results with violations when routing failed. If the option is set to false, router will leave open when routing failed.

Examples

The following example sets the option to be false. Router will leave open when routing failed.

```
prompt> set_app_options -name package.route.allow_violation_on_failure  
-value false
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)

package.route.d2d_shielding_rules

Specify shielding rules for die-to-die routing.

p

Data Types

*string***Default** Empty string

Description

This option specifies shielding rules for die-to-die routing connections. It defines which pg net would be considered as shielding net in the specified layer. Router would not consider shielding without the setting. The value format is

```
{
{
{layer metal layer}
{shield_net pg net}
}
[...]
```

layer metal layer

Specify the metal layer.

shield_net shield net

Specify pg net, should be power or ground nets.

Examples

The following example creates a d2d shielding rule with VDD on each layer.

```
prompt> set_app_options -name package.route.d2d_shielding_rules -value
"{{layer RDL1} {shield_net VDD}} {{layer RDL2} {shield_net VDD}} {{layer
RDL3} {shield_net VDD}} {{layer RDL4} {shield_net VDD}}"
```

The following example creates a d2d shielding rule with VDD on RDL1 and RDL3, VSS on RDL2 and RDL4.

```
prompt> set_app_options -name package.route.d2d_shielding_rules -value
"{{layer RDL1} {shield_net VDD}} {{layer RDL2} {shield_net VSS}} {{layer
RDL3} {shield_net VDD}} {{layer RDL4} {shield_net VSS}}"
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)

package.route.d2s_bridge_style_max_search_distance

Specify the max search distance of pin-pairing in d2s bridge style routing.

Data Types

Float

Default 0.0

Description

This option specifies the max search distance of pin-pairing in d2s bridge style routing. The value is in user unit. The default value is 0.0, which means router would automatically determine the max search distance based on the package technology. If the value is set to a non-zero value, router would use the specified value to determine if two pins can be paired for routing in d2s bridge style routing. For example, if the value is set to a small number, router would only pair pins that are close to each other.

Examples

The following example sets the max search distance to 1.5 user unit.

```
prompt> set_app_options -name  
package.route.d2s_bridge_style_max_search_distance -value 1.5
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)

package.route.d2s_pg_via_sharing_rules

Specify PG via sharing rules in die-to-substrate routing.

Data Types

string

Default Empty string

Description

This option specifies PG via sharing rules in die-to-substrate routing. The value format is

```
{  
{  
{cluster_distance distance}}
```

p

```
[{cluster_angle angle}]
[{nets net1 net2...}]
[{coordinates {{x1 y1} {x2 y2} {x3 y3}...}...}]
[{exclude_coordinates {{x1 y1} {x2 y2} {x3 y3}...}...}]
[{layers layer1 layer2...}]
}
[...]
}
```

cluster_distance *distance*

Specify the distance of clustering for via sharing.

cluster_angle *angle*

Specify the angle of clustering direction for via sharing. Without angle, clustering direction is radical.

nets *list_of_nets*

Specify the nets to apply the rule.

coordinates *list_of_regions*

Specify the regions to apply the rule.

exclude_coordinates *list_of_regions*

Specify the regions to not apply the rule.

layers *list_of_layers*

Specifies the layers to apply the rule.

Examples

The following example creates clusters that each shape within a cluster can find another shape in 0 degrees within 40 user-units. The rule is applied to shapes belong to net1 and net2 on layer1 and layer2 within region {{0 0} {100 100}} but not {{20 20} {40 40}}.

```
prompt> set_app_options -name package.route.d2s_pg_via_sharing_rules
-value "{{cluster_distance 40} {cluster_angle 0} {nets net1 net2}
{coordinates {0 0} {100 100}} {exclude_coordinates {20 20} {40 40}}
{layers layer1 layer2}}"
```

The following example creates two rules. The cluster condition are both with 40 user-units without angle which means in each cluster, a shape can find another shape within 40 user-units. The first rule is applied to shapes belong to net1 on layer 1 while the second rule is applied to shapes belong to net2 on layer2.

```
prompt> set_app_options -name package.route.d2s_pg_via_sharing_rules
-value "{{cluster_distance 40} {nets net1} {layers layer1}}
{{cluster_distance 40} {nets net2} {layers layer2}}"
```

p

The following example shows how to use pre-defined list variables in appOption value for options. To use variable in this appOption, the outmost scope of value must be "" instead of {}.

```
prompt> set nets {net1 net2}
prompt> set_app_options -name package.route.d2s_pg_via_sharing_rules
      -value "{{cluster_distance 40} {nets $nets}}"
```

The following example shows how to form pre-defined variables into list in appOption value for options. To use variable in this appOption, the outmost scope of value must be "" instead of {}.

```
prompt> set region1 {{0 0} {100 100}}
prompt> set region2 {{200 0} {300 100}}
prompt> set_app_options -name package.route.d2s_pg_via_sharing_rules
      -value "{{cluster_distance 40} {coordinates [list $region1 $region2]}}"
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)

package.route.d2s_via_escaping_angles

Specify via escaping angle constraints in die-to-substrate routing.

Data Types

string

Default Empty string

Description

This option is deprecated, please consult the man page for the new option *package.route.d2s_via_insertion_patterns*. This option specifies via escaping angle constraints in die-to-substrate routing. The value format is *{{{llx1 lly1} {urx1 ury1} routing_layer_1 cut_layer_1 angle_1} [...]}}*. It defines the via escaping angle on *routing_layer_n* when escaping *cut_layer_n* vias in the specified region *{llx1 lly1} {urxn ury1}*. If angle is 0, router would try to escape from the source shape on *routing_layer_n* to the *cut_layer_n* target via to the right direction. If there are conflicts in constraints, the latest one has higher priority. If angle is *auto*, it would reset previous constraints on *routing_layer_n* to the *cut_layer_n* target via in the specified region *{llx1 lly1} {urxn ury1}*.

p

Examples

The following example creates a 90-degree via escaping angle constraint in region {0 0} {200 300} on RDL layer for VIA2 layer via.

```
prompt> set_app_options -name package.route.d2s_via_escaping_angles
        -value {{{0 0} {200 300} RDL2 VIA2 90}}}
```

The following example creates a 90-degree via escaping angle constraint in larger region {0 0} {200 300} on RDL2 layer for VIA2 vias and a 0-degree via escaping angle constraint in smaller region {50 50} {100 100} on RDL2 layer for VIA2 vias. It creates a 90-degree constraint in the larger region, but in the smaller region the angle is 0-degree.

```
prompt> set_app_options -name package.route.d2s_via_escaping_angles
        -value {{{0 0} {200 300} RDL2 VIA2 90} {{50 50} {100 100} RDL2 VIA2 0}}}
```

The following example creates a 90-degree via escaping angle constraint in larger region {0 0} {200 300} on RDL2 layer for VIA2 vias and reset the constraint in smaller region {50 50} {100 100} on RDL2 layer for VIA2 vias. It creates a 90-degree constraint in the larger region, but there is no constraint in the smaller region due to auto angle.

```
prompt> set_app_options -name package.route.d2s_via_escaping_angles
        -value {{{0 0} {200 300} RDL2 VIA2 90} {{50 50} {100 100} RDL2 VIA2
        auto}}}
```

The following example creates a 90-degree via escaping angle constraint in larger region {0 0} {200 300} on RDL2 layer for VIA2 vias and a 0-degree via escaping angle constraint in smaller region {50 50} {100 100} on RDL4 layer for VIA4 vias. Since the two constraints are on different layers, they do not have conflicts. Router would try to follow the angles based on layer and region.

```
prompt> set_app_options -name package.route.d2s_via_escaping_angles
        -value {{{0 0} {200 300} RDL2 VIA2 90} {{50 50} {100 100} RDL4 VIA4 0}}}
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)

package.route.d2s_via_escaping_counts

Specify via escaping count constraints in die-to-substrate routing.

Data Types

string

p

Default Empty string**Description**

This option is deprecated, please consult the man page for the new option `package.route.d2s_via_insertion_counts`. This option specifies via escaping count constraints in die-to-substrate routing. It allows router creating multiple vias stacking on top of large pins or vias.

The value format is `{{layer_1 cut_layer_1 via_count_1 net_type_1} [...]}}`. It defines the number of `cut_layer_n` vias to create when router escapes `net_type_n` nets from `layer_n` pins or vias. The valid value of `layer_n` could be either metal layer or cut layer. The valid value of `via_count_n` is 2 or 3. The valid value of `net_type_n` is `signal` or `pg`. The `cut_layer_n` vias should be able to stack on `layer_n` in package techfile rules. Otherwise, this constraint would be ignored.

If router gets DRC violations on the multiple vias, it will try to rotate the vias together. If still fails, router would give up multiple vias solution and then use single via solution instead. Router would not create paths to connect between the multiple vias as they are supposed to be connected on large pins or vias on the specified layer. During routing, router could choose one of vias to connect to other shapes dependent on routing costs.

Examples

The following example creates a VIA1 layer 2-via escaping constraint on TSV layer for signal nets.

```
prompt> set_app_options -name package.route.d2s_via_escaping_counts
        -value {{TSV VIA1 2 signal}}
```

The following example creates two via escaping count constraints. The first one is VIA1 layer 2-via escaping constraint on TSV layer for signal nets. The second one is VIA1 layer 3-via escaping constraint on TSV layer for PG nets.

```
prompt> set_app_options -name package.route.d2s_via_escaping_counts
        -value {{TSV VIA1 2 signal} {TSV VIA1 3 pg}}
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)

package.route.d2s_via_insertion_counts

Specify via insertion counts constraints in die-to-substrate routing.

p

Data Types*string***Default** Empty string**Description**

This option specifies via insertion counts constraints in die-to-substrate routing. It allows router creating multiple vias stacking on top of large pins or vias. If router gets DRC violations on the multiple vias, it will try to rotate, shift or enlarge the via pattern together. If it still fails, router would leave pattern with drc violation if app option "package.route.allow_violation_on_failure" is set to true. The expected pattern would be the most compact one and the via count is the first value from option "counts". During routing, router could choose one of vias to connect to other shapes dependent on routing costs. When there are multiple constraints applied to same set of nets, router will honor the latest one. The rotation can be controlled by app option "package.route.d2s_via_insertion_patterns". Router always generate right polygon pattern. The definition of 0 degrees rotation is the pattern with one horizontal edge at the bottom of the pattern. The pattern is rotated counter-clockwise. Error markers will be generated if there are failed patterns. The error file name is "route_package_d2s.err" and the error name is "d2s_via_insertion_counts failures".

The value format is

```
{
{
{layer layer}
{cut_layers list_of_cut_layers}
{counts list_of_count}
[{net_type signal | pg}]
}
[...]
```

layer layer

Specifies the layer to apply the constraint.

cut_layers list_of_cut_layers

Specifies a list of cut layers to apply the constraint. Router would determine whether to enable multi-via insertion based-on *list_of_cut_layers* and *layer. layer* specifies the starting layer to insert multi-via; *list_of_cut_layers* determines the maximum number of layers require multi-via patterns. If there are multiple layers in *list_of_cut_layers* and multi-layer vias are required to reach the assigned primary layer of routing nets, router would search the feasible multi-via patterns on multi-layer simultaneously. Router would rotate, lengthen, or shift multi-via patterns on the specified cut layers at the same time to try to find a DRC clean solution.

p

counts *list_of_count*

Specifies the counts of *cut layer* vias to insert, when inserting via from *layer*. Router will try them one by one. If there are multiple cut layers in *list_of_cut_layers*, all cut layers would apply the same count at the same time. The minimum count can be specified is 2; the maximum count can be specified is 10.

net_type signal | pg

Specifies the net type to apply the constraint. Router will apply constraint to all types of nets without this option.

Examples

The following example creates a VIA1 layer 2-via insertion constraint on TSV layer for signal nets.

```
prompt> set_app_options -name package.route.d2s_via_insertion_counts
-value {{{layer TSV} {cut_layers VIA1} {counts 2} {net_type signal}}}
```

The following example creates a VIA1 layer 3-via or 2-via insertion constraint on TSV layer for pg nets. Router tries to insert 3 vias first. If all 3 vias solutions are invalid, router tries to insert 2 vias.

```
prompt> set_app_options -name package.route.d2s_via_insertion_counts
-value {{{layer TSV} {cut_layers VIA1} {counts 3 2} {net_type pg}}}
```

The following example creates two via insertion counts constraints. The first one is VIA1 layer 2-via insertion constraint on TSV layer for signal nets. The second one is VIA1 layer 3-via insertion constraint on TSV layer for PG nets.

```
prompt> set_app_options -name package.route.d2s_via_insertion_counts
-value {{{layer TSV} {cut_layers VIA1} {counts 2} {net_type signal}}
{{{layer TSV} {cut_layers VIA1} {counts 3} {net type pg}}}
```

The following example creates a VIA1 layer 2-via insertion constraint on TSV layer for signal nets. Router could leave results with DRC violations if it fails to insert vias.

```
prompt> set_app_options -name package.route.allow_violation_on_failure
-value true
prompt> set_app_options -name package.route.d2s_via_insertion_counts
-value {{{layer TSV} {cut_layers VIA1} {counts 2} {net_type signal}}}
```

The following example creates a VIA1 layer 2-via insertion constraint on TSV layer for signal nets. The rotation of two via pattern is set to 90 degrees.

```
prompt> set_app_options -name package.route.d2s_via_insertion_patterns
-value {{{layer FS_TIV} {cut_layers VIA1} {angle 90}}}
prompt> set_app_options -name package.route.d2s_via_insertion_counts
-value {{{layer TSV} {cut_layers VIA1} {counts 2} {net_type signal}}}
```

p

The following example creates VIA1 and VIA2 layers 3-via or 2-via insertion constraint on TSV layer for pg nets. Router tries to insert 3 VIA1 and 3 VIA2 vias first. If all 3 vias solutions are invalid, router tries to insert 2 VIA1 and 2 VIA2 vias.

```
prompt> set_app_options -name package.route.d2s_via_insertion_counts
-value {{{layer TSV} {cut_layers VIA1 VIA2} {counts 3 2} {net_type pg}}}
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)

package.route.d2s_via_insertion_patterns

Specify via insertion pattern constraints in die-to-substrate routing.

Data Types

string

Default Empty string

Description

This option specifies via insertion pattern constraints in die-to-substrate routing. The value format is

```
{
{
{layer metal layer}
{cut_layer cut layer}
{angle angle}
[{coordinates {{x1 y1} {x2 y2} {x3 y3}...}...}]
}
[...]
```

layer metal layer

Specifies the metal layer to apply the constraint.

cut_layer cut layer

Specifies the cut layer, when inserting via with cut_layer from *metal layer*, should apply the constraint.

p

angles *list_of_angles*

Specifies the angles which router would try to insert staggering via with *cut_layer* from *metal_layer*. If multiple angles are specified, router would pick user defined angles by order and use the first angle that is feasible. If router cannot find out feasible angle in the used specified angles, router will leave routing pattern created by 1st angle with violations.

coordinates *list_of_regions*

Specify the regions to apply the constraint.

If there are conflicts in constraints, the latest one has higher priority. If angle is *auto*, it would reset previous constraints on *metal_layer_n* to the *cut_layer_n* target via in the specified region $\{\{llyn\} \{urxn\} \{uryn\}\}$.

Examples

The following example creates a 90-degree via insertion angle constraint in region $\{\{0\ 0\} \{200\ 300\}\}$ on RDL layer for VIA2 layer via.

```
prompt> set_app_options -name package.route.d2s_via_insertion_patterns
-value {{{coordinates {{0 0} {200 300}}}} {layer RDL2} {cut_layer VIA2}
{angles 90}}}
```

The following example creates a 90-degree via insertion angle constraint in larger region $\{\{0\ 0\} \{200\ 300\}\}$ on RDL2 layer for VIA2 vias and a 0-degree via insertion angle constraint in smaller region $\{\{50\ 50\} \{100\ 100\}\}$ on RDL2 layer for VIA2 vias. It creates a 90-degree constraint in the larger region, but in the smaller region the angle is 0-degree.

```
prompt> set_app_options -name package.route.d2s_via_insertion_patterns
-value {{{coordinates {{0 0} {200 300}}}} {layer RDL2} {cut_layer VIA2}
{angle 90}} {{{coordinates {{50 50} {100 100}}}} {layer RDL2} {cut_layer
VIA2} {angles 0}}}
```

The following example creates a 90-degree via insertion angle constraint in larger region $\{\{0\ 0\} \{200\ 300\}\}$ on RDL2 layer for VIA2 vias and reset the constraint in smaller region $\{\{50\ 50\} \{100\ 100\}\}$ on RDL2 layer for VIA2 vias. It creates a 90-degree constraint in the larger region, but there is no constraint in the smaller region due to auto angle.

```
prompt> set_app_options -name package.route.d2s_via_insertion_patterns
-value {{{coordinates {{0 0} {200 300}}}} {layer RDL2} {cut_layer VIA2}
{angle 90}} {{{coordinates {{50 50} {100 100}}}} {layer RDL2} {cut_layer
VIA2} {angles auto}}}
```

The following example creates a 90-degree via insertion angle constraint in larger region $\{\{0\ 0\} \{200\ 300\}\}$ on RDL2 layer for VIA2 vias and a 0-degree via insertion angle constraint in smaller region $\{\{50\ 50\} \{100\ 100\}\}$ on RDL4 layer for VIA4 vias. Since the two constraints are on different layers, they do not have conflicts. Router would try to follow the angles based on layer and region.

p

```
prompt> set_app_options -name package.route.d2s_via_insertion_patterns
-value {{{coordinates {{0 0} {200 300}}} {layer RDL2} {cut_layer VIA2}
{angle 90}} {{{coordinates {{50 50} {100 100}}} {layer RDL4} {cut_layer
VIA4} {angles 0}}}
```

The following example creates a 90-degree via insertion angle constraint in region {{0 0} {100 100}} and region {{200 0} {300 100}} on RDL2 layer for VIA2 vias. The option *coordinates* takes list of regions, so the two regions and be written in a single constraint. The following example also shows how to use pre-defined list variables in appOption value for options. To use variable in this appOption, the outmost scope of value must be "" instead of {}.

```
prompt> set_region1 {{0 0} {100 100}}
prompt> set_region2 {{200 0} {300 100}}
prompt> set_app_options -name package.route.d2s_via_insertion_patterns
-value "{{{coordinates [list $region1 $region2]} {layer RDL2} {cut_layer
VIA2} {angles 90}}}"
```

The following example creates a multi via insertion angles constraint on RDL2 layer for VIA2 vias.

```
prompt> set_app_options -name package.route.d2s_via_insertion_patterns
-value "{{{{{layer RDL2} {cut_layer VIA2} {angles 90 135 180 auto}}}"
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)

package.route.ndr_vias

Specify non-default rule via constraints in die-to-substrate routing.

Data Types

string

Default Empty string

Description

This option specifies non-default rule via constraints in die-to-substrate routing. It can only take net names and via def names instead of object collections. Auto router only supports custom via defs, so the via def names must be existing custom via defs in database. The value format is

p

```
{
{
{nets net1 net2...}
{vias via_def1 via_def2...}
}
[...]
```

nets *list_of_nets*

Specify the nets to apply ndr vias.

vias *list_of_via_defs*

Specify the ndr vias for nets.

Examples

The following example creates a ndr via constraint with custom via def VIA1_50 for net n1.

```
prompt> set_app_options -name package.route.ndr_vias -value "{{nets n1}
{vias VIA1_50}}"
```

The following example creates a ndr via constraint with custom via defs VIA1_50, VIA2_50, and VIA3_50 for net n1.

```
prompt> set_app_options -name package.route.ndr_vias -value "{{nets n1}
{vias VIA1_50 VIA2_50 VIA3_50}}"
```

The following example creates a ndr via constraint with custom via defs VIA1_50, VIA2_50, and VIA3_50 for net n1, n2, and n3.

```
prompt> set_app_options -name package.route.ndr_vias -value "{{nets n1 n2
n3} {vias VIA1_50 VIA2_50 VIA3_50}}"
```

The following example creates two ndr via constraints. The first one is with custom via defs VIA1_50, VIA2_50, and VIA3_50 for net n1, n2, and n3. The second one is with custom via defs VIA2_50 for net n4.

```
prompt> set_app_options -name package.route.ndr_vias -value "{{nets n1 n2
n3} {vias VIA1_50 VIA2_50 VIA3_50}} {{nets n4} {vias VIA2_50}}"
```

The following example creates two ndr via constraints. The first one is with custom via defs VIA1_50 and VIA2_50 for all PG nets. The second one is with custom via defs VIA2_30 and VIA2_30 for all signal nets. To use variable in this appOption, the outmost scope of value must be "" instead of {}.

```
prompt> set_signal_net_names [get_attribute -name name [get_nets -filter
{net_type == signal}]]
prompt> set_pg_net_names [get_attribute -name name [get_nets -filter
{net_type == power || net_type == ground}]]
```


p

```
prompt> set_app_options -name package.route.ndr_vias -value "{{nets
$pg_net_names} {vias VIA1_50 VIA2_50}} {{nets $signal_net_names} {vias
VIA1_30 VIA2_30}}"
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)

package.route.pg_plane_spacing_model

Specify the PG plane spacing model.

Data Types

Enum

Default "optimistic"

Description

This option specifies the PG plane spacing model.

The default value is *optimistic*, router would apply ordinary spacing model for the PG nets and layers defined in *package.common.pg_nets_layers* app option.

If the value is set to *pessimistic*, router would apply plane spacing model for any shapes on the PG nets and layers defined in *package.common.pg_nets_layers* app option regardless of shape sizes.

Examples

The following example sets the PG plane spacing model to pessimistic.

```
prompt> set_app_options -name package.route.pg_plane_spacing_model -value
pessimistic
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)

pin_check.analyze.analyze_lib_cell_placement_cmd

Specifies the user defined `analyze_lib_cell_placement` command and options.

Data Types

string

Default ""

Description

This application option specifies the user defined *analyze_lib_cell_placement* command and options, excepts the `-lib_cells` option.

The option value is used in *check_libcell_pin_access -mode analyze_lib_cell*.

Examples

The following example specifies the user defined command and options.

```
prompt> set_app_options -as_user_default \  
         -name pin_check.analyze.analyze_lib_cell_placement_cmd \  
         -value "analyze_lib_cell_placement -trials 0 -threshold 0.5" \  
prompt> check_libcell_pin_access -mode analyze_lib_cell
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [check_libcell_pin_access](#)

pin_check.analyze.analyze_lib_pin_check_routability

Specifies whether to enable generating report of pin access by router.

Data Types

boolean

Default false

Description

This application option specifies whether to enable report pin access by router. If the value is true, generating report of pin access by router. If the value is false, generating report of pin access by `report_cell_pin_access` command.

p

Examples

The following example enables generating report of pin access by router.

```
prompt> set_app_options -as_user_default \\  
          -name pin_check.analyze.analyze_lib_pin_check_routability -value  
          true
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [check_libcell_pin_access](#)

pin_check.analyze.composite_cell_ratio

Specifies the ratio (in integer) of each reference cell in *pin_check.analyze.composite_cells* to be added into the test design for the *composite_cell* mode pin accessibility analysis.

Data Types

list

Default {}

Description

This application option is used for *check_libcell_pin_access -mode composite_cell*, which specifies the ratio (in integer) of each reference cell (specified in *pin_check.analyze.composite_cells*) to be added into the test design.

When cell ratio is provided, the tool checks *pin_check.analyze.composite_core_width* and *pin_check.analyze.composite_core_height* to calculate the instance count by the reference cell's size.

The list size of this option and *pin_check.analyze.composite_cells* must be equal.

This option is mutually exclusive of *pin_check.analyze.composite_inst_count*.

The list value is in integer.

Examples

The following example specifies five cells with "ratio" for composite_cell mode checking.

```
prompt> set_app_options -as_user_default \\  
          -name pin_check.analyze.composite_cells \\  
          -value {A B C D E}
```

p

```
prompt> set_app_options -as_user_default \\
        -name pin_check.analyze.composite_cell_ratio \\
        -value {1 2 1 2 1}
...
prompt> check_libcell_pin_access -mode composite_cell
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [check_libcell_pin_access](#)

pin_check.analyze.composite_cells

Specifies the reference cells to be added into the test design for the *composite_cell* mode pin accessibility analysis.

Data Types

list

Default {}

Description

This application option is used for *check_libcell_pin_access -mode composite_cell*, which specifies the cells in reference libraries to be added into the test design .

This option requires *pin_check.analyze.composite_inst_count* or *pin_check.analyze.composite_cell_ratio* to calculate the instance count of each cell. The list size must be equal in the options.

If both *composite_inst_count* and *composite_cell_ratio* are empty, the tool creates 10 instances for each specified cell by default.

Examples

The following example specifies five cells with "instance count" for *composite_cell* mode checking.

```
prompt> set_app_options -as_user_default \\
        -name pin_check.analyze.composite_cells \\
        -value {A B C D E}
prompt> set_app_options -as_user_default \\
        -name pin_check.analyze.composite_inst_count \\
        -value {10 2 4 40 1000}
```

p

```
...
prompt> check_libcell_pin_access -mode composite_cell
```

The following example specifies five cells with "cell ratio" for composite_cell mode checking.

```
prompt> set_app_options -as_user_default \\  
        -name pin_check.analyze.composite_cells \\  
        -value {A B C D E}  
prompt> set_app_options -as_user_default \\  
        -name pin_check.analyze.composite_cell_ratio \\  
        -value {1 2 1 2 1}  
...  
prompt> check_libcell_pin_access -mode composite_cell
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [check_libcell_pin_access](#)

pin_check.analyze.composite_core_height

Specifies the design height for the *composite_cell* mode pin accessibility analysis when calculate the cell instance count by *pin_check.analyze.composite_cell_ratio*.

Data Types

float

Default 100.0

Description

This application option specifies the design height for *check_libcell_pin_access -mode composite_cell*.

The tool decides the design size by this option only when calculating the cell instance count based on *pin_check.analyze.composite_cell_ratio*.

The distance should be specified in length units as configured by the *set_user_units -type length -input* command, which is micrometers (um) by default.

The default is 100.0.

p

Examples

The following example specifies five cells with "cell ratio" and "core height" for composite_cell mode checking.

```
prompt> set_app_options -as_user_default \\  
        -name pin_check.analyze.composite_cells \\  
        -value {A B C D E}  
prompt> set_app_options -as_user_default \\  
        -name pin_check.analyze.composite_cell_ratio \\  
        -value {1 2 1 2 1}  
prompt> set_app_options -as_user_default \\  
        -name pin_check.analyze.composite_core_height \\  
        -value 50.0  
...  
prompt> check_libcell_pin_access -mode composite_cell
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [check_libcell_pin_access](#)

pin_check.analyze.composite_core_width

Specifies the design width for the *composite_cell* mode pin accessibility analysis when calculate the cell instance count by *pin_check.analyze.composite_cell_ratio*.

Data Types

float

Default 100.0

Description

This application option specifies the design width for *check_libcell_pin_access -mode composite_cell*.

The tool decides the design size by this option only when calculating the cell instance count based on *pin_check.analyze.composite_cell_ratio*.

The distance should be specified in length units as configured by the *set_user_units -type length -input* command, which is micrometers (um) by default.

The default is 100.0.

p

Examples

The following example specifies five cells with "cell ratio" and "core width" for `composite_cell` mode checking.

```

prompt> set_app_options -as_user_default \\  

        -name pin_check.analyze.composite_cells \\  

        -value {A B C D E}  

prompt> set_app_options -as_user_default \\  

        -name pin_check.analyze.composite_cell_ratio \\  

        -value {1 2 1 2 1}  

prompt> set_app_options -as_user_default \\  

        -name pin_check.analyze.composite_core_width \\  

        -value 50.0  

...  

prompt> check_libcell_pin_access -mode composite_cell

```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [check_libcell_pin_access](#)

pin_check.analyze.composite_inst_count

Specifies the instance count of each reference cell in `pin_check.analyze.composite_cells` to be added into the test design for the `composite_cell` mode pin accessibility analysis.

Data Types

list

Default {}

Description

This application option is used for `check_libcell_pin_access -mode composite_cell`, which specifies the instance count of each reference cell (specified in `pin_check.analyze.composite_cells`) to be added into the test design.

The list size of this option and `pin_check.analyze.composite_cells` must be equal.

This option is mutually exclusive of `pin_check.analyze.composite_cell_ratio`.

The list value is in integer.

p

Examples

The following example specifies five cells with "instance count" for `composite_cell` mode checking.

```
prompt> set_app_options -as_user_default \\  
        -name pin_check.analyze.composite_cells \\  
        -value {A B C D E}  
prompt> set_app_options -as_user_default \\  
        -name pin_check.analyze.composite_inst_count \\  
        -value {10 2 4 40 1000}  
...  
prompt> check_libcell_pin_access -mode composite_cell
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [check_libcell_pin_access](#)

pin_check.analyze.composite_pin_dist

Specifies the maximum distance between each output pin to its corresponding input pin to connect the netlist in the *composite_cell* mode pin accessibility analysis.

Data Types

float

Default 0.0

Description

This application option is used for *check_libcell_pin_access -mode composite_cell*, which specifies the maximum distance between each output pin to its corresponding input pins of each net.

This option takes effect only when *pin_check.analyze.composite_fanout_number* is larger than 0.

The distance should be specified in length units as configured by the *set_user_units -type length -input* command, which is micrometers (um) by default.

The default is 0.0.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [check_libcell_pin_access](#)

pin_check.analyze.report_cell_pin_access_even_odd_place

Specifies whether to run the *report_cell_pin_access* command on cells at even site and odd site.

Data Types

boolean

Default false

Description

This application option specifies whether to run the *report_cell_pin_access* command on cells at even site and odd site. The default is false.

Examples

The following example enables *report_cell_pin_access* on cells at even site and odd site.

```
prompt> set_app_options -as_user_default \\  
         -name pin_check.analyze.report_cell_pin_access_even_odd_place \\  
         -value true  
prompt> check_libcell_pin_access -mode analyze_lib_pin
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [check_libcell_pin_access](#)

pin_check.analyze.report_cell_pin_access_exclude_lib_pins

Specifies the pins to be excluded in *report_cell_pin_access*.

p

Data Types

list

Default {}**Description**

This application option specifies the pins to be excluded when running *report_cell_pin_access* command.

Examples

The following example excludes the pins in *report_cell_pin_access*.

```
prompt> set_app_options -as_user_default \\  
        -name pin_check.analyze.report_cell_pin_access_exclude_lib_pins  
        \\  
        -value {cell11/pinA cell11/pinB ...}  
prompt> check_libcell_pin_access -mode analyze_lib_pin
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [check_libcell_pin_access](#)

pin_check.analyze.report_cell_pin_access_max_pin_dist

Specifies the distance and only reports pins sharing common tracks within the distance.

Data Types

float

Default 1.0**Description**

This application option specifies the max distance for risky pins which share common tracks within the distance.

The distance should be specified in length units as configured by the *set_user_units -type length -input* command, which is micrometers (um) by default.

Examples

The following example specifies the max distance for risky pins to be reported.

p

```
prompt> set_app_options -as_user_default \\  
        -name pin_check.analyze.report_cell_pin_access_max_pin_dist \\  
        -value 2.0  
prompt> check_libcell_pin_access -mode analyze_lib_pin
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [check_libcell_pin_access](#)

pin_check.analyze.report_cell_pin_access_mode

Specifies the extension mode for the lowest pin layer in *place.legalize.advanced_layer_access_check*, which is referenced by the *report_cell_pin_access* command when running *check_libcell_pin_access -mode analyze_lib_pin*.

Data Types

enumerated

Default "both"

Description

This application option specifies the extension mode for the lowest pin layer in *place.legalize.advanced_layer_access_check*. Possible values are *pin_extend0*, *pin_extend1* and *both*. *both* is the default.

This option is reflected in *check_libcell_pin_access -mode analyze_lib_pin*, which calls the *report_cell_pin_access* command and generates a report file for the risky pins called *risky_pin_access_[pin_extMode].rpt*.

Examples

The following example specifies the pin extension mode for *report_cell_pin_access*.

```
prompt> set_app_options -as_user_default \\  
        -name pin_check.analyze.report_cell_pin_access_mode \\  
        -value pin_extend1  
prompt> check_libcell_pin_access -mode analyze_lib_pin
```

p

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [check_libcell_pin_access](#)

pin_check.analyze.report_cell_pin_access_pin_extension_by_layer_name

Setting pin extend upper metal pitch with respect to the given routing layer.

Data Types

string

Default ""

Description

This application option setting pin extend upper metal pitch with respect to the given routing layer.

If not setting the value of routing layer, default extend 2 upper metal pitch.

This option is used with *pin_check.analyze.report_cell_pin_access_mode*.

Examples

The following example setting pin extend upper metal pitch of routing layer M0, M1, M2 and M3 to 1.0.

```
prompt> set_app_options -as_user_default \\  
           -name pin_check.analyze.report_cell_pin_access_mode -value  
           pin_extend1  
prompt> set_app_options -as_user_default \\  
           -name  
           pin_check.analyze.report_cell_pin_access_pin_extension_by_layer_name  
           -value {{M0 1.0} {M1 1.0} {M2 1.0} {M3 1.0}}
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)

- [report_app_options](#)
- [check_libcell_pin_access](#)

pin_check.common.check_routability

Specifies whether to run check_routability in the pin accessibility checking flow.

Data Types

boolean

Default false

Description

This application option specifies whether to run check_routability in the pin accessibility checking flow. The default is false.

Examples

The following example runs check_routability in the pin accessibility checking flow.

```
prompt> set_app_options -as_user_default \\  
          -name pin_check.common.check_routability -value true  
prompt> check_libcell_pin_access -mode cell_by_cell
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [check_libcell_pin_access](#)

pin_check.common.congested_netlist

Specifies whether to create crossover connections and make test cells more congested.

Data Types

boolean

Default false

Description

This application option specifies whether to create crossover connections and make test cells more congested.

p

Examples

The following example enables the congested connections.

```
prompt> set_app_options -as_user_default \\  
        -name pin_check.common.congested_netlist -value true
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [check_libcell_pin_access](#)

pin_check.common.grd

Specifies whether to enable parallel runs on GRD.

Data Types

boolean

Default false

Description

This application option specifies whether to enable parallel runs on GRD. If the value is true, the parallel runs are on different machines. If the value is false, the parallel runs are on the same machine.

This option is used with *pin_check.common.num_cells_per_run* or *pin_check.common.num_jobs*.

Examples

The following example enables the parallel runs on GRD.

```
prompt> set_app_options -as_user_default \\  
        -name pin_check.common.num_jobs -value 5  
prompt> set_app_options -as_user_default \\  
        -name pin_check.common.grd -value true  
rompt> check_libcell_pin_access -mode cell_by_cell
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)

p

- [report_app_options](#)
- [check_libcell_pin_access](#)

pin_check.common.grd_command

Specifies the GRD command to submit jobs for parallel runs.

Data Types

string

Default ""

Description

This application option specifies the command for parallel runs on GRD when *pin_check.common.grd* is enabled. The default is empty.

If this option is not specified, use "qsub -cwd -l mem_free=8G,qsc=currentQSC" to submit jobs.

Please specify a proper command according to system environment.

Examples

The following example specifies the GRD command for parallel runs.

```
prompt> set_app_options -as_user_default \\  
-name pin_check.common.num_jobs -value 5  
prompt> set_app_options -as_user_default \\  
-name pin_check.common.grd -value true  
prompt> set_app_options -as_user_default \\  
-name pin_check.common.grd_command \\  
-value "qsub -cwd -l mem_free=64G,qsc=s"  
rompt> check_libcell_pin_access -mode cell_by_cell
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [check_libcell_pin_access](#)

pin_check.common.max_cores

Specifies the number of threads to be used for routing.

p

Data Types

positive integer

Default 8**Description**

This application option specifies the number of threads to be used for routing by `set_host_options -max_cores`. The default is 8.

Examples

The following example sets `max_cores` to be 16.

```
prompt> set_app_options -as_user_default \\  
        -name pin_check.common.max_cores -value 16
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [check_libcell_pin_access](#)

pin_check.common.num_cells_per_run

Specifies the number of cells to divide pin accessibility checking into multiple jobs.

Data Types

positive integer

Default 0**Description**

This application option specifies the number of cells to divide pin accessibility checking into multiple jobs. The default value 0 indicates no multiple job.

If the number of target cells is less than the option value, it remains single job in pin accessibility checking.

This option is mutually exclusive of `pin_check.common.num_jobs`.

This option is used with `pin_check.common.grd`. When the GRD option is true, `check_libcell_pin_access` runs multiple jobs on different machines; otherwise the command runs multiple jobs on the same machine.

p

This option affects placing and routing in the *check_libcell_pin_access* command, but there is no effect for preparing only or reporting only. This option has no effect in analyzing mode either (*-mode analyze_lib_cell* | *analyze_lib_pin*).

Examples

The following example divides the pin accessibility checking by 10 cells. If there are 45 target cells, the checking command invokes 5 jobs for P&R. If there are only 5 target cells, the checking command remains one single job.

```
prompt> set_app_options -as_user_default \\  
        -name pin_check.common.num_cells_per_run -value 10  
prompt> check_libcell_pin_access -mode cell_by_cell
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [check_libcell_pin_access](#)

pin_check.common.num_jobs

Specifies the number of jobs to divide the pin accessibility checking.

Data Types

positive integer

Default 1

Description

This application option specifies the number of jobs to divide pin accessibility checking. The default value 1 indicates single job.

This option is mutually exclusive of *pin_check.common.num_cells_per_run*.

This option is used with *pin_check.common.grd*. When the GRD option is true, *check_libcell_pin_access* runs multiple jobs on different machines; otherwise the command runs multiple jobs on the same machine.

This option affects placing and routing in the *check_libcell_pin_access* command, but there is no effect for preparing only or reporting only. This option has no effect in analyzing mode either (*-mode analyze_lib_cell* | *analyze_lib_pin*).

p

Examples

The following example divides the pin accessibility checking into 10 jobs.

```
prompt> set_app_options -as_user_default \\  
        -name pin_check.common.num_jobs -value 10  
prompt> check_libcell_pin_access -mode cell_by_cell
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [check_libcell_pin_access](#)

pin_check.common.output_verilog

Specifies whether to output the dummy verilog file for the test block.

Data Types

boolean

Default false

Description

This application option specifies whether to output the dummy verilog file cell_[refCellName].v which is used for creating test block cell_[refCellName].design.

Examples

The following example outputs the verilog file for each test block.

```
prompt> set_app_options -as_user_default \\  
        -name pin_check.common.output_verilog -value true  
prompt> check_libcell_pin_access -mode cell_by_cell
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [check_libcell_pin_access](#)

pin_check.common.parallel_run_setup_file

Specifies the Tcl setup file for the parallel run environment.

Data Types

string

Default ""

Description

This application option specifies the Tcl setup file for the parallel run environment. The option is referenced when *pin_check.common.num_cells_per_run* or *pin_check.common.num_jobs* is effective.

Examples

The following example specifies a Tcl setup file, which sourcing the Synopsys default cshrc file.

```
[parallel_setup.tcl]
# 1: Define aliases for environment management.
# 2: Define and load Modules environment for /global/apps.
# 3: Define additional features.
source /global/etc/csh.cshrc

prompt> set_app_options -as_user_default \\  
         -name pin_check.common.parallel_run_setup_file \\  
         -value parallel_setup.tcl
prompt> check_libcell_pin_access -mode cell_by_cell
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [check_libcell_pin_access](#)

pin_check.common.pg_output_tcl

Specifies a Tcl file to output PG routing script when PG routing is enabled.

Data Types

string

Default "create_pg_route_pin_access_check.tcl"

p

Description

This application option specifies a Tcl file to output PG routing script when PG routing is enabled by *pin_check.common.route_pg*.

By default the output Tcl file name is "create_pg_route_pin_access_check.tcl".

Examples

The following example specifies the PG script output file.

```
prompt> set_app_options -as_user_default \\  
        -name pin_check.common.pg_output_tcl -value pg_output.tcl  
prompt> check_libcell_pin_access -mode cell_by_cell
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [check_libcell_pin_access](#)

pin_check.common.pg_script_file

Specifies the Tcl file which to be sourced before or after placement.

Data Types

string

Default ""

Description

This application option specifies the Tcl file which to be sourced before or after placement.

When this PG script file exists, the tool looks up the setting in *pin_check.common.route_pg*. If *pin_check.common.route_pg* is *before_placement*, sources this PG script file before placement; otherwise sources the file after placement.

Examples

The following example sources pg_script.tcl after placement by default.

```
prompt> set_app_options -as_user_default \\  
        -name pin_check.common.pg_script_file -value pg_script.tcl  
prompt> check_libcell_pin_access -mode cell_by_cell
```

The following example sources pg_script.tcl before placement.

p

```
prompt> set_app_options -as_user_default \\  
        -name pin_check.common.pg_script_file -value pg_script.tcl  
prompt> set_app_options -as_user_default \\  
        -name pin_check.common.route_pg -value before_placement  
prompt> check_libcell_pin_access -mode cell_by_cell
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [check_libcell_pin_access](#)

pin_check.common.random_netlist

Specifies whether to create random connections.

Data Types

boolean

Default false

Description

This application option specifies whether to create random connections.

Examples

The following example enables the random connections.

```
prompt> set_app_options -as_user_default \\  
        -name pin_check.common.random_netlist -value true
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [check_libcell_pin_access](#)

pin_check.common.route_pg

Specifies whether to create horizontal and vertical PG straps for the pin accessibility checking.

Data Types

enumerated

Default "none"

Description

This application option specifies whether to create PG straps for the pin accessibility checking. Possible values are *none*, *before_placement* and *after_placement*. *none* is the default.

When the option is *none*, no create PG straps. When the option is *before_placement*, creates PG straps before placement. When the option is *after_placement*, creates PG straps after placement.

Examples

The following example enables PG routing before placement.

```
prompt> set_app_options -as_user_default \\  
         -name pin_check.common.route_pg -value before_placement
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [check_libcell_pin_access](#)

pin_check.common.seed

Specifies the random seed number to randomize pin connections in netlist.

Data Types

positive integer

Default 0

p

Description

This application option specifies the random seed number to randomize pin connections in netlist.

Examples

The following example changes the random seed number.

```
prompt> set_app_options -as_user_default -name pin_check.common.seed  
-value 8
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [check_libcell_pin_access](#)

pin_check.common.tech_node

Specifies the technology node name for mega switch.

Data Types

string

Default ""

Description

This application option specifies the technology node name for mega switch.

Examples

The following example sets N3 for the pin accessibility checking.

```
prompt> set_app_options -as_user_default -name pin_check.common.tech_node  
-value 3
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [check_libcell_pin_access](#)

pin_check.input.cells

Specifies the library cells which are performed the pin accessibility checking.

Data Types

list

Default {all}

Description

This application option specifies the target cells in reference libraries for the pin accessibility checking.

The option accepts "all" to check all library cells pin accessibility, which is the default.

Please use *pin_check.input.exclude_cells* to remove the unwanted cells.

Examples

The following example selects specific cells for the pin accessibility checking.

```
prompt> set_app_options -as_user_default \\  
        -name pin_check.input.cells -value {A B}
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [check_libcell_pin_access](#)

pin_check.input.checking_multiplier

Specifies the multiple number of the cell instances to check against the target library cell. This option is only used in congested_random_placement mode.

Data Types

positive integer

Default 1

Description

This application option specifies the multiple number of the cell instances to check against the target library cell. This option is only used in `congested_random_placement` mode. The default is 1.

```
+---+---+---+ +---+---+---+
| B | A | B | | B | A | B | ...
+---+---+---+ +---+---+---+
```

Examples

The following example checks against specific cells with multiplier 2.

```
+---+---+---+ +---+---+---+
| B | A | B | | B | A | B | ...
+---+---+---+ +---+---+---+
| B |   | B | | B |   | B |
+---+ +---+ +---+ +---+
```

```
prompt> set_app_options -as_user_default \\  
-name pin_check.input.checking_multiplier -value 2  
prompt> chec_libcell_pin_access -mode congested_random_placement
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [check_libcell_pin_access](#)

pin_check.input.exclude_cells

Specifies the library cells which are excluded from the pin accessibility checking.

Data Types

list

Default {}

Description

This application option specifies the excluded cells for the pin accessibility checking.

The excluded cells are removed from the cells of the `pin_check.input.cells` application option.

p

Examples

The following example removes specific cells for the pin accessibility checking.

```
prompt> set_app_options -as_user_default \  
        -name pin_check.input.exclude_cells -value {A B}
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [check_libcell_pin_access](#)

pin_check.input.exclude_for_checking

Specifies the library cells to be excluded from the against cells for pin accessibility checking.

Data Types

list

Default {}

Description

This application option specifies the excluded cells from the against cells for the pin accessibility checking.

The excluded cells are removed from the cells of the *pin_check.input.include_for_checking* application option.

Examples

The following example checks against all library cells except cell {A B} for pin accessibility checking.

```
prompt> set_app_options -as_user_default \  
        -name pin_check.include_for_checking.cells -value {all}  
prompt> set_app_options -as_user_default \  
        -name pin_check.exclude_for_checking.cells -value {A B}
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)

- [report_app_options](#)
- [check_libcell_pin_access](#)

pin_check.input.filter_cells_by_pin_count

Specifies the number of pin to skip creating the test design for the library cell with pin count less than the threshold value.

Data Types

positive integer

Default 1

Description

This application option specifies the threshold value of pin to filter the target cells and against cells for pin accessibility checking.

By default, the tool checks the library cells at least with 1 signal pin. The library cells without pins are ignored in the pin accessibility checking.

User can specify 0 to include non-pin cells for checking, except *check_libcell_pin_access -mode single_cell*.

Examples

The following example checks library cells at least with 2 pins.

```
prompt> set_app_options -as_user_default \  
         -name pin_check.filter_cells_by_pin_count -value 2  
prompt> check_libcell_pin_access -mode single_cell
```

The following example checks library cells including cells without pins.

```
prompt> set_app_options -as_user_default \  
         -name pin_check.filter_cells_by_pin_count -value 0  
prompt> check_libcell_pin_access -mode design_under_test
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [check_libcell_pin_access](#)

pin_check.input.include_for_checking

Specifies the against library cells to about the target cell for pin accessibility checking.

Data Types

list

Default {}

Description

This application option specifies the agaist cells in reference libraries for the pin accessability checking.

The option accepts "all" to check against all library cells for the target cell. The target cell is set in the *pin_check.input.cells* application option.

The option accepts "self_only" to check against the target cell itself.

Examples

The following example checks against specific cells for all library cells.

```
prompt> set_app_options -as_user_default \\  
-name pin_check.input.cells -value {all} # the default  
prompt> set_app_options -as_user_default \\  
-name pin_check.input.include_for_checking -value {A B}
```

The following example checks against all library cells for {A B}.

```
prompt> set_app_options -as_user_default \\  
-name pin_check.input.cells -value {A B}  
prompt> set_app_options -as_user_default \\  
-name pin_check.input.include_for_checking -value {all}
```

The following example checks against the target cell itself.

```
prompt> set_app_options -as_user_default \\  
-name pin_check.input.cells -value {A B}  
prompt> set_app_options -as_user_default \\  
-name pin_check.input.include_for_checking -value {self_only}
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [check_libcell_pin_access](#)

pin_check.place.allow_empty_rows

Specifies whether to allow empty rows in cell placement.

Data Types

boolean

Default true

Description

This application option specifies whether to allow empty rows in cell placement. The default is false.

When the option is true, leaves one empty for every placed row. This option only supports single height cell.

Disable this option when needing to place cells in consecutive rows.

Examples

The following example disables empty rows.

```
prompt> set_app_options -as_user_default \\  
          -name pin_check.place.allow_empty_rows -value false  
rompt> check_libcell_pin_access -mode cell_by_cell
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [check_libcell_pin_access](#)

pin_check.place.auto_reuse

Specifies whether to use auto_reuse flow.

Data Types

boolean

Default false

Description

This application option enables auto_reuse flow. The default is false.

p

When *place.legalize.auto_reuse* is true, the tool will create a tmp block as the reuse design and use reuse flow. The new flow can reduce the run time to create new block for each cell by using reuse design. The tmp block will be created according to the largest cell in target cell list.

Examples

The following example enables the auto_reuse flow for cell_by_cell mode.

```
prompt> set_app_options -as_user_default \\  
        -name pin_check.place.auto_reuse -value true  
rompt> check_libcell_pin_access -mode cell_by_cell
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [check_libcell_pin_access](#)

pin_check.place.cell_spacing_rule_file

Specifies the Tcl constraint file to be honored during cell placement.

Data Types

string

Default ""

Description

This application option specifies the Tcl constraint file that contains cell spacing rules to be honored during cell placement.

The constraint file is generated in the *report_routing* step of *check_libcell_pin_access* when *pin_check.report.derive_cell_spacing_constraints* is specified.

In the *place* step, the constraint file is sourced before placement. Then it calls *report_placement_spacing_rules* to get the spacing needed for certain cells. When placing cells, keeps spacing for those cells to avoid violation.

Examples

The following example specifies a spacing rule file to be honored during cell placement.

```
prompt> set_app_options -as_user_default \\  
        -name pin_check.place.cell_spacing_rule_file \\  
        -value <rule_file>
```

p

```
-value spacing_rule.tcl
prompt> check_libcell_pin_access -mode cell_by_cell -from place
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [check_libcell_pin_access](#)

pin_check.place.check_legality

Specifies whether to enable legality checking after placement.

Data Types

boolean

Default true

Description

This application option specifies whether to perform *check_legality* after placement. The default is true.

Examples

The following example disables *check_legality* feature.

```
prompt> set_app_options -as_user_default \\  
          -name pin_check.place.check_legality -value false  
rompt> check_libcell_pin_access -mode cell_by_cell
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [check_libcell_pin_access](#)

pin_check.place.core_margin

Specifies the horizontal spacing between design boundary and core area.

p

Data Types

float

Default 2.0**Description**

This application option specifies the horizontal spacing between design boundary and core area. The default is 2.0.

The value is used in *initialize_floorplan -core_offset*.

The distance should be specified in length units as configured by the *set_user_units -type length -input* command, which is micrometers (um) by default.

Examples

The following example specifies the horizontal spacing between design boundary and core area.

```
prompt> set_app_options -as_user_default \\  
         -name pin_check.place.core_margin -value 5.0  
prompt> check_libcell_pin_access -mode cell_by_cell
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [check_libcell_pin_access](#)

pin_check.place.design_under_test_multiplier

Specifies the multiple number for target cells in design_under_test mode.

Data Types

positive integer

Default 1**Description**

This application option specifies the multiple number for target cells in design_under_test mode. The default is 1.

```
+---+---+---+  
  | B | B | B |
```


p

```

+---+---+---+
| B | A | B | ...
+---+---+---+
| B | B | B |
+---+---+---+

```

Examples

The following example multiplies 2 of target cell in design_under_test mode.

```

+---+---+---+---+
| B | B | B | B |
+---+---+---+---+
| B | A | A | B | ...
+---+---+---+---+
| B | B | B | B |
+---+---+---+---+

```

```

prompt> set_app_options -as_user_default \\  

        -name pin_check.place.design_under_test_multiplier -value 2  

prompt> check_libcell_pin_access -mode design_under_test

```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [check_libcell_pin_access](#)

pin_check.place.fill_empty_rows_with_blockages

Specifies the metal layers to be filled with zero spacing routing blockages in the empty rows.

Data Types

list

Default {}

Description

This application option specifies a list of metal layers where to create zero spacing routing blockages in the empty rows.

Examples

The following example specifies the layers needed to fill blockages in the empty rows.

p

```
prompt> set_app_options -as_user_default \\  
        -name pin_check.place.fill_empty_rows_with_blockages -value {M1  
        M2}  
rompt> check_libcell_pin_access -mode cell_by_cell
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [check_libcell_pin_access](#)

pin_check.place.fix_variant_cell_placement

Specifies whether to only place variant cell at legal location.

Data Types

boolean

Default false

Description

This application option disables variant-aware legalization for illegal cells. The default is false.

By default, when *place.legalize.enable_variant_aware* is true, the legalizer may replace a cell that is placed in an illegal location with a variant cell.

User can stop the legalizer changing cell with a different cell master by setting this option as true.

Examples

The following example disables the variant-aware legalization for cells in illegal location.

```
prompt> set_app_options -as_user_default \\  
        -name pin_check.place.fix_variant_cell_placement -value true  
rompt> check_libcell_pin_access -mode cell_by_cell
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)

- [report_app_options](#)
- [check_libcell_pin_access](#)

pin_check.place.flip_first_row

Specifies whether to flip first site row.

Data Types

boolean

Default false

Description

This application option specifies whether to flip first site row in floorplan. The default is false.

This option controls *initialize_floorplan -flip_first_row*.

Examples

The following example flips first site row in floorplan.

```
prompt> set_app_options -as_user_default \\  
          -name pin_check.place.flip_first_row -value true  
rompt> check_libcell_pin_access -mode cell_by_cell
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [check_libcell_pin_access](#)

pin_check.place.group_spacing_by_site_width

Specifies the spacing (in multiple of tile width) between each cell group.

Data Types

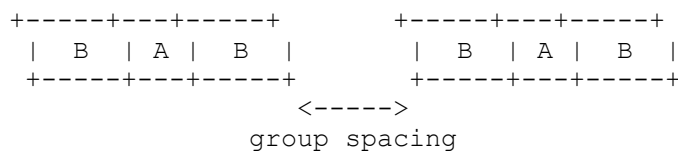
positive integer

Default 10

p

Description

This application option specifies the spacing (in multiple of tile width) between each cell group. The default is 10.



The actual spacing will be the option value multiple tile width.

Examples

The following example specifies 5 multiple tile width as the group spacing.

```

prompt> set_app_options -as_user_default \\  

        -name pin_check.place.group_spacing_by_site_width -value 5  

prompt> check_libcell_pin_access -mode cell_by_cell
  
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [check_libcell_pin_access](#)

pin_check.place.init_utilization

Specifies the initial design utilization ratio.

Data Types

float

Default 0.25

Description

This application option specifies the initial design utilization ratio. The value is used in *initialize_floorplan -core_utilization*. The default is 0.25.

Examples

The following example specifies the initial design utilization ratio.

p

```
prompt> set_app_options -as_user_default \  
        -name pin_check.place.init_utilization -value 0.5  
rompt> check_libcell_pin_access -mode cell_by_cell
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [check_libcell_pin_access](#)

pin_check.place.left_offset_by_site_width

Specifies the offset distance (in multiple of tile width) of the leftmost cell to core.

Data Types

positive integer

Default 6

Description

This application option specifies the offset distance (in multiple of tile width) of the leftmost cell to core. The default is 6.

The actual spacing will be the offset number multiple tile width.

Examples

The following example specifies 10 multiple tile width as the left offset.

```
prompt> set_app_options -as_user_default \  
        -name pin_check.place.left_offset_by_site_width -value 10  
prompt> check_libcell_pin_access -mode cell_by_cell
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [check_libcell_pin_access](#)

pin_check.place.legalize_placement

Specifies whether to enable *legalize_placement* in placing step.

Data Types

boolean

Default true

Description

This application option specifies whether to legalize placement within the site row where cells are currently placed. The default is true.

Examples

The following example disables *legalize_placement* feature.

```
prompt> set_app_options -as_user_default \\  
         -name pin_check.place.legalize_placement -value false  
rompt> check_libcell_pin_access -mode cell_by_cell
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [check_libcell_pin_access](#)

pin_check.place.macro_spacing

Specifies the spacing between each macro/pad cell.

Data Types

float

Default 5.0

Description

This application option specifies the spacing between each macro/pad cell. The default is 5.0.

The distance should be specified in length units as configured by the *set_user_units -type length -input* command, which is micrometers (um) by default.

p

```

+---+ +---+ +---+           +---+ +---+ +---+
| A | | A | | A |           | A | | A | | A |
+---+ +---+ +---+           +---+ +---+ +---+
      <->           <----->
      macro         group
      spacing       spacing

```

Examples

The following example specifies 2.0um as the macro spacing.

```

prompt> set_app_options -as_user_default \
        -name pin_check.place.macro_spacing -value 2.0
prompt> check_macro_pin_access -mode horizontal

```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [check_macro_pin_access](#)

pin_check.place.move_core_to_origin

Specifies whether to move core boundary to {0 0}.

Data Types

boolean

Default false

Description

This application option specifies whether to move core boundary to {0 0}. The default is false.

Examples

The following example enables to move core boundary to {0 0}.

```

prompt> set_app_options -as_user_default \
        -name pin_check.place.move_core_to_origin -value true
prompt> check_libcell_pin_access -mode cell_by_cell

```

p

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [check_libcell_pin_access](#)

pin_check.place.preplace_option_file

Specifies the Tcl setup file needed for placement.

Data Types

string

Default ""

Description

This application option specifies the Tcl file to be sourced before placement. The file must at least define the routing deriction for each metal layer. This option is required for the place step in *check_libcell_pin_access*.

Examples

The following example specifies a Tcl file required by placement.

```
[preplace_options.tcl]
set_attribute -objects [get_layers M1] -name routing_direction -value
    vertical -quiet
set_attribute -objects [get_layers M2] -name routing_direction -value
    horizontal -quiet
...
```

```
prompt> set_app_options -as_user_default \\  
         -name pin_check.place.preplace_option_file \\  
         -value preplace_options.tcl  
prompt> check_libcell_pin_access -mode cell_by_cell
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [check_libcell_pin_access](#)

pin_check.place.remove_duplicate_cell_pairs

Specifies whether to remove repeating cell pairs in the test block.

Data Types

boolean

Default false

Description

This application option specifies whether to remove repeating cell pairs in the test block cell_[refCellName].design. The default is false.

Examples

The following example enables to remove repeating cell pairs.

```
prompt> set_app_options -as_user_default \\  
          -name pin_check.place.remove_duplication_cell_pairs -value true  
prompt> check_libcell_pin_access -mode cell_by_cell
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [check_libcell_pin_access](#)

pin_check.place.reuse_design

Specifies an existing design to reuse the same floorplan, PG mesh and app options in pin accessibility checking.

Data Types

string

Default ""

Description

Specifies an existing design in the format of path/libName:blockName.

The tool will reuse the PG mesh, *.legalize.* options and route.* options of the design in pin accessibility checking, instead of relying on the PG scripts, preplace option settings or route option settings from user.

p

The default is "".

Examples

The following example specified test.nlib:top.design for reusing.

```
prompt> set_app_options -as_user_default \\  
        -name pin_check.place.reuse_design  
        -value ./tmp/test.nlib:top.design  
prompt> check_libcell_pin_access -mode cell_by_cell
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [check_libcell_pin_access](#)

pin_check.place.right_offset_by_site_width

Specifies the offset distance (in multiple of tile width) of the rightmost cell to core.

Data Types

positive integer

Default 6

Description

This application option specifies the offset distance (in multiple of tile width) of the rightmost cell to core. The default is 6.

The actual spacing will be the offset number multiple tile width.

Examples

The following example specifies 10 multiple tile width as the right offset.

```
prompt> set_app_options -as_user_default \\  
        -name pin_check.place.right_offset_by_site_width -value 10  
prompt> check_libcell_pin_access -mode cell_by_cell
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)

- [report_app_options](#)
- [check_libcell_pin_access](#)

pin_check.place.row_pattern

Specifies the row pattern name for creating site rows.

Data Types

string

Default ""

Description

This application option specifies the row pattern name for creating site rows. When using this option, the library technology file must be PRF format, which contains "STAR_PHYSICAL_RULE_SECTION".

This option is required for checking libraries with different height site rows. Generally the N3 library contains hybrid site rows.

Examples

The following example specifies the row pattern name.

```
prompt> set_app_options -as_user_default \\  
          -name pin_check.place.row_pattern -value H169_H221_H221_H169  
prompt> check_libcell_pin_access -mode cell_by_cell
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [check_libcell_pin_access](#)

pin_check.place.use_site_row

Specifies whether to create floorplan with site row.

Data Types

boolean

Default false

p

Description

This application option specifies whether to create floorplan with site rows. The default is false.

This option controls *initialize_floorplan -use_site_row*.

Examples

The following example creates site row in floorplan.

```
prompt> set_app_options -as_user_default \\  
        -name pin_check.place.use_site_row -value true  
prompt> check_libcell_pin_access -mode cell_by_cell
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [check_libcell_pin_access](#)

pin_check.place.vertical_core_margin

Specifies the spacing between top and bottom design boundary to core area.

Data Types

float

Default 2.0

Description

This application option specifies the spacing between top and bottom design boundary to core area. The default is 2.0.

The value is used in *initialize_floorplan -core_offset*.

The distance should be specified in length units as configured by the *set_user_units -type length -input* command, which is micrometers (um) by default.

Examples

The following example specifies the vertical spacing for core area.

```
prompt> set_app_options -as_user_default \\  
        -name pin_check.place.vertical_core_margin -value 5.0  
prompt> check_libcell_pin_access -mode cell_by_cell
```

p

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [check_libcell_pin_access](#)

pin_check.report.derive_cell_spacing_constraints

Specifies the Tcl file to output the cell spacing constraints.

Data Types

string

Default ""

Description

This application option specifies the Tcl file to output cell spacing constraints. The cell spacing constraints are generated in the step of reporting routing DRC in *check_libcell_pin_access*.

It queries the end of line violations on metal2 and via1 layers, and find left/right cells which overlap the violation area. The spacing labels for the neighbor cells are output in the Tcl constraint file.

```
set_placement_spacing_label -name leftLabel -side side1 -lib_cells
leftCells
set_placement_spacing_label -name rightLabel -side side2 -lib_cells
rightCells
set_placement_spacing_rule -labels {leftLabel rightLabel} {0 0}
...
```

The constraint file can be read by *pin_check.place.cell_spacing_rule_file* to update the cell placement.

Examples

The following example specifies the Tcl file to output the cell spacing constraints.

```
prompt> set_app_options -as_user_default \\  
-name pin_check.report.derive_cell_spacing_constraints \\  
-value spacing_rule.tcl  
prompt> check_libcell_pin_access -mode cell_by_cell -to report_route
```

User can source the constraint file later and honor the spacing rule in cell placement.

p

```
prompt> set_app_options -as_user_default \\  
        -name pin_check.place.cell_spacing_rule_file \\  
        -value spacing_rule.tcl  
prompt> check_libcell_pin_access -mode cell_by_cell -from place
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [check_libcell_pin_access](#)

pin_check.report.icv_home_dir

Specifies the home directory of the ICV executable.

Data Types

string

Default ""

Description

This application option specifies the home directory of the ICV executable. If the option is empty, gets the value from env ICV_HOME_DIR. If the option is not empty, replaces env ICV_HOME_DIR by the value.

The tool adds the ICV executable path into env PATH, such as \$ICV_HOME_DIR/bin/LINUX.64. The ICV executable path is required for running *signoff_check_drc* when *pin_check.report.signoff_check_drc_stages* is not empty.

Examples

The following example specifies the home directory of the ICV executable.

```
prompt> set_app_options -as_user_default \\  
        -name pin_check.report.icv_home_dir -value  
        "/remote/code/Hercules/dev/image_root"  
prompt> set_app_options -as_user_default \\  
        -name pin_check.report.signoff_check_drc_stages -value {placement  
        signal_routing}  
prompt> check_libcell_pin_access -mode cell_by_cell
```

p

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [check_libcell_pin_access](#)

pin_check.report.signoff_check_drc_options

Specifies the command line options to be used in the ICV in-design *signoff_check_drc*.

Data Types

string

Default ""

Description

This application option specifies the command line options to be used in the ICV in-design *signoff_check_drc*.

Examples

The following example specifies the command line options for *signoff_check_drc*.

```
prompt> set_app_options -as_user_default \\
        -name pin_check.report.icv_home_dir -value
        "/remote/code/Hercules/dev/image_root"
prompt> set_app_options -as_user_default \\
        -name pin_check.report.signoff_options_file -value
        signoff_options.tcl
prompt> set_app_options -as_user_default \\
        -name pin_check.report.signoff_check_drc_stages -value
        {placement}
prompt> set_app_options -as_user_default \\
        -name pin_check.report.signoff_check_drc_options -value
        "-select_layers { M1 M3 }"
prompt> check_libcell_pin_access -mode cell_by_cell
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [check_libcell_pin_access](#)

pin_check.report.signoff_check_drc_stages

Specifies the running stages for the ICV in-design *signoff_check_drc*.

Data Types

list

Default {}

Description

This application option specifies the running stages for the ICV in-design *signoff_check_drc*. The running stage can be *placement*, *pg_routing* and *signal_routing*. The option accepts a list of stages.

Examples

The following example specifies the running stages for *signoff_check_drc*.

```
prompt> set_app_options -as_user_default \\  
        -name pin_check.report.icv_home_dir -value  
        "/remote/code/Hercules/dev/image_root"  
prompt> set_app_options -as_user_default \\  
        -name pin_check.report.signoff_options_file -value  
        signoff_options.tcl  
prompt> set_app_options -as_user_default \\  
        -name pin_check.report.signoff_check_drc_stages -value {placement  
        signal_routing}  
prompt> check_libcell_pin_access -mode cell_by_cell
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [check_libcell_pin_access](#)

pin_check.report.signoff_options_file

Specifies the Tcl file to be sourced for ICV in-design *signoff_check_drc*.

Data Types

string

Default ""

p

Description

This application option specifies the Tcl file to be sourced for ICV in-design *signoff_check_drc*.

Must at least specify DRC runset in the Tcl file. Can also specify layer map file and application options for *signoff_check_drc*.

Examples

The following example specifies the Tcl file for DRC runset.

```
prompt> set_app_options -as_user_default \\  
          -name pin_check.report.icv_home_dir -value  
          "/remote/code/Hercules/dev/image_root"  
prompt> set_app_options -as_user_default \\  
          -name pin_check.report.signoff_options_file -value  
          signoff_options.tcl  
prompt> set_app_options -as_user_default \\  
          -name pin_check.report.signoff_check_drc_stages -value  
          {placement}  
prompt> check_libcell_pin_access -mode cell_by_cell
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [check_libcell_pin_access](#)

pin_check.route.allow_metal1_routing

Specifies whether to allow metal1 for routing.

Data Types

boolean

Default true

Description

This application option specifies whether to allow metal for routing. The default is true.

Examples

The following example disables the metal1 routing.

p

```
prompt> set_app_options -as_user_default \\  
        -name pin_check.route.allow_metall_routing -value false  
prompt> check_libcell_pin_access -mode cell_by_cell
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [check_libcell_pin_access](#)

pin_check.route.effort

Specifies the routing effort.

Data Types

enumerated

Default "medium"

Description

This application option specifies the routing effort. Possible values are *low*, *medium* and *high*. *medium* is the default.

If *pin_check.route.max_detail_route_iterations* is 0, this option is used to decide the max routing iterations. For *low* effort, the iteration is 10; otherwise the iteration is 40.

The effort option is used in normal routing flow. When effort is *high*, it performs *check_routes* to find DRC violations after *route_auto*, and runs *route_detail -incremental true* automatically for cells with DRC violations.

Examples

The following example specifies the routing effort.

```
prompt> set_app_options -as_user_default -name pin_check.route.effort  
        -value high  
prompt> check_libcell_pin_access -mode cell_by_cell
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)

- [report_app_options](#)
- [check_libcell_pin_access](#)

pin_check.route.incremental_search_repair

Specifies whether to run incremental detail routing.

Data Types

boolean

Default false

Description

This application option specifies whether to run incremental detail routing. The default is false.

When this option is turned on, it calls *route_detail -incremental true*. User can use *pin_check.route.max_detail_route_iterations* to control the *-max_number_iterations* option in the incremental detail routing.

Examples

The following example enables the incremental routing.

```
prompt> open_lib myTestLib
prompt> set_app_options -as_user_default \\  
          -name pin_check.route.incremental_search_repair -value true
prompt> check_libcell_pin_access -mode cell_by_cell -from route
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [check_libcell_pin_access](#)

pin_check.route.max_detail_route_iterations

Specifies the maximum number of detail routing iterations.

Data Types

positive integer

p

Default 40**Description**

This application option specifies the maximum number of detail routing iterations. The default is 40.

If the user specified value is 0, the `max_detail_route_iterations` value is automatically decided by `pin_check.route.effort`. If the effort is *low*, uses 10 for the maximum iterations; otherwise uses 40.

The option value is used in `route_auto -max_detail_route_iterations` or `route_detail -incremental true -max_number_iterations`.

Examples

The following example specifies the maximum detail routing iterations as 20.

```
prompt> set_app_options -as_user_default \\  
         -name pin_check.route.max_detail_route_iterations -value 20  
prompt> check_libcell_pin_access -mode cell_by_cell
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [check_libcell_pin_access](#)

pin_check.route.post_route_tcl

Specifies the Tcl file to be sourced after routing.

Data Types

string

Default ""**Description**

This application option specifies the Tcl file to be sourced after routing.

Examples

The following example specifies a Tcl file to be sourced after routing.

p

```
prompt> set_app_options -as_user_default \\  
        -name pin_check.route.post_route_tcl -value post_route.tcl  
prompt> check_libcell_pin_access -mode cell_by_cell
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [check_libcell_pin_access](#)

pin_check.route.reroute_drc_cells_iterations

Specifies the number of rerouting iteration for cells with DRC violations.

Data Types

positive integer

Default 0

Description

This application option specifies the number of rerouting iteration for cells with DRC violations. The default is 0.

When the iteration number is larger than 0, the *pin_check.route.reroute_drc_cells_only* or *pin_check.route.incremental_search_repair* application option must be turned on.

Examples

The following example sets the reroute iteration as 5.

```
prompt> open_lib myTestLib  
prompt> set_app_options -as_user_default \\  
        -name pin_check.route.reroute_drc_cells_only -value true  
prompt> set_app_options -as_user_default \\  
        -name pin_check.route.reroute_drc_cells_iterations -value 5  
prompt> check_libcell_pin_access -mode cell_by_cell -from route
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)

p

- [report_app_options](#)
- [check_libcell_pin_access](#)

pin_check.route.reroute_drc_cells_only

Specifies whether only to rip up and reroute violated nets in cells with DRC violations.

Data Types

boolean

Default false

Description

This application option specifies whether only to rip up and reroute violated nets in cells with DRC violations. The default is false.

When this option is turned on, please also specifies *pin_check.route.reroute_drc_cells_iterations* to be at least 1.

Examples

The following example specifies to reroute cells with DRC violations only.

```
prompt> open_lib myTestLib
prompt> set_app_options -as_user_default \\  
         -name pin_check.route.reroute_drc_cells_only -value true
prompt> set_app_options -as_user_default \\  
         -name pin_check.route.reroute_drc_cells_iterations -value 1
prompt> check_libcell_pin_access -mode cell_by_cell -from route
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [check_libcell_pin_access](#)

pin_check.route.route_option_file

Specifies the Tcl file to be sourced before routing.

Data Types

string

p

Default ""**Description**

This application option specifies the Tcl file to be sourced before routing. User can specify the routing related application options in the file.

Examples

The following example specifies a Tcl file for routing setup.

```
prompt> set_app_options -as_user_default \\  
        -name pin_check.route.route_option_file -value route_option.tcl  
prompt> check_libcell_pin_access -mode cell_by_cell
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [check_libcell_pin_access](#)

pin_check.route.signal_net_only

Specifies whether only to route signal nets in *check_macro_pin_access* command.

Data Types

boolean

Default false**Description**

The command *check_macro_pin_access* creates netlist for both signal and PG ports. By default it routes for all pins.

This application option specifies whether only to route signal nets in the macro/pad related cells. The default is false.

Examples

The following example specifies to route signal port_type pins only.

```
prompt> create_pin_check_lib myTestLib -technology a.tf -ref_libs MACRO  
prompt> set_app_options -as_user_default \\  
        -name pin_check.route.signal_net_only -value true  
prompt> check_macro_pin_access -mode vertical
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [check_macro_pin_access](#)

place.coarse.advanced_node_congestion_driven_max_util

Adjust congestion driven max util based on the technology node.

Data Types

Boolean

Default false

Description

When this application option is set to *true*, the coarse placer will adjust congestion driven max util for supported advanced technology nodes. It supports the following technologies: 3 with variable rows, 5 with uniform rows, 7 with siterow height 0.24 or 0.3, and s5.

set_technology can be used to set and list supported technology nodes.

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design. Use *report_app_options* to show the value of application options.

See Also

- [set_technology](#)
- [clock_opt](#)
- [create_placement](#)
- [get_app_option_value](#)
- [place_opt](#)
- [refine_placement](#)
- [report_app_options](#)
- [set_app_options](#)

place.coarse.auto_density_control

Enables automatic density control for the coarse placement engine.

Data Types

Enum

Default enhanced

Description

When active, the coarse placer attempts to use a higher cell density in areas where that does not create routing congestion.

Possible values are "true", "false", or "enhanced".

If application option *place.coarse.max_density* is 0, automatic density control selects an appropriate value for the maximum cell density. Message *PLACE-027* is issued to report the selected settings.

If *place.coarse.auto_density_control* is enhanced, *auto_density_control* further refines the the maximum density object for better overall clumping of the design, while also reducing the likelihood of density hotspots. Overall clumping can reduce design wirelength and power.

Automatic density control requires effort high for congestion analysis. Therefore, if *place.coarse.auto_density_control* is true or enhanced, effort high is used for application option *place.coarse.congestion_analysis_effort* and any user setting for this application option is ignored.

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design. Use *report_app_options* to show the value of application options.

See Also

- [clock_opt](#)
- [create_placement](#)
- [get_app_option_value](#)
- [place_opt](#)
- [refine_placement](#)
- [report_app_options](#)
- [set_app_options](#)

place.coarse.auto_timing_control

Enables automatic timing control for timing-driven coarse placement.

Data Types

Boolean

Default true

Description

When *true*, timing-driven coarse placement attempts to focus timing optimization on the most critical timing paths to find a good balance between reducing the worst slack and reducing the total negative slack.

If automatic timing control is enabled, the *place.coarse.tns_driven_placement* application option has no effect.

Possible values are *true* or *false*.

Application options are set using the *set_app_options* command. They can be specified globally or with respect to a block. Use the *report_app_options* command to show the value of application options.

See Also

- [clock_opt](#)
- [create_placement](#)
- [get_app_option_value](#)
- [place_opt](#)
- [refine_placement](#)
- [report_app_options](#)
- [set_app_options](#)

place.coarse.balance_registers

Enables register balancing during buffering aware timing driven coarse placement.

Data Types

boolean

Default true

p

Description

Enables explicit register balancing in *create_placement -buffering_aware_timing_driven* and in the initial_place step of *place_opt*. For selected registers, this feature will work to adjust the placement such that the input and output slacks are better balanced. If both slacks are better than an automatically determined margin, balancing is not pursued.

The selected registers are registers with a narrow fan-out and fan-in cone. A fan-out/in cone is considered narrow if the register drives/is driven by at most 3 other registers, top level ports or fixed pins and has at most 2 side fan-in/out nets.

See Also

- [create_placement](#)
- [place_opt](#)
- [set_app_options](#)
- [report_app_options](#)

place.coarse.channel_detect_mode

Enables channel detect mode in coarse placement.

Data Types

Boolean

Default true

Description

Enables enhanced channel detect mode in coarse placement. It automatically detects channels and control the logic cell density inside channels.

Possible values are "true" or "false"

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design. Use *report_app_options* to show the value of application options.

See Also

- [create_placement](#)
- [get_app_option_value](#)
- [report_app_options](#)

p

- [set_app_options](#)
- [place_opt](#)
- [refine_placement](#)

place.coarse.clock_gate_latency_aware

Makes the coarse placer aware of the effect of clock latency on clock gate enable timing.

Data Types

String

Default "false"

Description

This option controls whether timing-driven placement will estimate and consider clock latency when evaluating enable timing of clock gates. This option only has effect during timing driven (including buffering-aware timing driven) pre-CTS placement steps. The placer will take into account that moving a clock gate away from its sinks will enforce the clock to arrive earlier at the clock gate, due to increased downstream latency, and thus increase the criticality of the enable timing path. The app option can be set to one of the values "false" or "auto". The value "false" is the default.

With value "auto", the feature will automatically be enabled if timing-driven or buffering-aware placement is performed. With value "false", clock gate latency awareness during coarse placement will be disabled.

When the feature is enabled, estimated clock latencies will be annotated as "set_clock_latency -offset" statements at the end of each pre-CTS timing-driven placement step.

Clock gate input pins which are on clock network and have timing check arcs starting from them to other input pin of the cell will be annotated with estimated latencies. This generally includes cells such as ICGs, clock dividers, discrete clock gates.

Note that clock gate latency aware timing driven pre-CTS placement is incompatible with pre-CTS trial tree construction because trial tree construction forces clock timing to be in propagated mode. Therefore clock gate latency aware placement will not be enabled when app option place_opt.flow.trial_clock_tree or place_opt.flow.optimize_icgs is set. Warning PLACE-080 will be printed in that case.

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design. Use *report_app_options* to show the value of application options.

The value "true" for this app options is obsolete and will be interpreted as "auto".

See Also

- [create_placement](#)
- [place_opt](#)
- [PLACE-080](#)
- [report_app_options](#)
- [set_app_options](#)

place.coarse.cong_restruct

Enables congestion driven restructuring in place_opt.

Data Types

Enum

Default "on"

Description

Enables congestion driven restructuring during place_opt. If the value of *place.coarse.cong_restruct_strategy* is "original", net restructuring is interleaved between iterations of incremental placement. If the value of *place.coarse.cong_restruct_strategy* is "embed", net restructuring is performed inside the wire length driven coarse placer.

Possible values are "on" or "off".

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design. Use *report_app_options* to show the value of application options.

See Also

- [create_placement](#)
- [place_opt](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

place.coarse.cong_restruct_depth_aware

Sets a limit on the amount of relative connection depth difference that CDR can utilize.

p

Data Types

Boolean

Default `false`

Description

Sets a limit on the amount of relative connection depth difference that CDR can utilize. When this app option is set to `true`, any netlist change is guaranteed to not change the depth of the reconnected pin by more than 3, measured from the root of the associative commutative tree, not counting any buffers or inverters. As a consequence, any path traversing the tree will not increase its depth by more than 3. If the path incurs more than one CDR netlist change, the effect will be additive. If this app option is being set to `true`, a reduction in CDR's QoR gain is possible: wire length and congestion may have less improvement. When both `place.coarse.restruct` is set to `on` and `place.coarse.cong_restruct_strategy` is set to `embed`, this app option affects both the `place_opt` and `create_placement` commands. This app option has no effect if the value of the app option `place.coarse.cong_restruct_strategy` is `original` and it has reduced effect (the limit of 3 is not strictly observed) if the value of the app option `place.coarse.cong_restruct_effort` is `ultra`.

Possible values are `true` and `false`. The default value of `false` means that there is no depth change limit set for the netlist changes performed by CDR.

Application options are set using the `set_app_options` command, and they may be specified globally or with respect to a design. Use `report_app_options` to show the value of application options.

See Also

- [create_placement](#)
- [place_opt](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

`place.coarse.cong_restruct_effort`

Sets the effort level for congestion-driven restructuring.

Data Types

Enum

p

Default "medium"

Description

Sets the effort level for congestion-driven restructuring. This affects *place_opt* command when *place.coarse.cong_restruct* is set to "on", as well as the *create_placement* command.

Possible values are "low", "medium", "high" or "ultra".

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design. Use *report_app_options* to show the value of application options.

See Also

- [create_placement](#)
- [place_opt](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

place.coarse.cong_restruct_iterations

Sets the number of iterations of restructuring and *create_placement*.

Data Types

Integer

Default 1

Description

Sets the number of iterations of restructuring and *create_placement*, when the *place.coarse.cong_restruct_original_strategy* app option is set to *false*. For embed strategy it internally uses 1 as default value. This affects *place_opt* command when *place.coarse.cong_restruct* is set to "on", as well as the *create_placement* command.

Possible values are integer values from 1 to 1000.

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design. Use *report_app_options* to show the value of application options.

See Also

- [create_placement](#)
- [place_opt](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

place.coarse.cong_restruct_strategy

Sets the strategy type for congestion-driven restructuring.

Data Types

Enum

Default "embed"

Description

Sets the strategy type for congestion-driven restructuring. The "original" option performs net restructuring to improve congestion without logic gate alteration. The "embed" option performs the same type of restructuring as the "original" option, but embedded inside the coarse placer.

If this value is set to "embed" and the *place_opt* or *create_placement* commands are used, to turn on congestion-driven restructuring, the app_option "place.coarse.cong_restruct" should be programmed to "on".

Possible values of this app option are "original" and "embed".

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design. Use *report_app_options* to show the value of application options.

See Also

- [create_placement](#)
- [place_opt](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

place.coarse.congestion_analysis_effort

Determines the accuracy of the router used by congestion driven placement.

Data Types

Enum

Default "high"

Description

This option is deprecated and scheduled for removal in a future release. It has no effect anymore; the coarse placer always uses effort high.

Possible values are "high", "medium" or "low".

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design. Use *report_app_options* to show the value of application options.

See Also

- [create_placement](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)
- [place_opt](#)
- [refine_placement](#)
- [clock_opt](#)

place.coarse.congestion_driven_max_util

Specifies the maximum design utilization after congestion driven padding.

Data Types

float

Default 0.93

Description

Specifies the maximum design utilization after congestion driven padding during congestion driven placement. Possible values are floating point numbers between 0.0 and

p

1.0. Setting this value to a number following the overall design utilization causes it to be ignored.

Application options are set using the *set_app_options* command, and they might be specified globally or with respect to a design. Use the *report_app_options* command to show the value of application options.

See Also

- [create_placement](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)
- [place_opt](#)
- [refine_placement](#)

place.coarse.congestion_expansion_direction

Specify congestion driven cell expansion direction.

Data Types

String

Default horizontal

Description

It has two modes: {horizontal, both}. In the horizontal (default) mode, cells are all expanded horizontally. In the ``both" mode, the placer automatically expands cells according to routing congestion.

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design. Use *report_app_options* to show the value of application options.

See Also

- [create_placement](#)
- [place_opt](#)
- [clock_opt](#)
- [get_app_option_value](#)

- [report_app_options](#)
- [set_app_options](#)

place.coarse.congestion_layer_aware

Controls whether the tool considers the congestion of each layer separately during coarse placement.

Data Types

Boolean

Default `true`

Description

This *place.coarse.congestion_layer_aware* application option controls whether coarse placement considers the congestion of each layer separately.

When you set this application option to *true*, the tool considers the congestion of each layer separately during coarse placement, which improves the accuracy of congestion reduction.

When this is set to *false*, the tool combines the congestion information of all layers during coarse placement.

See Also

- [clock_opt](#)
- [create_placement](#)
- [get_app_option_value](#)
- [place_opt](#)
- [report_app_options](#)
- [set_app_options](#)

place.coarse.continue_on_missing_scandef

When this option is true scandef checking prior to coarse placement is disabled.

Data Types

Boolean

Default `false`

Description

Disables scandef checking prior to coarse placement.

Possible values are "true" or "false"

Application options are set using the `set_app_options` command, and they may be specified globally or with respect to a design. Use `report_app_options` to show the value of application options.

See Also

- [create_placement](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)
- [place_opt](#)
- [refine_placement](#)

place.coarse.detect_detours

Enables detour aware coarse placement.

Data Types

Boolean

Default true

Description

Detour aware placement makes the coarse placer aware of detouring that needs to happen because otherwise nets cannot be buffered properly. This can improve wirelength and timing after buffering.

Possible values are *true* or *false*.

Application options are set using the `set_app_options` command, and they may be specified globally or with respect to a design. Use `report_app_options` to show the value of application options.

See Also

- [clock_opt](#)
- [create_placement](#)

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)
- [place_opt](#)
- [refine_placement](#)

place.coarse.dft_enable_at_speed_modeling

Enables dft modeling for at-speed scan chains for coarse placement.

Data Types

Boolean

Default false

Description

Dft at-speed modeling makes the placement aware of the scan chains operating at the functional clock speed. Scan connectivities of start/stop cells are not ignored for such chains. This consideration helps in optimal placement of the dft modules along with other logical modules. This can improve wirelength and timing through the at-speed scan chains start/stop scan connections.

Possible values are *true* or *false*.

When set to "true", the modeling automatically identifies at-speed scan chains and depending on the start and stop cell types, connectivity is not ignored.

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design. Use *report_app_options* to show the value of application options.

See Also

- [clock_opt](#)
- [create_placement](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

p

- [place_opt](#)
- [refine_placement](#)

place.coarse.enable_advanced_mv_cell_placement

Enable advance feature for isolation and level shifter placement.

Data Types

Boolean

Default "true"

Description

This option control whether placer will be using advance multi voltage cell placement fetature. Under this fetaure placer try to optimize usage of dual rail buffer later in the flow, by minimize the length of net segment which required dual rail buffer. This feature also minimize the net length of multi voltage cell those are not bufferable.

This application option is deprecated and scheduled for removal in a future release.

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design. Use *report_app_options* to show the value of application options.

See Also

- [create_placement](#)
- [place_opt](#)
- [report_app_options](#)
- [set_app_options](#)

place.coarse.enable_dft_modeling

Enables dft modeling for coarse placement.

Data Types

String

Default auto

p

Description

Dft modeling aware placement makes the placement aware of connection to the dft modules through the start/stop cells connections of scan chains. This consideration helps in optimal placement of the dft modules along with other logical modules. This can improve wirelength and timing after buffering.

Possible values are *auto* or *false*.

When set to "auto", the modeling is adjusted automatically, depending on the start and stop cell types and connectivity, as well as the flow stage.

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design. Use *report_app_options* to show the value of application options.

See Also

- [clock_opt](#)
- [create_placement](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)
- [place_opt](#)
- [refine_placement](#)

place.coarse.enable_direct_congestion_mode

Enables direct congestion optimization during coarse placement.

Data Types

Boolean

Default false

Description

Adds a net density cost to the set of objectives that the coarse placement is optimizing for. This is done dynamically, in contrast to the cell expansion congestion optimization strategy, which runs external to the core placer software and keeps cell expansions static during the placement optimization. The net density objective measures the number of wires in a region and places a limit on the observed sum, leading the placer to spread the nets and the connected cells. This feature is expected to improve the placement response

p

to types of congestion which cannot be resolved via cell expansion, e.g. at macro corners, from repeater chains hugging macros, through-traffic congestion. A significant increase in the coarse placer runtime is expected, since a net density calculation is being added for each placement evaluation.

Possible values for this app option are *true* and *false*. The default value of *false* means that there is no net density awareness during coarse placement. If the app option is *true*, the net density objective gets employed in incremental placements.

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design. Use *report_app_options* to show the value of application options.

See Also

- [create_placement](#)
- [place_opt](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

place.coarse.enable_enhanced_soft_blockages

Enables enhanced soft blockage behavior.

Data Types

Boolean

Default "false"

Description

Enables enhanced soft blockage behavior. By default, if placement uses the current placement as a seed, it does not move cells that overlap a soft blockage. When this app options is set to "true", that behavior changes and placement can move cells that overlap a soft blockage, while still allowing these cells to overlap soft blockages.

Possible values are "true" or "false"

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design. Use *report_app_options* to show the value of the application options.

See Also

- [create_placement](#)
- [get_app_option_value](#)
- [place_opt](#)
- [refine_placement](#)
- [report_app_options](#)
- [set_app_options](#)

place.coarse.enable_spare_cell_placement

Enables spare cell placement.

Data Types

Boolean

Default "true"

Description

Enables spare cell placement. When this app option is set to "true", cells that are unconnected will be distributed evenly over the placeable area.

Possible values are "true" or "false"

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design. Use *report_app_options* to show the value of the application options.

See Also

- [create_placement](#)
- [get_app_option_value](#)
- [place_opt](#)
- [refine_placement](#)
- [report_app_options](#)
- [set_app_options](#)

place.coarse.enhanced_low_power_effort

Sets the effort level for enhanced low power placement.

Data Types

Enum

Default "low"

Description

Sets the effort level for "enhanced low power placement" feature. This affects the *place_opt* and *clock_opt* commands, as well as all the versions of *create_placement* and *refine_placement*, irrespective of the setting for the *place.coarse.low_power_placement* application option.

Enhanced low power placement reduces net switching power by reducing the wire length of the nets with non-zero toggle rate. It can work with either the activities from a SAIF, the user set information (via *set_switching_activity*), or the tool's default propagated activities. It is recommended to use a SAIF to ensure the feature works on accurate power/activity context.

Possible values are "none", "low", "medium", "high" or "auto". Higher values for the effort level causes the coarse placer to focus on optimizing the net switching power, at the possible cost of other metrics (for example, wire length and routability). The "auto" setting attempts to guess a reasonable effort level, usually improves the net switching power without degrading the wire length by more than a few percent.

The net switching power that the coarse placer optimizes is the average over all valid and active dynamic power scenarios. This feature gets automatically disabled if no valid power scenarios are active.

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design. Use the *report_app_options* command to show the value of application options.

See Also

- [create_placement](#)
- [place_opt](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

place.coarse.enhanced_low_power_version

Sets the version of the low power placement solution.

Data Types

Integer

Default "1"

Description

Sets the version of the "enhanced low power placement" feature used during coarse placement. This affects the *compile_fusion*, *place_opt* and *clock_opt* commands, as well as all the versions of *create_placement* and *refine_placement*, irrespective of the setting for the app option *place.coarse.enhanced_low_power_effort*.

Enhanced low power placement (ELPP) reduces the net switching power by reducing the wire lengths of nets with non-zero toggle rate, with likely degradations in other metrics (for example, wire length, timing and routability).

Setting the value of this *enhanced_low_power_version* variable to 1 makes ELPP use its original (2018) version, while setting it to 2 makes ELPP use the revamped (2024) version. The 2024 revamped version makes the ELPP usage more consistent throughout the flow and changes the balance of the composite cost priorities inside the placer, likely ending up with better net switching power and less wire length degradation than version 1, but possibly with slightly degraded timing or congestion.

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design. Use the *report_app_options* to show the value of application options.

See Also

- [create_placement](#)
- [place_opt](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

place.coarse.extreme_power_mode

Switches the coarse placer to an extreme power mode.

p

Data Types

*Integer***Default** "0"

Description

Switches the coarse placer to an "extreme power" mode. This affects all mega-commands employing the coarse placer, as well as all the versions of *create_placement* and *refine_placement*.

When operating in an extreme power mode, the coarse placer focuses on improving the net switching power and total power, at the likely expense of other QoR objectives. The result is expected to bring a couple of percent of net switching power improvement (as measured against a placer mode that does not optimize for power), with limited degradations in the other QoR metrics.

Application options are set using the *set_app_options* command, and they can be specified globally or with respect to a design. Use the *report_app_options* command to show the value of application options.

Clock net length improvement feature will be disabled as part of extreme power mode 1, unless explicitly set to true by a user.

See Also

- [create_placement](#)
- [place_opt](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

place.coarse.fix_cells_on_soft_blockages

Treat cells on soft blockages as fixed for coarse placement.

Data Types

*Boolean***Default** "true"

p

Description

When *true*, cells that are on top of soft blockages prior to placement are treated as fixed for coarse placement. This only applies to incremental placements. Also, this application option has no effect if application option *place.coarse.enable_enhanced_soft_blockages* is enabled.

Possible values are "true" or "false".

This application option is deprecated and scheduled for removal in a future release.

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design. Use *report_app_options* to show the value of application options.

See Also

- [clock_opt](#)
- [create_placement](#)
- [get_app_option_value](#)
- [place_opt](#)
- [refine_placement](#)
- [report_app_options](#)
- [set_app_options](#)

place.coarse.fix_hard_macros

Specifies that the placer should treat all hard macros in the design as fixed

Data Types

Boolean

Default false

Description

This option controls whether the *create_placement* command places hard macros or treats them as fixed objects.

Application options are set using the *set_app_options* command, and they might be specified globally or with respect to a design. Use *report_app_options* to show the value of application options. This application option is global only.

See Also

- [create_placement](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

place.coarse.fix_ios

Specifies I/O cells as fixed for coarse placement.

Data Types

Boolean

Default true

Description

This application options treats I/O cells as fixed for coarse placement.

Valid values are "true" or "false".

Application options are set using the *set_app_options* command, and they might be specified globally or with respect to a design. Use the *report_app_options* command to show the value of application options.

See Also

- [create_placement](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)
- [place_opt](#)
- [refine_placement](#)

place.coarse.fix_soft_macros

Specifies that the placer should treat all soft macros in the design as fixed

Data Types

Boolean

p

Default true**Description**

This option controls whether the *create_placement* command places soft macros or treat them as fixed objects.

Application options are set using the *set_app_options* command, and they might be specified globally or with respect to a design. Use *report_app_options* to show the value of application options. This application option is global only.

See Also

- [create_placement](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

place.coarse.handle_early_data

Forces coarse placement to complete on over-utilized voltage areas, exclusive movebound regions and designs.

Data Types*Boolean***Default** false**Description**

When this value is set to true then coarse placer internally will try to modify the overutilized regions of user created voltage areas and exclusive movebounds by modifying partial blockages, fix cells, hard blockages and category blockages. This option should be turned on only for early data flow, and all warnings should be cleaned before moving on to normal place_opt and clock_opt flow. Modifications are not saved in NDM and can't be viewed in GUI. Please be aware that this is likely to cause high density areas and may have cells sitting on top of blockages and fix cells.

Possible values are "true" or "false"

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design. Use *report_app_options* to show the value of application options.

See Also

- [create_placement](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)
- [place_opt](#)
- [refine_placement](#)

place.coarse.icg_auto_bound

Enables the placer to use automatic placement bounds for integrated clock gating and downstream sequential.

Data Types

Boolean

Default false

Description

This option controls whether placement uses the automatic group bounds created for integrated clock gating and sequential. The integrated clock gating automatic bounds are created before each placement invocations, and removed at the end of placement. The tool does not bound the cells already belonging to any existing group bounds.

Application options are set using the *set_app_options* command, and they might be specified globally or with respect to a design. Use *report_app_options* to show the value of application options.

See Also

- [create_placement](#)
- [place_opt](#)
- [report_app_options](#)
- [set_app_options](#)

place.coarse.icg_auto_bound_fanout_limit

Specifies fanout limit for the integrated clock gating automatic bound creation.

Data Types

integer

Default 100

Description

This application option limits the maximum number of fanouts that can be included in an integrated clock gating automatic bound.

Application options are set using the *set_app_options* command, and they might be specified globally or with respect to a design. Use the *report_app_options* command to show the value of application options.

See Also

- [create_placement](#)
- [place_opt](#)
- [report_app_options](#)
- [set_app_options](#)

place.coarse.increased_cell_expansion

When this option is true the congestion driven placement will reduce densities in congested areas more aggressively.

Data Types

Boolean

Default false

Description

This option is only impacts congestion driven placement. It is most effective when used on designs that have congestion hot spots in open areas of the block.

Possible values are "true" or "false"

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design. Use *report_app_options* to show the value of application options.

See Also

- [create_placement](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)
- [place_opt](#)
- [refine_placement](#)

place.coarse.ir_enable_er

Enable effective resistance placement during IRDP

Data Types

Boolean

Default true

Description

When this is set to true, IRDP will be effective resistance aware.

Acceptable values are true and false.

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design. Use *report_app_options* to show the value of application options.

See Also

- [create_placement](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)
- [place_opt](#)
- [clock_opt](#)

place.coarse.ir_enable_ir

Enable IR drop aware placement during IRDP

Data Types

Boolean

Default true

Description

When this is set to true, IRDP will be IR drop aware.

Acceptable values are true and false.

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design. Use *report_app_options* to show the value of application options.

See Also

- [create_placement](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)
- [place_opt](#)
- [clock_opt](#)

place.coarse.ir_enable_ol

Enable IR drop outliers handling during IRDP

Data Types

Boolean

Default true

Description

When this is set to true, IRDP will use IR drop outlier handling.

Acceptable values are true and false.

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design. Use *report_app_options* to show the value of application options.

See Also

- [create_placement](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)
- [place_opt](#)
- [clock_opt](#)

place.coarse.ir_enable_pd

Enable power density aware placement during IRDP

Data Types

Boolean

Default true

Description

When this is set to true, IRDP will be power density aware.

Acceptable values are true and false.

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design. Use *report_app_options* to show the value of application options.

See Also

- [create_placement](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)
- [place_opt](#)
- [clock_opt](#)

place.coarse.ir_verbose

Enable verbose mode

Data Types

Boolean

Default false

Description

When this is set to true the IR aware placement runs in verbose mode for debugging purposes.

Acceptable values are true and false.

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design. Use *report_app_options* to show the value of application options.

See Also

- [create_placement](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)
- [place_opt](#)
- [refine_placement](#)

place.coarse.irdp_cts_spacing

Enable clock spacing rules at the start of clock_opt build clock stage for IRDP power integrity flow.

Data Types

Boolean

Default true

Description

When this is set to true, clock spacing rules will be enabled at the start of clock_opt build clock stage for IRDP power integrity flow.

Acceptable values are true and false.

p

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design. Use *report_app_options* to show the value of application options.

See Also

- [create_placement](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)
- [place_opt](#)
- [clock_opt](#)

place.coarse.max_density

Sets the maximum density for the coarse placement engine.

Data Types

Float

Default 0.0

Description

When this value is set, the coarse placer attempts to limit local cell density to less than this setting. When the value is 0, the tool will automatically determine an appropriate value if automatic density control is active (controlled by application option *place.coarse.auto_density_control*) and otherwise the placer will try to spread cells evenly.

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design. Use *report_app_options* to show the value of application options.

See Also

- [create_placement](#)
- [get_app_option_value](#)
- [place_opt](#)
- [clock_opt](#)
- [refine_placement](#)

- [report_app_options](#)
- [set_app_options](#)

place.coarse.max_pins_per_square_micron

Sets the maximum pins per square micron

Data Types

Float

Default 0.0 (calculate internally).

Description

Use this app option to set the maximum pins per square micron. If value is 0, the placer will calculate the value internally.

Possible values are between 0.0 and 1000000.0.

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design. Use *report_app_options* to show the value of application options.

See Also

- [create_placement](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)
- [place_opt](#)
- [refine_placement](#)

place.coarse.ndr_area_aware

Makes the coarse placer aware of the area requirements by NDR nets.

Data Types

Boolean

Default false

Description

The `app_option` takes into consideration of wire "area" rather than just wirelength, when the placer is optimizing for wirelength, amongst other objectives. NDR nets usually require larger widths or spacing, the placer weights them more, resulting in shorter wirelengths.

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

place.coarse.perturbation_effort

Sets the perturbation effort in the core placer.

Data Types

String

Default default

Description

Use this option to change the default perturbation effort used in the core placer. For example, you may want to use a lower effort later in the flow.

Possible values are "default", "low", "medium", or "high".

Application options are set using the `set_app_options` command, and they may be specified globally or with respect to a design. Use `report_app_options` to show the value of application options.

See Also

- [create_placement](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)
- [place_opt](#)
- [refine_placement](#)

place.coarse.pin_cost_aware

Enables pin cost aware coarse placement

Data Types

Boolean

Default false

Description

When this value is set to true, the placer tries to control the maximum local pin cost in order to improve pin access routability.

This is a generalization of the pin density aware placement (see *place.coarse.pin_density_aware(3)*), where we use a model derived from the observed detailed router behavior to predict the routing length per layer needed to access each pin, the pin cost. Together with the effective track availability per layer, this defines a restriction for the coarse placer.

Possible values are "true" or "false"

Since the pin cost model is technology specific, it is required to use the *set_technology* command. Currently, only the 7, 7+, 5, s5 and s4 nodes are supported for the pin cost aware placement.

If both *place.coarse.pin_cost_aware* and *place.coarse.pin_density_aware* are set to "true", and the technology is set to one of the supported nodes, then the pin cost awareness takes precedence.

Application options are set using the *set_app_options* command. Use *report_app_options* to show the value of application options.

See Also

- [set_technology](#)
- [create_placement](#)
- [place_opt](#)
- [clock_opt](#)
- [refine_placement](#)
- [set_app_options](#)
- [report_app_options](#)
- [get_app_option_value](#)

place.coarse.pin_density_aware

Enables pin density-aware coarse placement

Data Types

Boolean

Default false

Description

When this value is set to true, the placer tries to control the maximum local pin density.

Possible values are "true" or "false"

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design. Use *report_app_options* to show the value of application options.

See Also

- [create_placement](#)
- [get_app_option_value](#)
- [place_opt](#)
- [refine_placement](#)
- [report_app_options](#)
- [set_app_options](#)

place.coarse.power_density_effective_resistance_layer

The lowest layer that is used for connections of the power grid to the standard cell row

Data Types

String

Default ""

Description

If you do not specify this option, the tool will use the lowest metal layer from the technology file. The layer is used to determine the connection point of the standard cell to the power grid. The effective resistance is calculated from this point.

See Also

- [create_placement](#)
- [get_app_option_value](#)
- [place_opt](#)
- [refine_placement](#)
- [report_app_options](#)
- [set_app_options](#)

place.coarse.power_density_fast

Enable faster power density aware placement

Data Types

Boolean

Default true

Description

When this is set to true, power density aware placement is enabled which is not effective resistance aware.

Acceptable values are true and false.

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design. Use *report_app_options* to show the value of application options.

See Also

- [create_placement](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

place.coarse.power_density_verbose

Enable verbose mode

p

Data Types

Boolean

Default false

Description

When this is set to true the power density aware placement runs in verbose mode for debugging purposes.

Acceptable values are true and false.

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design. Use *report_app_options* to show the value of application options.

See Also

- [create_placement](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

place.coarse.seq_array_icg_aware

Makes the coarse placer aware of sequential arrays.

Data Types

Boolean

Default false

Description

A sequential array is a group of registers that can be arranged as a 2-d array such that cells in each row connect to a clock net driven by an icg cell, and cells in each column connect to the same data input. Sequential arrays can cause congestion. When this application option is set to *true*, the coarse placer will address the issue by reducing the wirelength of the clock nets connecting the sequential arrays.

p

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

place.coarse.spread_repeater_paths

Enable the placer to spread repeater-paths around macro & blockage boundaries and corners.

Data Types

Boolean

Default "true"

Description

This option control whether placer will explicitly spread repeater-paths around macro & blockage boundaries and corners, minimizing clumping and hugging on those areas. A repeater-path is defined as single chain of repeater logic, consisting of buffer, inverter, and/or pipeline-register.

This feature does not spread non-repeater logic cells.

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design. Use *report_app_options* to show the value of application options.

See Also

- [create_placement](#)
- [place_opt](#)
- [report_app_options](#)
- [set_app_options](#)

place.coarse.target_routing_density

Controls the target routing density during congestion driven placement.

Data Types

Float

p

Default 0.0**Description**

Controls the target routing density for congestion driven placement. The default value of 0.0 allows the placer to determine the value based on which router is used.

By default, congestion driven placement only allows target routing density values in the range from 0.8 to 1.0, and automatically maps the user setting to the nearest value within this range.

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design. Use *report_app_options* to show the value of application options.

See Also

- [create_placement](#)
- [get_app_option_value](#)
- [place_opt](#)
- [refine_placement](#)
- [report_app_options](#)
- [set_app_options](#)

place.coarse.tns_driven_placement

Specifies that the timing driven placer should optimize total (instead of worst) negative slack of the design.

Data Types*Boolean***Default** "false"**Description**

This option controls whether the coarse placer (either run through *create_placement* explicitly, or when run as part of *place_opt* or *clock_opt* flows), tries to reduce the total negative slack of the design, instead of optimizing the worst slack of the design. It has no effect if application option *place.coarse.auto_timing_control* is enabled.

This application option is deprecated and scheduled for removal in a future release.

p

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design. Use *report_app_options* to show the value of application options. This application option is global only.

See Also

- [create_placement](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

place.coarse.use_tall_blockages

When this option is true the placer will have a greater tendency to keep cells within their original channel or floorplan region.

Data Types

Boolean

Default false

Description

This option is best used with incremental placement. When this option is true the placer will have a greater tendency to keep cells within the channel or floorplan region that they started.

Possible values are "true" or "false"

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design. Use *report_app_options* to show the value of application options.

See Also

- [create_placement](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)
- [place_opt](#)
- [refine_placement](#)

place.coarse.utilization_warning_threshold

Sets threshold for reporting movebounds and voltage areas with utilization more than the threshold.

Data Types

Float

Default 90.0

Description

Use this app option to set threshold for reporting high cell density movebounds and voltage areas in a design.

Possible values are between 0.0 and 1000000.0.

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design. Use *report_app_options* to show the value of application options.

See Also

- [create_placement](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)
- [place_opt](#)
- [refine_placement](#)

place.coarse.wide_cell_use_model

Enables handling of wide cells in N7 designs with narrow M1 PG straps.

Data Types

Boolean

Default "false"

Description

Enables handling of wide cells in N7 designs with narrow M1 PG straps by reducing their cell densities, and hence minimizing cell displacements during legalization. Cells wider

p

than half the PG pitch that cannot straddle the PG straps require a lower density, since only one cell could occupy a sub-row bordered by the PG straps.

Possible values are "true" or "false".

Application options are set using the `set_app_options` command, and they may be specified globally or with respect to a design. Use `report_app_options` to show the value of application options.

See Also

- [create_placement](#)
- [place_opt](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

place.common.pnet_aware_density

Specifies the target density under partially blocked power and ground rails.

Data Types

float

Default 1.0

Description

This application option specifies the target cell density under the power and ground rails on the layers specified by the `place.common.pnet_aware_layers` application option. The minimum width of blocked rails is controlled by the `place.common.pnet_aware_min_width` application option.

Examples

The following example partially blocks the area under the M1 and M2 power rails with a target utilization of 10%.

```
set_app_options -name place.common.pnet_aware_layers -value "M1 M2"  
set_app_options -name place.common.pnet_aware_density -value 0.1
```

See Also

- [create_placement](#)
- [place_opt](#)

p

- [clock_opt](#)
- [route_opt](#)

place.common.pnet_aware_layers

Specifies the layers which should be partially blocked for power and ground.

Data Types

String

Default <empty string>

Description

This app_option specifies the routing layer(s) under which power and ground rails will be considered to be partially blocked for the purposes of coarse placement and optimization. The amount of blockage and the minimum width are controlled by the companion app options *place.common.pnet_aware_density* and *place.common.pnet_aware_min_width*.

Examples

Partially block the area under layers M1 and M2 power rails with a target utilization of 10%.

```
set_app_options -name place.common.pnet_aware_layers -value "M1 M2"  
set_app_options -name place.common.pnet_aware_density -value 0.1
```

place.common.pnet_aware_min_width

Specifies the minimum width of rails that should be partially blocked for power and ground.

Data Types

float

Default 0.0

Description

In conjunction with the companion app options *place.common.pnet_aware_layers* and *place.common.pnet_aware_density*, this app_option specifies the minimum width of rails which should be partially blocked. Rails whose width is less than this number will be ignored for the purposes of partial blockage.

Examples

Partially block the area under M1 and M2 power rails with a target utilization of 10% but ignore rails whose width is smaller than 0.05 user units..

p

```
set_app_options -name place.common.pnet_aware_layers -value "M1 M2"
set_app_options -name place.common.pnet_aware_density -value 0.1 set_app_options
-name place.common.pnet_aware_min_width -value 0.05
```

place.common.use_placement_model

Turns on placement modeling during optimization.

Data Types

boolean

Default false

Description

When this app option is true, a model of the placement area is built which can be used by coarse placement and optimization to help predict the effects of various characteristics of the design on legalization. Specifically, it models the fragmentation in the placement area caused by dense power grids, multi-row cells and inbound cells. It replaces older app options that were used to model these effects. These app options are place.common.pnet_aware_layers, place.common.pnet_aware_min_width, place.common.pnet_aware_density and place.utilization.multi_row_penalty. If place.common.use_placement_model is true, the values of these app options will be ignored and the computed model will be used instead.

Examples

```
set_app_options -name place.common.use_placement_model -value true
```

place.diode.enable_backside_layer_antenna_fix

Enable antenna diode insertion on backside layer for diode mode 23.

Data Types

boolean

Default false

Description

Enable fixing antenna violation on backside layer for diode mode 23. This app is set to true if this feature is required.

Examples

```
set_app_options -name place.diode.enable_backside_layer_antenna_fix
-value true
```

place.floorplan.density_aware_hard_movebounds

Specifies that the placer should treat all hard movebounds in the design as partitions.

Data Types

Boolean

Default "false"

Description

This option controls whether the *create_placement* command will treat hard movebounds as partitions.

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design. Use *report_app_options* to show the value of application options. This application option is global only.

See Also

- [create_placement](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

place.floorplan.sliver_size

Specifies floorplan sliver size

Data Types

length

Default 0

Description

Specifies the minimum channel size to allow standard cell placement. The default is 0. The option type is length, which means that it must be specified with the length unit.

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design. Use *report_app_options* to show the value of the application options.

See Also

- [create_placement](#)
- [get_app_option_value](#)
- [place_opt](#)
- [refine_placement](#)
- [report_app_options](#)
- [set_app_options](#)

place.legalize.access_check_enclosure_waiving_distance

Specifies the distance that test via can be out of the testing region in legalizer access check.

Data Types

float

Default 0.0

Description

When checking pin access, we defined a span which is the test region to put a test via. The assumption is that if the enclosure of test via is beyond the span, it is very possible to violate end to end spacing with neighbor shapes. It is a pessimistic assumption but not a bug because legalizer cannot always predict router behavior precisely. If it is known for user that we can allow some enclosure beyond the span and will not cause a violation, we can set this option. The value is the distance of enclosure exceed the span.

Examples

```
set_app_options -list {place.legalize.access_check_enclosure_waiving_distance 0.7}
```

See Also

- [legalize_placement](#)
- [check_legality](#)

place.legalize.access_check_extra_layer

Enable checking of an additional metal and via layer in access checking.

p

Data Types

boolean

Default false

Description

Enable checking of an additional metal and via layer in pin access check. Typically only one layer beyond cell's top pin layer is used for checking. Some Technology DRC rules require an additional layer for multi-layer DRC checks. Turning on this option may lead to measurable runtime impact. This option is only available if Advanced prerouted net check (NPLDRC) is used for legalization and legality checking.

Acceptable values are true or false(default).

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design. Use *report_app_options* to show the value of application options.

See Also

- [legalize_placement](#)
- [check_legality](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

place.legalize.advanced_ignore_horizontal_keepout_margin

Data Types

boolean

Default false

Description

Ignore the horizontal keepout margin. Vertical keepout is very difficult for cells to be legalized properly because row resolution is very big compared to site resolution in horizontal keepout (moving a cell w/ vertical KO impacts multiple rows at once, not just the row the cell is going into). Users are encouraged to use horizontal keepout rather than vertical keepouts. To enable ignoring, mark the app option value as true.

See Also

- [legalize_placement](#)
- [check_legality](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

place.legalize.advanced_ignore_vertical_keepout_margin

Data Types

boolean

Default true

Description

This ignores the vertical keepout margin by default in advanced legalizer. Vertical keepout makes it very difficult for cells to be legalized properly because row resolution is very big compared to site resolution in horizontal keepout (moving a cell w/ vertical KO impacts multiple rows at once, not just the row the cell is going into). Users are encouraged to use horizontal keepout rather than vertical keepouts. If vertical constraint is essential, consider using libcell to libcell vertical spacing rule instead. To stop ignoring, mark the app option value as false.

See Also

- [legalize_placement](#)
- [check_legality](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

place.legalize.advanced_layer_access_check

Specify the strategy for checking the accessibility of standard cell pins during legality checking and legalization.

p

Data Types*list < string >***Default** empty list**Description**

Note that this option is only used if you also use `set place.legalize.enable_advanced_prerouted_net_check` to true. It is not usually recommended or necessary for older technologies (16 nanometer or higher).

During standard cell placement, it is necessary to verify that the cell is placed in such a way that its pins can be routed. Fixed preroutes in your design (such as the power grid) may interfere with this pin accessibility.

This app option lets you control the access checks for pins during placement. Depending on your technology rules, cell library, and power grid, it may be necessary to adjust these access checks to make them stricter or looser. For most situations, the default settings of this app option are the correct ones.

This app option requires you to specify a tcl list, surrounded by curly brackets {}. You can specify settings for each layer of metal in your technology. The metal you specify corresponds the layer that is used to connect to your pins. For example, if you specify a setting for Metal2, then this will affect access checks between a Metal2 route and a Metal1 pin.

```
set_app_option -name place.legalize.advanced_layer_access_check -value \
  {Metal2 pin_access2}
```

You can specify access checks for many metal layers. And you can specify more than one access check for a layer. This is done by using nested curly braces. For example:

```
set_app_option -name place.legalize.advanced_layer_access_check \
  -value {{Metal2 pin_access2 track_space3.5} {Metal3 pin_extend2}}
```

There are three types of access checks you can set. They are listed below.

Pin access count

The pin access count setting specifies how many via "access points" must be available on each standard cell pin. For example, if you set `pin_access2`:

```
set_app_option -name place.legalize.advanced_layer_access_check -value \
  {Metal2 pin_access2}
```

this means that the placer will try to create situations where the connection between a Metal2 route and a Metal1 pin could connect in at least two places. The only accepted value for the pin access count option is `pin_access2`. If you do not specify `pin_access2`, then the default behavior will be to accept situations with one access point. One access point is almost always the correct setting.

Pin extension

When the router connects to a standard cell pin, it often extends that pin before connecting to it. By doing this, it creates more opportunity to make a connection. And when a pin is under a power strap, it is often necessary to extend the pin to make any connection at all.

You can control what assumptions the placer makes about the ability to extend pins. For example, if you set `pin_extend0`:

```
set_app_option -name place.legalize.advanced_layer_access_check -value \\  
  {Metal3 pin_extend0}
```

you are telling the placer to assume that, when making a connection from Metal3 routes to Metal2 pins, assume that the Metal2 pins will not be extended. If you set `pin_extend1`, the placer will assume that the pin can be extended up to 1 routing track. If you set `pin_extend2`, the placer will assume that the pin can be extended up to 2 routing tracks. By default, the lowest layer pins will have a setting of `pin_extend0`. Higher layer pins will have `pin_extend1`.

Note that the placer will not assume pin extension in a case where it will violate routing DRC rules.

Track capacity

When the placer is evaluating the routability of standard cells, it needs to make sure that there is sufficient routing resource to connect to all of the cell's pins. This is done by evaluating "track capacity". For example, if a cell has 6 Metal1 pins and only 5 Metal2 tracks on which to make connections to those pins, this might cause a problem. The placer needs to enforce some rules to figure out if the route can be successful.

By default, the placer enforces the "track_drc" rule:

```
set_app_option -name place.legalize.advanced_layer_access_check -value \\  
  {Metal2 track_drc}
```

This means that when connecting Metal2 routes to Metal1 pins, it must be possible to make the connections using the available Metal2 tracks and:

- The metal2 routes stay within the confines of the cell area
- The metal2 routes do not violate routing DRC rules

Note that the real router is not restricted to route within the confines of the cell. But placement is using this restrictive model to better guarantee that the route will at least be possible.

p

Other possible settings of the track capacity are:

- `track_none` -- Turn off track capacity checking.
- `track_space2` -- Allow one connection from a Metal2 track to a Metal1 pin for every 2 Metal1 tracks in the cell.
- `track_space2.5` -- Allow one connection from a Metal2 track to a Metal1 pin for every 2.5 Metal1 tracks in the cell.
- `track_space3` -- Allow one connection from a Metal2 track to a Metal1 pin for every 3 Metal1 tracks in the cell.
- `track_space3.5` -- Allow one connection from a Metal2 track to a Metal1 pin for every 3.5 Metal1 tracks in the cell.
- `track_space4` -- Allow one connection from a Metal2 track to a Metal1 pin for every 4 Metal1 tracks in the cell.

See Also

- [legalize_placement](#)
- [check_legality](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

place.legalize.advanced_legalizer_auto_flow

Enable/disable the automatic detection of non-abutting site row designs styles

Data Types

boolean

Default `true`

Description

When this option is set to true (the default), the advanced legalizer will automatically detect floorplan styles with non-abutting, non-aligned site rows. For these floorplan styles, which are typically found for older technology nodes, the advanced legalizer will iterate over the voltage areas or voltage area regions that have site rows which are not aligned to each other. The legalizer might even iterate over individual site definitions in case they are non-overlapping and not aligned within a voltage area region.

See Also

- [legalize_placement](#)
- [check_legality](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

place.legalize.advanced_legalizer_hierarchical

Enable hierarchical legalization and placement rule checking for the advanced legalizer.

Data Types

Boolean

Default true

Description

Use this application option to enable hierarchical legalization and placement rule checking. This option only applies to the advanced legalizer. It affects the mega flow commands as well as `legalize_placement` and `check_legality`. When enabled, legalization and rule checking are traversing physical hierarchies which are marked as editable. Displacement and violation reports are printed separately for each hierarchy. A summary report at the end combines all hierarchical reports.

The application option is design specific. Use *report_app_options* to show the value of application options.

See Also

- [legalize_placement](#)
- [check_legality](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

place.legalize.advanced_legalizer_verbose_flow

Enable/disable verbose output when automatically detecting non-abutting site row designs

styles

Data Types

boolean

Default false

Description

When this option is set to true (the default), the advanced legalizer will print additional information while automatically detecting floorplan styles with non-abutting, non-aligned site rows. The automatic detection is controlled by app option *place.legalize.advanced_legalizer_auto_flow*.

See Also

- [legalize_placement](#)
- [check_legality](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

place.legalize.advanced_libpin_access_check

Specify the strategy for checking the accessibility of standard cell pins during legality checking and legalization.

Data Types

list < string >

Default empty list

Description

Note that this option is only used if you also use set *place.legalize.enable_advanced_prerouted_net_check* to true. It is not usually recommended or necessary for older technologies (16 nanometer or higher).

During standard cell placement, it is necessary to verify that the cell is placed in such a way that its pins can be routed. Fixed preroutes in your design (such as the power grid) may interfere with this pin accessibility.

p

This app option lets you control the access checks for selected libcell pins during placement. Depending on your technology rules, cell library, and power grid, it may be necessary to adjust these access checks to make them stricter or looser. option are the correct ones.

This app option requires you to specify a tcl list, surrounded by curly brackets {}. You can specify settings for each libcell pin in your technology. For example:

```
set_app_option -name place.legalize.advanced_libpin_access_check -value
  \\
    {mybuf16/Y:2 mybuf24/Y:3}
```

The numbers in the example (2 and 3) specify how many via "access points" must be available on each standard cell pin. In this example, the placer will create situations where the connection to the mybuf16/Y pins could connect in at least two places. And it will create situations where the connection to the mybuf14/Y pins could connect in at least three places.

See Also

- [legalize_placement](#)
- [check_legality](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

place.legalize.align_pin_only

Force Pin Color Alignment to check on pins only, allowing routing blockages within library cells to not align with tracks when pin track alignment is enabled.

Data Types

boolean

Default false

Description

By default if Pin Color Alignment rule engine is enabled, all metal shapes within a cell must align with a track, and if colored, the correct color track. This option allows user to loosen this restriction for routing blockages only. With this option library cells perform Pin Color Alignment checks on pins only. Library cells may have aligned and unaligned routing blockages. This option is used for legalization and legality checking.

Acceptable values are true or false(default).

p

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design. Use *report_app_options* to show the value of application options.

See Also

- [legalize_placement](#)
- [check_legality](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

place.legalize.allow_tiecells_on_buffer_only_blockages

Allow the placement of tie-cells on buffer-only blockages during legalization.

Data Types

boolean

Default `true`

Description

Use this app option to allow the placement of tie-cells on buffer-only category blockages in legalization. This functionality is only supported by the advanced legalizer. When setting this app option to *true* (the default), then tie-cells together with buffers and inverters are the only cells allowed on buffer-only type category blockages. In case this app option is set to *false*, then only buffers and repeaters are allowed on buffer-only blockages.

For the definition of category blockages see the *create_placement_blockage* command.

This app option has only an affect if category blockages are enabled for the advanced legalizer (see app option *place.legalize.enable_category_blockages*).

See Also

- [legalize_placement](#)
- [get_app_option_value](#)
- [set_app_options](#)
- [report_app_options](#)
- [create_placement_blockage](#)

place.legalize.allow_touch_track_for_access_check

Use less conservative estimate when checking pin access.

Data Types

Boolean

Default true

Description

Use a less conservative estimate of routeability by considering a routing track that is exactly the distance of pre-route width plus potential via enclosure to not be impeded. This setting is used by pin accessibility checking during legalization.

Acceptable values are true(default) or false.

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design. Use *report_app_options* to show the value of application options.

See Also

- [legalize_placement](#)
- [check_legality](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

place.legalize.allow_unaligned_blockage

Allow routing blockages within library cells to not align with tracks when pin track alignment is enabled.

Data Types

boolean

Default false

Description

By default if pin-track alignment is enabled, all metal shapes within a cell must align with a track, and if colored, the correct color track. This option allows user to loosen this

p

restriction for routing blockages only. With this option library cells may have aligned and unaligned routing blockages. This option is used for legalization and legality checking.

Acceptable values are true or false(default).

Application options are set using the `set_app_options` command, and they may be specified globally or with respect to a design. Use `report_app_options` to show the value of application options.

See Also

- [legalize_placement](#)
- [check_legality](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

place.legalize.always_continue

Setup legalizer behavior

Data Types

Boolean

Default true

Description

When this application option is set to false, the legalizer will abort when it cannot find legal locations for all cells in the design. If the applicaiton option is set to true, the legalizer will try to legalize as many cells as it can. The cells that cannot be legalized will be left in their original locations, untouched by the legalizer.

See Also

- [legalize_placement](#)
- [get_app_option_value](#)
- [set_app_options](#)
- [report_app_options](#)

place.legalize.auto_adjust_drc_rules_for_short_pin

Automatically disable Color-Based End-Of-Line Alignment rule in legalizer when design has short pins.

Data Types

boolean

Default false

Description

When this option is set to true, Advanced Legalizer will do pre-checking on each cells. If there's short pins, it will skip Color-Based End-Of-Line Alignment in PG DRC rule. For cases that do not need Color-Based End-Of-Line Alignment rule, it will improve runtime and won't hurt correctness for some special cases which spend too much time on Color-Based End-Of-Line Alignment rule.

See Also

- [legalize_placement](#)
- [check_legality](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

place.legalize.auto_disable_end_to_end_rules

Automatically disable End-To-End-Keepout rule in legalizer when no PG line end in the cells.

Data Types

boolean

Default false

Description

When this option is set to true, Advanced Legalizer will do pre-checking on each cells. If there's no PG line end right in the cell boundary, it will skip End-To-End Keepout in PG DRC rule. It will improve runtime and won't hurt correctness for some special cases which spend too much time on End-To-End Keepout rule.

See Also

- [legalize_placement](#)
- [check_legality](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

place.legalize.avoid_backside_pins_under_preroute_layers

Specifies the list of back-side routing layers of pre-routed net shapes that library pins which are specified with application option `place.legalize.avoid_backside_pins_under_preroute_libpins` may not be placed under during legality checking and legalization.

Data Types

list < string >

Default empty list

Description

The option is for back-side layer. Specifies the list of pre-routed net shapes under which given library pins are restricted from being placed. This option is used during legalization, legality checking, and legality query during optimization. This application option must be used in tandem with option "`place.legalize.avoid_backside_pins_under_preroute_libpins`" which specifies the list of library pins affected.

Examples

```
set_app_options -list {place.legalize.avoid_backside_pins_under_preroute_layers  
{ bmetal1 bmetal2 } }
```

place.legalize.avoid_backside_pins_under_preroute_libpins

Specifies the list of library back-side pins that may not be placed under back-side pre-routed net shapes during legality checking and legalization.

Data Types

list < string >

Default empty list

p

Description

The option is for back-side layer. Specifies the list of library pins that are restricted from being placed under pre-routed net shapes. This option is used during legalization, legality checking, and legality query during optimization. This application option must be used in tandem with option "place.legalize.avoid_backside_pins_under_preroute_layers" which specifies the list of pre-route routing layers to avoid.

Examples

```
set_app_options -list {place.legalize.avoid_backside_pins_under_preroute_libpins { libA/
libCellA1/X libB/libCellB1/Y libB/libCellB2/Y } }
```

place.legalize.avoid_backside_pins_under_preroute_libpins_under_net

Specifies the list of library back-side pins that may not be placed under specific back-side pre-routed net shapes during legality checking and legalization.

Data Types

list of pairs of strings

Default empty list

Description

The option is for back-side layer. Specifies the list of library pins that are restricted from being placed under specific pre-routed net shapes. This option is used during legalization, legality checking, and legality query during optimization. This application option must be used in tandem with option "place.legalize.avoid_backside_pins_under_preroute_layers" which specifies the list of pre-route routing layers to avoid.

Examples

```
set_app_options -name
place.legalize.avoid_backside_pins_under_preroute_libpins_under_net
-value {{lib782_i0m_180h_50pp_pwm_lvt/i0mcabf00ab1d06x5/vcc_in VSS}
{lib782_i0m_180h_50pp_pwm_lvt/i0mcabf00ab1d06x5/vcc_in VDD}}
```

place.legalize.avoid_backside_pins_under_preroute_on_no_via_region_pin

Allow back-side pins without via region not be placed under back-side pre-routed net shapes.

p

Data Types*bool***Default** false**Description**

The option is for back-side pins. By default the value is false, and only libpins with via region will be checked. When the option is true, libpins without via region will also be checked as well. For the libpin has via region, use the via region to check overlap with pre-route shape. If the libpin does not have via region, use the pin shape bbox to check overlap with pre-route shape.

Examples

```
set_app_options -name
place.legalize.avoid_backside_pins_under_preroute_on_no_via_region_pin -value true
```

place.legalize.avoid_backside_pins_under_preroute_width_threshold

Specifies the minimum back-side pre-route shape width, where pre-routes wider than the threshold will be avoided in placement of specified back-side library pins.

Data Types*float***Default** 0.0**Description**

The option is for back-side layer. Specifies the threshold width for pre-routed net shapes. Shapes wider than the threshold will be considered. By default all pre-routed net shapes on the given layers are considered. This option is used during legalization, legality checking, and legality query during optimization. This application option must be used with both other options "place.legalize.avoid_backside_pins_under_preroute_libpins" and "place.legalize.avoid_backside_pins_under_preroute_layers".

Examples

```
set_app_options -list
{place.legalize.avoid_backside_pins_under_preroute_width_threshold 0.4 }
```

place.legalize.avoid_pins_under_preroute_layers

Specifies the list of routing layers of pre-routed net shapes that library pins which are specified with application option `place.legalize.avoid_pins_under_preroute_libpins` may not be placed under during legality checking and legalization.

Data Types

list < string >

Default empty list

Description

Specifies the list of pre-routed net shapes under which given library pins are restricted from being placed. This option is used during legalization, legality checking, and legality query during optimization. This application option must be used in tandem with option "`place.legalize.avoid_pins_under_preroute_libpins`" which specifies the list of library pins affected.

Application options are set using the `set_app_options` command, and they may be specified globally or with respect to a design. Use `report_app_options` to show the value of application options.

Examples

```
set_app_options -list {place.legalize.avoid_pins_under_preroute_layers { Metal2 Metal3 } }
```

See Also

- [legalize_placement](#)
- [check_legality](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

place.legalize.avoid_pins_under_preroute_libpins

Specifies the list of library pins that may not be placed under pre-routed net shapes during legality checking and legalization.

Data Types

list < string >

Default empty list

p

Description

Specifies the list of library pins that are restricted from being placed under pre-routed net shapes. This option is used during legalization, legality checking, and legality query during optimization. This application option must be used in tandem with option "place.legalize.avoid_pins_under_preroute_layers" which specifies the list of pre-route routing layers to avoid.

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design. Use *report_app_options* to show the value of application options.

Examples

```
set_app_options -list {place.legalize.avoid_pins_under_preroute_libpins { libA/libCellA1/X  
libB/libCellB1/Y libB/libCellB2/Y } }
```

See Also

- [legalize_placement](#)
- [check_legality](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

place.legalize.avoid_pins_under_preroute_libpins_under_net

Specifies the list of library pins that may not be placed under specific pre-routed net shapes during legality checking and legalization.

Data Types

list of pairs of strings

Default empty list

Description

Specifies the list of library pins that are restricted from being placed under specific pre-routed net shapes. This option is used during legalization, legality checking, and legality query during optimization. This application option must be used in tandem with option "place.legalize.avoid_pins_under_preroute_layers" which specifies the list of pre-route routing layers to avoid.

p

Examples

```
set_app_options -name place.legalize.avoid_pins_under_preroute_libpins_under_net
-value {{lib782_i0m_180h_50pp_pwm_lvt/i0mcabf00ab1d06x5/vcc_in VSS}
{lib782_i0m_180h_50pp_pwm_lvt/i0mcabf00ab1d06x5/vcc_in VDD}}
```

place.legalize.avoid_pins_under_preroute_on_no_via_region_pin

Allow pins without via region not be placed under pre-routed net shapes.

Data Types

bool

Default false

Description

By default the value is false, and only libpins with via region will be checked. When the option is true, libpins without via region will also be checked as well. For the libpin has via region, use the via region to check overlap with pre-route shape. If the libpin does not have via region, use the pin shape bbox to check overlap with pre-route shape.

Examples

```
set_app_options -name place.legalize.avoid_pins_under_preroute_on_no_via_region_pin
-value true
```

place.legalize.avoid_pins_under_preroute_width_threshold

Specifies the minimum pre-route shape width, where pre-routes wider than the threshold will be avoided in placement of specified library pins.

Data Types

float

Default 0.0

Description

Specifies the threshold width for pre-routed net shapes. Shapes wider than the threshold will be considered. By default all pre-routed net shapes on the given layers are considered. This option is used during legalization, legality checking, and legality query during optimization. This application option must be used with both other options "place.legalize.avoid_pins_under_preroute_libpins" and "place.legalize.avoid_pins_under_preroute_layers".

p

Application options are set using the `set_app_options` command, and they may be specified globally or with respect to a design. Use `report_app_options` to show the value of application options.

Examples

```
set_app_options -list {place.legalize.avoid_pins_under_preroute_width_threshold0.4 }
```

See Also

- [legalize_placement](#)
- [check_legality](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

place.legalize.backside_prerouted_net_check_exclude_lib_cells

Specifies the list of library cells to exclude from backside pre-routed net checking.

Data Types

list of strings

Default empty list

Description

Specifies the list of library cells to exclude from backside pre-routed net checking. Pre-routed net checking is performed during detailed placement and legality checks. With this app_option user can mark instances of specific library cells as not requiring the checking. Using this option will likely result in DRCs caused by placement of these instances. User should only use this option if s/he has contingency plans for resolving such DRCs. If option is used, `check_legality` will not report DRC violations on instances of the specified library cells.

Examples

```
set_app_options -list {place.legalize.backside_prerouted_net_check_exclude_lib_cells  
{AOI22 AOI12}}
```

place.legalize.check_maximum_number_of_arrayed_cuts

Check Maximum Number of Arrayed Cuts rule in `legalize_placement` and `check_legality`.

Data Types

Boolean

Default `false`

Description

When a standard cell is placed in your design, the legalizer checks to see that Maximum Number of Arrayed Cuts rule is obeyed.

By default (*false*), all layers with the related techfile syntaxes are ignored.

When set to *true*, all layers with the related techfile syntaxes are checked.

See Also

- [legalize_placement](#)
- [check_legality](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

place.legalize.check_metal_dpt_odd_cycle_on_layers

Set layers for metal layer odd cycle in Advanced Legalizer PG DRC rule.

Data Types

string

Default `""`

Description

When setting layers in this option, NPLDRC will check double pattern rule for odd cycle case on no mask metals.

See Also

- [legalize_placement](#)
- [check_legality](#)

place.legalize.check_vt_min_area_for_multi_height_cells

Enable/disable the min area checks on VT implant layers for multi-height standard cells

Data Types

boolean

Default false

Description

When this option is set to true, the advanced legalizer will perform min area rule checks on VT implant layers that have a min-area constraint defined. The check is only done for multi-height standard cells. This rule check can also be enabled using the app option *place.legalize.enable_advanced_legalizer_rules -value {min_area}*.

See Also

- [legalize_placement](#)
- [check_legality](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

place.legalize.color_shift_layers

Specifies the metal layers on which to perform color shifting.

Data Types

list

Default {}

Description

This application option specifies the routing metal layers on which to perform color shifting. Specify the layers by using the layer names from the technology file.

This check is performed during legalization and legality checking when both of the following application options are set to *true*:

- *place.legalize.enable_color_shifting*
- *place.legalize.enable_advanced_prerouted_net_check*

p

For example, to enable pin color alignment checking on the m1 and m2 metal layers, use the following commands:

```
set_app_options -list {place.legalize.enable_color_shifting true}
set_app_options -list {place.legalize.enable_advanced_prerouted_net_check true}
set_app_options -list {place.legalize.color_shift_layers {m1 m2}}
```

Application options are set using the *set_app_options* command, and they can be specified globally or with respect to a block. To show the current value of an application option, use the *get_app_option* command. To show the value of multiple application options, use the *report_app_options* command.

See Also

- [legalize_placement](#)
- [check_legality](#)

place.legalize.consider_pmj_vl_user_routes

This app_option enables checking of cell placement against PMJ/VL user-route shapes of fixed cells.

Data Types

Boolean

Default "false"

Description

Enable checking of cell placement against PMJ/VL user-route shapes of fixed cells. The assumption is that those shapes and cells will not be moved or deleted by other engine. Thus, legalizer should consider them as pre-route shapes and check PG rules. Note that if the user-route shapes connected the the pin shapes, the pin shapes will be considered as well and merged with user-route shapes.

place.legalize.constrain_shape_search_extent

Constrain search space for prerouted net shapes.

Data Types

Boolean

Default False

p

Description

This application option directs the preroute net legality checker to constrain the search space around the checked cell. By default the tool collects all the shapes within the sectors (internal pnet representation). With the option set to True only the sector shapes that lie within the immediate area around the checked cell are selected. This option is intended to be used only in situations where PG grid is constructed in a way that creates very large sectors. A warning message NPLDRC-007 is issued if such a sector is found.

Acceptable values are true or false (default).

Application options are set using the `set_app_options` command, and they may be specified globally or with respect to a design. Use `report_app_options` to show the value of application options.

See Also

- [legalize_placement](#)
- [check_legality](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)
- [NPLDRC-007](#)

place.legalize.cross_row_vt_constraints

Define cross row VT implant layer constraints.

Data Types

list of string integer pairs

Default empty list

Description

Use this app option to define cross row VT implant constraints, and the value should be within the range of -5 to +5. A cross row constraint is forbidding touching, slight overlaps and slight spacing between implants of cells in neighbouring rows. For example, if a cell has a VTN implant at its top, and another cell is touching this cell at a corner in the next higher row also having a VTN implant (but at its bottom), then those implants have a touching corner. If such a touching corner is forbidden according to the design rules, then the user can explicitly define this rule using {VTN 0} as an entry to the list of constraints of this app option. The zero value defines the difference of the cell edges measured in sites.

p

So a constraint value of {VTN 1} means that it is not allowed to have both cells one site away from each other if they have VTN implants facing each other. A value of -1 would denote a forbidden overlap of 1 site. Multiple layer - value pairs can be specified as a list as an app option value.

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design. Use *report_app_options* to show the value of application options.

Examples

```
set_app_options -name place.legalize.cross_row_vt_constraints -value {{VTN -1}}
```

defines a single constraint for implant layer VTN not allowing single site cell overlap.

```
set_app_options -name place.legalize.cross_row_vt_constraints -value {{VTN -1} {VTP 0}}
```

defines a constraint for implant layer VTN not allowing single site cell overlap. Implant layer VTP does not allow corner touching cells.

See Also

- [legalize_placement](#)
- [check_legality](#)
- [get_app_option_value](#)
- [set_app_options](#)
- [report_app_options](#)

place.legalize.disable_advanced_access_check_on_libcells

Disable access checking of specified libcells.

Data Types

list of strings

Default Empty list

Description

Disable pin access checks on specified libcells. By default access check is performed for all libcells. Using this option may result in reduced routeability of instances of the specified libcells.

p

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design. Use *report_app_options* to show the value of application options.

Examples

```
set_app_options -name  
  place.legalize.disable_advanced_access_check_on_libcells -value {AOI22  
  AOI12}
```

See Also

- [legalize_placement](#)
- [check_legality](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

place.legalize.disable_advanced_access_check_on_libpins

Disable access checking of specified libcells.

Data Types

list of strings

Default Empty list

Description

Disable pin access checks on specified libpins. By default access check is performed for all libpins of all libcells. Using this option may result in reduced routeability of instances of the libcells of the specified libpins. Related option `place.legalize.disable_advanced_access_check_on_libcells` disables access check on all pins of the specified libcells.

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design. Use *report_app_options* to show the value of application options.

Examples

```
set_app_options -name  
  place.legalize.disable_advanced_access_check_on_libpins -value {AOI22/X  
  AOI12/Y}
```

p

See Also

- [legalize_placement](#)
- [check_legality](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

place.legalize.disable_advanced_legalizer_rules_on_instcells

Disable specific advanced legalizer rule checking against specified instcells.

Data Types

list of pairs of strings

Default Empty list

Description

Disable specific advanced legalizer rule engine on specified instcells. By default advanced legalizer rule engines check against all instcells. The rule name and the instcell name support wildcards. Using this option may result in reduced routeability of instances of the specified instcells.

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design. Use *report_app_options* to show the value of application options.

Examples

```
set_app_options -name
place.legalize.disable_advanced_legalizer_rules_on_instcells -value
{ {pg_drc *AOI_22*} }
```

See Also

- [legalize_placement](#)
- [check_legality](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

place.legalize.disable_advanced_legalizer_rules_on_libcells

Disable specific advanced legalizer rule checking against specified libcells.

Data Types

list of pairs of strings

Default Empty list

Description

Disable specific advanced legalizer rule engine on specified libcells. By default advanced legalizer rule engines check against all libcells. The rule name and the libcell name support wildcards. Using this option may result in reduced routeability of instances of the specified libcells.

Application options are set using the `set_app_options` command, and they may be specified globally or with respect to a design. Use `report_app_options` to show the value of application options.

Examples

```
set_app_options -name
place.legalize.disable_advanced_legalizer_rules_on_libcells -value
{ {pg_drc *AOI_22*} }
```

See Also

- [legalize_placement](#)
- [check_legality](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

place.legalize.disable_backside_drc_rules_on_libcells

Disable backside checking of specified DRC rules on specified libcells.

Data Types

list of pairs of strings

Default Empty list

p

Description

Disable backside checking of specified DRC rule categories on specified libcells. The value of the app_option is a list of string pairs, where the first string is the rule category name and the second is the libcell name. Alternately one may use wildcard "*" to disable the given rule category on all applicable combinations. This app_option applies to Advanced Prerouted Net Check only.

Examples

```
set_app_options -name
place.legalize.disable_backside_drc_rules_on_libcells -value { {short
*AOI_22*} }
```

place.legalize.disable_drc_rules

Disable checking of specified DRC rules on specified layers.

Data Types

list of pairs of strings

Default Empty list

Description

Disable checking of specified DRC rule categories on specified layers. The rule categories represent one or more closely related technology file rule. The value of the app_option is a list of string pairs, where the first string is the rule category name and the second is the exact layer name as defined in the technology file. Alternately one may use keyword "all" to disable the given rule category on all applicable layers. This app_option applies to Advanced Prerouted Net Check only. The category name needs to match one from the list given below exactly.

Acceptable values for rule category are: short, minimum_spacing, forbidden_pitch, color_based_end_of_line_alignment, color_based_end_of_line_keepout, color_based_metal_corner_keepout, color_based_end_of_line_spacing, color_based_span_spacing, stack_via_keepout, dpt_via_spacing, end_of_line_forbidden_spacing, end_of_line_to_end_of_line_spacing, fat_metal_forbidden_spacing, fat_metal_spacing, dpt_same_color_spacing, general_via_spacing, metal_span_orthogonal_spacing, metal_span_spacing, one_neighbor_end_of_line_spacing, one_neighbor_end_to_end_spacing, paired_via_spacing, preferred_and_nonpreferred_fat_metal_spacing, preferred_and_nonpreferred_end_of_line_spacing, preferred_direction_forbidden_spacing, same_mask_preferred_and_nonpreferred_end_of_line_spacing, signal_routing_max_width, two_neighbor_end_of_line_enclosure, two_neighbor_end_of_line_spacing, two_sided_end_of_line_spacing,

p

u_shape_spacing, zero_neighbor_end_of_line_enclosure,
 mask2_cut_end_spacing, no_cut_metal_end_of_line_to_end_of_line_spacing,
 no_cut_metal_same_mask_cut_end_spacing,
 no_cut_metal_same_mask_pref_and_nonpref_eol_spacing, mask1_span_spacing,
 mask2_span_spacing, four_via_region_forbidden_metal, separated_via_spacing,
 group_via_rules, group_via_region, end_to_end_keepout, three_via_region_forbidden,
 dpt_odd_cycle

Application options are set using the `set_app_options` command, and they may be specified globally or with respect to a design. Use `report_app_options` to show the value of application options.

Examples

```
set_app_options -name place.legalize.disable_drc_rules -value
  {{color_based_span_spacing M2} {end_to_end_keepout M3} {group_via_region
  all}}
```

See Also

- [legalize_placement](#)
- [check_legality](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

place.legalize.disable_drc_rules_in_access_check

Disable checking of specified DRC rules on specified layers in access check only.

Data Types

list of pairs of strings

Default Empty list

Description

Disable checking of specified DRC rule categories on specified layers in access check only. The rule is still enabled during regular DRC checking. The rule categories represent one or more closely related technology file rule. The value of the app_option is a list of string pairs, where the first string is the rule category name and the second is the exact layer name as defined in the technology file. Alternately one may use keyword "all" to disable the given rule category on all applicable layers. This app_option applies to

p

Advanced Prerouted Net Check only. The category name needs to match one from the list given below exactly.

Acceptable values for rule category are: short, minimum_spacing, forbidden_pitch, color_based_end_of_line_alignment, color_based_end_of_line_keepout, color_based_metal_corner_keepout, color_based_end_of_line_spacing, color_based_span_spacing, stack_via_keepout, dpt_via_spacing, end_of_line_forbidden_spacing, end_of_line_to_end_of_line_spacing, fat_metal_forbidden_spacing, fat_metal_spacing, dpt_same_color_spacing, general_via_spacing, metal_span_orthogonal_spacing, metal_span_spacing, one_neighbor_end_of_line_spacing, one_neighbor_end_to_end_spacing, paired_via_spacing, preferred_and_nonpreferred_fat_metal_spacing, preferred_and_nonpreferred_end_of_line_spacing, preferred_direction_forbidden_spacing, same_mask_preferred_and_nonpreferred_end_of_line_spacing, signal_routing_max_width, two_neighbor_end_of_line_enclosure, two_neighbor_end_of_line_spacing, two_sided_end_of_line_spacing, u_shape_spacing, zero_neighbor_end_of_line_enclosure, mask2_cut_end_spacing, no_cut_metal_end_of_line_to_end_of_line_spacing, no_cut_metal_same_mask_cut_end_spacing, no_cut_metal_same_mask_pref_and_nonpref_eol_spacing, mask1_span_spacing, mask2_span_spacing, four_via_region_forbidden_metal, separated_via_spacing, group_via_rules, group_via_region, end_to_end_keepout, three_via_region_forbidden, dpt_odd_cycle

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design. Use *report_app_options* to show the value of application options.

Examples

```
set_app_options -name place.legalize.disable_drc_rules_in_access_check
-value {{color_based_span_spacing M2} {end_to_end_keepout M3}
{group_via_region all}}
```

See Also

- [legalize_placement](#)
- [check_legality](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

place.legalize.disable_drc_rules_in_multi_cell_check

Disable checking of specified DRC rules on specified layers in multi_cell check only.

Data Types

list of pairs of strings

Default Empty list

Description

Disable checking of specified DRC rule categories on specified layers in multi_cell check only. The rule is still enabled during regular DRC checking. The rule categories represent one or more closely related technology file rule. The value of the app_option is a list of string pairs, where the first string is the rule category name and the second is the exact layer name as defined in the technology file. Alternately one may use keyword "all" to disable the given rule category on all applicable layers. This app_option applies to Advanced Prerouted Net Check only. The category name needs to match one from the list given below exactly.

Acceptable values for rule category are: short, minimum_spacing, forbidden_pitch, color_based_end_of_line_alignment, color_based_end_of_line_keepout, color_based_metal_corner_keepout, color_based_end_of_line_spacing, color_based_span_spacing, stack_via_keepout, dpt_via_spacing, end_of_line_forbidden_spacing, end_of_line_to_end_of_line_spacing, fat_metal_forbidden_spacing, fat_metal_spacing, dpt_same_color_spacing, general_via_spacing, metal_span_orthogonal_spacing, metal_span_spacing, one_neighbor_end_of_line_spacing, one_neighbor_end_to_end_spacing, paired_via_spacing, preferred_and_nonpreferred_fat_metal_spacing, preferred_and_nonpreferred_end_of_line_spacing, preferred_direction_forbidden_spacing, same_mask_preferred_and_nonpreferred_end_of_line_spacing, signal_routing_max_width, two_neighbor_end_of_line_enclosure, two_neighbor_end_of_line_spacing, two_sided_end_of_line_spacing, u_shape_spacing, zero_neighbor_end_of_line_enclosure, mask2_cut_end_spacing, no_cut_metal_end_of_line_to_end_of_line_spacing, no_cut_metal_same_mask_cut_end_spacing, no_cut_metal_same_mask_pref_and_nonpref_eol_spacing, mask1_span_spacing, mask2_span_spacing, four_via_region_forbidden_metal, separated_via_spacing, group_via_rules, group_via_region, end_to_end_keepout, three_via_region_forbidden, dpt_odd_cycle

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design. Use *report_app_options* to show the value of application options.

p

Examples

```
set_app_options -name place.legalize.disable_drc_rules_in_access_check
-value {{color_based_span_spacing M2} {end_to_end_keepout M3}
{group_via_region all}}
```

See Also

- [legalize_placement](#)
- [check_legality](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

place.legalize.disable_drc_rules_on_libcells

Disable checking of specified DRC rules on specified libcells.

Data Types

list of pairs of strings

Default Empty list

Description

Disable checking of specified DRC rule categories on specified libcells. The rule categories represent one or more closely related technology file rule. The value of the app_option is a list of string pairs, where the first string is the rule category name and the second is the libcell name. Alternately one may use wildcard "*" to disable the given rule category on all applicable combinations. This app_option applies to Advanced Prerouted Net Check only.

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design. Use *report_app_options* to show the value of application options.

Examples

```
set_app_options -name place.legalize.disable_drc_rules_on_libcells -value
{ {short *AOI_22*} }
```

See Also

- [legalize_placement](#)
- [check_legality](#)

p

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

place.legalize.disable_ignore_unextended_access_shape

Don't ignore the unextended access shape in NPLDRC access check.

Data Types

boolean

Default false

Description

By default, NPLDRC will ignore unextended shapes in access check. The unextended shape is due to either there is no track extensions in the design or the extension failed (might because there is no grid on track). The behavior to skip the shape could cause false or missing violation. Thus adding this option to control the behavior.

place.legalize.drc_verbose_level

This application option sets the verbosity level of text output of the legalization pg_drc checker.

Data Types

Integer

Default "5"

Description

This application option sets the verbosity level of text output of the legalization pg_drc checker. Default value is "5" -- medium verbosity. For minimal verbosity, level "1" should be used. For maximal verbosity, use level "7" or above. This is a global-scope option.

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design. Use *report_app_options* to show the value of application options.

See Also

- [legalize_placement](#)
- [check_legality](#)

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

place.legalize.enable_advanced_legalizer

Enable/disable Advanced (2D) Legalizer

Data Types

boolean

Default false

Description

When this option is set to true, it enables Advanced Legalizer through out the flow.

See Also

- [legalize_placement](#)
- [check_legality](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

place.legalize.enable_advanced_legalizer_rules

Enable additional Advanced Legalizer rules not detected by auto rule detection.

Data Types

list of string

Default false

Description

This app options enable additional rules not detected by Advanced Legalizer's auto rule detection.

See Also

- [legalize_placement](#)
- [check_legality](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

place.legalize.enable_advanced_legalizer_sanity_checker

Data Types

list of string

Default false

Description

This app option has no effect anymore. It is deprecated in 19.12-SP2 and will be removed in version 19.12-SP4. Please use the app option `place.legalize.enable_early_data_support` instead.

See Also

- [legalize_placement](#)
- [check_legality](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

place.legalize.enable_advanced_prerouted_net_check

Turn on the advanced preroute checks.

Data Types

bool

Default false

p

Description

Note that this option is only recommended for use in advanced node designs. It is not usually recommended or necessary for older technologies (16 nanometer or higher). This feature is currently only in limited release and is not necessarily supported if you run into trouble. Please ask your Synopsys representative before you use this option.

This option turns on extra legalization checks for modern technologies. This includes extra specialized DRC checks and extra checks to better ensure cell routability.

See Also

- [legalize_placement](#)
- [check_legality](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

place.legalize.enable_advanced_track_capacity_check

This app_option enables checking of cell placement such that routing access of all pins is considered simultaneously with respect to the cell's location, without regard to nearby prerouted net shapes. This is a check that is additional to the standard checking of track capacity of cells' pins with respect to prerouted net shapes. This check is only available in Advanced Legalizer flow, and is only needed for 10nm and below technologies. This check is performed during legalization and legality checking. This is off by default in IC CompilerII.

Data Types

Boolean

Default "false"

Description

Enable checking of pin access of all cell's pins simultaneously at cells current location in the floorplan without regard to the presense of prerouted net shapes.

Possible values are "true" or "false". Default is "false".

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design. Use *report_app_options* to show the value of application options.

place.legalize.enable_allowable_orient

Enable/disable allowable orientation.

Data Types

boolean

Default false

Description

When this app option is set to true, legalization considers allowable orientations of instance along with the legal orientations of the site row. Otherwise, only the legal orientations of the site row are considered.

See Also

- [legalize_placement](#)
- [check_legality](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

place.legalize.enable_backside_drc_rules_on_libcells

Enable backside checking of specified DRC rules on specified libcells.

Data Types

list of pairs of strings

Default Empty list

Description

Enable backside checking of specified DRC rule categories on specified libcells. The value of the app_option is a list of string pairs, where the first string is the rule category name and the second is the libcell name. Alternately one may use wildcard "*" to disable the given rule category on all applicable combinations. This app_option applies to Advanced Prerouted Net Check only.

Examples

```
set_app_options -name  
place.legalize.enable_backside_drc_rules_on_libcells -value  
{ {end_to_end_keepout *AOI_22*} }
```

place.legalize.enable_backside_pg_drc

This app_option enables checking of cell placement against pre-routed nets on back-side routing layers, including Power and Ground nets, during legalization and legality checking.

Data Types

Boolean

Default "false"

Description

Enable checking of cell placement against pre-routed net shapes.

place.legalize.enable_build_cell_runtime_enhancement

Enable runtime enhancement of building cell initialization.

Data Types

boolean

Default false

Description

Enable runtime enhancement of building cell initialization. This enables to speed up the initialization runtime if the app-option is set as true.

Examples

```
set_app_options -name  
place.legalize.enable_build_cell_runtime_enhancement -value true
```

place.legalize.enable_category_blockages

Enables category blockage support for the advanced legalizer.

Data Types

Boolean

Default false

Description

This application option controls whether category placement blockages are honored by the advanced legalizer.

p

By default (*false*), category placement blockages, including the predefined *allow_buffer_only* and *register* categories, are considered only by the coarse placer and are not honored by the advanced legalizer.

Setting this application option to *true* enables advanced legalizer support for category blockages, including *allow_buffer_only* and *register* blockages.

Category blockages can be defined on both cells and library cells. For category blockages with *blocked_percentage* less than 100 (using the *-blocked_percentage* option of the *create_placement_blockage* command), legalizer does not impose any density limit on allowed cells. For disallowed cells of a category blockage, the cells are always be blocked by the category blockage, regardless the value of *blocked_percentage*. Category blockages with a 100% percentage blockage value are considered as full blockages, like placement blockages of type *hard*.

User-defined categories on cells take precedence over those defined on library cells. See the *create_placement_blockage* man page for details on category blockages.

Note that, by default, the advanced legalizer is disabled. To enable the advanced legalizer, set the *place.legalize.enable_advanced_legalizer* application option to *true*.

See Also

- [legalize_placement](#)
- [get_app_option_value](#)
- [set_app_options](#)
- [report_app_options](#)
- [create_placement_blockage](#)

place.legalize.enable_cldr

enable CLDR integration to improve DRC convergence after RO/HRO

Data Types

boolean

Default *false*

Description

CLDR is a DRC convergence application to fix pin access DRC based on ICC2 router. The option is only available in RO/HRO commands. Other than RO/HRO, CLDR won't be triggered.

p

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design. Use *report_app_options* to show the value of application options.

Examples

```
set_app_options -name place.legalize.enable_cldr -value true
```

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

place.legalize.enable_clo_ccode_spacing_check

Enable CCODE Spacing rule in CLO for context aware legalization.

Data Types

boolean

Default false

Description

Enable ccode_spacing in Advanced Legalizer only in CLO and disable in all legalization. This app_option is for Context-Aware CLO flow.

Acceptable values are true or false(default).

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design. Use *report_app_options* to show the value of application options.

place.legalize.enable_color_aware_placement

This app option controls whether color rules should be honored during check_legality and legalize_placement.

Data Types

Boolean

Default "false"

p

Description

Enable/disable color legalizer engine.

Possible values are "true" or "false". Default is "false".

Application options are set using the `set_app_options` command, and they may be specified globally or with respect to a design. Use `report_app_options` to show the value of application options.

place.legalize.enable_color_shifting

Enables color shifting of the cell placement such that the colors of the cell-internal pins and metal shapes align with the routing tracks on the same metal layer.

Data Types

Boolean

Default false

Description

This application option enables color shifting of the cell-internal pins and metal shapes such that they align with routing tracks of the appropriate color. The checks consider both metal and track color.

By default (*false*), this check is not enabled.

When set to *true*, color shifting is performed during legalization and when the following application options are also set:

- `place.legalize.color_shift_layers`

Use this application option to specify the layers on which to perform color shifting.

- `place.legalize.enable_advanced_prerouted_net_check`

This application option must be set to *true* to enable color shifting.

For example, to enable color shifting on the m1 and m2 metal layers, use the following commands:

```
set_app_options -list {place.legalize.enable_color_shifting true}
set_app_options -list {place.legalize.enable_advanced_prerouted_net_check true}
set_app_options -list {place.legalize.color_shift_layers {m1 m2}}
```

Application options are set using the `set_app_options` command, and they can be specified globally or with respect to a block. To show the current value of an application

p

option, use the *get_app_option* command. To show the value of multiple application options, use the *report_app_options* command.

See Also

- [legalize_placement](#)
- [check_legality](#)

place.legalize.enable_cut_metal_color_alignment

Enables checking of the cell placement such that the colors of the cell-internal cut metal shapes align with the routing tracks on the respective metal layer.

Data Types

Boolean

Default false

Description

This application option enables checking of the cell-internal cut metal shapes such that they align with routing tracks of the appropriate color. The checks consider both cut metal and their respective metal layer track color.

By default (*false*), this check is not enabled.

When set to *true*, this check is performed during legalization and legality checking when the following application options are also set:

- *place.legalize.pin_color_alignment_layers*

Use this application option to include the cut metal layers on which to perform the cut metal color alignment checking.

- *place.legalize.pin_color_alignment_check*

This application option must be set to *true* to enable the pin color alignment checking of cut metal layers and metal layers.

For example, to enable cut metal color alignment checking on the m1 metal layer and cm1 cut metal layer, use the following commands:

```
set_app_options -list {place.legalize.enable_pin_color_alignment_check
true}
set_app_options -list {place.legalize.enable_cut_metal_color_alignment
true}
set_app_options -list {place.legalize.pin_color_alignment_layers {m1
cm1}}
```

p

Application options are set using the `set_app_options` command, and they can be specified globally or with respect to a block. To show the current value of an application option, use the `get_app_option` command. To show the value of multiple application options, use the `report_app_options` command.

See Also

- [legalize_placement](#)
- [check_legality](#)

place.legalize.enable_cutmetal_color_shifting

Enables color shifting of the cell placement such that the colors of the cell-internal cut metal shapes align with the routing tracks on the respective metal layer.

Data Types

Boolean

Default `false`

Description

This application option enables color shifting of the cell-internal cut metal shapes such that they align with routing tracks of the respective metal layer of the appropriate color. The checks consider both cut metal and track color of the respective metal layer.

By default (*false*), this check is not enabled.

When set to *true*, color shifting is performed during legalization and when the following application options are also set:

- `place.legalize.color_shift_layers`

Use this application option to specify the layers on which to perform color shifting.

- `place.legalize.enable_color_shifting`

This application option must be set to *true* to enable color shifting.

For example, to enable color shifting on the m1 metal layer and cm1 cut metal layer, use the following commands:

```
set_app_options -list {place.legalize.enable_color_shifting true}
set_app_options -list {place.legalize.enable_cutmetal_color_shifting
  true}
set_app_options -list {place.legalize.color_shift_layers {m1 cm1}}
```


p

Application options are set using the `set_app_options` command, and they can be specified globally or with respect to a block. To show the current value of an application option, use the `get_app_option` command. To show the value of multiple application options, use the `report_app_options` command.

See Also

- [legalize_placement](#)
- [check_legality](#)

place.legalize.enable_drc_rules

Enable checking of specified DRC rules on specified layers.

Data Types

list of pairs of strings

Default Empty list

Description

Enable checking of specified DRC rule categories on specified layers. The rule categories represent one or more closely related technology file rule. The value of the `app_option` is a list of string pairs, where the first string is the rule category name and the second is the exact layer name as defined in the technology file. Alternately one may use keyword "all" to enable the given rule category on all applicable layers. This `app_option` applies to Advanced Prerouted Net Check only. The category name needs to match one from the list given below exactly.

Acceptable values for rule category are: short, minimum_spacing, forbidden_pitch, color_based_end_of_line_alignment, color_based_end_of_line_keepout, color_based_metal_corner_keepout, color_based_end_of_line_spacing, color_based_span_spacing, stack_via_keepout, dpt_via_spacing, end_of_line_forbidden_spacing, end_of_line_to_end_of_line_spacing, fat_metal_forbidden_spacing, fat_metal_spacing, dpt_same_color_spacing, general_via_spacing, metal_span_orthogonal_spacing, metal_span_spacing, one_neighbor_end_of_line_spacing, one_neighbor_end_to_end_spacing, paired_via_spacing, preferred_and_nonpreferred_fat_metal_spacing, preferred_and_nonpreferred_end_of_line_spacing, preferred_direction_forbidden_spacing, same_mask_preferred_and_nonpreferred_end_of_line_spacing, signal_routing_max_width, two_neighbor_end_of_line_enclosure, two_neighbor_end_of_line_spacing, two_sided_end_of_line_spacing, u_shape_spacing, zero_neighbor_end_of_line_enclosure, mask2_cut_end_spacing, no_cut_metal_end_of_line_to_end_of_line_spacing,

p

```
no_cut_metal_same_mask_cut_end_spacing,
no_cut_metal_same_mask_pref_and_nonpref_eol_spacing, mask1_span_spacing,
mask2_span_spacing, four_via_region_forbidden_metal, separated_via_spacing,
group_via_rules, group_via_region, end_to_end_keepout, three_via_region_forbidden,
dpt_odd_cycle
```

Application options are set using the `set_app_options` command, and they may be specified globally or with respect to a design. Use `report_app_options` to show the value of application options.

Examples

```
set_app_options -name place.legalize.enable_drc_rules
  {{color_based_span_spacing M2} {end_to_end_keepout M3} {group_via_region
  all}}
```

See Also

- [legalize_placement](#)
- [check_legality](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

place.legalize.enable_drc_rules_in_access_check

Enable checking of specified DRC rules on specified layers in access check only.

Data Types

list of pairs of strings

Default Empty list

Description

Enable checking of specified DRC rule categories on specified layers in access check only. The rule is still enabled during regular DRC checking. The rule categories represent one or more closely related technology file rule. The value of the app_option is a list of string pairs, where the first string is the rule category name and the second is the exact layer name as defined in the technology file. Alternately one may use keyword "all" to enable the given rule category on all applicable layers. This app_option applies to Advanced Prerouted Net Check only. The category name needs to match one from the list given below exactly.

p

Acceptable values for rule category are: short, minimum_spacing, forbidden_pitch, color_based_end_of_line_alignment, color_based_end_of_line_keepout, color_based_metal_corner_keepout, color_based_end_of_line_spacing, color_based_span_spacing, stack_via_keepout, dpt_via_spacing, end_of_line_forbidden_spacing, end_of_line_to_end_of_line_spacing, fat_metal_forbidden_spacing, fat_metal_spacing, dpt_same_color_spacing, general_via_spacing, metal_span_orthogonal_spacing, metal_span_spacing, one_neighbor_end_of_line_spacing, one_neighbor_end_to_end_spacing, paired_via_spacing, preferred_and_nonpreferred_fat_metal_spacing, preferred_and_nonpreferred_end_of_line_spacing, preferred_direction_forbidden_spacing, same_mask_preferred_and_nonpreferred_end_of_line_spacing, signal_routing_max_width, two_neighbor_end_of_line_enclosure, two_neighbor_end_of_line_spacing, two_sided_end_of_line_spacing, u_shape_spacing, zero_neighbor_end_of_line_enclosure, mask2_cut_end_spacing, no_cut_metal_end_of_line_to_end_of_line_spacing, no_cut_metal_same_mask_cut_end_spacing, no_cut_metal_same_mask_pref_and_nonpref_eol_spacing, mask1_span_spacing, mask2_span_spacing, four_via_region_forbidden_metal, separated_via_spacing, group_via_rules, group_via_region, end_to_end_keepout, three_via_region_forbidden, dpt_odd_cycle

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design. Use *report_app_options* to show the value of application options.

Examples

```
set_app_options -name place.legalize.enable_drc_rules_in_access_check
  {{color_based_span_spacing M2}} {end_to_end_keepout M3} {group_via_region
  all}}
```

See Also

- [legalize_placement](#)
- [check_legality](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

place.legalize.enable_drc_rules_in_multi_cell_check

Enable checking of specified DRC rules on specified layers in multi_cell check only.

Data Types*list of pairs of strings***Default** Empty list**Description**

Enable checking of specified DRC rule categories on specified layers in multi_cell check only. The rule is still enabled during regular DRC checking. The rule categories represent one or more closely related technology file rule. The value of the app_option is a list of string pairs, where the first string is the rule category name and the second is the exact layer name as defined in the technology file. Alternately one may use keyword "all" to enable the given rule category on all applicable layers. This app_option applies to Advanced Prerouted Net Check only. The category name needs to match one from the list given below exactly.

Acceptable values for rule category are: short, minimum_spacing, forbidden_pitch, color_based_end_of_line_alignment, color_based_end_of_line_keepout, color_based_metal_corner_keepout, color_based_end_of_line_spacing, color_based_span_spacing, stack_via_keepout, dpt_via_spacing, end_of_line_forbidden_spacing, end_of_line_to_end_of_line_spacing, fat_metal_forbidden_spacing, fat_metal_spacing, dpt_same_color_spacing, general_via_spacing, metal_span_orthogonal_spacing, metal_span_spacing, one_neighbor_end_of_line_spacing, one_neighbor_end_to_end_spacing, paired_via_spacing, preferred_and_nonpreferred_fat_metal_spacing, preferred_and_nonpreferred_end_of_line_spacing, preferred_direction_forbidden_spacing, same_mask_preferred_and_nonpreferred_end_of_line_spacing, signal_routing_max_width, two_neighbor_end_of_line_enclosure, two_neighbor_end_of_line_spacing, two_sided_end_of_line_spacing, u_shape_spacing, zero_neighbor_end_of_line_enclosure, mask2_cut_end_spacing, no_cut_metal_end_of_line_to_end_of_line_spacing, no_cut_metal_same_mask_cut_end_spacing, no_cut_metal_same_mask_pref_and_nonpref_eol_spacing, mask1_span_spacing, mask2_span_spacing, four_via_region_forbidden_metal, separated_via_spacing, group_via_rules, group_via_region, end_to_end_keepout, three_via_region_forbidden, dpt_odd_cycle

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design. Use *report_app_options* to show the value of application options.

Examples

```
set_app_options -name place.legalize.enable_drc_rules_in_access_check
  {{color_based_span_spacing M2} {end_to_end_keepout M3} {group_via_region
  all}}
```

See Also

- [legalize_placement](#)
- [check_legality](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

place.legalize.enable_drc_rules_on_libcells

Enable checking of specified DRC rules on specified libcells.

Data Types

list of pairs of strings

Default Empty list

Description

Enable checking of specified DRC rule categories on specified libcells. The rule categories represent one or more closely related technology file rule. The value of the app_option is a list of string pairs, where the first string is the rule category name and the second is the libcell name. Alternately one may use wildcard "*" to enable the given rule category on all applicable combinations. This app_option applies to Advanced Prerouted Net Check only.

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design. Use *report_app_options* to show the value of application options.

Examples

```
set_app_options -name place.legalize.enable_drc_rules_on_libcells -value  
{ {end_to_end_keepout *AOI_22*} }
```

See Also

- [legalize_placement](#)
- [check_legality](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

place.legalize.enable_early_data_support

Data Types

boolean

Default false

Description

This app option when set to true enables dirty data handling for advanced legalizer. For `legalize_placement`, it does handling of scenarios that may cause long runtime or perceived hangs. For `check_legality` and other features that use advanced legalizer, it issues warning and error messages if any such scenario is encountered.

See Also

- [legalize_placement](#)
- [check_legality](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

place.legalize.enable_end_on_pref_grid_center

Enable End On Pref Grid Center on extension of PG rule violation in legalizer.

Data Types

boolean

Default false

Description

By default, all shapes are extended to preferred grid if it is defined. With the option on, legalizer will detect if there is syntax of End On Pref Grid Center in tech file and keep the line ends which are already aligned on center of preferred grid. For other line ends, they are still extended to preferred grid as before.

See Also

- [legalize_placement](#)
- [check_legality](#)

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

place.legalize.enable_fast_via_ladder_mode

This is to automatically generate PG blockage constraints for via ladder check.

Data Types

boolean

Default false

Description

PG blockage is an idea to improve runtime of via ladder check. It allows user to specify how much PG can overlap against via ladder pin bbox. Instead of precise checking, legalizer could quickly move cell away from PG region to save runtime. With this option, tool looks for all via ladder cut layers and if there are more than 1 cut rows, then it sets 0% overlap constraints on the routing layer above; if there is only one cut row, it sets a 25% overlap constraint on the routing layer above.

See Also

- [legalize_placement](#)
- [check_legality](#)

place.legalize.enable_gui_drc_error_status_support

Enable DRC error fix for GUI error browser.

Data Types

boolean

Default false

Description

This app options enable GUI error browser shows DRC violated cells as an error. This app option will be ODB in future release.

See Also

- [check_legality](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

place.legalize.enable_has_rectangle_only_rule

Check Has Rectangle Only rule in legalize_placement and check_legality.

Data Types

Boolean

Default false

Description

When a standard cell is placed in your design, the legalizer checks to see that Maximum Number of Arrayed Cuts rule is obeyed.

By default (*false*), all layers with the related techfile syntaxes are ignored.

When set to *true*, all layers with the related techfile syntaxes are checked.

See Also

- [legalize_placement](#)
- [check_legality](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

place.legalize.enable_high_local_density_handling

Data Types

boolean

Default false

p

Description

This app option when set to true enables dirty data handling of designs with high local density. This handling would skip legalization for cells that belong to high density areas, preventing long runtime and hangs in some cases. This app option only has an effect in case DCM is not enabled, see `set_qor_strategy` command for the `early_design` mode.

See Also

- [check_legalizer_sanity](#)
- [legalize_placement](#)
- [check_legality](#)
- [get_app_option_value](#)
- [set_app_options](#)
- [set_qor_strategy](#)

place.legalize.enable_incr_legalization_after_clo

Enable/disable Incremental Legalizer after CLO (Concurrent Legalization and Optimization)

Data Types

Boolean

Default false

Description

When this option is set to true, it enables an internal incremental legalizer inside CLO, using the same infrastructure. It should result in a placement without `check_legality` violations after CLO.

See Also

- [legalize_placement](#)
- [check_legality](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

place.legalize.enable_legal_index_rules

Enable/disable legal index rules for placement legalization and legality checking.

Data Types

boolean

Default false

Description

When this option is set to true, legal index rules would be considered for placement legalization and legality check. This should only be used with designs that are prepped with legal index setup.

See Also

- [legalize_placement](#)
- [check_legality](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

place.legalize.enable_movable_rectilinear_cells

Enable/disable the support of rectilinear standard cells in the advanced legalizer

Data Types

boolean

Default true

Description

When this option is set to true (the default), the advanced legalizer will support multi-height standard cells whose outline shape is not rectangular. The widths of these cells can be different on different site rows. Every site row coverage needs to be a rectangular shape.

See Also

- [legalize_placement](#)
- [check_legality](#)

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

place.legalize.enable_multi_cell_access_check

Enable multi cell pin access checks.

Data Types

boolean

Default false

Description

When this option is set to true, the multi cell access check will consider impact on a cell's pin access due to presence of abutting neighbor cells as well as any prerouted net PG net shapes in the vicinity. User is required to set related option place.legalize.enable_multi_cell_pnet_check to TRUE to enable this option to have an effect. Default value is FALSE.

See Also

- [legalize_placement](#)
- [check_legality](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

place.legalize.enable_multi_cell_check_on_libcell

Do multi-cell check on specified libcells.

Data Types

list of strings

Default empty list

Description

Specify a list of libcell name and Legalizer will only check multi-cell check on those libcells.

place.legalize.enable_multi_cell_pin_access_check

Enable/disable pin connectivity analysis for multi cell pin access checks

Data Types

boolean

Default false

Description

When this option is set to true, the multi cell access check will consider logical connectivity of pins of neighboring cells. If two pins of abutting cells are logically connected, a direct routing connection can be established which avoids the necessity to route on higher layers. These logical connections simplify the routing to access pin shapes. This app option only works together with option *place.legalize.enable_multi_cell_access_check*. Enabling this option will filter out potential violations due to logical connectivity.

See Also

- [legalize_placement](#)
- [check_legality](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

place.legalize.enable_multi_cell_pnet_check

Enable checking of DRC rules that involve objects in more than one cell as well as any prerouted net shapes that are in the vicinity.

Data Types

Boolean

Default False

Description

Enable checking multiple cells at once for DRC violations that may or may not involve additional power/ground grid shapes. This option is only available if Advanced Legalizer is used for legalization and legality checking.

Acceptable values are true or false (default).

p

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design. Use *report_app_options* to show the value of application options.

See Also

- [legalize_placement](#)
- [check_legality](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

place.legalize.enable_multi_cell_track_capacity_check

Enable multi cell track capacity checking during multi cell pin access checks.

Data Types

boolean

Default false

Description

When this option is set to true, the multi cell access check will analyze the track capacity of the cell's pins along with that of the abutting neighbor cells. Track capacity estimates the sufficiency of routing tracks in the area to access all pins in questions. User is required to set related options *place.legalize.enable_multi_cell_pnet_check* and *place.legalize.enable_multi_cell_access_check* to TRUE to enable this option to have an effect. Default value is FALSE.

See Also

- [legalize_placement](#)
- [check_legality](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

place.legalize.enable_ndr_pin_check

Enable automatic checking of pin accessibility for pins with NDR constraints in legality checker and placement legalizer.

Data Types

boolean

Default false

Description

When this option is set to true the tool automatically analyzes the netlist to collect a list of instance pins that require NDR routing. Accessibility of routing access to these pins with respect to preroutes on specified metal layers is considered for legality checking and placement legalization. The app_option "place.legalize.avoid_pins_under_preroute_layers" is required to indicate the metal layers that need consideration. In order to ensure proper tool behavior during optimization, app_option "place.legalize.use_nll_query_cm" must be set to "true".

See Also

- [legalize_placement](#)
- [check_legality](#)

place.legalize.enable_non_default_via_def_access_check

Enable non-default via def in access checking.

Data Types

boolean

Default false

Description

By default, the access checker in legalizer will use only default via def contactcode as a test via. For some advanced node, only non-default via def is routable. Enabling the option will make legalizer consider non-default via def. Note that there might be runtime increase due to much more choice for legalizer to test.

See Also

- [legalize_placement](#)
- [check_legality](#)

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

place.legalize.enable_non_preferred_direction_span_check

Check Color-Based Span Spacing rule on non-preferred routing direction in `legalize_placement` and `check_legality`.

Data Types

Boolean

Default `false`

Description

When a standard cell is placed in your design, the legalizer checks if Color-Based Span Spacing rule is obeyed on non-preferred routing direction.

By default (*false*), all layers with the related techfile syntaxes are ignored on non-preferred routing direction.

When set to *true*, all layers with the related techfile syntaxes are checked on non-preferred routing direction.

See Also

- [legalize_placement](#)
- [check_legality](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

place.legalize.enable_npldrc_pin_color_alignment_check

Enables checking of the cell placement such that the colors of the cell-internal pins and metal shapes align with the routing tracks on the same metal layer in NPLDRC.

Data Types

Boolean

Default `true`

Description

This application option enables checking of the cell-internal pins and metal shapes such that they align with routing tracks of the appropriate color. The checks consider both metal and track color.

By default (*true*), this check is performed during legalization and legality checking when the following application options are also set:

- [place.legalize.enable_pin_color_alignment_check](#)

When set to *false*, the pin color alignment check is not performed in NPLDRC. This application option doesn't affect the check in pin_color_align engine.

See Also

- [legalize_placement](#)
- [check_legality](#)

place.legalize.enable_odS17_checking

Enable OD.S.17 checking

Data Types

boolean

Default false

Description

When this option is set to true (the default), legalizer will enable OD.S.17 checking

See Also

- [legalize_placement](#)
- [check_legality](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

place.legalize.enable_odS17_fixing

Enable additional OD.S.17 fixing

p

Data Types*boolean***Default** `false`**Description**

When this option is set to `true` (the default), legalizer will enable additional OD.S.17 fixing by inserting blockages to break up the continuous horizontal boundaries.

See Also

- [legalize_placement](#)
- [check_legality](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

place.legalize.enable_pin_color_alignment_check

Enables checking of the cell placement such that the colors of the cell-internal pins and metal shapes align with the routing tracks on the same metal layer.

Data Types

Boolean

Default `false`**Description**

This application option enables checking of the cell-internal pins and metal shapes such that they align with routing tracks of the appropriate color. The checks consider both metal and track color.

By default (*false*), this check is not enabled.

When set to *true*, this check is performed during legalization and legality checking when the following application options are also set:

- *place.legalize.pin_color_alignment_layers*

Use this application option to specify the layers on which to perform the pin color alignment checking.

p

- *place.legalize.enable_prerouted_net_check*

This application option must be set to *true* to enable the pin color alignment checking.

For example, to enable pin color alignment checkkng on the m1 and m2 metal layers, use the following commands:

```
set_app_options -list {place.legalize.enable_pin_color_alignment_check true}
set_app_options -list {place.legalize.enable_prerouted_net_check true}
set_app_options -list {place.legalize.pin_color_alignment_layers {m1 m2}}
```

Application options are set using the *set_app_options* command, and they can be specified globally or with respect to a block. To show the current value of an application option, use the *get_app_option* command. To show the value of multiple application options, use the *report_app_options* command.

See Also

- [legalize_placement](#)
- [check_legality](#)

place.legalize.enable_pin_via_region_check

Enable tool to validate the legality of the via_regions on pins.

Data Types

boolean

Default false

Description

When this option is set to true, pg_drc check will double check that pin access points determined by the pins' via_regions are indeed legitimate. By default the tool uses the via_region access points as a given. Setting this option to true prevents legalization from being derailed by errors in via_region generation, at a cost of possible runtime impact. Default value is false.

See Also

- [legalize_placement](#)
- [check_legality](#)

place.legalize.enable_prerouted_net_check

This app_option enables checking of cell placement against pre-routed nets, including Power and Ground nets, during legalization and legality checking. This is on by default in IC CompilerII. Disabling pre-routed net checking throughout the flow will likely introduce design rule errors and routing problems.

Data Types

Boolean

Default "true"

Description

Enable checking of cell placement against pre-routed net shapes.

Possible values are "true" or "false". Default is "true".

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design. Use *report_app_options* to show the value of application options.

place.legalize.enable_rectangular_od_shapes

Enables rectangular OD shape checking and not allow OD jog.

Data Types

Boolean

Default false

Description

When the application option is set to true, it only allow rectangular OD shapes and does not allow OD jog in the cell. When the option is false, it checks all OD shapes as default.

See Also

- [legalize_placement](#)
- [check_legality](#)

place.legalize.enable_safety_register_groups_dual_taps

Enable/disable enforced tap cell space reservation on both sides of TMR registers

Data Types

boolean

Default true

Description

When this option is set to true (the default), the advanced legalizer will automatically reserve enough space to place tap cells on both sides of TMR safety registers. If this option is set to false and tap cells are required around TMR registers, then neighboring registers can share a common tap cell between them.

See Also

- [legalize_placement](#)
- [check_legality](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)
- [create_safety_register_rule](#)
- [create_safety_register_group](#)

place.legalize.enable_same_net_pnet_check

This app_option enables pg_drc rule checking when the shapes are same net.

Data Types

Boolean

Default "false"

Description

Possible values are "true" or "false". Default is "false". This app_option enables pg_drc rule checking when the shapes are same net. e.g. the violation will be reported between PG stripe and cell PG pin which have same net owner.

place.legalize.enable_sav_rule

Enable SAV rule in PG DRC check in Advanced Legalizer.

Data Types

boolean

Default false

Description

The SAV rules is off by default in PG DRC for runtime concern. When this option is set to true, enable the rule in PG DRC check.

See Also

- [legalize_placement](#)
- [check_legality](#)

place.legalize.enable_threaded_advanced_legalizer

Enable/disable threading in the Advanced Legalizer.

Data Types

boolean

Default true

Description

When this option is set to true, it enables threading for the Advanced Legalizer. If you use the Advanced Legalizer, various phases of it will utilize threads up to the maximum specified with *set_host_options*. Different phases can use different numbers of threads.

See Also

- [legalize_placement](#)
- [set_host_options](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

place.legalize.enable_track_capacity_check

This app_option enables checking of cell placement such that routing access of all pins is considered simultaneously with respect to all nearby prerouted shapes. This check is

p

useful for 10nm and below technologies. This check is performed during legalization and legality checking. This is off by default in IC CompilerII.

Data Types

Boolean

Default "false"

Description

Enable checking of pin access of all cell's pins simultaneously.

Possible values are "true" or "false". Default is "false".

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design. Use *report_app_options* to show the value of application options.

place.legalize.enable_unspecified_boundary_rule_check

Enable/disable unspecified boundary rule check

Data Types

boolean

Default false

Description

When this option is set to true, it enables the distinguish of boundary cells which is unspecified in current boundary rule set during check_legality -chipfinishing.

Examples

The following example enable the check of unspecified boundary cell.

```
prompt> set_app_options\\
         -name place.legalize.enable_unspecified_boundary_rule_check
         -value true
```

See Also

- [check_legality](#)
- [set_boundary_cell_rules](#)
- [report_boundary_cell_rules](#)
- [set_tap_boundary_wall_cell_rules](#)

p

- [report_tap_boundary_wall_cell_rules](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

place.legalize.enable_variant_aware

Controls whether variant-aware legalization is enabled.

Data Types

Boolean

Default "false"

Description

Variant-aware legalization enables the legalizer to replace a cell that is placed in an illegal location with a variant cell.

-If the cell has no variants, it is moved to a legal location.
-If the cell has variants, the legalizer tries to replace the cell with a variant until it is legalized or there are no more variants to try.
-If cell overlapping happens after swapping in a different variant size, the legalizer performs another legalization.

To use variant-aware legalization, the cell library must contain cell groups that define the cell variants. The cell groups are defined during library preparation by using the *create_cell_group* command. The following example defines a cell group that has two variants:

```
create_cell_group -name ABC [get_lib_cells "*/$libName/ABC_1 */$libName/ABC_2"]
```

See Also

- [legalize_placement](#)
- [check_legality](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

place.legalize.enable_vertical_abutment_rules

Enable/disable vertical abutment rules for placement legalization and legality checking.

Data Types

boolean

Default false

Description

When this option is set to true, vertical abutment rules would be considered for placement legalization and legality check. This should only be used with designs that are prepped with vertical abutment information.

See Also

- [legalize_placement](#)
- [check_legality](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

place.legalize.enable_via_dpt_odd_cycle

Enable via odd cycle rule in PG DRC check in Advanced Legalizer.

Data Types

boolean

Default false

Description

If it's double pattern design and all via has no color, there might be same color via odd cycle violation. When this option is set to true, enable the rule in PG DRC check.

See Also

- [legalize_placement](#)
- [check_legality](#)

place.legalize.enable_via_ladder_checks

Enable/disable via ladder check in legality checker and placement legalizer.

Data Types

boolean

Default false

Description

When this option is set to true, EM and PMJ via ladder constraints are considered for legality checking and placement legalization.

See Also

- [legalize_placement](#)
- [check_legality](#)

place.legalize.extend_preroute_shape_search_space

Extend halo around the checked cell to search for prerouted net shapes.

Data Types

Boolean

Default False

Description

This application option directs the preroute net legality checker to extend the search space (halo) around the checked cell. By default the halo is one unit site width wide. The tool searches for prerouted net shapes that intersect this halo in order to perform legality checks during legalization and `check_legality`. Setting the value of this option to true, extends the search halo to be four times the unit site width.

Acceptable values are true or false (default).

Application options are set using the `set_app_options` command, and they may be specified globally or with respect to a design. Use `report_app_options` to show the value of application options.

See Also

- [legalize_placement](#)
- [check_legality](#)

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

place.legalize.filler1_continuous_vertical_cell_edge_for_exception_cells

Specifies module names of exception lib cells for vertically consecutive filler1 rule (That is, 1-site gap parallel run length constraint).

Data Types

string

Default ""

Description

Specifies module names exception lib cells for vertically consecutive filler1 rule (That is, 1-site gap parallel run length constraint). If there exists any cell instance of the lib cells specified in the application option, is next to the 1-site gap column whose parallel run length exceeds the threshold, then the rule violation is ignored/waived. The module names are by default auto completed. For example, the value "FILL1 TAP_FILL1" would be automatically treated as "FILL1* TAPFILL1*", so that lib cell names of "FILL1_ABC", "FILL16_DEFG", "TAP_FILL1_IJK", "TAP_FILL12_XYZ", and so on, are set as exception lib cells. If customers would like to disable the string autocomplete feature, enable the application option, *place.legalize.exception_cells_complete_regexp*, so that the autocomplete feature only works when customers *explicitly* put an asterisk at the end of a partial string.

If lib cells specified in the application option are successfully found and treated as exception lib cells in advanced legalizer, a few LGL-096 information should be shown in the rule engine initialization log of *check_legality*, *legalize_placement*, and so on, such as follows.

Information: Lib-cell 'FILL1_ABC (Consecutive 1X filler rule check)' is vertical stacked length exception lib-cell. (LGL-096)

Information: Lib-cell 'FILL116_DEFG (Consecutive 1X filler rule check)' is vertical stacked length exception lib-cell. (LGL-096)

On the other hand, any lib cell specified in the application option not found in advanced legalizer, would just be ignored.

p

Examples

The following example sets "FILL1_*" as the exception lib cells for vertically consecutive filler1 rule, so that lib cells "FILL1_SVT", "FILL1_HVT", "FILL1_LVT", and so on, are treated as exception lib cells for vertically consecutive filler1 rule.

```
prompt> set_app_options\\
        -name
place.legalize.filler1_continuous_vertical_cell_edge_for_exception_cells
        -value "FILL1_"
or
prompt> set_app_options\\
        -name place.legalize.exception_cells_complete_regexp -value true
prompt> set_app_options\\
        -name
place.legalize.filler1_continuous_vertical_cell_edge_for_exception_cells
        -value "FILL1_*
```

See Also

- [check_legality](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

place.legalize.filler_lib_cells

Specifies the filler cells used by the *check_legality* command.

Data Types

list

Default null

Description

The *place.legalize.filler_lib_cells* application option specifies the filler cells that are available for filling gaps in the current block. This set of filler cells is used by the *check_legality* command.

To ensure that the *check_legality* command checks all placed cell violations, you might need to specify boundary cells, such as floorplan edge cells, macro boundary cells, and end-of-row cells in addition to filler cells.

p

Specify the library cells as a Tcl list of cell names. For example,

```
set_app_options -name place.legalize.filler_lib_cells \\  
-value {DCAP_HVT DCAP_LVT DCAP_RVT}
```

See Also

- [check_legality](#)

place.legalize.force_clo_always_legal

Force CLO to check extra legality rules spacing/vt_cross_row_overlap for neighboring cells to prevent illegal placement after CLO.

Data Types

boolean

Default false

Description

Perform extra checks for neighboring cells in CLO. Apply this app_option when CLO leaves spacing or vt-related check_legality violations.

Acceptable values are true or false (default).

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design. Use *report_app_options* to show the value of application options.

See Also

- [legalize_placement](#)
- [check_legality](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

place.legalize.half_height_sitedef_name

Definition of the site def for half height filler cells

Data Types

string

Default empty

Description

Use this option to specify the name of the site def of half height filler cells. This app option is only considered by the advanced legalizer.

See Also

- [legalize_placement](#)
- [check_legality](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

place.legalize.half_row_offset_cell_legalization

Data Types

boolean

Default false

Description

Setting this application option as true tells Advanced Legalizer about the existence of half row offset cells in the design, and hence triggering handling of legalization of these cells. Cells are set as half row offset and standard using "half_row_offset_cell_sitedef_name" and "standard_cell_sitedef_name".

See Also

- [legalize_placement](#)
- [check_legality](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

place.legalize.half_row_offset_cell_sitedef_name

Data Types

string

Default ""

Description

This application option defines placement site name to which half row offset cells are associated with. When this option is set with the name of a valid NDM site definition, the cells having similar site def are considered half row offset for legalization.

See Also

- [legalize_placement](#)
- [check_legality](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

place.legalize.high_local_density_threshold

Data Types

double

Default 2.0

Description

This app option sets value for high local density threshold. This value will be the threshold exceeding which an area will be considered as having high local density. The default value is 2.0(200%). If the app option `place.legalize.enable_high_local_density_handling` is set during `legalize_placement`, then all the cells in the high local density region will remain unlegalized, whereas if not, set user will be warned about high local density area which could lead to high runtime.

See Also

- [check_legalizer_sanity](#)
- [legalize_placement](#)
- [check_legality](#)

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

place.legalize.honor_router_pin_connect_option

Honor pin connection constraints set by option
route.common.connect_within_pins_by_layer_name or library pin "connect_within_pin"
attribute.

Data Types

bool

Default false

Description

True setting of this option instructs the legalization DRC checker to take router constraint into consideration during legalization/legality checking. "Connect within pin" access violations are flagged if cell placement cannot satisfy its setting.

See Also

- [legalize_placement](#)
- [check_legality](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

place.legalize.ignore_pin_color_alignment_width_threshold_on_layers

Specifies the minimum cell metal (pin or routing blockage) width(s), where metal wider than the threshold will be waived for pin color alignment checks in placement on specified layer(s).

Data Types

list of pairs of {float string}

Default empty list

p

Description

Specifies the threshold widths for cell metal shapes (pins or routing blockages). Shapes wider than the maximum of the user-set threshold, the metal layer's "defaultWidth", and track's "width" parameters will be considered as shapes that do not need pin color alignment checks on the specified layer.

Application options are set using the `set_app_options` command, and they may be specified globally or with respect to a design. Use `report_app_options` to show the value of application options.

Examples

```
set_app_options -list
{place.legalize.ignore_pin_color_alignment_width_threshold_on_layers { {0.14 M2} {0.11 M3} }
```

See Also

- [legalize_placement](#)
- [check_legality](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

place.legalize.ignore_site_rows

Ignore all site rows of the given list of site defs

Data Types

list of names

Default empty list

Description

Use this option to instruct the advanced legalizer to ignore all site rows of the given site defs. Cells of the given site defs are also ignored for legalization and rule checking.

Examples

The following example shows how to ignore cells and rows of site defs unit140 and unit280:

```
prompt> set_app_options -name place.legalize.ignore_site_rows -value
{unit140 unit280}
```


See Also

- [legalize_placement](#)
- [check_legality](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)
- [report_site_defs](#)
- [get_site_rows](#)

place.legalize.ignore_unaligned_shapes_on_user_fixed_cells

Force Pin Color Alignment to check on pins only if the cells are user-fixed, allowing routing blockages within library cells to not align with tracks when pin track alignment is enabled.

Data Types

boolean

Default false

Description

By default if Pin Color Alignment rule engine is enabled, all metal shapes within a cell must align with a track, and if colored, the correct color track. This option allows user to loosen this restriction for routing blockages only on user-fixed cells. With this option library cells perform Pin Color Alignment checks on pins only. Library cells may have aligned and unaligned routing blockages. This option is used for legalization and legality checking.

Acceptable values are true or false(default).

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design. Use *report_app_options* to show the value of application options.

See Also

- [legalize_placement](#)
- [check_legality](#)
- [get_app_option_value](#)

- [report_app_options](#)
- [set_app_options](#)

place.legalize.include_user_specified_via_ladder

Enable/disable UCVP in candidates for via ladder check in legality checker and placement legalizer.

Data Types

boolean

Default false

Description

When this option is set to true, UCVP in candidates are considered for via ladder check.

See Also

- [legalize_placement](#)
- [check_legality](#)

place.legalize.large_displacement_threshold

Setup customized displacement threshold.

Data Types

Integer

Default 3

Description

User can define what is the large displacement instead of using tool's default large displacement threshold. Displacement report will be based on user's displacement threshold setup. The threshold is defined in cell row heights. For example, user may set 20 row heights as large displacement. This helps to filter out unwanted displacement messages.

User may consider to use another application option `place.legalize.report_displacement_limit` to change the maximum number of reported displacements.

p

Application options are set using the `set_app_options` command, and they may be specified globally or with respect to a design. Use `report_app_options` to show the value of application options.

See Also

- [legalize_placement](#)
- [get_app_option_value](#)
- [set_app_options](#)
- [report_app_options](#)

place.legalize.legalize_only_selected_cells

Direct legalization when run with `-cells` option, to only consider the selected cells, completely ignoring all other cells in the design.

Data Types

boolean

Default `false`

Description

When this option is set to true, `legalize_placement -cells` will legalize only the selected cells and ignore all other cells in the design. As a result it is possible that the placement of the selected cells will overlap unselected cells. By default the unselected cells are fixed in place. Default value is false.

See Also

- [legalize_placement](#)

place.legalize.legalizer_search_and_repair

In certain situations after the invocation of `legalize_placement`, violations remain. This app option allows users to force search-and-repair step of legalizer to try cleaning up these violation during post legalization. Note that this does not guarantee that all violations will get cleaned up.

Data Types

Boolean

Default `"false"`

p

Description

Enable/disable search-and-repair step of legalize placement.

Possible values are "true" or "false". Default is "false".

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design. Use *report_app_options* to show the value of application options.

place.legalize.libcell_based_color_shifting

Enables color shifting of the cell placement such that the colors of the cell-internal pins and metal shapes align with the routing tracks on the same metal layer with respect to each library cell.

Data Types

Boolean

Default false

Description

This application option enables color shifting of the cell-internal pins and metal shapes such that they align with routing tracks of the appropriate color with respect to each library cell. The checks would honor respective layers defined for each library cell

By default (*false*), this check is not library-cell-based.

When set to *true*, color shifting is performed based on each library cell when the following application options are also set:

- *place.legalize.enable_color_shifting*

This application option must be set to *true* to enable color shifting.

For example, to enable color shifting on layers based on each library cell, use the following commands:

```
set_app_options -list {place.legalize.enable_color_shifting true}
set_app_options -list {place.legalize.libcell_based_color_shifting true}
```

Application options are set using the *set_app_options* command, and they can be specified globally or with respect to a block. To show the current value of an application option, use the *get_app_option* command. To show the value of multiple application options, use the *report_app_options* command.

p

See Also

- [legalize_placement](#)
- [check_legality](#)

place.legalize.limit_legality_checks

Controls whether the legalizer exhaustively checks all locations for a cell.

Data Types

Boolean

Default false

Description

This application option controls whether the legalizer exhaustively checks all locations for a cell.

By default (*false*), the legalizer checks locations for a cell until it finds a legal site or has checked all sites in the design. In some cases, issues with a design cause a cell to be illegal in all locations in the design. It can be very expensive for the legalizer to exhaustively check all locations, and it is frustrating when the legalizer runs for a very long time only to give up because cells could not be legalized.

When this application option is set to *true*, the legalizer stops checking locations for a cell after 5000 failed attempts. This limits the legalization runtime, but might leave illegally placed cells at the end. The number of trials (5000 by default) can be changed using the app option *place.legalize.max_legality_failures*.

See Also

- [legalize_placement](#)
- [get_app_option_value](#)
- [set_app_options](#)
- [report_app_options](#)

place.legalize.max_legality_failures

Controls the number of legalization trials for a cell in case the checks are limited.

Data Types

Int

p

Default 5000**Description**

This application option controls the number of legalization trials for a cell in case the number of trials is configured to be limited. The number of trials is limited by enabling the app option *place.legalize.limit_legality_checks*.

By default, the legalizer checks 5000 locations for a cell in case *place.legalize.limit_legality_checks* is set to true. Use this app option to set the number of trials to a different value.

See Also

- [legalize_placement](#)
- [get_app_option_value](#)
- [set_app_options](#)
- [report_app_options](#)

place.legalize.min_max_prerouted_net_check_layers

Specifies the minimum and maximum metal layers to be included in pre-routed net checking.

Data Types

list of strings

Default empty list**Description**

Specifies the minimum and maximum metal layers to be included in pre-routed net checking. Pre-routed net checking is performed during detailed placement and legality checks. By default all metal layers are checked. With this app_option user can specify to check only the metal layers in the range given by minimum and maximum metal layers. The input layer names must be defined in the Technology file and must correspond to metal routing layers. The minimum layer is required to be less than or equal to the maximum metal layer. Using this option may result in DRCs caused by lack of checking on layers specified by the app_option. User should only use this option if s/he has contingency plans for resolving such DRCs. If option is used, check_legality will not report DRC violations on layers outside the specified range.

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design. Use *report_app_options* to show the value of application options.

Examples

```
set_app_options -list {place.legalize.min_max_prerouted_net_check_layers {M1 M3}}
```

See Also

- [legalize_placement](#)
- [check_legality](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

place.legalize.ndr_backside_pin_check_exclude_libpins

Provide list of library pin names to exclude from automatic checking of pin accessibility for pins with NDR constraints on backside in legality checker and placement legalizer.

Data Types

list_of_strings

Default empty list

Description

When this option is set with a list of library pin names, instances containing such pins are excluded from automatic NDR accessibility checking. NDR pin check is enabled by non-default app_option "place.legalize.enable_ndr_backside_pin_check".

place.legalize.ndr_pin_check_exclude_libpins

Provide list of library pin names to exclude from automatic checking of pin accessibility for pins with NDR constraints in legality checker and placement legalizer.

Data Types

list_of_strings

Default empty list

Description

When this option is set with a list of library pin names, instances containing such pins are excluded from automatic NDR accessibility checking. NDR pin check is enabled by non-default app_option "place.legalize.enable_ndr_pin_check".

See Also

- [legalize_placement](#)
- [check_legality](#)

place.legalize.num_tracks_for_access_check

Set the minimum number of unimpeded routing tracks intersecting a pin's via region for the pin to be considered legally placed.

Data Types

Integer

Default 1

Description

Sets the minimum number of unimpeded routing tracks intersecting a pin's via region for the pin to be considered legally placed. This setting is used by pin accessibility checking during legalization.

Acceptable values are integers greater than 0.

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design. Use *report_app_options* to show the value of application options.

See Also

- [legalize_placement](#)
- [check_legality](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

place.legalize.optimize_orientations

Setup orientation legalization

Data Types

Boolean

Default false

Description

When this is set to true, legalizer will consider flipping the orientations of cells in order to reduce displacements during legalization.

See Also

- [legalize_placement](#)
- [get_app_option_value](#)
- [set_app_options](#)
- [report_app_options](#)

place.legalize.optimize_pin_access

Enable/disable pin access optimization during legalization

Data Types

boolean

Default true

Description

When this option is set to true (the default), pin access optimization is performed at the end of legalization.

See Also

- [legalize_placement](#)
- [check_legality](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

place.legalize.optimize_pin_access_access_points

Enable/disable optimization to maximize the number of available access points on horizontal pin shapes

Data Types

boolean

Default false

Description

When this option is set to true, pin access optimization which is executed at the end of legalization will try to maximize the number of access points on pins with horizontal pin shapes. Alignment to next higher vertical tracks and blockage from PG straps is considered. Note that using this option will increase runtime during pin access optimization. This app option is only considered by the advanced legalizer.

See Also

- [legalize_placement](#)
- [check_legality](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

place.legalize.optimize_pin_access_allow_row_swap

Allow cells to move in neighboring rows for pin access optimization

Data Types

boolean

Default false

Description

When this option is set to true, pin access optimization which is executed at the end of legalization is allowed to move cells into neighboring rows. Using this option might increase the average displacement but can help to improve routability. This app option is only considered by the advanced legalizer.

See Also

- [legalize_placement](#)
- [check_legality](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

place.legalize.optimize_pin_access_cells_verbose

Print information about which cells are considered in incremental mode of pin access optimization

Data Types

boolean

Default false

Description

When this option is set to true, pin access optimization will print the names of cells which have been moved by the previous legalization step and hence are considered in the incremental mode. At most 100 cells are printed. This app option is only considered by the advanced legalizer.

See Also

- [legalize_placement](#)
- [check_legality](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

place.legalize.optimize_pin_access_drc_variants

Perform variant swapping in routing DRC regions during pin access optimization

Data Types

boolean

Default false

Description

When this option is set to true, pin access optimization performs same footprint variant swapping at given routing DRC marker locations to prevent routing DRCs. This app option is only considered by the advanced legalizer.

See Also

- [legalize_placement](#)
- [check_legality](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

place.legalize.optimize_pin_access_ignore_lowest_routing_layer

Restrict the selection of routing layers for pin alignment during pin access optimization as part of the advanced legalizer.

Data Types

Boolean

Default false

Description

Use this app option to disable the lowest routing layer during pin access optimization. This app option is supported only by the advanced legalizer. For some advanced nodes, a routing layer is defined below the lowest input and output pin layers. You can use this option to ignore that lowest layer for pin access optimization such that only the next higher vertical and possibly horizontal layers are used for pin alignment.

Examples

The following example disables the lowest routing layer:

```
prompt> set_app_options \\  
        -name place.legalize.optimize_pin_access_ignore_lowest_routing_layer  
        \\  
        -value true
```

See Also

- [legalize_placement](#)
- [set_ignored_layers](#)
- [get_app_option_value](#)
- [set_app_options](#)
- [report_app_options](#)

place.legalize.optimize_pin_access_incremental

Enables or disables the incremental mode of pin access optimization

Data Types

boolean

Default true

Description

When this option is set to true (the default), pin access optimization which is executed at the end of legalization in incremental mode. The incremental mode allows only the movement and change of cells that have been moved or altered in orientation or its variant by the legalization step. In case more than 80% of the cells have been moved or altered then all cells are considered by pin access optimization.

See Also

- [legalize_placement](#)
- [check_legality](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

place.legalize.optimize_pin_access_same_row_variants

Enables swapping of lib-cell variants to improve alignment of horizontal pins

Data Types

boolean

Default false

Description

When this option is set to true, pin access optimization will perform only lib-cell variant swapping to increase the alignment of horizontal pin shapes on the same horizontal track for neighboring cells. Orientation Flipping and swapping of neighbor cell locations are also performed. This app option is only considered by the advanced legalizer.

p

See Also

- [legalize_placement](#)
- [check_legality](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

place.legalize.optimize_pin_access_strategies

Enable additional optimizations for pin access optimization as part of the advanced legalizer.

Data Types

list_of_string

Default ""

Description

Use this app option to enable additional optimizations during pin access optimization. This app option is supported only by the advanced legalizer. Three additional option values are available. They can all be specified at the same time. The first optimization "horizontal_align" is applicable for technologies where the standard cells have horizontal pin shapes. Pin access optimization will try to vertically align the horizontal pin shapes with vertical or horizontal pin shapes of cells that are in neighboring cell rows. The second optimization triggered by the app option value "avoid_high_pin_density" will move cells to nearby locations which have a high pin density and whose top and bottom neighboring cells won't allow any extensions of vertical pin shapes to increase the number of access points. These cell placements are typically very difficult or impossible to route for the detail router. The last value is "wide_variants". This option performs wide variant swapping on a routed design. Cells which have a wider variant are supposed to have a lower pin density. The legalizer will switch to a wider variant in regions with lower layer DRC markers. Those DRCs typically indicate pin access problems which might get fixed by swapping to a wider variant lib cell.

The default of this app option is to disable all three optimizations.

Examples

The following example enables the first two optimizations:

```
prompt> set_app_options \\  
-name place.legalize.optimize_pin_access_strategies \\  
-value "horizontal_align avoid_high_pin_density"
```

See Also

- [legalize_placement](#)
- [get_app_option_value](#)
- [set_app_options](#)
- [report_app_options](#)

place.legalize.optimize_pin_access_using_cell_spacing

Enable that cell legalization optimizes pin access using cell spacing.

Data Types

Boolean

Default "false"

Description

Use this app option to enable that cell legalization optimizes pin access using cell spacing. This feature will redistribute the locally available space in areas with high pin densities, with the goal of improving routability.

See Also

- [get_app_option_value](#)
- [set_app_options](#)
- [report_app_options](#)

place.legalize.optimize_pin_access_verbose

Enable/disable verbose mode for pin access optimization during legalization

Data Types

boolean

Default false

Description

When this option is set to true, pin access optimization at the end of legalization prints statistics about aligned pins and some relevant technology data.

See Also

- [legalize_placement](#)
- [check_legality](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

place.legalize.output_violated_info

Controls whether the *check_legality* command writes legality errors to the error database.

Data Types

Boolean

Default true

Description

By default (*true*), the *check_legality* command write all legality violations to the error database. You can use the error browser in the GUI to view and interact with these violations in the layout view.

See Also

- [check_legality](#)

place.legalize.output_violated_info_file

Controls whether the *check_legalita* command writes out a Tcl file.

Data Types

Boolean

Default false

Description

This application option controls whether the *check_legalita* command writes out a Tcl file that can be used by the *legalize_placement* or *change_selection* command or with existing scripts.

By default (*false*), the *check_legality -verbose* command only shows the violated cells in the log file.

p

When this application option is set to *true*, the *check_legality* also writes a Tcl file named *check_legality_violations.tcl.pid*, where *pid* is the process ID of the current session. If you rerun the *check_legality* command, it overwrites the Tcl file with the latest information.

If you source a Tcl file generated in a previous session, the tool issues the following warning message: "The tcl file is generated by different session so the content in the tcl file may be out of date." You should generate the Tcl file within same session to keep the information up-to-date.

You should not source a generated Tcl file with many violations directly, as the runtime might be long. If you want to legalize all violated cells and keep the existing legal cells unchanged, use the *legalize_placement -incremental* command, which first runs the *check_legality* command to get all violated cells and then performs legalization on the violated cells while keeping all legal cells unchanged.

The generated Tcl file contains all violated cells based on different categories. The default setting includes all types of violations and all moveable cells. You can change this by modifying two variables in the Tcl file, *GLOBAL_VIOLATION_TYPE* and *GLOBAL_VIOLATION_MODE*. For information about the valid values for these variables, see the header section at the beginning of the Tcl file.

When you source the Tcl file, it creates a variable named *CHECK_LEGALITY_VIOLATED_CELLS* that contains the violated cells. You can use this variable to legalize the violated cells (*legalize_placement -cells \$CHECK_LEGALITY_VIOLATED_CELLS*), select the cells in the GUI (*change_selection \$CHECK_LEGALITY_VIOLATED_CELLS*), or customize it with existing scripts. For example, if you want to show only moveable cells with an offrow violation, change the setting of the *GLOBAL_VIOLATION_TYPE* variable to Offrow and the setting of the *GLOBAL_VIOLATION_MODE* variable to Moveable, and then source the Tcl file. In this case, the *CHECK_LEGALITY_VIOLATED_CELLS* variable will include all moveable cells with an offrow violation.

This app option is only applicable for classic legalizer. It is not valid for Advanced Legalizer.

See Also

- [legalize_placement](#)

place.legalize.pg_blockage_specs

This is to let user set PG blockage constraints for specific via ladder rule, libCell, layer and overlap ratio.

Data Types

string

p

Default N/A**Description**

PG blockage is an idea to improve runtime of via ladder check. It allows user to specify how much PG can overlap against via ladder pin bbox. Instead of precise checking, legalizer could quickly move cell away from PG region to save runtime. Syntax Example `set_app_options -name place.legalize.pg_blockage_specs -value \{{VR:VPEM_M1_M4_1_4_2_1 M11 0.4}} \{{VR:VPEM_M1_M4_1_4_2_1 M3 0.0}} \{{LIBCELL:HDBULT08_FSDNR2PQB_LBSKY2_12 M2 0.0}}`

See Also

- [legalize_placement](#)
- [check_legality](#)

place.legalize.pin_alignment_for_clock_mesh_layer

Consider special layer for clock net in user_define rule.

Data Types

String

Default ""**Description**

Basically user_define only consider power and ground net for now. This option is an additional usage in user_define rule which will include the defined layer and check clock net on that layer. The usage:

```
set_app_options -name place.pin_alignment_for_clock_mesh_layer \\  
-value {M1}
```

place.legalize.pin_color_alignment_layers

Specifies the metal layers on which to perform pin color alignment checking.

Data Types

list

Default {}

p

Description

This application option specifies the routing metal layers on which to perform pin color alignment checking. Specify the layers by using the layer names from the technology file.

This check is performed during legalization and legality checking when both of the following application options are set to *true*:

- *place.legalize.enable_pin_color_alignment_check*
- *place.legalize.enable_prerouted_net_check*

For example, to enable pin color alignment checking on the m1 and m2 metal layers, use the following commands:

```
set_app_options -list {place.legalize.enable_pin_color_alignment_check
  true}
set_app_options -list {place.legalize.enable_prerouted_net_check true}
set_app_options -list {place.legalize.pin_color_alignment_layers {m1 m2}}
```

Application options are set using the *set_app_options* command, and they can be specified globally or with respect to a block. To show the current value of an application option, use the *get_app_option* command. To show the value of multiple application options, use the *report_app_options* command.

See Also

- [legalize_placement](#)
- [check_legality](#)

place.legalize.pin_color_alignment_width_threshold

Specifies the minimum cell metal (pin or routing blockage) width, where metal wider than the threshold will be considered "fat" for pin color alignment checks in placement.

Data Types

float

Default 0.0

Description

Specifies the threshold width for cell metal shapes (pins or routing blockages). Shapes wider than the maximum of the user-set threshold, the metal layer's "defaultWidth", and track's "width" parameters will be considered "fat". Shapes that are "fat" are aligned to opposite color tracks. Those that are not "fat" are aligned to same color tracks.

p

Application options are set using the `set_app_options` command, and they may be specified globally or with respect to a design. Use `report_app_options` to show the value of application options.

Examples

```
set_app_options -list {place.legalize.pin_color_alignment_width_threshold 0.4}
```

See Also

- [legalize_placement](#)
- [check_legality](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

place.legalize.pin_color_alignment_width_threshold_on_layers

Specifies the minimum cell metal (pin or routing blockage) width(s), where metal wider than the threshold will be considered "fat" for pin color alignment checks in placement on specified layer(s).

Data Types

list of pairs of {float string}

Default empty list

Description

Specifies the threshold widths for cell metal shapes (pins or routing blockages). Shapes wider than the maximum of the user-set threshold, the metal layer's "defaultWidth", and track's "width" parameters will be considered "fat" on the specified layer. Shapes that are "fat" are aligned to opposite color tracks. Those that are not "fat" are aligned to same color tracks.

Application options are set using the `set_app_options` command, and they may be specified globally or with respect to a design. Use `report_app_options` to show the value of application options.

Examples

```
set_app_options -list {place.legalize.pin_color_alignment_width_threshold_on_layers {{0.4 M0} {0.5 M1} {0.1 M3}}}
```

See Also

- [legalize_placement](#)
- [check_legality](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

place.legalize.post_route_feedback_threshold_for_pin_access

Specifies the maximum number of DRCs to enable PBPAC post-route mode.

Data Types

integer

Default 500

Description

Specifies the maximum number of DRCs to enable PBPAC post-route mode.

Too many DRCs may seriously increase PBPAC runtime in legalization due to unnecessary and pessimistic cell movements. Tool will automatically turns off PBPAC post-route mode when the number of DRCs is greater than the threshold.

If the number of the last mile DRCs are still bigger than the threshold, the automatic turn-off will deprive tool of a chance to fix the remaining DRCs. In that case, set the threshold to bigger than the number of DRCs, or set it to 0, where 0 means no auto turn-off of PBPAC post-route mode.

Note that the DRCs mentioned in this document do not mean all types of DRCs, but are limited to the following DRCs PBPAC can fix:

End of line spacing,

Short

Diff net via-cut spacing

Color conflicts between shapes

End extension for via enclosure

Less than minimum area

Forbid cut metal overlap

Diff net spacing

Two via region forbidden metal

Examples

```
set_app_options -list {place.legalize.post_route_feedback_threshold_for_pin_access  
1500}
```

See Also

- [legalize_placement](#)
- [check_legality](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

place.legalize.preclump

Setup pre-clumping of multi-row cells

Data Types

Boolean

Default false

Description

Set this app option to true to enable pre-clumping of multi-row cells. This clumps together multi-row cells so as to avoid small gaps between them. The small gaps are often not usable for placing single-row cells so should be avoided in order to legalize at high density.

Pre-clumping may degrade the displacement of multi-row cells while improving the achievable density. Use `place.legalize.preclump_threshold_sites` to control the amount of clumping.

See Also

- [legalize_placement](#)
- [get_app_option_value](#)
- [set_app_options](#)
- [report_app_options](#)

place.legalize.preclump_threshold_sites

Setup pre-clumping legalization

Data Types

Integer

Default 10

Description

Set this app option to control the gap width limit for pre-clumping. Gaps of width less than the threshold will be removed by pre-clumping. A higher threshold causes more aggressive pre-clumping and greater degradation in displacement of multi-row cells, but it may be worth it in order to legalize at high density. The threshold is in units of site widths.

See Also

- [legalize_placement](#)
- [get_app_option_value](#)
- [set_app_options](#)
- [report_app_options](#)

place.legalize.preroute_shape_merge_distance

Set the maximum distance in user units between two or more preroute shapes, such that the legalizer considers them as a single shape.

Data Types

Float

Default 0.0

Description

Sets the maximum distance in user units between two or more preroute shapes, such that the legalizer considers them as a single shape. The default value is 0.0, which corresponds to no shape merging. This setting is used by pin accessibility checking during legalization.

Acceptable values are floating point numbers greater than 0.

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design. Use *report_app_options* to show the value of application options.

See Also

- [legalize_placement](#)
- [check_legality](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

place.legalize.prerouted_net_check_exclude_lib_cells

Specifies the list of library cells to exclude from pre-routed net checking.

Data Types

list of strings

Default empty list

Description

Specifies the list of library cells to exclude from pre-routed net checking. Pre-routed net checking is performed during detailed placement and legality checks. With this app_option user can mark instances of specific library cells as not requiring the checking. Using this option will likely result in DRCs caused by placement of these instances. User should only use this option if s/he has contingency plans for resolving such DRCs. If option is used, check_legality will not report DRC violations on instances of the specified library cells.

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design. Use *report_app_options* to show the value of application options.

Examples

```
set_app_options -list {place.legalize.prerouted_net_check_exclude_lib_cells {AOI22  
AOI12}}
```

See Also

- [legalize_placement](#)
- [check_legality](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

place.legalize.reduce_conservatism_in_eol_check

Use less conservative values for endOfLine check.

Data Types

Boolean

Default false

Description

Use less conservative version of endOfLine spacing (1 neighbor v. 2 neighbor) for End-of-line spacing checking of cell internal metal and pins vis-a-vis prerouted PG shapes. This option is not appropriate for all library cell and design styles and should only be used with caution.

Acceptable values are true or false(default).

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design. Use *report_app_options* to show the value of application options.

See Also

- [legalize_placement](#)
- [check_legality](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

place.legalize.report_displacement_limit

Setup customized displacement message in the log.

Data Types

Integer

Default 10

Description

User can control how many displacement messages are printed in the run log instead of using tool's default value. For example, user may want to set 1000 so show until 1000 displacement messages of cells.

p

User may consider to use another application option `place.legalize.large_displacement_threshold` to control the large displacement limit.

Application options are set using the `set_app_options` command, and they may be specified globally or with respect to a design. Use `report_app_options` to show the value of application options.

See Also

- [legalize_placement](#)
- [get_app_option_value](#)
- [set_app_options](#)
- [report_app_options](#)

place.legalize.report_libcell_displacement_limit

Enable reporting of lib-cell specific displacements.

Data Types

Integer

Default 0

Description

Use this app option to enable the reporting of lib-cell specific displacement at the end of legalization. The displacement is reported as an average per cell for a given lib-cell in user units. This app option takes as value an integer number. 0 (the default) means that no lib-cell based reporting is printed. A value larger than 0 defines the number of lib-cells reported.

Two tables are printed. The first table is sorted according to the largest average displacement per lib-cell. The second table only considers cells with a displacement larger than 3 row heights. The table is sorted according to the number of cells per lib-cell.

Application options are set using the `set_app_options` command, and they may be specified globally or with respect to a design. Use `report_app_options` to show the value of application options.

See Also

- [legalize_placement](#)
- [get_app_option_value](#)

- [set_app_options](#)
- [report_app_options](#)

place.legalize.return_tcl_errors

Data Types

boolean

Default true

Description

Returns error if legalize_placement command fails. If set to false, will return ok even if the legalize_placement fails.

See Also

- [legalize_placement](#)
- [check_legality](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

place.legalize.revert_redundant_min_area_patching

Revert unnecessary shape extensions emulated in legalization that could already meet minArea rule without the patching

Data Types

boolean

Default false

Description

When this option is set to true, pg_drc check will check if the shape extensions is necessary or not. If the extension is unnecessary, the extension/patching is reverted. Default value is false.

p

See Also

- [legalize_placement](#)
- [check_legality](#)

place.legalize.safety_enables_advanced_legalizer

Automatically enable the Advanced Legalizer in case functional safety (FuSa) constraints are defined on the design.

Data Types

boolean

Default true

Description

When this option is set to true (the default), then the Advanced Legalizer will be used even if it is not enabled by the app option *place.legalize.enable_advanced_legalizer*. The classic legalizer does not support FuSa constraints so the Advanced Legalizer has to be applied. You can use this app option to disable the automatic enabling of the Advanced Legalizer.

See Also

- [legalize_placement](#)
- [check_legality](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

place.legalize.specify_clo_window_size_on_libcells

This app_option specifies custom CLO window size configurations for library cells. It supports both exact name matching and wildcard pattern matching using '*' characters. Note that the original CLO window is controlled by the app_option 'place.legalize.max_cellmap_displacement_rows', whose default value is 4. Also, the specified value is capped between 1 and 8, to avoid too-few solutions and excessive runtime.

Data Types

list of pairs of strings

p

Default Empty list**Description**

The value of the `app_option` is a list of string pairs, where the first string is libcell name and the second is the CLO window size (in integer). This `app_option` impacts the CLO behavior.

Examples

To specify the CLO window size for libCell_A to 3 (row-height), libCell_B to 5, and libCell_C to 7:

```
set_app_options -name place.legalize.specify_clo_window_size_on_libcells
-value { {libCell_A 3} {libCell_B 5} {libCell_C 7} }
```

As another example, the below Tcl corresponds to the below Log (when CLO is executed in the flow).

Tcl:

```
set_app_options -name
place.legalize.specify_clo_window_size_on_libcells -value
{ {*_FSDPQ_CBDY2_* 30} {HDBULT*_ND*_D* 2} }
```

Log:

```
Warning: The CLO window size for libcell HDBULT11_FSDPQ_CBDY2_1 is set
to 30, but it should be between 1 and 8. Adjusting to 8. (LGL-488)
```

```
Information: The CLO window size for libcell HDBULT11_FSDPQ_CBDY2_1 is
set to 8.
```

```
Warning: The CLO window size for libcell HDBULT11_FSDPQ_CBDY2_4 is set
to 30, but it should be between 1 and 8. Adjusting to 8. (LGL-488)
```

```
Information: The CLO window size for libcell HDBULT11_FSDPQ_CBDY2_4 is
set to 8.
```

```
Warning: The CLO window size for libcell HDBULT08_FSDPQ_CBDY2_1 is set
to 30, but it should be between 1 and 8. Adjusting to 8. (LGL-488)
```

```
Information: The CLO window size for libcell HDBULT08_FSDPQ_CBDY2_1 is
set to 8.
```

```
Warning: The CLO window size for libcell HDBULT08_FSDPQ_CBDY2_4 is set
to 30, but it should be between 1 and 8. Adjusting to 8. (LGL-488)
```

```
Information: The CLO window size for libcell HDBULT08_FSDPQ_CBDY2_4 is
set to 8.
```

```
Information: The CLO window size for libcell HDBULT11_ND4_D1_1 is set
to 2.
```

```
Information: The CLO window size for libcell HDBULT11_ND3_D1_1 is set
to 2.
```

```
Information: The CLO window size for libcell HDBULT08_ND4_D1_1 is set
to 2.
```

```
Information: The CLO window size for libcell HDBULT08_ND3_D1_1 is set
to 2.
```

place.legalize.standard_cell_sitedef_name

Data Types

string

Default ""

Description

This application option defines placement site name to which general standard cells are associated with. When this option is set with the name of a valid NDM site definition, the cells having similar site def are considered standard for legalization.

See Also

- [legalize_placement](#)
- [check_legality](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

place.legalize.stream_effort

Setup stream placement effort

Data Types

Boolean

Default low

Description

Set this app option to high to enable extra exploration within the stream placement algorithm. This improves the accuracy of stream placement and may result in better displacements at the expense of runtime.

See Also

- [legalize_placement](#)
- [get_app_option_value](#)

- [set_app_options](#)
- [report_app_options](#)

place.legalize.stream_effort_limit

Setup stream placement limit

Data Types

Integer

Default Inf

Description

Use this app option to set a limit on the runtime of stream placement. This comes at the expense of displacement. Stream placement handles the worst cells first. If the limit is reached, stream placement will stop improving cells. The remaining high-displacement cells will not have their displacement improved by stream placement.

The limit is not measured in units of time but in units of effort relative to design size. As a rule of thumb, set the limit to 10 for fast runtime at the expense of displacement. Set it to 100 for reasonable runtime and reasonable displacement.

See Also

- [legalize_placement](#)
- [get_app_option_value](#)
- [set_app_options](#)
- [report_app_options](#)

place.legalize.stream_place

Perform stream placement

Data Types

Boolean

Default false

Description

Set this app option to true to enable stream placement. Stream placement improves the largest displacements in the design. For each large-displacement cell, it attempts to move

p

out other cells to make room, such that a single very large displacement is replaced by a stream of small displacements.

Stream placement can be expensive in runtime if there are regions of high density where many cells are displaced by long distances. See `place.legalize.stream_effort_limit` to control runtime.

See Also

- [legalize_placement](#)
- [get_app_option_value](#)
- [set_app_options](#)
- [report_app_options](#)

place.legalize.support_min_area_constraint

Direct legalization to model router shape extensions necessary to meet minArea rule as defined in the technology file

Data Types

boolean

Default false

Description

When this option is set to true, `pg_drc` check will model router shape extensions on layers where the metal shape does not meet the minArea requirement as defined in the technology file. Default value is false.

See Also

- [legalize_placement](#)
- [check_legality](#)

place.legalize.support_off_track_via_region

Enable `pg_drc` checks of advanced legalizer to properly support pins which have `via_regions` that do not overlap routing tracks.

Data Types

bool

p

Default false**Description**

True setting of this option instructs the legalization DRC checker to take into consideration pin via_regions that do not overlap routing tracks during legalization/legality checking. Via_regions are set during library cell creation. For some combinations of technology DRC rules and library cell metal shape configuration it is not possible to drop a routing via on a pin directly on a routing track. For such pins, the via_region is considered "off-track". PG_drc check, by default, checks whether it is possible to drop a via on track. This option instructs the tool to check for off-track connections as well.

See Also

- [legalize_placement](#)
- [check_legality](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

place.legalize.tap_cover_drop_edges

Specifies the power type(s) of top/bottom boundary edge(s) to be dropped for tap coverage violations.

Data Types*string***Default** "VDD VSS"**Description**

Specifies the power type(s) of top/bottom boundary edge(s) to be dropped for tap coverage violations. The application option is for advanced legalizer (AL) latch-up violation checker (check_legality -chipfinishing [hard | all]). Total 4 values are available: "", "VDD", "VSS", and "VDD VSS". The value means the power type of top/bottom boundary edge(s) that tap_coverage violation can be ignored and hidden. The boundaries can be core area boundaries, VA boundaries, hard placement blockage boundaries, and so on so forth. For example, if "VDD VSS" are specified, all the tap coverage violations on the PWELL/ NWELL touching top/bottom boundary edges are not shown.

p

Examples

The following example sets the power type of top/bottom boundary edge(s) as VDD to be dropped for tap coverage violations.

```
prompt> set_app_options\\  
        -name place.legalize.tap_cover_drop_edges -value "VDD"
```

See Also

- [check_legality](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

place.legalize.use_eol_spacing_for_access_check

Use endOfLine spacing parameters when checking pin access.

Data Types

Boolean

Default false

Description

Use endOfLine spacing parameters instead of minSpacing (default) when determining whether a via can be dropped on a pin in a way that will not violate spacing between it and an existing PG pre-route (via or strap). This setting is used by pin accessibility checking during legalization.

Acceptable values are true or false(default).

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design. Use *report_app_options* to show the value of application options.

See Also

- [legalize_placement](#)
- [check_legality](#)
- [get_app_option_value](#)

p

- [report_app_options](#)
- [set_app_options](#)

place.legalize.use_nll_query_cm

This application option enables checking of advanced node rules during optimization using similar engine as the legalizer.

Data Types

Boolean

Default "false"

Description

Enable checking of advanced node rules during optimization using similar engine as legalizer during optimization. The check will be enabled for all optimizations done during place_opt, clock_opt and route_opt. Some of the examples of advance node rules are layer VTH, coloring rule etc.

Possible values are "true" or "false". Default is "false".

Please note that advanced node rule checks are done when this application option is not set too. This option enables a new engine which does more comprehensive checks.

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design. Use *report_app_options* to show the value of application options.

place.legalize.user_rule_check_layers

Set layers for user_define rule in Advanced Legalizer

Data Types

string

Default ""

Description

When using user_define in advanced legalizer, users need to set all layers in this option. The layers need to be aligned with those defined in pattern file.

place.legalize.user_rule_pattern_file

Set pattern file for user_define rule in Advanced Legalizer

Data Types

string

Default ""

Description

When using user_define in advanced legalizer, users need to set the pattern file.

place.legalize.via0_alignment_grid_name

Enable alignment of standard cells to named via0 grid in legality checker and placement legalizer.

Data Types

string

Default ""

Description

The via0 grid must be created by the user with "create_grid" command, and the name of the grid provided as input to this application option. When this option is set with the name of a valid NDM grid object, via0 alignment is considered for legality checking and placement legalization. By default via0 alignment is not enabled.

See Also

- [legalize_placement](#)
- [check_legality](#)
- [create_grid](#)

place.legalize.via_ladder_auto_stagger

Specifies whether via ladder insertion is free to stagger by any amount on any layer to achieve a successful insertion in legalizer via ladder check.

Data Types

boolean

p

Default false**Description**

Specifies whether via ladder insertion is free to stagger by any amount on any layer to achieve a successful insertion.

The option only impacts attempts to insert via ladder template types that have no user-set staggering values. If a template has user-set values, we will honor those.

I.e. auto stagger is only active for via ladder templates defined in the tech file with an empty or zeroed maxNumStaggerTracksTbl, or for Tcl-defined templates with empty or zeroed max_num_stagger_tracks_table . Staggering for other templates would occur just as it does today, regardless of the option setting.

When auto stagger is active for an insertion attempt, the tool will analyze the track configurations and layout around the pin to determine if staggering should be enabled on a level of the via ladder and by how much. This relieves the need for a user to specify any staggering values in the template.

See Also

- [legalize_placement](#)
- [check_legality](#)

place rp.allow_non_rp_cells

Allows non relative placement cells in the unused areas of relative placement groups.

Data Types

Boolean

Default false**Description**

When *true*, non relative placement cells are allowed in the unused areas of relative placement groups during coarse placement and refine placement. When *false* (the default), non relative placement cells are not allowed in the unused spaces of relative placement groups. Note that non relative placement cells are allowed in unused spaces during legalization and optimization irrespective of *place rp.allow_non_rp_cells* is set to *true* or *false*.

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design. To determine the current value of this application option on a specific design, use *get_app_option_value -block \$block -name*

p

place_rp.allow_non_rp_cells. Use *report_app_options* to show the value of multiple application options.

This application option is obsolete. An option *allow_non_rp_cells* is added in command *set_rp_group_options* to allow non relative placement cells in the unused spaces of the relative placement group.

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)
- [set_clock_gating_check](#)
- [set_rp_group_options](#)

place_rp.allow_non_rp_cells_on_blockages

Allows overlap of non relative placement cells with relative placement blockages and unused spaces.

Data Types

Boolean

Default false

Description

When *true*, non relative placement cells are allowed to overlap with relative placement blockages during coarse placement, refine placement, legalization and optimization. Please note that non relative placement cells are allowed in unused spaces of relative placement groups too. When *false* (the default), non relative placement cells can not overlap with relative placement blockages.

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design. To determine the current value of this application option on a specific design, use *get_app_option_value -block \$block -name place_rp.allow_non_rp_cells_on_blockages*. Use *report_app_options* to show the value of multiple application options.

This application option is obsolete. An option *allow_non_rp_cells_on_blockages* is added in command *set_rp_group_options* to allow non relative placement cells to overlap with the blockages of the relative placement group.

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)
- [set_clock_gating_check](#)
- [set_rp_group_options](#)

place.rp.disable_rp_legalization

Disables RP legalization when set to true.

Data Types

Boolean

Default false

Description

When *true*, disables the relative placement legalization step during *legalize_placement* and *legalize_rp_groups* command.

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

place.rp.disable_rp_two_pass_flow

Disable RP two pass flow irrespective of any constraint on the RP groups.

Data Types

Boolean

Default false

Description

When *true*, RP two pass flow will be disabled irrespective of any constraint on the RP groups but it may degrade QOR. This app option is intended to be used during flow establishment where efficient runtime is the key parameter.

p

Application options are set using the `set_app_options` command, and they may be specified globally or with respect to a design. To determine the current value of this application option on a specific design, use `get_app_option_value -block $block -name place_rp.allow_non_rp_cells`. Use `report_app_options` to show the value of multiple application options.

This application option is obsolete. An option `allow_non_rp_cells` is added in command `set_rp_group_options` to allow non relative placement cells in the unused spaces of the relative placement group.

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)
- [set_clock_gating_check](#)
- [set_rp_group_options](#)

place_rp.enable_horizontal_cell_shifting

Modifies the behavior of placement of cells in the RP groups.

Data Types

Boolean

Default false

Description

While placing or legalizing rp groups, if `place_rp.enable_horizontal_cell_shifting` is set to *true* then cells within rp groups will not be offset vertically around obstacles such as fixed cells, instead rp cells will be forced to shift horizontally to accommodate these obstacles and the rp groups will not leave vertical gaps between rp rows.

See Also

- [set_rp_group_options](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

place.rp.force_user_orients_only

Tries only for the user specified cell orientations during placement.

Data Types

Boolean

Default false

Description

If *place.rp.force_user_orients_only* is set to *TRUE* then, only orients specified by the user will be tried during cell placement for that particular cell on which orient has been specified. In order to cater the orientation specified by the user, the cell may offset a little in X/Y direction if it's not legaled at the intended location.

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)
- [set_clock_gating_check](#)
- [set_rp_group_options](#)

place.rp.hard_allow_non_rp_cells

Avoids non relative placement cells in the unused areas of relative placement groups during legalization.

Data Types

Boolean

Default false

Description

When *true*, non relative placement cells are not allowed in the unused areas of relative placement groups during legalization. When *false* (the default), non relative placement cells are allowed in unused spaces during legalize_placement.

If, a relative placement group has *allow_non_rp_cells* attribute as *true* then, *allow_non_rp_cells* attribute will have higher priority compared to *place.rp.hard_allow_non_rp_cells* App Option.

p

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)
- [set_rp_group_options](#)

place rp.hierarchical_place_around_fix_cells

Allows placement of sub hierarchy relative placement group around fix cells.

Data Types

Boolean

Default false

Description

When *true*, sub hierarchy relative placement groups are placed around the fixed cells depending upon the command option *place_around_fixed_cells* value on it's group, which is set using command *set_rp_group_options*. When *false* (the default), sub hierarchy relative placement groups are placed around fixed cells based on the command options *place_around_fixed_cells* value on top relative placement group.

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design. To determine the current value of this application option on a specific design, use *get_app_option_value -block \$block -name place rp.hierarchical_place_around_fix_cells*. Use *report_app_options* to show the value of multiple application options

See Also

- [set_rp_group_options](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)
- [set_clock_gating_check](#)

place.rules.min_od_filler_size

User can define what is the minimum OD filler cell size that is available to be used as filler cells. The size is specified in number of sites.

Data Types

Integer

Default 2

Description

Set minimum OD filler cell size in number of sites

Examples

Setting the minimum OD filler size to 1 site

```
set_app_options -list {place.rules.min_od_filler_size 1}
```

place.rules.min_tpo_filler_size

User can define what is the minimum TPO filler cell size that is available to be used as filler cells. The size is specified in number of sites.

Data Types

Integer

Default 1

Description

Set minimum TPO filler cell size in number of sites

Examples

Setting the minimum TPO filler size to 1 site

```
set_app_options -list {place.rules.min_tpo_filler_size 1}
```

place.rules.min_vt_filler_size

User can define what is the minimum VT filler cell size that is available to be used as filler cells. The size is specified in number of sites.

Data Types

Integer

p

Default 1**Description**

Set minimum VT filler cell size in number of sites

Examples

Setting the minimum VT filler size to 1 site

```
set_app_options -list {place.rules.min_vt_filler_size 1}
```

place.rules.vertical_abutment_mode

This app_option sets the mode for vertical abutment constraint generation to be used during legalization and legality checking. User has option of three modes - 1. "user" which relies solely on user provided values of attributes "vertical_abut_pattern_top" and "vertical_abut_pattern_bottom" that are set on library cells and the value of app_option "vertical_abutment_forbidden_pairs"; 2. "auto" which generates constraints automatically by processing all library cells; and 3. "merge" which combines both the user-supplied constraints and the automatically generated ones. This option is only valid if vertical abutment rules are enabled with app_option "place.legalize.enable_vertical_abutment_rules".

Data Types

String

Default "user"**Description**

Set vertical abutment rule generation mode..

Possible values are "user", "auto" or "merge". Default is "user".

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design. Use *report_app_options* to show the value of application options.

place.utilization.multi_row_penalty

Specifies the amount of extra width to add to multi-row cells when performing utilization calculations.

Data Types

float

p

Default 0.0**Description**

This app option tries to compensate for the difficulty in legalization caused by the presence of multi-row cells which leads to wasted space. It causes the optimization steps to consider multi-row cells to be wider than their actual width when computing chip and local utilization. One-half the amount is applied to each side of the cell. The addition is applied the cell boundary itself and not to any keepouts on the cell. If the keepout on a side is larger than the extra width added to the side, then this app option will have no effect there. It may affect the other side if the keepout is smaller or not present.

Examples

Apply a width penalty of 0.1 user units to all multi-row cells.

```
set_app_options -name place.utilization.multi_row_penalty -value 0.1
```

place_opt.final_place.effort**Data Types**

string

Default medium**Description**

Specifies the CPU effort level for the final coarse placement invoked in *place_opt*. Allowed values are *low*, *medium*, or *high*.

See Also

- [create_placement](#)
- [place_opt](#)

place_opt.flow.clock_aware_placement

Enables ICG timing criticality driven clock-aware placement in *place_opt*.

Data Types

Boolean

Default "false"

p

Description

This option controls whether `place_opt` will utilize clock-aware placement, guided by ICG's enable timing criticality. With this option, `place_opt` will try to improve ICG enable timing (measured after `clock_opt`) by placing the timing critical ICGs and their fanout cells at better locations for ICG enable paths. This option can be combined with `place_opt` ICG optimization flow (`place_opt.flow.optimize_icgs`).

Application options are set using the `set_app_options` command, and they may be specified globally or with respect to a design. Use `report_app_options` to show the value of application options.

See Also

- [place_opt](#)
- [report_app_options](#)
- [set_app_options](#)

place_opt.flow.cts_icg_resynthesis

Enable ICG resynthesis flow in `place_opt`

Data Types

Boolean

Default false

Description

`place_opt` will run clockgate resynthesis prior to final timing-driven placement. It includes timing aware clockgate merging, reclustering, splitting and relocation, which are designed to optimize for latency and dynamic power.

The resynthesis optimization calculates clock latency internally and does not require switching the Timer clock mode from ideal to propagated.

Subsequent clockgate optimizations in `clock_opt/build_clock` are incremental and will fine-tune the decisions made in `place_opt`.

See Also

- [set_app_options](#)
- [place_opt](#)

p

place_opt.flow.dcg_spg_mode

Enables the enhanced SPG flow for improved QoR and correlation.

Data Types

Boolean

Default true

Description

This option has been deprecated and it will be removed in subsequent releases.

This application option enables the enhanced SPG flow for improved QoR and correlation. For this option to have an effect, the *place_opt.flow.do_spg* application option must be set to *true*.

To use the previous SPG flow or a flow that involves a tool other than Design Compiler Graphical, set the *place_opt.flow.dcg_spg_mode* application option to *false*.

See Also

- [set_app_options](#)

place_opt.flow.do_path_opt

Runs path_opt in and after place_opt

Data Types

Boolean

Default false

Description

This option enables running path_opt in and after place_opt for better timing QOR. False is the default.

See Also

- [place_opt](#)
- [set_app_options](#)

place_opt.flow.do_spg

Run the SPG flow inside place_opt.

p

Data Types*Boolean***Default** false**Description**

This option has been deprecated and it will be removed in subsequent releases.

`place_opt` will run the SPG flow that takes in a netlist that has gone through SPG flow in Design Compiler and optimizes it further. It assumes the netlist has been placed and high fanout nets have been buffered, and skips `initial_place` and `initial_drc` steps.

See Also

- [set_app_options](#)

place_opt.flow.enable_ccd

Enables concurrent clock and data optimization (CCD) during `place_opt`.

Data Types*Boolean***Default** true**Description**

When this option is set to *true*, concurrent clock and data optimization (CCD) will be enabled during `place_opt`. The data path optimizations are interleaved with useful skew computations at the early stage of `final_opto`. The skew offsets are forwarded to `clock_opt` stage in the form of clock balance points, for finalizing clock trees in `clock_opt`. When `place_opt.flow.trial_clock_tree` is set to *true*, `place_opt` CCD integration will be based on propagated early clock tree. In this case, `clock_opt`-like setups are recommended (such as `clock_opt` scenarios, uncertainties, NDRs, etc). Otherwise, `place_opt` CCD will work on ideal clocks based on `set_clock_latency`, if any, provided by user. The useful skew computation incrementally updates the existing user offsets (both balance points and clock latencies). If user input for initial skew offsets are provided, it is highly recommended that the value for `set_clock_balance_points` is matched with `set_clock_latency`. For example, if a user ideal latency for a certain clock sink is -200ps, when all other clock sinks have 0ps (tool default), which means the 200ps faster clock arrival for the clock sink pin, it should have +200ps as `clock_balance_points`, to instruct CTS to implement 200ps faster arrival for the pin. In this way a better correlation between ideal and propagated clock is expected. The magnitude of skew offsets added by `place_opt` CCD are bounded by `place_opt.flow.enable_ccd_useful_skew_max_postpone` and `place_opt.flow.enable_ccd_useful_skew_max_prepone`.

See Also

- [place_opt](#)
- [clock_opt](#)

place_opt.flow.enable_ccd_fmax_flow**Data Types***boolean***Default** false**Description**

Specifies whether the *place_opt* command enables the CCD Fmax flow. When this application option is set to true, CCD in *place_opt* does a skewing aware of the data path optimization potential, puts more focus to optimize using useful skew optimization the violations which cannot be optimized efficiently by data optimization. It also reduces the useful skew overhead on the clock tree in terms of latency or clock area/power by coming up with the offsets that are only really needed to improve the overall timing. If this application option is set to false, CCD Fmax flow is disabled.

See Also

- [place_opt](#)

place_opt.flow.enable_dpa_ccd_power

Enables Data Path Aware (DPA) Concurrent Clock Data (CCD) optimization for power during *place_opt*.

Data Types*boolean***Default** false**Description**

The *place_opt.flow.enable_dpa_ccd_power* application option controls whether the Data Path Aware (DPA) Concurrent Clock Data (CCD) flow for power optimization is enabled during *place_opt*. If *true*, DPA CCD for power is enabled.

During DPA CCD for power, useful skew is also used to improve the power of data paths. Using this flow increases the number of clock tree sinks which are skewed, but this allows

p

data path optimization during *place_opt* to provide further power reduction that would not be possible otherwise.

See Also

- [place_opt](#)

place_opt.flow.enable_dpa_ccd_timing**Data Types***boolean***Default** false**Description**

Specifies whether the *place_opt* command enables the DPA CCD timing feature. If this app option is set to true, data-path optimization over-optimizes all positive paths in order to provide extra borrowable slack for paths that are limiting the frequency. If this app option is set to false, DPA CCD timing will be disabled.

See Also

- [place_opt](#)

place_opt.flow.enable_dpa_ccd_timing_with_delay_potential**Data Types***boolean***Default** false**Description**

Specifies whether the *place_opt* command enables the DPA CCD timing feature with the delay potential prediction. If this app option is set to true, a delay potential predictor is used to identify the paths with positive slack that can further be optimized. Data-path over-optimizes only the positive paths with delay potential, as they can be easily closed, to provide extra borrowable slack for paths that are limiting the frequency. If this app option is set to false, DPA CCD timing with delay potential will be disabled.

See Also

- [place_opt](#)

place_opt.flow.enable_dps

Enables Dynamic Power Shaping (DPS) during *place_opt*.

Data Types

Boolean

Default false

Description

When this option is set to *true*, Dynamic Power Shaping optimization (DPS) will be enabled during *place_opt*. This will reduce transient power by utilizing usefull skew. The skew offsets are forwarded to *clock_opt* stage in the form of clock balance points, for finalizing clock trees in *clock_opt*.

See Also

- [place_opt](#)

place_opt.flow.enable_fast_cus_in_final_opto

Data Types

boolean

Default false

Description

This application option enables several recipe changes in *place_opt* *final_opto* flow, which improve the elapsed time. The changes increase the incrementality and interleaving of skew optimization with datapath optimization. The resulting flow trajectory might be different, but timing and power are expected to be on par by the end of the routing optimization stage.

See Also

- [place_opt](#)

place_opt.flow.enable_incremental_multibit_banking

Controls banking of sequential cells to multibit sequential cells during the *final_opto* stage of the *place_opt* command.

Data Types

Boolean

Default false

Description

This application option controls banking of single bit and multibit sequential cells to multibit sequential cells during the *final_opto* stage of the *place_opt* command. The tool also modifies the netlist and does scan chain related changes.

Note that the *place_opt.flow.enable_multibit_banking* application option must also be set to *true* to enable the incremental banking.

See Also

- [place_opt](#)

place_opt.flow.enable_irap

Enables IR aware placement (IRAP) during *place_opt*.

Data Types

Boolean

Default false

Description

When this option is set to *true*, IR aware placement optimization (IRAP) will be enabled during *place_opt*. This will reduce the IR drop by reducing power density in high IR drop spots.

See Also

- [place_opt](#)

place_opt.flow.enable_multibit_banking

Enable banking of sequential cells to multi-bit sequential cells.

Data Types

Boolean

Default false

p

Description

This app option allows users to enable banking of single bit and multi bit sequential cells to multi-bit sequential cells during *place_opt*. The tool also modifies the netlist and does scan chain related changes. The banking is being done during *initial_opto* stage.

See Also

- [place_opt](#)

place_opt.flow.enable_multibit_debanking

Enable debanking of multibit sequential cells to improve total negative slack.

Data Types

Boolean

Default false

Description

This app option allows users to enable debanking of multibit sequential cells to improve total negative slack. The debanking is done to lower width multibit and/or single bit sequential cells. The tool also modifies the netlist and does scan chain related changes. The debanking is being done during *final_opto* stage of comamnd *place_opt*.

See Also

- [place_opt](#)

place_opt.flow.enable_multibit_rewiring

Enable rewiring to swap the nets on bits of multibit cells to improve usage of mixed drive strength and mixed VT cells.

Data Types

Boolean

Default false

Description

This app option allows users to enable rewiring among bits of multibit cells to increase the number of mixed drive strength cells and mixed VT cells. Such kind of cells have few bits with higher drive strength and other with with lower drive strength. Usage of such cells leads to better total power because of lower power consumption on lower drive strength

p

bits. The target of swapping bits is to drive critical nets or bits having higher load with high drive strength or higher VT bits, reducing need of going to a multibit cell with all higher drive strength bits. The rewiring is done both during timing optimization and during power/area optimization. The benefits of this feature will be seen mostly in the designs having mixed drive strength or mixed VT multibit lib cells in the library.

See Also

- [place_opt](#)

place_opt.flow.enable_multisource_clock_trees

Enables integrated tap assignment and structural multisource clock tree synthesis in `place_opt`

Data Types

Boolean

Default `false`

Description

When true, enables integrated multisource clock tree synthesis within *place_opt* internally. MSCTS option sets are picked up from `set_multisource_clock_tap_options` for Regular MSCTS and `set_multisource_clock_subtree_options` for Structural MSCTS. While using this feature, care must be taken that all required multisource CTS related setup such as global clock tree (Htree) creation, clock mesh creation, annotation on first level net and pin(s), tap/subtree driver insertion, definition of options for tap assignment and structural subtree synthesis, removing constraints like `user_dont_touch`, `fixed`, etc. to allow cell cloning and required CTS related settings are provided before *place_opt*. Integrated Regular MSCTS: With integrated tap assignment, *place_opt* can see the clock structure changes done as part of tap assignment especially ICG clones, and perform improved placement and optimization. Integrated tap assignment can be performed in ideal or propagated (trial clock tree and `optimize_icgs`) modes. All features honoured by *synthesize_multisource_clock_taps* and *place_opt* are supported in integrated regular MSCTS. In this flow, there is no requirement to run standalone *synthesize_multisource_clock_taps*. The database after *place_opt* with integrated tap assignment can be taken directly to `clock_opt build_clock`. Integrated Structural MSCTS: With integrated subtree synthesis, *place_opt* can see the clock structure changes done as part of structural subtree synthesis, especially ICG clones, and perform improved optimization. Integrated structural subtree synthesis can be performed only in ideal mode. All steps of *synthesize_multisource_clock_subtrees* and features honoured by *place_opt* are supported in integrated structural MSCTS. In integrated SMSCTS in *place_opt flow with CCD*, *offsets derived by CCD during final_opto can be implemented by a subsequent round of subtree optimization after*

p

place_opt. This can be performed in an integrated manner by *clock_opt build_clock* command by setting the app_option *clock_opt.flow.enable_multisource_clock_trees* to true. Alternatively, this incremental round of subtree synthesis can be done by a call to *synthesize_multisource_clock_subtrees*, which will operate incrementally and automatically start from optimize step.

See Also

- [place_opt](#)
- [synthesize_multisource_clock_taps](#)

place_opt.flow.enable_power

Data Types

boolean

Default true

Description

Specifies whether the *place_opt* command enables power optimization. By default, power type is determined based on scenario settings as shown in the table below. If this app option is set to false, power optimization will be disabled regardless of scenario settings.

Leakage-scenario Dynamic-scenario -> FlowType

False False -> Area

True False -> Leakage

False True -> Total

True True -> Total

See Also

- [place_opt](#)

place_opt.flow.enable_self_gating

Enable self-gating in *place_opt* to improve power

Data Types

Boolean

Default false

Description

Self-gating is an optimization technique used to reduce dynamic power consumption by turning off the clock signal of registers during clock cycles when the data remains unchanged. This app option allows users to enable self gating step in `place_opt` to improve power. The self-gating step will be enabled with the app option set to true and also with active dynamic scenarios.

The names of new cells created by self-gating in `place_opt` do not include the prefix specified by option `opt.common.user_instance_name_prefix`, as the naming conventions defined for clock gating optimizations are followed instead.

See Also

- [place_opt](#)

place_opt.flow.estimate_clock_gate_latency

Enable automatic clock gate latency estimation in the `place_opt` flow.

Data Types

Boolean

Default "true"

Description

This app option is *deprecated in version 2019.12-SP2* and will be removed in a future release. It will be on by default.

This option controls whether clock gate latency estimation is automatically performed after certain steps in the `place_opt` flow. With automatic clock gate latency estimation, `set_clock_latency -offset` assertions will be generated that reflect the estimated clock tree latency. It will make the `place_opt` flow aware of the reduced timing budget for clock gate enable paths. It is an alternative to manual `set_clock_latency` annotation at clock gate input pins or to manual `set_clock_gate_latency` settings. The annotated latencies will automatically be kept in sync with the state of the clock tree in the design.

In order to have accurate estimations, it is important to setup CTS constraints like routing-rules and buffer selection before `place_opt`. The latency estimator will consider these settings when doing the estimations.

Clock gate latency estimations will take place after major logical or physical clock reconstruction steps. These steps include placement, multi-source CTS and multi-bit banking. Enabling clock gate latency estimations will also enable clock gate latency aware placement if app option `place.coarse.clock_gate_latency_aware` was set to "auto".

p

Note that clock gate latency estimation is incompatible with pre-CTS trial tree construction because trial tree construction forces clock timing to be in propagated mode. Therefore clock gate latency estimation will not be performed when app option `place_opt.flow.trial_clock_tree` or `place_opt.flow.optimize_icgs` is set.

To inspect the applied offset latencies, please use `write_script` since offset latencies are not listed in the output of `write_sdc`.

Application options are set using the `set_app_options` command, and they may be specified globally or with respect to a design. Use `report_app_options` to show the value of application options.

See Also

- [place_opt](#)
- [write_script](#)
- [report_app_options](#)
- [set_app_options](#)

place_opt.flow.hold_area_budgeting

Enable the enhanced Hold Area Budgeting feature in the `place_opt` final_place stage

Data Types

string

Default false

Description

This application option enables the enhanced Hold Area Budgeting feature during *place_opt*. This feature models and reserves the amount of space required for hold fixing during `clock_opt` in the `place_opt` stage itself, thus creating a more convergent flow. Valid values are "false" and "enhanced".

See Also

- [place_opt](#)

place_opt.flow.merge_clock_gates

Enable ICG merging optimization during `place_opt`.

p

Data Types

Boolean

Default `true`

Description

When this app_option is set to *true*, ICG merging (*merge_clock_gates*) will be run internally inside *place_opt* initial_place stage. Here clock gate merging happens as a first step of initial_place. In case of SPG flow, ICG merging happens as a first step in *place_opt* initial_opto stage. If the design flow has already performed ICG merging, it will be skipped inside *place_opt*. If you are running the *place_opt* command in Fusion Compiler, the default value for this app-option is *false*.

See Also

- [place_opt](#)
- [merge_clock_gates](#)

place_opt.flow.optimize_icgs

Enable ICG optimization in place_opt

Data Types

Boolean

Default `false`

Description

place_opt will run automatic ICG optimization that performs trial clock synthesis, timing aware ICG splitting for critical enable paths, and clock-aware placement.

The trial clock tree synthesis is used regardless of the value of *place_opt.flow.trial_clock_tree*, for critical ICG enable paths identification and all the optimization steps in *place_opt*. Note that the end-of-*place_opt* timing QoR cannot be directly comparable to conventional ideal clock based *place_opt* QoR, and the trial clock tree is automatically removed and re-synthesized by *clock_opt* or *synthesize_clock_trees*.

The *optimize_icgs* flow does not merge ICGs, however, user can optionally run *merge_clock_gates* before running *place_opt*.

The ICG oriented clock-aware placement is automatically used regardless of the value of *place_opt.flow.clock_aware_placement*, which will further help ICG enable path timing improvement.

p

The aggressiveness of ICG splitting can be controlled by *place_opt.flow.optimize_icgs_critical_range*.

In order to maximize the benefit from the well-correlated trial clock tree in *place_opt*, apply as much clock-related settings as possible for *place_opt*, such as clock NDRs or post-clock uncertainties.

Top-level design support is limited. With read-only sub-blocks, ICG optimization will be limited to *current_block*, and *place_opt* will immediately exit with an Error message if modifiable sub-blocks are detected.

See Also

- [set_app_options](#)
- [place_opt](#)

place_opt.flow.optimize_icgs_critical_range

Critical range for ICG optimization in *place_opt*

Data Types

Float

Default 0.75

Description

This is a sub option for *place_opt.flow.optimize_icgs*, whose acceptable values are between 1.0 and 0.0. When set to X, only ICGs whose EN violations are within $\{EN_WNS, EN_WNS \cdot (1-X)\}$ will be considered for splitting. The larger the value is, the more aggressive ICG splitting is expected by including more ICGs for splitting consideration.

See Also

- [set_app_options](#)
- [place_opt](#)

place_opt.flow.optimize_icgs_use_timing_driven_splitting

Enable timing-driven ICG optimization in *place_opt*

Data Types

Boolean

Default true

p

Description

This is a sub option for *place_opt.flow.optimize_icgs*. This option will only take effect when *place_opt.flow.optimize_icgs* is enabled in the *place_opt* flow. When set to true, more runtime efficient timing-driven ICG splitting will be used in the ICG optimization flow in *place_opt*.

See Also

- [set_app_options](#)
- [place_opt](#)

place_opt.flow.optimize_layers

Enables layer optimization in *place_opt*

Data Types

String

Default false

Description

This option has been deprecated; this option will be removed in a subsequent release.

However, the *place_opt* command always performs layer optimization that is tightly integrated with buffering, which ensures that critical signals are promoted to low-RC layers and buffered to take advantage of those layers.

See Also

- [set_app_options](#)

place_opt.flow.optimize_layers_critical_range

Set critical range for layer optimization when *place_opt.flow.optimize_layers* is set to true.

Data Types

Float

Default 0.0001

Description

It accepts values from {0 ~ 1.0}. For a given value *y*, it optimizes nets that have slacks between {*y**WNS, WNS}, where WNS is the worst negative slack. Set it to a lower value

p

to fix paths that are in less critical range. It may cause a lot more nets to be promoted and use much more routing resource. So, lower the critical range only if necessary.

See Also

- [set_app_options](#)

place_opt.flow.optimize_layers_overlap_bins

Enable overlap bins when *place_opt.flow.optimize_layers* is set to true or auto.

Data Types

Boolean

Default false

Description

When the switch is enabled, layer bins can be overlapped. For example: {M10, M9}, {M9, M8}, {M8, M7}, {M7, M6}, {M6, M5} When it is disabled, layer bins do not overlap. For example: {M10, M9}, {M8, M7}, {M6, M5}

The feature might help improve QoR when routing resources is unbalanced. For example, if M10 routing capacity is low, but M9 is high. Enable overlap bin: Nets can be promoted to {M9, M8} to improve QoR. Disable overlap bin: Nets promote to {M8, M7}

See Also

- [set_app_options](#)

place_opt.flow.optimize_ndr

Enable NDR optimization in place_opt

Data Types

Boolean

Default false

Description

Place_opt flows will run automatic non-default-routing rule optimization that assigns long timing critical nets to NDR to improve timing.

See Also

- [set_app_options](#)

place_opt.flow.optimize_ndr_critical_range

Set critical range for Non-default routing (NDR) based optimization when *place_opt.flow.optimize_ndr* is set to true.

Data Types

Float

Default 0.7

Description

It accepts values from {0 ~ 1.0}. For a given value *y*, it optimizes nets that have slacks between {*y**WNS, WNS}, where WNS is the worst negative slack. Set it to a lower value to fix paths that are in less critical range. It may cause a lot more nets to have NDRs and use much more routing resource. So, lower the critical range only if necessary.

See Also

- [set_app_options](#)

place_opt.flow.optimize_ndr_max_nets

Set maximum number of nets to be routed with Non-default routing (NDR) rule when *place_opt.flow.optimize_ndr* is set to true.

Data Types

integer

Default 2000

Description

During NDR-based optimization, the number of maximum nets to be assigned NDRs is given by this value. This number limits the number of potential candidates that can benefit from applying NDRs, and must be a non-negative integer.

See Also

- [set_app_options](#)

place_opt.flow.remove_dr_shapes

deletes the detail route shapes on the optimizable nets during compile flow

Data Types

Boolean

Default true

Description

If turned to false, compile flow will not delete the detail route shapes on the optimizable nets

See Also

- [set_app_options](#)

place_opt.flow.repeatability

place_opt will trade off repeatability for some runtime speedup.

Data Types

Boolean

Default true

Description

If turned to false, place_opt will run a little faster, although the QoR at the end of the place_opt flow will no longer be repeatable. It is not recommended to set this to false, since the runtime difference is not substantial.

See Also

- [place_opt](#)
- [set_app_options](#)

place_opt.flow.trial_clock_tree

Enables early clock tree synthesis in the *place_opt* command.

Data Types

string

Default false

p

Description

This option controls whether the *place_opt* command performs early clock tree synthesis, which is useful for low-power placement and the integrated clock gating (ICG) optimization flow.

Valid values are *false* (default) and *true*.

When set to *true*, the *place_opt* command performs early clock tree synthesis, and then uses the trial clock tree structures during placement and optimization and uses the propagated clocks throughout the *place_opt* flow. Therefore, the *place_opt* QoR is not directly comparable to conventional ideal clock based *place_opt* flows.

Note that when the *place_opt.flow.optimize_icgs* application option is *true*, the *place_opt* command performs early clock tree synthesis regardless of the setting of the *place_opt.flow.trial_clock_tree* application option.

The trial clock tree is automatically removed and resynthesized by the *clock_opt* or *synthesize_clock_trees* command, where the finalized clock trees are built.

To maximize the benefit from the well-correlated trial clock tree in the *place_opt* command, apply as many clock-related settings, such as clock nondefault routing rules or post-clock uncertainties, as possible before running the *place_opt* command.

See Also

- [place_opt](#)
- [report_app_options](#)
- [set_app_options](#)

place_opt.initial_drc.global_route_based

Run global route based buffering (GRopto flow) during the initial_drc place_opt stage.

Data Types

integer

Default 0

Description

This app option enables the GRopto flow, which enables the place_opt initial_drc stage, which does high fanout net synthesis, to work with globally routed nets to determine the topology for buffering the trees. After buffering, all global routes are deleted, and the modified topologies will anchor buffers and guide downstream optimization. Currently, the

p

GRopto flow only affects the *place_opt* initial_drc stage of buffering. To enable the flow, set the app option value to 1.

place_opt.initial_place.buffering_aware

Runs buffering-aware timing-driven placement for the initial placement step inside the *place_opt* command.

Data Types

Boolean

Default true

Description

The *place_opt* command will run buffering-aware timing-driven placement to generate a better initial placement for later timing optimization steps in the *place_opt* command. Buffering-aware timing-driven placement performs timing-driven placement using a timing model that estimates the effect of later buffering of long nets and high-fanout nets. The netlist is not changed.

See Also

- [set_app_options](#)

place_opt.initial_place.effort

Data Types

string

Default high

Description

Specifies the CPU effort level for the initial coarse placement invoked in *place_opt*. This effort option also controls both coarse placements in two-pass initial placement, if *place_opt* is directed to use a two-pass initial placement. Allowed values are *low*, *medium*, or *high*.

See Also

- [create_placement](#)
- [place_opt](#)

place_opt.initial_place.two_pass

Runs the two-pass flow for initial placement step inside the *place_opt* command.

Data Types

Boolean

Default false

Description

The *place_opt* command will run a two-pass flow to generate a better initial placement. The two-pass flow performs wirelength-driven placement, followed by DRC fixing and another timing-driven placement.

See Also

- [set_app_options](#)

place_opt.place.congestion_effort

Data Types

string

Default medium

Description

Specifies the effort level for the congestion alleviation in *place_opt*. Allowed values are *none*, *medium*, or *high*. Expect a significant increase in runtime for high effort. If set to *none*, *place_opt* will not run in a congestion mode.

See Also

- [create_placement](#)
- [place_opt](#)

plan.3dic.bump_alignment_threshold

Specifies the alignment threshold of two bumps center.

Data Types

Float

Default -1.0

p

Description

This option allows user to specify the alignment threshold in user units between two bumps center. In the 3DIC design, the two vertical neighboring dies may have different technology, the bumps in these two dies may be sanpped to different grid, then the center of bumps in a bump pair that between these two dies may have slight misalignment. If the value of app option is negative, the default threshold will be used with the value that calculated by `grid_resolution/length_precision`, otherwise the specified threshold value will be used. `check_3d_design` will check the misalignment of bump pair to see if it is less than this threshold, if not, `check_3d_design` will report a violation. User can set this app option with proper value to let `check_3d_design` ignore this kind of misalignment. The default value is -1.0.

Application options are set using the `set_app_options` command, and they may be specified globally or with respect to a design. Use `report_app_options` to show the value of application options. This application option is design specific.

See Also

- [check_3d_design](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

plan.3dic.bump_model

Specifies the bump cell model that is used in current 3dic design.

Data Types

Enum

Default "net"

Description

This option specifies the bump cell model that is used in current 3dic design. The valid values are "net", "terminal". The default value is "net".

p

There are two kinds of bump model are supported:

- Net-based: Bump cell pin is connected to a logical net. Bump cells are written into an output Verilog netlist.
- Terminal-based: Die terminal is created that is linked to the bump's layer shape. Terminal is associated with a port: Terminal – Port – Bump. Bump cells are physical only, and not written into the netlist.

See Also

- [set_app_options](#)

plan.3dic.chip_enclosure

Specifies the enclosure (edge_to_edge) range between two vertical neighboring chips.

Data Types

unsigned float pair

Default {0.0 MAX_DISTANCE}

Description

This option specifies the enclosure (edge_to_edge) range between two vertical neighboring chips. *check_3d_design* will calculate the chip enclosure distance between two vertical neighboring chips in the 3d top design, if the distance is not in the range app option specified, the command will report a violation. User can set this app option with proper range to let *check_3d_design* do this kind of check. The default range is {0.0 MAX_DISTANCE}.

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design. Use *report_app_options* to show the value of application options. This application option is design specific.

See Also

- [check_3d_design](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

plan.3dic.chip_horizontal_misalignment

Specifies the horizontal misalignment(center_to_center) range between two chips with the same z_level.

Data Types

unsigned float pair

Default {0.0 0.0}

Description

This option specifies the horizontal misalignment(center_to_center) range between two chips with the same z_level. *check_3d_design* will calculate the horizontal misalignment center-to-center distance between two chips with the same z_level, if the distance is not in the range app option specified, the command will report a violation. User can set this app option with proper range to let *check_3d_design* do this kind of check. The default range is {0.0 0.0}.

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design. Use *report_app_options* to show the value of application options. This application option is design specific.

See Also

- [check_3d_design](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

plan.3dic.chip_horizontal_spacing

Specifies the horizontal spacing range between two chips with the same z_level.

Data Types

unsigned float pair

Default {0.0 0.0}

Description

This option specifies the horizontal spacing range between two chips with the same z_level. *check_3d_design* will calculate the horizontal spacing distance between two chips with the same z_level, if the distance is not in the range app option specified, the

p

command will report a violation. User can set this app option with proper range to let *check_3d_design* do this kind of check. The default range is {0.0 0.0}.

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design. Use *report_app_options* to show the value of application options. This application option is design specific.

See Also

- [check_3d_design](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

plan.3dic.chip_vertical_misalignment

Specifies the vertical misalignment(center_to_center) range between two chips with the same *z_level*.

Data Types

unsigned float pair

Default {0.0 0.0}

Description

This option specifies the vertical misalignment(center_to_center) range between two chips with the same *z_level*. *check_3d_design* will calculate the vertical misalignment center-to-center distance between two chips with the same *z_level*, if the distance is not in the range app option specified, the command will report a violation. User can set this app option with proper range to let *check_3d_design* do this kind of check. The default range is {0.0 0.0}.

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design. Use *report_app_options* to show the value of application options. This application option is design specific.

See Also

- [check_3d_design](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

plan.3dic.chip_vertical_spacing

Specifies the vertical spacing range between two chips with the same `z_level`.

Data Types

unsigned float pair

Default {0.0 0.0}

Description

This option specifies the vertical spacing range between two chips with the same `z_level`. *check_3d_design* will calculate the vertical spacing distance between two chips with the same `z_level`, if the distance is not in the range app option specified, the command will report a violation. User can set this app option with proper range to let *check_3d_design* do this kind of check. The default range is {0.0 0.0}.

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design. Use *report_app_options* to show the value of application options. This application option is design specific.

See Also

- [check_3d_design](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

plan.3dic.error_physical_contact

Specify error scenario for physical contact pair.

Data Types

Enum

Default ""

Description

This option specifies the error scenario for physical contact pair. The option value is honored by *check_3d_design*. The option value format is {{bump_type bump_type} {bump_type bump_type} ...}, the valid bump_type value is: dummy, probe, pg, signal. User can specify multiple pair values for this option, for example {{dummy pg} {dummy signal}}.

p

Examples

The following example specify the error scenario that a dummy bump physical contacts with a pg bump, `check_3d_design` will flag this error scenario.

```
prompt> set_app_options -name plan.3dic.error_physical_contact -value  
{{dummy pg}}
```

See Also

- [set_app_options](#)

plan.3dic.forbidden_spacing_of_connected_bump

Specifies the min Manhattan spacing distance between bumps with connected nets and die zone.

Data Types

unsigned float

Default 0.0

Description

This option specifies the min Manhattan spacing distance between bumps with connected nets and die zone. `check_3d_design` will compare the Manhattan spacing distance between the boundary of bump with connected nets and boundary of die, if the distance is less than the value app option specified, the command will report a violation. User can set this app option with proper value to let `check_3d_design` do this kind of check. The default value is 0.0.

Application options are set using the `set_app_options` command, and they may be specified globally or with respect to a design. Use `report_app_options` to show the value of application options. This application option is design specific.

See Also

- [check_3d_design](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

plan.3dic.forbidden_spacing_of_unconnected_bump

Specifies the min Manhattan spacing distance between bumps with unconnected nets and die zone.

Data Types

unsigned float

Default 0.0

Description

This option specifies the min Manhattan spacing distance between bumps with unconnected nets and die zone. *check_3d_design* will compare the Manhattan spacing distance between the boundary of bump with connected nets and boundary of die, if the distance is less than the value app option specified, the command will report a violation. User can set this app option with proper value to let *check_3d_design* do this kind of check. The default value is 0.0.

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design. Use *report_app_options* to show the value of application options. This application option is design specific.

See Also

- [check_3d_design](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

plan.3dic.ignore_logic_net

Ignore logic0 and logic1 type net in *check_3d_design* and *assign_3d_interchip_nets*.

Data Types

boolean

Default true

Description

This option specifies if the logic0 and logic1 type net will be checked and assigned. *check_3d_design* will ignore the logic0 and logic1 type net check in *check_3d_design*, *assign_3d_interchip_nets* will also ignore the logic0 and logic1 type net to do net

p

assignment. If the value of app option was set to false, logic0 and logic1 type nets will be checked and assigned. User can set this app option with proper value to let *check_3d_design* implement this kind of check and *assign_3d_interchip_nets* implement net assignment. The default value is true.

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design. Use *report_app_options* to show the value of application options. This application option is design specific.

See Also

- [check_3d_design](#)
- [assign_3d_interchip_nets](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

plan.3dic.max_bump_cluster_size

Specifies the maximum {width height} for the bump cluster.

Data Types

unsigned float pair

Default {0.0 0.0}

Description

This option specifies the maximum {width height} for the bump cluster. *check_3d_design* will check the width and height of all bump clusters, if the length is not in the range app option specified, the command will report a violation. User can set this app option with proper range to let *check_3d_design* do this kind of check. The default value is {0.0 0.0}.

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design. Use *report_app_options* to show the value of application options. This application option is design specific.

See Also

- [check_3d_design](#)
- [get_app_option_value](#)

p

- [report_app_options](#)
- [set_app_options](#)

plan.3dic.min_bump_shape_spacing

Specifies the min distance to the shape of bump.

Data Types

Pair list

Default NA.

Description

This option allows user to specify the min distance in user units to the shape of bump. When command '*create_bump_array*', '*create_bond_pad_array*' and '*commit_pseudo_bumps*' set the command option '-min_spacing' and it includes 'bump' element. During the process of these commands, these commands will check if the distance between any new shape of new created bump cells/bond pads and existed bump/bond pad shape is less than the specified app option value with the same layer. The new bump cell/bond pad will be skipped to create and report warning messages.

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design. Use *report_app_options* to show the value of application options. This application option is design specific.

Examples

```
prompt> set_app_options -name plan.3dic.min_bump_shape_spacing -value  
{{AP 80} {MB 100}}
```

See Also

- [create_bump_array](#)
- [create_bond_pad_array](#)
- [commit_pseudo_bumps](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

plan.3dic.min_bump_zone_street_keepout_margin

Specifies the margin distance of bump zone keepout street.

Data Types

FLOAT

Default `FLOAT_MAX`

Description

This option specifies the margin distance of bump zone keepout street that be used in command *check_3d_design*.

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design. Use *report_app_options* to show the value of application options. This application option is global specific.

See Also

- [check_3d_design](#)

plan.3dic.min_chip_enclosure

Specifies the minimal chip enclosure distance for 3d top design.

Data Types

unsigned float

Default `0.0`

Description

This option specifies the minimal chip enclosure distance for 3d top design. *check_3d_design* will calculate the chip enclosure distance in the 3d top design, if the distance is less than the value app option specified, the command will report a violation. User can set this app option with proper value to let *check_3d_design* do this kind of check. The default value is 0.0.

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design. Use *report_app_options* to show the value of application options. This application option is design specific.

See Also

- [check_3d_design](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

plan.3dic.min_chip_spacing

Specifies the minimal chip spacing for 3d top design.

Data Types

unsigned float

Default 0.0

Description

This option specifies the minimal chip spacing for 3d top design. *check_3d_design* will calculate the chip spacing distance in the 3d top design, if the distance is less than the value app option specified, the command will report a violation. User can set this app option with proper value to let *check_3d_design* do this kind of check. The default value is 0.0.

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design. Use *report_app_options* to show the value of application options. This application option is design specific.

See Also

- [check_3d_design](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

plan.3dic.min_mim_pad_pad_via_probe_cell_spacing

Specifies the min spacing distance between probe cell and interface pad layer via when hybrid bond connect by below way in MIM scenario. Top route layer<-->RV<-->pad layer<-->Interface pad layer via<-->Interface Layer

p

Data Types*Float***Default** `FLOAT_MAX`**Description**

This option specifies the min spacing distance between probe cell and the shape on interface pad layer which belongs to interface pad layer via. This option only set the min distance between above two objects when hybrid bond connect among 'Top route layer', 'RV', 'pad layer', 'Interface pad layer via', and 'Interface Layer' in MIM scenario. The default value is `FLOAT_MAX`. If this option is not specified, the corresponding rv rule will not be checked in command 'check_rv_rules'.

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design. Use *report_app_options* to show the value of application options. This application option is global specific.

plan.3dic.min_mim_pad_pad_via_rv_spacing

Specifies the min spacing distance between RV and interface pad layer via when hybrid bond connect by below way in MIM scenario. Top route layer<-->RV<-->pad layer<-->Interface pad layer via<-->Interface Layer

Data Types*Float***Default** `FLOAT_MAX`**Description**

This option specifies the min spacing distance between RV and the shape on interface pad layer which belongs to interface pad layer via. This option only set the min distance between above two objects when hybrid bond connect among 'Top route layer', 'RV', 'pad layer', 'Interface pad layer via', and 'Interface Layer' in MIM scenario. The default value is `FLOAT_MAX`. If this option is not specified, the corresponding rv rule will not be checked in command 'check_rv_rules'.

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design. Use *report_app_options* to show the value of application options. This application option is global specific.

plan.3dic.min_mim_top_pad_via_pad_layer_spacing

Specifies the min spacing distance between pad layer and interface pad layer via when hybrid bond connect by below way in MIM scenario. 1.Top route layer<-->Interface pad layer via<-->Interface Layer 2.Top route layer<-->RV<-->Pad layer

Data Types

Float

Default FLOAT_MAX

Description

This option specifies the min spacing distance between the shape on pad layer which belongs to RV and the shape on interface pad layer which belongs to interface pad layer via. This option only set the min distance between above two objects when hybrid bond connect among 'Top route layer', 'Interface pad layer via', and 'Interface Layer' in MIM scenario. The default value is FLOAT_MAX. If this option is not specified, the corresponding rv rule will not be checked in command 'check_rv_rules'.

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design. Use *report_app_options* to show the value of application options. This application option is global specific.

plan.3dic.min_mim_top_pad_via_plate_hole_spacing

Specifies the min spacing distance between plate hole edge and interface pad layer via when hybrid bond connect by below way in MIM scenario. 1.Top route layer<-->Interface pad layer via<-->Interface Layer 2.Top route layer<-->RV<-->Pad layer

Data Types

Float

Default FLOAT_MAX

Description

This option specifies the min spacing distance between plate hole edge and the shape on interface pad layer which belongs to interface pad layer via. This option only set the min distance between above two objects when hybrid bond connect among 'Top route layer', 'Interface pad layer via', and 'Interface Layer' in MIM scenario. The default value is FLOAT_MAX. If this option is not specified, the corresponding rv rule will not be checked in command 'check_rv_rules'.

p

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design. Use *report_app_options* to show the value of application options. This application option is global specific.

plan.3dic.min_mim_top_plate_hole_bottom_layer_width

Specifies the min width distance between RV and the edge of hole on MIM bottom layer whose mask_name is 'mimbottom' when hybrid bond connect by below way in MIM scenario. 1.Top route layer<-->Interface pad layer via<-->Interface Layer 2.Top route layer<-->RV<-->Pad layer

Data Types

Float

Default FLOAT_MAX

Description

This option specifies the min width distance between RV and the edge of hole on MIM bottom layer whose mask_name is 'mimbottom'. This option only set the min distance between above two objects when hybrid bond connect among 'Top route layer', 'Interface pad layer via', and 'Interface Layer' in MIM scenario. The default value is FLOAT_MAX. If this option is not specified, the corresponding rv rule will not be checked in command 'check_rv_rules'.

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design. Use *report_app_options* to show the value of application options. This application option is global specific.

plan.3dic.min_mim_top_plate_hole_middle_1_layer_width

Specifies the min width distance between RV and the edge of hole on MIM middle layer whose mask_name is 'mimmiddle1' when hybrid bond connect by below way in MIM scenario. 1.Top route layer<-->Interface pad layer via<-->Interface Layer 2.Top route layer<-->RV<-->Pad layer

Data Types

Float

Default FLOAT_MAX

Description

This option specifies the min width distance between RV and the edge of hole on MIM middle layer whose mask_name is 'mimmiddle1'. This option only set the min distance between above two objects when hybrid bond connect among 'Top route layer', 'Interface

p

pad layer via', and 'Interface Layer' in MIM scenario. The default value is `FLOAT_MAX`. If this option is not specified, the corresponding rv rule will not be checked in command 'check_rv_rules'.

Application options are set using the `set_app_options` command, and they may be specified globally or with respect to a design. Use `report_app_options` to show the value of application options. This application option is global specific.

plan.3dic.min_mim_top_plate_hole_top_layer_width

Specifies the min width distance between RV and the edge of hole on MIM top layer whose mask_name is 'mimtop' when hybrid bond connect by below way in MIM scenario.

1.Top route layer<-->Interface pad layer via<-->Interface Layer 2.Top route layer<-->RV<-->Pad layer

Data Types

Float

Default `FLOAT_MAX`

Description

This option specifies the min width distance between RV and the edge of hole on MIM top layer whose mask_name is 'mimtop'. This option only set the min distance between above two objects when hybrid bond connect among 'Top route layer', 'Interface pad layer via', and 'Interface Layer' in MIM scenario. The default value is `FLOAT_MAX`. If this option is not specified, the corresponding rv rule will not be checked in command 'check_rv_rules'.

Application options are set using the `set_app_options` command, and they may be specified globally or with respect to a design. Use `report_app_options` to show the value of application options. This application option is global specific.

plan.3dic.min_pad_pad_via_pad_layer_spacing

Specifies the min spacing distance between pad layer and interface pad layer via when hybrid bond connect by below way without MIM scenario. Top route layer<-->RV<-->pad layer<-->Interface pad layer via<-->Interface Layer

Data Types

Float

Default `FLOAT_MAX`

p

Description

This option specifies the min spacing distance between the shape on pad layer who belongs to RV and the shape on interface pad layer which belongs to interface pad layer via. This option only set the min distance between above two objects when hybrid bond connect among 'Top route layer', 'RV','pad layer','Interface pad layer via', and 'Interface Layer' without MIM scenario. The default value is FLOAT_MAX. If this option is not specified, the corresponding rv rule will not be checked in command 'check_rv_rules'.

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design. Use *report_app_options* to show the value of application options. This application option is global specific.

plan.3dic.min_pad_pad_via_rv_spacing

Specifies the min spacing distance between RV and interface pad layer via when hybrid bond connect by below way without MIM scenario. Top route layer<-->RV<-->pad layer<-->Interface pad layer via<-->Interface Layer

Data Types*Float***Default** FLOAT_MAX**Description**

This option specifies the min spacing distance between RV and the shape on interface pad layer which belongs to interface pad layer via. This option only set the min distance between above two objects when hybrid bond connect among 'Top route layer', 'RV','pad layer','Interface pad layer via', and 'Interface Layer' without MIM scenario. The default value is FLOAT_MAX. If this option is not specified, the corresponding rv rule will not be checked in command 'check_rv_rules'.

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design. Use *report_app_options* to show the value of application options. This application option is global specific.

plan.3dic.min_probe_bumps_quad_spacing

Specifies the min spacing distance between one and the nearest quad probe bumps.

Data Types*FLOAT***Default** FLOAT_MAX

p

Description

This option specifies the min spacing distance between one and the nearest quad probe bumps that be used in command *check_3d_design*. Probe bumps quad must be exactly 2x2 bumps, all 4 identical bumps shape, size, layer, perfect equal square spacing, all 4 connected to the same net.

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design. Use *report_app_options* to show the value of application options. This application option is global specific.

See Also

- [check_3d_design](#)

plan.3dic.min_routing_blockage_spacing

Specifies the min distance to the routing blockage.

Data Types

Pair list

Default NA.

Description

This option allows user to specify the min distance in user units to the routing blockage. When command '*create_bump_array*', '*create_bond_pad_array*' and '*commit_pseudo_bumps*' set the command option '-min_spacing' and it includes 'route_blockage' element. During the process of these commands, these commands will check if the distance between any new shape of new created bump cells/bond pads and existed route blockage is less than the specified app option value with the same layer. The new bump cell/bond pad will be skipped to create and report warning messages.

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design. Use *report_app_options* to show the value of application options. This application option is design specific.

Examples

```
prompt> set_app_options -name plan.3dic.min_routing_blockage_spacing
-value {{AP 80} {MB 100}}
```

See Also

- [create_bump_array](#)
- [create_bond_pad_array](#)

p

- [commit_pseudo_bumps](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

plan.3dic.min_routing_shape_spacing

Specifies the min distance to the routing shape.

Data Types

Pair list

Default NA.

Description

This option allows user to specify the min distance in user units to the routing shape. When command '*create_bump_array*', '*create_bond_pad_array*' and '*commit_pseudo_bumps*' set the command option '-min_spacing' and it includes 'route_shape' element. During the process of these commands, these commands will check if the distance between any new shape of new created bump cells/bond pads and existed route shape is less than the specified app option value with the same layer. The new bump cell/bond pad will be skipped to create and report warning messages.

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design. Use *report_app_options* to show the value of application options. This application option is design specific.

Examples

```
prompt> set_app_options -name plan.3dic.min_routing_shape_spacing -value  
{{AP 80} {MB 100}}
```

See Also

- [create_bump_array](#)
- [create_bond_pad_array](#)
- [commit_pseudo_bumps](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

plan.3dic.min_stitching_zone_spacing

Specifies the min distance to the stitching zone.

Data Types

Float

Default

1.

Description

This option allows user to specify the min distance in user units to the stitching zone. When command '*create_bump_array*', '*create_bond_pad_array*' and '*commit_pseudo_bumps*' set the command option '-min_spacing' and it includes 'stitching_zone' element. During the process of these commands, these commands will check if the distance between any new shape of new created bump cells/bond pads and existed stitching_zone is less than the specified app option value. The new bump cell/bond pad will be skipped to create and report warning messages.

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design. Use *report_app_options* to show the value of application options. This application option is design specific.

Examples

```
prompt> set_app_options -name plan.3dic.min_stitching_zone_spacing -value 100
```

See Also

- [create_bump_array](#)
- [create_bond_pad_array](#)
- [commit_pseudo_bumps](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

plan.3dic.min_top_pad_via_pad_layer_spacing

Specifies the min spacing distance between pad layer and interface pad layer via when hybrid bond connect by below way without MIM scenario. 1.Top route layer<-->Interface pad layer via<-->Interface Layer 2.Top route layer<-->RV<-->Pad layer

p

Data Types*Float***Default** FLOAT_MAX**Description**

This option specifies the min spacing distance between the shape on pad layer who belongs to RV and the shape on interface pad layer which belongs to interface pad layer via. This option only set the min distance between above two objects when hybrid bond connect among 'Top route layer', 'Interface pad layer via', and 'Interface Layer' without MIM scenario. The default value is FLOAT_MAX. If this option is not specified, the corresponding rv rule will not be checked in command 'check_rv_rules'.

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design. Use *report_app_options* to show the value of application options. This application option is global specific.

plan.3dic.min_top_pad_via_rv_spacing

Specifies the min spacing distance between RV and interface pad layer via when hybrid bond connect by below way without MIM scenario. 1.Top route layer<-->Interface pad layer via<-->Interface Layer 2.Top route layer<-->RV<-->Pad layer

Data Types*Float***Default** FLOAT_MAX**Description**

This option specifies the min spacing distance between RV and the shape on interface pad layer which belongs to interface pad layer via. This option only set the min distance between above two objects when hybrid bond connect among 'Top route layer', 'Interface pad layer via', and 'Interface Layer' without MIM scenario. The default value is FLOAT_MAX. If this option is not specified, the corresponding rv rule will not be checked in command 'check_rv_rules'.

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design. Use *report_app_options* to show the value of application options. This application option is global specific.

plan.3dic.pattern_marker_cell_margin_bottom

Specifies the range of enclosure distance between the pattern marker cell and the bottom edge of chip.

p

Data Types

*list_of_pairs***Default** null

Description

This option specifies the range of enclosure distance between the pattern marker cell and the bottom edge of chip that be used in command *check_3d_design*.

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design. Use *report_app_options* to show the value of application options. This application option is global specific.

See Also

- [check_3d_design](#)

plan.3dic.pattern_marker_cell_margin_left

Specifies the range of enclosure distance between the pattern marker cell and the left edge of chip.

Data Types

*list_of_pairs***Default** null

Description

This option specifies the range of enclosure distance between the pattern marker cell and the left edge of chip that be used in command *check_3d_design*.

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design. Use *report_app_options* to show the value of application options. This application option is global specific.

See Also

- [check_3d_design](#)

plan.3dic.pattern_marker_cell_margin_right

Specifies the range of enclosure distance between the pattern marker cell and the right edge of chip.

p

Data Types*list_of_pairs***Default** null**Description**

This option specifies the range of enclosure distance between the pattern marker cell and the right edge of chip that be used in command *check_3d_design*.

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design. Use *report_app_options* to show the value of application options. This application option is global specific.

See Also

- [check_3d_design](#)

plan.3dic.pattern_marker_cell_margin_top

Specifies the range of enclosure distance between the pattern marker cell and the top edge of chip.

Data Types*list_of_pairs***Default** null**Description**

This option specifies the range of enclosure distance between the pattern marker cell and the top edge of chip that be used in command *check_3d_design*.

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design. Use *report_app_options* to show the value of application options. This application option is global specific.

See Also

- [check_3d_design](#)

plan.3dic.pattern_marker_lib_cell_names

Specifies two lib cell names for pattern marker.

p

Data Types*list_of_pairs***Default** null**Description**

This option specifies two lib cells for pattern marker that be used in command *check_3d_design*. The cell with specified lib cell must be unique, and two marker cell must be placed in opposite corners.

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design. Use *report_app_options* to show the value of application options. This application option is global specific.

See Also

- [check_3d_design](#)

plan.3dic.probe_num_max

Specifies the max number of probe bumps per die.

Data Types*unsigned int***Default** UINT_MAX**Description**

This option specifies the max number of probe bumps per die. The default value is UINT_MAX. This option is used in command *check_3d_design -advanced_bump_rules*.

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design. Use *report_app_options* to show the value of application options. This application option is global specific.

See Also

- [check_3d_design](#)

plan.3dic.probe_num_min_discrete_power_ground

Specifies the min number of probe bumps that connect to power or ground net.

p

Data Types*unsigned int***Default** 0**Description**

This option specifies the min number of probe bumps that connect to power or ground net per die. The default value is 0. This option is used in command *check_3d_design -advanced_bump_rules*.

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design. Use *report_app_options* to show the value of application options. This application option is global specific.

See Also

- [check_3d_design](#)

plan.3dic.qprobe_min_orthogonal_pitch_sprobe

Specifies the min pitch(center to center) between the probe bump in the quad bumps and single probe bump.

Data Types*Float***Default** 0.0**Description**

This option specifies the min pitch(center to center) between the probe bump in the quad bumps and single probe bump. Probe bumps quad must be exactly 2x2 bumps, all 4 identical bumps shape, size, layer, perfect equal square spacing, all 4 connected to the same net. The default value is 0.0. Each probe bump can be reconginze as single probe bump. This option is used in command *check_3d_design -advanced_bump_rules*.

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design. Use *report_app_options* to show the value of application options. This application option is global specific.

See Also

- [check_3d_design](#)

plan.3dic.qprobe_min_pitch

Specifies the min pitch(center to center) between two quad probe bumps.

Data Types

Float

Default 0.0

Description

This option specifies the min pitch(center to center) between two quad probe bumps. This function of app option is same as *plan.3dic.min_probe_bumps_quad_spacing*. Probe bumps quad must be exactly 2x2 bumps, all 4 identical bumps shape, size, layer, perfect equal square spacing, all 4 connected to the same net. The default value is 0.0. This option is used in command *check_3d_design -advanced_bump_rules*.

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design. Use *report_app_options* to show the value of application options. This application option is global specific.

See Also

- [check_3d_design](#)

plan.3dic.qprobe_num_min

Specifies the min number of quad probe bumps.

Data Types

unsigned int

Default 0

Description

This option specifies the min number of quad probe bumps per die. Probe bumps quad must be exactly 2x2 bumps, all 4 identical bumps shape, size, layer, perfect equal square spacing, all 4 connected to the same net. The default value is 0. This option is used in command *check_3d_design -advanced_bump_rules*.

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design. Use *report_app_options* to show the value of application options. This application option is global specific.

p

See Also

- [check_3d_design](#)

plan.3dic.qprobe_pitch_ubump

Specifies the min pitch(center to center) between the quad probe bumps and common bump(not probe or dummy).

Data Types*Float***Default** 0.0**Description**

This option specifies the min pitch(center to center) between the quad probe bumps and common bump(not probe or dummy). Probe bumps quad must be exactly 2x2 bumps, all 4 identical bumps shape, size, layer, perfect equal square spacing, all 4 connected to the same net. The default value is 0.0. This option is used in command *check_3d_design -advanced_bump_rules*.

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design. Use *report_app_options* to show the value of application options. This application option is global specific.

See Also

- [check_3d_design](#)

plan.3dic.sprobe_min_pitch_non_dummy_ubump_other

Specifies the min pitch(center to center) between the probe bump and common bump(not probe or dummy) in other(not square) pattern.

Data Types*Float***Default** 0.0**Description**

This option specifies the min pitch(center to center) between the probe bump and common bump(not probe or dummy) in other(not square) pattern. The default value is 0.0. This option is used in command *check_3d_design -advanced_bump_rules*.

p

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design. Use *report_app_options* to show the value of application options. This application option is global specific.

See Also

- [check_3d_design](#)

plan.3dic.sprobe_min_pitch_non_dummy_ubump_square

Specifies the min pitch(center to center) between the probe bump and common bump(not probe or dummy) in square pattern.

Data Types*Float***Default** 0.0**Description**

This option specifies the min pitch(center to center) between the probe bump and common bump(not probe or dummy) in square pattern. The default value is 0.0. This option is used in command *check_3d_design -advanced_bump_rules*.

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design. Use *report_app_options* to show the value of application options. This application option is global specific.

See Also

- [check_3d_design](#)

plan.3dic.sprobe_min_pitch_other

Specifies the min pitch(center to center) between the probe bumps in the non-dummy region in other pattern.

Data Types*Float***Default** 0.0

p

Description

This option specifies the min pitch(center to center) between the probe bumps in the non-dummy region in other pattern. The default value is 0.0. This option is used in command *check_3d_design -advanced_bump_rules*.

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design. Use *report_app_options* to show the value of application options. This application option is global specific.

See Also

- [check_3d_design](#)

plan.3dic.sprobe_min_pitch_square

Specifies the min pitch(center to center) between the probe bumps in the non-dummy region in square pattern.

Data Types

Float

Default 0.0

Description

This option specifies the min pitch(center to center) between the probe bumps in the non-dummy region in square pattern. The default value is 0.0. This option is used in command *check_3d_design -advanced_bump_rules*.

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design. Use *report_app_options* to show the value of application options. This application option is global specific.

See Also

- [check_3d_design](#)

plan.3dic.sprobe_min_pitch_true_dummy_region

Specifies the min pitch(center to center) between the probe bumps in the true dummy region.

Data Types

Float

p

Default 0.0**Description**

This option specifies the min pitch(center to center) between the probe bumps in the true dummy region. If these bumps surround by probe bump are all dummy, it means this probe bump is in dummy region. The default value is 0.0. This option is used in command *check_3d_design -advanced_bump_rules*.

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design. Use *report_app_options* to show the value of application options. This application option is global specific.

See Also

- [check_3d_design](#)

plan.3dic.sprobe_min_pitch_true_dummy_ubump

Specifies the min pitch(center to center) between the probe bump and the unprobed dummy bump.

Data Types*Float***Default** 0.0**Description**

This option specifies the min pitch(center to center) between the probe bump and the unprobed dummy bump. The default value is 0.0. This option is used in command *check_3d_design -advanced_bump_rules*.

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design. Use *report_app_options* to show the value of application options. This application option is global specific.

See Also

- [check_3d_design](#)

plan.auto_group.balance_area

Creates partitions minimizing the area variation between them.

Data Types

string

Default none

Description

When this app option is set to all, the partitioner will balance the area of partitions such that the variation between them is minimized.

There will be trade off in terms of user/tool specified max partition size.

See Also

- [auto_partition_design](#)

plan.auto_group.enable_feedthrough_aware_in_distributed

This app-option is used to enable feedthrough flavor of auto-partitioning when the execution strategy is set to distributed.

Data Types

bool

Default false

Description

When the app-option is set to true, feedthrough flavor of auto-partitioning is enabled when the execution strategy is set to distributed by the user through the app-option `plan.auto_group.execution_strategy`. By default, feedthrough aware flavor of auto-partitioning is disabled when execution strategy is set to distributed in auto-partitioning.

See Also

- [set_auto_partition_constraints](#)
- [auto_partition_design](#)

plan.auto_group.enable_macro_balancing

Balances the macro count among partitions created by auto-partition.

Data Types

bool

p

Default false**Description**

When the app-option is set to true, macro count is balanced among partitions created by auto-partition. User may specify a threshold macro count using the app-option *plan.auto_group.threshold_macro_count*, which will be treated as the threshold count for number of macros in a partition. Additionally, user can specify macro lib cells that need to be balanced while ignoring the other macros for macro balancing. By default, macro balancing is disabled in auto-partition.

See Also

- [auto_partition_design](#)

plan.auto_group.enable_weighted_result_grading

This app-option is used to enable the weightage based result grading in auto-partition.

Data Types*Bool***Default** True**Description**

Enables a new weightage-based system to grade auto-partition results. When the app-option is set to true, tool will assign the weight to each grading criteria set via app option *plan.auto_group.results_grading_criteria*. and grade the auto-partition results based on the weight of each criteria. By default, weightage based result grading is enabled.

See Also

- [auto_partition_design](#)

plan.auto_group.execution_strategy

This app-option sets the strategy used in making auto-partition runs.

Data Types*String***Default** auto

p

Description

This app-option sets the strategy used in making auto-partition runs. When the app options is set to *auto*, the tool will default to serial run of auto partitioning. When the app option is set to distributed, the tool will start the auto partition in distributed manner. When the app option is set to config, the tool will start the runs mentioned in file specified in *flow_config_file* app-option in distributed manner. When app-option is set to user, the tool launches distributed runs for user settings. This will give priority to user settings of *multiple_results*, *partition_type* and *prioritize* factors.

See Also

- [set_auto_partition_constraints](#)
- [auto_partition_design](#)

plan.auto_group.flow_config_file

This app-option sets the file name required for distributed run of auto-partitioning when the execution strategy is set to config.

Data Types

String

Default ""

Description

If this app-option value is set to file containing the desired runs, and execution strategy is set to config then tool will starts the desired runs in distributed manner. By default, the value of app-option is empty.

See Also

- [set_auto_partition_constraints](#)
- [auto_partition_design](#)

plan.auto_group.generate_csv

In addition to generating results in ascii format also generates the results in csv format which is used by the auto-partition gui.

Data Types

bool

Default false

Description

By setting this app-option to true, auto-partition additionally writes result data in csv format which is used by the auto-partition gui.

See Also

- [auto_partition_design](#)

plan.auto_group.generate_stats_effort_level

Specifies the effort level for generating the auto partition result statistics'.

Data Types

string

Default "low"

Description

The auto partitioning default uses low effort level to generate the auto partition result statistics'. This option enables to generate the stats at medium effort level.

See Also

- [set_auto_partition_constraints](#)
- [auto_partition_design](#)

plan.auto_group.grade_results

Grades the auto-partition results as per the default/user provided criteria.

Data Types

bool

Default false

Description

By setting this app-option to true, the multiple results generated by auto-partition are graded according to the default or user provided criteria. By default, the results are graded according to the user-provided maximum cell count of partition, maximum pin count of partition, maximum group count and total pin count of a partition result in the

p

mentioned order. The user may provide criteria for result grading using app-option 'plan.auto_group.results_grading_criteria'.

See Also

- [explore_logic_hierarchy](#)

plan.auto_group.group_ncells_max

Specifies the maximum number of cells in a partition.

Data Types

integer

Default 0

Description

During auto partitioning, this application option specifies the maximum number of cells in a partition. This application option along with the plan.auto_group.groups_count_max application option, guides you in creating multiple partitions.

This is not a hard constraint in partitioning.

The size of result partition/s may be greater than that specified in the app-option in specific cases in the following scenarios:

- a) pincount is set as the first factor in the app-option plan.auto_group.prioritize_factors.
- b) Partition candidate specified by user in command option is bigger in size than max partition size.
- c) Size of hierarchies specified in -include_hierarchies_in_same_partition command option of command set_auto_partition_constraints is more than the max partition size.
- d) Partition has only one hierarchical instance in the result partition and there is no logical hierarchy below the hierarchical instance.
- e) MIMs are enabled and the size of Auto-Partition recognized MIM is more than the max partition size. Since MIM become stand alone partitions, the size of these partitions will be more than the max partition size.
- f) Black-box settings are given by the user in which cell-count of black-box is more than the max partition size.

See Also

- [explore_logic_hierarchy](#)
- [set_auto_partition_constraints](#)
- [auto_partition_design](#)

plan.auto_group.group_npins_max

Specifies the maximum number of pins of a partition

Data Types

integer

Default 0

Description

During auto partitioning, this app option specifies the maximum number of pins of a partition. This app option along with plan.auto_group.prioritize_factors guide the partitioner in creating multiple partitions. If the user has specified pincount as a higher priority via app option plan.auto_group.prioritize_factors and does not specify this app option, an average max pin count is calculated by the tool, In case pincount is set as a lower priority, the pin balancing is skipped.

See Also

- [explore_logic_hierarchy](#)

plan.auto_group.groups_count_max

Specifies the maximum number of partitions.

Data Types

integer

Default 10

Description

During auto partitioning, this application option specifies the maximum number of partitions to be created. This application option along with the plan.auto_group.group_ncells_max guides you in creating multiple partitions.

This is not a hard constraint in partitioning.

p

The number of result partitions may be more than the number specified in the above app-option in specific cases in the following scenarios:

- a) Design is MV and MV awareness is enabled in auto-partition. Partition clusters may be created by the internally by the tool to maintain MV compatibility in partitions.
- b) Partition Clusters in the design are more.
- c) pincount is set as the first factor in the app-option `plan.auto_group.prioritize_factors`.
- d) MIMs are enabled and count of Auto-Partition recognized MIM is substantial. Since, MIM become stand alone partitions, the MIMs will increase the partition count.
- e) Substantial Black-box settings are given by the user in which cell-count of black-box is large enough to make it a single partition.

See Also

- [explore_logic_hierarchy](#)
- [auto_partition_design](#)
- [set_auto_partition_constraints](#)

plan.auto_group.macro_lib_cells

If macro balancing is enabled, balances the count of specified macro lib cells in the partitions created by auto-partition.

Data Types

bool

Default false

Description

If macro balancing is enabled, balances the count of specified macro lib cells in the partitions created by auto-partition. The count of specified macros in each partition is printed in the auto-partition result statistics in the auto-partition result file. If `macro_balancing` is specified as a criteria in result grading, the results will be graded based on the count of the specified macros in the partitions.

See Also

- [auto_partition_design](#)

plan.auto_group.multiple_results

Controls generation of multiple auto-partition results

Data Types

bool

Default true

Description

Auto partition generates multiple results by using different sets of candidate hierarchies and also by using the partitioner differently. If there are N1 sets of candidate hierarchies and N2 different ways of using the partitioner, then setting this app option to true will give N1*N2 results. Setting the app option to false will use only one set of candidate hierarchies and give N2 results.

See Also

- [explore_logic_hierarchy](#)

plan.auto_group.partition_name_prefix

Specifies the prefix to use for the names of the partitions created during auto-partitioning.

Data Types

string

Default "snps_partition_"

Description

User may specify the partition name prefix for partitions generated by guided partitioning using this app-option.

See Also

- [auto_partition_design](#)

plan.auto_group.partition_only_cluster

Auto-Partition performs partition only inside clusters.

Data Types

bool

Default false

Description

Partitioning is performed only inside clusters by auto-partition. It excludes other logic in the design from the partitioning process.

See Also

- [set_auto_partition_constraints](#)
- [auto_partition_design](#)

plan.auto_group.partition_size_increase_percentage_for_result_optimization

Specifies the percentage for relaxing the partition size for auto partitioning result optimization.

Data Types

double

Default 0

Description

Through this application option, user can specify the percentage by which the partition size may be relaxed in case it is required for auto partitioning result optimization.

See Also

- [auto_partition_design](#)

plan.auto_group.partition_type

Specifies the type of partitioning to be run.

Data Types

String

Default auto

Description

This app-option specifies the type of partitioning to be run on the design. When the application options is set to *auto*, the tool will default to distance partition type for a mapped design and connectivity for an unmapped design. The value can be set to

p

connectivity, distance or feedthrough. When partition type is connectivity, partitioning is based on connectivity between modules. When set to distance, partitioning also takes into account distance between modules. Feedthrough partition-type aims to reduce the number of feedthroughs in partitions.

See Also

- [explore_logic_hierarchy](#)

plan.auto_group.physical_aware

Enables partitioning with physical data

Data Types

bool

Default false

Description

The partitioner default uses connectivity information. This option enables physical information, both physical distance and feedthrough, in addition to logical connectivity. This application option is deprecated and is scheduled for removal in a future release. Please use `plan.auto_group.partition_type` instead.

See Also

- [explore_logic_hierarchy](#)

plan.auto_group.print_inter_partition_feedthroughs

Enables printing of feedthrough index between partitions.

Data Types

bool

Default false

Description

This app-option enables the printing of feedthrough index of partition in auto-partition results.

See Also

- [set_auto_partition_constraints](#)
- [auto_partition_design](#)

plan.auto_group.print_inter_partition_wirelength

Enables printing of total wirelength between partitions.

Data Types

bool

Default false

Description

This app-option enables the printing of total wirelength in auto-partition results.

See Also

- [set_auto_partition_constraints](#)
- [auto_partition_design](#)

plan.auto_group.prioritize_factors

Set priority of factors 'pincount' and 'cellcount' during partitioning

Data Types

string

Default cellcount pincount

Description

During auto partitioning, this app option specifies the priority of factors determining partition result. The higher priority factor's max count is strictly adhered to during partitioning, while the max count of the second priority factor is relaxed by 30%. For example, If the value is set as 'pincount cellcount', max pincount is strictly adhered to while max cellcount is relaxed by 30% during partitioning. This app option along with plan.auto_group.group_npins_max and plan.auto_group.group_ncells_max guide the partitioner in creating multiple partitions. The default value of this app option is "cellcount pincount".

See Also

- [explore_logic_hierarchy](#)

plan.auto_group.remove_apd_file

This app option is used to remove file that documents commands to apply the auto-partition solution that has been selected.

Data Types

bool

Default "true"

Description

This app option is used to remove file that documents commands to apply the auto-partition solution that has been selected.

See Also

- [auto_partition_design](#)

plan.auto_group.result_directory

Specifies the output directory for the auto-partition result files.

Data Types

string

Default 0

Description

This app option specifies the output directory for the auto-partition result files.

See Also

- [explore_logic_hierarchy](#)

plan.auto_group.results_grading_criteria

Sets user grading criteria to grade auto-partition results.

Data Types

string

p

Default ""**Description**

This app-option specifies the criteria for grading the auto-partition results. The allowed values are `partition_cell_count`, `partition_pin_count`, `total_group_count`, `total_pin_count`. The order in which the values are specified determines the order in which the criteria is used for grading. The default value is the `partition_cell_count` and `partition_pin_count` (order is determined by `plan.auto_group.prioritize_factors`) followed by `total_group_count` and then `total_pin_count`.

See Also

- [auto_partition_design](#)

plan.auto_group.threshold_macro_count

Specifies the macro count threshold for each partition.

Data Types

integer

Default 0**Description**

If macro balancing is enabled in auto partition, this application option specifies the threshold macro count for a partition. This application option when used with the *plan.auto_group.enable_macro_balancing* application option, guides you in creating multiple partitions. In case, user does not provide a threshold macro count through the app-option, tool auto calculates the macro threshold count using the max partition count (user specified/tool calculated).

See Also

- [auto_partition_design](#)

plan.budget.add_missing_feedthrough_clocks

When writing budgets, declare clock in blocks where new clock feedthrough have been added

Data Types

Boolean

Default false

Description

When creating constraints for blocks, it is sometimes desirable include `create_clock` definitions on at places where blocks feed through your block. Even though the clock net may not fan out to any flops in your block, some tools might treat the clock feedthrough differently because a `create_clock` statement exists. For example, the clock might be shielded.

There is another app option (`plan.budget.include_feedthrough_clocks`) which signals the `split_constraints` command to declare `create_clock` statements on clock feedthroughs. But `split_constraints` will only see clock feedthroughs that are defined in your original verilog netlist. If clock feedthroughs are added after `split_constraints`, no `create_clock` will be declared.

If you set this option to true, the `write_budgets` command will add a `create_clock` statement for clock feedthroughs that do not yet have one. Since `write_budgets` is generally run after new clock feedthroughs are created, you can properly update your block constraints simply by applying the budget.

Note that `plan.budget.add_missing_feedthrough_clocks` app option will only work if you *also* set `plan.budget.include_feedthrough_clocks`. If you do not set the `plan.budget.include_feedthrough_clocks`, the tool will assume that feedthrough clocks should never get `create_clock` statements.

See Also

- [write_budgets](#)

plan.budget.all_design_subblocks

Controls the behavior of the `write_budgets` and `split_constraints` commands. When true, the `-design_subblocks` option is automatically set for all blocks.

Data Types

boolean

Default false

Description

When generating budget constraints, it is assumed that any subblock of a budgeted block is represented by an abstract. The budget will "cut out" constraints that are not visible on the boundary of the abstract. Setting this option causes the internal constraints of subblocks to be kept in the budget.

When calculating "design_subblock" constraints, it is necessary for the `write_budgets` command to time all of the internal paths of each budgeted block. If you are using

p

abstracts to represent some of your blocks, timing the internal paths of the block will not be possible. So "full" constraints cannot be generated for abstracted blocks. If you want to generate "full" constraints, you must also use a full design view (not an abstract) for the block.

Memory Considerations

When you set this option to "true", you must change all of your blocks to a full design view. For large designs, this could result in a very large use of memory and runtime. To avoid this memory impact, you should consider using the *split_constraints* command to generate the internal constraints of your budgeted block. The *split_constraints* command is specifically designed to use much less memory and runtime. After you have the results of *split_constraints*, you run *write_budgets* its default incremental setting. See the man page for details.

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)
- [split_constraints](#)
- [write_budgets](#)

plan.budget.all_full_budgets

Controls the behavior of the *write_budgets* command to choose between writing only incremental boundary constraints or full constraints for a budgeted block.

Data Types

Boolean

Default false

Description

For each budgeted block in your design, the *write_budgets* command is capable of generating budgets in one of two styles: "full constraints" or "incremental constraints". Full constraints are a complete set of constraints for a block. Incremental constraints preserve a block's internal constraints but updates the constraints on the block interface.

p

- If *plan.budget.all_full_budgets* is set to "true", then full constraints will be generated for all budgeted blocks.
- If *plan.budget.all_full_budgets* is set to "false", then incremental constraints will be used by default. However, if you create a budget for the top level, using the *-top* option of *write_budgets*, the top will always have a full budget. Also, you can dictate that some budgeted blocks should get a full budget by using the *-full_budget_blocks* option of *write_budgets*.

"Incremental" constraints apply only to the interface timing paths of a budgeted block. These constraints are intended to be overlaid on top of the existing internal constraints of the block. Using these constraints, you can incrementally change the block's interface boundary constraints without disturbing the internal constraints. If you do not have existing internal constraints for your block, you can obtain them by using the *split_constraints* command.

"Full" constraints include constraints for both the interface of the budgeted block *and* the internal block paths. If you use these constraints, any existing constraints on the block will be removed and replaced.

When calculating "full" constraints, it is necessary for the *write_budgets* command to time all of the internal paths of each budgeted block. If you are using abstracts to represent some of your blocks, timing the internal paths of the block will not be possible. So "full" constraints cannot be generated for abstracted blocks. If you want to generate "full" constraints, you must also use a full design view (not an abstract) for the block.

Memory Considerations

When you set this option to "true", you will be required to change all of your blocks to a full design view. For large designs, this could result in a very large use of memory and runtime. To avoid this memory impact, you should consider using the *split_constraints* command to generate the internal constraints of your budgeted block. The *split_constraints* command is specifically designed to use much less memory and runtime. After you have the results of *split_constraints*, you run *write_budgets* its default incremental setting. Refer to the man page for details.

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)
- [split_constraints](#)
- [write_budgets](#)

plan.budget.allow_disconnected_gclocks

Allows problems with generated clocks to be ignored during split_constraints.

Data Types

Boolean

Default false

Description

When creating split constraints, it is necessary to propagate clocks across the block boundaries. And if you have a generated clock source in your block, it is necessary to propagate the master clock from outside of the block to inside of the block.

If split_constraints is unable to propagate a master clock to the generated clock, you will receive either an ABS-207 or ABS-286 error message, and the output constraints will not be usable.

If you set this app option to true, split_constraints will work around the problem by automatically changing your generated clock into a regular clock. This will allow you to continue with your flow, but it is not recommended for the long term. When the generated clock in your budget has been changed into a regular clock, the clock latency will no longer be propagated to this clock. This will be a serious problem for post-CTS optimization.

Ultimately, you need to fix the underlying problem and set this app option back to false. The man pages for ABS-207 and ABS-286 give you tips for this.

See Also

- [split_constraints](#)

plan.budget.allow_top_only_exceptions

Generate budgets that allow you to change top-level exceptions without regenerating or modifying your block-level constraints.

Data Types

Boolean

Default false

Description

When running write_budgets, it is expected that the constraints used at the top level of your design are consistent with the constraints that applied to your lower-level blocks. This

p

allows the budgeter to incrementally apply constraints to the boundary of your block, while assuming that the constraints in the core of your block are already correct.

There are times, however, that you might want to adjust your top-level constraints without touching your block. For example, you might want to use `set_multicycle_path` on a path the crosses between two of your blocks.

If you set this app option to true, the budgeter will support changing exceptions (`set_false_path` and `set_multicycle_path`) at the top level, without regenerating or modifying your block-level constraints. The only rule is that your added constraint may not name any pins **inside** of your blocks. For example, it is OK to add the following top-level exception.

```
set_multicycle_path -from block1/out1 -to block2/in1
```

It is not supported if your exception names pins inside the block. For example, this is not supported:

```
set_multicycle_path -from block1/reg1/CK -to block2/reg1/D
```

If you want to change constraints that reach inside of the block, you need to also update your block constraints. This can be done manually, or with the help of the *split_constraints* command. Alternatively, you could consider using "full budgeting", but that carries a memory and CPU penalty. Please refer to the man page for *plan.budget.all_full_budgets* for details.

See Also

- [split_constraints](#)
- [write_budgets](#)

plan.budget.bbt_fixed_delay

Control whether delays added with bbt commands are treated as fixed by automatic budgeting.

Data Types

bool

Default true

Description

Black-box timing (bbt) delays can be added to your circuit using the *create_blackbox_delay* and *create_blackbox_constraint* commands. When automatic budgeting runs, it must decide whether to treat these bbt delays as "fixed" or "variable". If the bbt delay is variable, budgeting will assume that the delay might be improved by

p

synthesis. So budgeting will tighten the block constraint, if necessary, to meet top-level timing.

If the bbt delay is fixed, budgeting will not try not to tighten constraints on bbt delays. The constraint will only be tightened if there is no choice. For example, a constraint will only be tightened on a fixed bbt delay if the budgeted path has negative slack and there is no variable delay in the path to optimize. When this happens, you will receive ABS-220 warnings.

By default, bbt delays are treated as "fixed" by budgeting. Set this option to false to enable "variable" bbt delays.

See Also

- [compute_budget_constraints](#)
- [create_blackbox_constraint](#)
- [create_blackbox_delay](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

plan.budget.calculate_hold_budgets

Controls the behavior of the *compute_budget_constraints* command. When true, the budgeting computation will include the computation of hold budgets which can be used for hold optimization at the block boundaries.

Data Types

Boolean

Default false

Description

When computing budgets, only setup budgets on the block interface paths are computed by default. If you set this option to true, the hold budgets are also computed.

Hold budgets are chosen so that, as much as possible, hold buffers are added to the segments near the start of a timing path. The hold budgets on other segments are set so that those segments do not get too much faster and cause a hold violation on the overall path. Use the *plan.budget.hold_buffer_margin* app option to control this behavior.

p

If adding hold buffers would cause a setup budget constraint to be violated in a segment, the hold buffer constraints will direct synthesis to buffer a different segment further towards the end of the path.

Additionally, the *-launch_hold_fix*, *-capture_hold_fix*, and *-feed_hold_fix* options to the *set_budget_options* command give more control on where hold time violations can be repaired. These options are useful to avoid repairing hold violations in certain blocks or at the top level.

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)
- [compute_budget_constraints](#)
- [set_budget_options](#)
- [write_budgets](#)

plan.budget.clip_unoptimized_values

Instructs the budgeter to reduce very high delay values caused by overloaded or high-fanout logic nets.

Data Types

Boolean

Default false

Description

By default, budgeting computation and reporting are based on the actual timing of your design. If your design has not yet been optimized, it is likely that it will have some delays that are very, very large. The paths with very large delays may result in suboptimal budgets.

If you set this app option to true, very large delays and capacitances will automatically be reduced (clipped) to more reasonable values. With the reduced delays, your budget will be more useful. The budget can be used during early design stages, when you are working through the details of your flow.

Note that the clipping of delays does not give you an accurate prediction of the eventual logic optimization. It merely helps you get a budget that is generally useful. If you want a highly accurate budget, based on an accurate predictions of optimized delays, you should

use the `estimate_timing` command before budgeting. Please refer to the *estimate_timing* man page for details.

See Also

- [estimate_timing](#)
- [compute_budget_constraints](#)
- [report_budget](#)
- [write_budgets](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

plan.budget.compute_rise_fall_latencies

Controls whether latencies are calculated separately for rise and fall by the `compute_budget_constraints` command.

Data Types

true | *false*

Default `false`

Description

When `true`, the `compute_budget_constraints` command will calculate separate nominal latency values for the rise and fall edge of each clock. Likewise, a separate nominal CRP value will be calculated.

When `false`, the `compute_budget_constraints` command will combine the latencies of rise and fall together, and the same nominal value will be used for both edges.

See Also

- [compute_budget_constraints](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

plan.budget.delay_margin_percent

Specifies how margins are timed when evaluating budget paths.

Data Types

integer

Default 0

Description

The *write_budgets* command writes out a set of constraints for each block that have been previously computed using the *compute_budget_constraints* command. These constraints include input and output delays for the ports of the block that have timed paths going through them. If this app option is set to a non-zero value, these input and output delays are modified. They are tightened (when the number is positive) or relaxed (when the number is negative) by the percentage indicated by the app option value.

See Also

- [compute_budget_constraints](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [report_budget](#)
- [set_app_options](#)

plan.budget.enable_data_check

Causes *write_budgets* to write out data-to-data check constraints

Data Types

Boolean

Default false

Description

When app option is true, the *write_budgets* command includes data-to-data check constraints in the scripts generated by *write_budgets*.

See Also

- [write_budgets](#)
- [split_constraints](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

plan.budget.enable_tcm_flow

Support to use TCM demotion (as the alternative to the FC command *split_constraints*) as part of the FC budgeting solution.

Data Types

Boolean

Default `false`

Description

When app option is true, the *split_constraints -light* command includes TCM generated SDC constraints in the constraints files which can be loaded to the design by using *split_constraints*.

See Also

- [write_budgets](#)
- [split_constraints](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

plan.budget.estimate_timing_mode

Controls whether automatic budgeting computes slack allocation based on `estimate_timing` delays.

Data Types

true | false | auto

p

Default auto**Description**

The automatic budgeter can run in one of two modes. In regular mode, automatic budgeting creates constraints based on the worst delays across all active timing corners. In `estimate_timing` mode, the automatic budgeter will base the budget only on delays in the `estimated_corner`. This is desirable when some or all of your design is unoptimized.

Setting the option to "auto" will cause the budgeter to use `estimate_timing` mode when the `estimated_corner` is present in the design and regular mode otherwise.

Setting this option to "true" will cause the budgeter to use `estimate_timing` mode. If design does not have any `estimated_corner`, the command runs *estimate_timing* to estimate delay. It stores estimated timing numbers to the current corner. After *write_budgets* command all these estimated values will be removed.

Setting this option to "false" will cause the budgeter to use regular mode.

See Also

- [compute_budget_constraints](#)
- [estimate_timing](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

plan.budget.extend_paths

Treats `set_input_delay` and `set_output_delay` on block boundaries as if the path started inside of the block

Data Types

boolean

Default false**Description**

It is sometimes desirable to place `set_input_delay` and `set_output_delay` constraints on the boundary pins of a budgeted block. This is useful when you are trying to analyze the timing of top-level timing paths, even when the connectivity inside of your blocks is not defined. For example, you may choose to define a `set_input_delay` constraint on the output pin of a block that is not yet defined.

p

The budgeter is capable of understanding `set_input_delay` and `set_output_delay` constraints that have been defined on the boundary pins of a budgeted block. But by default, these constraints will only be applied to the path that lies outside of the block. This is consistent with the way normal timing analysis is performed. A `set_input_delay` constraint on the output of a block does not imply any timing paths inside of the block. So the budgeter will not generate any boundary constraints for the output pin of the block.

If you set this app option to true, the budgeter will treat `set_input_delay` on the output of a block (or `set_output_delay` on the input of a block) specially. A budget constraint *will* be generated for the block output. The constraint will be calculated as if there were actually a timing path inside of the block. For example, if the constraint on the block output was:

```
set_input_delay -clock sysclock 1.0 block1/OUT1
```

then budgeting will compute a budget as if there were a flop in the block clocked by "sysclk" and a path with delay 1.0 led from the flop to the output.

See Also

- [compute_budget_constraints](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [report_budget](#)
- [set_app_options](#)
- [set_input_delay](#)
- [set_output_delay](#)
- [write_budgets](#)

plan.budget.hold_buffer_margin

Controls the behavior of the *write_budgets* command. When creating budgets for hold time, causes one budget segment to be over-buffered to avoid buffers in other segments.

Data Types

Boolean

Default 0.01

Description

Hold constraints are chosen so that, as much as possible, hold buffers are added to the segments near the start of a timing path. The hold constraint on other segments are set so

p

that those segments do not get too much faster and cause a hold violation on the overall path.

By setting this option to a certain delay, one segment of the path will be constrained so that it meets the path hold constraint with the given delay margin. The extra hold slack created in the selected segment will be allocated to other segments on the same path. The other segments can then get slightly faster without violating their respective hold constraints.

It is a good idea to set this app option to a small positive value. This will make it less likely that hold buffers will be needed in multiple stages of the path, and hold buffers will only be needed on the selected segment where the margin is applied.

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)
- [write_budgets](#)

plan.budget.include_all_clock_inputs

When splitting constraints, declare clocks at block inputs, even if they don't feed clock pins.

Data Types

Boolean

Default false

Description

When creating split constraints, it is necessary to propagate clocks across the block boundaries. By default, if a clock enters a block and does not control any clocked elements, the `split_constraints` command will not include a clock source for the clock. This behavior is useful because it avoids situations where extra unneeded clock sources are included in the block constraints.

If you set this option to true, the clock source will be declared regardless of whether it is used in the block.

p

You can tell whether a branch of a clock tree is used by looking at the "is_clock_used_as_clock" attribute on leaf pins in your tree. For example, if the following returns false:

```
get_attribute [get_pin my_block/clock_buf/Y] is_clock_used_as_clock
```

then the clock is not used by that pin or any of that pin's fanouts.

There are many reasons why a clock is not used. For example:

- The clock does not fan out to any clock pins. (Note that D pins on flops are not clock pins.)
- The clock goes into a multiplexor where the control chooses the other input.
- `set_case_analysis` is used to set constant value is set in the middle of the clock tree. This will mean the downstream flops will not toggle.
- The clock goes through an AND gate where the other input is a constant zero.
- There is a `set_clock_sense` directive in the middle of the clock tree that prevents the downstream flops from being triggered.

See Also

- [split_constraints](#)

plan.budget.include_feedthrough_clocks

When splitting constraints, declare clock in blocks where they feed through

Data Types

Boolean

Default false

Description

When creating split constraints, it is necessary to propagate clocks across the block boundaries. By default, if a clock goes through a block without connecting to any clocked elements, the clock will not be declared for the given block.

If you set this option to true, a clock will be declared as long as it fans out to a clocked element somewhere downstream (not necessarily in the given block).

Note that this app option only applies to the `split_constraints` command. It therefore only works on feedthroughs that are in your original verilog network. Please see the man page for `plan.budget.add_missing_feedthrough_clocks` for details about clock feedthroughs that are added later in the flow.

See Also

- [split_constraints](#)

plan.budget.minimize_virtual_clocks

When this option is true, budgeting tries to minimize the number of virtual clocks that are created for each block.

Data Types

bool

Default true

Description

This option affects the output of *write_budgets* command. When it is true, budgeting tries to minimize the number of virtual clocks that are created in the budgeted constraints for the block. First, clocks that are not used in any *set_input_delay/set_output_delay* commands are removed. Any other constraints that refer to this virtual clock are either modified or removed as appropriate.

Budgeting also tries to combine virtual clocks that were created for different instances of an MIB but have the same delay values and exceptions associated with them for all instances.

When the value of the app option is false, some virtual clocks that are not associated with any other constraints might be created. In the case of MIBs, a distinct set of virtual clocks will be created for each instance.

See Also

- [write_budgets](#)

plan.budget.minimum_unfixed_segment_delay

Specifies how much extra delay will be allocated by budgeting to budget segments that have some fixed delay.

Data Types

float

Default 0

Description

The *compute_budget_constraints* can be used to automatically allocate timing budgets to all the budget segments of timing paths in a hierarchical design. When this app option is non-zero, the budgeting computation special handles budget segments that have some fixed delay associated with them. It estimates how much extra delay will be allocated to these segments and if it is less than the value indicated by the app option, it increases the fixed delay value of the segment, seen by the budgeting engine. This makes it more likely that the budget of the segment will be at least the original fixed delay plus the value of the app option.

This will also change the values seen when using the *report_budget* command. The new fixed delay for the segment will be the delay modified by the value of the app option.

See Also

- [compute_budget_constraints](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [report_budget](#)
- [set_app_options](#)

plan.budget.pessimistic_driving_cells

Controls how the driving cell delay is counted at the input of a budgeted block.

Data Types

Boolean

Default true

Description

At a boundary between block A and block B, there are two delays associated with the driving cell:

- In block A, there is an actual cell that drives the block output. Part of that cell's delay is caused by the load contributed to the output port.
- In block B, there is a delay associated with the *set_driving_cell* constraint that is set on the input port.

By default, budgeting charges both blocks with their respective driving cell delays for setup timing. This is pessimistic, as the same delay is counted twice.

p

If you set this option to *false*, the budgeter maintains greater control over driving cell pessimism. It adjusts delays so that block B is charged only with the driving cell delay that is caused by an excess of capacitive load. The excess is determined by your budget constraints. For example, consider an input pin has the following boundary constraints:

```
set_boundary_budget_constraints -name pin1_constr \\  
    -driving_cell BUFX4 -library my_lib  
set_boundary_budget_constraints -name pin1_constr -corner default \\  
    -fanin_capacitance 0 -load_capacitance 0.005
```

The budget is adjusted so that block B is charged with driving cell delay only when it has input network capacitance greater than 0.005.

Note that this option applies only to setup budgets at block inputs. Hold budgets at block inputs always use the *-load_capacitance* information, because the timing would be optimistic without it.

See Also

- [write_budgets](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

plan.budget.remove_unused_virtual_clocks

When this option is true, budgeting does not output virtual clocks that are not used by any other statements in the file. All the virtual clocks created are used either in a *set_input_delay/set_output_delay* statement or in another constraint.

Data Types

bool

Default true

Description

This option affects the output of *write_budgets* command. When it is true, the output does not include virtual clocks that are created but not used subsequently.

When the value of the app option is false, some virtual clocks that are not associated with any other constraints might be created.

See Also

- [write_budgets](#)

plan.budget.segment_fixed_delay_threshold

This app option works together with the *plan.budget.minimum_unfixed_segment_delay* app option. Specifically, this app option determines which budget segments will be considered as having fixed delay, when the value of the *plan.budget.minimum_unfixed_segment_delay* is non-zero.

Data Types

float

Default -1

Description

When the app option *plan.budget.minimum_unfixed_segment_delay* is non-zero, the budgeting computation special handles budget segments that have some fixed delay associated with them, if the fixed delay is at least the value specified by this app option. This is done to avoid giving extra budget to budget segments with very small fixed delay, that can be segments between abutted pins and would not benefit from extra budget.

When the value of this app option is -1 (the default), the threshold is set to 0.5% of the allowable delay of the path. This value will work well with most designs. If more control is needed, the user can set it to a preferred small delay value.

See Also

- [compute_budget_constraints](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [report_budget](#)
- [set_app_options](#)

plan.budget.split_latencies

Redefine clock latencies during split_constraints

Data Types

none | *prects* | *postcts*

Default none

p

Description

During the `split_constraints` commands, chip-level SDC constraints are split up and separate constraint files are written for child subblocks. By default (when this `app_option` is set to "none"), clock latencies are pushed from the chip level down to each block, without change. For example, if the source clock latency of clock CLK1 is 2 at the top level, the source clock latency will still be 2 at the port boundary in the split constraints for each block.

If you set this `app_option` to "prects", then `split_constraints` will modify the clock latencies in the block -- based on the budget constraints you have set. For example, if you have set:

```
# Set the total (network+source) latency in all blocks
  set_latency_budget_constraints -clock CLK1 \
  -early_latency 0.50 -late_latency 0.60
  # Set the source latency in block B1
  set_latency_budget_constraints -clock CLK1:B1 -source \
  -early_latency 0.42 -late_latency 0.50
```

The latencies of clock CLK1 in block B1 will be modified. The source latency of the clock will be 0.42/0.50. And the network latency of the clock will be adjusted so that the total latency of the block will be 0.50/0.60. If you do not specify a source or total latency with the `set_latency_budget_constraints` command, the chip level value for the latency will be used.

Setting this `app_option` to "postcts" is similar to "prects", except that no latencies will be written for generated clocks with sources inside the block. It is assumed that latencies should propagate to the generated clock, and explicitly setting the latency of a generated clock will cause propagation to fail.

See Also

- [split_constraints](#)
- [set_latency_budget_constraints](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

plan.budget.time_with_margins

Specifies how margins are timed when evaluating budget paths.

Data Types

ignore | *regular* | *fixed*

p

Default fixed**Description**

The *compute_budget_constraints* command evaluates paths in your current circuit and allocates delays among the budgeted blocks. If you have set extra timing margins with the *set_budget_margins* command, those margins can be taken into account when deciding budgets.

If you set this app option to "ignore", the margins are ignored during path analysis. If you set this app option to "regular", the margins are treated like extra optimizable delays during path analysis. If you set this app option to "fixed", the margins are treated like extra fixed, unoptimizable, delays during path analysis.

See Also

- [compute_budget_constraints](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [report_budget](#)
- [set_app_options](#)
- [set_budget_margins](#)

plan.budget.use_ccd_latency

When this option is true, budgeting takes into account latency constraints defined by CCD on startpoints and endpoints of budgeted paths.

Data Types*bool***Default** false**Description**

This option affects the output of *write_budgets* command. When it is true, the output will include path margin constraints for block pins, if the startpoint or endpoint of the budgeted path outside the block has latency constraints defined on it that specify a latency that is different from the latency applied to the virtual clock of the constraint.

The margin will better align the path available delay in the block with the available delay reported at top-level.

See Also

- [compute_budget_constraints](#)
- [write_budgets](#)

plan.budget.write_ocv_files

Causes `write_budgets` to write AOCVM or POCVM tables into files

Data Types

Boolean

Default `true`

Description

The `write_budgets` command can be used to automatically generate timing constraints for design blocks in a hierarchical design. When this app option is true, the command will also write out AOCVM and POCVM tables, if this information is available in the design.

The tables will be written in the budgeting directory, under a directory called `_OCVM_`.

See Also

- [write_budgets](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

plan.budget.write_via_variation_files

Controls the behavior of the `split_constraints` and `write_budgets` commands. When true, the output split or budget directory will include via variation information.

Data Types

bool

Default `false`

Description

When the top-level design has via variation information, the `split_constraints` and `write_budgets` commands can write the information out and create scripts that apply the information to the blocks when the `split_constraints` or `write_budgets` results are loaded in.

p

The via variation information is written out in a subdirectory called *VIA_VARIATION* under the *split_constraints/budgeting* directory. This directory will contain a set of .ivm files with via variation information for all the relevant corners.

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)
- [split_constraints](#)
- [write_budgets](#)

plan.busplan.auto_bus_creation_min_distance

Specifies whether to filter out buses shorter than specified distance.

Data Types

typed length

Default 0

Description

This controls the length of the shortest bus that will be created during automatic bus create. Any bus whose manhattan length is less than this specified distance will be filtered out and not created. Automatic bus creation is done using command *create_busplans -auto*.

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design. Use *report_app_options* to show the value of application options.

See Also

- [create_busplans](#)
- [report_app_options](#)
- [set_app_options](#)

plan.busplan.auto_bus_creation_min_width

Specifies to filter out buses with width less than specified width.

p

Data Types

*integer***Default** 2

Description

This controls the width (ie bit count) of the smallest bus that will be created during automatic bus create. Any bus whose width is less than this specified width will be filtered out and not created. Automatic bus creation is done using command *create_busplans -auto*.

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design. Use *report_app_options* to show the value of application options.

See Also

- [create_busplans](#)
- [report_app_options](#)
- [set_app_options](#)

plan.busplan.channels_include_hm

Specifies whether to treat hard macros as blocks when creating busplan channels

Data Types

*true | false***Default** false

Description

This option determines whether busplan channels used by *route_busplans -quick* are constructed taking hard macros into consideration, or not. If disabled, hard macros are treated as empty space - busplans can be routed on top of their location(s). If enabled, the area used by hard macros will be marked as unavailable for *route_busplans -quick*.

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design. Use *report_app_options* to show the value of application options.

See Also

- [route_busplans](#)
- [report_app_options](#)
- [set_app_options](#)

plan.busplan.channels_include_placement_blockage

Specifies whether to include placement blockages when creating busplan channels

Data Types

true | *false*

Default *false*

Description

This option determines whether busplan channels used by *route_busplans -quick* are constructed taking placement blockages into consideration, or not. If disabled, placement blockages are ignored - busplans can be routed on top of their location(s), if not otherwise unavailable (such as a block or hard macro at that location). If enabled, the area covered by placement blockages will be marked as unavailable for *route_busplans -quick*.

Enabling this option and creating placement blockages can be used to prevent *route_busplans -quick* from routing busplans through/over those locations.

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design. Use *report_app_options* to show the value of application options.

See Also

- [route_busplans](#)
- [report_app_options](#)
- [set_app_options](#)

plan.busplan.load_all_HMs

Specifies to load all HMs at start/end instances.

Data Types

bool

p

Default false**Description**

This controls whether to load all hard macros (HMs) as start/end instances. All HMs includes HMs in nested physical blocks. By default only top level HMs are loaded.

Loading a HM (or actually any cell) as a start/end instance means that *create_busplans* will consider that HM as a possible starting or ending point for a busplan.

It is recommended to set this app option to the desired value before doing any *create_busplans* command. In other words, this should be set before creating the first busplan.

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design. Use *report_app_options* to show the value of application options.

See Also

- [create_busplans](#)
- [report_app_options](#)
- [set_app_options](#)
- [write_busplans](#)

plan.busplan.max_fanout

Specifies the size of high fanout nets for auto bus creation.

Data Types*int***Default** 4000**Description**

During automatic bus creation, netlist tracing is done to find buses. Tracing through high fanout nets is runtime expensive and likely unnecessary to find buses, so if a net is high fanout, then it stops the tracing. This app option controls the definition of what is a high fanout net for this tracing.

Automatic bus creation is done using command *create_busplans -auto*.

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design. Use *report_app_options* to show the value of application options.

See Also

- [create_busplans](#)
- [report_app_options](#)
- [set_app_options](#)

plan.busplan.movebound_aspect_ratio

Specifies the aspect ratio to use when writing out movebounds for buses

Data Types

float

Default 2.0

Description

This application option controls the aspect ratio to use when writing out movebounds with the *write_busplans -implementation_script* command. The default is to make a 2 to 1 aspect ratio where the longer side is perpendicular to the signal direction. For example, by setting this app option to 1.0 you will get square movebounds.

Be warned that the movebound aspect ratio can deviate from desired aspect ratio due to channel sizes. If the channel size is small, the movebound may be distorted as it tries to fit in channel. However, if the channel size is large enough, the aspect ratio will be respected.

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design. Use *report_app_options* to show the value of application options.

See Also

- [create_busplans](#)
- [report_app_options](#)
- [set_app_options](#)
- [write_busplans](#)

plan.busplan.movebound_utilization

Specifies the utilization to use when writing out movebounds for buses

p

Data Types

*float***Default** 0.4

Description

This application option controls the utilization to use when writing out movebounds with the *write_busplans -implementation_script* command. By setting a lower utilization, the movebound area will increase. A higher utilization decreases the movebound area.

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design. Use *report_app_options* to show the value of application options.

See Also

- [create_busplans](#)
- [report_app_options](#)
- [set_app_options](#)
- [write_busplans](#)

plan.busplan.routing_corridor_pitch

Specifies the pitch to use when writing out routing corridors for buses

Data Types

*length***Default** 0

Description

This application option controls the pitch to use when writing out routing corridors with the *write_busplans -implementation_script* command. The routing corridor app option, `plan.busplan.write_routing_corridors` must be true for this to be active. The width of the routing corridor is the pitch times the number of bits in the bus. By default the pitch used is maximum of the vertical and horizontal layer pitch as defined in the net estimation rule for the bus times the app option `plan.busplan.routing_corridor_pitch_multiplier`.

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design. Use *report_app_options* to show the value of application options.

p

See Also

- [create_busplans](#)
- [report_app_options](#)
- [set_app_options](#)
- [write_busplans](#)

plan.busplan.routing_corridor_pitch_multiplier

Specifies a multiplicative factor on the pitch to use when writing out routing corridors for buses

Data Types

float

Default 2.0

Description

This application option controls a multiplicative factor on the pitch use when writing out routing corridors with the *write_busplans -implementation_script* command. This is needed because it may be difficult for the router to route the bus in the minimum sized routing corridor. This app option will be applicable independent of the app option `plan.busplan.routing_corridor`. However, the routing corridor app option, `plan.busplan.write_routing_corridors` must be true for this to be active. The width of the routing corridor is the pitch times the number of bits in the bus. Note that the pitch takes into account all the layers in the `net_estimation_rule` associated with the bus.

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design. Use *report_app_options* to show the value of application options.

See Also

- [create_busplans](#)
- [report_app_options](#)
- [set_app_options](#)
- [write_busplans](#)

plan.busplan.use_soft_movebounds

Specifies to use soft movebounds when writing out movebounds for buses

p

Data Types

*bool***Default** false

Description

This application option controls the whether to write soft movebounds with the *write_busplans -implementation_script* command. By default hard movebounds are written out.

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design. Use *report_app_options* to show the value of application options.

See Also

- [create_busplans](#)
- [report_app_options](#)
- [set_app_options](#)
- [write_busplans](#)

plan.busplan.use_timing

Specifies the amount of timing analysis to use during register planning.

Data Types

none | *clock* | *full***Default** clock

Description

This option controls whether to invoke the timing engine to determine the clock and other timing related information during register planning.

The default value of "clock" ensures that sufficient timing update is performed to automatically determine the clock that drives each bus in the plan. The clock period can then be used to determine a good register spacing for flip-flop on your bus.

The option "none" specifies that no timing update will be triggered. The "none" option reduces the startup time for the tool, but requires that you manually set parameters to determine register spacing on the bus with the *set_net_estimation_rule -parameter register_spacing* command. This allows register planing to determine the register spacing

p

without using the timing engine. See the man page for *DPUI-511* for more possibilities for determining register spacing.

If you set this option to "full", you will trigger a full timing update. This will take more CPU time than the "clock" option.

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design. Use *report_app_options* to show the value of application options.

See Also

- [create_busplans](#)
- [report_app_options](#)
- [set_app_options](#)
- [set_net_estimation_rule](#)

plan.busplan.write_bundles

Specifies whether to write out bundles for buses

Data Types

Boolean

Default false

Description

This application option controls whether bundles are written out with the *write_busplans -implementation_script* command. When this option is true, the output script will contain the top-level nets of each stage of the bus in a *create_bundle* command. Nets inside blocks will not be included because that is not supported by the *create_bundle* command.

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design. Use *report_app_options* to show the value of application options.

See Also

- [create_busplans](#)
- [report_app_options](#)
- [set_app_options](#)
- [write_busplans](#)

plan.busplan.write_max_layer

Specifies whether to write out max_layer constraint for buses.

Data Types

Boolean

Default false

Description

This application option controls whether the max_layer constraint for nets is written out with the *write_busplans -implementation_script* command. When this application option is true, the output script will set the max_layer attribute on all the nets in the bus. The max_layer used is the highest layer set on the net_estimation_rule of the bus.

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design. Use *report_app_options* to show the value of application options.

See Also

- [create_busplans](#)
- [report_app_options](#)
- [report_net_estimation_rules](#)
- [set_app_options](#)
- [write_busplans](#)

plan.busplan.write_min_layer

Specifies whether to write out the min_layer constraint for buses.

Data Types

Boolean

Default false

Description

This application option controls whether the min_layer constraint for nets is written with the *write_busplans -implementation_script* command. When this application option is true, the output script will set the min_layer attribute on all the nets in the bus. The min_layer used is the lowest layer set on the net_estimation_rule of the bus.

p

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design. Use *report_app_options* to show the value of application options.

See Also

- [create_busplans](#)
- [report_app_options](#)
- [report_net_estimation_rules](#)
- [set_app_options](#)
- [write_busplans](#)

plan.busplan.write_routing_corridors

Specifies whether to write out routing corridors for buses.

Data Types

Boolean

Default false

Description

This application option controls whether routing corridors are written out with the *write_busplans -implementation_script* command. With this option, the output script will have all the top-level nets of the bus in a *create_routing_corridor* command. The boundary of the routing corridor will follow the path as defined in the busplan. To more accurately control the boundary, you must define the busplan path such that each segment is a horizontal or vertical line.

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design. Use *report_app_options* to show the value of application options.

See Also

- [create_busplans](#)
- [report_app_options](#)
- [set_app_options](#)
- [write_busplans](#)

plan.clock_trunk.block_fanout_limit

Specifies the block clock pin fanout limit for clock trunk computation in clock trunk planning (CTP).

Data Types

integer

Default 1

Description

This application option effects the node selection criteria of block clock pins while computing the clock trunk during CTP. It specifies the number of fanouts of a node inside a block, including block ports, beyond which the node is considered to be "high-fanout". Clock trunk tracing for end-points stops at nodes marked as "high-fanout".

See Also

- [report_app_options](#)
- [set_app_options](#)
- [synthesize_clock_trunks](#)
- [report_clock_trunk_endpoints](#)

plan.clock_trunk.default_driving_cell

Specifies the driving cell per timing mode for clock trunk planning (CTP) with the *synthesize_clock_trunks* command.

Data Types

string

Default Empty.

Description

This application option affects the behavior of the *synthesize_clock_trunks* command.

Use this application option to provide a global driving cell specification per timing mode for clock trunk planning when the *set_driving_cell* constraint is not specified in the CTS_CONSTRAINT file. The CTS_CONSTRAINT file is specified by the *set_constraint_mapping_file* command. The driving cell specified by this application option applies to all clock input pins of all blocks. The format also supports a "default" driving cell which is applicable to timing modes for which driving cell has not been explicitly specified.

The precedence order for specifying driving cells for the *synthesize_clock_trunks* command is:

1. *set_driving_cell* constraint specified in CTS_CONSTRAINT file provided with *set_constraint_mapping_file*
2. *set_app_options -name plan.clock_trunk.default_driving_cell -value specification_string*

If a driving cell is not specified for a particular timing mode, the default driving cell is automatically selected by the clock trunk planning engine for that mode.

The value string of this application options follows a specific format as described below.

Syntax

```
{ model_name { driving_cell_string }
  mode2_name { driving_cell_string }
  ... {} { driving_cell_string } }
```

where *driving_cell_string* is of data-type "string" and it supports some of the options of *set_driving_cell* command:

```
-lib_cell lib_cell_name
[-rise]
[-fall]
[-min]
[-max]
[-library lib_name]
[-pin pin_name]
[-from_pin from_pin_name]
[-input_transition_rise rising_transition]
[-input_transition_fall falling_transition]
```

```
lib_cell_name      string
lib_name           string
pin_name           string
from_pin_name      string
rising_transition   float
falling_transition  float
```

Examples

Here is an example format for this application option:

```
prompt> set_app_option -name plan.clock_trunk.default_driving_cell -value
\\
      { functional { -library slow \\
                    -lib_cell BUF8X1 \\
                    -input_transition_rise 0.4 \\
                    -input_transition_fall 0.2 } \\
      test      { -library slow -lib_cell BUX2X1 } \\
      {}        { -library slow -lib_cell BUF4X1 } }
```

p

This example implies following driving cell specification for clock trunk planning:

- (a) For "functional" mode: library cell slow/BUF8X1 with 0.4 time units for rising transitions and 0.2 timing units for falling transitions
- (b) For "test" mode: library cell slow/BUF2X1
- (c) For timing modes other than "functional" or "test": library cell slow/BUF4X1

See Also

- [report_app_options](#)
- [set_app_options](#)
- [set_constraint_mapping_file](#)
- [set_driving_cell](#)
- [synthesize_clock_trunks](#)

plan.clock_trunk.enable_mph_constraints**Data Types**

Boolean

Default false

Description

Enable generation of MPH constraints for `synthesize_clock_trunks` command. Only designs having MPH depth more than 2 are considered.

See Also

- [place_pins](#)
- [report_app_options](#)
- [set_app_options](#)

plan.clock_trunk.enable_new_flow

Enables the new flow for clock trunk planning (CTP).

Data Types

boolean

p

Default false**Description**

This option (with true value) can be used to enable new CTP flow. New CTP Flow adds support for: + Non-abutted designs + Thin-channel designs + MPH This option can also be used with fully-abutted designs to overcome the limitations of predecessor CTP.

This is false by default.

See Also

- [report_app_options](#)
- [set_app_options](#)
- [synthesize_clock_trunks](#)
- [report_clock_trunk_endpoints](#)
- [set_clock_trunk_endpoints](#)

plan.clock_trunk.endpoint_limit

Specifies the endpoint limit for endpoint computation in clock trunk planning (CTP).

Data Types

integer

Default 100**Description**

This application option specifies the limit on total number of endpoints computed during clock trunk planning. Both automatically identified endpoints as well as those set manually using `set_clock_trunk_endpoints` are counted for verifying this limit. An error message (CTP-038) is generated in case this limit is exceeded.

See Also

- [report_app_options](#)
- [set_app_options](#)
- [synthesize_clock_trunks](#)
- [report_clock_trunk_endpoints](#)
- [set_clock_trunk_endpoints](#)

plan.clock_trunk.fanout_limit

Specifies the fanout limit for clock trunk computation in clock trunk planning (CTP).

Data Types

integer

Default 1

Description

This application option effects the node selection criteria while computing the clock trunk during CTP. It specifies the number of fanouts beyond which a node is considered to be "high-fanout". Clock trunk tracing for end-points stops at nodes marked as "high-fanout".

See Also

- [report_app_options](#)
- [set_app_options](#)
- [synthesize_clock_trunks](#)
- [report_clock_trunk_endpoints](#)

plan.clock_trunk.flow_control

Data Types

string

Default none

Description

This app option is a master app option to control the different flows of clock trunk planning. The default value of the app option is 'none'. When set to 'none', existing behavior will persist. When the app option value is set to 'optimize_subblocks', THO mode clock trunk planning will take place. When the app option value is set to 'push_down', push down mode of clock trunk place will take place.

See Also

- [place_pins](#)
- [report_app_options](#)
- [set_app_options](#)

plan.clock_trunk.report_truncation_limit

Specifies the limit on number of reported endpoints.

Data Types

unsigned int

Default none

Description

This app option limits the reported endpoints of `report_clock_trunk_qor` to the set number. By default, the value is none, which does not truncate the report.

See Also

- [report_app_options](#)
- [set_app_options](#)
- [synthesize_clock_trunks](#)
- [report_clock_trunk_endpoints](#)
- [set_clock_trunk_endpoints](#)

plan.clock_trunk.seed_clock_pin_placement

During block level CTP, it automatically places block clock pins.

Data Types

boolean

Default true

Description

This option can be used to disable finding/refining of block clock ports during block level CTP. This is TRUE by default. We recommend changing it to false only for multiple-instantiated blocks where clock-ports should be placed but we still need to identify the anchor-buffers.

See Also

- [report_app_options](#)
- [set_app_options](#)

p

- [synthesize_clock_trunks](#)
- [report_clock_trunk_endpoints](#)
- [set_clock_trunk_endpoints](#)

plan.clock_trunk.topo_constr_pin_place_script_name

Name of the Tcl script generated during constraint generation step of CTP pin-placement / port-punching feature.

Data Types

String

Default read_pin_constr_place_pins.tcl

Description

This application option is used to specify name of the tcl script which is generated when *pin_constr_generation* step is executed during CTP pin-placement / port-punching feature. This script contains the commands to read auto-generated topo constraints and run *place_pins*. The script is automatically sourced when *read_pin_constr_and_place_pins* step is executed for *synthesize_clock_trunks*.

See Also

- [place_pins](#)
- [report_app_options](#)
- [set_app_options](#)
- [synthesize_clock_trunks](#)

plan.commit.inner_keepout_margin

Data Types

hard | *soft* | *none*

Default "hard"

Description

This application option allows you to control the inner keepout margin type when running the *commit_block* command. By default, an inner keepout margin is created in the block during commit, and the type is "hard". You can also use this application option to create the margin as "soft", or avoid creating a keepout margin by specifying "none".

p

To control the size, see the man page for the `plan.commit.inner_keepout_margin_size` application option.

This option affects the `commit_block` command only.

Examples

```
set_app_options -list {plan.commit.inner_keepout_margin none}
```

See Also

- [commit_block](#)
- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)

plan.commit.inner_keepout_margin_size

Data Types

list <float>

Default Equal to one cell row height

Description

This application option allows you to control the size of the inner keepout margin when running the `commit_block` command. By default, an inner keepout margin is created in the block during commit, and the size is one cell row height. Use this option to specify a size for the inner keepout margin other than once cell row height.

If only one value is specified, the value is used for all sides. If two values are specified, the first value is applied to all vertical edges and then second value is applied to all horizontal edges. If four values are specified, the values are applied to left, bottom, right, top edges, in sequence.

To control the type of keepout margin (hard, soft, or none), use the `plan.commit.inner_keepout_margin` application option.

This option affects the `commit_block` command only.

Examples

```
set_app_options -list {plan.commit.inner_keepout_margin_size 0.3}
```

Note that the user distance unit MUST append the value of the distance.

See Also

- [commit_block](#)
- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)

plan.commit.outer_keepout_margin

Data Types

hard | *soft* | *none*

Default "hard"

Description

This option allows you to control the type of outer keepout margin created by the *commit_block* command. By default, an outer keepout margin is created in the block during commit, and the type is "hard". Use this application option to create a "soft" margin or create no margin ("none").

To control the size, use the *plan.commit.outer_keepout_margin_size* application option.

This option affects the *commit_block* command only.

Examples

```
set_app_options -list {plan.commit.outer_keepout_margin none}
```

See Also

- [commit_block](#)
- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)

plan.commit.outer_keepout_margin_size

Data Types

list <float>

Default Equal to twice the minimum track size.

Description

This application option allows you to control the size of the outer keepout margin when running the *commit_block* command. By default, an outer keepout margin is created in the block during commit, and the size is twice the size of the minimum track size. Use this application option to specify an outer keepout margin size other than the default.

If only one value is specified, the value is used for all sides. If two values are specified, the first value is applied to all vertical edges and then second value is applied to all horizontal edges. If four values are specified, the values are applied to left, bottom, right, top edges, in sequence.

To control the type of outer keepout that is created, (hard, soft, or none), use the *plan.commit.outer_keepout_margin* application option.

This option affects the *commit_block* command only.

Examples

```
set_app_options -list {plan.commit.outer_keepout_margin_size 0.3}
```

Note that the user distance unit MUST append the value of the distance.

See Also

- [commit_block](#)
- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)

plan.dff.keep_path_detail

Turns on dff path object detail retention.

Data Types

Boolean

p

Default true**Description**

This controls the storage/caching of `dff_path` detail data during path tracing using the `compute_dff_connections` command. The default is to cache intermediate data. This allows for rapid retrieval of the *detail* attribute during subsequent processing. The `get_dff_connections` command is used to return `dff_path` objects.

This option takes effect during the NEXT call to `compute_dff_connections` - simply changing its value has no immediate effect.

The only reason to disable this option is if the design is very large, storing the intermediate detail data is using too much memory and dff features need to be used. Disabling this value will have a very large impact on obtaining detail values of `dff_path` objects or loading detail data in the DFF Connections GUI.

Application options are set using the `set_app_options` command, and they may be specified globally or with respect to a design. Use `report_app_options` to show the value of application options.

See Also

- [compute_dff_connections](#)
- [report_app_options](#)
- [set_app_options](#)

plan.distributed_run.block_host

Specifies which host option to be used for the block during distributed runs.

Data Types*string***Default** ""**Description**

This option allows the user to specify different host options for different blocks in the design. Once this option is set, all distributed commands respect it. It allows the user to control the type of host that distributed jobs for a specific block will run on, and to allocate more resources to bigger or more difficult blocks. By default the block run will use the host option specified in the distributed command.

The option is persistent and will be saved with the block. The host option definition, however is not. The used host options need to be specified again for each new session.

p

Application options are set using the `set_app_options` command, and they may be specified globally or with respect to a block. This application option needs to be associated with a block. Use `report_app_options` to show the value of application options.

Examples

In the following example, 3 different host options are defined:

```
prompt> set_host_options -name small_block \\  
        -submit_command "bsub -q normal"  
prompt> set_host_options -name large_block \\  
        -submit_command "bsub -q huge"  
prompt> set_host_options -name special_block \\  
        -submit_command "rsh" local_machine
```

Then the `block_host` app option is set on specific blocks:

```
prompt> set_app_options -block [get_block block4] \\  
        -name plan.distributed_run.block_host -value large_block  
prompt> set_app_options -block [get_block block5] \\  
        -name plan.distributed_run.block_host -value large_block  
prompt> set_app_options -block [get_block block2] \\  
        -name plan.distributed_run.block_host -value special_block
```

Then, a distributed command is run:

```
create_placement -floorplan -host_options small_block
```

All the jobs associated with blocks that do not have the `plan.distributed_run.block_host` app option specified will run using the `small_block` host option. The jobs for blocks `block4` and `block5` will use the `large_block` host option. The job from `block2` will use the `special_block` host option.

See Also

- [set_host_options](#)
- [run_block_script](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

plan.distributed_run.block_priority

Assigns a hard-coded priority to a specific block for `run_block_script` jobs.

p

Data Types*Int (-10 to 11)***Default** 11**Description**

This option allows the user to control the scheduling priorities of blocks for `run_block_script` jobs.

By default, `run_block_script` treats all blocks as having equal priority (although dependencies are honored) and the default scheduling order of such equal-priority blocks is not always well defined. If not all task of a given job can be executed at once due to license limitations, then using priorities can avoid some sub-optimal schedules.

Priorities for blocks can range from -10 (highest priority) to 10 (lowest priority). Setting block to the default priority - 11 - disables the hard-coded priority assignment.

Note: changing the value of this app option requires a `save_block` call to be permanent. Since `run_block_script` does not save blocks, setting a priority and calling `run_block_script` without saving the block will result in a 1-time change to the priority. When the block is re-loaded after `run_block_script` completes, the previously saved priority (if any) will be restored.

Examples

Change the `block_priority` setting of block A to -10 to run this block before any others (subject to dependency constraints, if applicable, being satisfied).

```
prompt> set_app_options -name plan.distributed_run.block_priority
-block [get_block A] -value -10
```

Reset (disable) the user-defined block priority:

```
prompt> set_app_options -name plan.distributed_run.block_priority
-block [get_block A] -value 11
```

See Also

- [run_block_script](#)

plan.distributed_run.enable_rbs_improvements

Turns on enhancements for distributed block jobs.

Data Types

Boolean

p

Default false**Description**

Set this application option to true to enable enhancements for distributed block job runs, for example when using the command `run_block_script`. This application option only enables the overall enhancement, the individual enhancements are enabled using separate application options.

- Enable the ability to distinguish between different `run_block_script` jobs by creating a sub directory in `work_dir/` named the same as the job name specified by the user. This is controlled by the application option "`plan.distributed_run.rbs_unique_directory_naming`". See there for usage details.
- Enable heartbeat messaging with detailed statistics. This is enabled by the application option "`plan.distributed_run.enable_rbs_heartbeat_messaging`". See there for usage details.
- Allow the user to specify a time the tool will wait after a distributed session has completed and at-least one of the tasks have failed. This is enabled by the application option "`plan.distributed_run.gui_wait_on_fail_timeout`". See there for usage details.

Examples

The following example shows how to enable distributed process enhancements

```
prompt> set_app_options -name
plan.distributed_run.enable_rbs_improvements -value true
```

See Also

- [run_monitor_gui](#)
- [run_block_script](#)

plan.distributed_run.gui_wait_on_fail_timeout

Specifies the tool wait time in seconds for your input after a distributed session fails to complete without errors.

Data Types

int

Default 30**Description**

This application option specifies the wait time in seconds for your input after a distributed session is complete and at least one of the tasks has failed.

p

You can use the process monitor GUI to restart the tasks or kill the overall job. Use the `run_monitor_gui` command to invoke the application. The application will show all running/completed/failed tasks and wait for your action on the failed tasks. You can perform two actions in the monitor GUI:

1. Right-click on the block name (task) and select "Restart Task". This re-runs the block.
2. Right-click on the job name and select "Kill Job". This aborts the wait and the tool continues with normal processing, reloading the library, and the blocks.

If you do not respond within the specified wait time, the tool moves ahead without restarting or rescheduling the failed tasks. The following settings are supported for `plan.distributed_run.gui_wait_on_fail_timeout`:

- 0: No wait. Regardless of whether jobs fail or not, the tool continues without waiting.
- -1: If `run_block_script` fails, the tool goes into a waiting state indefinitely.
- >0: If `run_block_script` fails, the tool waits for the specified value in seconds.

By default, the wait time is 30 seconds.

Note that you need to enable the overall `run_block_script` enhancements as well using the application option "`plan.distributed_run.enable_rbs_improvements`".

Examples

The following example sets the wait-on-fail time to 300 seconds (5 minutes)

```
prompt> set_app_options -name
plan.distributed_run.enable_rbs_improvements -value true
prompt> set_app_options -name
plan.distributed_run.gui_wait_on_fail_timeout -value 300
```

See Also

- [run_monitor_gui](#)
- [run_block_script](#)

plan.distributed_run.heartbeat_interval_time

Defines the time interval in seconds to print the block status for distributed blocks jobs.

Data Types

int

Default 300

p

Description

Use this application option to configure the time interval in seconds to print the block status for distributed block jobs. The default setting is 300, which means that the status table will be printed every 300 seconds (5 minutes). The heartbeat message is only printed if enabled using the application option `plan.distributed_run.enable_rbs_heartbeat_messaging`.

Examples

The following example sets the interval between status message updates to 10 minutes (600 seconds):

```
prompt> set_app_options plan.distributed_run.heartbeat_interval_time  
-value 600
```

See Also

- [run_monitor_gui](#)
- [run_block_script](#)

plan.distributed_run.lib_cell_purpose_list

Lib cells to be copied from Top level for block.

Data Types

string

Default ""

Description

Sending the lib cells list from master to worker process.

Examples

The following example shows how to send lib cells list from master to worker.

```
prompt> set_app_options -name plan.distributed_run.lib_cell_purpose_list  
-value cell_list
```

See Also

- [run_block_script](#)

plan.distributed_run.main_is_idle

Determines whether the main process should be idle during distributed operation

p

Data Types

*Boolean***Default** true

Description

This option allows the user to specify the behavior of the main process during distributed runs. By default, the main process watches for status changes in worker processes, but does nothing else. This leads to the main process becoming "idle" in CPU terms.

Some computing environments detect and terminate idle processes. This could lead to a session with an unusually long distributed run from being identified as "idle" and terminated, potentially losing hours of useful work.

Disabling this setting will cause the main process to run CPU instructions instead of going into an "idle" state while waiting for worker process status changes. These instructions do nothing useful, other than preventing the process from becoming idle and being terminated.

Examples

Change the default main process "idle" setting to disabled

```
prompt> set_app_options -list  
{plan.distributed_run.main_is_idle false}
```

or

```
prompt> set_app_options -name  
plan.distributed_run.main_is_idle -value false
```

Then, when a distributed command is run, the main process will not become idle.

plan.distributed_run.priority_dependencies

Enable heuristic scheduling of run_block_script jobs based on task dependencies.

Data Types

*Bool***Default** false

Description

This option allows the user to enable heuristic scheduling of blocks for run_block_script jobs based on task dependencies.

p

By default, `run_block_script` treats all blocks as having equal priority (although dependencies are honored) and the default scheduling order of such equal-priority blocks is not always well defined. If not all tasks of a given job can be executed at once due to license limitations, then using priorities can avoid some sub-optimal schedules.

Priorities for blocks can range from -10 (highest priority) to 10 (lowest priority). The default priority of a block starts at 0 (medium priority). Scheduling based on task dependencies can increase the priority (lower the priority value) of one or more blocks. Scheduling based on block sizes (see `plan.distributed_run.priority_size(3)`) can decrease the priority (raise the priority value) of one or more blocks. In this way, these two options can work together to assign a priority to a block based on both criteria.

The current heuristic computes a "dependency length" for each block. This represents the number of levels of tasks that are "waiting" for this task to finish. The priority is then adjusted by: $-(3 + 3 \times \text{dependency_length})$. The default priority is 0, but this may have been adjusted by the size-based priority scheduling (`plan.distributed_run.priority_size`), if active. Assuming a block started with a priority 0 and had a dependency length of 2, its new priority would be: $0 - (3 + 3 \times 2) = -9$.

This quickly raises the priority of tasks that are holding up the processing for other tasks, even if they are small in size. For example, a very small block may have been assigned a low priority of 7 or 9 by `plan.distributed_run.priority_size`, but if it has several levels of dependencies, it will still be assigned a medium to high priority. The two options working together will tend to raise the priorities of large blocks that may require the longest runtimes, and blocks that are holding up processing, while reducing the priorities of small blocks that are not holding up any other tasks.

Note: computed priorities will range from -9 to 10, so that using `plan.distributed_run.block_priority` with a value of -10 can take precedence over any computed priority.

Examples

Enable the `priority_dependencies` setting.

See Also

- [run_block_script](#)

plan.distributed_run.priority_size

Enable heuristic scheduling of `run_block_script` jobs based on task size.

Data Types

Bool

p

Default false**Description**

This option allows the user to enable heuristic scheduling of blocks for `run_block_script` jobs based on task size.

By default, `run_block_script` treats all blocks as having equal priority (although dependencies are honored) and the default scheduling order of such equal-priority blocks is blocks is not always well defined. If not all tasks of a given job can be executed at once due to license limitations, then using priorities can avoid some sub-optimal schedules.

Priorities for blocks can range from -10 (highest priority) to 10 (lowest priority). The default priority of a block starts at 0 (medium priority). Scheduling based on task size can decrease the priority (raise the priority value) of one or more blocks. Scheduling based on block dependencies (see `plan.distributed_run.priority_dependencies(3)`) can increase the priority (lower the priority value) of one or more blocks. In this way, these two options can work together to assign a priority to a block based on both criteria.

The current heuristic computes a "block size" value for each block based on standard cell count, port count and net count. Block priorities are distributed over the range of the block size values, such that the largest block priority is set to 1, and the smallest block priority is set to 10. Other blocks are assigned priorities proportional to their size within the block size value range, with values > 1 , < 10 .

This lowers the priority of small blocks so that the largest blocks, which presumably will have the longest runtimes, are scheduled sooner. To avoid the problem of assigning too low a priority to small blocks that have other tasks waiting on them, also enable `plan.distributed_run.priority_dependencies(3)`.

Examples

Enable the `priority_size` setting.

See Also

- [run_block_script](#)

plan.distributed_run.reuse_rbs_workers

Determines whether workers started by `run_block_script` are used for more than one task.

Data Types

Boolean

Default true

p

Description

This option allows the user to control the usage of worker processes started by `run_block_script`. With the new default setting (true), workers are re-used as long as there are still tasks to process. The previous behavior was equivalent to setting this value false.

There are two conditions where this setting can affect the processing. First, when many workers are started, and due to the volume of jobs sent to the compute farm, some jobs are dropped. Or, due to some hosts being overloaded or having the wrong OS configuration, not all jobs start. In the past, this would result in `run_block_script` waiting indefinitely for workers that would never start, and the script would have to be canceled and rerun. In this situation, with the new default setting of true, any workers that do manage to start will keep processing tasks until the entire command is finished. This can avoid a failure due to missing a few workers out of many.

The second condition is where the number of concurrent blocks that are able to run is limited due to licensing, and with the old behavior, a new worker needed to be started anytime a previous worker finished. This is inefficient, and depending on other tasks that may have been started on the same queue in the meantime, can result in a long delay in finished a command, even though a worker had already been obtained.

Disabling this setting will revert to the old behavior of one task per worker. This should only be needed if the new functionality does not function as expected.

Examples

Change the `reuse_rbs_workers` setting to false.

```
prompt> set_app_options -list  
{plan.distributed_run.reuse_rbs_workers false}
```

or

```
prompt> set_app_options -name  
plan.distributed_run.reuse_rbs_workers -value false
```

See Also

- [run_block_script](#)

plan.distributed_run.set_pvt_configuration

This controls sending of `set_pvt` configuration in worker initialization.

Data Types

Boolean

Default true

p

Description

Handle the transmission of `set_pvt` configuration from master to the worker during initialization.

Examples

The following example shows how to pass `set_pvt` configuration from master to worker.

```
prompt> set_app_options -name plan.distributed_run.set_pvt_configuration
        -value true
```

See Also

- [run_block_script](#)

plan.distributed_run.watchdog_timer_delay

Specifies the initial delay for the watchdog timer during distributed runs.

Data Types

integer (range 15 - 9999999)

Default 60

Description

This option allows the user to specify the initial delay (after startup of the distributed worker session) for the watchdog timer check during distributed runs. After the initial delay, the watchdog timer repeats the check at an interval specified by the *plan.distributed_run.watchdog_timer_interval* application option.

The watchdog timer periodically checks that distributed workers are still able to communicate with the main session and terminates the distributed worker session if communication to the main session is lost, in order to reduce wasted compute farm resources. Disable the watchdog timer to stop this check.

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a block. This application option is a global option. Use *report_app_options* to show the value of application options.

Examples

Change the default watchdog timer delay to 4 minutes:

```
prompt> set_app_options -list {plan.distributed_run.watchdog_timer_delay
        240}
```

or

p

```
prompt> set_app_options -name plan.distributed_run.watchdog_timer_delay  
-value 240
```

Then, a distributed command is run:

```
create_placement -floorplan -host_options small_block
```

All distributed jobs started after setting the value will be started with the new value of the app option.

See Also

- [set_host_options](#)
- [run_block_script](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

plan.distributed_run.watchdog_timer_enabled

Specifies the state of the watchdog timer during distributed runs.

Data Types

Boolean

Default true

Description

This option allows the user to specify the state for the watchdog timer (in the distributed worker session) during distributed runs.

The watchdog timer periodically checks that distributed workers are still able to communicate with the main session and terminates the distributed worker session if communication to the main session is lost, in order to reduce wasted compute farm resources. Disable the watchdog timer to stop this check.

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a block. This application option is a global option. Use *report_app_options* to show the value of application options.

Examples

Change the default watchdog timer state to disabled

p

```
prompt> set_app_options -list  
{plan.distributed_run.watchdog_timer_enabled false}
```

or

```
prompt> set_app_options -name  
plan.distributed_run.watchdog_timer_enabled -value false
```

Then, a distributed command is run, and the watchdog timer will be disabled:

```
create_placement -floorplan -host_options small_block
```

All distributed jobs started after setting the value will be started with the new value of the app option.

See Also

- [set_host_options](#)
- [run_block_script](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

plan.distributed_run.watchdog_timer_interval

Specifies the repeating interval for the watchdog timer during distributed runs.

Data Types

integer (range 15 - 9999999)

Default 30

Description

This option allows the user to specify the repeating interval (after the initial delay, which is specified using *plan.distributed_run.watchdog_timer_delay*) of the watchdog timer check during distributed runs.

The watchdog timer periodically checks that distributed workers are still able to communicate with the main session and terminates the distributed worker session if communication to the main session is lost, in order to reduce wasted compute farm resources. Disable the watchdog timer to stop this check.

p

Application options are set using the `set_app_options` command, and they may be specified globally or with respect to a block. This application option is a global option. Use `report_app_options` to show the value of application options.

Examples

Change the default watchdog timer interval to 10 minutes:

```
prompt> set_app_options -list
{plan.distributed_run.watchdog_timer_interval 600}
```

or

```
prompt> set_app_options -name
plan.distributed_run.watchdog_timer_interval -value 600
```

Then, a distributed command is run:

```
create_placement -floorplan -host_options small_block
```

All distributed jobs started after setting the value will be started with the new value of the app option.

See Also

- [set_host_options](#)
- [run_block_script](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

plan.estimate_timing.abstract_add_feedthrough_buffer

Perform actual feedthrough buffer insertion when creating estimated abstract for child blocks from the top level.

Data Types

bool

Default true

Description

This app option controls the feedthrough buffer creation when creating estimated abstract views for child blocks from the top level, either implicitly through `estimate_timing` command or explicitly by calling `create_abstract -all_blocks | -blocks`.

p

When this app option is true, *add_feedthrough_buffer* would be performed on the child blocks before *create_abstract -estimate_timing*.

There are 3 other app options that can be used to control what options to use for the *add_feedthrough_buffer* operation in the blocks.

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

plan.estimate_timing.abstract_feedthrough_buffer_side

Specifies the location to use when inserting a feedthrough buffer during estimated timing abstract creation for child blocks from the top level.

Data Types

string

Default ""

Description

This application option works with the *plan.estimate_timing.abstract_add_feedthrough_buffer* application option to control whether to add buffer on input side or output side or both of a feedthrough net. This is similar to the *-buffer_side* option of the *add_feedthrough_buffers* command.

See Also

- [add_feedthrough_buffers](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

plan.estimate_timing.abstract_feedthrough_logical

Control the *-logical* option for *add_feedthrough_buffers* when creating estimated abstract for child blocks from the top level.

Data Types

bool

Default false

Description

This app option works with `plan.estimate_timing.abstract_add_feedthrough_buffer` and controls whether to buffer logical feedthroughs that can lead to assign statement.

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

plan.estimate_timing.abstract_feedthrough_user_buffer

Controls the *-user_buffer* option for *add_feedthrough_buffers* when creating estimated abstract for child blocks from the top level.

Data Types

string

Default ""

Description

This app option works with `plan.estimate_timing.abstract_add_feedthrough_buffer` and controls the buffer that is used for feedthrough buffering.

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

plan.estimate_timing.enable_fast_ml

Enable the fast mode for machine learning (ML) based *estimate_timing* command.

Data Types

bool

Default false

Description

This app option controls the fast mode for ML support feature in *estimate_timing* command.

See Also

- [estimate_timing](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

plan.estimate_timing.enable_ml_cell_sizing

Enable cell sizing for fast mode of machine learning (ML) based *estimate_timing* command.

Data Types

bool

Default false

Description

This app option controls the cell sizing for fast mode of ML support feature in *estimate_timing* command.

See Also

- [estimate_timing](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

plan.estimate_timing.enable_multi_threading

Enable multi-threading in *estimate_timing* command.

Data Types

bool

Default false

Description

This app option controls enabling of multi-threading in *estimate_timing* command.

See Also

- [estimate_timing](#)
- [create_abstract](#)
- [compute_budget_constraints](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

plan.estimate_timing.honor_routes

Consider the layers and topology of existing routes for net delay estimation.

Data Types

bool

Default false

Description

This app option controls the route support feature in *estimate_timing* command or *create_abstract -estimate_timing*.

When this app option is true, net's existing global route or detail route will be honored. If the route is partial, missing/incomplete segments will be estimated. The existing route will be used as guidance for virtual buffer locations.

See Also

- [estimate_timing](#)
- [create_abstract](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

plan.estimate_timing.maximum_cell_delay

Specifies a maximum cell delay for the *estimate_timing* command.

Data Types

float

Default -1.0

Description

This application option species a maximum cell delay for the *estimate_timing* and *create_abstract -estimate_timing* commands. If the cell delay value calculated by the *estimate_timing* command for a given cell is larger than the value specified by this application option, the estimated value is replaced by application option value.

Use the *plan.estimate_timing.maximum_net_delay* application option to specify a maximum net delay value.

When the calculated delay value of a cell is replaced by the application option value, the *is_et_delay_clipped* attribute is set to true on the output pins of the cell.

See Also

- [create_abstract](#)
- [estimate_timing](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

plan.estimate_timing.maximum_net_delay

Specifies a maximum net delay for the *estimate_timing* command.

Data Types

float

Default -1.0

Description

This application option species a maximum net delay for the *estimate_timing* and *create_abstract -estimate_timing* commands. If the net delay value calculated by the

p

estimate_timing command for a given net is larger than the value specified by this application option, the estimated value is replaced by application option value.

Use the *plan.estimate_timing.maximum_cell_delay* application option to specify a maximum cell delay value.

When the calculated delay value of a net is replaced by the application option value, the *is_et_delay_clipped* attribute is set to true on the output pins of the net driver.

See Also

- [create_abstract](#)
- [estimate_timing](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

plan.estimate_timing.ml_model_dir

Specifies the location of machine learning (ML) model directory during ML based *estimate_timing* command. This directory contains the trained ML model for conducting ML based *estimate_timing* command.

Data Types

string

Default ""

Description

This application option works with the ML based *estimate_timing* command.

See Also

- [estimate_timing](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

plan.estimate_timing.optimize_layers

Perform layer promotion when estimating timing for child blocks or top level.

Data Types

bool

Default false

Description

This app option controls the layer promotion feature in *estimate_timing* command or *create_abstract -estimate_timing*.

When this app option is true, layer promotion would be performed on the child blocks by *create_abstract -estimate_timing* or on top level by *estimate_timing*.

See Also

- [estimate_timing](#)
- [create_abstract](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

plan.estimate_timing.optimize_layers_count

Define how many layers from top most will be used by layer promotion when estimating timing for child blocks or top level.

Data Types

integer

Default 4

Description

This app option controls the total counts of layers used by layer promotion in *estimate_timing* command or *create_abstract -estimate_timing*.

When this app option is set, tool will use those number of layers counted from top most layers without ignored layers setting. If this app option is not set, tool will use the default value count of layers.

See Also

- [estimate_timing](#)
- [create_abstract](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

plan.estimate_timing.use_ml_model

Use machine learning (ML) techniques to estimate delay during *estimate_timing* command.

Data Types

bool

Default false

Description

This app option controls the ML support feature in *estimate_timing* command.

See Also

- [estimate_timing](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

plan.explore.default_threshold

Specifies the default threshold used in *explore_logic_hierarchy* for the current session. If the command line option *explore_logic_hierarchy -threshold* is specified, the current operation will take the threshold from the command.

Data Types

float

Default 0.03

p

Description

During hierarchy exploration (using the command *explore_logic_hierarchy*), the default threshold is 0.03 such that any logical hierarchy will be ignored if its size is smaller than 3 percent of total size (for option *-create_module_boundary*) or the module boundary size (for option *-expand*). This app option is to change this default value for the session.

See Also

- [explore_logic_hierarchy](#)

plan.explore.place_empty_modules

Specifies that empty modules should be placed by *explore_logic_hierarchy -place*.

Data Types

boolean

Default true

Description

During hierarchy exploration, module boundaries get placed using the command *explore_logic_hierarchy -place*. Some module boundaries can be empty and thus have no area. By default, some small area is given to empty modules so they can be placed.

By setting this app option to false, empty module boundaries will not be placed.

See Also

- [explore_logic_hierarchy](#)

plan.floorplan.core_area_accommodate_macro

Specifies whether the core area can accommodate the soft/hard macro largest size.

Data Types

Enum

Default "tolerate"

Description

This option allows you to specify the behavior when core area cannot accommodate the largest macro size.

p

If the value is "tolerate", the *initialize_floorplan* command will continue without adjusting the core area length and only report warning messages.

If the value is "error", the *initialize_floorplan* command will stop if the core area cannot accommodate the largest macro size and report errors.

If the value is "repair", the *initialize_floorplan* command will adjust the core area to accommodate the largest macro size and report which sides have been adjusted.

See Also

- [initialize_floorplan](#)

plan.floorplan.enable_adjust_core_area_for_hybrid_design

Specifies how to adjust the core area for a hybrid design.

Data Types

Enum

Default "false"

Description

This option allows you to specify the behavior when calculating the core area for a hybrid design.

If the value is "false", the *initialize_floorplan -row pattern* command uses the existing calculation method to calculate the core area and the core area is derived based on the core utilization.

If the value is "true", the *initialize_floorplan -row pattern* command calculates the maximum value of the core area based on the utilization of each site in hybrid row design.

If the value is "balance", the *initialize_floorplan -row pattern* command derives the core area based on the balanced cell ratio, which is equal to the site_row ratio in the row_pattern.

The default is "false".

See Also

- [initialize_floorplan](#)

plan.floorplan_rule.derive_keepout_behavior

Specifies what to do in presence of existing keepouts when deriving macro keepouts.

p

Data Types*Enum***Default** "override"**Description**

This option allows user to specify the behavior when deriving macro keepouts, if there are existing macro hard keepouts. If the value is "skip", *fix_floorplan_rules -derive_macro_keepout* will leave the existing hard keepouts unchanged. If the value is "override", *fix_floorplan_rules -derive_macro_keepout* will replace the existing keepouts with the calculated value. If the value is "enlarge", *fix_floorplan_rules -derive_macro_keepout* will replace the existing keepouts with the maximum of the calculated value and the existing value. The default value is "override".

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design. Use *report_app_options* to show the value of application options. This application option is design specific.

See Also

- [fix_floorplan_rules](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

plan.floorplan_rule.fill_in_partial_row

Specifies fixer need to fill in the partial site_row that seized by std_cell_area or boundary_cell_region with placement_blockage .

Data Types*enum***Default** true**Description**

This option allows user to specify the behavior of *fix_floorplan_rules*. if the value is "true" and the fixer is allowed to adjust *std_cell_area* or *boundary_cell_region* shape, the fixer will always fill in the partial site_row that seized by *std_cell_area* or *boundary_cell_region* which is not snapped to site_row grid, no matter the design has floorplan rule violation or not. if the value is "false", the fixer will not fill in the partial site_row occupied by *std_cell_area* or *boundary_cell_region* with *placement_blockage*. if the value is

p

"std_cell_area", fix_floorplan_rules only align the std_cell_area to site_row_grid. if the value is "boundary_cell_region", fix_floorplan_rules only align the boundary_cell_region to site_row_grid.

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design. Use *report_app_options* to show the value of application options. This application option is design specific.

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

plan.floorplan_rule.pre_opt_switch

Specifies the pre_opt switch status when checking floorplan rules.

Data Types

Enum

Default off

Description

This option allows user to specify the pre_opt switch status to control the *check_floorplan_rules* behavior. Assume the design has two floorplan rules, they have the same settings except the rule name and "-project_cell_groups" option (one with and one without). Let's called the rule without "-project_cell_groups" option as "Ground Rule" (Rule1), the rule with "-project_cell_groups" option as "Relaxed Rule" (Rule2). If the app option value is "on", checker skips all "Ground Rule" and check "Relaxed Rule" as the "Relaxed Rule" without "-project_cell_groups" setting. If the app option value is "off", checker works as usual. The default value is "off".

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design. Use *report_app_options* to show the value of application options. This application option is design specific.

See Also

- [check_floorplan_rules](#)
- [get_app_option_value](#)

p

- [report_app_options](#)
- [set_app_options](#)

plan.floorplan_rule.remove_existing_blockages

Specifies if user needs to remove existing blockages firstly when fix_floorplan_rules.

Data Types

enum

Default true

Description

This option allows user to specify the behavior of fix_floorplan_rules command. The "for_floorplan_rule" blockages are created by the fix_floorplan_rule command, which are surround the macros and used to fix some floorplan rule violations (for example, blockage to macro halo rule, std_cell_area to macro spacing/length rule). In next possible step, create_placement command might move the macros, but the "for_flooprlan_rule" blockages won't be moved with the macros. This causes some new floorplan rule violations which are hard to fix unless you remove the related blockages. If this option is set to "true", the fixer removes the "for_flooprlan_rule" blockages firstly. If this option is set to "false", the "for_flooprlan_rule" blockages won't be removed firstly.

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design. Use *report_app_options* to show the value of application options. This application option is design specific.

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

plan.flow.design_view_only

Specifies whether a block should stay as design view. In this case, the tool will not create an abstract view for the block.

Data Types

Boolean

Default false

p

Description

In a multi-level physical hierarchy design, only one level of physical hierarchy can be an abstract view. By default, the lowest level of blocks with disk write permission is the abstract view.

Use this application option to change this behavior and keep the block as a design view. If you do not want a block at the lowest level of physical hierarchy to be an abstract view, set this application option to true on that block reference. If `plan.flow.design_view_only` is true for the block, the block will be kept as design view. For a higher level block, if all its descendant blocks have this setting to false, then this higher level block can be an abstract view.

Using this application option, you can control exactly which blocks will use and abstract view.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)

plan.flow.initialize_floorplan_fix_core_area_from_lowestleft

Specifies whether to adjust the lowest site_row.

Data Types

Enum

Default "false"

Description

This option is to support fixing floorplan of rectilinear shape from the lowestleft point (tool will start looking for the lowest point and then followed by left most point) deduced from the block shape.

If the value is "false", the *initialize_floorplan* command will fix from left-low point.

If the value is "true", the *initialize_floorplan* command will fix from low-left point.

See Also

- [initialize_floorplan](#)

plan.flow.overwrite_work_dir

Remove all files in the work directory of the command before running.

Data Types

bool

Default false

Description

There are several commands in design planning that would run operations inside child blocks of the current block. These commands need to create files for each child block under a work directory. Running such a command repeated can produce many log files on disk under a particular work directory.

This app option controls the command to remove all existing files in its corresponding work directory before starting a new run.

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

plan.flow.segment_rule

Specifies the segment parity rule type to honor in design planning.

Data Types

list

Default ""

Description

There are four types of segment parity rule in design planning: `horizontal_even`, `horizontal_odd`, `vertical_even`, and `vertical_odd` as follows:

horizontal_even: All horizontal edges of the placeable area and all placeable site rows must be an even number of the site width.

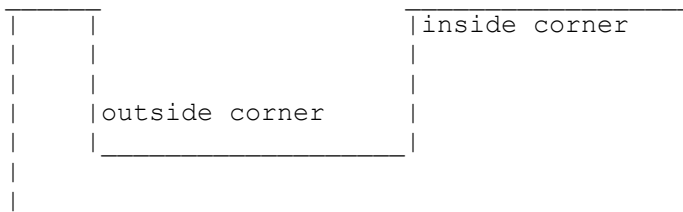
horizontal_odd: All horizontal edges between two inside corners or two outside corners, and all placeable site rows, must be an odd number of the site width. All horizontal edges between one inside corner and one outside corner must be an even number of the site width.

p

vertical_even: All vertical edges of the placeable area and all placeable site columns must be an even number of the site height.

vertical_odd: All vertical edges between two inside corners or two outside corners, and all placeable site rows, must be an odd number of the site width. All vertical edges between one inside corner and one outside corner must be even number of the site width.

This application option accepts the four keywords: "horizontal_even", "horizontal_odd", "vertical_even" and "vertical_odd". You can specify either some of them or none of them. "horizontal_even" and "horizontal_odd" can't be used together, "vertical_even" and "vertical_odd" can't be used together, otherwise, only the first one of them will be used.



See Also

- [get_app_option_value](#)
- [initialize_floorplan](#)
- [report_app_options](#)
- [set_app_options](#)
- [shape_blocks](#)

plan.flow.target_site_def

Specify the site def to honor in design planning if segment parity rule is honored.

Data Types

string

Default ""

Description

The site def specified in this app option will be used by `initialize_floorplan` and be used to find the site rows by `shape_blocks` for the segment parity rule.

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)
- [initialize_floorplan](#)
- [shape_blocks](#)

plan.flow.track_pattern_support_prf_based_initialize_floorplan_for_site_def

Specifies whether the tool will honor command option -site_def option for the PRF based initialize_floorplan.

Data Types

Boolean

Default false

Description

During the floorplan_rule honored floorplan initialization, the track pattern can be defined both under site section and row_pattern section. Once app_option plan.flow.track_pattern_support_prf_based_initialize_floorplan_for_site_def set as true, the tool will honor command option -site_def option for the PRF based initialize_floorplan.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)

plan.interconnection.enforce_repeater_hiers

Data Types

Boolean

Default true

p

Description

This option controls topology repeaters placement in TIP flow. In repeater optimization, tool recognizes the existing repeaters in the netlist and reconcile them in terms of physical or logical hierarchies to the topology repeaters.

By default, this option is set to true to honor existing repeaters in its physical/logical hierarchies.

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design. Use *report_app_options* to show the value of application options.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)

plan.interconnection.exclusive

Data Types

Boolean

Default false

Description

This application option controls how the command *optimize_topology_plans* treats the pin constraints.

If its value is false (the default), the individual pin constraints, bundle pin constraints, block pin constraints, and topological pin constraints are loaded together with the topology plans. The combination of all of them are honored by *optimize_topology_plans*. If they have conflict settings, the priority from the highest to the lowest is according to the following order:

- topology plans and the associated net estimation rules
- topological pin constraints
- individual pin constraints
- bundle pin constraints
- block pin constraints

p

If its value is true, then only topology plans and the associated net estimation rules are loaded and honored by command *optimize_topology_plans*. The topological pin constraints, individual pin constraints, bundle pin constraints, and block pin constraints are ignored.

By default, this option is false.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)

plan.interconnection.keep_polarity_within_block

Data Types

Boolean

Default true

Description

This option controls topology repeaters polarity in TIP flow. Tool recognizes inverter repeaters and maintain the polarity in each block or as the full path.

By default, this option is set to true to maintain the polarity within the block. Set this option to false to allow changing the polarity of sub-blocks of the path as long as the full path polarity is maintained.

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design. Use *report_app_options* to show the value of application options.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)

plan.macro.align_pins_on_parallel_edges

Specifies whether to align pin shapes along macro edges in the direction parallel to that edge during color matching of pin shapes and tracks.

p

Data Types

Boolean

Default false

Description

This application option specifies whether pins are aligned in both the x- and y-directions, or are aligned only with the direction for the layer they occupy. If this application option value is false, the tool expects pin shapes to be close to an edge of a macro and to align the pin shapes in the direction perpendicular to the macro edge. For example, if pin shapes are placed along the left edge of a macro, the tool aligns the pin shapes on a layer with a horizontal preferred routing direction with horizontal tracks on that layer. For these pins, alignment is not enforced with vertical tracks. This is particularly useful for the designs where each pin has two pin shapes: one on a horizontal layer and one on a vertical layer. For these two pin shapes, the tool selects which shapes to align based on the macro edge direction and ignores the other shapes. This avoids many false errors, as every pin shape on the vertical layer is likely not to be aligned, but that usually does not matter, as the routing will be on the horizontal layer.

If the option value is true, the tool check all possible alignment violations.

The default is false.

Application options are set using the `set_app_options` command, and they may be specified globally or with respect to a design. Use `report_app_options` to show the value of application options. This application option is design specific.

Examples

```
prompt> set_app_options -block [current_design] \\  
-list { plan.macro.align_pins_on_parallel_edges true }
```

See Also

- [create_placement](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [report_placement](#)
- [set_app_options](#)

plan.macro.apply_spacing_rules_to_boundary

Specifies whether to apply spacing rules between macro and boundary/blockage.

p

Data Types

Boolean

Default true

Description

During the execution of the *create_placement -floorplan* command, the spacing rules (which are specified using *plan.macro.spacing_rule_widths* and *plan.macro.spacing_rule_heights* options) are applied between two macros. This app options specifies whether the spacing rules are also applied between macro and boundary/blockage of type *hard_macro*. The default value is true.

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design. Use *report_app_options* to show the value of application options. This application option is design specific.

See Also

- [create_placement](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

plan.macro.auto_buffer_channels

Specifies whether to insert buffer channels of automatically calculated variable size.

Data Types

Boolean

Default false

Description

When true, during the execution of the command *create_placement -floorplan*, macro placement will insert channels for future buffer placement once the width of the macro stack reaches the buffer-to-buffer distance from the database. The size of the channels will vary; it will be related to the buffer size from the database and the number of pins whose nets are expected to need buffering. This means that, in general, channels further away from core edges will be larger, as there will be more macro pins to buffer.

The buffer size and the buffer-to-buffer distance will be obtained using the default rule, unless another net estimation rule is specified using *plan.macro.buffer_rule_name*. To view the buffer size and the buffer-to-buffer distance, use *report_net_estimation_rule*.

p

There is a similar functionality (buffer channel insertion) where values for max macro stack size and buffer channel size are provided by user through application options instead of being auto-calculated. When this option is true, the values of those application options will be ignored. The names of the options that will be ignored are *plan.macro.max_buffer_stack_width*, *plan.macro.max_buffer_stack_height*, *plan.macro.buffer_channel_width*, and *plan.macro.buffer_channel_height*.

If the option *plan.macro.immediately_buffer_ios* is true, the tool will insert buffers before the first row/column of macros to provide space for immediate buffering of IOs.

Note that the buffer channels for the macros along the middle of the core/block edges will be inserted only in the direction parallel to the edge. For example, if the macros are being placed along the middle of the horizontal edge, only horizontal channels will be created. For macros near the corners, channels will be created in both directions. The tool will automatically determine the cut off line for corner vs. middle of the edge.

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design. Use *report_app_options* to show the value of application options. This application option is design specific.

See Also

- [create_placement](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

plan.macro.auto_macro_array_max_height

Specifies the maximum height of an auto-generated macro array.

Data Types

length

Default 0m

Description

This option specifies the maximum height of an auto-generated macro array created during *create_placement -floorplan*; note that the value of 0 is an exception, it means no limit. The default is 0. The option type is length, which means that it must be specified with the length unit.

p

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design. Use *report_app_options* to show the value of application options. This application option is design specific.

See Also

- [create_placement](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

plan.macro.auto_macro_array_max_num_cols

Maximum number of columns in an auto-generated macro array.

Data Types

integer

Default 0

Description

This option specifies the maximum number of columns in an auto-generated macro array created during *create_placement -floorplan*; note that the value of 0 is an exception, it means no limit. The default is 0.

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design. Use *report_app_options* to show the value of application options. This application option is design specific.

See Also

- [create_placement](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

plan.macro.auto_macro_array_max_num_rows

Maximum number of rows in an auto-generated macro array.

p

Data Types

*integer***Default** 0

Description

This option specifies the maximum number of rows in an auto-generated macro array created during *create_placement -floorplan*; note that the value of 0 is an exception, it means no limit. The default is 0.

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design. Use *report_app_options* to show the value of application options. This application option is design specific.

See Also

- [create_placement](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

plan.macro.auto_macro_array_max_width

Specifies the maximum width of an auto-generated macro array.

Data Types

*length***Default** 0m

Description

This option specifies the maximum width of an auto-generated macro array created during *create_placement -floorplan*; note that the value of 0 is an exception, it means no limit. The default is 0. The option type is length, which means that it must be specified with the length unit.

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design. Use *report_app_options* to show the value of application options. This application option is design specific.

p

See Also

- [create_placement](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

plan.macro.auto_macro_array_minimize_channels

Specifies the way macros belonging to auto generated arrays are oriented during hard macro placement.

Data Types

Enum

Default "pins_in"

Description

This option controls the orientation of macros belonging to arrays created during *create_placement -floorplan*. Possible values are "none", "pins_in", and "pins_out". The default is "pins_in".

The option makes a difference only for macros with pins on one side only. For simplicity, let's assume that the sides with pins are vertical (similar argument, with obvious changes, will apply if they are horizontal).

If the value of the option is "none", there is no restriction how macros are flipped to minimize wirelength. This will frequently produce situations where, within an array, all macro sides with pins are facing the same way, typically towards the middle of the design, thus creating a channel between any two columns of macros. If the value is "pins_in", macros are flipped so that sides with the pins on neighboring macros are facing each other; the opposite sides without pins will be back-to-back, hence producing channels every two columns. If the value is "pins_out", macros are flipped so that sides without the pins on neighboring macros are placed back-to-back; the opposite sides with pins will be facing each other. The values pins_in and pins_out are similar in that they both reduce number of channels approximately by half and they both produce pins facing and opposite sides back-to-back, but what is done first - pins facing each other or back-to-back opposite sides - causes different results when there is an even number of columns. With pins_in, the first and the last column will have pins facing inside the array, while with pins_out, the first and the last column will have pins facing outside the array, producing one channel less than pins_in. The exception is that in pins_out case, if the first or last column is against the boundary, the macros in that column will be flipped so that pins will face away from the boundary.

p

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design. Use *report_app_options* to show the value of application options. This application option is design specific.

See Also

- [create_placement](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

plan.macro.auto_macro_array_size

Specifies the amount of array packing to use during hard macro placement.

Data Types

Enum

Default "high"

Description

This controls the amount of macro array packing to use during *create_placement -floorplan*. Possible values are "none", "low", "medium", and "high". The default is "high". If the value is "none", no array packing happens; "high" produces more array packing, i.e. potentially larger arrays than "low", with "medium" being in between.

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design. Use *report_app_options* to show the value of application options. This application option is design specific.

See Also

- [create_placement](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

plan.macro.auto_pin_blockages_min_pins

Specifies the minimal number of clustered pins to use the full width of pin blockages created during freeform macro placement.

p

Data Types

*integer***Default** 5

Description

This option specifies the minimal number of clustered pins to use the full width of the blockages around pins at the end of the *create_placement -floorplan* command if *plan.macro.style* is freeform. When less than the option value of pins are clustered together, the created blockage will be small; otherwise, the blockage size is determined by the *plan.macro.auto_pin_blockages_width* option.

Application options are set using the *set_app_options* command, and they might be specified globally or with respect to a design. Use *report_app_options* to show the value of application options. This application option is design-specific.

See Also

- [create_placement](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

plan.macro.auto_pin_blockages_width

Specifies the width (if the pins are on the vertical block edges) or height (if pins are on horizontal block edges) of pin blockages created during freeform macro placement.

Data Types

*length***Default** -1m

Description

When positive, this option specifies the width of the blockages around pins at the end of the *create_placement -floorplan* command if *plan.macro.style* is freeform. When negative, the width is auto-determined. Note that, in either case, isolated pins (or pins in a small group) may have smaller blockages; to check how we determine whether a group of pins is small, see *plan.macro.auto_pin_blockages_min_pins* option. For more information about these blockages, see the *plan.macro.create_auto_pin_blockages* application option. The default for the *plan.macro.auto_pin_blockages_width* option is -1.

p

Application options are set using the *set_app_options* command, and they might be specified globally or with respect to a design. Use *report_app_options* to show the value of application options. This application option is design-specific.

See Also

- [create_placement](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

plan.macro.buffer_channel_height

Specifies height of buffer channels between macros.

Data Types

length

Default 0m

Description

During the execution of the command *create_placement -floorplan*, once the height of the macro stack reaches the value specified using the *plan.macro.max_buffer_stack_height* option, a channel will be created for the buffer placement. This option specifies the height of that channel. The default value is 0, which means no channel will be created.

This option will be ignored if *plan.macro.auto_buffer_channels* option is true, as in that case the value will be automatically calculated.

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design. Use *report_app_options* to show the value of application options. This application option is design specific.

See Also

- [create_placement](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

plan.macro.buffer_channel_width

Specifies width of buffer channels between macros.

Data Types

length

Default 0m

Description

During the execution of the command *create_placement -floorplan*, once the width of the macro stack reaches the value specified using the *plan.macro.max_buffer_stack_width* option, a channel will be created for the buffer placement. This option specifies the width of that channel. The default value is 0, which means no channel will be created.

This option will be ignored if *plan.macro.auto_buffer_channels* option is true, as in that case the value will be automatically calculated.

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design. Use *report_app_options* to show the value of application options. This application option is design specific.

See Also

- [create_placement](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

plan.macro.buffer_rule_name

Specifies which net estimation rule to use for calculating buffer size and buffer-to-buffer distance when inserting buffer channels.

Data Types

string

Default Empty

Description

During the execution of the command *create_placement -floorplan*, if the value of *plan.macro.auto_buffer_channels* option is true, buffer channels will be inserted using buffer size and buffer-to-buffer distances (i.e. buffer spacings) read from the database.

p

This option specifies which net estimation rule to use to get these values. Such rules are created using the `set_net_estimation_rule` command and can be viewed using the `report_net_estimation_rule` command.

If the option is not specified, or is reset to the default value (empty string), or if a rule with this name does not exist, the default rule's values will be used. Furthermore, if either one of the buffer size and buffer-to-buffer distance can not be found using this rule, the value will be obtained using the default rule.

This option will be ignored if `plan.macro.auto_buffer_channels` option is false.

Application options are set using the `set_app_options` command, and they may be specified globally or with respect to a design. Use `report_app_options` to show the value of application options. This application option is design specific.

See Also

- [create_placement](#)
- [set_net_estimation_rule](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

plan.macro.congestion_iters

Specifies the number of congestion reduction iterations to do for macro placement.

Data Types

integer

Default 1

Description

The `create_placement -floorplan -congestion` command adjusts the width of the channels between the macros based on congestion estimation.

The `plan.macro.congestion_iters` application option specifies the number of times the congestion is estimated and the channels are adjusted. The default value is 1, which means the channel widths are updated once.

This is a block-level application option.

p

See Also

- [create_placement](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

plan.macro.conservative_auto_pin_blockage_size

Specifies whether to have smaller auto-generated port blockages.

Data Types

Boolean

Default false

Description

Auto-generated port blockages are created by the *create_placement -floorplan* command, if *plan.macro.create_auto_pin_blockages* app option is set to true. The size of the blockages is either automatically calculated or user defined by *plan.macro.auto_pin_blockages_width* app option. The automatically calculated size of the blockages tend to be larger for large groups of pins. If this option (*plan.macro.conservative_auto_pin_blockage_size*) is true, the auto calculation algorithm will be more conservative, i.e., it will generally produce smaller blockages for larger groups of pins.

The default value of this option is false.

Application options are set using the *set_app_options* command, and can be specified globally or with respect to a design. Use *report_app_options* to show the value of application options. This application option is design level.

See Also

- [create_placement](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

plan.macro.create_auto_pin_blockages

Specifies whether to create blockages around pins when running freeform macro placement.

Data Types

Boolean

Default true

Description

When true, this option creates blockages around pins during the *create_placement -floorplan* command if *plan.macro.style* is *freeform*. These blockages are of type *hard_macro* and they keep hard macros away from the design pins. *plan.macro.auto_pin_blockages_width* application option controls the thickness of the blockages. Whether these blockages are automatically removed at the end of placement is controlled by the *plan.macro.remove_auto_pin_blockages* application option. The default for the *plan.macro.create_auto_pin_blockages* option is true.

These placement blockages are created with a special attribute, "is_derived", so that they can be selected with the *get_placement_blockages -filter "is_derived==true"* command. Note that automatic channel blockages also get this attribute.

Application options are set using the *set_app_options* command, and they might be specified globally or with respect to a design. Use *report_app_options* to show the value of application options. This application option is design-specific.

See Also

- [create_placement](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

plan.macro.create_temporary_pg_grid

Specifies a command or script to use to create a temporary power and ground (PG) grid for congestion estimation during macro placement.

Data Types

String

p

Description

When executing `create_placement -floorplan -congestion`, the tool uses a routing-based congestion estimation to try to reduce the congestion over and between macros. To enhance the precision of the congestion estimation, you can let the tool do a temporary PG grid insertion right before the congestion estimation routing is done by using a script. In the same script, you can also insert temporary power switches.

This option is meant only for physically flat designs. For physically hierarchical designs, see the man page for the option, `plan.macro.estimate_pg`.

If the option is set to the name of a Tcl script file, the tool sources the file. Otherwise, the input is treated as a Tcl command. The specified file is sourced and executed just before congestion estimation routing. After the congestion estimation routing is done, the objects belonging to the temporary grid should be removed using the script or command specified using the `plan.macro.remove_temporary_pg_grid` option; if that option is not specified, all power and ground shapes, vias, and power switches are removed from the design.

Note that the generated power and ground grid is only temporary for the congestion estimation. To save run time, set the `-ignore_drc` option in the `compile_pg` command in the script to source. Setting this option suspends any checks for DRC violations.

By default, no temporary PG grid is created.

This application option can be set using the `set_app_options` command. It should be set for the top level design.

See Also

- [create_placement](#)
- [compile_pg](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

plan.macro.cross_block_connectivity_planning

Specifies if and in what way cross block connectivity planning should be done for macro placement.

Data Types

soft | hard | false | unset

Default unset

Description

During the execution of the command *create_placement -floorplan*, the tool can analyze the connectivity between sub-blocks and try to keep a section of the boundary between blocks free of macros. The resulting free edge can then be used to place pins later in the flow. To determine the edge width required for the pins, block level pin constraints are taken into account.

When using the value *soft*, this will be interpreted as a soft constraint, which can be violated if other considerations are deemed more important. When using *hard*, it is interpreted as a hard constraint, which might lead to a solution that has the free edge space required, but might have to sacrifice other properties to achieve this. The value *false* means that no cross block communication planning is done. The default value, *unset*, means inherit the value from the parent block, if there is a parent; if there is no parent (i.e. this is the top block), *unset* is interpreted as *false*. Therefore, if this app option is not set for any block, no cross block communication planning is done.

Similar behavior can be achieved for other placement regions, namely movebounds and voltage areas, using attribute *interface_connectivity_planning*. That attribute is very much like this app option, with the same values (including the default), and the same behavior. Note that blocks can be considered parents of placement regions and vice versa, so, for example, if, for a given movebound, the attribute value is *unset*, it will inherit the value of the block it belongs to.

This application option is design specific. When set for a design, it will also be applied for all sub-designs. The behavior for sub-designs can be overwritten by setting a different value for a sub-designs.

Application options are set using the *set_app_options* command. Use *report_app_options* to show the value of application options.

See Also

- [create_placement](#)
- [set_block_pin_constraints](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

plan.macro.estimate_pg

Specifies a command or script to use to create a temporary power and ground (PG) grid for congestion estimation during macro placement

p

Data Types

Boolean

Default false

Description

When executing *create_placement -floorplan -congestion*, the tool will use a routing-based congestion estimation to try to reduce the congestion over and between macros. To enhance the precision of the congestion estimation, the tool can do a temporary power and ground (PG) grid insertion right before the congestion estimation routing is done and remove it afterwards. Setting this option to *true* enables the temporary PG grid creation.

For this feature to work, the hierarchical PG grid creation must already have been set up the same way as is needed for *run_block_compile_pg*. This can be done using *characterize_block_pg* and *set_constraint_mapping_file*. Please consult the documentation for these commands for more information.

Note that the generated PG grid is only temporary for the congestion estimation. To save run time, only limited DRC checking and fixing is done. All of the created PG grid is removed after the congestion estimation.

This application option can be set using the *set_app_options* command, and should be set for the top design.

See Also

- [create_placement](#)
- [run_block_compile_pg](#)
- [set_constraint_mapping_file](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

plan.macro.floorplan_finishing

Specifies a command or script to use preform floorplan finishing during freeform macro placement.

Data Types

String

p

Description

When running freeform or hybrid macro placement, this command or script can be used to insert boundary cells, tab cells, switch cells, etc. We call this floorplan finishing.

When running the freeform or hybrid macro placement flow this command or script, if specified, is run after macro legalization and before the standard cell placement. This avoids that this has to be done after the freeform placement has finished which would require another standard cell placement to resolve overlaps between the floorplan finishing cells and the regular standard cells.

The tool keeps track of all the cells, nets and blockages that are inserted by the floorplan finishing. When a later incremental freeform or hybrid macro placement is run, that step will have to remove the floorplan finishing to allow free macro movement. This will be done automatically. To re-do the floorplan finishing after this incremental step, this app option will be used again.

This is done both for stand alone macro placement using *create_placement -floorplan* and using integrated macro placement in *place_opt*, *compile_fusion* or *rtl_opt*.

Note that this option is not supported for on-edge or island style macro placement.

If the option is set to the name of a Tcl script file, the tool sources the file. Otherwise, the input is treated as a Tcl command.

By default, no floorplan finishing is done.

This application option can be set using the *set_app_options* command. It should be set for the top level design.

Examples

The following example specifies a script file to source to do the floorplan finishing

```
set_app_options -name plan.macro.floorplan_finishing -value
scripts/fp_finish.tcl
```

The following example specifies a Tcl command to do the floorplan finishing

```
set_app_options -name plan.macro.floorplan_finishing -value
compile_boundary_cells
```

The following example specifies a Tcl inline script to do the floorplan finishing

```
set_app_options -name plan.macro.floorplan_finishing -value {
compile_boundary_cells
```


p

```
create_tap_cells -lib_cell myLib/Cell11 -distance 30  
}
```

See Also

- [create_placement](#)
- [place_opt](#)
- [compile_fusion](#)
- [set_app_options](#)
- [get_app_option_value](#)
- [report_app_options](#)

plan.macro.grid_error_behavior

Specifies what to do if there is a failure to assign grids to macros.

Data Types

Enum

Default "continue"

Description

This options allows user to specify the behavior if assigning grids to macro fails. If the value is "continue", *create_placement -floorplan* will print warnings and continue execution. If the value is "macro_place_only", after printing warnings, the command will stop as soon as macro locations have been decided, basically the same as if *plan.macro.macro_place_only* option has been set to true. If the value is "quit", the command will print errors and stop immediately. The default value is "continue".

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design. Use *report_app_options* to show the value of application options. This application option is design specific.

See Also

- [create_placement](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

plan.macro.grouping_by_hierarchy

Specifies whether to group macros based on hierarchy or not.

Data Types

Boolean

Default false

Description

This app option controls whether to group macros based on logical hierarchy or not during macro placement, i.e, command *create_placement -floorplan*.

When this option is set true, The grouper will put all macros under one hierarchy into same group if the ratio of the size of the hierarchy and the size of the block exceeds *plan.macro.hierarchy_area_threshold*. Here, size of a hierarchy includes both macros and standard cells. This process is recursive, which means, if a hierarchy contains child hierarchies which fulfill above condition, all macros of child hierarchies will be grouped together separately.

This option might conflict with move bounds or voltage area settings. For example, If two macros of same leaf hierarchy are assigned to two different move bounds, the tool will not be able to group macros based on hierarchy. In this case, the tool will issue a warning message and use other method to group macros.

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design. Use *report_app_options* to show the value of application options.

See Also

- [create_placement](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

plan.macro.hierarchy_area_threshold

Specifies hierarchy area ratio threshold which is used when grouping macros based on hierarchy.

Data Types

Float

p

Default 0.05**Description**

This app options is used by macro grouper to group macros by hierarchy. If the ratio of the size of one hierarchy and the size of its block exceeds this value, the grouper will group all macros under this hierarchy into one group. Here, the size of a hierarchy includes both macros and standard cells size. See *plan.macro.grouping_by_hierarchy(3)* for detail descriptions.

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design. Use *report_app_options* to show the value of application options.

See Also

- [create_placement](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

plan.macro.immediately_buffer_ios

Specifies whether to place buffer channels immediately after IOs, i.e. before placing first row/column of macros.

Data Types*Boolean***Default** false**Description**

During the execution of the command *create_placement -floorplan*, if *plan.macro.auto_buffer_channels* is true, buffer channels are inserted once the width of the macro stack reaches the calculated value. When this option is true, channels will also be created immediately next to IOs, before placing first row/column of macros.

This option will be ignored if *plan.macro.auto_buffer_channels* option is not true.

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design. Use *report_app_options* to show the value of application options. This application option is design specific.

p

See Also

- [create_placement](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

plan.macro.integrated

Controls whether place_opt places hard macros using the integrated macro placement flow.

Data Types

string

Default false

Description

This application option controls whether integrated macro placement is enabled.

When disabled, macros will be considered fixed during place_opt.

When enabled, the macro placement will be updated as part of the standard cell placement steps throughout the compile flow, up to and including final_place.

Valid values are

- *false* (default)

Macros are considered fixed during the place_opt flow.

- *true*

The macro placement will be updated as part of the standard cell placement steps throughout the place_opt flow, up to and including final_place. At the initial_place stage, the placement is done from scratch, and any pre-existing unfixed placement is ignored.

- *two_pass*

Same as *true*, but this indicates that the script is running a manual two-pass flow, running an extra *create_placement -incremental -timing -congestion* somewhere between initial_place and final_place. This means the integrated macro placement can do less work at the initial_place stage. \br Note that this extra *create_placement* call will also do integrated macro placement. \br Note that setting this option will not change the flow to be a two-pass flow. It only reduces the macro placement effort level at the initial_place stage.

p

Note that the integrated macro placement only supports the freeform and hybrid macro placement styles, see also the *plan.macro.style* application option. If the style is set differently, the integrated macro placement will be disabled.

Also note that, if the *place.coarse.fix_hard_macros* application option is set to *true*, *place_opt* does not place hard macros, regardless of the setting for this application option.

See Also

- [place_opt](#)
- [set_app_options](#)
- [get_app_option_value](#)
- [report_app_options](#)

plan.macro.integrated_incremental

Controls whether the *place_opt* integrated macro placement flow works incrementally.

Data Types

boolean

Default `false`

Description

This application option controls whether integrated macro placement is incremental, and only has an effect if *plan.macro.integrated* is enabled (not set to *false*).

When this application option is disabled (*false*), the initial_place of an integrated macro placement *place_opt* run will be done from scratch, and any pre-existing unfixed placement is ignored.

When this application option is enabled (*true*), initial_place of an integrated macro placement *place_opt* run will be done incrementally, starting from the pre-existing placement.

See Also

- [place_opt](#)
- [set_app_options](#)
- [get_app_option_value](#)
- [report_app_options](#)

plan.macro.legalize_effort

Specifies how much effort to spend legalizing macro placement.

Data Types

Enum

Default high

Description

In certain situations, the macro placement may not be legal. In particular, macros may not be placed on grid; blockages or keepout margins of type *hard_macro* will not be respected; in some cases, there may be overlaps.

This option specifies the amount of effort that will be spent legalizing macro placement. The legalization happens during the execution of the *create_placement -floorplan -incremental* command, and also during *create_placement -floorplan* command without *-incremental*, if *plan.macro.style* is "freeform".

The higher the effort, typically the more time the command will spend legalizing, and the more chance it will solve the problem in difficult cases.

If the value is "none", there will be no legalizing.

If the value is "low", the legalizing will be quite quick, but it will work only in simpler cases.

If the value is "medium", the legalizing will mostly succeed, but may fail on more difficult cases.

If the value is "high", the legalizing will mostly succeed, but may fail on more difficult cases.

If the value is "high", the command will work hard to legalize, and it should work on all but the most difficult cases.

If the value is "ultra", the command will work extra hard to find a solution, and it may run for a long time in difficult cases.

The default value of this option is "high".

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design. Use *report_app_options* to show the value of application options. This application option is design specific.

See Also

- [create_placement](#)
- [get_app_option_value](#)

- [report_app_options](#)
- [set_app_options](#)

plan.macro.macro_place_only

Specifies whether to stop the placement when macro locations are decided.

Data Types

Boolean

Default false

Description

When true, this option stops the *create_placement -floorplan* command as soon as macro locations are decided. This allows you to inspect macro locations with minimum runtime. Note that the standard cell placement will not be useful. The default value is false.

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design. Use *report_app_options* to show the value of application options. This application option is global only.

See Also

- [create_placement](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

plan.macro.macros_on_edge

Specifies whether to place macros along partition edges.

Data Types

Boolean

Default true

Description

When true, the *create_placement -floorplan* command places macros along the edges of the partitions. Partitions include blocks, voltage areas, and movebounds. This generally creates a larger contiguous area in which to place standard cells.

p

This option is deprecated. See *plan.macro.style* for new options and backward compatibility.

When this option is false, the placer can place macros away from partition edges. This can create a better placement for designs that contain smaller macros which need to be placed between objects on different sides of the partition. You can identify this type of macro for the placer by creating a zero-dimension group bound, adding the macro to the group bound, and setting the *is_off_edge* property of the group bound to true. For partitions that contain this type of group bound, only the specified macros will be considered for placement away from the edges. For other partitions, the placer determines which macros should be placed off-edge.

The default value of this option is true.

Application options are set using the *set_app_options* command, and can be specified globally or with respect to a design. Use *report_app_options* to show the value of application options. This application option is design level.

See Also

- [create_placement](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

plan.macro.mark_auto_pin_blockages_as_pin_protect

Specifies whether to set *is_pin_protect* attribute to true for auto-generated port blockages.

Data Types

Boolean

Default false

Description

When this app option is true, auto-generated port blockages will have their *is_pin_protect* attribute set to true; otherwise it is set to false. Auto-generated port blockages are created by the *create_placement -floorplan* command, if *plan.macro.create_auto_pin_blockages* app option is set to true.

The default value of this option is false.

p

Application options are set using the *set_app_options* command, and can be specified globally or with respect to a design. Use *report_app_options* to show the value of application options. This application option is design level.

See Also

- [create_placement](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

plan.macro.max_buffer_stack_height

Specifies max height of macro stack before buffer channel will be inserted.

Data Types

length

Default 0m

Description

During the execution of the command *create_placement -floorplan*, once the height of the macro stack reaches the value specified using this option, a channel will be created for the buffer placement; the height of that channel is specified using *plan.macro.buffer_channel_height* option. The default value is 0, which means no channel will be created.

This option will be ignored if *plan.macro.auto_buffer_channels* option is true, as in that case the value will be automatically calculated.

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design. Use *report_app_options* to show the value of application options. This application option is design specific.

See Also

- [create_placement](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

plan.macro.max_buffer_stack_width

Specifies max width of macro stack before buffer channel will be inserted.

Data Types

length

Default 0m

Description

During the execution of the command *create_placement -floorplan*, once the width of the macro stack reaches the value specified using this option, a channel will be created for the buffer placement; the width of that channel is specified using *plan.macro.buffer_channel_width* option. The default value is 0, which means no channel will be created.

This option will be ignored if *plan.macro.auto_buffer_channels* option is true, as in that case the value will be automatically calculated.

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design. Use *report_app_options* to show the value of application options. This application option is design specific.

See Also

- [create_placement](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

plan.macro.max_group_area_percentage

Specifies maximum macro group area percentage.

Data Types

float

Default 1.0

Description

For on edge macro placement, the first step of command *create_placement -floorplan* is to create groups. It then pack groups along edges of blocks. The command tries to create balanced groups in terms of size, but sometimes it is possible to create a big macro group.

p

In such case, these big macro group will be normally packed with high height, which might extend to the center of standard cell area. This might affect QoR such as congestion and timing. To prevent this problem, users sets *plan.macro.max_group_area_percentage* to set the maximum area of macro group. The percentage value is based on total macro area. The command *create_placement -floorplan* has three ways to group macros. They are default, by_connectivity, and by_hierarchy. *plan.macro.max_group_area_percentage* option only work for default and by_connectivity grouping. It does not work for by_hierarchy grouping. Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design. Use *report_app_options* to show the value of application options. This application option is design specific.

See Also

- [create_placement](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

plan.macro.min_macro_keepout

Minimum hard_macro keepout size when auto-generating macro keepouts.

Data Types

length

Default -1m

Description

During *create_placement -floorplan* each hard macro edge will have a default hard_macro keepout created based on the number of pins on that edge. This it to ensure routeability to the hard macro pins. This option specifies the minimum size of that hard_macro keepout. The default size is -1m, which tells the tool to use 1 site height. The option type is length, which means that it must be specified with the length unit.

User set hard_macro keepouts will override this. Use *create_keepout_margin -type hard_macro* to do this.

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design. Use *report_app_options* to show the value of application options. This application option is design specific.

See Also

- [create_placement](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)
- [create_keepout_margin](#)

plan.macro.ml_apply_final_opto

Specifies whether to force ML macro placement in high effort to run `compile_fusion` or `place_opt` to `final_opto`.

Data Types

Boolean

Default `false`

Description

When true, the ML macro placement command `place_macro_ml` in high effort runs `compile_fusion` or `place_opt` to `final_opto`. By default, the ML macro placement command `place_macro_ml` in high effort runs `compile_fusion` or `place_opt` to `final_place`.

See Also

- [place_macro_ml](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

plan.macro.ml_apply_upto_initial_opto

Specifies whether to force ML macro placement in high effort to run `compile_fusion` or `place_opt` to `initial_opto`.

Data Types

Boolean

Default `false`

p

Description

When true, the ML macro placement command `place_macro_ml` in high effort runs `compile_fusion` or `place_opt` to `initial_opto`. By default, the ML macro placement command `place_macro_ml` in high effort runs `compile_fusion` or `place_opt` to `final_place`.

See Also

- [place_macro_ml](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

plan.macro.ml_create_auto_pin_blockages

Specifies whether to create blockages around pins when running ML macro placement.

Data Types

Boolean

Default `false`

Description

When true, this option creates blockages around pins during the `place_macro_ml` command. These blockages are of type `hard_macro` and they keep hard macros away from the design pins.

These placement blockages are created with a special attribute, "is_derived". The blockage creation honors the app_options `plan.macro.mark_auto_pin_blockages_as_pin_protect` to specify if the created pin blockages have the attribute "is_pin_protect". To specify the created blockage size manually, please specify the following app_options. `plan.macro.ml_auto_pin_blockages_size` The first value of the app_options is for blockage width, and the second value is for blockage height. For example, `set_app_options -name plan.macro.ml_auto_pin_blockages_size -value "1um 2um"`

Application options are set using the `set_app_options` command, and they might be specified globally or with respect to a design. Use `report_app_options` to show the value of application options. This application option is design-specific.

See Also

- [place_macro_ml](#)
- [create_placement](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

plan.macro.ml_data_driven

Specifies whether to force ML macro placement to use only QoR values (congestion, timing, and power, etc.) for ML selection.

Data Types

Boolean

Default false

Description

When true, the ML macro placement command `place_macro_ml` uses only QoR values (congestion, timing, and power, etc.) for ML selection. When false, the ML macro placement command `place_macro_ml` uses QoR values and other extracted features of explored floorplans for ML selection.

See Also

- [place_macro_ml](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

plan.macro.ml_data_with_spacing_rule

Specifies whether to honor spacing rule during ML data creation.

Data Types

Boolean

Default false

p

Description

By default, ML macro placement honors specified spacing rules after the best floorplan is selected. After that, incremental macro placement is performed to honor the spacing rule to the selected macro placement. When true, this option enables ML macro placement to honor the specified spacing rule during ML data creation.

See Also

- [place_macro_ml](#)
- [create_placement](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

plan.macro.ml_detail_summary

Specifies whether to report detail summary of ML macro placement results.

Data Types

Boolean

Default false

Description

When true, the ML macro placement command `place_macro_ml` reports detail summary in the directory `mlmp_dir/summary_dir`. In addition, it also invoke `qorSum` to create HTML reports with QoR data of explored floorplans.

See Also

- [place_macro_ml](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

plan.macro.ml_discard_congested_results

Specifies whether to terminate the submitted process if the post-placement congestion is very high when running ML macro placement.

Data Types

Boolean

Default true

Description

When the app_option is set, MLMP terminates the submitted process if the post-placement congestion is very high.

See Also

- [place_macro_ml](#)
- [create_placement](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

plan.macro.ml_distribute_evenly

Specifies whether to distribute macros around boundaries evenly when running ML macro placement.

Data Types

Boolean

Default true

Description

When true, this option enable the function of distributing macros around boundaries evenly creates blockages around pins during the *place_macro_ml* command.

When the app_option is true, MLMP set the following three app_options.
plan.macro.look_for_free_edge_space plan.macro.max_stack_width
plan.macro.max_stack_height

See Also

- [place_macro_ml](#)
- [create_placement](#)
- [get_app_option_value](#)

- [report_app_options](#)
- [set_app_options](#)

plan.macro.ml_ffm_power_optimization

Specifies whether to force ML macro placement to use advanced setting for power optimization.

Data Types

Boolean

Default false

Description

When true, the command `place_macro_ml` uses dvanced setting for power optimization.

See Also

- [place_macro_ml](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

plan.macro.ml_honor_voltage_areas

Specifies whether to honor voltage area during ML data creation.

Data Types

Boolean

Default false

Description

When true, this option enables ML macro placement to honor voltage areas during ML data creation.

See Also

- [place_macro_ml](#)
- [create_placement](#)

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

plan.macro.ml_keep_optimization_setting

Specifies to force the ML macro placement command `place_macro_ml` in high effort does not reset optimization setting.

Data Types

boolean

Default `false`

Description

When true, the ML macro placement command `place_macro_ml` in high effort does not reset optimization setting. By default, the ML macro placement command `place_macro_ml` in high effort reset the following optimization setting when performing `compile_fusion` or `place_opt`.

`place.coarse*` `place_opt*` `refine*` `clock_opt*` `ccd*` `multibit*` `power*` `opt*` `route*`

See Also

- [place_macro_ml](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

plan.macro.ml_mode_weight

Specifies the weight of modes in ML macro placement.

Data Types

string

Default `""`

Description

When set, the ML macro placement command `place_macro_ml` uses the specified weight for each mode specified in the option `-mode` for ML selection.

p

For example, `set_app_options -name plan.macro.ml_mode_weight -value {1.0 0.8}`
`set_host_options -name mlmp -submit_command sh -max_cores 8 place_macro_ml`
`-host_option mlmp -mode {congestion timing}`

The above setting forces ML macro macro placement to weight 1.0 for congestion and 0.8 for timing during ML selection. The default weight is {2 1 0.5}, meaning the first mode with the weight 2, the second mode with the weight 1, and the third mode with th weight 0.5.

See Also

- [place_macro_ml](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

plan.macro.ml_place_existing_floorplan

Specifies whether to place standard cells of exising floorplan when the app_options plan.macro.ml_use_existing_floorplan is true.

Data Types

Boolean

Default true

Description

When true, the command `place_macro_ml` places standard cells of exising floorplan when the app_options plan.macro.ml_use_existing_floorplan is true. Note that the app_options needs to work with the app_options plan.macro.ml_use_existing_floorplan, which uses existing floorplan as one of the candidate of ML selection.

See Also

- [place_macro_ml](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

plan.macro.ml_place_opt_setting

Specify a place_opt setting script for ML macro placement with high effort to use in order to take users' setting of place_opt.

Data Types

String

Default By default, ML macro placement with high effort does not apply specific place_opt setting to run place_opt.

Description

ML macro placement with high effort performs place_opt for selected explored floorplans to obtain corresponding congestion and timing information. By default, default setting is used to perform place_opt. With this app_option, users can put their specific setting to a script. After that, ML macro placement with high effort can source the specified script before running place_opt. Note that only app_options setting are accepted in the setting file.

This application option can be set using the *set_app_options* command.

Examples

```
prompt> set_app_options -name plan.macro.ml_place_opt_setting -value  
"place_opt_setting.tcl"
```

See Also

- [place_macro_ml](#)
- [place_opt](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

plan.macro.ml_use_advanced_concav_blockage

Specifies whether to create hard macro placement blockage at concave corners of design boundaries using advanced algorithm.

Data Types

Boolean

Default false

Description

When true, this option enables ML macro placement to create hard macro placement blockage at concave corners of design boundaries using advanced algorithm.

See Also

- [place_macro_ml](#)
- [create_placement](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

plan.macro.ml_use_concav_blockage

Specifies whether to create hard macro placement blockage at concave corners of design boundaries.

Data Types

Boolean

Default false

Description

When true, this option enables ML macro placement to create hard macro placement blockage at concave corners of design boundaries.

See Also

- [place_macro_ml](#)
- [create_placement](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

plan.macro.ml_use_existing_floorplan

Specifies whether to uses existing floorplan as one of the candidates for ML selection.

p

Data Types

Boolean

Default false

Description

When true, the command `place_macro_ml` uses existing floorplan as one of the candidates for ML selection. For existing floorplan, by default, it replace standard cells. To force the command does not replace standard cells for existing floorplan, please set the `app_option plan.macro.ml_place_existing_floorplan` to false.

See Also

- [place_macro_ml](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

plan.macro.pin_shape_track_match_threshold

Specifies the threshold size to use when color matching pin shapes and tracks.

Data Types

list

Default {layerName 0m}

Description

This options specifies the threshold size when color matching macro pins to tracks. The threshold applies to width for vertical layers and height for horizontal layers. Pin shapes larger than this size will not be considered for color matching with tracks. The value is specified as a list of layer_name and length pairs, for example, { {layer_name1 size1} {layer_name2 size2} ... }

If the value is not specified for a given layer, the tool uses the min width for the layer as the size. The default is an empty list.

Application options are set using the `set_app_options` command, and they may be specified globally or with respect to a design. Use `report_app_options` to show the value of application options. This application option is design specific.

p

Examples

The following example specifies threshold of 23 nm on layer M2 and 25nm on layer M3.

```
prompt> set_app_options -block [current_design] \\  
        -list { plan.macro.pin_shape_track_match_threshold { {M2 23nm} {M3  
        25nm} } }
```

See Also

- [create_placement](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

plan.macro.remove_auto_pin_blockages

Specifies whether to remove the temporary blockages that were placed around the pins during freeform macro placement to allow easy access to the pins.

Data Types

Boolean

Default false

Description

When true, this option removes blockages around pins at the end of the *create_placement -floorplan* command if *plan.macro.style* is freeform. For more information about these blockages, see the *plan.macro.create_auto_pin_blockages* application option. The default for the *plan.macro.remove_auto_pin_blockages* option is false.

Application options are set using the *set_app_options* command, and they might be specified globally or with respect to a design. Use *report_app_options* to show the value of application options. This application option is design-specific.

See Also

- [create_placement](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

plan.macro.remove_temporary_pg_grid

Specifies a command or script to use to remove a temporary power and ground (PG) grid for congestion estimation during macro placement.

Data Types

String

Description

When executing *create_placement -floorplan -congestion*, the tool uses a routing-based congestion estimation to try to reduce the congestion over and between macros. To enhance the precision of the congestion estimation, you can let the tool do a temporary PG grid and power switch insertion right before the congestion estimation routing is done, and then remove it after the congestion is estimated.

Use this application option to remove the temporary power grid and power switches that were inserted using a script or command.

If the option is set to the name of a Tcl script file, the tool sources the file. Otherwise, the input is treated as a Tcl command. The script is sourced and executed just after congestion estimation routing. If the option is not specified, the tool tries to remove all power and ground shapes, vias, and power switches from the design.

By default, no command or script is specified.

This option is meant only for physically flat designs.

You can set this application option using the *set_app_options* command. It should be set for the top-level design.

See Also

- [create_placement](#)
- [compile_pg](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

plan.macro.spacing_rule_heights

Sets the spacing rules heights for macro placement.

p

Data Types

pair of distances

Default 0m 0m

Description

The option specifies two values: the first value is the *exact* distance and the second one is the *at least* distance. To satisfy the rule, the vertical distance between any two macros must be equal to the *exact* distance or be greater than or equal to the *at least* distance. For a *non-rectangular* macro, distances are interpreted with respect to its bounding box. Both distances are expected to be non-negative, and the *at least* distance is expected to be greater than the *exact* distance. The default value is { 0um 0um }, which means there is no spacing rule to enforce. This option is respected by *create_placement -floorplan* command.

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design. Use *report_app_options* to show the value of application options. This application option is design specific.

Examples

The following example makes sure macros are either exactly 2um apart or at least 10um apart:

```
prompt> set_app_options -name plan.macro.spacing_rule_heights -value  
{ 2um 10um }
```

The following example makes sure macros are either abutted or at least 10um apart:

```
prompt> set_app_options -name plan.macro.spacing_rule_heights -value  
{ 0um 10um }
```

The following example makes sure macros are at least 10um apart (note that the two values are the same):

```
prompt> set_app_options -name plan.macro.spacing_rule_heights -value  
{ 10um 10um }
```

See Also

- [create_placement](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

plan.macro.spacing_rule_widths

Sets spacing rules widths for macro placement.

Data Types

pair of distances

Default 0m 0m

Description

The option specifies two values: the first value is the *exact* distance and the second one is the *at least* distance. To satisfy the rule, the horizontal distance between any two macros must be equal to the *exact* distance or be greater than or equal to the *at least* distance. For a *non-rectangular* macro, distances are interpreted with respect to its bounding box. Both distances are expected to be non-negative, and *at least* distance is expected to be greater than the *exact* distance. The default value is { 0um 0um }, which means there is no spacing rule to enforce. This option is respected by *create_placement -floorplan* command.

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design. Use *report_app_options* to show the value of application options. This application option is design specific.

Examples

The following example makes sure macros are either exactly 2um apart or at least 10um apart:

```
prompt> set_app_options -name plan.macro.spacing_rule_widths -value { 2um
    10um }
```

The following example makes sure macros are either abutted or at least 10um apart:

```
prompt> set_app_options -name plan.macro.spacing_rule_widths -value { 0um
    10um }
```

The following example makes sure macros are at least 10um apart (note that the two values are the same):

```
prompt> set_app_options -name plan.macro.spacing_rule_widths -value
    { 10um 10um }
```

See Also

- [create_placement](#)
- [get_app_option_value](#)

p

- [report_app_options](#)
- [set_app_options](#)

plan.macro.style

Specifies macro placement style.

Data Types

Enum

Default on_edge

Description

This option specifies macro placement style. Following values are supported: on_edge, islands, freeform, hybrid, and all_islands. The default value is on_edge.

On edge style is the default style. It places macros along edges of the partitions. Partitions include blocks, voltate area, and movebounds. This generally creates a larger contiguous area in which to place standard cells.

In islands style, the placer can place some macros away from partition edges. But majority macros are still along edges of partitions. This can create a better placement for designs that contain smaller macros which need to be placed between objects on different sides of the partition. You can identify this type of macro for the placer by creating a zero-dimension group bound, adding the macro to the group bound, and setting the *is_off_edge* property of the group bound to true. For partitions that contain this type of group bound, only the specified macros will be considered for placement away from the edges. For other partitions, the placer determines which macros should be placed off-edge.

In freeform style, the placer places macros and standard cells simultaneously. Macros will be treated as big standard cells and scattered in the partition. There are three steps to perform freeform macro placement.

1. Coarse placer place macros and standard cells simultaneously.
2. Remove macro overlaps.
3. Replace standard cells.

See *plan.macro.legalize_effort* for different options to remove macro overlaps.

In hybrid style, some macros (likely a minority) are designated to be placed along edges of the partitions, and the others are to be placed using freeform placement. See *set_macro_constraints* command for details on how to designate macros for on edge or freeform placement.

Note that hybrid style is quite different from islands style, in several aspects, including a) how the decision which macros to place along edges is made, b) expected area of floating macros (very small in island, not limited in hybrid), and, most importantly, c) how

p

are floating macros placed - in island style, groups of floating macros are tightly packed in a structured way, while in hybrid style, the floating macros are placed individually and loosely in freeform style.

In all_islands style, all macros are going to be placed tightly in various islands. This style is meant for use with shell designs only.

With introduction of this option, the old application option *plan.macro.macros_on_edge* is deprecated. Following policy is adopted to keep backward compatibility before the option *plan.macro.macros_on_edge* is dropped off.

If value of *plan.macro.macros_on_edge* is true (its default value), its value will be ignored and macro placement style will be determined by *plan.macro.style*. If value of *plan.macro.macros_on_edge* is false, *plan.macro.style* will be ignored and the code falls back to old behavior.

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design. Use *report_app_options* to show the value of application options. This application option is design specific.

See Also

- [create_placement](#)
- [set_macro_constraints](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

plan.mtcmos.allow_legalizer_pin_color_alignment

This application option allows the legalizer to fix the pin color alignment during power switch insertion.

Data Types

Boolean

Default false

Description

When set to true, the legalizer can fix the pin color alignment during power switch insertion. Otherwise, inserted power switches might have pin alignment issues. The default value is false.

p

This application option is set using the *set_app_options* command, and may be specified globally. To determine the current value of this application option on a specific design, use *get_app_option_value -name opt_name*. Use *report_app_options* to show the value of multiple application options.

See Also

- [create_power_switch_ring](#)

plan.mtcmos.allow_partial_power_strap_overlap

Specifies whether power switches are allowed if not fully covered by the strap to which it should align.

Data Types

Boolean

Default false

Description

When set to true, power switches are allowed if they are not fully covered by the power straps to which they should align. Otherwise, power switches will be dropped. The default value is false.

This application option is set using the *set_app_options* command, and may be specified globally. To determine the current value of this application option on a specific design, use *get_app_option_value -name opt_name*. Use *report_app_options* to show the value of multiple application options.

See Also

- [create_power_switch_array](#)

plan.mtcmos.allow_power_switch_outside_core

Specifies whether power switches can be placed outside the core region.

Data Types

Boolean

Default false

p

Description

When set to true, power switches can be placed outside the core region. Otherwise, power switches will be dropped if they are outside the core region. The default value is false.

This application option is set using the *set_app_options* command, and may be specified globally. To determine the current value of this application option on a specific design, use *get_app_option_value -name opt_name*. Use *report_app_options* to show the value of multiple application options.

See Also

- [create_power_switch_ring](#)

plan.mtcmos.honor_min_vt_spacing_rule

Specifies if min Vth spacing rule will be checked.

Data Types

Boolean

Default false

Description

When set to true, min Vth spacing rules will be checked for inserted power switch cells. Note that this app option is only valid when option *-snap_to_site_row* is true. By default, the value is false.

This application option is set using the *set_app_options* command, and may be specified globally. To determine the current value of this application option on a specific design, use *get_app_option_value -name opt_name*. Use *report_app_options* to show the value of multiple application options.

See Also

- [create_power_switch_array](#)

plan.mtcmos.honor_pin_color_alignment

Specifies if pin of power switch cells should be snapped to right color track.

Data Types

Boolean

Default false

p

Description

When set to true, pin of power switch cells will be snapped to right color track. Power switch will be shifted toward right and find closest location to align correct color track. Note that this app option is only valid when option `-snap_to_site_row` is true. By default, the app option value is false.

This application option is set using the `set_app_options` command, and may be specified globally. To determine the current value of this application option on a specific design, use `get_app_option_value -name opt_name`. Use `report_app_options` to show the value of multiple application options.

See Also

- [create_power_switch_array](#)

plan.mtcmos.ignore_all_blockages

If this app option is specified, no blockages are honored during power switch insertion.

Data Types

Boolean

Default false

Description

When set to true, no blockages are honored for power switch insertion. By default, the value is false.

This application option is set using the `set_app_options` command, and may be specified globally. To determine the current value of this application option on a specific design, use `get_app_option_value -name opt_name`. Use `report_app_options` to show the value of multiple application options.

See Also

- [create_power_switch_ring](#)
- [create_power_switch_array](#)

plan.mtcmos.mark_supply_net_for_filler_cells

This app option decides if the tool should mark supply nets for inserted filler cells.

p

Data Types*Boolean***Default** false**Description**

When set to true, the tool will mark supply nets info for filler cells. By default, the value is false.

This application option is set using the *set_app_options* command, and may be specified globally. To determine the current value of this application option on a specific design, use *get_app_option_value -name opt_name*. Use *report_app_options* to show the value of multiple application options.

See Also

- [create_power_switch_ring](#)

plan.mtcmos.ring_pattern_cell_orientation

Specifies the reference orientation for a library cell defined in the power switch ring pattern.

Data Types

list

Default ""**Description**

This application option is used to specify the reference orientations of specific power switch library cells. The reference orientation is the orientation along the bottom ring edge. The orientation along other edges will be rotated following the rule defined for *create_power_switch_ring*. The application option only works for library cells defined in pattern. If not specified, all the library cells in the pattern will follow the orientation specified by the *-orient* option of *create_power_switch_ring* command.

The format to specify the library cell and orientation is:

```
{{power_switch_lib_cell_name1 power_switch_orientation_value1} ...}
```

Legal orientations are: R0 | R90 | R270 | R180 | MX | MY | MXR90 | MYR90.

p

Examples

In the following example, switch cell HEADX4_LVT is specified with an orientation of R90 along the bottom edge. Switch cell HEADX2_LVT is specified with an orientation of R0 along the bottom edge.

```
prompt> set_power_switch_placement_pattern -name pattern2 \\  
-placement_type ring \\  
-pattern {HEADX2_LVT {SPACE 10} {{HEADX4_LVT {SPACE 8}} 3} }  
prompt> set_app_options -name plan.mtcmos.ring_pattern_cell_orientation  
\\  
-value {{HEADX4_LVT R90}}  
prompt> create_power_switch_ring -power_switch sw1 -pattern pattern2  
-orient R0
```

See Also

- [create_power_switch_ring](#)
- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)
- [set_power_switch_placement_pattern](#)

plan.mtcmos.snap_after_placement

Specifies whether power switch cells should be snapped after placement based on original pitch.

Data Types

Boolean

Default false

Description

When set to true, the tool will not automatically update pitch to multiples of site width and height. The cells are snapped to the closest site after being placed at original locations.

Note that this app option is only valid when you specify *create_power_switch_array -snap_to_site_row true*. By default, the value is false.

This application option is set using the *set_app_options* command, and may be specified globally. To determine the current value of this application option on a specific design, use

p

get_app_option_value -name opt_name. Use *report_app_options* to show the value of multiple application options.

See Also

- [create_power_switch_array](#)
- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)

plan.mtcmos.snap_to_legal_index

Specifies whether power switch cells should be snapped to a legal index.

Data Types

Boolean

Default false

Description

When set to true, power switch cells will be snapped to a legal index. Note that this application option is only valid when you specify *create_power_switch_array -snap_to_site_row true*. By default, the value is false.

This application option is set using the *set_app_options* command, and may be specified globally. To determine the current value of this application option on a specific design, use *get_app_option_value -name opt_name*. Use *report_app_options* to show the value of multiple application options.

See Also

- [create_power_switch_array](#)
- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)

plan.mtcmos.snap_to_siterow_orient

Specifies if power switch cells should only be snapped to siterows with specified orientation.

p

Data Types*string* R0 | MX |**Default** ""**Description**

When set to R0, power switch cells will be snapped to siterows with R0 site orientation. When set to MX, power switch cells will be snapped to siterows with MX site orientation. If not specified, power switch cells can be snapped to all siterows. By default, the value is an empty string "". Note that this app option is only valid when you specify *create_power_switch_array -snap_to_site_row true*. If the power switch is a double height cell and needs to be snapped to every other row. This app option can be used to control the snapping.

This application option is set using the *set_app_options* command, and may be specified globally. To determine the current value of this application option on a specific design, use *get_app_option_value -name opt_name*. Use *report_app_options* to show the value of multiple application options.

See Also

- [create_power_switch_array](#)
- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)

plan.mtcmos.snap_to_via0_grid

Specifies if power switch cells should be snapped to via0 grid.

Data Types*Boolean***Default** false**Description**

When set to true, power switch cells will be snapped to via0 grid. Note that this app option is only valid when option *-snap_to_site_row* is true. By default, the value is false.

This application option is set using the *set_app_options* command, and may be specified globally. To determine the current value of this application option on a specific design, use *get_app_option_value -name opt_name*. Use *report_app_options* to show the value of multiple application options.

See Also

- [create_power_switch_array](#)

plan.mtcmos.treat_keepout_as_blockage

Specifies whether power switch insertion should treat keepout regions as blockages.

Data Types

Boolean

Default true

Description

When set to true, keepout regions are treated as blockages for power switch insertion. Otherwise, keepout regions will not be treated as blockages. By default, the value is true.

This application option is set using the *set_app_options* command, and may be specified globally. To determine the current value of this application option on a specific design, use *get_app_option_value -name opt_name*. Use *report_app_options* to show the value of multiple application options.

See Also

- [create_power_switch_array](#)
- [create_power_switch_ring](#)
- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)

plan.mtcmos.treat_voltage_area_as_blockage

Specifies whether power switch insertion should treat other voltage areas as blockages.

Data Types

Boolean

Default true

p

Description

When set to true, voltage areas not associated with current power switch strategy are treated as blockages for power switch insertion. Otherwise, voltage areas will not be treated as blockages. By default, the value is true.

This application option is set using the `set_app_options` command, and may be specified globally. To determine the current value of this application option on a specific design, use `get_app_option_value -name opt_name`. Use `report_app_options` to show the value of multiple application options.

See Also

- [create_power_switch_array](#)
- [create_power_switch_ring](#)
- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)

plan.outline.dense_module_depth

Specifies the number of levels from the top of the module hierarchy to make dense modules.

Data Types

integer

Default 1

Description

This application option specifies the number of levels from the top of the module hierarchy to make dense modules.

The default is 1, which means just the top level of the module hierarchy. A value of 2 makes the top module and its immediate children dense, and so on. A value of -1 makes all non-sparse modules dense.

This application option is set using the `set_app_options` command, and may be specified globally. To determine the current value of this application option on a specific design, use `get_app_option_value -name opt_name`. Use `report_app_options` to show the value of multiple application options.

See Also

- [expand_outline](#)
- [get_app_option_value](#)
- [get_app_options](#)
- [read_verilog_outline](#)
- [report_app_options](#)
- [set_app_options](#)

plan.outline.glue_cell_count

Specifies the target for the total number of leaf cells in small dense modules used as glue logic between partitions.

Data Types

integer

Default 100000

Description

This application option specifies the target for the total number of leaf cells in small dense modules used as glue logic between partitions.

Module hierarchies in the outline design are marked dense from the top of the partition on down, as long as the target is not exceeded. Inverter and buffer cells are not included in the count.

A value of -1 skips marking any glue modules as dense.

See Also

- [expand_outline](#)
- [get_app_option_value](#)
- [get_app_options](#)
- [read_verilog_outline](#)
- [report_app_options](#)
- [set_app_options](#)

plan.outline.keep_port_depth

Specifies the number of levels from the top of the module hierarchy to preserve module port interfaces.

Data Types

integer

Default 1

Description

This application option specifies the number of levels from the top of the module hierarchy to preserve module port interfaces.

The default is 1, which means just the top level of the module hierarchy. A value of 2 preserves the port interfaces of the top module and its immediate children, and so on. A value of -1 preserves the port interfaces of all modules.

This application option is set using the *set_app_options* command, and may be specified globally. To determine the current value of this application option on a specific design, use *get_app_option_value -name opt_name*. Use *report_app_options* to show the value of multiple application options.

See Also

- [expand_outline](#)
- [get_app_option_value](#)
- [get_app_options](#)
- [read_verilog_outline](#)
- [report_app_options](#)
- [set_app_options](#)

plan.outline.large_threshold

Specifies the number of leaf cells a module can have before it is considered a large module that should be represented as a sparse module.

Data Types

integer

Default 10000

p

Description

This application option specifies the number of leaf cells a module can have before it is considered a large module that should be represented as a sparse module. Inverter and buffer cells are not included in the count.

This application option is set using the `set_app_options` command, and may be specified globally. To determine the current value of this application option on a specific design, use `get_app_option_value -name opt_name`. Use `report_app_options` to show the value of multiple application options.

See Also

- [expand_outline](#)
- [get_app_option_value](#)
- [get_app_options](#)
- [read_verilog_outline](#)
- [report_app_options](#)
- [set_app_options](#)

plan.outline.message_limit

Specifies the maximum number of 'Module not found in Verilog' warnings to issue for each of the `-dense_modules`, `-port_modules`, or `-sparse_modules` options.

Data Types

integer

Default 4

Description

This application option specifies the maximum number of 'Module not found in Verilog' warnings to issue for each of the `-dense_modules`, `-port_modules`, or `-sparse_modules` options.

The default is four warnings for each option. A value of -1 removes the limit and issues a warning for each module that is specified, but not found in the Verilog netlist.

This application option is set using the `set_app_options` command, and may be specified globally. To determine the current value of this application option on a specific design, use `get_app_option_value -name opt_name`. Use `report_app_options` to show the value of multiple application options.

See Also

- [expand_outline](#)
- [get_app_option_value](#)
- [get_app_options](#)
- [read_verilog_outline](#)
- [report_app_options](#)
- [set_app_options](#)

plan.outline.partition

Selects whether the tool uses the allocation or partition method to determine dense or sparse modules when reading the outline Verilog modules.

Data Types

Boolean

Default `true`

Description

This application option selects whether the tool uses the allocation or partition method to determine dense or sparse modules when reading the outline Verilog modules.

By default (*true*), the tool uses the partition method. When set to *false*, the tool uses the allocation method.

This application option is set using the *set_app_options* command, and may be specified globally. To determine the current value of this application option on a specific design, use *get_app_option_value -name opt_name*. Use *report_app_options* to show the value of multiple application options.

See Also

- [expand_outline](#)
- [get_app_option_value](#)
- [get_app_options](#)
- [read_verilog_outline](#)
- [report_app_options](#)
- [set_app_options](#)

plan.outline.target_block_size

Specifies the target number of cells in each outline partition block.

Data Types

integer

Default 1000000

Description

This application option specifies the target number of cells in each outline partition block.

The partition outline method makes modules at the top of the module hierarchy dense until the number cells below that module is less than a target number of cells. Below that threshold, modules are marked sparse. Inverter and buffer cells are included in the hierarchical count.

This application option is set using the *set_app_options* command, and may be specified globally. To determine the current value of this application option on a specific design, use *get_app_option_value -name opt_name*. Use *report_app_options* to show the value of multiple application options.

See Also

- [expand_outline](#)
- [get_app_option_value](#)
- [get_app_options](#)
- [read_verilog_outline](#)
- [report_app_options](#)
- [set_app_options](#)

plan.outline.target_cell_count

Data Types

integer

Default 1000000

Description

This application option specifies the target total number of leaf cells in the modules of an outline design read by the *read_verilog_outline* command.

p

Modules in the outline design are marked dense from the top of the module hierarchy on down, as long as the target is not exceeded. Inverter and buffer cells are not included in the count.

This application option is set using the *set_app_options* command, and may be specified globally. To determine the current value of this application option on a specific design, use *get_app_option_value -name opt_name*. Use *report_app_options* to show the value of multiple application options.

See Also

- [expand_outline](#)
- [get_app_option_value](#)
- [get_app_options](#)
- [read_verilog_outline](#)
- [report_app_options](#)
- [set_app_options](#)

plan.path_analysis.enable_timing

Specifies whether timing evaluation should be done during path analysis.

Data Types

Boolean

Default true

Description

The *analyze_paths* command is used to analyze path connectivity between macros, ports, and hierarchies in your floorplan. By default, both the number and timing criticality of paths path examined. By setting this app option to false, you can disable the timing part of path analysis. Doing this can save well over half of the analysis runtime, and give you results in just a few minutes.

Set this option to true to get timing analysis for you paths between objects.

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design. Use *report_app_options* to show the value of application options.

See Also

- [report_app_options](#)
- [set_app_options](#)

plan.pgcheck.check_non_routing_layer_pins

Specifies if `check_pg_connectivity` command should check pins of non- routing layers, e.g. N-Well layer (NW), contact layer (CO) etc.

Data Types

Boolean

Default `false`

Description

When set to true, `check_pg_connectivity` will check pins of non-routing layers, including N-well layer(NW), contact layer(CO), etc. By default, the value is false.

This application option is set using the `set_app_options` command, and may be specified globally. To determine the current value of this application option on a specific design, use `get_app_option_value -name opt_name`. Use `report_app_options` to show the value of multiple application options.

plan.pgcheck.check_rdl_routing

Specifies if objects with `shape_use` of `rdl` should be checked or not.

Data Types

Boolean

Default `false`

Description

When set to true, `check_pg_connectivity` will check objects with `shape_use` of `rdl`. Otherwise objects with `shape_use` of `rdl` will be ignored. By default, the value is false.

This application option is set using the `set_app_options` command, and may be specified globally. To determine the current value of this application option on a specific design, use `get_app_option_value -name opt_name`. Use `report_app_options` to show the value of multiple application options.

plan.pgcheck.check_signal_ports

Specifies if signal ports assigned with tied-off nets should be checked by `check_pg_connectivity`.

Data Types

Boolean

Default `true`

Description

When set to `true`, tied-off signal ports will be checked by `check_pg_connectivity`. Otherwise, these ports will not be checked. By default, the value is `true`.

This application option is set using the `set_app_options` command, and may be specified globally. To determine the current value of this application option on a specific design, use `get_app_option_value -name opt_name`. Use `report_app_options` to show the value of multiple application options.

plan.pgcheck.fast_checking

Specifies if fast intersection detection engine will be used for `check_pg_connectivity`.

Data Types

Boolean

Default `true`

Description

When set to `true`, `check_pg_connectivity` will use fast intersection detection engine to check PG connectivity. When set to `false`, `check_pg_connectivity` uses previous intersection detection engine. By default, the value is `true`.

This application option is set using the `set_app_options` command, and may be specified globally. To determine the current value of this application option on a specific design, use `get_app_option_value -name opt_name`. Use `report_app_options` to show the value of multiple application options.

plan.pgcheck.ignore_floating_on_layers

Specifies the names of layers on which floating objects will not be reported.

Data Types

String

p

Default ""**Description**

If specified, `check_pg_connectivity` will not reporting floating objects on specified layers. Otherwise, all floating objects will be reported. By default, the value is not specified.

This application option is set using the `set_app_options` command, and may be specified globally. To determine the current value of this application option on a specific design, use `get_app_option_value -name opt_name`. Use `report_app_options` to show the value of multiple application options.

plan.pgcheck.ignore_unfixed_std_cells

Specifies if unfixed std cells should be checked by `check_pg_connectivity`.

Data Types

Boolean

Default false**Description**

When set to true, `check_pg_connectivity` will only check std cells with fixed status. By default, the value is false.

This application option is set using the `set_app_options` command, and may be specified globally. To determine the current value of this application option on a specific design, use `get_app_option_value -name opt_name`. Use `report_app_options` to show the value of multiple application options.

plan.pgcheck.max_checking_level

Specifies if the `check_pg_missing_vias` command should check missing via between different hierarchy.

Data Types

Integer

Default 0**Description**

When set to a positive number, the `check_pg_missing_vias` command will check if there is any vias missing in the wire intersections within the level up to the specified number. Data inside soft block will be loaded. Top-level is considered as level 0. Data inside soft macro

p

is considered next level. `check_pg_missing_vias` will only check missing vias in top-level by default.

See Also

- [check_pg_missing_vias](#)

plan.pgcheck.return_isolated_object_only

Specifies if `check_pg_connectivity` command only returns completely isolated floating objects.

Data Types

Boolean

Default false

Description

When set to true, `check_pg_connectivity` will only return floating objects that are completely isolated. If two objects are floating but connected to each other, both objects will not be returned. Otherwise all floating objects are returned. By default, the value is false.

This application option is set using the `set_app_options` command, and may be specified globally. To determine the current value of this application option on a specific design, use `get_app_option_value -name opt_name`. Use `report_app_options` to show the value of multiple application options.

plan.pgcheck.treat_terminal_as_voltage_source

Specifies if the `check_pg_connectivity` command should treat terminals as voltage sources.

Data Types

Boolean

Default false

Description

When set to true, the `check_pg_connectivity` command assumes that all terminals are voltages sources. Any PG objects connected to a terminal will NOT be considered as floating. By default, the value is false.

p

This application option is set using the *set_app_options* command, and may be specified globally. To determine the current value of this application option on a specific design, use *get_app_option_value -name opt_name*. Use *report_app_options* to show the value of multiple application options.

See Also

- [check_pg_connectivity](#)

plan.pgcheck.use_pin_as_bridge

Specifies if pins in macro or standard cells must be considered as bridges to connect PG straps by using the *check_pg_connectivity* command.

Data Types

Boolean

Default true

Description

When set to true, the *check_pg_connectivity* command assumes that all the pin shapes of the same port in a macro or standard cell are internally connected. In this case, PG straps connecting to disjoint pins are considered as connected using the pins as bridges. Otherwise, you assume the disjoint PG pins are not connected. By default, the value is true.

This application option is set using the *set_app_options* command, and might be specified globally. To determine the current value of this application option on a specific design, use *get_app_option_value -name opt_name*. Use *report_app_options* to show the value of multiple application options.

plan.pgcheck.via_site_threshold

Specifies the minimum overlap ratio between intersecting metal layers before a via is reported as missing by the *check_pg_missing_vias* command.

Data Types

Double

Default 0.0

Description

This application option specifies the minimum overlap ratio required for intersecting metal routes before a missing via is reported by the *check_pg_missing_vias* command. Valid

p

values are between 0.0 and 1.0 inclusive. A value of 1.0 specifies that the intersecting metal routes must fully overlap before a missing via is reported. A value of 0.5 specifies that the intersecting metal routes must be at least 50% overlapped before a missing via is reported. A value of 0.0 specifies that missing vias are reported for any overlapping metal routes.

By default, the value is 0.0, and the *check_pg_missing_vias* command reports a missing via for any metal intersection, even for metal routes that are only slightly overlapped.

This application option is set using the *set_app_options* command, and may be specified globally. To determine the current value of this application option on a specific design, use *get_app_option_value -name opt_name*. Use *report_app_options* to show the value of multiple application options.

See Also

- [check_pg_missing_vias](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

plan.pgroute.always_align_track_color_layers

This app option is a super set of *plan.pgroute.color_metal_cut_internally_for_drc*. This app_option controls if wire color should always follow track color in specified layer when metal string is set in *plan.pgroute.color_metal_cut_internally_for_drc*.

Data Types

string

Default all

Description

By default, wires should follow track color in all layers. User can specify one or multiple layers in the app_option. The input should be metal layer name. For the layers that are specified, the wires will always follow track color. For the layers are not specified, follow track color when the wire is default width, and use inverse color when the wire is not default width.

Examples

The following setting specifies M1 and M3 should always follow track color

p

```
prompt> set_app_options -name  
plan.pgroute.always_align_track_color_layers -value {M1 M3}
```

plan.pgroute.assign_drc_color_mode

Assigns color mode to uncolored shapes during design rule checking.

Data Types

Integer

Default -1

Description

For color-related design rules, the uncolored shapes might be ignored or treated as certain color during each rule checking. This application option can force the design rule checker to treat uncolored shapes as colored shapes to relax or tighten the spacing requirements. However, it does not actually color the shape as a specific color.

The valid values are -1, 0, and 1.

-1: Keep uncolored shape uncolored.

0: Treat uncolored shape as different color from the neighboring shape.

1: Treat uncolored shape as same color as the neighboring shape.

By default, an uncolored shape is left uncolored for rule checking.

When set to *0*, an uncolored shape is treated as *different color* when checking against the neighboring shapes. If a neighboring shape is color 1, the uncolored shape is treated as color 2. It is treated as color 1 if neighboring shape is color 2.

When set to *1*, an uncolored shape is treated as *same color* as the neighboring shape. It is treated as color 1 if neighboring shape is color 1. It is treated as color 2 if neighboring shape is color 2.

This application option is set using the `set_app_options` command, and can be specified globally. To determine the current value of this application option on a specific design, use `get_app_option_value -name opt_name`. Use `report_app_options` to show the value of multiple application options.

See Also

- [compile_pg](#)
- [create_pg_vias](#)

plan.pgroute.auto_connect_pg_net

Specifies whether or not PG route calls `connect_pg_net` under-the-hood.

Data Types

Boolean

Default `false`

Description

When true, PG route commands call `connect_pg_net` command internally to update the PG net-pin logic connectivity. By default, the value is false and `connect_pg_net` is not called.

This application option is set using the `set_app_options` command, and may be specified globally. To determine the current value of this application option on a specific design, use `get_app_option_value -name opt_name`. Use `report_app_options` to show the value of multiple application options.

See Also

- [set_pg_strategy](#)

plan.pgroute.color_metal_cut_internally_for_drc

Specifies whether to automatically color metal and cut layers internally before design rule checking and fixing. The color information can optionally be submitted to design.

Data Types

string

Default `{metal cut}`

Description

By default, the tool colors metal and cut layers internally before design rule checking and fixing. Without color information, some color related rules might not apply. This application option affects only newly created objects in PG router commands. For existing objects with no color, this application option does not apply.

When the `metal_commit` keyword is specified in the app option, the tool will color the wires as described above for the keyword metal and subsequently submit the color the the design.

The valid values are: metal, cut, and none.

p

Choose metal to color PG wires based on their track colors. When the wire width is the default width, the PG wire color follows the track color. The color applied depends on the `plan.pgroute.always_align_track_color_layers` application option setting. Non default width wires have opposite color of the track color.

Choose cut to color PG cuts using mask one at first and PG DRC fixing engine recolors as necessary to fix any DRC violations. If there is violation, the tool tries mask_two. If none of the masks are DRC clean, the tool continues with regular PG DRC fixing.

Choose none to not color anything before DRC checking. When none is combined with other options, none takes precedence.

To summarize,

- metal: Color metal layers based on track information
- cut: Color cut layer for DRC
- none: Not color metal or cut layers

Examples

The following setting specifies metal layers to be colored internally.

```
prompt> set_app_options -name
plan.pgroute.color_metal_cut_internally_for_drc -value {metal}
```

See Also

- [compile_pg](#)
- [create_pg_strap](#)

plan.pgroute.connect_user_route_shapes

Specifies if PG route command to create vias to connect existing user_route shapes.

Data Types

Boolean

Default false

Description

When set to true, PG route commands will create vias between compiled shapes and existing user_route shapes. Otherwise, user_route shapes are ignored.

This application option is set using the `set_app_options` command, and may be specified globally. To determine the current value of this application option on a specific design, use

p

get_app_option_value -name opt_name. Use *report_app_options* to show the value of multiple application options.

See Also

- [compile_pg](#)

plan.pgroute.create_via_cellalign_maxpin_only

Specifies if PG router creates vias over the maximum piece of pin shape only.

Data Types

Boolean

Default true

Description

When set to true, for cell alignment straps, PG router only create vias over the largest piece of pin shape. Otherwise via will be created over all intersected pin shape.

This application option is set using the *set_app_options* command. To determine the current value of this application option on a specific design, use *get_app_option_value -name opt_name*. Use *report_app_options* to show the value of multiple application options.

See Also

- [compile_pg](#)
- [create_pg_vias](#)

plan.pgroute.decide_rail_width_from_routing_area

Decide rail width from PG pins of cells in routing area specified in PG strategy.

Data Types

Boolean

Default false

Description

When set to true, PG rail creation command *compile_pg* will decide rail width from PG pins of cells in routing area specified in PG strategy. With the app_option, tool will find cells in the routing area to decide rail width. The app-option works only when cells in the routing areas are placed.

p

By default, tool decides rail width from cells in a design, which is independent on locations of the cells and routing area.

This application option is set using the `set_app_options` command. To determine the current value of this application option on a specific design, use `get_app_option_value -name opt_name`. Use `report_app_options` to show the value of multiple application options.

See Also

- [create_pg_std_cell_conn_pattern](#)
- [set_pg_strategy](#)
- [compile_pg](#)

plan.pgroute.default_tag

Specify default tag for PG commands.

Data Types

string

Default {}

Description

This app option is used to specify default tag for PG commands when PG commands do not specify tag. If PG commands already specify tag in command option, then the tag in command option takes precedence.

Examples

All shapes and vias generated in `compile_pg` will have tag value "defaultTag".

```
prompt> set_app_options -name plan.pgroute.default_tag defaultTag
prompt> compile_pg
```

All vias generated in `create_pg_vias` will have tag value "newTag" since `create_pg_vias` specified its own tag.

```
prompt> set_app_options -name plan.pgroute.default_tag defaultTag
prompt> create_pg_vias -tag newTag
```

See Also

- [compile_pg](#)
- [create_pg_vias](#)

plan.pgroute.derive_cut_net_from_pin

Enable the feature to derive net information for cut without net information if the cut is overlapped by pin.

Data Types

Bool

Default true

Description

When set to true, PG router DRC engine will derive net information for cut without net information. Cut here stands for wire shape or regular routing blockage on via cut layer. When cut without net information overlaps with pins in the same hierarchy, PG derive engine will treat the cut has the same net as overlapped pin. By default, the value is true.

This application option is set using the *set_app_options* command. To determine the current value of this application option on a specific design, use *get_app_option_value -name opt_name*. Use *report_app_options* to show the value of multiple application options.

See Also

- [compile_pg](#)
- [create_pg_vias](#)
- [create_pg_strap](#)

plan.pgroute.derive_pg_mask_engine_version

Specifies the version of *derive_pg_mask_constraint*. The larger number means newer engine. It affects runtime.

Data Types

Integer

Default 2

Description

When the number is set to two, which is the default, *derive_pg_mask_constraint* uses new algorithm to derive color of wire and via enclosure. The valid values are: 1 and 2. Version 2 engine should be faster than version 1.

p

This application option is set using the *set_app_options* command, and can be specified globally. To determine the current value of this application option on a specific design, use *get_app_option_value -name opt_name*. Use *report_app_options* to show the value of multiple application options.

See Also

- [derive_pg_mask_constraint](#)

plan.pgroute.disable_floating_removal

Force PG route command to not remove floating wires/vias during straps/vias creation.

Data Types

Boolean

Default false

Description

When set to true, PG route command *compile_pg* does not remove floating wires/vias. This option is set in multi-level physical hierarchy PG creation flow to maintain PG connectivity between physical hierarchy. By default, the value is false.

This application option is set using the *set_app_options* command. To determine the current value of this application option on a specific design, use *get_app_option_value -name opt_name*. Use *report_app_options* to show the value of multiple application options.

See Also

- [characterize_block_pg](#)
- [compile_pg](#)

plan.pgroute.disable_stapling_via_fixing

Disable DRC fixing engine for stapling vias.

Data Types

Boolean

Default false

p

Description

If the `app_option` is true, when DRC checking engine detects DRC violation for a stapling via, DRC fixing engine will not fix it. Instead, the via will not be created. The `app_option` is to reduce runtime for stapling via creation. By default, the value is false.

This application option is set using the `set_app_options` command. To determine the current value of this application option on a specific design, use `get_app_option_value -name opt_name`. Use `report_app_options` to show the value of multiple application options.

See Also

- [compile_pg](#)
- [create_pg_vias](#)

plan.pgroute.disable_strategy_reevaluation

Do not re-evaluate Tcl variables used in the PG strategy during `compile_pg`.

Data Types

Boolean

Default false

Description

When set to true, the `compile_pg` PG route command does not re-evaluate Tcl variables used in PG strategy during `compile_pg`. By default, the value is false and the `compile_pg` command re-evaluates all Tcl variables defined in PG strategy. This is done to return the latest values for the corresponding Tcl variables used in the PG strategy.

This application option is set using the `set_app_options` command, and may be specified globally. To determine the current value of this application option on a specific design, use `get_app_option_value -name opt_name`. Use `report_app_options` to show the value of multiple application options.

See Also

- [compile_pg](#)
- [get_app_options](#)
- [report_app_options](#)

- [set_app_options](#)
- [set_pg_strategy](#)

plan.pgroute.disable_trimming

Force PG route command to not trim dangling wires during PG wire creation.

Data Types

Boolean

Default false

Description

When set to true, PG route command *compile_pg* does not trim dangling wires. This option is set in multi-level physical hierarchy PG creation flow to maintain PG connectivity between physical hierarchy. By default, the value is false.

This application option is set using the *set_app_options* command. To determine the current value of this application option on a specific design, use *get_app_option_value -name opt_name*. Use *report_app_options* to show the value of multiple application options.

See Also

- [characterize_block_pg](#)
- [compile_pg](#)

plan.pgroute.disable_via_creation

Do not create any vias during compile_pg.

Data Types

Boolean

Default false

Description

When set to true, PG route command *compile_pg* does not create any vias even though via masters are specified. By default, the value is false.

This application option is set using the *set_app_options* command, and may be specified globally. To determine the current value of this application option on a specific design, use

p

get_app_option_value -name opt_name. Use *report_app_options* to show the value of multiple application options.

See Also

- [compile_pg](#)

plan.pgroute.discard_stackvia_with_drc

Specifies if PG route command to discard the whole stacked via when the stack via is not DRC clean.

Data Types

Boolean

Default false

Description

When set to true, PG route commands do not perform via DRC fixing, and discard the whole stacked via when any via in the stacked via is not DRC clean. This is for runtime speed-up. By default, DRC fixing will be performed for stacked vias which are not DRC clean.

This application option is set using the *set_app_options* command, and may be specified globally. To determine the current value of this application option on a specific design, use *get_app_option_value -name opt_name*. Use *report_app_options* to show the value of multiple application options.

See Also

- [compile_pg](#)
- [create_pg_vias](#)
- [create_pg_strap](#)

plan.pgroute.drc_check_fast_mode

Specifies whether fast mode is used when the PG router is performing DRC checks.

Data Types

Boolean

Default false

p

Description

When this application option is true, PG route commands use low-effort or fast-mode DRC checking to improve runtime. Specifically, the DRC checking engine skips the top and bottom layers of stacked vias and checks only the layers between them. For example, if your design contains a stacked via between layers M2 and M5 and this application option is true, the DRC checking engine checks only the layers V2, M3, V3, M4, and V4; checking for layers M2 and M5 is skipped. As a result, runtime might be slightly reduced. For the greatest possible runtime reduction, you should use multithreading.

When this application option is true, certain DRC violations might not be checked. For example, when the M2 and M5 layer of the stacked via is larger than the M2 and M5 wires, the DRC checking engine might miss certain DRC rules, such as the minimum edge rule.

This application option is set using the *set_app_options* command, and can be specified globally. To determine the current value of this application option on a specific design, use *get_app_option_value -name opt_name*. Use *report_app_options* to show the value of multiple application options.

See Also

- [compile_pg](#)
- [create_pg_strap](#)
- [create_pg_vias](#)

plan.pgroute.enable_rdl_45_shapes

Enable via creations on 45 degree RDL shapes.

Data Types

Boolean

Default false

Description

By default, *create_pg_vias* only handle manhattan geometries. To allow via creations on 45 degree RDL shapes, users need to set this app option to true. Once the feature is enabled, pg router will use rectangle shapes as input and calculate the intersection area with 45 degree RDL shapes. If an intersection area (potential via site) is a valid rectangle then via will be created.

There are some limitations for the current implementation. The intersections between two 45 degree RDL shape is not supported. Also, by default *create_pg_vias* does not merge

p

input shapes. This may produce intersected via sites and the vias created may not be as expected.

This application option is set using the *set_app_options* command, and may be specified globally. To determine the current value of this application option on a specific design, use *get_app_option_value -name opt_name*. Use *report_app_options* to show the value of multiple application options.

See Also

- [create_pg_vias](#)

plan.pgroute.fix_via_drc_multiple_viadev

Fix DRC of via instances using multiple contact codes defined in via master list.

Data Types

Boolean

Default false

Description

By default, PG router uses all available contact codes to fix DRC of via masters. Once a contact code is selected to create a DRC-clean via master, the contact code is used for all corresponding via instances, and will not be changed during DRC fixing of via instances, which may result in failing to fix DRC of via instances for some cases. To enable PG router to use multiple contact code to fix DRC of via instances, please apply the following setting: (1) apply the app_option *plan.pgroute.fix_via_drc_multiple_viadev*, (2) define a via master rule and use the option *-contact_code* to specify list of available *contactCode*, and (3) specify the defined via master rule in the option *-via_masters* of *create_pg_vias*, PG patterns, or PG strategy via rules. Note that the app_option may increase runtime since it will use all defined/available contact codes to fix DRC.

This application option is set using the *set_app_options* command. To determine the current value of this application option on a specific design, use *get_app_option_value -name opt_name*. Use *report_app_options* to show the value of multiple application options.

Examples

```
set_app_options -name plan.pgroute.fix_via_drc_multiple_viadev -name true

set_pg_via_master_rule vrule -contact_code \ {VIA67_BW114 VIA67_BW76
VIA67_BW38_UW38 VIA67_BW21}

create_pg_vias -from_layer M6 -to_layer M7 -nets vccint -via_masters {vrule}
```

p

See Also

- [create_pg_vias](#)
- [create_pg_mesh_pattern](#)
- [create_pg_composite_pattern](#)
- [set_pg_strategy_via_rule](#)
- [compile_pg](#)

plan.pgroute.high_capacity_mode

Enables high capacity mode to significantly reduce the memory required during specific PG router commands.

Data Types

Enum

Default 0

Description

Determine the approach to reducing total memory usage. One of the following values are valid:

0 - Feature is disabled. This is the default.

1 - Enables high capacity mode for the *compile_pg* and *create_pg_vias* PG router commands and significantly reduces memory required by the multi-threading PG router when performing via DRC checking and fixing. The floating wire and vias removal engine is off in this mode.

2 - Enables high capacity mode for the *compile_pg* PG router command and further reduces memory required by processing wires and vias in a divide and conquer approach. The floating wire and vias removal engine is off in this mode. In addition, when you activate the *start_dpx_workers* command, the distributed processing feature gets enabled in this mode. If more than 30 cores are available, the divide and conquer approach enables additional threading operations for improved runtime.

For backwards compatibility, the values *false* and *true* can be used and corresponds to the values 0 and 1.

This application option is set using the *set_app_options* command. To determine the current value of this application option on a specific design, use *get_app_option_value -name opt_name*. Use *report_app_options* to show the value of multiple application options.

p

See Also

- [compile_pg](#)
- [create_pg_vias](#)
- [start_dpx_workers](#)

plan.pgroute.hmpin_connection_target_layers

Specify layer names of wires which HM pins are connected to.

Data Types

String

Default ""

Description

When the string is set, PG route command *compile_pg* with hard macros/pads connectin pattern will connect to wires with only the specified layers. By default, the value is "".

This application option is set using the *set_app_options* command. To determine the current value of this application option on a specific design, use *get_app_option_value -name opt_name*. Use *report_app_options* to show the value of multiple application options.

See Also

- [create_pg_macro_conn_pattern](#)
- [compile_pg](#)

plan.pgroute.honor_cover_cell

Specifies whether PG routing should honor cover cell.

Data Types

Boolean

Default true

Description

Use this application option to specify whether PG routing should honor routing blockage inside cover cell. By default, tool honors routing blockage inside cover cell.

Examples

The following setting honors cover cell in PG routing.

```
prompt> set_app_options -name plan.pgroute.honor_cover_cell -value true
```

See Also

- [compile_pg](#)

plan.pgroute.honor_ndr_same_net_spacing

Specifies if PG route commands should honor NDR spacing rules between same nets.

Data Types

Boolean

Default false

Description

When set to true, PG route commands honor NDR spacing rules between same net shapes. Otherwise, NDR spacing is not checked against same net shapes. By default, the value is false.

This application option is set using the *set_app_options* command, and may be specified globally. To determine the current value of this application option on a specific design, use *get_app_option_value -name opt_name*. Use *report_app_options* to show the value of multiple application options.

See Also

- [compile_pg](#)
- [create_pg_vias](#)
- [create_pg_strap](#)

plan.pgroute.honor_ndr_spacing_drc

Specifies if PG route commands should honor NDR spacing rules for DRC check.

Data Types

Boolean

Default false

p

Description

When set to true, PG route commands honor net based NDR spacing for DRC check. Otherwise, NDR rules are not honored. By default, the value is false.

This application option is set using the *set_app_options* command, and may be specified globally. To determine the current value of this application option on a specific design, use *get_app_option_value -name opt_name*. Use *report_app_options* to show the value of multiple application options.

See Also

- [compile_pg](#)
- [create_pg_vias](#)
- [create_pg_strap](#)

plan.pgroute.honor_signal_route_drc

Specifies if PG route commands should honor signal route for DRC check.

Data Types

Boolean

Default false

Description

When set to true, PG route commands honor the existing signal route for DRC check. Otherwise, signal route is ignored for DRC check. By default, the value is false.

This application option is set using the *set_app_options* command, and may be specified globally. To determine the current value of this application option on a specific design, use *get_app_option_value -name opt_name*. Use *report_app_options* to show the value of multiple application options.

See Also

- [compile_pg](#)
- [create_pg_vias](#)
- [create_pg_strap](#)

plan.pgroute.honor_std_cell_drc

Specifies if PG route commands should honor internal shapes of standard cells for DRC check.

Data Types

Boolean

Default false

Description

When set to true, PG route commands honor the internal shapes of standard cells for DRC check. Otherwise, standard cell is ignored for DRC check. By default, the value is false.

This application option is set using the *set_app_options* command, and may be specified globally. To determine the current value of this application option on a specific design, use *get_app_option_value -name opt_name*. Use *report_app_options* to show the value of multiple application options.

See Also

- [compile_pg](#)
- [create_pg_vias](#)
- [create_pg_strap](#)

plan.pgroute.load_frame_view

Specifies if forcing PG route commands to load frame view for instances (macros, blocks, standard cells etc).

Data Types

Boolean

Default true

Description

When set to true, PG route commands load frame view of instances (macros, blocks, standard cells) for DRC checking. Otherwise, the current view of instances gets loaded. By default, the value is true.

This application option is set using the *set_app_options* command, and might be specified globally. To determine the current value of this application option on a specific design, use

p

get_app_option_value -name opt_name. Use *report_app_options* to show the value of multiple application options.

See Also

- [compile_pg](#)

plan.pgroute.max_extension_distance

Specify the maximum distance that an extension can reach.

Data Types

Double

Default -1

Description

When PG strategy defines extension, for example, first target, inner ring, outer ring, etc, tool makes extension no matter how far the target object is if the app option is not specified. This app option defines the range for the extension. If the extension is greater than the specified value, the extension will be dropped.

The default value is -1, which means no limit for the extension.

This application option is set using the *set_app_options* command. To determine the current value of this application option on a specific design, use *get_app_option_value -name opt_name*. Use *report_app_options* to show the value of multiple application options.

See Also

- [set_pg_strategy](#)
- [compile_pg](#)

plan.pgroute.max_undo_steps

Specifies the maximum number of undo steps for the *compile_pg* PG route command.

Data Types

Integer

Default 5

p

Description

This app option specifies the maximum number of undo steps of *compile_pg* PG route command. By default, the number of steps is five and the last five steps of *compile_pg* command can be reversed.

This application option is set using the *set_app_options* command, and may be specified globally. To determine the current value of this application option on a specific design, use *get_app_option_value -name opt_name*. Use *report_app_options* to show the value of multiple application options.

See Also

- [compile_pg](#)
- [create_pg_strap](#)
- [create_pg_vias](#)

plan.pgroute.maximize_total_cut_area

Uses the contactCode with the maximum total cut area.

Data Types

string

Default {}

Description

By default, tool uses default contactCode for via creation when via master is not specified. This application option limits the choice of the type of contactCode to use when creating vias with the maximum total cut area. The following inputs are supported: all, default, Vs, Vh, Vv.

- all: all possible contactCode.
- default: contactCode with property isDefaultContact equals to true.
- Vs: contactCode with square cuts.
- Vh: contactCode with horizontal bar
- Vv: contactCode with vertical bar

Examples

The following setting specifies the contactCode with the maximum total cut areas.

p

```
prompt> set_app_options -name plan.pgroute.maximize_total_cut_area -value
"all"
```

The following setting specifies only to use the square_cut contactCode with the maximum total cut areas.

```
prompt> set_app_options -name plan.pgroute.maximize_total_cut_area -value
"Vs"
```

The following setting specifies only to use square_cut or horizontal_bar contactCode with the maximum total cut areas.

```
prompt> set_app_options -name plan.pgroute.maximize_total_cut_area -value
"Vs Vh"
```

The following setting specifies only to use contactCode that has isDefaultContact to true for this application option.

```
prompt> set_app_options -name plan.pgroute.maximize_total_cut_area -value
default
```

See Also

- [compile_pg](#)
- [create_pg_strap](#)
- [create_pg_vias](#)

plan.pgroute.maximum_cell_gap_for_alignment_strap

Specify the maximum distance between two cells to use a continuous alignment strap.

Data Types

Double

Default -1

Description

When set to non-negative value, PG route command *compile_pg* with power switch alignment pattern or physical cell alignment pattern uses a separate alignment strap if distance between two neighboring cells is higher than the specified value. For example, if the value is set to 0.0, only abutted cells share the same straps. Please do not specify strategy extension with this option. By default, the value is -1.

This application option is set using the *set_app_options* command. To determine the current value of this application option on a specific design, use *get_app_option_value*

p

-name opt_name. Use *report_app_options* to show the value of multiple application options.

See Also

- [create_pg_special_pattern](#)
- [compile_pg](#)

plan.pgroute.merge_shapes_for_via_creation

Specifies whether the `create_pg_vias` command needs to merge overlapping shapes for via connection.

Data Types

Boolean

Default `true`

Description

When set to true, the `create_pg_vias` command merges overlapping shapes for via connection.

This application option is set using the *set_app_options* command, and can be specified globally. To determine the current value of this application option on a specific design, use *get_app_option_value -name opt_name*. Use *report_app_options* to show the value of multiple application options.

See Also

- [create_pg_vias](#)

plan.pgroute.merge_shapes_in_pad_cell

Allow metal shapes to be merged in various pad cell types.

Data Types

string

Default `{}`

Description

By default, PG router does not merge metal shapes in various pad cell types while performing DRC checks. This may create spacing violation if the metal shape is not a rectangle but a rectilinear polygon. Three different types of pad cells are supported. User

p

can selectively choose to enable which type or types of pad cells for merging. They are "io_pad", "corner_pad", and "flip_chip_pad".

This application option is set using the *set_app_options* command. To determine the current value of this application option on a specific design, use *get_app_option_value -name opt_name*. Use *report_app_options* to show the value of multiple application options.

Examples

```
set_app_options -name plan.pgroute.merge_shapes_in_pad_cell -value "io_pad
corner_pad flip_chip_pad"
```

See Also

- [compile_pg](#)

plan.pgroute.minimum_layer_length

Define minimum wire length requirements on the specified layers.

Data Types

list_of {string float}

Default {}

Description

To ensure sufficient via connections, wire length on certain layers must be longer than the minimum length defined by the technology. The purpose of this app option is specify minimum wire length requirements on specific layers. Floating wires shorter than the specified length will be removed prior to via insertion.

This application option is set using the *set_app_options* command, and may be specified globally. To determine the current value of this application option on a specific design, use *get_app_option_value -name opt_name*. Use *report_app_options* to show the value of multiple application options.

Examples

The following setting define a minimum length of 2.8 on layer ME4.

```
set_app_options -name plan.pgroute.minimum_layer_length -value {{ME4
2.8}}
```

The following setting define a minimum length of 2.8 on layer ME4 and 4.0 on layer ME6.

```
set_app_options -name plan.pgroute.minimum_layer_length -value {{ME4 2.8}
{ME6 4}}
```

p

See Also

- [compile_pg](#)

plan.pgroute.no_patch_fix_rectangle_only_rule

Specifies if forcing PG route commands not using patch method to fix rectangle only violation for vias.

Data Types

Boolean

Default true

Description

When set to true, PG route commands will not patch the newly created via that violates the rectangle only rule. The via will be discarded for DRC fixing. If it is set to false, PG route commands will patch the concave corner to make the merged metal shape to be a rectangle. By default, the value is true.

This application option is set using the *set_app_options* command, and may be specified globally. To determine the current value of this application option on a specific design, use *get_app_option_value -name opt_name*. Use *report_app_options* to show the value of multiple application options.

See Also

- [compile_pg](#)
- [create_pg_vias](#)

plan.pgroute.optimize_track_alignment

Optimize track alignment to save routing resource.

Data Types

Boolean

Default true

Description

When track alignment option is set and this option is true, A wire will be aligned to a track such that overlapping wires among different layers are moved to be closer to each other.

plan.pgroute.overlap_route_boundary

Specifies if PG route command allows PG wires being created partially overlapped with routing area boundary.

Data Types

Boolean

Default false

Description

When set to true, PG route command *compile_pg* allows PG wires being created partially overlapped with routing area boundary. Otherwise, these PG wires will not be created. By default, the value is false.

This application option is set using the *set_app_options* command, and may be specified globally. To determine the current value of this application option on a specific design, use *get_app_option_value -name opt_name*. Use *report_app_options* to show the value of multiple application options.

See Also

- [compile_pg](#)

plan.pgroute.overlap_route_boundary_exclude_design_boundary

Specifies if PG route command allows PG wires being created partially overlapped with routing area boundary which excludes the design boundary.

Data Types

Boolean

Default false

Description

When set to true, PG route command *compile_pg* allows PG wires being created partially overlapped with routing area boundary and excludes the design boundary. Otherwise, these PG wires will not be created. By default, the value is false.

This application option is set using the *set_app_options* command, and may be specified globally. To determine the current value of this application option on a specific design, use *get_app_option_value -name opt_name*. Use *report_app_options* to show the value of multiple application options.

See Also

- [compile_pg](#)

plan.pgroute.siterow_name

Specify site row name to use in PG routing.

Data Types

string

Default {}

Description

By default, tool tries to find the right site row to use automatically. However, when there are multiple non-default site row exists in the design, tool is unable to determine the right site row to use. This app option is used to specify the right site row to use.

Examples

The following setting specifies site row name "CORE" to use in PG routing.

```
prompt> set_app_options -name plan.pgroute.siterow_name -value CORE
```

See Also

- [compile_pg](#)

plan.pgroute.skip_balance_tree

Runtime optimization.

Data Types

Boolean

Default false

Description

This application option is not used. Changing its value does not impact the tool's behavior or runtime.

plan.pgroute.snap_stdcell_rail

Specifies if PG route command allows PG straps being extended or snapped to standard cell rails.

p

Data Types

Boolean

Default false

Description

When set to true, PG route command *compile_pg* allows PG straps being extended or snapped to standard cell rails. Otherwise, PG straps are not extended or snapped to standard cell rails. By default, the value is false.

This application option is set using the *set_app_options* command, and may be specified globally. To determine the current value of this application option on a specific design, use *get_app_option_value -name opt_name*. Use *report_app_options* to show the value of multiple application options.

See Also

- [compile_pg](#)

plan.pgroute.stapling_via_fast_mode

Specifies whether fast mode is used when *create_pg_stapling_vias* is used for non-adjacent layer via stapling.

Data Types

Boolean

Default true

Description

When this application option is true, *create_pg_stapling_vias* does not check DRC against neighboring stapling vias to be created, based on the assumption that neighboring stapling vias should not introduce DRC violations against each other. For example, if user creates PG stapling vias between layer M1 and M3, and pitch is 0.5um, suppose the command is to check the via stack at (10, 10), the DRC engine won't check against potential via stacks at (9.5, 10) and (10.5 10), if the app option is set to true.

This application option is set using the *set_app_options* command, and can be specified globally. To determine the current value of this application option on a specific design, use *get_app_option_value -name opt_name*. Use *report_app_options* to show the value of multiple application options.

See Also

- [create_pg_stapling_vias](#)

plan.pgroute.std_rail_from_refname

use specified reference of standard cells to compute width and shift of standard cell rails.

Data Types

string

Default {}

Description

By default, the tool selects a reference of standard cells, and uses pin geometries of the reference to compute width and shift of to-be-created standard cell rails. The `app_option` forces the tool to use pin geometries of the specified reference of standard cells to create standard cell rails.

Examples

The following setting is to use cell reference SVF_TAPDS to create standard cell rails.

```
set_app_options -name plan.pgroute.std_rail_from_refname -value
"SVF_TAPDS"
```

See Also

- [compile_pg](#)
- [set_pg_strategy](#)
- [create_pg_std_cell_conn_pattern](#)

plan.pgroute.treat_cell_type_as_macro

Create vias also on cells with the specified types. The supported types are: `lib_cell`, `filler`, `well_tap`, `end_cap` and `diode`.

Data Types

string

Default ""

Description

When specified, PG via creation command `create_pg_vias` and `compile_pg` will also create vias on pins of cells with the specified cell type. The `app_option` also works for pin connection of the selected cells specified by `create_pg_macro_conn_pattern`. The supported types are: `lib_cell`, `filler`, `well_tap`, `end_cap` and `diode`.

p

By default, vias are not created on pins of any non-macro cells.

This application option is set using the *set_app_options* command. To determine the current value of this application option on a specific design, use *get_app_option_value -name opt_name*. Use *report_app_options* to show the value of multiple application options.

See Also

- [create_pg_vias](#)
- [create_pg_macro_conn_pattern](#)
- [set_pg_strategy](#)
- [compile_pg](#)

plan.pgroute.treat_fat_blockage_as_fat_metal

Specifies whether PG route commands should honor routing blockages as real metal or cut shapes.

Data Types

Boolean

Default true

Description

When true, which is the default, PG route commands honor routing blockages as real metal or cut shapes for DRC check. When false, only overlapping and layer minimum spacing rules are checked for routing blockages. The tool applies the application option setting to routing blockages inside physical blocks, macros, or standard cells.

This application option is set using the *set_app_options* command. To determine the current value of this application option on a specific design, use *get_app_option_value -name opt_name*. Use *report_app_options* to show the value of multiple application options.

See Also

- [compile_pg](#)
- [create_pg_vias](#)
- [create_pg_strap](#)
- [check_pg_drc](#)

plan.pgroute.treat_fixed_stdcell_as_macro

Create vias also on pins of fixed standard cells. Connect pins of fixed standard cells.

Data Types

Boolean

Default false

Description

When set to true, PG via creation command *create_pg_vias* will also create vias on pins of fixed standard cells. The app_option also works for pin connection of standard cell specified by *create_pg_macro_conn_pattern*.

By default, vias are not created on pins of any standard cells.

This application option is set using the *set_app_options* command. To determine the current value of this application option on a specific design, use *get_app_option_value -name opt_name*. Use *report_app_options* to show the value of multiple application options.

See Also

- [create_pg_vias](#)
- [create_pg_macro_conn_pattern](#)
- [set_pg_strategy](#)
- [compile_pg](#)

plan.pgroute.treat_multiple_pins_as_one_target

Treat multiple pins as target for number keyword in *create_pg_special_pattern*.

Data Types

Boolean

Default false

Description

When set the value to true, the two special pattern *-insert_power_switch_alignment_straps* and *-insert_physical_cell_alignment_straps* will be impacted in special pattern. The number keyword in the two options means how many straps should connect to each pin. With the app_option on, all pins are treated as one target. In other word, it means how many straps should go through the bounding box of all pins instead of original behavior.

p

This application option is set using the *set_app_options* command, and may be specified globally. To determine the current value of this application option on a specific design, use *get_app_option_value -name opt_name*. Use *report_app_options* to show the value of multiple application options.

See Also

- [create_pg_special_pattern](#)

plan.pgroute.trim_by_blockage

Specifies whether the tool should consider routing blockage to decide the intersection region.

Data Types

Boolean

Default true

Description

When set to true, the *create_pg_vias* command considers routing blockage to initialize the intersection region for via creation. When there is partial overlapping with routing blockage, the command will still find the residual intersection for via creation. When it is false, the *create_pg_vias* command will not create any via when there is routing blockage overlapping with intersection region.

This application option is set using the *set_app_options* command, and may be specified globally. To determine the current value of this application option on a specific design, use *get_app_option_value -name opt_name*. Use *report_app_options* to show the value of multiple application options.

See Also

- [create_pg_vias](#)

plan.pgroute.use_fallback_spacing_rules

Uses fallback spacing rules during PG routing.

Data Types

Boolean

Default false

p

Description

This is for the minimal cut-to-cut distance estimation during maximized total cut area PG via creation. By default, the tool only looks at the minimum cut spacing rule in the technology file. If this application option is not set, a warning message is issued. If this application option is set, then the combined rules for the General Cut Spacing Rule is used, which is the same as the rules used by the DRC engine.

Examples

The following setting specifies the use of General Cut Spacing Rule during PG via creation.

```
prompt> set_app_options -name plan.pgroute.use_fallback_spacing_rules  
-value true
```

See Also

- [create_pg_vias](#)

plan.pgroute.use_shape_pattern

Instructs PG routing commands to infer shape_pattern objects instead of single shapes.

Data Types

Boolean

Default false

Description

By default, the PG routing commands create single shapes in the database. This might lead to a huge amount of database objects. To reduce the number of database objects that are in the memory, you can specify that the PG routing commands should infer shape_pattern objects. Shape pattern objects are regular repetitions of single shapes. Typically, PG structures are very repetitive and shape patterns can reduce the database size significantly. Note that the shape_pattern inference might come at a slight runtime increase and that the peak memory of the PG routing commands is not reduced by this option.

See Also

- [compile_pg](#)
- [create_pg_strap](#)
- [get_shape_patterns](#)

plan.pgroute.use_via_matrix

Instructs PG routing commands to infer via_matrix objects instead of single vias or single via arrays.

Data Types

Boolean

Default false

Description

By default, the PG routing commands create single vias or single via arrays in the database. This might lead to a huge amount of database objects. To reduce the number of database objects that are in the memory, you can specify that the PG routing commands should infer via_matrix objects. Via matrix objects are regular repetitions of single vias or single via arrays. Typically, PG structures are very repetitive and via matrixes can reduce the database size significantly. Note that the via matrix inference might come at a slight runtime increase and that the peak memory of the PG routing commands is not reduced by this option.

See Also

- [compile_pg](#)
- [create_pg_vias](#)
- [get_via_matrixes](#)

plan.pgroute.verbose

Specifies PG route command in verbose mode.

Data Types

Boolean

Default false

Description

When set to true, PG route command *compile_pg* may output more detailed information. By default, the value is false.

This application option is set using the *set_app_options* command, and may be specified globally. To determine the current value of this application option on a specific design, use *get_app_option_value -name opt_name*. Use *report_app_options* to show the value of multiple application options.

p

See Also

- [compile_pg](#)

plan.pgroute.via_array_size_control

Controls via array dimensions inside metal intersection during automatic via creation.

Data Types

string

Default {none}

Description

By default, the tool tries to fit the entire via array bounding box into metal intersection. To maximize number of cuts in array, the user may specify one of these values: {force_both_enclosures_in_intersection} - bottom and top via enclosures should fit in intersection {try_both_enclosures_in_intersection} - try to fit bottom and top via enclosures in intersection {try_one_enclosure_in_intersection} - try to fit at least one of enclosures in intersection {try_cuts_in_intersection} - try to fit cuts in intersection {enforce_via_rule_dimensions} - comply with via_rule dimensions regardless of intersection size {none} - do not consider this app option, this is default The tool may reduce or enlarge via array dimensions to fix DRC.

This application option is set using the *set_app_options* command, and may be specified globally. To determine the current value of this application option on a specific design, use *get_app_option_value -name opt_name*. Use *report_app_options* to show the value of multiple application options.

See Also

- [compile_pg](#)
- [create_pg_strap](#)
- [create_pg_vias](#)

plan.pgroute.via_site_threshold

Specifies the minimum via site threshold for PG via creation during PG route.

Data Types

Double

Default 0.0

p

Description

This app option specifies the minimum via site ratio threshold for PG via creation during PG route. By default, the value is 0.0, i.e. vias are created for all via sites. This app option can be used to control if vias are needed between partially intersected PG wires or between PG wires and macro pins. To elaborate it, if two wires are partially intersected, the threshold set by this app option is applied to decide whether a via should be created. The value is from 0 to 1. If it is 1, which means it has to be fully intersected to place a via.

The definition of via site ratio is as follows.

Suppose there are two boxes: ba and bb , which intersects each other. The corresponding intersection box is bx .

For vertical $bbox$, the via site ratio is $bx_width/bbox_width$ For horizontal $bbox$, the via site ratio is $bx_height/bbox_height$

For example, suppose ba is vertical and bb is horizontal. via_site_ratio of $ba = bx_width/ba_width$ via_site_ratio of $bb = bx_height/bb_height$

If minimum via site ratio is defined, via is created only when via site ratio of two corresponding intersected boxes are larger than the minimum via site ratio.

When minimum via site ratio is 1.0, vias are not created for partially overlapping wires.

This application option is set using the *set_app_options* command, and may be specified globally. To determine the current value of this application option on a specific design, use *get_app_option_value -name opt_name*. Use *report_app_options* to show the value of multiple application options.

See Also

- [compile_pg](#)
- [create_pg_vias](#)
- [create_pg_strap](#)

plan.pins.align_to_buffer_on_feedthrough**Data Types**

Boolean

Default "false"

Description

Enable this app-option to instruct place_pins to place the pins to be aligned with connected buffer cells.

p

When this app-option is set to true, the tool will trace the netlist connection to find the directly connected buffer cells and place the pins to be aligned to those cells. The buffer cells can be on driver side or on load side of the pins. When there are buffers on both driver/load side, the tool will use buffers on driver side to align the pins.

There is another app-option `plan.pins.buffer_cell_module_name` which can be used together with this option. When the `buffer_cell_module_name` is specified, pin placement will treat the cells which have same module name same as specified name as buffer even though those cells might not be "buffer" in the database.

This option affects the `place_pins` command.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [place_pins](#)
- [report_app_options](#)
- [set_app_options](#)

plan.pins.allow_feedthroughs_for_internal_channel_nets

Data Types

boolean

Default "false"

Description

This application option allows the design planning global router to route the nets that are totally inside reserved channels. This option is only valid when the `plan.pins.reserved_channel_threshold` application option is specified. If this application option is not specified, the nets that are totally inside reserved channels may not be routed for pin placement purpose.

This option affects the `place_pins` command only.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [place_pins](#)

- [report_app_options](#)
- [set_app_options](#)
- [set_block_pin_constraints](#)
- [set_individual_pin_constraints](#)

plan.pins.allow_mixed_feedthroughs

Data Types

boolean

Default "false"

Description

This application option allows the design planning global router to create the mixed feedthrough routing when feedthroughs constraints are disallowed. When the feedthroughs are allowed, this app option has no impact on the routing results. This app option only has impact when the feedthroughs are not allowed on either net or physical blocks. When the net or physical blocks have feedthroughs disallowed, with this app option set to true, the router can create mixed feedthroughs routings.

This option affects the *place_pins*, *route_global -virtual_flat* and *route_group -global_planning* command only.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)
- [place_pins](#)
- [route_global](#)
- [route_group](#)
- [set_block_pin_constraints](#)
- [set_individual_pin_constraints](#)

plan.pins.allow_pins_in_narrow_macro_channels

Data Types

boolean

Default "false"

Description

When this option is set to true, design planning global route can route through the narrow channel areas between block boundary and hardmacros. Also when this set to true, pin placement can also place the pins at the location inside the narrow channel areas between block boundary and hardmacros. By default, this option is set to false. This option needs to be used together with *plan.pins.reserved_channel_threshold*.

This option affects commands `place_pins`, `route_global -virtual_flat` and `route_group -global_planning`.

Examples

The following example sets this option to true.

```
prompt> set_app_options -list
{plan.pins.allow_pins_in_narrow_macro_channels true}
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [place_pins](#)
- [set_block_pin_constraints](#)
- [set_individual_pin_constraints](#)

plan.pins.allow_routing_over_readonly_blocks

Controls how readonly design view and abstract view physical blocks are treated by hierarchical global router.

Data Types

String

Default auto

p

Description

This application option controls how the hierarchical router treats physical blocks which are set uneditable via hierarchical edit control. This app option does not apply to physical blocks in frameview, i.e. the router always considers frameview physical blocks as hard macros, thus can route over them.

When *auto* (the default), the router automatically determines if a readonly physical block should be routed over. If this physical block has all of its signal pins placed and feedthrough is disallowed in block pin constraints, then router will not be able to route over this physical block, and the area of this block regarded as blockage. Otherwise, the physical block can be routed over.

If set to *true*, the router will be able to route over all readonly physical blocks.

If set to *false*, the router will not route over any readonly physical block.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_editability](#)

plan.pins.black_box_design_style

Data Types

boolean

Default "false"

Description

Specifies whether the tool should treat the design as black box design. When this option is set to true, during place_pins, the tool will use bigger gcell size to improve the runtime.

By default, this option is false.

This option affects commands plan_pins only and doesn't affect route_global -virtual_flat nor route_group -global_planning.

See Also

- [get_app_option_value](#)
- [get_app_options](#)

- [report_app_options](#)
- [place_pins](#)

plan.pins.buffer_cell_module_name

Data Types

string

Default ""

Description

This app-option has to be used together with "plan.pins.align_to_buffer_on_feedthrough". Otherwise it will be ignored.

When this option is specified, pin placement will treat the cells which have same module name same as specified name as buffer even though those cells might not be "buffer" in the database.

This option affects the *place_pins* command.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [place_pins](#)
- [report_app_options](#)
- [set_app_options](#)

plan.pins.create_double_pattern_long_pin

Data Types

Boolean

Default "true"

Description

Directs the *place_pins* and *check_pin_placement* commands to create long pin when double pattern technology is used to help detail routing. The default value is true (creating long pin). This app option only applies when double pattern technology applied in the current design.

p

This option affects the *place_pins* (during pin creation) and *check_pin_placement* (during pin size checking) commands.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [place_pins](#)
- [set_individual_pin_constraints](#)
- [read_pin_constraints](#)
- [report_app_options](#)
- [set_app_options](#)

plan.pins.detour_supernet_mode**Data Types***Boolean***Default** "false"**Description**

Directs the *check_pin_placement* commands to skip over repeaters and consider the flat net during detour computation.

By default this option is false, and in default behavior detour checking stops at repeater pins. Using this allows users to trace back to combination drivers/loads associated with the supernet.

Note that this app-option is ignored if the *plan.pins.path_based_pin_placement* is set to true. That is the path based pin placement mode is given priority over this supernet mode based detour checking option.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [check_pin_placement](#)
- [report_app_options](#)
- [set_app_options](#)

plan.pins.enable_color_span_rules

Data Types

Boolean

Default "true"

Description

This option is deprecated and scheduled for removal in a future release. This behavior is now on-by-default and controlled by the *plan.pins.enable_span_rules* app-option.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [place_pins](#)
- [check_pin_placement](#)
- [report_app_options](#)
- [set_app_options](#)

plan.pins.enable_feedthrough_timestamps

Data Types

Boolean

Default "false"

Description

Specify if a timestamp attribute is added to the feedthroughs created by *place_pins* or *push_down_objects*. The timestamp is determined by the system time when either one of these two commands are started. The timestamp can then be recognized by command *write_shadow_eco* such that the feedthroughs created by certain *place_pins* or *push_down_objects* may be written out in the shadow eco script.

Please note, the smallest time resolution on Unix/Linux is 1 second. Therefore, if the time interval between two feedthrough creation commands is less than 1 second, then the timestamps applied on the feedthroughs created by both commands should be identical. To guarantee different timestamps from different commands, add extra wait time (e.g. the Unix/Linux shell command *sleep*) if necessary.

By default, this option is false.

p

This option affects the *place_pins* and *push_down_objects* commands.

See Also

- [place_pins](#)
- [push_down_objects](#)
- [set_app_options](#)
- [report_app_options](#)

plan.pins.enable_span_rules

Data Types

Boolean

Default "true"

Description

Directs the *place_pins* and *check_pin_placement* commands to load the basic and color-based span rules along with corresponding exceptions from technology file and honor them during pin creation/checking.

Option controls span rules for both signal-to-signal and signal-to-PG shapes.

This span rule support is ON by default.

Note that the type of span rules supported is the "simple two dimensional span rule". It is listed under the property *spanTblYMinSpacing* and *spanTblXMinSpacing* in the technology file.

There is also support for same color and different color-based span rules. Some technologies have individual mask1 and mask2 specific span rules instead of the same color rules, and that is also supported. The tool expects to find either same color span rules or both mask1 and mask2 span rules in the technology file.

Currently, the exception tables listed under the following properties are supported and the tool expects to find only one of these defined for a given span table of a metal layer: *spanTbl(X/Y)ExcludedSpacingRange2* *spanTbl(X/Y)ExcludedSpacingRange3* (*diff/same/mask1/mask2/*)*ColorSpanTbl(X/Y)ExcludedSpacingRange*

Users can set this application option to false if they have created their tracks with appropriate spacing to avoid span rule violations.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [place_pins](#)
- [check_pin_placement](#)
- [report_app_options](#)
- [set_app_options](#)

plan.pins.exclude_clocks_from_feedthroughs

Data Types

boolean

Default "true"

Description

Specifies whether feedthrough port creation is excluded on clock nets.

By default, this option is true (clock nets are excluded).

This option affects commands `plan_pins`, `route_global -virtual_flat` and `route_group -global_planning`.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [place_pins](#)
- [set_block_pin_constraints](#)
- [set_individual_pin_constraints](#)

plan.pins.exclude_inout_nets_from_feedthroughs

Data Types

boolean

p

Default "true"**Description**

Specifies whether feedthrough port creation is excluded on nets that are connected to multiple driver pins, or nets that are without driver pins, or nets that are connected to inout pins. When this option is set to true, the feedthrough port and feedthrough net are disabled on all the nets that are connected to multiple driver pins, or nets that are without driver pins, or nets that are connected to inout pins.

By default, this option is true.

This option affects commands `plan_pins`, `route_global -virtual_flat` and `route_group -global_planning`.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [place_pins](#)
- [set_block_pin_constraints](#)
- [set_individual_pin_constraints](#)

plan.pins.fast_route**Data Types***boolean***Default** "false"**Description**

Performs fast global route based pin assignment.

When you set this application option to true, the pin assignment engine uses the fast global routing mode to route the nets for pin assignment purpose. The fast global routing mode uses maze routing algorithm, which is similar to what the Zroute global router uses to derive optimized routing solutions. All the pin constraints and most of the routing rules are supported. This includes non-default routing rules, routing corridors, routing guides, routing blockages, and pin guides.

However, timing driven mode is currently not supported in fast routing mode. If you set this `plan.pins.fast_route` application option to true and also turn on the timing driven mode in

`place_pins` , `place_pins` uses the fast global routing mode and ignores the timing driven feature.

This `plan.pins.fast_route` application option affects the `place_pins` command.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [place_pins](#)
- [route_global](#)
- [set_block_pin_constraints](#)
- [set_individual_pin_constraints](#)

`plan.pins.force_net_port_name_match`

Data Types

boolean

Default "false"

Description

With this app option set to true, this application option forces net has same name to one of its connected ports after push down,

This option affects the *push_down_objects* command only.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)
- [push_down_objects](#)

plan.pins.ignore_pin_spacing_constraint_to_pg

Data Types

Boolean

Default "false"

Description

Directs the *place_pins* and *check_pin_placement* commands to only honor or check the layers minimum spacing or NDR spacing rules (if they exist) for spacing between pins and PG pre-routes. Note that pin to pin spacing constraints are still honored as per usual and are unaffected by this option.

The default value is false, and with the default behavior the relevant user set spacing constraints on pins/blocks/nets etc. also apply to spacing between pins and PG pre-routes.

This option affects the *place_pins* (during pin creation) and *check_pin_placement* (during pin spacing checking) commands.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [place_pins](#)
- [set_individual_pin_constraints](#)
- [read_pin_constraints](#)
- [report_app_options](#)
- [set_app_options](#)

plan.pins.incremental

Data Types

boolean

Default "false"

Description

Performs incremental pin placement. Use this option to rerun pin placement after making changes to the floorplan or pin constraints.

p

The placer uses the existing pin locations to incrementally improve the pin placement QoR. The QoR considerations are wirelength, congestion, and pin alignment. If QoR improvement can be achieved by moving a pin to a new location and/or new layer within the displacement range and layer range specified by *pin_range* and *layer_range*, the new location and/or new layer will replace the original ones. Otherwise, the original location and layer will remain unchanged.

If the floorplan update is too large or newly applied pin constraints are too tight such that the original pin locations cause constraints violations, the tool tries to fix those violations within the displacement range and layer range specified by *pin_range* and *layer_range*. If constraints violations cannot be fixed within specified displacement and layer range, constraints will be relaxed and warning messages will issued.

Under any circumstances, the pins should remain at the original edges.

If there are missing pins, the command places the missing pins and ignores the *pin_range* and *layer_range* values for those missing pins.

This option affects commands `place_pins`, `route_global -virtual_flat` and `route_group -global_planning`.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [place_pins](#)
- [set_block_pin_constraints](#)
- [set_individual_pin_constraints](#)

plan.pins.layer_range

Sets layer range for incremental pin placement

Data Types

int

Default "2"

Description

Specifies the allowable layer displacement from the original layer for the pin when using the *plan.pins.incremental* option. By default, the *layer_range* is 2 and pins can be moved up or down by 2 metal layers.

This option affects commands `place_pins`, `route_global -virtual_flat` and `route_group -global_planning`.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [place_pins](#)
- [set_block_pin_constraints](#)
- [set_individual_pin_constraints](#)

plan.pins.maximize_black_box_pin_grouping

Data Types

Boolean

Default "false"

Description

Specifies whether the tool should group block pins together for black box designs. When this option is set to true, the tool will try to place the block pins together that have same block to block connections. When this option is set to false, the tool will try to spread out the pins on boundaries.

By default, this option is false.

This option affects commands `plan_pins`, `route_global -virtual_flat` and `route_group -global_planning`.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [place_pins](#)

plan.pins.minimize_feedthroughs

Data Types

boolean

Default "false"

Description

When this option is true, the design planning global router tries to minimize the number of feedthrough routes. In general, the number of feedthrough routes should be less than the routing result when this application option is false. By default, this option is false.

In channel based designs, when this app option is set to true, more routing will go through channel areas instead of routing through physical blocks. As a results, users may see increased wirelength and reduced number of feedthroughs in channel based designs.

In fully abutted design, when using this option, it may also cause wirelength increase so it is not recommended.

This option affects the *place_pins*, *route_global -virtual_flat*, and *route_group -global_planning* commands.

Examples

The following example sets this application option to true to minimize feedthroughs.

```
prompt> set_app_options -list {plan.pins.minimize_feedthroughs true}
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [place_pins](#)
- [report_app_options](#)
- [route_global](#)
- [route_group](#)
- [set_app_options](#)
- [set_block_pin_constraints](#)
- [set_individual_pin_constraints](#)

plan.pins.new_cell_name_tag

Data Types

string

Default "PUSHDOWN"

Description

Specifies the naming suffix for newly pushed down cells in `push_down_objects` command. When this option is specified, the tool uses *new_cell_name_tag* as a suffix for newly pushed down cells. In particular, the new name is "cell_name__name_tag", where "name_tag" is the string specified by this option. By default, newly created cells are named "cell_name__PUSHDOWN", where cell_name is the name of the original cell.

The following symbols "\V[]" are not allowed in the suffix name.

This option affects `push_down_objects` command.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [push_down_objects](#)

plan.pins.new_port_name_tag

Data Types

string

Default "PUSHDOWN"

Description

Specifies the naming suffix for newly created feedthrough ports, feedthrough nets in `place_pins`, `push_down_objects`, and virtual flat mode `route_global` commands. When this option is specified, the tool uses *new_port_name_tag* as a suffix for newly created feedthrough ports and feedthrough nets. In particular, the new name is "net_name__name_tag_#", where "name_tag" is the string specified by this option. By default, newly created feedthrough ports and nets are named "net_name__FEEDTHRU_#" in `place_pins`, "net_name__PUSHDOWN_#" in `push_down_objects`, and "net_name__VF_GROUTE_#" in virtual flat mode `route_global`,

p

where `net_name` is the name of the original net that connected this feedthrough port or feedthrough net, and `"#"` is an index number.

This option affects commands `plan_pins`, `push_down_objects`, and `route_global`.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [place_pins](#)
- [push_down_objects](#)
- [route_global](#)

plan.pins.new_port_name_tag_style

Data Types

string

Default "suffix"

Description

Specifies the naming style is suffix or prefix for newly created feedthrough ports and feedthrough nets in `place_pins` or `push_down_objects` commands.

When this option is specified as "suffix", the tool puts the new port name tag as suffix for newly created feedthrough ports and feedthrough nets. In particular, the new name is `"net_name__name_tag_#"`, where `"name_tag"` is the string specified by `"place_pins -new_port_name_tag"` or specified by `"set_push_down_object_options -new_port_name_tag"` and `"#"` is an index number.

When this option is specified as "prefix", the tool puts the new port name tag as prefix for newly created feedthrough ports and feedthrough nets. In particular, the new name is `"name_tag_#__net_name"`, where `"name_tag"` is the string specified by `"plan.pins.new_port_name_tag"` application option, and `"#"` is an index number.

By default, this option is set to "suffix"

This option affects commands `plan_pins`, `push_down_objects`.

p

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [place_pins](#)
- [push_down_objects](#)

plan.pins.new_split_nets_use_block_names**Data Types**

Boolean

Default "false"**Description**

When true, the tool changes how feedthrough net names are generated. All new feedthroughs split nets which occur between feedthrough blocks (but which do NOT connect to top-level ports) to embed a sequence of block master names into the new split net name. This generated embedded string begins with the master name of the driver block, followed by all load block master names, separated by "__".

For example, the following feedthrough net name:

```
FOO_FEEDTHRU_0
```

is modified by inserting the block names as follows:

```
FOO__BLOCKA__BLOCKB__FEEDTHRU_0
```

The following bus feedthrough name:

```
FOO_FEEDTHRU_0 ---> converts to ---> BLOCKA__BLOCKB__FOO__FEEDTHRU_0
```

is modified by inserting block names as follows:

```
FOO_FEEDTHRU_0[0] ---> converts to --->
BLOCKA__BLOCKB__FOO__FEEDTHRU_0[0]
```

This option affects the *place_pins* and *push_down_objects* commands.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [place_pins](#)
- [push_down_objects](#)
- [report_app_options](#)
- [set_app_options](#)

plan.pins.path_based_pin_placement

Data Types

boolean

Default "false"

Description

This option is used to control pin placement to place pins close to driver/load registers' location. When set to true, place_pins will traverse the netlist from block ports and find the registers on the load side and on the driver side and use those registers location to guide the routing and place the pins. This option affects commands place_pins, route_global -virtual_flat and route_group -global_planning.

When this option is set to true and the directly connected pins are located inside narrow channels, the pin placement based on the registers location might not happen since the tool prioritizes routing within narrow channels and derives a solution which minimizes routing within such narrow channels.

This option doesn't affect the nets that skipped routing during pin placement with command "place_pins -nets_to_exclude_from_routing". In addition, the tool will ignore this app option for the nets that don't connect to any physical block pins.

Examples

The following example sets this option to true.

```
prompt> set_app_options -list {plan.pins.path_based_pin_placement true}
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)

- [report_app_options](#)
- [place_pins](#)
- [route_global](#)
- [route_group](#)
- [check_pin_placement](#)

plan.pins.pin_range

Sets pin location range for incremental pin placement

Data Types

float

Default "10"

Description

Specifies the allowable displacement distance from the original pin location when using the *plan.pins.incremental* option. The unit is in micron. The default is 10 micron.

This option affects commands `place_pins`, `route_global -virtual_flat` and `route_group -global_planning`.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [place_pins](#)
- [set_block_pin_constraints](#)
- [set_individual_pin_constraints](#)

plan.pins.reserved_channel_threshold

Data Types

float

Default "site row's height"

p

Description

Specifies the maximum channel size to channel routing. The units are in microns. You must specify a positive value for *channel_size*. The default is same as default site row's height.

A "channel" is the space between a block or module on which pins can be placed and an adjacent module, hard macro, or I/O pad. For channel formation, all modules and cells (except standard cells) are considered. At least one side of the channel must be formed by a block. The space between two hard macros is not considered as a channel by this option. The chip boundary is also considered during channel creation.

The *plan.pins.reserved_channel_threshold* option modifies the behavior of global route and pin placement within channels. There is a higher cost for pins on modules that face the channel, unless they are perfectly aligned across the channel, or are pins belonging to the special nets (that don't have feedthrough allowed). Nets that are allowed for feedthrough routing will create feedthrough instead of entering the channels. This creates feedthrough pins for these nets.

This option is useful for floorplans with only narrow channel spaces between the modules. In this case, feedthroughs should generally be allowed, except for the special nets where feedthroughs are excluded.

When this option is specified, global route will use higher effort to avoid routings inside channel area to achieve better pin alignment.

This option affects commands `place_pins`, `route_global -virtual_flat` and `route_group -global_planning`.

Examples

The following example sets the *plan.pins.reserved_channel_threshold* option to 100 micron.

```
prompt> set_app_options -global {plan.pins.reserved_channel_threshold
100}
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [place_pins](#)
- [set_block_pin_constraints](#)
- [set_individual_pin_constraints](#)

plan.pins.retain_bus_name

Data Types

string

Default "true"

Description

This option is used to control how the new created bus names should look like.

Valid values are

- *true* (default)

When set to "true", causes all new feedthroughs nets and ports created during design planning operations `place_pins` and `push_down_objects` to keep bus notation.

- *false*

When set to "false", causes all new feedthroughs nets and ports created during design planning operations `place_pins` and `push_down_objects` to use a scalarized (non-bus) notation in the names when it detects that the originally-derived name would, when written out to Verilog by `write_verilog`, result in an escaped name (preceded by a back-slash).

The name substitution would be of the form:

NAME[0] ---> converts to ---> NAME__0

This change only applies to ports and nets that are not marked in the database as bus ports/nets. If a port/net has a bus-style notation, AND it is marked as a bus in the database, then it will not substitute the name with a non-bus notation since when written out to Verilog, it will not require a back-slash at the beginning of the name. Fast run time. Runs a maximum of three phases.

- *force_scalarized_names*

When set to "force_scalarized_names", causes all new feedthroughs nets and ports created during design planning operations `place_pins` and `push_down_objects` to use a scalarized (non-bus) notation in the names. This applies to even valid bus nets. i.e., even all the bus bits creating same feedthrough ports on a block, the tool still scalarized those feedthrough port names and skip setting bus attribute on those new created feedthrough ports.

This option affects commands `place_pins` and `push_down_objects`.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [place_pins](#)
- [push_down_objects](#)

plan.pins.strict_alignment

Data Types

Boolean

Default "false"

Description

Enable this app-option to instruct `place_pins` to strictly align all the connected pins during pin placement.

It is recommended to use this app-option only when there is scope for pins to be placed perfectly aligned with each other. `place_pins` would ignore any pin/routing blockages, existing fixed pins or pre-routes that are present at the locations where the pins must be placed in order to be strictly aligned. Therefore, this app-option has to be used with caution and only when the strict alignment would not lead to any pin QoR degradation or DRC violations.

Note: that With the app-option enabled, `place_pins` will not align the pins which have user specified pin constraints applied on them that don't allow strict alignment to happen.

When this app option is specified, the `place_pins` command will consider only the pins that can be aligned with their connected pins in the given floorplan setup and assigns them locations which result in perfect alignment. The connected pins are the pins that directly connected to the pin being placed or the pins connected through feedthroughs connections. The pin placement will ignore the tracks and place the pins strictly align with other placed connected pins on same layer.

This option affects the *place_pins* command.

See Also

- [get_app_option_value](#)
- [get_app_options](#)

p

- [place_pins](#)
- [set_individual_pin_constraints](#)
- [read_pin_constraints](#)
- [report_app_options](#)
- [set_app_options](#)

plan.pins.synthesize_abutted_pins

Data Types

Boolean

Default "false"

Description

Specify whether the *place_pins* or *push_down_objects* commands create abutted pins for block pins that do not connect to any other pins, or for the unused feedthrough pins on MIB blocks. Pin synthesis is only performed for pins on the abutted block edges. Pin synthesis is performed as a post-processing step to normal *place_pins* or *push_down_objects* operation, though it is done under the hood of these commands. No pin legalization is performed during pin synthesis, Warnings are issued if the synthesized pin is getting created violating blockages. For the pins that have power/ground port type, the tool will skip synthesize the abutted pins for them. In addition, if the pins connecting to other pins/ports in the design that cannot be connected through existing feedthrough pins, the tool will skip synthesize abutted pins for them.

When this app option is specified, the *place_pins* command creates a new pin on neighboring abutted block and connects the new pin to the original single pin. The pin is placed such that there is no single pin violation in fully abutted design.

Synthesize abutted pins apply to top level ports and hard macro pins as well. The tool will treat the hard macro pins same as other block pins and create synthesized pins on the abutted block edges. To create synthesized pins on the abutted child blocks for top level ports, user needs to use *place_pins -ports* and pass the selected top level ports for abutted pin synthesis. Or user can use *place_pins -self* to place all the terminals while create synthesized pins on abutted child blocks . If the top level terminals already placed and user doesn't want to move those top level terminals, user can set fixed status on the top level terminals.

For block pins that connect to a tie high or tie low net, *place_pins* and *push_down_objects* first breaks the top-level connections. Next, *place_pins* and *push_down_objects* create a new pin on neighboring abutted block for each pin and connects those two pins together to eliminate any single pin violation in a abutted design. After running *place_pins* and

p

push_down_objects in a abutted design at the top level, the original pins that connected to a tie low or tie high net are disconnected from the tie low or tie high net and are connected to new neighboring block pins. When the original pin connects to a tie low net inside a neighboring child block, the new pin will be connected to tie low net. If the original pin connects to tie high net inside the neighboring child block, the new pin will be connected to tie high net.

The abutted pin synthesizer is done under the hood of *place_pins* after it places pins as it would do without this option set, and that the tool will not do additional legalization for synthesized pins, they will be placed at the locations where single pin violations need to be resolved.

The widths of the synthesized abutted pins are the same as the pins they abut to. The lengths of the synthesized abutted pins should follow the same rules as in command *place_pins*.

By default, this option is false.

This option affects the *place_pins* and *push_down_objects* commands.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [place_pins](#)
- [set_individual_pin_constraints](#)
- [read_pin_constraints](#)
- [push_down_objects](#)
- [report_app_options](#)
- [set_app_options](#)

plan.pins.synthesize_abutted_pins_name_tag

Data Types

string

Default ""

Description

Specifies the naming suffix for newly created synthesizer abutted pins, nets in *place_pins*. When this option is specified, the tool uses *synthesize_abutted_pins_name_tag* as a

p

suffix for newly created synthesize abutted pin and nets. In particular, the new name is "net_name__name_tag_#", where "name_tag" is the string specified by this option.

By default, this option is set to ""(empty, synthesize abutted pin use same name)

This option affects the *place_pins* and *push_down_objects* commands.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [place_pins](#)
- [push_down_objects](#)

plan.pins.synthesize_abutted_pins_name_tag_style

Data Types

string

Default "suffix"

Description

Specifies the naming style is suffix or prefix for newly created synthesize abutted pins and nets in *place_pins* commands.

When this option is specified as "suffix", the tool puts the synthesize abutted pin name tag as suffix for newly created synthesize abutted pins and nets. In particular, the new name is "net_name__name_tag_#", where "name_tag" is the string specified by "plan.pins.synthesize_abutted_pins_name_tag" and "#" is an index number.

When this option is specified as "prefix", the tool puts the new synthesize abutted pin name tag as prefix for newly created feedthrough ports and feedthrough nets. In particular, the new name is "name_tag_#__net_name", where "name_tag" is the string specified by "plan.pins.synthesize_abutted_pins_name_tag" application option, and "#" is an index number.

By default, this option is set to "suffix"

This option affects the *place_pins* and *push_down_objects* commands.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [place_pins](#)
- [push_down_objects](#)

plan.pins.treat_pg_as_shield

Data Types

Boolean

Default "false"

Description

Directs the *place_pins* and *check_pin_placement* commands to only honor or check the layers minimum spacing or NDR spacing rules (if they exist) for spacing between signal pins and PG pin shapes. Also directs both the commands to treat the PG pin shapes as stop or shield for spacing between signal pins. That means signal pin spacing is not checked/honored across PG pins. Note that spacing between signal pins is still honored as per usual and are unaffected by this option.

The default value is false, and with the default behavior the relevant user set spacing constraints on pins/blocks/nets etc. also apply to spacing between pins and any existing PG pins.

This option affects the *place_pins* (during pin creation) and *check_pin_placement* (during pin spacing checking) commands.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [place_pins](#)
- [set_individual_pin_constraints](#)
- [read_pin_constraints](#)
- [report_app_options](#)
- [set_app_options](#)

plan.place.auto_create_blockage_channel_heights

Specifies a pair of channel heights for creating blockages for horizontal channels.

Data Types

pair (typed length, typed length)

Default -1m, -1m

Description

This application option is used to give you more control on the blockages created by the `plan.place.auto_create_blockages` option. See the `plan.place.auto_create_blockages` man page for more information.

This option sets two lengths in a pair. These two values are denoted as `length1` and `length2` in this document. For horizontal channel, if the channel height is less than `length1`, a hard blockage is created. If the channel height is between `length1` and `length2`, depending on the value of `plan.place.auto_create_blockages`, a soft or partial blockage is created. In case a partial blockage is created, the blockage percentage is calculated using the following formula: $100 - 100 * (\text{channel width} - \text{length1}) / (\text{length2} - \text{length1})$. Therefore, bigger the channel width, smaller the blockage percentage will be.

The default is (-1m,-1m). In this case, the tool sets `length1` as 0, and determines a good `length2` for the partial or soft blockages to be created; no hard blockage is created. You must specify a unit for the values; for example, for micrometer (or microns), specify `um`. However, for negative values, the units do not matter because the value is invalid; that is, it is equivalent to this option not being set. For negative values, the tool determines the best possible width for the placement blockage.

Be careful while setting this option as it takes a typed length. The default length type is "m" which is millimeters, so "10" means 10 millimeters. Most likely you might want to set a length of "10 um", which is 10 microns.

Application options are set using the `set_app_options` command, and they might be specified globally or with respect to a design. Use `report_app_options` to show the value of application options.

See Also

- [create_placement](#)
- [derive_placement_blockages](#)
- [get_app_option_value](#)
- [get_placement_blockages](#)

p

- [report_app_options](#)
- [set_app_options](#)

plan.place.auto_create_blockage_channel_widths

Specifies a pair of channel widths for creating blockages for vertical channels.

Data Types

pair (typed length, typed length)

Default -1m, -1m

Description

This application option is used to give you more control on the blockages created by the `plan.place.auto_create_blockages` option. See the `plan.place.auto_create_blockages` man page for more information.

This option sets two lengths in a pair. These two values are denoted as `length1` and `length2` in this document. For vertical channel, if the channel width is less than `length1`, a hard blockage is created. If the channel width is between `length1` and `length2`, depending on the value of `plan.place.auto_create_blockages`, a soft or partial blockage is created. In case a partial blockage is created, the blockage percentage is calculated using the following formula: $100 - 100 * (\text{channel width} - \text{length1}) / (\text{length2} - \text{length1})$. Therefore, bigger the channel width, smaller the blockage percentage will be.

The default is (-1m,-1m). In this case, the tool sets `length1` as 0, and determines a good `length2` for the partial or soft blockages to be created; no hard blockage is created in this case. You must specify a unit for the values; for example, for micrometer (or microns), specify `um`. However, for negative values, the units do not matter because the value is invalid; that is, it is equivalent to this option not being set. For negative values, the tool determines the best possible width for the placement blockage.

Be careful while setting this option as it takes a typed length. The default length type is "m" which is millimeters, so "10" means 10 millimeters. Most likely you might want to set a length of "10 um", which is 10 microns.

Application options are set using the `set_app_options` command, and they might be specified globally or with respect to a design. Use `report_app_options` to show the value of application options.

See Also

- [create_placement](#)
- [derive_placement_blockages](#)

p

- [get_app_option_value](#)
- [get_placement_blockages](#)
- [report_app_options](#)
- [set_app_options](#)

plan.place.auto_create_blockages

Specifies whether to create auto-derived soft and/or partial placement blockages.

Data Types

Enum

Default "auto"

Description

This app option controls whether to create placement blockages in channels between hard and soft objects during *create_placement -floorplan*. Hard objects include hard macros, hard keepout margins, block boundaries, VA boundaries, and hard placement blockages. Soft objects include soft placement blockages and soft keepout margins.

For greater control on these blockages see the man page of following app options. *plan.place.auto_create_blockage_channel_widths*, *plan.place.auto_create_blockage_channel_heights*,

These app options will each specify a pair of values. One is for vertical channel, and one is for horizontal channels. For example, app option *plan.place.auto_create_blockage_channel_widths* will specify (length1,length2) for vertical channels. If the width of a vertical channel is less than length1, a hard placement blockage will be created; if the width of a vertical channel is between length1 and length2, a soft or partial blockage will be created, depending on the setting of *plan.place.auto_create_blockages*. The app options *plan.place.auto_create_blockage_channel_heights* has same behavior.

Following paragraph will not distinguish vertical and horizontal channels. Also, since the code will always create hard placement blockages for channels whose width is less than length1, we will only discuss behavior of soft and partial blockages.

The app option *plan.place.auto_create_blockages* supports five values : auto, soft, partial, hybrid, and none. The default is "auto". "Auto" has different meaning for different macro styles.

p

- `on_edge` or `islands`. The tool will create soft placement blockage between all adjacent hard/soft objects.
- `freeform`. The tool will no create any blockages
- `hybrid`. The tool will create soft placement blockage for macros placed on edge, and will not create any blockage for macros placed with freeform style.

Other values of this option behave as following.

- `soft`. It will create soft blockages
- `partial`. It will create partial blockages
- `hybrid`. This option only works with hybrid macro placement style. It will create soft blockages for macros placed on edge, and create partial blockages for macros placed with freeform style.
- `none`. No blockages will be created.

These soft and partial placement blockages created have a special attribute, `"is_derived"`, so that they can be selected with the `get_placement_blockages -filter "is_derived==true"` command. The `derive_placement_blockages` command can also be used to create these blockages.

Note that automatic pin blockages, as controlled by `plan.macro.create_auto_pin_blockages`, can also create blockages with this attribute set.

It is important to have these blockages to control the amount of standard cells that get placed in the narrow channels between hard macros. Typically these blockages are added after design planning is finished and the block is ready to go to implementation.

For freeform macro placement, it is desired to place standard cells to the narrow channels between hard macros. But sometime user wants to control utilization of these channels. For example, user might want to insert power switch in these channels. To achieve this goal, user can use partial option to create partial placement blockages.

Note that the `"is_derived"` attribute will be removed if you move or change the blockage. This is because if you change a blockage, that means you have taken ownership of the blockage and do not want the tool to automatically delete it in the next run of `create_placement -floorplan` or `derive_placement_blockages`.

Application options are set using the `set_app_options` command, and they may be specified globally or with respect to a design. Use `report_app_options` to show the value of application options.

See Also

- [create_placement](#)
- [derive_placement_blockages](#)
- [get_app_option_value](#)
- [get_placement_blockages](#)
- [report_app_options](#)
- [set_app_options](#)

plan.place.auto_create_blockages_enable_default_va

Specifies whether to create blockages at the top level voltage area the same way as in the other voltage areas during macro placement.

Data Types

Boolean

Default false

Description

This app option controls whether to create blockages at the top level voltage area the same way as in the other voltage areas during macro placement, i.e. command *create_placement -floorplan*; otherwise, whether or not the blockages will be inserted is determined in a more conservative manner. Note that the same blockage creation algorithm is used in *derive_placement_blockages* command. Frequently, the top level voltage area (DEFAULT_VA) has narrow channels that should not be blocked, so the option currently defaults to false. If that is not the case in the design, you can set it to true.

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design. Use *report_app_options* to show the value of application options.

See Also

- [create_placement](#)
- [derive_placement_blockages](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

plan.place.auto_create_hard_keepout

Specifies whether to create hard keepouts during hard macro placement.

Data Types

Boolean

Default true

Description

This application option controls whether the tool automatically creates hard keepout margins for hard macros during *create_placement -floorplan*. If true, then a default hard_macro keepout margin is created on all hard macros that do not have a user set hard keepout. If false, then no default hard keepout is created (that is, it is zero). The size of the default hard keepout created is related to the track size and layer minimal spacing.

The purpose of hard keepout is to prevent standard cells from getting too close to macros.

Application options are set using the *set_app_options* command, and they can be specified globally or with respect to a design. This application option is design-specific. Use *report_app_options* to show the value of application options.

See Also

- [create_placement](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

plan.place.auto_generate_blockages

Specifies whether to create auto-derived soft placement blockages.

Data Types

Boolean

Default true

Description

This option is deprecated and scheduled for removal in a future release. Please use *plan.place.auto_create_blockages* instead.

p

This app option controls whether to create soft placement blockages in channels between hard and soft objects during *create_placement -floorplan*. Hard objects include hard macros, hard keepout margins, block boundaries, VA boundaries, and hard placement blockages. Soft objects include soft placement blockages and soft keepout margins. If true, the tool creates a soft placement blockage between all adjacent hard/soft objects. These placement blockages have a special attribute, "is_derived", so that they can be selected with the *get_placement_blockages -filter "is_derived==true"* command. The *derive_placement_blockages* command can also be used to create these blockages.

It is important to have these soft blockages so that standard cells do not get placed in the narrow channels between hard macros. Typically these blockages are added after design planning is finished and the block is ready to go to implementation.

Note that the "is_derived" attribute will be removed if you move or change the blockage. This is because if you change a blockage, that means you have taken ownership of the blockage and do not want the tool to automatically delete it in the next run of *create_placement -floorplan* or *derive_placement_blockages*.

For greater control on these blockages see the man page for the *plan.place.auto_generate_hard_blockage_channel_width* and *plan.place.auto_generate_soft_blockage_channel_width* app options.

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design. Use *report_app_options* to show the value of application options.

See Also

- [create_placement](#)
- [derive_placement_blockages](#)
- [get_app_option_value](#)
- [get_placement_blockages](#)
- [report_app_options](#)
- [set_app_options](#)

plan.place.auto_generate_blockages_pad_rectilinear

Specifies whether to put blockages in notches of slightly rectilinear macros.

Data Types

Boolean

Default false

p

Description

This is a control for when creating automatically generated blockages. To understand what are automatically generated blockages look at *plan.place.auto_generate_blockages*.

This controls whether or not to create blockages in the notches of slightly rectilinear macros. This will cause the rectilinear macro to be seen by placer as its rectangular bounding box.

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design. Use *report_app_options* to show the value of application options.

See Also

- [create_placement](#)
- [derive_placement_blockages](#)
- [get_placement_blockages](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

plan.place.auto_generate_blockages_smooth_rectilinear

Specifies whether to consider slightly rectilinear macros as rectangular.

Data Types

Boolean

Default true

Description

This is a control for when creating automatically generated blockages. To understand what are automatically generated blockages look at *plan.place.auto_generate_blockages*.

This controls whether slightly rectilinear macros are considered to be the shape of their rectangular bounding box when determining blockages. Considering them as their bounding box generally leads to a better set of placement blockages so that swimming pools are not created. However sometimes it is better to not do this as you get more blockages than you may expect.

p

It is recommended to use the default setting unless you see something wrong with auto generated blockages around slightly rectilinear macros, in which case you can change the setting to see if result is better.

Application options are set using the `set_app_options` command, and they may be specified globally or with respect to a design. Use `report_app_options` to show the value of application options.

See Also

- [create_placement](#)
- [derive_placement_blockages](#)
- [get_placement_blockages](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

plan.place.auto_generate_hard_blockage_channel_width

Specifies the maximum channel width for creating a hard blockage.

Data Types

pair(typed length, typed length)

Default -1m, -1m

Description

This app options is deprecated. Please use app option *plan.place.auto_create_blockage_channel_widths*, *plan.place.auto_create_blockage_channel_heights*.

plan.place.auto_generate_partial_blockage_horizontal_channel_width

Specifies the minimum and maximum channel width for creating a partial horizontal blockage.

Data Types

pair(typed length, typed length)

Default -1m, -1m

p

Description

This app options is deprecated. Please use app option

plan.place.auto_create_blockage_channel_widths,

plan.place.auto_create_blockage_channel_heights.

plan.place.auto_generate_partial_blockage_vertical_channel_width

Specifies the minimum and maximum channel width for creating a vertical partial blockage.

Data Types

pair(typed length, typed length)

Default -1m, -1m

Description

This app options is deprecated. Please use app option

plan.place.auto_create_blockage_channel_widths,

plan.place.auto_create_blockage_channel_heights.

plan.place.auto_generate_soft_blockage_channel_width

Specifies the maximum channel width for creating a soft blockage.

Data Types

pair(typed length, typed length)

Default -1m, -1m

Description

This app options is deprecated. Please use app option

plan.place.auto_create_blockage_channel_widths,

plan.place.auto_create_blockage_channel_heights.

plan.place.auto_max_density

Specifies whether to automatically use max_density during standard cell placement.

Data Types

Boolean

p

Default false**Description**

This controls whether to allow clumping of standard cells during placement of a block that has subblocks. Typically when the utilization is very low and for better QoR it is needed to allow standard cells to clump (ie not spread out evenly).

Another control for clumping of standard cells is the app option `place.max_density`. The `plan.place.auto_max_density` app option essentially sets the `place.max_density` app option to 0.7 automatically when it thinks utilization is low. However if you set the `place.max_density` app option then that setting will override this setting.

The default value of this parametre is false, in which case the placer will use setting of `place.coarse.auto_density_control`.

Application options are set using the `set_app_options` command, and they may be specified globally or with respect to a design. Use `report_app_options` to show the value of application options.

See Also

- [create_placement](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

plan.place.congestion_driven_mode

Specifies the type of congestion reduction to perform during floorplan placement.

Data Types

Enum

Default "macro"**Description**

The `create_placement -floorplan -congestion` command can reduce congestion over and between the macros, and reduce congestion in the standard cell area.

This option controls where congestion reduction is applied. Possible values are "macro", "std_cell", and "both". The default value "macro" enables the macro congestion reduction but disables the standard cell congestion reduction. The value "std_cell" will do the reverse, while the value "both" will enable both strategies. To disable both strategies,

p

remove the "-congestion" command option, in which case the value of this option is irrelevant.

Note that run time penalty for enabling the standard cell congestion reduction will generally be much larger than the penalty for enabling the macro congestion reduction.

This application option must be set on a block using the `set_app_options` command. This means that if the design has physical hierarchy, and you want the standard cell congestion reduction to be enabled for some or all of the sub-blocks, the app option has to be set explicitly for these sub-blocks. If the same setting should be used for all blocks, you could also use the `-as_user_default` option of `set_app_options`.

Examples

The flowing example enables both the macro and standard cell congestion reduction strategies.

```
prompt> set_app_options -list {plan.place.congestion_driven_mode both}
```

See Also

- [create_placement](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

plan.place.default_keepout

Specifies whether to use default keepouts during hard macro placement.

Data Types

Boolean

Default true

Description

This controls what hard_macro keepout margin is considered on hard macros during `create_placement -floorplan`. If true, then a default hard_macro keepout margin is created on all hard macros which do not have a user set hard_macro keepout. If false, then no default hard_macro keepout is created (i.e., it is zero). The default hard_macro keepout created is pin count based so sides with more pins will have a larger keepout.

It is important to have hard_macro keepouts to ensure hard macros are placed far enough from each other so that routing can access the hard macro pins.

p

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design. Use *report_app_options* to show the value of application options.

See Also

- [create_placement](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

plan.place.hierarchy_by_name

Specifies whether to recreate logical hierarchy from instance names during macro placement.

Data Types

Boolean

Default false

Description

This app option controls whether to recreate logical hierarchy using names during macro placement, i.e. command *create_placement -floorplan*. This option is intended for designs that were flattened, meaning their logical hierarchy has been at least partially removed; on designs with logical hierarchy intact, this option will likely not make a difference.

Using this option on a flattened design is likely to speed up the global macro placement phase of the command, and can also improve the quality of the result. Some increase in memory usage is expected.

Note that if this option is set to true, *plan.place.trace_mode* option can not be set to "dfa".

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design. Use *report_app_options* to show the value of application options.

See Also

- [create_placement](#)
- [get_app_option_value](#)

- [report_app_options](#)
- [set_app_options](#)

plan.place.max_utilization_for_auto_blockages

Specifies maximum utilization to allow partial blockages.

Data Types

float

Default 0.95

Description

Specifies maximal utilization that the automatically inserted blockages can cause during the execution of the command *create_placement -floorplan*. If the blockages would cause utilization higher than this value, the command does not insert the blockages and instead issues warning *DPP-236*.

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design. Use *report_app_options* to show the value of application options. This application option is design specific.

See Also

- [create_placement](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

plan.place.poly_rule

Specifies the narrow placeable site threshold used to check poly rule.

Data Types

integer

Default 0

Description

Before the execution of the command *report_placement -poly_rule*, user must specify a positive narrow placeable site threshold. If user use the default value 0 as the threshold, the command *report_placement -poly_rule* will report an error.

p

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design. Use *report_app_options* to show the value of application options. This application option is design specific.

See Also

- [report_placement](#)

plan.place.poly_rule_vertical_edge_expanding

Specifies the offset value of vertical edge expanding or shrinking used to check poly rule. The unit is site count.

Data Types*float***Default** 0**Description**

Before running the *report_placement -poly_rule* command, you can specify an offset value for expanding or shrinking the vertical edge adjustment on two sides, besides the placeable site threshold. Use positive values to expand the adjustment, use negative values to shrink the adjustment. By default, the adjustment value is 0 and no vertical edge adjustment is applied during poly rule checking. If the value is 0.5, it means expand 0.5 site width at left and right side of the placeable area.

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design. Use *report_app_options* to show the value of application options. This application option is design specific.

See Also

- [report_placement](#)

plan.place.timing_driven_mode

Specifies the type of timing reduction to perform during floorplan placement.

Data Types*Enum***Default** "std_cell"

p

Description

This option controls the behavior of *create_placement -floorplan -timing_driven* command. The default value "std_cell" will apply timing driven only during standard cell placement, "macro" will do it just for macro placement, and the value "both" will enable both strategies. To disable both strategies, remove the "-timing_driven" command option, in which case the value of this option is irrelevant. Note that this option only applies if the value of *plan.macro.style* is "on_edge", which is the default.

Examples

The following example enables both the macro and standard cell timing driven placement.

```
prompt> set_app_options -list {plan.place.timing_driven_mode both}
```

See Also

- [create_placement](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

plan.place.trace_mode

Control tracing method for connectivity extraction in macro placement.

Data Types

string

Default dfa

Description

This option specifies which tracing method to use for connectivity extraction during macro placement. The connectivity extraction is either performed by *create_placement -floorplan*, or by *create_abstract_placement* in case of the high capacity flow.

There are two possible values for this app option, "normal" or "dfa". The value "normal" picks a tracing method that extracts direct connectivity and connectivity via buffers and inverters. The default value "dfa" enables an enhanced tracing method that traces connectivity through combinatorial standard cells. This is similar to what is available in the data flow flylines (*compute_dff_connections*). Note that this analysis requires more runtime.

p

See Also

- [create_placement](#)
- [create_abstract](#)
- [compute_dff_connections](#)

plan.pna.ignore_objects_below_layer

Specifies the layer name below which PNA will not consider.

Data Types

String

Default ""

Description

Specifies the layer name below which PNA will not consider. This is helpful if the design is too large and user only cares about IR-drop on top level meshes. For example, if a design has tens of millions of stapling vias between M1 and M2, PNA may be slow to calculate all the wires/vias. If set this app option to M2, PNA will ignore stapling vias and rails below M2, but still gives a fairly accurate IR-drop estimation of upper level PG mesh. By default, PNA will consider all wires/vias.

This application option is set using the *set_app_options* command, and may be specified globally. To determine the current value of this application option on a specific design, use *get_app_option_value -name opt_name*. Use *report_app_options* to show the value of multiple application options.

See Also

- [analyze_power_plan](#)
- [set_virtual_pad](#)

plan.pna.power_switch_resistance

Specify power switch resistance value for the specific power switch lib cell.

Data Types

list

Default ""

p

Description

This application option is used to specify the power switch resistance value for the specific power switch lib cell. And the power switch resistance value will be used to perform PNA.

The format to specify the power switch resistance value for a power switch lib cell is

```
{{power_switch_lib_cell_name1 power_switch_resistance_value1} ...}
```

It will not check if the specified power switch lib cell can be found.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)

plan.shadow.exclude_non_hierarchical_objects

Data Types

boolean

Default "false"

Description

Exclude non hierarchical objects during *write_shadow_eco*.

The shadow ECO scripts created by *write_shadow_eco* contain commands to create and connect the feedthroughs. By default, these commands may refer to some standard cell pins. For example, disconnect standard cell pin from an original net and connect it a feedthrough net.

If user does not want to touch the original standard cell pins in the shadow ECO scripts, this option can be set to *true*. Then all the commands in the shadow ECO scripts that touches the original standard cell pins are commented out. User can examine those commented out commands and decide if they should keep commented out or not.

The default value is false.

See Also

- [write_shadow_eco](#)

plan.shadow.exclude_non_shadow_buffers

Data Types

boolean

Default "false"

Description

Exclude non shadow buffers during *write_shadow_eco*.

During *write_shadow_eco*, the tool automatically detects buffers along the feedthrough network. The following kind of buffers can be identified:

- buffers inserted by *add_buffer_on_route* or *add_buffers*
- buffers with *pushed_down* or *copied_down* shadow status
- buffers that can be 100% determined as being inserted after feedthrough creation

The identified buffers are regarded as being inserted after feedthrough creation or during *push_down_objects* and are written out for re-creation by command *create_cells* as ECO cells.

By setting this option to true, the buffer identification feature will be disabled and all the non-shadow buffers are regarded as original in the netlist and are not included in the output shadow ECO scripts. However, the buffers with shadow status (and all the standard cells with shadow status) are still written out in the shadow ECO scripts.

The default value is false.

See Also

- [write_shadow_eco](#)

plan.shaping.import_tcl_shaping_constraints

Specify whether *shape_blocks* imports constraints from Tcl.

Data Types

Boolean

Default false

Description

Traditionally, *shape_blocks* has offered a *-file* option to import shaping constraints via a file using a text-based constraint format. When this *app_option* is set to true, *shape_blocks*

will import constraints that are created via Tcl using *create_shaping_constraint*, *create_shaping_channel* and *create_group*.

See Also

- [shape_blocks](#)
- [create_group](#)
- [create_shaping_channel](#)
- [create_shaping_constraint](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

plan.shaping.new_mib_abutted_flow

Specify whether *shape_blocks* to adopt new MIB abutted flow during the abutted MIB block shaping.

Data Types

Boolean

Default false

Description

Adopt new MIB abutted flow during the abutted MIB block shaping. When this app_option is set to true, *shape_blocks* will cluster all MIB blocks (with same ref_block) together as one pseudo group during the abutted block shaping.

See Also

- [shape_blocks](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

plan.shaping.report_import_constraints

Print overview for shaping constraints imported by *shape_blocks*.

p

Data Types

Enum

Default "off"

Description

If this app_option is set to "log", *shape_blocks* will print an overview of the imported shaping constraints to the log (---IMPORTED CONSTRAINTS REPORT---). If this app_option is set to "file", *shape_blocks* will print the overview of the imported shaping constraints to a file named *shape_blocks_imported_constraints.txt*. If that file already exists, a unique number will be appended to the file name to make it unique. You can also set the app_option to "both", in which case the overview is printed to the log and to a file.

To get a report of the shaping constraints defined through *create_shaping_constraint*, *create_shaping_channel* and *create_group*, use the command *report_shaping_constraints* instead.

See Also

- [shape_blocks](#)
- [report_shaping_constraints](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

plan.shaping.snap_to_boundary_grid

Specifies that the block grid will be considered as the boundary grid.

Data Types

Boolean

Default false

Description

This option allows user to use the block grid (refer to *create_grid*) as the boundary grid so that *shape_blocks* will create the shape with its vertexes are on the grid. To achieve this, the fixed snapping point of the block grid has to be at (0, 0).

See Also

- [shape_blocks](#)
- [create_grid](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

power.annotate_repeater_tree

Determines how the activity is annotated in a repeater tree

Data Types

string

Default specified

Description

This app-option allows you to annotate activity at the root of a repeater tree if there is activity annotation for points inside the repeater tree. It can take three values - *root*, *specified*, and *both*. When its value is *specified*, annotation is done at the point for which activity is specified. When it is set to *root*, activity is annotated on the root of the repeater chain for this specified point. When value is *both*, both the specified point as well as the root are annotated with the specified activity. The default is specified.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)
- [read_saif](#)
- [set_switching_activity](#)

power.built_in_name_mapping_mode

Enable built in name mappings during read_saif.

Data Types

Integer

Default 0

Description

When doing `read_saif`, to associate a SAIF file name with a netlist object, the tool will always look for the exact name match. If exact name matching fails then tool will use built in name mapping rules as described below:

value = 0: Exact name matching.

value = 1: All available name mapping rules will be used (will be runtime intensive).

value = 2: '.#' to ['#]', '_#' to ['#].', and '['#]' to '_#_'.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)
- [read_saif](#)

power.clock_network

Specifies what objects contribute to the `clock_network` power in `report_power` reports.

Data Types

string

Default `cell_based`

Description

The app option specifies what nets and pins contribute to the power of the *clock_network* power group. Valid values are *cell_based* and *pin_based*.

If the value of the app option is *cell_based*, then cells are classified as clock cells or non-clock cells based on the fact whether they have at least one clock output or not. If a cell is classified as a clock cell, then all its internal, switching and leakage power is counted as `clock_network` power.

p

If the value is *pin_based*, then pins are classified as clock pins or non-clock pins based on the fact whether they are marked as 'is_clock_used_as_clock'. All internal and switching power of clock pins is counted as clock_network power. The internal and switching power of non-clock pins is accounted in the power_group where the cell would be if it was not in the clock network. For leakage power, the cell based classification is used. So if a cell has at least one clock output pin, then its leakage is counted as clock_network leakage. The exception to this is an ETM (Extracted Timing Model) cell. For an ETM, if the cell based classification is clock network then it will be ignored in pin based mode. Instead the pin based classification will be used.

E.g. the power of an enable pin of a clock-gating AND gate is counted as 'combinational' power, because the AND gate would be a 'combinational' cell if it was not in the clock network. The power of the clock input pin and the output pin are counted as 'clock_network' power. The leakage of the AND gate is counted as 'clock_network' power because the AND gate has a clock output.

For the accounting of clock sink pin power, please see app option 'power.clock_network_include_clock_sink_pin_power'.

By default, the option value is 'cell_based'. For correlation with the 'report_clock_qor -type power' command please set the app option value to 'pin_based'.

See Also

- [report_power](#)
- [report_clock_qor](#)

power.clock_network_include_clock_gating_network

Data Types

Boolean

Default false

Description

This application option affects the categorization of power in report_power.

When the application option is set to true, discrete logic structure that functions as clock gating belongs to the clock_network power group.

Only the typical clock gating logic is considered a qualified clock gating network included in the clock network. The typical clock gating logic starts from the output of a level-sensitive latch driven by a specific clock. The clock gating logic possibly goes through some buffers, and returns to the specified clock network through one of the input pins of a cell. That input pin must be a clock check enabled pin, either inferred or manually

p

set. Therefore, the results can be affected by clock gating check related commands or application options. When the clock gating network is included in the clock network, the latch is regarded as part of the clock_network power group, but not the register power group.

Note that total power is not affected, just the distribution over categories.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)
- [report_power](#)

power.clock_network_include_clock_sink_pin_power

Data Types

string

Default register

Description

If this app option is set to a power group, report_power includes the internal power of all the clock pins of the cells for the specified power group into the clock_network power group. Only the internal power of the clock pins will be included in the clock_network power group, rest of the cell power which includes internal power of pins other than clock pins, leakage power, and the switching power will be included in the power group to which the cell belongs. When this option is 'off', all of the cell internal power is reported in the power group to which the cell belongs. When this option is 'all', internal power of all the clock pins for every cell will be included in the clock_network power group.

Valid power groups that can be specified for this app option are: "off | all | io_pad | memory | black_box | register | sequential | combinational | <user_power_groups>" User needs to specify a valid power group name. In case the user has specified a name for which there is no corresponding power group value, the report_power command will ignore that name.

See Also

- [get_app_option_value](#)
- [get_app_options](#)

- [report_app_options](#)
- [set_app_options](#)
- [report_power](#)

power.default_static_probability

Specifies the default static probability value.

Data Types

float

Default 0.5

Description

The `power.default_static_probability` app option determines the static probability value of nets that have not been annotated, simulated, propagated or derived from static timing analysis.

The value of the `power.default_static_probability` app option should be between 0.0 and 1.0, both inclusive. If the value of the `power.default_static_probability` app option is 0.0 or 1.0, the value of the `power.default_toggle_rate` app option should be 0.0. If the value of the `power.default_toggle_rate` app option is 0.0, the value of the `power.default_static_probability` should be either 0.0 or 1.0.

See Also

- [get_app_option_value](#)
- [get_switching_activity](#)
- [report_app_options](#)
- [set_app_options](#)
- [set_switching_activity](#)

power.default_toggle_rate

Specifies the default toggle rate value.

Data Types

float

Default 0.1

p

Description

The `power.default_toggle_rate` app option is used to determine the toggle rate value of nets that have not been annotated, simulated, propagated or derived from static timing analysis.

The value of the `power.default_toggle_rate` app option should be greater than or equal to 0.0. If the value of the `power.default_static_probability` app option is 0.0 or 1.0, the value of the `power.default_toggle_rate` app option should be 0.0. If the value of the `power.default_toggle_rate` app option is 0.0, then the value of the `power.default_static_probability` app option should be either 1.0 or 0.0.

See Also

- [get_app_option_value](#)
- [get_switching_activity](#)
- [report_app_options](#)
- [set_app_options](#)
- [set_switching_activity](#)

power.default_toggle_rate_reference_clock

Specifies whether to use the fastest or the related clock for determining default toggle rates.

Data Types

string

Default related

Description

The `power.default_toggle_rate_reference_clock` app option determines whether to use the fastest or the related clock for determining the default toggle rates.

If the toggle rate is not annotated, simulated, propagated or derived from static timing analysis, then it is determined by using the following formula:

$$tr = def_tr * fclk$$

where `fclk` is the frequency of the clock (fastest or related) and `def_tr` is the value of the `power.default_toggle_rate` app option.

Two values, fastest and related, are allowed for the value of the `power.default_toggle_rate_reference_clock` app option. If the value "fastest" is specified,

p

the fastest clock in the design is used. If the value "related" is specified, the clock that is used depends on which clock the object for which switching activity is required is associated with.

See Also

- [get_app_option_value](#)
- [get_switching_activity](#)
- [report_app_options](#)
- [set_app_options](#)
- [set_switching_activity](#)

power.enable_activity_persistency

Enables reading/writing of power analysis side files during open_block/save_block.

Data Types

string

Default lazy

Description

The app option determines whether 'open_block/save_block' will read/write power analysis side files or not. Possible values for this app option are: "off" | "lazy" | "on". Value "lazy" and "on" is exactly same. The app option will be made boolean in future release. Power analysis side files contains two files: activity_data.misc containing switching activity data and common_data.misc. common_data.misc contains data for commands: set_threshold_voltage_group_type, set_power_clock_scaling and set_power_derate.

If the app option value is 'off', power analysis side files will not be read/written during 'open_block/save_block'.

If the app option value is 'on' or 'lazy', power analysis side files will be read/written during 'open_block/save_block'.

By default, the app option value is 'lazy'.

Following points needs to be noted: 1) All the design blocks are saved during save_lib too, hence save_lib also writes the power analysis side files. 2) If a block already contains side files and block is not modified, then save_block copies the side files irrespective of the value of the app option. 3) All the app options are persistent, so the value of app option when block is opened depends on the value of the app option when block was saved. 4) The app option can only be set after the block is opened (and not after open_lib).

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)
- [set_switching_activity](#)
- [read_saif](#)
- [set_threshold_voltage_group_type](#)
- [set_power_clock_scaling](#)

power.enable_ignore_create_generated_clock

Data Types

boolean

Default false

Description

When set to true, the generated clock activity set by the `create_generated_clock` command is ignored. Instead, the propagated activity from the upstream master clock after getting propagated through the netlist is used. The generated clock activity is overridden by the propagated clock activity. If the generated Clock is defined at the output of the black box cell, by default the generated clock activity is used and the propagated activity is ignored.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)
- [report_power](#)
- [propagate_switching_activity](#)

power.enable_nldm_cap

Enables NLDM-based capacitance for power calculation.

Data Types

Boolean

Default false

Description

The power.enable_nldm_cap application option determines whether NLDM-based capacitance values are used for power calculations or not. When this option is set to "true", NLDM-based capacitance values are used in the power analysis.

This option is useful for designs where NLDM capacitance data is available and needs to be incorporated into the power calculation process.

See Also

- [get_app_option_value](#)
- [report_power](#)
- [report_app_options](#)
- [set_app_options](#)

power.enhanced_sdpd_power_calculation

Data Types

boolean

Default false

Description

If SAIF has partial SDPD data specified for a cell, then with this app option set to true, tool will do sdpd estimation for missing sdpd data during power calculation. If set to false, pins or paths with missing SDPD data will have their power reported as zero.

By default the app option is 'false'.

See Also

- [report_power_calculation](#)
- [report_power](#)

power.get_power_clock_scaling_single_objects_only

Determines whether 'get_power_clock_scaling' command has new behavior or not which are described later. By default, the option is set to false.

Data Types

bool

Default false

Description

If the app option is true, then both the options (scenario list and clock list) in get_power_clock_scaling command can accept only single objects. If either one or both the option lists have more than one object, appropriate warning (POW-030) will be issued to the user and the return status will be 0 indicating failure. Following cases describe the changed behavior for app option set to true.

Both scenario and clock object are specified in get_power_clock_scaling :- The scaling data, for the specified clock for the specified scenario, will be the output as a tcl list. For outputs see later Examples section.

Only scenario is specified :- The global scenario specific ratio value will be the output as a tcl list. This will not include the clock specific scaling data for the scenario.

Only clock object is specified :- The scaling data for the specified clock will be the output for the current scenario.

Nothing is specified :- The global scenario specific ratio value will be the output for the current scenario. This will not include the clock specific scaling data for the current scenario.

If the app option is set to false, then 'get_power_clock_scaling' behaves exactly like 'report_power_clock_scaling'. This is also shown in examples. By default, the option is 'false'.

Examples

```
prompt> set_power_clock_scaling {CLK CLK2 C_A} -period 2.6
prompt> set_power_clock_scaling -ratio 1
prompt> current_scenario scn1
prompt> set_power_clock_scaling -ratio 5
prompt> set_power_clock_scaling -period 3 {CLK1}
prompt> get_power_clock_scaling
```

```
-----
Clock Scaling Data
-----
```

```
Scenario:default, Clock name:CLK, Period:2.600000
Scenario:default, Clock name:CLK2, Period:2.600000
```

p

```

Scenario:default, Clock name:C_A, Period:2.600000
Scenario:scn1, Clock name:CLK1, Period:3.000000
-----
Scaling Ratios for Scenarios without Clocks
-----
Scenario:default, Ratio:1.000000
Scenario:scn1, Ratio:5.000000

prompt> set_app_options -list
        {power.get_power_clock_scaling_single_objects_only true}
prompt> current_scenario scn1
prompt> get_power_clock_scaling
prompt> {ratio 5.0}
prompt> get_power_clock_scaling {CLK1}
prompt> {period 3.0}
prompt> get_power_clock_scaling -scenarios default {CLK}
prompt> {period 2.6}
prompt> get_power_clock_scaling -scenarios default
prompt> {ratio 1.0}

```

See Also

- [get_power_clock_scaling](#)
- [report_power_clock_scaling](#)

power.get_switching_activity_format

Determines the output format of 'get_switching_activity' command.

Data Types

String

Default singleton

Description

The app option sets the format that the 'get_switching_activity' command uses to print the output. There are two supported formats: singleton and list.

If the app option is set to "singleton" then the format of the get_switching_activity looks as follows:

```
(mode = default, corner = default, probability = 0.8, toggle_rate =
0.233333, type = default)
```

If the app option is set to "list" then the format of the get_switching_activity looks as follows:

```
{"default" "I1" 0.233333 0.8 default}
```

By default, the option value is 'singleton'.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)
- [get_switching_activity](#)

power.handle_clock_network_power_as_ideal

Data Types

boolean

Default false

Description

When this app option is set to true, the tool will handle clock network power as ideal. It means that for ideal clock networks, the capacitance used to calculate internal power is zero and switching power is zero. Timer will determine if a clock network is ideal. This app option can only be used in pre-CTS stages. If enabled, it is the user's responsibility to set it to false before entering CTS stage to ensure it does not affect results of other stages.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)
- [report_power](#)
- [report_power_calculation](#)

power.idpp_use_user_map_only

Enables Indesign PrimePower flow to use only user specified mapping file.

p

Data Types

Boolean

Default false**Description**

In Indesign PrimePower flow, FC writes ptpx mapping file which is being read by PrimePower. If this app option is enabled then FC will not write the ptpx mapping file, PrimePower will read mapping file specified by the user.

See Also

- [update_indesign_activity](#)

power.ignore_scan_out_activity**Data Types***boolean***Default** false**Description**

If power.ignore_scan_out_activity app option is set to true, scan-only(DFT) pins will be excluded from the seq-pin column in both –driver and –essential options in report_activity. If power.ignore_scan_out_activity app option is set to false, scan-only(DFT) pins will be included in the seq-pin column.

By default the app option is 'false'.

See Also

- [report_activity](#)

power.ignore_unconnected_driver**Data Types***boolean***Default** true**Description**

When set to true, this app option ignores the unconnected drivers in 'report_activity -driver' report. To include unconnected drivers set this app option to false.

By default the app option is 'true'.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)
- [report_activity](#)

power.include_hard_macro_input_pin_annotation

Writes hard macro input pin mappings in ptpx saif map and does not reset input pin annotation when doing reset_switching_activity -non_essential. Disabling the app option will exclude only hard macro cells having attributes 'is_hard_macro=true' but 'is_memory_cell=false' from ptpx saif map and saif annotation.

Data Types

Boolean

Default true

Description

When this app option is 'true', hard macro input pin mappings will be written in the ptpx saif map. Also it will not reset hard macro input pin annotation when doing reset_switching_activity -non_essential. By default, this app option is true.

See Also

- [reset_switching_activity](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)
- [saif_map](#)

power.include_ideal_network_swcap

Data Types

boolean

Default false

Description

If the app option is specified as 'true' then all the sink terms are also included of an ideal net to compute the corresponding swcap power. By Default the app option is 'false'

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)
- [report_power](#)

power.include_memory_input_pin_annotation

Writes memory input pin mappings in ptpx saif map and does not reset input pin annotation when doing reset_switching_activity -non_essential.

Data Types

Boolean

Default true

Description

When this app option is 'true', memory input pin mappings will be written in the ptpx saif map but, it does not reset input pin annotation when doing reset_switching_activity -non_essential. If set to false, *saif_map -type ptpx -write_map* will not write any memory input pin mappings.

See Also

- [reset_switching_activity](#)
- [get_app_option_value](#)
- [report_app_options](#)

- [set_app_options](#)
- [saif_map](#)

power.include_primary_input_swcap

Data Types

boolean

Default false

Description

If the app option is specified as 'true' then all the primary input nets are included while computing swcap power of a design. By Default the app option is 'false'

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)
- [report_power](#)

power.indesign_primepower_filter_x

Ignores all transitions from and to X values in the gate level saif generated by PrimePower in Indesign PrimePower flow with CAPP enabled.

Data Types

boolean

Default False

Description

If this app_option is set to true & CAPP (Cycle Accurate Peak Power) flow is enabled, InDesign PrimePower flow will run with the PrimePower app_var "power_filter_x_transition_from_activity_file" turned on. With this app_var, all transitions from and to X values in the gate level saif generated by PrimePower are ignored. By default, this app_option is false.

See Also

- [get_app_option_value](#)
- [get_app_options](#)

power.indesign_primepower_use_capp

Enables CAPP flow in PrimePower through Indesign PrimePower flow.

Data Types

boolean

Default False

Description

When this option is enabled, PrimePower uses CAPP (Cycle Accurate Peak Power) flow for time based analysis else PrimePower uses C-CAPP(Concurrent Cycle Accurate Peak Power) flow. Executing C-CAPP will require "PP-Elite" license.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)

power.input_pin_probability_scaling

Specifies whether input pin probability scaling should be done or not.

Data Types

Boolean

Default false

Description

The app option determines whether input pin probability needs to be scaled or not. Input pin probability needs to be scaled when incomplete when-clause set is specified for the input pins. Incomplete when-clause set means that the specified when-clauses for the input pin do not cover all states. The default value for this app option is 'false'.

power.internal_mode

Determines whether 'report_power' command reports internal power or not. By default, 'report_power' reports internal power numbers.

Data Types

string

Default on

Description

The app option determines whether 'report_power' and 'report_power_calculation' commands report internal power or not. If the option is 'on', internal power is reported by 'report_power' and 'report_power_calculation' commands. If the option is 'off', internal power is not reported by 'report_power' and 'report_power_calculation' commands. By default, the option is 'on'.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)
- [report_power](#)
- [report_power_calculation](#)

power.leakage_mode

Determines whether 'report_power' command reports leakage power or not. The option also specifies strategy for leakage power calculation.

Data Types

string

Default state

Description

The app option determines whether 'report_power' and 'report_power_calculation' commands report leakage power or not. If the app option value is not 'off', it specifies the strategy for leakage power calculation. If the option value is 'off', 'report_power' will not report leakage power. If the option value is 'average', 'report_power' reports leakage power

p

and average leakage data will be used during leakage power calculation. If the option value is 'state', 'report_power' reports leakage power and state based leakage data will be used during leakage power calculation. If the option value is 'unconditional', 'report_power' reports leakage power and unconditional leakage data will be used during leakage power calculation. By default, the option value is 'state'.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)
- [report_power](#)
- [report_power_calculation](#)

power.power_annotation_persistency

Enables the writing of annotated power during save_block.

Data Types

Boolean

Default false

Description

The app option enables whether during 'save_block' the annotated power is written to the power analysis side files or not. Possible values for this app option are "false" or "true".

If the app option value is "false", the annotated power will not be written to the power analysis side file called "activity_data.misc". If the app option value is "true", the annotated power will be written to the "activity_data.misc" file.

During reading, annotated power will be read regardless of the app option, if it is existing in the power analysis side file.

By default, the app option value is 'false'.

Following points need to be noted: if the app option power.enable_activity_persistency is set to 'off', no power side files are written and therefore the annotated power is also not saved, regardless of the setting of this app option power.power_annotation_persistency.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)

power.propagation_effort

Specifies the effort level to be used for the propagation of switching activity during power calculation by 'report_power' command.

Data Types

string

Default medium

Description

Specifies the effort level to be used during the propagation of switching activity. On higher effort levels, the tool uses more randomly generated switching activities to propagate the switching activity. The default value is *medium*.

Specifying *low* effort results in the fastest runtime and the lowest accuracy of power estimates. Specifying *medium* or *high* effort results in a longer run that has increased levels of accuracy. The analysis effort is considered only during the estimation of switching activity information, and, therefore, has an effect only when the design is not fully annotated with switching activity.

Specifying *off* means no propagation of switching activity will be performed. This gives only meaningful results if the design is fully annotated with switching activity, or if you are only interested in leakage power and the leakage strategy is *unconditional* or *average* in *power.leakage_mode*.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)
- [report_power](#)

power.ptpx_saif_map_get_all_driver_mappings

Writes multiple mappings for an object in ptpx saif map.

Data Types

Boolean

Default true

Description

saif_map mapping gives a relation between a gate object to corresponding rtl object. A gate object can correspond to multiple rtl objects(in different hierarchies). By default, saif_map writes only one mapping for each gate object.

When this app option is true, saif_map will be writing mappings for all rtl objects to the gate object.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)
- [saif_map](#)

power.ptpx_saif_map_include_register_output_pin

Enables the inclusion of every register output pin in ptpx saif map.

Data Types

Boolean

Default false

Description

By default, *saif_map -write_map* writes only the objects where the name mappings have changed. If you want to write out all the registers from the design in the saif map, the option needs to be set to true. The default is false. This option is valid only when you have not specified a saif file using the *-saif_file* option with *saif_map -write_map*.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)
- [saif_map](#)

power.report_user_power_groups

Specifies whether 'report_power' considers user power groups to be exclusive or inclusive while generating the summary report of the design.

Data Types

string

Default exclusive

Description

The app option determines whether 'report_power' considers user power groups to be exclusive or inclusive while generating the summary report of the design. If app option value is 'exclusive', then each instantiated cell will be in exactly one power group, it can be a default power group or a user specified power group. As a consequence, since a cell is in exactly one power group, the total power is simply the sum of the power over all user and default power groups. If app option value is 'inclusive', then each instantiated cell can be in one or more power groups. The power contribution of a cell will be added to its default power group and also to all the user specified power groups to which the cell belongs. As power of every cell is present in its respective default power group, total power is sum of the power over all the default power groups. By default, the app option value is 'exclusive'.

See Also

- [get_app_options](#)
- [set_app_options](#)
- [report_power](#)
- [report_power_groups](#)
- [get_power_group](#)
- [get_power_group_objects](#)

power.saif_glitch_handling

Allows the transport and inertial glitch to be annotated from saif file.

Data Types

boolean

Default true

Description

With this option turned on, read_saif takes into consideration the transport and inertial glitch from the saif file. By default, the option is true.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)

power.saif_map_disable_rtl_name

Disables object_rtl_name attribute when choosing rtl name for registers and primary ports at the time of 'saif_map -start'.

Data Types

Boolean

Default false

Description

By default, rtl name for registers and primary ports is based on object_rtl_name attribute. If set to true, object name at the time of 'saif_map -start' is taken as the rtl name.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)

p

- [set_app_options](#)
- [saif_map](#)

power.scale_dynamic_power_at_power_off

Indicates if the dynamic power is scaled according to the static probability of the corresponding power supply net. When this application option is set to *true*, the dynamic power is scaled by the power-on probability of the power supply net.

Data Types

Boolean

Default false

Description

The statistical activity information can be set using the *set_supply_net_probability* command or from the default setting (unswitched supply net, probability is 1.0). The tool is built with power management awareness and can reflect power saving technology used in the design in the calculated power results. If the power supply net is switched off during simulation, the tool can report the correct dynamic and static (leakage) power based on the statistical activity information.

If the given statistical activity information does not contain any toggle happened at the power-off state, dynamic power does not need to be scaled by the power-on probability. However, if the input activity information includes toggles happened at the power-off state, both dynamic and leakage power is scaled. Under such situation, you need to set the *power.scale_dynamic_power_at_power_off* application option to *true*, so that the tool applies the scaling to dynamic power as well.

See Also

- [set_supply_net_probability](#)
- [reset_supply_net_probability](#)

power.scale_leakage_power_at_power_off

Indicates if the leakage power is scaled according to the static probability of the corresponding power supply net. When this application option is set to *true*, the leakage power is scaled by the power-on probability of the power supply net.

Data Types

Boolean

p

Default true**Description**

The statistical activity information can be set using the *set_supply_net_probability* command or from default setting (unswitched supply net, probability is 1.0). The tool is built with power management awareness and can reflect power saving technology used in the design in the calculated power results. If the power supply net is switched off during simulation, the tool can report the correct dynamic and static (leakage) power based on the statistical activity information.

By default, leakage power is scaled. However, if you do not want to scale the leakage power, you need to set the *power.scale_leakage_power_at_power_off* application option to *false*, so that the tool does not apply the scaling to leakage power.

See Also

- [set_supply_net_probability](#)
- [reset_supply_net_probability](#)

power.swcap_mode

Determines whether 'report_power' command reports swcap power or not. By default, 'report_power' reports swcap power numbers.

Data Types*string***Default** on**Description**

The app option determines whether 'report_power' and 'report_power_calculation' commands report swcap power or not. If the option is 'on', swcap power is reported by 'report_power' and 'report_power_calculation' commands. If the option is 'off', swcap power is not reported by 'report_power' and 'report_power_calculation' commands. By default, the option is 'on'.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)

- [report_power](#)
- [report_power_calculation](#)

power.swcap_power_capacitance

Specifies the strategy for calculating the capacitive load used for switch cap power calculation.

Data Types

max | *average*

Default max

Description

The power.swcap_power_capacitance app option determines the strategy for calculating the capacitive load used for switch cap power calculation. The strategy can be either "max" or "average", with "max" being the default.

The capacitance extraction engine computes, for each net, two capacitance value: one for rise toggles and one for fall toggles.

When this option is set to "max", the largest of these values is used for the calculation of switch cap power.

When this option is set to "average", the average of these values is used for the calculation of switch cap power.

Note that this option does not influence the internal power calculation. For internal power calculation, the corresponding rise/fall capacitance is always used for a rise respectively fall toggle.

See Also

- [get_app_option_value](#)
- [report_power](#)
- [report_app_options](#)
- [set_app_options](#)

power.table_extrapolation

Determines how the power tables are extrapolated during power analysis

p

Data Types

string

Default on**Description**

This application option determines how the power tables are extrapolated during power analysis. If set to "on", linear extrapolation is done. If set to "clip", no extrapolation is done. If set to "clip1", linear extrapolation is done for a limited range, after which it is clipped. The limited range is the size of the last range in the table. If set to "estimate", for all non capacitance variables, clipping is done. For the capacitance variable, the value is clipped and multiplied by the actual capacitance value divided by the max capacitance value in the table.

See Also

- [report_power](#)

power.timer_implied_lower_than_propagated**Data Types***boolean***Default** false**Description**

If the app option is specified as 'true' then propagated activity becomes higher in priority than timer_implied activity (case_analysis value implied by timer) and overrides it. By default the app option is 'false'.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)

power.use_c1cn_pin_capacitance**Data Types***boolean*

Default false

Description

When this app option is set to true, the tool will use the cell's segmented c1cp cap module in the library to calculate pin cap. App option "time.enable_ccs_rcv_cap" needs to be true to consume library c1cn cap module for pin cap calculation. In case the c1cn model is not available in the library, the tool would fall back to use nldm model to calculate pin cap when seeing no c1cn model in the library and warning messages will be printed out

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)
- [report_power](#)
- [report_power_calculation](#)

power.use_ccs_rcv_cap

Data Types

boolean

Default false

Description

This app option enables the use of Synopsys Composite Current-Source (CCS) receiver model capacitance during power analysis. By default, CCS receiver model is not enabled. CCS receiver caps are used in power analysis and reported in report_power_calculation when this option is on.

By default the app option is 'false'.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)

p

- [report_power](#)
- [report_power_calculation](#)

power.use_enhanced_multidriver_net_driver_type

Data Types

boolean

Default false

Description

The app option is for supporting simultaneously multi driven net, like with multi-source CTS. This is not yet supported in PTPX.

If the app option is specified as 'true': - on a net with non tristate drivers, the drivers are assumed to drive simultaneously the net. Each driver will take care of the full toggle rate of the net. The full load of the net will be equally divided over the non tristate drivers. - on a net with only tristate drivers, the drivers are assumed to drive sequentially, i.e. only one driver will be active in time. The full load is driven by each driver. The full toggle rate will be equally divided over the drivers.

If the app option is specified as 'false': - all drivers are assumed to drive the net sequentially.

By default the app option is 'false'. This is compatible with PTPX.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)
- [report_power](#)

power.use_generated_clock_scaling_factor

Determines whether the scaling factor for the generated clocks or master clocks is to be used. By default, the master clock's scaling factor is used for activity scaling for the generated clocks. This app option will only affect the scaling for generated clocks.

p

Data Types

*string***Default** off

Description

The app option determines whether the scaling factor for the generated clocks is to be used. By default, for generated clock, the master clock's scaling factor is used for activity scaling. This app option will only affect the scaling data for the generated clocks. For non-generated clocks, their scaling data is used, if present. If the scaling data is not present for non-generated clocks, no scaling is applied. If the option is 'off', the top most master clock for the generated clock is fetched. If the scaling data for that clock is present it is used for activity scaling, and if it is not present no scaling is applied. If the option is 'on', the scaling data for the generated clock is used. If the scaling data for the generated clock is not present, the master clock for the generated clock is fetched. If the scaling data for the master clock is present it is used for scaling. If it is not present, and the master clock is again a generated clock its master clock is fetched. This is repeated until we find the master clock for which the scaling data is defined. If we do not find the scaling data no scaling is applied. By default, the option is 'off'.

See Also

- [set_power_clock_scaling](#)
- [get_power_clock_scaling](#)
- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)

preroute.common.mv_isolation

Enables multi-voltage isolation of load pins in preroute flow.

Data Types

*integer***Default** 1

Description

Enables multi-voltage isolation in preroute flow. When the option value is set to 1, during initial_drc phase the load pins belonging to the same voltage are clustered together and

q

isolated from other pins on different voltage domains by introducing isolation buffers. If set to 0, the multi-voltage isolation step will be skipped.

See Also

- [place_opt](#)
- [set_app_options](#)
- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [create_mv_cells](#)

q

query_objects_format**Data Types**

string

Default application specific

Description

This variable sets the format that the *query_objects* command uses to print its result. There are two supported formats: Legacy and Tcl.

The Legacy format looks like this:

```
{"or1", "or2", "or3"}
```

The Tcl format looks like this:

```
{or1 or2 or3}
```

Please see the man page for *query_objects* for complete details.

See Also

- [query_objects](#)

r

rail.3dic_generate_collaterals_distributed

Specifies whether to generate per-die collaterals for Redhawk-SC and convert per-die Redhawk-SC results to RAIL_DATABASE in remote machines.

Data Types

boolean

Default false

Description

Specifies whether to generate per-die collaterals for Redhawk-SC and convert per-die Redhawk-SC results to RAIL_DATABASE in remote machines. This app-option should be used with the *set_host_options* command to set a specific machine or farm.

If *true*, the tool generates per-die collaterals for Redhawk-SC and converts per-die Redhawk-SC results to RAIL_DATABASE in remote machines. If *false*, the tool generates per-die collaterals for Redhawk-SC and converts per-die Redhawk-SC results to RAIL_DATABASE in the local machine.

NOTE: Requires *set_host_options -submit_protocol sge -submit_command {...} -target Redhawk*

Examples

The following example generates per-die collaterals for Redhawk-SC and converts per-die Redhawk-SC results to RAIL_DATABASE in remote machines.

```
prompt> set_host_options -submit_protocol sge -submit_command {qsub -P
normal -l mem_free=5g} -target Redhawk
prompt> set_app_options -name rail.3dic_generate_collaterals_distributed
-value true
prompt> analyze_3d_rail -voltage_drop static -nets {VDD VSS}
```

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)
- [set_host_options](#)

r

rail.3dic_generate_collaterals_incremental

Specifies whether to avoid re-generating already generated per-die collaterals.

Data Types

boolean

Default false

Description

Specifies whether to avoid re-generating already generated per-die collaterals.

If *true*, the tool avoids re-generating already generated per-die collaterals. If *false*, the tool re-generates already generated per-die collaterals.

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)
- [set_host_options](#)

rail.3dic_generate_unique_die_only

Specifies whether to reuse collaterals and views for dies of the same master.

Data Types

boolean

Default false

Description

Specifies whether to reuse collaterals and views for dies of the same master.

If *true*, the tool reuses collaterals and views for dies of the same master. If *false*, the tool generates redundant collaterals and views for dies of the same master.

See Also

- [get_app_option_value](#)
- [report_app_options](#)

r

- [set_app_options](#)
- [set_host_options](#)

rail.allow_redhawk_license_checkout

Allows Redhawk to check out license.

Data Types

boolean

Default false

Description

Allows Redhawk to check out license.

See Also

- [analyze_rail](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

rail.analysis_type

Specifies what type of voltage drop analysis to be performed during IR drop driven optimization.

Data Types

string

Default dynamic

Description

Specifies type of voltage drop analysis to be performed during IR drop driven optimization. Accepted values are static, dynamic, dynamic_vcd, or dynamic_vectorless.

Default is dynamic.

string

Optional.

r

See Also

- [analyze_rail](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

rail.apl_files

Specifies RedHawk APL files.

Data Types

string

Default <empty>

Description

For RedHawk flow, generates APL_FILES block in GSR file generated by command `analyze_rail`.

This option can also be used in `3dic_shell`.

When used in `3dic_shell`, requires per-die syntax: { *DIE1* { *DIE1_VALUE* } *DIE2* { *DIE2_VALUE* } ... }

Redhawk-SC SYNTAX

```
{ file
  file
  .. }
```

Redhawk SYNTAX

```
{ file keyword
  file keyword
  .. }
```

file

Must exist.

keyword

Must be one of: `current`, `cdev`, `pwcap`, `avm`, `current_avm`, `cap_avm`, `cap`.

r

See Also

- [analyze_rail](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

rail.cell_subckt_files

Specifies cell subckt files for IR-drop analysis.

Data Types

string

Default <empty>

Description

Specifies cell subckt files for IR-drop analysis.

This option can also be used in 3dic_shell.

When used in 3dic_shell, requires per-die syntax: { *DIE1* { *DIE1_VALUE* } *DIE2* { *DIE2_VALUE* } ... }

Syntax

```
{
  FILE1
  FILE2
  ..
}
```

FILE

FILE cell subckt file name

Examples

This example specifies 2 subckt files using icc2_shell:

```
prompt> set_app_options -name rail.cell_subckt_files -value {
        ./cell_subckt_file1
        ./cell_subckt_file2
      }
```

This example specifies 2 subckt files for si_interposer_inst die using 3dic_shell:

r

```
prompt> set_app_options -name rail.cell_subckt_files -value
{ si_interposer_inst {
    ./cell_subckt_file1
    ./cell_subckt_file2
  }
}
```

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

rail.custom_script_variables

Specifies user define python variables.

Data Types

string

Syntax

```
{{(<TWF | LEF | DEF | SPEF | PLOC> | analysis_view> | liberty_view> | ...
  <USER_VARIABLE>)}} ...}
```

Default <empty>

Description

Specifies python variables for each collateral type to generate in a python collateral index file when generating in-design collaterals Or specifies non-default VIEW name.

Examples

The following example specifies python variables my_ploc, my_lef_files, my_def_files, my_spef_files, my_tw_files respectively for collateral types PLOC, LEF, DEF, SPEF, TWF.

```
prompt> set_app_options -name rail.custom_script_variables -value
{PLOC my_ploc LEF my_lef_files DEF my_def_files SPEF my_spef_files TWF
my_tw_files}
```


r

See Also

- [analyze_rail](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

rail.database

Specifies the path for saving RedHawk rail analysis results.

Data Types

string

Default RAIL_DATABASE

Description

Specifies the path for saving RedHawk rail analysis results.

This is enabled when the rail.enable_redhawk application option is set to true.

See Also

- [analyze_rail](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

rail.def_files

Specifies RedHawk DEF files.

Data Types

string

Default <empty>

Description

For RedHawk flow, generates a DEF_FILES block in the GSR file that is generated by the *analyze_rail* command.

r

This option can also be used in `3dic_shell`.

When used in `3dic_shell`, requires per-die syntax: `{ DIE1 { DIE1_VALUE } DIE2 { DIE2_VALUE } ... }`

Syntax

```
{ file keyword  
  file keyword  
  .. }
```

file

Must exist.

keyword

Must be either *block* or *top*.

See Also

- [analyze_rail](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

rail.display_eco_shapes

To enable Redhawk virtual PG grid analysis and display those virtual PG grids as ECO shapes after rail analysis is done.

Data Types

Boolean

Default `false`

Description

To enable Redhawk virtual PG grid analysis through *mesh add* commands. User can prepare a Redhawk script file which contains *mesh add* commands, and then pass to Redhawk Fusion flow. Those virtual PG grids will be added during rail analysis, and displayed as ECO shapes on GUI when the result is loaded back.

Those displayed ECO shapes are not ICC2 layout objects, so they will not be saved to ICC2 physical database when design is saved. However, they can be restored in *open_rail_result* once this switch is turned on. To query those objects, user can issue command *gui_get_annotations -group rheco*.

r

This switch is only honored in Redhawk instead of Redhawk-SC.

Examples

In the following example user has prepared a Redhawk script file `mesh.tcl` which contain *mesh add* commands, then he passes this file through *set_rail_command_options*:

```
prompt> sh cat mesh.tcl
mesh add -layer METAL6 -horizontal -net VDD -window {0 0 1000 410} -width
10 -space 100 -pitch 100
mesh add -layer METAL6 -horizontal -net VSS -window {0 0 1000 410} -width
10 -space 100 -pitch 100 -offset 50
mesh vias -toplayer METAL5 -bottomlayer METAL4 -net VDD
mesh vias -toplayer METAL5 -bottomlayer METAL4 -net VSS
prompt> set_rail_command_options -script_file rh.tcl -command
setup_design -order after_the_command
prompt> analyze_rail -voltage_drop static -nets {VDD VSS}
```

See Also

- [analyze_rail](#)
- [report_app_options](#)
- [set_app_options](#)

rail.dump_icc2_results_style

Specifies the `dump_icc2_results` style.

Data Types

string

Default 2.0

Description

Specifies the `dump_icc2_results` style. Supported values are "1.0", "1.1" and "2.0".

If `rail.dump_icc2_results_style` set to "1.0", Redhawk-SC dumps results in old map style with single-partition results. If `rail.dump_icc2_results_style` set to "1.1", Redhawk-SC dumps results in new map style with single-partition results. If `rail.dump_icc2_results_style` set to "2.0", Redhawk-SC dumps results in new map style with multiple-partition results.

Here are descriptions of *old/new map style* and *single/multiple-partition* results:

Old map style Parasitic-based maps use non-zero area nodes and edges to approximate layout geometry. This was the default map style prior to 2022.03-SP3.

r

New map style Parasitic-based maps use zero area nodes and edges overlaid upon actual layout geometry. This map style most closely resembles Redhawk-SC map style. In this style, nodes are represented as dots, lateral (XY-direction) edges are represented as segments with arrows, and vertical (Z-direction) edges are represented as circles or Xs. Segment arrow direction indicates lateral current direction. Circles represent vertical current flowing upwards out of the screen. Xs represent vertical current flowing inwards into the screen. This is the default map style starting 2022.03-SP3. This map style requires Redhawk-SC 2022_R1.0.p1 or later. If an older Redhawk-SC version is used to perform analysis, or if an existing RAIL_DATABASE generated from an older version Redhawk-SC is to be loaded, the user must first manually switch to style "1.0".

Single-partition results Results are dumped in serial using one Redhawk-SC worker.

Multiple-partition results Results are dumped in parallel using multiple Redhawk-SC workers. Redhawk-SC divides the layout into different sized partitions based on local design complexity. There's no way to control how they are partitioned.

See Also

- [analyze_rail](#)
- [open_rail_result](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

rail.effective_resistance_instance_file

Specifies a file containing a list of cell instances for RedHawk effective P/G grid resistance calculation.

Data Types

string

Default <empty>

Description

It passes the file to RedHawk as *perform res_calc -instFile FILE*.

perform res_calc -instFile FILE calculates the effective P/G grid resistance from all pads to selected instances, nets or locations, and provides an absolute resistance value in Ohms.

r

Syntax

file

file

Must exist.

FILE FORMAT

```
<name1> <name2>..
or
<name1>,<name2>,... (no space in this form)
```

See Also

- [analyze_rail](#)
- [report_app_options](#)
- [set_app_options](#)

rail.em_only_tech_file

Specifies RedHawk-SC EM-only tech file.

Data Types

string

Default <empty>

Description

For RedHawk-SC flow, generates *em_only_tech_file_name* argument used by *create_tech_view* command in *analyze_rail.py* file generated by command *analyze_rail*.

This option can also be used in *3dic_shell*.

When used in *3dic_shell*, requires per-die syntax: { *DIE1* { *DIE1_VALUE* } *DIE2* { *DIE2_VALUE* } ... }

Syntax

```
{ file }
```

file

Must exist.

r

See Also

- [analyze_rail](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

rail.em_temperature

Specifies the EM temperature used for running RedHawk analysis.

Data Types

string

Default <empty>

Description

For RedHawk flow, generates the TEMPERATURE_EM line in the GSR file that is generated by the *analyze_rail* command.

This option can also be used in 3dic_shell.

When used in 3dic_shell, requires per-die syntax: { *DIE1* { *DIE1_VALUE* } *DIE2* { *DIE2_VALUE* } ... }

Syntax

```
{ numeric }
```

See Also

- [analyze_rail](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

rail.enable_3dic

Specifies whether to enable 3DIC flow.

Data Types

Boolean

Default false

Description

Specifies whether to enable 3DIC flow.

See Also

- [get_app_option_value](#)
 - [report_app_options](#)
 - [set_app_options](#)
-

rail.enable_early_emir

Specifies whether to enable early EMIR flow.

Data Types

Boolean

Default false

Description

Specifies whether to enable early EMIR flow.

See Also

- [get_app_option_value](#)
 - [report_app_options](#)
 - [set_app_options](#)
 - [create_taps](#)
-

rail.enable_eff_vd_flow

Specifies whether to enable Eff-VD flow.

Data Types

boolean

Default false

Description

Specifies whether to enable Eff-VD flow.

r

Eff-VD flow loads results from Redhawk-SC in the form of text reports instead of gz files, optimizing for performance and scalability.

If *true*, the tool performs Eff-VD flow. If *false*, the tool performs default in-design flow.

See Also

- [analyze_rail](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)
- [open_rail_result](#)

rail.enable_hold_analysis

Sets hold analysis to this value before `update_timing` within `analyze_rail`.

Data Types

Boolean

Default `true`

Description

Sets hold analysis to this value before `update_timing` within `analyze_rail`. Applicable only when `rail.enable_update_timing_one_scenario` is set to `true`. Previous user setting is restored after `analyze_rail`.

See Also

- [analyze_rail](#)

rail.enable_missing_cells_report

Enable writing missing cells report.

Data Types

boolean

Default `false`

Description

Enable writing missing cells report.

r

Missing cells report includes two lists: no_lef and no_liberty.

Missing cells report is only supported for 2D flow voltage drop analysis using Redhawk-SC.

Examples

```
prompt> set_app_options -name rail.missing_cells_report -value true
```

See Also

- [analyze_rail](#)
- [set_app_options](#)

rail.enable_new_rail_scenario

Specifies whether to launch rail scenarios.

Data Types

boolean

Default false

Description

Specifies whether to launch rail scenarios.

If rail.enable_new_rail_scenario set to true, and "-rail_scenarios" is used, the tool launches rail scenarios specified by "-rail_scenarios". If rail.enable_new_rail_scenario set to true, and "-rail_scenarios" is not used, the tool launches rail scenarios associated with a design scenario with "-ir_drop" set to true. If rail.enable_new_rail_scenario set to false, and "-rail_scenarios" is used, the tool errors out. If rail.enable_new_rail_scenario set to false, and "-rail_scenarios" is not used, analyze_rail honors other command options.

See Also

- [analyze_rail](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

rail.enable_node_voltage

Specifies whether to enable node voltage report.

r

Data Types

boolean

Default false**Description**

Specifies whether to enable node voltage report

See Also

- [analyze_rail](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)
- [open_rail_result](#)

rail.enable_parallel_run_rail_scenario

Specifies whether to launch rail scenarios in parallel using "-multiple_script_files".

Data Types

boolean

Default true**Description**

Specifies whether to launch rail scenarios in parallel using "-multiple_script_files".

If rail.enable_parallel_rail_scenario set to true, and other criteria for launching rail scenarios are met, the tool launches rail scenarios in parallel using "-multiple_script_files". If rail.enable_parallel_rail_scenario set to false, and other criteria for launching rail scenarios are met, the tool launches rail scenarios serially.

See Also

- [analyze_rail](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

r

rail.enable_rail_probes_collision_test

Specifies whether to enable rail probes collision test.

Data Types

Boolean

Default false

Description

Specifies whether to enable rail probes collision test.

NOTE: Enabling this app-option slows down probe generation noticeably.

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

rail.enable_save

Enable saving redhawk-sc database(db).

Data Types

boolean

Default false

Description

Enable saving redhawk-sc database(db).

This app-option requires *rail.allow_redhawk_license_checkout* set to true.

Examples

```
prompt> set_app_options -name rail.enable_save -value true
```

See Also

- [analyze_rail](#)
- [set_app_options](#)
- [set_host_options](#)

r

rail.enable_scheduler_window

Enable scheduler window when running Redhawk_SC.

Data Types

bool

Default true

Description

Enable scheduler window when running Redhawk_SC.

rail.enable_setup_analysis

Sets setup analysis to this value before update_timing within analyze_rail.

Data Types

Boolean

Default true

Description

Sets setup analysis to this value before update_timing within analyze_rail. Applicable only when rail.enable_update_timing_one_scenario is set to true. Previous user setting is restored after analyze_rail.

See Also

- [analyze_rail](#)
-

rail.enable_write_spef_bg

Enable write SPEF in parallel with DEF and STA for rail analysis using analyze_rail.

Data Types

boolean

Default false

Description

Enable write SPEF in parallel with DEF and STA for rail analysis using analyze_rail.

This results in a slight speedup of up to 1.5x for data-preparation in analyze_rail in some cases.

r

NOTE: There is a known stability issue that causes written out SPEF to become empty, but occurrence is rare.

See Also

- [analyze_rail](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

rail.extra_gsr_option_file

Specifies an external GSR file to be included at the tail of the generated GSR file for Redhawk during IR driven optimization.

Data Types

string

Default <empty>

Description

Generates extra INCLUDE statement at the end of the GSR file generated from *analyze_rail* during IR drop driven optimization.

Default is blank.

Syntax

```
{ file }
```

file

Optional.

See Also

- [analyze_rail](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

r

rail.frequency

Specifies the frequency used for running RedHawk analysis.

Data Types

string

Default <empty>

Description

For RedHawk flow, generates the `FREQ` line in the GSR file that is generated by the `analyze_rail` command.

This option can also be used in `3dic_shell`.

When used in `3dic_shell`, requires per-die syntax: { *DIE1* { *DIE1_VALUE* } *DIE2* { *DIE2_VALUE* } ... }

Syntax

```
{ numeric }
```

See Also

- [analyze_rail](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

rail.generate_file

Specifies Redhawk/Redhawk-SC collateral types to generate for streamline flow.

Data Types

string

Syntax

```
{ <TWF | LEF | DEF | SPEF | PLOC> ... }
```

Default <empty>

Description

Specifies Redhawk/Redhawk-SC collateral types to generate for streamline flow.

r

Examples

The following example specifies to generate collateral types PLOC, LEF, DEF, SPEF, TWF.

```
prompt> set_app_options -name rail.generate_file -value {PLOC LEF DEF  
SPEF TWF}
```

See Also

- [analyze_rail](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

rail.generate_file_type

Specifies the type of collateral index file to generate when generating in-design collaterals.

Data Types

string

Default tcl

Description

Specifies the type of collateral index file to generate when generating in-design collaterals. Supported values are "tcl" and "python".

If rail.generate_file_type set to "tcl", and "-generate_file" is used with valid collateral keywords, the tool generates ICC2 script "set_input_files.tcl" along with the actual collaterals, which can be later sourced by ICC2 to apply generated collaterals for future use. If rail.generate_file_type set to "python", and "-generate_file" is used with valid collateral keywords, the tool generates Redhawk-SC script "input_files.py" along with the actual collaterals, which can be later included by Redhawk-SC to apply generated collaterals for future use.

See Also

- [analyze_rail](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

r

rail.generate_file_variables

Specifies python variables for each collateral type to generate in a python collateral index file when generating in-design collaterals.

Data Types

string

Syntax

```
{ [{<TWF | LEF | DEF | SPEF | PLOC> <USER_VARIABLE>}] ... }
```

Default <empty>

Description

Specifies python variables for each collateral type to generate in a python collateral index file when generating in-design collaterals.

Examples

The following example specifies python variables my_ploc, my_lef_files, my_def_files, my_spef_files, my_tw_files respectively for collateral types PLOC, LEF, DEF, SPEF, TWF.

```
prompt> set_app_options -name rail.generate_file_variables -value
        {PLOC my_ploc LEF my_lef_files DEF my_def_files SPEF my_spef_files TWF
        my_tw_files}
```

See Also

- [analyze_rail](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

rail.gpd_dir

Specifies RedHawk_sc gpd directory.

Data Types

string

Default <empty>

r

Description

For RedHawk_sc flow, specifies GPD directory and stop generate spf files by the *analyze_rail* command.

Syntax

```
{ file}
```

file

Must exist.

See Also

- [analyze_rail](#)
 - [get_app_option_value](#)
 - [report_app_options](#)
 - [set_app_options](#)
-

rail.ignore_filler_cells

Makes RedHawk ignore filler cells.

Data Types

boolean

Default false

Description

Makes RedHawk ignore filler cells.

This is enabled when rail.enable_redhawk is true.

See Also

- [analyze_rail](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

r

rail.instance_power_file

Specifies a RedHawk 3-column instance power file.

Data Types

string

Default <empty>

Description

For RedHawk flow, generates an INSTANCE_POWER_FILES line in the GSR file that is generated by the *analyze_rail* command.

For RedHawk-SC flow, generates an instance_power_file argument in the input_files.py file that is generated by the *analyze_rail* command.

This option can also be used in 3dic_shell.

When used in 3dic_shell, requires per-die syntax: { *DIE1* { *DIE1_VALUE* } *DIE2* { *DIE2_VALUE* } ... }

Syntax

```
{ file }
```

file

Must exist.

FILE FORMAT

```
<CELL> <POWER> <PORT>
```

NOTE: Power is in Watts

See Also

- [analyze_rail](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

rail.instance_voltage_max_lines

Specifies the line limit for instance voltage report.

r

Data Types*integer***Default** 0**Description**

Specifies the line limit for instance voltage report.

See Also

- [analyze_rail](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)
- [open_rail_result](#)

rail.ipf_leakage_data_precedence

Specify the leakage data precedence.

Data Types*string***Default** power_view**Description**

Specify the precedence order for leakage current and power calculation. Possible values are 'power_view' and 'liberty'. Default is 'power_view'.

Syntax

```
{ leakage_data_precedence }
```

leakage_data_precedence

The precedence order for leakage current and power calculation.

See Also

- [analyze_rail](#)
- [get_app_option_value](#)

r

- [report_app_options](#)
- [set_app_options](#)

rail.layer_voltage_max_lines

Specifies the line limit for layer voltage report.

Data Types

integer

Default 0

Description

Specifies the line limit for layer voltage report.

See Also

- [analyze_rail](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)
- [open_rail_result](#)

rail.lef_files

Specifies RedHawk LEF files.

Data Types

string

Default <empty>

Description

For RedHawk flow, generates a LEF_FILES block in the GSR file that is generated by the *analyze_rail* command.

This option can also be used in 3dic_shell.

When used in 3dic_shell, requires per-die syntax: { *DIE1* { *DIE1_VALUE* } *DIE2* { *DIE2_VALUE* } ... }

This option supports wildcards.

r

Syntax

```
{ file1
  file2
  .. }
```

file

Must exist.

See Also

- [analyze_rail](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

rail.lib_files

Specifies RedHawk/RedHawk-SC lib files.

Data Types

string

Default <empty>

Description

For RedHawk flow, generates LIB_FILES block in GSR file generated by command `analyze_rail`.

For RedHawk-SC flow, non-custom lib files generate *liberty_file_names* argument used by *create_liberty_view* command in `analyze_rail.py` file generated by command `analyze_rail`. Custom lib files generate *pgarc_file_names* argument used by *create_liberty_view* command in `analyze_rail.py` file generated by command `analyze_rail`.

This option can also be used in `3dic_shell`.

When used in `3dic_shell`, requires per-die syntax: { *DIE1* { *DIE1_VALUE* } *DIE2* { *DIE2_VALUE* } ... }

This option supports wildcards.

Syntax

```
{ file1
  file2 [CUSTOM]
  .. }
```

r

file

Must exist.

See Also

- [analyze_rail](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

rail.macro_models

Specifies RedHawk macro models.

Data Types

string

Default <empty>

Description

Generates a GDS_CELLs block in that GSR file that is generated by the *analyze_rail* command.

This is enabled when rail.enable_redhawk is true.

Syntax

```
{ cell1 path1
  cell2 path2 [mmx]
  .. }
```

path

Must exist.

See Also

- [analyze_rail](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

r

rail.nets

Specifies on which nets to perform Rail Analysis in optimization flow.

Data Types

string

Default <empty>

Description

Specifies on which nets to perform Rail Analysis in optimization flow.

See Also

- [analyze_rail](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

rail.node_voltage_max_lines

Specifies the line limit for node voltage report.

Data Types

integer

Default 0

Description

Specifies the line limit for node voltage report.

See Also

- [analyze_rail](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)
- [open_rail_result](#)

r

rail.node_voltage_sort_fields

Specifies the sort fields for node voltage report.

Data Types

string

Default node_voltage net layer

Description

Specifies the sort fields for node voltage report.

See Also

- [analyze_rail](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)
- [open_rail_result](#)

rail.pad_files

Specifies RedHawk tap files.

Data Types

string

Default <empty>

Description

For RedHawk flow, generates PAD_FILES block in GSR file generated by command analyze_rail.

This option can also be used in 3dic_shell.

When used in 3dic_shell, requires per-die syntax: { *DIE1* { *DIE1_VALUE* } *DIE2* { *DIE2_VALUE* } ... }

Syntax

```
{ file1
  file2
  .. }
```


r

file

Must exist.

See Also

- [analyze_rail](#)
 - [get_app_option_value](#)
 - [report_app_options](#)
 - [set_app_options](#)
-

rail.product

Specifies which engine performs Rail Analysis.

Data Types

Enum (string)

Default redhawk for 2D flow and redhawk_sc for 3D flow

Description

Specifies which engine performs Rail Analysis.

Possible values are 'redhawk' and 'redhawk_sc'.

See Also

- [analyze_rail](#)
 - [get_app_option_value](#)
 - [report_app_options](#)
 - [set_app_options](#)
-

rail.rail_probes_epsilon

Specifies collision radius for rail probes collision test.

Data Types

Float

Default 0.001

r

Description

Specifies collision radius for rail probes collision test.

This app-option requires app-option rail.enable_rail_probes_collision_test set to true.

Unit is in um.

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

rail.redhawk_path

Specifies the path to the RedHawk executable.

Data Types

string

Default <empty>

Description

Specifies the path to the RedHawk executable.

This is enabled when rail.enable_redhawk is set to true.

Syntax

```
{ path }
```

path

Must exist.

See Also

- [analyze_rail](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

r

rail.redhawk_sc_simulation_cycles

Data Types

Unsigned Integer

Default 10

Description

Use the 'rail.redhawk_sc_simulation_cycles' app option to override the RedHawk SC Simulation Cycles.

By default, the cycles are equal to 5 frames and a frame is equal to 2. So, by default, the cycles are 10.

The minimum admitted value is 1.

See Also

- [analyze_rail](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

rail.result_file

Specifies result file and analysis mode to load SigmaDvD flow or Eff-VD flow results.

Data Types

string

Default <empty>

Description

Specifies result file and analysis mode to load SigmaDvD flow or Eff-VD flow results.

This option cannot be used in 3dic_shell.

Syntax

```
{ file [analysis_mode] }
```

file

Must exist.

r

analysis_mode

Optional. Can be one of static, dynamic, dynamic_vectorless, dynamic_vcd, sigma_dvd, sigma_pd, or sigma_av.

To load static, dynamic, dynamic_vectorless, dynamic_vcd results, app-option *rail.enable_eff_vd_flow* must be turned on.

See Also

- [analyze_rail](#)
- [open_rail_result](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

rail.scenario_name

Specifies scenario to use for timing and extraction when invoke the *analyze_rail* command.

Data Types

string

Default <empty>

Description

Specifies scenario to use for timing and extraction when invoke the *analyze_rail* command.

Syntax

```
{ scenario_name }
```

scenario_name

The name of the scenario to use.

See Also

- [analyze_rail](#)
- [get_app_option_value](#)

r

- [report_app_options](#)
 - [set_app_options](#)
-

rail.search_distance_x

Define search distance in horizontal direction in micron such that when overlap current is calculated for a standard cell, all neighboring cells within this search distance will be counted during calculation.

Data Types

Boolean

Default 50

Description

For overlap current calculation on a standard cell, `rail.search_distance_x` defines horizontal distance in micron from which all neighboring cells fall in the range are counted during calculation. The overlap current is reported in command *report_hot_spots*.

See Also

- [analyze_rail](#)
 - [generate_hot_spots](#)
 - [report_hot_spots](#)
-

rail.search_distance_y

Define search distance in vertical direction in micron such that when overlap current is calculated for a standard cell, all neighboring cells within this search distance will be counted during calculation.

Data Types

Boolean

Default 50

Description

For overlap current calculation on a standard cell, `rail.search_distance_y` defines vertical distance in micron from which all neighboring cells fall in the range are counted during calculation. The overlap current is reported in command *report_hot_spots*.

r

See Also

- [analyze_rail](#)
- [generate_hot_spots](#)
- [report_hot_spots](#)

rail.sigma_av_drop_threshold

Filters SigmaAV drop report results by deltaV threshold (only reports cells that exceed threshold) in SigmaAV flow.

Data Types

integer

Default {}

Description

Filters SigmaAV drop report results by deltaV threshold (only reports cells that exceed threshold) in SigmaAV flow.

See Also

- [analyze_rail](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

rail.sigma_dvd_top_n_victims

Limits the number of instances reported in SigmaDvD flow.

Data Types

integer

Default 1000000

Description

Limits the number of instances reported in SigmaDvD flow.

r

See Also

- [analyze_rail](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

rail.spef_files

Specifies RedHawk SPEF files.

Data Types

string

Default <empty>

Description

For RedHawk flow, generates a CELL_RC_FILE block in that GSR file that is generated by the *analyze_rail* command.

This option can also be used in 3dic_shell.

When used in 3dic_shell, requires per-die syntax: { *DIE1* { *DIE1_VALUE* } *DIE2* { *DIE2_VALUE* } ... }

Syntax

```
{ cell1 file1
  cell2 file2
  .. }
```

file

Must exist.

See Also

- [analyze_rail](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

r

rail.sta_file

Specifies a RedHawk STA file.

Data Types

string

Default <empty>

Description

For RedHawk flow, generates a STA_FILE block in the GSR file that is generated by the *analyze_rail* command.

This option can also be used in 3dic_shell.

When used in 3dic_shell, requires per-die syntax: { *DIE1* { *DIE1_VALUE* } *DIE2* { *DIE2_VALUE* } ... }

Syntax

```
{ top_module_name file }
```

file

Must exist.

See Also

- [analyze_rail](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

rail.switch_model_files

Specifies RedHawk switch model files.

Data Types

string

Default <empty>

r

Description

Generates a SWITCH_MODEL_FILES block in the GSR file that is generated by the *analyze_rail* command.

This is enabled when rail.enable_redhawk is set to true.

This option can also be used in 3dic_shell.

When used in 3dic_shell, requires per-die syntax: { *DIE1* { *DIE1_VALUE* } *DIE2* { *DIE2_VALUE* } ... }

Syntax

```
{ file1
  file2
  .. }
```

file

Must exist.

See Also

- [analyze_rail](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

rail.switching_activity

Specifies an optional switching activity file to be used during IR drop driven optimization.

Data Types

string

Default <empty>

Description

Specifies an optional switching activity file. The *file_format* argument can be *VCD* (a VCD file generated from a gate-level simulation), *SAIF*, or *ICC_ACTIVITY*.

A VCD or SAIF file can reference a top-level design in a different hierarchy than the one used in the current session. The *strip_path* argument removes the specified string from the beginning of the object name. Using this option modifies the object names in the VCD or SAIF file to make them compatible with the object names in the design library.

r

Default is blank.

For example, use the following option to load a VCD file and strip the `/top_design/cup_module` prefix from all object names.

```
set_app_options -name rail.switching_activity -value "VCD
top_design.vcd /top_design/cup_module"
```

For example, use the following option to load a VCD file without a strip path.

```
set_app_options -name rail.switching_activity -value "VCD top_design.vcd"
```

If you have annotated switching activity on the design in the IC Compiler II environment, set the *file_format* argument to *ICC_ACTIVITY*. When you specify this format, the tool creates a temporary SAIF file that contains the user-annotated switching activity and passes it to the RedHawk tool. The *file_name* argument does not apply to this format.

When the *file_format* argument is set to *VCD* or *SAIF*, the tool generates a *VCD_FILE* or *SAIF_FILE* block in the GSR file, respectively.

When the *strip_path* argument is specified, the tool generates the *FRONT_PATH* component in the *VCD_FILE* block, or the *ROOT_INSTANCE* component in the *SAIF_FILE* block. Both *VCD_FILE* and *SAIF_FILE* blocks are in the GSR file.

When the *time_window* argument is specified, the tool generates the *SELECT_RANGE* component in the *VCD_FILE* block in the GSR file.

Syntax

```
{file_format [file_name] [strip_path] [time_window]}
```

string

Optional.

See Also

- [analyze_rail](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

rail.tech_file

Specifies a RedHawk ATF file.

r

Data Types

string

Default <empty>**Description**

For RedHawk flow, generates a TECH_FILE line in the GSR file that is generated by the *analyze_rail* command.

This option can also be used in 3dic_shell.

When used in 3dic_shell, requires per-die syntax: { *DIE1* { *DIE1_VALUE* } *DIE2* { *DIE2_VALUE* } ... }

Syntax

```
{ file }
```

file

Must exist.

See Also

- [analyze_rail](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

rail.technology

Specifies a RedHawk technology.

Data Types

string

Default <empty>**Description**

For RedHawk flow, generates a TECHNOLOGY line in the GSR file that is generated by the *analyze_rail* command.

For RedHawk-SC flow, specifies the technology node used for analysis in nanometers. Affects default step size and max duration.

r

Default is blank.

Syntax

```
{ string }
```

string

Must exist.

See Also

- [analyze_rail](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

rail.temperature

Specifies the temperature used for running RedHawk analysis.

Data Types

string

Default <empty>

Description

For RedHawk flow, generates the TEMPERATURE line in the GSR file that is generated by the *analyze_rail* command.

This option can also be used in 3dic_shell.

When used in 3dic_shell, requires per-die syntax: { *DIE1* { *DIE1_VALUE* } *DIE2* { *DIE2_VALUE* } ... }

Syntax

```
{ numeric }
```

See Also

- [analyze_rail](#)
- [get_app_option_value](#)

r

- [report_app_options](#)
- [set_app_options](#)

rail.toggle_rate

Specifies RedHawk-SC toggle rate.

Data Types

string

Default <empty>

Description

Generates *data_toggle_rate*, *default_combinational_pin_toggle_rate*, *default_sequential_output_pin_toggle_rate*, and *default_clock_pin_toggle_rate* arguments used by *create_switching_activity_view* command in *analyze_rail.py* file generated by command *analyze_rail*.

Default is blank.

Syntax

```
{ clock <numeric> data <numeric> combinational <numeric> sequential  
  <numeric> }
```

numeric

Must exist.

See Also

- [analyze_rail](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

rail.write_def_cell_as_via

Treat via PCELL shapes as genuine vias when invoke *write_def*.

Data Types

string

r

Default <empty>**Description**

Treat via PCELL shapes as genuine vias when invoke write_def.

During fusion/3d flow collateral generation, invokes write_def with "-flatten_lib_cells {MASTER1 MASTER2 ...}" for dies specified (not visible in generated script; please verify by comparing FANOUT die DEF with/without app-option).

This option can only be used in 3dic_shell.

When used in 3dic_shell, requires per-die syntax: { *DIE1* { *DIE1_VALUE* } *DIE2* { *DIE2_VALUE* } ... }

Syntax

```
{ DIE1 { MASTER1 MASTER2 ... } DIE2 { MASTER1 MASTER2 ... } ... }
```

DIE1

DIE1 cell instance name.

MASTER1

MASTER1 cell master name.

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

rail.write_def_flatten_lib_cells

Flatten teardrop PCELL shapes when invoke write_def.

Data Types

string

Default <empty>**Description**

Flatten teardrop PCELL shapes when invoke write_def.

During fusion/3d flow collateral generation, invokes write_def with "-flatten_lib_cells {MASTER1 MASTER2 ...}" for dies specified (not visible in generated script; please verify by comparing FANOUT die DEF with/without app-option).

r

This option can only be used in 3dic_shell.

When used in 3dic_shell, requires per-die syntax: { *DIE1* { *DIE1_VALUE* } *DIE2* { *DIE2_VALUE* } ... }

Syntax

```
{ DIE1 { MASTER1 MASTER2 ... } DIE2 { MASTER1 MASTER2 ... } ... }
```

DIE1

DIE1 cell instance name.

MASTER1

MASTER1 cell master name.

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

refine_opt.flow.clock_aware_placement

Enables ICG timing criticality driven clock-aware placement in refine_opt.

Data Types

Boolean

Default "false"

Description

This option controls whether refine_opt will utilize clock-aware placement, guided by ICG's enable timing criticality. With this option, *refine_opt* will try to improve ICG enable timing (measured after clock_opt) by placing the timing critical ICGs and their fanout cells at better locations for ICG enable paths.

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design. Use *report_app_options* to show the value of application options.

r

See Also

- [place_opt](#)
 - [report_app_options](#)
 - [set_app_options](#)
-

refine_opt.flow.do_path_opt

Run path_opt in refine_opt

Data Types

bool

Default true

Description

This option enables running path_opt in refine_opt for better timing QOR. true is the default.

See Also

- [set_app_options](#)
-

refine_opt.flow.exclusive

Control the exclusive optimization flows.

Data Types

String

Default none

Description

Enable exclusive area recovery, electrical DRC, power or hold optimization in refine_opt.

The exclusive flow can be: area, drc, power, hold or none.

See Also

- [set_app_options](#)

r

refine_opt.flow.optimize_layers

Enable layer optimization in *refine_opt*.

Data Types

String

Default false

Description

It has two modes: {true, false}. *refine_opt* flows will run layer optimization that assigns long timing critical nets to upper metal layers to improve timing.

See Also

- [set_app_options](#)

refine_opt.flow.optimize_layers_critical_range

Set critical range for layer optimization when *refine_opt.flow.optimize_layers* is set to true.

Data Types

Float

Default 0.9

Description

It accepts values from {0 ~ 1.0}. For a given value y, it optimizes nets that have slacks between {y*WNS, WNS}, where WNS is the worst negative slack. Set it to a lower value to fix paths that are in less critical range. It may cause a lot more nets to be promoted and use much more routing resource. So, lower the critical range only if necessary.

See Also

- [set_app_options](#)

refine_opt.flow.optimize_layers_overlap_bins

Enable overlap bins when *refine_opt.flow.optimize_layers* is set to true or auto.

Data Types

Boolean

Default false

r

Description

When the switch is enabled, layer bins can be overlapped. For example: {M10, M9}, {M9, M8}, {M8, M7}, {M7, M6}, {M6, M5} When it is disabled, layer bins do not overlap. For example: {M10, M9}, {M8, M7}, {M6, M5}

The feature might help improve QoR when routing resources is unbalanced. For example, if M10 routing capacity is low, but M9 is high. Enable overlap bin: Nets can be promoted to {M9, M8} to improve QoR. Disable overlap bin: Nets promote to {M8, M7}

See Also

- [set_app_options](#)

refine_opt.flow.optimize_ndr

Enable NDR optimization in refine_opt

Data Types

Boolean

Default false

Description

refine_opt flows will run automatic non-default-routing rule optimization that assigns long timing critical nets to NDR to improve timing.

See Also

- [set_app_options](#)

refine_opt.flow.optimize_ndr_critical_range

Set critical range for Non-default routing (NDR) based optimization when *refine_opt.flow.optimize_ndr* is set to true.

Data Types

Float

Default 0.9

Description

It accepts values from {0 ~ 1.0}. For a given value y, it optimizes nets that have slacks between {y*WNS, WNS}, where WNS is the worst negative slack. Set it to a lower value to

r

fix paths that are in less critical range. It may cause a lot more nets to have NDRs and use much more routing resource. So, lower the critical range only if necessary.

See Also

- [set_app_options](#)

refine_opt.flow.optimize_ndr_max_nets

Set maximum number of nets to be routed with Non-default routing (NDR) rule when *refine_opt.flow.optimize_ndr* is set to true.

Data Types

integer

Default 2000

Description

During NDR-based optimization, the number of maximum nets to be assigned NDRs is given by this value. This number limits the number of potential candidates that can benefit from applying NDRs, and must be a non-negative integer.

See Also

- [set_app_options](#)

refine_opt.flow.optimize_rp_groups

Enable RP group optimization in *refine_opt*.

Data Types

String

Default false

Description

It has two modes: {true, false}. *refine_opt* flows will improve relatively-placed groups using *path_opt* to improve timing.

See Also

- [set_app_options](#)

r

refine_opt.flow.repeatability

refine_opt will trade off repeatability for some runtime speedup.

Data Types

Boolean

Default true

Description

If turned to false, refine_opt will run a little faster, although the QoR at the end of the refine_opt flow will no longer be repeatable. It is not recommended to set this to false, since the runtime difference is not substantial.

See Also

- [set_app_options](#)

refine_opt.hold.effort

Control the optimization effort for hold fixing in refine_opt.

Data Types

String

Default none

Description

Enable higher effort hold fixing in refine_opt flows.

The hold fixing effort can be: none, low, medium, or high. Setting to "none" disables hold fixing.

The high effort option may result in slight degradation in setup TNS.

See Also

- [set_app_options](#)

refine_opt.path_opt.fixed_cell_disturbance

Allow fixed cells to move in path_opt

r

Data Types*bool***Default** false**Description**

This option allows fixed cells to move in *path_opt* for better timing QOR. false is the default.

See Also

- [set_app_options](#)

refine_opt.place.congestion_effort**Data Types***string***Default** medium**Description**

Specifies the effort level for the congestion alleviation in *refine_opt*. Allowed values are *none*, *medium*, or *high*. Expect a significant increase in runtime for high effort. If set to *none*, *refine_opt* will not run in a congestion mode.

See Also

- [create_placement](#)

refine_opt.place.effort**Data Types***string***Default** medium**Description**

Specifies the CPU effort level for coarse placements invoked in *refine_opt*. Allowed values are *medium* or *high*.

See Also

- [create_placement](#)

r

report.safety.enable_physical_checks_in_rtl

Controls physical checks performed by the report_safety_status command in RTL-A.

Data Types

Boolean

Default false

Description

This option can be used to enable physical checks performed by the report_safety_status command for legalized design. The option is disabled by default because RTL-A does not run legalization in the standard flow.

See Also

- [report_safety_status](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

route.auto_via_ladder.allow_neighbor_ripup_in_dense_area

Specifies whether to try reroute an inserted via ladder in dense area. In dense area, there could be insertion ordering issue. Try to reroute might have a better total insertion rate.

This option controls via ladder insertion during the automatic via ladder updating that can be activated with route.auto_via_ladder.update_during_route true or the refresh_via_ladders command.

Data Types

Boolean

Default false

Description

Specifies whether to try reroute an inserted via ladder in dense area. In dense area, there could be insertion ordering issue. Try to reroute might have a better total insertion rate.

By default (*false*), for each via ladder pin, there is only one trail of insertion. Pin ordering issue could affect total via ladder insertion rate - earlier inserted via ladder could block neighbor via ladders. Switch insertion order might increase the total insertion rate.

r

If the option is set to *true*, while there's a insertion failure, tool would detect whether if there's neighboring inserted via ladder could affect the insertion failure. If yes, tool would rip up the inserted via ladder, and re-insert the via ladders with the switch order.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_via_ladder_constraints](#)
- [remove_via_ladder_constraints](#)
- [insert_via_ladders](#)
- [remove_via_ladders](#)
- [verify_via_ladders](#)
- [refresh_via_ladders](#)
- [set_via_ladder_rules](#)
- [route_auto](#)
- [route_eco](#)
- [route_opt](#)
- [route_group](#)

route.auto_via_ladder.allow_patching

Specifies whether via ladder shapes can be patched to honor minimum length rules.

The default is true.

This option controls via ladder insertion during the automatic via ladder updating that can be activated with `route.auto_via_ladder.update_during_route` true or the `refresh_via_ladders` command.

Data Types

Boolean

Default true

r

Description

Specifies whether via ladder shapes can be patched to honor the tech file minLength rule in the Layer section of the tech file or the upperMetalMinLengthTbl rules given in the ViaRule section of the tech file. The applicable minimum length on a layer will be the maximum of these two values. Generalized patching of other DRC types is not supported during via ladder insertion. But the upperMetalMinLengthTbl tech file entry can sometimes be used to cover for other rule types.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_via_ladder_constraints](#)
- [remove_via_ladder_constraints](#)
- [insert_via_ladders](#)
- [remove_via_ladders](#)
- [verify_via_ladders](#)
- [refresh_via_ladders](#)
- [set_via_ladder_rules](#)
- [route_auto](#)
- [route_eco](#)
- [route_opt](#)
- [route_group](#)

route.auto_via_ladder.allow_samenet_intersection

Specifies whether via ladder shapes on a net are allowed to intersect preroute shapes on the same net.

This option controls via ladder insertion during the automatic via ladder updating that can be activated with route.auto_via_ladder.update_during_route true or the refresh_via_ladders command.

Data Types

Boolean

r

Default false**Description**

Specifies whether via ladder shapes on a net are allowed to intersect preroute shapes on the same net.

By default (*false*), a via ladder is inserted on a pin but preroute shapes on the same net will be treated as blockages to be avoided. If the option is set to *true* a via ladder is allowed to touch other shapes on the same net as long as the result is DRC clean.

This option alone does NOT encourage connections to same net shapes. Please see documentation for the companion option `route.auto_via_ladder.bias_top_layer_to_samenet_connection`.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_via_ladder_constraints](#)
- [remove_via_ladder_constraints](#)
- [insert_via_ladders](#)
- [remove_via_ladders](#)
- [verify_via_ladders](#)
- [refresh_via_ladders](#)
- [set_via_ladder_rules](#)
- [route_auto](#)
- [route_eco](#)
- [route_opt](#)
- [route_group](#)

route.auto_via_ladder.allow_via_array_as_single_cut

Specifies whether via arrays can be used in via ladders to prevent "Needs fat contact" DRCs.

The default is true.

r

This option only applies when `route.auto_via_ladder.strictly_honor_cut_table` is false.

This option controls via ladder insertion during the automatic via ladder updating that can be activated with `route.auto_via_ladder.update_during_route` true or the `refresh_via_ladders` command.

The option can be applied to `insert_via_ladders` by setting `route.auto_via_ladder.apply_app_options_to_insert_via_ladders_command` true.

Data Types

Boolean

Default true

Description

Specifies whether via arrays can be used in via ladders to prevent "Needs fat contact" DRCs. To enable use of via arrays, `route.auto_via_ladder.allow_via_array_as_single_cut` must be set to true, and `route.auto_via_ladder.strictly_honor_cut_table` must be set to false.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_via_ladder_constraints](#)
- [remove_via_ladder_constraints](#)
- [verify_via_ladders](#)
- [refresh_via_ladders](#)
- [set_via_ladder_rules](#)
- [route_auto](#)
- [route_eco](#)
- [route_opt](#)
- [route_group](#)

r

route.auto_via_ladder.apply_app_options_to_insert_via_ladders_command

Override insert_via_ladders command line options with route.auto_via_ladder.* app option settings.

Data Types

Boolean

Default true

Description

This option impacts the standalone insert_via_ladders command, and does not impact the automatic via ladder updating that can be activated with route.auto_via_ladder.update_during_route true or the refresh_via_ladders command.

When set to true (the default) the route.auto_via_ladder.* options will apply to insert_via_ladders and verify_via_ladders.

Those commands historically took option settings from the command line, but this interface will be made obsolete in a future release.

For any option that appears as both a command line option and a route.auto_via_ladder.* app option, the app option setting will be used when route.auto_via_ladder.apply_app_options_to_insert_via_ladders_command is set to true (the default).

Legacy command line options to the insert_via_ladders and verify_via_ladders commands will be honored only when route.auto_via_ladder.apply_app_options_to_insert_via_ladders_command false. Any command line option will eventually be obsoleted if it also exists as an app option. Use of the app option interface is strongly recommended.

Note: some command line options like -nets do not exist as app options, and will always be honored.

The default is *true*.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_via_ladder_constraints](#)

r

- [remove_via_ladder_constraints](#)
- [insert_via_ladders](#)
- [remove_via_ladders](#)
- [verify_via_ladders](#)
- [refresh_via_ladders](#)
- [set_via_ladder_rules](#)
- [route_auto](#)
- [route_eco](#)
- [route_opt](#)
- [route_group](#)

route.auto_via_ladder.auto_stagger

Specifies whether via ladder insertion is free to stagger by any amount on any layer to achieve a successful insertion.

Data Types

Boolean

Default `true`

Description

Specifies whether via ladder insertion is free to stagger by any amount on any layer to achieve a successful insertion.

The option only impacts attempts to insert via ladder template types that have no user-set staggering values. If a template has user-set values, we will honor those.

I.e. auto stagger is only active for via ladder templates defined in the tech file with an empty or zeroed `maxNumStaggerTracksTbl`, or for Tcl-defined templates with empty or zeroed `max_num_stagger_tracks_table`. Staggering for other templates would occur just as it does today, regardless of the option setting.

When auto stagger is active for an insertion attempt, the tool will analyze the track configurations and layout around the pin to determine if staggering should be enabled on a level of the via ladder and by how much. This relieves the need for a user to specify any staggering values in the template.

The default is *true*.

r

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_via_ladder_constraints](#)
- [remove_via_ladder_constraints](#)
- [insert_via_ladders](#)
- [remove_via_ladders](#)
- [verify_via_ladders](#)
- [refresh_via_ladders](#)
- [set_via_ladder_rules](#)
- [route_auto](#)
- [route_eco](#)
- [route_opt](#)
- [route_group](#)

route.auto_via_ladder.auto_stagger_for_em_via_ladder_max_number_stagger_tracks_per_level

Specifies the maximum number of tracks allowed for staggering at any level when creating a via ladder with a ViaRule marked as for_electromigration. The limit applies when auto-staggering is used (route.auto_via_ladder.auto_stagger true). The default value is 15, and the maximum is 30.

Data Types

list

Default 15**Description**

Auto-staggering can be activated during via ladder insertion with "route.auto_via_ladder.auto_stagger true." This feature allows via ladder insertion to extend the rows of any via ladder level to avoid DRCs. This staggering gives the tool more flexibility and can improve insertion rates. The length of staggering is defined as a number of routing tracks.

r

The option controls the maximum length of staggering allowed for via ladders with a ViaRule marked as for_electromigration. The default value is 15 tracks.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_via_ladder_constraints](#)
- [remove_via_ladder_constraints](#)
- [insert_via_ladders](#)
- [remove_via_ladders](#)
- [verify_via_ladders](#)
- [refresh_via_ladders](#)
- [set_via_ladder_rules](#)
- [route_auto](#)
- [route_eco](#)
- [route_opt](#)
- [route_group](#)

route.auto_via_ladder.auto_stagger_max_number_stagger_tracks_per_level

Specifies the maximum number of tracks allowed for staggering at any level when creating a via ladder.

The limit applies when auto-staggering is used (route.auto_via_ladder.auto_stagger true). The default value is 9, and the maximum is 30.

Data Types

list

Default 9

r

Description

Auto-staggering can be activated during via ladder insertion with "route.auto_via_ladder.auto_stagger true." This feature allows via ladder insertion to extend the rows of any via ladder level to avoid DRCs. This staggering gives the tool more flexibility and can improve insertion rates. The length of staggering is defined as a number of routing tracks.

The option controls the maximum length of staggering allowed for via ladders. The default value is 9 tracks.

Note that this sets initial allowable staggering for all via ladders, but the allowable staggering may be overridden for specific classes of ViaRules. For example, see route.auto_via_ladder.auto_stagger_for_em_via_ladder_max_number_stagger_tracks_per_level.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_via_ladder_constraints](#)
- [remove_via_ladder_constraints](#)
- [insert_via_ladders](#)
- [remove_via_ladders](#)
- [verify_via_ladders](#)
- [refresh_via_ladders](#)
- [set_via_ladder_rules](#)
- [route_auto](#)
- [route_eco](#)
- [route_opt](#)
- [route_group](#)

route.auto_via_ladder.bias_top_layer_to_samenet_connection

Specifies whether via ladder insertion should prefer to create a top level of a via ladder that intersects existing same net preroutes.

r

This option controls via ladder insertion during the automatic via ladder updating that can be activated with `route.auto_via_ladder.update_during_route` true or the `refresh_via_ladders` command.

Data Types

Boolean

Default false

Description

Specifies whether via ladder insertion should prefer to create a top level of a via ladder that intersects existing same net preroutes.

This option only has an impact when `route.auto_via_ladder.allow_samenet_intersection` is true and the top layer of a via ladder consists of 1 row.

If preconditions are met and this option is true we will always strongly prefer to create a via ladder top row so that it overlaps any existing samenet geometry on the top layer.

The default is *false*.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_via_ladder_constraints](#)
- [remove_via_ladder_constraints](#)
- [insert_via_ladders](#)
- [remove_via_ladders](#)
- [verify_via_ladders](#)
- [refresh_via_ladders](#)
- [set_via_ladder_rules](#)
- [route_auto](#)
- [route_eco](#)
- [route_opt](#)
- [route_group](#)

r

route.auto_via_ladder.check_drcs_on_existing_via_ladders

If true, this option will enable DRC checking of existing via ladders during via ladder insertion. An eligible DRC on an existing via ladder will trigger removal of the via ladder and an attempt to re-insert a new via ladder.

The default is false.

Data Types

Boolean

Default false

Description

This option is intended to clean any via ladder that is correctly connected to a pin and compliant with via ladder constraints, but has unacceptable physical DRCs. For example, this can occur during ECO if nearby cell layout is changed and creates a hard DRC against a via ladder that is otherwise compliant.

The option does not affect `verify_via_ladders`, as that command does not check physical DRCs.

If true, this option will enable DRC checking of existing via ladders during via ladder insertion. An eligible DRC on an existing via ladder will trigger removal of the via ladder and an attempt to re-insert a new via ladder.

Other `route.auto_via_ladder.*` option settings will determine eligibility of a DRC to trigger via ladder removal. E.g. `ignore_routing_shape_drcs`, `ignore_rippable_shapes`, `ignore_min_max_layer_constraints`, `ndr_mode`, `relax_pin_layer_metal_spacing_rules`, and `relax_line_end_via_enclosure_rule`. A DRC is eligible to trigger via ladder removal if the DRC would not be allowed during via ladder insertion based on all option settings.

If a DRC is found to trigger via ladder removal and `route.auto_via_ladder.verbose` is set to true, the log file will include a VIA LADDER INFO line showing the related pin and coordinates, followed by the text: "Existing via ladder has DRCs and will be removed." The next line will show the triggering DRC and coordinates, e.g. VIA LADDER INFO: Diff net spacing [m6] (273.864 576.752) (274.236 576.868) An attempt to re-insert a DRC clean via ladder will follow. Successful insertion is not guaranteed. If insertion fails, a "Needs via ladder" DRC will be created.

Use of this option will increase runtime of via ladder insertion, proportional to the number of existing via ladders in the design. Each of these via ladders will be rechecked for physical DRCs.

r

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_via_ladder_constraints](#)
- [remove_via_ladder_constraints](#)
- [insert_via_ladders](#)
- [remove_via_ladders](#)
- [verify_via_ladders](#)
- [refresh_via_ladders](#)
- [set_via_ladder_rules](#)
- [route_auto](#)
- [route_eco](#)
- [route_opt](#)
- [route_group](#)

route.auto_via_ladder.check_drcs_on_existing_via_ladders_in_route_eco_only

If true, this option will enable DRC checking of existing via ladders during via ladder insertion. An eligible DRC on an existing via ladder will trigger removal of the via ladder and an attempt to re-insert a new via ladder. The checking will be done only when via ladder insertion is invoked from within `route_eco`.

The default is false.

Data Types

Boolean

Default false

Description

This option is intended to clean any via ladder that is correctly connected to a pin and compliant with via ladder constraints, but has unacceptable physical DRCs. For example,

r

this can occur during ECO if nearby cell layout is changed and creates a hard DRC against a via ladder that is otherwise compliant.

The option does not affect `verify_via_ladders`, as that command does not check physical DRCs.

If true, this option will enable DRC checking of existing via ladders during via ladder insertion inside the `route_eco` command. An eligible DRC on an existing via ladder will trigger removal of the via ladder and an attempt to re-insert a new via ladder.

Other `route.auto_via_ladder.*` option settings will determine eligibility of a DRC to trigger via ladder removal. E.g. `ignore_routing_shape_drcs`, `ignore_min_max_layer_constraints`, `ndr_mode`, and `relax_pin_layer_metal_spacing_rules`. A DRC is eligible to trigger via ladder removal if the DRC would not be allowed during via ladder insertion based on all option settings.

If a DRC is found to trigger via ladder removal and `route.auto_via_ladder.verbose` is set to true, the log file will include a VIA LADDER INFO line showing the related pin and coordinates, followed by the text: "Existing via ladder has DRCs and will be removed." The next line will show the triggering DRC and coordinates, e.g. VIA LADDER INFO: Diff net spacing [m6] (273.864 576.752) (274.236 576.868) An attempt to re-insert a DRC clean via ladder will follow. Successful insertion is not guaranteed. If insertion fails, a "Needs via ladder" DRC will be created.

Use of this option will increase runtime of via ladder insertion, proportional to the number of existing via ladders in the design. Each of these via ladders will be rechecked for physical DRCs.

This option is similar to `route.auto_via_ladder.check_drcs_on_existing_via_ladders` but DRC checks will only be done during the via ladder refresh step at the beginning of `route_eco`.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_via_ladder_constraints](#)
- [remove_via_ladder_constraints](#)
- [insert_via_ladders](#)
- [remove_via_ladders](#)
- [verify_via_ladders](#)

r

- [refresh_via_ladders](#)
- [set_via_ladder_rules](#)
- [route_auto](#)
- [route_eco](#)
- [route_opt](#)
- [route_group](#)

route.auto_via_ladder.clean

Specifies whether all existing via ladders in the design should be removed before proceeding with via ladder insertion.

The default is false.

This option controls via ladder insertion during the automatic via ladder updating that can be activated with `route.auto_via_ladder.update_during_route` true or the `refresh_via_ladders` command.

Data Types

Boolean

Default false

Description

If true, all existing via ladders are removed before inserting via ladders.

If false, existing via ladders are not removed. Insertion of a new via ladder is only attempted on pins that do not already have a via ladder.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_via_ladder_constraints](#)
- [remove_via_ladder_constraints](#)
- [insert_via_ladders](#)
- [remove_via_ladders](#)

r

- [verify_via_ladders](#)
- [refresh_via_ladders](#)
- [set_via_ladder_rules](#)
- [route_auto](#)
- [route_eco](#)
- [route_opt](#)
- [route_group](#)

route.auto_via_ladder.connect_within_metal

Specifies whether via ladder shape is limited in pin shapes and its extensions on pin layer.

The default is false.

This option controls via ladder insertion during the automatic via ladder updating that can be activated with `route.auto_via_ladder.update_during_route` true or the `refresh_via_ladders` command.

Data Types

Boolean

Default false

Description

Specifies whether via ladder shape is limited to be contained inside of bounding box of pin shapes and its extensions on pin layer. Pin layer refers to top layer of the pin. Extension refer to "special_router_extension".

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_via_ladder_constraints](#)
- [remove_via_ladder_constraints](#)
- [insert_via_ladders](#)
- [remove_via_ladders](#)

r

- [verify_via_ladders](#)
- [refresh_via_ladders](#)
- [set_via_ladder_rules](#)
- [route_auto](#)
- [route_eco](#)
- [route_opt](#)
- [route_group](#)

route.auto_via_ladder.connect_within_metal_for_em_via_ladder

Specifies whether via ladder shape is limited in pin shapes and its extensions on pin layer. Unlike `route.auto_via_ladder.connect_within_metal_for_via_ladder`, which controls "is_via_ladder" shapes on all types of via_rule. This option only affects via_rule with "for_electro_migration" is TRUE.

The default is true.

This option controls via ladder insertion during the automatic via ladder updating that can be activated with `route.auto_via_ladder.update_during_route` true or the `refresh_via_ladders` command.

Data Types

String

Default true

Description

Specifies whether via ladder shape is limited to be contained inside of bounding box of pin shapes and its extensions on pin layer. Pin layer refers to top layer of the pin. Extension refer to "special_router_extension".

- *same_as_global*

Follow the behavior of global option,
`route.auto_via_ladder.connect_within_metal_for_via_ladder`.

- *false*

Regardless of the behavior of global option,
`route.auto_via_ladder.connect_within_metal_for_via_ladder`. "is_via_ladder" shape for "for_electro_migration" via_rule can stick out of pin shapes or pin extensions.

r

- *true* (default)

Regardless of the behavior of global option, `route.auto_via_ladder.connect_within_metal_for_via_ladder`. "is_via_ladder" shape for "for_electro_migration" via_rule cannot stick out of pin shapes or pin extensions.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_via_ladder_constraints](#)
- [remove_via_ladder_constraints](#)
- [insert_via_ladders](#)
- [remove_via_ladders](#)
- [verify_via_ladders](#)
- [refresh_via_ladders](#)
- [set_via_ladder_rules](#)
- [route_auto](#)
- [route_eco](#)
- [route_opt](#)
- [route_group](#)

route.auto_via_ladder.connect_within_metal_for_via_ladder

Specifies whether via ladder shape is limited in pin shapes and its extensions on pin layer. Unlike `route.auto_via_ladder.connect_within_metal`, which controls both "is_via_ladder" shape and "is_pattern_must_join" shape, this option only affects "is_via_ladder" shape.

The default is `same_as_global`.

This option controls via ladder insertion during the automatic via ladder updating that can be activated with `route.auto_via_ladder.update_during_route` true or the `refresh_via_ladders` command.

Data Types

String

r

Default `same_as_global`**Description**

Specifies whether via ladder shape is limited to be contained inside of bounding box of pin shapes and its extensions on pin layer. Pin layer refers to top layer of the pin. Extension refer to "special_router_extension".

- *same_as_global* (default)

Follow the behavior of global option, `route.auto_via_ladder.connect_within_metal`.

- *false*

Regardless of the behavior of global option, `route.auto_via_ladder.connect_within_metal`. "is_via_ladder" shape can stick out of pin shapes or pin extensions.

- *true*

Regardless of the behavior of global option, `route.auto_via_ladder.connect_within_metal`. "is_via_ladder" shape cannot stick out of pin shapes or pin extensions.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_via_ladder_constraints](#)
- [remove_via_ladder_constraints](#)
- [insert_via_ladders](#)
- [remove_via_ladders](#)
- [verify_via_ladders](#)
- [refresh_via_ladders](#)
- [set_via_ladder_rules](#)
- [route_auto](#)
- [route_eco](#)
- [route_opt](#)
- [route_group](#)

r

route.auto_via_ladder.connect_within_pin_mode

Controls how via ladder's via must be within the pin shape.

This option controls via ladder insertion during the automatic via ladder updating that can be activated with `route.auto_via_ladder.update_during_route` true or the `refresh_via_ladders` command.

Data Types

string pair list

Default { {all off} }

Description

Controls how via ladder's via must be within the pin shape.

Format of string pair list is : { {*TARGET_TYPE MODE*} {} ... }

For *TARGET_TYPE*, valid keywords are :

- *all*

Apply the setting on all types of via ladder, including pattern must join.

- *via_ladder*

Apply the setting on all types of via ladder, excluding pattern must join.

- *pattern_must_join*

Apply the setting on pattern must join.

For *MODE*, valid keywords are :

- *off*

No restriction on the via drop location.

- *via_cut*

The via-cut should be fully contained by the pin shape.

- *enclosure_within_metal*

The via-enclosure should be fully contained by the pin shape and pin extension.

If this option is specified, `route.auto_via_ladder.connect_within_metal` and `route.auto_via_ladder.connect_within_metal_for_via_ladder` will be ignored.

r

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_via_ladder_constraints](#)
- [remove_via_ladder_constraints](#)
- [insert_via_ladders](#)
- [remove_via_ladders](#)
- [verify_via_ladders](#)
- [refresh_via_ladders](#)
- [set_via_ladder_rules](#)
- [route_auto](#)
- [route_eco](#)
- [route_opt](#)
- [route_group](#)

route.auto_via_ladder.default_width_ndr_promotable_on_nonreserved_tracks

Specifies whether non-reserved tracks with a width constraint can be used for via ladders on NDR nets when the width specified in the NDR is the same as the default width.

The default is true.

This option controls via ladder insertion during the automatic via ladder updating that can be activated with `route.auto_via_ladder.update_during_route` true or the `refresh_via_ladders` command.

Data Types

Boolean

Default true

Description

Specifies whether non-reserved tracks with a width constraint can be used for via ladders on NDR nets when the width specified in the NDR is the same as the default width.

r

If true (the default), via ladder insertion on an NDR net is able to use a non-reserved track if the width of the track layer in the NDR is equal to the default width. Via ladder insertion will create a wire with a width equal to the width constraint given on the track.

If false, via ladder insertion on an NDR net can only use tracks with a width that matches the NDR width constraint for that layer.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_via_ladder_constraints](#)
- [remove_via_ladder_constraints](#)
- [insert_via_ladders](#)
- [remove_via_ladders](#)
- [verify_via_ladders](#)
- [refresh_via_ladders](#)
- [set_via_ladder_rules](#)
- [route_auto](#)
- [route_eco](#)
- [route_opt](#)
- [route_group](#)

route.auto_via_ladder.enable_em_to_performance_via_ladder_update

Specifies whether a forElectromigration via ladder in the layout should be removed during via ladder updating if it matches via ladder constraints but is preceded by a forHighPerformance via ladder in the via ladder constraint for the pin.

The default is true.

This option controls via ladder insertion during the automatic via ladder updating that can be activated with `route.auto_via_ladder.update_during_route` true or the `refresh_via_ladders` command.

r

Data Types

Boolean

Default true

Description

The tech file ViaRule via ladder templates contain a forHighPerformance flag and a forElectromigration flag.

During the automatic via ladder update, the via ladder verification step will identify which ViaRule matches a given via ladder in the layout.

If the enable_em_to_performance_via_ladder_update option is false, a properly connected via ladder in the layout will be removed only if it does not match the via ladder constraints for the pin.

If the enable_em_to_performance_via_ladder_update option is true (the default), a properly connected via ladder will also be removed if it DOES match the via ladder constraints for the pin AND is marked as forElectromigration AND a ViaRule marked as forHighPerformance appears in the via ladder constraint preference list as a more preferred ViaRule.

This enables via ladder insertion to potentially upgrade a forElectromigration via ladder to a forHighPerformance via ladder, even if the existing forElectromigration via ladder matches via ladder constraints and would normally not be touched.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_via_ladder_constraints](#)
- [remove_via_ladder_constraints](#)
- [insert_via_ladders](#)
- [remove_via_ladders](#)
- [verify_via_ladders](#)
- [refresh_via_ladders](#)
- [set_via_ladder_rules](#)
- [route_auto](#)

r

- [route_eco](#)
- [route_opt](#)
- [route_group](#)

route.auto_via_ladder.generate_center_track_on_off_grid_pattern_must_join_pin_shapes

Specifies whether via ladder commands should auto-generate a default track on any pattern_must_join shape found to be off-track.

Data Types

Boolean

Default false

Description

Specifies whether via ladder commands should auto-generate a default track on any pattern_must_join shape found to be off-track. If appropriate, a track will be generated through the center of the pin shape, running in the preferred direction of the pin layer.

The generated track is visible only during via ladder commands and cannot be used by regular routing commands.

The default is *false*.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_via_ladder_constraints](#)
- [remove_via_ladder_constraints](#)
- [insert_via_ladders](#)
- [remove_via_ladders](#)
- [verify_via_ladders](#)
- [refresh_via_ladders](#)
- [set_via_ladder_rules](#)

r

- [route_auto](#)
- [route_eco](#)
- [route_opt](#)
- [route_group](#)

route.auto_via_ladder.generate_track_width_by_layer_name

Specifies the width constraint to place on tracks auto-generated only for via ladder insertion and verification.

Data Types

list

Default ""

Description

The format to specify the track width for auto-generated tracks on a routing layer during via ladder commands is:

```
{layer width}
```

You must specify the routing layers by using the layer names in the technology file.

The width must be a float value between 0.0 and 20.0.

For each layer, we will auto-generate a track with the specified width in between any two adjacent default width tracks, at exactly the halfway point. We will consider tracks with no width constraint, or a width constraint that matches the default width. Tracks with only width constraints > default width will not be considered during auto-generation.

This will be done globally on all specified layers. Only one width can be specified per layer. For any layer L, we will call the user-specified width W(L).

Additionally, we will look at all pins with via ladder constraints, and generate non-redundant tracks on the pin layer that pass through the center of wide rects of the pin. "Wide rects" will be determined using the user-specified width for the layer W(L) supplied in the new app option.

r

For example, we might have a pin with two rects as follows:

```
|-----| | A | * | * * * * | * * * * |-----|-----| > > | > > > >
B > > > > > | > > |-----|
```

The user track passes through the thin rect B, with the track shown as “> > >”. We will auto-generate a track like “* * * *”. [Note: the track would be slightly lower than depicted in the ASCII drawing.]

To do so, we will generate maximal rects for the polygon of the pin.

```
|-----| | C | * | * * * * | * * * * |=====
> > | > > > > > > > | > > B > > > > > |=====|=====
```

For this pin, the maximal rects will be rect C and rect B. Rect C spans the full vertical range of the pin (outlined with “-” and “|”), and Rect B spans the full horizontal range of the pin (outlined with “=”). In this example, the width of rect B is < W(L) and the width of rect C is > W(L).

Width is measured in the dimension orthogonal to the routing tracks.

We will ignore rect B, since its width is < W(L). We will generate a track that passes through the center point of rect C. (Note: this location will be lower than what is shown in the drawing).

Use of auto-generated tracks can be controlled with the following fields in the ViaRule:

```
lower_layer_uses_auto_generated_tracks_table
upper_layer_uses_auto_generated_tracks_table
```

A via ladder row runs along the track for the upper layer of a via ladder level, and vias are allowed at the intersection of the lower layer track and the upper layer track. These two tables together specify the legal locations for the vias.

For example:

```
cut_layer_name_table = (VIA1, VIA2, VIA3, VIA4)
lower_layer_uses_auto_generated_tracks_table = (1,1,1,0)
upper_layer_uses_auto_generated_tracks_table = (1,1 0,0)
```

The VIA1 level has a lower layer of M1 (pin layer) and an upper layer of M2. The via ladder rows are dropped at the intersection points of M1 and M2 tracks, and the row of vias runs along the M2 track, connected by M2 wire.

For any level of the via ladder below the top level, the tool can generate a via ladder row on either i) only auto-generated tracks or ii) only user tracks.

In no case is a mix of user and auto-generated tracks to be used for the rows on any one level of the via ladder.

r

The top of the via ladder must always use user tracks so that the router can connect to the top wire.

Let's say this ViaRule spans VIA1, VIA2, VIA3, VIA4 (pin layer == M1, top of via ladder M5).

Using the example setting above:

We would use only auto-generated tracks on the pin layer M1.

The rows on M2 would use only auto-generated tracks (all vias at intersection points of auto-generated tracks on M1 and M2).

The rows on M3 would use only auto-generated tracks (all vias at intersection points of auto-generated tracks on M2 and M3).

The rows on M4 would use user tracks, with vias tapping into M3 on auto-generated tracks.

The rows on M5 would use only user tracks. This upper_layer_uses_auto_generated_tracks_table field is ignored. Top level rows are never allowed on auto-generated tracks.

Examples

The following example shows how to auto-generate tracks with a width constraint for via ladder commands:

```
prompt> set_app_options -block [current_block] \\  
-name route.auto_via_ladder.generate_track_width_by_layer_name \\  
-value { {M4 0.01} {M5 0.02} }
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)
- [set_via_ladder_constraints](#)
- [remove_via_ladder_constraints](#)
- [insert_via_ladders](#)
- [remove_via_ladders](#)
- [verify_via_ladders](#)

r

- [refresh_via_ladders](#)
- [set_via_ladder_rules](#)
- [route_auto](#)
- [route_eco](#)
- [route_opt](#)
- [route_group](#)

route.auto_via_ladder.ignore_min_max_layer_constraints

If true, minimum and maximum layer constraints will be ignored during via ladder insertion.

Minimum and maximum layer constraints never apply to via ladders if the effective min/max layer mode is "soft" or "allow_pin_connection." The constraints are only relevant if the effective min/max layer mode is "hard."

The default for route.auto_via_ladder.ignore_min_max_layer_constraints is true, and min/max layer constraints will be ignored even in "hard" mode.

If set to false, via ladder insertion will honor min/max layer constraints in "hard" mode.

This option controls via ladder insertion during the automatic via ladder updating that can be activated with route.auto_via_ladder.update_during_route true or the refresh_via_ladders command.

Data Types

Boolean

Default true

Description

If true (the default), this option will ignore all minimum and maximum layer constraints during via ladder insertion. Any "Min-max layer" DRC will not prevent successful insertion of a via ladder.

If false, a via ladder will not be inserted if minimum or maximum layer constraints would be violated.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)

r

- [set_via_ladder_constraints](#)
- [remove_via_ladder_constraints](#)
- [insert_via_ladders](#)
- [remove_via_ladders](#)
- [verify_via_ladders](#)
- [refresh_via_ladders](#)
- [set_via_ladder_rules](#)
- [route_auto](#)
- [route_eco](#)
- [route_opt](#)
- [route_group](#)

route.auto_via_ladder.ignore_routing_shape_drcs

Specifies whether detailed routing shapes should be ignored during via ladder insertion DRC checking.

This option controls via ladder insertion during the automatic via ladder updating that can be activated with `route.auto_via_ladder.update_during_route` true or the `refresh_via_ladders` command.

Data Types

Boolean

Default true

Description

If true, via ladder insertion will ignore any detailed routing shape that the detailed router would be allowed to reroute. DRCs are not checked against detailed routing shapes during via ladder insertion.

If false, DRCs are checked against all shapes. The recommended setting for optimal via ladder topology is `route.auto_via_ladder.ignore_routing_shape_drcs` true.

The default is true.

r

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_via_ladder_constraints](#)
- [remove_via_ladder_constraints](#)
- [insert_via_ladders](#)
- [remove_via_ladders](#)
- [verify_via_ladders](#)
- [refresh_via_ladders](#)
- [set_via_ladder_rules](#)
- [route_auto](#)
- [route_eco](#)
- [route_opt](#)
- [route_group](#)

route.auto_via_ladder.misalign_adjacent_pattern_must_join

Specifies whether to discourage adjacent pattern must join structures to be inserted on the same track.

This option controls via ladder insertion during the automatic via ladder updating that can be activated with `route.auto_via_ladder.update_during_route` true or the `refresh_via_ladders` command.

Data Types

Boolean

Default false

Description

Specifies whether to discourage adjacent pattern must join structures to be inserted on the same track. For limited routing resource case, access to pattern must join structures (PMJs) could mostly rely on same layer connection. Misalign the top layer metal of PMJ would help increase accessibility.

r

By default (*false*), PMJ top layer metal could be inserted on any track as long as the shape itself is DRC clean.

If the option is set to *true*, while there's another adjacent PMJ or via ladder within certain distance, tool will try to avoid using the same track. Please note it's a soft constraint. If there's no other DRC clean pattern, tool could still use the same track.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_via_ladder_constraints](#)
- [remove_via_ladder_constraints](#)
- [insert_via_ladders](#)
- [remove_via_ladders](#)
- [verify_via_ladders](#)
- [refresh_via_ladders](#)
- [set_via_ladder_rules](#)
- [route_auto](#)
- [route_eco](#)
- [route_opt](#)
- [route_group](#)

route.auto_via_ladder.ndr_mode

Controls how via ladders on nets with nondefault routing rules are created with default width wires.

This option controls via ladder insertion during the automatic via ladder updating that can be activated with `route.auto_via_ladder.update_during_route` true or the `refresh_via_ladders` command.

Data Types

enumerated

Default `top_layer_only`

r

Description

Controls how via ladders on nets with nondefault routing rules are created with default width wires.

If *full*, all layers of the via ladder will honor the NDR.

If *top_layer_only* (the default), only the top layer of the via ladder will honor the NDR. All lower layers will use default width wires.

If *none*, the NDR is ignored and all layers of the via ladder will use default width wires.

This option replaces and overrides `route.auto_via_ladder.ndr_on_top_layer_only`. That legacy option will be obsoleted in a future release.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_via_ladder_constraints](#)
- [remove_via_ladder_constraints](#)
- [insert_via_ladders](#)
- [remove_via_ladders](#)
- [verify_via_ladders](#)
- [refresh_via_ladders](#)
- [set_via_ladder_rules](#)
- [route_auto](#)
- [route_eco](#)
- [route_opt](#)
- [route_group](#)

route.auto_via_ladder.pattern_must_join_max_number_stagger_tracks

Specifies the maximum staggering number of tracks at each level. The default value is 0, and the maximum is 9.

r

Data Types

list

Default 0

Description

It controls the same functionality as `max_num_stagger_tracks_table` of `viaRule`. If the value set to `{3 0}`, then the pattern must join structure is allowed to extend by 3 tracks while constructing the first level of pattern must join structure. Then there would be longer metal for the second level of pattern must join structure to drop via.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_via_ladder_constraints](#)
- [remove_via_ladder_constraints](#)
- [insert_via_ladders](#)
- [remove_via_ladders](#)
- [verify_via_ladders](#)
- [refresh_via_ladders](#)
- [set_via_ladder_rules](#)
- [route_auto](#)
- [route_eco](#)
- [route_opt](#)
- [route_group](#)

route.auto_via_ladder.pattern_must_join_over_pin_layer

Specifies number of levels of pattern must join structure. The default value is 1, and the maximum is 2.

Data Types

Integer

r

Default 1**Description**

If the value set to 1, a pattern must join structure would be inserted according to pin shapes number. If the value set to 2, a single via would be added on top of the original 1-level pattern must join structure. The via at level-2 would have is_pattern_must_join attribute. And router would connect it through its upper via enclosure.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_via_ladder_constraints](#)
- [remove_via_ladder_constraints](#)
- [insert_via_ladders](#)
- [remove_via_ladders](#)
- [verify_via_ladders](#)
- [refresh_via_ladders](#)
- [set_via_ladder_rules](#)
- [route_auto](#)
- [route_eco](#)
- [route_opt](#)
- [route_group](#)

route.auto_via_ladder.pin_access_aware

Species whether via ladder insertion should check accessibility of neighboring pin.

The default is true.

This option controls via ladder insertion during the automatic via ladder updating that can be activated with route.auto_via_ladder.update_during_route true or the refresh_via_ladders command.

r

Data Types

Boolean

Default true

Description

Specifies whether via ladder insertion should check accessibility of neighboring pin.

If this option is true, via ladder insertion would prefer to build a via ladder with less influence to neighbor pin's pin access location.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_via_ladder_constraints](#)
- [remove_via_ladder_constraints](#)
- [verify_via_ladders](#)
- [refresh_via_ladders](#)
- [set_via_ladder_rules](#)
- [route_auto](#)
- [route_eco](#)
- [route_opt](#)
- [route_group](#)

route.auto_via_ladder.relax_pin_layer_metal_spacing_rules

Specifies whether some metal spacing rules can be ignored on the pin layer during via ladder insertion whenever the involved via ladder shape is fully contained within the pin.

The default is true.

This option controls via ladder insertion during the automatic via ladder updating that can be activated with `route.auto_via_ladder.update_during_route` true or the `refresh_via_ladders` command.

r

Data Types

Boolean

Default true

Description

Specifies whether some metal spacing rules can be ignored on the pin layer during via ladder insertion whenever the involved via ladder shape is fully contained within the pin. This includes "Diff net spacing," "End to end forbidden spacing," and "End to end space keepout" DRC types.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_via_ladder_constraints](#)
- [remove_via_ladder_constraints](#)
- [insert_via_ladders](#)
- [remove_via_ladders](#)
- [verify_via_ladders](#)
- [refresh_via_ladders](#)
- [set_via_ladder_rules](#)
- [route_auto](#)
- [route_eco](#)
- [route_opt](#)
- [route_group](#)

route.auto_via_ladder.report_all_via_ladder_violations

Species whether to print all the violation entries of via ladder verification report.

The default is true.

r

This option controls via ladder verification during the automatic via ladder updating that can be activated with `route.auto_via_ladder.update_during_route` true or the `refresh_via_ladders` command.

The option does not impact standalone via ladder verification done with `verify_via_ladders`.

Data Types

Boolean

Default true

Description

Specifies whether to print all the violation entries of via ladder verification report.

If false, the report only prints the first 40 entries for each violation category.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_via_ladder_constraints](#)
- [remove_via_ladder_constraints](#)
- [verify_via_ladders](#)
- [refresh_via_ladders](#)
- [set_via_ladder_rules](#)
- [route_auto](#)
- [route_eco](#)
- [route_opt](#)
- [route_group](#)

route.auto_via_ladder.report_all_via_ladders

Specifies whether to print all the entries of via ladder verification report.

The default is true.

r

This option controls via ladder verification during the automatic via ladder updating that can be activated with `route.auto_via_ladder.update_during_route` true or the `refresh_via_ladders` command.

The option does not impact standalone via ladder verification done with `verify_via_ladders`.

Data Types

Boolean

Default true

Description

Species whether to print all the entries of via ladder verification report.

If false, the report only prints the first 40 entries for each category.

If false, there's dependency with another app-option `route.auto_via_ladder.report_all_via_ladder_violations` Set it to true could report all the entries for violation categories.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_via_ladder_constraints](#)
- [remove_via_ladder_constraints](#)
- [verify_via_ladders](#)
- [refresh_via_ladders](#)
- [set_via_ladder_rules](#)
- [route_auto](#)
- [route_eco](#)
- [route_opt](#)
- [route_group](#)

r

route.auto_via_ladder.report_via_ladder_in_area_with_drc_threshold

Specifies how many DRCs in the vicinity of a via ladder should trigger creation of a “Via ladder in area with DRCs” DRC in `verify_via_ladders`. This DRC is for reporting purposes only and has no impact on routing QoR. It is intended as a tool to identify cases where the presence of a via ladder may be contributing to DRC convergence issues.

The default is -1, meaning “Via ladder in area with DRCs” DRCs will not be created.

If set to a non-negative number, the immediate area around any valid via ladder will be analyzed for physical DRCs. These DRCs will be counted, and if the count exceeds the set threshold, a “Via ladder in area with DRCs” DRC will be created.

Additional details will be reported if `route.auto_via_ladder.verbose` is true. The verbose output will include a list of DRC types and counts in the vicinity of via ladders.

The option will not trigger re-checking of DRCs. Only DRCs in the existing error cell are counted. For example, `route_detail` or `check_routes` can be run to create a DRC error cell, this option can be set, and then `verify_via_ladders` can be run to perform the analysis.

Data Types

Integer

Default -1

Description

If set to a non-negative number, the immediate area around any valid via ladder will be analyzed for physical DRCs. Via ladders that would be flagged as incorrect or Misplaced will not be analyzed.

The immediate area will be calculated per layer as the bounding box of the via ladder on that layer, bloated by min spacing. Physical DRCs within those bounding boxes will be counted.

If the total count across all layers exceeds the threshold, a “Via ladder in area with DRCs” DRC will be flagged.

The DRC bounding box will be the bounding box of the pin on which the via ladder is attached, and the DRC layer will be the pin layer.

The app option will apply in both `verify_via_ladders` and `refresh_via_ladders` as long as `route.auto_via_ladder.apply_app_options_to_insert_via_ladders_command` is true.

r

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_via_ladder_constraints](#)
- [remove_via_ladder_constraints](#)
- [verify_via_ladders](#)
- [refresh_via_ladders](#)
- [set_via_ladder_rules](#)
- [route_auto](#)
- [route_eco](#)
- [route_opt](#)
- [route_group](#)

route.auto_via_ladder.shift_vias_on_transition_layers

Specifies whether via ladder cuts on transition layers should be shifted off the routing tracks for improved DRC compliance.

The default is true.

This option controls via ladder insertion during the automatic via ladder updating that can be activated with `route.auto_via_ladder.update_during_route` true or the `refresh_via_ladders` command.

Data Types

Boolean

Default true

Description

If false, via ladder cuts are always centered at the intersection of routing tracks between adjacent layers.

If true (the default), via ladder cuts may be shifted off track on transition layers for improved DRC compliance.

r

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_via_ladder_constraints](#)
- [remove_via_ladder_constraints](#)
- [insert_via_ladders](#)
- [remove_via_ladders](#)
- [verify_via_ladders](#)
- [refresh_via_ladders](#)
- [set_via_ladder_rules](#)
- [route_auto](#)
- [route_eco](#)
- [route_opt](#)
- [route_group](#)

route.auto_via_ladder.strictly_honor_cut_table

Specifies whether via ladder insertion must strictly honor the cuts specified in the ViaRule cutNameTbl.

The default is false.

This option controls via ladder insertion during the automatic via ladder updating that can be activated with route.auto_via_ladder.update_during_route true or the refresh_via_ladders command.

Data Types

Boolean

Default false

Description

If false (the default), via ladder insertion will usually honor the cuts specified in the tech file ViaRule via ladder template, but via ladder insertion is free to choose other cuts if those cuts may improve compliance with fat via rules.

r

If true, via ladder insertion will strictly honor the cutNameTbl in the ViaRule via ladder template at all times.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_via_ladder_constraints](#)
- [remove_via_ladder_constraints](#)
- [insert_via_ladders](#)
- [remove_via_ladders](#)
- [verify_via_ladders](#)
- [refresh_via_ladders](#)
- [set_via_ladder_rules](#)
- [route_auto](#)
- [route_eco](#)
- [route_opt](#)
- [route_group](#)

route.auto_via_ladder.top_layer_to_samenet_min_nonzero_distance

Specifies the preferred distance of via ladder shapes from top layer preroutes when a direct connection is not possible.

This option only has an impact when `allow_samenet_intersection` is true and `bias_top_layer_to_samenet_connection` is true.

Please see documentation for `bias_top_layer_to_samenet_connection`.

This option controls via ladder insertion during the automatic via ladder updating that can be activated with `route.auto_via_ladder.update_during_route` true or the `refresh_via_ladders` command.

Data Types

float

r

Default 0**Description**

Specifies the preferred distance of via ladder shapes from top layer preroutes when a direct connection is not possible.

This option only has an impact when `allow_samenet_intersection` is true and `bias_top_layer_to_samenet_connection` is true.

Please see documentation for `bias_top_layer_to_samenet_connection`.

If a legal directly-overlapping top layer connection is not possible and a `top_layer_to_samenet_min_nonzero_distance` value is specified, we will try to create a top layer row at least this distance from any samenet layout on the top layer. If this distance is not supplied or is 0, a row may be created any legal distance from samenet layout when a direct connection is impossible.

The distance is specified in user units.

If distance is supplied and a legal solution cannot be found, we will aim to create any legal via ladder that satisfies all other constraints. I.e. we would create a via ladder as if `bias_top_layer_to_samenet_connection` false.

The default is 0.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_via_ladder_constraints](#)
- [remove_via_ladder_constraints](#)
- [insert_via_ladders](#)
- [remove_via_ladders](#)
- [verify_via_ladders](#)
- [refresh_via_ladders](#)
- [set_via_ladder_rules](#)
- [route_auto](#)
- [route_eco](#)

r

- [route_opt](#)
- [route_group](#)

route.auto_via_ladder.update_during_route

Activates an automatic via ladder updating step at the beginning of any routing command that runs through global routing, track assignment, and detailed routing.

Data Types

Boolean

Default true

Description

The option activates a via ladder refresh step at the beginning of any routing command that runs through global routing, track assignment, and detailed routing. This includes `route_eco`, `route_group`, and `route_auto`. This via ladder refresh step ensures that via ladders in a design are compliant with via ladder constraints. This refresh step is a compound sequence that will execute `verify_via_ladders`, remove any via ladders that would lead to "Misplaced via ladder" DRCs, and `insert_via_ladders` to insert via ladders where needed.

The `refresh_via_ladders` command executes the same internal flow as the automatic via ladder updating in routing commands, but does so as a standalone command.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_via_ladder_constraints](#)
- [remove_via_ladder_constraints](#)
- [insert_via_ladders](#)
- [remove_via_ladders](#)
- [verify_via_ladders](#)
- [refresh_via_ladders](#)
- [set_via_ladder_rules](#)
- [route_auto](#)

r

- [route_eco](#)
- [route_opt](#)
- [route_group](#)

route.auto_via_ladder.update_on_route_eco_nets_only

Specifies whether if via ladder update only applied on specified nets during route_eco.

The default is false.

Data Types

Boolean

Default false

Description

Specifies whether if via ladder update only applied on specified nets during route_eco.

If true, only the nets specified in route_eco -nets would be updated. Pins on other nets won't be updated during via ladder stage.

If false, all the pins with via ladder constraint would be updated.

This option only affects route_eco, the other commands would keep original behavior.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_via_ladder_constraints](#)
- [remove_via_ladder_constraints](#)
- [set_via_ladder_rules](#)
- [route_eco](#)

route.auto_via_ladder.update_on_route_group_nets_only

Specifies whether if via ladder update only applied on specified nets during route_group.

The default is true.

r

Data Types

Boolean

Default true

Description

Specifies whether if via ladder update only applied on specified nets during route_group.

If true, only the nets specified in route_group -nets would be updated. Pins on other nets won't be updated during via ladder stage.

If false, all the pins with via ladder constraint would be updated.

This option only affects route_group, the other commands would keep original behavior.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_via_ladder_constraints](#)
- [remove_via_ladder_constraints](#)
- [set_via_ladder_rules](#)
- [route_group](#)

route.auto_via_ladder.update_pattern_must_join_during_route

According to the pin attribute, pattern must join, it will activate an automatic via ladder updating step at the beginning of any routing command that runs through global routing, track assignment, and detailed routing.

Data Types

Boolean

Default true

Description

For pin with pattern must join attribute, an auto-generated via ladder template is attached to the end of its via ladder constraints. Similar to the other option, route.auto_via_ladder.update_during_route. The option activates a via ladder refresh step at the beginning of any routing command that runs through global routing, track

r

assignment, and detailed routing. This includes `route_eco`, `route_group`, and `route_auto`. This via ladder refresh step ensures that via ladders in a design are compliant with via ladder constraints. This refresh step is a compound sequence that will execute `verify_via_ladders`, remove any via ladders that would lead to "Misplaced via ladder" DRCs, and `insert_via_ladders` to insert via ladders where needed.

The `refresh_via_ladders` command executes the same internal flow as the automatic via ladder updating in routing commands, but does so as a standalone command.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_via_ladder_constraints](#)
- [remove_via_ladder_constraints](#)
- [insert_via_ladders](#)
- [remove_via_ladders](#)
- [verify_via_ladders](#)
- [refresh_via_ladders](#)
- [set_via_ladder_rules](#)
- [route_auto](#)
- [route_eco](#)
- [route_opt](#)
- [route_group](#)

route.auto_via_ladder.user_debug

Specifies whether to print additional information about via ladder insertion failures.

The default is false.

This option controls via ladder insertion during the automatic via ladder updating that can be activated with `route.auto_via_ladder.update_during_route` true or the `refresh_via_ladders` command.

r

Data Types

Boolean

Default false

Description

If true, information on track restrictions and flagged DRC types will be output any time via ladder insertion fails on a pin. This information can be useful to understanding why via ladder insertion has failed.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_via_ladder_constraints](#)
- [remove_via_ladder_constraints](#)
- [insert_via_ladders](#)
- [remove_via_ladders](#)
- [verify_via_ladders](#)
- [refresh_via_ladders](#)
- [set_via_ladder_rules](#)
- [route_auto](#)
- [route_eco](#)
- [route_opt](#)
- [route_group](#)

route.auto_via_ladder.verbose

Specifies whether the success or failure status of each attempted via ladder insertion is printed.

The default is false.

r

This option controls via ladder insertion during the automatic via ladder updating that can be activated with `route.auto_via_ladder.update_during_route` true or the `refresh_via_ladders` command.

Data Types

Boolean

Default false

Description

Specifies whether the success or failure status of each attempted via ladder insertion is printed.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_via_ladder_constraints](#)
- [remove_via_ladder_constraints](#)
- [insert_via_ladders](#)
- [remove_via_ladders](#)
- [verify_via_ladders](#)
- [refresh_via_ladders](#)
- [set_via_ladder_rules](#)
- [route_auto](#)
- [route_eco](#)
- [route_opt](#)
- [route_group](#)

route.common.advance_node_timing_driven_mode

Specifies timing-driven routing is running `advance_node` mode.

Data Types

boolean

r

Default false**Description**

This application option specifies advance node timing-driven routing is running in advance node mode. The advance node mode timing driven route is more resistance aware and reduces the high resistive metal stack usage. The advance node mode is useful when the metal stacks resistance are significantly different and For some nodes, the `advance_node_timing_driven_mode` is set to true through `set_technology`. This option takes effect in all routing stages.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [route_global](#)
- [route_track](#)
- [route_detail](#)
- [route_auto](#)
- [set_technology](#)

route.common.allow_pg_as_shield

Controls whether the router treats power and ground nets as shields.

Data Types

Boolean

Default true**Description**

This application option controls whether the router treats power and ground nets as shields.

When *true* (the default), the router treats power and ground nets as shields and does not reserve additional space around the nets to be shielded.

If set to *false*, the router tries to reserve space around the nets to be shielded if they are close to a power or ground net.

Note that this option changes the routing behavior.

r

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)

route.common.check_shield

Controls whether the shield data checker is enabled in all Zroute commands (except `check_routability`).

Data Types

Boolean

Default True

Description

When this option is true, all Zroute commands (except `check_routability`) do various data checks related to shields at the start of command execution. When an illegal or suspicious input shape is detected, an error or warning would be issued. This data checker checks the attributes of routing and shield shapes for known problems in shielding flow. It does not check the correctness of shielding shapes.

For `check_routability` command, shield data checker is enabled by *`check_routability -check_shield true`*.

Examples

The following example enables the shield data checker in all Zroute commands (except `check_routability`).

```
prompt> set_app_options \  
-name route.common.check_shield \  
-value true
```

See Also

- [check_routability](#)
- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)

r

route.common.clock_net_max_layer_mode

Specifies the default mode for clock nets with net-based maximum routing layer constraints set by the *set_routing_rule* command.

Data Types

string

Default unknown

Description

This application option specifies the default mode for clock nets with net-based maximum routing layer constraints set by the *set_routing_rule* command. This setting applies only to clock nets whose maximum routing layer mode is not set by the *set_routing_rule* command.

The definition for each mode is

- *unknown* (the default)

Defer to general *route.common.net_max_layer_mode* to determine effective mode

- *hard* (the default)

No routing can occur above the maximum layer. DRC violations are flagged on metal above the maximum layer.

- *soft*

Routing above the maximum layer is discouraged, but not disallowed. DRC violations are not flagged on any metal above the maximum layer.

- *allow_pin_connection*

Routing is allowed above the maximum layer only for pin connections or to fix DRC violations in extremely congested areas. DRC violations are flagged on metal above the maximum layer in the following cases:

- The metal is not along a path that connects to a pin.
- The metal is along a path that connects to a pin and the path is long.

The detail router considers a path to be long if its length exceeds approximately 10 average track pitches. The *check_routes* command considers a path to be long if its length exceeds approximately 15 average track pitches.

Note that the *route.common.number_of_vias_over_net_max_layer* option can relax the maximum layer constraint, but only for vias.

r

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)

route.common.clock_net_min_layer_mode

Specifies the default mode for clock nets with net-based minimum routing layer constraints set by the *set_routing_rule* command.

Data Types

string

Default unknown

Description

This application option specifies the default mode for clock nets with net-based minimum routing layer constraints set by the *set_routing_rule* command. This setting applies only to clock nets whose minimum routing layer mode is not set by the *set_routing_rule* command.

The definition for each mode is

- *unknown* (the default)

Defer to general *route.common.net_min_layer_mode* to determine effective mode

- *soft*

Routing below the minimum layer is discouraged, but not disallowed. DRC violations are not flagged on any metal below the minimum layer.

- *allow_pin_connection*

r

Routing is allowed below the minimum layer only for pin connections or to fix DRC violations in extremely congested areas. DRC violations are flagged on metal below the minimum layer in the following cases:

- The metal is not along a path that connects to a pin.
- The metal is along a path that connects to a pin and the path is long.

The detail router considers a path to be long if its length exceeds approximately 10 average track pitches. The *check_routes* command considers a path to be long if its length exceeds approximately 15 average track pitches.

- *hard*

No routing can occur below the minimum layer. DRC violations are flagged on metal below the minimum layer.

Note that the *route.common.number_of_vias_under_net_min_layer* option can relax the minimum layer constraint, but only for vias.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)

route.common.clock_net_min_max_layer_distance_threshold

Applies to clock nets only and overrides the allowable path length for routing below (above) the min (max) layer when routing in allow_pin_connection mode.

If 0.0 (unset) then control will be deferred to the more general option common.route.min_max_layer_distance_threshold that impacts all nets.

If that option is also unset, then an internally hard-coded distance threshold will be used as described below.

Data Types

Float

Default 0.0 (no impact)

r

Description

The `clock_net_min_max_layer_distance_threshold` option applies only to routing impacted by `min/max layer mode allow_pin_connection`.

The man page descriptions for the following common options can be consulted for a description of that mode: `net_min_layer_mode`, `net_max_layer_mode`, `global_min_layer_mode`, `global_max_layer_mode`.

When routing a clock net in that mode, the router will flag violations on some paths that are routed below (above) the min (max) layer. By default (if neither `clock_net_min_max_layer_distance_threshold` nor `min_max_layer_distance_threshold` specifies a distance threshold), DRC violations are flagged on metal below the minimum layer if the metal is along a path that connects to a pin and the path is long. A path is considered long by the detail router if its length exceeds approximately 10 average track pitches. A path is considered long by `check_routes` if its length exceeds approximately 15 average track pitches.

The option `clock_net_min_max_layer_distance_threshold` will override the default value of 10 or 15 average track pitches for clock nets only. The option value is a distance in the `unitLengthName` type specified in the tech file.

If the option `clock_net_min_max_layer_distance_threshold` value is set, the set value will be used by both the detailed router and `check_routes`. This can result in minor differences in "Min-max layer" DRC counts between detailed routing and `check_routes`. This is because slight variations in internal net representations can cause path lengths to vary slightly across the commands. Path lengths very close to the length threshold may be reported as a violation in one command but not the other. This is expected behavior.

For clock nets the precedence to determine the effective distance threshold is:

1) `clock_net_min_max_layer_distance_threshold`. If unset (0.0), then check: 2) `min_max_layer_distance_threshold`. If unset (0.0), then: 3) Use hard-coded values of 10 or 15 average track pitches as described above.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)

route.common.clock_topology

Specifies the clock routing topology.

r

Data Types

string

Default normal

Description

This application option specifies the clock routing topology.

Valid values are

- *normal* (default)
Use normal routing for clock nets.
- *comb*
Use comb routing for clock nets with a mesh or trunk.

Examples

The following example sets the *route.common.clock_topology* application option to *comb*.

```
prompt> set_app_options -name route.common.clock_topology -value comb
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)

route.common.color_based_dpt_flow

This application option implies if color based dpt flow is enabled.

Data Types

Boolean

Default false

Description

When *false* (the default), color based dpt flow is disabled. When *true*, color based dpt flow is enabled.

Application options are set using the *set_app_options* command, and specified globally.

r

route.common.comb_distance

Specifies the number of global routing cells within which to connect clock pins to the clock mesh.

Data Types

integer

Default 2

Description

This application option specifies the number of global routing cells within which to connect clock pins to the clock mesh. You must specify a value between 0 and 10, inclusive.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)

route.common.concurrent_redundant_via_effort_level

Specifies the effort level for the concurrent redundant via insertion flow.

Data Types

string

Default low

Description

This application option specifies the effort level for the concurrent redundant via insertion flow.

Valid values are *low* (the default), *medium*, and *high*.

The default effort level is *low*. The higher effort levels result in a better redundant via conversion rate at the expense of runtime.

This option takes effect only if the *route.common.concurrent_redundant_via_mode* option is set to a value other than *off*.

r

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)

route.common.concurrent_redundant_via_mode

Specifies the concurrent redundant via insertion mode for initial routing.

Data Types

string

Default off

Description

This application option specifies the concurrent redundant via insertion mode for initial routing.

The valid modes are

- *off* (the default)

Concurrent redundant via insertion is turned off.

- *reserve_space*

Concurrent redundant via insertion reserves space for redundant vias but does not instantiate them. The space reservation is done by generating internal soft rules for redundant vias and performing soft-rule-based optimization. The actual instantiation is done later by explicitly calling the *add_redundant_vias* command or by setting the *route.common.post_detail_route_redundant_via_insertion* option to a value other than *off*.

- *insert_at_high_cost*

In addition to the internal soft rules generated in *reserve_space* mode, concurrent redundant via insertion creates hard design rules for redundant vias and tries to insert them. Use this mode if only you need a nearly 100 percent conversion rate. It can lead to very long runtimes if the design is congested or the technology parameters are not friendly to redundant via insertion. This mode should be used with caution.

r

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)

route.common.connect_floating_shapes

Specifies whether the router should connect just the pins or the pins and all floating shapes.

Data Types

Boolean

Default false

Description

This application option specifies whether the router should connect just the pins or the pins and all floating shapes.

When *false* (the default), the router connects just the pins. The router might utilize the floating shapes; however, it does not have to connect the floating shapes.

When *true*, the router connects the pins and all floating shapes.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)

route.common.connect_within_pins_by_layer_name

Specifies, for each layer, whether to connect to pins by using vias or wires contained within the pin shapes.

Data Types

list

Default ""

r

Description

This application option specifies, for each layer, whether to connect to pins by using vias or wires contained within the pin shapes. You can specify this option only when the top-level block is open.

The syntax for specifying the setting for each layer is

```
{routing_layer mode}
```

You must specify the routing layers by using the layer names in the technology file.

The valid mode values are

- *off* (the default)
Connect by using wires or vias anywhere on the pins.
- *via_standard_cell_pins*
Connect to standard cell pins by using vias contained within the pin shapes.
- *via_wire_standard_cell_pins*
Connect to standard cell pins by using vias and wires contained within the pin shapes.
- *via_all_pins*
Connect to any pins by using vias contained within the pin shapes.
- *via_wire_all_pins*
Connect to any pin by using vias and wires contained within the pin shapes.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)

route.common.create_nets_for_floating_pins_pattern_must_join_via_ladder

Specifies whether to create new net for floating pattern must join pin.

The default is false.

Data Types

Boolean

r

Default false**Description**

Specifies whether to create new net for floating pattern must join pin.

If true: At the beginning of all Zroute commands, the tool will create new NDM net for pin without a net, if it has pattern_must_join attribute or via ladder constraint.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_via_ladder_constraints](#)
- [remove_via_ladder_constraints](#)
- [insert_via_ladders](#)
- [remove_via_ladders](#)
- [verify_via_ladders](#)
- [refresh_via_ladders](#)
- [set_via_ladder_rules](#)
- [route_auto](#)
- [route_eco](#)
- [route_opt](#)
- [route_group](#)
- [check_routes](#)

route.common.crosstalk_driven

Controls whether crosstalk driven flow is enabled for routing commands.

Data Types

Boolean

Default false

r

Description

This application option enables (true) or disables (false) crosstalk driven capability in global router, track assignment and detailed router. Internal soft rules are applied on crosstalk critical nets during crosstalk driven routing.

The option enables several other router options if they are not set by the user. The key options are: `route.track.timing_driven`, `route.track.crosstalk_driven`, `route.detail.timing_driven`. For correct functionality it is important not to disable these options explicitly in the script.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [route_detail](#)
- [route_track](#)
- [route_auto](#)
- [route_eco](#)

route.common.eco_fix_drc_in_changed_area_only

Enable change area only eco detail route.

Data Types

Boolean

Default false

Description

The application option enable change area only eco detail routing. The change regions are auto detected in `route_opt` and `route_eco` command. For `route_opt` command, cell/GR/VL changes are auto detected. For `route_eco` command, GR/VL changes are auto detected. DRC checking are limited to the changes. There is no additional routing constraint to ensure DRC converged.

r

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)

route.common.eco_route_concurrent_redundant_via_effort_level

Specifies the effort level for the concurrent redundant via insertion flow in ECO routing.

Data Types

string

Default low

Description

This application option specifies the effort level for the concurrent redundant via insertion flow in ECO routing.

The valid values are *low*, *medium*, and *high*. The default effort level is *low*. The higher effort levels result in a better redundant via conversion rate at the expense of runtime.

This option takes effect only if the *route.common.eco_concurrent_redundant_via_mode* option is set to a value other than *off*.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)

route.common.eco_route_concurrent_redundant_via_mode

Specifies the concurrent redundant via insertion mode for ECO routing.

Data Types

string

Default off

r

Description

This application option specifies the concurrent redundant via insertion mode for ECO routing.

The definition for each mode is

- *off* (the default)

Concurrent redundant via insertion is turned off.

- *reserve_space*

Concurrent redundant via insertion reserves space for redundant vias in ECO routing but does not instantiate them. The space reservation is done by generating internal soft rules for redundant vias and performing soft-rule-based optimization. The actual instantiation is done later by explicitly calling the *add_redundant_vias* command or by setting the *route.common.post_detail_route_redundant_via_insertion* option to a value other than *off*.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)

route.common.eco_route_fix_existing_drc

Specifies if we should rip-up reroute to fix the existing DRCs during eco route.

Data Types

Boolean

Default *true*

Description

This application option controls whether the router rips up and reroutes to fix the existing DRCs (i.e., DRCs stored in the error cell of the block) during eco route.

When *true* (the default), the router rips up and reroutes to fix all the DRCs, both existing DRCs and the new DRCs found after DRC check during eco route. When *false*, the router does not rip-up reroute for existing DRCs but does rip-up reroute for new DRCs found during DRC check. It is possible that when we rip-up reroute for new DRCs, the existing DRCs could get resolved in the process. This option is most useful when we have lots of existing DRCs in the block and we perform small eco changes and do not want the router

r

to spend huge runtime to resolve the existing DRCs. This setting is expected to reduce the eco route run time but it will most likely result in higher final DRCs.

Application options are set using the `set_app_options` command, and specified globally.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)

route.common.enable_auto_dirty_data_handling

App option to enable dirty data handling features, aimed at improving the Zroute runtime performance in designs containing dirty data.

Data Types

boolean

Default true

Description

When this app option is enabled, any dirty data will be logged and handled accordingly to accelerate runtime performance in Zroute.

See Also

- [get_app_option_value](#)
- [set_app_options](#)
- [route_detail](#)
- [route_group](#)
- [route_auto](#)

route.common.enable_explicit_cut_metal_generation

Enable the explicit cut metal generation flow.

Data Types

boolean

Default false

r

Description

With the enablement of this application option, cut metal shapes are generated explicitly and updated incrementally during detail routing. Please note that this option is only needed if there are cut metal rules like same color cut metal space. And this option is not to be used with the `create_cut_metals` standalone command.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)

route.common.enable_multi_thread

Controls whether multi-threading is enabled for routing commands.

Data Types

Boolean

Default `true`

Description

When *true* (the default), multi-threading is enabled for routing commands. The number of used threads is controlled by host options settings. When *false*, multi-threading is disabled for routing commands.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [report_host_options](#)
- [set_host_options](#)

route.common.exclude_routing_constraints_for_short_nets_threshold

Set the threshold of short nets to waive NDR, minimum, and maximum layer constraints.

r

Data Types

float

Default 0.0**Description**

This application option specifies the threshold of short nets to waive NDR, minimum, and maximum layer constraints during routing. The default value 0, disables the feature. The value must be a number between 0 to 10 and the unit is micron.

See Also

- [set_routing_rule](#)
- [report_routing_rules](#)
- [create_routing_rule](#)

route.common.extra_nonpreferred_direction_wire_cost_multiplier_by_layer_name

Specifies the cost multiplier for routing in the nonpreferred direction.

Data Types

list

Default ""**Description**

This application option specifies the cost multiplier for routing in the nonpreferred direction. You can specify this option only when the top-level block is open.

The format to specify the cost multiplier for a routing layer is

```
{layer multiplier}
```

You must specify the routing layers by using the layer names in the technology file.

The multiplier must be a float value between 0.0 and 20.0. The multiplier affects the routing cost in the nonpreferred direction as follows:

```
cost = baseCost * (1 + multiplier)
```

If you specify a large multiplier value, it can prevent the router from routing in the nonpreferred direction. If your design requires nonpreferred direction routing, use smaller multiplier values.

r

If you do not specify the multiplier for a layer, the tool uses the default of 0.0.

Examples

The following example shows how to set the non-preferred direction wire cost multiplier by layer name.

```
prompt> set_app_options -block [current_block] \\  
-name  
route.common.extra_nonpreferred_direction_wire_cost_multiplier_by_layer_  
name \\  
-value { {M4 3} {M5 2} }
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)

route.common.extra_preferred_direction_wire_cost_multiplier_by_layer_name

Specifies the cost multiplier for routing in the preferred direction.

Data Types

list

Default ""

Description

This application option specifies the cost multiplier for routing in the preferred direction. You can specify this option only when the top-level block is open.

The format to specify the cost multiplier for a routing layer is

```
{layer multiplier}
```

You must specify the routing layers by using the layer names in the technology file.

The multiplier must be a float value between 0.0 and 20.0. The multiplier affects the routing cost in the preferred direction as follows:

```
cost = baseCost * (1 + multiplier)
```

If you do not specify the multiplier for a layer, the tool uses the default of 0.0.

r

Examples

The following example shows how to set the preferred direction wire cost multiplier by layer name.

```
prompt> set_app_options -block [current_block] \\  
-name  
route.common.extra_preferred_direction_wire_cost_multiplier_by_layer_n  
ame \\  
-value { {M4 3} {M5 2} }
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)

route.common.extra_via_cost_multiplier_by_layer_name

Specifies the cost multiplier for via layers.

Data Types

list

Default ""

Description

This application option specifies the cost multiplier for via layers. The router uses via layers with a lower cost before via layers with a higher cost. To influence the layers used for vias, you can set relative costs for the via layers; the higher the cost, the less likely that the router will place vias on that layer.

You can specify this option only when the top-level block is open.

The format to specify the cost multiplier for a via layer is

```
{layer multiplier}
```

You must specify the via layers by using the layer names in the technology file.

The multiplier must be a float value between 0.0 and 20.0. It affects the via cost on the specified layer as follows:

```
viaCost = baseViaCost * (1 + multiplier)
```

r

If you do not specify the multiplier for a layer, the tool uses the default of 0.0.

Examples

The following example sets the extra via cost multiplier to 0.5 on the VIA12 layer and to 1.5 on the VIA23 layer. Increasing the cost of the VIA12 and VIA23 layers avoids the placement of vias on these layers.

```
prompt> set_app_options \\  
        -name route.common.extra_via_cost_multiplier_by_layer_name \\  
        -value {{VIA12 0.5} {VIA23 1.5}}
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)

route.common.extra_via_off_grid_cost_multiplier_by_layer_name

Specifies the cost multiplier for vias that are off grid with respect to the specified metal layers.

Data Types

list

Default ""

Description

This application option specifies the cost multiplier for vias. You can specify this option only when the top-level block is open.

The format to specify the cost multiplier for a via layer is

```
{layer multiplier}
```

You must specify the via layers by using the layer names in the technology file.

The multiplier must be a float value between 0.0 and 20.0. It affects the via cost on the specified layer as follows:

```
viaCost = baseViaCost * (1 + multiplier)
```

If you do not specify the multiplier for a layer, the tool uses the default of 0.0.

r

Examples

If the technology file defines the VIA12 via layer between the M1 and M2 metal layers and the VIA23 via layer between the M2 and M3 metal layers, the following example sets the extra off-grid via cost multiplier on the VIA12 and VIA23 layers to 0.5. Note that you specify the metal layer, but the multiplier is set on the adjacent via layers.

```
prompt> set_app_options \\  
-name route.common.extra_via_off_grid_cost_multiplier_by_layer_name \\  
-value {{M2 0.5}}
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)

route.common.filter_redundant_via_mapping

Specifies if redundant via insertion should simplify the redundant via mapping to potentially improve runtime.

Data Types

Boolean

Default true

Description

Specifies if redundant via insertion should simplify the redundant via mapping to potentially improve runtime.

If true (default), redundant via insertion may ignore some vias in the mapping if they are judged to be more likely to create DRCs than other vias in the same via mapping weight group.

For example, a via A in the same weight group as via B may be removed from the mapping if via A has larger surrounds on both layers. Some DRC rules may also be considered when deciding if a via should be filtered from the mapping. The aim is to simplify the via mapping and achieve faster runtimes without negatively impacting redundant via insertion rates.

If false, the redundant via mapping is not filtered.

r

See Also

- [add_via_mapping](#)
- [add_redundant_vias](#)
- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)

route.common.focus_scenario

Specify the dominant scenario for timing driven route.

Data Types

string

Default none

Description

This application option specifies the dominant scenario for timing driven route. Timing driven route assigns layer based on the dominant scenario's unit resistance and unit capacitance. By default, the tool picked the dominant scenario based on timing QoR per scenario.

values are scenario name This option takes effect only if timing-driven routing is enabled.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)

route.common.forbid_new_metal_by_layer_name

Controls whether ECO routing with the *route_eco* command can remove, but not add, metal on the specified layers. You must specify the routing layers by using the layer names in the technology file.

Data Types

list

Default ""

r

Description

This application option controls whether ECO routing with the *route_eco* command can remove, but not add, metal on the specified routing layers. You can set this option only when the block is open.

Note that for all other routing commands, this option acts the same as the *route.common.freeze_layer_by_layer_name* option and controls whether routing cannot add or remove metal on the specified layers.

The format for specifying the setting for a layer is

```
{routing_layer true|false}
```

You must specify the routing layers by using the layer names in the technology file.

When *false* (the default), the router can add or remove metal from the layer.

When *true*, the ECO router can remove, but not add, metal from the layer. The ECO router can remove metal if it is dangling or is not fully contained within fixed user shapes, such as pins. All other routing phases cannot add or remove metal from the layer. If the router adds metal on the layer, and that metal is not fully contained within a fixed user shape, the router reports a "Forbidden metal" DRC violation.

You would use this option if you want to freeze a layer when running core commands, such as *route_opt*, but give the router the flexibility to remove some shapes during ECO routing. If a layer is frozen with the *route.common.freeze_layer_by_layer_name* option, the router cannot touch any shapes, which can result in dangling wires and unfixable shorts if the placement changes during ECO. The *route.common.forbid_new_metal_by_layer_name* option gives the router the flexibility to remove such shapes while otherwise treating the layer as frozen.

If you specify both the *route.common.freeze_layer_by_layer_name* and *route.common.forbid_new_metal_by_layer_name* options, the stricter *route.common.freeze_layer_by_layer_name* option takes precedence.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)

route.common.freeze_layer_by_layer_name

Controls whether routing layers are frozen to prevent them from changing during routing.

r

Data Types

list

Default ""**Description**

This application option controls whether routing layers are frozen to prevent them from changing during routing. You can specify this option only when the block is open.

Use the following format to specify the setting for each layer:

```
{routing_layer true|false}
```

You must specify the routing layers by using the layer names in the technology file.

When *false* (the default), the layer is not frozen.

When *true*, the layer is frozen.

Examples

The following example sets the *route.common.freeze_layer_by_layer_name* application option to *true* for the m3 and m4 metal layers.

```
prompt> set_app_options \\  
-name route.common.freeze_layer_by_layer_name \\  
-value {{m3 true} {m4 true}}
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)

route.common.freeze_via_to_frozen_layer_by_layer_name

Controls the treatment of vias that touch the specified frozen layers on one side.

Data Types

list

Default ""**Description**

This application option controls the treatment of vias that touch the specified frozen layers on one side.

r

Use the following format to specify the setting for each layer:

```
{routing_layer true|false}
```

You must specify the routing layers by using the layer names in the technology file. The specified layers must be frozen layers.

By default, all the frozen layers are set as *false*, which indicates that the vias whose surroundings on a frozen layer are enclosed by other metal shapes can be removed or rerouted if they do not make a change on the frozen metal layer. Special vias such as H-shape vias are not removed or rerouted even though they do not change the frozen metal layer.

When you set this option to *true* on one or more frozen layers, the vias adjacent to these frozen layers are frozen and cannot be changed. Note that no change is allowed on a cut layer adjacent to two frozen metal layers.

Examples

The following example shows how to set the option freeze via on frozen layer by layer name:

```
prompt> set_app_options -block [current_block] \\  
-name route.common.freeze_via_to_frozen_layer_by_layer_name \\  
-value { {MET4 true} {MET5 true} }
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)

route.common.global_backside_max_layer_mode

Controls the application of the maximum backside routing layer constraint set for the design.

Data Types

string

Default soft

r

Description

This application option controls the application of the maximum backside routing layer constraint set for the design. This global constraint is set by using the *set_ignored_layers* command.

The definition for each mode is

- *soft* (the default)

Routing above the maximum backside layer is discouraged, but not disallowed. DRC violations are not flagged on any metal above the maximum backside layer.

- *allow_pin_connection*

Routing is allowed above the maximum backside layer only for pin connections or to fix DRC violations in extremely congested areas. DRC violations are flagged on metal above the maximum backside layer in the following cases:

- The metal is not along a path that connects to a pin.
- The metal is along a path that connects to a pin and the path is long.

The detail router considers a path to be long if its length exceeds approximately 10 average track pitches. The *check_routes* command considers a path to be long if its length exceeds approximately 15 average track pitches.

- *hard*

No routing can occur above the maximum backside layer. DRC violations are flagged on metal above the maximum backside layer.

Note that the *route.common.number_of_vias_over_global_backside_max_layer* option can relax the maximum backside layer constraint, but only for vias.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)

route.common.global_backside_min_layer_mode

Controls the application of the minimum backside routing layer constraint for the design.

r

Data Types

string

Default soft

Description

This application option controls the application of the minimum backside routing layer constraint for the design. This global constraint is set by using the *set_ignored_layers* command.

The definition for each mode is

- *soft* (the default)

Routing below the minimum backside layer is discouraged, but not disallowed. DRC violations are not flagged on any metal below the minimum backside layer.

- *allow_pin_connection*

Routing is allowed below the minimum backside layer only for pin connections or to fix DRC violations in extremely congested areas. DRC violations are flagged on metal below the minimum backside layer in the following cases:

- The metal is not along a path that connects to a pin.
- The metal is along a path that connects to a pin and the path is long.

The detail router considers a path to be long if its length exceeds approximately 10 average track pitches. The *check_routes* command considers a path to be long if its length exceeds approximately 15 average track pitches.

- *hard*

No routing can occur below the minimum backside layer. DRC violations are flagged on metal below the minimum backside layer.

Note that the *route.common.number_of_vias_under_global_backside_min_layer* option can relax the minimum backside layer constraint, but only for vias.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)

r

route.common.global_max_layer_mode

Controls the application of the maximum routing layer constraint set for the design.

Data Types

string

Default soft

Description

This application option controls the application of the maximum routing layer constraint set for the design. This global constraint is set by using the *set_ignored_layers* command.

The definition for each mode is

- *soft* (the default)

Routing above the maximum layer is discouraged, but not disallowed. DRC violations are not flagged on any metal above the maximum layer.

- *allow_pin_connection*

Routing is allowed above the maximum layer only for pin connections or to fix DRC violations in extremely congested areas. DRC violations are flagged on metal above the maximum layer in the following cases:

- The metal is not along a path that connects to a pin.
- The metal is along a path that connects to a pin and the path is long.

The detail router considers a path to be long if its length exceeds approximately 10 average track pitches. The *check_routes* command considers a path to be long if its length exceeds approximately 15 average track pitches.

- *hard*

No routing can occur above the maximum layer. DRC violations are flagged on metal above the maximum layer.

Note that the *route.common.number_of_vias_over_global_max_layer* option can relax the maximum layer constraint, but only for vias.

See Also

- [get_app_option_value](#)
- [get_app_options](#)

r

- [report_app_options](#)
- [set_app_options](#)

route.common.global_min_layer_mode

Controls the application of the minimum routing layer constraint for the design.

Data Types

string

Default `soft`

Description

This application option controls the application of the minimum routing layer constraint for the design. This global constraint is set by using the *set_ignored_layers* command.

The definition for each mode is

- *soft* (the default)

Routing below the minimum layer is discouraged, but not disallowed. DRC violations are not flagged on any metal below the minimum layer.

- *allow_pin_connection*

Routing is allowed below the minimum layer only for pin connections or to fix DRC violations in extremely congested areas. DRC violations are flagged on metal below the minimum layer in the following cases:

- The metal is not along a path that connects to a pin.
- The metal is along a path that connects to a pin and the path is long.

The detail router considers a path to be long if its length exceeds approximately 10 average track pitches. The *check_routes* command considers a path to be long if its length exceeds approximately 15 average track pitches.

- *hard*

No routing can occur below the minimum layer. DRC violations are flagged on metal below the minimum layer.

Note that the *route.common.number_of_vias_under_global_min_layer* option can relax the minimum layer constraint, but only for vias.

r

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)

route.common.high_resistance_flow

Specifies whether to enable the high-resistance feature during timing driven route.

Data Types

Boolean

Default true

Description

When set to true, enables the high-resistance feature for timing driven routing. The high-resistance feature is especially targeted for emerging nodes that are 20 nm and below where the net resistance is large comparing to the cell resistance. This feature tries to reduce the resistance by maintaining a balance between metal wire resistance and via resistance through changes in routing.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)

route.common.ignore_var_spacing_to_blockage

Controls whether the spacing defined in a nondefault routing rule is ignored against blockages.

Data Types

Boolean

Default false

Description

Note that this option changes the routing behavior.

r

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)

route.common.ignore_var_spacing_to_pg

Controls whether the spacing defined in a nondefault routing rule is ignored against all wires on power and ground nets.

Data Types

Boolean

Default false

Description

Note that this option changes the routing behavior.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)

route.common.ignore_var_spacing_to_shield

Controls whether the spacing defined in a nondefault routing rule is ignored against shield wires on power and ground nets.

Data Types

Boolean

Default true

Description

Note that this option changes the routing behavior.

r

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)

route.common.mark_clock_nets_minor_change

Controls whether changes on clock tree nets are limited during routing.

Data Types

Boolean

Default true

Description

This application option controls whether changes on clock tree nets are limited during routing.

When *true* (the default), the router can make only limited changes to nets marked as clock tree nets in the database.

When *false*, nets marked as clock tree nets in the database are routed normally (the router can make any changes to these nets).

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)

route.common.min_edge_offset_for_macro_pin_connection_by_layer_name

Specifies the minimum edge offset for macro pin connections for one or more metal layers.

Data Types

list

Default ""

r

Description

Specifies the minimum edge offset for macro pin connections for one or more metal layers. The minimum edge for macro pin connection rule enforces exactly aligned connections between metal wires and macro pins where they connect. If the macro pin and wire have different widths, the short edges resulting at the connection point must have at least the specified length. The unit of offset is in microns.

The settings specified in this option override the *minEdgeOffsetForMacroPinConnection* values specified in the Layer section of the technology file for the specified layers. The technology file values are used for any layers not specified in this option.

Use the following format to specify the setting for each layer:

```
{routing_layer offset}
```

You must specify the routing layers by using the layer names in the technology file.

Examples

The following example sets the *min_edge_offset_for_macro_pin_connection_by_layer_name* application option to 0.5 for the M3 and M4 metal layers.

```
prompt> set_app_options \\  
-name min_edge_offset_for_macro_pin_connection_by_layer_name \\  
-value {{M3 0.5} {M4 0.5}}
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)

route.common.min_edge_offset_for_top_level_pin_connection_by_layer_name

Specifies the minimum edge offset for top level pin connections for one or more metal layers.

Data Types

list

Default ""

r

Description

Specifies the minimum edge offset for top level pin connections for one or more metal layers. The minimum edge for top level pin connection rule enforces exactly aligned connections between metal wires and macro pins where they connect. If the top level pin and wire have different widths, the short edges resulting at the connection point must have at least the specified length.

Use the following format to specify the setting for each layer:

```
{routing_layer offset}
```

You must specify the routing layers by using the layer names in the technology file.

Examples

The following example sets the *min_edge_offset_for_top_level_pin_connection_by_layer_name* application option to 0.5 for the M3 and M4 metal layers.

```
prompt> set_app_options \\  
-name min_edge_offset_for_top_level_pin_connection_by_layer_name \\  
-value {{M3 0.5} {M4 0.5}}
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)

route.common.min_max_layer_distance_threshold

Overrides the allowable path length for routing below (above) the min (max) layer when routing in allow_pin_connection mode.

Data Types

Float

Default 0.0 (no impact)

Description

The min_max_layer_distance_threshold option applies only to routing impacted by min/max layer mode allow_pin_connection.

r

The man page descriptions for the following common options can be consulted for a description of that mode: `net_min_layer_mode`, `net_max_layer_mode`, `global_min_layer_mode`, `global_max_layer_mode`.

When routing in that mode, the router will flag violations on some paths that are routed below (above) the min (max) layer. For example, DRC violations are flagged on metal below the minimum layer if the metal is along a path that connects to a pin and the path is long. A path is considered long by the detail router if its length exceeds approximately 10 average track pitches. A path is considered long by `check_routes` if its length exceeds approximately 15 average track pitches.

The option `min_max_layer_distance_threshold` will override the default value of 10 or 15 average track pitches. The option value is a distance in the `unitLengthName` type specified in the tech file.

If the option `min_max_layer_distance_threshold` value is set, the set value will be used by both the detailed router and `check_routes`. This can result in minor differences in "Min-max layer" DRC counts between detailed routing and `check_routes`. This is because slight variations in internal net representations can cause path lengths to vary slightly across the commands. Path lengths very close to the length threshold may be reported as a violation in one command but not the other. This is expected behavior.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)

route.common.min_shield_length_by_layer_name

Specifies the minimum length of the wires to be shielded. The values are in microns by specified layers. The router skips shielding the wires with lengths less than specified values.

Data Types

list

Default The default length for each layer is 4-layer-pitch. Using -1 could set it back to the default value while a layer value.

Description

This application option specifies the minimum length of the wires to be shielded for each layer.

r

The format for specifying the minimum shield length for a layer is

```
{layer length}
```

Wires with a length that is less than the minimum shield length are always unshielded. After creating a shield for a net, when reporting shielding ratio in *create_net_shielding* command, wires that are less than the specified minimum shield length are not counted.

Specify the layer by using the layer name from the technology file.

Examples

The following example sets the *route.common.min_shield_length_by_layer_name* option on M1, M2, and M5.

```
prompt> set_app_options -list \\  
        {route.common.min_shield_length_by_layer_name \\  
          { {M1 0.1} {M2 0.18} {M5 0.256} }}
```

See Also

- [create_net_shielding](#)
- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [route_auto](#)
- [route_detail](#)
- [route_opt](#)
- [set_app_options](#)

route.common.ndr_by_delta_voltage

Voltage spacing is calculated based on the delta voltage for analog PG nets

Data Types

Boolean

Default false

Description

This application option controls whether the voltage spacing is calculated based on the delta voltage for analog PG nets.

r

When set to *true*, identify special analog PG nets *net_types* should be ground or power. Compute delta voltage max/min *voltage_range_min/voltage_range_max* of all *signal_nets* in the block against the negative voltage on the special nets. Apply NDR rule to the special PG nets by the calculated delta voltage

If set to *false*(the default), the router uses the absolute voltage.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)

route.common.net_max_layer_mode

Specifies the default mode for the net-based maximum routing layer constraints set by the *set_routing_rule* command.

Data Types

string

Default *hard*

Description

This application option specifies the default mode for the net-based maximum routing layer constraints set by the *set_routing_rule* command. This setting applies only to nets whose maximum routing layer mode is not set by the *set_routing_rule* command.

The definition for each mode is

- *hard* (the default)
No routing can occur above the maximum layer. DRC violations are flagged on metal above the maximum layer.
- *soft*
Routing above the maximum layer is discouraged, but not disallowed. DRC violations are not flagged on any metal above the maximum layer.
- *allow_pin_connection*

r

Routing is allowed above the maximum layer only for pin connections or to fix DRC violations in extremely congested areas. DRC violations are flagged on metal above the maximum layer in the following cases:

- The metal is not along a path that connects to a pin.
- The metal is along a path that connects to a pin and the path is long.

The detail router considers a path to be long if its length exceeds approximately 10 average track pitches. The *check_routes* command considers a path to be long if its length exceeds approximately 15 average track pitches.

Note that the *route.common.number_of_vias_over_net_max_layer* option can relax the maximum layer constraint, but only for vias.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)

route.common.net_max_layer_mode_soft_cost

Specifies the default cost for routing above the net-based maximum routing layer when using the soft maximum layer mode.

Data Types

string

Default medium

Description

This application option specifies the default cost for routing above the net-based maximum routing layer when using the soft maximum layer mode. Valid values are *low*, *medium*, and *high*.

This setting applies only to nets whose soft mode maximum routing layer cost is not set by the *set_routing_rule* command.

r

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)

route.common.net_min_layer_mode

Specifies the default mode for the net-based minimum routing layer constraints set by the *set_routing_rule* command.

Data Types

string

Default soft

Description

This application option specifies the default mode for the net-based minimum routing layer constraints set by the *set_routing_rule* command. This setting applies only to nets whose minimum routing layer mode is not set by the *set_routing_rule* command.

The definition for each mode is

- *soft* (the default)

Routing below the minimum layer is discouraged, but not disallowed. DRC violations are not flagged on any metal below the minimum layer.

- *allow_pin_connection*

Routing is allowed below the minimum layer only for pin connections or to fix DRC violations in extremely congested areas. DRC violations are flagged on metal below the minimum layer in the following cases:

- The metal is not along a path that connects to a pin.
- The metal is along a path that connects to a pin and the path is long.

The detail router considers a path to be long if its length exceeds approximately 10 average track pitches. The *check_routes* command considers a path to be long if its length exceeds approximately 15 average track pitches.

r

- *hard*

No routing can occur below the minimum layer. DRC violations are flagged on metal below the minimum layer.

Note that the *route.common.number_of_vias_under_net_min_layer* option can relax the minimum layer constraint, but only for vias.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)

route.common.net_min_layer_mode_soft_cost

Specifies the default cost for routing below the net-based minimum routing layer when using the soft minimum layer mode.

Data Types

string

Default medium

Description

This application option specifies the default cost for routing below the net-based minimum routing layer when using the soft minimum layer mode. Valid values are *low*, *medium*, and *high*.

This setting applies only to nets whose soft mode minimum routing layer cost is not set by the *set_routing_rule* command.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)

route.common.number_of_secondary_pg_pin_connections

Specifies the maximum number of secondary power and ground (PG) pins in a cluster.

r

Data Types

integer

Default 0**Description**

This application option specifies the maximum number of secondary PG pins in a cluster.

The definition of a cluster is a set of connected secondary PG pins. Each cluster has one connection with a PG strap or ring. If the value is greater than 0, a cluster should not contain more than the specified number of secondary PG pins.

By default, the setting is 0, which means that there is no maximum.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)

route.common.number_of_vias_over_global_backside_max_layer

Allows the router to use stacked vias to access pins or fixed routes above the global backside maximum layer.

Data Types

integer

Default 1**Description**

This application option allows the router to use stacked vias to access pins or fixed routes above the global backside maximum layer. If the global backside maximum layer mode is set to *soft*, this option is ignored, because in that case access above the maximum layer is allowed.

The value range is between 0 and 15. This value specifies the number of via layers that can be traversed to access a pin without triggering a DRC violation. For example, if the maximum layer is bm2 and you specify a value of 2, the tool does not flag violations on vias from bm2 to bm1 or bm1 to bm0 if these vias are needed to access pins on layers above the maximum layer.

r

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)

route.common.number_of_vias_over_global_max_layer

Allows the router to use stacked vias to access pins or fixed routes above the global maximum layer.

Data Types

integer

Default 1

Description

This application option allows the router to use stacked vias to access pins or fixed routes above the global maximum layer. If the global maximum layer mode is set to *soft*, this option is ignored, because in that case access above the maximum layer is allowed.

The value range is between 0 and 15. This value specifies the number of via layers that can be traversed to access a pin without triggering a DRC violation. For example, if the maximum layer is M6 and you specify a value of 2, the tool does not flag violations on vias from M6 to M7 or M7 to M8 if these vias are needed to access pins on layers above the maximum layer.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)

route.common.number_of_vias_over_net_max_layer

Allows the router to use stacked vias to access pins or fixed routes above the net-specific maximum layer.

Data Types

integer

Default 1

r

Description

This application option allows the router to use stacked vias to access pins or fixed routes above the net-specific maximum layer. If the net-specific maximum layer mode is set to *soft*, this option is ignored, because in that case access above the maximum layer is allowed.

The value range is between 0 and 15. This value specifies the number of via layers that can be traversed to access a pin without triggering a DRC violation. For example, if the maximum layer is M6 and you specify a value of 2, the tool does not flag violations on vias from M6 to M7 or M7 to M8 if these vias are needed to access pins on layers above the maximum layer.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)

route.common.number_of_vias_under_global_backside_min_layer

Allows the router to use stacked vias to access pins or fixed routes below the global backside minimum layer.

Data Types

integer

Default 1

Description

This application option allows the router to use stacked vias to access pins or fixed routes below the global backside minimum layer. If the global backside minimum layer mode is set to *soft*, this option is ignored, because in that case access below the minimum layer is allowed.

The value range is between 0 and 15. This value specifies the number of via layers that can be traversed to access a pin without triggering a DRC violation. For example, if the minimum layer is bm4 and you specify a value of 1, violations are not flagged on vias from bm4 to bm5 if these vias are needed to access pins on bm5.

r

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)

route.common.number_of_vias_under_global_min_layer

Allows the router to use stacked vias to access pins or fixed routes below the global minimum layer.

Data Types

integer

Default 1

Description

This application option allows the router to use stacked vias to access pins or fixed routes below the global minimum layer. If the global minimum layer mode is set to *soft*, this option is ignored, because in that case access below the minimum layer is allowed.

The value range is between 0 and 15. This value specifies the number of via layers that can be traversed to access a pin without triggering a DRC violation. For example, if the minimum layer is M2 and you specify a value of 1, violations are not flagged on vias from M1-to-M2 if these vias are needed to access pins on M1.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)

route.common.number_of_vias_under_net_min_layer

Allows the router to use stacked vias to access pins or fixed routes below the net-specific minimum layer.

Data Types

integer

Default 1

r

Description

This application option allows the router to use stacked vias to access pins or fixed routes below the net-specific minimum layer. If the net-specific minimum layer mode is set to *soft*, this option is ignored, because in that case access below the minimum layer is allowed.

The value range is between 0 and 15. This value specifies the number of via layers that can be traversed to access a pin without triggering a DRC violation. For example, if the minimum layer is M2 and you specify a value of 1, violations are not flagged on vias from M1-to-M2 if these vias are needed to access pins on M1.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)

route.common.pg_shield_distance_threshold

Specifies the distance threshold in microns for the router to consider the existing power and ground structure as shielding when calculating the shielding ratio.

Data Types

float

Default 0

Description

This application option specifies the distance threshold in microns for the router to consider the existing power and ground structure as shielding when calculating the shielding ratio.

Note that this option does not change the routing behavior. It affects only the shielding ratio report generated by the *create_shields* and *report_shields* commands.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)

r

route.common.post_detail_route_fix_soft_violations

Enables automatic optimization of soft rules that are fixable postroute, after the detail routing step.

Data Types

Boolean

Default false

Description

This application option enables automatic optimization of soft rules that are fixable postroute, after the detail routing step.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)

route.common.post_detail_route_redundant_via_insertion

Enables automatic redundant via insertion after each detail routing step.

Data Types

string

Default off

Description

This application option enables automatic redundant via insertion after each detail routing step.

Valid values are *off* (the default), *low*, *medium*, and *high*.

If the value is set to *low*, *medium*, or *high*, the tool performs redundant via insertion after each detail routing change, including initial detail routing, ECO routing, and incremental routing. Enabling this option keeps the redundant vias in the design up-to-date with routing changes. The *low*, *medium*, or *high* setting specifies the effort level used for redundant via insertion.

r

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)

route.common.post_eco_route_fix_soft_violations

Enables automatic optimization of soft rules that are fixable postroute, after the ECO routing step.

Data Types

Boolean

Default false

Description

This application option enables automatic optimization of soft rules that are fixable postroute, after the ECO routing step.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)

route.common.post_group_route_fix_soft_violations

Enables automatic optimization of soft rules that are fixable postroute, after the group routing step.

Data Types

Boolean

Default false

Description

This application option enables automatic optimization of soft rules that are fixable postroute, after the group routing step.

r

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)

route.common.post_incremental_detail_route_fix_soft_violations

Enables automatic optimization of soft rules that are fixable postroute, after the incremental routing step.

Data Types

Boolean

Default false

Description

This application option enables automatic optimization of soft rules that are fixable postroute, after the incremental routing step.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)

route.common.post_route_timing_driven_mode

Specifies if the timing data should be refreshed during the post route flow after hyper_route_opt.

Data Types

int

Default 0

Description

This application option specifies if the timing data should be updated and reloaded for each post route steps. The option should generally be set 1 if the user wants to save the runtime during the post route. if the user wants more precise timing after running the hyper_route_opt, set this mode to 0.

r

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)

route.common.rc_driven_setup_effort_level

Enables the router to automatically bias layer preference based on RC values for better setup timing.

Data Types

string

Default medium

Description

This application option enables the router to automatically bias layer preference based on RC values for better setup timing.

Valid values are *off*, *low*, *medium* (the default), and *high*.

This option takes effect only if timing-driven routing is enabled.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)

route.common.redundant_via_exclude_weight_group_by_layer_name

Specifies for each layer a set of redundant via insertion weight groups to selectively exclude.

Data Types

list

Default ""

r

Description

Specifies for each layer a set of redundant via insertion weight groups to selectively exclude.

Works in conjunction with `route.common.redundant_via_include_weight_group_by_layer_name`.

These options allow selective use of layer-specific weight groups during redundant via insertion, without any need for a user to modify the via mapping.

When `redundant_via_exclude_weight_group_by_layer_name` is used, the weight groups listed for each layer will not be tried. Layers without entries will have their complete via mapping applied.

If both options are used, the "include" option specifies the initial set of active weight groups, and the "exclude" option will deactivate weight groups from that set.

The active weight groups are fully controlled as follows:

Only "include" specified: NONE + {include} Only "exclude" specified: ALL - {exclude} Both "include" and "exclude": NONE + {include} - {exclude}

Example:

```
set_app_options -name
route.common.redundant_via_exclude_weight_group_by_layer_name \ -value { { V1 { 3 } }
\ { V2 { 2 1 } } \ { J3 { 10 } } }
```

You must specify the routing layers by using the layer names in the technology file.

See Also

- [add_via_mapping](#)
- [add_redundant_vias](#)
- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)

route.common.redundant_via_include_weight_group_by_layer_name

Specifies for each layer a set of redundant via insertion weight groups to selectively apply.

r

Data Types

list

Default ""**Description**

Specifies for each layer a set of redundant via insertion weight groups to selectively apply.

Works in conjunction with

`route.common.redundant_via_exclude_weight_group_by_layer_name` .

These options allow selective use of layer-specific weight groups during redundant via insertion, without any need for a user to modify the via mapping.

When `redundant_via_include_weight_group_by_layer_name` is used, only the weight groups listed for each layer will be tried. Layers without entries are skipped completely.

The `redundant_via_exclude_weight_group_by_layer_name` option subtracts weight groups from the applicable set. I.e. all weight groups will be tried EXCEPT those mentioned in the "exclude" list.

If both options are used, the "include" option specifies the initial set of active weight groups, and the "exclude" option will deactivate weight groups from that set.

The active weight groups are fully controlled as follows:

Only "include" specified: NONE + {include} Only "exclude" specified: ALL - {exclude} Both "include" and "exclude": NONE + {include} - {exclude}

Example:

```
set_app_options -name
route.common.redundant_via_include_weight_group_by_layer_name \ -value { { V1 { 3 2
1 } } \ { V2 { 2 1 } } \ { J3 { 2 1 } } }
```

You must specify the routing layers by using the layer names in the technology file.

See Also

- [add_via_mapping](#)
- [add_redundant_vias](#)
- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)

r

route.common.relax_soft_spacing_outside_min_max_layer

Automatically relax the soft rule outside the min/max layers for the global route and track assignment.

Data Types

Boolean

Default `true`

Description

This application option has impact when the net has min/max layer set and the net has soft NDR rule attached. i.e. `spacing_weight_levels` in the NDR rule is not set to `hard`. The option is intended to improve the correlation between GR and DR.

When set to *true*(the default), we removed the NDR requirement outside the min/max layer. GR/TA read the modified NDR soft rule and DR reads the original NDR rule.

If set to *false*, there is no change on the NDR rule for GR/TA.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)

route.common.reroute_clock_shapes

Specifies whether the router can reroute fixed clock wires and vias.

Data Types

Boolean

Default `false`

Description

This application option specifies whether the router can reroute fixed clock wires and vias.

r

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)

route.common.reroute_user_shapes

Specifies whether the router can reroute user-created wires and vias.

Data Types

Boolean

Default false

Description

This application option specifies whether the router can reroute user-created wires and vias.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)

route.common.reshield_modified_nets

Specifies whether to automatically unshield or reshield modified shielded nets during routing.

Data Types

string

Default off

Description

This application option specifies whether to automatically unshield or reshield modified shielded nets during routing.

The definition for each mode is

r

- *off* (the default)

The router does not unshield or reshield modified shielded nets.

- *reshield*

The router unshields modified shielded nets, automatically reshields them, and then reconnects the floating shielding. If the router cannot reconnect the floating shielding, it removes it. Shielded nets might have been modified by routing or outside of routing by manual editing, route optimization, and other commands.

- *unshield*

The router deletes existing shielding that is associated with modified shielded nets and reconnects the floating shielding if necessary. The nets might be reshielded at a later stage.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)

route.common.rotate_default_vias

Specifies whether the router can rotate default vias.

Data Types

Boolean

Default true

Description

This application option specifies whether the router can rotate default vias.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)

r

route.common.route_soft_rule_effort_level

Specifies the effort level for rip up and reroute to fix soft rule design rule violations.

Data Types

string

Default medium

Description

This application option specifies the effort level for rip up and reroute to fix soft rule design rule violations.

The definition for each effort level is

- *off*

The router does not perform rip up and reroute passes to resolve soft rule design rule violations.

- *min*

The router uses the smallest number of rip up and reroute passes to resolve soft rule design rule violations.

- *low*

The router uses a small number of rip up and reroute passes to resolve soft rule design rule violations.

- *medium* (the default)

Uses a medium number of rip up and reroute passes to resolve soft rule design rule violations.

- *high*

Treats soft rule design rule violations the same as regular design rule violations during rip up and reroute.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)

r

route.common.route_top_boundary_mode

Controls the routing behavior near the top boundary.

Data Types

string

Default stay_inside

Description

Controls the routing behavior near the top boundary.

The definition for each mode is

- *stay_half_min_space_inside*
The routing must be inside the boundary by half of the minimum spacing.
- *stay_inside* (the default)
The routing must be inside or abut the boundary.
- *ignore*
Wires can extend outside the boundary.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)

route.common.routing_rule_effort_level

Specifies detailed, layer-by-layer effort levels on the various spacings of soft nondefault routing rules.

Data Types

list

Default ""

Description

This application option specifies detailed, layer-by-layer effort levels on the various spacings of soft nondefault routing rules. For each nondefault routing rule, you can specify

r

the effort for each spacing value defined for a layer. The rule name corresponds to a previously defined nondefault routing rule. For each layer defined in the *-spacings* option of the *create_routing_rule* command, you specify the effort level for each of the spacings defined for that layer.

The definition for each effort level is

- *low*

Uses a small number of rip up and reroute passes to resolve soft rule design rule violations.

- *med*

Uses a medium number of rip up and reroute passes to resolve soft rule design rule violations.

- *high*

Treats soft rule design rule violations the same as regular design rule violations during rip up and reroute.

For example, suppose you have defined the following two soft routing rules:

```
create_routing_rule soft_rule_1 \\  
    -spacings {M1 {0.12 0.24 }  
              M2 {0.14 0.28 }  
              M3 {0.14 0.28 }  
              M4 {0.14 0.28 }  
              M5 {0.14 0.28 }} \\  
    -spacing_weight_levels {M1 {hard high}  
                           M2 {hard high}  
                           M3 {hard high}  
                           M4 {hard high}  
                           M5 {hard high}}  
  
create_routing_rule soft_rule_2 \\  
    -spacings {M1 {0.24 0.36}  
              M2 {0.28 0.42}  
              M3 {0.28 0.42}  
              M4 {0.28 0.42}  
              M5 {0.28 0.42}} \\  
    -spacing_weight_levels {M1 {high medium}  
                           M2 {high medium}  
                           M3 {high medium}  
                           M4 {high medium}  
                           M5 {high medium}}
```


r

You would specify the detailed effort levels for these rules as shown in the following example:

```
set_app_options \\  
-name route.common.routing_rule_effort_level \\  
-value { {soft_rule_1 { {M1 {high med}}  
                        {M2 {high med}}  
                        {M3 {high med}}  
                        {M4 {high med}}  
                        {M5 {high med}}  
                      }  
        {soft_rule_2 { {M1 {high med}}  
                      {M2 {low high}}  
                      {M3 {med high}}  
                      {M4 {med high}}  
                      {M5 {high high}}  
                    }  
        }  
      }
```

Examples

The following example shows how to set the routing rule effort level for a rule `soft_rule_2` on MET4 layer.

```
prompt> set_app_options -block [current_block] \\  
-name route.common.routing_rule_effort_level \\  
-value { { soft_rule_2 { MET4 HIGH MED} } }
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)

route.common.separate_tie_off_from_secondary_pg

Specifies whether to separate the tie-off connections from the secondary PG power connections.

Data Types

Boolean

Default false

r

Description

By default (*false*), tie-off connections and secondary PG power connections are considered the same nets, so nondefault routing rules, minimum and maximum layer constraints, and other constraints on the net apply to both tie-off connections and secondary PG power connections.

When you separate the tie-off connections from the secondary PG power connections by setting this option to *true*, you can apply nondefault routing rules, minimum and maximum layer constraints, and other constraints only to the secondary PG power connections.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)

route.common.shielding_nets

Specifies the power and ground nets to be used for shielding by the *create_shields* command.

Data Types

list

Default ""

Description

This application option specifies the power and ground nets to be used for shielding by the *create_shields* command.

When routing signal nets, if the *route.common.allow_pg_as_shield* application option is *true* (the default), the router tries to route the shielded nets adjacent to any of the specified power and ground nets while considering other routing metrics such as congestion and wire length.

If the *route.common.allow_pg_as_shield* application option is *false*, this option has no effect.

If you do not specify this option, by default, the *create_shields* command uses the ground net for shielding.

You can also specify the power or ground net used for shielding by using the *-with_ground* option with the *create_shields* command; however, this option supports only a single power or ground net.

r

If you specify both the *route.common.shielding_nets* application option and the *create_shields -with_ground* option, the tool issues a warning message and uses the *route.common.shielding_nets* setting.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)

route.common.single_connection_to_pins

Specifies whether the router can connect to a pin only once.

Data Types

string

Default off

Description

This application option specifies whether the router can connect to a pin only once.

Valid values are *off*, *standard_cell_pins*, *macro_cell_pins*, *top_cell_pins*, *standard_and_macro_cell_pins*, *standard_and_top_cell_pins*, *macro_and_top_cell_pins*, *standard_and_macro_and_pad_cell_pins*, and *all_pins*.

If the value is *off* (the default), the router is allowed to connect to a pin any number of times. If the value is *all_pins*, all the pins in the design have the single-connection constraint and the router can connect to any pin only once. The other values specify the subset of pins that have the single-connection constraint.

The default is *off*, except for nets that use a nondefault width routing rule. For those nets, the value is always *all_pins*.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)

route.common.soft_rule_weight_to_effort_level_map

Specifies a global mapping from soft weight level to soft effort level.

r

Data Types

list

Default ""

Description

This application option specifies a global mapping from soft weight level to soft effort level.

The weight level must be one of the following values: *low*, *medium*, or *high*.

The effort level must be one of the following values: *off*, *low*, *medium*, or *high*.

The definition for each effort level is

- *off* (the default)

The router does not perform rip up and reroute passes to resolve soft rule design rule violations.

- *low*

The router uses a small number of rip up and reroute passes to resolve soft rule design rule violations.

- *medium*

The router uses a medium number of rip up and reroute passes to resolve soft rule design rule violations.

- *high*

The router treats soft rule design rule violations the same as regular design rule violations during rip up and reroute.

This mapping is applied whenever soft rules are defined using the *-spacing_weight_levels* option of the *create_routing_rule* command.

Each weight level should appear only once in the map.

The following mappings are not allowed:

- Low weight with medium effort
- Low weight with high effort

For example, suppose you have defined the following mapping:

```
set_app_options \\  
  -name soft_rule_weight_to_effort_level_map \\  
  -value { {low low}  
           {medium medium}  
           {high medium} }
```

r

This means that

- Soft rules with low weight should use low effort.
- Soft rules with medium weight should use medium effort.
- Soft rules with high weight should use medium effort.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)

route.common.support_staple_vias_during_routing

Support vias with staple via property during routing.

Data Types

boolean

Default false

Description

Enabling the application option makes Zroute treat vias with staple via property like other vias in the design without this property.

By default, Zroute ignores vias with staple-via property. When app option `route.common.verbose_level` is set, ZRT-659 message is issued for each via ignored by router due to their staple-via property. When app option `route.common.support_staple_vias_during_routing` is set, vias with stable via property are not ignored by router. They are handled like other routing vias without this property.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [route_opt](#)

r

- [route_auto](#)
- [route_detail](#)

route.common.threshold_noise_ratio

Specifies the static noise threshold as a ratio of the voltage.

Data Types

float

Default 0.35

Description

This application option specifies the static noise threshold as a ratio of the voltage. This is used by the router to determine the crosstalk criticality of a net.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)

route.common.tie_off_mode

Controls whether the router can connect tie-off nets to any power or ground (PG) structures, such as straps, rails, or pins, or only to a PG rail.

Data Types

string

Default all

Description

This application option controls whether the router can connect tie-off nets to any power or ground (PG) structures, such as straps, rails, or pins, or only to a PG rail.

The definition for each mode is

- *all* (the default)

Tie-off nets can be connected to any PG structures.

r

- *rail_only*

Tie-off nets can be connected only to a PG rail.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)

route.common.track_auto_fill

Specifies whether to fill empty space with routing tracks.

Data Types

Boolean

Default true

Description

This application option specifies whether to fill empty space with routing tracks.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)

route.common.treat_all_cover_cells_as_lib_cells_for_routing

Treat all the cover cells of the top design like lib cells.

Data Types

boolean

Default false

Description

Enabling the application option would make Zroute treat cover cells like lib cells for all the routing commands.

r

By default, PG-net pins of cover cells are not connected by router. Instead, they are treated as pre-routes of corresponding PG net. When user sets “route.common.treat_all_cover_cells_as_lib_cells_for_routing” to TRUE, cover cells will be treated similar as standard cells, processing of PG-net pins of those cover cells will be done similar as standard cell pins.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [route_opt](#)
- [route_auto](#)
- [route_detail](#)

route.common.treat_via_array_as_big_via

Specifies whether to treat via arrays as big via

Data Types

Boolean

Default false

Description

This application option specifies whether to treat via arrays as big via.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)

route.common.verbose_level

Sets the verbosity level for the routing log file.

Data Types

integer

r

Default 0**Description**

This application option sets the verbosity level for the routing log file. Valid values are 0, 1, or 2.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)

route.common.via_array_mode

Specifies the types of rotated via arrays that can be used during signal routing and redundant via insertion when using the default routing rules.

Data Types

string

Default all**Description**

This application option specifies the types of rotated via arrays that can be used during signal routing and redundant via insertion when using the default routing rules.

The definition for each mode is

- *off*
Do not rotate or swap via arrays.
- *swap*
Use 1 x N and N x 1 via arrays as rotated equivalent vias.
- *rotate*
Rotate line via arrays instead of swapping row and column numbers.
- *all* (the default)
Use all four possible via arrays.

r

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)

route.common.via_on_grid_by_layer_name

Controls whether vias and their associated via enclosures must be routed on-grid. This option controls the same functionality as the *onGrid* attribute for a via layer in the technology file.

Data Types

list

Default ""**Description**

If you use this option, the tool ignores all *onGrid* attributes defined in the technology file (both for metal layers and via layers). The default setting for any unspecified *onGrid* attribute is *false*; therefore, on-grid routing is required only for the via layers specified with a setting of *true* in this option and metal layers specified with a setting of *true* in the *route.common.wire_on_grid_by_layer_name* option. You can set this option only when the block is open. This option controls only *onGrid* attributes; it has no impact if the technology file defines *onWireTrack* attributes.

Use the following syntax to specify the setting for each layer:

```
{routing_layer true|false}
```

You must specify the routing layers by using the layer names in the technology file.

Examples

The following example sets the *route.common.via_on_grid_by_layer_name* application option to *true* for the V3 and V4 cut layers.

```
prompt> set_app_options \
        -name route.common.via_on_grid_by_layer_name \
        -value {{V3 true} {V4 true}}
```

r

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)

route.common.wide_macro_pin_as_fat_wire

Specifies whether a wide macro pin or pad pin should have an implicit depth the same as its width.

Data Types

Boolean

Default false

Description

This application option specifies whether a wide macro pin or pad pin should have an implicit depth the same as its width. If this option is set to false, we assume the depth of the pin is the same as the default width of that layer and the pin shape would not trigger the fat metal spacing rule.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)

route.common.wire_on_grid_by_layer_name

Controls whether wires must be routed on-grid. This option controls the same functionality as the *onGrid* attribute for a metal layer in the technology file.

Data Types

list

Default ""

Description

If you use this option, the tool ignores all *onGrid* attributes defined in the technology file (both for metal layers and via layers). The default setting for any unspecified *onGrid*

r

attributes is *false*; therefore, on-grid routing is required only for the metal layers specified with a setting of *true* in this option and via layers specified with a setting of *true* in the *-via_on_grid_by_layer_name* option. You can specify this option only when the block is open. This option is to control *onGrid* attributes only. It has no impact if *onWireTrack* is defined in the technology file.

Use the following format to specify the setting for each layer:

```
{routing_layer true|false}
```

You must specify the routing layers by using the layer names in the technology file.

Examples

The following example sets the *route.common.wire_on_grid_by_layer_name* application option to *true* for the M3 and M4 metal layers.

```
prompt> set_app_options \\  
-name route.common.wire_on_grid_by_layer_name \\  
-value {{M3 true} {M4 true}}
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)

route.common.write_instance_via_color

Specifies the via color should be written to the instance cell.

Data Types

boolean

Default false

Description

This application option specifies the via color should be written to the instance cell. The option should generally be set true if the library has uncolored vias, and via coloring is required.

r

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)

route.detail.allow_default_rule_nets_via_ladder_lower_layer_connection

Routing is normally required to connect to the top layer of a via ladder. If true, this option waives that requirement on default rule nets.

The default is false.

Data Types

Boolean

Default false

Description

Routing is normally required to connect to the top layer of a via ladder. If true, this option waives that requirement on default rule nets.

The connection to a via ladder may then be on any layer of the via ladder, or directly to the pin below the via ladder.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_via_ladder_constraints](#)
- [remove_via_ladder_constraints](#)
- [insert_via_ladders](#)
- [remove_via_ladders](#)
- [verify_via_ladders](#)
- [refresh_via_ladders](#)
- [set_via_ladder_rules](#)

r

- [route_auto](#)
- [route_eco](#)
- [route_opt](#)
- [route_group](#)
- [route_detail](#)

route.detail.allow_ndr_nets_via_ladder_lower_layer_connection

Routing is normally required to connect to the top layer of a via ladder. If true, this option waives that requirement on NDR nets.

The default is false.

Data Types

Boolean

Default false

Description

Routing is normally required to connect to the top layer of a via ladder. If true, this option waives that requirement on NDR nets.

The connection to a via ladder may then be on any layer of the via ladder, or directly to the pin below the via ladder.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_via_ladder_constraints](#)
- [remove_via_ladder_constraints](#)
- [insert_via_ladders](#)
- [remove_via_ladders](#)
- [verify_via_ladders](#)
- [refresh_via_ladders](#)
- [set_via_ladder_rules](#)

r

- [route_auto](#)
- [route_eco](#)
- [route_opt](#)
- [route_group](#)
- [route_detail](#)

route.detail.antenna

Enables (true) or disables (false) antenna analysis.

Data Types

Boolean

Default true

Description

This application option enables (true) or disables (false) antenna analysis.

Examples

The following example disables antenna analysis.

```
prompt> set_app_options \  
-name route.detail.antenna -value false
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [route_opt](#)
- [route_auto](#)
- [route_detail](#)

route.detail.antenna_fixing_preference

Specifies the preferred technique used to fix antenna violations: layer hopping or diode insertion.

r

Data Types

string

Default `hop_layers`

Description

This application option specifies the preferred technique used to fix antenna violations.

Valid values are

- *hop_layers* (the default)
Selects layer hopping as the preferred technique to fix antenna violations.
- *use_diodes*
Selects diode insertion as the preferred technique to fix antenna violations.

This option is used only when the user enables diode insertion and does not disable layer hopping. Related app options for them are `route.detail.hop_layers_to_fix_antenna` (default value true) `route.detail.insert_diodes_during_routing` (detail value false)

This is only a preferred method to resolve antenna violations. Setting the option value to one does not automatically disable the other. Zroute still attempts layer hopping and diode insertion techniques to resolve antenna violations.

Examples

The following example sets layer hopping as the preferred technique for fixing antenna violations:

```
prompt> set_app_options \  
-name route.detail.antenna_fixing_preference -value hop_layers
```

The following example sets diode insertion as the preferred technique for fixing antenna violations:

```
prompt> set_app_options \  
-name route.detail.antenna_fixing_preference -value use_diodes
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [route_auto](#)

r

- [route_detail](#)
- [set_app_options](#)

route.detail.antenna_on_iteration

Specifies the detail routing iteration at which antenna analysis and optimization is turned on.

Data Types

integer

Default 1

Description

This application option specifies the detail routing iteration at which antenna analysis and optimization is turned on.

The range is from 1 to 19 (the routing iteration number starts at 0).

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [route_opt](#)
- [route_auto](#)
- [route_detail](#)

route.detail.antenna_verbose_level

Specifies the verbosity level to print extra information for antenna analysis and optimization.

Data Types

integer

Default 1

r

Description

This application option specifies the verbosity level to print extra information for antenna analysis and optimization. This option does not affect the actual analysis or optimization.

The range is from 0 to 3.

When this option is set to 2, the tool prints antenna calculation information similar to the following:

```
ReportedNet n94696; NetTopLayer metal3; ICell post_clock29650; MasterPort A;
AntennaMode 4; DiodeMode 4; gClass 0; GateType: gate; ; DiodeProtection 0.00000000;
wlay metal1; WireSideWallArea 0.01596000; GateArea 0.00185600; AllowedRatio
2147483648.00000000; ActualRatio 8.59913826; clay via1; CutSideWallArea 0.00000000;
GateArea 0.00185600; AllowedRatio 2147483648.00000000; ActualRatio 0.00000000;
```

When this option is set to 3, antenna calculation information is printed for violating pins only on layer with antenna violation.

See Also

- [check_routes](#)
- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)
- [route_auto](#)
- [route_detail](#)

route.detail.check_antenna_for_open_nets

Check and fix antenna violations on nets that are not completely connected.

Data Types

boolean

Default false

Description

This application option enables checking and fixing of antenna violations on open nets.

Valid values are false and true. When a net is open, antenna violations reported for its pins will be based on partially connected net.

r

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [route_opt](#)
- [route_auto](#)
- [route_detail](#)

route.detail.check_antenna_on_diode_pins

Enables (true) or disables (false) analysis and optimization of antenna violations of diode pins.

Data Types

Boolean

Default false

Description

This application option enables (true) or disables (false) analysis and optimization of antennas of diode pins.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [route_opt](#)
- [route_auto](#)
- [route_detail](#)

route.detail.check_antenna_on_pg

Enables (true) or disables (false) analysis and optimization of antenna violations of power and ground nets.

r

Data Types

Boolean

Default false**Description**

This application option enables (true) or disables (false) analysis and optimization of antennas on power and ground nets.

For designs with high fanout power or ground nets, enabling this option can result in longer runtime for routing commands to analyze and optimize antenna violations for these nets.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [route_opt](#)
- [route_auto](#)
- [route_detail](#)

route.detail.check_patchable_drc_from_fixed_shapes

Enables (true) or disables (false) checking of patchable DRCs for

Data Types

Boolean

Default false**Description**

This application option enables (true) or disables (false) checking of patchable DRCs for fixed routes.

When this option is enabled, patchable DRCs (minimum area, minimum length, and end-of-line spacing) are checked between fixed routes. Violations are resolved by metal patching.

r

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [route_opt](#)
- [route_auto](#)
- [route_detail](#)

route.detail.check_pin_min_area_min_length

Enables (true) or disables (false) checking for minimum area and minimum length rules on pins.

Data Types

Boolean

Default true

Description

This application option enables (true) or disables (false) checking for minimum area and minimum length rules on pins.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [route_opt](#)
- [route_auto](#)
- [route_detail](#)

route.detail.check_port_min_area_min_length

Controls checking for minimum area and minimum length on physical ports.

Data Types

Boolean

r

Default true**Description**

This application option enables (true) or disables (false) checking for minimum area and minimum length rules on physical ports (terminals).

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [route_opt](#)
- [route_auto](#)
- [route_detail](#)

route.detail.check_routes_frozen_mode_include_shape_with_attribute_locked

Enables (true) or disables (false) checking of frozen shape with physical_status locked

Data Types

Boolean

Default false**Description**

This application option enables (true) or disables (false) checking of frozen shape with physical_status locked.

When this option is enabled, check_routes -check_from_frozen_shapes mode would check frozen shape with physical_status locked.

When this option is disabled, check_routes -check_from_frozen_shapes mode only check frozen shape with its net physical_status locked.

See Also

- [get_app_option_value](#)
- [get_app_options](#)

r

- [report_app_options](#)
- [check_routes](#)

route.detail.continue_after_large_design_rule_value_error

Controls whether to continue on large design rule values.

Data Types

Boolean

Default false

Description

The tool has an internal threshold for the `fatTblExtensionRange`, `fatTblXExtensionRange`, `fatTblYExtensionRange`, and `protrusionLengthLimitTbl` attributes in the technology file.

By default (*false*), the detail router fails if the value specified for any of these attributes exceeds the internal threshold.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [route_opt](#)
- [route_auto](#)
- [route_detail](#)

route.detail.default_diode_protection

Specifies the diode protection value used for pins during antenna analysis if the value is not specified in the frame view of the cell.

Data Types

float

Default 0.0

r

Description

This application option specifies the diode protection value used for pins during antenna analysis if the value is not specified in the frame view of the cell. This option has no effect when the frame view specifies a diode protection value.

Depending on the diode mode, the diode protection value can be used to calculate antenna ratio requirements, equivalent metal area, or equivalent gate area.

The range is from 0.0 to 1,000,000.00. There is no unit for diode protection.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [route_opt](#)
- [route_auto](#)
- [route_detail](#)

route.detail.default_gate_size

Specifies the gate size used for pins during antenna analysis if the gate size is not specified in the frame view of the cell.

Data Types

float

Default -1.0

Description

This application option specifies the gate size used for pins during antenna analysis if the gate size is not specified in the frame view of the cell.

The range is from -1.0 to 1,000,000.00. The unit is the square of the unitLengthName specified in the technology file.

The tool uses this option only when you set it to a value of 0.0 or greater and the frame view does not specify the gate size for the pin. This option has no effect when the frame view specifies a gate size value.

r

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [route_opt](#)
- [route_auto](#)
- [route_detail](#)

route.detail.default_nwell_size

Specifies the nwell size used for pins during antenna analysis if the nwell size is not specified in the frame view of the cell.

Data Types

float

Default -1.0

Description

This application option specifies the nwell size used for pins during antenna analysis if the nwell size is not specified in the frame view of the cell.

The range is from -1.0 to 1,000,000.00. The unit is the square of the unitLengthName specified in the technology file.

The tool uses this option only when you set it to a value of 0.0 or greater and the frame view does not specify the nwell size for the pin. This option has no effect when the frame view specifies a nwell size value.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [route_opt](#)
- [route_auto](#)
- [route_detail](#)

r

route.detail.default_port_external_antenna_area

Specifies the antenna area value used for ports (top-level pins) during antenna analysis if the antenna area is not specified in the frame view of the cell.

Data Types

float

Default 0.0

Description

This application option specifies the antenna area value used for ports (top-level pins) during antenna analysis if the antenna area is not specified in the frame view of the cell. The antenna area is the metal area from the gate input pin to the external port. If any antenna violations are detected using this option value, they are reported and fixed during detail route. When the design with this option setting is used as sub-block (like macro) for parent-level design, antenna analysis in parent-level design will be impacted as well.

The range is from 0.0 to 1,000,000.00. The unit is the square of the unitLengthName specified in the technology file.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [route_opt](#)
- [route_auto](#)
- [route_detail](#)

route.detail.default_port_external_gate_size

Specifies the gate size used for ports (top-level pins) during antenna analysis if the gate size is not specified in the frame view of the cell.

Data Types

float

Default -1.0

r

Description

This application option specifies the gate size used for ports (top-level pins) during antenna analysis if the gate size is not specified in the frame view of the cell.

The range is from -1.0 to 1,000,000.00. The unit is square of unitLengthName specified in the technology file.

The tool uses this option only when you set it to a value of 0.0 or greater and the frame view does not specify the gate size for the port. This option has no effect when the frame view specifies a gate size value.

If the gate size is not set for a port, Zroute issues a ZRT-311 message and does not perform antenna analysis on the port.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [route_opt](#)
- [route_auto](#)
- [route_detail](#)

route.detail.default_port_external_nwell_size

Specifies the nwell size used for ports (top-level pins) during antenna analysis if the nwell size is not specified in the frame view of the cell.

Data Types

float

Default -1.0

Description

This application option specifies the nwell size used for ports (top-level pins) during antenna analysis if the nwell size is not specified in the frame view of the cell.

The range is from -1.0 to 1,000,000.00. The unit is square of unitLengthName specified in the technology file.

r

The tool uses this option only when you set it to a value of 0.0 or greater and the frame view does not specify the nwell size for the port. This option has no effect when the frame view specifies a nwell size value.

If the nwell size is not set for a port, Zroute issues a ZRT-311 message and does not perform antenna analysis on the port.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [route_opt](#)
- [route_auto](#)
- [route_detail](#)

route.detail.delete_redundant_diodes_during_routing

Specifies whether detail route can delete redundant diodes that are not needed anymore to fix antenna violations.

Data Types

Boolean

Default false

Description

This application option specifies whether the router can delete existing redundant diodes if they are not needed to fix antenna violations. Redundant diodes are detected automatically during detail route and deleted. When user sets route.detail.antenna_verbose_level to greater than 0, names of such deleted diodes are printed in the log file.

- Spare diodes are left untouched as they do not connect to any nets. - Port protection diodes that are inserted by detail route are not removed.

For redundant diodes to be detected and deleted, following app options are to be enabled:
- route.detail.antenna and - route.detail.insert_diodes_during_routing

r

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [route_opt](#)
- [route_auto](#)
- [route_detail](#)

route.detail.detail_route_special_design_rule_fixing_stage

Specifies the stage at which to fix special design rules when running the *route_detail*, *route_auto*, and *route_opt* commands.

Data Types

string

Default *late_routing*

Description

This application option specifies the stage at which to fix special design rules when running the *route_detail*, *route_auto*, and *route_opt* commands.

The special design rules include the iso-dense via enclosure rule. Fixing the special design rules requires significant routing resources, so you might not want to fix them at all stages.

You can fix these rules at the following stages:

- *late_routing* (the default) Check and fix these rules in the postroute stage, after the other DRC rules.
- *early_routing*
Check and fix these rules along with the other DRC rules.
- *none*
Never check and fix these rules.

r

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [route_opt](#)
- [route_auto](#)
- [route_detail](#)

route.detail.diagonal_min_width

Specifies whether to use diagonal distance for minimum width violations.

Data Types

Boolean

Default true

Description

This application option specifies whether to use diagonal distance when checking minimum width violations.

The minimum width is specified in the tech file using the minWidth field of the "Layer" section. The minWidth value uses the unit of length specified in the tech file.

If *true* (the default), diagonal distance is used.

If *false*, manhattan distance is used.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [route_opt](#)
- [route_auto](#)
- [route_detail](#)

r

route.detail.diode_insertion_mode

Specifies whether to fix antenna violations by using both new and spare diodes, spare diodes only, or new diodes only.

Data Types

string

Default new_and_spare

Description

This application option specifies whether to fix antenna violations by using both new and spare diodes (*new_and_spare*), spare diodes only (*spare*), or new diodes only (*new*).

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [route_opt](#)
- [route_auto](#)
- [route_detail](#)

route.detail.diode_libcell_names

Specifies the diode library cells to use for antenna fixing.

Data Types

list

Default ""

Description

This application option specifies the diode library cells to use for antenna fixing. When you specify the library cells, specify only the library cell name; do not include the library name. For example,

```
prompt> set_app_options -name route.detail.diode_libcell_names \\  
-value "$diode_library_cell_name"
```

The default is an empty list, in which case the tool automatically detects the diode cells.

r

Examples

The following example shows how to set the diode library cell to use for Zroute diode insertion.

```
prompt> set_app_options -block [current_block] \  
-name route.detail.diode_libcell_names -value {ANTENNA ANTENNA2}
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)
- [route_opt](#)
- [route_auto](#)
- [route_detail](#)

route.detail.diode_preference

Specifies the preference for fixing antenna violations by inserting a new diode or using an existing spare diode.

Data Types

string

Default none

Description

This application option specifies the preference for fixing antenna violations by inserting a new diode or using an existing spare diode.

The default is *none*. In this case, the tool selects the method to use based on whether an empty location for a new diode or an existing spare diode is closer to the required location.

Examples

The following example sets *route.detail.diode_preference* application option to *spare*.

```
prompt> set_app_options \  
-name route.detail.diode_preference \  
-value spare
```


r

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [route_opt](#)
- [route_auto](#)
- [route_detail](#)

route.detail.drc_convergence_effort_level

Specifies the effort level to use before early termination of detail routing when design rule checking (DRC) does not converge.

Data Types

string

Default medium

Description

This application option specifies the effort level to use before early termination of detail routing when design rule checking (DRC) does not converge.

Set the effort level to *high* to run more detail routing iterations when DRC does not converge. Set the effort level to *low* to run fewer detail route iterations when DRC does not converge.

This option is ignored if the *route.detail.force_max_number_iterations* application option is set to *true*.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [route_opt](#)
- [route_auto](#)
- [route_detail](#)

r

route.detail.eco_max_number_of_iterations

Controls the number of detail route iterations during the *route_eco* and *route_opt* commands.

Data Types

integer

Default -1

Description

This application option specifies the maximum number of detail routing iterations during the *route_eco* and *route_opt* commands.

You can specify an integer between -1 and 1000.

The default is -1, which means that the *route_eco* and *route_opt* commands determine the number of iterations. For the default number of detail route iterations for each command, see the man page.

If you use the *-max_detail_route_iterations* option with the *route_eco*, its setting overrides this application option.

Examples

The following example performs a maximum of 10 detail route iterations during ECO routing.

```
prompt> set_app_options \\  
-name route.detail.eco_max_number_of_iterations -value 10  
prompt> route_eco
```

The following example performs a maximum of seven detail route iterations during the *route_opt* command.

```
prompt> set_app_options \\  
-name route.detail.eco_max_number_of_iterations -value 7  
prompt> route_opt
```

The following example performs a maximum of five detail route iterations during ECO routing.

```
prompt> set_app_options \\  
-name route.detail.eco_max_number_of_iterations -value 10  
prompt> route_eco -max_detail_route_iterations 5
```

r

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)
- [route_eco](#)
- [route_opt](#)

route.detail.eco_route_special_design_rule_fixing_stage

Specifies the stage at which to fix special design rules when running the *route_eco* command.

Data Types

string

Default `late_routing`

Description

This application option specifies the stage at which to fix special design rules when running the *route_eco* command.

The special design rules include the iso-dense via enclosure rule. Fixing the special design rules requires significant routing resources, so you might not want to fix them at all stages.

You can fix these rules at the following stages:

- *late_routing* (the default) Check and fix these rules in the postroute stage, after the other DRC rules.
- *early_routing*
Check and fix these rules along with the other DRC rules.
- *none*
Never check and fix these rules.

r

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [route_opt](#)
- [route_auto](#)
- [route_detail](#)

route.detail.eco_route_use_soft_spacing_for_timing_optimization

Enables (true) or disables (false) soft-rule-based timing optimization during ECO routing.

Data Types

Boolean

Default true

Description

This application option enables (true) or disables (false) soft-rule-based timing optimization during ECO routing.

This option works only if the *route.detail.timing_driven* application option is set to *true*.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [route_opt](#)
- [route_auto](#)
- [route_detail](#)

route.detail.elapsed_time_limit

Specifies the limit for elapsed wall-clock time in minutes.

r

Data Types

integer

Default -1**Description**

This application option specifies the limit for elapsed wall-clock time in minutes.

The range is from -1 to MAX_INT.

The default is -1, which means no limit.

Examples

The following example sets the maximum elapsed time to 60 minutes.

```
prompt> set_app_options \\  
-name route.detail.elapsed_time_limit \\  
-value 60
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [route_opt](#)
- [route_auto](#)
- [route_detail](#)

route.detail.enable_fixing_selective_design_rule_violations_on_fixed_shapes

Valid option values are minimum_area, true, false. 'false' disables this check. Rest of the options enable checking for select design rule violations on fixed preroutes, pins, obstructions.

Data Types

String

Default false

r

Description

This application option enables checking for select design rule violations on fixed preroutes, pins and routing blockages. If the design rules are violated, enabling this option would make detail route fix the DRCs too. When this option is set to true or minimum_area, the design rules being checked are "Less than minimum area", "Less than minimum area in no cut metal area", "Via enclosed metal minimum area".

Option value 'true' is scheduled to be removed in 2017.09 release.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [route_opt](#)
- [route_auto](#)
- [route_detail](#)

route.detail.enable_ndr_tapering_on_voltage_rule

Enable the tapering on the nets with voltage spacing rule.

Data Types

bool

Default true

Description

An net with a NDR rule can also have attributes *voltage_range_max* and *voltage_range_min* set to certain values. This means that the net has to honor the voltage spacing rule defined by a table in the tech file layer section. This application option specifies if the tool honors the taper definition from the NDR rule for the nets with the voltage dependent spacing rule.

- *true* (the default)

The tool will try to taper the NDR nets with voltage dependent spacing rule.

- *false*

If the net has voltage spacing rule, the tapering is turned off.

r

Examples

The following example enable the taper for the nets with voltage spacing rule.

```
prompt> set_app_options \\  
        -name route.detail.enable_ndr_tapering_on_voltage_rule -value true
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [route_auto](#)
- [route_detail](#)
- [set_app_options](#)

route.detail.enable_separate_pgate_ngate_antenna_ratio_check

Specifies this option to perform separate pgate, ngate, gate based antenna ratio checks during detail route.

When this option is specified, the antenna ratio checks would be: if (ngate_area > 0 and nratio antenna rule defined) then check nratio if (pgate_area > 0 && pratio antenna rule defined) then check pratio if (gate_area > 0 && gate-ratio antenna rule defined) then check gate area based ratio

The default behavior would be: if (pgate_area = 0 and ngate_area > 0) then check nratio; if (ngate_area = 0 && pgate_area > 0) then check pratio; if (ngate_area > 0 && pgate_area > 0) or (ngate_area = 0 && pgate_area = 0) then check gate area based ratio

Data Types

Boolean

Default false

Description

This application option is used to perform separate pgate, ngate, gate area based antenna ratio checks during detail route.

r

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [route_opt](#)
- [route_auto](#)
- [route_detail](#)

route.detail.fat_metal_forbidden_pitch_effort_level

Specify the build-in effort level to reduce the Saumsung fat metal forbidden pitch DRCs.

Data Types

string

Default "off"

Description

This application option specifies the effort level to reduce the Saumsung fat metal forbidden pitch DRCs. The option values are (off, low, medium, high). The number of extra DRC checks iterations for Saumsung fat metal forbidden pitch is increased. The runtime will also increase.

Examples

The following example shows how to use the command.

```
prompt> set_app_options \\  
        -block [current_block] -list  
        {route.detail.fat_metal_forbidden_pitch_effort_level high}
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [route_detail](#)

r

route.detail.force_end_on_preferred_grid

Controls whether all metal ends must be snapped to the preferred grid if the preferred grid is defined for the layer.

Data Types

Boolean

Default false

Description

This application option controls whether every metal end must be snapped to the preferred grid if both the preferred grid and the required technology file attributes (end to end forbidden space, end of line one neighbor rule, uncut shape condition or end off preferred grid center rule) are defined for the layer with preferred grid.

By default (*false*), only cut metal ends must be on the preferred grid. The router has the flexibility to do random routing for natural line ends and can use patching to fix DRC violations on fixed preroutes, pins, and routing blockages.

When *true*, all metal ends must be snapped to the preferred grid.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [route_opt](#)
- [route_auto](#)
- [route_detail](#)

route.detail.force_max_number_iterations

Controls whether the maximum number of iterations must be run when DRCs do not converge.

Data Types

Boolean

Default false

r

Description

This application option controls whether the maximum number of iterations must be run when DRCs do not converge.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [route_opt](#)
- [route_auto](#)
- [route_detail](#)

route.detail.generate_extra_off_grid_pin_tracks

Specifies whether to generate extra off-grid routing tracks for pin connections. If set to true, the router more aggressively generates off-grid dynamic tracks.

Data Types

Boolean

Default false

Description

This application option specifies whether to generate extra off-grid routing tracks for pin connections.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [route_opt](#)
- [route_auto](#)
- [route_detail](#)

r

route.detail.generate_off_grid_feed_through_tracks

Specifies the effort level used to generate extra off-grid routing tracks for feedthrough nets between blockages.

Data Types

string

Default low

Description

This application option specifies the effort level used to generate extra off-grid routing tracks for feedthrough nets between blockages. If the blockages block the adjacent routing tracks, we need to add few off-grid tracks between them in order to allow the routing to go through the gap between the blockages. The higher the effort, the more extra off-grid tracks added.

Valid values are *low*, *medium*, and *high*.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [route_opt](#)
- [route_auto](#)
- [route_detail](#)

route.detail.group_route_special_design_rule_fixing_stage

Specifies the stage at which to fix special design rules when running the *route_group* command.

Data Types

string

Default late_routing

Description

This application option specifies the stage at which to fix special design rules when running the *route_group* command.

r

The special design rules include the iso-dense via enclosure rule. Fixing the special design rules requires significant routing resources, so you might not want to fix them at all stages.

You can fix these rules at the following stages:

- *late_routing* (the default) Check and fix these rules in the postroute stage, after the other DRC rules.

- *early_routing*

Check and fix these rules along with the other DRC rules.

- *none*

Never check and fix these rules.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [route_opt](#)
- [route_auto](#)
- [route_detail](#)

route.detail.hop_layers_to_fix_antenna

Controls whether the router performs layer hopping to fix antenna violations.

Data Types

Boolean

Default true

Description

This application option controls whether the router performs layer hopping to fix antenna violations.

See Also

- [get_app_option_value](#)
- [get_app_options](#)

r

- [report_app_options](#)
- [route_opt](#)
- [route_auto](#)
- [route_detail](#)

route.detail.ignore_contained_metal_for_needs_fat_contact_overlap_check

App option to control needs fat contact DRC analysis when overlapping metal shape is contained within overhang of via being checked.

Data Types

boolean

Default false

Description

When this app option is set to true and a metal shape is contained within via overhang, that metal shape is not considered for need fat contact DRC analysis or fixing.

See Also

- [get_app_option_value](#)
- [set_app_options](#)
- [route_detail](#)
- [route_group](#)
- [route_auto](#)

route.detail.ignore_drc

Controls whether the router ignores specific routing design rule checks (DRCs).

Data Types

list

Default ""

r

Description

This application option controls whether the router ignores specific routing design rule checks (DRCs). When a DRC is ignored, the router neither analyzes nor fixes those violations.

The valid DRC keywords are

- *same_net_metal_space*
Ignore all same net metal spacing violations.
- *same_net_enclosed_cut_space*
Ignore same net enclosed cut spacing violations.
- *all_same_net_drc_for_frozen_net*
Ignore all same net violations on frozen nets.

The default is *false* for all DRC types, which means that none of them are ignored.

Examples

The following example tells the router to ignore the same net metal spacing violations but not the same net enclosed cut spacing violations.

```
prompt> set_app_options \\  
-name route.detail.ignore_drc  
-value { {same_net_metal_space true} \\  
        {same_net_enclosed_cut_space false} }
```

If an empty list is specified then none of the DRC types are ignored.

```
prompt> set_app_options \\  
-name route.detail.ignore_drc  
-value { }
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [route_opt](#)
- [route_auto](#)
- [route_detail](#)

r

route.detail.ignore_self_fatness_for_fat_via_check

Set this app option for fat via DRC to ignore fatness of via being checked.

Data Types

boolean

Default false

Description

This option is used in fat via DRC check of Zroute. When use sets app option `route.detail.ignore_contained_metal_for_needs_fat_contact_overlap_check` or defines "`fatTblSelectExactContactCodeNumber = 1`" for a layer in tech-file, Zroute considers fatness of the via in the design.

Setting app option `route.detail.ignore_self_fatness_for_fat_via_check` to TRUE makes this DRC check ignore fatness of the via for rule analysis and fixing.

See Also

- [get_app_option_value](#)
- [set_app_options](#)
- [route_detail](#)
- [route_group](#)
- [route_auto](#)

route.detail.incremental_detail_route_special_design_rule_fixing_stage

Specifies the stage at which to fix special design rules during incremental detail routing (the *route_detail -incremental true* command).

Data Types

string

Default late_routing

Description

This application option selects the stage at which to fix special design rules during incremental detail routing (the *route_detail -incremental true* command).

r

The special design rules include the iso-dense via enclosure rule. Fixing the special design rules requires significant routing resources, so you might not want to fix them at all stages.

You can fix these rules at the following stages:

- *late_routing* (the default) Check and fix these rules in the postroute stage, after the other DRC rules.

- *early_routing*

Check and fix these rules along with the other DRC rules.

- *none*

Never check and fix these rules.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [route_opt](#)
- [route_auto](#)
- [route_detail](#)

route.detail.insert_diodes_during_routing

Specifies whether the router can use diode insertion to fix antenna violations.

Data Types

Boolean

Default false

Description

This application option specifies whether the router can use diode insertion to fix antenna violations.

See Also

- [get_app_option_value](#)
- [get_app_options](#)

r

- [report_app_options](#)
- [route_opt](#)
- [route_auto](#)
- [route_detail](#)

route.detail.insert_redundant_vias_layer_order_low_to_high

Sets the layer order in which vias are processed during redundant via insertion.

Data Types

Boolean

Default `false`

Description

This application option chooses the layer order for processing vias during redundant via insertion.

Valid values are *false* (the default) and *true*.

If the value is set to *false* (default), the tool will insert redundant vias starting on higher layers first, and then process lower layers. If the value is set to *true*, the tool will insert redundant vias starting on lower layers first, and then process upper layers.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [add_redundant_vias](#)

route.detail.macro_pin_antenna_mode

Specifies how macro cell pins are treated for antenna considerations.

Data Types

string

Default `normal`

r

Description

This application option specifies how macro cell pins are treated for antenna considerations.

The valid modes are

- *normal* (the default)

Treat macro cell pins as normal standard cell output pins.

- *jump*

Treat macro cell output pins as if they are connected to a huge antenna and break the antenna, which requires a jumper at the top metal layer.

- *top_layer*

Connect all input pins to macro cell output pins only through the highest routing layer of the net.

Examples

The following example sets the macro pin antenna mode to *jump*.

```
prompt> set_app_options \\  
        -name route.detail.macro_pin_antenna_mode -value jump
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [route_opt](#)
- [route_auto](#)
- [route_detail](#)

route.detail.max_antenna_fixing_pin_count_threshold

Specifies the maximum number of pins on a net on which antenna fixing is performed. Antenna fixing of power and ground nets is not affected by this option.

Data Types

integer

Default 50

r

Description

This application option specifies the maximum number of pins on a net on which antenna fixing happens. If a net has more than the specified number of pins, antenna fixing is skipped; otherwise, antenna fixing is performed.

The range is from -1 to 1,000,000.

The default is 50, which means antenna fixing is done for nets with up to 50 pins. If antenna fixing is intended for all nets, set this option value to -1.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [route_opt](#)
- [route_auto](#)
- [route_detail](#)

route.detail.max_antenna_pin_count

Specifies the maximum number of pins on a net on which antenna checking is performed.

Data Types

integer

Default -1

Description

This application option specifies the maximum number of pins on a net on which antenna checking is performed. If a net has more than the specified number of pins, antenna checking is skipped; otherwise, antenna checking is performed.

The range is from -1 to 1,000,000.

The default is -1, which means no limit.

See Also

- [get_app_option_value](#)
- [get_app_options](#)

r

- [report_app_options](#)
- [route_opt](#)
- [route_auto](#)
- [route_detail](#)

route.detail.merge_gates_for_antenna

Controls whether gates are merged for antenna analysis.

Data Types

Boolean

Default `true`

Description

This application option controls whether gates are merged for antenna analysis.

By default (*true*), when two or more gates share metal on a metal or via layer, antenna analysis uses the sum of the areas of the shared gates.

When set to *false*, the gates are not merged and antenna analysis uses only the individual gate area.

For example, assume that gate1 and gate2 share gate areas. When this application option is *true*, the antenna ratio for gate1 is calculated by using the sum of the gate areas: $M1b / (gate1_area + gate2_area)$. When this application option is *false*, the antenna ratio for gate1 is calculated by using only the gate1 area: $M1b / gate1_area$.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)
- [route_opt](#)
- [route_auto](#)
- [route_detail](#)

r

route.detail.min_max_layer_effort_level

Specifies the effort level applied to fixing "Min/max layer" DRCs.

Data Types

string

Default high

Description

This application option specifies the effort level for Min/max layer DRC fixing.

Valid values are *off*, *low*, *medium*, and *high* (the default).

The default effort level is *high*.

Setting this to "off" prevents all enforcement of min/max layer constraints in DR. In "low" effort mode, the min max layer constraints are not enforced after DR iteration 20. In "medium" effort mode, min max layer constraints are not enforced after DR iteration 40.

In the default "high" effort mode, they are always enforced.

Min/max layer DRCs are always checked and flagged, but if the min/max layer constraints are not enforced, there is no cost penalty for routing outside the min/max layer range, and no ripup-and-reroute to fix the DRCs.

Lower effort modes can be used to force the detailed router to give priority to fixing physical DRCs during later DR iterations at the possible expense of reduced compliance with min/max layer constraints.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)

route.detail.ndr_layer_based_taper_distance_threshold

Distance threshold for layer based tapering in the unit of 10-15 DPT layer average pitch. The default value is -1.0. Any negative number means that there is no distance limit for the layer based tapering. Once this option is set to a positive value, you can find this distance in the log info ZRT-706. The max value for the option is 20.0.

Data Types

Float

r

Default -1.0**Description**

This application option specifies the distance threshold for layer based tapering. This is to discourage running long wires on the lower layers.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [route_opt](#)
- [route_auto](#)
- [route_detail](#)

route.detail.ndr_spacing_length_threshold_factor

Auto-derives the length threshold of routing rules when it is not specified.

Data Types

int

Default "0"**Description**

This application option auto-derives the length threshold of routing rules when it is not specified.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [create_routing_rule](#)

route.detail.optimize_partition_size_for_drc

Optimize partition sizes within detail routing to help DRC convergence.

r

Data Types

Boolean

Default `false`

Description

This application option specifies whether to optimize the partition sizes to help DRC convergence. When *false* (the default), the partition sizes are not adjusted within detailed routing. When *true*, the partition sizes may be adjusted to help DRC convergence.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [route_opt](#)
- [route_auto](#)
- [route_detail](#)

route.detail.optimize_tie_off_effort_level

Specifies the effort level used to optimize wire length and via counts for tie-off nets.

Data Types

string

Default `low`

Description

This application option specifies the effort level used to optimize wire length and via counts for tie-off nets.

The valid values are

- *off*
No extra effort to optimize wire length and via counts for tie-off nets.
- *low* (the default)
Low effort to optimize wire length and via counts for tie-off nets.

r

- *high*

High effort to optimize wire length and via counts for tie-off nets.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [route_opt](#)
- [route_auto](#)
- [route_detail](#)

route.detail.optimize_wire_via_effort_level

Specifies the effort level used to optimize wire length and via counts.

Data Types

string

Default low

Description

This application option specifies the effort level used to optimize wire length and via counts.

The valid values are

- *off*

No extra effort to optimize wire length and via counts.

- *low* (the default)

Low effort to optimize wire length and via counts.

- *medium*

Medium effort to optimize wire length and via counts.

- *high*

High effort to optimize wire length and via counts.

r

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [route_opt](#)
- [route_auto](#)
- [route_detail](#)

route.detail.pin_taper_mode

Specifies the pin tapering mode. This option can be used when pins are thinner than the default width. The tool can force the directly connected shape to the pin to use pin width instead of the default width. Be careful to not use this option when the pin width is larger than the default width, it can create fuse routing patterns.

Data Types

string

Default `default_width`

Description

This application option specifies the pin tapering mode.

The valid values are

- *default_width* (the default)
Nondefault rule connections to pins are tapered to the default rule width.
- *pin_width*
Nondefault rule connections to pins are tapered to the pin width.
- *off*
Pin tapering is not performed.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)

r

- [route_opt](#)
- [route_auto](#)
- [route_detail](#)

route.detail.port_antenna_mode

Specifies how the ports (top-level pins) are treated for antenna considerations.

Data Types

string

Default float

Description

This application option specifies how the ports (top-level pins) are treated for antenna considerations.

The valid values are

- *float* (the default)
Treat ports as floating wires.
- *jump*
Treat ports as if they are connected to a huge antenna and break the antenna, which requires a jumper at the top metal layer.
- *top_layer*
Connect all input pins to ports only through the highest routing layer of the net.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [route_opt](#)
- [route_auto](#)
- [route_detail](#)

r

route.detail.post_process_special_design_rule_fixing_stage

Specifies the stage at which to fix special design rules during post-processing commands such as *add_redundant_vias*.

Data Types

string

Default `late_routing`

Description

This application option specifies the stage at which to fix special design rules during post-processing commands such as *add_redundant_vias*.

The special design rules include the iso-dense via enclosure rule. Fixing the special design rules requires significant routing resources, so you might not want to fix them at all stages.

You can fix these rules at the following stages:

- *late_routing* (the default) Check and fix these rules in the postroute stage, after the other DRC rules.
- *early_routing*
Check and fix these rules along with the other DRC rules.
- *none*
Never check and fix these rules.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [route_opt](#)
- [route_auto](#)
- [route_detail](#)

route.detail.repair_shorts_over_macros_effort_level

Controls the effort level applied to the automatic repairing of shorts over macros.

r

Data Types

enumerated

Default off**Description**

This application option controls the effort level applied to the automatic repairing of shorts over macros.

Valid values are *off* (the default), *low*, *medium*, and *high*.

If enabled, any shorts over macros that remain after detail routing can trigger the deletion of the involved wires and a full ECO rerouting of the involved nets (global routing, track assignment, and detail routing). Higher effort levels can increase the number of ECO repair iterations, which might decrease the final DRC count at the expense of runtime.

This option can significantly increase runtime if a design has many shorts over macros that are difficult to repair.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [route_opt](#)
- [route_auto](#)
- [route_detail](#)

route.detail.repair_shorts_over_macros_verbose

Controls the verbosity of output during the automatic repairing of shorts over macros that can be triggered by the `route.detail.repair_shorts_over_macros_effort_level` app option.

Data Types

Boolean

Default false**Description**

This option controls the verbosity of output during the automatic repairing of shorts over macros that can be triggered by the `route.detail.repair_shorts_over_macros_effort_level` app option.

r

If true, the log file will include details of each net and DRC that is triggering deletion of wire during the repairing of shorts over macros.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [route_opt](#)
- [route_auto](#)
- [route_detail](#)

route.detail.report_antenna_for_must_join_port_root

When there are multiple ports with must join port setting, report antenna violations for the root port only.

Data Types

Boolean

Default false

Description

This application option controls how to report antenna violations on ports with must join setting. When this option is set, antenna violations of the must join root ports are the only ones reported. For other ports without must join port setting, this option setting has no effect.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [route_opt](#)
- [route_auto](#)
- [route_detail](#)

r

route.detail.report_drc_totals_by_layer

Report total number of DRCs per layer.

Data Types

Boolean

Default false

Description

This application option allows DRC report to include the by layer breakdown DRCs per layer.

Examples

```
@@DRC COUNTS BY LAYER:
M1 : 355
M2 : 6
M3 : 1
M4 : 9
M5 : 7
M6 : 6
M7 : 4
M8 : 1
M9 : 1
M1-VIA1 : 32
VIA1 : 3
VIA0-M1 : 75
VIA1-M2 : 3
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)

route.detail.report_drc_types_by_layer

Report number of DRC based on DRC type and layer.

Data Types

Boolean

Default false

r

Description

This application option controls if needs to report total number of each type of DRC on each layer.

Examples

```
@@DRC COUNTS BY TYPE OF EACH LAYER:  
M2 : Short : 2  
M4 : Short : 2  
M5 : Short : 1  
M6 : Short : 2
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)

route.detail.report_hier_antenna_for_must_join_port_root

When there are multiple ports with must join port setting, report hierarchical antenna properties for the root port only.

Data Types

Boolean

Default false

Description

This application option controls how to report hierarchical antenna properties on ports with must join setting. When this option is set, antenna properties of the must join root ports are the only ones reported. For other ports without must join port setting, this option setting has no effect.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [route_opt](#)
- [route_auto](#)
- [route_detail](#)

r

route.detail.report_ignore_drc

Enables an additional DRC report and specifies the DRC types to be suppressed from this report.

Data Types

list

Default ""

Description

This application option enables an additional DRC report and specifies the DRC types to be suppressed from this report.

The additional report is printed after the DRC summary and includes all DRC violations, except internal-only types and those specified in the list.

The valid DRC name values are

```
"Access preference area"
"Antenna"
"Asymmetric via-cut to metal spacing"
"Cluster bbox"
"Comb routing direct connection"
"Comb routing maximum connections"
"Comb routing daisy-chain distance"
"Compute all drcs"
"Connection not within pin"
"Convex metal cut enclosure"
"Convex metal cut keepout"
"Crossing top-cell boundary"
"Dangling wire"
"Dense end of line enclosure"
"Dense short metal spacing"
"Diff net fat extension spacing"
"Diff net spacing"
"Diff net var rule spacing"
"Diff net via-cut spacing"
"Diff net via-cut to metal spacing"
"DFM end to end space keepout"
"Dog bone spacing"
"Edge-line via spacing"
"Enclosed unaligned via center spacing"
"Enclosed via spacing"
"End of line enclosure"
"End of line spacing"
"End of line to concave corner distance"
"End of line wire via enclosure"
"End to end space keepout"
"Fat metal via asymmetric enclosure"
```


r

```
"Fat shape"
"Fat wire via keepout area"
"Fat wire via keepout enclosure"
"Forbidden metal"
"Forbidden Pitch"
"Frozen layer"
"Greater than maximum width"
"h-shape"
"Illegal Bridge"
"Illegal width route"
"Jog Wire via Enclosure"
"Jog wire via keepout area"
"Joint wire via enclosure"
"Less than minimum area"
"Less than minimum edge length"
"Less than minimum enclosed area"
"Less than minimum length"
"Less than minimum width"
"Less than NDR width"
"Less than taper width"
"Line-end to short-edge (minEdgeLengthTbl)"
"Macro pin connection by offset"
"Maximum utilization"
"Min-max layer"
"Minimum extension"
"Minimum utilization"
"MisAligned via wire"
"Multiple pin connections"
"NDR contact code mismatch"
"Needs fat contact"
"Needs fat contact on extension"
"Needs redundant via"
"Non-preferred direction route"
"Off-grid"
"Offset via"
"One grid jog in non-preferred direction"
"Over max stack level"
"Protrusion length"
"Redundant via"
"Same net fat extension spacing"
"Same net spacing"
"Same net via-cut spacing"
"Secondary pg pin direct connection"
"Secondary pg pin maximum connections"
"Self aligned via cluster"
"Self aligned via cut spacing"
"Self aligned via invalid enclosure"
"Self aligned via with neighbor cut spacing"
"Self aligned zig-zag via with neighbor cut spacing"
"Self aligned zig-zag via cut spacing"
"Self aligned diagonal via cut spacing"
"Shielding unintended overlap"
"Short"
```

r

```
"Small jog"
"Soft spacing"
"Special notch spacing"
"T junction"
"T shape wire via enclosure"
"Tie to rail"
"Tie to rail directly"
"Two sided end/joint spacing"
"U shape spacing"
"Via-cut to metal spacing"
"Via-Metal Concave corner rule"
"Voltage area"
"Wide Metal Jog"
```

Note that the double quotation marks are required for any DRC name that includes spaces.

By default, the additional report is not generated.

Examples

The following example generates the additional DRC report and ignores "Fat wire via keepout enclosure" violations.

```
prompt> set_app_options -name route.detail.report_ignore_drc \\  
          -value {"Fat wire via keepout enclosure"}
```

The following example generates the additional DRC report and ignores the "Illegal Bridge" and "Multiple pin connections" violations.

```
prompt> set_app_options -name route.detail.report_ignore_drc \\  
          -value {"Illegal Bridge" "Multiple pin connections"}
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [route_opt](#)
- [route_auto](#)
- [route_detail](#)

route.detail.report_soft_drc_separately

Enables to exclude soft DRCs from the table "TOTAL VIOLATION" and report a new table "TOTAL SOFT VIOLATION" for the soft DRCs.

r

Data Types

Boolean

Default false

Description

This application option enables you to exclude soft DRCs from the TOTAL VIOLATION table and report it to the new table named TOTAL SOFT VIOLATIONS for the soft DRCs.

The new table is printed at the end of the DRC summary and includes all the soft DRCs.

Examples

The following example generates the DRC report and the soft DRC report.

```
icc2_shell> set_app_options -name route.detail.report_soft_drc_separately  
\\  
          -value true
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [route_opt](#)
- [route_auto](#)
- [route_detail](#)

route.detail.reuse_filler_locations_for_diodes

Specifies whether to reuse filler cell locations for inserting diodes.

Data Types

Boolean

Default true

Description

This application option specifies whether to reuse filler cell locations for inserting diodes.

Locations for diodes are chosen close to the violating pin. They may be an empty locations or location of filler cells. Detail route does not prefer empty location over filler cells or vice versa.

r

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [route_opt](#)
- [route_auto](#)
- [route_detail](#)

route.detail.save_after_iterations

Saves the intermediate cell after the specified iterations.

Data Types

list

Default ""

Description

This application option saves the intermediate cell after the specified iterations.

Examples

The following example shows how to save the design after DR iterations 0 and 1.

```
prompt> set_app_options \\  
        -block [current_block] -list {route.detail.save_after_iterations { 0  
        1 } }
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [route_opt](#)
- [route_auto](#)
- [route_detail](#)

r

route.detail.save_cell_prefix

Specifies the prefix to be used in the saved cell name.

Data Types

string

Default DR

Description

This application option specifies the prefix to be used in the saved cell name.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [route_opt](#)
- [route_auto](#)
- [route_detail](#)

route.detail.set_new_diodes_of_clock_nets_as_application_fixed

When this option is set to true, diodes inserted for antenna fixing of clock nets are marked as application fixed.

Data Types

Boolean

Default false

Description

New diodes inserted in current routing command to resolve antenna violations of clock nets are marked as application_fixed. This is done to discourage moving of these diodes later in the flow.

See Also

- [get_app_option_value](#)
- [get_app_options](#)

r

- [report_app_options](#)
- [route_opt](#)
- [route_auto](#)
- [route_detail](#)

route.detail.shift_diode_locations_for_port_protection

Specify this app option to shift port protection diodes and reconnect so they can protect the top-level port of the net.

Data Types

Boolean

Default false

Description

When a port protection diode is connected to top-level port above the layer of top-level port, it cannot protect it. When this app option is specified, - check_routes issues warnings related to such diodes. - detail route shift such diodes to near the ports and connects them within the layer of top-level port of the net.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [route_opt](#)
- [route_auto](#)
- [route_detail](#)

route.detail.skip_antenna_analysis_for_ground_nets

Setting this option to true makes routing commands skip antenna analysis and fixing for ground nets.

Data Types

bool

Default false

r

Description

By default, diode modes including diode mode 23 perform antenna analysis of PG nets. Setting this option to true makes router skip antenna analysis and fixing for ground nets.

Examples

```
set_app_options -list {route.detail.skip_antenna_analysis_for_ground_nets true}
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [route_opt](#)
- [route_auto](#)
- [route_detail](#)

route.detail.skip_antenna_analysis_for_nets

Specifies the nets to be skipped for antenna analysis.

Data Types

list

Default ""

Description

This application option is for user to specify nets to be skipped fro antenna analysis. You must specify the nets as a Tcl list; this option does not take a Tcl collection as input.

By default, the router analyzes the antenna violations for all signal and clock nets.

Examples

The following example shows how to skip antenna analysis for a net.

```
prompt> set_app_options \\  
        -block [current_block] -list  
        {route.detail.skip_antenna_analysis_for_nets n3567}
```

r

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [route_opt](#)
- [route_auto](#)
- [route_detail](#)

route.detail.skip_antenna_analysis_for_tie_low_pins

Setting this option to true makes routing commands skip antenna analysis and fixing for tie low pins.

Data Types

bool

Default false

Description

By default, diode modes including diode mode 23 perform antenna analysis of tie low pins. Setting this option to true makes router skip antenna analysis and fixing for tie low pins.

Examples

```
set_app_options -list {route.detail.skip_antenna_analysis_for_tie_low_pins true}
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [route_opt](#)
- [route_auto](#)
- [route_detail](#)

route.detail.skip_antenna_fixing_for_nets

Specifies the nets to be skipped for antenna fixing.

r

Data Types

list

Default ""

Description

This application option specifies the nets to be skipped for antenna fixing. You must specify the nets as a Tcl list; this option does not take a Tcl collection as input.

By default, the router fixes the antenna violations for all signal and clock nets.

Examples

The following example shows how to skip antenna fixing for a net.

```
prompt> set_app_options \\  
        -block [current_block] -list  
        {route.detail.skip_antenna_fixing_for_nets n3567}
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [route_opt](#)
- [route_auto](#)
- [route_detail](#)

route.detail.timing_driven

Specifies whether timing-driven routing is enabled.

Data Types

Boolean

Default false

Description

This application option specifies whether timing-driven routing is enabled.

r

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [route_opt](#)
- [route_auto](#)
- [route_detail](#)

route.detail.top_layer_antenna_fix_threshold

Specifies the threshold for fixing antenna violations on the top routing layer.

Data Types

integer

Default -1

Description

This application option specifies the threshold for fixing antenna violations on the top routing layer. Antenna violations on the top routing layer are fixed if they are within $((\text{safe_ratio}) * (1 + \text{top_layer_antenna_fix_threshold} * 0.1))$.

The range is from -1 to 10.

The default is -1, which means no limit.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [route_opt](#)
- [route_auto](#)
- [route_detail](#)

r

route.detail.topology_eco_effort_level

Specify the build-in post-detail-route topology eco effort level for secondary PG cluster control.

Data Types

string

Default "off"

Description

This application option specifies the build-in post-detail-route topology eco effort level for secondary PG cluster control. The option values are (off, low, medium, high). They define the maximum post-detail-route topology eco iterations to be (0, 1, 3, 5) respectively. This app option is off by default. It should only be used as a last resort because of the potential runtime impact. It will cost extra run time but reduction in secondary PG pin connections is not guaranteed if turned on. Instead of using this app option, users should review the whether the number of available tapping points for secondary PG connections is sufficient to satisfy the fanout limit.

This app option will trigger ECO routing at the end of Detail Routing to try to resolve secondary PG pin max fanout violations. Improvement in results cannot be guaranteed.

Examples

The following example shows how to use the command.

```
prompt> set_app_options \\  
        -block [current_block] -list {route.detail.topology_eco_effort_level  
        off}
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [route_detail](#)

route.detail.use_default_width_for_min_area_min_len_stub

Specifies whether to use default width stubs to fix minimum area and minimum length violations.

r

Data Types

Boolean

Default false

Description

This application option specifies whether to use default width stubs to fix minimum area and minimum length violations.

When *false* (the default), stubs are the same width as the via surround. When *true*, stubs are the default width.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [route_opt](#)
- [route_auto](#)
- [route_detail](#)

route.detail.use_lower_hierarchy_for_port_diodes

Controls the logic hierarchy in which diodes are inserted to fix antenna violations of physical ports (terminals).

Data Types

Boolean

Default false

Description

This application option controls the logic hierarchy in which diodes are inserted to fix antenna violations of physical ports (terminals).

By default (*false*), the diodes are inserted at the top level.

If set to *true* and the port belongs to a voltage area, the diodes are inserted in the logic hierarchy associated with the voltage area. If the port does not belong to a voltage area, the diodes are inserted at the top level regardless of the setting of this option.

r

Note that the diodes are inserted in the same physical block as the boundary pin regardless of the setting of this option.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [route_opt](#)
- [route_auto](#)
- [route_detail](#)

route.detail.use_wide_wire_effort_level**Data Types**

String

Default off**Description**

Specifies the effort level to use NDR vias and wires as much as possible within taper limit. The pin tapering is defined by the NDR rule and by the option "route.detail.use_wire_wire_to_xx_pin". This effort level works only on the pin connection where tapering is allowed. Once this option is set to low or high, we should see less tapering but longer runtime. This is because that we would spend more time to optimize the NDR pin connection to use bigger NDR vias and wires. The higher the effort, the longer the runtime and the higher the NDR wire and via usage.

Valid values are *off*, *low*, and *high*.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [route_opt](#)
- [route_auto](#)
- [route_detail](#)

r

route.detail.use_wide_wire_to_input_pin

Specifies whether pin tapering to input pins is disallowed.

Data Types

Boolean

Default false

Description

This application option specifies whether pin tapering to input pins is disallowed.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [route_opt](#)
- [route_auto](#)
- [route_detail](#)

route.detail.use_wide_wire_to_macro_pin

Specifies whether pin tapering to macro pins is disallowed.

Data Types

Boolean

Default false

Description

This application option specifies whether pin tapering to macro pins is disallowed.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [route_opt](#)

r

- [route_auto](#)
- [route_detail](#)

route.detail.use_wide_wire_to_output_pin

Specifies whether pin tapering to output pins is disallowed.

Data Types

Boolean

Default false

Description

This application option specifies whether pin tapering to output pins is disallowed.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [route_opt](#)
- [route_auto](#)
- [route_detail](#)

route.detail.use_wide_wire_to_pad_pin

Controls pin tapering to pad pins.

Data Types

string

Default same_as_macro_pin

Description

This application option controls pin tapering to pad pins.

The valid values are

r

- *same_as_macro_pin* (the default)

The setting is taken from the *route.detail.use_wide_wire_to_macro_pin* application option.

- *true*

Disallow pin tapering to pad pins.

- *false*

Allow pin tapering to pad pins.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [route_opt](#)
- [route_auto](#)
- [route_detail](#)

route.detail.use_wide_wire_to_port

Controls pin tapering to ports.

Data Types

string

Default *same_as_macro_pin*

Description

This application option controls pin tapering to ports.

The valid values are

- *same_as_macro_pin* (the default)

The setting is taken from the *route.detail.use_wide_wire_to_macro_pin* application option.

- *true*

Disallow pin tapering to ports.

- *false*

Allow pin tapering to ports.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [route_opt](#)
- [route_auto](#)
- [route_detail](#)

route.detail.user_defined_partition

Specifies the coordinates, in database units, of a rectangular partition to be added for routing.

Data Types

list

Default ""

Description

This application option specifies the coordinates, in database units, of a rectangular partition `{llx lly urx ury}` to be added for routing.

In general, the detail router automatically generates partitions to fix the DRCs in the design. In some situations, it might not generate the right-sized partitions to fix the DRCs. In this case, you can assist Zroute by adding the right-sized partitions. This option is intended only for advanced users. There is a limit on the size of the partition as it can cause huge memory and runtime overhead. As a rough guide, the area of the partition should be less than 600 pitches by 600 pitches on metal1.

Examples

The following example adds a user-defined partition for routing.

```
prompt> set_app_options \\  
-name route.detail.user_defined_partition \\  
-value {100 200 500 3000}
```

r

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [route_opt](#)
- [route_auto](#)
- [route_detail](#)

route.detail.var_spacing_to_same_net

Specifies if nondefault spacing is to be applied to shapes of the same net.

Data Types

Boolean

Default false

Description

This application option specifies if nondefault spacing is to be applied to shapes of the same net.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [route_opt](#)
- [route_auto](#)
- [route_detail](#)

route.detail.via_ladder_upper_layer_via_connection_mode

Controls the allowed types of via connections to the top layer of a via ladder.

Data Types

string

r

Default above**Description**

This application option controls the type of connection allowed by via to the top layer of a via ladder.

An illegal connection will result in a "Needs upper layer connection on via ladder" DRC.

Allowed settings are "any" and "above." The default is "above."

When set to "above" (the default), a connection to a via ladder must be made either by a wire connecting to the top layer, or by a via tapping into the top layer from above.

For example, if the top layer of a via ladder is M6, a legal connection will be a wire on M6 or a via from M7->M6.

When set to "any," a connection to a via ladder must be made either by a wire connecting to the top layer, or by a via tapping into the top layer from above or below.

For example, if the top layer of a via ladder is M6, a legal connection will be a wire on M6 or a via from M7->M6 or a via from M5->M6.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [route_detail](#)
- [insert_via_ladders](#)

route.global.advance_node_timing_driven_effort

Specifies the effort level for advance node timing-driven global routing.

Data Types

string

Default high**Description**

This application option specifies the effort level for advance node timing-driven global routing. The effort levels trade off between timing QoR and runtime in timing-driven global routing, with the high effort level emphasizing timing QoR and some runtime overhead.

r

This option takes effect only if the *route.global.timing_driven* application option is set to *true* and normal global routing flow.

Valid values are: *low*, *medium* and *high*(default).

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [route_global](#)
- [route_auto](#)

route.global.avoid_blocked_unbufferable_macro_area_in_opt

This option is to avoid blocked unbufferable macro area for server routing in optimization stage.

Data Types

Boolean

Default false

Description

This option is to avoid blocked unbufferable macro area for server routing in optimization stage.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)

route.global.coarse_grid_refinement

Enables the coarse grid refinement technique in global routing.

Data Types

Boolean

Default true

r

Description

This application option specified whether the tool uses a uniform or nonuniform coarse grid for global routing.

By default (*true*), the tool uses a nonuniform coarse grid. Nonuniform grids have an advantage of providing better QoR in difficult routing areas, such as channels, at the expense of slightly increased runtime.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [route_opt](#)
- [route_global](#)
- [route_auto](#)
- [route_eco](#)
- [route_group](#)

route.global.connect_pins_outside_routing_corridor

Enables (true) or disables (false) to connect to the pins outside the routing corridor.

Data Types

Boolean

Default true

Description

This application option enables (true) or disables (false) the global routing to connect the pins outside the routing corridor.

See Also

- [create_routing_corridor](#)
- [create_routing_corridor_shape](#)
- [report_routing_corridors](#)

r

route.global.crosstalk_driven

Enables (true) or disables (false) crosstalk-driven global routing.

Data Types

Boolean

Default false

Description

This application option enables (true) or disables (false) crosstalk-driven global routing.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [route_opt](#)
- [route_global](#)
- [route_auto](#)
- [route_eco](#)
- [route_group](#)

route.global.custom_track_modeling_enhancement

Turns on/off custom track modeling enhancement in global routing.

Data Types

Boolean

Default false

Description

This application option enables enhanced modeling of custom track during global routing.

Default of the option is (*false*), but we recommend setting it to (*true*) if custom track is present in the design.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [route_opt](#)
- [route_global](#)
- [route_auto](#)
- [route_eco](#)
- [route_group](#)

route.global.deterministic

Specifies the determinstic mode of global router in multi-threaded routing.

Data Types

string

Default on

Description

This application option is deprecated and has no effect. The global router is always deterministic.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [route_opt](#)
- [route_global](#)
- [route_auto](#)
- [route_eco](#)
- [route_group](#)

r

route.global.diode_insertion_dist_below_io_pin_layer

Specify diode insertion distance below io pin layer in gcells. The distance ranges from 5 to 100.

Data Types

Integer

Default 100

Description

This application option is used together with option `enable_diode_insertion_for_io_pin` for IO pin diode insertion. It can specify distance range for the connection to IO pins.

Examples

The following example sets the `route.global.diode_insertion_dist_below_io_pin_layer` option to 100.

```
prompt> set_app_options -as_user_default -list  
{route.global.diode_insertion_dist_below_io_pin_layer 100}
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [route_opt](#)
- [route_global](#)
- [route_auto](#)
- [route_eco](#)
- [route_group](#)

route.global.double_pattern_utilization_by_layer_name

Specifies the double patterning utilization percentage for each layer.

Data Types

list

Default ""

r

Description

This application option specifies the double-patterning utilization percentage for each layer.

The format for specifying the double-patterning utilization for a layer is

```
{layer utilization}
```

Specify the layer by using the layer name from the technology file.

The range for the utilization percentage is from 0 to 100. The default is 80 for the lowest two routing layers, 85 for the third routing layer, and 90 for any higher double-patterning layers. The value is always 100 for non-double-patterning layers.

For advanced nodes, the utilization setting is no longer supported and ZRT-138 message is suppressed.

Examples

The following example sets the *route.global.double_pattern_utilization_by_layer_name* option on layers M1, M2, and M5.

```
prompt> set_app_options -list \\  
        {route.global.double_pattern_utilization_by_layer_name \\  
          { {M1 85} {M2 85} {M5 70} }}
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [route_opt](#)
- [route_global](#)
- [route_auto](#)
- [route_eco](#)
- [route_group](#)

route.global.eco_honor_target_dly

Controls whether global routing honors the timing for existing target shapes in minDly net eco routing.

r

Data Types

Boolean

Default false**Description**

This application option controls whether global routing honors the timing for existing target shapes in minDly net eco routing.

By default (*false*), global routing does not consider the timing for existing target shapes in minDly net eco routing.

If the option is set to *true*, global routing honors the timing for existing target shapes in minDly net eco routing.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [route_opt](#)
- [route_global](#)
- [route_auto](#)
- [route_eco](#)
- [route_group](#)

route.global.effort_level

Specifies the global routing effort level.

Data Types

string

Default medium**Description**

This application option specifies the global routing effort level.

Valid values are

r

- *minimum*
Very fast runtime. Runs a maximum of two phases.
- *low*
Fast runtime. Runs a maximum of three phases.
- *medium* (default)
Medium runtime. Runs a maximum of four phases.
- *high*
Slow runtime, but better quality of results. Runs a maximum of five phases.
- *ultra*
Slower runtime. Use on really congested designs. Runs a maximum of seven phases.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [route_opt](#)
- [route_global](#)
- [route_auto](#)
- [route_eco](#)
- [route_group](#)

route.global.enable_buffer_aware

Enable buffer distance aware mode in route_global/route_group

Data Types

Boolean

Default false

Description

This option enables buffer distance aware mode in route_global/route_group.

r

User is suggested to set proper value for app option `route.global.max_buffer_distance` along with `route.global.enable_buffer_aware` in order to get expected result.

If `route.global.max_buffer_distance` is not set, the tool will derive the distance. The tool derived distance might not always get expected result.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [route_global](#)
- [route_group](#)

route.global.enable_clock_clustering

Enable clustering for clock routing.

Data Types

bool

Default false

Description

This application option enables clustering for clock net routing. The pins in the same cluster can be connected by lower layer metal wires without honoring the min routing layer constraint.

See Also

- [route_group](#)

route.global.enable_diode_insertion_for_io_pin

Enables (true) or disables (false) diode insertion for IO pin global routing.

Data Types

Boolean

Default false

r

Description

This application option enables (true) or disables (false) diode insertion for IO pin in global routing.

Examples

The following example sets the *route.global.enable_diode_insertion_for_io_pin* option to *true*.

```
prompt> set_app_options -as_user_default -list  
{route.global.enable_diode_insertion_for_io_pin true}
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [route_opt](#)
- [route_global](#)
- [route_auto](#)
- [route_eco](#)
- [route_group](#)

route.global.enable_gr_graph_lock

Enable low-level lock for GR multithread. It can help to improve the scalability of PG net routing.

Data Types

Boolean

Default true

Description

This application option controls whether to turn on low level locking feature in global routing.

By default (*true*), global routing turns off low level locking feature.

If the option is set to *false*, global routing turns off low level locking feature.

r

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [route_opt](#)
- [route_global](#)
- [route_auto](#)
- [route_eco](#)
- [route_group](#)

route.global.enhancement_on_non_preferred_direction_area

Models fat wires in non-preferred direction areas

Data Types

Boolean

Default false

Description

This application option considers non-preferred direction routing areas when modelling fat wires in the global router. By default (*false*), the fat wire modelling is solely dependent on the fat wire layer's preferred direction.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [route_global](#)
- [route_auto](#)
- [route_eco](#)
- [route_group](#)

r

route.global.exclude_blocked_gcells_from_congestion_report

Controls whether blocked global routing cells (gcells) are included in the overflow percentage calculation.

Data Types

Boolean

Default false

Description

This application option controls whether blocked global routing cells (gcells) are included in the overflow percentage calculation.

By default (*false*), the total number of global routing cells is used to calculate the overflow percentages in the congestion report.

If the option is set to *true*, the overflow percentage calculation excludes the global routing cells that are blocked for routing. In cases where the global router has to route through blocked global routing cells to find a solution, those global routing cells are counted in the overflow number, but not in the total number.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [route_opt](#)
- [route_global](#)
- [route_auto](#)
- [route_eco](#)
- [route_group](#)

route.global.export_soft_congestion_maps

Exports soft congestion maps into the database.

Data Types

Boolean

r

Default false**Description**

This application option specifies if the tool exports the soft congestion maps into the database. Soft congestion stored in the soft congestion maps is contributed by external or internal soft routing rules. The external soft routing rules can be specified by `create_routing_rule` command whereas the internal soft routing rules are generated by global router automatically for modeling purpose. By default, the soft congestion maps are not exported into the database.

See Also

- [route_global](#)
- [route_auto](#)
- [route_group](#)
- [create_routing_rule](#)
- [report_congestion](#)

route.global.extra_blocked_layer_utilization_reduction

Specifies the percentage of additional utilization to be reduced for areas where either the upper or lower layer is blocked and switching to the opposite layer would take more than one track resource.

Data Types

integer

Default 0**Description**

This application option specifies the percentage of additional utilization to be reduced for areas where either the upper or lower layer is blocked and switching to the opposite layer would take more than one track resource. You must specify a value from 0 to 100, inclusive.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)

r

- [route_opt](#)
- [route_global](#)
- [route_auto](#)
- [route_eco](#)
- [route_group](#)

route.global.force_full_effort

Forces the maximum number of global routing iterations based on the effort level.

Data Types

Boolean

Default false

Description

This application option forces the maximum number of global routing iterations based on the effort level to try to achieve better results with longer runtime. The effort level is determined from the setting of the *route.global.effort_level* application option or the *route_global -effort_level* option.

By default (*false*), the global router might stop before the maximum number of iterations, based on the incremental congestion reduction.

If set to *true*, the global router always performs the maximum number of routing phases based on the specified effort level. For example, if this option is set to *true* and the *route.global.effort_level* option is set to *high*, the global router always runs five routing phases.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [route_opt](#)
- [route_global](#)
- [route_auto](#)

r

- [route_eco](#)
- [route_group](#)

route.global.force_rerun_after_global_route_opt

Forces routing to be run after GR based optimization has been completed.

Data Types

Boolean

Default false

Description

This application option forces global router to execute inside `route_global` and `route_auto` commands after global-route-based optimization (GRO).

The default flow will skip global routing, inside `route_global` and `route_auto` commands, after the `global_route_opt` stage of `clock_opt` has been run. This is the recommended flow. Running additional global routing after GRO may change the timing picture and reduce the effectiveness of the GRO result.

If set to true, the global router may be run in either full or incremental mode. Running in incremental mode (`-reuse_existing_global_route true`) after GRO will limit the extent of the routing change.

Additionally, this application option can also be used to force the `route_group` command to execute after global-route-based optimization (GRO).

The default flow will skip the `route_group` command entirely after the `global_route_opt` stage of `clock_opt` has been run. This is the recommended flow. Forcing `route_group` command after `global_route_opt` will delete the global routing data in the design. In this case, user must ensure rerunning global routing.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [route_global](#)
- [route_auto](#)
- [route_group](#)

r

route.global.insert_gr_via_ladders

Inserts global route segment (glink) based via ladders on pins with via ladder constraints.

Data Types

Boolean

Default false

Description

This application option specifies if global router inserts global route segment (glink) based via ladders on pins with via ladder constraints.

Via ladder rules are specified in a *ViaRule* section of the technology file or by using the *create_via_rule* command.

Via ladder constraints are assigned to pins by using the *set_via_ladder_constraints* or *set_via_ladder_rules* command.

By default, the global router doesn't insert glink based via ladders.

See Also

- [route_global](#)
- [insert_via_ladders](#)
- [set_via_ladder_constraints](#)
- [set_via_ladder_rules](#)

route.global.interactive_multithread_mode

Controls whether multi-threading is enabled for the router in the interactive mode.

Data Types

Boolean

Default false

Description

When *true*, multi-threading is enabled for the router in the interactive mode. The number of used threads is controlled by host options settings. When *false*, multi-threading is disabled.

r

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [report_host_options](#)
- [set_host_options](#)

route.global.macro_area_iterations

Specifies number of iterations to honor macro track utilization.

Data Types

integer

Default 0

Description

This application option specifies the number of iterations that consider macro related track utilization settings during GR congestion map updates.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [route_opt](#)
- [route_global](#)
- [route_auto](#)
- [route_eco](#)
- [route_group](#)

route.global.macro_area_track_utilization

Specifies the percentage of tracks to be used over a macro.

r

Data Types

integer

Default 0

Description

This application option specifies the percentage of tracks to be used over a macro. You must specify a value between 0 and 100, inclusive. The default value 0 means that the router will try to automatically detect the best macro utilization for the design.

Examples

The following example sets the macro area utilization target to 75%.

```
prompt> set_app_options -list \\  
      {route.global.macro_area_track_utilization \\  
        75}
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [route_opt](#)
- [route_global](#)
- [route_auto](#)
- [route_eco](#)
- [route_group](#)

route.global.macro_boundary_track_utilization

Specifies the percentage of tracks to be used along macro boundaries.

Data Types

integer

Default 100

Description

This application option specifies the percentage of tracks to be used along macro boundaries. You must specify a value between 0 and 100, inclusive.

r

Examples

The following example sets the macro boundary utilization target to 75%.

```
prompt> set_app_options -list \\  
        {route.global.macro_boundary_track_utilization \\  
          75}
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [route_opt](#)
- [route_global](#)
- [route_auto](#)
- [route_eco](#)
- [route_group](#)

route.global.macro_boundary_width

Specifies the width of the macro boundary in terms of global route cells.

Data Types

integer

Default 5

Description

This application option specifies the width of the macro boundary in terms of global route cells. You must specify a value between 0 and 50, inclusive.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [route_opt](#)
- [route_global](#)

r

- [route_auto](#)
- [route_eco](#)
- [route_group](#)

route.global.macro_corner_track_utilization

Specifies the percentage of tracks to be used along macro corners.

Data Types

integer

Default 100

Description

This application option specifies the percentage of tracks to be used along macro corners. You must specify a value between 0 and 100, inclusive.

Examples

The following example sets the macro corner utilization target to 75%.

```
prompt> set_app_options -list \\  
        {route.global.macro_corner_track_utilization \\  
          75}
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [route_opt](#)
- [route_global](#)
- [route_auto](#)
- [route_eco](#)
- [route_group](#)

route.global.max_buffer_distance

Controls maximum buffer distance in microns for max_buffer_distance feature.

r

Data Types

*Int***Default** 0

Description

This option works with app option `route.global.enable_buffer_aware` true, which run global router in buffer distance aware mode.

The valid value for `max_buffer_distance` is from 0 to 10000. User is required to set proper buffer distance to guide global router. Global router will try to avoid routes longer than `max_buffer_distance` in the non-bufferable regions. This is a soft constraint; it may not be fully honored if it will cause large detour or congestion.

If `route.global.max_buffer_distance` is not set (default value 0), the tool will derive the distance. The tool derived distance might not always get expected result.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [route_global](#)
- [route_group](#)

route.global.net_type_based_blockage_boundary_gcell_enhancement

Enables the enhancement for the gCells on the boundary of net-type routing blockages.

Data Types

Boolean

Default false

Description

This application option specified whether the gCells partially covered by net-type routing blockages can be routed into.

By default (*false*), if a gCell is partially covered by a net-type routing blockage, the whole gCell is considered blocked even if there are some free tracks left. This is more conservative.

r

If the option is set to *true*, the gCell partially covered by a net-type routing blockage can be routed into if there is any free track not covered. This can reduce detour.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [route_opt](#)
- [route_global](#)
- [route_auto](#)
- [route_eco](#)
- [route_group](#)

route.global.power_aware_routing

Enables (true) or disables (false) power aware global routing.

Data Types

Boolean

Default false

Description

This application option enables (true) or disables (false) power aware global routing.

Examples

The following example sets the *route.global.power_aware_routing* option to *true*.

```
prompt> set_app_options -as_user_default -list {route.global.power_aware_routing true}
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [route_opt](#)
- [route_global](#)

r

- [route_auto](#)
- [route_eco](#)
- [route_group](#)

route.global.preroute_connection_enhancement

Enable better connection to the customized long detailed routing shape on high layers.

Data Types

boolean

Default false

Description

The enhancement is to make better connection to the customized long detailed routing shape on high layers. The default value is false.

See Also

- [route_global](#)
- [route_group](#)

route.global.support_excl_range2

Consider all versions of exclusion spacing ranges in fat parallel routing rule

Data Types

Boolean

Default false

Description

This application option considers both versions of exclusion spacing ranges when modelling the fat parallel routing rule around preroutes during global routing.

By default (*false*), the tool considers only the original version of the exclusion spacing range that is not length-dependent. With the option enabled, the length-dependent version of the exclusion spacing ranges is also considered, by making some assumptions to make it as accurate as possible.

r

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [route_global](#)
- [route_auto](#)
- [route_eco](#)
- [route_group](#)

route.global.timing_driven

Enables (true) or disables (false) timing-driven global routing.

Data Types

Boolean

Default false

Description

This application option enables (true) or disables (false) timing-driven global routing.

Examples

The following example sets the *route.global.timing_driven* option to *true*.

```
prompt> set_app_options -as_user_default -list {route.global.timing_driven true}
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [route_opt](#)
- [route_global](#)
- [route_auto](#)

r

- [route_eco](#)
- [route_group](#)

route.global.timing_driven_effort_level

Specifies the effort level for timing-driven global routing.

Data Types

string

Default high

Description

This application option specifies the effort level for timing-driven global routing. The effort levels trade off between timing QoR and DRC convergence, with the high effort level emphasizing timing QoR and the low effort level emphasizing DRC convergence.

This option takes effect only if the *route.global.timing_driven* application option is set to *true*.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [route_opt](#)
- [route_global](#)
- [route_auto](#)
- [route_eco](#)
- [route_group](#)

route.global.voltage_area_corner_track_utilization

Specifies the percentage of tracks to be used in the region within five global routing cells of a voltage area corner.

Data Types

integer

r

Default 100**Description**

This application option specifies the percentage of tracks to be used in the region within five global routing cells of a voltage area corner. You must specify a value between 0 and 100, inclusive.

A smaller utilization reduces the number of global routes that can go through a global routing cell by artificially setting the congestion in that area higher. If the neighboring global routing cells have lower congestion, the routing might be pushed to them; however, this is a function of many other metrics in the global router.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [route_opt](#)
- [route_global](#)
- [route_auto](#)
- [route_eco](#)
- [route_group](#)

route.track.allow_wire_outside_gcell

Specify if to allow track assignment to assign wires outside the current gcell.

Data Types

Boolean

Default true**Description**

This application option specifies if track assignment is allowed to assign wires outside the current gcell.

r

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)

route.track.crosstalk_driven

Enables (true) or disables (false) crosstalk-driven track assignment.

Data Types

Boolean

Default false

Description

This application option enables (true) or disables (false) crosstalk-driven track assignment.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)

route.track.timing_driven

Enables (true) or disables (false) timing-driven track assignment.

Data Types

Boolean

Default false

Description

This application option enables (true) or disables (false) timing-driven track assignment.

Examples

The following example sets the *route.track.timing_driven* application option to *true*.

```
prompt> set_app_options \  
-name route.track.timing_driven \  
-value true
```

r

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)

route_opt.flow.enable_cbuf_density_fll_checks**Data Types***boolean***Default** false**Description**

Enables density based FLL checks in many of the internal APIS for CBUF buffering engine for both *route_opt* and *hyper_route_opt* command. Enables cache based FLL bailout for the entire CBUF engine. Earlier this bailout was only limited to a single driver scope. To avoid runtime in non legalizable nodes. Enables bufferability based bailout for UBX and MAXCAP buffering heuristics to avoid runtime in cases nodes are not bufferable.

See Also

- [route_opt](#)

route_opt.flow.enable_cbuf_small_area_pass**Data Types***boolean***Default** false**Description**

Enables extra LDRC pass just like inverter pass or loadIsolation passes in cBuf LDRC. This pass is intended to fix legalizer issues existing during normal LDRC passes, it will try legalizing the smallest available lib cell and see if we can place that without any errors. Later on sizing engine can size the cell to whatever limit possible.

See Also

- [route_opt](#)

r

route_opt.flow.enable_ccd

Data Types

boolean

Default false

Description

Specifies whether concurrent clock and data optimization is enabled in the *route_opt* command. Enabling concurrent clock and data optimization helps setup timing closure.

See Also

- [route_opt](#)

route_opt.flow.enable_ccd_clock_drc_fixing

Data Types

string

Default auto

Description

Specifies whether timing-driven postroute clock drc fixing feature is turned on during the *route_opt* command.

When enable, this feature will fix clock transition and capacitance violations similar to clock drc fixing feature of the *synthesize_clock_trees -postroute* command, but with minimal impact on the timing QoR (both of setup and hold).

Allowable options are "auto", "always_on" and "always_off".

In *auto* mode, this feature is enabled when *enable_ccd* is on and disabled when *enable_ccd* is off. Note that when cell EM analysis is enabled in any of the scenarios, this feature is still enabled when *enable_ccd* is off.

In *always_on* mode, this feature is enabled regardless of *enable_ccd*.

In *always_off* mode, this feature is disabled regardless of *enable_ccd*.

This option needs to be turned on before running the *route_opt* command.

See Also

- [route_opt](#)

r

route_opt.flow.enable_clock_power_recovery

Enables power or area recovery from clock cells and registers during the *route_opt* command; applies to both the concurrent clock and data (CCD) and non-CCD flows.

Data Types

string

Default auto

Description

This option turns on clock power recovery or clock area recovery in *route_opt*. When power recovery mode is run, CCD will work on improving the power depending upon the scenario settings for power. If only leakage power scenarios are enabled, CCD will work on improving leakage power alone. If only dynamic power scenarios are enabled, CCD will work on improving dynamic power alone. If both leakage and dynamic power scenarios are enabled, CCD will work on improving total power. When area recovery mode is run, CCD will work on improving the area.

During power or area recovery, CCD performs clock cell sizing, clock repeater removal, clock repeater sizing and flip-flop sizing.

The valid values and the respective behavior are as follows:

- *auto* (default value)

Power recovery CCD will be run for designs with scenarios enabled for dynamic power if *route_opt.flow.enable_ccd* is true. There will not be any power recovery CCD run if *route_opt.flow.enable_ccd* is set as false or if scenarios are not enabled for dynamic power.

- *power*

Power recovery CCD will be run for designs with scenarios enabled for dynamic power or leakage power or both even if *route_opt.flow.enable_ccd* is set as false.

- *area*

Area recovery CCD will be run instead of power recovery CCD. This will be run irrespective of whether *route_opt.flow.enable_ccd* is set as true or false.

- *none*

This will disable power recovery or area recovery CCD irrespective of whether *route_opt.flow.enable_ccd* is set as true or false.

r

See Also

- [route_opt](#)

route_opt.flow.enable_irdrivenopt

Enables IR driven optimization during `route_opt`.

Data Types

Boolean

Default `false`

Description

When this option is set to *true*, IR driven optimization will be enabled during `route_opt`. This will reduce the IR drop by sizing datapath cells in high IR drop hotspots.

See Also

- [route_opt](#)

route_opt.flow.enable_ldrc_driver_side_buffer_filters**Data Types**

boolean

Default `false`

Description

Specifies whether we have enabled legalizer calls (for delay-fixing) and density-prefilter (for tran-fxing) during driver side buffering in LDRC/DELAY in both the `route_opt` and `hyper_route_opt` command. Set to `false` is the default behaviour. Setting `true` will enable both filters in LDRC/DELAY.

See Also

- [route_opt](#)

route_opt.flow.enable_ml_opto**Data Types**

boolean

r

Default false**Description**

Specifies whether the *route_opt* command runs ML timing-driven flow that reduces total power. Expect improvement in total power when the power flow is enabled and time.pba_optimization mode is set to path.

See Also

- [route_opt](#)

route_opt.flow.enable_multi_vector_ccd_pba_tns

Enable Multi-Vector Concurrent Clock and Data Optimization (CCD) for TNS fixing in hyper_route_opt phase 2 PBA timing mode flow.

Data Types*boolean***Default** false**Description**

This application option allows you to enable multi-vector CCD to improve total negative slack in *hyper_route_opt* phase2 PBA timing mode.

See Also

- [hyper_route_opt](#)

route_opt.flow.enable_multi_vector_ccd_pba_wns

Enable Multi-Vector Concurrent Clock and Data Optimization (CCD) for WNS fixing in hyper_route_opt phase 2 PBA timing mode flow.

Data Types*boolean***Default** false**Description**

This application option allows you to enable multi-vector CCD to improve worst negative slack in *hyper_route_opt* phase2 PBA timing mode.

r

See Also

- [hyper_route_opt](#)

route_opt.flow.enable_multibit_debanking

Enables debanking of multibit sequential cells to improve total negative slack.

Data Types

boolean

Default false

Description

This application option allows you to enable debanking of multibit sequential cells to improve total negative slack in *hyper_route_opt* command. The debanking is done to replace multibit sequential cells with lower bit multibit and single bit cells.

See Also

- [hyper_route_opt](#)

route_opt.flow.enable_new_pba_optimization**Data Types**

boolean

Default true

Description

Specifies whether the *route_opt* command flow optimizes the design using PBA timing.

See Also

- [route_opt](#)

route_opt.flow.enable_noise_closure**Data Types**

boolean

Default false

r

Description

Specifies whether tool identifies noise fixing as one of the focus costs to be fixed in some of the engines like LDRC/SEC-OPT during post-route optimization for both the *route_opt* and *hyper_route_opt* command. Set to false is the default behaviour. Setting true will enable noise fixing in LDRC and also few other optimization engines.

Enabling this option helps to fix static noise violations during all stages of *route_opt* and *hyper_route_opt* flows. Enabling this option may incur additional run time.

See Also

- [route_opt](#)
- [hyper_route_opt](#)

route_opt.flow.enable_power**Data Types**

boolean

Default true

Description

Specifies whether the *route_opt* command enables power optimization. By default, power type is determined based on scenario settings as shown in the table below. If this app option is set to false, power optimization will be disabled regardless of scenario settings.

Leakage-scenario Dynamic-scenario -> FlowType

False False -> Area

True False -> Leakage

False True -> Total

True True -> Total

See Also

- [route_opt](#)

route_opt.flow.enable_targeted_ccd_wns_optimization**Data Types**

boolean

r

Default false**Description**

Specifies whether targeted concurrent clock and data optimization is enabled in the *route_opt* command. Enabling targeted concurrent clock and data optimization helps setup timing closure on user-specified end points and/or path groups.

See Also

- [route_opt](#)

route_opt.flow.enable_voltage_drop_opt_ccd**Data Types***boolean***Default** false**Description**

Specifies whether IR Driven concurrent clock and data optimization (CCD) is enabled in the *route_opt* command. This helps reduce IR-drop by utilizing sizing and relocation.

See Also

- [route_opt](#)

route_opt.flow.size_only_mode

Controls whether the *route_opt* command runs the size-only flow.

Data Types*string***Default** ""**Description**

Controls whether the *route_opt* command runs the size-only flow to avoid inserting repeaters.

The valid values for this option are

r

- *true_footprint*

Allows resizing a cell to those library cells such that no legalization and eco routing is needed at the end of `route_opt/hyper_route_opt`. The tool will analyze the library cell to ensure that the cell is legal, the same physical characteristics (pin location/shapes) are kept and no new DRC violation is introduced after resizing. To apply *true_footprint* sizing, CLO needs to be enabled. `size_only` mode is only supported by 2nd `route_opt` and after. Turning it on at 1st `route_opt` will issue ROPT-017, and the flow will be converted back to regular `route_opt`.

- *footprint*

Allows resizing only to those library cells that have the same *cell_footprint* attribute as the original cell. You must ensure that the library cells have the correct *cell_footprint* attributes.

- *equal*

Allows resizing a cell to those library cells that have exactly the same physical area as the original cell.

- *equal_or_smaller*

Allows resizing a cell to those library cells that have the same or smaller physical area than the original cell.

- *none*

Disables the size-only mode.

- "" (empty string, the default)

Disables the size-only mode.

If you specify any other values, the tool ignores this option and disables size-only mode.

See Also

- [route_opt](#)

rtl_opt.clockgate.fanin_sequential

Controls whether the *rtl_opt* command executes fanin-based sequential clock gating optimizations.

Data Types

Boolean

Default false

r

Description

This application option controls whether the *rtl_opt* command executes fanin-based sequential clock gating optimizations.

When the value of this application option is set to *true* and the *RTL-ATL-New-Tech-Power* license is available, fanin-based sequential clock gating insertion is performed in the execution of the *rtl_opt* command.

Fanin-based sequential clock gating is the process of extracting and propagating enable conditions from upstream sequential elements in the data path for the purpose of enabling clock gating insertion for additional registers. A register can be gated during a given clock cycle if all the registers in the transitive fanin did not change state in the previous clock cycle.

To create a delayed version of the enable signal extracted from the transitive fanin registers, the tool creates a new register and uses the output pin of this register as the driver of the enable pin for the new clock gate. This new register is referred to as the "delay register".

If the transitive fanin registers do not share exactly the same enable signal, the tool searches for, or synthesizes, a logic cover of all the enables, and uses that as the enable signal to be delayed.

If the register to be clock gated has an enable signal already, either as a synchronous enable pin, as a feedback loop to the data pin, or as the enable of a regular clock gate, a special circuitry is added at the input pins of the delay register to guarantee the logical correctness of the transformation.

Given that new logic is created to implement fanin-based sequential clock gating, a small increase in combinational and sequential area is expected.

Fanin-based sequential clock gating insertion occurs at two different stages of the *rtl_opt* flow. During the *conditioning* stage, the analysis is limited to the local hierarchy of the registers. During the *estimation* stage, the tool performs analysis across hierarchical boundaries.

By default, the name of fanin-based sequential clock gates inserted by the tool begins with the "seq_clock_gate_fi_" prefix. However, you can set this up by using the `compile.clockgate.fanin_sequential_name_prefix` application option. The name of the delay register inserted by the tool as part of the transformation begins with the "seqcg_delay_reg_" prefix, but it can be set up by the use of the `compile.clockgate.sequential_delay_register_name_prefix` application option. In both cases, the remaining part of the name follows the standard clock-gating naming conventions.

r

CONFIGURATION OPTIONS

Fanin-based sequential clock gates inserted by the tool honor the settings in the *set_clock_gate_style* command, which applies to all types of clock gates. However, following configuration commands are specific to fanin-based sequential clock gating:

Use the *set_fanin_sequential_clock_gating_options* command to specify structural options for fanin-based sequential clock gating, such as the minimum bitwidth and maximum fanout of a bank. And use the *report_fanin_sequential_clock_gating_options* command to report these settings.

Use the *set_fanin_sequential_clock_gating_objects* command to control which objects should be included or excluded from fanin-based sequential clock gating insertion. And use the *report_fanin_sequential_clock_gating_objects* command to report these settings.

REQUIRED CONDITIONS

Regardless of settings made with the *set_fanin_sequential_clock_gating_options* and *set_fanin_sequential_clock_gating_objects* commands, the following conditions prevent fanin-based sequential clock gating insertion for a given register:

- The register under analysis contains objects other than edge-triggered flip-flops in its transitive fanin.
- The register under analysis, or any registers in its transitive fanin, have asynchronous state changes.
- The enable extracted from registers in the transitive fanin depends on their output pins.
- The registers in the transitive fanin are controlled by a different base clock, or triggered by the opposite clock edge.
- At least one of the registers in the transitive fanin is always-enabled.
- Either the next state function of the register under analysis, or the registers in its transitive fanin, depending on logic that contains a *dont_care* condition in the RTL.
- The register under analysis is multibit and its slices do not share the same enable signal.
- The tools finds that transformation is not likely to provide a power reduction benefit given the switching activity conditions.

REPORTING

When fanin-based sequential clock gating is enabled, the *report_clock_gating* command automatically displays an additional summary table showing the gating statistics.

Use the *report_clock_gating -ungated* command to report the reasons why fanin-based sequential clock gating is not inserted for each ungated register.

r

Use the `get_clock_gates` command to get a collection with all types of clock gates, including fanin-based sequential clock gates. And use the `get_clock_gates -type fanin_seqcg` command to get a collection with fanin-based sequential clock gates only.

FORMAL VERIFICATION SUPPORT

The use of the *Formality* tool is required for synthesis verification when fanin-based sequential clock is enabled. Third party formal verification tools are not supported. When fanin-based sequential clock gating is enabled, there is limited support for the ECO flow. Please refer to the *Formality* documentation for further details.

See Also

- [get_clock_gates](#)
- [report_clock_gating](#)
- [report_fanin_sequential_clock_gating_objects](#)
- [report_fanin_sequential_clock_gating_options](#)
- [set_clock_gate_style](#)
- [set_fanin_sequential_clock_gating_objects](#)
- [set_fanin_sequential_clock_gating_options](#)

rtl_opt.clockgate.self_gating

Controls whether the `rtl_opt -to estimation` command executes self-gating optimizations.

Data Types

Boolean

Default false

Description

This application option controls whether the `rtl_opt -to estimation` command executes self-gating optimizations.

When the value of this application option is *true*, self-gating optimization is performed in the execution of the `rtl_opt -to estimation` command, at the *estimation* stage.

Given that new logic is created to implement self-gating, a small increase in combinational and sequential area is expected.

r

The name prefix of inserted self-gates can be controlled by using application option `compile.clockgate.self_gate_name_prefix`. The remaining part of the name follows the standard clock-gating naming conventions.

CONFIGURATION OPTIONS

Self-gates inserted by the tool honor the settings in the `set_clock_gate_style` command, which applies to all types of clock gates. However, following configuration commands are specific to self-gating:

Use the `set_self_gating_objects` command to control which objects should be included, excluded or forced from self-gating insertion, as well as the type of comparator logic (XOR, NAND, OR, AUTO) to use on them. And use the `report_self_gating_objects` command to report these settings.

Use the `set_self_gating_options` command to specify structural options for self-gating, that are instance-specific at the hierarchical level, such as the minimum and maximum bitwidth of a bank, or how the feature interacts with the presence of tool-inserted clock gates. And use the `report_self_gating_options` command to report these settings.

Use the `set_global_self_gating_options` command to control options for self-gating behavior that are applicable throughout the entire design. And use the `report_global_self_gating_options` command to report these settings.

REPORTING

When self-gating is enabled, the `report_clock_gating` command automatically displays an additional summary table showing the gating statistics.

Use the `report_clock_gating -ungated` command to report the reasons why self-gating is not inserted for each ungated register.

Use the `get_clock_gates` command to get a collection with all types of clock gates, including self-gates. And use the `get_clock_gates -type self_gate` command to get a collection with self-gates only.

NOTES

Inheritance of timing exceptions is not supported in self-gating. Timing violations introduced in the self-gate enable pin can be avoided by excluding affected registers using the `set_self_gating_objects -exclude object_list` command.

See Also

- [get_clock_gates](#)
- [report_clock_gating](#)
- [report_global_self_gating_options](#)

r

- [report_self_gating_objects](#)
- [report_self_gating_options](#)
- [set_global_self_gating_options](#)
- [set_self_gating_objects](#)
- [set_self_gating_options](#)

rtl_opt.commit.enable_respective_lib

Specifies if rtl_opt should commit blocks to different design libraries, one design per library.

Data Types

String

Default false

Description

By default, tool will commit the sub-block into same top design library. If user set this application-option to true, tool will commit each sub-block to different design library.

See Also

- [commit_block](#)

rtl_opt.conditioning.disable_boundary_optimization_and_auto_ungrouping

Specify if auto-ungrouping and boundary optimization across logical hierarchy boundaries must be disabled to preserve the hierarchies.

Data Types

String

Default false

Description

By default, the rtl_opt command automatically ungroups logical hierarchies during optimization to merge sub-designs of a given level of the hierarchy into the parent design. It removes hierarchical boundaries to improve timing by reducing the levels of logic and to improve area by sharing logic.

r

By default, the command also does boundary optimization across the logical hierarchy boundaries to propagate equal-opposite logic, phase inversion, constants and unloaded ports.

To disable automatic ungrouping and boundary optimization, set this application-option to true. During rtl_opt runtime, this is equivalent to having set the following application options to false:

```
compile.flow.autoungroup
compile.flow.boundary_optimization
compile.flow.constant_and_unloaded_propagation_with_no_boundary_opt
```

Changing this application option will not directly effect the setting of above three compile.flow.* application options. The application option will take effect implicitly when the rtl_opt command is executed.

Note that this app-option's priority is higher than the above three compile.flow.* application options when you set them both in rtl_opt flow.

See Also

- [report_app_options](#)
- [set_app_options](#)

rtl_opt.conditioning.enable_constraint_name_tracking

Specify the enabling of constraints name tracking regardless of its cell_pin name changing during the block_conditioning stage of the RTLA hierarchical flow.

Data Types

String

Default False

Description

When a user applies a constraint that refers to GTECH pins, which can happen when the constraints are loaded, the constraints will be lost when the blocks are re-loaded into the top level after their distributed runs have completed due to the cell_pin name will be changed during the block_conditioning step. Specify this app_option as true for enabling of constraints name tracking regardless of its cell_pin name changing during the block_conditioning stage of the RTLA hierarchical flow.

See Also

- [report_app_options](#)
- [set_app_options](#)

rtl_opt.conditioning.enable_initial_block_budgets

Specify if the tool has to generate initial block IO budgets to use during block conditioning stage of rtl_opt and how the budgets must be computed.

Data Types

String

Default percentage

Description

Specify whether the tool has to compute and use initial block IO budgets during block conditioning stage of rtl_opt flow since the actual budgets are not available at this stage of the flow.

Further, specify if percentage based or logic-depth based block IO budgets are to be used. Depending on the method indicated, initial budgets will be generated during split_constraints.

The valid values: false, percentage, logic_depth.

false: Disable the generation and use of initial block IO budgets during block conditioning stage.

percentage: Compute percentage based block IO budgets during split_constraints. Control what percentage of the available path delay through the pin should be allocated for IO delay by the budgeter using app-option rtl_opt.conditioning.initial_percent_block_budgets.

logic_depth: Compute initial block IO budgets based on logic-depth of the timing path.

See Also

- [report_app_options](#)
- [set_app_options](#)
- [split_constraints](#)

rtl_opt.conditioning.feedthrough_percent_block_budgets

Specify the input and output delay percentage which will be used to calculate the delay value for feedthrough pins of the block.

r

Data Types

*List***Default** {50 50}

Description

Specify the input and output delay percentage which will be used to calculate the delay value for feedthrough pins of the block. i.e. {20 30} would assign a 20% budget for inputs and 30% for outputs.

See Also

- [report_app_options](#)
- [set_app_options](#)
- [split_constraints](#)

rtl_opt.conditioning.ignore_repeaters_for_logic_depth_budgets

Calculate Budget by ignoring repeater cells.

Data Types

*String***Default** false

Description

Calculate Budget by ignoring repeater cells, this option only works when the value of 'rtl_opt.conditioning.enable_initial_block_budgets' is 'logic_depth'. The default value is false.

See Also

- [report_app_options](#)
- [set_app_options](#)
- [split_constraints](#)

rtl_opt.conditioning.initial_percent_block_budgets

Specify what percentage of the available path delay through the block input and output pins should be allocated for IO delays by the budgeter during percentage based IO budgeting.

r

Data Types

List

Default {30 30}

Description

Using this application option, user can control what percentage of the available path delay through the block pins should be allocated for IO delays by the budgeter when percentage based block IO budgeting is enabled through the app-option *rtl_opt.conditioning.enable_initial_block_budgets* to generate initial block IO budgets to use during block-conditioning stage of rtl_opt.

By default, 30% of available time between launch and capture endpoints (pre-dominantly it would be clock-period) will be allocated for both input and output delays of the block IOs.

Different percentage values could be allocated for input and output ports.

See Also

- [report_app_options](#)
- [set_app_options](#)
- [split_constraints](#)

rtl_opt.estimated.enable_dtdp

Specifies if direct timing driven placement is to be enabled in the RTLA estimation flow.

Data Types

String

Default false

Description

By default, tool will not enable direct timing driven placement. If user set this application-option to true, tool will enable the direct timing driven placement in the RTLA estimation.

rtl_opt.estimated.placement_congestion_effort

Specifies the placement congestion effort in the RTLA estimation flow. Allowed values are "high", "medium", "none".

Data Types

String

r

Default none**Description**

By default, the congestion effort in the RTL A estimation flow is "none". The user may set it to a higher value based on the effort it needs for the congestion alleviation.

rtl_opt.estimation.tieopt

Specifies if tie cells optimization is to be enabled in the RTL A estimation flow.

Data Types*String***Default** false**Description**

By default, tool will not enable tie cell optimization. If user set this application-option to true, tool will enable the tie cells optimization in the RTL A estimation.

rtl_opt.explore_design.run_baseline

Specifies whether add existing baseline to design exploration run.

Data Types*boolean***Default** false**Description**

By default, the explore design option does not include the existing baseline. However, if this application option is set to true, the existing baseline is incorporated into the design exploration run.

rtl_opt.floorplan.enable_feedthroughs

Specify if feedthrough creation is allowed on all the block instances in the design during floorplanning stage of rtl_opt flow.

Data Types*String***Default** true

r

Description

By default, tool will create feedthroughs on all the block instances in the design as necessary during block-level pin assignment step within the floorplanning stage of the hierarchical rtl_opt flow.

Feedthroughs are inserted on the block boundary using virtual-flat global route based pin placement during top-down hierarchical auto-floorplanning or design planning stage.

Set the application-option to false to disable feedthrough creation on all the blocks. Further, if user intends to allow feedthroughs on only certain blocks then set this application-option to false and use *set_block_pin_constraints -allow_feedthroughs* pin constraint to define user requirements.

See Also

- [set_block_pin_constraints](#)

rtl_opt.floorplan.skip_stages

Specify if any of the floorplanning steps must be skipped during hierarchical top-down design planning when some of the floorplan information is already loaded through DEF.

Data Types

String

Default ""

Description

During hierarchical rtl_opt flow, the floorplanning stage performs auto-floorplanning which includes various design planning steps comprising of floorplan initialization, block and voltage-area shaping, hard-macro placement, top-level port placement, block-level pin placement with feedthrough creation, timing estimation and finally budgeting.

In some cases, users might want to load partial floorplan information through a DEF file which could define hard-macro, port locations etc. In such scenarios, to ensure that the tool doesn't disturb those user-intended floorplan details, corresponding floorplanning step could be skipped though this app-option.

Valid floorplaning stages: *initialize_floorplan*, *shape_blocks*, *hier_placement*, *top_level_pin_assignment*, *block_level_pin_assignment* and *budgeting*.

See Also

- [report_app_options](#)
- [set_app_options](#)

r

rtl_opt.flow.enable_app_options_mapping

Specifies if FC to RTLA app options mapping is to be enabled in the rtl_opt command flow.

Data Types

String

Default false

Description

By default, tool will not enable FC to RTLA app options mapping. If user set this application-option to true, tool will enable the FC to RTLA app options mapping in the rtl_opt flow.

rtl_opt.flow.enable_cts

Specify if FAST CTS must be enabled during rtl_opt to build a quick and adequate clock tree structure for the purpose clock network power estimation purpose only.

Data Types

boolean

Default false

Description

By default, rtl_opt command won't perform FAST CTS which is designed purely for the purpose of building a clock tree buffer structure which could be used for clock network power estimation only.

Please set this application option to true prior to running rtl_opt command.

Please note that the clock tree built by FAST CTS engine will not be used when reporting timing QoR as the goal of this clock structure is for power estimation only.

Please apply all necessary CTS configuration(including Non-default routing rules and DRCs such as max-transition, max-capacitance & max-fanout) before rtl_opt to be honered during FAST CTS step embedded into rtl_opt (runs post the estimation stage of rtl_opt is completed) and provide more accurate clock power QoR estimation. Standalone method to perform FAST CTS is not available.

rtl_opt.flow.fully_abutted_style_floorplan

Specify if the top-level floorplan style is of fully-abutted type with no channels between the block instances in the design so that rtl_opt hierarchical flow is updated accordingly.

r

Data Types

*String***Default** false

Description

Fully abutted style floorplan design are those which have no channels between the block instances in the design or otherwise those that don't have any logic gates at the top-level. In such cases, top-conditioning and top-estimation stages of the rtl_opt hierarchical flow need not be performed.

When this application-option is enabled, rtl_opt hierarchical flow will be automatically updated to skip conditioning and estimation steps for top-level.

See Also

- [report_app_options](#)
- [set_app_options](#)

rtl_opt.place.run_floorplan_mode_global_route

Specify if floorplan mode global router should be used in congestion driven placement.

Data Types

*String***Default** true

Description

In congestion driven placement in rtl_opt flow, placer uses global router to route the design to determine where the congestion happens. If this app option is set to true, a faster floorplan mode global routing will be used to estimate the congestions. This will give much faster runtime in global routing stage. If this app option is set to false, then a higher effort global routing will be used to estimate the congestions which could lead to slower runtime.

See Also

- [report_app_options](#)
- [set_app_options](#)

rtlfp.flow.default_tip_flipflop

Specify default flip flop library cell to be used in compile_physical_rtl

s

Data Types

String

Default ""**Description**

This option specifies the default flip flop library cell to be used in *compile_physical_rtl* command. When this option is specified, user does not need to specify flip flop library cell in the feedthrough input file to *compile_physical_rtl* command. Otherwise user will need to specify one in the feedthrough input file.

rtlfp.flow.tip_rtl_mode

Enables TIP RTL mode.

Data Types

Boolean

Default false**Description**

When this option is enabled, user will be able to use TIP RTL Flow where it is possible to insert feedthroughs and pipeline registers in RTL as per design plan.

Upon enabling this option, user will also be able to generate feedthroughs and pipeline registered inserted RTLs using *write_shadow_rtl* command.

See Also

- [write_shadow_eco](#)

S

sdc_version

Specifies the SDC version that was written. Used in context of reading an SDC file.

Data Types

string

Default latest version

Description

The *sdc_version* variable is meaningful only within the context of reading an SDC file. Setting it outside an SDC file has no effect, other than to produce an informational message.

The *write_sdc* command writes a command to the SDC file to set the *sdc_version* variable to the version that was written. You do not have any control over the version of SDC that is written. The latest version is written. When *read_sdc* reads the SDC file, it validates the version specified in the file (if present) with the version requested by the command.

See Also

- [read_sdc](#)
- [write_sdc](#)

selection_logging_no_core_action

This variable determines how the *change_selection_no_core* command behaves.

Data Types

String

Default ""

Description

This variable determines how the Tcl command *change_selection_no_core* behaves.

selection_logging_too_many_objects_action

This variable determines how the *change_selection_too_many_objects* command behaves.

Data Types

String

Default ""

Description

This variable determines how the *change_selection_too_many_objects* Tcl command behaves.

sh_allow_tcl_with_set_app_var

Allows the *set_app_var* and *get_app_var* commands to work with application variables.

Data Types

string

Default application specific

Description

Normally the *get_app_var* and *set_app_var* commands only work for variables that have been registered as application variables. Setting this variable to *true* allows these commands to set a Tcl global variable instead.

These commands issue a CMD-104 error message for the Tcl global variable, unless the variable name is included in the list specified by the *sh_allow_tcl_with_set_app_var_no_message_list* variable.

See Also

- [get_app_var](#)
- [set_app_var](#)

sh_allow_tcl_with_set_app_var_no_message_list

Suppresses CMD-104 messages for variables in this list.

Data Types

string

Default application specific

Description

This variable is consulted before printing the CMD-104 error message, if the *sh_allow_tcl_with_set_app_var* variable is set to *true*. All variables in this Tcl list receive no message.

See Also

- [get_app_var](#)
- [set_app_var](#)

sh_arch

Indicates the system architecture of your machine.

Data Types

string

Default platform-dependent

Description

The *sh_arch* variable is set by the application to indicate the system architecture of your machine. Examples of machines being used are sparcoS5, amd64, and so on. This variable is read-only.

sh_auto_complete_display_count

Controls the tool language used by the application interactive console.

Data Types

integer

Default -1

Description

This variable sets the threshold for the number of tab completion matches. If the number of matches exceeds the threshold then results will display in "bucketed" format, which groups the results into a maximum number of prefixes that is less than the threshold and displays each prefix with a count of the number of matches with that prefix.

To determine the current value of this variable, use the *get_app_var sh_auto_complete_display_count* command.

sh_command_abbrev_mode

Sets the command abbreviation mode for interactive convenience.

Data Types

string

Default application specific

Description

This variable sets the command abbreviation mode as an interactive convenience. Script files should not use any command or option abbreviation, because these files are then susceptible to command changes in subsequent versions of the application.

Although the default value is *Anywhere*, it is recommended that the site startup file for the application set this variable to *Command-Line-Only*. It is also possible to set the value to *None*, which disables abbreviations altogether.

To determine the current value of this variable, use the `get_app_var sh_command_abbrev_mode` command.

See Also

- [sh_command_abbrev_options](#) Turns off abbreviation of command dash option names when false.
- [get_app_var](#)
- [set_app_var](#)

sh_command_abbrev_options

Turns off abbreviation of command dash option names when false.

Data Types

boolean

Default application specific

Description

When command abbreviation is currently off (see `sh_command_abbrev_mode`) then setting this variable to false will also not allow abbreviation of command dash options. This variable also impacts abbreviation of the values specified to command options that expect values to be one of an allowed list of values.

This variable exists to be backward compatible with previous tool releases which always allowed abbreviation of command dash options and option values regardless of the command abbreviation mode.

It is recommended to set the value of this variable to false.

To determine the current value of this variable, use the `get_app_var sh_command_abbrev_options` command.

See Also

- [sh_command_abbrev_mode](#) Sets the command abbreviation mode for interactive convenience.
- [get_app_var](#)
- [set_app_var](#)

sh_command_log_file

Specifies the name of the file to which the application logs the commands you executed during the session.

Data Types

string

Default empty string

Description

This variable specifies the name of the file to which the application logs the commands you run during the session. By default, the variable is set to an empty string, indicating that the application's default command log file name is to be used. If a file named by the default command log file name cannot be opened (for example, if it has been set to read only access), then no logging occurs during the session.

This variable can be set at any time. If the value for the log file name is invalid, the variable is not set, and the current log file persists.

To determine the current value of this variable, use the *get_app_var sh_command_log_file* command.

See Also

- [get_app_var](#)
- [set_app_var](#)

sh_continue_on_error

Allows processing to continue when errors occur during script execution with the *source* command.

Data Types

Boolean

Default application specific

Description

It is recommended to use the `-continue_on_error` option to the `source` command instead of this variable because that option only applies to a single script, and not the entire application session.

When set to `true`, the `sh_continue_on_error` variable allows processing to continue when errors occur. Under normal circumstances, when executing a script with the `source` command, Tcl errors (syntax and semantic) cause the execution of the script to terminate.

When `sh_continue_on_error` is set to `false`, script execution can also terminate due to new error and warning messages based on the value of the `sh_script_stop_severity` variable.

To determine the current value of the `sh_continue_on_error` variable, use the `get_app_var sh_continue_on_error` command.

See Also

- [get_app_var](#)
- [set_app_var](#)
- [source](#)
- [sh_script_stop_severity](#) Indicates the error message severity level that would cause a script to stop running before it completes.

sh_deprecated_is_error

Raise a Tcl error when a deprecated command is executed.

Data Types

Boolean

Default application specific

Description

When set this variable causes a Tcl error to be raised when an deprecated command is executed. Normally only a warning message is issued.

See Also

- [get_app_var](#)
- [set_app_var](#)

sh_dev_null

Indicates the current null device.

Data Types

string

Default platform dependent

Description

This variable is set by the application to indicate the current null device. For example, on UNIX machines, the variable is set to `/dev/null`. This variable is read-only.

See Also

- [get_app_var](#)

sh_disabled_is_error

Raise a Tcl error when a disabled command is executed.

Data Types

Boolean

Default application specific

Description

When set this variable causes a Tcl error to be raised when a disabled command is executed. When false, only a warning message is issued.

Disabled commands have no effect.

See Also

- [get_app_var](#)
- [set_app_var](#)

sh_enable_page_mode

Displays long reports one page at a time (similar to the UNIX *more* command).

Data Types

Boolean

Default application specific

Description

This variable displays long reports one page at a time (similar to the UNIX *more* command), when set to *true*. Consult the man pages for the commands that generate reports to see if they are affected by this variable.

To determine the current value of this variable, use the *get_app_var sh_enable_page_mode* command.

See Also

- [get_app_var](#)
- [set_app_var](#)

sh_enable_stdout_redirect

Allows the redirect command to capture output to the Tcl stdout channel.

Data Types

Boolean

Default application specific

Description

When set to *true*, this variable allows the redirect command to capture output sent to the Tcl stdout channel. By default, the Tcl *puts* command sends its output to the stdout channel.

See Also

- [get_app_var](#)
- [set_app_var](#)

sh_help_shows_group_overview

Changes the behavior of the "help" command.

Data Types

string

Default application specific

Description

This variable changes the behavior of the *help* command when no arguments are specified to help. Normally when no arguments are specified an informational message with a list of available command groups is displayed.

When this variable is set to false the command groups and the commands in each group is printed instead. This variable exists for backward compatibility.

See Also

- [help](#)
- [set_app_var](#)

sh_language

Controls the tool language used by the application interactive console.

Data Types

string

Default tcl

Description

This variable controls the language used by the application interactive console. Some applications support languages in addition to Tcl as an extension language.

To determine the current value of this variable, use the *get_app_var sh_language* command.

See Also

- [get_app_var](#)
- [set_app_var](#)

sh_new_variable_message

Controls a debugging feature for tracing the creation of new variables.

Data Types

Boolean

Default application specific

Description

The `sh_new_variable_message` variable controls a debugging feature for tracing the creation of new variables. Its primary debugging purpose is to catch the misspelling of an application-owned global variable. When set to `true`, an informational message (CMD-041) is displayed when a variable is defined for the first time at the command line. When set to `false`, no message is displayed.

Note that this debugging feature is superseded by the new `set_app_var` command. This command allows setting only application-owned variables. See the *set_app_var command man page* for details.

Other variables, in combination with `sh_new_variable_message`, enable tracing of new variables in scripts and Tcl procedures.

Warning: This feature has a significant negative impact on CPU performance when used with scripts and Tcl procedures. This feature should be used only when developing scripts or in interactive use. When you turn on the feature for scripts or Tcl procedures, the application issues a message (CMD-042) to warn you about the use of this feature.

To determine the current value of this variable, use the `get_app_var sh_new_variable_message` command.

See Also

- [get_app_var](#)
- [set_app_var](#)
- [sh_new_variable_message_in_proc](#) Controls a debugging feature for tracing the creation of new variables in a Tcl procedure.
- [sh_new_variable_message_in_script](#) Controls a debugging feature for tracing the creation of new variables within a sourced script.

sh_new_variable_message_in_proc

Controls a debugging feature for tracing the creation of new variables in a Tcl procedure.

Data Types

Boolean

Default `false`

Description

The `sh_new_variable_message_in_proc` variable controls a debugging feature for tracing the creation of new variables in a Tcl procedure. Its primary debugging purpose is to catch the misspelling of an application-owned global variable.

Note that this debugging feature is superseded by the new `set_app_var` command. This command allows setting only application-owned variables. Please see the `set_app_var` command man page for details.

Note that the `sh_new_variable_message` variable must be set to `true` for this variable to have any effect. Both variables must be set to `true` for the feature to be enabled. Enabling the feature simply enables the `print_proc_new_vars` command. In order to trace the creation of variables in a procedure, this command must be inserted into the procedure, typically as the last statement. When all of these steps have been taken, an informational message (CMD-041) is generated for new variables defined within the procedure, up to the point that the `print_proc_new_vars` commands is executed.

Warning: This feature has a significant negative impact on CPU performance. This should be used only when developing scripts or in interactive use. When you turn on the feature, the application issues a message (CMD-042) to warn you about the use of this feature.

To determine the current value of this variable, use the `get_app_var sh_new_variable_message_in_proc` command.

See Also

- [get_app_var](#)
- [set_app_var](#)
- [sh_new_variable_message](#) Controls a debugging feature for tracing the creation of new variables.
- [sh_new_variable_message_in_script](#) Controls a debugging feature for tracing the creation of new variables within a sourced script.

sh_new_variable_message_in_script

Controls a debugging feature for tracing the creation of new variables within a sourced script.

Data Types

Boolean

Default false

Description

The `sh_new_variable_message_in_script` variable controls a debugging feature for tracing the creation of new variables within a sourced script. Its primary debugging purpose is to catch the misspelling of an application-owned global variable.

Note that this debugging feature is superseded by the new `set_app_var` command. This command allows setting only application-owned variables. See the `set_app_var` command man page for details.

Note that the `sh_new_variable_message` variable must be set to `true` for this variable to have any effect. Both variables must be set to `true` for the feature to be enabled. In that case, an informational message (CMD-041) is displayed when a variable is defined for the first time. When `sh_new_variable_message_in_script` is set to `false` (the default), no message is displayed at the time that the variable is created. When the `source` command completes, however, you see messages for any new variables that were created in the script. This is because the state of the variables is sampled before and after the `source` command. It is not because of inter-command sampling within the script. So, this is actually a more efficient method to see if new variables were created in the script.

For example, given the following script `a.tcl`:

```
echo "Entering script"
set a 23
echo a = $a
set b 24
echo b = $b
echo "Exiting script"
```

When `sh_new_variable_message_in_script` is `false` (the default), you see the following when you source the script:

```
prompt> source a.tcl
Entering script
a = 23
b = 24
Exiting script
Information: Defining new variable 'a'. (CMD-041)
Information: Defining new variable 'b'. (CMD-041)
prompt>
```

Alternatively, when `sh_new_variable_message_in_script` is `true`, at much greater cost, you see the following when you source the script:

```
prompt> set sh_new_variable_message_in_script true
Warning: Enabled new variable message tracing -
        Tcl scripting optimization disabled. (CMD-042)
true
prompt> source a.tcl
Entering script
Information: Defining new variable 'a'. (CMD-041)
```

```
a = 23
Information: Defining new variable 'b'. (CMD-041)
b = 24
Exiting script
prompt>
```

Warning: This feature has a significant negative impact on CPU performance. This should be used only when developing scripts or in interactive use. When you turn on the feature, the application issues a message (CMD-042) to warn you about the use of this feature.

To determine the current value of this variable, use the *get_app_var* *sh_new_variable_message_in_script* command.

See Also

- [get_app_var](#)
- [set_app_var](#)
- [sh_new_variable_message](#) Controls a debugging feature for tracing the creation of new variables.
- [sh_new_variable_message_in_proc](#) Controls a debugging feature for tracing the creation of new variables in a Tcl procedure.

sh_obsolete_is_error

Raise a Tcl error when an obsolete command is executed.

Data Types

Boolean

Default application specific

Description

When set this variable causes a Tcl error to be raised when an obsolete command is executed. Normally only a warning message is issued.

Obsolete commands have no effect.

See Also

- [get_app_var](#)
- [set_app_var](#)

sh_output_log_file

This variable is used to name the file to record console output. Set the variable to the empty string to disable output capture.

Data Types

String

Default application specific

Description

This variable can be used to save almost all console output to a log file. The log file is useful for bug reproduction and reporting.

The first time the variable is set to a valid filename, all previously logged output is written to the specified file, overwriting any previous contents of the file. All subsequent logged output is appended to the file unless the variable is later reset.

If the variable is later changed to an empty string, all logged output is disabled.

If the variable is later set to a non-empty string that is an invalid filename, the new value is ignored and the old value is restored.

If the variable is later set to a non-empty string that is a valid filename, all subsequent logged output is written to this file. Any previous contents of the file are overwritten.

The log file may not capture the banner text from the beginning of the session, and does not capture the stack trace that is printed following a fatal error. You can use the UNIX tee command or redirect the output to capture this information.

This variable is not available in de_shell mode.

See Also

- [printvar](#)

sh_product_version

Indicates the version of the application currently running.

Data Types

string

Description

This variable is set to the version of the application currently running. The variable is read only.

To determine the current value of this variable, use the `get_app_var sh_product_version` command.

See Also

- [get_app_var](#)

sh_script_stop_severity

Indicates the error message severity level that would cause a script to stop running before it completes.

Data Types

string

Default application specific

Description

When a script is run with the `source` command, there are several ways to get it to stop running before it completes. One is to use the `sh_script_stop_severity` variable. This variable can be set to *none*, *W*, or *E*.

- When set to *E*, the generation of one or more error messages by a command causes a script to stop.
- When set to *W*, the generation of one or more warning or error messages causes a script to stop.
- When set to *none*, the generation messages does not cause the script to stop.

Note that `sh_script_stop_severity` is ignored if `sh_continue_on_error` is set to *true*.

To determine the current value of this variable, use the `get_app_var sh_script_stop_severity` command.

See Also

- [get_app_var](#)
- [set_app_var](#)
- [source](#)
- [sh_continue_on_error](#) Allows processing to continue when errors occur during script execution with the `source` command.

sh_source_emits_line_numbers

Indicates the error message severity level that causes an informational message to be issued, listing the script name and line number where that message occurred.

Data Types

string

Default application specific

Description

When a script is executed with the *source* command, error and warning messages can be emitted from any command within the script. Using the *sh_source_emits_line_numbers* variable, you can help isolate where errors and warnings are occurring.

This variable can be set to *none*, *W*, or *E*.

- When set to *E*, the generation of one or more error messages by a command causes a CMD-082 informational message to be issued when the command completes, giving the name of the script and the line number of the command.
- When set to *W*, the generation of one or more warning or error messages causes a the CMD-082 message.

The setting of *sh_script_stop_severity* affects the output of the CMD-082 message. If the setting of *sh_script_stop_severity* causes a CMD-081 message, then it takes precedence over CMD-082.

To determine the current value of this variable, use the *get_app_var sh_source_emits_line_numbers* command.

See Also

- [get_app_var](#)
- [set_app_var](#)
- [source](#)
- [sh_continue_on_error](#) Allows processing to continue when errors occur during script execution with the *source* command.
- [sh_script_stop_severity](#) Indicates the error message severity level that would cause a script to stop running before it completes.
- [CMD-081](#)
- [CMD-082](#)

sh_source_logging

Indicates if individual commands from a sourced script should be logged to the command log file.

Data Types

Boolean

Default application specific

Description

When you source a script, the *source* command is echoed to the command log file. By default, each command in the script is logged to the command log file as a comment. You can disable this logging by setting *sh_source_logging* to *false*.

To determine the current value of this variable, use the *get_app_var sh_source_logging* command.

See Also

- [get_app_var](#)
- [set_app_var](#)
- [source](#)

sh_source_uses_search_path

Indicates if the *source* command uses the *search_path* variable to search for files.

Data Types

Boolean

Default application specific

Description

When this variable is set to *true* the *source* command uses the *search_path* variable to search for files. When set to *false*, the *source* command considers its file argument literally.

To determine the current value of this variable, use the *get_app_var sh_source_uses_search_path* command.

See Also

- [get_app_var](#)
- [set_app_var](#)
- [source](#)

sh_tcllib_app_dirname

Indicates the name of a directory where application-specific Tcl files are found.

Data Types

string

Description

The *sh_tcllib_app_dirname* variable is set by the application to indicate the directory where application-specific Tcl files and packages are found. This is a read-only variable.

See Also

- [get_app_var](#)

sh_user_man_path

Indicates a directory root where you can store man pages for display with the *man* command.

Data Types

list

Default empty list

Description

The *sh_user_man_path* variable is used to indicate a directory root where you can store man pages for display with the *man* command. The directory structure must start with a directory named *man*. Below *man* are directories named *cat1*, *cat2*, *cat3*, and so on. The *man* command will look in these directories for files named *file.1*, *file.2*, and *file.3*, respectively. These are pre-formatted files. It is up to you to format the files. The *man* command effectively just types the file.

These man pages could be for your Tcl procedures. The combination of defining help for your Tcl procedures with the *define_proc_attributes* command, and keeping a manual page for the same procedures allows you to fully document your application extensions.

The *man* command will look in *sh_user_man_path* after first looking in application-defined paths. The user-defined paths are consulted only if no matches are found in the application-defined paths.

To determine the current value of this variable, use the *get_app_var sh_user_man_path* command.

See Also

- [define_proc_attributes](#)
- [get_app_var](#)
- [man](#)
- [set_app_var](#)

shell.common.app_option_persistency

Specifies whether to enable persistency of user-defaults of all app-options across different sessions of the tool.

Data Types

Boolean

Default false

Description

This app option, if set to true, indicates that the option values of user-defaults of all app-options, are written out into a file in the user's home directory as a hidden file upon tool exit. This hidden file is sourced back during the start of a new session to restore the app-options values from the previous session into the new session.

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

shell.common.auto_sdp

Generates Synopsys Diagnostic Platform automatically for the commands, when set to true, for the commands mentioned with this application option.

Data Types

Boolean

Default false

Description

When the application option value is true, Synopsys Diagnostic Platform generates automatically for the commands mentioned with the `shell.common.auto_sdp_commands` application option. By default, Synopsys Diagnostic Platform generates for the `compile_fusion` command.

shell.common.auto_sdp_commands

Generates Synopsys Diagnostic Platform (sdp) automatically for the commands for the commands mentioned with this application option.

Data Types

list

Default compile_fusion

Description

This application option can be used to set the list of commands to generate Synopsys Diagnostic Platform automatically.

shell.common.auto_sdp_crte_timeperiod

Sets the Synopsys Diagnostic Platform (sdp) crte data collection time period with this value.

Data Types

int

Default 600

Description

When the application option value is set with some value, then that value is set to Synopsys Diagnostic Platform crte data collection time period.

shell.common.auto_sdp_delete

Removes the previous Synopsys Diagnostic Platform (sdp) directory, when set to true.

Data Types

Boolean

Default false

Description

When the application option value is set to true, previous Synopsys Diagnostic Platform directory is removed before creating the new Synopsys Diagnostic Platform directory.

shell.common.auto_sdp_stack_trace_frequency

Sets the Synopsys Diagnostic Platform (sdp) stack tracer frequency with this value.

Data Types

int

Default 100

Description

When the application option value is set with some value, then that value is set to Synopsys Diagnostic Platform stack tracer frequency.

shell.common.build_arch

Specifies the machine architecture used for building the application. This is read-only value.

Data Types

string

Default Not applicable

Description

This application option indicates the architecture of the machine that was used to build the application. This also usually implies that the application can be run only on a machine of this architecture.

For an executable built for linux64 architecture, this app-option will return a value of *linux64*. For an executable built for ARM64 or AArch64 architecture, this app-option will return a value of *linuxaarch64*. Value of this option is specific to this particular executable of the product. The value is read-only and always fixed for a particular executable.

To determine the current value of this application option, use the *get_app_option_value -name shell.common.build_arch* command.

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

shell.common.check_error_list

Specifies the error codes for which the *check_error* command checks.

Data Types

list

Default empty list

Description

This application option specifies the error codes for which the *check_error* command checks. The *check_error* command returns a 1 if any of the specified error codes have been generated by a previous command in the current session.

You can use this capability to stop batch jobs in which the specified error codes occur.

To determine the current value of this application option, use the *get_app_option_value -name shell.common.check_error_list* command.

See Also

- [check_error](#)

shell.common.collection_result_display_limit

Sets the maximum number of objects that can be displayed by any command that displays a collection.

Data Types

integer

Default 100

Description

This application option sets the maximum number of objects that can be displayed by any command that displays a collection. The default is 100.

When a command (for example, *add_to_collection*) is issued at the command prompt, its result is implicitly queried, as though *query_objects* had been called. You can limit the number of objects displayed by setting this application option to an appropriate integer. A value of -1 displays all objects; a value of 0 displays the collection handle id instead of the names of any objects in the collection.

To determine the current value of this application option, use the *get_app_option_value -name shell.common.collection_result_display_limit* command.

See Also

- [collections](#)
- [query_objects](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

shell.common.copyright_year

Specifies the copyright year for the application. This is read-only value.

Data Types

string

Default Not applicable

Description

This application option exposes the copyright year of the application to the interpreter. It is a read-only value.

To determine the current value of this application option, use the *get_app_option_value -name shell.common.copyright_year* command.

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

shell.common.enable_deterministic_mode

Directs the tool to run in deterministic mode.

Data Types

Boolean

Default false

Description

Fusion Compiler is expected to produce identical results across multiple runs with the same input data and the same max_cores setting. This is "flow determinism".

Some features that enable strict determinism may not yet be on by default. The shell.common.enable_deterministic_mode mega-switch application option enables all of these.

If your flow is not producing deterministic results to an expected level of TNS variation, it is recommended to enable this app option. There may be a small runtime penalty when running in deterministic mode. PPA will be neutral, but your exact flow trajectory may be different.

To determine the current value of this application option, use *get_app_option_value -name shell.common.enable_deterministic_mode*.

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

shell.common.enable_line_editing

Enables the command line editing capabilities.

Data Types

Boolean

Default true

Description

If the application option is set to true, it enables advanced UNIX-like shell capabilities.

This application option needs to be set in the .synopsys_fc.setup file to take effect.

Key Bindings The *list_key_bindings* command displays current key bindings and the edit mode. To change the edit mode, set the *shell.common.line_editing_mode* application option either in the *.synopsys_fc.setup* file or directly in the shell.

Command Completion The editor can complete commands, options, variables, and files given a unique abbreviation. You must type part of a word and press the Tab key to get the complete command, option, variable, or file. For command options, you must type '-' and press the Tab key to get the options list.

If no match is found, the terminal bell rings. If the word is already complete a space is added to the end, if it is not already there, to speed typing and provide a visual indicator of successful completion. Completed text pushes the rest of the line to the right. If there are multiple matches, all the matching commands, options, variables, or files are listed.

Completion works in following context sensitive way:

The first token of a command line : completes commands

Token that begins with "-" after a command : completes command arguments

After a ">", "|" or a "sh" command : completes file names

After a set, unset or printvar command : completes the variables

After '\$' symbol : completes the variables

After the help command : completes command

After the man command : completes commands or variables

Any token which is not the first token and does not match any of the earlier rules:
completes file names

shell.common.enable_quiet_on_filter

Controls whether commands issue warning and error messages when no objects match the criteria specified in the *-filter* option. Under the hood, it enables the *-quiet* option whenever the *-filter* option is used.

Data Types

Boolean

Default *true*

Description

When this application option is *false*, commands print warning and error messages when no objects match the criteria provided in the *-filter* option. To avoid printing these warning and error messages, set this application option to *true*.

s

For example, the `get_cells` command issues warning and error messages if the cell name provided as the pattern does not exist and the application option is set to *false*.

```
prompt> get_cells -filter {name == A}
Warning: No cell objects matched 'A' (SEL-004)
Error: Nothing matched for collection (SEL-005)
```

When the application option is set to *true*, the warning and error messages are suppressed.

```
prompt> get_cells -filter {name == A}
```

To determine the current value of this application option, use `get_app_option_value -name shell.common.enable_quiet_on_filter`.

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

shell.common.enable_summary_error

Controls whether commands print the final summary error message, such as SEL-005.

Data Types

Boolean

Default false

Description

This application option controls whether commands print a final summary error message, such as SEL-005.

By default (*false*), commands do not print a final summary message. When this application option is *true*, commands might print a final summary error message.

For example, the `get_cells` command issues a summary error message if the cell name provided as the pattern does not exist and the application option is set to *true*.

```
prompt> get_cells -filter {name == A}
Warning: No cell objects matched 'A' (SEL-004)
Error: Nothing matched for collection (SEL-005)
```

When the application option is *false*, the summary error message is suppressed.

```
prompt> get_cells -filter {name == A}
Warning: No cell objects matched 'A' (SEL-004)
```

To determine the current value of this application option, use *get_app_option_value -name shell.common.enable_summary_error*.

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

shell.common.hyper_scale

Enables hyper scale enhancements

Data Types

Boolean

Default false

Description

This application option enables all non-default hyper scaling improvements. Setting this app_option to true may improve scaling on many core systems.

To determine the current value of this application option, use *get_app_option_value -name shell.common.hyper_scale*.

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

shell.common.implicit_find_mode

Sets the search order of few predefined object types that can be used when specified *object_list* for *get_attribute* and *set_attribute* commands is string.

Data Types

string

Default default

Description

The commands *get_attribute* and *set_attribute* allow users to specify *object_list* as strings. This requires finding of objects from the specified strings implicitly. Since multiple objects of different types can have same string representation, there is a need for a search order of the objects.

This application option sets the search order of few predefined object types. This application option can have two possible values, *default* and *DC*.

The search order for application option value of *default* is:

port cell net pin bound routing_guide rp_group lib lib_cell lib_pin clock path_group design block

The search order for application option value of *DC* is:

clock port pin cell net design bound lib lib_cell lib_pin module path_group

To determine the current value of this application option, use *get_app_option_value -name shell.common.implicit_find_mode*.

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)
- [get_attribute](#)
- [set_attribute](#)

shell.common.line_editing_mode

Enables vi or emacs editing mode in the tool's shell.

Data Types

string

Default emacs

Description

This application option can be used to set the command line editor mode to either vi or emacs. Valid values are emacs or vi.

Use *list_key_bindings* command to display the current key bindings and edit mode.

This application option can be set in the either `.synopsys_icc2.setup` file or directly in the shell. The `shell.common.enable_line_editing` application option must be set to true.

shell.common.man_path

Specifies the search path for man pages for the application. This is a read-only application option.

Data Types

string

Default `<root>/doc/FC/man <root>/doc/snps_tcl/man <root>/doc/snps_gui/man`

Description

This application option exposes the list of directories to be searched for man pages. It is a read-only value.

To determine the current value of this application option, use `get_app_option_value -name shell.common.man_path`.

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

shell.common.message_style

Sets the messaging style.

Data Types

string

Default `platform`

Description

This application option sets the messaging style. Valid values for this application option are *platform* or *legacy*.

When the app_option value is *platform*, (i) The `suppress_message` command will suppress Error messages without '-force' command option. (ii) The 'occurrences' in command `get_message_info` and `print_message_info` represent the number of times the message is actually printed. (iii) Message limit in `set_message_info` is applied for every

s

source. For example, if a message limit is 2, for every source command, the message, if thrown, can be printed maximum twice.

When the `app_option` value is *legacy*, (i) The `suppress_message` command will suppress Error messages with '-force' command option. (ii) The 'occurrences' in command `get_message_info` and `print_message_info` represent sum of the number of times the message is actually printed and the number of times it was suppressed. (iii) Message limit in `set_message_info` is same across multiple source command. For example, if a message limit is 2, the message, if thrown, can be printed maximum twice, even for multiple script sourcing.

See Also

- [get_message_info](#)
- [print_message_info](#)
- [suppress_message](#)
- [set_message_info](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

shell.common.product_abbreviation

Specifies the product abbreviation for the application. This is read-only value.

Data Types

string

Default Not applicable

Description

This application option exposes the product abbreviation of the application to the interpreter. The abbreviation is a short string without spaces or punctuation that is used in the generation of the names of shared setup files. It is a read-only value.

To determine the current value of this application option, use the `get_app_option_value -name shell.common.product_abbreviation` command.

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

shell.common.product_base_build_date

The product base build date for the application. (Read-Only)

Data Types

string

Default not applicable

Description

This app option exposes the base build date of the application to the interpreter. It is a read-only value.

To determine the current value of this application option, use *get_app_option_value -name shell.common.product_base_build_date*.

See Also

- [get_app_option_value](#)
- [report_app_options](#)

shell.common.product_build_date

The product build date for the application. (Read-Only)

Data Types

string

Default not applicable

Description

This app option exposes the build date of the application to the interpreter. It is a read-only value.

To determine the current value of this application option, use *get_app_option_value -name shell.common.product_build_date*.

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

shell.common.product_name

Specifies the product name for the application. This is read-only value.

Data Types

string

Default Not applicable

Description

This application option exposes the product name of the application to the interpreter. It is a read-only value.

To determine the current value of this application option, use the *get_app_option_value -name shell.common.product_name* command.

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

shell.common.product_version

Specifies the product version for the application. This is read-only value.

Data Types

string

Default Not applicable

Description

This application option exposes the version of the application to the interpreter. It is a read-only value.

To determine the current value of this application option, use the *get_app_option_value -name shell.common.product_version* command.

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

shell.common.rename_program_name

Sets the `synopsys_program_name` for the application.

Data Types

string

Default Current `synopsys_program_name` when the product is started.

Description

This application option sets the `synopsys_program_name` for the application. Valid values for this application option are *dcr shell*, *icc2 shell*, or *fc shell*.

To determine the current value of this application option, use *get_app_option_value -name shell.common.rename_program_name*.

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

shell.common.report_default_significant_digits

Specifies the default number of significant digits used to display values in reports.

Data Types

int

Default 2

Description

This application option sets the default number of significant digits for many reports. Allowed values are 0-13; the default is 2. Most report commands have a *-significant_digits* option that overrides the value of this application option.

Not all reports respond to this application option. Check the man page of individual reports to determine whether they support this feature.

To determine the current value of this application option, use this command:

```
get_app_option_value -name  
shell.common.report_default_significant_digits
```

See Also

- [get_app_option_value](#)

shell.common.sh_language

Controls the tool language used by the application interactive console.

Data Types

string

Default tcl

Description

This app-option controls the language used by the application interactive console. Some applications support languages in addition to Tcl as an extension language.

To determine the current value of this application option, use *get_app_option_value -name shell.common.sh_language*.

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

shell.common.shell_abbreviation

Specifies the shell abbreviation name for the tool. This is a read-only application option.

Data Types

string

Default icc2 for Fusion Compiler

Description

This application option exposes the shell abbreviation of the tool to the interpreter. The shell abbreviation is used in setup files to denote the values that are specific to this particular executable of the product. It is a read-only value.

To determine the current value of this application option, use *get_app_option_value -name shell.common.shell_abbreviation*.

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

shell.common.shell_mode

Internal use only : the shell mode for the application. (Read-Only)

Data Types

string

Default not applicable

Description

This application exposes the shell mode name of the application to the interpreter. It is a read-only value, and intended for internal use.

To determine the current value of this application option, use *get_app_option_value -name shell.common.shell_mode*.

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

shell.common.shell_name

Specifies the shell name for the application. This is read-only value.

Data Types

string

Default Not applicable

Description

This application option exposes the shell name of the application to the interpreter. The shell name is used in the prompt and setup files to denote the values that are specific to this particular executable of the product. It is a read-only value.

To determine the current value of this application option, use the *get_app_option_value -name shell.common.shell_name* command.

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

shell.common.single_line_messages

When set to true, Error, Warning, and Information messages which contain newlines are written without newlines.

Data Types

Boolean

Default false

Description

To determine the current value of this application option, use *get_app_option_value -name shell.common.single_line_messages*.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [set_app_options](#)

shell.common.store_messages

Sets the recoding style of the messages.

Data Types

string

Default *first*

Description

This application option sets the recording style of the messages. Valid values for this application option are *first* or *last*.

When the app_option value is *first*, the messages printed first will be recorded till the limit reached.

When the app_option value is *last*, after the number of recorded messages reached the limit, the messages printed later will override the previously recorded message.

See Also

- [set_msg](#)
- [get_msg](#)
- [get_app_option_value](#)
- [get_app_options](#)
- [set_app_options](#)
- [report_app_options](#)

shell.common.tmp_dir_path

Sets the directory that the tool uses for temporary storage.

Data Types

string

Default */tmp* (the local /tmp partition)

Description

Specifies a directory for the tool to use as temporary storage. By default, the value is */tmp*. You can set this application option to any directory with proper read/write permissions, such as *."*, the current directory.

To determine the current value of this application option, use *get_app_option_value -name shell.common.tmp_dir_path*.

See Also

- [get_app_option_value](#)
- [set_app_options](#)

shell.common.track_subprocess_memory

When set to true, a new line will be printed in the exit summary reporting the peak memory of the session including subprocess's memory if there are any subprocess's memory to report.

Data Types

Boolean

Default *true*

Description

To determine the current value of this application option, use *get_app_option_value -name shell.common.track_subprocess_memory*.

When the application option value is true, the command *get_mem -include_subprocesses* will return the peak memory of the session including subprocess's memory. Otherwise, will continue to return the peak memory of the session without subprocess's memory.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [set_app_options](#)

shell.dc_compatibility.black_box_is_leaf_cell

Treats black box as leaf cell.

Data Types

boolean

Default *false*

Description

When this app option is true, black box cells(cells whose `is_black_box` attribute value is equal to true) and does not contain any black box cells will be treated as leaf cells and will be return by `get_flat_cells` command. And the pins on these cells will be return by `get_flat_pins` command.

When this app option is false, black box cells will be treated as leaf cells if their `is_hierarchical` attribute is false.

To determine the current value of this application option, use `get_app_option_value -name shell.dc_compatibility.black_box_is_leaf_cell`.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)
- [get_attribute](#)

shell.dc_compatibility.enable_regex_behavior

Enables DC regex behavior

Data Types

boolean

Default *false*

Description

When this app option is true, pin object regexp queries will be based on the DC regexp behavior.

When this app option is false, pin object regexp queries will be based on the ICC2 regexp behavior.

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

shell.dc_compatibility.remove_libcell_attribute

Controls the behavior of remove_attributes for library cell dont_use and dont_touch attributes.

Data Types

boolean

Default *false*

Description

When this app option is true, removing attribute dont_touch or dont_use from library cell will set these attributes to false, regardless of the attribute value originally defined in the library.

When this app option is false, removing attribute dont_touch or dont_use from library cell will restore these attributes to the value that was originally defined in the library.

To determine the current value of this application option, use *get_app_option_value -name shell.dc_compatibility.remove_libcell_attribute*.

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)
- [remove_attributes](#)
- [set_dont_touch](#)
- [set_lib_cell_purpose](#)

shell.dc_compatibility.return_tcl_errors

Determines whether or not commands will return Tcl Errors.

Data Types

boolean

Default *true*

Description

When this value is true, commands may return Tcl Errors which will halt the execution of a script. To avoid scripts from aborting prematurely, proper checking is required before passing invalid arguments to a command.

For example, `set_lib_cell_purpose` will error if it is passed a empty collection. To avoid the `foreach` loop from aborting early, we need to put in the proper checking for size of the input collection:

```
foreach cell $lib_cells_list { set lc [get_lib_cells -quiet */$cell] if {[sizeof_collection $lc] > 0}
{ set_lib_cell_purpose -include all $lc } }
```

When this value is false, Tcl Errors will be suppressed. This allows for the shorter form:

```
foreach cell $lib_cells_list { set_lib_cell_purpose -include all [get_lib_cells */$cell] }
```

The app variable `sh_continue_on_error` has a similar function, but doesn't help in the case when nested commands are within the script body of `foreach`, `if-then-else`, `while`, `type` constructs.

To determine the current value of this application option, use `get_app_option_value -name shell.dc_compatibility.return_tcl_errors`.

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)
- [sh_continue_on_error](#) Allows processing to continue when errors occur during script execution with the `source` command.

shell.synthesis.logic_level_report_group_format

Controls the format of the columns to be displayed during the Logic level path report section of the `report_logic_levels` command.

Data Types

List

Default slack ll llthreshold startpoint endpoint

Description

This application option specifies columns and their order to be displayed in path group detail section of `report_logic_levels`.

The default specification is shown in the DEFAULT section above. The headings and order of the columns displayed correspond to the keywords specified in the syntax. For example, "slack" specifies the WNS column, "ll" the LOGIC LEVELS column, and so on.

Table 1

Default Output Format				
WNS	LOGIC LEVELS	THRESHOLD	START POINT	END
POINT				
-----	-----	-----	-----	-----
-2.3006	13	5	REG_BLK/DATA_OUT_reg[0]/CK	
UPC_BLK/DATA_OUT_reg[7]/D				
-2.2972	11	5	REG_BLK/DATA_OUT_reg[0]/CK	
UPC_BLK/DATA_OUT_reg[3]/D				
-2.2899	10	5	REG_BLK/DATA_OUT_reg[0]/CK	
UPC_BLK/DATA_OUT_reg[3]/D				
-2.2891	11	5	REG_BLK/DATA_OUT_reg[0]/CK	
UPC_BLK/DATA_OUT_reg[9]/D				
-2.2879	12	5	REG_BLK/DATA_OUT_reg[0]/CK	
UPC_BLK/DATA_OUT_reg[5]/D				

There are 10 possible columns that can be displayed; only 5 are displayed in the default format. You can create a customized output format by specifying any number of the available columns.

Following are the available columns and their header values.

slack	WNS
ll_buf_inv	ALL LOGIC LEVELS
ll	LOGIC LEVELS
llthreshold	THRESHOLD
startpoint	START POINT
endpoint	END POINT
clockcycles	CLOCK CYCLES
startclk	START CLOCK
endclk	END CLOCK
infeasible	INFEASIBLE

Application options are set using the `set_app_options` command. To determine the current value of this application option, use:

```
get_app_option_value -name
    shell.synthesis.logic_level_report_group_format.
Use report_app_options to show the value of multiple application options.
```

Examples

The following example sets slack, ll, ll_buf_inv, clockcycles, startclk & endclk to be reported in path group detail section of report_logic_levels.

```
prompt> set_app_options \\  
-name shell.synthesis.logic_level_report_group_format \\  
-value "slack ll ll_buf_inv clockcycles startclk endclk"
```

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)
- [report_logic_levels](#)

shell.synthesis.logic_level_report_summary_format

Controls the format of the columns to be displayed during the Summary section of the *report_logic_levels* command.

Data Types

List

Default group period wns num_paths max_level

Description

This application option controls the format of the columns to be displayed during the *report_logic_levels* command.

The default specification is shown in the DEFAULT section above. The headings and order of the columns displayed correspond to the keywords specified in the syntax. For example, "group" specifies the GROUP column, "period" the REQUIRED PERIOD column, and so on.

Table 1

Default Output Format				
GROUP	REQUIRED PERIOD	WNS	NUM OF PATHS	MAX LEVEL
All Path Groups	n/a	n/a	2310	13
CLOCK	0.0100	-2.3289	1600	13
custom_group1	0.0100	-2.3174	166	12
custom_group2	0.0100	-2.3006	544	13

There are 8 possible columns that can be displayed; only 5 are displayed in the default format. You can create a customized output format by specifying any number of the available columns.

Following are the available columns and their header values.

wns	WNS
group	GROUP
period	REQUIRED PERIOD
std_dev	STANDARD DEVIATION
num_paths	NUM OF PATHS
max_level	MAX LEVEL
min_level	MIN LEVEL
avg_level	AVERAGE LEVEL

Application options are set using the `set_app_options` command. To determine the current value of this application option, use:

```
get_app_option_value -name  
    shell.synthesis.logic_level_report_summary_format.  
Use report_app_options to show the value of multiple application options.
```

Examples

The following example sets `group`, `num_paths`, `max_level`, `min_level` & `avg_level` to be reported in *summary* section of `report_logic_levels`.

```
prompt> set_app_options \  
    -name shell.synthesis.logic_level_report_summary_format \  
    -value "group num_paths max_level min_level avg_level"
```

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)
- [report_logic_levels](#)

shell.undo.enabled

Specifies whether undo history will be recorded for undoable commands.

Data Types

boolean

Default true

Description

This application option enables the recording of undo history for undoable commands, i.e. enables the undo feature.

When *true*, change information will be recorded up to *shell.undo.max_levels* commands and *shell.undo.max_memory* memory overhead, whichever limit is reached first. Recording of undo history will incur modest runtime overhead, proportional to the extent of changes made by the commands.

When *false*, no change information will be recorded and no commands will be undoable. Slightly better runtime performance may be observed.

Undoability applies only to undoable commands, which you can determine using the *get_undo_info -command <command_name>* command. When an explicitly non-undoable command is invoked, the undo history is cleared, and the system state cannot be undone to previous points.

Note that changing the value of this application option to *false* clears all existing undo history.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [get_undo_info](#)

shell.undo.max_levels

Specifies the maximum number of commands to record undo change history for.

Data Types

integer

Default 100

Description

This application option specifies the maximum number of commands in the undo history to maintain. A larger value enables a longer undo history, at the expense of greater memory overhead.

The actual number of commands in the undo history may be lower, depending on when the undo history was last cleared and whether the history was truncated due to the *shell.undo.max_memory* limit being reached.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [get_undo_info](#)

shell.undo.max_memory

Specifies the maximum memory overhead, in bytes, to allow for the storage of change information for undoability of commands.

Data Types

integer

Default 1000000000

Description

This application option specifies the maximum memory, in bytes, to allow for the overhead of undo history storage. A higher value enables a longer undo history, up to but not exceeding the number of commands specified by *shell.undo.max_levels*.

Keeping an undo history of at least one command takes precedence, so this memory limit may be exceeded if the last executed undoable command requires a large amount of memory to enable its undoability.

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [get_undo_info](#)

signoff.antenna.contact_layer

Option to specify contact layer for antenna property extraction flow.

Data Types

string

Default Empty.

s

Description

If you do not specify this option, the tool tries to get the contact layer from the technology file. If the technology file does not have the mask name set for contact layer, the user needs to pass the contact layer from command line to successfully run the flow.

This option also supports the boolean NOT operation. If it is required to derive contact layer from more than one layers using OR/NOT operation, then it could be passed using logical unary NOT operator.

Examples

The following example sets the contact layer as layer number 30 in the design in antenna property extraction during library creation:

```
prompt> set_app_options -name signoff.antenna.contact_layer -value 30
```

The following example sets the contact layer as NOT of layer number 30 and layer number 30:1 in the design in antenna property extraction during library creation:

```
prompt> set_app_options -name signoff.antenna.contact_layer -value
{ {30} !{30:1} }
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)

signoff.antenna.contact_layers_between_m0_diffusion

Specifies the contact layers (contact layers are the layers that connect metal0 to diffusion). You can specify the layers by using the layer names from the technology file.

Data Types

string

Default Empty.

Description

Specifies the contact layers (contact layers are the layers that connect metal0 to diffusion). You can specify the layers by using the layer names from the technology file.

The *contact_layers_between_m0_diffusion* and *contact_layer* options are mutually exclusive; you can specify only one.

Examples

The following example sets the `contact_layers_between_m0_diffusion` as layer number 28 in antenna property extraction during library creation:

```
prompt> set_app_options -name  
        signoff.antenna.contact_layers_between_m0_diffusion\  
        -value 28
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)

signoff.antenna.custom_runset_file

Option to specify custom runset file in case of advanced gate layers.

Data Types

string

Default Empty.

Description

The custom runset file is passed when the metal layers are derived from boolean operations of more than layers. For instance, at advanced nodes, METAL1 layers can be derived from CUT layers. In those cases, this option could be used to pass the custom runset file that would be included as a part of main runset in antenna flow.

The custom runset follows syntax of standard ICV runset and would contain first assign section and then boolean operation in a sequential order.

Examples

The following example pass the custom runset include file in antenna property extraction during library creation:

```
prompt> set_app_options -name signoff.antenna.custom_runset_file -value  
        custom.rs
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)

signoff.antenna.diffusion_layer

Option to specify diffusion layer for antenna property extraction flow.

Data Types

string

Default Empty.

Description

If you do not specify this option, the tool tries to get the diffusion layer from the technology file. If the technology file does not have the mask name set for diffusion layer, the user needs to pass the diffusion layer from command line to successfully run the flow.

This option also supports the boolean NOT operation. If it is required to derive diffusion layer from more than one layers using OR/NOT operation, then it could be passed using logical unary NOT operator.

Examples

The following example sets the diffusion layer as layer number 6 in the design in antenna property extraction during library creation:

```
prompt> set_app_options -name signoff.antenna.diffusion_layer -value 6
```

The following example sets the diffusion layer as NOT of layer number 6 and layer number 2 in the design in antenna property extraction during library creation:

```
prompt> set_app_options -name signoff.antenna.diffusion_layer -value  
{ {6} !{2} }
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)

- [report_app_options](#)
- [set_app_options](#)

signoff.antenna.enabled

Option to run antenna property extraction flow during *check_workspace* command. Enable the antenna property extraction flow.

Data Types

boolean

Default false

Description

If you do not specify this option, the tool does not extract the antenna properties during library creation.

Examples

The following example runs the antenna property extraction flow during *check_workspace*.

```
prompt> set_app_options -name signoff.antenna.enabled -value true
prompt> check_workspace
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)

signoff.antenna.exclude_non_mask_layers

Option to specify to exclude non mask layers in antenna extraction flow.

Data Types

bool

Default false.

s

Description

If you do not specify this option, the tool includes the non mask layers as input layers. Whether the layer is non mask or not is decided based on the field "nonMask" in the layer/ datatype section in the techfile.

If the app option is set to true, the input layer excludes the layer and layer purpose that have nonMask field set to 1.

Examples

The following example excludes the non Mask layer from the input layer in antenna property extraction during library creation:

```
prompt> set_app_options -name signoff.antenna.exclude_non_mask_layers  
-value true
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)

signoff.antenna.extract_via_antenna_property

Specifies whether to extract antenna properties for via layers. This option is used during antenna extraction.

Data Types

bool

Default false

Description

This application option specifies whether to extract antenna properties for via layers. By default, this option is false and the flow extracts antenna properties only for metal layers.

Examples

The following example sets the extract via antenna prop to true.

```
prompt> set_app_options -list  
{signoff.antenna.extract_via_antenna_property {true}}
```


See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)

signoff.antenna.gate_class1_layers

Specifies the marking layers for gate thickness class 1. You can specify the layers by using the layer names from the technology file.

Data Types

string

Default Empty.

Description

Specifies the marking layers for gate thickness class 1. You can specify the layers by using the layer names from the technology file.

Examples

The following example sets the gate 1 layer as layer number 3 in antenna property extraction during library creation:

```
prompt> set_app_options -name signoff.antenna.gate_class1_layers -value 3
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)

signoff.antenna.gate_class2_layers

Specifies the marking layers for gate thickness class 2. You can specify the layers by using the layer names from the technology file.

Data Types

string

Default Empty.

Description

Specifies the marking layers for gate thickness class 2. You can specify the layers by using the layer names from the technology file.

Examples

The following example sets the gate 2 layer as layer number 4 in antenna property extraction during library creation:

```
prompt> set_app_options -name signoff.antenna.gate_class2_layers -value 4
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)

signoff.antenna.gate_class3_layers

Specifies the marking layers for gate thickness class 3. You can specify the layers by using the layer names from the technology file.

Data Types

string

Default Empty.

Description

Specifies the marking layers for gate thickness class 3. You can specify the layers by using the layer names from the technology file.

Examples

The following example sets the gate 3 layer as layer number 5 in antenna property extraction during library creation:

```
prompt> set_app_options -name signoff.antenna.gate_class3_layers -value 5
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)

signoff.antenna.hierarchical_gate_class_include_file

Specifies the location of the gate class include file for advanced gate class marker layers.

Data Types

string

Default null

Description

The *signoff.antenna.hierarchical_gate_class_include_file* application option specifies the file name of a valid, partial IC Validator runset to use for identifying the marking layers.

The runset should have the following syntax:

```
OD1 = assign ({13,0});
OD2 = assign ({113,0},{83,0});
OD3 = assign ({84,0},{26,0});
OD4 = assign ({109,0});

GATES1 = GATES and OD1;
GATES2 = GATES and OD2;
GATES3 = GATES and OD3;
```

This example shows a complex relationship between four different marking layers, OD1, OD2, OD3, and OD4. The intermediate layer names OD1 through OD4 can be named differently.

If you do not need all three marking layers, you can define only the GATES1 or GATES2 layers, and set the *signoff.antenna.max_gate_class_name* application option accordingly. This application option specifies the maximum gate class name when the gate class layers are passed through a custom runset. This option must be specified when the *signoff.antenna.hierarchical_gate_class_include_file* is used.

Examples

The following example sets the gate class include file as `complex_gates.rs`.

```
prompt> set_app_options \\
        -name signoff.antenna.hierarchical_gate_class_include_file \\
        -value complex_gates.rs
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)

signoff.antenna.m0_layers_for_diffusion_connection

Specifies the m0 layers for diffusion connection. You can specify the layers by using the layer names from the technology file.

Data Types

string

Default Empty.

Description

Specifies the m0 layers for diffusion connection. You can specify the layers by using the layer names from the technology file.

The *m0_layers_for_diffusion_connection* and *contact_layer* options are mutually exclusive; you can specify only one.

Examples

The following example sets the poly layer as layer name M0_OD in antenna property extraction during library creation:

```
prompt> set_app_options -name
        signoff.antenna.m0_layers_for_diffusion_connection \
        -value M0_OD
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)

signoff.antenna.m0_layers_for_poly_connection

Specifies the m0 layers for poly connection. You can specify the layers by using the layer names from the technology file.

Data Types

string

Default Empty.

Description

Specifies the m0 layers for poly connection. You can specify the layers by using the layer names from the technology file.

The *m0_layers_for_poly_connection* and *contact_layers* options are mutually exclusive; you can specify only one.

Examples

The following example sets the *m0_layers_for_poly_connection* layer as layer number 19 in antenna property extraction during library creation:

```
prompt> set_app_options -name  
signoff.antenna.m0_layers_for_poly_connection -value 19
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)

signoff.antenna.max_gate_class_name

Specifies the maximum gate class name when the gate class layers are passed through custom runset.

Data Types

string

Default null

Description

Specifies the maximum gate class name when the gate class layers are passed through custom runset. This option is necessary to specify when `signoff.antenna.hier_gate_class_include_file` is passed.

If the marking layers have a more complex structure than a simple OR relationship, you can provide a partial ICV runset. To do so, you need to define the

The many gate class definitions to expect and must have one of the following values: GATES1, GATES2, or GATES3.

Examples

The following example sets the max gate class name as GATES1.

```
prompt> set_app_options -name signoff.antenna.max_gates_class_name -value  
GATES1
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)

signoff.antenna.poly_layer

Option to specify poly layer for antenna property extraction flow.

Data Types

string

Default Empty.

Description

If you do not specify this option, the tool tries to get the poly layer from the technology file. If the technology file does not have the mask name set for poly layer, the user needs to pass the poly layer from command line to successfully run the flow.

This option also supports the boolean NOT operation. If it is required to derive poly layer from more than one layers using OR/NOT operation, then it could be passed using logical unary NOT operator.

Examples

The following example sets the poly layer as layer number 17 in the design in antenna property extraction during library creation:

```
prompt> set_app_options -name signoff.antenna.poly_layer -value 17
```

The following example sets the poly layer as NOT of layer number 17 and layer number 17:1 in the design in antenna property extraction during library creation:

```
prompt> set_app_options -name signoff.antenna.poly_layer -value  
{ {17} !{17:1} }
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)

signoff.antenna.report_diodes

Controls whether the tool reports the location of generated protection diodes.

Data Types

bool

Default true

Description

Controls whether the tool reports the location of generated protection diodes.

The default is *true*.

Examples

The following example sets the report_diodes option to false in antenna property extraction during library creation:

```
prompt> set_app_options -name signoff.antenna.report_diodes -value false  
prompt> check_workspace
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)

signoff.antenna.top_cell_pin_only

Controls whether the tool extracts hierarchical antenna properties only for nets that connect to pins in the top-level of the hard macro.

Data Types

bool

Default `true`

Description

Controls whether the tool extracts hierarchical antenna properties only for nets that connect to pins in the top-level of the hard macro.

The default is *true*.

If you set this option to *false*, the command extracts hierarchical antenna properties for nets that connect to pins in the top-level of the hard macro, as well as to pins in the child cells contained within the hard macro.

Examples

The following example sets the `top_cell_pin_only` option to `true` in antenna property extraction during library creation:

```
prompt> set_app_options -name signoff.antenna.top_cell_pin_only -value  
false  
prompt> check_workspace
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)

signoff.antenna.treat_source_drain_as_diodes

Controls whether the tool treats the MOS source and drain as protection diodes and considers any MOS source and drain regions that are connected to a net to be a part of the diode protection area available to that net.

Data Types

bool

Default `true`

Description

Controls whether the tool treats the MOS source and drain as protection diodes and considers any MOS source and drain regions that are connected to a net to be a part of the diode protection area available to that net.

The default is *true*.

Examples

The following example sets the `treat_source_drain_as_diodes` option to false in antenna property extraction during library creation:

```
prompt> set_app_options -name  
        signoff.antenna.treat_source_drain_as_diodes -value false  
prompt> check_workspace
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)

signoff.antenna.user_defined_options

Option for the antenna flow. Specifies strings as part of IC Validator command line options.

Data Types

string

Default `null`

Description

This application option specifies additional options for the IC Validator command line. The string that you specify is added to the command line used to invoke design rule checking in IC Validator. IC Compiler II does not perform any checking on the specified string.

By default, no user-defined options are included.

Examples

The following example specifies the user-defined options named `-turbo`.

```
prompt> set_app_options \\  
-name signoff.calculate_hier_antenna_property.user_defined_options \\  
-value "-turbo"
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)

signoff.antenna.v0_layers_between_m1_m0

Specifies the v0 layers (v0 layers are the layers that connect metal1 to metal0). You can specify the layers by using the layer names from the technology file.

Data Types

string

Default Empty.

Description

Specifies the v0 layers (v0 layers are the layers that connect metal1 to metal0). You can specify the layers by using the layer names from the technology file.

The options `v0_layers_between_m1_m0` and `contact_layers` options are mutually exclusive; you can specify only one.

Examples

The following example sets the `v0_layers_between_m1_m0` as layer number 50 in antenna property extraction during library creation:

```
prompt> set_app_options -name signoff.antenna.v0_layers_between_m1_m0  
-value 50
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)

signoff.calculate_hier_antenna_property.extract_via_antenna_prop

Option for the `signoff_calculate_hier_antenna_property` command. Specifies to extract the antenna properties for via layers.

Data Types

bool

Default false

Description

This application option specifies to extract the antenna properties for via layers as well. By default (false), the flow only extracts properties for metal layers. If this option is set to true, antenna properties for via layers will also be extracted.

Examples

The following example sets the extract via antenna property to true.

```
prompt> set_app_options -global \  
-name signoff.calculate_hier_antenna_property.extract_via_antenna_prop  
\  
-value true  
prompt> signoff_calculate_hier_antenna_property
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)
- [signoff_calculate_hier_antenna_property](#)

signoff.calculate_hier_antenna_property.user_defined_options

Option for the *signoff_calculate_hier_antenna_property* command. Specifies strings as part of IC Validator command line options.

Data Types

string

Default null

Description

This application option specifies additional options for the IC Validator command line. The string that you specify is added to the command line used to invoke design rule checking in IC Validator. IC Compiler II does not perform any checking on the specified string.

By default, no user-defined options are included.

Examples

The following example specifies the user-defined options named -turbo.

```
prompt> set_app_options -global \\  
        -name signoff.calculate_hier_antenna_property.user_defined_options \\  
        "-turbo"  
prompt> signoff_calculate_hier_antenna_property
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)
- [signoff_calculate_hier_antenna_property](#)

signoff.check_design.generate_runset_only

Option for the *signoff_check_design* command. Specifies whether to generate only runset or generate runset and run IC Validator with generated runset. By default, it generates the runset and runs IC Validator with the runset.

Data Types

bool

Default false

Description

When it is set as true, only the runset is generated. When it is set as false, runset is generated and IC Validator is run with the runset.

Examples

The following example generates only the runset for the *signoff_check_design* command.

```
prompt> set_app_options -global  
          {signoff.check_design.generate_runset_only true}  
prompt> signoff_check_design
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)
- [signoff_check_design](#)

signoff.check_design.max_errors_per_rule

Option for the *signoff_check_design* command. Specifies the maximum number of errors per rule for the IC Validator run.

Data Types

integer

Default Max Integer limit

Description

This application option specifies the maximum number of errors per rule for the IC Validator run. The value must be greater than or equal to 1. The default value is max integer limit.

Examples

The following example runs IC Validator for design rule checking and sets the maximum number of errors per rule to 8000.

```
prompt> set_app_options -global {signoff.check_design.max_errors_per_rule  
      8000}  
prompt> signoff_check_design
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)
- [signoff_check_design](#)

signoff.check_design.run_dir

Option for the *signoff_check_design* command. Specifies the path of working directory.

Data Types

string

Default signoff_check_design_run

Description

If you do not specify this option, the tool uses a directory called *signoff_check_design_run* under the current working directory. The path can be either relative or absolute.

Examples

The following example runs IC Validator under the working directory named *my_signoff_run*.

```
prompt> set_app_options -global {signoff.check_design.run_dir  
      "my_signoff_run"}  
prompt> signoff_check_design
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)
- [signoff_check_design](#)

signoff.check_design.runset

Specifies the runset to use for `signoff_check_design` utilities such as to check the shorts between signal and metal fill shapes. By default, the tool generates the runset on the fly. It is an option for the `signoff_check_design` command.

Data Types

string

Default null

Description

The runset is used while the tool invokes IC Validator for checking violations related to shorts and other specific checks.

Examples

The following example specifies the runset `drc.rs` for the `signoff_check_design` command.

```
prompt> set_app_options -global {signoff.check_design.runset "drc.rs"}
prompt> signoff_check_design
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)
- [signoff_check_design](#)

signoff.check_design.user_defined_options

Option for the *signoff_check_design* command. Specifies strings as part of IC Validator command line options.

Data Types

string

Default null

Description

This application option specifies additional options for the IC Validator command line. The string that you specify is added to the command line used to invoke design rule checking in IC Validator. IC Compiler II does not perform any checking on the specified string.

By default, no user-defined options are included.

Examples

The following example specifies the user-defined options named -turbo.

```
prompt> set_app_options -global  
      {signoff.check_design.user_defined_options "-turbo"}  
prompt> signoff_check_design
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)
- [signoff_check_design](#)

signoff.check_drc.auto_eco_threshold_value

Option for the *signoff_check_drc* command. Specifies the threshold value for the allowed amount of change in percent of the design after an ECO has been performed so that incremental drc checking will be performed. Default: 20%.

Data Types

integer

Default 20

Description

This application option specifies the threshold value for the allowed amount of change in percent of the design so that incremental drc checking will be performed. The default value is 20.

Examples

The following example runs incremental drc by setting a max allowed percent of design change to 30.

```
prompt> set_app_options -name signoff.check_drc.auto_eco_threshold_value  
-value 30  
prompt> signoff_check_drc -auto_eco true
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)
- [signoff_check_drc](#)

signoff.check_drc.enable_icv_explorer_mode

Option for the *signoff_check_drc* command. Enables the icv explorer mode while invoking ICV through *signoff_check_drc* command/

Data Types

string

Default true

Description

Enables the icv explorer mode. This enables the generation of heat map data for DRC errors. The heat map can be seen in ICC2 gui.

Examples

The following example enables the generation of heat map data for the *signoff_check_drc* command.

```
prompt> set_app_options -name signoff.check_drc.enable_icv_explorer_mode  
-value "true"  
prompt> signoff_check_drc
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)
- [signoff_check_drc](#)

signoff.check_drc.exclude_cells

Option for the *signoff_check_drc* command. Specifies a list of explicit IP cells to exclude from design rule checking.

Data Types

list_of_cells

Default null

Description

This application option specifies the explicit IP cells to exclude from design rule checking. You can specify a list of one or more cells by their names.

Examples

The following example excludes specific IP cells from design rule checking.

```
prompt> set_app_options -name signoff.check_drc.exclude_cells -value  
      {SEDFPQLX0P5MV0SM30P NOR2X1MV0SM30P XOR2X1MV0SM30P}  
prompt> signoff_check_drc
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)
- [signoff_check_drc](#)

signoff.check_drc.excluded_cell_types

Option for the *signoff_check_drc* command. Specifies a list of cell types to exclude from design rule checking. Cell types include {lib_cell, macro, pad, filler}.

Data Types

list_of_cell_types

Default null

Description

This application option specifies the cell types to exclude from design rule checking. You can specify a list of one or more cell types. The valid values for cell type are *lib_cell*, *macro*, *pad*, and *filler*. These cell types are defined as follows:

- *lib_cell* is for standard cells
- *macro* is for macro cells
- *pad* is for I/O pad cells
- *filler* is for filler cells

By default, all cell types are checked (no cell types are excluded).

Examples

The following example excludes standard cells and macro cells from design rule checking.

```
prompt> set_app_options -name signoff.check_drc.excluded_cell_types  
          -value {lib_cell macro}  
prompt> signoff_check_drc
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)
- [signoff_check_drc](#)

signoff.check_drc.fill_view_data

Option for the *signoff_check_drc* command. Defines the behavior for reading of FILL data during *signoff_check_drc* command.

Data Types

string

Default `read_if_uptodate`

Description

There are three possible modes for reading of fill data.

"`read_if_uptodate`" The fill view data is read only when its timestamp is newer than the timestamp of the design view.

"`read`" The FILL view data is read even though its timestamp is older than the timestamp of the design view.

"`discard`" Never read fill view data.

Examples

The following example reads the FILL data even though its timestamp is older than the timestamp of the design view.

```
prompt> set_app_options -name signoff.check_drc.fill_view_data -value  
      read  
prompt> signoff_check_drc
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)
- [signoff_check_drc](#)

signoff.check_drc.ignore_blockages_in_cells

Option for the *signoff_check_drc* command. Controls whether blockages in macros are read during the *signoff_check_drc* command.

Data Types

bool

Default `true`

Description

If set to true (default), the blockages from the FRAME view of macros will not be read. Only the pins will be read. The DESIGN views of standard library cells will be read.

If set to false, the blockages from the FRAME view of macros and standard library cells will be read along with the pins.

Examples

The following example runs design rule checking while attempting to read the cell instance data for all library cells from the DESIGN view and only the pins from the FRAME view of the macros:

```
prompt> set_app_options -name signoff.check_drc.ignore_blockages_in_cells  
-value true  
prompt> signoff_check_drc
```

The following example runs design rule checking while attempting to read the cell instance data for the M1 macro from the DESIGN view. If the DESIGN view for this macro cannot be found the FRAME view is used. The cell instance data for all remaining macros and standard library cells will be read from their FRAME views and include blockages and pins. All remaining library cell instance data will be read from their DESIGN views.

```
prompt> set_app_options -name signoff.check_drc.read_design_views -value  
{M1}  
prompt> set_app_options -name signoff.check_drc.ignore_blockages_in_cells  
-value false  
prompt> signoff_check_drc
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)
- [signoff_check_drc](#)

signoff.check_drc.ignore_child_cell_errors

Option for the *signoff_check_drc* command. Skips the reporting of DRC violations discovered for child cells.

Data Types

bool

Default false

Description

If true, the command skips the reporting of DRC violations discovered inside all cell instances (reference libraries).

By default (false), DRC violations inside cell instances are reported. Note that by default, when this option is unset, and child cell has DRC violations to begin with, violations will be reported at top-block level for only one child instance, rather than on all instances, to avoid redundant reporting.

Examples

The following example skips the reporting of DRC violations discovered inside all cell instances (reference libraries):

```
prompt> set_app_options -name signoff.check_drc.ignore_child_cell_errors
        -value true
prompt> signoff_check_drc
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)
- [signoff_check_drc](#)

signoff.check_drc.max_errors_per_rule

Option for the *signoff_check_drc* command. Specifies the maximum number of errors per rule reported for the IC Validator run.

Data Types

integer

Default 1000

Description

This application option specifies the maximum number of errors per rule to store in the error database that will be available to the Error Browser for viewing and reported in the IC Validator LAYOUT_ERRORS file. The value must be greater than or equal to 1. The default value is 1000.

Examples

The following example runs IC Validator for design rule checking and sets the maximum number of errors per rule to 8000.

```
prompt> set_app_options -name signoff.check_drc.max_errors_per_rule  
-value 8000  
prompt> signoff_check_drc
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)
- [signoff_check_drc](#)

signoff.check_drc.merge_base_error_name

Option for the *signoff_check_drc* command. Specifies the name of the error data to merge when *signoff_check_drc -auto_eco* is used and *signoff.check_drc.merge_incremental_error_data* is set to true. Default: *signoff_check_drc.err*.

Data Types

string

Default *signoff_check_drc.err*

Description

This application option specifies the name of the error data used as a base to merge when *signoff_check_drc -auto_eco* is used to specify an incremental DRC run and the *signoff.check_drc.merge_incremental_error_data* is set to true. Normally there is no need to set this application option but it is required when the previous full DRC run was run with the *signoff_check_drc -error_data* option set. The default value is *signoff_check_drc.err*.

Examples

The following example runs incremental drc and merges the result with a previous error data called "pre_eco_drc.err".

```
prompt> set_app_options -name  
signoff.check_drc.merge_incremental_error_data -value true
```

s

```
prompt> set_app_options -name signoff.check_drc.merge_base_error_name  
-value "pre_eco_drc.err"  
prompt> signoff_check_drc -auto_eco true
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)
- [signoff_check_drc](#)

signoff.check_drc.merge_incremental_error_data

Option for the *signoff_check_drc* command. Specifies the whether the *signoff_check_drc -auto_eco* command should merge incremental error data with the result of a previous *signoff_check_drc* result. Default: false.

Data Types

boolean

Default false

Description

This application option specifies whether or not to merge incremental error data from a *signoff_check_drc* run with a previous result when *signoff_check_drc -auto_eco* is set to true. The resulting name of the merged error data is *signoff_check_drc_incremental_merged.err*. The default value is false.

Examples

The following example runs incremental drc and merges the result with a previous *signoff_check_drc* run.

```
prompt> set_app_options -name  
signoff.check_drc.merge_incremental_error_data -value true  
prompt> signoff_check_drc -auto_eco true
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)

- [report_app_options](#)
- [set_app_options](#)
- [signoff_check_drc](#)

signoff.check_drc.read_design_views

Option for the *signoff_check_drc* command. Reads the data of cell instances from the given list of design views.

Data Types

list_of_macro_master_names

Default none

Description

If set, the cell instance data of each listed cell master is read from the design view. If the design view of a cell instance cannot be found, a warning message is printed and the command reads cell instance data from the FRAME view instead. For convenience, all cells can be specified using the `{*}` wildcard. Macro instances that are not selected by this option will have their instance data read from its FRAME view.

By default (none), the macro instance data is read from the FRAME view. If the FRAME view of a macro instance cannot be found, the command ignores this instance for design rule checking and continues to check the remaining cell instances. The command issues a warning message for the ignored cell instances in the error file.

Examples

The following example runs design rule checking, which attempts to read the cell instance data for all cells from the design view. If the design view cannot be found, the command reads the FRAME view instead.

```
prompt> set_app_options -name signoff.check_drc.read_design_views -value  
{*}  
prompt> signoff_check_drc
```

The following example runs design rule checking, which attempts to read the cell instance data for the M1 macro from the design view. If the design view for this macro cannot be found the command reads the FRAME view instead. The cell instance data for all other cells is read from the FRAME view. If the FRAME view cannot be found for any of those cells, the cell is ignored.

```
prompt> set_app_options -name signoff.check_drc.read_design_views -value  
{M1}  
prompt> signoff_check_drc
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)
- [signoff_check_drc](#)

signoff.check_drc.read_layout_views

Option for the *signoff_check_drc* command. Reads the data of cell instances from the given list of layout views.

Data Types

list_of_macro_master_names

Default none

Description

If set, the cell instance data of each listed cell master is read from the layout view. If the layout view of a cell instance cannot be found, application will read design view, if it exists, otherwise it will read the frame view. For convenience, all cells can be specified by the `{*}` wildcard. By default (none), the macro instance data is read from the FRAME view. If the FRAME view of a macro instance cannot be found, the command ignores this instance for design rule checking and continues to check the remaining cell instances. The command issues a warning message for the ignored cell instances in the error file.

Examples

The following example runs design rule checking, which attempts to read the cell instance data for all cells from the layout view.

```
prompt> set_app_options -name signoff.check_drc.read_layout_views -value  
{*}  
prompt> signoff_check_drc
```

The following example runs design rule checking, which attempts to read the cell instance data for the M1 macro from the layout view.

```
prompt> set_app_options -name signoff.check_drc.read_layout_views -value  
{M1}  
prompt> signoff_check_drc
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)
- [signoff_check_drc](#)

signoff.check_drc.run_dir

Option for the *signoff_check_drc* command. Specifies the path of the working directory.

Data Types

string

Default `signoff_check_drc_run`

Description

Specifies the path of the working directory. If you do not specify this option, the tool creates a directory called *signoff_check_drc_run* under the current working directory. The path can be either relative or absolute.

Examples

The following example runs IC Validator under the working directory named *my_signoff_run*.

```
prompt> set_app_options -name signoff.check_drc.run_dir -value  
"my_signoff_run"  
prompt> signoff_check_drc
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)
- [signoff_check_drc](#)

signoff.check_drc.runset

Option for the *signoff_check_drc* command. Specifies the foundry runset to use for design rule checking (DRC).

Data Types

string

Default null

Description

Specifies the foundry runset that will be used while the tool invokes IC Validator to perform design rule checking on the current design.

Examples

The following example specifies the DRC runset *drc.rs* for the *signoff_check_drc* command.

```
prompt> set_app_options -name signoff.check_drc.runset -value "drc.rs"  
prompt> signoff_check_drc
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)
- [signoff_check_drc](#)

signoff.check_drc.user_defined_options

Option for the *signoff_check_drc* command. Specifies strings as part of IC Validator command line options.

Data Types

string

Default null

Description

This application option specifies additional options for the IC Validator command line. The string that you specify is added to the command line used to invoke design rule checking in IC Validator. IC Compiler II does not perform any checking on the specified string.

By default, no user-defined options are included.

Examples

The following example specifies the user-defined options named -turbo.

```
prompt> set_app_options -name signoff.check_drc.user_defined_options  
-value "-turbo"  
prompt> signoff_check_drc
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)
- [signoff_check_drc](#)

signoff.check_drc_live.check_only_visible_layers

Controls whether Live DRC includes rules defined on non-displayed layers.

Data Types

enumerated

Default all_layers

Description

This application option specifies how Live DRC handles rules defined on non-displayed layers. Valid values are

- *all_layers* (the default)
Live DRC includes the rules for all layers, whether displayed or not.
- *visible_involved*
Live DRC includes the rules that involve the visible metal and via layers.

- *visible_only*

Live DRC includes the rules that use only the visible metal and via layers.

For example, if the only the M1 layer is visible,

- A rule that checks M1 versus M1 spacing would be included in all three modes
- A rule that checks the M1 enclosure of V1 would be included only in *all_layers* and *visible_involved* modes. The rule is excluded in *visible_only* mode because V1 is not visible.

Examples

The following example enables the check-only-visible-layers option of the *ignoff_check_live_drc* command.

```
prompt> set_app_options \\  
-name signoff.check_drc_live.check_only_visible_layers \\  
-value visible_only
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)
- [signoff_check_live_drc](#)

signoff.check_drc_live.default_layers

Specifies the layers with color masks to write into the default layers for Live DRC.

Data Types

string

Default {}

Description

This application option specifies the layers with color masks to write into the default layers for Live DRC.

Examples

The following example runs Live DRC using VIA1 and VIA2 as the default layers.

```
prompt> set_app_options \\  
-name signoff.check_drc_live.default_layers \\  
-value "VIA1 VIA2"  
prompt> signoff_check_live_drc \\  
-rect {{55.633 72.456} {62.766 67.848}}
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)
- [signoff_check_live_drc](#)
- [write_oasis](#)

signoff.check_drc_live.exclude_command_class

Controls whether Live DRC excludes rules related to specific classes.

Data Types

list

Default {{density true} {connectivity true}}

Description

This application option controls whether Live DRC excludes rules related to specific classes.

The syntax for specifying this application option is a list of class-Boolean pairs. Supported classes are *density* and *connectivity*.

By default, all rules related to density and connectivity are excluded.

Examples

The following example includes the density and connectivity rules for the *signoff_check_live_drc* command.

```
prompt> set_app_options \\  
-name signoff.check_drc_live.exclude_command_class \\  
-value {{ density false } { connectivity false }}
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)
- [signoff_check_live_drc](#)

signoff.check_drc_live.halo

Specifies the halo around the check area used by Live DRC.

Data Types

double

Default 1

Description

This application option specifies the distance in microns by which Live DRC expands the user-specified check area when determining which DRC violations to keep. Live DRC uses the *signoff.check_drc_live.keep_enclosing_results* application option with this option to determine which violations to keep.

The default is 1 micron.

Examples

The following example specifies a halo of 2 microns.

```
prompt> set_app_options -name signoff.check_drc_live.halo -value 2.0
prompt> signoff_check_live_drc -rect {{55.633 72.456} {62.766 67.84}}
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)
- [signoff_check_live_drc](#)

signoff.check_drc_live.hierarchy_level

Specifies the maximum hierarchical depth processed by Live DRC.

Data Types

enumerated

Default visible

Description

This application option specifies the maximum hierarchical depth processed by Live DRC. This application option determines how Live DRC sets the *-child_depth* option of the generated *signoff_check_live_drc* command.

By default (*visible*), Live DRC uses the current view level as the maximum hierarchical depth and sets the *-child_depth* option to this value.

To process all levels of hierarchy, set this application option to *all*, which sets the *-child_depth* option of the *signoff_check_live_drc* command to -1.

Examples

The following example checks for DRC violations through the entire hierarchy.

```
prompt> set_app_options \\  
          -name signoff.check_drc_live.hierarchy_level -value all  
prompt> signoff_check_live_drc -rect {{55.633 72.456} {62.766 67.84}}
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)
- [signoff_check_live_drc](#)

signoff.check_drc_live.igraph

Controls whether Live DRC uses igraph mode or legacy mode.

Data Types

Boolean

Default true

Description

This application option controls whether Live DRC uses igrph mode or legacy mode.

By default (*true*), Live DRC uses igrph mode, which does not have runset compile overhead and is faster than legacy mode.

If set to *false*, Live DRC uses legacy mode.

Examples

The following example enables legacy mode for Live DRC:

```
prompt> set_app_options -name signoff.check_drc_live.igrph -value true
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)
- [signoff_check_live_drc](#)

signoff.check_drc_live.keep_enclosing_results

Controls whether Live DRC keeps DRC violations that are outside of the checking region.

Data Types

Boolean

Default false

Description

This application option controls whether Live DRC keeps DRC violations that are outside of the checking region.

By default (*false*), Live DRC keeps only those violations that are fully within the checking region.

If you set this application option to *true*, Live DRC keeps the DRC violations that are fully inside the trim region. The trim region is the checking region plus half of the distance specified by the *signoff.check_drc_live.halo* application option. Violations that are fully outside the trim region are always discarded.

Examples

The following example keeps DRC violations that are within the trim region.

```
prompt> set_app_options \\  
        -name signoff.check_drc_live.keep_enclosing_results -value true  
prompt> signoff_check_live_drc -rect {{55.633 72.456} {62.766 67.84}}
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)
- [signoff_check_live_drc](#)

signoff.check_drc_live.run_dir

Specifies the path of the working directory for Live DRC.

Data Types

string

Default signoff_check_drc_live_run

Description

This application option specifies the path of the working directory. The path can be either relative or absolute.

If you do not specify this option, the tool creates a directory named signoff_check_drc_live_run under the current working directory.

Examples

The following example runs Live DRC in the working directory named my_signoff_run.

```
prompt> set_app_options -name signoff.check_drc_live.run_dir \\  
        -value "my_signoff_run"  
prompt> signoff_check_live_drc -rect {{55.633 72.456} {62.766 67.848}}
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)

- [report_app_options](#)
- [set_app_options](#)
- [signoff_check_live_drc](#)

signoff.check_drc_live.runset

Specifies the foundry runset to use for Live DRC. This is a required option.

Data Types

string

Default null

Description

This application option specifies the foundry runset used by Live DRC to perform design rule checking on the current block.

Examples

The following example specifies the DRC runset named drc.rs for Live DRC.

```
prompt> set_app_options -name signoff.check_drc_live.runset \\  
        -value "drc.rs"  
prompt> signoff_check_live_drc -rect {{55.633 72.456} {62.766 67.84
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)
- [signoff_check_live_drc](#)

signoff.color_macro.enabled

Option to run color macro pins flow during *check_workspace* command. Enable the color macro pins flow.

Data Types

boolean

Default false

Description

If you do not specify this option, the tool does not color the pins shapes during library creation.

Examples

The following example runs the color macro pins flow during *check_workspace*.

```
prompt> set_app_options -name signoff.color_macro.enabled -value true
prompt> check_workspace
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)

signoff.color_macro.ignore_fill_view_time_stamp

Option to ignore time stamp of fill internal design.

Data Types

boolean

Default false

Examples

The following example sets the option to ignore timestamp of fill internal design.

```
prompt> set_app_options -list
{signoff.color_macro.ignore_fill_view_time_stamp true}
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)

signoff.color_macro.input_gds

Specifies the input GDSII file.

Data Types

string

Default null

Description

This application option specifies the name of the input GDSII file for the *color_macro_pins* command.

This option has no effect when you run the color macro pins flow in the library manager (icc2_lm_shell).

Examples

The following example sets the name of the input GDSII file as hard_macro.gds.

```
prompt> set_app_options \\  
        -name signoff.color_macro.input_gds -value "hard_macro.gds"  
w
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)
- [color_macro_pins](#)

signoff.color_macro.layer_map

Specifies the layer mapping file for the *color_macro_pins* command.

Data Types

string

Default null

Description

This application option specifies the layer mapping file for the *color_macro_pins* command. It is required only when the color macro pins flow is run on GDSII files. Note that the layers are assigned based on NDM layers for the top cell. The *color_macro_pins* command reads data only from NDM layer numbers. So, the GDSII layers for the metal and boundary layers should be mapped to NDM layer numbers that exist in the top cell. The layer mapping file should map: - Un-colored input metal layers to datatype 0.

- Pre-colored metal layer in color 1 (if exists) to datatype 120.
- Pre-colored metal layer in color 2 (if exists) to datatype 121.
- Boundary layer to datatype 255.

This option has no effect when you run the color macro pins flow in the library manager (icc2_lm_shell).

Examples

The following example reads the layer mapping file named layer.map.

```
prompt> set_app_options \\  
        -name signoff.color_macro.layer_map -value "layer.map"
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)
- [color_macro_pins](#)

signoff.color_macro.macro_name

Specifies the name of macro block to read to run the color macro pins flow.

Data Types

string

Default null

Description

This application option specifies the name of macro block to read to run the color macro pins flow.

s

You must set this option to run the color macro pins flow in the library manager (`icc2_lm_shell`). When this option is set, the `check_workspace` command runs the color macro pins flow.

You must also set this option if you use the GDSII input flow in the application tool.

Examples

The following example reads the macro named `hard_macro` in the color macro pins flow during library creation.

```
prompt> set_app_options \\  
        -name signoff.color_macro.macro_name -value hard_macro
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)
- [color_macro_pins](#)

signoff.color_macro.output_gds

Specifies the name of the GDSII file generated by the `color_macro_pins` command.

Data Types

string

Default null

Description

This application option specifies the name of the GDSII file generated by the `color_macro_pins` command when the color macro pins flow is run on GDSII files.

This option has no effect when you run the color macro pins flow in the library manager (`icc2_lm_shell`).

Examples

The following example sets the name of the output GDSII file to `hard_macro.gds`.

```
prompt> set_app_options \\  
        -name signoff.color_macro.output_gds -value "hard_macro.gds"
```


See Also

- [color_macro_pins](#)
- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)

signoff.color_macro.preferred_routing_direction

Option to specify routing direction of metals.

Data Types

list of string string

Default Default directions are as described in design.

Description

Option to specify the preferred routing direction in color macro pins flow. If this is not specified for the block, the user could specify the preferred routing direction for first two layers. The rest of the layers will be alternate of the previous layer's routing direction.

The user specifies the routing direction with layer name and keyword '\H\' for horizontal and '\V\' for vertical.

Examples

The following example sets the preferred routing direction as given by user in color macro pins flow:

```
prompt> set_app_options -list
{signoff.color_macro.preferred_routing_direction \
  "{M1 H} {M2 V}"}
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)

signoff.color_macro.rule_type

Option for the color macro pins flow. Specifies the type of node and foundry.

Data Types

string

Default tsmc20

Description

If you do not specify this option, the tool uses tsmc20 as rule type to run the color macro pin flow. The valid values of rule_type is tsmc20, tsmc16, samsung20, samsung16 and samsung10.

Examples

The following example runs the color macro pins flow for rule type samsung20.

```
prompt> set_app_options -name signoff.color_macro.rule_type -value  
"samsung20"
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)

signoff.color_macro.run_dir

Option for the color macro pins flow. Specifies the path of working directory.

Data Types

string

Default pin_color_<design_name>

Description

If you do not specify this option, the tool uses a directory called pin_color_<design_name> under the current working directory. The path can be either relative or absolute.

Examples

The following example runs IC Validator under the working directory named *my_signoff_run_<design_name>*.

```
prompt> set_app_options -name signoff.color_macro.run_dir -value  
"my_signoff_run"
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)

signoff.color_macro.technology_file

Specifies the technology file for the *color_macro_pins* command.

Data Types

string

Default null

Description

This application option specifies the technology file for the *color_macro_pins* command. The technology file is required to get the layer details when the color macro pins flow is run on GDSII files.

This option has no effect when you run the color macro pins flow in the library manager (icc2_lm_shell).

Examples

The following example specifies the technology file as mytech.tf.

```
prompt> set_app_options \  
-name signoff.color_macro.technology_file -value "mytech.tf"
```

See Also

- [color_macro_pins](#)
- [get_app_option_value](#)
- [get_app_options](#)

- [report_app_options](#)
- [set_app_options](#)

signoff.color_macro.track_color_preference

Option to specify the color pattern in the design.

Data Types

list of string string

Default nocolor

Description

In the case of pre-color information not present in design, this option is used to specify the color pattern of tracks. By default, it uses the colors pre-assigned to the tracks.

There are only two valid values that could be passed with each layer - color1 and color2.

Examples

The following example sets the track color preference in color macro pins flow.

```
prompt> set_app_options -list
{signoff.color_macro.track_color_preference "{M1 color1} \
{M2 color2} {M3 color1}"}
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)

signoff.color_macro.track_offset

Option to specify the track information for the macro block.

Data Types

List of string double

Default zero.

Description

Option to derive correct track information for the macro block from the library. There is no easy way to derive track information from newly created library, this is an user option to provide track offset values. The offset is the distance between the first track and the bottom left corner of the boundary of the macro. The offset values is referred to bottom most track for horizontal tracks and left most track for vertical tracks.

Examples

The following example sets the track offset values for each layer's track in color macro pins flow:

```
prompt> set_app_options -list {signoff.color_macro.track_offset \
    "{M1 0.05} {M2 0.05} {M3 0.065} {M4 0.05}"}
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)

signoff.create_metal_fill.apply_nondefault_rules

Option for the *signoff_create_metal_fill* command. Specifies whether or not to honor of NDR rules during fill creation.

Data Types

boolean

Default false

Description

If the user specifies the option, NDR rules are honored during fill creation flow, ie. no fill will be created in NDR blocked area.

Examples

The following example enables honoring of NDR rules.

```
prompt> set_app_options -name
    signoff.create_metal_fill.apply_nondefault_rules -value true
prompt> signoff_create_metal_fill
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)
- [signoff_create_metal_fill](#)

signoff.create_metal_fill.auto_eco_threshold_value

Option for the *signoff_create_metal_fill* command. Specifies the maximum threshold in design change area which will allow incremental fill to be run. Default is 20%.

Data Types

integer

Default 20

Description

Specifies the maximum threshold in design change area which will allow incremental fill to be run. Default is 20%.

Examples

The following example sets the maximum change area to threshold to 30% and then invokes *signoff_create_metal_fill*.

```
prompt> set_app_options -name  
      signoff.create_metal_fill.auto_eco_threshold_value -value 30  
prompt> signoff_create_metal_fill -auto_eco true
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)
- [signoff_create_metal_fill](#)

signoff.create_metal_fill.base_layer_runset

Option for the *signoff_create_metal_fill* command. Specifies the foundry runset for base layers to use for metal fill insertion.

Data Types

string

Default null

Description

The foundry runset for base layers will be used while the tool invokes IC Validator for metal fill insertion in the current design.

Examples

The following example specifies the fill runset for base layers *base_fillfill.rs* for the *signoff_create_metal_fill* command.

```
prompt> set_app_options -name signoff.create_metal_fill.base_layer_runset  
          -value "base_fill.rs"  
prompt> signoff_create_metal_fill -foundry_fill_type feol
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)
- [signoff_create_metal_fill](#)

signoff.create_metal_fill.fill_over_net_on_adjacent_layer

Option for the *signoff_create_metal_fill* command. Insert metal fill on the vertical extension regions of critical nets on adjacent layers.

Data Types

bool

Default false

Description

The option only takes effect when *signoff_create_metal_fill* option *-nets* and/or *-timing_preserve_setup_slack_threshold* are specified.

If true, metal fills are inserted on adjacent layers of timing critical nets.

By default (false), the vertical extension regions of critical nets on adjacent layers are blocked with the minimum spacing.

Examples

The following example fills all empty regions with metal fill, except for the regions within two times the minimum spacing away from critical nets on the same layer. The adjacent layers of timing critical nets are also filled. The critical nets are n1, n2, and n3.

```
prompt> set_app_options -name  
      signoff.create_metal_fill.fill_over_net_on_adjacent_layer -value true  
prompt> signoff_create_metal_fill -nets {n1 n2 n3}
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)
- [signoff_create_metal_fill](#)

signoff.create_metal_fill.fill_shielded_clock

Option for the *signoff_create_metal_fill* command. Insert metal fill till the *space_to_net* value of the shieldings of a shielded clock net.

Data Types

bool

Default false

Description

If true, metal fills are inserted till the *space_to_net* value of the shieldings of a shielded timing critical clock net.

If the *space_to_clock* is set, that value will be used for inserting the metal fills.

Examples

The following example fills till the shielding of the timing critical clock nets using the `space_to_net` value.

```
prompt> set_app_options -name  
signoff.create_metal_fill.fill_shielded_clock -value true
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)
- [signoff_create_metal_fill](#)

signoff.create_metal_fill.fix_density_errors

Option for the `signoff_create_metal_fill` command. Specifies the option to enable honoring of density rules during fill insertion, removal and addition.

Data Types

boolean

Default false

Description

When this option is enabled, density rules will be honored during fill insertion, removal and addition. When doing timing aware fill insertion/removal, fill shapes near timing critical nets will be removed. Excessive removal of fill can cause density violations. Enabling 'fix_density_errors' will make sure that removal of fill is done to the extent that it does not violate density rules. When doing fill addition (-mode add), enabling this option will ensure that we add fill shapes to the extent that it satisfies minimum required density and no more than that. This option is useful to limit addition of fill in density violating windows only.

By default, the density rules to be honored are taken from the tech file. If the rules do not exist in tech file, `signoff_create_metal_fill` command will error out. For track fill, density rules can be specified through the user parameter file, through option 'track_fill_runset_include_file'. An example set of density rules would be:

```
m2_min_density = 30; m2_min_density_window = 50; m3_min_density = 30;  
m3_min_density_window = 50;
```

By default, for track fill, if density rule for a particular layer do not exist in tech file and user parameter file, the density window size is taken as 50 micron X 50 micron and min required density is 5 percent. Please note that values specified in user parameter file will take precedence over tech file values.

This app option will be ignored when `signoff_create_metal_fill` doesn't work in timing-aware mode or append mode (-mode add). Because in non-timing-aware mode, density is always taken into consideration. And in append mode, density is also considered by default, defining this option to be true is to limit addition of fill in density violating windows only.

Examples

The following example runs timing aware fill flow with honoring of density rules enabled.

```
prompt> set_app_options -name  
signoff.create_metal_fill.fix_density_errors -value true  
prompt> signoff_create_metal_fill -timing_preserve_setup_slack_threshold  
0.01
```

The following example runs timing aware fill removal on existing fill. Fill will be removed to the extent that it does not violate density rules.

```
prompt> set_app_options -name  
signoff.create_metal_fill.fix_density_errors -value true  
prompt> signoff_create_metal_fill -timing_preserve_setup_slack_threshold  
0.01 -mode remove
```

The following example adds track fill on top of existing fill. Since 'fix_density_errors' is enabled, fill addition will be limited to density violating windows only.

```
prompt> set_app_options -name  
signoff.create_metal_fill.fix_density_errors -value true  
prompt> signoff_create_metal_fill -mode add -track_fill generic
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)
- [signoff_create_metal_fill](#)

signoff.create_metal_fill.flat

Option for the `signoff_create_metal_fill` command. Force IC Validator generate flat fill.

Data Types

bool

Default false

Description

There are two modes of fill: hierarchical fill and flat fill.

If true, the command forces IC Validator to generate flat fill.

By default (false), the mode in the runset is used.

Examples

The following example forces IC Validator generate flat fill.

```
prompt> set_app_options -name signoff.create_metal_fill.flat -value true
prompt> signoff_create_metal_fill
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)
- [signoff_create_metal_fill](#)

signoff.create_metal_fill.max_density_threshold

Specifies the maximum density threshold value for a track fill layer. Used by the *signoff_create_metal_fill* command.

Data Types

list

Default {}

Description

The user provides a list of {layerName value} pairs to control max density threshold value for a track fill layer. When this option is given, the fill will be trimmed to bring its density close to max density threshold specified by user.

Examples

The following example sets a max density threshold of 40 percent for M2, M3, M4.

```
prompt> set_app_options \\  
    -name signoff.create_metal_fill.max_density_threshold \\  
    -value {{M2 40} {M3 40} {M4 40}}  
prompt> signoff_create_metal_fill -track_fill
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)
- [signoff_create_metal_fill](#)

signoff.create_metal_fill.read_design_views

Option for the *signoff_create_metal_fill* command. By default, *signoff_create_metal_fill* reads child cell data from the frame view. This option can be used to read child cell data from the design view.

Data Types

list_of_child_cell_names

Default none

Description

If set, the child cell data is read from the design view for the cells specified in the list. If the design view of a cell instance cannot be found, a warning message is printed and the command reads the frame view instead. For convenience, all cells can be specified by the `{*}` wildcard.

By default (none), the child cell data is read from the frame view. If the frame view of a child cell cannot be found, the command aborts.

Examples

The following example attempts to read the cell instance data for the ch1, ch2 child cells from the design view. If the design view for this macro cannot be found the command reads the frame view instead. The child cell data for all other cells is read from the frame view.

s

```
prompt> set_app_options -name signoff.create_metal_fill.read_design_views
        -value {ch1 ch2}
prompt> signoff_create_metal_fill
```

The following example attempts to read the cell instance data for all cells from the design view. If the design view cannot be found, the command reads the frame view instead.

```
prompt> set_app_options -name signoff.create_metal_fill.read_design_views
        -value {*}
prompt> signoff_create_metal_fill
```

The following example attempts to read the cell instance data for all cells, except ch1 from the design view. If the design view cannot be found, the command reads the frame view instead.

```
prompt> set_app_options -name signoff.create_metal_fill.read_design_views
        -value {* !ch1}
prompt> signoff_create_metal_fill
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)
- [signoff_create_metal_fill](#)

signoff.create_metal_fill.read_layout_views

Option for the *signoff_create_metal_fill* command. By default, *signoff_create_metal_fill* reads child cell data from the frame view. This option can be used to read child cell data from the layout view.

Data Types

list_of_child_cell_names

Default none

Description

If set, the child cell data is read from the layout view for the cells specified in the list. If the layout view of a cell instance cannot be found, application will read design view, if it exists, otherwise it will read the frame view. For convenience, all cells can be specified by the *{*}* wildcard.

s

By default (none), the child cell data is read from the frame view. If the frame view of a child cell cannot be found, the command aborts.

Examples

The following example attempts to read the cell instance data for the ch1, ch2 child cells from the layout view. The child cell data for all other cells is read from the frame view.

```
prompt> set_app_options -name signoff.create_metal_fill.read_layout_views  
-value {ch1 ch2}  
prompt> signoff_create_metal_fill
```

The following example attempts to read the cell instance data for all cells from the layout view.

```
prompt> set_app_options -name signoff.create_metal_fill.read_layout_views  
-value {*}  
prompt> signoff_create_metal_fill
```

The following example attempts to read the cell instance data for all cells, except ch1 from the layout view.

```
prompt> set_app_options -name signoff.create_metal_fill.read_layout_views  
-value {* !ch1}  
prompt> signoff_create_metal_fill
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)
- [signoff_create_metal_fill](#)

signoff.create_metal_fill.run_dir

Option for the *signoff_create_metal_fill* command. Specifies the path of the working directory.

Data Types

string

Default signoff_fill_run

s

Description

If you do not specify this option, the tool uses a directory called `signoff_fill_run` under the current working directory. The path can be either relative or absolute.

Examples

The following example runs IC Validator under the working directory named *my_signoff_run*.

```
prompt> set_app_options -name signoff.create_metal_fill.run_dir -value  
"my_signoff_run"  
prompt> signoff_create_metal_fill
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)
- [signoff_create_metal_fill](#)

signoff.create_metal_fill.runset

Option for the *signoff_create_metal_fill* command. Specifies the foundry runset to use for metal fill insertion.

Data Types

string

Default null

Description

The foundry runset will be used while the tool invokes IC Validator for metal fill insertion in the current design.

Examples

The following example specifies the fill runset *fill.rs* for the *signoff_create_metal_fill* command.

```
prompt> set_app_options -name signoff.create_metal_fill.runset -value  
"fill.rs"  
prompt> signoff_create_metal_fill
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)
- [signoff_create_metal_fill](#)

signoff.create_metal_fill.space_to_clock_nets

Option for the *signoff_create_metal_fill* command. Specifies the fill spacing parameters to clock nets.

Data Types

list

Default {}

Description

This option takes effect only when *signoff_create_metal_fill* option *-nets* is specified. Although a combination of clock nets as well as non-clock nets can be specified through option *-nets*, the clock net spacing will apply only for the clock nets. This option specifies the spacing that fill needs to keep from clock net shapes for each given metal layer. The spacing value can be an explicit value in micron or a multiple of minimum spacing value for the given metal layer.

The list format for specifying explicit spacing values looks like following: {{layer_name1 value1} {layer_name2 value2} ... }

Example: {{M2 0.4} {M3 0.5}}

For specifying spacing values as multiple of minimum spacing, the multiplier is appended with a 'x' or 'X': Example: {{M2 2X} {M3 3x}}

You must specify the layer name by using the layer name from technology file.

By default, this command uses the spacing value as two times of minimum spacing specified in the technology file for each layer.

Examples

The following example provides spacing info for clock nets. M1 fill is blocked till 3 times minimum spacing from M1 route shapes and M2 fill is blocked till 3 times minimum spacing from M2 route shapes. The clock nets are clk1, clk2, and clk3.

s

```
prompt> set_app_options -name
        signoff.create_metal_fill.space_to_clock_nets -value {{M1 3x} {M2 3x}}
prompt> signoff_create_metal_fill -nets {clk1 clk2 clk3}
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)
- [signoff_create_metal_fill](#)

signoff.create_metal_fill.space_to_clock_nets_on_adjacent_layer

Specifies the fill spacing on adjacent layers to clock nets. Option for the *signoff_create_metal_fill* command.

Data Types

list

Default {}

Description

This option only takes effect only when *signoff_create_metal_fill* option *-nets* is specified. Although a combination of clock nets as well as non-clock nets can be specified through option *-nets*, the clock net spacing will apply only for the clock nets. This option specifies the spacing that adjacent layer fill needs to keep from clock net shapes for each given metal layer. The spacing value can be an explicit value in micron or a multiple of minimum spacing value for the given metal layer.

The list format for specifying explicit spacing values looks like following:

```
{{layer_name1 value1} {layer_name2 value2} ... }
```

Example:

```
{{M2 0.4} {M3 0.5}}
```

For specifying spacing values as multiple of minimum spacing, the multiplier is appended with a 'x' or 'X': Example:

```
{{M2 2X} {M3 3x}}
```

s

You must specify the layer name by using the layer name from technology file. By default, this command uses the adjacent fill spacing value as one times of minimum spacing specified in the technology file for the adjacent layer.

Examples

In the following example, from a M3 clock net shape, M2 fill will be blocked till 3 times M2 minSpacing, and M4 fill will be blocked till 4 times M4 minSpacing. The clock nets are n1, n2, and n3.

```
prompt> set_app_options \\  
-name signoff.create_metal_fill.space_to_clock_nets_on_adjacent_layer  
\\  
-value {{M2 3x} {M3 2x} {M4 4x}}  
prompt> signoff_create_metal_fill -nets {n1 n2 n3}
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)
- [signoff_create_metal_fill](#)

signoff.create_metal_fill.space_to_nets

Option for the *signoff_create_metal_fill* command. Specifies the fill-to-critical-nets spacing for each given metal layer.

Data Types

list

Default null

Description

This option takes effect only when *signoff_create_metal_fill* option *-nets* and/or *-timing_preserve_setup_slack_threshold* are specified. This option specifies the spacing that fill needs to keep from critical net shapes for each given metal layer. The spacing value can be an explicit value in micron or a multiple of minimum spacing value for the given metal layer.

The list format for specifying explicit spacing values looks like following: {{layer_name1 value1} {layer_name2 value2} ... }

Example: {{M2 0.4} {M3 0.5}}

For specifying spacing values as multiple of minimum spacing, the multiplier is appended with a 'x' or 'X': Example: {{M2 2X} {M3 3x}}

You must specify the layer name by using the layer name from technology file.

By default, this command uses the spacing value as two times of minimum spacing specified in the technology file for each layer.

Examples

The following example specifies that M2 fill shapes should be kept at 3 times of minimum spacing from M2 net shapes and M3 fill shapes should be kept at .04 micron away from M3 net shapes. The critical nets are n1, n2, and n3.

```
prompt> set_app_options -name signoff.create_metal_fill.space_to_nets  
-value {{M2 3X} {M3 0.04}}  
prompt> signoff_create_metal_fill -nets {n1 n2 n3}
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)
- [signoff_create_metal_fill](#)

signoff.create_metal_fill.space_to_nets_on_adjacent_layer

Specifies the fill spacing on adjacent layers to critical nets. Option for the *signoff_create_metal_fill* command.

Data Types

list

Default {}

Description

This option only takes effect only when *signoff_create_metal_fill* option *-nets* and/or *-timing_preserve_setup_slack_threshold* are specified. This option specifies the spacing that adjacent layer fill needs to keep from critical net shapes for each given metal layer. The spacing value can be an explicit value in micron or a multiple of minimum spacing value for the given metal layer.

The list format for specifying explicit spacing values looks like following:

```
{{layer_name1 value1} {layer_name2 value2} ... }
```

Example:

```
{{M2 0.4} {M3 0.5}}
```

For specifying spacing values as multiple of minimum spacing, the multiplier is appended with a 'x' or 'X': Example:

```
{{M2 2X} {M3 3x}}
```

You must specify the layer name by using the layer name from technology file. By default, this command uses the adjacent fill spacing value as one times of minimum spacing specified in the technology file for the adjacent layer.

Examples

In the following example, from a M3 net shape, M2 fill will be blocked till 3 times M2 minSpacing, and M4 fill will be blocked till 4 times M4 minSpacing. The critical nets are n1, n2, and n3.

```
prompt> set_app_options \\  
-name signoff.create_metal_fill.space_to_nets_on_adjacent_layer \\  
-value {{M2 3x} {M3 2x} {M4 4x}}  
prompt> signoff_create_metal_fill -nets {n1 n2 n3}
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)
- [signoff_create_metal_fill](#)

signoff.create_metal_fill.tcd_fill

Option for the *signoff_create_metal_fill* command. Specifies to handle TCD structures of foundry metal fill runsets in *signoff_create_metal_fill* flow.

Data Types

Boolean

Default false

Description

This option specifies to add handling of TCD structures of foundry metal fill runsets for 28/20/16nm nodes in `signoff_create_metal_fill` flow.

By default, the `signoff_create_metal_fill` command only handles fill for routing layers and does not save fill data for non-routing layers.

TCD structures have data on typically non routing layers, So even if runset outputs these layers, data is not saved to ndm db.

This option supports insertion and removal of TCD structures in conjunction with all `signoff_create_metal_fill` flow options.

By default it is false.

Examples

The following example will create fill along with the TCD structures if runset outputs any.

```
prompt> set_app_options -name signoff.create_metal_fill.tcd_fill -value  
true  
prompt> signoff_create_metal_fill
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)
- [signoff_create_metal_fill](#)

signoff.create_metal_fill.user_defined_options

Specifies the string to be passed as the command line options to IC Validator. Option for the `signoff_create_metal_fill` command.

Data Types

string

Default null

Description

This application option specifies additional command line options for IC Validator. Note that IC Compiler II does not perform any validation on the specified string.

By default, no user-defined options are included.

Examples

The following example specifies the user-defined option named *-turbo*.

```
prompt> set_app_options -name  
        signoff.create_metal_fill.user_defined_options -value "-turbo"  
prompt> signoff_create_metal_fill
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)
- [signoff_create_metal_fill](#)

signoff.create_pg_augmentation.adjacent_layer_timing_protection_mode

Option to enable/disable the adjacent layer protection for timing critical nets. The shapes of adjacent layers are considered for upper and lower metal layers of layer shape of the timing critical nets. By default, this is set to true. Option for the *signoff_create_pg_augmentation* command.

Data Types

string

Default off

Description

There are two possible values for this option.

"false" Adjacent layer timing protection is disabled.

"true" Adjacent layer timing protection is enabled.

When enabled (i.e., not set to `false`), PGA shapes of adjacent layers of timing critical net shapes are removed for timing protection.

Examples

The following example disables the adjacent layer timing protection.

s

```
prompt> set_app_options -name
signoff.create_pg_augmentation.adjacent_layer_timing_protection_mode
-value "false"
prompt> signoff_create_pg_augmentation -node tsmc3 -mode add
-output_colored_shapes true -parameter_file ./test.rh -mode
trim_pga_by_timing -max_timing_paths 10000
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)
- [signoff_create_pg_augmentation](#)

signoff.create_pg_augmentation.enable_auto_set_width

Selects the width of PG augmentation shapes automatically for designs where the default width of PG augmentation does not allow the formation of both side via connections. Option for the *signoff_create_pg_augmentation* command.

Data Types

Boolean

Default true

Description

The *signoff.create_pg_augmentation.enable_auto_set_width* application option controls whether the tool automatically selects the width of PG augmentation shapes during the execution of the *signoff_create_pg_augmentation* command.

By default, this option is enabled (*true*), allowing the tool to adjust the width of augmentation shapes to ensure proper via connections on both sides. You can disable this feature by setting the option to *false*.

Examples

The following example disables the automatic width selection for PG augmentation shapes:

```
prompt> set_app_options -name
signoff.create_pg_augmentation.enable_auto_set_width
-value false
prompt> signoff_create_pg_augmentation
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)
- [signoff_create_pg_augmentation](#)

signoff.create_pg_augmentation.ground_net_name

Specifies the name of the ground net for PGA. Option for the *signoff_create_pg_augmentation* command.

Data Types

string

Default ""

Description

If you do not specify this option, the tool does not create PGA for any ground net. You must specify this or *signoff.create_pg_augmentation.power_net_name* application option.

Examples

The following example runs PGA for ground net "VSS".

```
prompt> set_app_options -name  
        signoff.create_pg_augmentation.ground_net_name  
        -value "VSS"  
prompt> signoff_create_pg_augmentation
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)
- [signoff_create_pg_augmentation](#)

signoff.create_pg_augmentation.parameter_file_check_mode

Option to enable the parsing of the user parameter file, which is essential for validating user-provided inputs in the parameter file. If this option is not specified, invalid user inputs in the parameter file can lead to errors. Option for the *signoff_create_pg_augmentation* command.

Data Types

string

Default off

Description

There are three possible modes to check parameter file.

"off" No validation is performed.

"warn" Prints a warning but does not exit, allowing the process to continue.

"error" Displays an error and exits the process.

When enabled (i.e., not set to `off`), the `track\_fill\_params.rh` will show enhanced user parameter values. This validation is crucial for defining custom recipes for PGA parameters, such as changes in PGA width/side spacing. If the user provides incorrect values, errors/warnings are flagged early in the process.

Examples

The following example sets parameter file check mode to error.

```
prompt> set_app_options -name  
        signoff.create_pg_augmentation.parameter_file_check_mode -value "error"  
prompt> signoff_create_pg_augmentation -node tsmc3 -mode add  
        -output_colored_shapes true -parameter_file ./test.rh
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)
- [signoff_create_pg_augmentation](#)

signoff.create_pg_augmentation.power_net_name

Specifies the name of power net for PGA. Option for the *signoff_create_pg_augmentation* command.

Data Types

string

Default ""

Description

If you do not specify this option, the tool does not create PGA for any power net. You must specify either this or *signoff.create_pg_augmentation.ground_net_name* application option.

Examples

The following example runs PGA for power net "VDD".

```
prompt> set_app_options -name  
        signoff.create_pg_augmentation.power_net_name  
        -value "VDD"  
prompt> signoff_create_pg_augmentation
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)
- [signoff_create_pg_augmentation](#)

signoff.create_pg_augmentation.run_dir

Specifies the path of the working directory. Option for the *signoff_create_pg_augmentation* command.

Data Types

string

Default signoff_create_pg_augmentation_run

Description

If you do not specify this option, the tool uses a directory called `signoff_create_pg_augmentation_run` under the current working directory. The path can be either relative or absolute.

Examples

The following example runs IC Validator under the working directory named `my_signoff_run`.

```
prompt> set_app_options -name signoff.create_pg_augmentation.run_dir  
-value "my_signoff_run"  
prompt> signoff_create_pg_augmentation
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)
- [signoff_create_pg_augmentation](#)

signoff.create_pg_augmentation.timing_driven

Enables PG augmentation shapes insertion followed by trim PGA for timing recovery. Option for the `signoff_create_pg_augmentation` command.

Data Types

Boolean

Default false

Description

The `signoff.create_pg_augmentation.timing_driven` application option enables 2 pass flow under the hood where PG augmentation shapes are inserted in first pass and then trimmed around critical paths/nets in second pass for timing recovery.

By default, ICV will be called twice with this app option and there will be 2 sub directories `rundir.create_pga` and `rundir.trim_pga_1` created inside main run directory for for ICV runs.

By default PG augmentation shapes will be trimmed around maximum 1000000 max violating paths extracted with setup slack < 0 by traversing source to sink topology of violating paths of nets and nets with transition setup slack < 0.

s

The *signoff_create_pg_augmentation* command options *-max_timing_path*, *-timing_preserve_setup_slack_threshold*, *-timing_preserve_transition_setup_slack_threshold*, will take precedence during trim run if options specified by the user.

Examples

The following example enables the auto timing_driven flow for pg_augmentation flow:

```
prompt> set_app_options -name
        signoff.create_pg_augmentation.timing_driven
        -value true
prompt> signoff_create_pg_augmentation
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)
- [signoff_create_pg_augmentation](#)

signoff.create_pg_augmentation.user_defined_options

Specifies the string to be passed as the command line options to IC Validator. Option for the *signoff_create_pg_augmentation* command.

Data Types

string

Default null

Description

This application option specifies the additional command line options for IC Validator. Note that IC Compiler II does not perform any validation on the specified string.

By default, no user-defined options are included.

Examples

The following example specifies the user-defined option named *-turbo*.

```
prompt> set_app_options -name
        signoff.create_pg_augmentation.user_defined_options -value "-turbo"
prompt> signoff_create_pg_augmentation
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)
- [signoff_create_pg_augmentation](#)

signoff.fix_drc.advanced_guidance_for_rules

Specifies whether the *signoff_fix_drc* command applies rule-specific heuristics during the automatic design rule (ADR) fixing flow.

Data Types

string

Default all

Description

This application option controls whether the *signoff_fix_drc* command applies rule-specific heuristics during the automatic design rule (ADR) fixing flow.

By default (*all*), the command applies all available rule-specific heuristics. To disable application of the rule-specific heuristics, set this application option to *off*.

Examples

The following example runs the ADR fixing flow using the rule-specific heuristics.

```
prompt> set_app_options \\  
        -name signoff.fix_drc.advanced_guidance_for_rules \\  
        -value "all"  
prompt> signoff_fix_drc
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)
- [signoff_fix_drc](#)

signoff.fix_drc.check_drc

Specifies whether the intermediate DRC runs must be done on complete design or the regions, which have been changed during automatic DRC repair (ADR) fixing flow. Option for the *signoff_fix_drc* command.

Data Types

string

Default local

Description

If you do not specify this option, the tool uses "local", which means that the regions, which have been changed during ADR fixing flow, are checked for DRCs. If you select "global", then it means that all regions of the design are checked for DRCs.

Examples

The following example runs ADR fixing flow with DRC being run on the complete design.

```
prompt> set_app_options -list {signoff.fix_drc.check_drc "global"}  
prompt> signoff_fix_drc
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)
- [signoff_fix_drc](#)

signoff.fix_drc.config_file

Specifies the path of configuration file and it is an option for the *signoff_fix_drc* command.

Data Types

string

Default auto

Description

If you do not specify this option, the tool uses "auto", which is automatically generated configuration file.

s

Examples

The following example runs with *my_config_file* as the configuration file.

```
prompt> set_app_options -list {signoff.fix_drc.config_file  
    "my_config_file"}  
prompt> signoff_fix_drc
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)
- [signoff_fix_drc](#)

signoff.fix_drc.custom_guidance

Specifies the way DPT fixing flow must be run. Option for the *signoff_fix_drc* command.

Data Types

string

Default off

Description

With default "off" value, DPT error fixing is not executed during ADR flow.

With value "dpt", this option enables the automated DRC repair (ADR) flow to fix only DPT errors. If this option is defined with value "dpt", "signoff_fix_drc -select_rules" with DPT rule names must be specified.

With value "all", this option enables ADR flow to fix DPT errors as part of the generic ADR flow (where other nonDPT errors are targeted as well). See information about signoff.fix_drc.dpt_rule_information option for more details.

Examples

The following example runs ADR for DPT error fixing.

```
prompt> set_app_options -list {signoff.fix_drc.custom_guidance dpt}  
prompt> signoff_fix_drc -select_rules {"*G0*"}
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)
- [signoff_fix_drc](#)

signoff.fix_drc.dpt_rule_information

Specifies the file that contains the list of double-patterning rules and the corresponding metal or via layer.

Data Types

string

Default ""

Description

The *signoff.fix_drc.dpt_rule_information* application option specifies the file that contains the list of double-patterning rules and the corresponding metal or via layer. The tool uses this application option only when the *signoff.fix_drc.custom_guidance* application option is set to *all*, which enables double-patterning fixes during the automatic DRC fixing flow.

Each line in the file uses the following syntax:

```
"<rule>" "<layer_name>"
```

The rule string must match the first part of the violation comment from the runset file (which should be unique, based on the violation comment naming conventions) and cannot include wildcards.

For example,

```
"M2 DPT odd cycle conflict" "metal2"
```

When you specify this application option, the tool uses the rules specified in this file in addition to the following default rules:

```
"M1.G0.1__M1.G0.2__M1.G0.3" "metal1"  
"M2.G0.1__M2.G0.2__M2.G0.3" "metal2"  
"M3.G0.1__M3.G0.2__M3.G0.3" "metal3"  
"M4.G0.1__M4.G0.2__M4.G0.3" "metal4"  
"M5.G0.1__M5.G0.2__M5.G0.3" "metal5"  
"M6.G0.1__M6.G0.2__M6.G0.3" "metal6"
```



```
"GRMx.C.2_M1" "metal1"  
"GRMx.C.2_M2" "metal2"  
"GRMx.C.2_M3" "metal3"  
"GRMx.C.2_M4" "metal4"  
  
"GRV1.C.6" "via1"  
"GRVx.C.6_V2" "via2"  
"GRVx.C.6_V3" "via3"
```

If you do not specify this option, the tool uses this default list as the double-patterning rules.

Examples

The following example runs the automatic DRC fixing flow with the additional double-patterning rules specified in the file named `my_dpt_rules`.

```
prompt> set_app_options -name signoff.fix_drc.custom_guidance \  
-value all  
prompt> set_app_options -name signoff.fix_drc.dpt_rule_information \  
-value my_dpt_rules  
prompt> signoff_fix_drc
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)
- [signoff_fix_drc](#)

signoff.fix_drc.fix_detail_route_drc

Specifies how the detail route DRCs are fixed while fixing Signoff DRCs. Option for the `signoff_fix_drc` command.

Data Types

string

Default global

Description

After fixing signoff DRCs, automatic DRC repair (ADR) also tries to fix detail route DRCs. This option specifies if the fixing need to be done only near to signoff DRC or full chip.

s

- *local* : Detail route DRCs near to Signoff DRCs are tried for fixing.
- *global* : Detail route DRCs in full chip are tried for fixing.

By default, full chip detail route DRCs are fixed.

Examples

The following example runs ADR with local detail route DRC fixing.

```
prompt> set_app_options -list {signoff.fix_drc.fix_detail_route_drc
      local}
prompt> signoff_fix_drc
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)
- [signoff_fix_drc](#)

signoff.fix_drc.init_drc_error_db

Specifies the initial DRC error database path for the *signoff_fix_drc* command.

Data Types

string

Default This is a required option.

Description

Specifies the path of the directory that contains the initial DRC error database generated by IC Validator. The path can be either relative or absolute. This option is required.

Examples

The following example runs ADR using the initial DRC error database from working directory named *my_signoff_check_run*.

```
prompt> set_app_options -list {signoff.check_drc.run_dir
      "my_signoff_check_run"}
prompt> set_app_options -list {signoff.fix_drc.initial_drc_error_db
      "my_signoff_check_run"}
prompt> signoff_check_drc
prompt> signoff_fix_drc
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)
- [signoff_check_drc](#)
- [signoff_fix_drc](#)

signoff.fix_drc.last_run_full_chip

Specifies if the last run during incremental_drc_level=high is run on complete design or not. It is option for the *signoff_fix_drc* command.

Data Types

Boolean

Default false

Description

If you do not specify this option, the tool runs the last drc on the sub design with changes due to automatic DRC repair (ADR). This causes inaccuracy during reporting, but is fast compared to complete design DRC.

Examples

The following example runs ADR with last DRC being complete design DRC.

```
prompt> set_app_options -list {signoff.fix_drc.last_run_full_chip true}
prompt> signoff_fix_drc
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)
- [signoff_fix_drc](#)

signoff.fix_drc.max_detail_route_iterations

Specifies the number of router search and repair loops for the *signoff_fix_drc* command to run after router guidance fixing.

Data Types

integer

Default 5

Description

Specifies the number of search and repair loops to run after route guidance fixing. This is mainly for cleaning up DRC violations generated during route guidance fixing. The router runs the specified number of detail route iterations if you set the *route.detail.force_max_number_iterations* option to true.

You can specify an integer between 0 and 1000. The default value is 5.

Examples

The following example runs 10 search and repair loop after route guidance fixing.

```
prompt> set_app_options -list  
      {signoff.fix_drc.max_detail_route_iterations 10}  
prompt> signoff_fix_drc
```

The following example forces the maximum number of detail route iterations.

```
prompt> set_app_options -list {route.detail.force_max_number_iterations  
      true}  
prompt> set_app_options -list  
      {signoff.fix_drc.max_detail_route_iterations 10}  
prompt> signoff_fix_drc
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)
- [signoff_fix_drc](#)

signoff.fix_drc.max_errors_per_rule

Specifies the maximum number of errors per rule for the IC Validator run. Option for the *signoff_fix_drc* command.

Data Types

integer

Default 1000

Description

This application option specifies the maximum number of errors per rule for the IC Validator run. The value must be greater than or equal to 1. The default is 1000.

If the number of errors exceeds the specified value, the entire rule is skipped during the signoff fixing flow. A reasonable value should be relative to the design size. For a typical design, the default of 1000 makes sense. For a huge design, targeting more than 1000 errors per rule might be a valid request.

Examples

The following example runs IC Validator for design rule checking and sets the maximum number of errors per rule to 8000.

```
prompt> set_app_options -list {signoff.fix_drc.max_errors_per_rule 8000}  
prompt> signoff_fix_drc
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)
- [signoff_fix_drc](#)

signoff.fix_drc.run_dir

Specifies the path of working directory. Option for the *signoff_fix_drc* command.

Data Types

string

Default signoff_fix_drc_run

Description

If you do not specify this option, the tool uses a directory called `signoff_fix_drc_run` under the current working directory. The path can be either relative or absolute.

Examples

The following example runs ADR under the working directory named *my_signoff_fix_run*.

```
prompt> set_app_options -list {signoff_fix_drc.run_dir  
      "my_signoff_fix_run"}  
prompt> signoff_fix_drc
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)
- [signoff_fix_drc](#)

signoff_fix_drc.target_clock_nets

Specifies if errors associated with clock nets must be targeted by automatic DRC repair (ADR). Option for the *signoff_fix_drc* command.

Data Types

Boolean

Default false

Description

If you do not specify this application option, the tool would not target errors associated with clock nets. If this application option is set to true, the tool would target errors associated with clock nets during ADR fixing.

Note: When the `signoff_fix_drc.target_clock_nets` option is set to false, the router does not have big routing topology changes on clock nets, but it does allow minor shape changes in local area for fixing DRC violations.

Examples

The following example enables targeting of errors associated with clock nets during ADR fixing.

s

```
prompt> set_app_options -list {signoff_fix_drc.target_clock_nets true}
prompt> signoff_fix_drc
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)
- [signoff_fix_drc](#)

signoff_fix_drc.user_defined_options

Specifies strings as part of IC Validator command line options and it is an option for the *signoff_fix_drc* command.

Data Types

string

Default null

Description

This application option specifies additional options for the IC Validator command line. The string that you specify is added to the command line used to invoke design rule checking in IC Validator. IC Compiler II does not perform any checking on the specified string.

By default, no user-defined options are included.

Examples

The following example specifies the user-defined options named -turbo.

```
prompt> set_app_options -list {signoff_fix_drc.user_defined_options
"-turbo"}
prompt> signoff_fix_drc
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)

- [set_app_options](#)
- [signoff_fix_drc](#)

signoff.fix_isolated_via.avoid_net_types

Specifies net types to be avoided for isolated via fixing. Option for the *signoff_fix_isolated_via* command.

Data Types

list

Default null

Description

This application option specifies the net types to be avoided for isolated via fixing. You can specify a list of one more net types. The valid values for net types are *clock*, *pg*, and *none*. These net types are defined as follows:

- *clock* is for clock nets
- *pg* is for power/ground nets
- *none* is for none of the nets

By default, both clock and pg nets are avoided for iso via fixing.

Examples

The following example specifies clock nets to be avoided for isolated via fixing.

```
prompt> set_app_options -name signoff.fix_isolated_via.avoid_net_types  
          -value {clock}  
prompt> signoff_fix_isolated_via
```

The following example specifies both clock and pg nets to be avoided for isolated via fixing.

```
prompt> set_app_options -name signoff.fix_isolated_via.avoid_net_types  
          -value {clock pg}  
prompt> signoff_fix_isolated_via
```

The following example specifies both clock and pg nets to be used for isolated via fixing:

```
prompt> set_app_options -name signoff.fix_isolated_via.avoid_net_types  
          -value {none}  
prompt> signoff_fix_isolated_via
```


See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)
- [signoff_fix_isolated_via](#)

signoff.fix_isolated_via.isolated_via_max_range

Specifies the isolated via range for each given via layer. Option for the *signoff_fix_isolated_via* command.

Data Types

list

Default null

Description

This option specifies the isolated via range for each given via layer. The list format is:

```
{{layer_name1 value1}  
 {layer_name2 value2}  
 {layer_name3 value3} ...}
```

You must specify the layer name by using the layer mask name from the technology file.

Examples

The following example specifies an isolated via range of *3.0* for *layer_name1*.

```
prompt> set_app_options \  
-name signoff.fix_isolated_via.isolated_via_max_range \  
-value {{layer_name1 3.0}}  
prompt> signoff_fix_isolated_via
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)

- [set_app_options](#)
- [signoff_fix_isolated_via](#)

signoff_fix_isolated_via.run_dir

Specifies the path of the working directory. Option for the *signoff_fix_isolated_via* command.

Data Types

string

Default signoff_fix_isolated_via_run

Description

If you do not specify this option, the tool uses a directory called *signoff_fix_isolated_via_run* under the current working directory. The path can be either relative or absolute.

Examples

The following example runs IC Validator under the working directory named *my_signoff_run*.

```
prompt> set_app_options -name signoff_fix_isolated_via.run_dir -value  
"my_signoff_run"  
prompt> signoff_fix_isolated_via
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)
- [signoff_fix_isolated_via](#)

signoff_fix_isolated_via.user_defined_options

Specifies the string to be passed as the command line options to the IC Validator tool.

Data Types

string

Default null

Description

This application option specifies additional command line options for the IC Validator tool. Note that the IC Compiler II tool does not perform any validation on the specified string.

By default, no user-defined options are included.

Examples

The following example specifies the user-defined option named "-turbo".

```
prompt> set_app_options -name  
        signoff_fix_isolated_via.user_defined_options -value "-turbo"  
prompt> signoff_fix_isolated_via
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)
- [signoff_fix_isolated_via](#)

signoff.physical.layer_map_file

Specifies the layer mapping file for the *signoff_check_drc* or *signoff_create_metal_fill* command. This file is to map IC Compiler II layers to runset (GDSII) stream layers.

Data Types

string

Default null

Description

This application option specifies the layer mapping file for the *signoff_check_drc* or *signoff_create_metal_fill* command. This file maps IC Compiler II layers to runset (GDSII) stream layers. The format of the file is same as the layer map file used by *read_gds* and *write_gds* commands.

For more information about the layer mapping file format, see the *Library Data Preparation for IC Compiler II User Guide*.

By default, no mapping file is required.

Examples

The following example specifies the layer mapping file *layer.map* for the *signoff_check_drc* or *signoff_create_metal_fill* command.

```
prompt> set_app_options -list {signoff.physical.layer_map_file  
    "layer.map"}
```

The following example shows a layer mapping file that maps IC Compiler II all the purpose on IC Compiler II layer 15 to GDSII data type 70 on GDSII layer 35.

```
15 35:70
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)
- [read_gds](#)
- [write_gds](#)
- [signoff_check_drc](#)
- [signoff_create_metal_fill](#)

signoff.physical.merge_exclude_libraries

Specifies the list of new data model (NDM) reference libraries that are excluded from default replacement from merge stream files for the *signoff_check_drc* or *signoff_create_metal_fill* command.

Data Types

list

Default null

Description

This application option specifies list of new data model reference libraries that are excluded from default replacement from merge stream files for the *signoff_check_drc* or *signoff_create_metal_fill* command. Cells in the stream files with the same name as in the current design does not replace the cells in the reference library, if reference library is set in merge exclude libraries. The reference library names can be relative or absolute path names of the current run.

Examples

The following example specifies the list of merge exclude libraries for the *signoff_check_drc* command.

```
prompt> set_app_options \\  
      -name signoff.physical.merge_exclude_libraries \\  
      -value {ref_lib_file1.ndm ref_lib_file2.ndm}  
prompt> signoff_check_drc
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)
- [signoff_check_drc](#)
- [signoff_create_metal_fill](#)

signoff.physical.merge_layer_map_file

Specifies the layer mapping file for *signoff_check_drc* or *signoff_create_metal_fill* command. This file is to map NDM layers to runset (GDSII) stream layers.

Data Types

string

Default null

Description

This application option specifies the layer mapping file for the *signoff_check_drc* or *signoff_create_metal_fill* command. This file maps NDM layers to runset (GDSII) stream layers. The format of the file is same as the layer map file used by *read_gds* and *write_gds* commands.

For more information about the layer mapping file format, see the *Library Data Preparation for IC Compiler II User Guide*.

By default, no mapping file is required.

Examples

The following example specifies the layer mapping file *layer.map* for the *signoff_check_drc* or *signoff_create_metal_fill* command.

```
prompt> set_app_options -list {signoff.physical.merge_layer_map_file  
"layer.map"}
```

See Also

- `get_app_option_value`
- `get_app_options`
- `report_app_options`
- `set_app_options`
- `read_gds`
- `write_gds`
- `signoff_check_drc`
- `signoff create metal fill`

signoff.physical.merge_stream_files

Specifies the list of stream (GDS or OASIS) files that are merged into the current design for the *signoff check drc* or *signoff create metal fill* command.

Data Types

list

Default null

Description

This application option specifies list of stream (GDS or OASIS) files that are merged into the current design for the *signoff_check_drc* or *signoff_create_metal_fill* commands. Cells in the stream files with the same name as in the current design replace the cells in the reference library. The file names can be relative or absolute path names of the current run. The results of the *signoff_check_drc* or *signoff_create_metal_fill* commands are expected to be the same as if run directly on a stream version of the entire design.

By default, no mapping file is required.

Examples

The following example specifies the list of stream files for the *signoff_check_drc* command.

```
prompt> set_app_options \\  
-name signoff.physical.merge stream files \\  
-
```

s

```

    -value {stream_file1.gds stream_file2.oas}
prompt> signoff_check_drc

```

The following example specifies the list of stream files for the *signoff_create_metal_fill* command from a file.

```

prompt> set file_handle [open stream_files.list r]
prompt> set stream_files [read $file_handle]
prompt> set_app_option -name signoff.physical.merge_stream_files \
    -value $stream_files
prompt> signoff_create_metal_fill

```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)
- [signoff_check_drc](#)
- [signoff_create_metal_fill](#)

signoff.physical.rename_cell_files

Specifies a file or a list of files that contains the mapping from original cell names to new cell names for the *signoff_check_drc* or *signoff_create_metal_fill* command.

Data Types

list

Default null

Description

This application option specifies a file or a list of files that contains the mapping from original cell names to new cell names for the *signoff_check_drc* or *signoff_create_metal_fill* command. The renaming works for both reference library cells and design blocks. These files are usually provided by the foundry or library cell provider.

The list of files should be the same as used in the *write_oasis -rename_cell* option. This option should be used with *signoff.physical.merge_stream_files*.

Examples

The following example specifies a file to rename cells before running *signoff_check_drc*

```
prompt> set_app_options \  
-name signoff.physical.rename_cell_files \  
-value {rename_file1.map}  
prompt> signoff_check_drc
```

See Also

- [signoff_create_metal_fill](#)
- [signoff_check_drc](#)
- [write_oasis](#)
- [write_gds](#)

signoff.report_metal_density.create_heat_maps

Option for the *signoff_report_metal_density* command. Enables the creation of heat maps for metal density in windows across the chip.

Data Types

bool

Default false

Description

When set to true, this option enables the creation of metal density heat maps in windows across the chip. ICC2 GUI overlays the heat map onto the current block and colors are assigned to each window based on the metal density. The metal density map can be overlaid using the main menu. (View -> Map -> ICV_Heatmap).

Examples

The following example populates the heat map data in the current block and can be overlaid using the view menu:

```
prompt> set_app_options -name  
signoff.report_metal_density.create_heat_maps -value "true"  
prompt> signoff_report_metal_density
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)

- [set_app_options](#)
- [signoff_report_metal_density](#)

signoff.report_metal_density.density_window

Specifies the size of density window. Option for the *signoff_report_metal_density* command.

Data Types

string

Default Value from the technology file

Description

This option sets the window size for the calculation of density values. The default for window size is taken from technology file.

The list format for specifying explicit window size values is as follows: {{layer_name1 value1} {layer_name2 value2} ... } Example: {{M2 10} {M3 20}} You must specify the layer name by using the layer name from technology file.

Examples

The following example sets the window size of 10 and 20 for layers M2 and M3.

```
prompt> set_app_options -name signoff.report_metal_density.density_window  
-value {{M2 10} {M3 20}}  
prompt> signoff_report_metal_density
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)
- [signoff_report_metal_density](#)

signoff.report_metal_density.density_window_step

Specifies the step size of the density window for the *signoff_report_metal_density* command.

Data Types

string

Default Half of the density window size.

Description

This option sets the window step size for the calculation of metal density. The list format for specifying explicit window step size values is as follows:

```
{{layer_name1 value1} {layer_name2 value2} ... }
```

Example:

```
{{M2 10} {M3 20}}
```

You must specify the layer name by using the layer name from technology file.

Examples

The following example sets the window step size of 10 and 20 for layers M2 and M3.

```
prompt> set_app_options \\  
        -name signoff.report_metal_density.density_window_step \\  
        -value {{M2 10} {M3 20}}  
prompt> signoff_report_metal_density
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)
- [signoff_report_metal_density](#)

signoff.report_metal_density.fill_view_data

Defines the behavior for reading of FILL data during signoff_report_metal_density command. Option for the *signoff_report_metal_density* command.

Data Types

string

Default read

Description

There are three following possible modes for reading of data:

"read_if_uptodate": The fill view data is read-only, when its timestamp is newer than the timestamp of the design view.

"read": Reads the FILL view data even though its timestamp is older than the timestamp of the design view.

"discard": Never reads fill view data.

Examples

The following example never reads the fill data:

```
prompt> set_app_options -name signoff.report_metal_density.fill_view_data  
-value discard  
prompt> signoff_report_metal_density
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)
- [signoff_report_metal_density](#)

signoff.report_metal_density.gradient_window_size

Specifies the size of the density gradient window for the *signoff_report_metal_density* command.

Data Types

string

Default Value from the technology file.

Description

This option sets the window size for the calculation of density gradient values. The list format for specifying explicit window size values is as follows:

```
{{layer_name1 value1} {layer_name2 value2} ... }
```

Example:

```
{{M2 10} {M3 20}}
```

You must specify the layer name by using the layer name from the technology file.

Examples

The following example sets the window size of 100 and 200 for layers M2 and M3.

```
prompt> set_app_options \\  
-name signoff.report_metal_density.gradient_window_size \\  
-value {{M2 100} {M3 200}}  
prompt> signoff_report_metal_density
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)
- [signoff_report_metal_density](#)

signoff.report_metal_density.min_density

Specifies the minimum required density for each metal layer. Option for the *signoff_report_metal_density* command.

Data Types

string

Default Value from the technology file

Description

This option sets the minimum required density for each metal layer. This value overrides the values mentioned in the technology file. This value is used for highlighting windows that fall short of required density. An asterisk(*) sign is marked above the violating windows.

The list format for specifying explicit minimum required density values is as follows: {{layer_name1 value1} {layer_name2 value2} ... } Example: {{M2 10} {M3 20}} You must specify the layer name by using the layer name from technology file.

Examples

The following example sets the minimum required density values of 10 and 20 for layers M2 and M3.

```
prompt> set_app_options -name signoff.report_metal_density.min_density  
-value {{M2 10} {M3 20}}  
prompt> signoff_report_metal_density
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)
- [signoff_report_metal_density](#)

signoff.report_metal_density.run_dir

Specifies the path of the working directory. Option for the *signoff_report_metal_density* command.

Data Types

string

Default signoff_report_metal_density_run

Description

If you do not specify this option, the tool uses a directory called *signoff_report_metal_density_run* under the current working directory. The path can be either relative or absolute.

Examples

The following example runs IC Validator under the working directory named *my_signoff_run*.

```
prompt> set_app_options -name signoff.report_metal_density.run_dir -value  
"my_signoff_run"  
prompt> signoff_report_metal_density
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)
- [signoff_report_metal_density](#)

signoff.report_metal_density.user_defined_options

Specifies the string to be passed as the command line options to IC Validator. Option for the *signoff_report_metal_density* command.

Data Types

string

Default null

Description

This application option specifies additional command line options for IC Validator. Note that IC Compiler II does not perform any validation on the specified string.

By default, no user-defined options are included.

Examples

The following example specifies the user-defined option named *-turbo*.

```
prompt> set_app_options -name  
        signoff.report_metal_density.user_defined_options -value "-turbo"  
prompt> signoff_report_metal_density
```

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [report_app_options](#)
- [set_app_options](#)
- [signoff_report_metal_density](#)

sim.fine_sim_path

Specifies the path to the the FineSim executable.

Data Types

string

Default <empty>

Description

Specifies the path to the FineSim executable.

Syntax

```
{ path }
```

path

Must exist.

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

sim.hspice_path

Specifies the path to the the HSPICE executable.

Data Types

string

Default <empty>

Description

Specifies the path to the HSPICE executable.

Syntax

```
{ path }
```

path

Must exist.

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

sim.wave_view_path

Specifies the path to the the WaveView executable.

Data Types

string

Default <empty>

Description

Specifies the path to the WaveView executable.

Syntax

```
{ path }
```

path

Must exist.

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

stream.merge_multi_thread

Enables the *merge_stream*, *write_gds* and *write_oasis* commands to control whether the multi-thread mechanism is enabled when merging GDSII and OASIS layout files.

Data Types

Boolean

Default false

Description

This application option is used to enable the multi-thread mechanism to improve the runtime when merging GDSII and OASIS layout files.

When the option is false, which is also the default, the multi-thread mechanism is disabled even if the value of the `max_cores` set by the command `set_host_options` is more than 1.

When the option is true, the multi-thread mechanism is enabled if the licence check for "ICWBEV_PLUS" or "ICValidator-Workbench" is successful, and if the value of the `max_cores` is more than 1. The `merge_stream`, `write_gds` and `write_oasis` commands result in errors if the licence check fails.

Examples

The following example sets this application option to true.

```
prompt> set_app_options -name stream.merge.multi_thread \\  
-value true
```

See Also

- [write_gds](#)
- [write_oasis](#)
- [merge_stream](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

synopsys_program_name

Indicates the name of the program currently running.

Data Types

string

Description

This variable is read only, and is set by the application to indicate the name of the program you are running. This is useful when writing scripts that are mostly common between some applications, but contain some differences based on the application.

To determine the current value of this variable, use `get_app_var synopsys_program_name`.

t

See Also

- [get_app_var](#)

synopsys_root

Indicates the root directory from which the application was run.

Data Types

string

Description

This variable is read only, and is set by the application to indicate the root directory from which the application was run.

To determine the current value of this variable, use *get_app_var synopsys_root*.

See Also

- [get_app_var](#)

t

tbcs.record.enable**Data Types**

boolean

Default false

Description

This application option enables the recording of *source* of Tbc files. With this application option enabled, you can use the *report_tbcs* command to display all the Tbc files.

Tbc file or Tbc script file is an ascii script file compressed with *synenc* utility, or a Tcl bytecode script created by TclPro Compiler. Tbc's are either created by the user or provided by Synopsys representatives. These are read and evaluated by this application using the *source* command. Tbc is short for Tcl Bytecode.

When *true*, *source* of Tbc file will be recorded by the application.

When *false*, the recording will be suspended

See Also

- [get_app_option_value](#)
- [get_app_options](#)
- [set_app_options](#)
- [report_tbc](#)
- [report_app_options](#)

techMap.mapCklnsts

Enables mapping of combinational cells on the clock path to CTS purpose cells only.

Data Types

boolean

Default `false`

Description

This app option controls the mapping of clock path combinational cells to CTS purpose cells. When turned on tool will map combinational cells to CTS prupose cells only. By default tool maps such cells to OPTO purpose cells.

The user needs to set the *CTS purpose* on the following logic:

- *AND2/NAND2* equivalent cells
- *Inverter*
- *Buffer*

Some examples that attend these requirements include:

- *AND2 + NAND2 + BUF + INV*
- *AND2 + OR2 + BUF + INV*
- *OR2 + NOR2 + BUF + INV*
- *NAND2 + NOR2 + BUF + INV*

The provided cells must also attend the following constraints:

- *smallest drive strength* available as CTS purpose
- only *single rail* cells\
- *dont_touch/dont_use* set as *false*

t

Both requirements (CTS basic cells and constraints) need to be met. Otherwise, `initial_map` will *error* out.

thermal.adaptive_grid_resolution

Specifies the adaptive grid resolution for the thermal engine.

Data Types

`pair<double, double>`

Default `<empty>`

Description

Specifies the adaptive grid resolution for the thermal engine. When the adaptive grid resolution is set, the thermal analysis will perform a higher resolution analysis on the hotspot. The first value define the width of the thermal grid, and the second one define the height of the thermal grid.

unit: um

Examples

The following example shows the usage.

```
prompt> set_app_options -name thermal.adaptive_grid_resolution -value {10  
20}
```

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)
- [analyze_thermal](#)

thermal.boundary_temperature

Specifies temperature to the design boundary.

Data Types

`string`

Default `<empty>`

Description

Specifies temperature to the design boundary. Use six values to define temperature to the top/bottom/left/right/front/back side boundaries.

Examples

The following example shows the usage.

```
prompt> set_app_options -name thermal.boundary_temperature -value {30 30  
40 40 50 50}
```

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

thermal.carrier_order

Specifies the order of carriers.

Data Types

string

Default <empty>

Description

Specifies the order of carriers. The order is bottom to top

Examples

The following example shows the usage.

```
prompt> set_app_options -name thermal.carrier_order -value { carrier  
carrier2 }
```

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)
- [analyze_thermal](#)

thermal.common.database

Specifies where to save the thermal analysis results.

Data Types

string

Default ./THERMAL_DATABASE

Description

Specifies where to save the thermal analysis results. Thermal Database result subfolder will match corresponding RedHawk-SC Electrothermal project/folder.

Examples

The following example shows how to define the location of the thermal database.

```
prompt> set_app_options -name thermal.common.database -value TDB_STATIC
```

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

thermal.common.temperature_unit

Specifies user temperature unit.

Data Types

enumerated

Default C

Description

Specifies user temperature unit. The temperature unit can be set either C/Celsius or K/Kelvin.

Examples

The following example shows how to define the user temperature unit.

```
prompt> set_app_options -name thermal.common.temperature_unit -value C
prompt> set_app_options -name thermal.common.temperature_unit -value
Celsius
```

t

```
prompt> set_app_options -name thermal.common.temperature_unit -value K
prompt> set_app_options -name thermal.common.temperature_unit -value
Kelvin
```

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

thermal.compact_model_file

Specifies the compact model input file to the thermal engine.

Data Types

string

Default <empty>

Description

Specifies the compact model input file to the thermal engine.

Examples

The following example shows the usage.

```
prompt> set_app_options -name thermal.compact_model_file -value FILE
```

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

thermal.component_dimension

Specifies component dimension.

Data Types

string

Default <empty>

Description

Specifies component dimension. The format is {COMPONENT width height}

unit: um

Examples

The following example shows the usage.

```
prompt> set_app_options -name thermal.component_dimension -value  
{ {heat_sink 30000 20000} {heat_sink_base 5000 6000} }
```

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)
- [analyze_thermal](#)

thermal.component_material

Specifies component material.

Data Types

string

Default <empty>

Description

Specifies component material.

Examples

The following example shows the usage.

```
prompt> set_app_options -name thermal.component_material -value  
{ {heat_sink Copper} {heat_sink_base Copper} }
```

See Also

- [get_app_option_value](#)
- [report_app_options](#)

- [set_app_options](#)
- [analyze_thermal](#)

thermal.component_thickness

Specifies component thickness.

Data Types

string

Default <empty>

Description

Specifies component thickness.

unit: um

Examples

The following example shows the usage.

```
prompt> set_app_options -name thermal.component_thickness -value  
{ {heat_sink 20} {heat_sink_base 50} }
```

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)
- [analyze_thermal](#)

thermal.database

Specifies where to save the thermal analysis results.

Data Types

string

Default ./THERMAL_DATABASE

Description

Specifies where to save the thermal analysis results.

Examples

The following example shows how to define the location of the thermal database.

```
prompt> set_app_options -name thermal.database -value THERMAL_DATABASE
```

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

thermal.engine

Specifies the thermal engine.

Data Types

string

Default helios

Description

Specifies the thermal engine "Helios" or "Compact".

Examples

The following example shows the usage.

```
prompt> set_app_options -name thermal.engine -value helios
```

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

thermal.grid_resolution

Specifies the grid resolution for the thermal engine.

Data Types

pair<double, double>

Default <empty>

Description

Specifies the grid resolution for the thermal engine. The first value define the width of the thermal grid, and the second one define the height of the thermal grid.

unit: um

Examples

The following example shows the usage.

```
prompt> set_app_options -name thermal.grid_resolution -value {10 20}
```

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

thermal.heat_sink_order

Specifies the order of heat sink.

Data Types

string

Default <empty>

Description

Specifies the order of heat sink. The order is bottom to top

Examples

The following example shows the usage.

```
prompt> set_app_options -name thermal.heat_sink_order -value { timmm  
heat_sink_base heat_sink }
```

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)
- [analyze_thermal](#)

t

thermal.heat_transfer_rate

Specifies the heat transfer rate to the design boundary.

Data Types

string

Default <empty>

Description

Specifies the heat transfer rate to the design boundary. Use two values to define the top and bottom side boundaries or use six values to define the top/bottom/left/right/front/back side boundaries.

Examples

The following example shows the usage.

```
prompt> set_app_options -name thermal.heat_transfer_rate -value {10000 0}
```

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

thermal.hier_block_metal_profile

Specifies the metal density to the macro cell which belong to the same lib cell.

Data Types

string

Default <empty>

Description

Specifies the metal density to the macro cell which belong to the same lib cell.

The metal density can be provide by a float number or a metal profile path

If none specified, all macros metal density are set to zero.

Syntax

```
{ LIB_CELL1 0.3 LIB_CELL2 METAL_PROFILE_PATH ... }
```

LIB_CELL1

LIB_CELL1 lib cell name

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

thermal.hier_block_power_profile

Specifies the power values to the macro cell which belong to the same lib cell.

Data Types

string

Default <empty>

Description

Specifies the power values to the macro cell which belong to the same lib cell.

The power value can be provide by a float number (unit: Watt) or a power profile path

If none specified, all macros power are set to zero.

Syntax

```
{ LIB_CELL1 0.3 LIB_CELL2 POWER_PROFILE_PATH ... }
```

LIB_CELL1

LIB_CELL1 lib cell name

thermal.material_conductivity

Specifies conductivity of materials.

Data Types

string

Default <empty>

t

Description

Specifies conductivity of materials. Conductivity has three directions: x, y, and z. If only one value is set, then x, y, and z will all be set to the same value.

Examples

The following example shows the usage.

```
prompt> set_app_options -name thermal.material_conductivity -value  
{ {material_1 1} {material_2 63 35 6} }
```

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)
- [analyze_thermal](#)

thermal.material_density

Specifies density of material.

Data Types

string

Default <empty>

Description

Specifies density of materials.

Examples

The following example shows the usage.

```
prompt> set_app_options -name thermal.material_density -value  
{ {material_1 1} {material_2 63} }
```

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)
- [analyze_thermal](#)

thermal.material_heat_capacity

Specifies heat capacity of material.

Data Types

string

Default <empty>

Description

Specifies heat capacity of material.

Examples

The following example shows the usage.

```
prompt> set_app_options -name thermal.material_heat_capacity -value  
{ {material_1 31} {material_2 68} }
```

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)
- [analyze_thermal](#)

thermal.metal_density

Specifies metal density to specific layer.

Data Types

string

Default <empty>

Description

Specifies metal density to specific layer.

If none specified, the density will follow metal profile.

Examples

The following example shows the usage.

t

```
prompt> set_app_options -name thermal.metal_density -value {M1 0.4 VIA2 0.8}
```

```
prompt> set_app_options -name thermal.metal_density -value {M1 0.4 all_metal 0.9 all_via 0.7}
```

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

thermal.metal_profile

Specifies the metal profile.

Data Types

string

Default <empty>

Description

Specifies the metal profile.

Examples

The following example shows the usage.

```
prompt> set_app_options -name thermal.metal_profile -value METAL_PROFILE
```

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

thermal.percentage_of_max_threshold

Specifies the threshold with the percentage of max temperature.

Data Types

float

Default 90

Description

Specifies the threshold with the percentage of max temperature to filter out the hotspot grids.

Examples

The following example shows the usage.

```
prompt> set_app_options -name thermal.percentage_of_max_threshold -value 50
```

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

thermal.power_layer

Specifies the layer where generating power.

Data Types

string

Default The lowest level of routing layer

Description

Specifies the layer where generating power.

Examples

The following example shows the usage.

```
prompt> set_app_options -name thermal.power_layer -value M1
```

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

thermal.power_profile

Specifies the power profile.

Data Types

string

Default <empty>

Description

Specifies the power profile.

Examples

The following example shows the usage.

```
prompt> set_app_options -name thermal.power_profile -value POWER_PROFILE
```

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

thermal.tech_file

Specifies the tech file.

Data Types

string

Default <empty>

Description

Specifies the tech file.

The tech file define the layer order, layer material and layer thickness.

Examples

The following example shows the usage.

```
prompt> set_app_options -name thermal.tech_file -value TECH_FILE
```

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

thermal.threshold

Specifies the threshold.

Data Types

float

Default 25

Description

Specifies the threshold to filter out the hotspot grids.

Examples

The following example shows the usage.

```
prompt> set_app_options -name thermal.threshold -value 50
```

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

thermal.use_layout

Determine to use layout to calculate via metal density.

Data Types

bool

Default true

Description

Determine to use layout to calculate via metal density.

Examples

The following example shows the usage.

```
prompt> set_app_options -name thermal.use_layout -value false
```

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

time.advanced_multi_input_switching_tied_input_mode

Specify the tied input mode for advanced MIS analysis

Data Types

string

Default none

Description

This variable controls the advanced MIS mode for tied input mode.

- *none* (default) - Perform advanced MIS analysis on all cells.
- *any_inputs* - Only perform advanced MIS analysis on cells with at least two input pins tied.
- *all_inputs* - Only perform advanced MIS analysis on cells with all input pins tied.

See Also

- [set_multi_input_switching_coefficient](#)

time.all_clocks_propagated

Specifies whether or not all subsequent clocks are created as propagated clocks.

Data Types

Boolean

Default false

t

Description

When true, all clocks subsequently created by *create_clock* or *create_generated_clock* are created as propagated clocks. When false (the default), clocks are created as nonpropagated clocks.

By default, *create_clock* and *create_generated_clock* create only nonpropagated clocks. You can subsequently define some or all clocks to be propagated clocks using the *set_propagated_clock* command. However, if you set this option to true, *create_clock* and *create_generated_clock* subsequently create only propagated clocks. Setting this option to true or false affects only clocks created after the setting is changed. Clocks created before the setting is changed retain their original condition (propagated or non-propagated).

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design. To determine the current value of this application option on a specific design, use *get_app_option -design \$design -name opt_name*. Use *report_app_options* to show the value of multiple application options.

See Also

- [create_clock](#)
- [create_generated_clock](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

time.allow_port_latency_for_feedthrough

Allow port latency to be computed on ports that have feedthrough connection.

Data Types

Boolean

Default false

Description

This application option controls the computation of port latency value on the ports that have feedthrough connection. By default, for the ports that have feedthrough connection, the port latency values for the top-level usage do not get computed and stored with the block-level design during *compute_clock_latency* command.

By setting this app-option to true, the port latency values will be computed for ports with feedthrough connection too. However, note that promotion of such port latencies values in

t

top-level shall cause balance point constraints to be created on the block hierarchical pins that can cause the sinks in the fanout of the feedthrough connection to be unaccounted for.

Application options are set using the `set_app_options` command. This app-option has to be set before `compute_clock_latency` command in block-level flow.

Use `get_app_option_value -design $design -name time.allow_port_latency_for_feedthrough` to get the current value of this application option on a specific design.

See Also

- [get_app_option_value](#)
- [set_app_options](#)
- [promote_clock_data](#)

time.ambiguous_checks_sequential_infer_pt_compatibility

Enable inferring of latches to flip flops to be compatible with PT.

Data Types

Boolean

Default false

Description

When set to *true*, positive level latches with rise edge triggered setup check and falling level latches with fall edge triggered setup check will be inferred as flip flops like PT.

When set to *false*, default behaviour of inferring it as level sensitive latch will continue.

Application options are set using the `set_app_options` command, and they may be specified with respect to a design. To determine the current value of this application option, use:

```
get_app_option_value -user_default -name
time.ambiguous_checks_sequential_infer_pt_compatibility
Use report_app_options to show the value of multiple application options.
```

See Also

- [get_app_option_value](#)
- [report_app_options](#)

- [set_app_options](#)
- [report_timing](#)
- [get_attribute](#)

time.analyze_subcircuit_use_pulse_waveform

Write out PULSE format waveform in analyze_subcircuit

Data Types

Boolean

Default false

Description

This application option controls which waveform format is used in the output spice deck file of analyze_subcircuit command. By default, PWL format is used in describing the waveform for start point clock pin in the spice deck file generated by analyze_subcircuit command.

If this application option is turned on, the waveform will be illustrated by PULSE format.

For the same subcircuit simulation, waveform format shall not impact the simulation result. Details of waveform format, please refer to user guide of spice simulator, e.g. HSPICE or PrimeSim.

Use the *get_app_option_value -name time.analyze_subcircuit_use_pulse_waveform* command to get the current value of this application option.

Use the *get_app_option_value -name time.analyze_subcircuit_use_pulse_waveform* command to retrieve the current value.

Use the *set_app_options -as_user_default {time.analyze_subcircuit_use_pulse_waveform true|false}* command to set a new value.

Use the *report_app_options* command to show the value of multiple application options.

See Also

- [get_app_option_value](#)
- [set_app_options](#)

t

time.analyze_subcircuit_use_real_pg_supply_name

Write out real power supply net name for cell instances in spice deck file generated by `analyze_subcircuit` command.

Data Types

Boolean

Default `false`

Description

This application option controls whether real power supply net name is used for cell instances in spice deck file generated by `analyze_subcircuit` command.

By default, `analyze_subcircuit` uses `VDD`, `VDD1`, `VDD2`... as power supply net name for cell instances in the generated spice deck file. This was a simplified solution to keep the voltage supply name unique in the spice deck file.

If this application option is turned on, real power supply net name will be used in the spice deck file. e.g. `VDD_TOP`, `VDD_L`, `VDD_H`, etc.

For subcircuit simulation, as long as the voltage supply value is correct, the name does not impact the simulation result. While real voltage name is good to track the real information of cell power supply, and can be easier for designs to check the correctness of spice deck file without finding out the name and value mapping.

Use the `get_app_option_value -name time.analyze_subcircuit_use_real_pg_supply_name` command to get the current value of this application option.

Use the `get_app_option_value -name time.analyze_subcircuit_use_real_pg_supply_name` command to retrieve the current value.

Use the `set_app_options -as_user_default {time.analyze_subcircuit_use_real_pg_supply_name true|false}` command to set a new value.

Use the `report_app_options` command to show the value of multiple application options.

See Also

- [get_app_option_value](#)
- [set_app_options](#)

t

time.analyze_subcircuit_use_specified_voltage

Data Types

Boolean

Default false

Description

By default, the spice deck file written out by analyze subcircuit uses effective voltage in cell supply voltage, for simulation.

when the app-option is set to true, will use specified voltage instead.

To know the value of a cell's effective voltage or specified voltage, use report_pvt, e.g.
report_pvt -object [get_cells U1]

See Also

- [analyze_subcircuit](#)
- [report_pvt](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

time.aocvm_analysis_mode

Configures advanced on-chip variation (AOCV) analysis.

Data Types

string

Default separate_launch_capture_depth

Description

This option sets the configuration for advanced on-chip variation (AOCV) analysis. To perform AOCV analysis, you must set the *time.aocvm_enable_analysis* application option to *true*.

In AOCV analysis, the derating factor for a path depends on the path depth, which is defined as the number of successive cell (or net) delay timing arcs in a path from the common point to the endpoint. The depth ranges and corresponding derating factors are

t

specified in a table, which is read into the tool with the *read_ocvm* command. Separate depth values are calculated for cells and nets, and for launch and capture paths.

When this option is set to *separate_launch_capture_dept*, both clock and data networks objects are included in the depth computation.

When this option is set to *combined_launch_capture_depth*, launch and capture depths are not calculated separately. The launch and capture paths are considered together and a combined depth is calculated for the entire path.

When this option is set to *separate_data_and_clock_metrics*, separate depths are computed for the clock and data paths.

See Also

- [read_ocvm](#)
- [remove_ocvm](#)
- [report_ocvm](#)
- [report_timing](#)

time.aocvm_enable_analysis

Enable the advanced on-chip variation (advanced OCV) analysis.

Data Types

Boolean

Default false

Description

When you set this option to true, the advanced OCV timing update is performed as part of the *update_timing* command or any other functionality needing a timing update.

When this option is set to its default value, the advanced OCV timing update is not performed.

See Also

- [read_ocvm](#)
- [remove_ocvm](#)
- [report_ocvm](#)
- [report_timing](#)

t

time.aocvm_enable_clock_network_only

Enables advanced on-chip variation (AOCV) analysis only for the clock network.

Data Types

Boolean

Default false

Description

When this option is set to *true*, advanced OCV derating is applied to arc delays in the clock network only. Constant (OCV) derating is applied to data paths, if constant derating factors have been annotated for data objects; otherwise they are not derated. Depth is calculated based on clock network topology only; delay timing arcs in the data network are excluded from depth calculations. This option is only effective when advanced on-chip variation analysis has been enabled by setting the *time.ocvm_enable_analysis* option to aocvm.

See Also

- [read_ocvm](#)
- [remove_ocvm](#)
- [report_ocvm](#)
- [report_timing](#)

time.aocvm_enable_single_path_metrics

Configures advanced on-chip variation (AOCV) analysis.

Data Types

Boolean

Default false

Description

This application option determines if advanced OCV analysis accounts for depth and distance metrics for cell arcs and net arcs separately. By default, separate accounts are kept and when looking up cell derates the cell metrics are used and when looking net derates the net metrics are used.

When this this application option is set to true, only cell arc metrics are computed and used to look up both cell and net derates.

t

To restrict advanced OCV analysis to use only cell arc metrics, enter the following command:

```
prompt> set_app_options -name time.aocvm_enable_single_path_metrics \\  
-value true
```

To determine the current value of this application option, enter the following command:

```
prompt> report_app_options time.aocvm_enable_single_path_metrics
```

See Also

- [read_ocvm](#)
- [remove_ocvm](#)
- [report_ocvm](#)
- [report_timing](#)

time.awp_compatibility_mode

Enables the advanced waveform propagation (AWP) analysis engine.

Data Types

Boolean

Default false

Description

This application option previously controlled the selection between two Advanced Waveform Propagation (AWP) engines during timing update.

As of the P-2019.03-SP2 release, the delay calculation AWP engine is aligned with the PrimeTime AWP engine and this option has no effect. This engine requires cell libraries built using version M-2016.12-SP2 or later.

This option will become obsolete in a future release.

See Also

- [get_app_option_value](#)
- [set_app_options](#)

time.case_analysis_propagate_through_icg

Determines whether case analysis is propagated through integrated clock gating cells.

t

Data Types

Boolean

Default false

Description

When this application option is set to false (the default), constants propagating throughout the design will stop propagating when an integrated clock gating cell is encountered. Regardless of whether the integrated clock gating cell is enabled or disabled, no logic values will propagate in the fanout of the cell.

When the value is true, constants propagated throughout the design will propagate through an integrated clock gating cell, provided the cell is enabled. An integrated clock gating cell is enabled when its enable pin (or test enable pin) is set to a high logic value. If the cell is disabled, then the disable logic value for the cell is propagated in its fanout. For example, for a latch_posedge ICG, when it is disabled, it will propagate a logic 0 in its fanout.

Application options are set using the `set_app_options` command, and they may be specified globally or with respect to a design. To determine the current value of this application option on a specific design, use `get_app_option_value -design $design -name time.case_analysis_propagate_through_icg`. Use `report_app_options` to show the value of multiple application options.

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)
- [remove_case_analysis](#)
- [set_case_analysis](#)

time.case_analysis_sequential_propagation

Specifies whether case analysis is propagated across sequential cells.

Data Types

string

Default never

t

Description

Determines whether case analysis is propagated across sequential cells. Allowed values are *never* (the default) or *always*. When set to *never*, case analysis is not propagated across the sequential cells. When set to *always*, case analysis is propagated across the sequential cells.

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design. To determine the current value of this application option on a specific design, use *get_app_option_value -design \$design -name time.case_analysis_sequential_propagation*. Use *report_app_options* to show the value of multiple application options.

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)
- [set_case_analysis](#)

time.clock_gating_propagate_enable

Allows the gating enable signal delay to propagate through the gating cell.

Data Types

Boolean

Default *true*

Description

When set to *true* (the default), the tool allows the delay and slew from the data line of the gating check to propagate. When set to *false*, the tool blocks the delay and slew from the data line of the gating check from propagating. Only the delay and slew from the clock line is propagated.

If the output goes to a clock pin of a latch, setting this option to *false* produces the most desirable behavior.

If the output goes to a data pin, setting this option to *true* produces the most desirable behavior.

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design. To determine the current value of this application option on a specific design, use *get_app_option_value -design \$design -name*

t

time.clock_gating_propagate_enable. Use *report_app_options* to show the value of multiple application options.

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

time.clock_gating_user_setting_only

When clock gating checks are not disabled, this application option controls whether auto-inferred clock-gating checks for setup and hold violations are made.

Data Types

Boolean

Default false

Description

When the *time.disable_clock_gating_checks* application option is set to *true*, the value of the *time.clock_gating_user_setting_only* application option has no effect.

When the *time.disable_clock_gating_checks* application option is set to *false* (default), this application option controls which set of clock-gating checks are created.

When the *time.clock_gating_user_setting_only* application option is set to *true*, it disables auto-inferred clock-gating checks on objects that have not also had a *set_clock_gating_check* command run against them.

This application option has no effect on library-specific clock-gating checks.

This application option has no equivalent application variable in *PrimeTime*.

Setting this application option to true may cause miscorrelation between *IC Compiler II* and *PrimeTime*.

Setting this application option to true may allow better correlation between *IC Compiler II* and *IC Compiler* since *IC Compiler* does not automatically infer clock-gating checks based pin commonality along the clock network.

See the Solvnet article 015769 "How Are Clock Gating Checks Inferred?" for further information on how clock-gating checks are inferred.

time.clock_gating_user_setting_only may only be set in the global and block-specific context and is persistent.

t

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design. To determine the current value of this application option on a specific design, use *get_app_option_value -design \$design -name time.use_pin_capacitance_ranges*. Use *report_app_options* to show the value of multiple application options.

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)
- [set_clock_gating_check](#)
- [remove_clock_gating_check](#)
- [set_disable_clock_gating_check](#)
- [report_clock_gating_checks](#)

time.clock_marking

Sets the clock marking view used by timing analysis.

Data Types

enumerated

Default native

Description

The *time.clock_marking* application option controls the value of the *is_clock_used_as_clock* and *is_clock_used_as_data* attributes for specific objects.

By default (*native*), the behavior is consistent with the implementation tool.

When set to *primetime*, the behavior matches the behavior of the PrimeTime tool.

The behavior is different between the implementation tool and the PrimeTime tool for the following objects on the clock tree network:

- An output port without any output delay constraint
 - when specified as native, the *is_clock_used_as_data* attribute value is false
 - when specified as primetime, the *is_clock_used_as_data* attribute value is true

t

- An output port with an output delay constraints but not constrained by either the `set_clock_balance_point` command or the `set_sense -clock_leaf` command
 - when specified as native, the `is_clock_used_as_clock` attribute value is false
 - when specified as primetime, the `is_clock_used_as_clock` attribute values is true
- For all the pins in indirect source latency cone of a generated clock.
 - when specified as native, the `is_clock_used_as_clock` attribute values is true
 - when specified as primetime, the `is_clock_used_as_clock` attribute values is false

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

time.clock_reconvergence_pessimism

Selects signal transition sense matching for computing clock reconvergence pessimism removal.

Data Types

String

Default normal

Description

Determines how the value of the clock reconvergence pessimism removal (CRPR) is computed with respect to transition sense. Allowed values are as follows:

- normal (default value)
- same_transition

When set to *normal*, the CRPR value is computed even if the clock transitions to the source and destination latches are in different directions on the common clock path. It is computed separately for rise and fall transitions and the value with smaller absolute value is used.

When set to *same_transition*, the CRPR value is computed only when the clock transition to the source and destination latches have a common path and the transition is in the same direction on each pin of the common path. If the source and destination latches are

t

triggered by different edge types, CRPR is computed at the last common pin at which the launch and capture edges match.

Application options are set using the `set_app_options` command, and they may be specified globally or with respect to a design. To determine the current value of this application option, use:

```
get_app_option_value -user_default -name
time.clock_reconvergence_pessimism
```

Use **report_app_options** to show the value of multiple application options.

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)
- [report_crpr](#)

time.convert_constraint_from_bc_wc

Specify conversion style to apply min/max values of timing constraints.

Data Types

Enum

Default "none"

Description

This application option can be used to specify how the "-min" and "-max" values of timing constraints are applied. Possible values are "none", "bc_only" and "wc_only". "none" is the default.

Behavior for conversion styles:

- none : do nothing. Apply the constraint as given.
- bc_only : if -max value is given without -min, ignore the value.
Otherwise the value will be used for both max and min.
- wc_only : if -min value is given without -max, ignore the value.
Otherwise value will be used for both max and min.

This application option is global only, so cannot be specified on a design.

See Also

- [set_clock_latency](#)
- [set_clock_transition](#)
- [set_input_transition](#)
- [set_drive](#)
- [set_driving_cell](#)
- [set_load](#)
- [set_input_delay](#)
- [set_output_delay](#)
- [set_timing_derate](#)

time.create_clock_no_input_delay

Data Types

Boolean

Default false

Description

This option is obsoleted in the tool (ICC2/FC). Please remove it from your script.

See Also

- [create_clock](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

time.crpr_different_transition_derate

Specifies the fraction of the common clock reconvergence pessimism (CRP) that is removed if the transitions on the launching and capturing clock paths are different.

Data Types

float

t

Default 1.0**Description**

When computing the common clock pessimism, the path to the common point is assumed to be correlated.

If the "normal" setting of the application option *time.clock_reconvergence_pessimism* is used, a pessimism value is computed for both the rising path and falling path to the common point, and the minimum is removed. This assumes the launch and capture paths, despite different transitions, are fully correlated.

However, the paths are actually only partially correlated. The gates and wires are the same, but the paths through the transistors of the gates may be different.

To reduce the amount of pessimism removed, set this application option to a factor less than 1.0. For example, setting it to 0.90 causes 90 percent of the calculated pessimism to be removed, resulting less reported slack and a more conservative analysis than removing 100 percent of the calculated pessimism.

Note that setting a factor of 0.0 results in no pessimism being removed for different transitions. This is not equivalent to using the "same_transition" setting for the *time.clock_reconvergence_pessimism* application option. In that setting, the tool continues to search for a common point with matching transitions. With the "normal" setting, the topological common point is always selected, irrespective of the whether the transitions match.

In parametric on-chip variation analysis, this application option derates just the nominal value of the common clock pessimism. To derate the variation, use the application option *time.crpr_different_transition_variation_derate*.

See Also

- [get_app_option_value](#)
- [set_app_options](#)
- [report_app_options](#)

time.crpr_different_transition_variation_derate

Specifies the fraction of the variation of the common clock reconvergence pessimism (CRP) that is removed if the transitions on the launching and capturing clock paths are different.

Data Types

float

t

Default 0.0**Description**

Parametric on-chip variation (POCV) analysis internally computes arrival, required, and slack values based on statistical distributions. The common clock pessimism removes both nominal and variation pessimism from the slack.

When computing the common clock pessimism, if the launch and capture transition (rise/fall) at the common point are the same, the variation at the common point is removed from the slack.

If the "normal" setting of the application option *time.clock_recovergence_pessimism* is used, and if the launch and capture transition (rise/fall) at the common point are different, the variation will not be removed. This is because the rising path and the falling path are not fully correlated.

If this is too pessimistic, a percentage of the variation can be removed by setting the application option *time.crpr_different_transition_variation_derate* to a non-zero value. i.e. to remove 10% of the variation, use the value 0.1. To allow the full credit of the variation, use 1.0.

See Also

- [get_app_option_value](#)
- [set_app_options](#)
- [report_app_options](#)

time.crpr_remove_clock_to_data_crp

Allows the removal of clock reconvergence pessimism (CRP) from paths that fan out directly from clock source to the data pins of sequential devices.

Data Types

Boolean

Default false**Description**

When this application option is set to true then CRP will be removed for all paths that fan out directly from clock source pins to the data pins of sequential devices.

When this option is set to false, only the CRP up to the clock source pin which fans out to the data pin of the sequential device would be removed. This is because the path up to a clock source pin is considered to be a clock path.

t

Consider the following example, where GCLK1 is a generated clock with CLK as its master clock. In this case, the CRP between pins A and B would be removed irrespective of the value of the application option. However, when the application option is set to true, additional CRP between pins B and C would also be removed.

```
+-- buff3 (D) --- FF1/D
                                     |
A (CLK) --- buff1 (B GCLK1) --- buff2 (C) --- +-- buff4 (E) --- FF1/CP
```

Application options are set using the `set_app_options` command, and they can be specified globally or with respect to a design. To determine the current value of this application option use:

```
get_app_option_value -user_default -name
time.crpr_remove_clock_to_data_crp
Use report_app_options to show the value of multiple application options.
```

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)
- [report_crpr](#)

time.delay_calc_enhanced_ccsn_waveform_analysis

Enables composite current source (CCS) advanced waveform propagation enhancement for advanced nodes with CCS noise libraries in the PrimeTime delay calculation.

Data Types

Boolean

Default true

Description

With this variable set to default (true), advanced waveform calculation enhancement is enabled in the PrimeTime delay calculation. This setting matches the PrimeTime `time.delay_calc_enhanced_ccsn_waveform_analysis` variable. For this setting to be effective, the CCS noise data must be well characterized and aligned with data model (NLDM) and CCS timing data.

See Also

- [get_app_option_value](#)
- [set_app_options](#)

time.delay_calc_waveform_analysis_constraint_arcs_compatibility

Specifies whether setup and hold constraint analysis uses nonlinear delay model (NLDM) lookup tables instead of waveform propagation.

Data Types

Boolean

Default true

Description

This variable specifies whether setup and hold constraint analysis uses nonlinear delay model (NLDM) lookup tables instead of waveform propagation, even when the *time.delay_calc_waveform_analysis_mode* variable is set to *full_design*.

Set the *time.delay_calc_waveform_analysis_constraint_arcs_compatibility* variable as follows:

- *true* (default) - Setup and hold constraint analysis uses nonlinear delay model (NLDM) lookup tables and does not consider the waveform distortion effect.
- *false* - If waveform propagation is enabled by the *time.delay_calc_waveform_analysis_mode*, setup and hold constraint analysis considers waveform distortion effects. This analysis requires CCS timing and noise data in the library.

See Also

- [extract_model](#)
- [report_timing](#)
- [update_timing](#)

time.delay_calc_waveform_analysis_mode

Controls usage of CCS-based waveform analysis for uncoupled and signal integrity calculation.

t

Data Types

string

Default disabled**Description**

You can set this application option to one of the following:

- *disabled* (default) - Disables CCS waveform analysis.
- *full_design* - Enables CCS waveform analysis on the entire design.

CCS waveform analysis requires libraries that contain CCS timing and CCS noise data.

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design.

To determine the current value of this application option on a specific block, use:

```
get_app_option_value -block $block -name time.delay_calc_waveform_analysis_mode
```

```
get_app_option_value -user_default -name time.delay_calc_waveform_analysis_mode
```

Use *report_app_options* to show the value of multiple application options.

See Also

- [get_app_option_value](#)
- [set_app_options](#)

time.delay_calculation_style

Controls how the timer calculates cell and net delays and slews.

Data Types*Enum***Default** "auto"**Description**

This option controls how the timer calculates cell and net delays and slews. Possible values are "auto", and "zero_interconnect". "auto" is the default. In "zero_interconnect" mode, cell delays will be calculated assuming zero interconnect capacitance, and net delays will be zero. Zero-interconnect analysis is always used for unplaced designs. In "auto" mode, tool uses its default engine for net delay calculations.

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design. To determine the current value of an

t

application option on a specific design, use `get_app_option_value -design $design -name <app_option_name>`. Use `report_app_options` to show the value of multiple application options.

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

time.deprioritize_io_path_groups

Specifies whether the tool should de-prioritize the default I/O path groups (**in2reg_default**, **in2out_default**, **reg2out_default**) if it's weight is default (1.0).

Data Types

Boolean

Default true

Description

This option reduces the weight of the default I/O path groups to a lower value if the path group weight is a default value (refer: time.enable_io_path_groups).

The optimization module then uses this deprioritized weight for costing and reporting. Setting this app-option to true / false has no costing or reporting impact unless time.enable_io_path_groups is set to true.

When *true* (default), this app-option deprioritizes the weight of I/O path groups.

When *false*, the weights of the I/O path groups is either the default value or the weight explicitly set by the user.

If the weight is explicitly set to 1.0 and this app-option is set to true, tool will still deprioritize it. Path groups with weight non-default are not deprioritized with this app-option.

Examples

```
prompt> set_app_options \\  
-name time.enable_io_path_groups -value true  
prompt> set_app_options \\  
-name time.deprioritize_io_path_groups -value true  
prompt> group_path -name "**in2reg_default**" -weight 5.00  
prompt> report_path_group
```

Path_Group	Weight	From/Through/To
------------	--------	-----------------

```
-----
**async_default**          1.00      -
**clock_gating_default**   1.00      -
**default**                1.00      -
**in2out_default**         0.10      -
**in2reg_default**         5.00      -
**reg2out_default**        0.10      -
clk                        1.00      -to [get_clocks {clk}]

prompt> set_app_options \\  
-name time.enable_io_path_groups -value true
prompt> set_app_options \\  
-name time.deprioritize_io_path_groups -value true
prompt> group_path -name "***in2reg_default**" -weight 1.0
prompt> report_path_groups

Path_Group          Weight      From/Through/To
-----
---
**async_default**   1.00      -
**clock_gating_default** 1.00      -
**default**         1.00      -
**in2out_default**   0.10      -
**in2reg_default**   0.10      -
**reg2out_default**  0.10      -
clk                 1.00      -to [get_clocks {clk}]
```

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)
- [report_path_groups](#)

time.disable_case_analysis

Specifies whether case analysis is disabled.

Data Types

Boolean

Default false

Description

When *false* (the default), constant propagation is performed in the design from pins either that are tied to a logic constant value, or for which a *set_case_analysis* command is

t

specified. For example, a typical design has several pins set to a constant logic value. By default, this constant value propagates through the logic to which it connects. When the app option *time.disable_case_analysis* is *true*, case analysis and constant propagation are not performed.

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design. To determine the current value of this application option on a specific design, use *get_app_option_value -design \$design -name time.disable_case_analysis*. Use *report_app_options* to show the value of multiple application options.

See Also

- [get_app_option_value](#)
- [remove_case_analysis](#)
- [report_case_analysis](#)
- [report_app_options](#)
- [set_app_options](#)
- [set_case_analysis](#)

time.disable_case_analysis_ti_hi_lo

Determines whether case analysis is propagated from pins that are tied to a logic constant value.

Data Types

Boolean

Default false

Description

When this application option is set to its default of *false*, constant propagation is performed from pins that are tied to a logic constant value. For example, a typical design has several pins set to a constant logic value. By default, this constant value propagates through the logic to which it connects. When you set this application option to *true*, constant propagation is not performed from these pins.

This current value of this application option does not alter the propagation of logic values from pins where the logic value has been set by the *set_case_analysis* command.

If you set the *time.disable_case_analysis* application option to *true*, all constant propagation is disabled regardless of the current value of this application option.

t

Application options are set using the `set_app_options` command, and they can be specified globally or with respect to a design. To determine the current value of this application option on a specific design, use the `get_app_option_value -design $design -name time.disable_case_analysis_ti_hi_lo` command. Use the `report_app_options` command to show the value of multiple application options.

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)
- [remove_case_analysis](#)
- [set_case_analysis](#)

time.disable_clock_gating_checks

Disables clock gating checks, including both automatically inferred and library-defined clock gating checks.

Data Types

Boolean

Default false

Description

Set this application option to *true* to disable all clock gating checks, including both automatically inferred and library-defined clock gating checks, as well as inferred checks modified by the `set_clock_gating_check` command.

When this application option is set to *false* (the default), clock gating checks are enabled.

For more information, see SolvNet article 015769, "How Are Clock Gating Checks Inferred?"

This application option can be set in the global and block-specific context and is persistent.

See Also

- [get_app_option_value](#)
- [remove_clock_gating_check](#)
- [report_app_options](#)

t

- [set_app_options](#)
- [set_clock_gating_check](#)
- [set_disable_clock_gating_check](#)

time.disable_cond_default_arcs

Disables the default, non-conditional timing arc between pins that have conditional arcs.

Data Types

Boolean

Default false

Description

When *true*, disables nonconditional timing arcs between any pair of pins that have at least one conditional arc of the same arc sense. When *false* (the default), these nonconditional timing arcs are not disabled. This application option is primarily intended to deal with the situation between two pins that have conditional arcs, where there is always a default timing arc with no condition.

Set this application option to *true* when the specified conditions cover all possible state-dependent delays, so that the default arc is useless. For example, consider a 2-input XOR gate with inputs as A and B and with output as Z. If the delays between A and Z are specified with 2 arcs with respective conditions 'B' and 'B~', the default arc between A and Z is useless and should be disabled.

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design. To determine the current value of this application option on a specific design, use *get_app_option_value -design \$design -name time.disable_cond_default_arcs*. Use *report_app_options* to show the value of multiple application options.

See Also

- [get_app_option_value](#)
- [report_disable_timing](#)
- [report_app_options](#)
- [set_app_options](#)

t

time.disable_flr_for_same_disable_object

Enable skipping of adding file line reference when same object is disabled from different lines and constraint files.

Data Types

Boolean

Default false

Description

When set to *true*, it skips adding file line reference of constraint set_disable_timing when same object is disabled from different lines/constraint files. Only first file line reference is stored and that will be shown in write_sdc instead of all file/line numbers.

When set to *false*, default behaviour of adding all file line references will continue and write_sdc will dump all file/line numbers.

Application options are set using the *set_app_options* command, and they may be specified with respect to a design. To determine the current value of this application option, use:

```
get_app_option_value -user_default -name  
time.disable_flr_for_same_disable_object
```

Use **report_app_options** to show the value of multiple application options.

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)
- [set_disable_timing](#)

time.disable_internal_inout_net_arcs

Controls whether bidirectional feedback paths across nets are disabled or not.

Data Types

Boolean

Default true

t

Description

When *true* (the default), the tool automatically disables bidirectional feedback paths that involve more than one cell; no path segmentation is required. Note that only the feedback net arc between non-bidirectional driver and load is disabled. When *false*, these bidirectional feedback paths are enabled.

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design. To determine the current value of this application option on a specific design, use *get_app_option_value -design \$design -name time.disable_internal_inout_net_arcs*. Use *report_app_options* to show the value of multiple application options.

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

time.disable_recovery_removal_checks

Disable or enable the timing analysis of recovery and removal checks in the design.

Data Types

Boolean

Default false

Description

When *true*, disables recovery and removal timing analysis. When *false* (the default), the tool performs recovery and removal checks; for descriptions of these checks, see the man page for the *report_constraint* command.

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design. To determine the current value of this application option on a specific design, use *get_app_option_value -design \$design -name time.disable_recovery_removal_checks*. Use *report_app_options* to show the value of multiple application options.

t

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

time.early_launch_at_latch_loop_breaker

Removes clock latency pessimism from the launch times for paths which begin at the data pins of transparent latch loop breakers.

Data Types

Boolean

Default true

Description

When a latch loop breaker is in its transparent phase, data arriving at the D-pin passes through the element as though it were combinational. To model this scenario, whenever IC Compiler II determines that time borrowing occurs at such a D-pin, paths which originate at the D-pin are created.

Sometimes there is a difference between the launching and capturing latch latencies, due either to reconvergent paths in the clock network or different min and max delays of cells in the clock network. For setup paths, IC Compiler II uses the late value to launch and the early value to capture. This achieves the tightest constraint and avoids optimism. However, for paths starting from latch D-pins this is pessimistic since data simply passes through and thus does not even "see" the clock edge at the latch.

When this application option is set to *true* (the default), such pessimism is eliminated by using the early latch latency to launch such paths. Note that only paths which originate from a latch D-pin are affected. When the application option is set to *false*, late clock latency is used to launch all setup paths in the design.

You should use this form of pessimism removal because it does not cause the runtime of the analysis to increase. However, you should disable it when clock reconvergence pessimism removal (CRPR) is enabled (the *time.remove_clock_reconvergence_pessimism* application option is *true*). CRPR may not be applied to paths which have been launched using an early latency or the results may be optimistic. Since CRPR is a more sophisticated and accurate means of pessimism removal, the user should disable *time.early_launch_at_borrowing_latches* when CRPR is enabled so that CRPR applies to all paths in the design. In this mode, note that the D-pin launch time is not modified by the open edge CRP - since late launch latency is used at

t

the path startpoint, to additionally add CRP would be pessimistic, representing a "double-counting" of early-late differences.

To determine the current value of this application option, use the `get_app_option_value -name time.early_launch_at_latch_loop_breaker` command.

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

time.edge_specific_source_latency

Controls whether or not the generated clock source latency computation considers edge relationship.

Data Types

Boolean

Default true

Description

When true, only paths with the same sense relationship derived from generated clock definition are considered.

When this application option is set to *false*, all paths that fanout to a generated clock source pin are considered and the worst path is selected for generated clock source latency computation.

When this application option is *true* (the default), and there is no path with the sense relationship that matches the generated clock definition a warning message (GRF-011) is issued and zero latency is used.

See Also

- [create_generated_clock](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

t

time.enable_auto_mux_clock_exclusivity

Enables automatic inference of MUX cells for clock exclusivity.

Data Types

Boolean

Default false

Description

If you set this app option to true, the tool automatically identifies the MUX cells on the clock network and forces the clocks that go through different data lines of the MUX cells to be exclusive.

By default, it is set to false and the tool does not automatically infer the exclusivity of clocks from MUX cells in the clock network. Please note that if there is a generated clock defined after the exclusivity point, the exclusivity states will be lost. This is similar to the existing behavior of the generated clocks. When a generated clock is created at a pin, all other clocks arriving at that pin are blocked unless they also have generated clock versions created at that pin.

Automatic mux inference works up to a maximum number of mux levels only. Beyond that all exclusive pins are excluded to avoid large memory increase. Limit can be controlled by app option, time.max_exclusive_mux_levels.

See Also

- [set_clock_exclusivity](#)
- [set_disable_auto_mux_clock_exclusivity](#)
- [report_clock](#)

time.enable_ccs_rcv_cap

Enables Synopsys Composite Current-Source (CCS) receiver model during timing analysis.

Data Types

Boolean

Default false

t

Description

This application option controls delay calculation using CCS receiver model during timing analysis. By default, CCS receiver model is not enabled. Turning this option on clears the existing delay data in the timer. And CCS receiver caps are reported in `report_timing`, `report_delay_calculation`, `report_stage` and `report_constraint` when this option is on.

Application options are set using the `set_app_options` command, and they may be specified globally or with respect to a design.

To determine the current value of this application option on a specific block, use: `get_app_option_value -block $block -name time.enable_ccs_rcv_cap`
`get_app_option_value -user_default -name time.enable_ccs_rcv_cap` Use `report_app_options` to show the value of multiple application options.

See Also

- [get_app_option_value](#)
- [set_app_options](#)

time.enable_clock_propagation_through_preset_clear

Enables propagation of clock signals through preset and clear pins.

Data Types

Boolean

Default false

Description

When this option is set to *true*, clock signals will be propagated through the preset and clear pins of a sequential device. Naturally, this will only occur when clock signals are incident on such pins.

If clock reconvergence pessimism removal (CRPR) is enabled it will consider any sequential devices in the fanout of such pins for analysis.

Application options are set using the `set_app_options` command, and they may be specified globally or with respect to a design. To determine the current value of this application option on a specific design, use `get_app_option_value -design $design -name time.enable_clock_propagation_through_preset_clear`. Use `report_app_options` to show the value of multiple application options.

t

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

time.enable_clock_propagation_through_three_state_enable_pins

Allow clocks to propagate through the enable pin of a three-state cell.

Data Types

Boolean

Default false

Description

When *true*, clocks can propagate through the enable pins of three-state cells. When *false* (the default), the tool will not propagate clocks between a pair of pins if there is at least one timing arc with a disable sense between those pins.

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design. To determine the current value of this application option on a specific design, use *get_app_option_value -design \$design -name time.enable_clock_propagation_through_three_state_enable_pins*. Use *report_app_options* to show the value of multiple application options.

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

time.enable_clock_to_data_analysis

Enables ideal clock transition and network latency for clock-to-data interface pins

Data Types

Boolean

Default false

t

Description

When set to *true*, this application option enables timing analysis of clock-to-data pins when the clock is ideal. The ideal clock latency is used as the data arrival time at the clock-to-data pin, and the ideal transition is used as data transition time at that pin. If more than one clock drives the data pin, the worst latency and transition values apply. Note that there will be symbol 'U' in *report_timing* which means using ideal clock transition or network latency for clock_to_data interface pin.

When the application option is *false* (the default), the tool uses the actual propagated clock latency and transition time at the clock-to-data pin, if any, and ignores any ideal clock latency and transition time set for the clock.

You can set the ideal latency and ideal transition time of a clock by using the *set_clock_latency* and *set_clock_transition* commands.

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design. To determine the current value of this application option on a specific block, use: *get_app_option_value -block \$block -name time.enable_clock_to_data_analysis* *get_app_option_value -user_default -name time.enable_clock_to_data_analysis* Use the *report_app_options* command to show the value of multiple application options.

See Also

- [create_clock](#)
- [create_generated_clock](#)
- [set_clock_latency](#)
- [set_clock_transition](#)

time.enable_constraint_variation

Enables constraint variation in parametric on-chip variation (POCV) analysis.

Data Types

Boolean

Default false

Description

If you set this option to true and set the option *time.pocvm_enable_analysis* to true, constraint variation is considered in parametric on-chip variation (POCV) analysis.

See Also

- [read_ocvm](#)
- [remove_ocvm](#)
- [report_ocvm](#)
- [report_timing](#)

time.enable_cumulative_incremental_derate

Use cumulative derate with *set_timing_derate -increment*

Data Types

bool

Default false

Description

Use cumulative derate with *set_timing_derate -increment*

time.enable_incr_update_for_clock_balance_points

Specifies whether the tool propagates selected clocks incrementally.

Data Types

Boolean

Default false

Description

When this application option is set to *true*, tool propagates selected clocks downstream instead of all clocks on newly added nodes in incremental mode during clock optimization step.

time.enable_io_path_groups

Specifies whether I/O path groups are enabled or disabled.

Data Types

Boolean

Default true

t

Description

When *true*, this option enables I/O path group costing and reporting. I/O path group is a new special group path under `group_path` category. This enables user to group and report boundary and I/O paths. It comes under inbuilt group paths.

I/O path group has following three sub-groups, each having their reserved names :

1. Input port to register (**in2reg_default **)
2. Register to Output port (**reg2out_default **)
3. Input port to Output port (**in2out_default **)

I/O path group has following properties:

1. They belong to special inbuilt path group category.
2. By default they are enabled.
3. Default weight of each I/O path group is 1.0, which is configurable.
4. They exists event before timer is created and can be listed using commands: **report_constraints** or **report_path_groups**.
5. They cannot be removed using **remove_path_groups** command.
6. Toggling or setting value of this app option invalidates timer.

When *false*, the tool disables I/O path group costing and reporting.

Application options are set using the `set_app_options` command, and they may be specified globally or with respect to a design. To determine the current value of this application option on a specific design, use: `get_app_option_value -block $block -name time.enable_io_path_groups` `get_app_option_value -user_default -name time.enable_io_path_groups` Use `report_app_options` to show the value of multiple application options.

App-option `time.deprioritize_io_path_groups` can be used to deprioritize the default I/O path groups for optimization.

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)
- [report_path_groups](#)

time.enable_low_accuracy_reports

Disable use of signoff timing for timing reports.

t

Data Types

Boolean

Default false**Description**

When this application option is set to its default of *false*, signoff accurate delay calculation is used for postroute netlist. When this application option is set to *true*, signoff accuracy is disabled. This setting is ignored during optimization and can cause discrepancy in timing used for optimization and finally used for reporting.

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

time.enable_ml_delay_calculation

Controls usage of ML-based acceleration for uncoupled delay calculation

Data Types

Boolean

Default true**Description**

You can set this application option to one of the following:

- *true* (default) - Turn on ML acceleration for delay calculation.
- *false* - Turn off ML acceleration for delay calculation.

ML-based delay calculation acceleration requires either or both of AWP and CCS receiver enabled.

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design.

To determine the current value of this application option on a specific block, use:
get_app_option_value -block \$block -name time.enable_ml_delay_calculation
get_app_option_value -user_default -name time.enable_ml_delay_calculation Use
report_app_options to show the value of multiple application options.

See Also

- [get_app_option_value](#)
- [set_app_options](#)

time.enable_multi_input_switching_analysis

Enables or disables multi-input switching (MIS) analysis.

Data Types

Boolean

Default false

Description

Set this application option to one of these values:

- *false* (default) - Disables MIS analysis.
- *true* - Enables MIS analysis, which uses the delay coefficients specified by the *set_multi_input_switching_coefficient* command.

See Also

- [report_multi_input_switching_coefficient](#)
- [reset_multi_input_switching_coefficient](#)
- [set_multi_input_switching_coefficient](#)

time.enable_multi_input_switching_timing_window_filter

Enables or disables the window alignment and overlap checking for multi-input switching (MIS) analysis.

Data Types

Boolean

Default true

t

Description

Set this application option to one of these values:

- *true* (default) - Enables window alignment and overlap checking during MIS analysis. The tool collects the arrival windows of input pins, performs overlap checking, and derives the actual coefficient factor.
- *false* - Ignores all arrival window information and applies the specified coefficient factor directly for MIS analysis.

See Also

- [report_multi_input_switching_coefficient](#)
- [reset_multi_input_switching_coefficient](#)
- [set_multi_input_switching_coefficient](#)

time.enable_netlist_constant_transition

Determines whether transition propagation happens on network controlled by netlist constant.

Data Types

Boolean

Default false

Description

When this application option is set to its default of *false*, netlist constant drivers are considered disabled and transition propagation does not occur in the fanout drivers if the netlist constant is controlling and arcs from other input pins are disabled. When this application option is set to *true*, transition propagation occurs at the fanout driver even though netlist constant is propagated.

Application options are set using the *set_app_options* command, and they can be specified globally or with respect to a design. To determine the current value of this application option on a specific design, use the *get_app_option_value -design \$design -name time.enable_netlist_constant_transition* command. Use the *report_app_options* command to show the value of multiple application options.

See Also

- [get_app_option_value](#)
- [report_app_options](#)

- [set_app_options](#)
- [remove_case_analysis](#)
- [set_case_analysis](#)

time.enable_new_exceptions_report

Specifies whether the tool reports output in new format when user use the *report_exceptions* or *report_timing -exception <options>* commands.

Data Types

Boolean

Default false

Description

When this application option is set to *false* (the default), the *report_exceptions* or *report_timing -exception <options>* commands reports only in old format.

When this application option is set to *true*, the *report_exceptions* or *report_timing -exception <options>* commands will report in new format.

See Also

- [report_exceptions](#)
- [report_timing](#)

time.enable_non_sequential_checks

Enables or disables library nonsequential data checks in the design during timing analysis.

Data Types

Boolean

Default true

Description

This application option enables or disables analysis of library nonsequential data checks in the design. The nonsequential arcs defined in the library will not be used for constraint checking if this option is set to false. This option does not affect the data checks defined by the *set_data_check* command.

Application options are set using the *set_app_options* command, and they can be specified with respect to a design. Use *get_app_option_value -name*

t

time.enable_non_sequential_checks to retrieve the current value. Use *set_app_options -design <design_name> {time.enable_non_sequential_checks true|false}* to set the new value for a given design. Use *report_app_options* to show the value of multiple application options.

See Also

- [set_data_check](#)
- [get_app_option_value](#)
- [set_app_options](#)

time.enable_preset_clear_arcs

Specifies whether preset and clear timing arcs are enabled or not.

Data Types

Boolean

Default `false`

Description

When *true*, permanently enables asynchronous preset and clear timing arcs, so that you use them to analyze timing paths. When *false* (the default), the tool disables all preset and clear timing arcs.

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design. To determine the current value of this application option on a specific design, use *get_app_option_value -design \$design -name time.enable_preset_clear_arcs*. Use *report_app_options* to show the value of multiple application options.

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

time.enable_propagation_for_noise

Enables noise propagation across combinational-cells from input pin to output-pins during timing analysis.

t

Data Types*Boolean***Default** false**Description**

This application option controls noise propagation across combinational-cells during noise analysis. By default, noise propagation is not enabled. For allowing noise propagation, enable this application option by setting its value to true.

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design.

Use *get_app_option_value -design \$design -name time.enable_propagation_for_noise* to get the current value of this application option on a specific design.

Use *get_app_option_value -name time.enable_propagation_for_noise* to retrieve the current value. Use *set_app_options -as_user_default {time.enable_propagation_for_noise true/false}* to set the new value. Use *report_app_options* to show the value of multiple application options.

See Also

- [get_app_option_value](#)
- [set_app_options](#)

time.enable_si_timing_windows

Enables timing window analysis during crosstalk analysis for detail-routed design.

Data Types*Boolean***Default** true**Description**

This application option controls timing window analysis during crosstalk analysis.

When timing window analysis is not enabled, infinite windows are used for all the victim nets and aggressor nets, and all unfiltered aggressors will be considered and contribute to the final crosstalk delta delay.

When timing window analysis is enabled, the overlap between arrival time of the signals on the victim net and aggressor nets is considered during crosstalk analysis. Non-overlapping aggressors will be excluded and will not contribute to the delta delay; partially-

t

overlapping aggressors will partially contribute to the delta delay; fully-overlapping aggressors will fully contribute to the delta delay. To see the results of overlapping timing windows, use the *report_delay_calculation -crosstalk* command.

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design.

Use *get_app_option_value -design \$design -name time.enable_si_timing_windows* to get the current value of this application option on a specific design.

Use *get_app_option_value -name time.enable_si_timing_windows* to retrieve the current value. Use *set_app_options -as_user_default {time.enable_si_timing_windows true|false}* to set the new value. Use *report_app_options* to show the value of multiple application options.

See Also

- [get_app_option_value](#)
- [set_app_options](#)
- [report_delay_calculation](#)

time.enable_slew_variation

Enables transition variation in parametric on-chip variation (POCV) analysis.

Data Types

Boolean

Default false

Description

If you set this application option to *true* and set the *time.pocvm_enable_analysis* application option to *true*, transition variation is considered in parametric on-chip variation (POCV) analysis.

See Also

- [read_ocvm](#)
- [report_ocvm](#)

time.enable_sps_delcalc

Enable Fusion Compiler CTPM flow.

t

Data Types

Boolean

Default false

Description

When set to *true*, this variable enables Fusion Compiler CTPM flow.

CTPM flow enables analysis for timing and power impact assessment of a given target. This target can be defined through a spice model representing next PDK, spot model, different spice corner.

CTPM files aims to bridge the gap between the base and the target by capturing and gap through spice process parameters placed inside of a Compact Timing Power Model (CTPM) database. When CTPM is consumed in a base STA session, the QoR of this session is shifted such that to match QoR of the target. This targeting analysis is done without the libraries characterized at that target, saving designers cycles on waiting for official library characterization. Requirements are the sensitivity Fusion libraries, which represent timing behavior of each arc for a given physical spice parameter perturbation range.

See Also

- [read_ctpm](#)
- [reset_ctpm](#)
- [report_ctpm](#)

time.enable_static_noise

Enables Noise-Analysis during timing analysis.

Data Types

Boolean

Default false

Description

This application option controls noise analysis during timing analysis. By default, noise analysis is not enabled. For performing noise analysis, following conditions are to be met: 1) Scenario needs to be enabled for Noise. 2) Timing analysis is enabled for si analysis. 3) Timing analysis is enabled for noise analysis.

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design.

t

Use `get_app_option_value -design $design -name time.enable_static_noise` to get the current value of this application option on a specific design.

Use `get_app_option_value -name time.enable_static_noise` to retrieve the current value. Use `set_app_options -as_user_default {time.enable_static_noise true|false}` to set the new value. Use `report_app_options` to show the value of multiple application options.

See Also

- [get_app_option_value](#)
- [set_app_options](#)

time.enable_thermal_analysis

Enables thermal aware timing analysis on the design.

Data Types

Boolean

Default false

Syntax

`set_app_options -name time.enable_thermal_analysis -value <true/false>`

Description

Set the app option `time.enable_thermal_analysis` to enable Thermal Timing analysis in FC.

This app option can be turned on in FC shell prior to `Place_opt` or `Route_opt` to enable Thermal timing driven Optimizations. Intermediate timing updates are incremental thermal aware timing updates. The final timing reports are thermal aware timing reports.

Examples

Thermal timing driven Route opt script: The following example shows an Thermal aware timing enabled FC Route_opt script.

```
#####
###
#
#
#   File : route_opt_thermal.tcl
#
#
#   FC Thermal Aware Route Opt Script
#
```


t

```

#
#
#   FC Thermal STA build: X-2025.06-DEV
#
#
#####
###
open_lib 0205_new_mmmc
open_block 0205_b4core_mmmc
link_block -force

# The Thermal aware STA settings are the highlighted lines below
#Enable Power, Thermal analysis on all scenarios - m1::c1, m2::c1, m2::c2
set_scenario_status [get_scenarios ] -leakage true -dynamic true
set_scenario_status [get_scenarios ] -active true -thermal true
report_scenarios -all

#Set Ambient temperature same as Nominal temperature
set_app_options -name thermal.ambient_temperature -value 125

# Turn on Thermal Analysis
set_app_options -list {time.enable_thermal_analysis true}

# Read Thermal derate side file for lib cells
read_thermal_info -derate ./thermal_derate_side_file_test_0126 -corners
[all_corners]

# Input the Thermal tech file
set_app_options -name thermal.tech_file
-value /u/knikro/k_data/kelvin_labs/lab1/lef_def/sample.sttf

# Thermal Aware Route Opt
route_opt

# Update timing after route_opt (thermal aware)
update_timing

# report_timing (Critical path) after route_opt (thermal aware)
report_timing -sig 3 -nosp -input -net -phy -derate

```

See Also

- [report_timing](#)

time.enable_via_variation

Enables via variation analysis.

Data Types

Boolean

Default false

Description

Setting this option to *true* enables the via variation analysis. Using this data in POCV timing analysis produces more accurate results by taking into account the effects of interconnect via variation.

To use this feature, POCV analysis must be enabled (*time.pocvm_enable_analysis* option set to *true*) and you must specify the variation parameters in a text file, and read in that data with the *read_ivm* command.

See Also

- [read_ivm](#)
- [report_ivm](#)

time.estnet_blockage_aware

Enable blockage detour in virtual net estimation.

Data Types

Boolean

Default true

Description

If you set this app option to false, virtual router would ignore the routing blockages to avoid the detour around the routing blockages when to generate virtual router topology.

time.filter_power_and_ground_terms

Filters power and ground ports while doing *write_sdc*.

Data Types

Boolean

Default false

t

Description

This app option filters out ground and power ports while writing constraints using `write_sdc` when used with `set_ideal_network`. Application options are set using the `set_app_options` command.

The default value is false. Example:

```
prompt> set_app_options -name constraint.filter_power_and_ground_terms
        -value true
```

See Also

- [get_app_option_value](#)
- [set_app_options](#)

time.frequency_based_max_cap

Enable max_capacitance constraint from the frequency based table on the lib_pin in the library.

Data Types

Boolean

Default false

Description

When set to *true*, the max_capacitance constraint is read from the maxCap table in the library. The maxCap table in the library specifies frequency based values for the lib_pins. The frequency, used in constraint query, is the highest of the clock frequencies at which a pin is operating. The clocks passing through the pin(on the clock network) and the launching clocks of the paths through the pin are used to get the highest frequency. Reports and optimization will reflect frequency based max_capacitance constraints and costs in the flow.

When the option is set to true and the lib_pin doesn't have a maxCap table then the default max_capacitance constraint is used.

When the app option is set to true and the max-capacitance constraint is specified for the lib_pin along with maxCap table, the tightest value of lib_pin value and the table value is used.

When set to *false*, default max_capacitance constraint on the lib_pin is used and the maxCap table is ignored.

t

Application options are set using the *set_app_options* command, and they may be specified with respect to a design. To determine the current value of this application option, use:

```
get_app_option_value -user_default -name time.frequency_based_max_cap
Use report_app_options to show the value of multiple application options.
```

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)
- [report_constraint](#)
- [set_max_capacitance](#)

time.gclock_source_network_num_master_registers

Specifies the maximum number of register clock pins clocked by the master clock allowed in generated clock source latency paths.

Data Types

int

Default 10000000

Description

This option specifies the maximum number of register clock pins clocked by the master clock allowed in generated clock source latency paths. This option does not affect the number of registers traversed in a single path that do not have a clock assigned or are clocked by another generated clock that has the same primary master as the generated clock in question.

Register clock pins or transparent-D pins of registers clocked by unrelated clocks are not traversed in determining generated clock source latency paths. An unrelated clock is any clock that primary master clock differs from the generated clock whose source latency paths are being computed.

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design. To determine the current value of this application option on a specific design, use *get_app_option_value -design \$design -name time.gclock_source_network_num_master_registers*. Use *report_app_options* to show the value of multiple application options.

t

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

time.high_fanout_net_pin_capacitance

Sets the pin capacitance of a high fanout net (HFN).

Data Types

Typed

Default 1pF

Description

This application option specifies the pin capacitance of a single terminal of a high fanout net. The default value is 1pF.

This application option is used during net estimation. If a net has a number of terminals that is greater than or equal to *time.high_fanout_net_threshold*, it is processed as a high fanout net. For a high fanout net, actual net estimation is not performed in the design flow. For a high fanout net, zero wire capacitance, saturated load capacitance, and default thresholds are considered. The saturated load capacitance is calculated as: Saturated Load Capacitance = $\{time.high_fanout_net_threshold * time.high_fanout_net_pin_capacitance\}$

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design.

Use *get_app_option_value -design \$design -name time.high_fanout_net_pin_capacitance* to get the current value of this application option on a specific design.

Use *get_app_option_value -name time.high_fanout_net_pin_capacitance* to retrieve the current value. Use *set_app_options {time.high_fanout_net_pin_capacitance capacitance_value>}* to set the new value.

Examples: prompt> *get_app_option_value -name time.high_fanout_net_pin_capacitance 1pF* prompt> *set_app_options {time.high_fanout_net_pin_capacitance 1uF}* prompt> *get_app_option_value -name time.high_fanout_net_pin_capacitance 1uF*

Use *report_app_options* to show the value of multiple application options.

t

See Also

- [get_app_option_value](#)
- [set_app_options](#)

time.high_fanout_net_threshold

Sets the minimum number of net terminals or loads to classify a net as a high fanout net (HFN).

Data Types

int

Default 1000

Description

This application option specifies the minimum number of net terminals which a net should have to be considered as a high fanout net. The default value is 1000. This application option is used during net estimation.

If a net has number of terminals greater than or equal to *time.high_fanout_net_threshold*, it is processed as a high fanout net. For a high fanout net, actual net estimation is not performed in the design flow. For a high fanout net, zero wire capacitance, saturated load capacitance, and default thresholds are considered during net estimation. The saturated load capacitance is calculated as:

Saturated Load Capacitance = {*time.high_fanout_net_threshold* *
time.high_fanout_net_pin_capacitance}

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design.

Use *get_app_option_value -design \$design -name time.high_fanout_net_threshold* to get the current value of this application option on a specific design.

Use *get_app_option_value -name time.high_fanout_net_threshold* to retrieve the current value. Use *set_app_options {time.high_fanout_net_threshold <value>}* to set the new value. Use *report_app_options* to show the value of multiple application options.

See Also

- [get_app_option_value](#)
- [set_app_options](#)

t

time.ideal_clock_zero_default_transition

Specifies whether or not a zero transition value is assumed for sequential devices clocked by ideal clocks.

Data Types

Boolean

Default `true`

Description

Specifies how transitions are treated at clock pins of a flip-flop. If *set_clock_transition* command is used and the clock is ideal, the specified transition value will be used. If the clock transition is not set by the *set_clock_transition* command, and the clock is ideal, then this option will have the following effect. When *true* (the default), the tool uses a zero transition value for ideal clocks. When *false*, the tool uses a propagated transition value. Note that *set_clock_transition* will override the effect of this option for ideal clocks.

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design. To determine the current value of this application option on a specific design, use *get_app_option_value -design \$design -name time.ideal_clock_zero_default_transition*. Use *report_app_options* to show the value of multiple application options.

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

time.max_exclusive_mux_levels

Maximum number of levels of exclusive MUX cells allowed for automatic inference of exclusivity.

Data Types

Integer

Default `3`

Description

Automatic mux inference works up to a maximum number of MUX levels only. Beyond that all exclusive pins are excluded to avoid large memory increase. This app option sets

t

the limit to 3, by default. Tool doesn't restrict larger values but a value of 8 or smaller is recommended for memory overhead.

See Also

- [set_clock_exclusivity](#)
- [report_clock](#)

time.max_parallel_computations

Sets the maximum degree of parallelism used for parallel timing updates.

Data Types

integer

Default 0

Description

This application option specifies the degree of parallelism used for parallel timing updates. Increasing the parallelism results in reduced runtime at the expense of memory.

When this option has a value of 0 (the default), the number of cores used for timing update is specified by the value of the *-max_cores* option of the *set_host_options* command.

When this option has a value of 1, parallel timing update is disabled.

When this option has a value that is larger than 1 but smaller than the value of the *set_host_options -max_cores* option, the number of cores being used must not exceed this limit and a smaller memory footprint is achieved.

When this option has a value that is larger than the value of the *set_host_options -max_cores* option, the number of cores being used must not exceed the limit set with the *set_host_options* command and better runtime speedup can be achieved.

The actual degree of parallelism (number of threads) must not exceed the value specified in this option, but will not always be exactly the same. The tool derives the number internally to achieve the best performance.

See Also

- [set_host_options](#)

time.max_uncompressed_net_drivers

Use compressed net arcs if the number of drivers exceeds this threshold.

t

Data Types*int***Default** 100**Description**

This application option specifies the minimum number of net drivers which a net should have to receive compressed net arcs. The default value is 100. This application option is used primarily for mesh nets, for which all drivers are switching simultaneously.

If a net has number of drivers greater than or equal to *time.max_uncompressed_net_arcs*, then instead of receiving a net arc between every driver and every load, only one driver (with alphabetically first path name) is selected to have a net arc to every load. The remaining drivers receive a net arc connecting to only a single load. This prevents large numbers of redundant arcs from being created on mesh nets.

See Also

- [get_app_option_value](#)
- [set_app_options](#)

time.multi_input_switching_analysis_mode

Controls the multi-input switching (MIS) analysis mode.

Data Types*string***Default** lib_cell**Description**

Multi-input switching (MIS) allows timing analysis engine to model multi-input switching effects, which occur when multiple inputs transition simultaneously (as compared to the one-input-at-a-time stimulus used during library cell characterization).

Three analysis modes are available. This variable specifies the mode to use.

- *lib_cell* (default) - Uses library cell coefficients to perform multi-input switching analysis. In this mode, you must supply pre-characterized delay coefficients with the *set_multi_input_switching_coefficient* command.
- *advanced* - Enables advanced MIS analysis. In this mode, the tool calculates the multi-input switching speedup of a cell using its input slew/waveform and output load without the need for user-supplied annotated coefficients. See below for the supported library cell types and prerequisites.

t

This variable takes effect only when the *time.enable_multi_input_switching_analysis* is set to *true*.

time.multi_input_switching_precedence

Controls the precedence of multi-input switching (MIS) analysis modes.

Data Types

string

Default lib_cell

Description

This variable controls the precedence of calculation of MIS speedup for a cell when different modes of MIS analysis apply to the same cell.

- *lib_cell* (default) - User set *set_multi_input_switching_coefficient* takes precedence over advanced MIS calculated coefficients.
- *advanced* - Computed advanced MIS calculation takes precedence over user set *set_multi_input_switching_coefficient* coefficients.

See Also

- [set_multi_input_switching_coefficient](#)

time.nonlinear_scaling

Uses nonlinear model for scaling during delay calculation.

Data Types

Boolean

Default false

Description

This application option controls how scaling is done in native delay calculator. It does not affect PrimeTime delay calculator's behavior. By default, native delay calculator uses linear interpolation when scaling across voltage, temperature or process. If this application option is turned on, native delay calculator uses nonlinear interpolation for scaling, which is similar to PrimeTime delay calculator's method. Setting this option to *true* clears the existing delay data stored in the timer.

Using this application option requires at least three libraries in a scaling group. It improves correlation of native delay calculator with signoff Static Time Analysis for low voltage and

t

advanced nodes. However, it also increase the library loading time as well as memory usage for native delay calculator.

Use the `get_app_option_value -design $design -name time.nonlinear_scaling` command to get the current value of this application option on a specific design.

Use the `set_app_options -as_user_default {time.nonlinear_scaling true|false}` command to set a new value.

See Also

- [get_app_option_value](#)
- [set_app_options](#)

time.ocvm_enable_distance_analysis

Enables distance analysis support in advanced on-chip variation (AOCV) analysis or parametric on-chip variation (POCV) analysis.

Data Types

Boolean

Default false

Description

When this option is set to *true*, the tool calculates derating values for timing arcs based on both the distance of the path (physical range of the path) and the depth of the path (the number of successive gates in the path).

When this option is *false*, the tool will disable the pocv distance derating.

This option setting has an effect only under the following conditions:

- AOCV has been enabled by setting the *time.aocvm_enable_analysis* application option to *true* or POCV has been enabled by setting the *time.pocvm_enable_analysis* application option to *true*.
- The AOCV/POCV data tables contain distance dependency information.
- The tool has access to information about the physical locations of the cells and nets in the timing paths, so it can calculate the physical span of each timing arcs.

See Also

- [read_ocvm](#)
- [remove_ocvm](#)

t

- [report_ocvm](#)
- [report_timing](#)

time.ocvm_precedence_compatibility

Control the fallback to on-chip variation (OCV) derates when advanced OCV (AOCV) or parametric on-chip variation (POCV) is enabled.

Data Types

Boolean

Default false

Description

When this option is set to true, the OCV derate settings are completely ignored if AOCV or POCV tables are available.

When this option is set to false, for each object, both OCV and AOCV or POCV derate settings are considered, and the more specific setting is used. This is the default order of usage, from highest to lowest priority:

- i. OCV leaf cell derate setting
 - ii. AOCV or POCV lib-cell derate setting
 - iii. OCV lib-cell derate setting
 - iv. AOCV or POCV hier-cell derate setting
 - v. OCV hier-cell derate setting
 - vi. AOCV or POCV design derate setting
 - vii. OCV design derate

This option is effective only when *time.aocvm_enable_analysis* or *time.pocvm_enable_analysis* is set to true.

See Also

- [read_ocvm](#)
- [report_ocvm](#)
- [set_timing_derate](#)

time.open_block_exclude_file_line_info

Excludes file_line_info from being read into the constraint.

Data Types

Boolean

t

Default false**Description**

This application option controls whether or not `file_line_info` from being read as part of the constraint. When set to `false` (the default), the `file_line_info` will be read.

Use the `get_app_option_value -design $design -name time.open_block_exclude_file_line_info` command to get the current value of this application option on a specific design.

Use the `get_app_option_value -name time.open_block_exclude_file_line_info` command to retrieve the current value.

Use the `set_app_options -name time.open_block_exclude_file_line_info -value true|false` command to set a new value.

Use the `report_app_options` command to show the value of multiple application options.

See Also

- [get_app_option_value](#)
- [set_app_options](#)

time.pba_derate_only_mode

Specifies that only the path derates are reevaluated during path-based analysis.

Data Types

Boolean

Default false**Description**

This application option applies to the path-based analysis performed during the `get_timing_paths`, `report_timing` and `report_qor` commands when you specify the `-pba_mode` option. This application option controls whether regular path-based analysis (path-specific slew propagation) effects are performed during a path-based analysis in addition to adjusting the deratings on the path according to the path-based conditions.

If the application option is set to `false` (the default), the tool performs both regular path-based analysis and derating adjustment during a path-based analysis. This option removes the maximum amount of pessimism from the design, but the runtime can be long.

If you set the application option to `true`, the tool only adjusts the derating according to the path-based conditions during the path-based analysis.

t

This application option is useful in the advanced OCV flow and parametric OCV flow, and is recommended in these flows to achieve the fastest runtime.

Note that advanced or parametric OCV path-based analysis is applied only if user-specified advanced or parametric OCV information exists.

Also note that in parametric OCV analysis, the random variation pessimism of graph-based analysis over path-based analysis is zero. Therefore, running path-based analysis with this application option set to *true* in parametric OCV removes only the pessimism resulting from systematic variation, that is, distance-based derating.

See Also

- [get_timing_paths](#)
- [report_timing](#)

time.pba_exhaustive_endpoint_path_limit

Defines a limit for exhaustive path-based analysis.

Data Types

integer (Range: 1 to 2000000)

Default 1000

Description

This application option applies to the exhaustive path-based analysis performed during the *get_timing_paths* or *report_timing* command when the *-pba_mode exhaustive* option is specified. This exhaustive analysis is computationally intensive, and it is intended to be used only when the design is approaching signoff.

In certain badly-behaved designs the exhaustive analysis can run for a long time. This application option restricts the exhaustive path search so that the number of paths recalculated at any endpoint is limited. You can adjust this limit; however, increasing the value to a larger number increases the runtime of the analysis.

There are several other measures that improve the runtime of the analysis.

- Use conservative values for the *-nworst* and *-max_paths* options
- Set a realistic threshold for the *-slack_lesser_than* option

When an advanced on-chip variation (OCV) analysis is being performed, there are several additional application option settings that improve the runtime of the analysis.

t

- Enable CRPR
- Enable path-based advanced OCV only mode

See Also

- [get_timing_paths](#)
- [report_timing](#)

time.pba_optimization_mode**Data Types***string***Default** none**Description**

Controls the use of path-based analysis during post-route optimization. The effort value can be either none, path, or exhaustive, and the meanings are the same as in the report_timing and get_timing_paths commands. That is, when you specify none (the default), path-based analysis is not applied. When you specify path, path-based analysis is applied to the worst GBA path to each endpoint. When you specify exhaustive, an exhaustive path search algorithm is applied to determine worst pba path to an endpoint.

To avoid long runtimes and memory increases, consider running pba optimization when design is ready for signoff. Initial iterations can be done with 'path' mode, which gives the best total power savings with the fastest path-based analysis runtime. Final passes can be done with computationally more expensive 'exhaustive' mode.

The app-option currently applies for *route_opt* command with PrimeTime Delay Calculation enabled. This app-option also controls the behavior of report_timing, report_global_timing and report_qor when -pba_mode option is not specified. The default reports are kept consistent with the time.pba_optimization_mode used for *route_opt*. GBA reports can be generated only by explicitly specifying *-pba_mode none* after post route optimization with path mode.

The app-option is not allowed to be changed to 'none' after route_opt has been run with time.pba_optimization_mode being 'path' or 'exhaustive'. Default timing reports and optimization continues to use the pba optimization mode used for route_opt.

See Also

- [route_opt](#)
- [report_qor](#)

- [report_timing](#)
- [get_timing_paths](#)

time.pocvm_corner_sigma

Specifies the standard deviation used to calculate worst-case values from statistical parameters during parametric on-chip variation analysis.

Data Types

float

Default 3

Description

Parametric on-chip variation (POCV) analysis internally computes arrival times, required times, and slack values based on statistical distributions. When performing comparisons between these statistical quantities, the tool needs to know what values in the distribution are considered worst-case.

The default behavior is to choose values at 3.0 standard deviations away from the mean value. In other words, for an arrival time distribution, the worst-case early arrival is 3.0 standard deviations below the mean, and the worst-case late arrival is 3.0 standard deviations above the mean.

You can use this app-option set a different number of standard deviations away from the mean to determine the worst-case values for timing reports. Use a lower value such as 2.5 for less worst-case variation and more relaxed timing constraints. Conversely, use a higher value such as 3.5 for more worst-case variation and more restrictive timing constraints.

If you specify a block-specific sigma with this application option and then specify a corner-specific sigma with the *set_pocvm_corner_sigma* command, then the corner-specific sigma value will overwrite the previous block-specific value for the specific corners. The last sigma value specified for each corner will be the sigma value that is used for POCV analysis.

See Also

- [set_pocvm_corner_sigma](#)

time.pocvm_enable_analysis

Enable the parametric on-chip variation (POCV) analysis.

Data Types

Boolean

Default false

Description

When you set this option to true, the parametric on-chip variation timing update is performed as part of the *update_timing* command or any other functionality needing a timing update.

When this option is set to its default value, the POCV timing update is not performed.

See Also

- [read_ocvm](#)
- [remove_ocvm](#)
- [report_ocvm](#)
- [report_timing](#)

time.pocvm_enable_constraint_variation

Enables constraint variation in parametric on-chip variation (POCV) analysis.

Data Types

Boolean

Default false

Description

If you set this option to true and set the option *time.pocvm_enable_analysis* to true, constraint variation is considered in parametric on-chip variation (POCV) analysis.

This application option is obsolete and is provided for backward compatibility. Use the *time.enable_constraint_variation* app option instead.

See Also

- [read_ocvm](#)
- [remove_ocvm](#)
- [report_ocvm](#)
- [report_timing](#)

t

time.pocvm_enable_delay_slew_correlation

Enables enhanced delay/slew correlation in moment-based parametric on-chip variation (POCV) analysis.

Data Types

Boolean

Default false

Description

In a basic POCV analysis, each cell instance is considered to be statistically independent. However, when slew variation is enabled, the output slew from one cell induces a correlated delay component in subsequent cells.

When this variable is set to *false* (the default), the tool performs a basic analysis of this effect.

When this variable is set to *true*, the tool performs a more advanced analysis of this effect that can improve accuracy in a moment-based (non-Gaussian) analysis.

This variable has an effect only when both of the following conditions are true:

- The *time.pocvm_enable_extended_moments* variable is set to *true* to enable moment-based POCV analysis.
- The *time.enable_slew_variation* variable is set to *true* to consider slew variation.

time.pocvm_enable_extended_moments

Enables extended statistical moments LVF support for parametric on-chip variation (POCV) analysis.

Data Types

Boolean

Default false

Description

Setting this application option to true enables extended statistical moments to be computed and propagated for timing quantities during parametric on-chip variation (POCV) timing analysis mode. That is, all operations in the implementation tool are performed in POCV mode using extended moments as described in Liberty LVF extension. The extended moments are mean_shift, std_dev and skewness as defined in Liberty LVF extension in order to model non Gaussian distribution.

t

Note: Setting this app option to true has any effect only if *timing_pocvm_enable_analysis* is also set to true (i.e. PrimeTime POCV analysis has been enabled as well).

time.pocvm_max_transition_clock_sigma

Specifies the POCV standard deviation used for reporting maximum transition violations by the *report_constraint -max_transition* command for pins in the clock network.

Data Types

float or "disabled"

Default disabled

Description

When this variable is set to its default value of "disabled" the *report_constraint -max_transition* command reports transition times with the sigma value of *time.pocvm_max_transition_sigma* variable for all pins.

If it is set to a float value, the sigma applied to clock network pins is *time.pocvm_max_transition_clock_sigma* (it is assumed that *time.pocvm_max_transition_clock_sigma* is larger than *time.pocvm_max_transition_sigma*. Since all clock network pins also carry data signals, *time.pocvm_max_transition_sigma* is applied to all pins if it is larger than *time.pocvm_max_transition_clock_sigma*.

time.pocvm_max_transition_sigma

Specifies the standard deviation to be used for parametric on-chip variation analysis when reporting corner values from computed statistical transition times for max corner analysis.

Data Types

float

Default 0.0

Description

Parametric on-chip variation analysis internally computes timing quantities (example arrival, required, transition times) based on statistical distributions.

By default, the tool doesn't use statistical distributions for transition values for *max_transition* violations.

During reporting of *max_transition* violations using *report_constraint* command, a less pessimistic corner can be evaluated without performing a full timing update by selecting

t

a corner value for reporting max_transition values only. This reporting of corner values of max_transition can be set using the time.pocvm_max_transition_sigma app option.

See Also

- [set_pocvm_corner_sigma](#)

time.pocvm_precedence

Controls the precedence of how multiple POCV tables that are file-based or library-based are applied to a timing arc

Data Types

string

Default file

Description

Set this app-option to one of these values:

- o file (the default) - POCV coefficient file takes precedence over LVF data available in library.
- o library - POCV LVF data available in library takes precedence over coefficient file.
- o lib_cell_in_file - the tool uses the following priority, in decreasing order of precedence, to determine the table that applies to the arc:
 - o Library cell table
 - o Hierarchical cell table
 - o Design table

This app-option is effective only when the option *time.pocvm_enable_analysis* is set to *true*.

time.port_input_threshold_fall

Specifies the threshold voltage that defines the startpoint of the falling cell or net delay calculation.

Data Types

float

Default 50.0

t

Description

This application option specifies the threshold voltage that defines the startpoint of the falling cell or net delay calculation. The value is a percentage of the voltage source. Allowed values are 0.0 through 100.0, inclusive.

This option is one of eight application options that affect delay and transition time computations for detailed RC networks.

- *time.port_slew_lower_threshold_rise* (default is 20.0) •
- time.port_slew_lower_threshold_fall* (default is 20.0) •
- time.port_slew_upper_threshold_rise* (default is 80.0) •
- time.port_slew_upper_threshold_fall* (default is 80.0) •
- time.port_input_threshold_rise* (default is 50.0) •
- time.port_input_threshold_fall* (default is 50.0) •
- time.port_output_threshold_rise* (default is 50.0) •
- time.port_output_threshold_fall* (default is 50.0)

These application options interpret the cell delays and transition times from the reference library. The trip-point values specified in the library override the values specified by these options, so normally it should not be necessary to set the options. The specified values are applied only to libraries that do not contain trip-point specifications.

The default values specify that a cell delay is defined from 50 percent of the voltage value for the input transition to 50 percent of the voltage value for the output transition. The default values also specify that a transition time, or slew, is defined from 20 percent to 80 percent of the voltage.

To determine the current value of this application option, use the *get_app_option_value -name time.port_input_threshold_fall* command. For a list of all calc application options and their current values, use the *report_app_options calc.** command.

Examples

The following example specifies that cell delays are computed from 50 percent of the input transition to 55 percent of the output transition. In addition, the example specifies that transition times represent the delay from 10 percent to 90 percent of the voltage source.

```
prompt> set_app_options -list {
    {time.port_slew_lower_threshold_fall 10}
    {time.port_slew_lower_threshold_rise 10}
    {time.port_slew_upper_threshold_fall 90}
    {time.port_slew_upper_threshold_rise 90}
    {time.port_input_threshold_rise      50}
    {time.port_input_threshold_fall      50}
    {time.port_output_threshold_rise     55}
    {time.port_output_threshold_fall     55}
}
```

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

time.port_input_threshold_rise

Specifies the threshold voltage that defines the startpoint of the rising cell or net delay calculation.

Data Types

float

Default 50.0

Description

This application option specifies the threshold voltage that defines the startpoint of the rising cell or net delay calculation. The value is a percentage of the voltage source. Allowed values are 0.0 through 100.0, inclusive.

This option is one of eight application options that affect delay and transition time computations for detailed RC networks.

- *time.port_slew_lower_threshold_rise* (default is 20.0) •
time.port_slew_lower_threshold_fall (default is 20.0) •
time.port_slew_upper_threshold_rise (default is 80.0) •
time.port_slew_upper_threshold_fall (default is 80.0) • *time.port_input_threshold_rise*
(default is 50.0) • *time.port_input_threshold_fall* (default is 50.0) •
time.port_output_threshold_rise (default is 50.0) • *time.port_output_threshold_fall*
(default is 50.0)

These application options interpret the cell delays and transition times from the reference library. The trip-point values specified in the library override the values specified by these options, so normally it should not be necessary to set the options. The specified values are applied only to libraries that do not contain trip-point specifications.

The default values specify that a cell delay is defined from 50 percent of the voltage value for the input transition to 50 percent of the voltage value for the output transition. The default values also specify that a transition time, or slew, is defined from 20 percent to 80 percent of the voltage.

To determine the current value of this application option, use the *get_app_option_value -name time.port_input_threshold_rise* command. For a list of all calc application options and their current values, use the *report_app_options calc.** command.

t

Examples

The following example specifies that cell delays are computed from 50 percent of the input transition to 55 percent of the output transition. In addition, the example specifies that transition times represent the delay from 10 percent to 90 percent of the voltage source.

```
prompt> set_app_options -list {
    {time.port_slew_lower_threshold_fall 10}
    {time.port_slew_lower_threshold_rise 10}
    {time.port_slew_upper_threshold_fall 90}
    {time.port_slew_upper_threshold_rise 90}
    {time.port_input_threshold_rise      50}
    {time.port_input_threshold_fall      50}
    {time.port_output_threshold_rise     55}
    {time.port_output_threshold_fall     55}
}
```

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

time.port_output_threshold_fall

Specifies the threshold voltage that defines the endpoint of the falling cell or net delay calculation.

Data Types

float

Default 50.0

Description

This application option specifies the threshold voltage that defines the endpoint of the falling cell or net delay calculation. The value is a percentage of the voltage source. Allowed values are 0.0 through 100.0, inclusive.

This option is one of eight application options that affect delay and transition time computations for detailed RC networks.

- *time.port_slew_lower_threshold_rise* (default is 20.0) •
- *time.port_slew_lower_threshold_fall* (default is 20.0) •
- *time.port_slew_upper_threshold_rise* (default is 80.0) •
- *time.port_slew_upper_threshold_fall* (default is 80.0) • *time.port_input_threshold_rise*

t

(default is 50.0) • *time.port_input_threshold_fall* (default is 50.0) •
time.port_output_threshold_rise (default is 50.0) • *time.port_output_threshold_fall*
 (default is 50.0)

These application options interpret the cell delays and transition times from the reference library. The trip-point values specified in the library override the values specified by these options, so normally it should not be necessary to set the options. The specified values are applied only to libraries that do not contain trip-point specifications.

The default values specify that a cell delay is defined from 50 percent of the voltage value for the input transition to 50 percent of the voltage value for the output transition. The default values also specify that a transition time, or slew, is defined from 20 percent to 80 percent of the voltage.

To determine the current value of this application option, use the *get_app_option_value -name time.port_output_threshold_fall* command. For a list of all calc application options and their current values, use the *report_app_options calc.** command.

Examples

The following example specifies that cell delays are computed from 50 percent of the input transition to 55 percent of the output transition. In addition, the example specifies that transition times represent the delay from 10 percent to 90 percent of the voltage source.

```
prompt> set_app_options -list {
    {time.port_slew_lower_threshold_fall 10}
    {time.port_slew_lower_threshold_rise 10}
    {time.port_slew_upper_threshold_fall 90}
    {time.port_slew_upper_threshold_rise 90}
    {time.port_input_threshold_rise      50}
    {time.port_input_threshold_fall      50}
    {time.port_output_threshold_rise     55}
    {time.port_output_threshold_fall     55}
}
```

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

time.port_output_threshold_rise

Specifies the threshold voltage that defines the endpoint of the rising cell or net delay calculation.

t

Data Types

float

Default 50.0

Description

This application option specifies the threshold voltage that defines the endpoint of the rising cell or net delay calculation. The value is a percentage of the voltage source. Allowed values are 0.0 through 100.0, inclusive.

This option is one of eight application options that affect delay and transition time computations for detailed RC networks.

- *time.port_slew_lower_threshold_rise* (default is 20.0) •
- time.port_slew_lower_threshold_fall* (default is 20.0) •
- time.port_slew_upper_threshold_rise* (default is 80.0) •
- time.port_slew_upper_threshold_fall* (default is 80.0) •
- time.port_input_threshold_rise* (default is 50.0) •
- time.port_input_threshold_fall* (default is 50.0) •
- time.port_output_threshold_rise* (default is 50.0) •
- time.port_output_threshold_fall* (default is 50.0)

These application options interpret the cell delays and transition times from the reference library. The trip-point values specified in the library override the values specified by these options, so normally it should not be necessary to set the options. The specified values are applied only to libraries that do not contain trip-point specifications.

The default values specify that a cell delay is defined from 50 percent of the voltage value for the input transition to 50 percent of the voltage value for the output transition. The default values also specify that a transition time, or slew, is defined from 20 percent to 80 percent of the voltage.

To determine the current value of this application option, use the *get_app_option_value -name time.port_output_threshold_rise* command. For a list of all calc application options and their current values, use the *report_app_options calc.** command.

Examples

The following example specifies that cell delays are computed from 50 percent of the input transition to 55 percent of the output transition. In addition, the example specifies that transition times represent the delay from 10 percent to 90 percent of the voltage source.

```
prompt> set_app_options -list {
    {time.port_slew_lower_threshold_fall 10}
    {time.port_slew_lower_threshold_rise 10}
    {time.port_slew_upper_threshold_fall 90}
    {time.port_slew_upper_threshold_rise 90}
    {time.port_input_threshold_rise      50}
    {time.port_input_threshold_fall      50}
```

t

```

    {time.port_output_threshold_rise      55}
    {time.port_output_threshold_fall     55}
}

```

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

time.port_slew_derate_from_library

Specifies the derating needed for the transition times in the reference library to match the transition times between the characterization trip points.

Data Types

float

Default 1.0**Description**

This application option specifies a floating-point number between 0.0 and 1.0 that indicates the derating needed for the transition times in the reference library to match the transition times between the characterization trip points.

The default is 1.0, which means that the transition times in the reference library are used without change.

The slew-derating values specified in the reference library override this value, so it should not be necessary to set this application option. The specified value applies only to libraries that do not contain slew-derating specifications.

Use a slew-derating value of 1.0 if the transition times specified in the library represent the exact transition times between the characterization trip points, which is usually the case.

Use a slew-derating value of less than 1.0 for libraries where the transition times have been extrapolated to the rail voltages. For example, if the transition times are characterized as between 30 percent and 70 percent and then extrapolated to the rails, the slew-derating value is $0.4 = (70 - 30) / 100$.

To determine the current value of this application option, use the *get_app_option_value -name time.port_slew_derate_from_library* command. For a list of all calc application options and their current values, use the *report_app_options calc.** command.

t

Examples

The following example specifies that transition times were computed by measuring the delay from 30 percent to 70 percent of the voltage source and then multiplying the measured transition times by $2.5 = (100-0)/(70-30)$ to extrapolate to 0-100 percent of the rail voltages.

```
prompt> set_app_options \\  
        -name time.port_slew_derate_from_library -value 0.4
```

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

time.port_slew_lower_threshold_fall

Specifies the threshold voltage that defines the endpoint of the falling slew calculation.

Data Types

float

Default 20.0

Description

This application option specifies the threshold voltage that defines the endpoint of the falling slew calculation. The value is a percentage of the voltage source. Allowed values are 0.0 through 100.0, inclusive.

This option is one of eight application options that affect delay and transition time computations for detailed RC networks.

- *time.port_slew_lower_threshold_rise* (default is 20.0) •
time.port_slew_lower_threshold_fall (default is 20.0) •
time.port_slew_upper_threshold_rise (default is 80.0) •
time.port_slew_upper_threshold_fall (default is 80.0) • *time.port_input_threshold_rise*
(default is 50.0) • *time.port_input_threshold_fall* (default is 50.0) •
time.port_output_threshold_rise (default is 50.0) • *time.port_output_threshold_fall*
(default is 50.0)

These application options interpret the cell delays and transition times from the reference library. The trip-point values specified in the library override the values specified by these options, so normally it should not be necessary to set the options. The specified values are applied only to libraries that do not contain trip-point specifications.

t

The default values specify that a cell delay is defined from 50 percent of the voltage value for the input transition to 50 percent of the voltage value for the output transition. The default values also specify that a transition time, or slew, is defined from 20 percent to 80 percent of the voltage.

To determine the current value of this application option, use the *get_app_option_value -name time.port_slew_lower_threshold_fall* command. For a list of all calc application options and their current values, use the *report_app_options calc.** command.

Examples

The following example specifies that cell delays are computed from 50 percent of the input transition to 55 percent of the output transition. In addition, the example specifies that transition times represent the delay from 10 percent to 90 percent of the voltage source.

```
prompt> set_app_options -list {
    {time.port_slew_lower_threshold_fall 10}
    {time.port_slew_lower_threshold_rise 10}
    {time.port_slew_upper_threshold_fall 90}
    {time.port_slew_upper_threshold_rise 90}
    {time.port_input_threshold_rise 50}
    {time.port_input_threshold_fall 50}
    {time.port_output_threshold_rise 55}
    {time.port_output_threshold_fall 55}
}
```

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

time.port_slew_lower_threshold_rise

Specifies the threshold voltage that defines the startpoint of the rising slew calculation.

Data Types

float

Default 20.0

Description

This application option specifies the threshold voltage that defines the startpoint of the rising slew calculation. The value is a percentage of the voltage source. Allowed values are 0.0 through 100.0, inclusive.

t

This option is one of eight application options that affect delay and transition time computations for detailed RC networks.

- *time.port_slew_lower_threshold_rise* (default is 20.0) •
- *time.port_slew_lower_threshold_fall* (default is 20.0) •
- *time.port_slew_upper_threshold_rise* (default is 80.0) •
- *time.port_slew_upper_threshold_fall* (default is 80.0) •
- *time.port_input_threshold_rise* (default is 50.0) •
- *time.port_input_threshold_fall* (default is 50.0) •
- *time.port_output_threshold_rise* (default is 50.0) •
- *time.port_output_threshold_fall* (default is 50.0)

These application options interpret the cell delays and transition times from the reference library. The trip-point values specified in the library override the values specified by these options, so normally it should not be necessary to set the options. The specified values are applied only to libraries that do not contain trip-point specifications.

The default values specify that a cell delay is defined from 50 percent of the voltage value for the input transition to 50 percent of the voltage value for the output transition. The default values also specify that a transition time, or slew, is defined from 20 percent to 80 percent of the voltage.

To determine the current value of this application option, use the *get_app_option_value -name time.port_slew_lower_threshold_rise* command. For a list of all calc application options and their current values, use the *report_app_options calc.** command.

Examples

The following example specifies that cell delays are computed from 50 percent of the input transition to 55 percent of the output transition. In addition, the example specifies that transition times represent the delay from 10 percent to 90 percent of the voltage source.

```
prompt> set_app_options -list {
    {time.port_slew_lower_threshold_fall 10}
    {time.port_slew_lower_threshold_rise 10}
    {time.port_slew_upper_threshold_fall 90}
    {time.port_slew_upper_threshold_rise 90}
    {time.port_input_threshold_rise      50}
    {time.port_input_threshold_fall      50}
    {time.port_output_threshold_rise     55}
    {time.port_output_threshold_fall     55}
}
```

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

t

time.port_slew_upper_threshold_fall

Specifies the threshold voltage that defines the startpoint of the falling slew calculation.

Data Types

float

Default 80.0

Description

This application option specifies the threshold voltage that defines the startpoint of the falling slew calculation. The value is a percentage of the voltage source. Allowed values are 0.0 through 100.0, inclusive.

This option is one of eight application options that affect delay and transition time computations for detailed RC networks.

- *time.port_slew_lower_threshold_rise* (default is 20.0) •
- *time.port_slew_lower_threshold_fall* (default is 20.0) •
- *time.port_slew_upper_threshold_rise* (default is 80.0) •
- *time.port_slew_upper_threshold_fall* (default is 80.0) •
- *time.port_input_threshold_rise* (default is 50.0) •
- *time.port_input_threshold_fall* (default is 50.0) •
- *time.port_output_threshold_rise* (default is 50.0) •
- *time.port_output_threshold_fall* (default is 50.0)

These application options interpret the cell delays and transition times from the reference library. The trip-point values specified in the library override the values specified by these options, so normally it should not be necessary to set the options. The specified values are applied only to libraries that do not contain trip-point specifications.

The default values specify that a cell delay is defined from 50 percent of the voltage value for the input transition to 50 percent of the voltage value for the output transition. The default values also specify that a transition time, or slew, is defined from 20 percent to 80 percent of the voltage.

To determine the current value of this application option, use the *get_app_option_value -name time.port_slew_upper_threshold_fall* command. For a list of all calc application options and their current values, use the *report_app_options calc.** command.

Examples

The following example specifies that cell delays are computed from 50 percent of the input transition to 55 percent of the output transition. In addition, the example specifies that transition times represent the delay from 10 percent to 90 percent of the voltage source.

```
prompt> set_app_options -list {
      {time.port_slew_lower_threshold_fall 10}
```

t

```

    {time.port_slew_lower_threshold_rise 10}
    {time.port_slew_upper_threshold_fall 90}
    {time.port_slew_upper_threshold_rise 90}
    {time.port_input_threshold_rise      50}
    {time.port_input_threshold_fall      50}
    {time.port_output_threshold_rise     55}
    {time.port_output_threshold_fall     55}
}

```

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

time.port_slew_upper_threshold_rise

Specifies the threshold voltage that defines the endpoint of the rising slew calculation.

Data Types

float

Default 80.0

Description

This application option specifies the threshold voltage that defines the endpoint of the rising slew calculation. The value is a percentage of the voltage source. Allowed values are 0.0 through 100.0, inclusive.

This option is one of eight application options that affect delay and transition time computations for detailed RC networks.

- *time.port_slew_lower_threshold_rise* (default is 20.0) •
- time.port_slew_lower_threshold_fall* (default is 20.0) •
- time.port_slew_upper_threshold_rise* (default is 80.0) •
- time.port_slew_upper_threshold_fall* (default is 80.0) •
- time.port_input_threshold_rise* (default is 50.0) •
- time.port_input_threshold_fall* (default is 50.0) •
- time.port_output_threshold_rise* (default is 50.0) •
- time.port_output_threshold_fall* (default is 50.0)

These application options interpret the cell delays and transition times from the reference library. The trip-point values specified in the library override the values specified by these options, so normally it should not be necessary to set the options. The specified values are applied only to libraries that do not contain trip-point specifications.

t

The default values specify that a cell delay is defined from 50 percent of the voltage value for the input transition to 50 percent of the voltage value for the output transition. The default values also specify that a transition time, or slew, is defined from 20 percent to 80 percent of the voltage.

To determine the current value of this application option, use the *get_app_option_value -name time.port_slew_upper_threshold_rise* command. For a list of all calc application options and their current values, use the *report_app_options calc.** command.

Examples

The following example specifies that cell delays are computed from 50 percent of the input transition to 55 percent of the output transition. In addition, the example specifies that transition times represent the delay from 10 percent to 90 percent of the voltage source.

```
prompt> set_app_options -list {
    {time.port_slew_lower_threshold_fall 10}
    {time.port_slew_lower_threshold_rise 10}
    {time.port_slew_upper_threshold_fall 90}
    {time.port_slew_upper_threshold_rise 90}
    {time.port_input_threshold_rise      50}
    {time.port_input_threshold_fall      50}
    {time.port_output_threshold_rise     55}
    {time.port_output_threshold_fall     55}
}
```

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

time.propagate_interclock_uncertainty

Enable propagation of launch clock uncertainty through latch loop breakers to end point.

Data Types

Boolean

Default false

Description

When *false*, (the default), the interclock uncertainty is calculated for each latch-to-latch path independently, from the clock at the launch latch to the clock at the capture latch, even when latches operate in transparent mode.

t

When true, clock uncertainty information is propagated through each latch operating in transparent mode or in latch loop breaker mode as though it were a combinational element. This allows an entire sequence of latch-to-latch stages to be considered a single path for interclock uncertainty calculation, provided that time borrowing occurs at the endpoint of each intermediate stage.

Operating with this variable set to true can lead to more accurate results for designs containing transparent latches, at the cost of some CPU time and memory resources.

For example, consider a pipeline containing latches A, B, and C, clocked by clocks 1, 2, and 3, respectively. The tool treats the paths between A and B and between B and C as distinct. In reality, however, if latch B is in transparent mode or latch loop breaker, data passes through it as though it were a combinational element or borrow from next stage. Regardless of whether interclock uncertainty has been applied between clocks 1 and 3, the default behavior is to apply the uncertainty between clocks 2 and 3 when calculating slack at latch C. It is more accurate, however, to apply the uncertainty between the clock at the path startpoint (clock 1, latch A) and the clock at the path endpoint (clock 3, latch C), if defined.

When set to *true*, the correct interclock uncertainty is applied, as though the path from latch A to latch C through the transparent latch B were a single, extended path. That is, the uncertainty is propagated through the transparent latch. To find out the startpoint of this extended path, use `report_timing -trace_latch_borrow`.

Application options are set using the `set_app_options` command, and they may be specified with respect to a design. To determine the current value of this application option, use:

```
get_app_option_value -user_default -name
time.propagate_interclock_uncertainty
Use report_app_options to show the value of multiple application options.
```

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)
- [report_timing](#)
- [set_clock_uncertainty](#)

time.rc_receiver_modeling_effort

Controls the computational effort for receiver modeling algorithm in PrimeTime delay calculation.

t

Data Types*string***Default** *high***Description**

When the app option *time.delay_calc_enhanced_ccsn_waveform_analysis* is set to *true* to enable enhanced waveform propagation analysis, PTDC performs adaptive receiver modeling to integrate CCS noise models with library receiver models (C1CN).

This app option controls how much effort is used for receiver modeling. Available options for this variable are *high* (the default) and *low*.

The default value of *high* provides the best accuracy. Normally, receiver modeling takes only a small fraction of the timing update runtime and the computational effort is minimal.

However, for large scaling library groups, the runtime for receiver modeling can be higher. To reduce the runtime in this case, you can set this variable to *low*.

This variable has an effect only when enhanced waveform propagation is enabled.

```
# enable advanced waveform propagation (default is disabled)
set_app_options -name time.delay_calc_waveform_analysis_mode -value
  full_design

# enable enhanced advanced waveform propagation (default is false)
set_app_options -name time.delay_calc_enhanced_ccsn_waveform_analysis
  -value true
```

See Also

- [update_timing](#)

time.remove_clock_reconvergence_pessimism

Enables or disables clock reconvergence pessimism removal.

Data Types*Boolean***Default** *false***Description**

When this option is set to *true*, the tool removes clock reconvergence pessimism from the slack calculation.

t

Clock reconvergence pessimism (CRP) is the difference in delay along the common part of the launching and capturing clock paths. The most common causes of CRP are reconvergent paths in the clock network, and the difference in early and late arrivals in the clock network.

CRP is independently calculated for rise and fall clock paths. You can use the `time.clock_reconvergence_pessimism` option to control CRP calculation method with respect to transition sense.

If the capturing device is a level-sensitive latch, two CRP values will be calculated:

- `crp_open`, which is the CRP corresponding to the opening edge of the latch
- `crp_close`, which is the CRP corresponding to the closing edge of the latch

CRP does not support paths that fan out directly from clock source pins to the data pins of sequential devices. To enable support for such paths, set the `time.crpr_remove_clock_to_data_crp` option to true.

Application options are set using the `set_app_options` command, and they may be specified globally or with respect to a design. To determine the current value of this application option, use the following command:

```
get_app_option_value -name time.remove_clock_reconvergence_pessimism
```

Use `report_app_options` to show the value of multiple application options.

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [report_crpr](#)
- [set_app_options](#)

time.reparent_enable_inherit_constraints

Enables preserving the timing constraint from hierarchical-parent-pins to move-cell-pins during reparent.

Data Types

Boolean

Default false

t

Description

Previously, if timing constraints were defined on a hierarchical-parent-pin after reparenting the move-cell to another hierarchy, the original timing constraint was not preserved for the move-cell.

When this application option is set to true, the tool copies those timing constraints defined in the original hierarchical-parent-pins of the move-cell to the move-cell-pins to avoid timing changes.

When this application option is set to false (the default), the tool does not preserve the timing constraint for move-cell, like in the previous behavior.

See Also

- [reparent_cells](#)

time.report_through_paths_as_non_io**Data Types**

Boolean

Default false

Description

When this application option is set to *true*, `report_global_timing` to report a in2reg latch through path in reg2reg category similar to `report_timing`

See Also

- [report_timing](#)
- [get_timing_paths](#)

time.report_timing_column_order

Specifies the column order for timing report.

Data Types

List

Default null

t

Description

This application option specifies the column orders of the timing report. You can specify a list of column names with specified orders, then the timing report will be generated as you specified.

The valid column names are: "Point", "Incr", "Path", "Cap", "Fanout", "Trans", "Derate", "Mean", "Sensit".

When this option is not set, the timing report is in a default column order.

Application options are set using the `set_app_options` command, and they may be specified globally or with respect to a design. To determine the current value of this application option, use:

```
get_app_option_value -name time.report_timing_column_order.  
Use report_app_options to show the value of multiple application options.
```

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)
- [report_timing](#)

time.report_unconstrained_paths

Specifies whether the tool searches for unconstrained paths when you use the `report_timing` or `get_timing_paths` command.

Data Types

Boolean

Default false

Description

When this application option is set to *false* (the default), the `report_timing` and `get_timing_paths` commands search only for constrained paths, and do not waste any time searching for unconstrained paths.

When this application option is set to *true*, the `report_timing` and `get_timing_paths` commands will continue searching for unconstrained paths when constrained paths cannot be found for some endpoints. Searching unconstrained paths can cause significantly longer runtimes.

t

Note that the *report_timing* and *get_timing_paths* commands only search for unconstrained paths to endpoints that have no constrained paths satisfying the path search options.

See Also

- [get_timing_paths](#)
- [report_timing](#)

time.report_with_clock_path_edrc

Report with clock path electrical drc constraints.

Data Types

Boolean

Default true

Description

Clock path constraints are set on clocks using *set_max_transition* and *set_max_capacitance* commands with option, *-clock_path*.

When this global option is set to *false*, the report commands, *report_constraint* and *report_qor*, ignore clock path constraints on clocks in reporting *max_transition* and *max_capacitance* violations. This is non-default reporting. When *max_transition* and *max_capacitance* are reported, such reports are generated with a comment, at the top of the report, as follows:

"Generated with option *time.report_with_clock_path_edrc false*."

When set to *true*, it generates normal(default) reports considering clock path constraints.

This can be used to generate reports that are comparable to other tools where clock path constraints are not supported in report commands.

The app option, when set to *true*(default), does not have any runtime overhead. When set to *false*, the report commands check and recompute costs of some nets to generate new costs and violation counts. This will increase runtime for non-default reports. But, this does not invalidate stored costs, so rest of the flow has no runtime impact.

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design. To determine the current value of this application option, use:

```
get_app_option_value -name time.report_with_clock_path_edrc.
Use report_app_options to show the value of multiple application options.
```

t

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)
- [report_qor](#)
- [report_constraint](#)
- [set_max_transition](#)
- [set_max_capacitance](#)

time.rh_file_use_clock_arrival

Use clock arrival window data for pins on clock network during write_rh_file.

Data Types

Boolean

Default false

Description

This application option controls which arrival window value shall be used for pins on clock network during write_rh_file (or write_timing_file command, which is an alias of write_rh_file).

If this application option is set to true, "clock_arrival_window" value will be used for timing window entry in RH file. That is the {min/max}_{rise/fall} values in the line of: T <pin_name> <is_clock> <min_rise> <max_rise> <min_fall> <max_fall> ... When set it false, "arrival_window" attribute value is used for the entry.

From V-2023.12 release, the default value of the app-option is set to false. This is to align the default behavior of same-name command in PT, to eliminate potential correlation issue in RedHawk enabled flow.

Use the *get_app_option_value -name time.rh_file_use_clock_arrival* command to get the current value of this application option.

Use the *get_app_option_value -name time.rh_file_use_clock_arrival* command to retrieve the current value.

Use the *set_app_options -as_user_default {time.rh_file_use_clock_arrival true|false}* command to set a new value.

Use the *report_app_options* command to show the value of multiple application options.

t

See Also

- [get_app_option_value](#)
- [set_app_options](#)

time.save_block_exclude_file_line_info

Excludes the file_line_info from being archived into NDM.

Data Types

Boolean

Default false

Description

This application option controls whether or not the file_line_info from being archived when constraint are to be saved into NDM. So the next time constraint is loaded from the disk, write_sdc will not contain file_line_info. The default (false) is that file_line_info will be archived.

Use the *get_app_option_value -design \$design -name time.save_block_exclude_file_line_info* command to get the current value of this application option on a specific design.

Use the *get_app_option_value -name time.save_block_exclude_file_line_info* command to retrieve the current value.

Use the *set_app_options -name time.save_block_exclude_file_line_info -value true|false* command to set a new value.

Use the *report_app_options* command to show the value of multiple application options.

See Also

- [get_app_option_value](#)
- [set_app_options](#)

time.save_mode_based_cts_exceptions

Specifies whether the tool saves cts exception constraints in older builds as moded constraints.

Data Types

Boolean

t

Default false**Description**

When this application option is set to *true*, cts exception constraints are saved as moded constraints.

See Also

- [write_script](#)
- [write_sdc](#)

time.scaling_calculation_compatibility

Enables the PrimeTime Delay Calculation Nonlinear Scaling Optimization feature to achieve better accuracy.

Data Types

Boolean

Default true**Description**

This app option is equivalent to the Primetime application variable "scaling_calculation_compatibility" and enables more accurate multiple libraries nonlinear scaling. To be consistent with PrimeTime, the multiple libraries nonlinear scaling feature is enabled by specifying a value of "false" for this app option.

Once this feature is enabled in a session, change this app option value will have no effect. If you wish to disable this feature once enabled, you are required to start a new session of the tool.

Use the *get_app_option_value -design \$design -name time.scaling_calculation_compatibility* command to get the current value of this application option on a specific design.

Use the *get_app_option_value -name time.scaling_calculation_compatibility* command to retrieve the current value.

Use the *set_app_options -as_user_default {time.scaling_calculation_compatibility true|false}* command to set a new value.

Use the *report_app_options* command to show the value of multiple application options.

See Also

- [get_app_option_value](#)
- [set_app_options](#)

time.scenario_auto_name_separator

Specifies the string used to format auto-generated scenario names.

Data Types

string

Default "::"

Description

If *create_scenario* is used to create a scenario without specifying the *-name* option, the new name for the scenario will be auto-generated by concatenating the mode name, the separator string, and the corner name. The default separator string is "::", so a command like *create_scenario -mode modeX -corner cornerY* creates a scenario named "modeX::cornerY". This app option can be used to specify a different separator string, such as "/", instead of "::".

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design. To determine the current value of this application option on a specific design, use *get_app_option_value -block \$block -name timer.enable_preset_clear_arcs*. Use *report_app_options* to show the value of multiple application options.

See Also

- [create_scenario](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [report_scenarios](#)
- [set_app_options](#)

time.si_enable_analysis

Enables crosstalk analysis during timing analysis.

t

Data Types*Boolean***Default** false**Description**

This application option controls crosstalk analysis during timing analysis. By default, crosstalk analysis is not enabled. Turning this option on clears the existing delay data and any stored parasitics in the timer. When set to "true", this application option also sets up the extraction of the routed nets along with their coupling caps.

Application options are set using the *set_app_options* command, and they may be specified globally or with respect to a design.

Use *get_app_option_value -design \$design -name time.si_enable_analysis* to get the current value of this application option on a specific design.

Use *get_app_option_value -name time.si_enable_analysis* to retrieve the current value. Use *set_app_options -as_user_default {time.si_enable_analysis true|false}* to set the new value. Use *report_app_options* to show the value of multiple application options.

See Also

- [get_app_option_value](#)
- [set_app_options](#)

time.si_filter_per_aggr_noise_peak_ratio

Specifies the threshold for the voltage bump introduced by an aggressor at a victim node, divided by VDD, below which the aggressor net can be filtered out.

Data Types

float

Default 0.01**Description**

Specifies the threshold for the voltage bump introduced by an aggressor at a victim node; the default is 0.01. The tool uses the *time.si_filter_per_aggr_noise_peak_ratio* and *time.si_filter_accum_aggr_noise_peak_ratio* application options to determine whether an aggressor net can be filtered.

t

An aggressor net, along with its coupling capacitors, is filtered when either of the following are true:

- The peak voltage of the voltage bump induced on the victim net divided by VDD is less than the value specified with the *time.si_filter_per_aggr_noise_peak_ratio* application option.
- The accumulated peak voltage of voltage bumps induced on the victim by aggressors to the victim net divided by VDD is less than the value specified with the *time.si_filter_accum_aggr_noise_peak_ratio* application option.

time.si_ignore_input_data_arrival

Ignore input data path arrival time and make an infinite timing window in signal integrity analysis.

Data Types

bool

Default false

Description

Ignore input data path arrival time and make an infinite timing window in signal integrity analysis.

time.si_xtalk_composite_aggr_mode

Enables composite aggressor in SI crosstalk analysis.

Data Types

String

Default disabled

Description

This application option enables the composite aggressor mode in SI crosstalk analysis to avoid over filtering. Traditionally, a victim net's aggressors with bump height below a user defined threshold will be filtered/screened during SI crosstalk analysis. With this application option enabled, filtered aggressors are grouped into a virtual composite aggressor, thus it provides a safer filtering mechanism. Statistical mode is supported to further reduce pessimism. User can set *time.si_xtalk_composite_aggr_quantile_high_pct* to consider the probability that all filtered aggressors will switch simultaneously to adversely affect the victim.

t

Application options are set using the `set_app_options` command, and they may be specified globally or with respect to a design.

Use `get_app_option_value -design $design -name time.si_xtalk_composite_aggr_mode` to get the current value of this application option on a specific design.

Use `get_app_option_value -name time.si_xtalk_composite_aggr_mode` to retrieve the current value. Use `set_app_options -as_user_default {time.si_xtalk_composite_aggr_mode statistical}` to turn on composite aggressor analysis mode.

See Also

- [get_app_option_value](#)
- [set_app_options](#)

time.si_xtalk_composite_aggr_noise_peak_ratio

Control the composite aggressor selection in crosstalk analysis.

Data Types

float

Default 0.01

Description

This application option specifies the threshold value as a ratio of the crosstalk bump to VDD. Below this threshold, aggressors are selected into the composite aggressor group. The default is 0.01, which means that all aggressor nets with crosstalk-bump-to-VDD ratio less than 0.01 is selected into the composite aggressor group.

This application option works with other filtering thresholds to determine which aggressors are selected into the composite aggressor group.

time.si_xtalk_composite_aggr_quantile_high_pct

Controls the composite aggressor creation for statistical analysis.

Data Types

float

Default 99.73

t

Description

This application option specifies the probability in percentage format that any given real combined bump height is less than or equal to the computed composite aggressor bump height. Given the specified probability, the resulting quantile value for the composite aggressor bump height is calculated.

The default of this application option is 99.73, which corresponds to a 3-sigma probability that the real bump height from any randomly-chosen combination of aggressors is covered by the composite aggressor bump height.

Use *get_app_option_value -design \$design -name time.si_xtalk_composite_aggr_quantile_high_pct* to get the current value of this application option on a specific design. Use *get_app_option_value -name time.si_xtalk_composite_quantile_high_pct* to retrieve the current value.

time.snapshot_storage_location

Specifies the search location for the *create_qor_snapshot*, *remove_qor_snapshot*, *report_qor_snapshot* and *query_qor_snapshot* utilities.

Data Types

string

Default snapshot

Description

This application option specifies the location in which to store snapshot data for the *create_qor_snapshot* command. The *report_qor_snapshot* command generates and reports snapshot data from this location. The *query_qor_snapshot* command generates and reports snapshot data from this location. The *remove_qor_snapshot* command deletes snapshot(s) from this location.

See Also

- [create_qor_snapshot](#)
- [remove_qor_snapshot](#)
- [report_qor_snapshot](#)
- [query_qor_snapshot](#)

t

time.special_path_group_precedence

Specifies the precedence rules to use for placing a path into the special path groups (*****async_default***** and *****clock_gating_default*****).

Data Types

enumerated

Default *native*

Description

This application option specifies the precedence rules used to assign asynchronous and clock-gating paths to path groups when the special path groups (*****async_default***** and *****clock_gating_default*****) are enabled.

The IC Compiler II tool supports the following precedence rules for assigning asynchronous and clock-gating paths to path groups when the special path groups are enabled:

native (the default)

User-defined path groups have higher precedence than the special path groups, so paths in user-defined groups are not moved to a special path group.

primetime

The special path groups have the highest precedence, so all asynchronous and clock-gating paths are moved to a special path group. This setting changes the path assignment to match the behavior of the PrimeTime tool.

By default, the *time.use_special_default_path_groups* application option is *false*, and the IC Compiler II tool does not use the special path groups, so the *time.special_path_group_precedence* application option has no effect.

Examples

The following example uses native IC Compiler II precedence with user control of special path grouping.

```
prompt> set_app_options -name time.use_special_default_path_groups \\  
-value true  
prompt> reset_app_options {time.special_path_group_precedence}  
prompt> group_path -name ICG -to [get_pins -hierarchical \\  
-filter is_clock_gating_enable]  
prompt> sizeof_collection [get_timing_paths \\  
-groups **clock_gating_default**]  
0
```

t

The following example uses PrimeTime-like special path group precedence:

```
prompt> set_app_options -name time.use_special_default_path_groups \\  
        -value true  
prompt> set_app_options -name timer.special_path_group_precedence \\  
        -value primetime  
prompt> sizeof_collection [get_timing_paths \\  
        -groups {**clock_gating_default**}]  
666
```

See Also

- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

time.through_path_max_segments

Specifies the maximum number of latches for reporting paths through latches.

Data Types

integer

Default 5

Description

When the *time.through_path_max_segments* application option is set to a nonzero value, the tool selects some latch data pins on paths with many transparent latches to behave as they would; this limits the reporting on the long path but speeds up analysis.

With a small nonzero value, the tool selects many transparent latch data pins in the design. With a larger value, the tool selects fewer latch data pins.

When you set *time.through_path_max_segments* to 0, the tool selects no latch data pins. Reporting on paths through many latches is allowed, but analysis might be slower.

time.timing_report_always_use_valid_start_end_points

Requires the -from, -rise_from, and -fall_from options to specify valid timing startpoints and the -to, -rise_to, and -fall_to options to specify valid timing endpoints.

Data Types

Boolean

Default true

t

Description

This variable affects the way the `report_timing`, `report_bottleneck`, and `get_timing_path` commands interpret the `-from`, `-rise_from`, `-fall_from`, `-to`, `-rise_to`, and `-fall_to` options.

When set to false, the `from_list` objects are interpreted as all pins found by given objects. Objects specified with an asterisk (*) wildcard or cell name might return invalid startpoints or end-points. For example, `-from [get_pins FF/*]` includes input, output, and asynchronous pins. The tool considers these invalid startpoints or end-points to be throughpoints and continues the path searching. Although it is convenient to use, it is not suggested because of longer run-times.

To report only valid startpoints and endpoints, set this variable to true (the default). It is always suggested to use input ports or register clock pins for the `from_list` objects and output ports or register datapins for the `to_list` objects.

See Also

- [get_timing_paths](#)
- [report_timing](#)

time.ungroup_allow_new_buffer

Enables ungrouping to automatically add new buffers to preserve timing constraints defined on hierarchical cells or pins.

Data Types

Boolean

Default false

Description

During ungroup, tool will figure out appropriate replacement leaf pins to receive the timing constraints copied from the original hierarchical pins or cells.

In some situations, if tool can't find any replacement pins, ungroup will be rejected unless you enable this application option to allow tool automatically adding new buffer as replacement pins.

When this application option has been set as true, if tool still could find appropriate replacement pins to preserve original timing constraints, then no buffer will be added. But, if tool can't find appropriate replacement pins, then tool will automatically add one new buffer and then copy the timing-constraints defined upon original hierarchical pins/cells to this new-added buffer.

When this application option has been set as false, then tool will never add any new buffer during ungroup.

The default value is false.

time.ungroup_enable_inherit_constraints

Enables preservation of timing constraints defined on hierarchical cells and pins during ungroup.

Data Types

Boolean

Default true

Description

Previously, if timing constraints have been defined on hierarchical pins and cells, ungroup will error out directly.

When this application option has been set as true, then tool will move those timing constraints defined upon original hierarchical pins/cells to corresponding replacement leaf-pins to avoid timing changes.

If tool can't find appropriate replacement leaf-pins to preserve those timing-constraints, tool will reject ungroup request unless you allow tool to adding new buffer automatically.

Please reference 'time.ungroup_allow_new_buffer' for more details.

When this application option has been set as false, then tool will bail out directly, just like previous behavior.

The default value is true.

time.use_lib_cell_generated_clock_name

Specifies how generated clock names are derived from library files.

Data Types

Boolean

Default false

t

Description

This application option specifies how generated clock names are derived from library files; you must set this application option before linking the design:

- *false* - Derives the name of the generated clock from the name of its source pin or the first source pin name in case of multisource generated clocks.
- *true* - Takes the generated clock name directly taken from the explicit specification in the library.

See Also

- [report_clock](#)

time.use_pt_delay

Enables PrimeTime Delay Calculation during timing analysis.

Data Types

Boolean

Default false

Description

This application option controls which delay calculator is used during timing analysis. By default, timing analysis uses the tool's native delay calculator. If this application option is turned on, timing analysis uses the PrimeTime delay calculator. Setting this option to *true* clears the existing delay data stored in the timer.

To use PrimeTime delay calculation, the library .db files must be available for the design and must be accessed using the specified search path. Further the design must be routed. If timing analysis fails to use PrimeTime delay calculation, it issues a warning (TIM-200) and reverts to the tool's native delay calculator.

Use the *get_app_option_value -design \$design -name time.use_pt_delay* command to get the current value of this application option on a specific design.

Use the *get_app_option_value -name time.use_pt_delay* command to retrieve the current value.

Use the *set_app_options -as_user_default {time.use_pt_delay true|false}* command to set a new value.

Use the *report_app_options* command to show the value of multiple application options.

See Also

- [get_app_option_value](#)
- [set_app_options](#)

time.use_pt_si

Enables Primetime crosstalk analysis during timing update.

Data Types

Boolean

Default false

Description

This application option controls if Primetime crosstalk analysis engine is used. By default, crosstalk analysis uses native engine. If this application option is turned on, Primetime crosstalk analysis engine will be used. It has higher accuracy but consumes more runtime and memory. Setting this option clears the existing delay data and any stored parasitics in the timer. This application option can only be used on routed designs with crosstalk analysis enabled.

Use *get_app_option_value -design \$design -name time.use_pt_si* to get the current value of this application option on a specific design.

Use *get_app_option_value -name time.use_pt_si* to retrieve the current value. Use *set_app_options -as_user_default {time.use_pt_si true|false}* to set the new value. Use *report_app_options* to show the value of multiple application options.

See Also

- [get_app_option_value](#)
- [set_app_options](#)

time.use_slew_variation_in_constraint_arcs

Enables transition variation impact in constraint variation in parametric on-chip variation (POCV) analysis.

Data Types

string

Default none

t

Description

You can set this application option to one of the following:

- *none* (default) - Do not consider transition variation impacts on constraint variation for setup or hold arcs.
- *setup* - Consider transition variation impact for setup arcs only.
- *hold* - Consider transition variation impact for hold arcs only.
- *setup_hold* - Consider transition variation impact for both setup and hold arcs.
- *false* - Do not consider transition variation impacts on constraint variation for setup or hold arcs. (Forward compatible)
- *true* - Consider transition variation impact for both setup and hold arcs. (Forward compatible)

This application option would only take effect when the following application options are also set:

```
icc2_shell> set_app_options -name time.pocvm_enable_analysis -value true
icc2_shell> set_app_options -name time.enable_slew_variation -value true
icc2_shell> set_app_options -name time.enable_constraint_variation -value
true
```

time.use_special_default_path_groups

Specifies whether to group paths belonging to special inbuilt path groups, such as: **async_default** and **clock_gating_default**.

Data Types

Boolean

Default false

Description

By default, the tool does not group or list paths belonging to special inbuilt path groups, such as: **async_default** and **clock_gating_default**, unlike PrmeTime, where its turned on by default and it has highest precedence.

This can be turned ON by setting the value to *true*.

When the value is set to *true*, paths belonging to **async** or **clock gating** are grouped into path group **async_default** and **clock_gating_default** respectively. Note that user specified path group has higher precedence than special inbuilt path groups and hence any path belonging to special inbuilt path group can be pulled into user specified path group.

t

In presence of special inbuild path group, the tool follows following precedence rule for assigning group paths:

In presence of different path groups, order of precedence from highest to lowest is:

a) User defined group path b) Special inbuilt group path (asyn and clock_gating only) `<set_app_options -name timer.use_special_default_path_groups -value true>` c) IO group path `<set_app_options -name timer.enable_io_path_groups -value true>` d) Default clock group

The timer.special_path_group_precedence app option is usually used to achieve exactly the PrimeTime special default path group behaviour. In presence of special_path_group_precedence set to PrimeTime, following applies: a) App option: "use_special_default_path_groups" is of no use b) User can not pull any paths belonging to async or clock gating path groups.

See Also

- [group_path](#)
- [get_app_option_value](#)
- [report_app_options](#)
- [set_app_options](#)

time.write_script_use_mode_based_cts_exceptions

Enables writing CTS Exceptions, balance-delays and ignore exceptions into mode files.

Data Types

Boolean

Default false

Description

Balance delay part of "set_clock_balance_point" command is scenario based command. In Previous releases it use to be mode based command and hence tool used to write into mode files. In order to write into old format set this app option to *true*.

When this option is set to its default value, the write_script command will write CTS-Exceptions into scenario files.

See Also

- [set_clock_balance_points](#)
- [remove_clock_balance_points](#)
- [report_clock_balance_points](#)

time.xtalk_use_small_xcap_filter

Specifies the maximum signal integrity (SI) calculation filter threshold for a victim net as a ratio between the net's total coupling capacitance and the net's total capacitance.

Data Types

float

Default 0

Description

This application option specifies the maximum SI calculation filter threshold for a victim net.

The threshold is compared with the ratio between the total coupling capacitance of the net and the total capacitance of the net. The tool performs maximum delta delay calculation on the net only if the ratio is above the threshold. Otherwise, the tool skips the calculation to reduce runtime, because the delta delay would be very small.

Application options are set using the *set_app_options* command, and they can be specified globally or with respect to a design.

Example:

```
prompt> set_app_options -name time.xtalk_use_small_xcap_filter \\  
-value 0.04
```

See Also

- [get_app_option_value](#)
- [set_app_options](#)

top_level.concurrent_top_block_eco_routing_for_tho

If set to true, both top and sub-blocks eco routing will happen concurrently as part of route_opt/hyper_route_opt in THO flow.

t

Data Types*boolean***Default** false**Description**

When this app option value is set to "true", both top and sub-blocks eco routing will happen concurrently as part of route_opt/hyper_route_opt in THO flow. This app option is effective only when top and sub-blocks eco-routing runs within route_opt/hyper_route_opt (i.e. when -detached_hier_route_eco option of init_hier_optimization is not used).

See Also

- [init_hier_optimization](#)
- [route_opt](#)
- [hyper_route_opt](#)

top_level.continue_flow_on_check_hier_design_errors

If set to true, command will continue even if top level design checks fail.

Data Types*string***Default** false**Description**

When this app option value is set to "true", command will continue even if top level design checks fail and HOPT-008 error message will be given.

See Also

- [check_hier_design](#)

top_level.enable_mib_optimize_subblocks

Enables the MIB subblocks for optimization in THO flow.

Data Types*boolean***Default** false

v

Description

When this app option value is set to true, MIB sub-blocks are enabled for optimization in THO flow.

top_level.tho_block_to_top_map_auto_clock

Set the clock mapping rule that shall be used for constraints promotion during initializations performed for pre-route and post-route Transparent Hierarchy Optimization(THO).

Data Types

String

Default {connected}

Description

The allowed values for the app-option include local, connected and all. This app-option will set the clock mapping required for clock constraints promotion during THO initializations. If this app option is not specified, then the clock map rule specified by `set_block_to_top_map` (if any) will be used. Please refer to `-auto_clock` command option of `promote_clock_data` command for more information on the app-option values.

See Also

- [init_hier_optimization](#)
- [promote_clock_data](#)
- [set_block_to_top_map](#)
- [report_block_to_top_map](#)

v

vl.flow.enable_convergence_flow

Enable convergence flow.

Data Types

boolean

Default false

v

Description

This application option specifies if the tool needs to enable performance via ladder optimization in convergence mode to prevent QoR fluctuation due to too many via ladder insertions. The default value of this application option is false.

vl.opt.enable_em_via_ladder

Enable electromigration via ladder.

Data Types

boolean

Default true

Description

This application option specifies if the tool needs to try to use electromigration via ladders during optimization process. The default value of this application option is true.

If this application option is set true, vl.opt.enable_em_via_ladder_on_clock option is ignored because this option is the superset of vl.opt.enable_em_via_ladder_on_clock.

vl.opt.enable_em_via_ladder_on_clock

TBD.

Data Types

boolean

Default false

Description

This application option specifies if the tool needs to try to use electromigration via ladders on clock pins, when vl.opt.enable_em_via_ladder is set to false. The default value of this application option is true.

If vl.opt.enable_em_via_ladder option is set to true, this application option is ignored because vl.opt.enable_em_via_ladder is the superset of this application option.

vl.opt.shrink_performance_vl

Enable shrinking performance via ladder to smaller structure.

w

Data Types

boolean

Default false

Description

This application option specifies if all performance via ladders need to be shrink to smaller via ladder constraint (one finger) due to pin access problem. The default value of the application option is false.

Noted that this application option is a short-term solution to shrink all performance via ladder conservatively.

If this application option is set true, vlinterface will transfer via ladder constraint to one finger for all performance via ladder to reduce short DRCs or pin access issues.

W

wildcards

Describes supported wildcard characters and ways in which they can be escaped.

Description

The following characters are supported as wildcards:

- Asterisks (*) substitute for a string of characters of any length.
- Question marks (?) substitute for a single character.

The following commands support wildcard characters:

```
get_app_options
get_blocks
get_bound_shapes
get_bounds
get_bundles
get_cells
get_clock_groups
get_clocks
get_corners
get_designs
get_drc_error_data
get_drc_error_type
get_drc_errors
get_edit_groups
get_ems_databases
get_ems_groups
```

w

```
get_ems_messages
get_ems_rules
get_generated_clocks
get_io_guides
get_io_rings
get_layers
get_lib_cells
get_lib_pins
get_libs
get_message_ids
get_modes
get_modules
get_nets
get_parasitic_techs
get_path_groups
get_pg_regions
get_pin_blockages
get_pin_guides
get_pins
get_placement_blockages
get_ports
get_power_domains
get_power_strategies
get_related_supply_nets
get_routing_blockages
get_routing_corridor_shapes
get_routing_corridors
get_routing_guides
get_scenarios
get_shapes
get_site_rows
get_skew_groups
get_supply_nets
get_supply_ports
get_terminals
get_tracks
get_vias
get_voltage_area_shapes
get_voltage_area
```

In addition to these commands, commands that perform an implicit get also support the wildcarding feature.

Escaping Wildcards

Wildcard characters must be escaped using double backslashes (\\) to remove their special regular expression meaning. See the EXAMPLES section for more information.

Escaping Escape Character (\\)

This is similar to that of escaping wildcard characters, but needs one escape character each to escape the escape character. See the EXAMPLES section for more information.

w

Examples

The following are examples of using wildcard characters.

Using Wildcards

The following example gets all nets in the current design that are prefixed by in and followed by any two characters:

```
prompt> get_nets in??
{"in11" "in21"}
```

The following example gets all cells in the current design that are prefixed by U and followed by a string of characters of any length:

```
prompt> get_cells U*
{"U1" "U2" "U3" "U4"}
```

Escaping Wildcards

The following are examples of escaping wildcard characters.

The following example gets design test?1 in the system.

```
prompt> get_designs {test\\?1}
{"test?1"}
```

The same example can be written using the Tcl *list* command:

```
prompt> get_designs [list {test\\?1}]
{"test?1"}
```

If neither curly braces nor the *list* command is used, the syntax is as follows:

```
prompt> get_designs test\\\\\\\\\\\\?1
{"test?1"}
```

Escaping Escape Character (\\)

The following are examples of escaping an escape character.

The following example finds design test\\1 in the system.

```
prompt> get_designs {test\\\\\\\\\\\\1}
{"test\\1"}
```

The same above example can be written using the Tcl *list* command:

```
prompt> get_designs [list {test\\\\\\\\1}]
{"test\\1"}
```

w

If neither curly braces nor the *list* command is used, the syntax is as follows:

```
prompt> get_designs test\\\\\\\\\\\\\\\\\\\\\\\\\\\\\\\\1  
{"test\\1"}
```

2

Fusion Compiler Attributes

This document describes the attributes supported by the Fusion Compiler tool.

a

alignment_marker_rule_attributes

Description of the application defined alignment_marker_rule attributes.

Description

Attributes are properties assigned to objects such as pins, cells, and nets. They can be read-only or settable. A read-only attribute is informational; you cannot set its value. To modify a settable attribute, use the *set_attribute* command.

To determine the value of an attribute, use the *get_attribute* command. To see a list of all alignment_marker_rule attributes available, use the *list_attributes -application -class alignment_marker_rule* command.

Attributes for alignment_marker_rule

This section describes the alignment_marker_rule attributes.

boundary

A settable polygon attribute that returns the blockage area. This will be a placement blockage of type alignment_marker.

full_name

A read-only string attribute that returns the full name of the alignment_marker_rule.

layer_number

A settable int attribute that returns the layer_number of alignment_marker_rule with reference_object of type layer_shape.

a

lib_cell

A settable `lib_cell` or reference cell block collection attribute that returns the `lib_cell` design or reference cell block of `alignment_marker_rule`. If a heterogeneous collection is specified, the command will return an error.

lib_cell_map

A read-only string attribute that returns the list of `lib_cells` or reference cell blocks of alignment marker cells used at each corner of the die.

margin

A settable `margin_list` attribute that returns the keepout margin values in the following order: left, bottom, right, and top. The format of a `margin_list` specification is `{l b r t}`. The margin or keepout will be applied around `reference_cells` specified for the `alignment_marker_rule`.

name

A read-only string attribute that returns the name of the `alignment_marker_rule`.

object_class

A read-only string attribute that returns the name of this object class. Returns 'alignment_marker_rule' for `alignment_marker_rule` objects.

object_id

A read-only unsigned attribute that returns the `object_id` of the `alignment_marker_rule`.

orientation

A settable string attribute (with allowed values: R0 R90 R180 R270 MX MXR90 MY MYR90) that returns the orientation of `alignment_marker_rule`.

orientation_map

A read-only string attribute that returns the orientations of the alignment marker cells used at each corner of the die.

projection_map

A settable `vector_of_pair_of_strings` attribute that returns the `projection_map` which is a list of pair of strings for the `alignment_marker_rule` of type `projection`.

purpose_number

A settable int attribute that returns the `purpose_number` of `alignment_marker_rule` with `reference_object` of type `layer_shape`.

a

quadrant

A settable string attribute (with allowed values: bottom_left bottom_right top_left top_right) that returns the quadrant with respect to the die corner where the alignment marker cell will be placed.

reference_cells

A settable lib_cell or reference cell block collection attribute that returns the reference cells whose instances are considered for rule of type blockage. The blockage or keepout will be applied around these instances depending on the margin attribute of the rule. If a heterogeneous collection is specified, the command will return an error.

reference_corner

A settable string attribute (with allowed values: bottom_left bottom_right top_left top_right) that returns the reference corner of the reference shape of alignment_marker_rule.

reference_die

A read-only string attribute (with allowed values: adjacent host invalid projection) that returns the reference die of alignment_marker_rule. The reference die is a chiplet object to define the alignment marker location.

reference_die_orientation

A settable string attribute (with allowed values: R0 R90 R180 R270 MX MXR90 MY MYR90) that returns the reference die's orientation.

reference_object

A settable string attribute (with allowed values: custom die_boundary invalid layer_shape scribe_line seal_ring stitching_zone) that returns the reference object of alignment_marker_rule. The reference object is a shape object in reference chiplet to define the alignment marker location.

reference_object_name

A settable string attribute that returns the reference object name of alignment_marker_rule for reference_object of type custom.

target_layer

A read-only string attribute that returns the target layer name of alignment_marker_rule with target_object of type layer_shape.

target_object

A read-only string attribute (with allowed values: custom die_boundary layer_shape scribe_line seal_ring stitching_zone) that returns the target object

a

of `alignment_marker_rule`. The target object is a shape object in target chiplet to define the alignment marker location.

target_object_name

A read-only string attribute that returns the target object name of `alignment_marker_rule` with `target_object` of type `custom`.

target_x_enclosure

A settable string attribute that returns the `target_x_enclosure` of `alignment_marker_rule`. It is the horizontal distance from alignment marker boundary to the vertical edge of target die across reference corner in pre-blow up die dimension. The string should specify value ranges in the form of `distance_pair_list` that contains either float or list of float pairs. For example, { (a b) [c d] [e f] (g h) } would mean : $a < x < b$, $c \leq x \leq d$, $e \leq x < f$, and $g < x \leq h$. Only non-negative float values are allowed.

target_y_enclosure

A settable string attribute that returns the `target_y_enclosure` of `alignment_marker_rule`. It is the vertical distance from alignment marker boundary to the horizontal edge of target die across reference corner in pre-blow up die dimension. The string should specify value ranges in the form of `distance_pair_list` that contains either float or list of float pairs. For example, { (a b) [c d] [e f] (g h) } would mean : $a < y < b$, $c \leq y \leq d$, $e \leq y < f$, and $g < y \leq h$. Only non-negative float values are allowed.

type

A read-only string attribute (with allowed values: `corner` `none` `projection` `step`) that returns the type of the `alignment_marker_rule`.

x_enclosure

A settable string attribute that returns the `x_enclosure` of `alignment_marker_rule`. It is the horizontal distance from alignment marker cell boundary to the vertical edge of reference die across reference corner in pre-blow up die dimension. The string should specify value ranges in the form of `distance_pair_list` that contains either float or list of float pairs. For example, { (a b) [c d] [e f] (g h) } would mean : $a < x < b$, $c \leq x \leq d$, $e \leq x < f$, and $g < x \leq h$. Only non-negative float values are allowed.

x_num

A settable int attribute that returns the `x_num` of `alignment_marker_rule`. It specifies pre shrink x num value.

a

x_offset

A settable string attribute that returns the *x_offset* of *alignment_marker_rule*. It specifies pre shrink horizontal offset from *ref_corner* to lower left corner of alignment marker cell placement after orient. The string should specify value ranges in the form of *distance_pair_list* that contains either float or list of float pairs. For example, { (a b) [c d] [e f] (g h) } would mean : $a < x < b$, $c \leq x \leq d$, $e \leq x < f$, and $g < x \leq h$.

x_spacing

A settable string attribute that returns the *x_spacing* of *alignment_marker_rule*. It is the horizontal distance from alignment marker cell boundary to alignment marker cell boundary to vertical edge of reference die boundary in pre-blow up die dimension. The string should specify value ranges in the form of *distance_pair_list* that contains either float or list of float pairs. For example, { (a b) [c d] [e f] (g h) } would mean : $a < x < b$, $c \leq x \leq d$, $e \leq x < f$, and $g < x \leq h$. Only non-negative float values are allowed.

x_step

A settable distance attribute that returns the *x_step* of *alignment_marker_rule*. It specifies pre shrink x step value.

y_enclosure

A settable string attribute that returns the *y_enclosure* of *alignment_marker_rule*. It is the vertical distance from alignment marker cell boundary to the horizontal edge of reference die across reference corner in pre-blow up die dimension. The string should specify value ranges in the form of *distance_pair_list* that contains either float or list of float pairs. For example, { (a b) [c d] [e f] (g h) } would mean : $a < y < b$, $c \leq y \leq d$, $e \leq y < f$, and $g < y \leq h$. Only non-negative float values are allowed.

y_num

A settable int attribute that returns the *y_num* of *alignment_marker_rule*. It specifies pre shrink y num value.

y_offset

A settable string attribute that returns the *y_offset* of *alignment_marker_rule*. It specifies pre shrink horizontal offset from *ref_corner* to lower left corner of alignment marker cell placement after orient. The string should specify value ranges in the form of *distance_pair_list* that contains either float or list of float pairs. For example, { (a b) [c d] [e f] (g h) } would mean : $a < y < b$, $c \leq y \leq d$, $e \leq y < f$, and $g < y \leq h$.

a

y_spacing

A settable string attribute that returns the *y_spacing* of *alignment_marker_rule*. It is the vertical distance from alignment marker cell boundary to alignment marker cell boundary to horizontal edge of reference die boundary in pre-blow up die dimension. The string should specify value ranges in the form of *distance_pair_list* that contains either float or list of float pairs. For example, { (a b) [c d] [e f] (g h) } would mean : $a < y < b$, $c \leq y \leq d$, $e \leq y < f$, and $g < y \leq h$. Only non-negative float values are allowed.

y_step

A settable distance attribute that returns the *y_step* of *alignment_marker_rule*. It specifies pre shrink y step value.

See Also

- [create_alignment_marker_rule](#)
- [get_attribute](#)
- [list_attributes](#)
- [set_attribute](#)

anchor_attributes

Description of the application defined anchor attributes.

Description

Attributes are properties assigned to objects such as pins, cells, and nets. They can be read-only or settable. A read-only attribute is informational; you cannot set its value. To modify a settable attribute, use the *set_attribute* command.

To determine the value of an attribute, use the *get_attribute* command. To see a list of all anchor attributes available, use the *list_attributes -application -class anchor* command.

Attributes for anchor

This section describes the anchor attributes.

anchor_object

A settable collection attribute that returns the anchor object of the anchor. The anchor object is the object to which the dependent object is anchored. This attribute may only be set if the anchor's *type* attribute is "location".

a

anchor_object_position

A settable string attribute (with allowed values: `bbox_center` `bbox_ll` `bbox_lr` `bbox_ul` `bbox_ur` `center`) that returns the anchoring reference point of the anchor object's shape. Please see the *create_anchor(2)* man page for a description of these values. This attribute may only be set if the anchor's *type* attribute is `"location"`.

anchor_orientation

A settable boolean attribute that determines whether to also lock the dependent object's orientation to that of the anchor object when the *type* attribute is `"location"`, or to the anchor x object when the *type* attribute is `"location_x_y"`. This attribute has no effect unless the dependent object's *object_class* has a settable `"orientation"` attribute.

anchor_x_object

A settable collection attribute that returns the anchor x object of the anchor. The anchor x object is the object to which the dependent object's x coordinate is anchored. This attribute may only be set if the anchor's *type* attribute is `"location_x_y"`.

anchor_x_object_position

A settable string attribute (with allowed values: `bbox_center` `bbox_ll` `bbox_lr` `bbox_ul` `bbox_ur` `center`) that returns the anchoring reference point of the anchor object's x shape. Please see the *create_anchor(2)* man page for a description of these values. This attribute may only be set if the anchor's *type* attribute is `"location_x_y"`.

anchor_y_object

A settable collection attribute that returns the anchor y object of the anchor. The anchor y object is the object to which the dependent object's y coordinate is anchored. This attribute may only be set if the anchor's *type* attribute is `"location_x_y"`.

anchor_y_object_position

A settable string attribute (with allowed values: `bbox_center` `bbox_ll` `bbox_lr` `bbox_ul` `bbox_ur` `center`) that returns the anchoring reference point of the anchor's object's y shape. Please see the *create_anchor(2)* man page for a description of these values. This attribute may only be set if the anchor's *type* attribute is `"location_x_y"`.

dependent_object

A settable collection attribute that returns the dependent (anchored) object of the anchor. As the anchor object(s) move, so does the dependent object. The object

a

must be in the same block as the anchor. Note that if the dependent object gets subsequently removed, this anchor will also be automatically removed. An object may be the `dependent_object` of no more than one anchor. It is an error to specify an object that is already the dependent object of an existing anchor.

dependent_object_position

A settable string attribute (with allowed values: `bbox_center` `bbox_ll` `bbox_lr` `bbox_ul` `bbox_ur` `center`) that returns the anchoring reference point of the anchor's dependent object's shape. Please see the *create_anchor(2)* man page for a description of these values.

full_name

A read-only string attribute that returns the full name of the anchor.

name

A settable string attribute that returns the name of the anchor.

object_class

A read-only string attribute that returns the name of this object class. Returns 'anchor' for anchor objects.

offset

A settable `distance_list` attribute that returns the offset to be maintained between the reference points defined by the *dependent_object_position* and the *anchor_object_position* attribute values when the *type* attribute is "location", or the *dependent_object_position* and the *anchor_x_object_position* and *anchor_y_object_position* attribute values when the *type* attribute is "location_x_y".

parent_block

A read-only block collection attribute that returns the owner block object for this anchor.

type

A settable string attribute (with allowed values: `location` `location_x_y`) that returns the anchor type.

See Also

- [create_anchor](#)
- [get_anchors](#)
- [get_attribute](#)

a

- [list_attributes](#)
- [set_attribute](#)

annotation_point_attributes

Description of the application defined annotation_point attributes.

Description

Attributes are properties assigned to objects such as pins, cells, and nets. They can be read-only or settable. A read-only attribute is informational; you cannot set its value. To modify a settable attribute, use the *set_attribute* command.

To determine the value of an attribute, use the *get_attribute* command. To see a list of all annotation_point attributes available, use the *list_attributes -application -class annotation_point* command.

Attributes for annotation_point

This section describes the annotation_point attributes.

anchor_object

A settable collection attribute that represents the object the annotation_point will follow when the annotation_point's type is *anchor*. This means as the anchor_object's placement changes, so will the annotation_point's location change to maintain it's relative positioning to the anchor_object. Only a single anchor_object may be specified for this attribute, and only object classes that have physical extent may serve as anchor_objects.

annotation_shape

A read-only annotation_shape collection attribute that returns a collection representing the annotation_shape that owns this annotation_point.

full_name

A read-only string attribute that returns the full name of the annotation_point.

index

A read-only unsigned attribute that returns this annotation_point's position in its owning annotation_shape's anchor_points collection. If the owning annotation_shape has N annotation_points, then this attribute will range from 0 to N-1 over the annotation_points in the collection.

location

A settable coord attribute that returns the annotation_point's physical location. If the annotation_point's type is *absolute*, then this attribute is settable by the

a

user. If the `annotation_point`'s type is *anchor*, then its location is automatically computed and hence not directly settable. The computation of the location when the `annotation_point`'s type is *anchor* involves starting with the `anchor_object`'s bounding box, then determining the coordinate relative to the bounding box using the `position_type` and `offset` attribute values.

name

A read-only string attribute that returns the name of the `annotation_point`.

object_class

A read-only string attribute that returns the name of this object class. Returns 'annotation_point' for `annotation_point` objects.

offset

A settable `pair_of_distance_or_percent` attribute that returns a pair of values, representing the horizontal and vertical location offsets, respectively. The offsets are relative to the location defined by the `anchor_object` and `position_type` attribute values. Each offset value may be an absolute distance, or may be a percentage, in which case the '%' character should be appended. When the value is a percentage, this will represent the percentage of the `anchor_object`'s bounding box extent in that direction.

physical_status

A settable string attribute (with allowed values: `locked` `unrestricted`) that returns the physical modification status of the `annotation_point`.

position_type

A settable string attribute (with allowed values: `bbox_center` `bbox_ll` `bbox_lr` `bbox_ul` `bbox_ur` `center`) that returns the `annotation_point`'s location positioning with respect to the `anchor_object`'s geometry. When the value is *center*, the location will be the center of the largest rectangle contained within the `anchor_object`. The remaining values specify the positioning with respect to the `anchor_object`'s bounding box.

type

A settable string attribute (with allowed values: `absolute` `anchor`) that returns the type of the `annotation_point`.

See Also

- [get_annotation_points](#)
- [get_attribute](#)

a

- [list_attributes](#)
- [set_attribute](#)

annotation_shape_attributes

Description of the application defined annotation_shape attributes.

Description

Attributes are properties assigned to objects such as pins, cells, and nets. They can be read-only or settable. A read-only attribute is informational; you cannot set its value. To modify a settable attribute, use the *set_attribute* command.

To determine the value of an attribute, use the *get_attribute* command. To see a list of all annotation_shape attributes available, use the *list_attributes -application -class annotation_shape* command.

Attributes for annotation_shape

This section describes the annotation_shape attributes.

annotation_points

A read-only annotation_point collection attribute that returns a collection representing the annotation_points of the annotation_shape.

bbox

A read-only rect attribute that returns the bounding-box of the points of the annotation_shape.

end_endcap

A settable string attribute (with allowed values: flush full_width half_width octagon round) that returns the end of endcap type of the annotation_shape.

full_name

A read-only string attribute that returns the full name of the annotation_shape.

height

A settable distance attribute that returns the height of the annotation_shape. This attribute is valid only for annotation_shape of type *symbol*.

layer_name

A settable string attribute that represent the layer name of the annotation_point where min_distance will be calculated, only available on min_distance_ruler type annotation_shapes.

a

min_distance_layer

A read-only string attribute that represent the layer name of the `annotation_point` used for `min_distance` calculation, or `BOUNDARY` if no `layer_name` is specified or the two annotation points do not share a common layer.

name

A settable string attribute that returns the name of the `annotation_shape`.

object_class

A read-only string attribute that returns the name of this object class. Returns 'annotation_shape' for `annotation_shape` objects.

physical_status

A settable string attribute (with allowed values: `locked` `unrestricted`) that returns a string specifying the physical modification status of the `annotation_shape`.

points

A read-only `coord_list` attribute that returns a coordinate list representing the locations of the corresponding `annotation_points` of the `annotation_shape`.

radius

A settable distance attribute that returns the radius of the `annotation_shape`. This attribute is valid only for `annotation_shapes` of type *circle*.

start_endcap

A settable string attribute (with allowed values: `flush` `full_width` `half_width` `octagon` `round`) that returns the start of endcap type of the `annotation_shape`. Valid values are `flush`, `full_width`, `half_width`, `octagon`, and `round`.

type

A read-only string attribute (with allowed values: `circle` `circular_ruler` `connector` `fit_symbol` `line` `polygon` `rect` `ruler` `symbol` `text`) that returns the type of the `annotation_shape`.

width

A settable distance attribute that returns the width of the `annotation_shape`. This attribute is valid only for `annotation_shape` of type *line* , *connector* and *symbol*.

See Also

- [get_annotation_shapes](#)
- [get_attribute](#)

b

- [list_attributes](#)
- [set_attribute](#)

b

block_attributes

Description of the application defined block attributes.

Description

Attributes are properties assigned to objects such as pins, cells, and nets. They can be read-only or settable. A read-only attribute is informational; you cannot set its value. To modify a settable attribute, use the *set_attribute* command.

To determine the value of an attribute, use the *get_attribute* command. To see a list of all block attributes available, use the *list_attributes -application -class block* command.

Attributes for block

This section describes the block attributes.

allowable_orientation

A settable orientations attribute that returns a list of allowed orientations for instances of this block.

always_on

A read-only boolean attribute that returns True if the block is a always on cell.

antenna_diode_type

A read-only string attribute that returns the antenna diode type of the block. The allowable values are "power", "ground" or "power_ground".

area

A read-only area attribute that returns the area of the block.

bbox

A read-only rect attribute that returns the bounding-box of a design. The format of a rectangle specification is `{{llx lly} {urx ury}}`, which specifies the lower-left and upper-right corners of the rectangle.

block

A read-only block collection attribute that returns the block of this design.

b

block_name

A read-only string attribute that returns the name of the block.

bond_style

A settable string attribute (with allowed values: bond_pad micro_bump none) that returns the connection style between dies in a 3DIC block.

boundary

A settable coord_list attribute that returns the boundary of a design. The format of a coordinate_list specification is {{x1 y1} {x2 y2}...}. If you want to clear an existing boundary, use the remove_attribute(2) command.

boundary_bbox

A read-only rect attribute that returns the bounding-box of a design. The format of a rectangle specification is {{llx lly} {urx ury}}, which specifies the lower-left and upper-right corners of the rectangle.

boundary_bounding_box

A read-only bounding_box collection attribute that returns the bounding-box of a design. The format of the collection specifies lower-left and upper-right corners of the rectangle.

boundary_polyrects

A read-only poly_list attribute that returns the boundary polyrects of a design. The no. of polyrects could be >1 only when disjoint.

bounding_box

A read-only bounding_box collection attribute that returns the bounding-box of a design. The format of the collection specifies lower-left and upper-right corners of the rectangle.

clock_gating_integrated_cell

A read-only string attribute that returns the name of the integrated clock gating if the block is of type integrated clock gating.

clock_path_target_lib_cell_exclusions

A read-only lib_cell collection attribute that contains the lib_cell objects in the target library subset's dont-use list assigned to this block to constrain the clock paths.

clock_path_target_lib_cell_subset

A read-only lib collection attribute that contains the lib_cell objects in the target library subset's only-here list assigned to this block to constrain the clock paths.

b

clock_path_target_library_subset

A read-only lib collection attribute that contains the lib objects in the target library subset assigned to this block to constrain the clock paths. This attribute contains only the libraries which were explicitly specified in the target library subset assigned to this block. It does not contain libraries which are inferred from the reference library list.

cmog_class

A read-only string attribute that returns the cmog_class value for the design.

compression_format

A read-only string attribute that returns possible values of gzip, zstd, and uncompressed.

core_area_area

A read-only double attribute that returns the area of the core_area.

core_area_bbox

A read-only rect attribute that returns the bounding-box of the core_area. The format of a rectangle specification is {{llx lly} {urx ury}}, which specifies the lower-left and upper-right corners of the rectangle.

core_area_boundary

A read-only coord_list attribute that returns the coordinates of the core_area. The format of a coordinate_list specification is {{x1 y1} {x2 y2}...}.

core_area_bounding_box

A read-only bounding_box collection attribute that contains the bounding-box of the core_area. The format of the collection specifies lower-left and upper-right corners of the rectangle.

data_path_target_lib_cell_exclusions

A read-only lib_cell collection attribute that returns the lib_cell objects in the target library subset's dont-use list assigned to this block to constrain the data paths.

data_path_target_lib_cell_subset

A read-only lib collection attribute that returns the lib_cell objects in the target library subset's only-here list assigned to this block to constrain the data paths.

data_path_target_library_subset

A read-only lib collection attribute that returns the lib objects in the target library subset assigned to this block to constrain the data paths. This attribute contains

b

only the libraries which were explicitly specified in the target library subset assigned to this block. It does not contain libraries which are inferred from the reference library list.

default_site

A read-only string attribute that returns the default site defs associated with the site available in the block. This attribute is not defined for design.

design

A read-only design collection attribute that returns this block as a design object.

design_name

A read-only string attribute that returns the name of this block as a design.

design_type

A settable string attribute (with allowed values: 3dic abstract analog analysis black_box bridge corner cover die diode end_cap feedthrough fill filler flip_chip_driver flip_chip_pad ftv interposer lib_cell macro module package pad pad_spacer pcb rdl_fanout rtl substrate thermal tsv vib well_tap) that returns the design type of this block. The design types "bridge" and "substrate" are deprecated.

die_separation

A read-only distance attribute that returns the distance between dies in a virtual block. This attribute exists only on virtual blocks and is set during the creation of the virtual block.

disable_timing

A read-only boolean attribute that returns True if the design is marked as disable_timing in the library. This attribute can be set in icc2_lm_shell.

dont_touch

A settable boolean attribute that returns True if the lib cell block is marked as dont_touch to prevent optimization from changing any instances of this lib cell block.

dont_use

A settable boolean attribute that returns true if the lib cell block is marked as dont_use to prevent optimization from inserting instances of this lib cell block.

excluded_purposes

A read-only string attribute that returns the purposes that are excluded as valid usages of the block. This is meant for lib_cells only.

b

extended_name

A read-only string attribute that returns the full path to the library plus the design name, separated by a ':' character, an optional label name, separated by a '/' character, and a view name, separated by a '.'. For example, "/disks/user1/work/top_lib:top_design/my_label.design".

file_size

A read-only string attribute that returns the size of the file on disk for this block.

foundry

A read-only string attribute that returns the foundry value for the design.

frame_update

A settable boolean attribute that enables updating the frame view of the lib cell during edit flow of the Library Manager. By default, the value is 'false'.

full_name

A read-only string attribute that returns the full name of a block, including the library name plus the block name, separated by a ':' character, an optional label name, separated by a '/' character, and a view name, separated by a '.' character. For example, "top_lib:top_design/my_label.design"

function_class

A read-only string attribute that returns the function class which represents the general function of the lib cell block.

has_mirror

A read-only boolean attribute that returns true if the 'mirror_name' attribute on the block is set to a non-empty string.

has_timing_model

A read-only boolean attribute that returns true if the timing data is available for the block.

height

A read-only distance attribute that returns the height of the boundary of the block.

hpc_core

A read-only string attribute that returns the HPC core value for the design.

included_purposes

A read-only string attribute that returns the purposes that are included as valid usages of the block. This is meant for lib_cells only.

inner_keepout_margin_clock

A settable margin_list attribute that returns the clock inner keepout.

inner_keepout_margin_filler_allowed

A settable margin_list attribute that returns the filler_allowed inner keepout.

inner_keepout_margin_hard

A settable margin_list attribute that returns the hard inner keepout.

inner_keepout_margin_hard_macro

A settable margin_list attribute that returns the hard_macro inner keepout.

inner_keepout_margin_route_blockage

A settable margin_list attribute that returns the route_blockage inner keepout.

inner_keepout_margin_route_blockage_layers

A read-only layer collection attribute that returns the route_blockage_layers inner keepout.

inner_keepout_margin_soft

A settable margin_list attribute that returns the soft inner keepout.

internal_power_derate_factor

A read-only float attribute that returns the actually stored internal derate values if and only if they exist. The value is based on current timer status, the attribute does not update the timer.

is_active_fill_valid

A read-only boolean attribute that returns true if this design's active fill is up-to-date, false if the design has been modified such that the active fill should be re-generated.

is_backside

A read-only boolean attribute that returns true if this block is a backside bump cell and false otherwise. A backside bump cell is a lib cell having design_type "flip_chip_pad" and either shapes on the backside metal layers, or shapes on the metal1 layer when its no_substrate attribute is true.

b

is_black_box

A read-only boolean attribute that returns true if this lib_cell has no logic function.

is_combinational

A read-only boolean attribute that returns true if the lib_cell is not sequential.

is_compressed

A settable boolean attribute that returns true if this design is saved in compressed format on disk, false otherwise.

is_current

A read-only boolean attribute that returns true if this design is the current_design for the session, false otherwise.

is_diff_level_shifter

A read-only boolean attribute that returns true if the lib_cell is a differential level shifter.

is_diode_cell

A read-only boolean attribute that returns true if the lib_cell is a power diode cell or ground diode cell.

is_enable_level_shifter

A read-only boolean attribute that returns true if the lib_cell is an enable level shifter cell.

is_etm_moded_cell

A read-only boolean attribute that returns true if design is ETM, and false otherwise.

is_extractable

A settable boolean attribute that returns true if the block has the design type of tsv or flip_chip_pad. This is used for user-supplied designs with 3dIC context.

is_fall_edge_triggered

A read-only boolean attribute that returns true if the block has timing behavior relative to the falling edge of a clock signal. This attribute can be set in icc2_lm_shell.

is_hierarchical

A read-only boolean attribute that always returns true.

b

is_implicit

A read-only boolean attribute that returns true if the implicit flag is set on the lib cell.

is_integrated_clock_gating_cell

A read-only boolean attribute that returns true if the block is defined in the library as an integrated clock gating cell. This attribute can be set in `icc2_lm_shell`.

is_isolation

A read-only boolean attribute that returns true if the lib_cell is an isolation cell.

is_level_shifter

A read-only boolean attribute that returns true if the lib_cell is a level shifter cell.

is_linked

A read-only boolean attribute that returns true if this design is already linked, and false otherwise.

is_locked_for_edit

A settable boolean attribute that returns true value if this reference block is locked to disallow edit (uneditable) whereas false value unlocks the edit-lock of this reference block to allow edit (editable).

is_mask_shiftable

A settable boolean attribute that returns true if this block enables multi-patterning mask shifting of cells. If "false", the mask_shift of any cell which references this block will be ignored.

is_memory_cell

A read-only boolean attribute that returns True if the lib_cell is defined in the library as a memory cell. This attribute can be set in `icc2_lm_shell`.

is_modified

A read-only boolean attribute that returns the value of whether this design has modified and un-saved data.

is_mux

A read-only boolean attribute that returns True if the lib_cell is a mux.

is_negative_level_sensitive

A read-only boolean attribute that returns True if the block has negative level-sensitive timing behavior, as in a negative D-latch. This attribute can be set in `icc2_lm_shell`.

b

is_open

A read-only boolean attribute that returns the value of whether this design is already loaded into memory.

is_port_punching_ok

A settable boolean attribute that returns True if the ability to add or remove ports is enabled in the lib cell.

is_positive_level_sensitive

A read-only boolean attribute that returns True if the block has positive level-sensitive timing behavior, as in a positive D-latch. This attribute can be set in `icc2_lm_shell`.

is_power_switch

A read-only boolean attribute that returns True if the block is a power switch cell.

is_remote

A read-only boolean attribute that returns True if a block is remote. The attribute should be always 'false' for lib cell.

is_retention

A read-only boolean attribute that returns True if the block is a retention cell.

is_rise_edge_triggered

A read-only boolean attribute that returns True if the block has timing behavior relative to the rising edge of a clock signal. This attribute can be set in `icc2_lm_shell`

is_sequential

A read-only boolean attribute that returns True if the block has sequential logic function.

is_soi

A read-only boolean attribute that returns True if the block is a SOI cell.

is_three_state

A read-only boolean attribute that returns True if the block has one or more three-state outputs. This attribute can be set in `icc2_lm_shell`.

keepouts

A read-only `keepout_margin` collection attribute that returns the keepouts for the block.

b

label_name

A read-only string attribute that returns the label for the block.

last_modified

A read-only string attribute that returns the date and time this design was last modified.

last_saved

A read-only string attribute that returns the date and time this design was last saved.

leakage_power_derate_factor

A read-only float attribute that returns the actually stored leakage derate values if and only if they exist. The value is based on current timer status, the attribute does not update the timer.

lib

A read-only lib collection attribute that returns the library of this design.

lib_name

A read-only string attribute that returns the name of library of this design.

logic_only

A settable boolean attribute that indicates whether this design is marked for the logic-only design flow. The attribute can only be set to "true" before the design is linked with *link_design*. Logic-only designs are used for Design Planning budgeting timing. They can not have physical design information, so they use less memory.

max_capacitance

A read-only float attribute that returns the float value for max capacitance constraint on the design.

max_capacitance_constraint

A read-only float attribute that returns the float value for max capacitance constraint on the design.

max_lvth_percentage

A read-only float attribute that returns the maximum limit on the percentage of low-Vth cells in the current design for optimization.

b

max_transition

A read-only float attribute that returns the float value for max transition constraint on the design.

max_transition_constraint

A read-only float attribute that returns the float value for max transition constraint on the design.

mb_bussed_ports

A read-only string attribute that returns the multibit bussed ports for the design.

mb_pin_map

A read-only string attribute that returns the multibit pin map for the design.

mb_scan_chain_ports

A read-only string attribute that returns the multibit scanchain ports for the design.

metrics_cong_cell_padding

A read-only double attribute that returns a metric derived by `compute_metrics` for this block. The amount of increase in apparent area of this hierarchy due to leaf cell padding added by the placer to enforce cell spreading to reduce congestion.

metrics_cong_cell_padding_local

A read-only double attribute that returns a metric derived by `compute_metrics` for this block. Only considers objects in this top block hierarchy, not child hierarchies. The amount of increase in apparent area of this hierarchy due to leaf cell padding added by the placer to enforce cell spreading to reduce congestion.

metrics_cong_drc_estimate

A read-only double attribute that returns a DRC estimate from leaf cells for top level.

metrics_cong_drc_estimate_local

A read-only double attribute that returns a DRC estimate from leaf cells local to top level.

metrics_cong_global_net_count

A read-only int attribute that returns a metric derived by `compute_metrics` for this block. Number of nets that pass over congestion hotspots in this hierarchy, but do not have local connections in that hotspot.

b

metrics_cong_global_net_count_local

A read-only int attribute that returns a metric derived by `compute_metrics` for this block. Only considers objects in this top block hierarchy, not child hierarchies. Number of nets that pass over congestion hotspots in this hierarchy, but do not have local connections in that hotspot.

metrics_cong_local_net_count

A read-only int attribute that returns a metric derived by `compute_metrics` for this block. Number of nets that pass over congestion hotspots in this hierarchy, and also have local pin connections in that area.

metrics_cong_local_net_count_local

A read-only int attribute that returns a metric derived by `compute_metrics` for this block. Only considers objects in this top block hierarchy, not child hierarchies. Number of nets that pass over congestion hotspots in this hierarchy, and also have local pin connections in that area.

metrics_cong_logic_cong

A read-only int attribute that returns a metric derived by `compute_metrics` for this block. Number of cells in congested areas that come from select op structures in the RTL. RTL could possibly be recoded to improve congestion in this hierarchy.

metrics_cong_logic_cong_local

A read-only int attribute that returns a metric derived by `compute_metrics` for this block. Only considers objects in this top block hierarchy, not child hierarchies. Number of cells in congested areas that come from select op structures in the RTL. RTL could possibly be recoded to improve congestion in this hierarchy.

metrics_cong_number_cells

A read-only int attribute that returns a metric derived by `compute_metrics` for this block. Number of leaf cells.

metrics_cong_number_cells_in_cong_area

A read-only int attribute that returns a metric derived by `compute_metrics` for this block. Number of cells overlapping global route congestion hotspots.

metrics_cong_number_cells_in_cong_area_local

A read-only int attribute that returns a metric derived by `compute_metrics` for this block. Only considers objects in this top block hierarchy, not child hierarchies. Number of cells overlapping global route congestion hotspots.

b

metrics_cong_number_cells_in_cong_channel

A read-only int attribute that returns a metric derived by `compute_metrics` for this block. Number of cells overlapping global route congestion hotspots in placement channels between macros.

metrics_cong_number_cells_in_cong_channel_local

A read-only int attribute that returns a metric derived by `compute_metrics` for this block. Only considers objects in this top block hierarchy, not child hierarchies. Number of cells overlapping global route congestion hotspots in placement channels between macros.

metrics_cong_number_cells_local

A read-only int attribute that returns a metric derived by `compute_metrics` for this block. Only considers objects in this top block hierarchy, not child hierarchies. Number of leaf cells.

metrics_cong_overflow

A read-only double attribute that returns a metric derived by `compute_metrics` for this block. Global routing overflows (gcells where routing demand exceeds capacity).

metrics_cong_overflow_local

A read-only double attribute that returns a metric derived by `compute_metrics` for this block. Only considers objects in this top block hierarchy, not child hierarchies. Global routing overflows (gcells where routing demand exceeds capacity).

metrics_cong_percent_cells_in_cong_area

A read-only double attribute that returns a metric derived by `compute_metrics` for this block. Percentage of cells overlapping global route congestion hotspots.

metrics_cong_percent_cells_in_cong_area_local

A read-only double attribute that returns a metric derived by `compute_metrics` for this block. Only considers objects in this top block hierarchy, not child hierarchies. Percentage of cells overlapping global route congestion hotspots.

metrics_cong_percent_of_cong_area

A read-only double attribute that returns a metric derived by `compute_metrics` for this block. Percentage of cell area overlapping global route congestion hotspots.

b

metrics_cong_percent_of_cong_area_local

A read-only double attribute that returns a metric derived by `compute_metrics` for this block. Only considers objects in this top block hierarchy, not child hierarchies. Percentage of cell area overlapping global route congestion hotspots.

metrics_cong_shape_dispersion

A read-only float attribute that returns a metric derived by `compute_metrics` for this block. Percentage spread of cells in this hierarchy. 0 indicates a contiguous placement area, as the number increases the placement shape is more fragmented (could be formed from multiple disjoint regions).

metrics_cong_shape_distortion

A read-only float attribute that returns a metric derived by `compute_metrics` for this block. Percentage measure of how the placement of logic in this hierarchy is impacted by macros. As the percentage increases, the placement shape is more distorted by macros.

metrics_cong_utilization

A read-only float attribute that returns a metric derived by `compute_metrics` for this block. Placement utilization of this hierarchy.

metrics_tim_bottleneck_count

A read-only int attribute that returns a metric derived by `compute_metrics` for this block. Number of cells in the hierarchy that have more negative slack paths passing through them than specified by `metrics.timing.bottleneck_path_count`.

metrics_tim_bottleneck_count_local

A read-only int attribute that returns a metric derived by `compute_metrics` for this block. Only considers objects in this top block hierarchy, not child hierarchies. Number of cells in the hierarchy that have more negative slack paths passing through them than specified by `metrics.timing.bottleneck_path_count`.

metrics_tim_logic_levels_viol

A read-only int attribute that returns a metric derived by `compute_metrics` for this block. Number of violating paths with logic level count greater than that specified by `metrics.timing.logic_level_threshold`.

metrics_tim_logic_levels_viol_local

A read-only int attribute that returns a metric derived by `compute_metrics` for this block. Only considers objects in this top block hierarchy, not child hierarchies. Number of violating paths with logic level count greater than that specified by `metrics.timing.logic_level_threshold`.

b

metrics_tim_net_length_viol

A read-only int attribute that returns a metric derived by `compute_metrics` for this block. Number of violating paths with nets exceeding the length specified by `metrics.timing.unbuffered_net_length_threshold`.

metrics_tim_net_length_viol_local

A read-only int attribute that returns a metric derived by `compute_metrics` for this block. Only considers objects in this top block hierarchy, not child hierarchies. Number of violating paths with nets exceeding the length specified by `metrics.timing.unbuffered_net_length_threshold`.

metrics_tim_nvp_internal

A read-only int attribute that returns a metric derived by `compute_metrics` for this block. Internal path number of failing setup endpoints.

metrics_tim_nvp_internal_local

A read-only int attribute that returns a metric derived by `compute_metrics` for this block. Only considers objects in this top block hierarchy, not child hierarchies. Internal path number of failing setup endpoints.

metrics_tim_nvp_io

A read-only int attribute that returns a metric derived by `compute_metrics` for this block. Input and output path number of failing setup endpoints.

metrics_tim_nvp_io_local

A read-only int attribute that returns a metric derived by `compute_metrics` for this block. Only considers objects in this top block hierarchy, not child hierarchies. Input and output path number of failing setup endpoints.

metrics_tim_nvp_r2r

A read-only int attribute that returns a metric derived by `compute_metrics` for this block. Register-to-register number of failing setup endpoints.

metrics_tim_nvp_r2r_local

A read-only int attribute that returns a metric derived by `compute_metrics` for this block. Only considers objects in this top block hierarchy, not child hierarchies. Register-to-register number of failing setup endpoints.

metrics_tim_nvp_total

A read-only int attribute that returns a metric derived by `compute_metrics` for this block. Overall design number of failing setup endpoints.

b

metrics_tim_nvp_total_local

A read-only int attribute that returns a metric derived by `compute_metrics` for this block. Only considers objects in this top block hierarchy, not child hierarchies. Overall design number of failing setup endpoints.

metrics_tim_path_length_viol

A read-only int attribute that returns a metric derived by `compute_metrics` for this block. Number of violating paths with path length longer then specified by `metrics.timing.path_length_threshold`.

metrics_tim_path_length_viol_local

A read-only int attribute that returns a metric derived by `compute_metrics` for this block. Only considers objects in this top block hierarchy, not child hierarchies. Number of violating paths with path length longer then specified by `metrics.timing.path_length_threshold`.

metrics_tim_tns_internal

A read-only float attribute that returns a metric derived by `compute_metrics` for this block. Internal path total negative setup slack.

metrics_tim_tns_internal_local

A read-only float attribute that returns a metric derived by `compute_metrics` for this block. Only considers objects in this top block hierarchy, not child hierarchies. Internal path total negative setup slack.

metrics_tim_tns_io

A read-only float attribute that returns a metric derived by `compute_metrics` for this block. Input and output path total negative setup slack.

metrics_tim_tns_io_local

A read-only float attribute that returns a metric derived by `compute_metrics` for this block. Only considers objects in this top block hierarchy, not child hierarchies. Input and output path total negative setup slack.

metrics_tim_tns_r2r

A read-only float attribute that returns a metric derived by `compute_metrics` for this block. Register-to-register total negative setup slack.

metrics_tim_tns_r2r_local

A read-only float attribute that returns a metric derived by `compute_metrics` for this block. Only considers objects in this top block hierarchy, not child hierarchies. Register-to-register total negative setup slack.

b

metrics_tim_tns_total

A read-only float attribute that returns a metric derived by `compute_metrics` for this block. Overall design total negative setup slack.

metrics_tim_tns_total_local

A read-only float attribute that returns a metric derived by `compute_metrics` for this block. Only considers objects in this top block hierarchy, not child hierarchies. Overall design total negative setup slack.

metrics_tim_wns_internal

A read-only float attribute that returns a metric derived by `compute_metrics` for this block. Internal path worst negative setup slack.

metrics_tim_wns_internal_local

A read-only float attribute that returns a metric derived by `compute_metrics` for this block. Only considers objects in this top block hierarchy, not child hierarchies. Internal path worst negative setup slack.

metrics_tim_wns_io

A read-only float attribute that returns a metric derived by `compute_metrics` for this block. Input and output path worst negative setup slack.

metrics_tim_wns_io_local

A read-only float attribute that returns a metric derived by `compute_metrics` for this block. Only considers objects in this top block hierarchy, not child hierarchies. Input and output path worst negative setup slack.

metrics_tim_wns_r2r

A read-only float attribute that returns a metric derived by `compute_metrics` for this block. Register-to-register worst negative setup slack.

metrics_tim_wns_r2r_local

A read-only float attribute that returns a metric derived by `compute_metrics` for this block. Only considers objects in this top block hierarchy, not child hierarchies. Register-to-register worst negative setup slack.

metrics_tim_wns_total

A read-only float attribute that returns a metric derived by `compute_metrics` for this block. Overall design worst negative setup slack.

b

metrics_tim_wns_total_local

A read-only float attribute that returns a metric derived by `compute_metrics` for this block. Only considers objects in this top block hierarchy, not child hierarchies. Overall design worst negative setup slack.

metrics_tim_zwl_viol_count

A read-only int attribute that returns a metric derived by `compute_metrics` for this block.

metrics_tim_zwl_viol_count_local

A read-only int attribute that returns a metric derived by `compute_metrics` for this block. Only considers objects in this top block hierarchy, not child hierarchies.

min_capacitance

A read-only float attribute that returns the float value for min transition constraint on the design.

min_capacitance_constraint

A read-only float attribute that returns the float value for min transition constraint on the design.

mirror_name

A settable string attribute that returns the `mirror_name` of the block.

multibit_width

A read-only int attribute that returns the multibit width for the design.

name

A read-only string attribute that returns the name of the block.

no_substrate

A settable boolean attribute that indicates whether this block has no substrate, and therefore no backside metal layers. When true, the block and cell `is_backside` attribute treats the `metal1` layer as a backside metal layer. When false, only the `bmetal` layers are treated a backside metal layers.

number_of_pins

A read-only int attribute that returns the number of pins on this `lib_cell`.

object_class

A read-only string attribute that returns the name of this object class. Returns 'block' for block objects.

b

open_count

A read-only int attribute that returns the number of times this design has been opened.

open_mode

A read-only string attribute that returns the open-mode of the design. Valid values are "read", "edit", and "new".

operating_condition_max

A read-only string attribute that returns the Max-delay operating condition.

operating_condition_min

A read-only string attribute that returns the min-delay operating condition.

outer_keepout_boundary

A read-only coord_list attribute that represents the outer keepout boundary.

outer_keepout_margin_assembly_die

A settable margin_list attribute that returns the assembly_die outer keepout.

outer_keepout_margin_clock

A settable margin_list attribute that returns the clock outer keepout.

outer_keepout_margin_filler_allowed

A settable margin_list attribute that returns the filler_allowed outer keepout.

outer_keepout_margin_hard

A settable margin_list attribute that returns the hard outer keepout.

outer_keepout_margin_hard_macro

A settable margin_list attribute that returns the hard_macro outer keepout.

outer_keepout_margin_route_blockage

A settable margin_list attribute that returns the route_blockage outer keepout.

outer_keepout_margin_route_blockage_layers

A read-only layer collection attribute that returns the route_blockage_layers outer keepout.

outer_keepout_margin_seal_ring

A settable margin_list attribute that returns the seal_ring outer keepout.

b

outer_keepout_margin_soft

A settable margin_list attribute that returns the soft outer keepout.

package_type

A settable string attribute (with allowed values: fanout hybrid interposer none stacked) that returns the packaging style for a 3D IC top-level design. It cannot be set on blocks for which design_type is not 3dic.

pad_cell

A read-only boolean attribute that returns True if the block is defined in the library as a pad cell.

read_from_product_name

A read-only string attribute that returns the name of the binary used to create this library.

read_from_product_version

A read-only string attribute that returns the version of the binary used to create this library.

read_from_schema_version

A read-only string attribute that returns the schema version of the library file (major_version). minor_version) format.

reference_orientation

A settable string attribute (with allowed values: R0 R90 R180 R270 MX MXR90 MY MYR90) that returns the orientation for instances of this block. This is meant for lib_cells.

register_blockage

A settable boolean attribute that returns True to disallow sequential topology repeaters (registers) placement in the block during repeater optimization in TIP flow.

register_boundary

A settable string attribute (with allowed values: both in none out) that returns whether the block requires input and/or output registers.

register_req_number

A settable int attribute that returns the number of the required (equal or more) sequential topology repeaters (registers) in the block. Treat negative value(-1) as register blockage in this block.

b

sb_degen_fid

A read-only string attribute that returns the single bit degenerate function id for the design.

sb_port_cnt

A read-only int attribute that returns the single bit port count for the design.

shaping_constraints

A read-only *shaping_constraint* collection attribute that contains the set of *shaping_constraint* objects defined for this block. Only applies to non lib cell blocks.

single_bit_degenerate

A read-only string attribute that returns the single bit degenerate value for the design.

site_def

A read-only *site_def* collection attribute that contains the site def associated with the block. This is meant for lib_cells and TSV designs only.

site_name

A read-only string attribute that returns the name of the site def associated with the block. This is meant for lib_cells and TSV designs only.

subset_name

A settable *name_list* attribute that returns the block subset name for LEQ. This attribute is valid only for blocks which are library cells. All library cells with no subset name also form a subset, which is used by the technology mapping function. Named subsets are used for special library cells (e.g., clock buffers that are specifically designed to match the delay of certain clock gates).

switch_list

A read-only string attribute that returns the effective view switch list of this design. The effective view switch list is the precedence-ordered list of design views that is applied when attempting to bind this design.

switching_power_derate_factor

A read-only float attribute that returns the actually stored switching derate values if and only if they exist. The value is based on current timer status, the attribute does not update the timer.

b

target_boundary_area

A settable area attribute that returns the desired or target boundary area for the block.

target_utilization

A settable float attribute that returns the desired or target utilization for the block.

technology_node

A read-only string attribute that returns the technology node for the design.

test_normal_func_id

A read-only string attribute that returns the name of the test normal function id of the block.

thickness

A settable distance attribute that returns the thickness of the block. The unit will be the same as the user unit for length.

threshold_voltage_group

A settable string attribute that returns the name of the category of the block based on its threshold voltage characteristics.

threshold_voltage_group_type

A read-only string attribute that returns the type of the category of the block based on its threshold voltage characteristics. The valid values are "low_vt", "normal_vt" and "high_vt".

top_module

A read-only module collection attribute that returns the top module of this design.

top_module_name

A settable string attribute that returns the name of the top module of this design. This attribute will change the name of the top module in the design. This is useful when managing multiple versions of modules, allowing use of the different versions by giving them different top-module names.

use_for_size_only

A read-only boolean attribute that returns true if the block is to be used for sizing only. This is meant for lib_cells only.

b

use_frame_blockage

A settable boolean attribute that enables the user to choose to not use blockages in certain frames. The default value is "true", which results in usage of the blockages in the frame.

user_function_class

A settable string attribute that returns the name of the user function class of the block. This is meant for lib_cells only.

valid_purposes

A read-only string attribute that returns the purposes that are valid usages of the block. This is meant for lib_cells only.

via0_alignment

A settable boolean attribute that indicates whether the block needs to be VIA0 aligned.

via_regions

A read-only via_region collection attribute that returns the via regions of the block. This is meant for lib_cells only.

view_name

A read-only string attribute (with allowed values: abstract conn design frame layout outline rtl timing) that returns the view name of this design.

virtual_blocks

A read-only block collection attribute that returns the virtual interface blocks in this block. This block must be the 3DIC top level block.

virtual_thickness

A settable distance attribute that returns the thickness representing the common thickness of the bump at the bottom of an interposer. This attribute is only available for 3dic design types.

voltage_max

A read-only float attribute that returns the voltage for max-delay analysis.

voltage_min

A read-only float attribute that returns the voltage for min-delay analysis.

width

A read-only distance attribute that returns the width of the boundary of the block.

b

z_span

A settable int attribute that sets the thickness in terms of number of die levels covered by a virtual interposer block. The valid values are between 1 to 255, both inclusive.

See Also

- [current_block](#)
- [get_blocks](#)
- [get_attribute](#)
- [list_attributes](#)
- [set_attribute](#)

block_pin_constraint_attributes

Description of the application defined block_pin_constraint attributes.

Description

Attributes are properties assigned to objects such as pins, cells, and nets. They can be read-only or settable. A read-only attribute is informational; you cannot set its value. To modify a settable attribute, use the *set_attribute* command.

To determine the value of an attribute, use the *get_attribute* command. To see a list of all block_pin_constraint attributes available, use the *list_attributes -application -class block_pin_constraint* command.

Attributes for block_pin_constraint

This section describes the block_pin_constraint attributes.

allowed_layers

A settable layer collection attribute that returns the metal layers which are allowed for pin placement. An empty collection means that all metal layers are allowed.

corner_keepout_distance

A settable distance attribute that returns the distance from the block corners where pin placement should begin. Value must be ≥ 0 . This is mutually exclusive with *corner_keepout_track_count*. Setting this attribute will unset *corner_keepout_track_count*.

b

corner_keepout_track_count

A settable int attribute that returns the number of wire tracks from the block corners where pin placement should begin. Value must be ≥ 0 . This defaults to 5 if neither `corner_keepout_track_count` nor `corner_keepout_distance` is specified. This is mutually exclusive with `corner_keepout_distance`. Setting this attribute will unset `corner_keepout_distance`.

excluded_sides

A settable unsigned int_list attribute that returns the block sides where pins cannot be placed. Each side number must be ≥ 1 . An empty list will reset this attribute. Sides are numbered starting with the leftmost vertical edge (side number 1) of the shape and increments by one going clockwise around to each edge in the shape. If there are multiple leftmost edges, then the starting edge is the lowest of the leftmost edges. This attribute is mutually exclusive with the `sides` attribute. Setting this attribute will unset the `sides` attribute.

full_name

A read-only string attribute that returns the name of the `block_pin_constraint`.

hard_constraints

A settable list attribute that returns the hard constraint types. This is a list containing one or more values. Valid values are `off`, `spacing`, `location`, `layer`, and `length`. Default is `off`.

is_feedthrough_allowed

A settable boolean attribute that returns whether feedthrough ports can be created during pin placement. Default is `false`.

length

A settable list attribute that returns the length of the pins. The length is parallel to the layer's preferred routing direction. The syntax is either a single length indicating the pin length for all layers, or a list of layer length pairs indicating the pin length for each layer (i.e., `{{layerName0 length0} {layerNameN lengthN}}`). Each layer name is a string. Each length is a float.

name

A read-only string attribute that returns the name of the `block_pin_constraint`.

object_class

A read-only string attribute that returns the name of this object class. Returns `'block_pin_constraint'` for `block_pin_constraint` objects.

b

parent_block

A read-only block collection attribute that returns the owner block object for this block_pin_constraint.

parent_block_cell

A read-only cell collection attribute that returns the parent block cell object for this block_pin_constraint.

parent_cell

A read-only cell collection attribute that returns the owner cell object for this block_pin_constraint.

pin_spacing_track_count

A settable int attribute that returns the minimum number of wire tracks between adjacent pins, or between a pin and a preroute wire that cuts across a block edge in the normal direction. Value must be ≥ 0 . This defaults to 1 if neither pin_spacing_track_count nor pin_spacing_distance is specified. This is mutually exclusive with pin_spacing_distance. Setting this attribute will unset pin_spacing_distance.

sides

A settable unsigned int_list attribute that returns the block sides where pins must be placed. Each side number must be ≥ 1 . An empty list will reset this attribute. Sides are numbered starting with the leftmost vertical edge (side number 1) of the shape and increments by one going clockwise around to each edge in the shape. If there are multiple leftmost edges, then the starting edge is the lowest of the leftmost edges. This attribute is mutually exclusive with the excluded_sides attribute. Setting this attribute will unset the excluded_sides attribute.

stacking_type

A settable string attribute (with allowed values: any none with_pg_pins_only with_signal_pins_only) that returns how signal pins can be stacked with other pins during pin placement. Default is any.

top_block

A read-only block collection attribute that returns the top block object for this block_pin_constraint.

width

A settable list attribute that returns the width of the pins. The width is perpendicular to the layer's preferred routing direction. This attribute is settable. The syntax is either a single width indicating the pin width for all layers, or a list

b

of layer width pairs indicating the pin width for each layer (i.e., {{layerName0 width0} {layerNameN widthN}}). Each layer name is a string. Each width is a float.

See Also

- [create_block_pin_constraint](#)
- [get_attribute](#)
- [list_attributes](#)
- [set_attribute](#)

bond_pad_def_attributes

Lists the predefined attributes for bond pad definitions.

Description

Attributes are properties assigned to objects such as pins, cells and nets. Definitions for bond pad definition attributes are provided in the subsections that follow.

Attributes are read-only unless marked as settable. Read-only attributes are informational, and you cannot set its value. A settable attribute can be modified with the *set_attribute* command.

To determine the value of an attribute, use the *get_attribute* command. To see a list of all attributes available for a class of objects use the *list_attributes -application* command.

Bond Pad Definition Attributes

boundary

A read-only coordinate list attribute that contains the boundary of the shape for the bond pad, or a pair of points representing it in the form of bounding box. The format of a coordinate_list specification is {{x1 y1} {x2 y2}...}.

dimensions

A read-only distance pair attribute that contains the width and height of the bond pad. It is used along with *-num_sides* to determine the exact shape and size of the bond pad.

full_name

A read-only string attribute that contains the full name of the bond pad definitions.

b

layer

A read-only layer collection attribute that contains the tech layer for the bond pad.

name

A read-only string attribute that contains the name of the bond pad definition.

num_sides

A read-only integer attribute that contains the number of sides of polygon of the bond pad shape. The value is between 4 and 1024.

object_class

A read-only string attribute with the value of "bond_pad_def".

See Also

- [get_bond_pad_defs](#)
- [get_attribute](#)
- [list_attributes](#)
- [set_attribute](#)

bound_attributes

Description of the application defined bound attributes.

Description

Attributes are properties assigned to objects such as pins, cells, and nets. They can be read-only or settable. A read-only attribute is informational; you cannot set its value. To modify a settable attribute, use the *set_attribute* command.

To determine the value of an attribute, use the *get_attribute* command. To see a list of all bound attributes available, use the *list_attributes -application -class bound* command.

Attributes for bound

This section describes the bound attributes.

allowed_design_type

A read-only string attribute (with allowed values: 3dic abstract analog analysis black_box bridge corner cover die diode end_cap feedthrough fill filler flip_chip_driver flip_chip_pad ftv interposer lib_cell macro module package pad pad_spacer pcb rdl_fanout rtl substrate thermal tsv vib well_tap) that returns

b

the type of cells that are allowed to be associated with the bound. The possible values are `abstract`, `analog`, `black_box`, `corner`, `cover`, `diode`, `end_cap`, `fill`, `filler`, `flip_chip_driver`, `flip_chip_pad`, `lib_cell`, `macro`, `module`, `pad`, `pad_spacer`, `physical_only` and `well_tap`.

bbox

A read-only `rect` attribute that returns the bounding-box of a bound. The format of a rectangle specification is `{{llx lly} {urx ury}}`, which specifies the lower-left and upper-right corners of the rectangle.

bound_groups

A read-only group collection attribute that contains the repelling bound group objects that belong to this bound. Only valid for repelling bounds.

bound_type

A read-only string attribute that indicates if the bound is a group bound or a move bound.

bounding_box

A read-only `bounding_box` collection attribute that returns the bounding-box of a bound. The format of the collection specifies lower-left and upper-right corners of the rectangle.

cells

A read-only cell collection attribute that contains the cells associated with this bound.

central_object

A read-only port, pin, or cell collection attribute that returns the anchor point object in the bound. It can be a cell, a pin or a port. This is applicable for diamond bounds only.

color

A settable string attribute that returns the color of the move bound. This value must be a 24-bit RGB hexadecimal string prefaced by '#', for example: `"#72c4f1"` or `"#ce8322"`.

dimensions

A settable `coord` attribute that returns the width of the bound as x-coordinate and height as y-coordinate. This is applicable for group bounds only.

b

effort

A settable string attribute (with allowed values: low medium high ultra) that returns the effort of bringing cells closer inside a group bound. This is applicable for group bounds only.

excluded_cells

A read-only cell collection attribute that contains cells which have been explicitly removed from this bound.

full_name

A read-only string attribute that returns the name of the bound.

groups

A read-only group collection attribute that returns the groups in which this bound is a member.

in_edit_group

A read-only boolean attribute that indicates whether a bound belongs to an edit_group.

interface_connectivity_planning

A settable string attribute (with allowed values: false hard soft unset) that determines whether to leave some space along edges of a move bound free of macros during automatic macro placement. It works together with the app option *plan.macro.cross_connectivity_planning*; see that option's man page for more details. The default is unset. This is applicable for move bounds only.

is_exclusive

A read-only boolean attribute that indicates whether a move bound should be treated as an exclusive area with respect to cells not associated with this bound. This is applicable for move bounds only.

is_hierarchical

A read-only boolean attribute that indicates whether a move bound can contain only cells which represent module boundary objects, which are created by the *explore_logic_hierarchy* command. No other cells or ports can be added to move bounds of this type. These move bounds are used during floorplanning and have no effect on other placement or legalization. This attribute is applicable only for move bounds.

is_macro_group

A read-only boolean attribute that indicates whether a move bound is macro_group move bound. This is applicable for move bounds only.

b

is_off_edge

A settable boolean attribute that determines whether macros belonging to the bound should be placed off edges when *plan. macro.macros_on_edge* app option is false. The default is false.

is_repelling

A read-only boolean attribute that indicates if the bound is attractive or repelling. True if the bound is repelling, false if it is attractive.

is_shadow

A read-only boolean attribute that indicates whether a bound is pushed down from above.

is_visible

A settable boolean attribute that returns whether the move bound is visible or not.

name

A settable string attribute that returns the name of the bound.

object_class

A read-only string attribute that returns the name of this object class. Returns 'bound' for bound objects.

parent_block

A read-only block collection attribute that returns the owner block object for this bound.

parent_block_cell

A read-only cell collection attribute that returns the parent block cell object for this bound.

parent_cell

A read-only cell collection attribute that returns the owner cell object for this bound.

physical_status

A settable string attribute (with allowed values: locked unrestricted) that returns the physical modification status of the bound. If status is unrestricted, the bound may be moved or rotated. If status is locked, the bound may not be moved or rotated.

b

ports

A read-only port collection attribute that returns the ports associated with this bound.

shadow_status

A settable string attribute (with allowed values: copied_down copied_up normal pulled_up pushed_down virtual_flat) that returns the shadow status of a bound.

shape_count

A read-only int attribute that returns the number of bound_shape objects in the bound. This is applicable for move bounds only.

shape_type

A read-only string attribute that reports the bound's shape type. Valid shape types are diamond and rectangle.

shapes

A read-only bound_shape collection attribute that returns the bound_shape objects in the bound. This is applicable for move bounds only.

shaping_constraints

A read-only shaping_constraint collection attribute that returns the shaping constraints associated with the bound. Only applies to move bounds.

standard_cell_utilization

A read-only double attribute that returns the standard cell utilization of a move bound. This attribute exists if and only if this bound is a move bound. The value is the ratio of the associated standard cell area to the bound area occupied by standard cells or no cells (i.e., the bound area occupied by macro cells is not included).

top_block

A read-only block collection attribute that returns the top block object for this bound.

type

A settable string attribute (with allowed values: hard soft) that returns the type of bound.

utilization

A read-only double attribute that returns the utilization of a move bound as ratio of area of associated instances to the area of the bound.

See Also

- [create_bound](#)
- [get_attribute](#)
- [list_attributes](#)
- [set_attribute](#)

bound_shape_attributes

Description of the application defined bound_shape attributes.

Description

Attributes are properties assigned to objects such as pins, cells, and nets. They can be read-only or settable. A read-only attribute is informational; you cannot set its value. To modify a settable attribute, use the *set_attribute* command.

To determine the value of an attribute, use the *get_attribute* command. To see a list of all bound_shape attributes available, use the *list_attributes -application -class bound_shape* command.

Attributes for bound_shape

This section describes the bound_shape attributes.

bbox

A read-only rect attribute that returns the bounding-box of a bound_shape. The format of a rectangle specification is `{{llx lly} {urx ury}}`, which specifies the lower-left and upper-right corners of the rectangle.

bound

A read-only bound collection attribute that returns the owning bound of this bound_shape.

boundary

A settable polygon attribute that returns the boundary of this bound_shape. The format of a boundary specification is `{{{llx1 lly1} {urx1 ury1}} {{llx2 lly2} {urx2 ury2} ...} ...}`.

bounding_box

A read-only bounding_box collection attribute that returns the bounding-box of a bound_shape. The format of the collection specifies lower-left and upper-right corners of the rectangle.

full_name

A read-only string attribute that returns the name of the bound_shape.

groups

A read-only group collection attribute that returns the groups in which this bound_shape is a member.

name

A read-only string attribute that returns the name of the bound_shape.

object_class

A read-only string attribute that returns the name of this object class. Returns 'bound_shape' for bound_shape objects.

parent_block

A read-only block collection attribute that returns the owner block object for this bound_shape.

parent_block_cell

A read-only cell collection attribute that returns the parent block cell object for this bound_shape.

parent_cell

A read-only cell collection attribute that returns the owner cell object for this bound_shape.

top_block

A read-only block collection attribute that returns the top block object for this bound_shape.

See Also

- [create_bound_shape](#)
- [get_attribute](#)
- [list_attributes](#)
- [set_attribute](#)

bounding_box_attributes

Description of the application defined bounding_box attributes.

b

Description

Attributes are properties assigned to objects such as pins, cells, and nets. They can be read-only or settable. A read-only attribute is informational; you cannot set its value. To modify a settable attribute, use the *set_attribute* command.

To determine the value of an attribute, use the *get_attribute* command. To see a list of all bounding_box attributes available, use the *list_attributes -application -class bounding_box* command.

Attributes for bounding_box

This section describes the bounding_box attributes.

area

A read-only area attribute that returns the area this rectangle covers.

center

A read-only coord attribute that returns the center of the rectangle.

height

A read-only distance attribute that returns the distance between the *ur_y* (top) and the *ll_y* (bottom) coordinate values.

ll

A read-only coord attribute that returns the lower-left corner of the rectangle.

ll_x

A read-only distance attribute that returns the distance of the lower-left corner of the rectangle in the x-direction.

ll_y

A read-only distance attribute that returns the distance of the lower-left corner of the rectangle in the y-direction.

object_class

A read-only string attribute that returns the name of this object class. Returns 'bounding_box' for bounding_box objects.

ur

A read-only coord attribute that returns the upper-right corner of the rectangle.

ur_x

A read-only distance attribute that returns the distance of the upper-right corner of the rectangle in the x-direction.

b

ur_y

A read-only distance attribute that returns the distance of the upper-right corner of the rectangle in the y-direction.

width

A read-only distance attribute that returns the distance between the ll_x (left) and the ur_x (right) values.

See Also

- [get_attribute](#)
- [list_attributes](#)
- [set_attribute](#)

budget_attributes

Summary of attributes available on tcl objects returned by `get_budgets`

Attributes of budget_clock object*object_class*

Returns the string "budget_clock"

full_name

Returns the name of the clock as seen in the output of the `report_budget` command.

name

Same as `full_name`

source

Pin or port where budget clock is defined. If this is a portion of a clock tree that crosses into a budgeted block, then the source is the hierarchical pin on the block where the clock entered. Will be undefined for top-level virtual clocks.

block

Budget block (a cell) when the clock is defined. Will be undefined for top-level clocks.

clock

Object corresponding to the clock defined by your SDC.

b

edge

Edge of the clock at the SDC source

Attributes of budget_path_type object

object_class

Returns the string "budget_path_type"

full_name

Returns the name of the path_type as seen in the output of the report_budget command.

name

Same as full_name

path_type_id

Returns an integer which is a unique identifier for this path type. You can use this identifier to check that two objects refer to the same data. This identifier is also used as an anchor in the output of "report_budget -html".

launch_clock

Returns a budget_clock object corresponding to the start of this type of path. Will be undefined if the path is not clocked.

capture_clock

Returns a budget_clock object corresponding to the end of this type of path. Will be undefined if the path is not clocked.

exception_type

Returns a short string corresponding to an exception applying to this path. For example, a "multicycle 2" path will have exception type "mcp2". Will be an empty string if there is no exception.

exception

Returns a long string corresponding to an exception applying to this path. The string will have SDC format. For example "set_multicycle_path 2". Will be an empty string if there is no exception.

level_sensitive_capture

Will return the string "true" if the path ends in a level sensitive latch and "false" otherwise.

b

from_latch_path

Will return the string "true" if the path comes from a level sensitive latch and "false" otherwise.

total_path_delay

Returns a float which is the total delay of the path based on the launch clock, the capture clock, and the exception.

budget_path_delay

Returns a float which is the portion of the total path delay used for budgeting. Margins and clock uncertainty are removed. clock, the capture clock, and the exception.

launch_uncertainty

Returns a float which represents the clock uncertainty applied to the the budgeted portion of the path at the launching end.

capture_uncertainty

Returns a float which represents the clock uncertainty applied to the the budgeted portion of the path at the capturing end.

Attributes of budget_pin object

Budget_segment objects are returned by the -budget_pin and -all_budget_pins options of the get_budgets command. You can use the *get_attribute* command to query the following attributes of this object:

object_class

Returns the string "budget_pin".

full_name

Returns the string "pin%d"

name

Same as full_name

pin_id

Returns an integer which is a unique identifier for this object. You can use this identifier to check that two objects refer to the same data. This identifier is also used as an anchor in the output of "report_budget -html".

pin

Returns the pin object that this data refers to.

b

direction

Returns a string which is "input" for input pins and "output" for output pins. This attribute will return the appropriate direction of the associated path for inout pins.

internal_frozen

Return the string "true" if the value of internal constraints have been frozen for this pin, and "false" otherwise.

feedthrough_frozen

Return the string "true" if the value of feedthrough constraints have been frozen for this pin, and "false" otherwise.

constraints

Return a collection of budget pin constraints that have been set for this pin. If no constraints are set, then this attribute will not be defined.

pin_data

Return a collection of budget pin data objects for paths that pass through this pin. If no paths pass through this pin, then this attribute will not be defined.

no_path_fanin_pins

Return a collection of budget pins that feed this pin through an unconstrained segment. Will be undefined if there are none.

no_path_fanout_pins

Return a collection of budget pins that are fed by this pin through an unconstrained segment. Will be undefined if there are none.

has_clock_fanin

Return true if this budget pin is fed by a clock.

missing_fanin_delay

Return true if this budget pin is not fed by a potential datapath start such as a set_input_delay or a clocked flop.

missing_fanout_delay

Return true if this budget pin does not feed a potential datapath end such as set_output_delay or a clocked flop.

has_input_delay

Return true if this budget pin has a set_input_delay constraint defined on it.

b

`has_output_delay`

Return true if this budget pin has a `set_output_delay` constraint defined on it.

`has_fanin_budget_pin`

Return true if this pin is fed by another budget pin through a combinational path.

`has_fanout_budget_pin`

Return true if this pin feeds another budget pin through a combinational path.

Attributes of `budget_pin_data` object

`object_class`

Returns the string "budget_pin_data".

`full_name`

Returns the string "pd%d"

`name`

Same as `full_name`

`pin_id`

Returns an integer which is a unique identifier for this object. You can use this identifier to check that two objects refer to the same data. This identifier is also used as an anchor in the output of "report_budget -html".

`path_type_id`

Returns an integer which is a unique identifier for the path type of this object. You can use this identifier to check that two objects refer to the same data. This identifier is also used as an anchor in the output of "report_budget -html".

`pin`

Returns the pin object that this data refers to.

`budget_pin`

Returns the `budget_pin` object that this data refers to.

`path_type`

Returns the `budget_path_type` object that applies to the timing path for this data.

`direction`

Returns a string which is "input" for input pins and "output" for output pins. This attribute will return the appropriate direction of the associated path for inout pins.

constraint

Returns the `budget_pin_constraint` object used to derive the budget for this path.
Will be undefined if the pin is unconstrained.

is_feedthrough

Returns true if the `pin_data` is for a path that feeds through the pin's block.

launch_budget

Returns a float which is the budgeted portion of the path on the launching side of this pin. Will be undefined if there is no budget.

launch_delay

Returns an estimate of the longest actual delay of the portion of the path on the launching side of this pin.

launch_slack

Returns a float which is `launch_budget` minus `launch_delay`. Will be undefined if there is no budget.

capture_budget

Returns a float which is the budgeted portion of the path on the capturing side of this pin. Will be undefined if there is no budget.

capture_delay

Returns an estimate of the longest actual delay of the portion of the path on the capturing side of this pin.

capture_slack

Returns a float which is `capture_budget` minus `capture_delay`. Will be undefined if there is no budget.

Attributes of `budget_pin_constraint` object

object_class

Returns the string "budget_pin_constraint".

full_name

Returns the string "cons%d"

name

Same as `full_name`

b

constraint_id

Returns an integer which is a unique identifier for this pin constraint. You can use this identifier to check that two objects refer to the same data.

pin

Returns the pin object being constrained.

direction

Returns a string which is "input" for input pins and "output" for output pins. This attribute will return the appropriate direction of the associated path for inout pins.

constraint_type

Returns a string which is one of "internal_percent", "internal_delay", "from_percent", "from_delay", "to_percent", "to_delay", or "none".

constraint_value

Returns a float value corresponding to the constraint type.

path_type

Return "feedthrough" for a constraint on a feedthrough path, "internal" for a constraint on an path start or end, and "" if the constraint applies to both.

from_clock

Return the name of the clock object specified to the set_budget_pin_options command. Return "" if none.

from_clock_edge

Return the string "rise", "fall" or "" depending on the options passed to the set_budget_pin_options command.

to_clock

Return the name of the clock object specified to the set_budget_pin_options command. Return "" if none.

to_clock_edge

Return the string "rise", "fall" or "" depending on the options passed to the set_budget_pin_options command.

frozen

Return the string "true" if the value of the constraint has been frozen, and "false" otherwise.

b

Attributes of budget_segment object**object_class**

Returns the string "budget_segment".

full_name

Returns the string "seg%d"

name

Same as full_name

from_pin_id

Returns an integer which is a unique identifier for the budget_pin_data object that fans into this segment. You can use this identifier to check that two objects refer to the same data. This identifier is also used as an anchor in the output of "report_budget. Will be undefined if this segment does not have a budget pin at its input.

to_pin_id

Returns an integer which is a unique identifier for the budget_pin_data object that fans out of this segment. You can use this identifier to check that two objects refer to the same data. This identifier is also used as an anchor in the output of "report_budget. Will be undefined if this segment does not have a budget pin at its output.

path_type_id

Returns an integer which is a unique identifier for the budget_path_type object that for the path that this segment is part of. You can use this identifier to check that two objects refer to the same data. This identifier is also used as an anchor in the output of "report_budget.

segment_id

Returns an integer which is a unique identifier for this budget_segment object. You can use this identifier to check that two objects refer to the same data. This identifier is also used as an anchor in the output of "report_budget.

path_type

Returns the budget_path_type object that for the path that this segment is part of.

from_block

Returns the name of the block where the segment starts. If the segment starts on a block boundary, the block on the other side of the boundary will be returned. For the top level, "" is returned.

b

through_block

Returns the name of the block where the segment passes through. For the top level, "" is returned.

to_block

Returns the name of the block where the segment ends. If the segment ends on a block boundary, the block on the other side of the boundary will be returned. For the top level, "" is returned.

from_pin

Return the pin object for the pin that is at the input of this segment. Will be undefined if this segment does not have a budget pin at its input.

to_pin

Return the pin object for the pin that is at out the output of this segment. Will be undefined if this segment does not have a budget pin at its output.

from_pin_data

Returns the budget_pin_data object that fans into this segment. Will be undefined if this segment does not have a budget pin at its input.

to_pin_data

Returns the budget_pin_data object that fans of of this segment. Will be undefined if this segment does not have a budget pin at its output.

budget

Returns the delay that has been budgeted to this segment. Will be undefined if this segment does not have a budget constraint.

delay

Returns an estimate of the longest actual delay for timing represented by this budget segment.

fixed_delay

Returns the portion of delay that is "fixed". This includes delay through macros, clock skew and ocv, and flop setup and hold times.

slack

Returns budget minus delay.

fixed_slack

Returns budget minus fixed_delay.

b

constraint

If a delay constraint has been set for this segment using the `-busplans` option of the `compute_budget_constraints` command or by the `set_segment_budget_constraints` command, the applied value is returned. Otherwise, the attribute is undefined.

rule_name

If a busplan rule has been used to constrain this segment, the name of that rule is returned. Otherwise, the attribute is undefined.

is_extended_path

If the given segment was created because of the `plan.budget.extend_paths` app option, return true. Otherwise, the attribute is undefined.

See Also

- [get_budgets](#)
- [budget_pin_attributes](#) Description of the application defined `budget_pin` attributes.
- [budget_pin_constraint_attributes](#) Description of the application defined `budget_pin_constraint` attributes.
- [budget_clock_attributes](#) Description of the application defined `budget_clock` attributes.
- [budget_pin_data_attributes](#) Description of the application defined `budget_pin_data` attributes.
- [budget_path_type_attributes](#) Description of the application defined `budget_path_type` attributes.
- [budget_segment_attributes](#) Description of the application defined `budget_segment` attributes.
- [budget_attributes](#) Summary of attributes available on tcl objects returned by `get_budgets`
- [get_attribute](#)
- [report_attributes](#)
- [list_attributes](#)
- [report_budget](#)
- [set_budget_options](#)
- [set_pin_budget_constraints](#)

- [compute_budget_constraints](#)
- [write_budgets](#)

budget_clock_attributes

Description of the application defined budget_clock attributes.

Description

Attributes are properties assigned to objects such as pins, cells, and nets. They can be read-only or settable. A read-only attribute is informational; you cannot set its value. To modify a settable attribute, use the *set_attribute* command.

To determine the value of an attribute, use the *get_attribute* command. To see a list of all budget_clock attributes available, use the *list_attributes -application -class budget_clock* command.

The budget_clock object is returned by the *get_budgets -clock* command. They can also be accessed as an attribute of budget_path_type objects.

Attributes for budget_clock

This section describes the budget_clock attributes.

block

A read-only collection attribute that returns the budget block (a cell) when the clock is defined. Will be undefined for top-level clocks.

clock

A read-only collection attribute that returns the object corresponding to the clock defined by your SDC.

edge

A read-only string attribute that returns the edge of the clock at the SDC source.

full_name

A read-only string attribute that returns the name of the clock as seen in the output of the report_budget command.

name

A read-only string attribute that returns the same as full_name.

object_class

A read-only string attribute that returns the name of this object class. Returns 'budget_clock' for budget_clock objects.

source

A read-only collection attribute that returns the pin or port where budget clock is defined. If this is a portion of a clock tree that crosses into a budgeted block, then the source is the hierarchical pin on the block where the clock entered. Will be undefined for top-level virtual clocks.

See Also

- [get_budgets](#)
- [collections](#)
- [get_attribute](#)
- [report_attributes](#)
- [report_budget](#)
- [set_budget_options](#)
- [set_pin_budget_constraints](#)
- [compute_budget_constraints](#)
- [write_budgets](#)

budget_path_type_attributes

Description of the application defined budget_path_type attributes.

Description

Attributes are properties assigned to objects such as pins, cells, and nets. They can be read-only or settable. A read-only attribute is informational; you cannot set its value. To modify a settable attribute, use the *set_attribute* command.

To determine the value of an attribute, use the *get_attribute* command. To see a list of all budget_path_type attributes available, use the *list_attributes -application -class budget_path_type* command.

budget_path_type objects are returned by the *get_budgets -path_types* command. They can also be accessed as attributes of budget_pin_data and budget_segment objects.

Attributes for budget_path_type

This section describes the budget_path_type attributes.

b

budget_path_delay

A read-only float attribute that returns a float which is the portion of the total path delay used for budgeting. Margins and clock uncertainty are removed.

capture_clock

A read-only collection attribute that returns a `budget_clock` object corresponding to the end of this type of path. Will be undefined if the path is not clocked.

capture_uncertainty

A read-only float attribute that returns the budgeted uncertainty applied at the path end.

exception

A read-only string attribute that returns a long string corresponding to an exception applying to this path. The string will have SDC format. For example "set_multicycle_path 2". Will be an empty string if there is no exception.

exception_type

A read-only string attribute that returns a short string corresponding to an exception applying to this path. For example, a "multicycle 2" path will have exception type "mcp2". Will be an empty string if there is no exception.

from_latch_path

A read-only boolean attribute that returns the string "true" if the path comes from a level sensitive latch and "false" otherwise.

full_name

A read-only string attribute that returns the name of the `path_type` as seen in the output of the `report_budget` command.

launch_clock

A read-only collection attribute that returns a `budget_clock` object corresponding to the start of this type of path. Will be undefined if the path is not clocked.

launch_uncertainty

A read-only float attribute that returns a float which represents the clock uncertainty applied to the budgeted portion of the path at the launching end.

level_sensitive_capture

A read-only boolean attribute that returns the string "true" if the path ends in a level sensitive latch and "false" otherwise.

b

name

A read-only string attribute that is the same as `full_name`.

object_class

A read-only string attribute that is the name of this object class. Returns 'budget_path_type' for `budget_path_type` objects.

path_type_id

A read-only int attribute that returns an integer which is a unique identifier for this path type. You can use this identifier to check that two objects refer to the same data. This identifier is also used as an anchor in the output of "report_budget -html".

total_path_delay

A read-only float attribute that returns a float which is the total delay of the path based on the launch clock, the capture clock, and the exception.

See Also

- [get_budgets](#)
- [collections](#)
- [get_attribute](#)
- [report_attributes](#)
- [report_budget](#)
- [set_budget_options](#)
- [set_pin_budget_constraints](#)
- [compute_budget_constraints](#)
- [write_budgets](#)

budget_pin_attributes

Description of the application defined `budget_pin` attributes.

Description

Attributes are properties assigned to objects such as pins, cells, and nets. They can be read-only or settable. A read-only attribute is informational; you cannot set its value. To modify a settable attribute, use the *set_attribute* command.

b

To determine the value of an attribute, use the *get_attribute* command. To see a list of all *budget_pin* attributes available, use the *list_attributes -application -class budget_pin* command.

budget_pin objects are returned by the *-budget_pin* and *-all_budget_pins* options of the *get_budgets* command.

Attributes for *budget_pin*

This section describes the *budget_pin* attributes.

constraints

A read-only collection attribute that returns a collection of budget pin constraints that have been set for this pin. If no constraints are set, then this attribute will not be defined.

direction

A read-only string attribute that returns a string which is "input" for input pins and "output" for output pins. This attribute will return the appropriate direction of the associated path for inout pins.

feedthrough_frozen

A read-only boolean attribute that returns the string "true" if the value of feedthrough constraints have been frozen for this pin, and "false" otherwise.

full_name

A read-only string attribute that returns the string "pin%d".

has_clock_fanin

A read-only boolean attribute that returns true if this budget pin is fed by a clock.

has_fanin_budget_pin

A read-only boolean attribute that returns true if this pin is fed by another budget pin through a combinational path.

has_fanout_budget_pin

A read-only boolean attribute that returns true if this pin feeds another budget pin.

has_input_delay

A read-only boolean attribute that returns true if this budget pin has a *set_input_delay* constraint defined on it.

b

has_output_delay

A read-only boolean attribute that returns true if this budget pin has a `set_output_delay` constraint defined on it.

internal_frozen

A read-only boolean attribute that returns the string "true" if the value of internal constraints have been frozen for this pin, and "false" otherwise.

missing_fanin_delay

A read-only boolean attribute that returns true if this budget pin is not fed by a potential datapath start such as a `set_input_delay` or a clocked flop.

missing_fanout_delay

A read-only boolean attribute that returns true if this budget pin does not feed a potential datapath end such as `set_output_delay` or a clocked flop.

name

A read-only string attribute that returns the same value as `full_name`.

no_path_fanin_pins

A read-only collection attribute that returns a collection of budget pins that feed this pin through an unconstrained segment. Will be undefined if there are none.

no_path_fanout_pins

A read-only collection attribute that returns a collection of budget pins that are fed by this pin through an unconstrained segment. Will be undefined if there are none.

object_class

A read-only string attribute that returns the name of this object class. Returns 'budget_pin' for budget_pin objects.

pin

A read-only collection attribute that returns the pin object that this data refers to.

pin_data

A read-only collection attribute that returns a collection of budget pin data objects for paths that pass through this pin. If no paths pass through this pin, then this attribute will not be defined.

pin_id

A read-only int attribute that returns an integer which is a unique identifier for this object. You can use this identifier to check that two objects refer to the same

b

data. This identifier is also used as an anchor in the output of "report_budget.html".

See Also

- [get_budgets](#)
- [collections](#)
- [get_attribute](#)
- [report_attributes](#)
- [report_budget](#)
- [set_budget_options](#)
- [set_pin_budget_constraints](#)
- [compute_budget_constraints](#)
- [write_budgets](#)

budget_pin_constraint_attributes

Description of the application defined budget_pin_constraint attributes.

Description

Attributes are properties assigned to objects such as pins, cells, and nets. They can be read-only or settable. A read-only attribute is informational; you cannot set its value. To modify a settable attribute, use the *set_attribute* command.

To determine the value of an attribute, use the *get_attribute* command. To see a list of all budget_pin_constraint attributes available, use the *list_attributes -application -class budget_pin_constraint* command.

budget_pin_constraint objects are returned by the *-pin_constraints* or *-all_pin_constraints* options of the *get_budgets* command.

Attributes for budget_pin_constraint

This section describes the budget_pin_constraint attributes.

constraint_id

A read-only int attribute that returns an integer which is a unique identifier for this pin constraint. You can use this identifier to check that two objects refer to the same data.

b

constraint_type

A read-only string attribute that returns a string which is one of "internal_percent", "internal_delay", "from_percent", "from_delay", "to_percent", "to_delay", or "none".

constraint_value

A read-only float attribute that returns a float value corresponding to the constraint type.

direction

A read-only string attribute that returns a string which is "input" for input pins and "output" for output pins. This attribute will return the appropriate direction of the associated path for inout pins.

from_clock

A read-only string attribute that returns the name of the clock object specified to the set_budget_pin_options command. Return "" if none.

from_clock_edge

A read-only string attribute that returns the string "rise", "fall" or "" depending on the options passed to the set_budget_pin_options command.

frozen

A read-only boolean attribute that returns True if the constraint has been frozen.

full_name

A read-only string attribute that returns the string "cons%d".

name

A read-only string attribute that returns the same value as full_name.

object_class

A read-only string attribute that returns the name of this object class. Returns 'budget_pin_constraint' for budget_pin_constraint objects.

path_type

A read-only string attribute that returns "feedthrough" for a constraint on a feedthrough path, "internal" for a constraint on an path start or end, and "" if the constraint applies to both.

pin

A read-only collection attribute that returns the pin object being constrained.

b

to_clock

A read-only string attribute that returns the name of the clock object specified to the `set_budget_pin_options` command. Return "" if none.

to_clock_edge

A read-only string attribute that returns the string "rise", "fall" or "" depending on the options passed to the `set_budget_pin_options` command.

See Also

- [get_budgets](#)
- [collections](#)
- [get_attribute](#)
- [report_attributes](#)
- [report_budget](#)
- [set_budget_options](#)
- [set_pin_budget_constraints](#)
- [compute_budget_constraints](#)
- [write_budgets](#)

budget_pin_data_attributes

Description of the application defined `budget_pin_data` attributes.

Description

Attributes are properties assigned to objects such as pins, cells, and nets. They can be read-only or settable. A read-only attribute is informational; you cannot set its value. To modify a settable attribute, use the *set_attribute* command.

To determine the value of an attribute, use the *get_attribute* command. To see a list of all `budget_pin_data` attributes available, use the *list_attributes -application -class budget_pin_data* command.

`budget_pin_data` objects are returned by the *-pin_data* and *-all_pin_data* options of the *get_budgets* command.

Attributes for budget_pin_data

This section describes the `budget_pin_data` attributes.

b

budget_pin

A read-only collection attribute that returns the `budget_pin` object that this data refers to.

capture_budget

A read-only float attribute that returns the budgeted portion of the path on the capturing side of this pin. Will be undefined if there is no budget.

capture_delay

A read-only float attribute that returns an estimate of the longest actual delay of the portion of the path on the capturing side of this pin.

capture_slack

A read-only float attribute that returns the slack in the budget after this pin.

constraint

A read-only collection attribute that returns the `budget_pin_constraint` object used to derive the budget for this path. Will be undefined in the pin is unconstrained.

direction

A read-only string attribute that returns a string which is "input" for input pins and "output" for output pins. This attribute will return the appropriate direction of the associated path for inout pins.

full_name

A read-only string attribute that returns the string "budget_pin_data".

hold_constraint

A read-only collection attribute that holds `budget_pin_constraint` for this data.

is_feedthrough

A read-only boolean attribute that returns true if the `pin_data` is for a path that feeds through the pin's block.

launch_budget

A read-only float attribute that returns a float which is the budgeted portion of the path on the launching side of this pin. Will be undefined if there is no budget.

launch_delay

A read-only float attribute that returns an estimate of the longest actual delay of the portion of the path on the launching side of this pin.

b

launch_slack

A read-only float attribute that returns a float which is launch_budget minus launch_delay. Will be undefined if there is no budget.

name

A read-only string attribute that is the same as full_name.

object_class

A read-only string attribute that returns the name of this object class. Returns 'budget_pin_data' for budget_pin_data objects.

path_type

A read-only collection attribute that returns the budget_path_type object that applies to the timing path for this data.

path_type_id

A read-only int attribute that is a unique identifier for the path type of this object. You can use this identifier to check that two objects refer to the same data. This identifier is also used as an anchor in the output of "report_budget -html".

pin

A read-only collection attribute that returns the pin object that this data refers to.

pin_id

A read-only int attribute that returns a unique identifier for this object. You can use this identifier to check that two objects refer to the same data. This identifier is also used as an anchor in the output of "report_budget -html".

See Also

- [get_budgets](#)
- [collections](#)
- [get_attribute](#)
- [report_attributes](#)
- [report_budget](#)
- [set_budget_options](#)
- [set_pin_budget_constraints](#)
- [compute_budget_constraints](#)
- [write_budgets](#)

b

budget_segment_attributes

Description of the application defined `budget_segment` attributes.

Description

Attributes are properties assigned to objects such as pins, cells, and nets. They can be read-only or settable. A read-only attribute is informational; you cannot set its value. To modify a settable attribute, use the `set_attribute` command.

To determine the value of an attribute, use the `get_attribute` command. To see a list of all `budget_segment` attributes available, use the `list_attributes -application -class budget_segment` command.

`budget_segment` objects are returned by the `-all_segments`, `-fanin_segments` and `-fanout_segments` option of the `get_budgets` command.

Attributes for budget_segment

This section describes the `budget_segment` attributes.

budget

A read-only float attribute that returns the delay that has been budgeted to this segment. Will be undefined if this segment does not have a budget constraint.

constraint

A read-only float attribute that returns the applied value if a delay constraint has been set for this segment using the `-busplans` option of the `compute_budget_constraints` command or by the `set_segment_budget_constraints` command. Otherwise, the attribute is undefined.

delay

A read-only float attribute that returns an estimate of the longest actual delay for timing represented by this budget segment.

fixed_delay

A read-only float attribute that returns the portion of delay that is 'fixed'. This includes delay through macros, clock skew and ocv, and flop setup and hold times.

fixed_slack

A read-only float attribute that returns `budget` minus `fixed_delay`.

b

from_block

A read-only string attribute that returns the name of the block where the segment starts. If the segment starts on a block boundary, the block on the other side of the boundary will be returned. For the top level, "" is returned.

from_pin

A read-only collection attribute that returns the pin object for the pin that is at the input of this segment. Will be undefined if this segment does not have a budget pin at its input.

from_pin_data

A read-only collection attribute that returns the `budget_pin_data` object that fans into this segment. Will be undefined if this segment does not have a budget pin at its input.

from_pin_id

A read-only int attribute that returns an integer which is a unique identifier for the `budget_pin_data` object that fans into this segment. You can use this identifier to check that two objects refer to the same data. This identifier is also used as an anchor in the output of `report_budget`. Will be undefined if this segment does not have a budget pin at its input.

full_name

A read-only string attribute that returns the string "seg%d".

is_extended_path

A read-only boolean attribute that returns True if segment is from `extend_paths` feature (may be undefined).

name

A read-only string attribute that returns the same value as `full_name`.

object_class

A read-only string attribute that returns the name of this object class. Returns 'budget_segment' for `budget_segment` objects.

path_type

A read-only collection attribute that returns the `budget_path_type` object for the path that this segment is part of.

path_type_id

A read-only int attribute that returns an integer which is a unique identifier for the `budget_path_type` object for the path that this segment is part of. You can use

b

this identifier to check that two objects refer to the same data. This identifier is also used as an anchor in the output of "report_budget.

rule_name

A read-only string attribute that returns the name of the busplan rule used to constrain this segment, if any. Otherwise, the attribute is undefined.

segment_id

A read-only int attribute that returns an integer which is a unique identifier for this budget_segment object. You can use this identifier to check that two objects refer to the same data. This identifier is also used as an anchor in the output of "report_budget.

slack

A read-only float attribute that returns budget minus delay.

through_block

A read-only string attribute that returns the name of the block where the segment passes through. For the top level, "" is returned.

to_block

A read-only string attribute that returns the name of the block where the segment ends. If the segment ends on a block boundary, the block on the other side of the boundary will be returned. For the top level, "" is returned.

to_pin

A read-only collection attribute that returns the pin object for the pin that is at the output of this segment. Will be undefined if this segment does not have a budget pin at its output.

to_pin_data

A read-only collection attribute that returns the budget_pin_data object that fans out of this segment. Will be undefined if this segment does not have a budget pin at its output.

to_pin_id

A read-only int attribute that returns an integer which is a unique identifier for the budget_pin_data object that fans out of this segment. You can use this identifier to check that two objects refer to the same data. This identifier is also used as an anchor in the output of "report_budget. Will be undefined if this segment does not have a budget pin at its output.

See Also

- [get_budgets](#)
- [budget_attributes](#) Summary of attributes available on tcl objects returned by `get_budgets`
- [get_attribute](#)
- [report_attributes](#)
- [report_budget](#)
- [set_budget_options](#)
- [set_pin_budget_constraints](#)
- [compute_budget_constraints](#)
- [write_budgets](#)

bump_bundle_attributes

Description of the application defined bump_bundle attributes.

Description

Attributes are properties assigned to objects such as pins, cells, and nets. They can be read-only or settable. A read-only attribute is informational; you cannot set its value. To modify a settable attribute, use the *set_attribute* command.

To determine the value of an attribute, use the *get_attribute* command. To see a list of all bump_bundle attributes available, use the *list_attributes -application -class bump_bundle* command.

Attributes for bump_bundle

This section describes the bump_bundle attributes.

anchor_cell

A read-only collection attribute that returns the anchor cell of the bump_bundle.

bbox

A read-only rect attribute that returns the bounding box of the contents of the bump_bundle.

bump_count

A read-only unsigned attribute that returns the number of bump instantiations in the bump_bundle.

bumps

A read-only collection attribute that returns the bumps of the bump_bundle.

chipselets

A read-only string attribute that returns a list of chipselet information describing the chipselet blocks on which bumps of the bump_bundle are instantiated.

full_name

A read-only string attribute that returns the full name of the bump_bundle.

name

A read-only string attribute that returns the name of the bump_bundle.

object_class

A read-only string attribute that returns "bump_bundle".

orientation

A read-only string attribute that returns the orientation of the bundle.

origin

A read-only coord attribute that returns the orientation of the bundle.

parent_block

A read-only collection attribute that returns the owner block object for the bump_bundle.

pattern

A read-only collection attribute that returns the bump_bundle_pattern the bump_bundle instantiates.

unique_bump_count

A read-only unsigned attribute that returns the number of unique physical bumps of the bump_bundle.

See Also

- [create_bump_bundle](#)
- [get_bump_bundles](#)
- [get_attribute](#)
- [list_attributes](#)
- [set_attribute](#)

bump_bundle_pattern_attributes

Description of the application defined bump_bundle_pattern attributes.

Description

Attributes are properties assigned to objects such as pins, cells, and nets. They can be read-only or settable. A read-only attribute is informational; you cannot set its value. To modify a settable attribute, use the *set_attribute* command.

To determine the value of an attribute, use the *get_attribute* command. To see a list of all bump_bundle_pattern attributes available, use the *list_attributes -application -class bump_bundle_pattern* command.

Attributes for bump_bundle_pattern

This section describes the bump_bundle_pattern attributes.

bumps

A read-only string attribute that returns a list of the bump specifications of the bump_bundle_pattern. Bump specifications are added to the bump_bundle_pattern using the *add_bump_to_bundle_pattern* command.

bump_count

A read-only unsigned attribute that returns the number of bump specifications in the bump_bundle_pattern.

bundles

A read-only collection attribute that returns the bundles that instantiate the bump_bundle_patterns.

chipslets

A read-only string attribute that returns a list of chiplet information. The chipslets are the sub-blocks where bumps are to be created and placed per the bump and mirror specifications of the bump_bundle_pattern.

full_name

A read-only string attribute that returns the full name of the bump_bundle_pattern.

mirrors

A read-only string attribute that returns a list of the mirror specifications of the bump_bundle_pattern. Mirror specifications are added to the bump_bundle_pattern using the *add_mirror_to_bundle_pattern* command.

mirror_count

A read-only unsigned attribute that returns the number of mirror specifications in the bump_bundle_pattern.

name

A read-only string attribute that returns the name of the bump_bundle_pattern.

net_groups

A read-only string attribute that returns a list of net_groups to which one or more bumps in the bump specifications are assigned.

object_class

A read-only string attribute that returns "bump_bundle_pattern".

parent_block

A read-only collection attribute that returns the owner block object for the bump_bundle_pattern.

ref_types

A read-only string attribute that returns a list of ref_types of the bumps in the bump_bundle_pattern's bump specifications. The ref_type of a bump is the name of the block the bump cell will reference.

See Also

- [create_bump_bundle_pattern](#)
- [add_bump_to_bundle_pattern](#)
- [add_mirror_to_bundle_pattern](#)
- [get_bump_bundle_patterns](#)
- [get_attribute](#)
- [list_attributes](#)
- [set_attribute](#)

bump_region_attributes

Description of the application defined bump_region attributes.

b

Description

Attributes are properties assigned to objects such as pins, cells, and nets. They can be read-only or settable. A read-only attribute is informational; you cannot set its value. To modify a settable attribute, use the *set_attribute* command.

To determine the value of an attribute, use the *get_attribute* command. To see a list of all bump_region attributes available, use the *list_attributes -application -class bump_region* command.

Attributes for bump_region

This section describes the bump_region attributes.

anchor

A read-only collection attribute that returns the anchor for which this object is the dependent_object. When non-empty, this object is the dependent_object of the anchor, and its location is therefore constrained by the anchor's anchor_object. See the *create_anchor(2)* command man page for more details about anchors.

array_size

A settable int_pair attribute that returns the number of columns and rows, respectively, of pseudo_bumps contained within the bump_region. When set by the user, the bump_region's boundary will be resized to accommodate the requested array_size, and the bump_region's *boundary_mode* attribute will automatically be set to "array_size".

bbox

A read-only rect attribute that returns the bounding box of the boundary and the contents of the bump_region.

boundary

A settable polygon attribute that returns the boundary of the bump_region. When set by the user, the bump_region's *boundary_mode* attribute will automatically be set to "boundary".

boundary_bbox

A read-only rect attribute that returns the bounding box of the boundary of the bump_region. This does not include the pseudo_bump boundaries.

boundary_margin

A settable distance_list attribute that returns the boundary margin of the bump_region. This is the inner margin distance between the bump_region's boundary and the pseudo_bumps contained by the bump_region. The value of this attribute may be a distance list of length 1, 2, or 4. If length 1, then that

b

distance will be enforced on all sides of the `bump_region`. If length 2, then the first value will be the distance enforced on the left and right of the `bump_region`, and the second value will be the distance enforced on the top and bottom of the `bump_region`. If length 4, then the values will be the distance enforced on the left, bottom, right, and top of the `bump_region`, respectively.

boundary_mode

A settable string attribute (with allowed values: `array_size` `boundary`). When "boundary", the `array_size` attribute of the `bump_region` will change as the boundary changes. When "array_size", the `array_size` attribute will, if possible, stay constant, and the boundary will automatically be adjusted to maintain the constant `array_size`.

bump_count

A read-only unsigned attribute that reports the number of non-hole pseudo_bumps contained by the `bump_region`.

content_bbox

A read-only rect attribute that returns the bounding box of the contents of the `bump_region`. This does not include the boundary.

content_type

A read-only string attribute (with allowed values: `bump` `bump_tsv` `tsv`) that returns the `content_type` of the `bump_region`.

full_name

A read-only string attribute that returns the full name of the `bump_region`.

has_pseudo_bumps

A read-only boolean attribute that indicates whether the `bump_region` has pseudo_bumps. This attribute is true unless the `bump_region`'s `content_type` is "tsv", which is set via the `-content_type` option of the `create_bump_region` command.

has_pseudo_tsvs

A read-only boolean attribute that indicates whether the `bump_region` has pseudo_tsvs. This attribute becomes true when the user sets the `tsv_pattern` attribute of the `bump_region`, or when the user sets the `-tsv_pattern` option of the `create_bump_region` command. It also becomes true when the user sets the `-content_type` option of the `create_bump_region` command to "bump_tsv" or "tsv". Once true, the attribute value remains true for the lifetime of the `bump_region`.

b

is_backside

A settable boolean attribute that indicates whether the bump_region is backside.

is_freeform

A read-only boolean attribute that indicates whether the bump_region is freeform. Freeform bump_regions have pseudo_bumps or pseudo_tsvs that are explicitly created and removed by the user, as opposed to having implicitly created pseudo_bumps or pseudo_tsvs based on the pitch and stagger of the referenced bump_region_pattern. A bump_region becomes freeform on creation if it references a bump_region_pattern of type "freeform".

is_frontside

A settable boolean attribute that returns whether the bump_region is frontside.

is_internal

A settable boolean attribute that returns whether the bump_region is internal with no external connections.

is_internal_ext

A settable boolean attribute that returns whether the bump_region is internal with external connections.

name

A settable string attribute that returns the name of the bump_region.

net_pattern

A settable string attribute that returns the default net assignments of the pseudo_bumps and pseudo_tsvs in the bump_region, in the form of a two dimensional list of nets representing the net pattern to be applied cyclically throughout the region. The format of the net pattern is `{{list_of_nets_on_row_0...} {list_of_nets_on_row_1...} ... {list_of_nets_on_row_n...}}`, which forms a two dimensional array of net assignments to apply to the pattern of pseudo_bumps. Each element of the list of nets on a row may be a collection of a net object, a net name, or the empty string ("") to indicate no net (a net "hole"). Each inner list of nets and net holes applies to the i'th row, cyclically, and the net assignments for that row will cycle through the list. If the net pattern contains any collections or substitution operators ("[]"), you must use the *list* command to form the lists, otherwise the substitutions will not be processed. See the *- net_pattern* argument section of the *create_bump_regions(2)* man page for net_pattern format examples.

b

object_class

A read-only string attribute that returns the name of this object class. Returns 'bump_region' for bump_region objects.

orientation

A settable string attribute (with allowed values: R0 R90 R180 R270 MX MXR90 MY MYR90) that returns the orientation of the bump_region.

parent_block

A read-only block collection attribute that returns the owner block object for this bump_region.

pattern

A settable bump_region_pattern collection attribute that returns the bump_region_pattern the bump_region references, which determines the spacing and shape of the pseudo_bumps of the bump_region. This attribute may be set to a different bump_region_pattern, but the new bump_region_pattern must have the same *type* attribute value as the original bump_region_pattern.

pattern_anchor

A settable string attribute (with allowed values: block region) that returns the pattern_anchor of the bump_region. If the value is "region", the pattern's pitch and stagger cycling starts at the lower-left of the bump_region boundary's bounding box. If the value is "block", the pattern's pitch and stagger cycling starts at the block origin.

pattern_offset

A settable distance_list attribute that returns a distance list of length 2 specifying the pattern offset of the bump region, where the first element is the offset in the x direction, and the second element the offset in the y direction. This value is useful when the pattern has multiple alternating pitch or stagger values, as it specifies a shift in the pattern position with respect to the region boundary. When the pattern offset is set to a positive value in a particular direction, then the pattern appears as though the region boundary was moved in that positive direction compared to the pattern (even though the boundary coordinates stay the same) - conversely, it would appear as though the pattern were shifted by that distance in the opposite direction within the boundary.

port_pattern

A settable string attribute that returns the default port assignments of the pseudo_bumps in the bump_region, in the form of a two dimensional list of ports representing the port pattern to be applied cyclically throughout the region. The

b

format of the port pattern is similar to the format of the *net_pattern* attribute, except with ports instead of nets.

tsv_count

A read-only unsigned attribute that returns an integer reporting the number of non-hole pseudo_tsvs contained by the bump_region.

tsv_pattern

A settable string attribute that enables pseudo_tsvs in the bump_region, and specifies the default pseudo_tsv_def and offset configuration of the pseudo_tsvs, in the form of a two dimensional list. See the *-tsv_pattern* argument section of the *create_bump_regions(2)* man page for tsv_pattern format description and examples.

z_offset

A settable integer attribute that specifies the z offset of the bump_region, which corresponds to the z position of the bump_region's contents.

The value cannot be negative.

See Also

- [create_bump_region](#)
- [get_bump_regions](#)
- [get_attribute](#)
- [list_attributes](#)
- [set_attribute](#)

bump_region_pattern_attributes

Description of the application defined bump_region_pattern attributes.

Description

Attributes are properties assigned to objects such as pins, cells, and nets. They can be read-only or settable. A read-only attribute is informational; you cannot set its value. To modify a settable attribute, use the *set_attribute* command.

To determine the value of an attribute, use the *get_attribute* command. To see a list of all bump_region_pattern attributes available, use the *list_attributes -application -class bump_region_pattern* command.

b

Attributes for bump_region_pattern

This section describes the bump_region_pattern attributes.

bump_radius

A settable distance attribute that returns the radius of the bumps in the bump_region_pattern.

bump_shape

A settable string attribute (with allowed values: circle hexagon octagon pentagon septagon square triangle) that returns the shape of the bumps in the bump_region_pattern.

column_stagger_y

A settable distance_list attribute that returns the y offset added to bumps in each column of the bump_region_pattern. The i'th value in the list applies to the i'th column, cyclically.

full_name

A read-only string attribute that returns the full name of the bump_region_pattern.

is_default

A settable boolean attribute that indicates whether the bump_region_pattern is the default bump_region_pattern in the block. The default bump_region_pattern is implicitly set as the reference pattern in every *create_bump_region* command invocation that does not specify a *-pattern* option.

name

A settable string attribute that returns the name of the bump_region_pattern.

object_class

A read-only string attribute that returns the name of this object class. Returns 'bump_region_pattern' for bump_region_pattern objects.

parent_block

A read-only block collection attribute that returns the owner block object for this bump_region_pattern.

pitch_x

A settable distance_list attribute that returns the spacing between subsequent columns of bumps in the bump_region_pattern, where the i'th distance value is the distance between the i'th and the (i+1)'th columns, cyclically.

b

pitch_y

A settable distance_list attribute that returns the spacing between subsequent rows of bumps in the bump_region_pattern, where the i'th distance value is the distance between the i'th and the (i+1)'th rows, cyclically.

row_stagger_x

A settable distance_list attribute that returns the x offset added to bumps in each row of the bump_region_pattern. The i'th value in the list applies to the i'th row, cyclically.

type

A read-only string attribute (with allowed values: freeform structured) that returns the type of the bump_region_pattern. Structured bump_region_patterns describe a regular pattern of pseudo_bumps in the bump_regions that reference the bump_region_pattern. Freeform bump_region_patterns describe pseudo_bump default values only, but are without pitch or stagger values. Bump_regions that reference freeform bump_region_patterns have *is_freeform* values of "true", and require explicit creation and removal of pseudo_bumps.

See Also

- [get_bump_region_patterns](#)
- [get_attribute](#)
- [list_attributes](#)
- [set_attribute](#)

bundle_attributes

Description of the application defined bundle attributes.

Description

Attributes are properties assigned to objects such as pins, cells, and nets. They can be read-only or settable. A read-only attribute is informational; you cannot set its value. To modify a settable attribute, use the *set_attribute* command.

To determine the value of an attribute, use the *get_attribute* command. To see a list of all bundle attributes available, use the *list_attributes -application -class bundle* command.

Attributes for bundle

This section describes the bundle attributes.

bundle_type

A read-only string attribute that returns the type of objects associated with this bundle.

constraint_groups

A read-only constraint_group collection attribute that returns the Custom Router constraint groups that contain the net.

expanded_objects

A read-only collection attribute that returns all the objects associated with this bundle but flattened out.

name

A read-only string attribute that returns the name of the bundle.

object_class

A read-only string attribute that returns the name of this object class. Returns 'bundle' for bundle objects.

object_count

A read-only int attribute that returns the number of objects associated with this bundle.

objects

A read-only collection attribute that returns all the objects associated with this bundle.

stop_hierarchy

A settable string attribute that returns the name of the cells whose internal connections will be ignored in design planning.

topological_constraints

A read-only topological_constraint collection attribute that returns the Custom Router topological constraints associated with the net.

See Also

- [create_bundle](#)
- [get_attribute](#)
- [list_attributes](#)
- [set_attribute](#)

busplan_bus_attributes

Description of the application defined busplan_bus attributes.

Description

Attributes are properties assigned to objects such as pins, cells, and nets. They can be read-only or settable. A read-only attribute is informational; you cannot set its value. To modify a settable attribute, use the *set_attribute* command.

To determine the value of an attribute, use the *get_attribute* command. To see a list of all busplan_bus attributes available, use the *list_attributes -application -class busplan_bus* command.

Attributes for busplan_bus

This section describes the busplan_bus attributes.

all_nets

A read-only collection attribute that returns all the nets associated with this busplan bus.

all_top_nets

A read-only collection attribute that returns all the top level nets associated with this busplan bus.

branching

A read-only boolean attribute that indicates if this busplan bus is a branching bus.

capture_block_budget

A settable float attribute that returns the time spent in the capture block.

clock_name

A read-only string attribute that returns the clock that is associated with this busplan bus.

constraint_to

A read-only string attribute that is another busplan bus which is constrained to this busplan bus.

constraint_type

A read-only string attribute that indicates if this busplan bus has a constraint and the type of constraint that it has. Constraints supported are "<", "<=", ">", ">=", and "=".

b

elements

A read-only collection attribute that returns the busplan elements of this busplan bus.

end_elements

A read-only collection attribute that returns the busplan elements at the end of this busplan bus.

full_name

A read-only string attribute that returns the name of this busplan bus.

horizontal_register_spacing

A read-only float attribute that returns the horizontal register spacing associated with this busplan bus.

in_MIB

A read-only boolean attribute that indicates if this bus goes through (ie feedthrough) at least one Multiple Instanciated Block (MIB).

in_MIB_block_list

A read-only collection attribute that returns a collection of MIBs which this busplan goes through (ie feedthrough).

in_MIB_parent

A read-only boolean attribute that indicates if this busplan bus is the MIB parent. MIB parent means this busplan bus is sharing MIBs pins with other busplan buses and this busplan bus is the largest such bus. It is a superset of the other busplan buses.

in_MIB_related_buses

A read-only string attribute that returns a collection of busplan buses which are MIB related to this busplan bus. MIB related means that these buses use the same pins on different instances of the same MIBs.

launch_block_budget

A settable float attribute that returns the time spent in the launch block.

name

A read-only string attribute that returns the name of this busplan bus.

object_class

A read-only string attribute that returns the name of this object class. Returns 'busplan_bus' for busplan_bus objects.

b

path_nets

A read-only collection attribute that returns the nets associated with the first bit of this busplan bus.

rule

A read-only collection attribute that returns the net estimation rule for this busplan bus.

rule_name

A read-only string attribute that returns the name of the net estimation rule for this busplan bus.

start_element

A read-only collection attribute that returns the busplan element at the start of this busplan bus.

timing_valid

A read-only boolean attribute that returns whether the timing on this busplan bus is up to date.

vertical_register_spacing

A read-only float attribute that returns the vertical register spacing associated with this busplan bus.

width

A read-only int attribute that returns the number of bits in this busplan bus.

wirelength

A read-only float attribute that returns the manhattan wirelength for this busplan bus.

worst_slack

A read-only float attribute that returns the worst slack on any path in this busplan. The path could be from start element to register, register to register, register to end element, or start element to end element (in case no registers).

See Also

- [create_busplans](#)
- [get_attribute](#)
- [list_attributes](#)
- [set_attribute](#)

busplan_element_attributes

Description of the application defined busplan_element attributes.

Description

Attributes are properties assigned to objects such as pins, cells, and nets. They can be read-only or settable. A read-only attribute is informational; you cannot set its value. To modify a settable attribute, use the *set_attribute* command.

To determine the value of an attribute, use the *get_attribute* command. To see a list of all busplan_element attributes available, use the *list_attributes -application -class busplan_element* command.

Attributes for busplan_element

This section describes the busplan_element attributes.

bits

A read-only string attribute that returns which bits of a bus group have objects.

block

A read-only collection attribute that returns the physical block containing the group.

bus

A read-only collection attribute that returns the bus that contains this element.

cell

A read-only collection attribute that returns the physical cell instance containing the group.

center

A read-only string attribute that returns the coordinate of the center of the element objects.

center_in_block

A read-only string attribute that returns the block coordinate of the center of the element objects.

cycle

A read-only int attribute that returns the register cycle.

b

fanin_element

A read-only collection attribute that returns the bus element that is a fanin of this group.

fanout_elements

A read-only collection attribute that contains bus elements that are at the fanout of this group.

fanout_nets

A read-only collection attribute that contains bus nets that fan out of this group.

fixed

A settable boolean attribute that returns whether the bus element is fixed for auto movement features.

full_name

A read-only string attribute that returns the full name of this object.

group_type

A read-only string attribute that returns the type of object in a group or virtual group.

in_MIB

A read-only boolean attribute that returns whether the element is in an MIB bus group.

in_MIB_end_elements

A read-only collection attribute that returns the feed through end group(s) of MIB bus element.

in_MIB_related_elements

A read-only collection attribute that returns the related MIB elements of this element.

in_MIB_start_element

A read-only collection attribute that returns the feed through start group of MIB bus element.

is_end

A read-only boolean attribute that returns whether this is a bus at end.

is_redundant

A read-only boolean attribute that indicates whether this register is redundant.

is_start

A read-only boolean attribute that indicates whether this is the bus source.

location

A read-only string attribute that returns the planned location for the element.

name

A read-only string attribute that returns the name of this object.

object_class

A read-only string attribute that returns the name of this object class. Returns 'busplan_element' for busplan_element objects.

objects

A read-only collection attribute that contains cells, ports, or pins in the group.

objects_in_block

A read-only collection attribute that contains block-level cells, ports, or pins in the group.

segment_stretch

A read-only string attribute that returns the distance to adjust the fanin segment to meet timing.

stage_slack

A read-only float attribute that returns the timing slack between blocks and/or flops.

type

A read-only string attribute that returns the type of this element (group, virtual, guide).

width

A read-only int attribute that returns the width of a group of objects.

See Also

- [get_attribute](#)
- [list_attributes](#)
- [set_attribute](#)

C

cell_array_pattern_attributes

Description of the application defined cell_array_pattern attributes.

Description

Attributes are properties assigned to objects such as pins, cells, and nets. They can be read-only or settable. A read-only attribute is informational; you cannot set its value. To modify a settable attribute, use the *set_attribute* command.

To determine the value of an attribute, use the *get_attribute* command. To see a list of all cell_array_pattern attributes available, use the *list_attributes -application -class cell_array_pattern* command.

Attributes for cell_array_pattern

This section describes the cell_array_pattern attributes.

block

A read-only block collection attribute that returns the set of allowable orientations for the lib cell.

full_name

A read-only string attribute that returns the full name of the cell_array_pattern.

halo

A settable float attribute that returns the halo of the cell_array_pattern. The attribute value is the multiplier of the cell_array_pattern width and height, which indicates the width of the halo in the respective horizontal and vertical directions.

horizontal_spacing

A settable int attribute that returns a string specifying the horizontal spacing of the cell_array_pattern, in units of sites. The attribute value is the number of sites along each row of the cell_array_pattern between each of eligible sites for the lib_cell placement within the pattern.

lib_cell_name

A settable string attribute that returns the name of the lib_cell to be instantiated within the pattern.

name

A settable string attribute that returns the name of the cell_array_pattern.

c

object_class

A read-only string attribute that returns the name of this object class. Returns 'cell_array_pattern' for cell_array_pattern objects.

row_column_ratio

A settable int_pair attribute that specifies the ratio of rows to columns in the cell_array_pattern. Each value in the integer pair must be ≥ 1 . Note that the actual number of rows and columns can be any positive integer multiple of these values.

type

A settable string attribute (with allowed values: checkerboard solid) that returns the type of the cell_array_pattern's nodes.

vertical_spacing

A settable int attribute that specifies the vertical spacing of the cell_array_pattern, in units of sites. The attribute value is the number of sites along each column of the cell_array_pattern between each of the eligible sites for the lib_cell placement within the pattern.

See Also

- [get_cell_array_patterns](#)
- [get_attribute](#)
- [list_attributes](#)
- [set_attribute](#)

cell_attributes

Description of the application defined cell attributes.

Description

Attributes are properties assigned to objects such as pins, cells, and nets. They can be read-only or settable. A read-only attribute is informational; you cannot set its value. To modify a settable attribute, use the *set_attribute* command.

To determine the value of an attribute, use the *get_attribute* command. To see a list of all cell attributes available, use the *list_attributes -application -class cell* command.

Attributes for cell

This section describes the cell attributes.

c

abstract_dont_touch

A read-only boolean attribute that returns true if the cell has a dont_touch status based on being inside an abstract.

actual_internal_power_derate_factor

A read-only float attribute that returns the internal power derate value used in power analysis for the specified cell. The value is based on current timer status, the attribute does not update the timer. For the cell without explicit internal power derating factor, the value returned in decreasing order of precedence will be the internal power derate value for its hierarchical parent, or the corresponding library cell, or the design.

actual_leakage_power_derate_factor

A read-only float attribute that returns the leakage power derate value used in power analysis for the specified cell. The value is based on current timer status, the attribute does not update the timer. For the cell without explicit leakage power derating factor, the value returned in decreasing order of precedence will be the leakage power derate value for its hierarchical parent, or the corresponding library cell, or the design.

actual_switching_power_derate_factor

A read-only float attribute that returns the switching power derate value used in power analysis for the specified cell. The value is based on current timer status, the attribute does not update the timer. For the cell without explicit switching power derating factor, the value returned in decreasing order of precedence will be the switching power derate value for its hierarchical parent, or the corresponding library cell, or the design.

alignment_marker_rule_name

A settable string attribute that returns the name of the alignment marker rule used to create this marker cell.

allowable_orientation

A read-only orientations attribute that returns the allowable orientations for the cell.

anchor

A read-only collection attribute that returns the anchor for which this object is the dependent_object. When non-empty, this object is the dependent_object of the anchor, and its location is therefore constrained by the anchor's anchor_object. See the *create_anchor(2)* command man page for more details about anchors.

c

annotated_internal_power

A read-only double attribute that returns the sum of the total annotated internal power for the cell.

annotated_leakage_power

A read-only double attribute that returns the sum of the total annotated leakage power for the cell.

annotated_switching_power

A read-only double attribute that returns the sum of the total annotated switching power for the cell.

area

A read-only area attribute that returns the area of the cell.

aspect_ratio

A read-only double attribute that returns the ratio of the height over the width of the cell.

base_name

A read-only string attribute that returns the simple name of this cell, excluding hierarchy. Same as name.

bbox

A read-only rect attribute that returns the bounding box of the cell, which is computed as the smallest rectangle that encloses all the physical shapes of the cell, including its keepout margins. When computing the bounding box, the tool considers not only the keepout margins and physical shapes that are fully inside the cell boundary, but also the physical shapes associated with the cell that are partially or fully outside the cell boundary. Even text can sometimes be included as a shape. The format of a rectangle specification is `{{llx lly} {urx ury}}`, which specifies the lower-left and upper-right corners of the rectangle.

block_grid

A settable string attribute that returns the block grid that the cell is associated with.

block_placement_type

A settable string attribute (with allowed values: center distributed regular) that describes how the cell should be treated by the *create_placement -floorplan* and *shape_blocks* commands. This attribute exists only for the blocks; it does not exist for standard cells or hard macros. Valid values are regular, center, and distributed. If *block_placement_type* is regular, it is shaped by *shape_blocks*,

c

and not moved by `create_placement -floorplan`. If `block_placement_type` is `center`, it is not shaped by `shape_blocks`, while `create_placement -floorplan` will place it near the middle of its parent block. If `block_placement_type` is `distributed`, it is not shaped by `shape_blocks`, while `create_placement -floorplan` will place it according to its connectivity. This attribute defaults to `regular`.

blockage_group_id

A settable `int` attribute that returns an integer in the range [0-8] that specifies the active group id of the reference block's boundary routing blockages. Defaults to 0.

boundary

A settable `coord_list` attribute that returns the user-specified boundary of the cell. The format of a coordinate list specification is `{{x1 y1} {x2 y2}...}`.

boundary_bbox

A read-only `rect` attribute that returns the rectangular boundary of the cell. For rectilinear cells, it is the smallest rectangle that covers the boundary of the cell. The format of a rectangle specification is `{{llx lly} {urx ury}}`, which specifies the lower-left and upper-right corners of the rectangle.

boundary_bounding_box

A read-only `bounding_box` collection attribute that returns the rectangular boundary of the cell. For rectilinear cells, it is the smallest rectangle that covers the boundary of the cell. The format of the collection specifies the lower-left and upper-right corners of the rectangle.

boundary_optimization

A read-only `string` attribute that indicates if boundary optimization is allowed.

boundary_polyrects

A read-only `poly_list` attribute that returns the boundary polyrects of a `lib_cell`. The no. of polyrects could be >1 only when disjoint

bounding_box

A read-only `bounding_box` collection attribute that returns the bounding box of the cell, which is computed as the smallest rectangle that encloses all the physical shapes of the cell, including its keepout margins. When computing the bounding box, the tool considers not only the keepout margins and physical shapes that are fully inside the cell boundary, but also the physical shapes associated with the cell that are partially or fully outside the cell boundary. Even text can sometimes be included as a shape. The format of the collection specifies the lower-left and upper-right corners of the rectangle.

c

bounds

A read-only bound collection attribute that returns the group and move bounds to which the cell belongs.

bump_check_suppression_id

A settable string attribute that suppresses a list of message IDs from being reported for that object. The attribute takes a comma-separated list of message IDs. For example "3DIC-123,3DIC-456".

bus

A read-only cell_bus collection attribute that returns the cell bus. Only defined for cells with the is_bus_bit attribute true.

busplan_trace_through

A settable boolean attribute that returns whether a cell is to be traced through for pipeline register planning.

clock_gating_integrated_cell

A read-only string attribute that returns the name of the integrated clock gating if the lib cell is of type integrated clock gating.

clock_path_target_lib_cell_exclusions

A read-only lib_cell collection attribute that returns the library cells in the target library subset's dont-use list assigned to the cell to constrain the clock paths. This attribute contains only the library cell exclusions which were specified in the target library subset assigned to the cell. It does not contain library cell exclusions that are inherited from target library subsets assigned to higher levels of hierarchy.

clock_path_target_lib_cell_subset

A read-only lib collection attribute that returns the library cells in the target library subset's only-here list assigned to the cell to constrain the clock paths. This attribute contains only the library cells which were specified in the target library subset assigned to the cell. It does not contain library cells which are inherited from target library subsets assigned to higher levels of hierarchy.

clock_path_target_library_subset

A read-only lib collection attribute that returns the lib objects in the target library subset assigned to the cell to constrain the clock paths. This attribute contains only the libraries which were explicitly specified in the target library subset assigned to the cell. It does not contain libraries which are inferred from the reference library list or inherited from target library subsets assigned to higher levels of hierarchy.

c

color

A settable int attribute that returns the color index of the cell.

connected_bumps

A settable cell collection attribute that returns the associated bump instances of this bump instance.

const_prop_off

A read-only boolean attribute that indicates if constant optimization is not allowed.

constant_propagation

A read-only boolean attribute that indicates if constant optimization is allowed.

cts_size_in_place

A read-only boolean attribute that indicates this is part of the clock network and cannot experience significant movement in the flow after CTS has marked this attribute. Only sizing and small movements are allowed.

data_path_target_lib_cell_exclusions

A read-only lib_cell collection attribute that returns the library cells in the target library subset's dont-use list assigned to the cell to constrain the data paths. This attribute contains only the library cell exclusions which were specified in the target library subset assigned to the cell. It does not contain library cell exclusions which are inherited from target library subsets assigned to higher levels of hierarchy.

data_path_target_lib_cell_subset

A read-only lib collection attribute that returns the library cells in the target library subset's only-here list assigned to the cell to constrain the data paths. This attribute contains only the library cells which were specified in the target library subset assigned to the cell. It does not contain library cells which are inherited from target library subsets assigned to higher levels of hierarchy.

data_path_target_library_subset

A read-only lib collection attribute that returns the lib objects in the target library subset assigned to the cell to constrain the data paths. This attribute contains only the libraries which were explicitly specified in the target library subset assigned to the cell. It does not contain libraries which are inferred from the reference library list or inherited from target library subsets assigned to higher levels of hierarchy.

c

del_unloaded_gate_off

A read-only boolean attribute that indicates if unloaded gate deletion is disabled.

design_type

A read-only string attribute (with allowed values: 3dic abstract analog analysis black_box bridge corner cover die diode end_cap feedthrough fill filler flip_chip_driver flip_chip_pad ftv interposer lib_cell macro module package pad pad_spacer pcb rdl_fanout rtl substrate thermal tsv vib well_tap) that returns the design type of the cell.

disable_timing

A read-only boolean attribute that returns True if the cell or any of its pins have been marked as *disable_timing* by the *set_disable_timing* command.

dont_retime

A read-only boolean attribute that indicates if retiming is disabled.

dont_touch

A settable boolean attribute that returns true if the cell is protected from change by the *set_dont_touch* command. When this attribute is set to true, the cell cannot be resized, removed, or replaced by the ECO fixing commands (*fix_eco_drc*, *fix_eco_power*, and *fix_eco_timing*). However, it can still be removed by the *remove_cell* command. This attribute exists for a cell only after the attribute is created by the *set_dont_touch* command.

dont_use_for_shift_registers

A settable boolean attribute that, if enabled, disables retiming of shift registers.

early_fall_cell_check_derate_factor

A read-only float attribute that returns the derate factor applied for timing checks (setup, hold, recovery, removal) on this cell for falling edge early analysis. Timing check derates are applied using *set_timing_derate -cell_check*.

early_fall_cell_check_derate_factor_source

A read-only string attribute that indicates the source scope of the *set_timing_derate -early -fall -cell_check* constraint on this cell. The source could be from a constraint on the design, a hierarchical cell, a library cell, or a leaf cell.

early_fall_cell_check_derate_file_line_info

A read-only string attribute that returns a Tcl list of two elements, the first showing the source file path and the second showing the source file line

c

number, for the `set_timing_derate -cell_check -early -fall` constraint applied on this cell.

early_fall_clk_cell_derate_factor

A read-only float attribute that returns the derate factor applied on this cell for falling edge early analysis if it is in a clock path. Timing derates on clock logic are applied using `set_timing_derate -clock`.

early_fall_clk_cell_derate_factor_file_line_info

A read-only string attribute that returns a Tcl list of two elements, the first showing the source file path and the second showing the source file line number, for the `set_timing_derate -clock -early -fall` constraint applied on this cell.

early_fall_clk_cell_derate_factor_source

A read-only string attribute that indicates the source scope of the `set_timing_derate -early -fall -clock` constraint on this cell. The source could be from a constraint on the design, a hierarchical cell, a library cell, or a leaf cell.

early_fall_data_cell_derate_factor

A read-only float attribute that returns the derate factor applied on this cell for falling edge early analysis if it is in the datapath. Timing derates on datapath are applied using `set_timing_derate -data`.

early_fall_data_cell_derate_factor_file_line_info

A read-only string attribute that returns a Tcl list of two elements, the first showing the source file path and the second showing the source file line number, for the `set_timing_derate -data -early -fall` constraint applied on this cell.

early_fall_data_cell_derate_factor_source

A read-only string attribute that indicates the source scope of the `set_timing_derate -early -fall -data` constraint on this cell. The source could be from a constraint on the design, a hierarchical cell, a library cell, or a leaf cell.

early_rise_cell_check_derate_factor

A read-only float attribute that returns the derate factor applied for timing checks (setup, hold, recovery, removal) on this cell for rising edge early analysis. Timing check derates are applied using `set_timing_derate -cell_check`.

early_rise_cell_check_derate_factor_file_line_info

A read-only string attribute that returns a Tcl list of two elements, the first showing the source file path and the second showing the source file line number, for the `set_timing_derate -cell_check -early -rise` constraint applied on this cell.

c

early_rise_cell_check_derate_factor_source

A read-only string attribute that indicates the source scope of the `set_timing_derate -early -rise -cell_check` constraint on this cell. The source could be from a constraint on the design, a hierarchical cell, a library cell, or a leaf cell.

early_rise_clk_cell_derate_factor

A read-only float attribute that returns the derate factor applied on this cell for rising edge early analysis if it is in a clock path. Timing derates on clock logic are applied using `set_timing_derate -clock`.

early_rise_clk_cell_derate_factor_file_line_info

A read-only string attribute that returns a Tcl list of two elements, the first showing the source file path and the second showing the source file line number, for the `set_timing_derate -clock -early -rise` constraint applied on this cell.

early_rise_clk_cell_derate_factor_source

A read-only string attribute that indicates the source scope of the `set_timing_derate -early -rise -clock` constraint on this cell. The source could be from a constraint on the design, a hierarchical cell, a library cell, or a leaf cell.

early_rise_data_cell_derate_factor

A read-only float attribute that returns the derate factor applied on this cell for rising edge early analysis if it is in the datapath. Timing derates on datapath are applied using `set_timing_derate -data`.

early_rise_data_cell_derate_factor_file_line_info

A read-only string attribute that returns a Tcl list of two elements, the first showing the source file path and the second showing the source file line number, for the `set_timing_derate -data -early -rise` constraint applied on this cell.

early_rise_data_cell_derate_factor_source

A read-only string attribute that indicates the source scope of the `set_timing_derate -early -rise -data` constraint on this cell. The source could be from a constraint on the design, a hierarchical cell, a library cell, or a leaf cell.

eco_change_status

A settable string attribute (with allowed values: `add_buffer` `add_buffer_on_route` `change_link` `create_cell` `eco_legalized` `eco_placed` `move_cell` `size_cell`) that returns the eco status of the cell.

c

eco_cluster_id

A settable int attribute that is annotated and reported by `report_eco_physical_changes` -type cluster, which groups eco cells by the distance between each other. It is the number of an eco cell cluster. You can get all eco cells in a cluster through this attribute.

eco_net_group_id

A settable int attribute that is set by `add_group_repeaters` automatically. After inserting repeaters on each net group, it annotates net group id onto the newly created repeaters of this net group.

eco_repeater_group_id

A read-only int attribute that is generated by `set_repeater_group` to define a group of repeaters with outline for placement. Command `place_group_repeaters` can accept repeater group IDs and place repeaters by groups.

effective_process_label_early

A read-only string attribute that returns the actual process label value used for early analysis for this cell in the `current_corner`. This can be different from the `set_process_label` constraint applied to this cell if that process label mismatches the one used for `current_corner`. If they mismatch, then the attribute `mismatched_process_label_min` on this cell will be true.

effective_process_label_late

A read-only string attribute that returns the actual process label value used for late analysis for this cell in the `current_corner`. This can be different from the `set_process_label` constraint applied to this cell if that process label mismatches the one used for `current_corner`. If they mismatch, then the attribute `mismatched_process_label_max` on this cell will be true.

effective_process_number_early

A read-only float attribute that returns the actual process number value used for early analysis for this cell in the `current_corner`. This can be different from the `set_process_number` constraint applied to this cell if that process number mismatches the one used for `current_corner`. If they mismatch, then the attribute `mismatched_process_number_min` on this cell will be true.

effective_process_number_late

A read-only float attribute that returns the actual process number value used for late analysis for this cell in the `current_corner`. This can be different from the `set_process_number` constraint applied to this cell if that process number mismatches the one used for `current_corner`. If they mismatch, then the attribute `mismatched_process_number_max` on this cell will be true.

effective_temperature_early

A read-only float attribute that returns the actual temperature value used for early analysis for this cell in the `current_corner`. This can be different from the `set_temperature` constraint applied to this cell if that temperature mismatches the one used for `current_corner`. If they mismatch, then the attribute `mismatched_temperature_min` on this cell will be true.

effective_temperature_late

A read-only float attribute that returns the actual temperature value used for late analysis for this cell in the `current_corner`. This can be different from the `set_temperature` constraint applied to this cell if that temperature mismatches the one used for `current_corner`. If they mismatch, then the attribute `mismatched_temperature_max` on this cell will be true.

effective_voltage_early

A read-only float attribute that returns the actual voltage value used for early analysis for this cell in the `current_corner`. This can be different from the `set_voltage` constraint applied to this cell if that voltage mismatches the one used for `current_corner`. If they mismatch, then the attribute `mismatched_voltage_min` on this cell will be true.

effective_voltage_late

A read-only float attribute that returns the actual voltage value used for late analysis for this cell in the `current_corner`. This can be different from the `set_voltage` constraint applied to this cell if that voltage mismatches the one used for `current_corner`. If they mismatch, then the attribute `mismatched_voltage_max` on this cell will be true.

equal_opposite_propagation

A read-only boolean attribute that controls `equal_opposite` propagation.

escaped_full_name

A read-only string attribute that returns the full name of the cell including hierarchy, with all literal hierarchy characters escaped with a backslash.

esd_sources

A read-only string attribute that contains the list of Electro Static Source (ESD) name and worst case voltage value assigned to this cell.

exclude_multibit

A read-only string attribute that defines the special spacing rules, as set by the `set_placement_spacing_rule` command

c

exclude_multibit_pamb

A read-only string attribute that returns 1 if the legalizer checks special spacing rules for the cell

exclude_multibit_rtl

A read-only string attribute that indicates whether the cells are being excluded from RTL banking step. The attribute is set by `set_multibit_options -stage rtl -exclude`.

flat_net_count

A read-only int attribute that is only available for hierarchical cell. The value is the total number of flat nets for the whole hierarchy tree under this cell.

for_floorplan_rules

A read-only boolean attribute that returns whether the cell is created for floorplan rules. This attribute exists only for the hard macros; it does not exist for standard cells. This attribute is not user-settable, it is only set for the cells added by the `fix_floorplan_rules` command. The default value is false.

fs_dont_use

A settable boolean attribute that indicates whether, in freeze-silicon eco flow, `place_freeze_silicon` and `map_freeze_silicon` will not map eco cells to spare cells with this attribute. You filter out the spare cells you don't want to use by this attribute.

fs_mapped_cell_name

A read-only string attribute that is for user manageable spare cell mapping in freeze-silicon eco flow. When option `-no_spare_cell_swapping` is turned on, `place_freeze_silicon` command will map eco cells to spare cells, move eco cells onto spare cells, but not swap out spare cells. The mapping relationship between an eco cell and its corresponding spare cell is annotated by this attribute.

full_name

A read-only string attribute that returns the full name of the cell including hierarchy.

glc_area

A read-only double attribute that returns the glue logic cell area queryable from glue logic wrapper hierarchy or glue logic model leaf cell area.

c

glitch_power

A read-only double attribute that returns the glitch power of the cell in watts. Glitch power is considered part of the dynamic power.

group_repeater_driver

A read-only string attribute that indicates the connection between cells in repeater groups. The value is the full name of the driver cell of this cell. It could be created by `create_repeater_groups`, `set_repeater_group`, or `place_group_repeater`. It is used by `place_group_repeater`.

group_repeater_loads

A read-only string attribute that indicates the connection between cells in repeater groups. The value is the full names of the load cells of this cell. It could be created by `create_repeater_groups`, `set_repeater_group`, or `place_group_repeater`. It is used by `place_group_repeater`.

group_repeater_placed

A read-only boolean attribute that indicates if a group of repeaters are placed or not. It is automatically set by command `place_group_repeater` and `unplace_group_repeater`.

groups

A read-only group collection attribute that returns the groups the cell is a part of.

hard_macro_count

A read-only int attribute that is only available for hierarchical cell. The value is the total number of hard macros directly under this cell, not including hard macros in its children's hierarchies.

has_child_logical_hierarchy

A read-only boolean attribute that is only available for hierarchical cells. True if the cell has other hierarchy under it in the logical hierarchy tree.

has_child_physical_hierarchy

A read-only boolean attribute that is only available for hierarchical cells. True if the cell has physical hierarchy under it in the logical hierarchy tree.

has_multi_power_rails

A read-only boolean attribute that returns true if the cell has multiple power rails.

has_timing_model

A read-only boolean attribute that returns true if the cell has a timing model.

c

has_variant_ref

A read-only boolean attribute that returns true if the cell has variant reference.

hdl_canonical_params

A read-only string attribute that exists on parameterized elaborated design. It shows the actual and default attribute values of the design with the values displayed in normalized hexadecimal form. The values are returned as a single string. If the design has no parameters, the attribute is still defined but returns an empty string.

hdl_file

A read-only string attribute that returns the original RTL file name for the cell.

hdl_group

A read-only group collection attribute that returns the hdl group the cell is a part of.

hdl_hier

A read-only string attribute that returns the original RTL hierarchy for the cell.

hdl_line

A read-only string attribute that returns the original RTL file line number for the cell.

hdl_origin

A read-only string attribute that returns the original RTL creator associated with the cell.

hdl_parameters

A read-only string attribute that exists on parameterized elaborated design. It shows the actual and default attribute values of the design. The values are returned as a single string. If the design has no parameters, the attribute is still defined but returns an empty string.

hdl_template

A read-only string attribute that returns the original RTL design name after the design was parameterized. This attribute does not work after you run the uniquify command.

height

A read-only distance attribute that returns the height of the cell.

c

hierarchy_hard_macro_count

A read-only int attribute that is only available for hierarchical cells. The value is the total number of hard macros for the whole hierarchy tree under this cell.

hierarchy_pad_cell_count

A read-only int attribute that is only available for hierarchical cells. The value is the total number of pad cells for the whole hierarchy tree under this cell.

hierarchy_physical_only_cell_count

A read-only int attribute that is only available for hierarchical cells. The value is the total number of physical only cells for the whole hierarchy tree under this cell.

hierarchy_std_cell_count

A read-only int attribute that is only available for hierarchical cells. The value is the total number of std cells in the given block. It includes the number inside of any child block.

hierarchy_switch_cell_count

A read-only int attribute that is only available for hierarchical cells. The value is the total number of switch cells for the whole hierarchy tree under this cell.

hierarchy_type

A read-only string attribute that returns the hierarchy type of the cell. The hierarchy type can be any combinations of partition, boundary, block or normal.

hold_fix_dont_touch

A read-only boolean attribute that returns true if the cell is dont_touch to preserve it to fix a hold violation.

in_edit_group

A read-only boolean attribute that returns true if the cell is in edit group.

inner_keepout_margin_filler_allowed

A settable margin_list attribute that returns the inner keepout margin of type filler_allowed.

inner_keepout_margin_hard

A settable margin_list attribute that returns the inner keepout margin of type hard.

c

inner_keepout_margin_hard_macro

A settable margin_list attribute that returns the inner keepout margin of type hard macro.

inner_keepout_margin_route_blockage

A settable margin_list attribute that returns the inner keepout margin of type route blockage.

inner_keepout_margin_route_blockage_layers

A read-only layer collection attribute that returns the inner keepout margin of type route blockage layers.

inner_keepout_margin_soft

A settable margin_list attribute that returns the inner keepout margin of type soft.

inst_marker

A settable int attribute that specifies that the cell needs to follow special spacing rules. The special spacing rules are defined by set_placement_spacing_rule -special_cells_only command. If the value of the attribute is set to 1, legalizer will check special spacing rules for the cell. Example usage of set_placement_spacing_rule command: set_placement_spacing_rule -special_cells_only -labels {label_one label_two} {1 1}

internal_clock_input_pin_power

A read-only double attribute that returns the total internal power associated with clock input pins for the cell. The value is based on current timer status, the attribute does not update the timer. This attribute exists only on non-hierarchical cells.

internal_power

A read-only double attribute that returns the internal power for the cell. The value is based on current timer status, the attribute does not update the timer.

internal_power_derate_factor

A read-only float attribute that returns the actually stored internal derate values if and only if they exist. The value is based on current timer status, the attribute does not update the timer.

io_guide

A read-only io_guide collection attribute that returns the IO guide the cell is associated with.

c

io_guide_neighbor_relative

A read-only boolean attribute that returns true if the IO guide is relative to origin or neighboring pad cell.

io_guide_offset

A settable distance attribute that returns the offset of blkInst from the IO guide shelf.

io_guide_spacing

A settable distance attribute that returns the spacing relative to IO guide or its neighbor.

io_ring_name

A read-only string attribute that returns the name of the IO ring

is_always_on_logic

A read-only boolean attribute that indicates whether a cell has an output related supply different from its parent domain main rail supply.

is_auto_floorplan_modified

A read-only boolean attribute that returns True if the cell is modified by the auto-floorplanning flow

is_backside

A read-only boolean attribute that returns True if this cell is a backside bump cell and false otherwise. A backside bump cell references a lib_cell having design_type "flip_chip_pad" and either containing backside metal shapes, containing metal1 shapes and having a no_substrate attribute value of true, or containing metal1 shapes and the parent block of this cell has a no_substrate attribute value of true.

is_bbt_object

A read-only boolean attribute that returns true if the cell is a black-box timing object.

is_black_box

A read-only boolean attribute that returns True if the cell is unresolved or refers to a lib_cell with no logic function

is_bus_bit

A read-only boolean attribute that returns True if the cell is part of a cell bus

c

is_clock_network_cell

A read-only boolean attribute that returns True if the cell is in the clock network of any clock in the current mode

is_combinational

A read-only boolean attribute that returns True if the cell is combinational logic (not sequential)

is_destructible

A settable boolean attribute that returns True if a probe_pad cell is destructible. Only bump cell labeled with is_probe_pad being true can be set as destructible. It is an error to set a non probe_pad cell as destructible

is_diff_level_shifter

A read-only boolean attribute that returns True if the cell is a differential level shifter cell

is_diode_cell

A read-only boolean attribute that returns True if the cell is a diode cell

is_direct_probe

A settable boolean attribute that returns True if the cell is a direct probe bump cell

is_dummy

A settable boolean attribute that returns True if the cell is a dummy bump cell. A cell's is_dummy flag is automatically set to true if its master lib-cell is set as dummy.

is_empty

A read-only boolean attribute that is only available for hierarchical cells. True if there are only shadow nets and cells in the whole hierarchy tree under this cell.

is_enable_level_shifter

A read-only boolean attribute that returns True if the cell is an enable level shifter

is_etm_moded_cell

A read-only boolean attribute that returns True if the cell is an extracted timing model cell

c

is_fall_edge_triggered

A read-only boolean attribute that returns True if the cell has timing behavior relative to the falling edge of a clock signal

is_fixed

A read-only boolean attribute that returns True if the physical status of the cell is either fixed or locked (but not application_fixed)

is_flipped

A settable boolean attribute that returns True if the cell should be flipped with respect to the flipping axis of the IO guide. This attribute exists only on macro and pad cells. This is only a placement constraint and does not directly change the cell orientation.

is_fs_eco_add

A settable boolean attribute that returns True if the cell has been created in freeze_silicon mode

is_glc

A read-only boolean attribute that returns true if this cell is a glue logic cell.

is_golden

A settable boolean attribute that is used for multiply instantiated blocks (MIBs). The cells with is_golden attribute are considered to have highest priority when perform design planning global routing and creating pins related to those cells.

is_hard_macro

A read-only boolean attribute that returns True if the cell is a hard macro

is_hierarchical

A read-only boolean attribute that returns True if the cell is hierarchical

is_ideal

A read-only boolean attribute that returns true if the cell has been marked ideal using the set_ideal_network command.

is_integrated_clock_gating_cell

A read-only boolean attribute that returns true if the cell is an integrated clockgating cell (ICG).

is_io

A read-only boolean attribute that returns true if the cell is of type IO.

c

is_isolation

A read-only boolean attribute that returns true if the cell is an isolation cell.

is_level_shifter

A read-only boolean attribute that returns true if the cell is a level shifter.

is_lft_buf

A read-only boolean attribute that specifies that the cell is a logical feedthrough buffer.

is_logical_black_box

A read-only boolean attribute that returns true if this cell is a logical black box.

is_mapped

A read-only boolean attribute that returns true if the cell is mapped.

is_memory_cell

A read-only boolean attribute that returns true if the cell refers to a lib_cell that is defined as a memory cell.

is_mirrored

A settable boolean attribute that returns true if the cell is created with the mirrored version of the block. Changing *is_mirrored* from "false" to "true" require the reference block to already have the "mirror_name" set. If the reference block doesn't already have a mirror_name, an auto-generated mirror_name in the format of <ref_block_name>_mz_%d will be set on the reference block. This attribute is inheritable from a parent instance. If a parent instance is set as mirrored, then all its sub-block instances are automatically mirrored.

is_multiply_instantiated_block

A read-only boolean attribute that returns true if the cell is an instance of a multiply instantiated block.

is_multiply_instantiated_module

A read-only boolean attribute that returns true if the cell is an instance of a multiply instantiated module.

is_mux

A read-only boolean attribute that returns True if the cell is a mux.

is_negative_level_sensitive

A read-only boolean attribute that returns True if the cell has negative level-sensitive timing behavior, as in a negative D-latch.

c

is_new_cell_allowed

A settable boolean attribute that returns True if new cell creation is allowed in the hierarchy.

is_outline

A read-only boolean attribute that returns True if the module of the cell has outline data.

is_physical

A read-only boolean attribute that returns True for physical cells.

is_physical_only

A read-only boolean attribute that returns True for physical only cells.

is_placed

A read-only boolean attribute that returns True if the cell has physical status of placed.

is_port_protection_diode

A read-only boolean attribute that returns True if the cell is a port protection diode.

is_positive_level_sensitive

A read-only boolean attribute that returns True if the cell has positive level-sensitive timing behavior, as in a positive D-latch.

is_power_domain_root_cell

A read-only boolean attribute that returns True if the cell is a power domain root cell.

is_power_switch

A read-only boolean attribute that returns True if the cell is a power switch cell

is_probe_pad

A settable boolean attribute that returns True if the cell is a probe pad bump cell. Only a probe_pad bump cell can be labeled as destructible. It is an error to set the is_probe_pad flag from true to false if the bump cell is already labeled as destructible.

is_retention

A read-only boolean attribute that returns True if the cell is a retention cell

c

is_rise_edge_triggered

A read-only boolean attribute that returns True if the cell has timing behavior relative to the rising edge of a clock signal

is_sequential

A read-only boolean attribute that returns True if the cell has sequential logic function

is_shadow

A read-only boolean attribute that returns True if the cell is pushed down from the top design

is_snap_point_fixed

A settable boolean attribute that returns True if the snap point on the cell reference is fixed. If fixed, the tool cannot move the snap point. This cell must be physical.

is_soft_macro

A read-only boolean attribute that returns True if the cell is a soft macro

is_soi

A read-only boolean attribute that returns True if the cell is a silicon-on- insulator (SOI) cell

is_spare_cell

A settable boolean attribute that returns True if the cell is a spare cell

is_spare_overridden

A read-only boolean attribute that returns True if the cell is a spare cell and overridden.

is_switch_place_holder_cell

A read-only boolean attribute that returns True if the cell is a place holder switch cell.

is_three_state

A read-only boolean attribute that returns True if the cell has one or more three state outputs.

is_tsv

A read-only boolean attribute that returns True if the cell is a through-silicon via (TSV) cell.

c

is_unbound

A read-only boolean attribute that returns True if the cell is unbound, meaning that it has no reference cell.

is_unmapped

A read-only boolean attribute that returns True if the cell is unmapped.

is_upf_mim_primary_instance

A settable boolean attribute that returns True if this cell's UPF is used as the resulting Multi-Instantiated Module (MIM) UPF of the module during split_constraints. It can only be set to true and only for one instance of each module.

is_user_always_on_logic

A read-only boolean attribute that specifies whether the buffer/inverter/tie cell has user-specified supply net connection to a supply that is different from domain primary.

keepouts

A read-only keepout_margin collection attribute that returns the area around the cell which cannot be used for placement.

late_fall_cell_check_derate_factor

A read-only float attribute that returns the derate factor applied for timing checks (setup, hold, recovery, removal) on this cell for falling edge late analysis. Timing check derates are applied using set_timing_derate -cell_check.

late_fall_cell_check_derate_factor_file_line_info

A read-only string attribute that returns a Tcl list of two elements, the first showing the source file path and the second showing the source file line number, for the set_timing_derate -cell_check -late -fall constraint applied on this cell.

late_fall_cell_check_derate_factor_source

A read-only string attribute that indicates the source scope of the set_timing_derate -late -fall -cell_check constraint on this cell. The source could be from a constraint on the design, a hierarchical cell, a library cell, or a leaf cell.

late_fall_clk_cell_derate_factor

A read-only float attribute that returns the derate factor applied on this cell for falling edge late analysis if it is in a clock path. Timing derates on clock logic are applied using set_timing_derate -clock.

c

late_fall_clk_cell_derate_factor_file_line_info

A read-only string attribute that returns a Tcl list of two elements, the first showing the source file path and the second showing the source file line number, for the `set_timing_derate -clock -late -fall` constraint applied on this cell.

late_fall_clk_cell_derate_factor_source

A read-only string attribute that indicates the source scope of the `set_timing_derate -late -fall -clock` constraint on this cell. The source could be from a constraint on the design, a hierarchical cell, a library cell, or a leaf cell.

late_fall_data_cell_derate_factor

A read-only float attribute that returns the derate factor applied on this cell for falling edge late analysis if it is in the datapath. Timing derates on datapath are applied using `set_timing_derate -data`.

late_fall_data_cell_derate_factor_file_line_info

A read-only string attribute that returns a Tcl list of two elements, the first showing the source file path and the second showing the source file line number, for the `set_timing_derate -data -late -fall` constraint applied on this cell.

late_fall_data_cell_derate_factor_source

A read-only string attribute that indicates the source scope of the `set_timing_derate -late -fall -data` constraint on this cell. The source could be from a constraint on the design, a hierarchical cell, a library cell, or a leaf cell.

late_rise_cell_check_derate_factor

A read-only float attribute that returns the derate factor applied for timing checks (setup, hold, recovery, removal) on this cell for rising edge late analysis. Timing check derates are applied using `set_timing_derate -cell_check`.

late_rise_cell_check_derate_factor_file_line_info

A read-only string attribute that returns a Tcl list of two elements, the first showing the source file path and the second showing the source file line number, for the `set_timing_derate -cell_check -late -rise` constraint applied on this cell.

late_rise_cell_check_derate_factor_source

A read-only string attribute that indicates the source scope of the `set_timing_derate -late -rise -cell_check` constraint on this cell. The source could be from a constraint on the design, a hierarchical cell, a library cell, or a leaf cell.

c

late_rise_clk_cell_derate_factor

A read-only float attribute that returns the derate factor applied on this cell for rising edge late analysis if it is in a clock path. Timing derates on clock logic are applied using `set_timing_derate -clock`.

late_rise_clk_cell_derate_factor_file_line_info

A read-only string attribute that returns a Tcl list of two elements, the first showing the source file path and the second showing the source file line number, for the `set_timing_derate -clock -late -rise` constraint applied on this cell.

late_rise_clk_cell_derate_factor_source

A read-only string attribute that indicates the source scope of the `set_timing_derate -late -rise -clock` constraint on this cell. The source could be from a constraint on the design, a hierarchical cell, a library cell, or a leaf cell.

late_rise_data_cell_derate_factor

A read-only float attribute that returns the derate factor applied on this cell for rising edge late analysis if it is in the datapath. Timing derates on datapath are applied using `set_timing_derate -data`.

late_rise_data_cell_derate_factor_file_line_info

A read-only string attribute that returns a Tcl list of two elements, the first showing the source file path and the second showing the source file line number, for the `set_timing_derate -data -late -rise` constraint applied on this cell.

late_rise_data_cell_derate_factor_source

A read-only string attribute that indicates the source scope of the `set_timing_derate -late -rise -data` constraint on this cell. The source could be from a constraint on the design, a hierarchical cell, a library cell, or a leaf cell.

leakage_power

A read-only double attribute that returns the leakage power for the cell instance. The value is based on current timer status, the attribute does not update the timer.

leakage_power_derate_factor

A read-only float attribute that returns the actually stored leakage derate values if and only if they exist. The value is based on current timer status, the attribute does not update the timer.

c

lib_cell

A read-only *lib_cell* collection attribute that returns the library cell for a cell. The collection contains exactly one library cell object. This attribute is defined only on leaf cells.

libcell_subset

A read-only string attribute that returns the subset name if the cell is associated with a *lib_cell* subset. This cell must be a leaf cell.

mask_shift

A settable string attribute that returns a list of layer mask shifts, formatted { { layer_name shift }. .. }, in which *layer_name* is the layer and *shift* indicates the amount to shift the mask. Valid shift values are 0 and 1.

matching_type

A read-only *matching_type* collection attribute that returns the *matching_type* which constrains this cell. A matching type specifies which bump cells are to be connected to which pad pins.

metrics_cong_cell_padding

A read-only double attribute that returns a metric derived by *compute_metrics* for this cell. The amount of increase in apparent area of this hierarchy due to leaf cell padding added by the placer to enforce cell spreading to reduce congestion.

metrics_cong_cell_padding_local

A read-only double attribute that returns a metric derived by *compute_metrics* for this cell. Only considers objects in this cell hierarchy, not child hierarchies. The amount of increase in apparent area of this hierarchy due to leaf cell padding added by the placer to enforce cell spreading to reduce congestion.

metrics_cong_drc_estimate

A read-only double attribute that returns the DRC estimate for leaf cells of the logical hierarchy.

metrics_cong_drc_estimate_local

A read-only double attribute that returns the DRC estimate from leaf cells local to the logical hierarchy.

metrics_cong_global_net_count

A read-only int attribute that returns a metric derived by *compute_metrics* for this cell. Number of nets that pass over congestion hotspots in this hierarchy, but do not have local connections in that hotspot.

c

metrics_cong_global_net_count_local

A read-only int attribute that returns a metric derived by `compute_metrics` for this cell. Only considers objects in this cell hierarchy, not child hierarchies. Number of nets that pass over congestion hotspots in this hierarchy, but do not have local connections in that hotspot.

metrics_cong_local_net_count

A read-only int attribute that returns a metric derived by `compute_metrics` for this cell. Number of nets that pass over congestion hotspots in this hierarchy, and also have local pin connections in that area.

metrics_cong_local_net_count_local

A read-only int attribute that returns a metric derived by `compute_metrics` for this cell. Only considers objects in this cell hierarchy, not child hierarchies. Number of nets that pass over congestion hotspots in this hierarchy, and also have local pin connections in that area.

metrics_cong_logic_cong

A read-only int attribute that returns a metric derived by `compute_metrics` for this cell. Number of cells in congested areas that come from select op structures in the RTL. RTL could possibly be recoded to improve congestion in this hierarchy.

metrics_cong_logic_cong_local

A read-only int attribute that returns a metric derived by `compute_metrics` for this cell. Only considers objects in this cell hierarchy, not child hierarchies. Number of cells in congested areas that come from select op structures in the RTL. RTL could possibly be recoded to improve congestion in this hierarchy.

metrics_cong_number_cells

A read-only int attribute that returns a metric derived by `compute_metrics` for this cell. Number of leaf cells.

metrics_cong_number_cells_in_cong_area

A read-only int attribute that returns a metric derived by `compute_metrics` for this cell. Number of cells overlapping global route congestion hotspots.

metrics_cong_number_cells_in_cong_area_local

A read-only int attribute that returns a metric derived by `compute_metrics` for this cell. Only considers objects in this cell hierarchy, not child hierarchies. Number of cells overlapping global route congestion hotspots.

c

metrics_cong_number_cells_in_cong_channel

A read-only int attribute that returns a metric derived by `compute_metrics` for this cell. Number of cells overlapping global route congestion hotspots in placement channels between macros.

metrics_cong_number_cells_in_cong_channel_local

A read-only int attribute that returns a metric derived by `compute_metrics` for this cell. Only considers objects in this cell hierarchy, not child hierarchies. Number of cells overlapping global route congestion hotspots in placement channels between macros.

metrics_cong_number_cells_local

A read-only int attribute that returns a metric derived by `compute_metrics` for this cell. Only considers objects in this cell hierarchy, not child hierarchies. Number of leaf cells.

metrics_cong_overflow

A read-only double attribute that returns a metric derived by `compute_metrics` for this cell. Global routing overflows (gcells where routing demand exceeds capacity).

metrics_cong_overflow_local

A read-only double attribute that returns a metric derived by `compute_metrics` for this cell. Only considers objects in this cell hierarchy, not child hierarchies. Global routing overflows (gcells where routing demand exceeds capacity).

metrics_cong_percent_cells_in_cong_area

A read-only double attribute that returns a metric derived by `compute_metrics` for this cell. Percentage of cells overlapping global route congestion hotspots.

metrics_cong_percent_cells_in_cong_area_local

A read-only double attribute that returns a metric derived by `compute_metrics` for this cell. Only considers objects in this cell hierarchy, not child hierarchies. Percentage of cells overlapping global route congestion hotspots.

metrics_cong_percent_of_cong_area

A read-only double attribute that returns a metric derived by `compute_metrics` for this cell. Percentage of cell area overlapping global route congestion hotspots.

c

metrics_cong_percent_of_cong_area_local

A read-only double attribute that returns a metric derived by `compute_metrics` for this cell. Only considers objects in this cell hierarchy, not child hierarchies. Percentage of cell area overlapping global route congestion hotspots.

metrics_cong_shape_dispersion

A read-only float attribute that returns a metric derived by `compute_metrics` for this cell. Percentage spread of cells in this hierarchy. 0 indicates a contiguous placement area, as the number increases the placement shape is more fragmented (could be formed from multiple disjoint regions).

metrics_cong_shape_distortion

A read-only float attribute that returns a metric derived by `compute_metrics` for this cell. Percentage measure of how the placement of logic in this hierarchy is impacted by macros. As the percentage increases, the placement shape is more distorted by macros.

metrics_cong_utilization

A read-only float attribute that returns a metric derived by `compute_metrics` for this cell. Placement utilization of this hierarchy.

metrics_tim_bottleneck_count

A read-only int attribute that returns a metric derived by `compute_metrics` for this cell. Number of cells in the hierarchy that have more negative slack paths passing through them than specified by `metrics.timing.bottleneck_path_count`.

metrics_tim_bottleneck_count_local

A read-only int attribute that returns a metric derived by `compute_metrics` for this cell. Only considers objects in this cell hierarchy, not child hierarchies. Number of cells in the hierarchy that have more negative slack paths passing through them than specified by `metrics.timing.bottleneck_path_count`.

metrics_tim_logic_levels_viol

A read-only int attribute that returns a metric derived by `compute_metrics` for this cell. Number of violating paths with logic level count greater than that specified by `metrics.timing.logic_level_threshold`.

metrics_tim_logic_levels_viol_local

A read-only int attribute that returns a metric derived by `compute_metrics` for this cell. Only considers objects in this cell hierarchy, not child hierarchies. Number of violating paths with logic level count greater than that specified by `metrics.timing.logic_level_threshold`.

c

metrics_tim_net_length_viol

A read-only int attribute that returns a metric derived by `compute_metrics` for this cell. Number of violating paths with nets exceeding the length specified by `metrics.timing.unbuffered_net_length_threshold`.

metrics_tim_net_length_viol_local

A read-only int attribute that returns a metric derived by `compute_metrics` for this cell. Only considers objects in this cell hierarchy, not child hierarchies. Number of violating paths with nets exceeding the length specified by `metrics.timing.unbuffered_net_length_threshold`.

metrics_tim_nvp_internal

A read-only int attribute that returns a metric derived by `compute_metrics` for this cell. Internal path number of failing setup endpoints.

metrics_tim_nvp_internal_local

A read-only int attribute that returns a metric derived by `compute_metrics` for this cell. Only considers objects in this cell hierarchy, not child hierarchies. Internal path number of failing setup endpoints.

metrics_tim_nvp_io

A read-only int attribute that returns a metric derived by `compute_metrics` for this cell. Input and output path number of failing setup endpoints.

metrics_tim_nvp_io_local

A read-only int attribute that returns a metric derived by `compute_metrics` for this cell. Only considers objects in this cell hierarchy, not child hierarchies. Input and output path number of failing setup endpoints.

metrics_tim_nvp_r2r

A read-only int attribute that returns a metric derived by `compute_metrics` for this cell. Register-to-register number of failing setup endpoints.

metrics_tim_nvp_r2r_local

A read-only int attribute that returns a metric derived by `compute_metrics` for this cell. Only considers objects in this cell hierarchy, not child hierarchies. Register-to-register number of failing setup endpoints.

metrics_tim_nvp_total

A read-only int attribute that returns a metric derived by `compute_metrics` for this cell. Overall design number of failing setup endpoints.

c

metrics_tim_nvp_total_local

A read-only int attribute that returns a metric derived by `compute_metrics` for this cell. Only considers objects in this cell hierarchy, not child hierarchies. Overall design number of failing setup endpoints.

metrics_tim_path_length_viol

A read-only int attribute that returns a metric derived by `compute_metrics` for this cell. Number of violating paths with path length longer then specified by `metrics.timing.path_length_threshold`.

metrics_tim_path_length_viol_local

A read-only int attribute that is a metric derived by `compute_metrics` for this cell. Only considers objects in this cell hierarchy, not child hierarchies. Number of violating paths with path length longer then specified by `metrics.timing.path_length_threshold`.

metrics_tim_repeater_levels_viol

A read-only int attribute that returns the repeater count violation for the current scenario. Works only on hierarchical cells. Returns the sum of all repeater count violating paths under the input hierarchy.

metrics_tim_repeater_levels_viol_local

A read-only int attribute that returns the repeater count violation for the current scenario. Works only on hierarchical cells. Returns the sum of all repeater count violating paths ending in the input hierarchy.

metrics_tim_tns_internal

A read-only float attribute that works on the current scenario.

metrics_tim_tns_internal_local

A read-only float attribute that returns a metric derived by `compute_metrics` for this cell. Only considers objects in this cell hierarchy, not child hierarchies. Internal path total negative setup slack.

metrics_tim_tns_io

A read-only float attribute that returns a metric derived by `compute_metrics` for this cell. Input and output path total negative setup slack.

metrics_tim_tns_io_local

A read-only float attribute that returns a metric derived by `compute_metrics` for this cell. Only considers objects in this cell hierarchy, not child hierarchies. Input and output path total negative setup slack.

c

metrics_tim_tns_r2r

A read-only float attribute that returns a metric derived by `compute_metrics` for this cell. Register-to-register total negative setup slack.

metrics_tim_tns_r2r_local

A read-only float attribute that returns a metric derived by `compute_metrics` for this cell. Only considers objects in this cell hierarchy, not child hierarchies. Register-to-register total negative setup slack.

metrics_tim_tns_total

A read-only float attribute that returns a metric derived by `compute_metrics` for this cell. Overall design total negative setup slack.

metrics_tim_tns_total_local

A read-only float attribute that returns a metric derived by `compute_metrics` for this cell. Only considers objects in this cell hierarchy, not child hierarchies. Overall design total negative setup slack.

metrics_tim_wns_internal

A read-only float attribute that returns a metric derived by `compute_metrics` for this cell. Internal path worst negative setup slack.

metrics_tim_wns_internal_local

A read-only float attribute that returns a metric derived by `compute_metrics` for this cell. Only considers objects in this cell hierarchy, not child hierarchies. Internal path worst negative setup slack.

metrics_tim_wns_io

A read-only float attribute that returns a metric derived by `compute_metrics` for this cell. Input and output path worst negative setup slack.

metrics_tim_wns_io_local

A read-only float attribute that returns a metric derived by `compute_metrics` for this cell. Only considers objects in this cell hierarchy, not child hierarchies. Input and output path worst negative setup slack.

metrics_tim_wns_r2r

A read-only float attribute that returns a metric derived by `compute_metrics` for this cell. Register-to-register worst negative setup slack.

c

metrics_tim_wns_r2r_local

A read-only float attribute that returns a metric derived by `compute_metrics` for this cell. Only considers objects in this cell hierarchy, not child hierarchies. Register-to-register worst negative setup slack.

metrics_tim_wns_total

A read-only float attribute that returns a metric derived by `compute_metrics` for this cell. Overall design worst negative setup slack.

metrics_tim_wns_total_local

A read-only float attribute that returns a metric derived by `compute_metrics` for this cell. Only considers objects in this cell hierarchy, not child hierarchies. Overall design worst negative setup slack.

metrics_tim_zwl_viol_count

A read-only int attribute that returns a metric derived by `compute_metrics` for this cell.

metrics_tim_zwl_viol_count_local

A read-only int attribute that returns a metric derived by `compute_metrics` for this cell. Only considers objects in this cell hierarchy, not child hierarchies.

mismatched_process_label_max

A read-only boolean attribute that returns True if there is a mismatch between the process label applied to this cell from the `set_process_label` constraint for max analysis, and the process label used for the `current_corner`.

mismatched_process_label_min

A read-only boolean attribute that returns True if there is a mismatch between the process label applied to this cell from the `set_process_label` constraint for min analysis, and the process label used for the `current_corner`.

mismatched_process_number_max

A read-only boolean attribute that returns True if there is a mismatch between the process number applied to this cell from the `set_process_number` constraint for max analysis, and the process number used for the `current_corner`.

mismatched_process_number_min

A read-only boolean attribute that returns True if there is a mismatch between the process number applied to this cell from the `set_process_number` constraint for min analysis, and the process number used for the `current_corner`.

c

mismatched_pvt_max

A read-only boolean attribute that returns True if there is any mismatch in the PVT conditions applied to this cell for max analysis, and the PVT conditions used for the `current_corner`.

mismatched_pvt_min

A read-only boolean attribute that returns True if there is any mismatch in the PVT conditions applied to this cell for min analysis, and the PVT conditions used for the `current_corner`.

mismatched_temperature_max

A read-only boolean attribute that returns True if there is a mismatch between the temperature applied to this cell from the `set_temperature` constraint for max analysis, and the temperature used for the `current_corner`.

mismatched_temperature_min

A read-only boolean attribute that returns True if there is a mismatch between the temperature applied to this cell from the `set_temperature` constraint for min analysis, and the temperature used for the `current_corner`.

mismatched_voltage_max

A read-only boolean attribute that returns True if there is a mismatch between the voltage applied to this cell from the `set_voltage` constraint for max analysis, and the voltage used for the `current_corner`.

mismatched_voltage_min

A read-only boolean attribute that returns True if there is a mismatch between the voltage applied to this cell from the `set_voltage` constraint for min analysis, and the voltage used for the `current_corner`.

mpn_size_only

A settable boolean attribute that returns true if the cell has a `size_only` constraint due to it being used for multiport net fixing (MPN).

multibit_debank_fail_reason

A read-only string attribute that indicates a reason ignored by the RTL debanking engine for a multibit cell.

multibit_debank_fail_reason_physical

A read-only string attribute that indicates a reason ignored by the physical debanking engine for a multibit cell.

c

multibit_fail_reason

A read-only string attribute that indicates a reason ignored by the RTL banking engine for a cell.

multibit_fail_reason_physical

A read-only string attribute that indicates a reason ignored by the physical banking engine for a cell.

multibit_width

A read-only int attribute that returns the bitwidth of a multibit cell.

mv_dont_touch

A read-only boolean attribute that returns true if the cell has dont_touch status due to multivoltage (MV) related reasons.

name

A settable string attribute that returns the simple name of this cell, excluding hierarchy. Same as base_name.

net_count

A read-only int attribute that is only available for hierarchical cells. The value is the total number of nets directly under this cell, not including its children's nets.

num_of_virtual_connection

A read-only int attribute that is for virtual connection based eco coarse placement. It is annotated by create_virtual_connection. After this command, you can identify target eco cells based on this attribute and apply place_eco_cells use_virtual_connection on them.

number_of_pins

A read-only int attribute that returns the number of pins on the cell.

object_class

A read-only string attribute that returns the name of this object class. Returns 'cell' for cell objects.

object_rtl_name

A settable string attribute that returns the rtl name of the cell.

orientation

A settable string attribute (with allowed values: R0 R90 R180 R270 MX MXR90 MY MYR90) that returns the orientation of the cell.

c

origin

A settable coord attribute that returns the origin of the cell.

outer_keepout_margin

A settable margin_list attribute that returns the outer keepout margin.

outer_keepout_margin_assembly_die

A read-only margin_list attribute that returns the assembly_die outer keepout.

outer_keepout_margin_clock

A settable margin_list attribute that returns the clock outer keepout.

outer_keepout_margin_filler_allowed

A settable margin_list attribute that returns the outer keepout margin of type filler_allowed.

outer_keepout_margin_hard

A settable margin_list attribute that returns the outer keepout margin of type hard.

outer_keepout_margin_hard_macro

A settable margin_list attribute that returns the outer keepout margin of type hard macro.

outer_keepout_margin_route_blockage

A settable margin_list attribute that returns the outer keepout margin of type route blockage.

outer_keepout_margin_route_blockage_layers

A read-only layer collection attribute that returns the outer keepout margin of type route blockage layers.

outer_keepout_margin_seal_ring

A read-only margin_list attribute that returns the seal_ring outer keepout.

outer_keepout_margin_soft

A settable margin_list attribute that returns the outer keepout margin of type soft.

outer_keepout_type

A settable string attribute (with allowed values: assembly_die clock filler_allowed hard hard_macro routing_blockage seal_ring soft) that returns the outer keepout margin type.

c

outline_port_count

A read-only int attribute that returns the outline port count of the cell.

outline_ref_count

A read-only string attribute that returns the outline reference count of the cell.

pad_cell

A read-only boolean attribute that returns True if the cell is of type pad cell.

pad_offset

A settable coord attribute that returns the offset location for an I/O pad cell that is not already associated with an I/O guide.

pane_early

A read-only int attribute that returns the timing pane number used for early analysis on this cell in current_corner.

pane_late

A read-only int attribute that returns the timing pane number used for early analysis on this cell in current_corner.

parent_block

A read-only block collection attribute that returns the owner block reference for the cell.

parent_block_cell

A read-only cell collection attribute that returns the owner block instance for the cell. This attribute is defined only for cells inside physical hierarchy. It is undefined for all top-level cells.

parent_cell

A read-only cell collection attribute that returns the instance name of the cell's owner in the logic hierarchy. If the cell exists at the top-level of the logic hierarchy, this attribute is undefined.

partition_top_primary_domain

A settable string attribute that returns the child power domain, whose primary will be used as the primary of top domain created by split_constraints when mv.hierarchical.restructure_upf_for_group feature is enable.

physical_hierarchy_level

A read-only int attribute that returns the depth of the physical hierarchy. This attribute is defined only for instances of physical hierarchy blocks. Blocks owned

c

by the top level have a *physical_hierarchy_level* attribute of 1. Blocks owned by those blocks have a *physical_hierarchy_level* attribute of 2, and so on deeper into the physical hierarchy.

physical_status

A settable string attribute (with allowed values: *application_fixed* *fixed* *legalize_only* *locked* *placed* *unplaced*) that returns the physical placement status of the cell. If status is *unplaced*, the cell has not been placed. If status is *placed*, the cell has been placed and may be moved. If status is *legalize_only*, the cell may be moved for legalization only. If status is *application_fixed*, the cell is fixed by the application for automatic change; the application may change this status if necessary. If status is *fixed*, the cell is fixed by the user for automatic changes; the application cannot change this status. If status is *locked*, the cell is fixed for automatic and manual changes, what had not been executed through 'set_attribute'.

pin_pair

A read-only string attribute that records the input and output pin mapping relationship when a lib cell has multiple input or output pins. It could be created by *create_repeater_groups*, *set_repeater_group*, or *place_group_repeater*. It is used by *place_group_repeater* and *set_repeater_group -check_connection*.

placement_attractions

A read-only *placement_attraction* collection attribute that returns a collection of placement attraction objects that the cell is a member of.

plc_ir_drop_bf

A read-only double attribute that returns the bloating factor for power density, effective resistance and IR drop aware placement.

power_domain

A read-only string attribute that returns the power domain name of the cell.

power_group

A read-only string attribute that returns the power groups of the cell. The value is based on current timer status, the attribute does not update the timer. This attribute depends on the setting of the *power.report_user_power_groups* application option. If the application option is set to *exclusive*, the command returns the actual power group. The actual power group is the user-specified power group present for that cell. If multiple user power groups are specified then first power group in the specified list is Actual power group. If a user-specified power group is not present for that cell, the actual power group is the user-specified power group present for the library cell of that cell. If multiple user power groups are present for the library cell, the first power group in the

c

specified list is the actual power group. If user-specified power group is not present for that library cell then actual power group is the default power group of that cell. If the application option value is *inclusive*, the list of all user-specified power groups and default power group of that cell is returned. If the cell has no user power groups specified, the corresponding library cell is checked for user-specified power groups.

power_strategy

A read-only collection attribute that returns the power strategy that is associated with the cell.

preserve_via_ladder_constraints

A settable boolean attribute that returns whether application may change via_ladder constraints. A true value of this attribute would prevent applications from changing user specified via_ladder constraints. This attribute defaults to false.

process_label_early

A read-only string attribute that returns the process label constraint for early analysis for this cell in the current_corner. Set by the set_process_label constraint.

process_label_early_file_line_info

A read-only string attribute that returns a Tcl list of two elements, the first showing the source file path and the second showing the source file line number, where the set_process_label constraint for early analysis was applied on this cell.

process_label_early_source

A read-only string attribute that indicates the source scope of the constraint that gave this cell its early analysis process label setting from the set_process_label command. The source could be from a constraint on the design, a hierarchical cell, a library cell, or a leaf cell.

process_label_late

A read-only string attribute that returns the process label constraint for late analysis for this cell in the current_corner. Set by the set_process_label constraint.

process_label_late_file_line_info

A read-only string attribute that returns a Tcl list of two elements, the first showing the source file path and the second showing the source file line number, where the set_process_label constraint for late analysis was applied on this cell.

c

process_label_late_source

A read-only string attribute that indicates the source scope of the constraint that gave this cell its late analysis process label setting from the `set_process_label` command. The source could be from a constraint on the design, a hierarchical cell, a library cell, or a leaf cell.

process_number_early

A read-only float attribute that returns the process label constraint for early analysis for this cell in the `current_corner`. Set by the `set_process_number` constraint.

process_number_early_file_line_info

A read-only string attribute that returns a Tcl list of two elements, the first showing the source file path and the second showing the source file line number, where the `set_process_number` constraint for early analysis was applied on this cell.

process_number_early_source

A read-only string attribute that indicates the source scope of the constraint that gave this cell its early analysis process number setting from the `set_process_number` command. The source could be from a constraint on the design, a hierarchical cell, a library cell, or a leaf cell.

process_number_late

A read-only float attribute that returns the process number constraint for late analysis for this cell in the `current_corner`. Set by the `set_process_number` constraint.

process_number_late_file_line_info

A read-only string attribute that returns a Tcl list of two elements, the first showing the source file path and the second showing the source file line number, where the `set_process_number` constraint for late analysis was applied on this cell.

process_number_late_source

A read-only string attribute that indicates the source scope of the constraint that gave this cell its late analysis process number setting from the `set_process_number` command. The source could be from a constraint on the design, a hierarchical cell, a library cell, or a leaf cell.

c

propagated_esd_sources

A read-only string attribute that contains the list of assigned and propagated worst-case Electro Static Sources (ESD) of this cell. The ESD name and voltage are listed.

psc_filler_is_used

A read-only boolean attribute that returns True if the cell was a programmable spare cell (psc) filler and is used now.

ref_block

A read-only block collection attribute that returns the reference lib_cell object of this cell. If a physical and timing view are present, this contains the timing view. Otherwise it contains the physical view.

ref_full_name

A read-only string attribute that returns the full_name of the block or lib_cell to which the cell instance refers. It consists of the reference lib_name and block_name, separated by a ':' character, and an optional layer_name, separated by a '/' character, and a view name, separated by a '.' character. For example, "bot_lib:bot/my_label.design".

ref_label_name

A read-only string attribute that returns the label name of the block or lib_cell to which this cell instance refers.

ref_lib_name

A read-only string attribute that returns the reference cell's library name.

ref_mirror_name

A read-only string attribute that returns the mirror name of the block to which the cell instance refers.

ref_module

A read-only module collection attribute that returns the module object referenced by the cell instance. This is an empty collection if the cell is not bound.

ref_module_name

A read-only string attribute that returns the name of the module, or the name of the top module of the block or lib_cell to which the cell instance refers.

ref_name

A read-only string attribute that returns the base_name of the block or lib_cell to which the cell instance refers.

c

ref_phys_block

A read-only block collection attribute that returns the physical view of the reference lib_cell object of this cell.

ref_via_def

A read-only via_def collection attribute that returns the reference via_def of the cell. This attribute may only exist on through-silicon via ("TSV") cells, and its existence is mutually exclusive with the *ref_block* attribute.

ref_view_name

A read-only string attribute (with allowed values: abstract conn design frame layout outline rtl timing) that returns the view name of the block or lib_cell to which the cell instance refers.

register_list

A settable string attribute that records the original names of the cells that were banked from for a multibit cell.

retention_cell_style

A read-only string attribute that returns the retention-cell type of the cell's reference library cell.

rotation

A settable float attribute that returns the all-angle rotation value of the cell in degrees. Allowed to be set only for cells instantiated within designs of type pcb, rdl_fanout or package.

rgb_core_internal_net_rbid

A settable int attribute that refers to its associated routing blockages' routing_blockage_group_id on a repelling_group_bound's core.

rgb_core_routing_blockages

A settable string attribute that stores the collection of routing_blockage_group_ids, representing the routing blockages associated with this repelling_group bound's core.

rp_column

A read-only int attribute that returns the column position of the cell in rp_group. IP *rp_is_free_placement* A Boolean attribute that is true if the cell is associated with a free placement group.

c

rp_is_free_placement

A read-only boolean attribute that returns true if the cell is associated with a free placement group. free placement refers to cells without a fixed relative location inside a relative_placement group.

rp_num_columns

A read-only int attribute that returns the number of columns occupied by the cell in a relative placement group.

rp_num_rows

A read-only int attribute that returns the number of rows occupied by the cell in a relative placement group.

rp_orientations

A settable orientation_list attribute that returns the user-specified orientations for the cell in a relative placement group.

rp_override_alignment

A settable string attribute (with allowed values: left right) that returns the alignment to be used to override the alignment of the relative placement cell in the column of its parent relative placement group.

rp_parent_group

A read-only string attribute that returns the parent relative placement group name of the cell.

rp_pin_name

A settable string attribute that returns the name of pin on which cell will be aligned in a rp_group column.

rp_row

A read-only int attribute that returns the row position of the cell in rp_group.

rp_size_in_place

A read-only boolean attribute that returns true if only relative placer (RP) is allowed to size these cells in-place. Coarse placer will not move the cells.

rp_top_group

A read-only string attribute that returns the top rp_group name of cell.

safety_core_internal_net_rbid

A settable int attribute that refers to its associated routing blockages' routing_blockage_group_id on a safety core.

c

safety_core_routing_blockages

A settable string attribute that stores the routing_blockage_group_ids, representing the routing blockages associated with this safety core.

safety_correction_signal

A read-only port, or pin collection attribute that stores the correction signal setting provided by functional safety commands.

safety_error_signal

A read-only port, or pin collection attribute that denotes the error signal associated with this safety critical register.

safety_requirement_id

A read-only string attribute that captures the requirement ID provided through the -requirement_id option of Functional Safety commands. Functional safety constraints need to be trackable back to a specific requirement and this attribute captures that information

scan_chains

A read-only scan_chain collection attribute that returns the scan_chains which contain this cell.

shadow_status

A settable string attribute (with allowed values: copied_down copied_up normal pulled_up pushed_down virtual_flat) that returns the shadow status of the cell.

shaping_constraints

A read-only shaping_constraint collection attribute that returns the set of shaping_constraint objects defined for this cell.

shrink_factor_x

A settable double attribute that can be set on a soft macro (sub-block) instance. Hierarchical relative location placement will expand (when factor >1) or shrink (when factor < 1) the horizontal dimensions of this soft macro cell by this factor.

shrink_factor_y

A settable double attribute that can be set on a soft macro (sub-block) instance. Hierarchical relative location placement will expand (when factor >1) or shrink (when factor < 1) the vertical dimensions of this soft macro cell by this factor.

single_bit_list

A settable string attribute that records the original names of the cells that were banked from for a multibit cell.

c

size_in_place

A read-only boolean attribute that indicates if the cell is only allowed to be sized in-place. It cannot be moved by the coarse placer.

size_only

A read-only boolean attribute that returns true if the cell is protected from removal by the `set_size_only` command. When this attribute is set to true, the ECO fixing commands (`fix_eco_drc`, `fix_eco_power`, and `fix_eco_timing`) can resize the cell but not remove it. The cell can still be removed by the `remove_cell` command.

snap_point

A settable coord attribute that returns the snap point on the cell reference.

spare_cell_mode

A settable string attribute (with allowed values: `auto` `false` `true`) that returns the spare cell mode of the cell.

spfm_loss

A settable float attribute that returns sequential cells. It indicates the safety criticality of the register that it is set on. Its value should be between 0 and 100.

spfm_target

A read-only float attribute that reflects the setting via `set_spfm_target -cells` command for sequential cells.

std_cell_count

A read-only int attribute that is only available for hierarchical cells. The value is the total number of std cells directly under this cell, not including its children's hierarchies.

supernet_disabled

A read-only boolean attribute that returns the supernet disabled status for the cell. For a non-repeater cell this is always false and for repeater it is true if the repeater has been explicitly disabled for supernet transparency using the `set_supernet_exceptions -disabled_cell` command.

supernet_transparent

A read-only boolean attribute that returns the supernet transparent status for the cell.

c

supernet_transparent_pins

A read-only pin collection attribute that returns the supernet transparent pins for the cell.

switching_power

A read-only double attribute that returns the switching power for the cell. The value is based on the current timer status; the attribute does not update the timer.

switching_power_derate_factor

A read-only float attribute that returns the actually stored switching derate values if and only if they exist. The value is based on the current timer status; the attribute does not update the timer.

target_boundary_area

A settable area attribute that returns the target boundary area associated with the current hierarchy block's physical hierarchy.

target_utilization

A settable float attribute that returns the target utilization associated with the current hierarchy block's physical hierarchy.

temperature_early

A read-only float attribute that returns the temperature constraint for early analysis for this cell in the current_corner. Set by the set_temperature constraint.

temperature_early_file_line_info

A read-only string attribute that returns a Tcl list of two elements, the first showing the source file path and the second showing the source file line number, where the set_temperature constraint for early analysis was applied on this cell.

temperature_early_source

A read-only string attribute that indicates the source scope of the constraint that gave this cell its early analysis temperature setting from the set_temperature command. The source could be from a constraint on the design, a hierarchical cell, a library cell, or a leaf cell.

temperature_late

A read-only float attribute that returns the temperature constraint for late analysis for this cell in the current_corner. Set by the set_temperature constraint.

c

temperature_late_file_line_info

A read-only string attribute that returns a Tcl list of two elements, the first showing the source file path and the second showing the source file line number, where the `set_temperature` constraint for late analysis was applied on this cell.

temperature_late_source

A read-only string attribute that indicates the source scope of the constraint that gave this cell its late analysis temperature setting from the `set_temperature` command. The source could be from a constraint on the design, a hierarchical cell, a library cell, or a leaf cell.

top_block

A read-only block collection attribute that returns the top block object for the cell.

total_power

A read-only double attribute that returns the total power for the cell. The value is based on the current timer status; the attribute does not update the timer.

UPF_enable_bias

A read-only boolean attribute that specifies whether the cell is under the scope of a bias block with bias functions.

UPF_is_hard_macro

A read-only boolean attribute that returns whether a completely implemented block, which can be used as is in other blocks. It forms a terminal boundary.

UPF_is_refinable_macro

A read-only boolean attribute that specifies whether it is a macro cell that is represented by the original RTL and UPF which will be later refined with implementation details by the integrator in a bottom up integration flow.

UPF_is_soft_macro

A read-only boolean attribute that returns whether a macro cell represented by the original RTL and UPF from which its implementation will be derived in a bottom-up flow. It forms a terminal boundary.

UPF_terminal_boundary

A read-only boolean attribute that specifies whether the cell is considered as a terminal boundary that the ports on the boundary are treated as drivers and receivers.

c

user_dont_touch

A settable boolean attribute that returns true if a user dont_touch constraint is applied on the cell with the set_dont_touch command.

user_power_group

A read-only string attribute that returns the user specified power groups of the cell instance.

user_size_only

A settable boolean attribute that returns true if a user size_only constraint is applied on the cell with the set_size_only command.

view_name

A read-only string attribute (with allowed values: abstract conn design frame layout outline rtl timing) that returns the view name selected by change_view for the cell reference. A cell with the default view name of "" uses the view switch list to select the reference block view.

voltage_area

A read-only voltage_area collection attribute for the cell reference. A cell with the default view name of "" uses

voltage_area_shape

A read-only voltage_area_shape collection attribute that returns the view switch list to select the reference block view.

voltage_early

A read-only float attribute that returns the voltage constraint for early analysis for this cell in the current_corner. Set by the set_voltage constraint.

voltage_early_file_line_info

A read-only string attribute that returns a Tcl list of two elements, the first showing the source file path and the second showing the source file line number, where the set_voltage constraint for early analysis was applied on this cell.

voltage_early_source

A read-only string attribute that indicates the source scope of the constraint that gave this cell its early analysis voltage setting from the set_voltage command. The source could be from a constraint on the design, a hierarchical cell, a library cell, or a leaf cell.

c

voltage_late

A read-only float attribute that returns the voltage constraint for late analysis for this cell in the current_corner. Set by the set_voltage constraint.

voltage_late_file_line_info

A read-only string attribute that returns a Tcl list of two elements, the first showing the source file path and the second showing the source file line number, where the set_voltage constraint for late analysis was applied on this cell.

voltage_late_source

A read-only string attribute that indicates the source scope of the constraint that gave this cell its late analysis voltage setting from the set_voltage command. The source could be from a constraint on the design, a hierarchical cell, a library cell, or a leaf cell.

width

A read-only distance attribute that returns the width of the cell.

z_offset

A settable int attribute that returns a zero or positive integer specifying the offset in the z-direction in 3DIC. If the specified die is at a lower physical hierarchy at another 3DIC_TOP design, the z_offset will be set at the instance/die level at that 3DIC_TOP container. If that lower level 3DIC_TOP design has an z_offset set, it will be subtracted from the z_offset value set at the leaf level die.

z_value

A settable distance attribute that returns a negative or positive distance specifying the position of a cell from the bottom of the die stack in the z-direction in 3DIC.

See Also

- [get_attribute](#)
- [list_attributes](#)
- [set_clock_uncertainty](#)

cell_bus_attributes

Description of the application defined cell_bus attributes.

Description

Attributes are properties assigned to objects such as pins, cells, and nets. They can be read-only or settable. A read-only attribute is informational; you cannot set its value. To modify a settable attribute, use the *set_attribute* command.

To determine the value of an attribute, use the *get_attribute* command. To see a list of all cell_bus attributes available, use the *list_attributes -application -class cell_bus* command.

Attributes for cell_bus

This section describes the cell_bus attributes.

bit_order

A read-only string attribute (with allowed values: down up) that returns the bit order of a cell bus. Possible values are *up* or *down*. It is *down* when *end* is less than *start* else it is *up*.

block

A read-only block collection attribute that returns the block of a cell bus.

cells

A read-only cell collection attribute that returns the cells of a cell bus.

end

A read-only int attribute that returns the end index of a cell bus.

full_name

A read-only string attribute that returns the name of a cell bus.

name

A read-only string attribute that returns the name of a cell bus.

object_class

A read-only string attribute that returns the name of this object class. Returns 'cell_bus' for cell_bus objects.

object_rtl_name

A settable string attribute that returns the rtl name of a cell bus.

start

A read-only int attribute that returns the start index of a cell bus.

See Also

- [get_attribute](#)
- [list_attributes](#)

clock_attributes

Description of the application defined clock attributes.

Description

Attributes are properties assigned to objects such as pins, cells, and nets. They can be read-only or settable. A read-only attribute is informational; you cannot set its value. To modify a settable attribute, use the *set_attribute* command.

To determine the value of an attribute, use the *get_attribute* command. To see a list of all clock attributes available, use the *list_attributes -application -class clock* command.

Attributes for clock

This section describes the clock attributes.

clock_latency_fall_max

A read-only float attribute that returns the user-specified clock network latency for max fall. The value defaults to 0.0.

clock_latency_fall_min

A read-only float attribute that returns the user-specified clock network latency for min fall. The value defaults to 0.0.

clock_latency_rise_max

A read-only float attribute that returns the user-specified clock network latency for max rise. The value defaults to 0.0.

clock_latency_rise_min

A read-only float attribute that returns the user-specified clock network latency for min rise. The value defaults to 0.0.

clock_network_pins

A read-only collection attribute that returns the value as a collection of pins in the network of this clock.

clock_source_latency_early_fall_max

A read-only float attribute that returns the user-specified clock network latency for early max fall.

c

clock_source_latency_early_fall_min

A read-only float attribute that returns the user-specified clock network latency for early min fall.

clock_source_latency_early_rise_max

A read-only float attribute that returns the user-specified clock network latency for early max rise.

clock_source_latency_early_rise_min

A read-only float attribute that returns the user-specified clock network latency for early min rise.

clock_source_latency_late_fall_max

A read-only float attribute that returns the user-specified clock network latency for late max fall.

clock_source_latency_late_fall_min

A read-only float attribute that returns the user-specified clock network latency for late min fall.

clock_source_latency_late_rise_max

A read-only float attribute that returns the user-specified clock network latency for late max rise.

clock_source_latency_late_rise_min

A read-only float attribute that returns the user-specified clock network latency for late min rise.

clock_source_latency_pins

A read-only collection attribute that returns the value of pins in the source latency network of this clock.

command_text

A read-only string attribute that returns the equivalent command(s) that can re-create the object.

file_line_info

A read-only string attribute that returns the Tcl or SDC source file name and line number pairs of the commands which were used to create this object, if it exists.

full_name

A read-only string attribute that returns the name of a clock.

c

generated_clocks

A read-only collection attribute that returns the value as a collection of generated clock objects. This attribute is defined for any clock that is the parent of one or more generated clocks.

hold_uncertainty

A read-only float attribute that returns the user-specified clock uncertainty for hold.

is_generated

A read-only boolean attribute that returns true if this clock is a generated clock, false otherwise.

master_clock

A read-only collection attribute that returns the master clock for this generated clock, if it exists.

max_time_borrow

A read-only float attribute that returns the upper limit for time borrowing, preventing the use of the entire pulse width for level-sensitive latches. Units are those of the logic library.

object_class

A read-only string attribute that returns the name of this object class. Returns 'clock' for clock objects.

period

A read-only float attribute that returns the period of the clock.

propagated_clock

A read-only boolean attribute that returns true if this clock has propagated clock latency, false if it has ideal latency.

setup_uncertainty

A read-only float attribute that returns the user-specified clock uncertainty for setup.

sources

A read-only collection attribute that returns the value as a collection of source objects for the clock.

waveform

A read-only string attribute that represents the clock waveform as a list of edges.

See Also

- [get_attribute](#)
- [list_attributes](#)
- [set_clock_uncertainty](#)

clock_balance_group_attributes

Specifies the description of the predefined attributes for clock balance group.

Description

Attributes are properties assigned to objects such as pins, cells and nets. Definitions for `clock_balance_group` attributes are provided in the following subsections.

Attributes are read-only unless marked as settable. Read-only attributes are informational, and you cannot set its value. A settable attribute can be modified with the `set_attribute` command.

To determine the value of an attribute, use the `get_attribute` command. To see a list of all attributes available for a class of objects, use the `list_attributes -application` command.

clock_balance_group Attributes*name*

Specifies a string attribute for the name of a `clock_balance_group`.

object_class

Specifies a string attribute with the value of "clock_balance_group". Each object class has a different object class value.

See Also

- [create_clock_balance_group](#)
- [get_attribute](#)
- [list_attributes](#)

clock_group_attributes

Description of the application defined `clock_group` attributes.

Description

Attributes are properties assigned to objects such as pins, cells, and nets. They can be read-only or settable. A read-only attribute is informational; you cannot set its value. To modify a settable attribute, use the *set_attribute* command.

To determine the value of an attribute, use the *get_attribute* command. To see a list of all *clock_group* attributes available, use the *list_attributes -application -class clock_group* command.

Attributes for clock_group

This section describes the *clock_group* attributes.

clock_group_type

A read-only string attribute that returns the type of exception. The valid values are "logically exclusive", "asynchronous", and "physically exclusive".

file_line_info

A read-only string attribute that returns the Tcl or SDC source file name and line number pair of the command, which is used to create this object, if it exists.

full_name

A read-only string attribute that returns a unique name for the *clock_group*. This is the name that you assign or the tool automatically assigns name, if you do not specify any.

group_count

A read-only int attribute that returns the number of individual groups given in the *set_clock_group* command.

is_allow_path

A read-only boolean attribute that returns true if the *-allow_paths* option is used in the *set_clock_group* command for an asynchronous clock group setting.

object_class

A read-only string attribute that returns the name of this object class. Returns 'clock_group' for *clock_group* objects.

See Also

- [get_clock_groups](#)
- [get_attribute](#)
- [list_attributes](#)

c

- [report_clocks](#)
- [set_clock_groups](#)

clock_group_group_attributes

Description of the application defined clock_group_group attributes.

Description

Attributes are properties assigned to objects such as pins, cells, and nets. They can be read-only or settable. A read-only attribute is informational; you cannot set its value. To modify a settable attribute, use the *set_attribute* command.

To determine the value of an attribute, use the *get_attribute* command. To see a list of all clock_group_group attributes available, use the *list_attributes -application -class clock_group_group* command.

Attributes for clock_group_group

This section describes the clock_group_group attributes.

object_class

A read-only string attribute that returns the name of this object class. Returns 'clock_group_group' for clock_group_group objects.

objects

A read-only collection attribute that returns the value of a collection of clocks contained in this group of the clock_group.

See Also

- [get_clock_groups](#)
- [get_clock_group_groups](#)
- [get_attribute](#)
- [list_attributes](#)
- [report_clocks](#)
- [set_clock_groups](#)

clock_path_attributes

Description of the application defined clock_path attributes.

Description

Attributes are properties assigned to objects such as pins, cells, and nets. They can be read-only or settable. A read-only attribute is informational; you cannot set its value. To modify a settable attribute, use the *set_attribute* command.

To determine the value of an attribute, use the *get_attribute* command. To see a list of all clock_path attributes available, use the *list_attributes -application -class clock_path* command.

All clock_path attributes are read-only.

All time-valued attributes will be based on the user output units in effect when the attribute value is gotten, not when the clock path was originally created.

The object-valued attributes (such as clock, mode, corner, etc.) are not guaranteed to have valid values if the design has been modified since the clock_path was created. However, the corresponding "name" attributes (for example, clock_name) will always be valid, even if the object itself no longer exists.

Attributes for clock_path

This section describes the clock_path attributes.

clock

A read-only collection attribute that returns the clock_path's clock.

clock_edge_type

A read-only string attribute that returns the clock_path's relevant edge, as measured at the clock source. The value will be either "rise" or "fall".

clock_edge_value

A read-only time attribute that returns the time of the clock_path's relevant clock edge, measured at the clock source.

clock_latency

A read-only time attribute that returns the latency of the clock at end of path from its ideal edge.

clock_name

A read-only string attribute that returns the name of the clock_path's launch clock.

corner

A read-only collection attribute that returns the clock_path's corner.

c

corner_name

A read-only string attribute that returns the name of clock_path's corner.

design_name

A read-only string attribute that returns the name of the clock_path's design.

driving_cell_adjustment

A read-only time attribute that returns the clock_path's driving_cell adjustment time, for paths starting at input ports.

endpoint

A read-only collection attribute that returns the clock_path's endpoint.

endpoint_name

A read-only string attribute that returns the name of the clock_path's endpoint.

full_name

A read-only string attribute that returns the clock_path's name, which is synthesized from some of its other attributes. The name is not guaranteed to be unique.

lib_name

A read-only string attribute that returns the name of the library containing the clock_path's design.

mode

A read-only collection attribute that returns the clock_path's mode.

mode_name

A read-only string attribute that returns the name of the clock_path's mode.

name

A read-only string attribute that returns the clock_path's name, which is synthesized from some of its other attributes. The name is not guaranteed to be unique.

object_class

A read-only string attribute that returns the name of this object class. Returns 'clock_path' for clock_path objects.

path_type

A read-only string attribute that returns the clock_path's delay type, either 'min' or 'max'.

c

points

A read-only collection attribute that returns the clock_path's points.

propagation_type

A read-only string attribute that is equivalent to the path_type attribute, with a value of "min" or "max".

scenario

A read-only collection attribute that returns the clock_path's scenario.

scenario_name

A read-only string attribute that returns the name of the clock_path's scenario.

source_clock

A read-only collection attribute that returns the clock's source clock, if the clock is generated.

source_clock_edge_type

A read-only string attribute that returns the clock_path's applicable clock edge, as measured at the clock source. The value will be either "rise" or "fall".

source_clock_edge_value

A read-only time attribute that returns the time of the applicable clock edge, measured at the clock source.

source_clock_name

A read-only string attribute that returns the name of the clock's source clock, if the clock is generated.

source_latency

A read-only time attribute that returns the clock_path's source latency.

startpoint

A read-only collection attribute that returns the clock_path's startpoint.

startpoint_name

A read-only string attribute that returns the name of the clock_path's startpoint.

See Also

- [get_attribute](#)
- [list_attributes](#)

c

- [get_timing_paths](#)
- [report_timing](#)
- [set_user_units](#)
- [report_user_units](#)

constraint_group_attributes

Description of the application defined constraint_group attributes.

Description

Attributes are properties assigned to objects such as pins, cells, and nets. They can be read-only or settable. A read-only attribute is informational; you cannot set its value. To modify a settable attribute, use the *set_attribute* command.

To determine the value of an attribute, use the *get_attribute* command. To see a list of all constraint_group attributes available, use the *list_attributes -application -class constraint_group* command.

Attributes for constraint_group

This section describes the constraint_group attributes.

attraction_type

A settable string attribute (with allowed values: default medium strong weak) that returns the attraction force for river style of bus routing type of analog constraints. This is for river style bus routing constraint group.

corner_type

A settable string attribute (with allowed values: auto cross default river) that returns the corner type for bus style of bus routing type of analog constraints. This is for bus style bus routing constraint group.

default_gap

A settable distance attribute that returns the gap between outermost shape of the group with other shapes in the design for differential group/pair, net shielding and bus routing type of analog constraints. This is for differential, shielding and both bus routing constraint groups.

default_width

A settable distance attribute that returns the width of shield material for net shielding type of analog constraints. This is for shielding constraint group.

c

disabled_layers

A settable layer collection attribute that returns the disallowed layers for net shielding type of analog constraints. This is for shielding constraint group.

enclose_pins

A settable string attribute (with allowed values: false true unset) that returns whether pins are enclosed by the shield for net shielding type of analog constraints. This is for shielding constraint group.

enclose_vias

A settable string attribute (with allowed values: false true unset) that returns whether vias are enclosed by the shield for net shielding type of analog constraints. This is for shielding constraint group.

full_name

A read-only string attribute that returns the full name of the constraint group.

group_shield

A read-only boolean attribute that returns whether a single set of shield shapes can be used to shield all nets of the constraint group for net shielding type of analog constraints.

groups

A read-only group collection attribute that returns the groups in which this constraint group is a member.

is_relative

A settable boolean attribute that specifies whether the tolerance value is in absolute (when false) or percentage for wire matching, resistance limit and wire length limit type of analog constraints. This is for wire matching constraint group.

jog_length

A settable distance attribute that defines the minimum length of jogs for river style bus routing type of analog constraints. This is for river style bus routing constraint group.

layer_gap

A settable distance attribute that defines the layer specific gap between route objects and shield for net shielding type of analog constraints. This is a subscripted attribute based on layer names. This is for shielding constraint group.

c

layer_lines

A settable int attribute that defines the layer specific number of lines allowed for reliability verification type of analog constraints. This is a subscripted attribute based on layer names. This is for reliability verification constraint group.

layer_max_gap

A settable distance attribute that defines the layer specific max gap between route objects and shield for net shielding type of analog constraints. This is a subscripted attribute based on layer names. This is for shielding constraint group.

layer_spacing

A settable distance attribute that defines the layer specific minimum spacing for trunk for differential group/pair and bus routing type of analog constraints. This is a subscripted attribute based on layer names. Only one layer name is applicable for river style bus routing type. This is for differential and both bus routing constraint group.

layer_width

A settable distance attribute that defines the layer specific minimum width for trunk for differential group/pair, net shielding, bus routing and reliability verification type of analog constraints. This is a subscripted attribute based on layer names. Only one layer name is applicable for river style bus routing type. This is for differential, shielding, both bus routing and reliability verification constraint group.

matching_style

A settable string attribute (with allowed values: length length_per_layer resistance) that specifies the parameter used for matching the nets for wire matching, resistance limit and wire length limit type of analog constraints. This is for wire matching, resistance limit and wire length limit constraint group and is a settable attribute for wire matching.

max_gap

A settable distance attribute that defines the max gap between route objects and shield for net shielding type of analog constraints. This is for shielding constraint group.

max_layer

A read-only layer collection attribute that defines the maximum metal layer out of the specified layers for differential group/pair and bus style bus routing type of analog constraints. This is for differential and bus style bus routing constraint group.

c

min_layer

A read-only layer collection attribute that defines the minimum metal layer out of the specified layers for differential group/pair and bus style bus routing type of analog constraints. This is for differential and bus style bus routing constraint group.

min_segment

A settable distance attribute that returns the minimum segment length to be shielded for net shielding type of analog constraints. This is for shielding constraint group.

min_value

A settable float attribute that returns the minimum value to be achieved for each of the constituents of the constraint group for resistance limit and wire length limit type of analog constraints. This is for resistance limit and wire length limit constraint group.

name

A settable string attribute that returns the name of the constraint group.

object_class

A read-only string attribute that returns the name of this object class. Returns 'constraint_group' for constraint_group objects.

objects

A read-only collection attribute that returns the members of the constraint group.

priority

A settable int attribute that returns the priority of its constituents for net priority type of analog constraints. This is for net priority constraint group.

reference_net

A settable net collection attribute that dictates the topology of the bus routing for river style bus routing type of analog constraints. This is for river style bus routing constraint group.

river_layer

A settable layer collection attribute that specifies the layer for river trunk for river style bus routing type of analog constraints. This is for river style bus routing constraint group.

sharing

A settable string attribute (with allowed values: false true unset) that specifies if a shield can be shared between constituents of the group for net shielding type of analog constraints. This is for shielding constraint group.

shield_net

A settable net collection attribute that specifies the net to connect to shield material for net shielding type of analog constraints. This is for shielding constraint group.

shield_net_2

A settable net collection attribute that specifies a second net to connect to shield material for net shielding type of analog constraints. This is for shielding constraint group.

shield_overhang

A read-only string attribute that returns the distance list applicable for shield overhang type of analog constraints defining the lower and upper overhang for shielding with a given metal and its width. This is a subscripted attribute based on layer names.

shield_placement

A settable string attribute (with allowed values: default double_interleave half_interleave interleave outside) that defines how the main bus trunk will be shielded when one or more constituent nets of the group are shielded for differential group/pair and only bus style of bus routing type of analog constraints.

shielding_style

A settable string attribute (with allowed values: above above_center_coaxial above_solid_coaxial above_split_coaxial below below_center_coaxial below_solid_coaxial below_split_coaxial center_coaxial coaxial offset_above offset_above_center_coaxial offset_above_solid_coaxial offset_above_split_coaxial offset_below offset_below_center_coaxial offset_below_solid_coaxial offset_below_split_coaxial offset_center_coaxial offset_coaxial offset_split_coaxial offset_tandem parallel parallel_multi_layer split_coaxial tandem) that specifies a collective set of shielding style, tandem layers, tandem offset and coaxial type for net shielding type of analog constraints. Below table captures the set. This is for shielding constraint group and is a settable attribute.

name	shield style	tandem layers	tandem offset	coaxial type

c

parallel	parallel	N/A	N/A	N/A
tandem	tandem	both	1	N/A
coaxial	coaxial	both	1	solid
split_coaxial	coaxial	both	1	split
center_coaxial	coaxial	both	1	
centerOnly				
below	tandem	below	1	N/A
above	tandem	above	1	N/A
offset_below	tandem	below	2	N/A
offset_above	tandem	above	2	N/A
offset_tandem	tandem	both	2	N/A
offset_coaxial	coaxial	both	2	solid
below_split_coaxial	coaxial	below	1	split
above_split_coaxial	coaxial	above	1	split
below_center_coaxial	coaxial	below	1	
centerOnly				
above_center_coaxial	coaxial	above	1	
centerOnly				
above_solid_coaxial	coaxial	above	1	solid
below_solid_coaxial	coaxial	below	1	solid
offset_above_solid_coaxial	coaxial	above	2	solid
offset_below_solid_coaxial	coaxial	below	2	solid
offset_split_coaxial	coaxial	both	2	split
offset_above_split_coaxial	coaxial	above	2	split
offset_below_split_coaxial	coaxial	below	2	split
offset_center_coaxial	coaxial	both	2	
centerOnly				
offset_above_center_coaxial	coaxial	above	2	
centerOnly				
offset_below_center_coaxial	coaxial	below	2	
centerOnly				

tolerance

A settable float attribute that defines the tolerance, either as absolute or percentage (governed by `is_relative`), of the specified parameter across all constituents of the group for wire matching type of analog constraints. This is for wire matching constraint group.

twist_interval

A settable distance attribute that defines the interval for twisting and is applicable only when twist style is not none for differential group/pair type of analog constraints. This is for differential constraint group.

twist_offset

A settable distance attribute that defines the offset for twisting and is applicable only when twist style is not none for differential group/pair type of analog constraints. This is for differential constraint group.

c

twist_style

A settable string attribute (with allowed values: default diagonal none orthogonal) that defines how the nets in the group will be twisted for differential group/pair type of analog constraints. This is for differential constraint group.

type

A read-only string attribute (with allowed values: bus_style differential_group differential_pair matched_wire net_priority reliability_verification resistance_limit river_style shield_overhang shielding wire_length_limit) that returns the type of the constraint group.

via_defs

A settable via_def collection attribute that specifies the via definitions to be used for net shielding type of analog constraints. This is for shielding constraint group.

See Also

- [create_wire_matching](#)
- [create_length_limit](#)
- [create_differential_group](#)
- [create_net_shielding](#)
- [create_net_priority](#)
- [create_bus_routing_style](#)
- [get_attribute](#)
- [list_attributes](#)
- [set_attribute](#)

core_area_attributes

Description of the application defined core_area attributes.

Description

Attributes are properties assigned to objects such as pins, cells, and nets. They can be read-only or settable. A read-only attribute is informational; you cannot set its value. To modify a settable attribute, use the *set_attribute* command.

To determine the value of an attribute, use the *get_attribute* command. To see a list of all core_area attributes available, use the *list_attributes -application -class core_area* command.

Attributes for core_area

This section describes the core_area attributes.

area

A read-only area attribute that returns the area of the core area of a block.

bbox

A read-only rect attribute that returns the bounding-box of a core_area. The format of a rectangle specification is `{{llx lly} {urx ury}}`, which specifies the lower-left and upper-right corners of the rectangle.

boundary

A settable coord_list attribute that returns the boundary of a core_area. The format of a boundary specification is `{{{llx1 lly1} {urx1 ury1}} {{llx2 lly2} {urx2 ury2} ...} ...}`. This is listed as a settable attribute, but it can be set only while site rows are not defined for the block. Once site rows for the block are defined, the core area is derived from them and can not be set anymore.

bounding_box

A read-only bounding_box collection attribute that returns the bounding-box of a core_area. The format of the collection specifies lower-left and upper-right corners of the rectangle.

full_name

A read-only string attribute that returns the name of the core_area.

name

A read-only string attribute that returns the name of the core_area.

object_class

A read-only string attribute that returns the name of this object class. Returns 'core_area' for core_area objects.

See Also

- [get_core_area](#)
- [get_attribute](#)
- [list_attributes](#)
- [set_attribute](#)

corner_attributes

Description of the application defined corner attributes.

Description

Attributes are properties assigned to objects such as pins, cells, and nets. They can be read-only or settable. A read-only attribute is informational; you cannot set its value. To modify a settable attribute, use the *set_attribute* command.

To determine the value of an attribute, use the *get_attribute* command. To see a list of all corner attributes available, use the *list_attributes -application -class corner* command.

Attributes for corner

This section describes the corner attributes.

file_line_info

A read-only string attribute that returns the Tcl or SDC source file name and line number pairs of the commands used to create this object, if it exists.

full_name

A read-only string attribute that returns the name of a corner.

is_current

A read-only boolean attribute that indicates whether this corner is the current corner.

is_default

A read-only boolean attribute that indicates whether this corner is the default corner.

object_class

A read-only string attribute that returns the name of this object class. Returns 'corner' for corner objects.

See Also

- [create_corner](#)
- [get_attribute](#)
- [list_attributes](#)

curved_poly_rect_attributes

Description of the application defined curved_poly_rect attributes.

Description

Attributes are properties assigned to objects such as pins, cells, and nets. They can be read-only or settable. A read-only attribute is informational; you cannot set its value. To modify a settable attribute, use the *set_attribute* command.

To determine the value of an attribute, use the *get_attribute* command. To see a list of all curved_poly_rect attributes available, use the *list_attributes -application -class curved_poly_rect* command.

Attributes for curved_poly_rect

This section describes the curved_poly_rect attributes.

bbox

A read-only rect attribute that returns the bounding box of the curved_poly_rect.

bbox_llx

A read-only distance attribute that returns the lower left horizontal coordinate of the bounding box of the curved_poly_rect.

bbox_lly

A read-only distance attribute that returns the lower left vertical coordinate of the bounding box of the curved_poly_rect.

bbox_urx

A read-only distance attribute that returns the upper right horizontal coordinate of the bounding box of the curved_poly_rect.

bbox_ury

A read-only distance attribute that returns the upper right vertical coordinate of the bounding box of the curved_poly_rect.

bounding_box

A read-only bounding_box collection attribute that returns the bounding-box of the curved_poly_rect. The format of the collection specifies lower-left and upper-right corners of the rectangle.

height

A read-only distance attribute that returns the height of the curved_poly_rect.

layer

A read-only layer collection attribute that returns the (optional) layer on which the `curved_poly_rect` resides.

layer_name

A read-only string attribute that returns the name of the layer on which the `curved_poly_rect` (optionally) resides.

mask

A settable string attribute (with allowed values: `no_mask` `mask_one` `mask_two` `mask_three` `mask_four` `same_mask`) that returns the mask constraint applied to the `curved_poly_rect`.

object_class

A read-only string attribute that returns the name of this object class. Returns 'curved_poly_rect' for `curved_poly_rect` objects.

point_list

A read-only `coord_list` attribute that returns the points of the `curved_poly_rect` boundary.

width

A read-only distance attribute that returns the width of the `curved_poly_rect`.

See Also

- [create_curved_poly_rect](#)
- [get_attribute](#)
- [list_attributes](#)
- [set_attribute](#)

d

density_rule_attributes

Description of the application defined `density_rule` attributes.

Description

Attributes are properties assigned to objects such as pins, cells, and nets. They can be read-only or settable. A read-only attribute is informational; you cannot set its value. To modify a settable attribute, use the *set_attribute* command.

To determine the value of an attribute, use the *get_attribute* command. To see a list of all *density_rule* attributes available, use the *list_attributes -application -class density_rule* command.

Attributes for *density_rule*

This section describes the *density_rule* attributes.

full_name

A read-only string attribute that returns the name of a density rule.

layer

A read-only layer collection attribute that returns the layer on which density rule exists.

layer_name

A settable string attribute that returns the name of the layer on which density rule exists.

max_density

A settable double attribute that returns the maximum percentage of metal allowed in the window.

max_gradient_density

A settable double attribute that returns the maximum percentage difference between the fill density of adjacent windows.

min_density

A settable double attribute that returns the minimum percentage of metal allowed in the window.

name

A read-only string attribute that returns the name of a density rule.

object_class

A read-only string attribute that returns the name of this object class. Returns 'density_rule' for *density_rule* objects.

window_size

A settable float attribute that returns the size of the window for the density check. By default, the window step size is half of the defined `windowSize`. The check setup is assumed to be half of the window size.

See Also

- [get_attribute](#)
- [list_attributes](#)

design_attributes

Description of the application defined design attributes.

Description

Attributes are properties assigned to objects such as pins, cells, and nets. They can be read-only or settable. A read-only attribute is informational; you cannot set its value. To modify a settable attribute, use the *set_attribute* command.

To determine the value of an attribute, use the *get_attribute* command. To see a list of all design attributes available, use the *list_attributes -application -class design* command.

Attributes for design

This section describes the design attributes.

allowable_orientation

A settable orientations attribute that returns the list of allowed orientations for instances of this design.

always_on

A read-only boolean attribute that returns True if the design is an always on cell.

antenna_diode_type

A read-only string attribute that returns the antenna diode type of the design. The allowable values are "power", "ground" or "power_ground".

area

A read-only area attribute that returns the area of the design.

bbox

A read-only rect attribute that returns the bounding-box of a design. The format of a rectangle specification is `{{llx lly} {urx ury}}`, which specifies the lower-left and upper-right corners of the rectangle.

block

A read-only block collection attribute that returns the block of this design.

block_name

A read-only string attribute that returns the name of this design as a block.

boundary

A settable coord_list attribute that returns the boundary of a design. The format of a coordinate_list specification is {{x1 y1} {x2 y2}...}.

boundary_bbox

A read-only rect attribute that returns the bounding-box of a design. The format of a rectangle specification is {{llx lly} {urx ury}}, which specifies the lower-left and upper-right corners of the rectangle.

boundary_bounding_box

A read-only bounding_box collection attribute that returns the bounding-box of a design. The format of the collection specifies lower-left and upper-right corners of the rectangle.

bounding_box

A read-only bounding_box collection attribute that returns the bounding-box of a design. The format of the collection specifies lower-left and upper-right corners of the rectangle.

clock_gating_integrated_cell

A read-only string attribute that returns the name of the integrated clock gating if the design is of type integrated clock gating.

cmog_class

A read-only string attribute that returns the cmog_class value for the design.

core_area_area

A read-only double attribute that returns the area of the core_area.

core_area_bbox

A read-only rect attribute that returns the bounding-box of the core_area. The format of a rectangle specification is {{llx lly} {urx ury}}, which specifies the lower-left and upper-right corners of the rectangle.

core_area_boundary

A read-only coord_list attribute that returns the coordinates of the core_area. The format of a coordinate_list specification is {{x1 y1} {x2 y2}...}.

core_area_bounding_box

A read-only bounding_box collection attribute that returns the bounding-box of the core_area. The format of the collection specifies lower-left and upper- right corners of the rectangle.

default_site

A read-only string attribute that returns the default site defs associated with the site available in the block. This attribute is not defined for design.

design

A read-only design collection attribute that returns this design as a design object.

design_name

A read-only string attribute that returns the name of this design as a design.

design_type

A settable string attribute (with allowed values: 3dic abstract analog analysis black_box bridge corner cover die diode end_cap feedthrough fill filler flip_chip_driver flip_chip_pad ftv interposer lib_cell macro module package pad pad_spacer pcb rdl_fanout rtl substrate thermal tsv vib well_tap) that returns the design type of this design. The design types "bridge" and "substrate" are deprecated.

disable_timing

A read-only boolean attribute that returns True if the design is marked as disable_timing in the library. This attribute can be set in icc2_lm_shell.

dont_touch

A settable boolean attribute that returns True if the lib_cell is marked as dont_touch to prevent optimization from changing any instances of this lib_cell.

dont_use

A settable boolean attribute that returns True if the lib_cell is marked as dont_use to prevent optimization from inserting instances of this lib_cell.

early_fall_cell_check_derate_factor

A read-only float attribute that returns an early timing derating factor, specified by the set_timing_derate command, that applies to the design.

early_fall_clk_cell_derate_factor

A read-only float attribute that returns an early timing derating factor, specified using the set_timing_derate command, that applies to the design.

early_fall_clk_net_delta_derate_factor

A read-only float attribute that returns an early timing derating factor, specified using the `set_timing_derate` command, that applies to the design.

early_fall_clk_net_derate_factor

A read-only float attribute that returns an early timing derating factor, specified using the `set_timing_derate` command, that applies to the design.

early_fall_data_cell_derate_factor

A read-only float attribute that returns an early timing derating factor, specified using the `set_timing_derate` command, that applies to the design.

early_fall_data_net_delta_derate_factor

A read-only float attribute that returns an early timing derating factor, specified using the `set_timing_derate` command, that applies to the design.

early_fall_data_net_derate_factor

A read-only float attribute that returns an early timing derating factor, specified using the `set_timing_derate` command, that applies to the design.

early_rise_cell_check_derate_factor

A read-only float attribute that returns an early timing derating factor, specified using the `set_timing_derate` command, that applies to the design.

early_rise_clk_cell_derate_factor

A read-only float attribute that returns an early timing derating factor, specified using the `set_timing_derate` command, that applies to the design.

early_rise_clk_net_delta_derate_factor

A read-only float attribute that returns an early timing derating factor, specified using the `set_timing_derate` command, that applies to the design.

early_rise_clk_net_derate_factor

A read-only float attribute that returns an early timing derating factor, specified using the `set_timing_derate` command, that applies to the design.

early_rise_data_cell_derate_factor

A read-only float attribute that returns an early timing derating factor, specified using the `set_timing_derate` command, that applies to the design.

early_rise_data_net_delta_derate_factor

A read-only float attribute that returns an early timing derating factor, specified using the `set_timing_derate` command, that applies to the design.

early_rise_data_net_derate_factor

A read-only float attribute that returns an early timing derating factor, specified using the `set_timing_derate` command, that applies to the design.

excluded_purposes

A read-only string attribute that returns the purposes that are excluded as valid usages of the block. This is meant for `lib_cells` only.

extended_name

A read-only string attribute that returns the full path to the library plus the design name, separated by a ':' character, an optional label name, separated by a '/' character, and a view name, separated by a '.'. For example, `"/disks/user1/work/top_lib:top_design/my_label.design"`.

foundry

A read-only string attribute that returns the foundry value for the design.

frame_update

A settable boolean attribute that provides user control to select which specific designs to update frame view in edit flow. The value is set to `"true"` if the design frame view needs to be updated and `"false"` otherwise. The default value is `"false."`

full_name

A read-only string attribute that returns the full name of a design, including the library name plus the design name, separated by a ':' character, an optional label name, separated by a '/' character, and a view name, separated by a '.'. For example, `"top_lib:top_design/my_label.design"`

function_class

A read-only string attribute that returns the function class value for the design.

glitch_power

A read-only double attribute that returns the glitch power of the design in watts. It is the sum of the glitch power of all the cells of the design.

has_timing_model

A read-only boolean attribute that indicates if the timing data is available for the design.

height

A read-only distance attribute that returns the height of the boundary of the block.

hpc_core

A read-only string attribute that returns the HPC core value for the design.

included_purposes

A read-only string attribute that returns the purposes that are included as valid usages of the block. This is meant for lib_cells only.

inner_keepout_margin_clock

A settable margin_list attribute that returns the clock inner keepout.

inner_keepout_margin_hard

A settable margin_list attribute that returns the hard inner keepout.

inner_keepout_margin_hard_macro

A settable margin_list attribute that returns the hard_macro inner keepout.

inner_keepout_margin_route_blockage

A settable margin_list attribute that returns the route_blockage inner keepout.

inner_keepout_margin_route_blockage_layers

A read-only layer collection attribute that returns the route_blockage_layers inner keepout.

inner_keepout_margin_soft

A settable margin_list attribute that returns the soft inner keepout.

internal_power

A read-only double attribute that returns the internal power of the design in watts. It is the sum of the internal power of all the cells of the design.

internal_power_derate_factor

A read-only float attribute that returns the actually stored internal derate values if and only if they exist. The value is based on current timer status, the attribute does not update the timer.

is_backside

A read-only boolean attribute that returns true if this design is a backside bump cell and false otherwise. A backside bump cell is a lib cell having design_type "flip_chip_pad" and either shapes on the backside metal layers, or shapes on the metal1 layer when its no_substrate attribute is true.

is_black_box

A read-only boolean attribute that returns true if this design has no logic function.

is_combinational

A read-only boolean attribute that returns true if the design is not sequential.

is_compressed

A settable boolean attribute that returns true if this design is saved in compressed format on disk, false otherwise.

is_current

A read-only boolean attribute that returns true if this design is the current_design for the session, false otherwise.

is_diff_level_shifter

A read-only boolean attribute that returns true if the design is a differential level shifter.

is_diode_cell

A read-only boolean attribute that returns true if the design is a power diode cell or ground diode cell.

is_enable_level_shifter

A read-only boolean attribute that returns true if the design is a enable level shifter cell.

is_etm_moded_cell

A read-only boolean attribute that returns true if design is ETM, and false otherwise.

is_fall_edge_triggered

A read-only boolean attribute that returns true if the design has timing behavior relative to the falling edge of a clock signal. This attribute can be set in icc2_lm_shell.

is_hierarchical

A read-only boolean attribute that always returns true.

is_implicit

A read-only boolean attribute that returns true if the implicit flag is set on the lib cell.

is_integrated_clock_gating_cell

A read-only boolean attribute that returns true if the design is defined in the library as an integrated clock gating cell. This attribute can be set in `icc2_lm_shell`.

is_isolation

A read-only boolean attribute that returns true if the design is a isolation cell.

is_level_shifter

A read-only boolean attribute that returns true if the design is a level shifter cell.

is_linked

A read-only boolean attribute that returns true if this design is already linked, and false otherwise.

is_mask_shiftable

A settable boolean attribute that returns true if the design is mask shiftable.

is_memory_cell

A read-only boolean attribute that returns true if the design is defined in the library as a memory cell. This attribute can be set in `icc2_lm_shell`.

is_modified

A read-only boolean attribute that returns true if this design has modified and unsaved data, false otherwise.

is_mux

A read-only boolean attribute that returns true if the design is a mux.

is_negative_level_sensitive

A read-only boolean attribute that returns true if the design has negative level-sensitive timing behavior, as in a negative D-latch. This attribute can be set in `icc2_lm_shell`.

is_open

A read-only boolean attribute that returns true if this design is already loaded into memory, false otherwise.

is_port_punching_ok

A settable boolean attribute that enables the ability to add or remove ports in the design when set to true.

is_positive_level_sensitive

A read-only boolean attribute that returns true if the design has positive level-sensitive timing behavior, as in a positive D-latch. This attribute can be set in `icc2_lm_shell`.

is_power_switch

A read-only boolean attribute that returns true if the design is a power switch cell.

is_remote

A read-only boolean attribute that returns true if a block is a remote design. The attribute should be always 'false' for lib cell.

is_retention

A read-only boolean attribute that returns true if the design is a retention cell.

is_rise_edge_triggered

A read-only boolean attribute that returns True if the design has timing behavior relative to the rising edge of a clock signal. This attribute can be set in `icc2_lm_shell`

is_sequential

A read-only boolean attribute that returns True if the design has sequential logic function.

is_soi

A read-only boolean attribute that returns True if the design is a SOI cell.

is_three_state

A read-only boolean attribute that returns True if the design has one or more three-state outputs. This attribute can be set in `icc2_lm_shell`.

keepouts

A read-only `keepout_margin` collection attribute that returns the keepouts for the block.

label_name

A read-only string attribute that returns a label for the block.

last_modified

A read-only string attribute that returns the date and time this design was last modified.

last_saved

A read-only string attribute that returns the date and time this design was last saved.

late_fall_cell_check_derate_factor

A read-only float attribute that returns a late timing derating factor, specified using the `set_timing_derate` command, that applies to the design.

late_fall_clk_cell_derate_factor

A read-only float attribute that returns a late timing derating factor, specified using the `set_timing_derate` command, that applies to the design.

late_fall_clk_net_delta_derate_factor

A read-only float attribute that returns a late timing derating factor, specified using the `set_timing_derate` command, that applies to the design.

late_fall_clk_net_derate_factor

A read-only float attribute that returns a late timing derating factor, specified using the `set_timing_derate` command, that applies to the design.

late_fall_data_cell_derate_factor

A read-only float attribute that returns a late timing derating factor, specified using the `set_timing_derate` command, that applies to the design.

late_fall_data_net_delta_derate_factor

A read-only float attribute that returns a late timing derating factor, specified using the `set_timing_derate` command, that applies to the design.

late_fall_data_net_derate_factor

A read-only float attribute that returns a late timing derating factor, specified using the `set_timing_derate` command, that applies to the design.

late_rise_cell_check_derate_factor

A read-only float attribute that returns a late timing derating factor, specified using the `set_timing_derate` command, that applies to the design.

late_rise_clk_cell_derate_factor

A read-only float attribute that returns a late timing derating factor, specified using the `set_timing_derate` command, that applies to the design.

late_rise_clk_net_delta_derate_factor

A read-only float attribute that returns a late timing derating factor, specified using the `set_timing_derate` command, that applies to the design.

late_rise_clk_net_derate_factor

A read-only float attribute that returns a late timing derating factor, specified using the `set_timing_derate` command, that applies to the design.

late_rise_data_cell_derate_factor

A read-only float attribute that returns a late timing derating factor, specified using the `set_timing_derate` command, that applies to the design.

late_rise_data_net_delta_derate_factor

A read-only float attribute that returns a late timing derating factor, specified using the `set_timing_derate` command, that applies to the design.

late_rise_data_net_derate_factor

A read-only float attribute that returns a late timing derating factor, specified using the `set_timing_derate` command, that applies to the design.

leakage_power

A read-only double attribute that returns the leakage power of the design in watts. It is the sum of the leakage power of all the cells of the design.

leakage_power_derate_factor

A read-only float attribute that returns the actually stored leakage derate values if and only if they exist. The value is based on current timer status, the attribute does not update the timer.

lib

A read-only lib collection attribute that returns the library of this design.

lib_name

A read-only string attribute that returns the name of library of this design.

logic_only

A settable boolean attribute that returns true if this design is marked for the logic-only design flow. The attribute can only be set to "true" before the design is linked with `link_design`. Logic-only designs are used for Design Planning budgeting timing. They can not have physical design information, so they use less memory.

max_capacitance

A read-only float attribute that returns the default maximum capacitance design rule limit for the design. The units must be consistent with those of the logic library used during optimization. It is set with the `set_max_capacitance` command.

max_capacitance_constraint

A read-only float attribute that returns the float value for max capacitance constraint on the design.

max_lvth_percentage

A read-only float attribute that returns the maximum limit on the percentage of low-Vth cells in the current design for optimization.

max_transition

A read-only float attribute that returns the default maximum transition design rule limit for the design. The units must be consistent with those of the logic library used during optimization. It is set with the `set_max_transition` command.

max_transition_constraint

A read-only float attribute that returns the float value for max transition constraint on the design.

mb_bussed_ports

A read-only string attribute that returns the multibit bussed ports for the design.

mb_pin_map

A read-only string attribute that returns the multibit pin map for the design.

mb_scan_chain_ports

A read-only string attribute that returns the multibit scanchain ports for the design.

min_capacitance

A read-only float attribute that returns the default minimum capacitance design rule limit for the design. The units must be consistent with those of the logic library used during optimization. It is set with the `set_min_capacitance` command.

min_capacitance_constraint

A read-only float attribute that returns the float value for min transition constraint on the design.

multibit_width

A read-only int attribute that returns the multibit width for the design.

name

A read-only string attribute that returns the name of the design.

no_substrate

A settable boolean attribute that indicates whether this design has no substrate, and therefore no backside metal layers. When true, the design and cell `is_backside` attribute treats the metal1 layer as a backside metal layer. When false, only the bmetal layers are treated a backside metal layers.

number_of_pins

A read-only int attribute that returns the number of pins on this lib_cell.

object_class

A read-only string attribute that returns the name of this object class. Returns 'design' for design objects.

open_count

A read-only int attribute that returns the number of times this design has been opened.

open_mode

A read-only string attribute that returns the open-mode of the design. Valid values are "read", "edit", and "new".

operating_condition_early

A read-only string attribute that is an alias for the *operating_condition_min* attribute.

operating_condition_late

A read-only string attribute that is an alias for the *operating_condition_max* attribute.

operating_condition_max

A read-only string attribute that returns the name of the maximum or single operating condition for the design. It is set with the `set_operating_conditions` command.

operating_condition_min

A read-only string attribute that returns the name of the minimum operating condition for the design. This attribute is not valid in single operating condition analysis.

outer_keepout_boundary

A read-only coord_list attribute that returns the outer keepout boundary.

outer_keepout_margin_clock

A settable margin_list attribute that returns the clock outer keepout.

outer_keepout_margin_hard

A settable margin_list attribute that returns the hard outer keepout.

outer_keepout_margin_hard_macro

A settable margin_list attribute that returns the hard_macro outer keepout.

outer_keepout_margin_route_blockage

A settable margin_list attribute that returns the route_blockage outer keepout.

outer_keepout_margin_route_blockage_layers

A read-only layer collection attribute that returns the route_blockage_layers outer keepout.

outer_keepout_margin_soft

A settable margin_list attribute that returns the soft outer keepout.

pad_cell

A read-only boolean attribute that returns true if the design is defined in the library as a pad cell.

read_from_product_name

A read-only string attribute that returns the name of the binary used to create this library.

read_from_product_version

A read-only string attribute that returns the version of the binary used to create this library.

read_from_schema_version

A read-only string attribute that returns the schema version of the library file, in (major_version... minor_version) format.

reference_orientation

A settable string attribute (with allowed values: R0 R90 R180 R270 MX MXR90 MY MYR90) that returns the orientation for instances of this block. This is meant for lib_cells.

sb_degen_fid

A read-only string attribute that returns the single bit degenerate function id for the design.

sb_port_cnt

A read-only int attribute that returns the single bit port count for the design.

single_bit_degenerate

A read-only string attribute that returns the single bit degenerate value for the design.

site_def

A read-only site_def collection attribute that returns the site def associated with the block. This is meant for lib_cells only.

site_name

A read-only string attribute that returns the name of the site def associated with the block. This is meant for lib_cells only.

subset_name

A settable name_list attribute that returns the block subset name for LEQ. This attribute is valid only for blocks which are library cells. All library cells with no subset name also form a subset, which is used by the technology mapping function. Named subsets are used for special library cells (e.g., clock buffers that are specifically designed to match the delay of certain clock gates).

switch_list

A read-only string attribute that returns the effective view switch list of this design. The effective view switch list is the precedence-ordered list of design views that is applied when attempting to bind this design.

switching_power

A read-only double attribute that returns the switching power of the design in watts. It is the sum of the switching power of all the cells of the design.

switching_power_derate_factor

A read-only float attribute that returns the actually stored switching derate values if and only if they exist. The value is based on current timer status, the attribute does not update the timer.

target_boundary_area

A settable area attribute that returns the desired or target boundary area for the block.

target_utilization

A settable float attribute that returns the desired or target utilization for the block.

technology_node

A read-only string attribute that returns the technology node for the design.

test_normal_func_id

A read-only string attribute that returns the name of the test normal function id of the design.

threshold_voltage_group

A settable string attribute that returns the name of the category of the design based on its threshold voltage characteristics.

threshold_voltage_group_type

A read-only string attribute that returns the type of the category of the design based on its threshold voltage characteristics. The valid values are "low_vt", "normal_vt" and "high_vt".

top_module

A read-only module collection attribute that returns the top module of this design.

top_module_name

A settable string attribute that returns the name of the top module of this design.

use_for_size_only

A read-only boolean attribute that returns True if the block is to be used for sizing only. This is meant for lib_cells only.

use_frame_blockage

A settable boolean attribute that enables the user to choose to not use blockages in certain frames. The default value is "true", which results in usage of the blockages in the frame.

user_function_class

A settable string attribute that returns the name of the user function class of the block. This is meant for lib_cells only.

valid_purposes

A read-only string attribute that returns the purposes that are valid usages of the block. This is meant for lib_cells only.

via0_alignment

A settable boolean attribute that indicates whether the design needs to be VIA0 aligned.

via_regions

A read-only `via_region` collection attribute that returns the via regions of the block. This is meant for `lib_cells` only.

view_name

A read-only string attribute (with allowed values: abstract conn design frame layout outline rtl timing) that returns the view name of this design.

voltage_max

A read-only float attribute that returns the voltage value of the maximum or single operating condition for the design. The operating condition is defined in a library or by the `create_operating_conditions` command. You associate operating conditions with a design using the `set_operating_conditions` command. This attribute represents the supply voltage value for the operating condition.

voltage_min

A read-only float attribute that returns the voltage value of the minimum operating condition for the design. The operating condition is defined in a library or by the `create_operating_conditions` command. You associate operating conditions with a design using the `set_operating_conditions` command. This attribute represents the supply voltage value for the operating condition.

width

A read-only distance attribute that returns the width of the boundary of the block.

See Also

- [current_design](#)
- [get_designs](#)
- [get_attribute](#)
- [list_attributes](#)
- [set_attribute](#)

design_checks_attributes

Describes the predefined attributes for `design_checks` attributes.

Description

Attributes are properties assigned to objects such as pins, cells and nets. Definitions for `design_checks` attributes are provided in the following subsections.

Attributes are read-only unless marked as settable. Read-only attributes are informational, and you cannot set its value. A settable attribute can be modified with the *set_attribute* command.

To determine the value of an attribute, use the *get_attribute* command. To see a list of all attributes available for a class of objects, use the *list_attributes -application -class ems_check* command.

design_checks Attributes:

current_definition

Specifies a string attribute representing the current check - or a group of checks performed by using the *design_check*.

default_definition

Specifies a string attribute representing the default setting of checks performed by using the *design_check*. Be aware that user-defined checks do not have default definitions.

enabled

Specifies a Boolean attribute representing the current enable status of the check. This is a settable attribute.

full_name

Specifies a string attribute for the full name of the *design_check*. This attribute is essential for *get_object_name* command.

group_members_count

Specifies a string (number) attribute describing the number of members in the group. If the design check is defined as a check, this number is 0. If the design check is defined as a group, this number is higher than 0.

group_name

Specifies a string attribute describing the group name. If the design check is defined as a check, this attribute is empty. If the design check is defined as a group, this attribute gives the group name.

help_info

Specifies a string attribute describing what is the check doing.

is_default

Specifies a Boolean attribute representing the current status of the command. If the default settings of the design check remains, it returns true. If the default settings are overridden by your settings, it returns false.

is_group_member

Specifies a Boolean attribute describing a check membership in a group of checks. If the design check is not a member in a group of checks, it returns false. If the design check is a member in a group of checks, it returns true.

is_group_name

Specifies a Boolean attribute describing if a design check is defined as group. If the design check is defined as a check, it returns false. If the design check is defined as a group, it returns true.

name

Specifies a string attribute for the name of the design_check.

object_class

Specifies a string attribute describing the check design class, which is "ems_check".

See Also

- [get_design_checks](#)
- [get_attribute](#)
- [list_attributes](#)
- [get_object_name](#)
- [set_attribute](#)

design_rule_attributes

Description of the application defined design_rule attributes.

Description

Attributes are properties assigned to objects such as pins, cells, and nets. They can be read-only or settable. A read-only attribute is informational; you cannot set its value. To modify a settable attribute, use the *set_attribute* command.

To determine the value of an attribute, use the *get_attribute* command. To see a list of all design_rule attributes available, use the *list_attributes -application -class design_rule* command.

For more details about the design_rule attributes, please refer to the tech reference manual.

Attributes for design_rule

This section describes the design_rule attributes.

enc_metal_x_min_spacing_tbl

A settable distance_list attribute that returns 2D values.

enc_metal_y_min_spacing_tbl

A settable distance_list attribute that returns 2D values.

fat_wire_via_enc_other_layer_excluded_height_tbl

A settable distance_list attribute that returns 2D values.

fat_wire_via_enc_other_layer_excluded_width_tbl

A settable distance_list attribute that returns 2D values.

fat_wire_via_enclosure_other_layer_tbl

A settable distance_list attribute that returns 2D values.

full_name

A read-only string attribute that returns the name of a pr rule.

min_spacing_x_parallel_length_threshold_tbl

A settable distance_list attribute that returns 2D values.

min_spacing_y_parallel_length_threshold_tbl

A settable distance_list attribute that returns 2D values.

misaligned_via_corner_min_spacing_tbl

A settable distance_list attribute that returns 2D values.

misaligned_via_min_spacing_tbl

A settable distance_list attribute that returns 2D values.

name

A read-only string attribute that returns the name of a pr rule.

object_class

A read-only string attribute that returns the name of this object class. Returns 'design_rule' for design_rule objects.

separated_cut_x_min_spacing_tbl

A settable distance_list attribute that returns 2D values.

separated_cut_y_min_spacing_tbl

A settable distance_list attribute that returns 2D values.

See Also

- [get_attribute](#)
- [list_attributes](#)

dff_connection_object_attributes

Description of the predefined attributes for dff_connection objects.

Description

Attributes are properties assigned to objects such as pins, cells and nets. Definitions for dff_connection attributes are provided in the subsections that follow.

Attributes are read-only unless marked as settable. Read-only attributes are informational, and you cannot set its value. A settable attribute can be modified with the *set_attribute* command.

To determine the value of an attribute, use the *get_attribute* command. To see a list of all attributes available for a class of objects use the *list_attributes -application* command.

dff_connection object Attributes

end

A collection attribute that contains the end pin or port of this connection.

detail

A collection attribute that contains all the intermediate instances and hierarchy pins of the path. Even if 0 registers and 0 gates are present, this attribute may still contain any hierarchy pins that are part of the path. Note: start and end are never included in the detail attribute.

full_name

A string attribute representing the name of the dff_connection object. Format of the string is: <name of start object>::<name of end object>.

hierarchy_pins

A collection attribute that contains any hierarchy pins traversed between the start and end pins. The start and end pins are NOT included in this collection, even if they are hierarchical. If no hierarchical pins are traversed, this attribute is undefined.

gates

An integer value containing the number of gates (combinational logic) traversed by this path.

name

Same as full_name.

object_class

A string attribute with the value of "dff_connection".

registers

An integer value containing the number of registers (sequential logic) traversed by this path.

start

A collection attribute that contains the start pin or port of this connection.

See Also

- [compute_dff_connections](#)
- [get_dff_connections](#)

drc_error_attributes

Description of the application defined drc_error attributes.

Description

Attributes are properties assigned to objects such as pins, cells, and nets. They can be read-only or settable. A read-only attribute is informational; you cannot set its value. To modify a settable attribute, use the *set_attribute* command.

To determine the value of an attribute, use the *get_attribute* command. To see a list of all drc_error attributes available, use the *list_attributes -application -class drc_error* command.

Attributes for drc_error

This section describes the drc_error attributes.

actual_spacing

A read-only errdm_distance attribute that returns the actual spacing, which violated the required spacing in the drc_error.

bbox

A read-only `errdm_rect` attribute that returns the bounding box about all shapes describing this `drc_error`.

brief_info

A read-only string attribute that returns a brief information about the `drc_error`.

direction

A read-only string attribute (with allowed values: `horizontal`, `lower_left_to_upper_right`, `unknown`, `upper_left_to_lower_right`, `vertical`) that returns the direction of the violation, for example in a spacing violation.

error_class

A read-only string attribute (with allowed values: `basic`, `open_locator`, `short`, `size`, `spacing`) that specifies the category of errors in which the `drc_error` belongs, example `short` or `spacing`. The `basic` category includes all errors that do not belong in others.

error_data

A read-only collection attribute that returns the name of the `drc_error_data` in which the `drc_error` was created.

error_id

A read-only int attribute that returns the identifier for the `drc_error`.

error_info

A settable string attribute that returns additional information specific to the `drc_error`.

error_type

A read-only collection attribute that returns the name of the `drc_error_type` in which the `drc_error` belongs.

full_name

A read-only string attribute that returns the full name of a `drc_error`. This is equivalent to the `name`.

layers

A read-only collection attribute that returns the layer names on which the `drc_error` was detected.

must_fix

A settable boolean attribute that indicates whether this drc_error was marked as a must_fix error.

name

A read-only string attribute that returns the name of a drc_error.

object_class

A read-only string attribute that returns the name of this object class. Returns 'drc_error' for drc_error objects.

objects

A read-only collection attribute that returns the design objects associated with the drc_error.

pin_edge

A read-only int attribute that returns the pin edge involved in the violation.

points

A read-only errdm_coord_list_list attribute that specifies a list of points, which describe the drc_error.

required_height

A read-only errdm_distance attribute that returns the required height of an object, which is violated in the drc_error.

required_spacing

A read-only errdm_distance attribute that returns the required spacing, which is violated in the drc_error.

required_width

A read-only errdm_distance attribute that returns the required width of an object, which is violated in the drc_error.

shape

A read-only string attribute that specifies a list of polyrects or polygons, which describe the drc_error.

shape_uses

A read-only route_type_list attribute that specifies a list of shape_uses associated with the layers on which the drc_error was detected.

status

A settable string attribute (with allowed values: error fixed ignored waived) that returns the current status of the `drc_error`.

verbose_info

A read-only string attribute that provides verbose information about the `drc_error`.

See Also

- [get_attribute](#)
- [list_attributes](#)
- [set_attribute](#)

drc_error_data_attributes

Description of the application defined `drc_error_data` attributes.

Description

Attributes are properties assigned to objects such as pins, cells, and nets. They can be read-only or settable. A read-only attribute is informational; you cannot set its value. To modify a settable attribute, use the *set_attribute* command.

To determine the value of an attribute, use the *get_attribute* command. To see a list of all `drc_error_data` attributes available, use the *list_attributes -application -class drc_error_data* command.

Note that you must open an external DRC error data file before you can query its attributes. You can query the attributes of an attached DRC error data file regardless of whether it is open or closed.

Attributes for drc_error_data

This section describes the `drc_error_data` attributes.

checker_name

A read-only string attribute that returns the name of the checker that produced the DRC error data file. This attribute is not set when the error data file was created by the tool.

checker_version

A read-only string attribute that returns the version of the checker that produced the DRC error data file. This attribute is not set when the error data file was created by the tool.

creation_date

A read-only string attribute that returns the date on which the DRC error data file was created.

file_name

A read-only string attribute that returns the name of the DRC error data file. For an attached error data file, this is the name of the error data file. For an external error data file, this is the path to the file.

full_name

A read-only string attribute that returns the full name of the DRC error data file. This is equivalent to the *name* attribute. For external DRC error data files, it includes only the file name, and not its path.

information

A read-only string attribute that provides additional information about the DRC error data file, as specified by the *-information* option of the *create_drc_error_data* command. This attribute is not set when the error data file was created by the tool.

modification_date

A read-only string attribute that returns the date on which the DRC error data file was last modified.

modified

A read-only boolean attribute that indicates whether the DRC error data file contains uncommitted changes.

name

A read-only string attribute that returns the name of the DRC error data file. For external DRC error data files, it includes only the file name, and not its path.

num_errors

A read-only int attribute that returns the number of reported *drc_error* instances in the DRC error data file.

num_types

A read-only int attribute that returns the number of *drc_error_type* instances in the DRC error data file.

object_class

A read-only string attribute that returns the name of this object class. Returns 'drc_error_data' for *drc_error_data* objects.

open_mode

A read-only string attribute that returns the mode in which the DRC error data file was opened, *read_only* or *read_write*. If the DRC error data file is not open, the attribute value is *closed*.

schema_version

A read-only string attribute that returns the schema version with which the DRC error data file was created.

See Also

- [get_drc_error_data](#)
- [get_attribute](#)
- [list_attributes](#)
- [open_drc_error_data](#)

drc_error_type_attributes

Description of the application defined drc_error_type attributes.

Description

Attributes are properties assigned to objects such as pins, cells, and nets. They can be read-only or settable. A read-only attribute is informational; you cannot set its value. To modify a settable attribute, use the *set_attribute* command.

To determine the value of an attribute, use the *get_attribute* command. To see a list of all drc_error_type attributes available, use the *list_attributes -application -class drc_error_type* command.

Attributes for drc_error_type

This section describes the drc_error_type attributes.

bbox

A read-only errdm_rect attribute that returns the bounding box about all drc_error instance shapes and points.

brief_format

A read-only string attribute (with allowed values: ascii message) that specifies a string attribute used internally to interpret the brief information string.

brief_info

A read-only string attribute that provides a brief information about the error type.

error_class

A read-only string attribute (with allowed values: basic open_locator short size spacing) that specifies the category of errors in which the `drc_error_type` belongs. The basic category includes all error types that do not belong in others.

error_data

A read-only collection attribute that returns the name of the `drc_error_data` in which the `drc_error_type` is created.

full_name

A read-only string attribute that returns the full name of a `drc_error_type`. This is equivalent to the name.

name

A read-only string attribute that returns the name of a `drc_error_type`.

num_detected_errors

A read-only int attribute that returns the number of detected `drc_error` instances in this `drc_error_type`. Some checkers allow you to set a limit on the number of reported error instances per type. The checker might detect more and record the actual number detected in this attribute.

num_errors

A read-only int attribute that returns the number of reported `drc_error` instances in this `drc_error_type`.

object_class

A read-only string attribute that returns the name of this object class. Returns 'drc_error_type' for `drc_error_type` objects.

severity

A read-only string attribute (with allowed values: error information recommendation unknown warning) that returns the severity level of the error type.

verbose_format

A read-only string attribute (with allowed values: ascii message) that specifies a string attribute used internally to interpret the verbose information string.

e

verbose_info

A read-only string attribute that returns verbose information about the error type.

See Also

- [get_attribute](#)
- [list_attributes](#)
- [set_attribute](#)

e

early_data_check_record_attributes

Lists the predefined attributes for early data check records.

Description

Attributes are properties assigned to objects such as pins, cells and nets. Definitions for early data check record attributes are provided in the subsections that follow.

Attributes are read-only unless marked as settable. Read-only attributes are informational, and you cannot set its value. A settable attribute can be modified with the *set_attribute* command.

To determine the value of an attribute, use the *get_attribute* command. To see a list of all attributes available for a class of objects use the *list_attributes -application* command.

Attributes for early_data_check_record

This section describes the *early_data_check_record* attributes.

check_name

A read-only string attribute that returns the name of early data check.

full_name

A read-only string attribute that returns the full name of a *early_data_check_record*.

name

A read-only string attribute that returns the name of a *early_data_check_record*.

object_class

A read-only string attribute with the value of "early_data_check_record".

policy

A read-only string attribute that returns the name of policy applied for failing check.

strategy

A read-only string attribute that returns the name of strategy applied for failing check. This will be empty if no strategy defined for applied policy.

See Also

- [get_early_data_check_records](#)
- [get_attribute](#)
- [list_attributes](#)
- [set_attribute](#)

edit_group_attributes

Description of the application defined edit_group attributes.

Description

Attributes are properties assigned to objects such as pins, cells, and nets. They can be read-only or settable. A read-only attribute is informational; you cannot set its value. To modify a settable attribute, use the *set_attribute* command.

To determine the value of an attribute, use the *get_attribute* command. To see a list of all edit_group attributes available, use the *list_attributes -application -class edit_group* command.

Attributes for edit_group

This section describes the edit_group attributes.

bbox

A read-only rect attribute that returns the bounding-box of an edit_group. The format of a rectangle specification is `{{llx lly} {urx ury}}`, which specifies the lower-left and upper-right corners of the rectangle.

bounding_box

A read-only bounding_box collection attribute that returns the bounding-box of an edit_group. The format of the collection specifies lower-left and upper-right corners of the rectangle.

bump_array_bbox

A read-only rect attribute that returns the bounding-box of an `edit_group` used for bump array. The format of a rectangle specification is `{{llx lly} {urx ury}}`, which specifies the lower-left and upper-right corners of the rectangle.

bump_array_bounding_box

A read-only collection attribute that returns the bounding-box of an `edit_group` used for bump array. The format of the collection specifies lower-left and upper-right corners of the rectangle.

delta

A read-only coord attribute that returns the distance between members of an `edit_group` used for bump array. The x-coordinate is the distance in x direction and y-coordinate is the distance in y direction.

full_name

A read-only string attribute that returns the name of the `edit_group`.

group_use

A settable string attribute (with allowed values: `bump_array` `io_group` `macro_array` `user`) that returns what the `edit_group` is used for.

hier_objects

A read-only collection attribute that returns the hierarchical objects associated with the `edit_group`. The hierarchical objects will be `edit_group` objects from lower levels of physical hierarchy.

is_flexible

A settable boolean attribute that indicates whether an `edit_group` is actually a macro array whose channels and macro orientations can be changed by automatic macro placement.

is_hierarchical

A read-only boolean attribute that returns "true" if the `edit_group`'s `hier_objects` attribute is non-empty, "false" otherwise.

is_placement_constraint

A settable boolean attribute that indicates if the `edit_group` is used as a placement constraint.

is_shadow

A read-only boolean attribute that indicates whether an `edit_group` is pushed down from above.

is_transformable

A read-only boolean attribute that determines whether the `edit_group` can be transformed based on the current group status and the presence of unsupported member types.

keep_macro_order

A settable boolean attribute that indicates whether the order of macros in a flexible `edit_group` will be kept by automatic macro placement. Only applies if `is_flexible` is true.

lib_cell

A read-only string attribute that returns the design name of the first member of the `edit_group`.

name

A settable string attribute that returns the name of the `edit_group`.

object_class

A read-only string attribute that returns the name of this object class. Returns 'edit_group' for `edit_group` objects.

objects

A read-only collection attribute that returns the objects associated with the `edit_group`.

orientation

A read-only string attribute that returns the orientation of `edit_group` used for bump array as N, S, W, E, FN, FS, FW or FE.

origin

A read-only coord attribute that returns the origin across all members of `edit_group` used for bump array with respect to top level bounding box origin.

parent_block

A read-only block collection attribute that returns the owner block object for this `edit_group`.

parent_block_cell

A read-only cell collection attribute that returns the parent block cell object for this `edit_group`.

parent_cell

A read-only cell collection attribute that returns the owner cell object for this edit_group.

parent_edit_group

A read-only edit_group collection attribute that returns the owner edit_group in which current edit_group belongs.

pattern

A read-only string attribute that returns whether the edit_group pattern is inline or of staggered types.

physical_status

A settable string attribute (with allowed values: locked unrestricted) that returns the physical modification status of the edit_group. If status is unrestricted, the edit_group may be freely modified. If status is locked, the edit_group is fixed for automatic and manual changes. This is a settable attribute.

repeat

A read-only coord attribute that returns a coordinate where x-coordinate specifies the number of members in the edit_group used for bump array having a unique bounding box upper right x-coordinate and similarly for y-coordinate.

shadow_status

A settable string attribute (with allowed values: copied_down copied_up normal pulled_up pushed_down virtual_flat) that returns the shadow status of an edit_group.

top_block

A read-only block collection attribute that returns the top block object for this edit_group.

ungroup_on_remove

A settable string attribute (with allowed values: first last never second-to-last) that determines when an edit_group should self delete.

See Also

- [create_edit_group](#)
- [get_attribute](#)
- [list_attributes](#)
- [set_attribute](#)

ems_check_attributes

Description of the application defined `ems_check` attributes.

Description

Attributes are properties assigned to objects such as pins, cells, and nets. They can be read-only or settable. A read-only attribute is informational; you cannot set its value. To modify a settable attribute, use the `set_attribute` command.

To determine the value of an attribute, use the `get_attribute` command. To see a list of all `ems_check` attributes available, use the `list_attributes -application -class ems_check` command.

Attributes for `ems_check`

This section describes the `ems_check` attributes.

current_definition

A read-only string attribute that returns the current check, or a group of checks, performed by using the `ems_check`.

default_definition

A read-only string attribute that returns the default setting of checks performed by using the `ems_check`. The user-defined checks do not have default definitions.

enabled

A settable boolean attribute that returns the current enable status of the check.

full_name

A read-only string attribute that returns the full name of the `ems_check`. This attribute is required for the `get_object_name` command.

group_members_count

A read-only string attribute that returns a string (number) that describes the number of members in the group. If the design check is defined as a check, this number is 0. If the design check is defined as a group, this number is higher than 0.

group_name

A read-only string attribute that describes the group name. If the design check is defined as a check, this attribute is empty. If the design check is defined as a group, this attribute gives the group name.

help_info

A read-only string attribute that describes what the check is doing.

is_default

A read-only boolean attribute that returns the current status of the command. If the default settings of the design check remains, it returns true. If the default settings are overridden by the user-defined settings, it returns false.

is_group_member

A read-only boolean attribute that indicates whether a check is a member of a group of checks. If the design check is not a member in a group of checks, it returns false. If the design check is a member in a group of checks, it returns true.

is_group_name

A read-only boolean attribute that indicates whether a design check is defined as a group. If the design check is defined as a check, it returns false. If the design check is defined as a group, it returns true.

name

A read-only string attribute that returns the name of the `ems_check`.

object_class

A read-only string attribute that returns the name of this object class. Returns 'ems_check' for `ems_check` objects.

See Also

- [get_design_checks](#)
- [get_attribute](#)
- [list_attributes](#)
- [get_object_name](#)
- [set_attribute](#)

ems_database_attributes

Description of the application defined `ems_database` attributes.

Description

Attributes are properties assigned to objects such as pins, cells, and nets. They can be read-only or settable. A read-only attribute is informational; you cannot set its value. To modify a settable attribute, use the *set_attribute* command.

To determine the value of an attribute, use the *get_attribute* command. To see a list of all *ems_database* attributes available, use the *list_attributes -application -class ems_database* command.

Attributes for *ems_database*

This section describes the *ems_database* attributes.

full_name

A read-only string attribute that returns the full path of the EMS database name.

msg_count

A read-only string attribute that returns the total number of messages the database stores.

name

A read-only string attribute that returns the name of the EMS database.

object_class

A read-only string attribute that returns the name of this object class. Returns 'ems_database' for *ems_database* objects.

See Also

- [get_attribute](#)
- [list_attributes](#)
- [get_object_name](#)
- [set_attribute](#)

ems_group_attributes

Describes the predefined attributes for *ems_group*.

Description

Attributes are properties assigned to objects such as *ems* rule, message, group and database. Definitions for *ems_group* attributes are provided in the following subsections.

e

Attributes are read-only unless marked as settable. Read-only attributes are informational, and you cannot set its value. A settable attribute can be modified with the *set_attribute* command.

To determine the value of an attribute, use the *get_attribute* command. To see a list of all attributes available for a class of objects, use the *list_attributes -application -class ems_group* command.

ems_group Attributes:*full_name*

Specifies a string attribute to represent full name of the group.

grouping

Specifies a string attribute to identify the parameters to have grouping function. "grouping" is an EMS display feature. You can identify the parameter that messages be arranged and displayed by it.

messages

Specifies a collection attribute to represent sequences of the messages under the group.

name

Specifies a string attribute to represent the name of the group.

object_class

Specifies a string attribute with the value of *ems_group*.

parameters

Specifies a string attribute to represent the parameters defined on the group. These parameters are the common parameters among those rules which belong to this group.

rules

Specifies a collection attribute to represent the rules belong to the group.

visible

Specifies a string attribute to identify the parameters to have "visible" function. "visible" is an EMS display feature. You can identify parameters that their value be displayed in separate columns and be able to sort if needed.

See Also

- [get_attribute](#)
- [list_attributes](#)
- [get_object_name](#)
- [set_attribute](#)

ems_message_attributes

Description of the application defined `ems_message` attributes.

Description

Attributes are properties assigned to objects such as pins, cells, and nets. They can be read-only or settable. A read-only attribute is informational; you cannot set its value. To modify a settable attribute, use the `set_attribute` command.

To determine the value of an attribute, use the `get_attribute` command. To see a list of all `ems_message` attributes available, use the `list_attributes -application -class ems_message` command.

Attributes for `ems_message`

This section describes the `ems_message` attributes.

object_class

A read-only string attribute that returns the name of this object class. Returns 'ems_message' for `ems_message` objects.

rule_name

A read-only string attribute that returns the rule name the message generated.

See Also

- [get_attribute](#)
- [list_attributes](#)
- [get_object_name](#)
- [set_attribute](#)

ems_rule_attributes

Description of the application defined `ems_rule` attributes.

Description

Attributes are properties assigned to objects such as pins, cells, and nets. They can be read-only or settable. A read-only attribute is informational; you cannot set its value. To modify a settable attribute, use the *set_attribute* command.

To determine the value of an attribute, use the *get_attribute* command. To see a list of all *ems_rule* attributes available, use the *list_attributes -application -class ems_rule* command.

Attributes for *ems_rule*

This section describes the *ems_rule* attributes.

full_name

A read-only string attribute that returns the full name of a rule.

is_enabled

A settable boolean attribute that returns the rule status. If "False" - "disabled", messages of the rule won't be created.

name

A read-only string attribute that returns the name of the rule.

object_class

A read-only string attribute that returns the name of this object class. Returns 'ems_rule' for *ems_rule* objects.

type

A read-only string attribute that returns the rule type. The type can be information, warning or error.

See Also

- [get_attribute](#)
- [list_attributes](#)
- [get_object_name](#)
- [set_attribute](#)

exception_attributes

Description of the application defined exception attributes.

Description

Attributes are properties assigned to objects such as pins, cells, and nets. They can be read-only or settable. A read-only attribute is informational; you cannot set its value. To modify a settable attribute, use the *set_attribute* command.

To determine the value of an attribute, use the *get_attribute* command. To see a list of all exception attributes available, use the *list_attributes -application -class exception* command.

Attributes for exception

This section describes the exception attributes.

command_text

A read-only string attribute that returns the equivalent commands that can re-create the object.

exception_type

A read-only string attribute that returns the type of exception. Valid values are *false_path*, *multicycle_path*, and *max_min_delay*.

file_line_info

A read-only string attribute that returns the Tcl or SDC source file name and line number pairs of the commands, which were used to create this object.

full_name

A read-only string attribute that returns a unique name for the exception.

group_count

A read-only int attribute that returns the number of groups in the exception.

has_from

A read-only boolean attribute that returns True if the exception has a -from group.

has_through

A read-only boolean attribute that returns True if the exception has a -through group.

has_to

A read-only boolean attribute that returns True if the exception has a -to group.

hold_fall_end

A read-only boolean attribute that returns True if the exception is multicycle and the hold fall multiplier is relative to the end clock.

hold_fall_multiplier

A read-only int attribute that returns the multiplier value for hold fall in Multicycle_path exceptions.

hold_rise_end

A read-only boolean attribute that returns True if the exception is multicycle and the hold rise multiplier is relative to the end clock.

hold_rise_multiplier

A read-only int attribute that returns the multiplier value for hold rise in Multicycle_path exceptions.

is_hold_fall_false

A read-only boolean attribute that returns True if the exception is false_path, and the hold fall timing is set to be false.

is_hold_rise_false

A read-only boolean attribute that returns True if the exception is false_path, and the hold rise timing is set to be false.

is_setup_fall_false

A read-only boolean attribute that returns True if the exception is false_path, and the setup fall timing is set to be false.

is_setup_rise_false

A read-only boolean attribute that returns True if the exception is false_path, and the setup rise timing is set to be false.

max_fall_delay

A read-only float attribute that returns the max_min_delay exceptions. Specifies the delay value for max_fall.

max_rise_delay

A read-only float attribute that returns the max_min_delay exceptions. Specifies the delay value for max_rise.

min_fall_delay

A read-only float attribute that returns the max_min_delay exceptions. Specifies the delay value for min_fall.

min_rise_delay

A read-only float attribute that returns the max_min_delay exceptions. Specifies the delay value for min_rise.

object_class

A read-only string attribute that returns the name of this object class. Returns 'exception' for exception objects.

setup_fall_multiplier

A read-only int attribute that returns the multicycle_path exceptions, specifying the multiplier value for setup fall.

setup_fall_start

A read-only boolean attribute that returns True if the exception is multicycle and the setup fall multiplier is relative to the start clock.

setup_rise_multiplier

A read-only int attribute that returns the multiplier value for setup rise in multicycle_path exceptions.

setup_rise_start

A read-only boolean attribute that returns True if the exception is multicycle and the setup rise multiplier is relative to the start clock.

through_group_count

A read-only int attribute that returns the number of -through, -rise_through, and -fall_through options specified with the exception command. For example, if the exception is specified as "-through {a1 a2 a3} -rise_through {b1 b2 b3}", the *through_group_count* attribute value is 2.

See Also

- [get_exceptions](#)
- [get_attribute](#)
- [list_attributes](#)
- [report_exceptions](#)
- [set_false_path](#)
- [set_max_delay](#)

- [set_min_delay](#)
- [set_multicycle_path](#)

exception_group_attributes

Description of the application defined exception_group attributes.

Description

Attributes are properties assigned to objects such as pins, cells, and nets. They can be read-only or settable. A read-only attribute is informational; you cannot set its value. To modify a settable attribute, use the *set_attribute* command.

To determine the value of an attribute, use the *get_attribute* command. To see a list of all exception_group attributes available, use the *list_attributes -application -class exception_group* command.

Attributes for exception_group

This section describes the exception_group attributes.

edge

A read-only string attribute that gives the edge type of the from or through or to group. It can be either of rise, fall, or both.

exception_object

A read-only collection attribute that returns exactly one exception. This exception is the one which contains this exception_object.

full_name

A read-only string attribute that returns the type of from or through or to group. It can return one of "from", "rise_from", "fall_from", ... "rise_to" or "fall_to".

has_clock

A read-only boolean attribute that returns True if the from or through or to group contains a clock.

object_class

A read-only string attribute that returns the name of this object class. Returns 'exception_group' for exception_group objects.

objects

A read-only collection attribute that returns heterogeneous objects of the type pins, ports, and clocks. This returns all the objects, which are in this exception_group object.

f

type

A read-only string attribute that returns the type of group. It can return either of the values from, to or through.

See Also

- [get_exceptions](#)
- [get_exception_groups](#)
- [get_attribute](#)
- [list_attributes](#)

f

failsafe_fsm_group_attributes

Lists the predefined attributes for failsafe fsm groups.

Description

Attributes are properties assigned to objects such as pins, cells and nets. Definitions for failsafe fsm group attributes are provided in the subsections that follow.

Attributes are read-only unless marked as settable. Read-only attributes are informational, and you cannot set its value. A settable attribute can be modified with the *set_attribute* command.

To determine the value of an attribute, use the *get_attribute* command. To see a list of all attributes available for a class of objects use the *list_attributes -application* command.

Failsafe Fsm Group Attributes*correction_signal*

A read-only pin or port collection attribute to which all the correction outputs are connected to.

decoder

A read-only cell collection attribute of the hierarchical or leaf-level cell which is the decoder logic for this failsafe fsm group.

encoder

A read-only cell collection attribute of the hierarchical or leaf-level cell which is the encoder logic for this failsafe fsm group.

f

error_signal

A read-only pin or port collection attribute of the design to which the error signal generated from the failsafe finite state machine logic is tied.

full_name

A read-only string attribute that contains the full name of the failsafe fsm group.

name

A settable string attribute that contains the name of the failsafe fsm group.

object_class

A read-only string attribute with the value of "failsafe_fsm_group".

parity_registers

A read-only pin collection attribute of the output pins of registers, which are parity registers for the failsafe fsm group.

registers

A read-only pin collection attribute of the output pins of registers, which are part of the failsafe FSM group.

requirement_id

A read-only string attribute that contains the tracking id of the failsafe fsm group in requirement management system.

rule

A read-only failsafe_fsm_rule collection attribute based on which this failsafe fsm group has been created.

split_pins

A settable pin collection attribute, which should be split to be a part of different branches of high-fanout tree.

synchronizer_registers

A read-only pin collection attribute of output pins of registers, which are the synchronizer registers for the failsafe fsm group.

taps

A settable cell collection attribute of tap cells for this failsafe fsm group.

See Also

- [get_failsafe_fsm_groups](#)
- [get_attribute](#)
- [list_attributes](#)
- [set_attribute](#)

failsafe_fsm_rule_attributes

Lists the predefined attributes for failsafe fsm rules.

Description

Attributes are properties assigned to objects such as pins, cells and nets. Definitions for failsafe fsm rule attributes are provided in the subsections that follow.

Attributes are read-only unless marked as settable. Read-only attributes are informational, and you cannot set its value. A settable attribute can be modified with the *set_attribute* command.

To determine the value of an attribute, use the *get_attribute* command. To see a list of all attributes available for a class of objects use the *list_attributes -application* command.

Failsafe Fsm Rule Attributes

distance_type

A read-only string attribute (with allowed values: xy radial) that indicates whether the separation distance was specified in the form of X & Y distances, or radial distance.

encoding_style

A read-only string attribute (with allowed values: hamming2 hamming3) that contains the encoding style of the failsafe finite state machine.

error_detection_synchronizer

A read-only boolean attribute for the error detection synchronizer for the failsafe fsm rule.

fit_rate_threshold

A read-only float attribute for the fit rate threshold for the failsafe fsm rule.

full_name

A read-only string attribute that contains the full name of the failsafe fsm rule.

f

isolation

A settable boolean attribute for indicating whether the failsafe finite state machine needs isolation using appropriate tap cell insertion.

name

A settable string attribute that contains the name of the failsafe fsm rule.

object_class

A read-only string attribute with the value of "failsafe_fsm_rule".

r_distance

A settable distance attribute for the radial separation distance of the failsafe fsm rule. Valid only if "distance_type" is "radial".

register_mapping

A read-only module collection attribute for the logic mapping modules of the failsafe fsm.

registers

A read-only pin collection attribute for the output pins of the registers on which this failsafe fsm rule is applied.

split_pin_types

A settable string list attribute (with allowed values: clock reset scan) for the pin types to be considered for splitting.

tap_mapping

A settable name list attribute of library cells, out of which the appropriate one is used for instantiating tap cells for the failsafe fsm.

xy_distance

A settable distance pair attribute for the separation distance of the failsafe fsm rule. Valid only if "distance_type" is "xy".

See Also

- [get_failsafe_fsm_rules](#)
- [get_attribute](#)
- [list_attributes](#)
- [set_attribute](#)

fill_cell_attributes

Description of the application defined fill_cell attributes.

Description

Attributes are properties assigned to objects such as pins, cells, and nets. They can be read-only or settable. A read-only attribute is informational; you cannot set its value. To modify a settable attribute, use the *set_attribute* command.

To determine the value of an attribute, use the *get_attribute* command. To see a list of all fill_cell attributes available, use the *list_attributes -application -class fill_cell* command.

Attributes for fill_cell

This section describes the fill_cell attributes.

bbox

A read-only rect attribute that returns the bounding box of the fill_cell.

bounding_box

A read-only bounding_box collection attribute that returns the bounding-box of the fill_cell. The format of the collection specifies lower-left and upper-right corners of the rectangle.

columns

A settable int attribute that returns the number of columns of the fill_cell. It is valid only if *is_arrayed* attribute is true.

full_name

A read-only string attribute that returns the full name of the fill_cell.

is_arrayed

A read-only boolean attribute that indicates if the fill_cell is an array.

name

A read-only string attribute that returns the name of the fill_cell.

object_class

A read-only string attribute that returns the name of this object class. Returns 'fill_cell' for fill_cell objects.

orientation

A settable string attribute (with allowed values: R0 R90 R180 R270 MX MXR90 MY MYR90) that returns the orientation of the fill_cell.

origin

A settable coord attribute that returns the origin of the fill_cell.

ref_name

A read-only string attribute that returns the base name of the block to which this fill_cell refers.

rotation

A settable float attribute that returns the all-angle rotation value of the fill cell in degrees. Allowed to be set only for fill cells instantiated within designs of type pcb, rdl_fanout or package.

rows

A settable int attribute that returns the number of rows of the fill_cell. It is valid only if *is_arrayed* attribute is true.

x_pitch

A settable distance attribute that returns the horizontal pitch for array elements if fill_cell is an array.

x_stagger

A settable distance attribute that returns the horizontal stagger for array elements if fill_cell is an array.

y_pitch

A settable distance attribute that returns the vertical pitch for array elements if fill_cell is an array.

y_stagger

A settable distance attribute that returns the vertical stagger for array elements if fill_cell is an array.

See Also

- [get_attribute](#)
- [list_attributes](#)

geo_mask_attributes

Description of the application defined geo_mask attributes.

Description

Attributes are properties assigned to objects such as pins, cells, and nets. They can be read-only or settable. A read-only attribute is informational; you cannot set its value. To modify a settable attribute, use the *set_attribute* command.

To determine the value of an attribute, use the *get_attribute* command. To see a list of all geo_mask attributes available, use the *list_attributes -application -class geo_mask* command.

Attributes for geo_mask

This section describes the geo_mask attributes.

bbox

A read-only rect attribute that returns the bounding box of the geo_mask.

bounding_box

A read-only bounding_box collection attribute that returns the bounding-box of the geo_mask. The format of the collection specifies lower-left and upper-right corners of the rectangle.

is_empty

A read-only boolean attribute that indicates whether the geo_mask contains any poly-rectangular regions.

object_class

A read-only string attribute that returns the name of this object class. Returns 'geo_mask' for geo_mask objects.

poly_rects

A read-only poly_rect collection attribute that contains the poly-rectangular regions in the geo_mask.

shape_count

A read-only int attribute that returns the number of poly-rectangular regions contained in the geo_mask.

See Also

- [create_geo_mask](#)
- [get_attribute](#)
- [list_attributes](#)
- [set_attribute](#)

grid_attributes

Description of the application defined grid attributes.

Description

Attributes are properties assigned to objects such as pins, cells, and nets. They can be read-only or settable. A read-only attribute is informational; you cannot set its value. To modify a settable attribute, use the *set_attribute* command.

To determine the value of an attribute, use the *get_attribute* command. To see a list of all grid attributes available, use the *list_attributes -application -class grid* command.

Attributes for grid

This section describes the grid attributes.

allowed_orientations

A settable string attribute that returns the list of orientations that cells associated with the grid are allowed to be placed. This attribute is only available for grid of type *block*, and is only settable for manual defined block grid.

full_name

A read-only string attribute that returns the full name of a grid.

groups

A read-only group collection attribute that contains the groups in which the grid is a member.

is_auto_derived

A read-only boolean attribute that indicates if the grid is a block grid that is derived from site rows and/or PG strategies.

name

A read-only string attribute that returns the name of a grid.

g

object_class

A read-only string attribute that returns the name of this object class. Returns 'grid' for grid objects.

type

A read-only string attribute that returns the type of the grid.

x_offset

A settable distance attribute that returns the grid point's offset to the block's origin on the X-axis. This attribute is not settable for auto derived block grid.

x_step

A settable distance attribute that returns the grid's pitch along the X-axis. This attribute is not settable for auto derived block grid.

y_offset

A settable distance attribute that returns the grid point's offset to the block's origin on the Y-axis. This attribute is not settable for auto derived block grid.

y_step

A settable distance attribute that returns the grid's pitch along the Y-axis. This attribute is not settable for auto derived block grid.

See Also

- [create_grid](#)
- [set_grid](#)
- [get_attribute](#)
- [list_attributes](#)
- [set_attribute](#)

group_attributes

Description of the application defined group attributes.

Description

Attributes are properties assigned to objects such as pins, cells, and nets. They can be read-only or settable. A read-only attribute is informational; you cannot set its value. To modify a settable attribute, use the *set_attribute* command.

To determine the value of an attribute, use the *get_attribute* command. To see a list of all group attributes available, use the *list_attributes -application -class group* command.

Attributes for group

This section describes the group attributes.

allow_duplicate_name

A settable boolean attribute that returns whether more than one group having this group's name can exist in the design.

design_name

A read-only string attribute that returns the name of the design.

full_name

A read-only string attribute that returns the name of the group.

groups

A read-only group collection attribute that returns the groups in which this group is a member.

is_hdl_group

A read-only boolean attribute that returns whether the group is an hdl group.

is_repelling_bound_group

A read-only boolean attribute that indicates if the group is a repelling bound group.

is_shaping

A read-only boolean attribute that indicates if this group is a shaping group. Shaping groups contain shapeable objects and provide grouping information for the Design Planning block shaper. Shaping groups are always sets, and they allow a user specified order, editable through the *set_shaping_group_order* command. The list of shapeable objects is voltage areas, move bounds, cells, and shaping groups.

member_cells

A read-only cell collection attribute that returns the cells of the hdl group.

name

A settable string attribute that returns the name of the group.

object_class

A read-only string attribute that returns the name of this object class. Returns 'group' for group objects.

objects

A read-only collection attribute that returns the objects in the non-hdl group.

remove_when

A settable string attribute (with allowed values: any_member_removed empty no_auto_removal one_member) that returns when a group should self remove.

shaping_constraints

A read-only shaping_constraint collection attribute that returns the set of shaping_constraint objects defined for this group.

statement_type

A settable string attribute (with allowed values: always_statement assign_statement case_statement for_statement function_statement generate_case_statement generate_for_statement generate_if_statement if_else_statement initial_statement no_statement while_statement) that returns the statement type of the hdl_group.

type

A settable string attribute (with allowed values: collection set) that returns the type of the group. This is settable attribute for empty groups and values are restricted to set and collection.

See Also

- [create_group](#)
- [get_attribute](#)
- [list_attributes](#)
- [set_attribute](#)

gui_annotation_attributes

Description of the predefined attributes for gui_annotation.

Description

Objects of type gui_annotation are created by the *gui_add_annotation* command. See the manpage for that command for more details.

To determine the value of an attribute use the *get_attribute* command.

GUI Annotation Attributes

client_data

A string attribute that can be used by client code to store arbitrary information

color

The color used to draw the annotation.

fill_pattern

The fill pattern used to fill the annotationshape

group

The group of the annotation.

info_tip

The tip text (or command) displayed when a user hovers over the annotation in the layout.

line_style

The line style to use when drawing the annotation

line_width

The width of the line in pixels.

object_class

The string attribute with the value "gui_annotation"

points

The points for the annotation

query_command

The command to use when the annotation is queried in the layout.

query_text

The text to display when the annotation is queried in the layout.

shape_type

The type of shape for this annotation. For example, rect, line, etc.

text

If the shape_type is text this attribute has the text to display

window

When set this annotation will only be drawn on the specified window.

See Also

- [gui_get_annotations](#)
- [gui_remove_all_annotations](#)

gui_object_attributes

Description of the predefined attributes for `gui_object`.

Description

Objects of type `gui_object` are created by the `gui_create_vm_objects` command. See the manpage for that command for more details.

All attributes are read-only. To determine the value of an attribute use the `get_attribute` command.

GUI Object Attributes

name

A string attribute for the name of the object

full_name

The same as `name`

object_class

A string attribute with the value of "gui_object"

layout_info_tip

The info-tip to use in the layout when the user hovers over this object.

query_text

The query text to display if the user queries this object in the layout.

core_object

A collection containing the actual object wrapped by this `gui_object`.

See Also

- [gui_create_vm_objects](#)
- [gui_create_vm](#)
- [gui_update_vm_annotations](#)

h

hier_tech_layer_attributes

Description of the predefined attributes for hier tech layer.

Description

Attributes are properties assigned to objects such as pins, cells and nets. Definitions for hier tech layer attributes are provided in the subsections that follow.

Attributes are read-only unless marked as settable. Read-only attributes are informational and you cannot set their values. A settable attribute can be modified with the *set_attribute* command.

To determine the value of an attribute, use the *get_attribute* command. To see a list of all the attributes of a class of objects, use the *list_attributes -application\P* command.

Hier Tech Layer Attributes

adjacent_cut_range

A read-only distance attribute. The value is adjacent cut range of hier tech layer.

blink

A settable integer attribute that returns signifying blink for the hier tech layer.

capacitance_multiplier

A read-only float attribute. The value is capacitance multiplier of hier tech layer.

check_manhattan_spacing

A read-only boolean attribute. The value is true if manhattan spacing is set on the hier tech layer.

color

A settable string attribute the returns signifying color.

default_width

A settable distance attribute. The value is default width of hier tech layer.

end_of_line1_neighbor_corner_keepout_width

A read-only distance attribute. The value is end of line1 neighbor corner keepout width of hier tech layer.

end_of_line1_neighbor_min_length

A read-only distance attribute. The value is end of line1 neighbor min length of hier tech layer.

end_of_line1_neighbor_min_spacing

A read-only distance attribute. The value is end of line1 neighbor min spacing of hier tech layer.

end_of_line1_neighbor_threshold

A read-only distance attribute. The value is end of line1 neighbor threshold of hier tech layer.

end_of_line1_neighbor_wire_min_threshold

A read-only distance attribute. The value is end of line1 neighbor wire min threshold of hier tech layer.

end_of_line_corner_keepout_width

A read-only distance attribute. The value is end of line corner keepout width of hier tech layer.

end_of_line_side_keepout_length

A read-only distance attribute. The value is end of line side keepout length of hier tech layer.

fat_table_dimension

A read-only integer attribute. The value is fat_table_dimension of hier tech layer.

fat_table_parallel_length

A read-only distance list attribute. The value is fat_table_parallel_length of hier tech layer.

fat_table_parallel_length_dimension

A read-only integer attribute. The value is fat_table_parallel_length_dimension of hier tech layer.

fat_table_spacing

A read-only distance list attribute. The value is fat_table_spacing of hier tech layer.

fat_table_threshold

A read-only distance list attribute. The value is fat_table_spacing of hier tech layer.

fixed_mask

A settable string attribute (with allowed values: mask_one mask_three mask_two no_mask) that returns fixed mask value as a string value for the hier tech layer.

full_name

A read-only string attribute representing the full name of the hier tech layer.

is_default_layer

A read-only boolean attribute. The value is true if current metal layer is the default hier tech layer.

is_destructible

A read-only boolean attribute that indicates a hier tech layer is for test-only, and will be removed after all testing are completed.

is_marker_layer

A read-only boolean attribute. The value is true if current metal layer is the marker layer.

is_routing_layer

A read-only boolean attribute. The value is true if current metal layer is a routing layer.

layer_number

A read-only string attribute. The value is layer number of hier tech layer.

layer_source

A settable string attribute that returns the source of the hier tech layer whether layer is from tech file or gdsII or other sources.

layer_type

A read-only string attribute (with allowed values: INTERCONNECT LOCAL_INTERCONNECT VIA_CUT LOCAL_VIA_CUT DIFFUSION NWELL

PWELL N_IMPLANT P_IMPLANT IMPLANT MIM_INTERCONNECT
MIM_VIA_CUT SADP_TRIM TRIM). The value is the type of the hier tech layer.

linestyle

A settable string attribute that returns the line style of the hier tech layer.

mask_name

A read-only string attribute representing the mask name.

mask_order

A settable integer attribute that returns the order number of the mask in the set of all layers.

mask_order_in_type

A read-only integer attribute that returns the order number of the mask in the layers of the same type.

max_num_adjacent_cut

A read-only integer attribute. The value is max number of adjacent cuts of hier tech layer.

min_area

A read-only area attribute. The value is min area of hier tech layer.

min_length

A read-only distance attribute. The value is min length of hier tech layer.

min_spacing

A read-only distance attribute. The value is min spacing of hier tech layer.

min_width

A read-only distance attribute. The value is min width of hier tech layer.

name

A read-only string attribute representing the name.

number_of_masks

A settable integer attribute indicating the number of multi-patterning masks supported on the hier tech layer.

object_class

A read-only string attribute with the value of "hier_tech_layer".

pattern

A settable string attribute which is the fill pattern for the hier tech layer.

pitch

A read-only distance attribute. The value is pitch of hier tech layer.

purpose_name

A read-only string attribute. The value is the purpose name of the hier tech layer.

purpose_number

A read-only integer attribute. The value is the purpose number of the hier tech layer.

purposes

A read-only layer collection attribute of all tech purposes associated with the hier tech layer. Returns NULL if there are none.

routing_direction

A settable string attribute. The value is the routing direction of the hier tech layer.

same_net_min_spacing

A settable distance attribute that returns the minimum width for shapes of the same net on a layer.

selectable

A settable integer attribute that returns true if the layer is selectable, false otherwise.

special_min_area

A read-only distance attribute. The value is special min area of hier tech layer.

stub_mode

A settable string attribute (with allowed values: PARALLEL ONE_EDGE TWO_EDGE THREE_EDGE TWO_EDGE_WITH_EXCLUSION INVALID) that returns the stub mode for layer.

stub_spacing

A read-only distance attribute. The value is stub spacing of hier tech layer.

stub_threshold

A read-only distance attribute. The value is stub threshold of hier tech layer.

i

tech_layer

A read-only layer collection attribute of tech layer of the hier tech layer.

track_offset

A settable distance attribute. The value is the track offset value of the hier tech layer.

visible

A settable integer attribute that returns 1 if the layer is visible, 0 otherwise.

x_default_width

A settable distance attribute that returns the horizontal default width.

x_legal_dim_tbl

A read-only distance list attribute that returns the horizontal legal dimensions.

y_default_width

A settable distance attribute that returns the vertical default width.

y_legal_dim_tbl

A read-only distance list attribute that returns the vertical legal dimensions.

See Also

- [get_layers](#)
- [get_attribute](#)
- [list_attributes](#)
- [set_attribute](#)

i

input_delay_attributes

Description of the application defined input_delay attributes.

Description

Attributes are properties assigned to objects such as pins, cells, and nets. They can be read-only or settable. A read-only attribute is informational; you cannot set its value. To modify a settable attribute, use the *set_attribute* command.

To determine the value of an attribute, use the *get_attribute* command. To see a list of all *input_delay* attributes available, use the *list_attributes -application -class input_delay* command.

Attributes for *input_delay*

This section describes the *input_delay* attributes.

clock_name

A read-only string attribute that returns the name of the relative clock.

file_line_info

A read-only string attribute that returns the Tcl or SDC source file name and line number pairs of the commands which were used to create this object, if it exists.

full_name

A read-only string attribute that returns a unique name for the input delay object.

is_clock_fall

A read-only boolean attribute that returns true if this *input_delay* is relative to the falling edge of the clock, false otherwise.

is_level_sensitive

A read-only boolean attribute that returns true if this *input_delay* represents a level-sensitive startpoint, false otherwise.

is_network_latency_included

A read-only boolean attribute that returns true if this *input_delay* includes clock network latency, false otherwise.

is_source_latency_included

A read-only boolean attribute that returns true if this *input_delay* includes clock source latency, false otherwise.

max_fall

A read-only float attribute that returns the maximum fall delay value.

max_rise

A read-only float attribute that returns the maximum rise delay value.

min_fall

A read-only float attribute that returns the minimum fall delay value.

i

min_rise

A read-only float attribute that returns the minimum rise delay value.

object_class

A read-only string attribute that returns the name of this object class. Returns 'input_delay' for input_delay objects.

See Also

- [set_input_delay](#)
- [get_attribute](#)
- [list_attributes](#)

io_guide_attributes

Description of the application defined io_guide attributes.

Description

Attributes are properties assigned to objects such as pins, cells, and nets. They can be read-only or settable. A read-only attribute is informational; you cannot set its value. To modify a settable attribute, use the *set_attribute* command.

To determine the value of an attribute, use the *get_attribute* command. To see a list of all io_guide attributes available, use the *list_attributes -application -class io_guide* command.

Attributes for io_guide

This section describes the io_guide attributes.

bbox

A read-only rect attribute that returns the bounding-box of an io_guide. The format of a rectangle specification is `{{lxl lly} {urx ury}}`, which specifies the lower-left and upper-right corners of the rectangle.

bounding_box

A read-only collection attribute that returns the bounding-box of an io_guide. The format of the collection specifies lower-left and upper-right corners of the rectangle.

cells

A read-only collection attribute that returns the instances associated with the IO guide.

i

end_offset

A settable distance attribute that returns the end offset of IO guide.

full_name

A read-only string attribute that returns the name of the io_guide.

in_edit_group

A read-only boolean attribute that indicates if the IO guide belongs to an edit group.

io_ring

A read-only collection attribute that returns the IO ring associated with the IO guide.

is_placed

A read-only boolean attribute that indicates if the io_guide is placed.

is_shadow

A read-only boolean attribute that indicates whether an IO guide is pushed down from above.

length

A read-only distance attribute that returns the length of the IO guide.

line

A settable coord_list attribute that returns the first and last points of the IO guide.

name

A settable string attribute that returns the name of the io_guide.

object_class

A read-only string attribute that returns the name of this object class. Returns 'io_guide' for io_guide objects.

origin

A settable coord attribute that returns the first point of the IO guide.

over_constrained

A read-only boolean attribute that indicates if the IO guide is over constrained.

parent_block

A read-only collection attribute that returns the owner block object for this IO guide.

parent_block_cell

A read-only collection attribute that returns the parent block cell object for this IO guide.

parent_cell

A read-only collection attribute that returns the owner cell object for this IO guide.

physical_status

A settable string attribute (with allowed values: locked unrestricted) that returns the physical modification status of the ioguide. Valid values are unrestricted, application_fixed, fixed, and locked. If status is unrestricted, the ioguide may be freely modified. If the status is application_fixed, the ioguide is fixed by the application for automatic changes (the application may change this status). If status is fixed, the ioguide is fixed by the user for automatic changes. If status is locked, the ioguide is fixed for automatic and manual changes.

power_constraint_cell

A settable collection attribute that returns the list of Power/Ground IO pad cells in the IO Guide.

power_constraint_offset

A settable string attribute that returns the list of the maximal distances permitted between starting edge of IO Guide and closest edge of the first IO power driver cell.

power_constraint_ratio

A settable string attribute that returns the list of the number of signal IO driver cells allowed between consecutive power driver cells of the same power supply or ground.

power_constraint_share

A settable string attribute that returns the list of the number of IO driver cells that can share a single bump.

power_constraint_space

A settable string attribute that returns the list of distances which signifies the maximal distance between two consecutive power driver cells.

shadow_status

A settable string attribute (with allowed values: copied_down copied_up normal pulled_up pushed_down virtual_flat) that returns the shadow status of an io_guide.

i

side

A read-only string attribute (with allowed values: bottom left right top) that returns the access direction of the IO guide.

signal_constraint_fixed

A read-only string attribute that returns the location of each individual IO driver cell which are fixed with respect to the IO Guide.

signal_constraint_insert

A read-only string attribute that returns the array of locations where other IO Pad Cells can be inserted in spaces between other IO Pad cells if allowed.

signal_constraint_location

A read-only string attribute that returns the location of each individual IO driver cell within the IO Guide.

signal_constraint_order

A read-only string attribute that returns the relative order of IO driver cells within the IO Guide.

signal_constraint_other

A read-only string attribute that indicates if other IO Pad Cells can be inserted before the first or the last IO Pad cell.

start_offset

A settable distance attribute that returns the start offset of IO guide.

top_block

A read-only collection attribute that returns the top block object for this IO guide.

See Also

- [create_io_guide](#)
- [get_attribute](#)
- [list_attributes](#)
- [set_attribute](#)

io_ring_attributes

Description of the application defined io_ring attributes.

Description

Attributes are properties assigned to objects such as pins, cells, and nets. They can be read-only or settable. A read-only attribute is informational; you cannot set its value. To modify a settable attribute, use the *set_attribute* command.

To determine the value of an attribute, use the *get_attribute* command. To see a list of all *io_ring* attributes available, use the *list_attributes -application -class io_ring* command.

Attributes for *io_ring*

This section describes the *io_ring* attributes.

bbox

A settable rect attribute that returns the bounding-box of an *io_ring*. The format of a rectangle specification is *{{llx lly} {urx ury}}*, which specifies the lower-left and upper-right corners of the rectangle.

bounding_box

A settable collection attribute that returns the bounding-box of an *io_ring*. The format of the collection specifies lower-left and upper-right corners of the rectangle.

cells

A read-only collection attribute that returns the instances associated with the IO ring.

corner_height

A settable distance attribute that returns the corner height of IO ring.

full_name

A read-only string attribute that returns the name of the *io_ring*.

in_edit_group

A read-only boolean attribute that indicates if the IO ring belongs to an edit group.

io_guides

A read-only collection attribute that returns the IO guides associated with the IO ring.

name

A settable string attribute that returns the name of the *io_ring*.

i

object_class

A read-only string attribute that returns the name of this object class. Returns 'io_ring' for io_ring objects.

offset

A settable distance attribute that returns the offset of IO ring from block boundary or outer ring.

outer_ring

A settable collection attribute that returns the outer ring, if any, relative to the IO ring.

over_constrained

A read-only boolean attribute that returns whether the IO ring is over constrained.

parent_block

A read-only collection attribute that returns the owner block object for this IO Ring.

parent_block_cell

A read-only collection attribute that returns the parent block cell object for this IO Ring.

parent_cell

A read-only collection attribute that returns the owner cell object for this IO Ring.

top_block

A read-only collection attribute that returns the top block object for this IO Ring.

See Also

- [create_io_ring](#)
- [get_attribute](#)
- [list_attributes](#)
- [set_attribute](#)

k

k

keepout_margin_attributes

Description of the application defined keepout_margin attributes.

Description

Attributes are properties assigned to objects such as pins, cells, and nets. They can be read-only or settable. A read-only attribute is informational; you cannot set its value. To modify a settable attribute, use the *set_attribute* command.

To determine the value of an attribute, use the *get_attribute* command. To see a list of all keepout_margin attributes available, use the *list_attributes -application -class keepout_margin* command.

Attributes for keepout_margin

This section describes the keepout_margin attributes.

boundary

A read-only coord_list attribute that returns the boundary of the owner the current keepout margin belongs to. The boundary value has the current keepout margin applied. The format of a coordinate_list specification is {{x1 y1} {x2 y2}...}.

full_name

A read-only string attribute that returns the full name of a keepout margin. The string value has the full name of the owner and the name of the keepout margin.

layers

A settable layer collection attribute that returns the layers for the route_blockage type keepout margin. This attribute is applicable only for the route_blockage type keepout margin.

margin

A settable margin_list attribute that returns the keepout margin values in the following order: left, bottom, right, and top. The format of a margin_list specification is {l b r t}.

max_padding_per_macro

A read-only distance attribute that returns the maximum padding per macro which was used to calculate the keepout margin. See the *create_keepout_margin -max_padding_per_macro* option for additional information.

min_padding_per_macro

A read-only distance attribute that returns the minimum padding per macro which was used to calculate the keepout margin. See the *create_keepout_margin -min_padding_per_macro* option for additional information.

name

A read-only string attribute that returns the name of the keepout margin.

object_class

A read-only string attribute that returns the name of this object class. Returns 'keepout_margin' for keepout_margin objects.

owner

A read-only design, or cell collection attribute that returns the design or the cell object the keepout margin belongs to.

scope

A read-only string attribute that returns one of the following values: inner, outer.

tracks_per_macro_pin

A read-only double attribute that returns the number of tracks per macro pin which was used to calculate the keepout margin. See the *create_keepout_margin -tracks_per_macro_pin* option for additional information.

type

A read-only string attribute (with allowed values: assembly_die, clock, filler_allowed, alignment_marker, hard, hard_macro, routing_blockage, seal_ring, soft) that returns the keepout margin type.

See Also

- [create_keepout_margin](#)
- [get_attribute](#)
- [list_attributes](#)
- [set_attribute](#)

layer_attributes

Description of the application defined layer attributes.

Description

Attributes are properties assigned to objects such as pins, cells, and nets. They can be read-only or settable. A read-only attribute is informational; you cannot set its value. To modify a settable attribute, use the *set_attribute* command.

To determine the value of an attribute, use the *get_attribute* command. To see a list of all layer attributes available, use the *list_attributes -application -class layer* command.

Attributes for layer

This section describes the layer attributes.

adjacent_cut_range

A read-only distance attribute that returns the value of the adjacent cut range of layer.

blink

A settable int attribute that returns an integer value signifying blink for the layer.

capacitance_multiplier

A read-only float attribute that returns the capacitance multiplier of the layer.

check_manhattan_spacing

A read-only boolean attribute that returns true if manhattan spacing is set on the layer.

color

A settable string attribute that returns a string value signifying color.

default_width

A settable distance attribute that returns the default width of the layer.

end_of_line1_neighbor_corner_keepout_width

A read-only distance attribute that returns the end of line1 neighbor corner keepout width of the layer.

|

end_of_line1_neighbor_min_length

A read-only distance attribute that returns the end of line1 neighbor min length of the layer.

end_of_line1_neighbor_min_spacing

A read-only distance attribute that returns the end of line1 neighbor min spacing of the layer.

end_of_line1_neighbor_threshold

A read-only distance attribute that returns the end of line1 neighbor threshold of layer.

end_of_line1_neighbor_wire_min_threshold

A read-only distance attribute that returns the end of line1 neighbor wire min threshold of layer.

end_of_line_corner_keepout_width

A read-only distance attribute that returns the end of line corner keepout width of the layer.

end_of_line_side_keepout_length

A read-only distance attribute that returns the end of line side keepout length of the layer.

fat_table_dimension

A read-only int attribute that returns the fat_table_dimension of layer.

fat_table_parallel_length_dimension

A read-only int attribute that returns the fat_table_parallel_length_dimension of layer.

fixed_mask

A settable string attribute (with allowed values: mask_one mask_three mask_two no_mask) that returns the fixed mask value as an integer value for the layer.

full_name

A read-only string attribute that returns the name.

is_default_layer

A read-only boolean attribute that returns true if the current metal layer is the default layer.

|

is_destructible

A read-only boolean attribute that indicates whether a layer is for test-only, and will be removed after all testing are completed.

is_marker_layer

A read-only boolean attribute that returns true if the current metal layer is the marker layer.

is_routing_layer

A read-only boolean attribute that returns true if the current metal layer is a routing layer.

layer_number

A read-only string attribute that returns the layer number of layer.

is_editable

A read-only boolean attribute that indicates whether a layer is for visible only and shapes on which are not to be edited.

lock_tracks

A settable boolean attribute that enables a warning when tracks on the given layer are created/removed. It indicates to the user that changing this layer's tracks is not advised. For example, the flag may be set after auto-generating a track structure from PRF rules.

layer_source

A settable string attribute that returns a string value for the source of the layer, whether the layer is from a tech file, gdsII, or other sources.

layer_type

A read-only string attribute that returns the type of the layer. Valid types include: INTERCONNECT, LOCAL_INTERCONNECT, VIA_CUT, LOCAL_VIA_CUT, DIFFUSION, N_WELL, P_WELL, N_IMPLANT, P_IMPLANT, IMPLANT, MIM_INTERCONNECT, MIM_VIA_CUT, SADP_TRIM, TRIM

linestyle

A settable string attribute that returns a string value for the line style of the layer.

mask_min_area_mask_num_tbl

This attribute is read-only and returns a list of integer value. Returns the mask numbers for color-based minimum area rule of the layer.

|

mask_min_area_tbl

This attribute is read-only and returns a list of float value. Returns minimum areas for color-based minimum area rule of the layer.

mask_min_area_tbl_size

This attribute is read-only and returns an integer value. Returns the number of color for color-based minimum area rule of the layer.

mask_name

A read-only string attribute that returns the mask name.

mask_order

A settable int attribute that returns an integer value. Returns the order number of the mask in the set of all layers.

mask_order_in_type

A read-only int attribute that returns an integer value. Returns the order number of the mask in the layers of the same type.

max_num_adjacent_cut

A read-only int attribute that returns the max number of adjacent cuts of layer.

min_area

A read-only area attribute that returns the min area of layer.

min_length

A read-only distance attribute that returns the floating point value of the min length of the layer.

min_spacing

A read-only distance attribute that returns the min spacing of layer.

min_width

A read-only distance attribute that returns the min width of layer.

name

A read-only string attribute that returns the name.

number_of_masks

A settable int attribute that returns the number of multi-patterning masks supported on the layer.

|

object_class

A read-only string attribute that returns the name of this object class. Returns 'layer' for layer objects.

pattern

A settable string attribute that returns the fill pattern for the layer.

pitch

A settable distance attribute that returns the pitch of layer. The specified pitch must be greater than or equal to zero.

purpose_name

A read-only string attribute that returns the purpose name of the layer.

purpose_number

A read-only int attribute that returns the purpose number of the layer.

purposes

A read-only layer collection attribute that returns a collection of all tech purposes associated with the layer. Returns NULL if there are none.

routing_direction

A settable string attribute that returns the routing direction of the layer.

same_net_min_spacing

A settable distance attribute that returns a float value. Returns the minimum width for shapes of the same net on a layer.

selectable

A settable int attribute that returns an integer value. Returns true if the layer is selectable, false otherwise.

special_min_area

A read-only distance attribute that returns the special min area of the layer.

stub_mode

A settable string attribute that returns a string value. Returns the stub mode for layer and the value could be one of PARALLEL, ONE_EDGE, TWO_EDGE, THREE_EDGE, TWO_EDGE_WITH_EXCLUSION or INVALID.

stub_spacing

A read-only distance attribute that returns the stub spacing of the layer.

stub_threshold

A read-only distance attribute that returns the stub threshold of the layer.

track_offset

A settable distance attribute that returns the track offset value of the layer.

visible

A settable int attribute that returns an integer value. Returns 1 if the layer is visible, 0 otherwise.

x_default_width

A settable distance attribute that returns the horizontal default width.

y_default_width

A settable distance attribute that returns the vertical default width.

x_legal_width_tbl

A settable list of distance attribute that returns the horizontal legal width table.

x_legal_width_tbl_size

This attribute is read-only and returns an integer value. Returns the size of the horizontal legal width table.

See Also

- [get_layers](#)
- [get_attribute](#)
- [list_attributes](#)
- [set_attribute](#)

lib_attributes

Description of the application defined lib attributes.

Description

Attributes are properties assigned to objects such as pins, cells, and nets. They can be read-only or settable. A read-only attribute is informational; you cannot set its value. To modify a settable attribute, use the *set_attribute* command.

To determine the value of an attribute, use the *get_attribute* command. To see a list of all lib attributes available, use the *list_attributes -application -class lib* command.

|

Attributes for lib

This section describes the lib attributes.

base_lib

A read-only collection attribute that is the base lib of a library which is sparse.

base_lib_path

A settable string attribute that returns the absolute path of the base lib of a library which is sparse.

capacitance_unit

A read-only string attribute that returns the capacitance unit for this library.

ccs_converted

A read-only boolean attribute that returns true if CCS converted to NLDM, false otherwise.

characterization_points

A read-only string attribute that returns the characterization points of this library.

compression_format

A read-only string attribute that returns possible values of gzip, zstd, and uncompressed.

current_unit

A read-only string attribute that returns the current unit for this library.

default_cell_leakage_power

A settable float attribute that returns the default cell leakage power for this library. This is a liberty attribute and specifies the default leakage power for those cells of the library that do not have the `cell_leakage_power` attribute. This attribute must be a non negative floating point number.

default_connection_class

A settable string attribute that returns the default connection class for this library. This is a liberty attribute and specifies a default value for connection class for all pins in the library.

default_max_capacitance

A settable float attribute that returns the default max capacitance for this library. This is a liberty attribute and specifies the default maximum capacitance of output pins of the library.

|

default_max_fanout

A settable float attribute that returns the library default maximum fanout design rule limit.

default_max_transition

A settable float attribute that returns the default max transition for this library. This is a liberty attribute and specifies the default maximum transition of output pins of the library.

extended_name

A read-only string attribute that returns the full path to the library directory of this library plus the library name, separated by a ':' character. For example, "/disks/user1/work/tech_lib:tech_lib".

full_name

A read-only string attribute that returns the name of a library.

half_node_scale_factor

A settable float attribute that returns the half_node_scale_factor for the library. Valid values are greater than or equal to 0.5 and less than 1.0. This attribute only takes effect when the app option "design.is_3dic_mode" is set to true. If the attribute is not set, the value is automatically set from the TLU+ files, if it exists there. The attribute value can be reset using clear_attribute. If the TLU+ file has a valid value, it will be used to set the attribute, next time the library is opened.

input_threshold_pct_fall

A read-only float attribute that returns a value between 0.0 and 100.0. It specifies the threshold point of an input pin signal falling from 1 to 0.

input_threshold_pct_rise

A read-only float attribute that returns a value between 0.0 and 100.0. It specifies the threshold point of an input pin signal rising from 0 to 1.

is_aggregate

A read-only boolean attribute that indicates if this library is an aggregate library.

is_aggregate_member

A read-only boolean attribute that indicates if this library belongs to an aggregate library.

is_compressed

A settable boolean attribute that returns true if this library is saved in compressed format on disk, false otherwise.

|

is_current

A read-only boolean attribute that returns true if this library is the current_lib for the session, false otherwise.

is_hidden

A read-only boolean attribute that returns true if this library is hidden, false otherwise.

is_implicit

A read-only boolean attribute that returns true if this library is implicitly opened as reference library, false otherwise.

is_lib_cell_lib

A read-only boolean attribute that returns true if this library is a lib-cell library created from icc2_lm shell.

is_locked_for_edit

A settable boolean attribute that returns true value if this reference lib is locked to disallow edit (uneditable) whereas false value unlocks the edit-lock of this reference lib to allow edit (editable).

is_mask_shiftable

A settable boolean attribute that enables multi-patterning mask shifting of cells when true. If "false", the mask_shift of any cell which references a lib_cell or block in this library will be ignored.

is_modified

A read-only boolean attribute that returns true if this library has modified and unsaved data, false otherwise.

is_sparse

A read-only boolean attribute that returns true if this library is sparse based upon a base library, false otherwise.

last_saved

A read-only string attribute that returns the date and time this library was last saved.

name

A read-only string attribute that returns the name of a library.

|

object_class

A read-only string attribute that returns the name of this object class. Returns 'lib' for lib objects.

open_count

A read-only int attribute that returns the number of times this design has been opened.

open_mode

A read-only string attribute that returns the open-mode of the library. Valid values are "read", "edit", and "new".

output_threshold_pct_fall

A read-only float attribute that returns a value between 0.0 and 100.0. It specifies the threshold point of an output pin signal falling from 1 to 0.

output_threshold_pct_rise

A read-only float attribute that returns a value between 0.0 and 100.0. It specifies the threshold point of an output pin signal rising from 0 to 1.

read_from_product_name

A read-only string attribute that returns the name of the binary used to create this library.

read_from_product_version

A read-only string attribute that returns the version of the binary used to create this library.

read_from_schema_version

A read-only string attribute that returns the schema version of the library file, in (major_version... minor_version) format.

ref_libs

A read-only string attribute that returns the list of reference libraries.

resistance_unit

A read-only string attribute that returns the resistance unit for this library.

save_mode

A read-only string attribute that returns the save mode of this library. Valid values are "file" and "directory".

|

scale_factor

A read-only int attribute that returns the length precision used in this library.

slew_derate_from_library

A read-only float attribute that specifies how the transition (slew) times in library need to be derated to match the transition times between characterization trip points.

slew_lower_threshold_pct_fall

A read-only float attribute that returns a value between 0.0 and 100.0. It specifies the lower trip point for measuring the transition (slew) time on a pin falling from 1 to 0.

slew_lower_threshold_pct_rise

A read-only float attribute that returns a value between 0.0 and 100.0. It specifies the lower trip point for measuring the transition (slew) time on a pin rising from 0 to 1.

slew_upper_threshold_pct_fall

A read-only float attribute that returns a value between 0.0 and 100.0. It specifies the upper trip point for measuring the transition (slew) time on a pin falling from 1 to 0.

slew_upper_threshold_pct_rise

A read-only float attribute that returns a value between 0.0 and 100.0. It specifies the upper trip point for measuring the transition (slew) time on a pin rising from 0 to 1.

source_file_name

A read-only string attribute that returns the full path to the source file of this library.

switch_list

A read-only string attribute that returns the effective view switch list of this library. The view switch list of a design is the precedence-ordered list of design views that is applied when attempting to bind it. If a design in this library does not have a view switch list explicitly set, then the effective view switch list of this library (this attribute value) is applied to the design.

tech

A read-only collection attribute that returns the technology of this library.

tech_lib

A read-only collection attribute that returns the technology library of this library.

tech_name

A read-only string attribute that returns the name of technology of this library.

temperature_unit

A read-only string attribute that returns the temperature unit for this library.

time_unit

A read-only string attribute that returns the time unit for this library.

use_hier_ref_libs

A settable boolean attribute that indicates whether blocks in this library should always be linked using this library's reference library list. If "false", then the reference library list of the parent block is used.

voltage_unit

A read-only string attribute that returns the voltage unit for this library.

See Also

- [get_attribute](#)
- [list_attributes](#)

lib_cell_attributes

Description of the application defined lib_cell attributes.

Description

Attributes are properties assigned to objects such as pins, cells, and nets. They can be read-only or settable. A read-only attribute is informational; you cannot set its value. To modify a settable attribute, use the *set_attribute* command.

To determine the value of an attribute, use the *get_attribute* command. To see a list of all lib_cell attributes available, use the *list_attributes -application -class lib_cell* command.

Attributes for lib_cell

This section describes the lib_cell attributes.

allowable_orientation

A settable orientations attribute that returns the set of allowable orientations for the lib cell.

|

always_on

A read-only boolean attribute that returns True for *always_on* cells with a secondary PG connection to stay active in a switched power domain.

analysis_area

A read-only double attribute that returns the area of *lib_cell*. Subscript: ([*analysis_name*])

analysis_avg_fall_delay

A read-only double attribute that returns the average fall delay of all the maximum arcs for all slew/load points. Subscript: (pvt[,*analysis_name*])

analysis_avg_fall_drive

A read-only double attribute that returns the average rise drive of all the maximum arcs for all slew/load points. Subscript: (pvt[,*analysis_name*])

analysis_avg_fall_hold

A read-only double attribute that returns the average rise hold of all the maximum arcs for all slew/load points. Subscript: (pvt[,*analysis_name*])

analysis_avg_fall_setup

A read-only double attribute that returns the average rise setup of all the maximum arcs for all slew/load points. Subscript: (pvt[,*analysis_name*])

analysis_avg_internal_power

A read-only double attribute that returns the average internal power of all slew/load points. Subscript: (pvt[,*analysis_name*])

analysis_avg_pin_cap

A read-only double attribute that returns the average pin cap of all pins on the libcell. Subscript: (pvt[,*analysis_name*])

analysis_avg_rise_delay

A read-only double attribute that returns the average rise delay of all the maximum arcs for all slew/load points. Subscript: (pvt[,*analysis_name*])

analysis_avg_rise_drive

A read-only double attribute that returns the average rise drive of all the maximum arcs for all slew/load points. Subscript: (pvt[,*analysis_name*])

analysis_avg_rise_hold

A read-only double attribute that returns the average rise hold of all the maximum arcs for all slew/load points. Subscript: (pvt[,*analysis_name*])

|

analysis_avg_rise_setup

A read-only double attribute that returns the average rise setup of all the maximum arcs for all slew/load points. Subscript:(pvt[,analysis_name])

analysis_avg_switch_power

A read-only double attribute that returns the average switch power of all slew/load points. Subscript:(pvt[,analysis_name])

analysis_avgsl_fall_delay

A read-only float attribute that returns the average fall delay for all the arcs given slew/load points of lib_cells. Subscript:(pvt,slew,load[,analysis_name])

analysis_avgsl_fall_drive

A read-only float attribute that returns the average fall drive for all the arcs given slew/load points of lib_cells. Subscript:(pvt,slew,load[,analysis_name])

analysis_avgsl_fall_hold

A read-only float attribute that returns the average fall hold for all the arcs given slew/load points of lib_cells. Subscript:(pvt,slew,load[,analysis_name])

analysis_avgsl_fall_setup

A read-only float attribute that returns the average fall setup for all the arcs given slew/load points of lib_cells. Subscript:(pvt,slew,load[,analysis_name])

analysis_avgsl_fall_transition

A read-only float attribute that returns the average fall transition time for all the arcs given slew/load points of lib_cells. Subscript:(pvt,slew,load[,analysis_name])

analysis_avgsl_rise_delay

A read-only float attribute that returns the average rise delay for all the arcs given slew/load points of lib_cells. Subscript:(pvt,slew,load[,analysis_name])

analysis_avgsl_rise_drive

A read-only float attribute that returns the average rise drive for all the arcs given slew/load points of lib_cells. Subscript:(pvt,slew,load[,analysis_name])

analysis_avgsl_rise_hold

A read-only float attribute that returns the average rise hold for all the arcs given slew/load points of lib_cells. Subscript:(pvt,slew,load[,analysis_name])

|

analysis_avgsl_rise_setup

A read-only float attribute that returns the average rise setup for all the arcs given slew/load points of lib_cells. Subscript:(pvt,slew,load[,analysis_name])

analysis_avgsl_rise_transition

A read-only float attribute that returns the average rise transition time for all the arcs given slew/load points of lib_cells. Subscript:(pvt,slew,load[,analysis_name])

analysis_internal_power

A read-only double attribute that returns the internal power of lib_cells. Subscript:(pvt,slew,load[,analysis_name])

analysis_leakage

A read-only double attribute that returns the cell leakage of lib_cell Subscripted:(pvt[,analysis_name])

analysis_max_fall_delay

A read-only float attribute that returns the max fall delay for all the arcs given slew/load points of lib_cells. Subscript:(pvt,slew,load[,analysis_name])

analysis_max_fall_drive

A read-only float attribute that returns the max fall drive for all the arcs given slew/load points of lib_cells. Subscript:(pvt,slew,load[,analysis_name])

analysis_max_fall_hold

A read-only float attribute that returns the min rise hold for all the arcs given slew/load points of lib_cells. Subscript:(pvt,slew,load[,analysis_name])

analysis_max_fall_setup

A read-only float attribute that returns the min rise setup for all the arcs given slew/load points of lib_cells. Subscript:(pvt,slew,load[,analysis_name])

analysis_max_fall_transition

A read-only float attribute that returns the max fall transition time for all the arcs given slew/load points of lib_cells. Subscript:(pvt,slew,load[,analysis_name])

analysis_max_internal_power

A read-only double attribute that returns the maximum internal power of all slew/load points. Subscript:(pvt[,analysis_name])

|

analysis_max_rise_delay

A read-only float attribute that returns the max rise delay for all the arcs given slew/load points of lib_cells. Subscript:(pvt,slew,load[,analysis_name])

analysis_max_rise_drive

A read-only float attribute that returns the max rise drive for all the arcs given slew/load points of lib_cells. Subscript:(pvt,slew,load[,analysis_name])

analysis_max_rise_hold

A read-only float attribute that returns the max rise hold for all the arcs given slew/load points of lib_cells. Subscript:(pvt,slew,load[,analysis_name])

analysis_max_rise_setup

A read-only float attribute that returns the max rise setup for all the arcs given slew/load points of lib_cells. Subscript:(pvt,slew,load[,analysis_name])

analysis_max_rise_transition

A read-only float attribute that returns the max rise transition time for all the arcs given slew/load points of lib_cells. Subscript:(pvt,slew,load[,analysis_name])

analysis_max_switch_power

A read-only double attribute that returns the maximum switch power of all slew/load points. Subscript:(pvt[,analysis_name])

analysis_min_fall_delay

A read-only float attribute that returns the min fall delay for all the arcs given slew/load points of lib_cells. Subscript:(pvt,slew,load[,analysis_name])

analysis_min_fall_drive

A read-only float attribute that returns the min fall drive for all the arcs given slew/load points of lib_cells. Subscript:(pvt,slew,load[,analysis_name])

analysis_min_fall_hold

A read-only float attribute that returns the min fall hold for all the arcs given slew/load points of lib_cells. Subscript:(pvt,slew,load[,analysis_name])

analysis_min_fall_setup

A read-only float attribute that returns the min fall setup for all the arcs given slew/load points of lib_cells. Subscript:(pvt,slew,load[,analysis_name])

analysis_min_fall_transition

A read-only float attribute that returns the min fall transition time for all the arcs given slew/load points of lib_cells. Subscript:(pvt,slew,load[,analysis_name])

|

analysis_min_internal_power

A read-only double attribute that returns the minimum internal power of all slew/load points. Subscript:(pvt[,analysis_name])

analysis_min_rise_delay

A read-only float attribute that returns the min rise delay for all the arcs given slew/load points of lib_cells. Subscript:(pvt,slew,load[,analysis_name])

analysis_min_rise_drive

A read-only float attribute that returns the min rise drive for all the arcs given slew/load points of lib_cells. Subscript:(pvt,slew,load[,analysis_name])

analysis_min_rise_hold

A read-only float attribute that returns the max fall hold for all the arcs given slew/load points of lib_cells. Subscript:(pvt,slew,load[,analysis_name])

analysis_min_rise_setup

A read-only float attribute that returns the max fall setup for all the arcs given slew/load points of lib_cells. Subscript:(pvt,slew,load[,analysis_name])

analysis_min_rise_transition

A read-only float attribute that returns the min rise transition time for all the arcs given slew/load points of lib_cells. Subscript:(pvt,slew,load[,analysis_name])

analysis_min_switch_power

A read-only double attribute that returns the minimum switch power of all slew/load points. Subscript:(pvt[,analysis_name])

analysis_norm_area

A read-only double attribute that returns the normalized value of cell area of all lib_cells in the family: Subscript:(pvt[,analysis_name])

analysis_norm_avg_fall_delay

A read-only double attribute that returns the normalized value of average fall delay of all the maximum arcs for all slew/load points of all lib_cells in the family. Subscript:(pvt[,analysis_name])

analysis_norm_avg_fall_drive

A read-only double attribute that returns the normalized value of average fall drive of all the arcs for all slew/load points of all lib_cells in the family. Subscript:(pvt[,analysis_name])

|

analysis_norm_avg_fall_hold

A read-only double attribute that returns the normalized value of average fall hold of all the arcs for all slew/load points of all lib_cells in the family. Subscript: (pvt[,analysis_name])

analysis_norm_avg_fall_setup

A read-only double attribute that returns the normalized value of average fall setup of all the arcs for all slew/load points of all lib_cells in the family. Subscript: (pvt[,analysis_name])

analysis_norm_avg_rise_delay

A read-only double attribute that returns the normalized value of average rise delay of all the maximum arcs for all slew/load points of all lib_cells in the family. Subscript: (pvt[,analysis_name])

analysis_norm_avg_rise_drive

A read-only double attribute that returns the normalized value of average rise drive of all the arcs for all slew/load points of all lib_cells in the family. Subscript: (pvt[,analysis_name])

analysis_norm_avg_rise_hold

A read-only double attribute that returns the normalized value of average rise hold of all the arcs for all slew/load points of all lib_cells in the family. Subscript: (pvt[,analysis_name])

analysis_norm_avg_rise_setup

A read-only double attribute that returns the normalized value of average rise setup of all the arcs for all slew/load points of all lib_cells in the family. Subscript: (pvt[,analysis_name])

analysis_norm_leakage

A read-only double attribute that returns the normalized value of cell leakage of all lib_cells in the family. Subscripted: (pvt[,analysis_name])

analysis_norm_max_fall_delay

A read-only double attribute that returns the normalized value of maximum fall delay of all the maximum arcs for all slew/load points of all lib_cells in the family. Subscript: (pvt[,analysis_name])

analysis_norm_max_fall_drive

A read-only double attribute that returns the normalized value of maximum fall drive of all the arcs for all slew/load points of all lib_cells in the family. Subscript: (pvt[,analysis_name])

|

analysis_norm_max_fall_hold

A read-only double attribute that returns the normalized value of maximum fall hold of all the arcs for all slew/load points of all lib_cells in the family. Subscript: (pvt[,analysis_name])

analysis_norm_max_fall_setup

A read-only double attribute that returns the normalized value of maximum fall setup of all the arcs for all slew/load points of all lib_cells in the family. Subscript: (pvt[,analysis_name])

analysis_norm_max_internal_power

A read-only double attribute that returns the normalized value of maximum cell internal power for all slew/load points for all lib_cells in the family. Subscript: (pvt[,analysis_name])

analysis_norm_max_rise_delay

A read-only double attribute that returns the normalized value of maximum rise delay of all the maximum arcs for all slew/load points of all lib_cells in the family. Subscript: (pvt[,analysis_name])

analysis_norm_max_rise_drive

A read-only double attribute that returns the normalized value of maximum rise drive of all the arcs for all slew/load points of all lib_cells in the family. Subscript: (pvt[,analysis_name])

analysis_norm_max_rise_hold

A read-only double attribute that returns the normalized value of maximum rise hold of all the arcs for all slew/load points of all lib_cells in the family. Subscript: (pvt[,analysis_name])

analysis_norm_max_rise_setup

A read-only double attribute that returns the normalized value of maximum rise setup of all the arcs for all slew/load points of all lib_cells in the family. Subscript: (pvt[,analysis_name])

analysis_norm_max_switch_power

A read-only double attribute that returns the normalized value of maximum cell switch power for all slew/load points for all lib_cells in the family. Subscript: (pvt[,analysis_name])

|

analysis_norm_min_fall_delay

A read-only double attribute that returns the normalized value of minimum fall delay of all the arcs for all slew/load points of all lib_cells in the family. Subscript: (pvt[,analysis_name])

analysis_norm_min_fall_drive

A read-only double attribute that returns the normalized value of minimum fall drive of all the arcs for all slew/load points of all lib_cells in the family. Subscript: (pvt[,analysis_name])

analysis_norm_min_fall_hold

A read-only double attribute that returns the normalized value of minimum fall hold of all the arcs for all slew/load points of all lib_cells in the family. Subscript: (pvt[,analysis_name])

analysis_norm_min_fall_setup

A read-only double attribute that returns the normalized value of minimum fall setup of all the arcs for all slew/load points of all lib_cells in the family. Subscript: (pvt[,analysis_name])

analysis_norm_min_internal_power

A read-only double attribute that returns the normalized value of minimum cell internal power for all slew/load points for all lib_cells in the family. Subscript: (pvt[,analysis_name])

analysis_norm_min_rise_delay

A read-only double attribute that returns the normalized value of minimum rise delay of all the arcs for all slew/load points of all lib_cells in the family. Subscript: (pvt[,analysis_name])

analysis_norm_min_rise_drive

A read-only double attribute that returns the normalized value of maximum fall drive of all the arcs for all slew/load points of all lib_cells in the family. Subscript: (pvt[,analysis_name])

analysis_norm_min_rise_hold

A read-only double attribute that returns the normalized value of average rise hold of all the arcs for all slew/load points of all lib_cells in the family. Subscript: (pvt[,analysis_name])

|

analysis_norm_min_rise_setup

A read-only double attribute that returns the normalized value of minimum rise setup of all the arcs for all slew/load points of all lib_cells in the family. Subscript: (pvt[,analysis_name])

analysis_norm_min_switch_power

A read-only double attribute that returns the normalized value of minimum cell switch power for all slew/load points for all lib_cells in the family. Subscript: (pvt[,analysis_name])

analysis_switch_power

A read-only double attribute that returns the switch power of lib_cells. Subscript: (pvt,slew,load[,analysis_name])

antenna_diode_type

A read-only string attribute that returns the antenna diode type of the lib cell. The allowable values are "power", "ground" or "power_ground".

area

A read-only area attribute that returns the area of a lib_cell.

auxiliary_pad_cell

A read-only boolean attribute that indicates whether more than one pad lib_cell (attribute is_pad is true) can be used to build a logical pad.

base_name

A read-only string attribute that returns the base name of the lib_cell, which excludes the library prefix.

bbox

A read-only rect attribute that returns the bounding box of the lib cell.

block

A read-only block collection attribute that returns the block object for the lib-cell.

block_name

A read-only string attribute that returns the block name of the lib cell.

bottom_spacing_labels

A read-only string list attribute to represent names of spacing labels that are bottom of the provided library cell.

|

boundary

A settable coord_list attribute that returns the boundary of the lib cell.

boundary_bbox

A read-only rect attribute that returns the bbox of the boundary of the lib cell.

boundary_bounding_box

A read-only bounding_box collection attribute that returns the bounding-box of a lib_cell. The format of the collection specifies lower-left and upper-right corners of the rectangle.

boundary_polyrects

A read-only poly_list attribute that returns the boundary polyrects of a lib_cell. The no. of polyrects could be >1 only when disjoint

bounding_box

A read-only bounding_box collection attribute that returns the bounding-box of a lib_cell. The format of the collection specifies lower-left and upper-right corners of the rectangle.

builtin_clock_gating_integrated_cell

A read-only string attribute that gives a list of integrated clockgating (ICG) logic to be combined in a sequential cell.

cell_footprint

A settable string attribute that returns the footprint class of the lib cell.

cell_leakage_power

A settable float attribute that returns the cell leakage power for this lib cell.

clock_gating_integrated_cell

A read-only string attribute that returns the name of the integrated clock gating if the lib cell is of type integrated clock gating.

contention_condition

A read-only string attribute that specifies the contention conditions for a lib_cell. Contention is a clash of 0 and 1 signals. In certain cells, it can be a forbidden condition and cause circuits to short. Value is a boolean expression representing the contention condition.

core_area_area

A read-only double attribute that returns the core area of the lib cell.

|

core_area_bbox

A read-only rect attribute that returns the bounding box of the core area of the lib cell.

core_area_boundary

A read-only coord_list attribute that returns the boundary of the core area of the lib cell.

core_area_bounding_box

A read-only bounding_box collection attribute that returns the bounding-box of the core_area. The format of the collection specifies lower-left and upper- right corners of the rectangle.

design

A read-only design collection attribute that returns the design object for the lib-cell.

design_name

A read-only string attribute that returns the design name of the lib_cell.

design_type

A settable string attribute (with allowed values: 3dic abstract analog analysis black_box bridge corner cover die diode end_cap feedthrough fill filler flip_chip_driver flip_chip_pad ftv interposer lib_cell macro module package pad pad_spacer pcb rdl_fanout rtl substrate thermal tsv vib well_tap) that returns the design type of the lib cell.

device_group

A settable string attribute that returns the name of the category of the lib- cell based on its device characteristics. This attribute is settabl.

device_group_type

A settable string attribute that returns the type of the category of the lib- cell based on its device characteristics. The valid values are "narrow", "normal", and "wide".

disable_timing

A read-only boolean attribute that returns True if the lib_cell is marked as disable_timing in the library. This attribute can be set in icc2_lm_shell.

dont_touch

A settable boolean attribute that returns True if the lib_cell is marked as dont_touch to prevent optimization from changing any instances of this lib_cell.

|

dont_use

A settable boolean attribute that returns True if the lib_cell is marked as dont_use to prevent optimization from inserting instances of this lib_cell. The value of this attribute will also affect the valid_purposes attribute : When dont_use is set to true, the valid_purposes attribute will return no value (empty). When dont_use is set to false, the valid_purposes attribute will return to default value (power hold cts optimization).

excluded_purposes

A read-only string attribute that returns the purposes that are excluded as valid usages of the lib cell.

extended_name

A read-only string attribute that returns the full path to the source. db file of the library containing this lib_cell plus the full name of the lib_cell, separated by a ':' character. For example, "/disks/user1/work/tech_lib.db:tech_lib/AN2".

frame_update

A settable boolean attribute that enables updating the frame view of the lib cell during edit flow of the Library Manager. By default, the value is 'false'.

full_name

A read-only string attribute that returns the full name of the lib_cell, which includes the library prefix.

function_class

A read-only string attribute that returns the function class which represents the general function of the lib cell.

has_multi_power_rails

A read-only boolean attribute that returns True if the lib cell has multiple power rails.

has_timing_model

A read-only boolean attribute that returns True if the timing data is available for the lib cell.

has_tristate_outputs

A settable boolean attribute that returns True if a lib_cell has tristate output pins

height

A read-only distance attribute that returns the height of the boundary of the lib cell.

|

included_purposes

A read-only string attribute that returns the purposes that are included as valid usages of the lib cell.

inner_keepout_margin_clock

A settable margin_list attribute that returns the sum of all repeater count violating paths under the input hierarchy.

inner_keepout_margin_filler_allowed

A settable margin_list attribute that specifies the inner keepout margin of type filler_allowed.

inner_keepout_margin_hard

A settable margin_list attribute that specifies the inner keepout region where standard cells and hard macros can't be put, in the format {left bottom right top}.

inner_keepout_margin_hard_macro

A settable margin_list attribute that specifies the inner keepout region where hard macros can't be put, in the format {left bottom right top}.

inner_keepout_margin_route_blockage

A settable margin_list attribute that specifies the inner keepout region where routes can't be put, in the format {left bottom right top}.

inner_keepout_margin_route_blockage_layers

A read-only layer collection attribute that returns the layers of routes that can't be put in the route_blockage inner keepout region.

inner_keepout_margin_soft

A settable margin_list attribute that specifies the inner keepout region where standard cells and hard macros can't be put but optimizer and legalizer may not honor the restriction, in the format {left bottom right top}.

input_voltage_range

A read-only string attribute that specifies the allowed voltage range of the level shifter input pin and the voltage range for all input pins of the lib_cell under all possible operating conditions (defined across multiple libraries). The attribute defines two floating values: the first is the lower bound, and second is the upper bound. The attribute is also used in low-power macro modeling.

|

internal_power_derate_factor

A read-only float attribute that returns the actually stored internal derate values if and only if they exist. The value is based on current timer status, the attribute does not update the timer.

is_a_flip_flop

A settable boolean attribute that returns True if the lib_cell is a flip-flop

is_a_flip_flop_bank

A settable boolean attribute that returns True if the lib_cell is part of a bank of flip-flops

is_a_latch

A settable boolean attribute that returns True if the lib_cell is a latch

is_ao_cell_without_primary_pg_pin

A read-only boolean attribute that specifies whether it is an always-on backplane library cell.

is_backside

A read-only boolean attribute that returns True if this lib_cell is a backside bump cell and false otherwise. A backside bump cell is a lib_cell having design_type "flip_chip_pad" and either shapes on the backside metal layers, or shapes on the metal1 layer when its no_substrate attribute is true.

is_black_box

A read-only boolean attribute that returns True if the lib_cell has no logic function

is_buffer

A read-only boolean attribute that returns True if the lib_cell is a buffer

is_clock_isolation_cell

A read-only boolean attribute that returns True if the lib_cell is a clock isolation cell

is_combinational

A read-only boolean attribute that returns True if the lib_cell is combinational (not sequential)

is_compressed

A settable boolean attribute that represents whether a block is compressible. lib_cells are not compressible, so the attribute is always 'false' for lib cell

|

is_current

A read-only boolean attribute that returns True if the lib_cell is set as the current design

is_decap_cell

A read-only boolean attribute that returns True if the lib_cell is a decap cell

is_diff_level_shifter

A read-only boolean attribute that returns True if the lib_cell is a differential level shifter

is_differential_level_shifter

A read-only boolean attribute that returns True if the lib_cell is a differential level shifter

is_diode_cell

A read-only boolean attribute that returns True if the lib_cell is a power or ground diode cell

is_dummy

A settable boolean attribute that returns True if the lib_cell is a dummy bump cell

is_enable_level_shifter

A read-only boolean attribute that returns True if the lib_cell is an enable level shifter cell

is_etm_moded_cell

A read-only boolean attribute that returns True if the lib_cell is an ETM cell

is_fall_edge_triggered

A read-only boolean attribute that returns True if the lib_cell has timing behavior relative to the falling edge of a clock signal This attribute can be set in icc2_lm_shell.

is_feedthrough_via_cell

A read-only boolean attribute that returns True if the lib_cell is a feedthrough via cell

is_filler_cell

A read-only boolean attribute that returns True if the lib_cell is a filler cell

|

is_hierarchical

A read-only boolean attribute that represents if a block is hierarchical. The attribute is always 'false' for a lib_cell.

is_implicit

A read-only boolean attribute that returns True if the implicit flag is set on the lib cell.

is_integrated_clock_gating_cell

A read-only boolean attribute that returns True if the lib_cell is an integrated clockgating cell (ICG)

is_inverter

A read-only boolean attribute that returns True if the lib_cell is an inverter

is_isolation

A read-only boolean attribute that returns True if the lib_cell is an isolation cell

is_level_shifter

A read-only boolean attribute that returns True if the lib_cell is a level shifter cell

is_linked

A read-only boolean attribute that returns True if the lib_cell is linked

is_macro_cell

A settable boolean attribute that returns True if the lib_cell is a macro cell

is_mask_shiftable

A settable boolean attribute that returns True if the lib cell is mask shiftable. This attribute can be set in icc2_lm_shell.

is_memory_cell

A read-only boolean attribute that returns True if the lib_cell is defined in the library as a memory cell. This attribute can be set in icc2_lm_shell.

is_modified

A read-only boolean attribute that returns True if the lib cell is modified since the last library save

is_mux

A read-only boolean attribute that returns True if the lib_cell is a mux

|

is_negative_level_sensitive

A read-only boolean attribute that returns True if the lib_cell has negative level-sensitive timing behavior, as in a negative D-latch. This attribute can be set in `icc2_lm_shell`.

is_open

A read-only boolean attribute that returns True if the lib_cell is open

is_pll_cell

A read-only boolean attribute that returns True if the lib_cell is a PLL cell.

is_port_punching_ok

A settable boolean attribute that returns True if the ability to add or remove ports is enabled in the lib_cell.

is_positive_level_sensitive

A read-only boolean attribute that returns True if the lib_cell has positive level-sensitive timing behavior, as in a positive D-latch. This attribute can be set in `icc2_lm_shell`.

is_power_switch

A read-only boolean attribute that returns True if the lib_cell is a power switch cell.

is_remote

A read-only boolean attribute that represents whether a block is remote. Should always be false for lib_cells.

is_repeater

A read-only boolean attribute that returns True if the lib_cell is a repeater.

is_retention

A read-only boolean attribute that returns True if the lib_cell is a retention cell.

is_rise_edge_triggered

A read-only boolean attribute that returns True if the lib_cell has timing behavior relative to the rising edge of a clock signal. This attribute can be set in `icc2_lm_shell`.

is_sequential

A read-only boolean attribute that returns True if the lib_cell has a sequential logic function.

|

is_shared_timing_model

A read-only boolean attribute that indicates whether the lib_cell is a shared timing variant that is linked to a master timing view from which it retrieves its timing information.

is_soi

A read-only boolean attribute that returns True if the lib_cell is a silicon-on-insulator (SOI) cell.

is_tap_cell

A read-only boolean attribute that returns True if the lib_cell is a tap cell.

is_three_state

A read-only boolean attribute that returns True if the lib_cell has one or more three_state outputs. This attribute can be set in icc2_lm_shell.

keepouts

A read-only keepout_margin collection attribute that contains all types of inner and outer keepout margins applicable for this lib cell.

label_name

A read-only string attribute that returns the label name of the block. Lib cell should not have a label name.

last_modified

A read-only string attribute that returns the last modification time of the lib cell. This attribute does not exist for timing view lib-cells of a fusion library.

last_saved

A read-only string attribute that returns the last saved time of the lib cell.

leakage_power_derate_factor

A read-only float attribute that returns the actually stored leakage derate values if and only if they exist. The value is based on current timer status, the attribute does not update the timer.

left_spacing_labels

A read-only name_list attribute that returns the names of spacing labels that are left to the provided library cell.

level_shifter_type

A read-only string attribute that returns the value 'HL' if the level shifter is of type 'high-to-low', 'LH' if the level shifter is of type 'low-to-high' and 'HL_LH' if the level

shifter is of type 'high-to-low low-to-high'. Empty string is returned in case input library cell is not a level-shifter.

lib

A read-only lib collection attribute that returns the library for this lib_cell.

lib_name

A read-only string attribute that returns the name of the library for this lib_cell.

mask_shift_layers

A read-only layer collection attribute that returns the layers of lib_cell which can have shifted masks. This attribute can be set in icc2_lm_shell.

master_timing_model

A read-only lib_cell collection attribute that returns the master timing view for which this shared timing variant retrieves its timing information.

multibit_width

A read-only int attribute that returns the bitwidth of the cell for a multibit cell.

name

A read-only string attribute that returns the name of this lib_cell. Same as base_name, a simple name without the library prefix.

no_substrate

A settable boolean attribute that indicates whether this lib_cell has no substrate, and therefore no backside metal layers. When true, the lib_cell and cell is_backside attribute treats the metal1 layer as a backside metal layer. When false, only the bmetal layers are treated a backside metal layers.

number_of_pins

A read-only int attribute that returns the number of pins on this lib_cell.

object_class

A read-only string attribute that returns the name of this object class. Returns 'lib_cell' for lib_cell objects.

object_id

A read-only string attribute that returns the object_id of the lib_cell.

open_count

A read-only int attribute that returns the open count of the lib_cell.

|

open_mode

A read-only string attribute that returns the open mode for the lib_cell.

outer_keepout_boundary

A read-only coord_list attribute that extends the boundary outwards to cover the maximum of all types of outer keepout margins.

outer_keepout_margin_assembly_die

A settable margin_list attribute that returns the assembly_die outer keepout.

outer_keepout_margin_clock

A settable margin_list attribute that returns the format {left bottom right top} specifying an outer keepout region where clocks can't be put.

outer_keepout_margin_filler_allowed

A settable margin_list attribute that returns the outer keepout margin of type filler_allowed.

outer_keepout_margin_hard

A settable margin_list attribute that returns the format {left bottom right top} specifying the outer keepout region where standard cells and hard macros can't be put.

outer_keepout_margin_hard_macro

A settable margin_list attribute that returns the format {left bottom right top} specifying the outer keepout region where hard macros can't be put.

outer_keepout_margin_route_blockage

A settable margin_list attribute that returns the format {left bottom right top} specifying the outer keepout region where routes can't be put.

outer_keepout_margin_route_blockage_layers

A read-only layer collection attribute that returns the layers of routes that can't be put in the route_blockage outer keepout region.

outer_keepout_margin_seal_ring

A settable margin_list attribute that returns the seal_ring outer keepout.

outer_keepout_margin_soft

A settable margin_list attribute that returns the format {left bottom right top} specifying the outer keepout region where standard cells and hard macros can't be put but optimizer and legalizer may not honor the restriction.

|

output_voltage_range

A read-only string attribute that specifies the allowed voltage range of the levelshifter input pin and the voltage range for all output pins of the lib_cell under all possible operating conditions (defined across multiple libraries). The attribute defines two floating values: the first is the lower bound, and second is the upper bound. The attribute is also used in low-power macro modeling.

pad_cell

A read-only boolean attribute that returns True if the lib_cell is defined in the library as a pad cell.

pad_type

A read-only string attribute (with allowed values: clock) that identifies a type of pad_cell or auxiliary_pad_cell that requires special treatment. The only supported type is 'clock', which identifies a pad cell as a clock driver.

power_switch_type

A read-only string attribute that returns the value "header" if the power switch is a header cell and "footer" if the power switch is a footer cell. Empty string is returned in case input library cell is not a power-switch.

psc_type_id

A settable int attribute that returns the layout compatible type of lib_cell when doing programmable space cell mapping.

reference_orientation

A settable string attribute (with allowed values: R0 R90 R180 R270 MX MXR90 MY MYR90) that returns the reference orientation of the lib cell which is the orientation needed to be applied to an IO Pad such that it can be legally placed on the bottom row of IO cells. The lib cell must be an IO Pad.

retention_cell

A read-only string attribute that identifies a retention cell. It's value is a random string.

retention_cell_style

A read-only string attribute that returns the retention-cell-type of the lib- cell.

right_spacing_labels

A read-only name_list attribute that returns the names of spacing labels that are right to the provided library cell.

|

shared_timing_models

A read-only lib_cell collection attribute that returns the shared timing views that link to this master timing view for their timing information.

single_bit_degenerate

A read-only string attribute that names a single-bit lib_cell that can be used by Fusion Compiler to link a multibit blackbox cell with the single-bit version.

site_def

A read-only site_def collection attribute that returns the site_def associated with the lib_cell.

site_name

A read-only string attribute that returns the name of the site_def for this lib_cell.

subset_name

A settable name_list attribute that returns the lib_cell subset name for LEQ. All lib_cells with no subset name also form a subset, which is used by the technology mapping function. Named subsets are used for special lib_cells (e.g., clock buffers that are specifically designed to match the delay of certain clock gates).

switch_cell_type

A read-only string attribute (with allowed values: coarse_grain fine_grain) that specifies the type of the switch cell for direct inference. When specified, the tool does not need to indirectly infer the switch cell type through other lib_cell information.

switch_list

A read-only string attribute that returns the view switch list of the block. Lib cell should not have view switch list.

switching_power_derate_factor

A read-only float attribute that returns the actually stored switching derate values if and only if they exist. The value is based on current timer status, the attribute does not update the timer.

synchronizer_stages

A read-only int attribute that indicates the cell is a synchronizer cell and defines its stages. Its value is the number of logic stages.

|

target_boundary_area

A settable area attribute that returns the desired or target boundary area for the block. `lib_cell` should not have target boundary area.

target_utilization

A settable float attribute that returns the desired or target utilization for the block. `lib_cell` should not have target utilization.

test_normal_func_id

A read-only string attribute that returns the name of the test normal function id of the `lib_cell`.

thickness

A settable distance attribute that returns the thickness of the `lib_cell`. The unit will be the same as the user unit for length.

threshold_voltage_group

A settable string attribute that returns the name of the category of the `lib_cell` based on its threshold voltage characteristics.

threshold_voltage_group_type

A read-only string attribute that returns the type of the category of the `lib_cell` based on its threshold voltage characteristics. The valid values are "low_vt", "normal_vt" and "high_vt".

top_module

A read-only module collection attribute that returns the top module of the `lib_cell`.

top_module_name

A settable string attribute that returns the top module name of the `lib_cell`.

top_spacing_labels

A read-only string list attribute to represent names of spacing labels that are top of the provided library cell.

use_for_size_only

A read-only boolean attribute that returns True if the `lib_cell` is to be used for sizing only.

use_frame_blockage

A settable boolean attribute that indicates whether the blockages are not to be used in frame view of the `lib cell`. This is 'true' otherwise.

|

user_function_class

A settable string attribute that returns the name of the user function class of the lib_cell.

user_multibit_width

A read-only int attribute that returns the bit width of the multibit lib_cell.

user_power_group

A read-only string attribute that returns the user specified power group of a lib_cell.

valid_purposes

A read-only string attribute that returns the purposes that are valid usages of the lib cell.

via0_alignment

A settable boolean attribute that indicates whether the lib_cell needs to be VIA0 aligned.

via_ladder_pin_order

A settable lib_pin collection attribute that returns the lib_pins in order of via ladder creation. The router generates via ladders on cell instances of this lib_cell according to this order. If a lib_pin does not appear in the collection, then it is not constrained by a creation order. If this attribute is undefined on a lib_cell, then the lib_cell has no creation order constraint.

via_regions

A read-only via_region collection attribute that returns the via regions of the lib_cell.

view_name

A read-only string attribute (with allowed values: abstract conn design frame layout outline rtl timing) that returns the view name of the lib_cell.

width

A read-only distance attribute that returns the width of the boundary of the lib_cell.

See Also

- [get_attribute](#)
- [list_attributes](#)

lib_pin_attributes

Description of the application defined lib_pin attributes.

Description

Attributes are properties assigned to objects such as pins, cells, and nets. They can be read-only or settable. A read-only attribute is informational; you cannot set its value. To modify a settable attribute, use the *set_attribute* command.

To determine the value of an attribute, use the *get_attribute* command. To see a list of all lib_pin attributes available, use the *list_attributes -application -class lib_pin* command.

Attributes for lib_pin

This section describes the lib_pin attributes.

alive_during_partial_power_down

A read-only boolean attribute that returns True if the lib_pin is alive during partial power down value.

alive_during_power_up

A read-only boolean attribute that returns True if the lib_pin is alive during power up.

allow_diode_insertion

A settable boolean attribute that indicates whether diodes can be inserted on this lib_pin

always_on

A read-only boolean attribute that returns True if the pin is an always-on signal pin.

antenna_area

A read-only string attribute that returns the antenna area of the lib_pin.

antenna_diode_related_ground_pins

A read-only port collection attribute that, for an antenna-diode cell, specifies the related ground pin of the cell. Apply the *antenna_diode_related_ground_pins* attribute to the input pin of the cell. For a cell with a built-in antenna-diode pin or port, the *antenna_diode_related_ground_pins* attribute specifies the related ground pins for the antenna-diode pin. Apply the *antenna_diode_related_ground_pins* attribute to the antenna-diode pin.

|

antenna_diode_related_power_pins

A read-only port collection attribute that specifies the related power pin of the cell for an antenna-diode cell. Apply the `antenna_diode_related_power_pins` attribute to the input pin of the cell. For a cell with a built-in antenna-diode pin or port, the `antenna_diode_related_power_pins` attribute specifies the related power pins for the antenna-diode pin. Apply the `antenna_diode_related_power_pins` attribute to the antenna-diode pin.

antenna_diode_type

A read-only `diode_cell_type` attribute (with allowed values: `ground power power_ground`) that specifies the type of pin in a macro cell. Can only be specified on macro `lib_pins`.

antenna_side_area

A read-only string attribute that returns a list of layer wise antenna side wall area in the format {layer value}. This is a settable attribute in library manager.

base_name

A read-only string attribute that returns the simple name of the `lib_pin`, excluding the library and `lib_cell` name prefix.

bbox

A read-only rect attribute that returns the bounding box of the `lib_pin`.

bias_type

A read-only string attribute (with allowed values: `device_layer routing_pin`) that returns the bias type of the `lib_pin`.

bound_name

A read-only string attribute that returns the name of the bound of the `lib_pin`.

bound_type

A read-only string attribute (with allowed values: `hard soft`) that returns the type of the bound of the `lib_pin`.

bounding_box

A read-only `bounding_box` collection attribute that returns the bounding-box of a `lib_pin`. The format of the collection specifies lower-left and upper-right corners of the rectangle.

bus_hold_function

A read-only string attribute that specifies a boolean function for the `bus_hold` programmable driver type for I/O pad cells.

|

capacitance

A settable float attribute that returns the capacitance of the lib_pin.

clamp_0_function

A read-only string attribute that specifies a boolean function which is the input condition for the enable pins of an enable level-shifter or isolation cell when the output clamps to 0.

clamp_1_function

A read-only string attribute that specifies a boolean function which is the input condition for the enable pins of an enable level-shifter or isolation cell when the output clamps to 1.

clamp_latch_function

A read-only string attribute that specifies a boolean function which is the input condition for the enable pins of an enable level-shifter or isolation cell when the output clamps to the previously latched value.

clamp_z_function

A read-only string attribute that specifies a boolean function which is the input condition for the enable pins of an enable level-shifter or isolation cell when the output clamps to z.

clock

A read-only boolean attribute that returns True if the lib_pin is defined as a clock in the library.

clock_gate_clear_pin

A read-only boolean attribute that marks the asynchronous clear pin of an integrated clock gating library cell.

clock_gate_clock_pin

A read-only boolean attribute that marks the clock input pin of an integrated clock gating library cell.

clock_gate_enable_pin

A read-only boolean attribute that marks the enable pin of an integrated clock gating library cell. The enable pin controls whether the input clock signal is propagated to the output pin.

clock_gate_obs_pin

A read-only boolean attribute that marks the observation output pin of an integrated clock gating library cell. The observation output can be used to

observe the enable input when an instance of this library cell is not propagating the clock in DFT mode.

clock_gate_out_pin

A read-only boolean attribute that marks the clock output pin of an integrated clock gating library cell.

clock_gate_preset_pin

A read-only boolean attribute that marks the asynchronous preset pin of an integrated clock gating library cell.

clock_gate_test_pin

A read-only boolean attribute that marks the DFT (test mode) input pin of an integrated clock gating library cell.

clock_isolation_cell_clock_pin

A read-only boolean attribute that returns true if the lib_pin is an input clock pin of a clock-isolation cell.

clock_pin

A settable boolean attribute that returns true if the pin connects to a primary clock signal.

connect_within_pin

A settable string attribute (with allowed values: none via via_wire) that constrains the router when routing to a blocked lib pin using via or wire.

connection_class

A settable string attribute that returns the connection class string for the pin.

cut_area

A read-only string attribute that returns the antenna cut area of the lib_pin.

data_in_type

A read-only string attribute (with allowed values: clear data load preset) that specifies the type of input data defined by the data_in attribute in a latch or latch_bank group.

design

A read-only design collection attribute that returns the lib cell of the lib_pin.

|

diff_area

A read-only string attribute that returns a list of layer wise diode protection value for an output in the format {layer value}. This is a settable attribute in library manager.

diode_protection

A settable string attribute that returns a list of layer wise diode protection value for an external port in the format {layer value}.

direction

A read-only string attribute (with allowed values: in inout internal out) that returns the direction of a pin. Possible values are *in*, *out*, *inout*, or *unknown*.

disable_timing

A read-only boolean attribute that returns True if the lib_pin is marked as disable_timing in the library.

drive_current

A settable float attribute that specifies the drive current strength for the pad pin.

escaped_full_name

A read-only string attribute that returns the full name of the lib_pin with any hierarchy characters escaped with a backslash.

extended_name

A read-only string attribute that returns the full path to the source. db file of the library containing this lib_pin plus the full name of the lib_pin, separated by a ':' character. For example, "/disks/user1/work/tech_lib.db:tech_lib/AN2/Z".

fanout_load

A settable float attribute that returns the internal fanout of the input lib_pin.

full_name

A read-only string attribute that returns the full name of the lib_pin, including the library and lib_cell name prefix.

function

A read-only string attribute that returns output or inout lib_pins for the logic function. The logic function is defined in the library.

|

gate_area

A settable string attribute that returns a list of layer wise gate area for an input pin in the format {layer value}. If library supports multiple gate oxide modes then oxide mode is also present like {oxide_mode layer value}.

gate_diffusion_length

A read-only string attribute that returns a list of layer wise gate diffusion length in the format {layer value}. This is a settable attribute in library manager.

group_bound_height

A read-only distance attribute that returns the height of the group bound associated with the lib_pin.

group_bound_width

A read-only distance attribute that returns the width of the group bound associated with the lib_pin.

has_ccs_receiver_fall

A read-only boolean attribute that returns True if CCS information is present in this lib_pin for receiver fall analysis.

has_ccs_receiver_rise

A read-only boolean attribute that returns True if CCS information is present in this lib_pin for receiver rise analysis.

has_ccsn_first_stage

A read-only boolean attribute that returns True if ccsn_first_stage group is present in this lib_pin.

has_ccsn_last_stage

A read-only boolean attribute that returns True if ccsn_last_stage group is present in this lib_pin.

illegal_clamp_condition

A read-only string attribute that specifies a boolean expression representing the invalid condition for the enable pins of an enable level-shifter or isolation cell. If it is not specified, then the invalid condition does not exist..

input_signal_level

A read-only string attribute that describes the voltage level of a pin in a lib_cell with multiple power supplies. It is used for an input or inout lib_pin definition. The value is the name of the power supply associated with that pin.

|

input_threshold_pct_fall

A read-only float attribute that returns a value between 0.0 and 100.0. It specifies the threshold point of input signal falling from 1 to 0 on this lib_pin. Note that this may be a library-level value.

input_threshold_pct_rise

A read-only float attribute that returns a value between 0.0 and 100.0. It specifies the threshold point of input signal rising from 0 to 1 on this lib_pin. Note that this may be a library-level value.

input_voltage

A read-only string attribute that returns the name of the voltage range group for this pin (input pin).

input_voltage_range

A read-only string attribute that specifies the allowed voltage range of the level shifter input pin and the voltage range for all input pins of the lib_cell under all possible operating conditions. The attribute defines two floating values: the first is the lower bound, and second is the upper bound. The attribute is also used in low-power macro modeling.

is_acknowledge_pin

A read-only boolean attribute that specifies whether it is acknowledge pin of a coarse-grain switch cell.

is_analog

A read-only boolean attribute that returns True if this is an analog signal pin.

is_async_pin

A read-only boolean attribute that returns True if this lib_pin is a preset or clear pin.

is_clear_pin

A read-only boolean attribute that returns True if this lib_pin is a clear (reset) pin.

is_clock_pin

A read-only boolean attribute that returns True if this lib_pin is a clock pin of a register. A clock pin is any pin that has a timing check from it to a data pin.

is_data_pin

A read-only boolean attribute that returns True for data pins of registers. A data pin is any pin that has a timing check to it.

|

is_diode

A read-only boolean attribute that returns True if this lib_pin is a diode port.

is_em_via_ladder_required

A settable boolean attribute that returns True if this lib_pin requires an EM via ladder.

is_fall_edge_triggered_clock_pin

A read-only boolean attribute that returns True for clock pins of registers with falling edge-triggered behavior.

is_fall_edge_triggered_data_pin

A read-only boolean attribute that returns True for data pins of registers with falling edge-triggered behavior.

is_fixed

A read-only boolean attribute that returns True if the physical status of the lib pin is either fixed or locked (but not application_fixed).

is_insulated

A read-only boolean attribute that returns True if the lib pin is bias port and is insulated.

is_isolated

A read-only boolean attribute that specifies whether the pin is internally isolated and does not require the insertion of an external isolation cell.

is_isolation_cell_data_pin

A read-only boolean attribute that specifies whether it is the data pin of an isolation cell.

is_isolation_cell_enable_pin

A read-only boolean attribute that specifies whether it is the enable input pin of an isolation cell.

is_negative_level_sensitive_clock_pin

A read-only boolean attribute that returns True for clock pins of latches with negative level-sensitive behavior.

is_negative_level_sensitive_data_pin

A read-only boolean attribute that returns True for data pins of latches with negative level-sensitive behavior.

|

is_net_driver

A read-only boolean attribute that returns True if the lib_pin drives a net.

is_net_load

A read-only boolean attribute that returns true if the lib_pin is a load.

is_pad

A settable boolean attribute that returns true if the lib pin is a pad IO.

is_physical

A settable boolean attribute that returns true if the lib pin is a physical port.

is_pll_feedback_pin

A read-only boolean attribute that returns true if the library pin is a feedback pin of a phase locked loop (PLL) cell.

is_pll_output_pin

A read-only boolean attribute that returns true if the library pin is an output pin of a phase locked loop (PLL) cell.

is_pll_reference_pin

A read-only boolean attribute that returns true if the library pin is a reference pin of a phase locked loop (PLL) cell.

is_positive_level_sensitive_clock_pin

A read-only boolean attribute that returns true for clock pins of latches with positive level-sensitive behavior.

is_positive_level_sensitive_data_pin

A read-only boolean attribute that returns true for data pins of latches with positive level-sensitive behavior.

is_preset_pin

A read-only boolean attribute that returns true if this lib_pin is an asynchronous preset pin.

is_rectangle_only_rule_waived

A settable boolean attribute that returns true if rectangle only rule need to be waived.

is_retention_pin

A read-only boolean attribute that specifies whether it is the retention pin of a retention cell.

|

is_rise_edge_triggered_clock_pin

A read-only boolean attribute that returns True for clock pins of registers with rising edge-triggered behavior

is_rise_edge_triggered_data_pin

A read-only boolean attribute that returns True for data pins of registers with rising edge-triggered behavior

is_scan

A read-only boolean attribute that returns True if the lib_pin is a scan port

is_secondary_pg

A read-only boolean attribute that returns True if this is a secondary PG pin

is_shadow

A read-only boolean attribute that returns True if the lib_pin has shadow status

is_std_cell_main_rail

A read-only boolean attribute that specifies whether the pg pin is main rail of std cell.

is_switch_pin

A read-only boolean attribute that specifies whether it is switch pin of a coarse-grain switch cell.

is_three_state

A read-only boolean attribute that returns True if this is a three-state lib_pin A library pin is a three-state pin if there are three-state enable and or three-state disable arcs from or to that pin.

is_three_state_enable_pin

A read-only boolean attribute that returns True if this is a three-state enable lib_pin A library pin is a three-state enable pin if there are three-state enable and or three-state disable arcs from that pin.

is_three_state_output_pin

A read-only boolean attribute that returns True if this is a three-state output lib_pin A library pin is a three-state output pin if there are three-state enable and or three-state disable arcs to that pin.

is_unbuffered

A read-only boolean attribute that returns true if the library pin is unbuffered See the Library Compiler documentation.

I

is_unconnected

A read-only boolean attribute that returns True if the pin is internally not connected, which means that the pin is not part of a feedthrough path and it does not have the `related_power_pin` or `related_ground_pin` attribute.

isolation_cell_async_clear_pin

A settable boolean attribute that returns whether it is the asynchronous clear pin of an isolation cell.

isolation_cell_async_preset_pin

A settable boolean attribute that returns whether it is the asynchronous preset pin of an isolation cell.

isolation_cell_data_pin

A read-only boolean attribute that specifies whether it is the data pin of an isolation cell

isolation_cell_enable_pin

A read-only boolean attribute that specifies whether it is the enable input pin of an isolation cell

physical_hierarchy_level

An integer attribute that specifies the depth of the physical hierarchy.

layer

A read-only layer collection attribute that returns all the tech layers associated with the `lib_pin`

layer_name

A read-only string attribute that returns all the tech layers associated with the `lib_pin`.

level_shifter_data_pin

A read-only boolean attribute that specifies whether it is the input data pin of a level shifter cell

level_shifter_enable_pin

A read-only boolean attribute that returns true if the level shifter enable flag is set on the `lib_pin`. This flag can only be set on a signal pin.

lib

A read-only lib collection attribute that returns the library of the `lib_pin`.

|

lib_cell

A read-only *lib_cell* collection attribute that returns the lib cell of the *lib_pin*.

max_area_car

A read-only string attribute that returns a list of layer wise maximum cumulative area ratio on the layer using the metal area at or below the layer excluding the pin area itself along with oxide mode in the format {oxide_mode layer value}.

max_capacitance

A read-only float attribute that returns the *max_capacitance* constraint set on the *lib_pin* in the library or by the *set_max_capacitance* command. Value is returned for the *current_corner*.

max_cut_area_car

A read-only string attribute that returns a list of layer wise maximum cumulative antenna ratio on the layer using the cut area at or below the layer excluding the pin area itself along with oxide mode in the format {oxide_mode layer value}.

max_fanout

A settable float attribute that returns the maximum fanout of the output and inout lib pin.

max_side_area_car

A read-only string attribute that returns a list of layer wise maximum cumulative antenna ratio on the layer using the metal side wall area at or below the layer excluding the pin area itself along with oxide mode in the format {oxide_mode layer value}.

max_transition

A read-only float attribute that returns the *max_transition* constraint set on the *lib_pin* by the *set_max_transition* command.

min_capacitance

A read-only float attribute that returns the *min_capacitance* constraint set on the *lib_pin* in the library or by the *set_min_capacitance* command. Value is returned for the *current_corner*.

mode1_area

A read-only string attribute that returns a list of layer wise metal area when connectivity is traced upto that metal layer along with oxide mode in the format {oxide_mode layer value}. This is a settable attribute in library manager.

|

mode2_ratio

A read-only string attribute that returns a list of layer wise maximum internal ratio of area of metal layer below the specified metal layer when connectivity is traced upto that metal layer along with oxide mode in the format {oxide_mode layer value}. This is a settable attribute in library manager.

mode3_area

A read-only string attribute that returns a list of layer wise total metal area along with oxide mode in the format {oxide_mode layer value}. This is a settable attribute in library manager.

mode4_area

A read-only string attribute that returns a list of layer wise side wall metal area when connectivity is traced upto that metal layer along with oxide mode in the format {oxide_mode layer value}. This is a settable attribute in library manager.

mode5_ratio

A read-only string attribute that returns a list of layer wise maximum internal ratio of side wall area of metal layer below the specified metal layer when connectivity is traced upto that metal layer along with oxide mode in the format {oxide_mode layer value}. This is a settable attribute in library manager.

mode6_area

A read-only string attribute that returns a list of layer wise total side wall metal area along with oxide mode in the format {oxide_mode layer value}. This is a settable attribute in library manager.

multicell_pad_pin

A read-only boolean attribute that returns True for any pin on a pad cell or auxiliary pad cell that is connected to another cell.

must_join_port

A read-only port collection attribute that returns the must join port of the lib pin.

n_gate_area

A read-only string attribute that returns a list of layer wise gate area of n-channel transistors for an input pin in the format {layer value}. If library supports multiple gate oxide modes then oxide mode is also present like {oxide_mode layer value}. This is a settable attribute in library manager.

|

n_gate_diffusion_length

A read-only string attribute that returns a list of layer wise gate diffusion length of n-channel transistors in the format {layer value}. This is a settable attribute in library manager.

name

A read-only string attribute that returns the simple name of the lib_pin, excluding the library and lib_cell name prefix.

net_name

A read-only string attribute that returns the net name of the lib pin.

nextstate_type

A settable int attribute that defines the type of next_state attribute used in the ff or ff_bank group.

object_class

A read-only string attribute that returns the name of this object class. Returns 'lib_pin' for lib_pin objects.

open_drain_function

A read-only string attribute that specifies a boolean function for the open_drain programmable driver type for I/O pad cells.

open_source_function

A read-only string attribute that specifies a boolean function for the open_source programmable driver type for I/O pad cells.

output_signal_level

A read-only string attribute that describes the voltage level of a pin in a lib_cell with multiple power supplies. It is used for an output or inout lib_pin definition. The value is the name of the power supply associated with that pin.

output_threshold_pct_fall

A read-only float attribute that returns the value of the threshold point on an output pin signal falling from 1 to 0.

output_threshold_pct_rise

A read-only float attribute that returns the value of the threshold point on an output pin signal rising from 0 to 1.

|

output_voltage

A read-only string attribute that returns the name of the voltage range group for this pin (output pin)

p_gate_area

A read-only string attribute that returns a list of layer wise gate area of p-channel transistors for an input pin in the format {layer value}. If library supports multiple gate oxide modes then oxide mode is also present like {oxide_mode layer value}. This is a settable attribute in library manager.

p_gate_diffusion_length

A read-only string attribute that returns a list of layer wise gate diffusion length of p-channel transistors in the format {layer value}. This is a settable attribute in library manager.

parent_block

A read-only block collection attribute that returns the owner block of the lib_pin.

parent_block_cell

A read-only cell collection attribute that returns the owner block instance. This attribute is not defined for lib_pin.

parent_cell

A read-only cell collection attribute that returns the instance name of the cell's owner in the logic hierarchy. This attribute is not defined for lib_pin.

pattern_must_join

A settable boolean attribute that marks pattern_must_join.

pattern_must_join_level

A settable int attribute that marks pattern_must_join_level.

permit_power_down

A read-only boolean attribute that returns True if the power pin can be powered down while in the isolation mode.

pg_function

A read-only string attribute that returns the PG pin. It is to model that internal PG pin is related to a source PG pin.

pg_type

A read-only string attribute (with allowed values: backup internal primary) that specifies the PG type of the pin.

|

physical_hierarchy_level

A read-only int attribute that specifies the depth of the physical hierarchy.

physical_status

A settable string attribute (with allowed values: fixed locked placed unplaced) that returns the physical placement status of the lib_pin. If status is unplaced, the lib_pin has not been placed. If status is placed, the lib_pin has been placed and may be moved. If status is fixed, the lib_pin is fixed for automatic changes. If status is locked, the lib_pin is fixed for automatic and manual changes.

pin_capacitance

A read-only float attribute that returns the capacitance of this lib_pin. Value is returned for the current_corner.

pin_class

A read-only int attribute that returns the pin class of the lib_pin.

pin_direction

A read-only string attribute (with allowed values: in inout internal out) that returns the direction of a pin. Valid values are *in*, *out*, *inout*, or *unknown*. This is the same as "direction". This attribute is supported for compatibility with other tools. The "direction" attribute is supported for both ports and pins and therefore easier to use for heterogeneous collections of pins and ports.

pin_function_class

A read-only string attribute that returns the pin function class of the lib_pin. This is the canonical representation of the logic function of the lib_pin.

pin_number

A read-only int attribute that returns the pin number of the lib_pin.

port_direction

A read-only string attribute that returns the direction of a lib_pin. Possible values are *in*, *out*, *inout*, or *unknown*.

port_type

A read-only string attribute (with allowed values: analog_ground, analog_power, analog_signal, clock, deep_nwell, deep_pwell, ground, nwell, power, pwell, reset, scan, signal, tie_high, tie_low, unset) that returns the port type of a lib_pin.

power_down_function

A read-only string attribute that returns the boolean condition under which the cell's output pin is switched off by the state of the power and ground pins

I

pull_down_function

A read-only string attribute that specifies a boolean function for the pull_down programmable driver type for I/O pad cells

pull_up_function

A read-only string attribute that specifies a boolean function for the pull_up programmable driver type for I/O pad cells

rail_name

A read-only string attribute that returns the rail name of the lib pin.

related_bias_pin

A read-only port collection attribute that defines all bias pins associated with a power or ground lib_pin.

related_bias_pin_name

A read-only string attribute that returns all bias pins associated with a power or ground lib_pin.

related_ground_pin

A read-only port collection attribute that returns the related ground pin of the lib_pin.

related_ground_pin_name

A read-only string attribute that returns the related ground pin of the lib_pin.

related_power_pin

A read-only port collection attribute that returns the related power pin of the lib_pin.

related_power_pin_name

A read-only string attribute that returns the related power pin of the lib_pin.

resistive_0_function

A read-only string attribute that specifies a boolean function for the resistive_0 programmable driver type for I/O pad cells

resistive_1_function

A read-only string attribute that specifies a boolean function for the resistive_1 programmable driver type for I/O pad cells

|

resistive_function

A read-only string attribute that specifies a boolean function for the resistive programmable driver type for I/O pad cells

shadow_status

A settable string attribute (with allowed values: copied_down copied_up normal pulled_up pushed_down virtual_flat) that returns the shadow status of this lib_pin.

signal_type

A read-only string attribute that returns the signal type of the test lib_pin.

single_connection

A read-only boolean attribute that indicates whether the router is allowed to connect to an instance of this lib_pin only once and cannot use it as a feedthrough. This is a settable attribute in library manager.

slew_derate_from_library

A read-only float attribute that returns the lib pin which specify how the transition times found in the library need to be derated to match the transition times between the characterization trip points.

slew_lower_threshold_pct_fall

A read-only float attribute that returns a value between 0.0 and 100.0. It specifies the lower trip point for measuring the transition (slew) time of a falling signal on this lib_pin. Note that this may be a library-level value.

slew_lower_threshold_pct_rise

A read-only float attribute that returns a value between 0.0 and 100.0. It specifies the lower trip point for measuring the transition (slew) time of a rising signal on this lib_pin. Note that this may be a library-level value.

slew_upper_threshold_pct_fall

A read-only float attribute that returns a value between 0.0 and 100.0. It specifies the upper trip point for measuring the transition (slew) time of a falling signal on this lib_pin. Note that this may be a library-level value.

slew_upper_threshold_pct_rise

A read-only float attribute that returns a value between 0.0 and 100.0. It specifies the upper trip point for measuring the transition (slew) time of a rising signal on this lib_pin. Note that this may be a library-level value.

|

switch_function

A read-only string attribute that returns the switch function expression of the lib_pin.

switch_pin

A read-only boolean attribute that returns True if the lib_pin is a switch pin of a coarse-grain switch cell

test_output_only

A read-only boolean attribute that is for any output lib pin described in statetable format. When set to true specify that a lib_pin is for test only.

three_state_function

A read-only string attribute that returns the output or inout lib_pins for the three_state logic function. The three_state logic function is defined in the library. If the three_state logic function evaluates to zero, then the lib_pin is enabled.

tied_to

A read-only string attribute that is tied to the lib pin associated with the input lib_pin.

top_block

A read-only block collection attribute that returns the lib cell of the lib_pin.

via_ladder_min_layer_for_bottom

A read-only layer collection attribute that returns the tech layer used as the layer constraint for bottom layer connection to a via ladder.

via_ladder_names

A settable string attribute that returns the names of the via ladder candidates of the lib_pin.

x_function

A read-only string attribute that describes the X state behavior of an output or inout pin as a boolean function X is a state other than 0, 1, or Z.

See Also

- [get_attribute](#)
- [list_attributes](#)

lib_timing_arc_attributes

Description of the application defined lib_timing_arc attributes.

Description

Attributes are properties assigned to objects such as pins, cells, and nets. They can be read-only or settable. A read-only attribute is informational; you cannot set its value. To modify a settable attribute, use the *set_attribute* command.

To determine the value of an attribute, use the *get_attribute* command. To see a list of all lib_timing_arc attributes available, use the *list_attributes -application -class lib_timing_arc* command.

Attributes for lib_timing_arc

This section describes the lib_timing_arc attributes.

analysis_fall_delay

A read-only float attribute that returns the fall delay of the arc. Subscript: (pvt,slew,load point[,analysis_name])

analysis_fall_drive

A read-only float attribute that returns the fall drive of the arc. Subscript: (pvt,slew,load point[,analysis_name])

analysis_fall_hold

A read-only float attribute that returns the fall hold of the arc. Subscript: (pvt,slew,load point[,analysis_name])

analysis_fall_setup

A read-only float attribute that returns the fall setup of the arc. Subscript: (pvt,slew,load point[,analysis_name])

analysis_fall_transition

A read-only float attribute that returns the fall transition of the arc. Subscript: (pvt,slew,load point[,analysis_name])

analysis_rise_delay

A read-only float attribute that returns the rise delay of the arc. Subscript: (pvt,slew,load point[,analysis_name])

analysis_rise_drive

A read-only float attribute that returns the rise drive of the arc. Subscript: (pvt,slew,load point[,analysis_name])

|

analysis_rise_hold

A read-only float attribute that returns the rise hold of the arc. Subscript: (pvt,slew,load point[,analysis_name])

analysis_rise_setup

A read-only float attribute that returns the rise setup of the arc. Subscript: (pvt,slew,load point[,analysis_name])

analysis_rise_transition

A read-only float attribute that returns the rise transition of the arc. Subscript: (pvt,slew,load point[,analysis_name])

clock_to_q

A read-only boolean attribute that indicates whether this arc corresponds to clock to Q or QN pin.

from_base_name

A read-only string attribute that returns the name of the from lib pin of the lib timing arc.

from_lib_pin

A read-only lib_pin collection attribute that returns the from library pin for a lib_timing_arc. If this lib_timing_arc is for a timing check such as setup or hold, the from pin is the clock pin of the check.

full_name

A read-only string attribute that returns the full name of the lib timing arc.

has_ccs_receiver_fall

A read-only boolean attribute that indicates whether CCS information is present in this lib_timing_arc for receiver fall analysis.

has_ccs_receiver_rise

A read-only boolean attribute that indicates whether CCS information is present in this lib_timing_arc for receiver rise analysis.

has_ccsn_first_stage

A read-only boolean attribute that returns True if ccsn_first_stage group is present in this lib_timing_arc.

has_ccsn_last_stage

A read-only boolean attribute that returns True if ccsn_last_stage group is present in this lib_timing_arc.

|

is_disabled

A read-only boolean attribute that indicates whether this library timing arc is disabled. Library timing arcs may be disabled either because the user has disabled it with the command *set_disable_timing* or because of mode settings.

is_user_disabled

A read-only boolean attribute that indicates whether this library timing arc is disabled because of a *set_disable_timing* command.

lib

A read-only lib collection attribute that returns the library of the from lib pin of the lib timing arc.

lib_cell

A read-only lib_cell collection attribute that returns the lib cell of the from lib pin of the lib timing arc.

mode

A read-only string attribute that returns the name of the mode this lib timing arc is active for. The attribute will only be defined for moded arcs.

name

A read-only string attribute that returns the name of the lib timing arc.

object_class

A read-only string attribute that returns the name of this object class. Returns 'lib_timing_arc' for lib_timing_arc objects.

sdf_cond

A read-only string attribute that returns a string representing the SDF condition of the library timing arc.

sense

A read-only string attribute that returns the sense of a lib_timing_arc. Possible values are

rising_edge	falling_to_rise
rising_to_rise	rising_to_fall
falling_edge	rise_to_rise
falling_to_fall	rise_to_fall
fall_to_rise	positive_unate
fall_to_fall	non_unate
negative_unate	clear_low
clear_high	preset_high
clear_both	

<pre> preset_low disable_high disable_both disable_low_rise disable_high_fall disable_both_fall enable_low enable_high_rise enable_both_rise enable_low_fall retain_rising_edge retain_fall_to_rise retain_rise_to_fall retain_positive_unate retain max_clock_tree_rise_to_rise max_clock_tree_negative_unate max_clock_tree_fall_to_rise min_clock_tree_positive_unate min_clock_tree_fall_to_fall min_clock_tree_rise_to_fall min_clock_tree_non_unate nonseq_setup_rise_clk_rise nonseq_setup_clk_fall nonseq_setup_fall_clk_fall nonseq_hold_rise_clk_rise nonseq_hold_clk_fall nonseq_hold_fall_clk_fall clock_gating_setup_rise_clk_rise clock_gating_setup_fall_clk_rise clock_gating_setup_clk_fall clock_gating_setup_rise_clk_fall clock_gating_setup_fall_clk_fall clock_gating_hold_rise_clk_rise </pre>	<pre> preset_both disable_low disable_high_rise disable_both_rise disable_low_fall enable_high enable_both enable_low_rise enable_high_fall enable_both_fall retain_rise_to_rise retain_falling_edge retain_fall_to_fall retain_negative_unate max_clock_tree_positive_unate max_clock_tree_fall_to_fall max_clock_tree_rise_to_fall max_clock_tree_non_unate min_clock_tree_rise_to_rise min_clock_tree_negative_unate min_clock_tree_fall_to_rise nonseq_setup_clk_rise nonseq_setup_fall_clk_rise nonseq_setup_rise_clk_fall nonseq_hold_clk_rise nonseq_hold_fall_clk_rise nonseq_hold_rise_clk_fall clock_gating_setup_clk_rise clock_gating_hold_clk_rise </pre>
---	--

to_base_name

A read-only string attribute that returns the name of the from lib pin of the lib timing arc.

to_lib_pin

A read-only lib_pin collection attribute that returns the to library pin for a lib_timing_arc. If this lib_timing_arc is for a timing check such as setup or hold, the from pin is the data pin of the check.

when

A read-only string attribute that returns the when value for the lib_timing_arc.

See Also

- [get_attribute](#)
- [list_attributes](#)

m

manufacturing_shape_attributes

Description of the application defined manufacturing_shape attributes.

Description

Attributes are properties assigned to objects such as pins, cells, and nets. They can be read-only or settable. A read-only attribute is informational; you cannot set its value. To modify a settable attribute, use the *set_attribute* command.

To determine the value of an attribute, use the *get_attribute* command. To see a list of all manufacturing_shape attributes available, use the *list_attributes -application -class manufacturing_shape* command.

Attributes for manufacturing_shape

This section describes the manufacturing_shape attributes.

bbox

A read-only rect attribute that returns the bounding-box of a manufacturing_shape. The format of a rectangle specification is `{{llx lly} {urx ury}}`, which specifies the lower-left and upper-right corners of the rectangle. This is the bounding box of the draw area specified by users when creating the manufacturing_shape.

boundary

A settable coord_list attribute that returns the boundary of a design. The format of a coordinate_list specification is `{{x1 y1} {x2 y2}...}`. If you want to clear an existing boundary, use the *remove_attribute(2)* command.

bounding_box

A read-only bounding_box collection attribute that returns the bounding-box of a manufacturing_shape. The format of the collection specifies lower-left and upper-right corners of the rectangle. This is the bounding box of the draw area specified by users when creating the manufacturing_shape.

full_name

A read-only string attribute that returns the name of a manufacturing_shape.

label

A settable string attribute that returns a label of a manufacturing_shape, available only to custom-type manufacturing_shape.

name

A settable string attribute that returns the name of a manufacturing_shape.

object_class

A read-only string attribute that returns the name of this object class. Returns 'manufacturing_shape' for manufacturing_shape objects.

parent_block

A read-only block collection attribute that returns a collection for the parent block of the voltage area.

parent_block_cell

A read-only cell collection attribute that returns a collection for the instance corresponding to the parent block of the voltage area.

parent_cell

A read-only cell collection attribute that returns a collection for the instance corresponding to the root hierarchy of the voltage area if it exists.

physical_status

A settable string attribute (with allowed values: locked unrestricted) that returns the physical modification status of the manufacturing_shape. If status is unrestricted, the manufacturing_shape may be freely modified. If status is locked, the manufacturing_shape is fixed for automatic and manual changes.

top_block

A read-only block collection attribute that returns a collection of the top block for the corresponding voltage area.

See Also

- [create_manufacturing_shape](#)
- [get_attribute](#)
- [list_attributes](#)
- [set_attribute](#)

matching_type_attributes

Description of the application defined matching_type attributes.

Description

Attributes are properties assigned to objects such as pins, cells, and nets. They can be read-only or settable. A read-only attribute is informational; you cannot set its value. To modify a settable attribute, use the *set_attribute* command.

To determine the value of an attribute, use the *get_attribute* command. To see a list of all matching_type attributes available, use the *list_attributes -application -class matching_type* command.

Attributes for matching_type

This section describes the matching_type attributes.

full_name

A read-only string attribute that returns the name of the matching_type.

name

A settable string attribute that returns the name of the matching_type.

object_class

A read-only string attribute that returns the name of this object class. Returns 'matching_type' for matching_type objects.

objects

A read-only cell, pin, or terminal collection attribute that returns the cells, pins, and terminals in the matching_type.

uniquify_number

A settable int attribute that returns the uniquify number of the matching_type. Valid values are in the range [-4, 8]. If the value is greater than 0, the value indicates the number of pad pins that might be assigned to a single bump cell, and no two bump cells might be assigned to the same pad pin. If the value is less than 0, the absolute value indicates the number of bump cells that must be assigned to each pad pin, and no two pad pins might be associated to the same bump cell. If the value = 0, only 1 bump cell is assigned to all pad pins.

See Also

- [get_attribute](#)
- [list_attributes](#)

material_attributes

Lists the predefined attributes for materials.

Description

Attributes are properties assigned to objects such as pins, cells and nets. Definitions for material attributes are provided in the subsections that follow.

Attributes are read-only unless marked as settable. Read-only attributes are informational, and you cannot set its value. A settable attribute can be modified with the *set_attribute* command.

To determine the value of an attribute, use the *get_attribute* command. To see a list of all attributes available for a class of objects use the *list_attributes -application* command.

Material Attributes

cte

A settable double attribute for the Coefficient of Thermal expansion of the material, popularly denoted by CTE. It is a double value with the units 1e-6/oC. Note that this physical property can vary with temperature, the value specified is the value at 25 oC, i.e. room temperature.

density

A settable double attribute for the density of the material. It is a double value in the units g/cm^3. Note that this physical property can vary with temperature, the value specified is the density at 25 oC, i.e. room temperature.

full_name

A read-only string attribute for the full name of the material.

lib_name

A read-only string attribute representing the owner library name.

name

A read-only string attribute for the name of the material.

nusselt_number

A settable double attribute for the Nusselt number of the material, popularly denoted by Nu. It is a double value without any units.

object_class

A read-only attribute with the value of "material".

prandtl_number

A settable double attribute for the Prandtl number of the material, popularly denoted by Pr. It is a double value without any units. Note that this physical property can vary with temperature, the value specified is at 25 oC, i.e. room temperature.

shear_modulus

A settable double attribute for the Shear Modulus value of the material, popularly denoted by G. It is a double value with the units GPa, i.e. 1e06 N/m². Note that this physical property can vary with temperature, the value specified is the value at 25 oC, i.e. room temperature.

specific_heat_capacity

A settable double attribute for the specific heat capacity of the material, popularly denoted by cp. It is a double value in the units J/g-oC, i.e. J/g-K. Note that this physical property can vary with temperature, the value specified is the specific heat capacity at 25 oC, i.e. room temperature.

thermal_conductivity

A settable double attribute for the thermal conductivity of the material, popularly denoted by k, i.e. its ability to conduct heat. It is a double value in the units W/m-oC, i.e. W/m-K. Note that this physical property can vary with temperature, the value specified is the thermal conductivity at 25 oC(degree Celsius), i.e. room temperature.

thermal_conductivity_anisotropic

A settable double list attribute for the anisotropic thermal conductivity of the material. It is a list of 3 double values in the units W/m-oC, i.e. W/m-K. Note that this physical property can vary with temperature, the value specified is the anisotropic thermal conductivity at 25 oC(degree Celsius), i.e. room temperature.

type

A read-only string attribute (with allowed values: conductor dielectric) for the type of the material.

youngs_modulus

A settable double attribute for the Young's Modulus of the material, popularly denoted by E. It is a double value with the units GPa. Note that this physical property can vary with temperature, the value specified is the value at 25 oC, i.e. room temperature.

See Also

- [get_materials](#)
- [get_attribute](#)
- [list_attributes](#)
- [set_attribute](#)

metal_area_attributes

Description of the application defined metal_area attributes.

Description

Attributes are properties assigned to objects such as pins, cells, and nets. They can be read-only or settable. A read-only attribute is informational; you cannot set its value. To modify a settable attribute, use the *set_attribute* command.

To determine the value of an attribute, use the *get_attribute* command. To see a list of all metal_area attributes available, use the *list_attributes -application -class metal_area* command.

Attributes for metal_area

This section describes the metal_area attributes.

bbox

A read-only rect attribute that returns the bounding-box of a metal area. The format of a rectangle specification is `{{llx lly} {urx ury}}`, which specifies the lower-left and upper-right corners of the rectangle.

boundary

A settable polygon attribute that returns the boundary of the metal area. The format of coordinate list specification is `{{x1 y1} {x2 y2} ...}`, which specifies a list of points defining the boundary of the shape.

bounding_box

A read-only bounding_box collection attribute that returns the bounding-box of a metal area. The format of the collection specifies lower-left and upper-right corners of the rectangle.

effective_shapes

A read-only poly_list attribute that returns the effective boundary of the metal area. The effective_shape is calculated by comparing the stacking_order of the metal area of the corresponding layer with all other metal areas in the

block of same layer, subtracting the overlapping region from the regions with lower stacking order. The format of the specification is $\{\{x_{11} y_{11}\} \{x_{12} y_{12}\} \dots\} \{\{x_{21} y_{21}\} \{x_{22} y_{22}\} \dots\} \dots\}$, which specifies a list of coordinates of rectilinear polygons.

full_name

A read-only string attribute that returns the name of the metal area.

generated_shape_are_out_of_date

A read-only boolean attribute that returns whether the realized shapes corresponding to the metal area need to be regenerated.

layer

A settable layer collection attribute that returns the layer on which the metal area exists.

layer_name

A read-only string attribute that returns the name of the layer on which the metal area exists.

name

A read-only string attribute that returns the name of the metal area.

net

A settable net collection attribute that represents the owner net of this metal area.

object_class

A read-only string attribute that returns the name of this object class. Returns 'metal_area' for metal_area objects.

parent_block

A read-only block collection attribute that returns the owner block object for this metal area.

parent_block_cell

A read-only cell collection attribute that returns the parent block cell object for this metal area.

parent_cell

A read-only cell collection attribute that returns the owner cell object for this metal area.

physical_status

A settable string attribute (with allowed values: `application_fixed` `fixed` `locked` `minor_change` `unrestricted`) that returns the physical modification status of the metal area. If status is `unrestricted`, the metal area may be freely modified. If the status is `application_fixed`, the metal area is fixed by the application for automatic changes (the application may change this status). If status is `fixed`, the metal area is fixed by the user for automatic changes. If status is `locked`, the metal area is fixed for automatic and manual changes.

stacking_order

A read-only int attribute that returns the number of metal areas above the bottom of the metal area position stack.

top_block

A read-only block collection attribute that returns the top block object for this metal area.

See Also

- [create_metal_area](#)
- [get_attribute](#)
- [list_attributes](#)
- [set_attribute](#)

metal_area_hole_attributes

Description of the application defined `metal_area_hole` attributes.

Description

Attributes are properties assigned to objects such as pins, cells, and nets. They can be read-only or settable. A read-only attribute is informational; you cannot set its value. To modify a settable attribute, use the *set_attribute* command.

To determine the value of an attribute, use the *get_attribute* command. To see a list of all `metal_area_hole` attributes available, use the *list_attributes -application -class metal_area_hole* command.

Attributes for metal_area_hole

This section describes the `metal_area_hole` attributes.

bbox

A read-only rect attribute that returns the bounding-box of a metal area hole. The format of a rectangle specification is `{{llx lly} {urx ury}}`, which specifies the lower-left and upper-right corners of the rectangle.

boundary

A settable polygon attribute that returns the boundary of the metal area hole. The format of coordinate list specification is `{{x1 y1} {x2 y2} ...}`, which specifies a list of points defining the boundary of the shape.

bounding_box

A read-only bounding_box collection attribute that returns the bounding-box of a metal area hole. The format of the collection specifies lower-left and upper-right corners of the rectangle.

full_name

A read-only string attribute that returns the name of the metal area hole.

metal_area

A read-only metal_area collection attribute that returns a collection representing the owner metal area of this metal area hole.

name

A read-only string attribute that returns the name of the metal area hole.

object_class

A read-only string attribute that returns the name of this object class. Returns 'metal_area_hole' for metal_area_hole objects.

parent_block

A read-only block collection attribute that returns the owner block object for this metal area hole.

parent_block_cell

A read-only cell collection attribute that returns the parent block cell object for this metal area hole.

parent_cell

A read-only cell collection attribute that returns the owner cell object for this metal area hole.

physical_status

A settable string attribute (with allowed values: `application_fixed` `fixed` `locked` `minor_change` `unrestricted`) that returns the physical modification status of the metal area hole. If status is `unrestricted`, the metal area hole may be freely modified. If the status is `application_fixed`, the metal area hole is fixed by the application for automatic changes (the application may change this status). If status is `fixed`, the metal area hole is fixed by the user for automatic changes. If status is `locked`, the metal area hole is fixed for automatic and manual changes.

top_block

A read-only block collection attribute that returns the top block object for this metal area hole.

See Also

- [create_metal_area_hole](#)
- [get_attribute](#)
- [list_attributes](#)
- [set_attribute](#)

mismatch_object_attributes

Description of the application defined `mismatch_object` attributes.

Description

Attributes are properties assigned to objects such as pins, cells, and nets. They can be read-only or settable. A read-only attribute is informational; you cannot set its value. To modify a settable attribute, use the *set_attribute* command.

To determine the value of an attribute, use the *get_attribute* command. To see a list of all `mismatch_object` attributes available, use the *list_attributes -application -class mismatch_object* command.

Attributes for mismatch_object

This section describes the `mismatch_object` attributes.

action

A read-only string attribute (with allowed values: `error` `ignore` `repair`) that returns the action associated to the mismatch object.

current_repair

A read-only string attribute that returns the repair strategy associated to the mismatch object.

full_name

A read-only string attribute that returns the name of a mismatch_object.

mismatch_type

A read-only collection attribute that returns the mismatch type associated to the mismatch object.

object_class

A read-only string attribute that returns the name of this object class. Returns 'mismatch_object' for mismatch_object objects.

object_name

A read-only string attribute that returns the name of the underlying object corresponding to which the mismatch object is created.

object_type

A read-only string attribute that returns the type of the underlying object corresponding to which the mismatch object is created.

See Also

- [get_mismatch_objects](#)
- [get_attribute](#)
- [list_attributes](#)
- [set_attribute](#)

mismatch_type_attributes

Description of the application defined mismatch_type attributes.

Description

Attributes are properties assigned to objects such as pins, cells, and nets. They can be read-only or settable. A read-only attribute is informational; you cannot set its value. To modify a settable attribute, use the *set_attribute* command.

To determine the value of an attribute, use the *get_attribute* command. To see a list of all mismatch_type attributes available, use the *list_attributes -application -class mismatch_type* command.

Attributes for mismatch_type

This section describes the mismatch_type attributes.

action

A settable string attribute that returns the action associated to each mismatch type under each config. Pre-defined configs are: auto_fix and default. The possible values for action can be : repair, error, accept and ignore. This is a subscripted attribute based on configs.

available_repairs

A read-only string attribute that returns all the repair strategies available for a mismatch type under each config.

category

A read-only string attribute (with allowed values: floorplan library netlist routing upf) that returns a string attribute about the category to which a mismatch type belongs to.

current_repair

A settable string attribute that returns a string attribute about the repair strategy registered for a mismatch type under each config. This is a subscripted attribute based on configs.

description

A read-only string attribute that returns a string attribute having the description of a mismatch type.

full_name

A read-only string attribute (with allowed values: bus_bit_naming bus_width_mismatch empty_logic_module exceed_ndr_limitation layer_missing_prefer_direction lib_cell_name_case lib_missing_logical_port lib_missing_physical_port lib_missing_physical_reference lib_missing_rail_pg lib_port_direction lib_port_missing_access_edge lib_port_missing_via_region lib_port_name_case lib_port_name_synonym lib_port_type macro_cell_contain_too_many_obs missing_logical_reference missing_port port_name_case port_name_synonym reference_name_case site_def_mismatch) that returns the name of a mismatch_type.

name

A read-only string attribute (with allowed values: bus_bit_naming bus_width_mismatch empty_logic_module exceed_ndr_limitation layer_missing_prefer_direction lib_cell_name_case lib_missing_logical_port lib_missing_physical_port lib_missing_physical_reference lib_missing_rail_pg

lib_port_direction lib_port_missing_access_edge lib_port_missing_via_region
lib_port_name_case lib_port_name_synonym lib_port_type
macro_cell_contain_too_many_obs missing_logical_reference
missing_port port_name_case port_name_synonym reference_name_case
site_def_mismatch) that returns the name of a mismatch_type.

object_class

A read-only string attribute that returns the name of this object class. Returns 'mismatch_type' for mismatch_type objects.

See Also

- [get_mismatch_types](#)
- [get_attribute](#)
- [list_attributes](#)
- [set_attribute](#)

mode_attributes

Description of the application defined mode attributes.

Description

Attributes are properties assigned to objects such as pins, cells, and nets. They can be read-only or settable. A read-only attribute is informational; you cannot set its value. To modify a settable attribute, use the *set_attribute* command.

To determine the value of an attribute, use the *get_attribute* command. To see a list of all mode attributes available, use the *list_attributes -application -class mode* command.

Attributes for mode

This section describes the mode attributes.

file_line_info

A read-only string attribute that returns the Tcl or SDC source file name and line number pairs of the commands, which are used to create this object, if it exists.

full_name

A read-only string attribute that returns the name of a mode.

is_current

A read-only boolean attribute that returns true if this mode is the current mode, otherwise false.

is_default

A read-only boolean attribute that returns true if this mode is the default mode, otherwise false.

name

A read-only string attribute that returns the name of the mode object.

object_class

A read-only string attribute that returns the name of this object class. Returns 'mode' for mode objects.

See Also

- [create_mode](#)
- [get_attribute](#)
- [list_attributes](#)

module_attributes

Description of the application defined module attributes.

Description

Attributes are properties assigned to objects such as pins, cells, and nets. They can be read-only or settable. A read-only attribute is informational; you cannot set its value. To modify a settable attribute, use the *set_attribute* command.

To determine the value of an attribute, use the *get_attribute* command. To see a list of all module attributes available, use the *list_attributes -application -class module* command.

Attributes for module

This section describes the module attributes.

full_name

A read-only string attribute that returns the full name of a module.

groups

A read-only group collection attribute that returns the groups this module belongs to.

is_new_cell_allowed

A settable boolean attribute that returns true if the new cell creation is allowed in the module.

is_outline

A read-only boolean attribute that returns true if this module is an outline module. That is, the design owning it an outline view.

name

A settable string attribute that returns the name of a module. While setting this attribute if specified name matches any existing module name in the same design then an error is issued.

object_class

A read-only string attribute that returns the name of this object class. Returns 'module' for module objects.

outline_def_file_name

A settable string attribute that returns the outline def file for this module.

outline_def_file_size

A read-only string attribute that returns the size of the outline def file.

outline_def_file_timestamp

A read-only string attribute that returns the timestamp of the outline def file.

outline_file_line_end

A settable int attribute that returns the end line of outline file.

outline_file_line_start

A settable int attribute that returns the start line of outline file.

outline_file_name

A settable string attribute that returns the outline file for this module, usually the source verilog file.

outline_file_size

A read-only int attribute that returns the size of the outline file.

outline_file_timestamp

A read-only int attribute that returns the timestamp of the outline file.

outline_port_count

A read-only int attribute that returns the ports on the module.

outline_ref_count

A read-only string attribute that returns the ref count of the module.

n

See Also

- [get_modules](#)
- [get_attribute](#)
- [list_attributes](#)
- [set_attribute](#)

n

net_attributes

Description of the application defined net attributes.

Description

Attributes are properties assigned to objects such as pins, cells, and nets. They can be read-only or settable. A read-only attribute is informational; you cannot set its value. To modify a settable attribute, use the *set_attribute* command.

To determine the value of an attribute, use the *get_attribute* command. To see a list of all net attributes available, use the *list_attributes -application -class net* command.

Attributes for net

This section describes the net attributes.

activity_type

A read-only string attribute that returns the switching activity type of a net. The value is based on the current timer status; the attribute does not update the timer.

actual_related_clock

A read-only string attribute that returns the clock referenced by the toggle rate. The value is based on current timer status; the attribute does not update the timer. If there is no clock related to this net, it specifies the fastest clock for the current mode.

antenna_rule_name

A settable string attribute that returns the antenna rule associated with the net.

ba_resistance_max

A read-only float attribute that returns the back-annotated resistance on the net for maximum conditions, set with the `set_resistance` or `read_parasitics` command.

ba_resistance_min

A read-only float attribute that returns the back-annotated resistance of the net for minimum conditions, set with the `set_resistance` or `read_parasitics` command.

backside_max_layer_mode

A settable string attribute (with allowed values: `allow_pin_connection` `extract_only` `hard` `soft` `unknown`) that returns the backside maximum routing layer mode for the net. When set to `unknown`, this returns the default backside maximum routing layer mode for all nets in the block. When set to `unknown` and if this net type is clock, this returns the default backside maximum routing layer mode for clock nets, if available, in the block. This attribute is settable if and only if the default net layer modes are set. See `set_min_max_layer_modes`.

backside_max_layer_mode_soft_cost

A settable string attribute (with allowed values: `high` `low` `medium` `unknown`) that returns the backside maximum routing layer mode soft cost for the net. This attribute is settable if and only if the default net layer modes are set. See `set_min_max_layer_modes`.

backside_min_layer_mode

A settable string attribute (with allowed values: `allow_pin_connection` `extract_only` `hard` `soft` `unknown`) that returns the backside minimum routing layer mode for the net. When set to `unknown`, this returns the default backside minimum routing layer mode for all nets in the block. When set to `unknown` and if this net type is clock, this returns the default backside minimum routing layer mode for clock nets, if available, in the block. This attribute is settable if and only if the default net layer modes are set. See `set_min_max_layer_modes`.

backside_min_layer_mode_soft_cost

A settable string attribute (with allowed values: `high` `low` `medium` `unknown`) that returns the backside minimum routing layer mode soft cost for the net. This attribute is settable if and only if the default net layer modes are set. See `set_min_max_layer_modes`.

base_name

A read-only string attribute that returns the name of the net. The name does not include the instance path prefix. This attribute is based on the value of the settable *name* attribute.

bbox

A read-only rect attribute that returns the lower-left and upper-right corners of the bounding box of the net.

bounding_box

A read-only bounding_box collection attribute that returns the lower-left and upper-right corners of the bounding box of the net.

bundles

A read-only bundle collection attribute that returns the bundles that contain the net. A net can be in multiple bundles, a single bundle, or no bundle.

bus

A read-only net_bus collection attribute that returns the net bus for this net, if its is_bus_bit attribute is true.

busplan_stop_trace_net

A settable boolean attribute that returns True if the net should not be traced through for pipeline register planning.

color

A settable string attribute that returns the color of the port (except for lib-pin). This value must be a 24-bit RGB hexadecimal string prefaced by '#', for example: "#72c4f1" or "#ce8322".

constraint_groups

A read-only constraint_group collection attribute that returns the Custom Router constraint groups that contain the net.

dont_touch

A settable boolean attribute that indicates whether the net is excluded from optimization. When this attribute is *true*, the net is not modified or replaced during optimization.

dr_length

A read-only double attribute that returns the total routed length of the net. The calculation considers only the path and line shapes of the net. The value could be derived from global, track, or detailed routes, depending on the design's

routing state. This value is equal to the sum of the *dr_x_length* and *dr_y_length* attributes.

dr_x_length

A read-only double attribute that returns the horizontal routed length of the net. The calculation considers only the path and line shapes of the net. This could be derived from global, track, or detailed routes, depending on the design's routing state.

dr_y_length

A read-only double attribute that returns the vertical routed length of the net. The calculation considers only path and line shapes of the net. This could be derived from global, track, or detailed routes, depending on the design's routing state.

eco_net_group_id

A settable int attribute that is for net grouping of command `create_group_repeater_guidance` and `add_group_repeater`. They group nets based on their routing topology, signal direction, and routing layers automatically. The net group IDs are annotated on this attribute. You can also manually group nets by setting this attribute, then run `add_group_repeater -honor_user_net_grouping_id`.

escaped_full_name

A read-only string attribute that returns the full name of the net with any literal hierarchy characters escaped with a backslash.

estimate_timing_net_rule

A settable string attribute that returns the estimation rule for the net. An estimation rule is defined by the *set_net_estimation_rule* command and is applied to a net by setting this attribute. A net can have one rule or no rule. A rule can be associated with multiple nets. The *estimate_timing* command honors this attribute and uses the layer- and buffer-specific estimation rule to estimate the delay, capacitance, and transition time for nets with this attribute.

feedthrough_sharing_matching_id

A settable int attribute that is for feedthrough sharing among different nets. During design planning, `place_pins` or `optimize_topology_plans` only share the feedthroughs among the nets with same matching ID

forbidden_preroute_access_layers

A settable layer collection attribute that returns the layers on which the secondary PG router cannot access fixed prerouted wires or vias when routing the net.

frontside_max_layer_mode

A settable string attribute (with allowed values: allow_pin_connection extract_only hard soft unknown) that returns the frontside maximum routing layer mode for the net. When set to unknown, this returns the default frontside maximum routing layer mode for all nets in the block. When set to unknown and if this net type is clock, this returns the default frontside maximum routing layer mode for clock nets, if available, in the block. This attribute is settable if and only if the default net layer modes are set. See set_min_max_layer_modes.

frontside_max_layer_mode_soft_cost

A settable string attribute (with allowed values: high low medium unknown) that returns the frontside maximum routing layer mode soft cost for the net. This attribute is settable if and only if the default net layer modes are set. See set_min_max_layer_modes.

frontside_min_layer_mode

A settable string attribute (with allowed values: allow_pin_connection extract_only hard soft unknown) that returns the frontside minimum routing layer mode for the net. When set to unknown, this returns the default frontside minimum routing layer mode for all nets in the block. When set to unknown and if this net type is clock, this returns the default frontside minimum routing layer mode for clock nets, if available, in the block. This attribute is settable if and only if the default net layer modes are set. See set_min_max_layer_modes.

frontside_min_layer_mode_soft_cost

A settable string attribute (with allowed values: high low medium unknown) that returns the frontside minimum routing layer mode soft cost for the net. This attribute is settable if and only if the default net layer modes are set. See set_min_max_layer_modes.

full_name

A read-only string attribute that returns the full name of the net including a prefix for the instance path to this net.

glitch_rate

A read-only double attribute that returns the rate at which glitch transitions occur on the net, equal to glitch_count/power_simulation_time.

groups

A read-only group collection attribute that returns the groups the net belongs to.

is_analog

A read-only boolean attribute that returns whether this is an analog signal net.

is_bbt_object

A read-only boolean attribute that returns true if the net is a black-box timing object.

is_bus_bit

A read-only boolean attribute that returns true if the net is part of a net bus.

is_global

A read-only boolean attribute that returns True if the net is tied to logic 0 or logic 1.

is_ideal

A read-only boolean attribute that returns True if the net is ideal.

is_in_bundle

A read-only boolean attribute that returns True if the net is part of a net bundle.

is_physical

A read-only boolean attribute that returns True if the net is a physical net.

is_rdl

A read-only boolean attribute that returns True if the net is a redistribution layer (RDL) net.

is_shadow

A read-only boolean attribute that returns True if the net is a shadow net. A shadow net is created when you push down routes from the top level or create feedthroughs on a block.

is_tie_high_net

A read-only boolean attribute that returns True if the net is tied to logic 1.

is_tie_low_net

A read-only boolean attribute that returns True if the net is tied to logic 0 or logic 1.

is_user_pg

A settable boolean attribute that returns True if the net is marked as a user-created PG net. When you read a Verilog netlist, the tool marks the PG and tie nets read from the netlist as user-created.

leaf_loads

A read-only port, or pin collection attribute that contains the leaf load pins and ports of a net.

major_activity_type

A read-only string attribute that returns the major switching activity type of the net.

max_layer

A settable layer collection attribute that returns the maximum routing layer for the net.

max_layer_is_user

A settable boolean attribute that returns true if the maximum routing layer is user-generated. If this attribute is *false*, the maximum routing layer is tool-generated.

max_layer_mode

A settable string attribute that specifies the maximum routing layer mode for the net. Valid values are *allow_pin_connection*, *extract_only*, *hard*, *soft*, and *unknown*. This attribute is not settable when the frontside and backside layer modes are set. See *set_min_max_layer_modes*.

max_layer_mode_soft_cost

A settable string attribute that specifies the maximum routing layer mode soft cost for the net. Valid values are *high*, *low*, *medium*, and *unknown*. This attribute is not settable when the frontside and backside layer modes are set. See *set_min_max_layer_modes*.

max_layer_name

A read-only string attribute that returns the maximum routing layer constraint for the net.

min_layer

A settable layer collection attribute that returns the minimum routing layer for the net.

min_layer_is_user

A settable boolean attribute that returns true if the minimum routing layer is user-generated. If this attribute is *false*, the minimum routing layer is tool-generated.

min_layer_mode

A settable string attribute that specifies the minimum routing layer mode for the net. Valid values are *allow_pin_connection*, *extract_only*, *hard*, *soft*, and *unknown*. This attribute is not settable when the frontside and backside layer modes are set. See *set_min_max_layer_modes*.

min_layer_mode_soft_cost

A settable string attribute that specifies the minimum routing layer mode soft cost for the net. Valid values are *high*, *low*, *medium*, and *unknown*. This attribute is not settable when the frontside and backside layer modes are set. See *set_min_max_layer_modes*.

min_layer_name

A read-only string attribute that returns the minimum routing layer constraint for the net.

minor_activity_type

A read-only string attribute that returns the minor switching activity type of the net.

name

A settable string attribute that returns the name of the net. The *base_name* and *full_name* attributes are derived from the value of this attribute.

net_type

A settable string attribute (with allowed values: *analog_ground* *analog_power* *analog_signal* *clock* *deep_nwell* *deep_pwell* *ground* *nwell* *power* *pwell* *reset* *scan* *signal* *tie_high* *tie_low* *unset*) that returns the type of the net.

net_type_policy

A settable string attribute (with allowed values: *auto* *override*) that returns the *net_type* setting policy of the net. If set to 'auto', then *net_type* will be set to *analog_signal*, *analog_ground* or *analog_power* if net is connected to *lib_pin* having liberty attribute "is_analog" true and *port_type* is signal, ground or power respectively. If set to 'override' then *net_type* will not be set automatically and existing *net_type* will prevail. *net_type* of PG and tie nets will not be changed due to analog pin connection.

number_of_flat_drivers

A read-only int attribute that returns the number of top-level ports and leaf pins that drive the flat net of this net.

number_of_flat_inouts

A read-only int attribute that returns the number of top-level ports and leaf pins on the flat net of this net that have an inout direction. These are considered to be both drivers and loads of the flat net and are included in the counts for the *number_of_flat_drivers* and *number_of_flat_loads* attributes.

number_of_flat_loads

A read-only int attribute that returns the number of top-level ports and leaf pins driven by the flat net of this net.

number_of_pins

A read-only int attribute that returns the number of ports and pins connected to this net.

number_of_wires

A read-only int attribute that returns the number of wires on this net. A wire is defined by two points and a width. For example, if the path shape has a centerline with N points, it consists of N-1 wires.

object_class

A read-only string attribute that returns the name of this object class. Returns 'net' for net objects.

parent_block

A read-only block collection attribute that returns the owner block reference for this net.

parent_block_cell

A read-only cell collection attribute that returns the owner block instance for this net in the physical hierarchy. This attribute is defined only for nets inside a physical hierarchy. It is undefined for all top-level nets.

parent_cell

A read-only cell collection attribute that returns the owner instance for this net in the logic hierarchy. This attribute is defined only for nets inside a logic hierarchy. It is undefined for top-level nets.

physical_hierarchy_level

A read-only int attribute that specifies the depth of the physical hierarchy.

physical_status

A settable string attribute (with allowed values: locked minor_change unrestricted) that returns the physical modification status of the net routes. If

'locked', the router cannot reroute this net. If 'minor_change', the router can make minor changes to this net's routes. If 'unrestricted', the router can freely reroute this net.

pin_bbox

A read-only rect attribute that returns the lower-left and upper-right corners of the bounding box that fully contains all pins and terminals connected to the net.

pin_bounding_box

A read-only bounding_box collection attribute that returns the lower-left and upper-right corners of the bounding box that fully contains all pins and terminals connected to the net.

rc_hold_criticality

A settable string attribute (with allowed values: high low medium none) that returns the hold time criticality preference of the net. The router uses this attribute to decide whether the net should be routed on lower RC layers for hold time optimization.

rc_setup_criticality

A settable string attribute (with allowed values: high low medium none) that returns the setup time criticality preference of the net. The router uses this attribute to decide whether the net should be routed on lower RC layers for setup time optimization.

related_clock

A read-only string attribute that returns the clock related to this net. If the net does not have a related clock, the attribute has a value of *NONE*. The value is based on current timer status; the attribute does not update the timer.

route_length

A read-only string attribute that returns the route length for each layer, in the format {{layer1 length1} {layer2 length2}. ..}. The route length calculation considers only the path and line shapes of the net. The sum of the per-layer lengths in this attribute equals the value specified in the dr_length net attribute.

routing_blockage_group_id

A settable int attribute that returns the ID of the routing blockage group associated with this net. The value must be between 0 and 8, inclusive. The default is 0, which means that the net is not associated with a routing blockage group. If this attribute has a nonzero value, the router honors all routing blockages that belong to the specified routing group when routing this net. Note that setting this attribute is the only way to associate specific nets with specific routing blockages.

routing_blockages

A read-only `routing_blockage` collection attribute that returns the routing blockages that have the same group ID as the `routing_blockage_group_id` attribute of this net. It does not include net-type-based routing blockages that affect this net.

routing_corridor

A read-only `routing_corridor` collection attribute that returns the routing corridors associated with this net.

routing_rule

A read-only string attribute that returns the name of the routing rule assigned to this net. If the net is assigned the default rule, the attribute has a value of *default*.

shadow_status

A settable string attribute (with allowed values: `copied_down` `copied_up` `normal` `pulled_up` `pushed_down` `virtual_flat`) that returns the shadow status if the `is_shadow` attribute is true.

size_only

A read-only boolean attribute that returns whether the net is restricted to size-only optimization.

skip_antenna_fixing

A settable boolean attribute that returns whether or not the router should skip fixing the net to comply with antenna rules.

static_probability

A read-only double attribute that returns the switching activity probability of the net. The value is based on the current timer status; the attribute does not update the timer.

stop_transit_cell

A settable boolean attribute that, when set, avoids buffers and inverters on this net from being classified as transit cells for a boundary net of a safety core.

subchannel_index

A read-only int attribute that specifies the index of subchannel which the net belongs to. It is only used in interposer bus routing. The default is -1. Sideband signals of UCle interface have their `subchannel_index` set to -2.

switching_power

A read-only double attribute that returns the switching power for the net. The value is based on the current timer status; the attribute does not update the timer.

synonyms

A settable string attribute that returns the synonym names of the net. An empty string indicates that the net has no synonyms. Synonyms allow the net to be referenced by one of its synonym names.

toggle_rate

A read-only double attribute that returns the switching activity toggle rate of the net. The value is based on the current timer status, the attribute does not update the timer.

top_block

A read-only block collection attribute that returns the top-level block object for the net.

topological_constraints

A read-only topological_constraint collection attribute that returns the Custom Router topological constraints associated with the net.

total_capacitance_max

A read-only float attribute that returns the sum of all pin capacitances and wire capacitance of the net for the max condition in the current_scenario. Uses the worst of the rise and fall max condition pin capacitances on each pin driven by the net. Value is returned for the current_corner.

total_capacitance_min

A read-only float attribute that returns the sum of all pin capacitances and wire capacitance of the net for the min condition in the current_scenario. Uses the best of the rise and fall min condition pin capacitances on each pin driven by the net. Value is returned for the current_corner.

total_ccs_capacitance_max_fall

A read-only float attribute that returns the sum of the wire capacitance and the max condition fall capacitance of the CCS receiver pins in the current_corner.

total_ccs_capacitance_max_rise

A read-only float attribute that returns the sum of the wire capacitance and the max condition rise capacitance of the CCS receiver pins in the current_corner.

total_ccs_capacitance_min_fall

A read-only float attribute that returns the sum of the wire capacitance and the min condition fall capacitance of the CCS receiver pins in the *current_corner*.

total_ccs_capacitance_min_rise

A read-only float attribute that returns the sum of the wire capacitance and the min condition rise capacitance of the CCS receiver pins in the *current_corner*.

user_dont_touch

A settable boolean attribute that indicates whether the *dont_touch* attribute is user-specified (*true*) or tool-generated (*false*).

voltage_range_max

A read-only float attribute that returns the maximum logic-one voltage of a net across all corners. For signal nets, it is based on the output voltage of the net's drivers. For power nets, it is based on the voltage set on the corresponding supply net.

voltage_range_min

A read-only float attribute that returns the minimum logic-one voltage of a net across all corners. For signal nets, it is based on the output voltage of the net's drivers. For power nets, it is based on the voltage set on the corresponding supply net.

vr_length

A read-only double attribute that returns the total virtual route length of the net. The calculation traverses all levels of the physical hierarchy and considers blockages. To disable one or more blocks for planning, use the *set_hierarchy_options* command. The value of this attribute is the same as the value returned by the *get_estimated_wirelength* command for the net. This value is equal to the sum of the *vr_x_length* and *vr_y_length* attributes.

vr_x_length

A read-only double attribute that returns the horizontal virtual route length of the net. The calculation traverses all levels of the physical hierarchy and considers blockages. To disable one or more blocks for planning, use the *set_hierarchy_options* command.

vr_y_length

A read-only double attribute that returns the vertical virtual route length of the net. The calculation traverses all levels of the physical hierarchy and considers blockages. To disable one or more blocks for planning, use the *set_hierarchy_options* command.

wire_capacitance_max

A read-only float attribute that returns the wire capacitance of the net for the max condition in the current_corner.

wire_capacitance_min

A read-only float attribute that returns the wire capacitance of the net for the min condition in the current_corner.

See Also

- [get_attribute](#)
- [list_attributes](#)

net_bus_attributes

Description of the application defined net_bus attributes.

Description

Attributes are properties assigned to objects such as pins, cells, and nets. They can be read-only or settable. A read-only attribute is informational; you cannot set its value. To modify a settable attribute, use the *set_attribute* command.

To determine the value of an attribute, use the *get_attribute* command. To see a list of all net_bus attributes available, use the *list_attributes -application -class net_bus* command.

Attributes for net_bus

This section describes the net_bus attributes.

bit_order

A read-only string attribute (with allowed values: down up) that returns the bit order of a net bus. It is *down* when *end* is less than *start* else it is *up*.

block

A read-only block collection attribute that returns the block of a net bus.

color

A settable string attribute that returns the color of the net bus. This value must be a 24-bit RGB hexadecimal string prefaced by '#', for example: "#72c4f1" or "#ce8322".

end

A read-only int attribute that returns the end index of a net bus.

full_name

A read-only string attribute that returns the name of a net bus.

name

A read-only string attribute that returns the name of a net bus.

nets

A read-only net collection attribute that returns the nets of a net bus.

object_class

A read-only string attribute that returns the name of this object class. Returns 'net_bus' for net_bus objects.

start

A read-only int attribute that returns the start index of a net bus.

See Also

- [get_attribute](#)
- [list_attributes](#)

net_estimation_rule_attributes

Description of the application defined net_estimation_rule attributes.

Description

Attributes are properties assigned to objects such as pins, cells, and nets. They can be read-only or settable. A read-only attribute is informational; you cannot set its value. To modify a settable attribute, use the *set_attribute* command.

To determine the value of an attribute, use the *get_attribute* command. To see a list of all net_estimation_rule attributes available, use the *list_attributes -application -class net_estimation_rule* command.

Attributes for net_estimation_rule

This section describes the net_estimation_rule attributes.

buffer

A read-only string attribute that returns buffer.

capture_block_budget

A read-only float attribute that returns the percent of sdc path reserved for capture block delay.

clock

A read-only string attribute that returns the clock in current mode.

corner

A read-only string attribute that returns the corner.

delay_derate

A read-only float attribute that returns the derate amount for the delay.

full_name

A read-only string attribute that returns the full name of this object.

h_buffer_pattern

A read-only string attribute that returns the horizontal buffer pattern.

h_buffer_spacing

A read-only float attribute that returns the horizontal buffer spacing.

h_bump_pitch

A read-only float attribute that returns the horizontal bump pitch.

h_ff_per_mm

A read-only float attribute that returns the horizontal ff_per_mm.

h_layer

A read-only string attribute that returns the horizontal layer.

h_ns_per_mm

A read-only float attribute that returns the horizontal ns_per_mm.

h_ohms_per_mm

A read-only float attribute that returns the horizontal ohms_per_mm.

h_register_pattern

A read-only string attribute that returns the horizontal register pattern.

h_register_spacing

A read-only float attribute that returns the horizontal register_spacing.

h_signal_pitch

A read-only float attribute that returns the horizontal signal pitch.

h_utilization

A read-only float attribute that returns the horizontal utilization.

h_xtalk_fraction

A read-only float attribute that returns the horizontal xtalk capacitance correction.

launch_block_budget

A read-only float attribute that returns the percent of sdc path reserved for launch block delay.

name

A read-only string attribute that returns the name of this object.

ndr

A read-only string attribute that returns ndr.

object_class

A read-only string attribute that returns the name of this object class. Returns 'net_estimation_rule' for net_estimation_rule objects.

register

A read-only string attribute that returns register.

sdc_stage_delay

A read-only float attribute that returns the clock period + clock uncertainty.

stage_margin

A read-only float attribute that returns the margin to account for skew and OCV at registers.

v_buffer_pattern

A read-only string attribute that returns the vertical buffer pattern.

v_buffer_spacing

A read-only float attribute that returns the vertical buffer spacing.

v_bump_pitch

A read-only float attribute that returns the vertical bump pitch.

v_ff_per_mm

A read-only float attribute that returns the vertical ff per mm.

o

v_layer

A read-only string attribute that returns the vertical layer.

v_ns_per_mm

A read-only float attribute that returns the vertical ns_per_mm.

v_ohms_per_mm

A read-only float attribute that returns the vertical ohms_per_mm.

v_register_pattern

A read-only string attribute that returns the vertical register pattern.

v_register_spacing

A read-only float attribute that returns the vertical register_spacing.

v_signal_pitch

A read-only float attribute that returns the vertical signal pitch.

v_utilization

A read-only float attribute that returns the vertical utilization.

v_xtalk_fraction

A read-only float attribute that returns the vertical xtalk capacitance correction.

See Also

- [get_attribute](#)
- [list_attributes](#)
- [set_attribute](#)

o

outline_attributes

Describes the predefined attributes for outline modules.

Description

Attributes are properties assigned to objects such as pins, cells and nets. Definitions for outline attributes are provided in the following subsections.

o

Attributes are read-only unless marked as settable. Read-only attributes are informational, and you cannot set its value. A settable attribute can be modified with the *set_attribute* command.

To determine the value of an attribute, use the *get_attribute* command. To see a list of all attributes available for a class of objects, use the *list_attributes -application* command.

Outline Attributes

is_outline

Specifies a Boolean attribute. The value is true, if this module is an outline module.

outline_file_name

Specifies a string attribute for the path name to the Verilog source file.

outline_file_line_start

Specifies an integer attribute for the starting line number of this module in the Verilog source file.

outline_file_line_end

Specifies an integer attribute for the ending line number of this module in the Verilog source file.

outline_file_size

Specifies an integer attribute for the size in bytes of the Verilog source file.

outline_file_timestamp

Specifies an integer attribute for the timestamp of the Verilog source file.

outline_port_count

Specifies an integer attribute for the number of ports on this module.

outline_ref_count

Specifies a list attribute. It contains information on the leaf cells in the design, but not instantiated in the outline design. The attribute value is a list of name-integer value pairs, where the name is the name of the leaf cell reference, and the integer value is the number of instances of this reference the full design module would have.

See Also

- [expand_outline](#)
- [get_attribute](#)

o

- [list_attributes](#)
- [read_verilog_outline](#)

output_delay_attributes

Description of the application defined output_delay attributes.

Description

Attributes are properties assigned to objects such as pins, cells, and nets. They can be read-only or settable. A read-only attribute is informational; you cannot set its value. To modify a settable attribute, use the *set_attribute* command.

To determine the value of an attribute, use the *get_attribute* command. To see a list of all output_delay attributes available, use the *list_attributes -application -class output_delay* command.

Attributes for output_delay

This section describes the output_delay attributes.

clock_name

A read-only string attribute that returns the name of the relative clock.

file_line_info

A read-only string attribute that returns the value containing one or more Tcl or SDC source file name and line number pairs of the commands which were used to create this object.

full_name

A read-only string attribute that returns a unique name for the output delay object.

is_clock_fall

A read-only boolean attribute that returns true if this output_delay is relative to the falling edge of the clock, false otherwise.

is_level_sensitive

A read-only boolean attribute that returns true if this output_delay represents a level-sensitive startpoint, false otherwise.

is_network_latency_included

A read-only boolean attribute that returns true if this output_delay includes clock network latency, false otherwise.

o

is_source_latency_included

A read-only boolean attribute that returns true if this output_delay includes clock source latency, false otherwise.

max_fall

A read-only float attribute that returns the maximum fall delay value.

max_rise

A read-only float attribute that returns the maximum rise delay value.

min_fall

A read-only float attribute that returns the minimum fall delay value.

min_rise

A read-only float attribute that returns the minimum rise delay value.

object_class

A read-only string attribute that returns the name of this object class. Returns 'output_delay' for output_delay objects.

See Also

- [set_output_delay](#)
- [get_attribute](#)
- [list_attributes](#)

overlap_blockage_attributes

Description of the application defined overlap_blockage attributes.

Description

Attributes are properties assigned to objects such as pins, cells, and nets. They can be read-only or settable. A read-only attribute is informational; you cannot set its value. To modify a settable attribute, use the *set_attribute* command.

To determine the value of an attribute, use the *get_attribute* command. To see a list of all overlap_blockage attributes available, use the *list_attributes -application -class overlap_blockage* command.

Attributes for overlap_blockage

This section describes the overlap_blockage attributes.

o

area

A read-only area attribute that returns the area of the `overlap_blockage`.

bbox

A read-only rect attribute that returns the bounding-box of the `overlap_blockage`. The format of a rectangle specification is `{{llx lly} {urx ury}}`, which specifies the lower-left and upper-right corners of the rectangle.

boundary

A read-only `coord_list` attribute that returns the boundary of the `overlap_blockage`. The format of a `coordinate_list` specification is `{{x1 y1} {x2 y2}...}`.

bounding_box

A read-only `bounding_box` collection attribute that returns the bounding-box of the `overlap_blockage`. The format of the collection specifies lower-left and upper-right corners of the rectangle.

full_name

A read-only string attribute that returns the name of the `overlap_blockage`.

groups

A read-only group collection attribute that returns the groups in which this `overlap_blockage` is a member.

name

A read-only string attribute that returns the name of the `overlap_blockage`.

object_class

A read-only string attribute that returns the name of this object class. Returns 'overlap_blockage' for `overlap_blockage` objects.

overlap_layer_name

A read-only string attribute that returns the name of the layer on which this `overlap_blockage` exists.

parent_block

A read-only block collection attribute that returns the owner block object for this `overlap_blockage`.

parent_block_cell

A read-only cell collection attribute that returns the parent block cell object for this `overlap_blockage`.

p

parent_cell

A read-only cell collection attribute that returns the owner cell object for this overlap_blockage.

top_block

A read-only block collection attribute that returns the top block object for this overlap_blockage.

See Also

- [get_attribute](#)
- [list_attributes](#)

p

parasitic_tech_attributes

Description of the application defined parasitic_tech attributes.

Description

Attributes are properties assigned to objects such as pins, cells, and nets. They can be read-only or settable. A read-only attribute is informational; you cannot set its value. To modify a settable attribute, use the *set_attribute* command.

To determine the value of an attribute, use the *get_attribute* command. To see a list of all parasitic_tech attributes available, use the *list_attributes -application -class parasitic_tech* command.

Attributes for parasitic_tech

This section describes the parasitic_tech attributes.

canonical_file_name

A read-only string attribute that returns the absolute path of the source file used to create this parasitic_tech.

full_name

A read-only string attribute that returns the full name of this parasitic_tech.

itf_technology_name

A read-only string attribute that returns the technology name of this parasitic_tech.

layer_map_file_name

A read-only string attribute that returns the layer mapping file used to create this `parasitic_tech`.

library_name

A read-only string attribute that returns the name of library of this `parasitic_tech`.

library_parasitic_name

A read-only string attribute that returns the library and parasitic name in `lib_name:parasitic_name` format.

name

A read-only string attribute that returns the name of a `parasitic_tech`.

object_class

A read-only string attribute that returns the name of this object class. Returns 'parasitic_tech' for `parasitic_tech` objects.

See Also

- [get_attribute](#)
- [list_attributes](#)
- [set_attribute](#)

path_group_attributes

Description of the application defined `path_group` attributes.

Description

Attributes are properties assigned to objects such as pins, cells, and nets. They can be read-only or settable. A read-only attribute is informational; you cannot set its value. To modify a settable attribute, use the `set_attribute` command.

To determine the value of an attribute, use the `get_attribute` command. To see a list of all `path_group` attributes available, use the `list_attributes -application -class path_group` command.

Attributes for path_group

This section describes the `path_group` attributes.

p

command_text

A read-only string attribute that returns the equivalent commands that can re-create the object.

file_line_info

A read-only string attribute that returns the Tcl or SDC source file name and line number pairs of the commands, which are used to create this object, if it exists.

full_name

A read-only string attribute that returns a unique name for the path_group.

mode

A read-only collection attribute that returns the mode that contains the path_group.

object_class

A read-only string attribute that returns the name of this object class. Returns 'path_group' for path_group objects.

weight

A read-only float attribute that returns Path_groups. Specifies the weight value of the group.

See Also

- [get_path_groups](#)
- [get_attribute](#)
- [list_attributes](#)
- [report_path_groups](#)

pg_region_attributes

Description of the application defined pg_region attributes.

Description

Attributes are properties assigned to objects such as pins, cells, and nets. They can be read-only or settable. A read-only attribute is informational; you cannot set its value. To modify a settable attribute, use the *set_attribute* command.

To determine the value of an attribute, use the *get_attribute* command. To see a list of all pg_region attributes available, use the *list_attributes -application -class pg_region* command.

Attributes for pg_region

This section describes the pg_region attributes.

bbox

A read-only rect attribute that returns the bounding-box of a pg_region. The format of a rectangle specification is {{llx lly} {urx ury}}, which specifies the lower-left and upper-right corners of the rectangle.

boundary

A settable coord_list attribute that returns the boundary of this pg_region. The format of a boundary specification is { {{llx1 lly1} {urx1 ury1}} {{llx2 lly2} {urx2 ury2}}...}.

bounding_box

A read-only bounding_box collection attribute that returns the bounding-box of a pg_region. The format of the collection specifies lower-left and upper-right corners of the rectangle.

full_name

A read-only string attribute that returns the full name of a pg_region.

groups

A read-only group collection attribute that returns the groups the voltage area shape object belongs to.

in_edit_group

A read-only boolean attribute that returns whether the pg_region belongs to an edit group.

is_shadow

A read-only boolean attribute that returns whether a pg_region is pushed down from above.

name

A settable string attribute that returns the name of a pg_region.

object_class

A read-only string attribute that returns the name of this object class. Returns 'pg_region' for pg_region objects.

parent_block

A read-only block collection attribute that returns the owner block object for this pg_region.

p

parent_block_cell

A read-only cell collection attribute that returns the parent block cell object for this pg_region.

parent_cell

A read-only cell collection attribute that returns the owner cell object for this pg_region.

physical_status

A settable string attribute (with allowed values: locked unrestricted) that returns the value of physical modification status for the pg_region object.

region

A settable coord_list attribute that returns the boundary of this pg_region. The format of a boundary specification is { {{llx1 lly1} {urx1 ury1}} {{llx2 lly2} {urx2 ury2}}...}.

shadow_status

A settable string attribute (with allowed values: copied_down copied_up normal pulled_up pushed_down virtual_flat) that returns the shadow status of a pg_region.

top_block

A read-only block collection attribute that is the top block object for this pg_region.

See Also

- [create_pg_region](#)
- [get_attribute](#)
- [list_attributes](#)
- [set_attribute](#)

physical_constraint_attributes

Lists the predefined attributes for physical constraints.

Description

Attributes are properties assigned to objects such as pins, cells and nets. Definitions for physical constraint attributes are provided in the subsections that follow.

p

Attributes are read-only unless marked as settable. Read-only attributes are informational, and you cannot set its value. A settable attribute can be modified with the *set_attribute* command.

To determine the value of an attribute, use the *get_attribute* command. To see a list of all attributes available for a class of objects use the *list_attributes -application* command.

Physical Constraint Attributes

density_type

A read-only string attribute (with allowed values: global gradient local) for the type of density calculation for physical constraint of type density.

directions

A read-only name list attribute (with allowed values: diagonal horizontal orthogonal vertical) for the list of directions for checking the physical constraint.

full_name

A read-only string attribute for the full name of the physical constraint.

layers

A read-only name list attribute of the list of layer name of the physical constraint.

max

A read-only string attribute for the maximum value of the physical constraint.

min

A read-only string attribute for the minimum value of the physical constraint.

name

A read-only string attribute for the name of the physical constraint.

object_class

A read-only string attribute with the value of "physical_constraint".

operand_specifications

A read-only name list attribute of the list of operand specification of the physical constraint. Operand specification is set of design objects on which constraint is set.

tolerance

A read-only string attribute for the tolerance value of the physical constraint.

p

type

A read-only string attribute (with allowed values: alignment area aspect_ratio count density diagonal enclosure gap_area height overlap parallel_run_length pitch spacing width) for the physical constraint type.

window_size

A read-only distance list attribute for the the window size of the physical constraint.

window_step

A read-only distance list attribute for the the window step of the physical constraint.

See Also

- [get_physical_constraints](#)
- [get_attribute](#)
- [list_attributes](#)
- [set_attribute](#)

pin_attributes

Description of the application defined pin attributes.

Description

Attributes are properties assigned to objects such as pins, cells, and nets. They can be read-only or settable. A read-only attribute is informational; you cannot set its value. To modify a settable attribute, use the *set_attribute* command.

To determine the value of an attribute, use the *get_attribute* command. To see a list of all pin attributes available, use the *list_attributes -application -class pin* command.

Attributes for pin

This section describes the pin attributes.

activity_type

A read-only string attribute that returns the switching activity type of a pin. The valid values are

```
simulated: Switching activity is retrieved from simulation files
annotated: The switching activity is annotated by the user
derived: The switching activity is retrieved from constraints
```

p

calculated: The switching activity is computed by activity propagation
 default: This value represents all other sources of switching activity

The value is based on the current timer status; the attribute does not update the timer.

actual_fall_transition_max

A read-only float attribute that returns the largest max condition falling transition time for the pin. The timer must be up-to-date to query this attribute value. The value is for the current_scenario only.

actual_fall_transition_min

A read-only float attribute that returns the smallest min condition falling transition time for the pin. The timer must be up-to-date to query this attribute value. The value is for the current_scenario only.

actual_related_clock

A read-only string attribute that returns a clock by which the toggle rate value is referenced. The value is based on current timer status, the attribute does not update the timer. If there is no clock related to this pin, it gives the fastest clock for the current mode..

actual_rise_transition_max

A read-only float attribute that returns the largest max condition rising transition time for the pin. The timer must be up-to-date to query this attribute value. The value is for the current_scenario only.

actual_rise_transition_min

A read-only float attribute that returns the smallest min condition rising transition time for the pin. The timer must be up-to-date to query this attribute value. The value is for the current_scenario only.

allow_diode_insertion

A settable boolean attribute that indicates whether diode insertion is to be skipped on the pin.

antenna_area

A settable string attribute that returns the antenna area.

antenna_side_area

A settable string attribute that returns the antenna side area.

arrival_window

A read-only string attribute that returns the minimum and maximum arrivals for rise and fall transitions. To get the `arrival_window` attribute on pins that are not endpoints, set the `timing_save_pin_arrival_and_slack` variable to true. Provides the arrival time of a signal at a pin or port on the datapath. It includes the clock name and the active edge associated with the signal, and the minimum and maximum rise or fall arrival times at the clock pin, based on the active edge of the clock. If multiple clocks or multiple active edges of the same clock launch a signal to this pin, the tool reports a separate arrival window for each clock or clock edge. The arrival windows for each clock signal or active edge are returned as a list of lists from this attribute. Example: {{{clk} pos_edge {min_r_f -- --} {max_r_f 2.32464 2.3004}}} {{v_clk} pos_edge {min_r_f -- --} {max_r_f 1.18975 1.07728}}}

bbox

A read-only rect attribute that returns the bounding box of the pin as a rectangle.

boundary_optimization

A read-only string attribute that indicates whether boundary optimization is allowed if enabled.

bounding_box

A read-only `bounding_box` collection attribute that returns the bounding-box of a pin. The format of the collection specifies lower-left and upper-right corners of the rectangle.

bump_access_dir

A settable `bump_access_dir` attribute that returns the accessible directions on a bump pin. This attribute is only defined for pins that are on a bump cell. For octagonal bump pins, there are eight possible access directions: N (North), NE (North East), E (East), SE (South East), S (South), SW (South West), W (West), and NW (North West). For rectangular bump pins, only the 90-degree edges, N (North), E (East), S (South), W (West) are available. There is also a "ALL" and "NONE" setting to enable and disable all directions. Note that "ALL" means 8 directions for octagonal bump pins and 4 directions for rectangular bump pins.

bus

A read-only `pin_bus` collection attribute that returns the pin bus for this pin, if its `is_bus_bit` attribute is *true*.

case_value

A read-only string attribute that specifies the value of case on this pin. The possible values are "0" or "1". This value may come from user defined case or

p

netlist defined constant value. The value may be because a case value was directly set on this pin or because the case value was propagated through logic to this pin. If there is no case assigned or propagated to this pin this attribute is not defined.

ccd_max_postpone

A settable float attribute that returns the user-specified maximum postpone value for concurrent clock and data optimization (CCD).

ccd_max_prepone

A settable float attribute that returns the user-specified maximum prepone value for concurrent clock and data optimization (CCD).

cell

A read-only cell collection attribute that returns the instance owning this pin.

clock_arrival_window

A read-only string attribute that provides the arrival time of a clock signal at a pin or port on the clock network. It includes the clock name, the active edge type, and the minimum and maximum rise or fall arrival times at the clock pin, based on the active edge of the clock. If multiple clocks or multiple active edges of the same clock arrive at the pin, the tool reports a separate arrival window for each clock or clock edge. The arrival windows for each clock signal or active edge are returned as a list of lists from this attribute. For example: {{{SYS_CLK} pos_edge {min_r_f 1.03046 --} {max_r_f 1.03067 --}} {{REF_CLK} pos_edge {min_r_f 1.05123 --} {max_r_f 1.05742 --}}}

clock_latency_fall_max

A read-only float attribute that returns the user-specified clock network latency for max fall. This attribute is only defined on those pins where *set_clock_latency* has been directly set.

clock_latency_fall_min

A read-only float attribute that returns the user-specified clock network latency for min fall. This attribute is only defined on those pins where *set_clock_latency* has been directly set.

clock_latency_offset_fall_max

A read-only float attribute that returns the value set on this pin using the 'set_clock_latency -offset -fall -max' command. Clock latency offsets can only be set on clock sink pins and ICG clock inputs, and are used to do ideal clock network modeling of a latency offset between the entire clock tree and that particular pin.

clock_latency_offset_fall_min

A read-only float attribute that returns the value set on this pin using the 'set_clock_latency -offset -fall -min' command. Clock latency offsets can only be set on clock sink pins and ICG clock inputs, and are used to do ideal clock network modeling of a latency offset between the entire clock tree and that particular pin.

clock_latency_offset_rise_max

A read-only float attribute that returns the value set on this pin using the 'set_clock_latency -offset -rise -max' command. Clock latency offsets can only be set on clock sink pins and ICG clock inputs, and are used to do ideal clock network modeling of a latency offset between the entire clock tree and that particular pin.

clock_latency_offset_rise_min

A read-only float attribute that returns the value set on this pin using the 'set_clock_latency -offset -rise -min' command. Clock latency offsets can only be set on clock sink pins and ICG clock inputs, and are used to do ideal clock network modeling of a latency offset between the entire clock tree and that particular pin.

clock_latency_rise_max

A read-only float attribute that returns the user-specified clock network latency for max rise. This attribute is only defined on those pins where *set_clock_latency* has been directly set.

clock_latency_rise_min

A read-only float attribute that returns the user-specified clock network latency for min rise. This attribute is only defined on those pins where *set_clock_latency* has been directly set.

clock_source_latency_early_fall_max

A read-only float attribute that returns the user-specified clock network latency for early max fall. This attribute is only defined on those pins where *set_clock_latency -source* has been directly set.

clock_source_latency_early_fall_min

A read-only float attribute that returns the user-specified clock network latency for early min fall. This attribute is only defined on those pins where *set_clock_latency -source* has been directly set.

p

clock_source_latency_early_rise_max

A read-only float attribute that returns the user-specified clock network latency for early max rise. This attribute is only defined on those pins where *set_clock_latency -source* has been directly set.

clock_source_latency_early_rise_min

A read-only float attribute that returns the user-specified clock network latency for early min rise. This attribute is only defined on those pins where *set_clock_latency -source* has been directly set.

clock_source_latency_late_fall_max

A read-only float attribute that returns the user-specified clock network latency for late max fall. This attribute is only defined on those pins where *set_clock_latency -source* has been directly set.

clock_source_latency_late_fall_min

A read-only float attribute that returns the user-specified clock network latency for late min fall. This attribute is only defined on those pins where *set_clock_latency -source* has been directly set.

clock_source_latency_late_rise_max

A read-only float attribute that returns the user-specified clock network latency for late max rise. This attribute is only defined on those pins where *set_clock_latency -source* has been directly set.

clock_source_latency_late_rise_min

A read-only float attribute that returns the user-specified clock network latency for late min rise. This attribute is only defined on those pins where *set_clock_latency -source* has been directly set.

clocks

A read-only collection attribute that returns a collection of clock objects that this pin is in the clock network of. The clocks in the collection are just those clocks that this pin fans out directly from the clock sources. It does not include the master clocks of any generated clock networks that this pin is in.

clocks_with_sense

A read-only string attribute that returns a string attribute in the form of a list of clock names and clock senses this pin is in the clock network of. The possible clock senses are: "positive", "negative", "rise_triggered_high_pulse", "fall_triggered_high_pulse",

p

"rise_triggered_low_pulse", "fall_triggered_low_pulse". For example, the following script:

```
foreach_in_collection pin $pinList {
    set pinName [get_attribute $pin full_name]
    set senses [ get_attribute $pin clocks_with_sense]
    echo "For pin: " $pinName " " $senses;
}
```

Might produce output such as:

```
For pin: mux/A { { "clk" "negative" } }
For pin: mux/B { { "clk" "positive" } }
For pin: mux/Z { { "clk" "negative" } { "clk" "positive" } }
```

constant_propagation

A read-only boolean attribute that indicates whether constant optimization is allowed if enabled.

constant_value

A read-only string attribute that returns the constant value on this pin. The possible values are "0" or "1". Constant values are caused by a logic constant in the netlist or by a library cell such as a pullup or pulldown having a constant logic function. A constant value is on the initial pin and on any pins that have their case propagated as a result of that. For pins with the attribute `constant_value` defined, the attribute `case_value` will also be defined and have the same value.

constraining_max_transition

A read-only float attribute that returns the most constraining maximum transition value from all sources at the pin. This is the actual max transition constraints enforced by the tool at this pin.

constraint_groups

A read-only `constraint_group` collection attribute that specifies the Custom Router constraint groups that contain the pin.

cts_fixed_balance_pin

A settable boolean attribute that guides CCD to not derive useful skew offsets and preserve the user balance point, if any.

diode_protection

A settable string attribute that returns a string value which is the diode protection value for the pin.

p

direction

A read-only string attribute that returns the direction of a pin. Possible values are *in*, *out*, *inout*, or *unknown*.

disable_timing

A read-only boolean attribute that returns true if the pin has been marked as *disable_timing* by the *set_disable_timing* command.

dont_estimate_clock_latency

A settable boolean attribute that marks clock gate clock pins which are to be excluded from receiving a clock latency from the estimate clock gate latency feature (ECGL).

dont_touch_network

A read-only boolean attribute that returns True if a *dont_touch_network* constraint impacts this pin.

dont_touch_network_no_propagate

A read-only boolean attribute that returns True if a *dont_touch_network-no_propagate* constraint impacts this pin.

early_fall_arrival

A read-only float attribute that returns the arrival time for the falling edge signal at this pin in early analysis.

early_fall_input_cap

A read-only float attribute that returns the early condition falling edge input capacitance for this pin. Returns 0 for output pins. Value is returned for the *current_corner*.

early_fall_load_cap

A read-only float attribute that returns the early condition falling edge capacitance seen for this pin and its connected net, including net wire capacitance, and pin capacitance for all pins or ports driven by this pin's net. Value is returned for the *current_corner*.

early_fall_transition

A read-only float attribute that returns the early analysis falling transition time for the pin. The timer must be up-to-date to query this attribute value. The value is for the *current_scenario* only.

p

early_rise_arrival

A read-only float attribute that returns the arrival time for the rising edge signal at this pin in early analysis.

early_rise_input_cap

A read-only float attribute that returns the early condition rising edge input capacitance for this pin. Returns 0 for output pins. Value is returned for the current_corner.

early_rise_load_cap

A read-only float attribute that returns the early condition rising edge capacitance seen for this pin and its connected net, including net wire capacitance, and pin capacitance for all pins or ports driven by this pin's net. Value is returned for the current_corner.

early_rise_transition

A read-only float attribute that returns the early analysis rising transition time for the pin. The timer must be up-to-date to query this attribute value. The value is for the current_scenario only.

eco_fl_status

A read-only string attribute that is for freeze layer eco flow. It indicates the status of the freeze layer objects that associated with the pin. Value "disconnected" means the pin is disconnected from a net and freeze layer objects are annotated on the pin. Value "updated" means the pin is connected to a new net and freeze layer objects of the pin have been updated to the new net." Value "connected" means the pin is connected to a new net but freeze layer objects are not updated to the new net successfully.

effective_process_label_early

A read-only string attribute that returns the actual process label value used for early analysis for this pin in the current_corner. This can be different from the set_process_label constraint applied to this pin's cell if that process label mismatches the one used for current_corner. If they mismatch, then the attribute mismatched_process_label_min on this cell will be true.

effective_process_label_late

A read-only string attribute that returns the actual process label value used for late analysis for this pin in the current_corner. This can be different from the set_process_label constraint applied to this pin's cell if that process label mismatches the one used for current_corner. If they mismatch, then the attribute mismatched_process_label_max on this cell will be true.

effective_process_number_early

A read-only float attribute that returns the actual process number value used for early analysis for this pin in the `current_corner`. This can be different from the `set_process_number` constraint applied to this pin's cell if that process number mismatches the one used for `current_corner`. If they mismatch, then the attribute `mismatched_process_number_min` on this cell will be true.

effective_process_number_late

A read-only float attribute that returns the actual process number value used for late analysis for this pin in the `current_corner`. This can be different from the `set_process_number` constraint applied to this pin's cell if that process number mismatches the one used for `current_corner`. If they mismatch, then the attribute `mismatched_process_number_max` on this cell will be true.

effective_temperature_early

A read-only float attribute that returns the actual temperature value used for early analysis for this pin in the `current_corner`. This can be different from the `set_temperature` constraint applied to this pin's cell if that temperature mismatches the one used for `current_corner`. If they mismatch, then the attribute `mismatched_temperature_min` on this cell will be true.

effective_temperature_late

A read-only float attribute that returns the actual temperature value used for late analysis for this pin in the `current_corner`. This can be different from the `set_temperature` constraint applied to this pin's cell if that temperature mismatches the one used for `current_corner`. If they mismatch, then the attribute `mismatched_temperature_max` on this cell will be true.

effective_voltage_early

A read-only float attribute that returns the actual voltage value used for early analysis for this pin in the `current_corner`. This can be different from the `set_voltage` constraint applied to this pin's cell if that voltage mismatches the one used for `current_corner`. If they mismatch, then the attribute `mismatched_voltage_min` on this cell will be true.

effective_voltage_late

A read-only float attribute that returns the actual voltage value used for late analysis for this pin in the `current_corner`. This can be different from the `set_voltage` constraint applied to this pin's cell if that voltage mismatches the one used for `current_corner`. If they mismatch, then the attribute `mismatched_voltage_max` on this cell will be true.

equal_opposite_propagation

A read-only boolean attribute that controls equal_opposite propagation.

escaped_full_name

A read-only string attribute that returns the full name of the pin with any literal hierarchy characters escaped with a backslash.

essential_activity_group

A read-only string attribute that returns the essential activity group of the specified pin.

estimated_analysis

A read-only string attribute that indicates what estimation techniques are applied by the command "estimate_timing" at the driver cell or net of the pin. It includes "SIZE", "BUFFERS" etc.

full_name

A read-only string attribute that returns the full name of the pin including a prefix for the instance path to this pin.

function

A read-only string attribute that returns leaf output or inout pins for the logic function. The logic function is defined in the library.

gate_area

A settable string attribute that returns a string value which is the value of antenna property gate area.

gate_diffusion_length

A settable string attribute that returns a string value which is the value of antenna property gate diffusion length.

glitch_rate

A read-only double attribute that returns the rate at which the glitch transitions occur on the pin within a specific time period.

groups

A read-only group collection attribute that indicates the groups the pin belongs to.

hdl_file

A read-only string attribute that returns the original RTL file name for the pin.

p

hdl_hier

A read-only string attribute that returns the original RTL hierarchy for the pin.

hdl_line

A read-only string attribute that returns the original RTL file line number for the pin.

hdl_origin

A read-only string attribute that returns the original RTL creator associated with the pin.

hold_uncertainty

A read-only float attribute that returns the user-specified clock uncertainty for hold. This attribute will exist only if *set_clock_uncertainty* was specified on this object.

ignore_ndr

A settable boolean attribute that determines whether the router should ignore the non-default routing rule for this pin's net and use the default rules instead.

implemented_strategy

A read-only collection attribute that returns the strategy implemented during insertion or association

is_async_pin

A read-only boolean attribute that returns true if the pin is asynchronous.

is_bus_bit

A read-only boolean attribute that returns true if the pin is part of a pin bus.

is_clear_pin

A read-only boolean attribute that returns true if the pin is clear.

is_clock_exclusivity

A read-only boolean attribute that returns true if the pin has been set as a clock exclusivity point either manually using the *set_clock_exclusivity* command, or automatically through the time. enable_auto_mux_clock_exclusivity app option.

is_clock_gating_clock

A read-only boolean attribute that returns true if the pin is the enable pin of a clock gating check.

p

is_clock_gating_enable

A read-only boolean attribute that returns True if pin is the enable pin of a clock gating check (alias of *is_clock_gating_pin*).

is_clock_gating_pin

A read-only boolean attribute that returns True if pin is the enable pin of a clock gating check (alias of *is_clock_gating_enable*).

is_clock_pin

A read-only boolean attribute that returns True for clock pins of registers. A clock pin is any pin that has a timing check from it to a data pin.

is_clock_used_as_clock

A read-only boolean attribute that returns True if this pin is part of at least one clock network and the clock network fanout of this pin reaches at least one register clock pin or output port.

is_clock_used_as_data

A read-only boolean attribute that returns True if this pin is part of at least one clock network and the clock network fanout of this pin reaches at least one register data pin. If the attribute values for *is_clock_used_as_data* is TRUE and *is_clock_used_as_clock* is FALSE, the clocks through this pin only acts a data and never as clock.

is_data_pin

A read-only boolean attribute that returns True for data pins of registers. A data pin is any pin that has a timing check to it. Data pins are legal endpoint for timing paths.

is_fall_edge_triggered_clock_pin

A read-only boolean attribute that returns True if pin is a fall edge triggered clock pin.

is_fall_edge_triggered_data_pin

A read-only boolean attribute that returns True if pin is a fall edge triggered data pin.

is_feedthrough_port

A read-only boolean attribute that is set on the feedthrough ports created by *place_pins* or *optimize_topology_plans* or *push_down_objects*.

p

is_fixed

A read-only boolean attribute that returns True if the physical status of this pin is fixed or locked. If this pin is a physical hierarchical pin, it is fixed/locked if its reference port is fixed/locked. If this pin is a leaf pin, it is fixed/locked if its cell is fixed/locked.

is_hierarchical

A read-only boolean attribute that returns True if the pin's cell is hierarchical.

is_ideal

A read-only boolean attribute that returns true if the pin has been marked ideal using the `set_ideal_network` command.

is_latch_loop_breaker

A read-only boolean attribute that returns true if the pin is the D pin of a loop-breaker latch.

is_negative_level_sensitive_clock_pin

A read-only boolean attribute that returns true if the pin is a low level sensitive clock pin.

is_negative_level_sensitive_data_pin

A read-only boolean attribute that returns true if the pin is a low level sensitive data pin.

is_net_driver

A read-only boolean attribute that returns true if the pin is a net driver.

is_net_load

A read-only boolean attribute that returns true if the pin is a net load.

is_positive_level_sensitive_clock_pin

A read-only boolean attribute that returns true if the pin is a high level sensitive clock pin.

is_positive_level_sensitive_data_pin

A read-only boolean attribute that returns true if the pin is a high level sensitive data pin.

is_preset_pin

A read-only boolean attribute that returns true if the pin is a preset pin.

p

is_rise_edge_triggered_clock_pin

A read-only boolean attribute that returns true if the pin is a rise edge triggered clock pin.

is_rise_edge_triggered_data_pin

A read-only boolean attribute that returns True if pin is a rise edge triggered data pin.

is_scan

A read-only boolean attribute that returns True if the pin is a scan pin.

is_three_state

A read-only boolean attribute that returns True if this pin is a three-state output (can become high impedance when not enabled).

is_three_state_enable_pin

A read-only boolean attribute that returns True if the pin is a three state enable pin.

is_three_state_output_pin

A read-only boolean attribute that returns True if this pin is a three-state output (can become high impedance when not enabled).

is_upward_via_connection_preferred

A settable boolean attribute that returns True if the router should connect to this pin from the lower layer, instead of from the upper layer or same layer.

is_user_latch_loop_breaker

A read-only boolean attribute that is true if the pin that has been specified by the user to be used as a loop breaker latch data pin. This is set using the `set_latch_loop_breaker` command, which guides the tool in selecting data pins as loop breaker data pins. This is useful for breaking sequential loops of transparent latches in a design, allowing for special analysis of these paths.

is_user_pg

A settable boolean attribute that returns True if this pin is marked as a user created PG pin. PG pins coming in from Verilog are marked as user created, unless this attribute is subsequently modified.

late_fall_arrival

A read-only float attribute that returns the arrival time for the falling edge signal at this pin in late analysis.

p

late_fall_input_cap

A read-only float attribute that returns the late condition falling edge input capacitance for this pin. Returns 0 for output pins. Value is returned for the `current_corner`.

late_fall_load_cap

A read-only float attribute that returns the late condition falling edge capacitance seen for this pin and its connected net, including net wire capacitance, and pin capacitance for all pins or ports driven by this pin's net. Value is returned for the `current_corner`.

late_fall_transition

A read-only float attribute that returns the late analysis falling transition time for the pin. The timer must be up-to-date to query this attribute value. The value is for the `current_scenario` only.

late_rise_arrival

A read-only float attribute that returns the arrival time for the rising edge signal at this pin in late analysis.

late_rise_input_cap

A read-only float attribute that returns the late condition rising edge input capacitance for this pin. Returns 0 for output pins. Value is returned for the `current_corner`.

late_rise_load_cap

A read-only float attribute that returns the late condition rising edge capacitance seen for this pin and its connected net, including net wire capacitance, and pin capacitance for all pins or ports driven by this pin's net. Value is returned for the `current_corner`.

late_rise_transition

A read-only float attribute that returns the late analysis rising transition time for the pin. The timer must be up-to-date to query this attribute value. The value is for the `current_scenario` only.

layer

A read-only layer collection attribute that returns layers on which the shapes of this pin are.

layer_name

A read-only string attribute that returns a list of layer names on which the shapes of this pin are.

p

lft_inv_guide_load

A read-only boolean attribute that indicates whether you need to add an inverter on the load side of this hierarchical pin to resolve the polarity issue caused by `commit_lft`.

lib_pin

A read-only `lib_pin` collection attribute that is the library pin for a pin. The collection contains exactly one library pin object. This attribute is defined only on leaf cells.

lib_pin_name

A read-only string attribute that returns the name of the library pin.

location

A read-only coord attribute that returns the physical location of the pin.

major_activity_type

A read-only string attribute that returns the major switching activity type of a pin. The valid values are

```
simulated: Switching activity is retrieved from simulation files
annotated: The switching activity is annotated by the user
derived: The switching activity is retrieved from constraints
calculated: The switching activity is computed by activity
propagation
default: This value represents all other sources of switching
activity
```

matching_type

A read-only `matching_type` collection attribute that returns the `matching_types` of the pin.

max_balance_delay

A read-only float attribute that returns the balance point delay value set by the user on a clock pin using `'set_clock_balance_points -delay $value -late'`.

max_capacitance

A read-only float attribute that returns the `max_capacitance` constraint set on the pin by the `set_max_capacitance` command. Value is returned for the `current_corner`.

max_capacitance_constraint

A read-only float attribute that returns the `max_capacitance` constraint set on the pin by the `set_max_capacitance` command. Value is returned for the `current_corner`.

max_fall_arrival

A read-only float attribute that returns the arrival time for the falling edge signal at this pin in max analysis.

max_fall_input_cap

A read-only float attribute that returns the max condition falling edge input capacitance for this pin. Returns 0 for output pins. Value is returned for the current_corner.

max_fall_load_cap

A read-only float attribute that returns the max condition falling edge capacitance seen for this pin and its connected net, including net wire capacitance, and pin capacitance for all pins or ports driven by this pin's net. Value is returned for the current_corner.

max_fall_local_slack

A read-only float attribute that returns the local slack for a pin of a transparent latch, considering the data arrival at the local pin and setup constraints at the local pin, without considering constraints downstream from the pin. If there are no constraints at the pin, the attribute is undefined. The normal slack (max_fall_slack or max_slack) considers the constraints downstream from the pin. This attribute gives the local slack for falling max path analysis.

max_fall_slack

A read-only float attribute that returns the worst slack at a pin for falling maximum path delays. This attribute is valid only after timing has been updated.

max_fall_transition

A read-only float attribute that returns the largest max condition falling transition time for the pin. The timer must be up-to-date to query this attribute value. The value is for the current_scenario only.

max_rise_arrival

A read-only float attribute that returns the arrival time for the rising edge signal at this pin in max analysis.

max_rise_input_cap

A read-only float attribute that returns the max condition rising edge input capacitance for this pin. Returns 0 for output pins. Value is returned for the current_corner.

max_rise_load_cap

A read-only float attribute that returns the max condition rising edge capacitance seen for this pin and its connected net, including net wire capacitance, and pin capacitance for all pins or ports driven by this pin's net. Value is returned for the `current_corner`.

max_rise_local_slack

A read-only float attribute that returns the local slack for a pin of a transparent latch, considering the data arrival at the local pin and setup constraints at the local pin, without considering constraints downstream from the pin. If there are no constraints at the pin, the attribute is undefined. The normal slack (`max_rise_slack` or `max_slack`) considers the constraints downstream from the pin. This attribute gives the local slack for rising max path analysis.

max_rise_slack

A read-only float attribute that returns the worst slack at a pin for rising maximum path delays. This attribute is valid only after timing has been updated.

max_rise_transition

A read-only float attribute that returns the largest max condition rising transition time for the pin. The timer must be up-to-date to query this attribute value. The value is for the `current_scenario` only.

max_slack

A read-only float attribute that returns the worst slack value between the `max_rise_slack` and `max_fall_slack` attributes.

max_time_borrow

A read-only float attribute that returns a floating-point number that establishes an upper limit for time borrowing; that is, it prevents the use of the entire pulse width for level-sensitive latches. Units are those used in the logic library. Set with the `set_max_time_borrow` command.

max_transition

A read-only float attribute that returns the `max_transition` constraint set on the pin by the `set_max_transition` command. The actual `max_transition` constraint enforced at the pin is in the `constraining_max_transition` attribute.

max_transition_constraint

A read-only float attribute that returns the `max_transition` constraint set on the pin by the `set_max_transition` command. The actual `max_transition` constraint enforced at the pin is in the `constraining_max_transition` attribute.

metrics_tim_logic_levels_viol

A read-only int attribute that returns if the worst timing path ending at this pin has logic level violation or not. Returns 0 or 1 accordingly. Works on current scenario.

metrics_tim_logic_levels_viol_local

A read-only int attribute that returns if the worst timing path ending at this pin has logic level violation or not. Returns 0 or 1 accordingly. Works on current scenario.

metrics_tim_logic_levels_viol_pg

A read-only string attribute that returns the space separated string of path-group-names which for which the worst timing paths ending at this pin has logic level violations. Something like "<pg1> <pg3> <pg4>" etc. Works on current scenario.

metrics_tim_net_length_viol

A read-only string attribute that returns the space separated string of path-group-names which for which the worst timing paths ending at this pin has net length violations. Something like <pg1> <pg3> <pg4> etc. Works on current scenario.

metrics_tim_path_length_viol

A read-only string attribute that returns the space separated string of path-group-names which for which the worst timing paths ending at this pin has path length violations. Something like <pg1> <pg3> <pg4> etc. Works on current scenario.

metrics_tim_repeater_levels_viol

A read-only int attribute that returns the repeater count violation for the current scenario. Works only on hierarchical cells. Returns the sum of all repeater count violating paths under the input hierarchy.

metrics_tim_repeater_levels_viol_local

A read-only int attribute that returns the repeater count violation for the current scenario. Works only on hierarchical cells. Returns the sum of all repeater count violating paths ending in the input hierarchy.

min_balance_delay

A read-only float attribute that returns the balance point delay value set by the user on a clock pin using 'set_clock_balance_points -delay \$value -early'.

p

min_capacitance

A read-only float attribute that returns the `min_capacitance` constraint set on the pin by the `set_min_capacitance` command. Value is returned for the `current_corner`.

min_capacitance_constraint

A read-only float attribute that returns the `min_capacitance` constraint set on the pin by the `set_min_capacitance` command. Value is returned for the `current_corner`.

min_fall_arrival

A read-only float attribute that returns the arrival time for the falling edge signal at this pin in min analysis.

min_fall_input_cap

A read-only float attribute that returns the min condition falling edge input capacitance for this pin. Returns 0 for output pins. Value is returned for the `current_corner`.

min_fall_load_cap

A read-only float attribute that returns the min condition falling edge capacitance seen for this pin and its connected net, including net wire capacitance, and pin capacitance for all pins or ports driven by this pin's net. Value is returned for the `current_corner`.

min_fall_slack

A read-only float attribute that returns the worst slack at a pin for falling minimum path delays. This attribute is valid only after timing has been updated.

min_fall_transition

A read-only float attribute that returns the smallest min condition falling transition time for the pin. The timer must be up-to-date to query this attribute value. The value is for the `current_scenario` only.

min_rise_arrival

A read-only float attribute that returns the arrival time for the rising edge signal at this pin in min analysis.

min_rise_input_cap

A read-only float attribute that returns the min condition rising edge input capacitance for this pin. Returns 0 for output pins. Value is returned for the `current_corner`.

p

min_rise_load_cap

A read-only float attribute that returns the min condition rising edge capacitance seen for this pin and its connected net, including net wire capacitance, and pin capacitance for all pins or ports driven by this pin's net. Value is returned for the `current_corner`.

min_rise_slack

A read-only float attribute that returns the worst slack at a pin for rising minimum path delays. This attribute is valid only after timing has been updated.

min_rise_transition

A read-only float attribute that returns the smallest min condition rising transition time for the pin. The timer must be up-to-date to query this attribute value. The value is for the `current_scenario` only.

min_slack

A read-only float attribute that returns the worst slack value between the `min_rise_slack` and `min_fall_slack` attributes.

minor_activity_type

A read-only string attribute that returns the minor switching activity type of a pin. The valid values are:

- **abstract**: Applies to the **annotated** activity type. Indicates that the switching activity was created during abstraction.
- **create_clock**: Applies to the **derived** activity type. Indicates that the switching activity was derived from a **create_clock** command in the SDC.
- **create_generated_clock**: Applies to the **derived** activity type. Indicates that the switching activity was derived from a **create_generated_clock** command in the SDC.
- **estimated**: Applies to the **calculated** activity type. Indicates that the switching activity was created during optimization.
- **implied**: Applies to the **calculated** activity type. Indicates that the switching activity was calculated from the propagation of activity through repeater cells.
- **logic_constant**: Applies to the **derived** activity type. Indicates that the switching activity was derived from the netlist.

p

- **propagated:** Applies to the **calculated** activity type. Indicates that the switching activity was calculated after the **propagate_switching_activity** command.
- **set_case_analysis:** Applies to the **derived** activity type. Indicates that the switching activity was derived from a **set_case_analysis** command in the SDC.
- **set_switching_activity:** Applies to the **annotated** activity type. Indicates that the switching activity was set by the user.
- **supply_constant:** Applies to the **derived** activity type. Indicates that the switching activity was derived from the netlist.
- **timer_implied:** Applies to the **derived** activity type. Indicates that the switching activity was propagated by the timer from a **set_case_analysis** command in the SDC.

mismatched_process_label_max

A read-only boolean attribute that returns True if there is a mismatch between the process label applied to this pin from the **set_process_label** constraint for max analysis, and the process label used for the **current_corner**.

mismatched_process_label_min

A read-only boolean attribute that returns True if there is a mismatch between the process label applied to this pin from the **set_process_label** constraint for min analysis, and the process label used for the **current_corner**.

mismatched_process_number_max

A read-only boolean attribute that returns True if there is a mismatch between the process number applied to this pin from the **set_process_number** constraint for max analysis, and the process number used for the **current_corner**.

mismatched_process_number_min

A read-only boolean attribute that returns True if there is a mismatch between the process number applied to this pin from the **set_process_number** constraint for min analysis, and the process number used for the **current_corner**.

mismatched_pvt_max

A read-only boolean attribute that returns True if there is any mismatch in the PVT conditions applied to this pin for max analysis, and the PVT conditions used for the **current_corner**.

p

mismatched_pvt_min

A read-only boolean attribute that returns True if there is any mismatch in the PVT conditions applied to this pin for min analysis, and the PVT conditions used for the current_corner.

mismatched_temperature_max

A read-only boolean attribute that returns True if there is a mismatch between the temperature applied to this pin from the set_temperature constraint for max analysis, and the temperature used for the current_corner.

mismatched_temperature_min

A read-only boolean attribute that returns True if there is a mismatch between the temperature applied to this pin from the set_temperature constraint for min analysis, and the temperature used for the current_corner.

mismatched_voltage_max

A read-only boolean attribute that returns True if there is a mismatch between the voltage applied to this pin from the set_voltage constraint for max analysis, and the voltage used for the current_corner.

mismatched_voltage_min

A read-only boolean attribute that returns True if there is a mismatch between the voltage applied to this pin from the set_voltage constraint for min analysis, and the voltage used for the current_corner.

mv_scan_avoid_tap

A settable boolean attribute that, if set, indicates that the pin should be skipped by MV aware scan tapping point selection during DFT Scan insertion step.

n_gate_area

A settable string attribute that returns a string value which is the value of antenna property n-gate area.

n_gate_diffusion_length

A settable string attribute that returns a string value which is the value of antenna property n-gate diffusion length.

name

A read-only string attribute that returns the name of the pin.

net

A read-only net collection attribute that returns the net connected to the pin.

p

net_name

A read-only string attribute that returns the name of the net connected to the pin.

object_class

A read-only string attribute that returns the name of this object class. Returns 'pin' for pin objects.

p_gate_area

A settable string attribute that returns a string value which is the value of antenna property p-gate area.

p_gate_diffusion_length

A settable string attribute that returns a string value which is the value of antenna property p-gate diffusion length.

pa_input_pin_fall_transition

A read-only float attribute that returns the fall transition used in power analysis at the node related to the specified pin. The value is based on current timer status, the attribute does not update the timer. This attribute is set during timing updates and is corner and scenario and instance specific. The value of the fall transition attribute is the same as what is computed for the cell's input pin by the report_delay_calculation command.

pa_input_pin_rise_transition

A read-only float attribute that returns the rise transition used in power analysis at the node related to the specified pin. The value is based on current timer status, the attribute does not update the timer. This attribute is derived during timing updates and is corner, scenario, and instance specific. The value of the rise transition attribute is the same as what is computed for the cell's input pin by the report_delay_calculation command.

pa_output_pin_cap

A read-only float attribute that returns the capacitance used in power analysis on the specified pin. The value is based on current timer status, the attribute does not update the timer. Value is returned for the current_corner.

pa_pin_voltage

A read-only float attribute that returns the voltage used in power analysis of the PG pin related to the specified signal pin of a cell. This attribute is set during timing updates and is corner and scenario specific. In Liberty, signal pins have a related_power_pin attribute to define the relationship between a signal pin and its corresponding PG pin. This relationship defines the voltage

p

level of the signal pin. The PG pin has a `voltage_name` attribute that specifies a name voltage value which corresponds to a `voltage_map` definition for the entire library. The `voltage_map` provides a floating point voltage value for the `voltage_name` attribute. The `pa_pin_voltage` attribute stores this information found from the linked reference library cell in fashion that is easier to query. The voltage of the cell comes from the reference library cell it is linked against. You can set the voltage of a cell in 1 of 2 fashions. The typical method is to set the top level operating condition. In implementation, the PVT, process, voltage, and temperature of a design is uniform for the entire design. Therefore, all standard cells in the design will share the same PVT. In UPF designs, the voltage can be varied on a per supply net basis. The `set_voltage` command is used to define the current operating voltage of a supply net. This voltage value should be defined as a potential state for the supply net in your PST, power state table. The voltage values used to define the current operating voltage of the supply nets in the design do not need to correspond to a defined PST state. The voltage values are used to tell optimization the voltage that is to be used by optimization. For instance, a design with 3 unique supply nets could be optimized at 0.85V, 0.75V, and 1.5V. Your UPF must specify that 0.85V, 0.75V and 1.5V are valid states for the individual supply nets. However, your UPF does not need to specify that the design will operating at a state of 0.85V, 0.75V and 1.5V simultaneously.

pane_early

A read-only int attribute that returns the timing pane number used for early analysis on this pin in `current_corner`.

pane_late

A read-only int attribute that returns the timing pane number used for early analysis on this pin in `current_corner`.

parent_block

A read-only block collection attribute that returns the block reference of the physical hierarchy owner of this pin. For example, if pin `a/b/c/d/reg/Q` exists inside physical hierarchy `a/b`, this attribute contains the reference cell information for the block instance `a/b`.

parent_block_cell

A read-only cell collection attribute that returns the block instance of the physical hierarchy owner of this pin. For example, if pin `a/b/c/d/reg/Q` exists inside physical hierarchy `a/b`, this attribute contains the block instance `a/b`. This attribute is defined only for pins inside physical hierarchy. It is undefined for all pins at the top level.

p

parent_cell

A read-only cell collection attribute that returns the instance name of the logical hierarchy owner of this pin's cell. For example, for pin *a/b/c/d/reg/Q*, it does not contain *a/b/c/d/reg*, but rather *a/b/c/d*. Note that this attribute does not contain the cell owner of the pin, but rather the cell owner of the pin's cell. To get the pin's direct cell owner, query the cell attribute of the pin. If the cell that owns this pin is at the top level of the logic hierarchy, this attribute is undefined.

pg_pin_weight

A settable float attribute that returns the PG pin weight value for routing. The valid range for this value is $0.1 \leq \text{value} \leq 25.0$. Values get rounded to 0.1 precision when setting this attribute. This attribute is defined only for physical pins.

pg_type

A read-only string attribute that returns the pg-type of the given ground pin or the power pin.

physical_hierarchy_level

A read-only int attribute that specifies the depth of the physical hierarchy.

physical_status

A read-only string attribute (with allowed values: *fixed* *locked* *placed* *unplaced*) that returns the physical placement status of the pin. If this pin is a physical hierarchical pin, its status is obtained from the reference port. If this pin is a leaf pin, its status is derived from its cell's placement status. If status is *'unplaced'*, the pin has not been placed. If status is *'placed'*, the pin has been placed and may be moved. If status is *'fixed'*, the pin is fixed for automatic changes. If status is *'locked'*, the pin is fixed for automatic and manual changes.

pin_alignment_matching_id

A settable int attribute that is used only when the app option *plan.pins.strict_alignment* set to true. It is used to align the pins even if they don't have connections.

pin_direction

A read-only string attribute that returns the direction of a pin. Possible values are *in*, *out*, *inout*, or *unknown*. This is the same as "direction". This attribute is supported for compatibility with other tools. The "direction" attribute is supported for both ports and pins and therefore easier to use for heterogeneous collections of pins and ports.

p

pin_has_async_check

A read-only boolean attribute that indicates whether the pin has recovery or removal check.

port

A read-only port collection attribute that returns the port corresponding to this pin, if it is hierarchical.

port_type

A settable string attribute (with allowed values: analog_ground analog_power analog_signal clock deep_nwell deep_pwell ground nwell power pwell reset scan signal tie_high tie_low unset) that returns the type of the reference port of this pin.

power_domain

A read-only string attribute that returns the power domain name of the given pin.

power_rail_voltage_max

A read-only float attribute that returns the maximum power rail voltage set on the pin. In the case of a bidirectional pin, the output voltage is returned by default. To return the input voltage of a bidirectional pin, query the power_rail_voltage_bidir_input_max attribute.

power_rail_voltage_min

A read-only float attribute that returns the minimum power rail voltage set on the pin. In the case of a bidirectional pin, the output voltage is returned by default. To return the input voltage of a bidirectional pin, query the power_rail_voltage_bidir_input_min attribute.

process_label_early

A read-only string attribute that returns the process label constraint for early analysis for this pin in the current_corner. Set by the set_process_label constraint.

process_label_early_file_line_info

A read-only string attribute that returns a Tcl list of two elements, the first showing the source file path and the second showing the source file line number, where the set_process_label constraint for early analysis was applied on this pin.

process_label_early_source

A read-only string attribute that indicates the source scope of the constraint that gave this pin its early analysis process label setting from the set_process_label

p

command. The source could be from a constraint on the design, a hierarchical cell, a library cell, or a leaf cell.

process_label_late

A read-only string attribute that returns the process label constraint for late analysis for this pin in the current_corner. Set by the set_process_label constraint.

process_label_late_file_line_info

A read-only string attribute that returns a Tcl list of two elements, the first showing the source file path and the second showing the source file line number, where the set_process_label constraint for late analysis was applied on this pin.

process_label_late_source

A read-only string attribute that indicates the source scope of the constraint that gave this pin its late analysis process label setting from the set_process_label command. The source could be from a constraint on the design, a hierarchical cell, a library cell, or a leaf cell.

process_number_early

A read-only float attribute that returns the process label constraint for early analysis for this pin in the current_corner. Set by the set_process_number constraint.

process_number_early_file_line_info

A read-only string attribute that returns a Tcl list of two elements, the first showing the source file path and the second showing the source file line number, where the set_process_number constraint for early analysis was applied on this pin.

process_number_early_source

A read-only string attribute that indicates the source scope of the constraint that gave this pin its early analysis process number setting from the set_process_number command. The source could be from a constraint on the design, a hierarchical cell, a library cell, or a leaf cell.

process_number_late

A read-only float attribute that returns the process number constraint for late analysis for this pin in the current_corner. Set by the set_process_number constraint.

p

process_number_late_file_line_info

A read-only string attribute that returns a Tcl list of two elements, the first showing the source file path and the second showing the source file line number, where the `set_process_number` constraint for late analysis was applied on this pin.

process_number_late_source

A read-only string attribute that indicates the source scope of the constraint that gave this pin its late analysis process number setting from the `set_process_number` command. The source could be from a constraint on the design, a hierarchical cell, a library cell, or a leaf cell.

propagated_clock

A read-only boolean attribute that returns true if clock edge times are delayed by propagating the values through the clock network. If this attribute is not present, ideal clocking is assumed. Set with `set_propagated_clock`.

related_clock

A read-only string attribute that returns a clock related to this pin if it exists, else returns "<NONE>". The value is based on current timer status, the attribute does not update the timer.

related_ground_pin

A read-only collection attribute that returns the related ground pin of the `lib_pin`.

related_power_pin

A read-only collection attribute that returns the related power pin of the `lib_pin`.

route_connection

A settable string attribute (with allowed values: `connect skip`) that determines whether the router should connect the pin.

safety_correction_signal

A read-only port, or pin collection attribute that stores the correction signal setting provided by functional safety commands.

safety_error_signal

A read-only port, or pin collection attribute that denotes the error signal associated with this safety critical register pin.

safety_requirement_id

A read-only string attribute that captures the requirement ID provided through the `-requirement_id` option of Functional Safety commands. Functional safety

p

constraints need to be trackable back to a specific requirement and this attribute captures that information

scan_chains

A read-only `scan_chain` collection attribute that contains the `scan_chains` which have this pin.

scan_signals

A read-only `scan_signal` collection attribute that contains the `scan_signals` which have this pin.

setup_uncertainty

A read-only float attribute that returns the user-specified clock uncertainty for setup. This attribute will exist only if `set_clock_uncertainty` was specified on this object.

signal_type

A read-only string attribute that identifies the types of scan pins.

single_connection

A settable boolean attribute that indicates whether the router is allowed to connect to this pin only once and cannot use it as a feedthrough. This is settable and removable. When set, this pin-specific setting has precedence over its reference `lib_pin`'s `single_connection` attribute value. When unset or removed, this pin inherits its reference `lib_pin`'s `single_connection` attribute value.

skew_groups

A read-only collection attribute that captures the user-defined skew group that the pin belongs to.

skip_via_ladder_insertion

A settable boolean attribute that determines whether the router should skip `via_ladder` insertion on this pin. A true value of this attribute would prevent router from inserting `via_ladder`. This attribute defaults to false.

spfm_loss

A settable float attribute that returns the output pins of sequential cells. It indicates the safety criticality of the register that it is set on. Its value should be between 0 and 100.

static_probability

A read-only double attribute that returns the switching activity probability of a pin. The value is based on current timer status, the attribute does not update the timer.

p

supply_connection_type

A read-only string attribute that returns how the supply net connection is determined. Possible values are "default", "user" and "derived".

temperature_early

A read-only float attribute that returns the temperature constraint for early analysis for this pin in the current_corner. Set by the set_temperature constraint.

temperature_early_file_line_info

A read-only string attribute that returns a Tcl list of two elements, the first showing the source file path and the second showing the source file line number, where the set_temperature constraint for early analysis was applied on this pin.

temperature_early_source

A read-only string attribute that indicates the source scope of the constraint that gave this pin its early analysis temperature setting from the set_temperature command. The source could be from a constraint on the design, a hierarchical cell, a library cell, or a leaf cell.

temperature_late

A read-only float attribute that returns the temperature constraint for late analysis for this pin in the current_corner. Set by the set_temperature constraint.

temperature_late_file_line_info

A read-only string attribute that returns a Tcl list of two elements, the first showing the source file path and the second showing the source file line number, where the set_temperature constraint for late analysis was applied on this pin.

temperature_late_source

A read-only string attribute that indicates the source scope of the constraint that gave this pin its late analysis temperature setting from the set_temperature command. The source could be from a constraint on the design, a hierarchical cell, a library cell, or a leaf cell.

toggle_rate

A read-only double attribute that returns the switching activity toggle rate of a pin. The value is based on current timer status, the attribute does not update the timer.

top_block

A read-only block collection attribute that returns the top block object for this pin.

p

UPF_clamp_value

A read-only string attribute that returns the clamp value to be used if the pin has an isolation strategy applied to it

UPF_driver_supply

A read-only collection attribute that returns the driver supply of a macro cell output pin, or the assumed driver supply of an external logic driving a primary input. Ignored if pin is not on a terminal boundary or not top level

UPF_feedthrough

A read-only collection attribute that identifies a set of pins on the interface of a module or cell that are directly connected to a pin inside the module or cell

UPF_is_analog

A read-only boolean attribute that identifies a signal pin on the interface of a module or cell that is an analog port.

UPF_literal_supply

A read-only collection attribute that identifies the supply set to be used to implement a literal constant value associated with an input pin of an instance.

UPF_receiver_supply

A read-only collection attribute that specifies the receiver supply of a macro cell input pin, or the assumed receiver supply of an external logic receiving a primary input. Ignored if pin is not on a terminal boundary or not top level

UPF_related_supply_set

A read-only collection attribute that returns the supply set that is related to.

UPF_unconnected

A read-only boolean attribute that identifies a pin on the interface of a module or cell that is not connected to either a source or a sink within the cell, and that is not connected to any other port on the interface of the module or cell.

user_case_value

A read-only string attribute that returns the case value set by the command *set_case_analysis*. This attribute is only defined on those pins that *set_case_analysis* has been directly set on. Use the attribute "case_value" for both directly set case values and propagated case values.

user_clock_sense

A read-only string attribute that returns the sense set by the command `set_clock_sense`. This attribute is only defined on those pins where the `set_clock_sense` command was used directly.

voltage_early

A read-only float attribute that returns the voltage constraint for early analysis for this pin in the current corner. Set by the `set_voltage` constraint.

voltage_early_file_line_info

A read-only string attribute that returns a Tcl list of two elements, the first showing the source file path and the second showing the source file line number, where the `set_voltage` constraint for early analysis was applied on this pin.

voltage_early_source

A read-only string attribute that indicates the source scope of the constraint that gave this pin its early analysis voltage setting from the `set_voltage` command. The source could be from a constraint on the design, a hierarchical cell, a library cell, or a leaf cell.

voltage_late

A read-only float attribute that returns the voltage constraint for late analysis for this pin in the current corner. Set by the `set_voltage` constraint.

voltage_late_file_line_info

A read-only string attribute that returns a Tcl list of two elements, the first showing the source file path and the second showing the source file line number, where the `set_voltage` constraint for late analysis was applied on this pin.

voltage_late_source

A read-only string attribute that indicates the source scope of the constraint that gave this pin its late analysis voltage setting from the `set_voltage` command. The source could be from a constraint on the design, a hierarchical cell, a library cell, or a leaf cell.

worst_max_fall_slack

A read-only float attribute that returns the worst slack at a pin for falling maximum path delays. This attribute is valid only after timing has been updated.

p

worst_max_rise_slack

A read-only float attribute that returns the worst slack at a pin for rising maximum path delays. This attribute is valid only after timing has been updated.

worst_max_slack

A read-only float attribute that returns the worst slack value between the `max_rise_slack` and `max_fall_slack` attributes.

worst_min_fall_slack

A read-only float attribute that returns the worst slack at a pin for falling minimum path delays. This attribute is valid only after timing has been updated.

worst_min_rise_slack

A read-only float attribute that returns the worst slack at a pin for rising minimum path delays. This attribute is valid only after timing has been updated.

worst_min_slack

A read-only float attribute that returns the worst slack value between the `min_rise_slack` and `min_fall_slack` attributes.

See Also

- [get_attribute](#)
- [list_attributes](#)
- [set_clock_uncertainty](#)

pin_blockage_attributes

Description of the application defined `pin_blockage` attributes.

Description

Attributes are properties assigned to objects such as pins, cells, and nets. They can be read-only or settable. A read-only attribute is informational; you cannot set its value. To modify a settable attribute, use the `set_attribute` command.

To determine the value of an attribute, use the `get_attribute` command. To see a list of all `pin_blockage` attributes available, use the `list_attributes -application -class pin_blockage` command.

Attributes for pin_blockage

This section describes the `pin_blockage` attributes.

p

all_objects

A read-only layer, port, pin, or net collection attribute that returns the layers blocked by the `pin_blockage`, and the ports, pins, or nets which are explicitly blocked by this `pin_blockage`. If the `pin_blockage` implicitly blocks all ports and pins, then this collection contains only the blocked layers.

bbox

A read-only rect attribute that returns the bounding-box of the `pin_blockage`. The format of a rectangle specification is `{{llx lly} {urx ury}}`, which specifies the lower-left and upper-right corners of the rectangle.

boundary

A settable polygon attribute that returns the boundary of the `pin_blockage`. The format of a `coordinate_list` specification is `{{x1 y1} {x2 y2}...}`.

bounding_box

A read-only `bounding_box` collection attribute that returns the bounding-box of the `pin_blockage`. The format of the collection specifies lower-left and upper-right corners of the rectangle.

feedthrough_only

A settable boolean attribute that returns whether this `pin_blockage` prevents feedthrough pins from being placed within its boundary.

full_name

A read-only string attribute that returns the name of the `pin_blockage`.

groups

A read-only group collection attribute that returns the groups in which this `pin_blockage` is a member.

in_edit_group

A read-only boolean attribute that returns whether the `pin_blockage` belongs to an `edit_group`.

is_shadow

A read-only boolean attribute that returns whether the `pin_blockage` is pushed down from above.

layer_names

A read-only string attribute that returns the names of the layers which are blocked by the `pin_blockage`.

layers

A settable layer collection attribute that returns the layers which are blocked by the `pin_blockage`.

name

A settable string attribute that returns the name of the `pin_blockage`.

nets

A settable net collection attribute that returns the nets which are explicitly blocked by the `pin_blockage`. A `pin_blockage` may explicitly block ports, pins, or nets. If nets, all of the nets' ports and pins are blocked. If none are explicitly blocked, then the `pin_blockage` blocks all ports and pins.

object_class

A read-only string attribute that returns the name of this object class. Returns 'pin_blockage' for `pin_blockage` objects.

parent_block

A read-only block collection attribute that returns the owner block object for this `pin_blockage`.

parent_block_cell

A read-only cell collection attribute that returns the parent block cell object for this `pin_blockage`.

parent_cell

A read-only cell collection attribute that returns the owner cell object for this `pin_blockage`.

physical_status

A settable string attribute (with allowed values: locked unrestricted) that returns the physical modification status of the `pin_blockage`. If status is unrestricted, the `pin_blockage` may be moved or rotated. If the status is locked, the `pin_blockage` may not be moved or rotated.

pins

A settable pin collection attribute that returns the pins which are explicitly blocked by the `pin_blockage`. A `pin_blockage` may explicitly block ports, pins, or nets. If nets, all of the nets' ports and pins are blocked. If none are explicitly blocked, then the `pin_blockage` blocks all ports and pins.

p

ports

A settable port collection attribute that returns the ports which are explicitly blocked by the `pin_blockage`. A `pin_blockage` may explicitly block ports, pins, or nets. If nets, all of the nets' ports and pins are blocked. If none are explicitly blocked, then the `pin_blockage` blocks all ports and pins.

shadow_status

A settable string attribute (with allowed values: `copied_down` `copied_up` `normal` `pulled_up` `pushed_down` `virtual_flat`) that returns the shadow status of the `pin_blockage`.

top_block

A read-only block collection attribute that returns the top block object for this `pin_blockage`.

See Also

- [get_attribute](#)
- [list_attributes](#)

pin_bus_attributes

Description of the application defined `pin_bus` attributes.

Description

Attributes are properties assigned to objects such as pins, cells, and nets. They can be read-only or settable. A read-only attribute is informational; you cannot set its value. To modify a settable attribute, use the `set_attribute` command.

To determine the value of an attribute, use the `get_attribute` command. To see a list of all `pin_bus` attributes available, use the `list_attributes -application -class pin_bus` command.

Attributes for pin_bus

This section describes the `pin_bus` attributes.

bit_order

A read-only string attribute (with allowed values: `down` `up`) that returns the bit order of a pin bus. Possible values are *up* or *down*. It is *down* when *end* is less than *start* else it is *up*.

block

A read-only block collection attribute that returns the block of a pin bus.

p

end

A read-only int attribute that returns the end index of a pin bus.

full_name

A read-only string attribute that returns the name of a pin bus.

name

A read-only string attribute that returns the name of a pin bus.

object_class

A read-only string attribute that returns the name of this object class. Returns 'pin_bus' for pin_bus objects.

pins

A read-only pin collection attribute that returns the pins of a pin bus.

start

A read-only int attribute that returns the start index of a pin bus.

See Also

- [get_attribute](#)
- [list_attributes](#)

pin_constraint_attributes

Description of the application defined pin_constraint attributes.

Description

Attributes are properties assigned to objects such as pins, cells, and nets. They can be read-only or settable. A read-only attribute is informational; you cannot set its value. To modify a settable attribute, use the *set_attribute* command.

To determine the value of an attribute, use the *get_attribute* command. To see a list of all pin_constraint attributes available, use the *list_attributes -application -class pin_constraint* command.

Attributes for pin_constraint

This section describes the pin_constraint attributes.

allow_feedthroughs

A read-only boolean attribute that returns whether feed through ports can be created in pin placement flow.

p

bundle_order

A settable string attribute (with allowed values: decreasing equal-distance increasing ordered) that returns the relative order of pins in the bundle.

exclude_sides

A read-only string attribute that returns the block sides on which the pin must not be placed. Applicable only for individual pin constraints.

full_name

A read-only string attribute that returns the full name of the pin constraint.

interleaving_bit_offset

A settable int attribute that returns the bit relative offset across layers when folding it.

interleaving_layer_range

A settable int attribute that returns the layer range of bundle pins folded into several layers.

keep_pins_together

A settable boolean attribute that indicates whether bundle pins are to be kept together. Applicable to pin constraint objects of type "bundle".

layers

A settable layer collection attribute that returns the set of metal layers allowed for pin placement.

length

A settable string attribute that returns the size of the pin.

location

A settable coord attribute that returns the co-ordinate in top level design where the pin is to be created.

name

A read-only string attribute that returns the name of the pin constraint.

object_class

A read-only string attribute that returns the name of this object class. Returns 'pin_constraint' for pin_constraint objects.

p

objects

A read-only net, pin, or port collection attribute that returns all the objects associated with this pin constraint.

off_edge

A read-only boolean attribute that indicates if a pin is to be created on the block boundary.

offset

A settable list_of_distance_or_percent attribute that returns the distance between the starting point of a specified edge and the pin's center location.

parent_block

A read-only block collection attribute that returns the type of objects associated with this pin constraint.

parent_cell

A read-only cell collection attribute that returns the owner cell object for this pin constraint.

pin_constraint_type

A read-only string attribute (with allowed values: bundle individual) that returns the type of pin constraint object - either "bundle" or "individual".

range

A settable string attribute that returns the position within a side within which the pins are to be placed.

sides

A read-only string attribute that returns the block sides on which the pin must be placed.

spacing_distance

A settable distance attribute that returns the minimum distance between adjacent pins or between a pin and a preroute wire that cuts across a block edge in the normal direction.

spacing_tracks

A settable int attribute that returns the minimum number of wire tracks between adjacent pins or between a pin and a preroute wire that cuts across block edge in the normal direction.

p

width

A settable string attribute that returns the width of the pin.

See Also

- [create_pin_constraint](#)
- [get_attribute](#)
- [list_attributes](#)
- [set_attribute](#)

pin_guide_attributes

Description of the application defined pin_guide attributes.

Description

Attributes are properties assigned to objects such as pins, cells, and nets. They can be read-only or settable. A read-only attribute is informational; you cannot set its value. To modify a settable attribute, use the *set_attribute* command.

To determine the value of an attribute, use the *get_attribute* command. To see a list of all pin_guide attributes available, use the *list_attributes -application -class pin_guide* command.

Attributes for pin_guide

This section describes the pin_guide attributes.

bbox

A read-only rect attribute that returns the bounding-box of the pin_guide. The format of a rectangle specification is *{{llx lly} {urx ury}}*, which specifies the lower-left and upper-right corners of the rectangle.

boundary

A settable polygon attribute that returns the boundary of the pin_guide. The format of a coordinate_list specification is *{{x1 y1} {x2 y2}...}*.

bounding_box

A read-only bounding_box collection attribute that returns the bounding-box of the pin_guide. The format of the collection specifies lower-left and upper- right corners of the rectangle.

p

full_name

A read-only string attribute that returns the name of the pin_guide.

groups

A read-only group collection attribute that returns the groups in which this pin_guide is a member.

in_edit_group

A read-only boolean attribute that returns whether the pin_guide belongs to an edit_group.

is_exclusive

A settable boolean attribute that returns whether the pin_guide is exclusive. If exclusive, the ports, pins, and ports/pins of nets constrained by this pin_guide may be placed within the pin_guide, but all others must be placed outside of the pin_guide.

is_shadow

A read-only boolean attribute that returns whether the pin_guide is pushed down from above.

layer_names

A read-only string attribute that returns the names of the layers which are constrained by the pin_guide.

layers

A settable layer collection attribute that returns the layers which are constrained by the pin_guide.

name

A settable string attribute that returns the name of the pin_guide.

nets

A settable net collection attribute that returns the nets which are constrained by the pin_guide. A pin_guide may constrain ports, pins, or nets. If nets, all of the nets' ports and pins are constrained.

object_class

A read-only string attribute that returns the name of this object class. Returns 'pin_guide' for pin_guide objects.

p

parent_block

A read-only block collection attribute that returns the owner block object for this pin_guide.

parent_block_cell

A read-only cell collection attribute that returns the parent block cell object for this pin_guide.

parent_cell

A read-only cell collection attribute that returns the owner cell object for this pin_guide.

parents

A settable cell collection attribute that returns the macro and partition cells to which the pin_guide applies. If the collection is empty, the pin_guide applies to the parent block.

physical_status

A settable string attribute (with allowed values: locked unrestricted) that returns the physical modification status of the pin_guide. If status is unrestricted, the pin_guide may be moved or rotated. If the status is locked, the pin_guide may not be moved or rotated.

pin_spacing

A settable int attribute that returns the number of wire tracks between pins within the pin_guide.

pins

A settable pin collection attribute that returns the pins which are constrained by the pin_guide. A pin_guide may constrain ports, pins, or nets. If nets, all of the nets' ports and pins are constrained.

ports

A settable port collection attribute that returns the ports which are constrained by the pin_guide. A pin_guide may constrain ports, pins, or nets. If nets, all of the nets' ports and pins are constrained.

shadow_status

A settable string attribute (with allowed values: copied_down copied_up normal pulled_up pushed_down virtual_flat) that returns the shadow status of the pin_guide.

p

top_block

A read-only block collection attribute that is the top block object for this pin_guide.

See Also

- [get_attribute](#)
- [list_attributes](#)

placement_affinity_attributes

Description of the predefined attributes for affinities.

Description

Attributes are properties assigned to objects such as pins, cells and nets. Definitions for placement_affinity attributes are provided in the subsections that follow.

Attributes are read-only unless marked as settable. Read-only attributes are informational, and you cannot set its value. A settable attribute can be modified with the *set_attribute* command.

To determine the value of an attribute, use the *get_attribute* command. To see a list of all attributes available for a class of objects use the *list_attributes -application* command.

Affinity Attributes*add_internal_logic*

Add_internal_logic setting selected when the affinity was created. Value is "true" or "false".

area

Will be defined for affinities that were created with the -area option. Attribute will consist of lower-left and upper-right coordinates.

cells

A collection attribute of the cells associated with this affinity.

edge

Will be defined for affinities that were created with the -edge option. Attribute will consist of two coordinates.

effort

Effort selected when the affinity was created. Value is one of "very_low", "low", "medium", or "high".

exclude_buffers

Exclude_buffer setting selected when the affinity was created. Value is one of "none", "edge" or "all".

full_name

A string attribute for the name of the affinity.

is_floating

Boolean value which is true when all of "location", "edge" and "area" are undefined.

location

Will be defined for affinities that were created with the -location option. Attribute will consist of a single coordinate.

name

A string attribute for the name of the affinity.

See Also

- [get_attribute](#)
- [list_attributes](#)
- [set_attribute](#)

placement_attraction_attributes

Description of the application defined placement_attraction attributes.

Description

Attributes are properties assigned to objects such as pins, cells, and nets. They can be read-only or settable. A read-only attribute is informational; you cannot set its value. To modify a settable attribute, use the *set_attribute* command.

To determine the value of an attribute, use the *get_attribute* command. To see a list of all placement_attraction attributes available, use the *list_attributes -application -class placement_attraction* command.

Attributes for placement_attraction

This section describes the placement_attraction attributes.

p

add_internal_logic

A settable boolean attribute that indicates whether the attraction was created with the `-add_internal_logic` option of the `create_placement_attraction` command.

bbox

A read-only rect attribute that returns the bounding-box of a `placement_attraction`. The format of a rectangle specification is `{{llx lly} {urx ury}}`, which specifies the lower-left and upper-right corners of the rectangle. If the attraction was created with no region, then this attribute will be undefined.

cells

A read-only cell collection attribute that contains the cells associated with this `placement_attraction`.

effort

A settable string attribute (with allowed values: `very_low` `low` `medium` `high`) that returns the effort of pulling cells to the `placement_attraction`.

exclude_buffers

A settable string attribute (with allowed values: `all` `edge` `none`) that indicates whether the attraction was created with the `-exclude_buffers` option of the `create_placement_attraction` command. The values it can take are restricted to `none`, `edge`, and `all`.

full_name

A read-only string attribute that returns the name of the `placement_attraction`.

name

A settable string attribute that returns the name of the `placement_attraction`.

object_class

A read-only string attribute that returns the name of this object class. Returns `'placement_attraction'` for `placement_attraction` objects.

points

A settable `coord_list` attribute that returns the list of coords that were specified to the `-region` option of the `create_placement_attraction` command. If `-region` was not specified, then this attribute is undefined. This attribute is settable with the same rules as the `-region` option of `create_placement_attraction`. You can also use the `remove_attribute` command to remove the "points" attribute and make this attraction floating.

See Also

- [create_placement_attraction](#)
- [get_attribute](#)
- [list_attributes](#)
- [set_attribute](#)

placement_blockage_attributes

Description of the application defined placement_blockage attributes.

Description

Attributes are properties assigned to objects such as pins, cells, and nets. They can be read-only or settable. A read-only attribute is informational; you cannot set its value. To modify a settable attribute, use the *set_attribute* command.

To determine the value of an attribute, use the *get_attribute* command. To see a list of all placement_blockage attributes available, use the *list_attributes -application -class placement_blockage* command.

Attributes for placement_blockage

This section describes the placement_blockage attributes.

anchor

A read-only collection attribute that returns the anchor for which this object is the dependent_object. When non-empty, this object is the dependent_object of the anchor, and its location is therefore constrained by the anchor's anchor_object. See the *create_anchor(2)* command man page for more details about anchors.

area

A read-only area attribute that returns the area of the placement_blockage.

bbox

A read-only rect attribute that returns the bounding-box of the placement_blockage. The format of a rectangle specification is {{llx lly} {urx ury}}, which specifies the lower-left and upper-right corners of the rectangle.

blockage_purpose

A settable string attribute (with allowed values: system user) that returns the purpose of the placement_blockage. Valid values are system and user.

blockage_type

A settable string attribute (with allowed values: `allow_buffer_only` `allow_rp_only` `category` `hard` `hard_macro` `overlap` `partial` `register` `routing_forbid_fill_only` `rp_group` `soft` `unknown`) that returns the type of the `placement_blockage`.

blocked_percentage

A settable int attribute that returns the percentage of the blockage area that is blocked. This attribute is applicable only for `allow_buffer_only`, `allow_rp_only`, `partial`, `category`, `register`, and `rp_group` `placement_blockages`.

boundary

A settable `coord_list` attribute that returns the boundary of the `placement_blockage`. The format of a `coordinate_list` specification is `{{x1 y1} {x2 y2}...}`.

bounding_box

A read-only `bounding_box` collection attribute that returns the bounding-box of the `placement_blockage`. The format of the collection specifies lower-left and upper-right corners of the rectangle.

category

A settable string attribute that returns the category name of a category `placement_blockage`.

for_floorplan_rules

A settable boolean attribute that indicates whether the `placement_blockage` is for floorplan rules.

full_name

A read-only string attribute that returns the name of the `placement_blockage`.

groups

A read-only group collection attribute that returns the groups the `placement_blockage` belongs to.

in_edit_group

A read-only boolean attribute that indicates whether the `placement_blockage` belongs to an `edit_group`.

is_derived

A settable boolean attribute that indicates whether the `placement_blockage` was automatically generated by the application. For more details, see *derive_placement_blockages* and *plan.place.auto_generate_blockages*.

is_pin_protect

A settable boolean attribute that indicates whether the application is allowed to stack macros on the edge of the hard_macro placement_blockage. If true, the macro placer is not allowed to stack macros on the edge of the blockage. This is used to prevent the macro placer from stacking macros against block pins.

is_shadow

A read-only boolean attribute that indicates whether the placement_blockage is pushed down from above.

name

A settable string attribute that returns the name of the placement_blockage.

object_class

A read-only string attribute that returns the name of this object class. Returns 'placement_blockage' for placement_blockage objects.

parent_block

A read-only block collection attribute that returns the owner block object for this placement_blockage.

parent_block_cell

A read-only cell collection attribute that returns the parent block cell object for this placement_blockage.

parent_cell

A read-only cell collection attribute that returns the owner cell object for this placement_blockage.

physical_status

A settable string attribute (with allowed values: locked unrestricted) that returns the physical modification status of the placement_blockage. If status is unrestricted, the placement_blockage may be moved or rotated. If the status is locked, the placement_blockage may not be moved or rotated.

shadow_status

A settable string attribute (with allowed values: copied_down copied_up normal pulled_up pushed_down virtual_flat) that returns the shadow status of the placement_blockage.

top_block

A read-only block collection attribute that returns the top block object for this placement_blockage.

See Also

- [get_attribute](#)
- [list_attributes](#)

poly_rect_attributes

Description of the application defined `poly_rect` attributes.

Description

Attributes are properties assigned to objects such as pins, cells, and nets. They can be read-only or settable. A read-only attribute is informational; you cannot set its value. To modify a settable attribute, use the `set_attribute` command.

To determine the value of an attribute, use the `get_attribute` command. To see a list of all `poly_rect` attributes available, use the `list_attributes -application -class poly_rect` command.

Attributes for `poly_rect`

This section describes the `poly_rect` attributes.

`area`

A read-only area attribute that reports the area contained within the `poly_rect` boundary.

`bbox`

A read-only rect attribute that returns the bounding box of the `poly_rect`.

`bbox_llx`

A read-only distance attribute that returns the lower left horizontal coordinate of the bounding box of the `poly_rect`.

`bbox_lly`

A read-only distance attribute that returns the lower left vertical coordinate of the bounding box of the `poly_rect`.

`bbox_urx`

A read-only distance attribute that returns the upper right horizontal coordinate of the bounding box of the `poly_rect`.

`bbox_ury`

A read-only distance attribute that returns the upper right vertical coordinate of the bounding box of the `poly_rect`.

bounding_box

A read-only `bounding_box` collection attribute that returns the bounding-box of the `poly_rect`. The format of the collection specifies lower-left and upper- right corners of the rectangle.

height

A read-only distance attribute that returns the height of the `poly_rect`.

is_rectangle

A read-only boolean attribute that indicates whether the `poly_rect` boundary forms a rectangle.

layer

A read-only layer collection attribute that returns the (optional) layer on which the `poly_rect` resides.

layer_name

A read-only string attribute that returns the name of the layer on which the `poly_rect` (optionally) resides.

mask

A read-only string attribute (with allowed values: `no_mask` `mask_one` `mask_two` `mask_three` `mask_four` `same_mask`) that returns the mask constraint applied to the `poly_rect`.

object_class

A read-only string attribute that returns the name of this object class. Returns 'poly_rect' for `poly_rect` objects.

point_list

A settable `coord_list` attribute that returns the points of the `poly_rect` boundary.

width

A read-only distance attribute that returns the width of the `poly_rect`.

See Also

- [create_poly_rect](#)
- [get_attribute](#)
- [list_attributes](#)
- [set_attribute](#)

port_attributes

Description of the application defined port attributes.

Description

Attributes are properties assigned to objects such as pins, cells, and nets. They can be read-only or settable. A read-only attribute is informational; you cannot set its value. To modify a settable attribute, use the *set_attribute* command.

To determine the value of an attribute, use the *get_attribute* command. To see a list of all port attributes available, use the *list_attributes -application -class port* command.

Attributes for port

This section describes the port attributes.

activity_type

A read-only string attribute that returns the switching activity type of a port. The value is based on current timer status, the attribute does not update the timer.

actual_fall_transition_max

A read-only float attribute that returns the largest max condition falling transition time for the port. The timer must be up-to-date to query this attribute value. The value is for the current_scenario only.

actual_fall_transition_min

A read-only float attribute that returns the smallest min condition falling transition time for the port. The timer must be up-to-date to query this attribute value. The value is for the current_scenario only.

actual_related_clock

A read-only string attribute that returns a clock by which the toggle rate value is referenced. The value is based on current timer status, the attribute does not update the timer. If there is no clock related to this port, it gives the fastest clock for the current mode.

actual_rise_transition_max

A read-only float attribute that returns the largest max condition rising transition time for the port. The timer must be up-to-date to query this attribute value. The value is for the current_scenario only.

actual_rise_transition_min

A read-only float attribute that returns the smallest min condition rising transition time for the port. The timer must be up-to-date to query this attribute value. The value is for the current_scenario only.

p

allow_diode_insertion

A settable boolean attribute that indicates whether diode insertion is to be skipped on the port.

antenna_area

A settable string attribute that returns the antenna area of the lib_pin.

antenna_diode_related_ground_pins

A settable port collection attribute that, for an antenna-diode cell, specifies the related ground pin of the cell. Apply the *antenna_diode_related_ground_pins* attribute to the input pin of the cell. For a cell with a built-in antenna-diode pin or port, the *antenna_diode_related_ground_pins* attribute specifies the related ground pins for the antenna-diode pin. Apply the *antenna_diode_related_ground_pins* attribute to the antenna-diode pin.

antenna_diode_related_power_pins

A settable port collection attribute that, for an antenna-diode cell, specifies the related power pin of the cell. Apply the *antenna_diode_related_power_pins* attribute to the input pin of the cell. For a cell with a built-in antenna-diode pin or port, the *antenna_diode_related_power_pins* attribute specifies the related power pins for the antenna-diode pin. Apply the *antenna_diode_related_power_pins* attribute to the antenna-diode pin.

antenna_side_area

A settable string attribute that returns a list of layer wise antenna side wall area in the format {layer value}. This is a settable attribute in library manager.

arrival_window

A read-only string attribute that returns the minimum and maximum arrivals for rise and fall transitions. To get the *arrival_window* attribute on pins that are not endpoints, set the *timing_save_pin_arrival_and_slack* variable to true. Provides the arrival time of a signal at a pin or port on the datapath. It includes the clock name and the active edge associated with the signal, and the minimum and maximum rise or fall arrival times at the clock pin, based on the active edge of the clock. If multiple clocks or multiple active edges of the same clock launch a signal to this pin, the tool reports a separate arrival window for each clock or clock edge. The arrival windows for each clock signal or active edge are returned as a list of lists from this attribute. Example: {{{clk} pos_edge {min_r_f -- --} {max_r_f 2.32464 2.3004}} {{v_clk} pos_edge {min_r_f -- --} {max_r_f 1.18975 1.07728}}}

p

bbox

A read-only rect attribute that reports the dimensions of the smallest rectangular area completely covering this port.

bias_type

A read-only string attribute (with allowed values: device_layer routing_pin) that specifies the type of bias for the port. It comes from the bias type of the net connected to this port.

bound_name

A read-only string attribute that reports the name of the bound in which the port is located. If not defined then the port is not located in any bound.

bound_type

A read-only string attribute (with allowed values: hard soft) that reports the type of the bound in which the port is located. Only defined if the port is located in a bound.

bounding_box

A read-only bounding_box collection attribute that returns the bounding-box of a port. The format of the collection specifies lower-left and upper-right corners of the rectangle.

bus

A read-only port_bus collection attribute that returns the port bus for this port, if is_bus_bit is true.

capacitance

A settable float attribute that returns the capacitance of the port.

case_value

A read-only string attribute that specifies the value of case on this port. The possible values are "0" or "1". This value may come from user defined case or netlist defined constant value. The value may be because a case value was directly set on this port or because the case value was propagated through logic to this port. If there is no case assigned or propagated to this port this attribute is not defined.

clock

A read-only boolean attribute that returns true if the lib_pin is defined as a clock in the library.

clock_arrival_window

A read-only string attribute that provides the arrival time of a clock signal at a pin or port on the clock network. It includes the clock name, the active edge type, and the minimum and maximum rise or fall arrival times at the clock pin, based on the active edge of the clock. If multiple clocks or multiple active edges of the same clock arrive at the pin, the tool reports a separate arrival window for each clock or clock edge. The arrival windows for each clock signal or active edge are returned as a list of lists from this attribute. For example: {{{SYS_CLK} pos_edge {min_r_f 1.03046 --} {max_r_f 1.03067 --}} {{REF_CLK} pos_edge {min_r_f 1.05123 --} {max_r_f 1.05742 --}}}}

clock_latency_fall_max

A read-only float attribute that returns the user-specified clock network latency for max fall. This attribute is only defined on those ports where *set_clock_latency* has been directly set.

clock_latency_fall_min

A read-only float attribute that returns the user-specified clock network latency for min fall. This attribute is only defined on those ports where *set_clock_latency* has been directly set.

clock_latency_offset_fall_max

A read-only float attribute that returns the value set on this port using the 'set_clock_latency -offset -fall -max' command. Clock latency offsets can only be set on clock sink pins and ICG clock inputs, and are used to do ideal clock network modeling of a latency offset between the entire clock tree and that particular pin.

clock_latency_offset_fall_min

A read-only float attribute that returns the same value as the attribute on pin.

clock_latency_offset_rise_max

A read-only float attribute that returns the same value as the attribute on pin.

clock_latency_offset_rise_min

A read-only float attribute that returns the same value as the attribute on pin.

clock_latency_rise_max

A read-only float attribute that returns the user-specified clock network latency for max rise. This attribute is only defined on those ports where *set_clock_latency* has been directly set.

clock_latency_rise_min

A read-only float attribute that returns the user-specified clock network latency for min rise. This attribute is only defined on those ports where *set_clock_latency* has been directly set.

clock_source_latency_early_fall_max

A read-only float attribute that returns the user-specified clock network latency for early max fall. This attribute is only defined on those ports where *set_clock_latency -source* has been directly set.

clock_source_latency_early_fall_min

A read-only float attribute that returns the user-specified clock network latency for early min fall. This attribute is only defined on those ports where *set_clock_latency -source* has been directly set.

clock_source_latency_early_rise_max

A read-only float attribute that returns the user-specified clock network latency for early max rise. This attribute is only defined on those ports where *set_clock_latency -source* has been directly set.

clock_source_latency_early_rise_min

A read-only float attribute that returns the user-specified clock network latency for early min rise. This attribute is only defined on those ports where *set_clock_latency -source* has been directly set.

clock_source_latency_late_fall_max

A read-only float attribute that returns the user-specified clock network latency for late max fall. This attribute is only defined on those ports where *set_clock_latency -source* has been directly set.

clock_source_latency_late_fall_min

A read-only float attribute that returns the user-specified clock network latency for late min fall. This attribute is only defined on those ports where *set_clock_latency -source* has been directly set.

clock_source_latency_late_rise_max

A read-only float attribute that returns the user-specified clock network latency for late max rise. This attribute is only defined on those ports where *set_clock_latency -source* has been directly set.

clock_source_latency_late_rise_min

A read-only float attribute that returns the user-specified clock network latency for late min rise. This attribute is only defined on those ports where *set_clock_latency -source* has been directly set.

clocks

A read-only collection attribute that contains a collection of clock objects that this port is in the clock network of. The clocks in the collection are just those clocks that this port fans out directly from the clock sources. It does not include the master clocks of any generated clock networks that this port is in.

clocks_with_sense

A read-only string attribute that returns a string in the form of a list of clock names and clock senses that propagate to this port. The possible clock senses are: "positive", "negative", "rise_triggered_high_pulse", "fall_triggered_high_pulse", "rise_triggered_low_pulse", "fall_triggered_low_pulse".

color

A settable string attribute that returns the color of the port (except for lib-pin). This value must be a 24-bit RGB hexadecimal string prefaced by '#', for example: "#72c4f1" or "#ce8322".

connect_within_pin

A settable string attribute (with allowed values: none via via_wire) that constrains the router when routing to a blocked pin on this port using a via or a wire. When set to via, the router cannot change the pin shape when using a via to connect to the pin. When set to via_wire, the router cannot change the pin shape when using a via or a wire to connect to the pin shape. When set to none, there is no such constraint on the router.

connection_class

A read-only string attribute that returns the connection class string for the port.

constant_value

A read-only string attribute that returns the constant value on this port. The possible values are "0" or "1". Constant values are caused by a logic constant in the netlist or by a library cell such as a pullup or pulldown having a constant logic function. A constant value is on the initial port and on any ports that have their case propagated as a result of that constant value. For ports with the attribute *constant_value* defined, the attribute *case_value* will also be defined and have the same value.

constraining_max_transition

A read-only float attribute that returns the most constraining maximum transition value from all sources at the port. This is the actual max transition constraints enforced by the tool at this port.

constraint_groups

A read-only constraint_group collection attribute that contains the same information as the attribute on pin.

cut_area

A read-only string attribute that returns the antenna cut area of the lib_pin.

design

A read-only design collection attribute that returns the design in which this port exists.

die_side

A read-only string attribute (with allowed values: back front unknown) that returns the side of the die on which the port resides. This is set based on the *preferred_pin* attribute if specified.

diff_area

A read-only string attribute that returns a list of layer wise diode protection value for an output in the format {layer value}. This is a settable attribute in library manager.

diode_lib_cell_name

A settable string attribute that specifies the diode lib cell to be used when inserting diodes during routing.

diode_protection

A settable string attribute that specifies a list of layer wise diode protection value for an external port in the format {layer value}.

direction

A settable string attribute (with allowed values: in inout internal out) that returns the direction of a port.

disable_timing

A read-only boolean attribute that returns True if the port has been marked as disable_timing by the set_disable_timing command.

dont_touch_network

A read-only boolean attribute that returns True if a dont_touch_network constraint impacts this port.

dont_touch_network_no_propagate

A read-only boolean attribute that returns True if a dont_touch_network -no_propagate constraint impacts this port.

drive_resistance_fall_max

A read-only float attribute that returns the linear drive resistance for falling delays and maximum conditions, associated with an input or inout port. Set with the set_drive command.

drive_resistance_fall_min

A read-only float attribute that returns the linear drive resistance for falling delays and minimum conditions, associated with an input or inout port. Set with the set_drive command.

drive_resistance_rise_max

A read-only float attribute that returns the linear drive resistance for rising delays and maximum conditions, associated with an input or inout port. Set with the set_drive command.

drive_resistance_rise_min

A read-only float attribute that returns floating point attributes for retrieving the values set with the set_drive command.

early_fall_arrival

A read-only float attribute that returns the arrival time for the falling edge signal at this port in early analysis.

early_fall_input_cap

A read-only float attribute that returns the early condition falling edge capacitance modeled on an output port from its set_load constraint. Returns 0 for input ports. Value is returned for the current_corner.

early_fall_load_cap

A read-only float attribute that returns the early condition falling edge capacitance seen for this port and its connected net, including net wire capacitance, input pin capacitance, and set_load capacitance on output ports. Value is returned for the current_corner.

early_fall_transition

A read-only float attribute that returns the early analysis falling transition time for the port. The timer must be up-to-date to query this attribute value. The value is for the `current_scenario` only.

early_rise_arrival

A read-only float attribute that returns the arrival time for the rising edge signal at this port in early analysis.

early_rise_input_cap

A read-only float attribute that returns the early condition rising edge capacitance modeled on an output port from its `set_load` constraint. Returns 0 for input ports. Value is returned for the `current_corner`.

early_rise_load_cap

A read-only float attribute that returns the early condition rising edge capacitance seen for this port and its connected net, including net wire capacitance, input pin capacitance, and `set_load` capacitance on output ports. Value is returned for the `current_corner`.

early_rise_transition

A read-only float attribute that returns the early analysis rising transition time for the port. The timer must be up-to-date to query this attribute value. The value is for the `current_scenario` only.

effective_process_label_early

A read-only string attribute that returns the actual process label value used for early analysis for this port in the `current_corner`. This can be different from the `set_process_label` constraint applied to this port's cell if that process label mismatches the one used for `current_corner`. If they mismatch, then the attribute `mismatched_process_label_min` on this cell will be true.

effective_process_label_late

A read-only string attribute that returns the actual process label value used for late analysis for this port in the `current_corner`. This can be different from the `set_process_label` constraint applied to this port's cell if that process label mismatches the one used for `current_corner`. If they mismatch, then the attribute `mismatched_process_label_max` on this cell will be true.

effective_process_number_early

A read-only float attribute that returns the actual process number value used for early analysis for this port in the `current_corner`. This can be different from the `set_process_number` constraint applied to this port's cell if that process number

p

mismatches the one used for `current_corner`. If they mismatch, then the attribute `mismatched_process_number_min` on this cell will be true.

effective_process_number_late

A read-only float attribute that returns the actual process number value used for late analysis for this port in the `current_corner`. This can be different from the `set_process_number` constraint applied to this port's cell if that process number mismatches the one used for `current_corner`. If they mismatch, then the attribute `mismatched_process_number_max` on this cell will be true.

effective_temperature_early

A read-only float attribute that returns the actual temperature value used for early analysis for this port in the `current_corner`. This can be different from the `set_temperature` constraint applied to this port's cell if that temperature mismatches the one used for `current_corner`. If they mismatch, then the attribute `mismatched_temperature_min` on this cell will be true.

effective_temperature_late

A read-only float attribute that returns the actual temperature value used for late analysis for this port in the `current_corner`. This can be different from the `set_temperature` constraint applied to this port's cell if that temperature mismatches the one used for `current_corner`. If they mismatch, then the attribute `mismatched_temperature_max` on this cell will be true.

effective_voltage_early

A read-only float attribute that returns the actual voltage value used for early analysis for this port in the `current_corner`. This can be different from the `set_voltage` constraint applied to this port's cell if that voltage mismatches the one used for `current_corner`. If they mismatch, then the attribute `mismatched_voltage_min` on this cell will be true.

effective_voltage_late

A read-only float attribute that returns the actual voltage value used for late analysis for this port in the `current_corner`. This can be different from the `set_voltage` constraint applied to this port's cell if that voltage mismatches the one used for `current_corner`. If they mismatch, then the attribute `mismatched_voltage_max` on this cell will be true.

escaped_full_name

A read-only string attribute that returns the full name of the port with any literal hierarchy characters escaped with a backslash.

essential_activity_group

A read-only string attribute that returns the essential activity group of the specified port.

estimated_analysis

A read-only string attribute that indicates what estimation techniques are applied by the command "estimate_timing" at the driver cell or net of the port. It includes "SIZE", "BUFFERS" etc.

extended_name

A read-only string attribute that reports the port name prepended with the source object name.

fanout_load

A settable float attribute that returns the fanout load on output ports. Set with the set_fanout_load command.

full_name

A read-only string attribute that returns the full name of the port including a prefix for the instance path to this port.

function

A read-only string attribute that returns the output or inout lib_pins for the logic function. The logic function is defined in the library.

gate_area

A settable string attribute that returns a list of layer wise gate area for an input pin in the format {layer value}. If library supports multiple gate oxide modes then oxide mode is also present like {oxide_mode layer value}.

gate_diffusion_length

A settable string attribute that returns a list of layer wise gate diffusion length in the format {layer value}. This is a settable attribute in library manager.

glitch_rate

A read-only double attribute that returns the rate at which the glitch transitions occur on the pin within a specific time period.

group_bound_height

A read-only distance attribute that reports the height of the group bound in which this port is located. Only defined if the port is located in a group bound.

p

group_bound_width

A read-only distance attribute that reports the width of the group bound in which this port is located. Only defined if the port is located in a group bound.

groups

A read-only group collection attribute that returns the groups of which this port is a member.

hdl_file

A read-only string attribute that returns the original RTL file name for the port.

hdl_line

A read-only string attribute that returns the original RTL file line number for the port.

hdl_origin

A read-only string attribute that returns the original RTL creator associated with the port.

hold_uncertainty

A read-only float attribute that returns the clock uncertainty (skew) of a clock used for hold (and other minimum delay) timing checks. Set with the `set_clock_uncertainty` command.

input_threshold_pct_fall

A read-only float attribute that returns a value between 0.0 and 100.0. It specifies the threshold point of input signal falling from 1 to 0 on this `lib_pin`. Note that this may be a library-level value.

input_threshold_pct_rise

A read-only float attribute that returns a value between 0.0 and 100.0. It specifies the threshold point of input signal rising from 0 to 1 on this `lib_pin`. Note that this may be a library-level value.

input_transition_fall_max

A read-only float attribute that returns the fixed transition time for max condition falling delays associated with an input or inout port. Set with the `set_input_transition` command.

input_transition_fall_min

A read-only float attribute that returns the fixed transition time for min condition falling delays associated with an input or inout port. Set with the `set_input_transition` command.

input_transition_rise_max

A read-only float attribute that returns the fixed transition time for max condition rising delays associated with an input or inout port. Set with the `set_input_transition` command.

input_transition_rise_min

A read-only float attribute that returns the fixed transition time for min condition rising delays associated with an input or inout port. Set with the `set_input_transition` command.

is_async_pin

A read-only boolean attribute that returns true if the port is asynchronous.

is_auto_floorplan_modified

A read-only boolean attribute that returns true if the port is modified by auto-floorplanning flow.

is_bus_bit

A read-only boolean attribute that returns true if the port is part of a port bus.

is_clear_pin

A read-only boolean attribute that returns true if the port is clear.

is_clock_exclusivity

A read-only boolean attribute that returns true if the port has been set as a clock exclusivity point either manually using the `set_clock_exclusivity` command, or automatically through the `enable_auto_mux_clock_exclusivity` app option.

is_clock_pin

A read-only boolean attribute that returns true if this `lib_pin` is a clock pin of a register. A clock pin is any pin that has a timing check from it to a data pin.

is_clock_used_as_clock

A read-only boolean attribute that returns true if this port is part of at least one clock network and the clock network fanout of this port reaches at least one register clock port or output port.

is_clock_used_as_data

A read-only boolean attribute that returns true if this port is part of at least one clock network and the clock network fanout of this port reaches at least one register data port. If the attribute values for `is_clock_used_as_data` is TRUE and `is_clock_used_as_clock` is FALSE, the clocks through this port only acts a data and never as clock.

p

is_data_pin

A read-only boolean attribute that returns True for data ports of registers. A data port is any port that has a timing check to it. Data ports are legal endpoint for timing paths.

is_diode

A read-only boolean attribute that returns True if the port is a diode pin based on its lib_cell.

is_fall_edge_triggered_clock_pin

A read-only boolean attribute that returns True if port is a fall edge triggered clock port.

is_fall_edge_triggered_data_pin

A read-only boolean attribute that returns True if port is a fall edge triggered data port.

is_feedthrough_port

A read-only boolean attribute that returns True if the port is a feedthrough port created by place_pins, optimize_topology_plans, or push_down_objects.

is_fixed

A read-only boolean attribute that returns True if the physical status of this port is fixed or locked.

is_ideal

A read-only boolean attribute that returns true if the port has been marked ideal using the set_ideal_network command.

is_latch_loop_breaker

A read-only boolean attribute that returns true if the pin is the D pin of a loop-breaker latch.

is_negative_level_sensitive_clock_pin

A read-only boolean attribute that returns True if the port is a low level sensitive clock port.

is_negative_level_sensitive_data_pin

A read-only boolean attribute that returns True if the port is a low level sensitive data port.

is_net_driver

A read-only boolean attribute that returns True if the port is a net driver.

p

is_net_load

A read-only boolean attribute that returns True if the port is a net load.

is_pad

A settable boolean attribute that returns True if the port is a pad IO based on its lib_cell.

is_physical

A settable boolean attribute that returns True if the lib pin is a physical port.

is_positive_level_sensitive_clock_pin

A read-only boolean attribute that returns True if the port is a high level sensitive clock port.

is_positive_level_sensitive_data_pin

A read-only boolean attribute that returns True if the port is a high level sensitive data port.

is_preset_pin

A read-only boolean attribute that returns True if the port is a preset port.

is_rectangle_only_rule_waived

A settable boolean attribute that returns True if the rectangle only rule needs to be waived for this port.

is_rise_edge_triggered_clock_pin

A read-only boolean attribute that returns True if port is a rise edge triggered clock port.

is_rise_edge_triggered_data_pin

A read-only boolean attribute that returns True if port is a rise edge triggered data port.

is_scan

A settable boolean attribute that returns True if the port is a scan port.

is_secondary_pg

A read-only boolean attribute that returns True if the port is secondary PG.

is_shadow

A read-only boolean attribute that returns True if the port is pushed down from above.

p

is_three_state

A read-only boolean attribute that returns True if this port is a three-state output (can become high impedance when not enabled).

is_three_state_enable_pin

A read-only boolean attribute that returns True if the port is a three-state enable port.

is_three_state_output_pin

A read-only boolean attribute that returns True if this port is a three-state output (can become high impedance when not enabled).

is_user_pg

A settable boolean attribute that returns True if this port is marked as a user-created PG port. PG ports coming in from Verilog are marked as user created, unless this attribute is subsequently modified.

late_fall_arrival

A read-only float attribute that returns the arrival time for the falling edge signal at this port in late analysis.

late_fall_input_cap

A read-only float attribute that returns the late condition falling edge capacitance modeled on an output port from its set_load constraint. Returns 0 for input ports. Value is returned for the current_corner.

late_fall_load_cap

A read-only float attribute that returns the late condition falling edge capacitance seen for this port and its connected net, including net wire capacitance, input pin capacitance, and set_load capacitance on output ports. Value is returned for the current_corner.

late_fall_transition

A read-only float attribute that returns the late analysis falling transition time for the port. The timer must be up-to-date to query this attribute value. The value is for the current_scenario only.

late_rise_arrival

A read-only float attribute that returns the arrival time for the rising edge signal at this port in late analysis.

p

late_rise_input_cap

A read-only float attribute that returns the late condition rising edge capacitance modeled on an output port from its set_load constraint. Returns 0 for input ports. Value is returned for the current_corner.

late_rise_load_cap

A read-only float attribute that returns the late condition rising edge capacitance seen for this port and its connected net, including net wire capacitance, input pin capacitance, and set_load capacitance on output ports. Value is returned for the current_corner.

late_rise_transition

A read-only float attribute that returns the late analysis rising transition time for the port. The timer must be up-to-date to query this attribute value. The value is for the current_scenario only.

layer

A read-only layer collection attribute that returns all the tech layers associated with the lib_pin.

layer_name

A read-only string attribute that returns all the tech layers associated with the lib_pin.

lib

A read-only lib collection attribute that returns the library in which the object on which the port is defined is defined.

lib_cell

A read-only lib_cell collection attribute that returns the lib cell design on which this port is defined. Only defined if the port is defined on a lib cell.

location

A read-only coord attribute that returns the physical location of the port.

major_activity_type

A read-only string attribute that returns the major switching activity type of this port.

max_area_car

A read-only string attribute that returns a list of layer wise maximum cumulative area ratio on the layer using the metal area at or below the layer excluding the pin area itself along with oxide mode in the format {oxide_mode layer value}.

max_balance_delay

A read-only float attribute that returns the same value as the attribute on pin.

max_capacitance

A read-only float attribute that returns the max_capacitance constraint set on the port by the *set_max_capacitance* command. Value is returned for the current_corner.

max_capacitance_constraint

A read-only float attribute that returns the max_capacitance constraint set on the port by the *set_max_capacitance* command. Value is returned for the current_corner.

max_cut_area_car

A read-only string attribute that returns a list of layer wise maximum cumulative antenna ratio on the layer using the cut area at or below the layer excluding the pin area itself along with oxide mode in the format {oxide_mode layer value}.

max_fall_arrival

A read-only float attribute that returns the arrival time for the falling edge signal at this port in max analysis.

max_fall_input_cap

A read-only float attribute that returns the max condition falling edge capacitance modeled on an output port from its set_load constraint. Returns 0 for input ports. Value is returned for the current_corner.

max_fall_load_cap

A read-only float attribute that returns the max condition falling edge capacitance seen for this port and its connected net, including net wire capacitance, input pin capacitance, and set_load capacitance on output ports. Value is returned for the current_corner.

max_fall_local_slack

A read-only float attribute that returns the local slack for a pin of a transparent latch, considering the data arrival at the local pin and setup constraints at the local pin, without considering constraints downstream from the pin. If there are no constraints at the pin, the attribute is undefined. The normal slack (max_rise_slack or max_slack) considers the constraints downstream from the pin. This attribute gives the local slack for falling max path analysis.

p

max_fall_slack

A read-only float attribute that returns the worst slack at a pin for falling maximum path delays. This attribute is valid only after timing has been updated.

max_fall_transition

A read-only float attribute that returns the largest max condition falling transition time for the port. The timer must be up-to-date to query this attribute value. The value is for the current_scenario only.

max_fanout

A settable float attribute that returns the maximum fanout load for the net connected to this port. PrimeTime ensures that the fanout load on this port is less than the specified value. Set with the set_max_fanout command

max_rise_arrival

A read-only float attribute that returns the arrival time for the rising edge signal at this port in max analysis.

max_rise_input_cap

A read-only float attribute that returns the max condition rising edge capacitance modeled on an output port from its set_load constraint. Returns 0 for input ports. Value is returned for the current_corner.

max_rise_load_cap

A read-only float attribute that returns the max condition rising edge capacitance seen for this port and its connected net, including net wire capacitance, input pin capacitance, and set_load capacitance on output ports. Value is returned for the current_corner.

max_rise_local_slack

A read-only float attribute that returns the local slack for a pin of a transparent latch, considering the data arrival at the local pin and setup constraints at the local pin, without considering constraints downstream from the pin. If there are no constraints at the pin, the attribute is undefined. The normal slack (max_rise_slack or max_slack) considers the constraints downstream from the pin. This attribute gives the local slack for rising max path analysis.

max_rise_slack

A read-only float attribute that returns the worst slack at a pin for rising maximum path delays. This attribute is valid only after timing has been updated.

max_rise_transition

A read-only float attribute that returns the largest max condition rising transition time for the port. The timer must be up-to-date to query this attribute value. The value is for the current_scenario only.

max_side_area_car

A read-only string attribute that returns a list of layer wise maximum cumulative antenna ratio on the layer using the metal side wall area at or below the layer excluding the pin area itself along with oxide mode in the format {oxide_mode layer value}.

max_slack

A read-only float attribute that returns the worst slack value between the max_rise_slack and max_fall_slack attributes.

max_transition

A read-only float attribute that returns the max_transition constraint set on the port by the *set_max_transition* command. The actual max_transition constraint enforced at the port is in the constraining_max_transition attribute.

max_transition_constraint

A read-only float attribute that returns the max_transition constraint set on the port by the *set_max_transition* command. The actual max_transition constraint enforced at the pin is in the constraining_max_transition attribute.

metrics_tim_logic_levels_viol

A read-only int attribute that returns if the worst timing path ending at this port has logic level violation or not. Returns 0 or 1 accordingly. Works on current scenario.

metrics_tim_logic_levels_viol_local

A read-only int attribute that returns if the worst timing path ending at this port has logic level violation or not. Returns 0 or 1 accordingly. Works on current scenario.

metrics_tim_net_length_viol

A read-only string attribute that returns the space separated string of path-group-names which for which the worst timing paths ending at this pin has net length violations. Something like <pg1> <pg3> <pg4> etc. Works on current scenario.

metrics_tim_path_length_viol

A read-only string attribute that returns the space separated string of path-group-names for which the worst timing paths ending at this pin has path length violations. Something like <pg1> <pg3> <pg4> etc. Works on current scenario.

metrics_tim_repeater_levels_viol

A read-only int attribute that returns if the worst timing path ending at this pin has repeater level violation or not. Returns 0 or 1 accordingly. Works on current scenario.

metrics_tim_repeater_levels_viol_local

A read-only int attribute that returns if the worst timing path ending at this pin has repeater level violation or not. Returns 0 or 1 accordingly. Works on current scenario.

min_balance_delay

A read-only float attribute that returns the balance point delay value set by the user on a clock port using 'set_clock_balance_points -delay \$value -early'.

min_capacitance

A read-only float attribute that returns the minimum capacitance value for input and bidirectional ports. Set with the set_min_capacitance command. Value is returned for the current_corner.

min_capacitance_constraint

A read-only float attribute that returns the minimum capacitance value for input and bidirectional ports. Set with the set_min_capacitance command. Value is returned for the current_corner.

min_fall_arrival

A read-only float attribute that returns the arrival time for the falling edge signal at this port in min analysis.

min_fall_input_cap

A read-only float attribute that returns the min condition falling edge capacitance modeled on an output port from its set_load constraint. Returns 0 for input ports. Value is returned for the current_corner.

min_fall_load_cap

A read-only float attribute that returns the min condition falling edge capacitance seen for this port and its connected net, including net wire capacitance, input pin capacitance, and set_load capacitance on output ports. Value is returned for the current_corner.

p

min_fall_slack

A read-only float attribute that returns the worst slack at a pin for falling minimum path delays. This attribute is valid only after timing has been updated.

min_fall_transition

A read-only float attribute that returns the smallest min condition falling transition time for the port. The timer must be up-to-date to query this attribute value. The value is for the current_scenario only.

min_rise_arrival

A read-only float attribute that returns the arrival time for the rising edge signal at this port in min analysis.

min_rise_input_cap

A read-only float attribute that returns the min condition rising edge capacitance modeled on an output port from its set_load constraint. Returns 0 for input ports. Value is returned for the current_corner.

min_rise_load_cap

A read-only float attribute that returns the min condition rising edge capacitance seen for this port and its connected net, including net wire capacitance, input pin capacitance, and set_load capacitance on output ports. Value is returned for the current_corner.

min_rise_slack

A read-only float attribute that returns the worst slack at a pin for rising minimum path delays. This attribute is valid only after timing has been updated.

min_rise_transition

A read-only float attribute that returns the smallest min condition rising transition time for the port. The timer must be up-to-date to query this attribute value. The value is for the current_scenario only.

min_slack

A read-only float attribute that returns the worst slack value between the min_rise_slack and min_fall_slack attributes.

minor_activity_type

A read-only string attribute that returns the minor switching activity type of this port.

mismatched_process_label_max

A read-only boolean attribute that returns True if there is a mismatch between the process label applied to this port from the `set_process_label` constraint for max analysis, and the process label used for the `current_corner`.

mismatched_process_label_min

A read-only boolean attribute that returns True if there is a mismatch between the process label applied to this port from the `set_process_label` constraint for min analysis, and the process label used for the `current_corner`.

mismatched_process_number_max

A read-only boolean attribute that returns True if there is a mismatch between the process number applied to this port from the `set_process_number` constraint for max analysis, and the process number used for the `current_corner`.

mismatched_process_number_min

A read-only boolean attribute that returns True if there is a mismatch between the process number applied to this port from the `set_process_number` constraint for min analysis, and the process number used for the `current_corner`.

mismatched_pvt_max

A read-only boolean attribute that returns True if there is any mismatch in the PVT conditions applied to this port for max analysis, and the PVT conditions used for the `current_corner`.

mismatched_pvt_min

A read-only boolean attribute that returns True if there is any mismatch in the PVT conditions applied to this port for min analysis, and the PVT conditions used for the `current_corner`.

mismatched_temperature_max

A read-only boolean attribute that returns True if there is a mismatch between the temperature applied to this port from the `set_temperature` constraint for max analysis, and the temperature used for the `current_corner`.

mismatched_temperature_min

A read-only boolean attribute that returns True if there is a mismatch between the temperature applied to this port from the `set_temperature` constraint for min analysis, and the temperature used for the `current_corner`.

mismatched_voltage_max

A read-only boolean attribute that returns True if there is a mismatch between the voltage applied to this port from the `set_voltage` constraint for max analysis, and the voltage used for the `current_corner`.

mismatched_voltage_min

A read-only boolean attribute that returns True if there is a mismatch between the voltage applied to this port from the `set_voltage` constraint for min analysis, and the voltage used for the `current_corner`.

mode1_area

A read-only string attribute that returns a list of layer wise metal area when connectivity is traced upto that metal layer along with oxide mode in the format {oxide_mode layer value}. This is a settable attribute in library manager.

mode2_ratio

A read-only string attribute that returns a list of layer wise maximum internal ratio of area of metal layer below the specified metal layer when connectivity is traced upto that metal layer along with oxide mode in the format {oxide_mode layer value}. This is a settable attribute in library manager.

mode3_area

A read-only string attribute that returns a list of layer wise total metal area along with oxide mode in the format {oxide_mode layer value}. This is a settable attribute in library manager.

mode4_area

A read-only string attribute that returns a list of layer wise side wall metal area when connectivity is traced upto that metal layer along with oxide mode in the format {oxide_mode layer value}. This is a settable attribute in library manager.

mode5_ratio

A read-only string attribute that returns a list of layer wise maximum internal ratio of side wall area of metal layer below the specified metal layer when connectivity is traced upto that metal layer along with oxide mode in the format {oxide_mode layer value}. This is a settable attribute in library manager.

mode6_area

A read-only string attribute that returns a list of layer wise total side wall metal area along with oxide mode in the format {oxide_mode layer value}. This is a settable attribute in library manager.

must_join_port

A read-only port collection attribute that returns the must join port of this pin.

n_gate_area

A settable string attribute that returns a list of layer wise gate area of n-channel transistors for an input pin in the format {layer value}. If library supports multiple gate oxide modes then oxide mode is also present like {oxide_mode layer value}. This is a settable attribute in library manager.

n_gate_diffusion_length

A settable string attribute that returns a list of layer wise gate diffusion length of n-channel transistors in the format {layer value}. This is a settable attribute in library manager.

name

A settable string attribute that returns the name of this port.

net

A read-only net collection attribute that returns the net to which this port is connected.

net_name

A read-only string attribute that returns the name of the net to which this port is connected.

nextstate_type

A settable int attribute that defines the type of next_state attribute used in the ff or ff_bank group.

number_of_diodes_inserted

A read-only unsigned attribute that returns the number of diodes already inserted by the router for the port.

number_of_diodes_required

A settable unsigned attribute that specifies the number of diodes to be inserted during routing.

object_class

A read-only string attribute that returns the name of this object class. Returns 'port' for port objects.

p

output_threshold_pct_fall

A read-only float attribute that returns the value of the threshold point on an output pin signal falling from 1 to 0.

output_threshold_pct_rise

A read-only float attribute that returns the value of the threshold point on an output pin signal rising from 0 to 1.

p_gate_area

A settable string attribute that returns a list of layer wise gate area of p-channel transistors for an input pin in the format {layer value}. If library supports multiple gate oxide modes then oxide mode is also present like {oxide_mode layer value}. This is a settable attribute in library manager.

p_gate_diffusion_length

A settable string attribute that returns a list of layer wise gate diffusion length of p-channel transistors in the format {layer value}. This is a settable attribute in library manager.

pa_input_pin_fall_transition

A read-only float attribute that returns the fall transition used in power analysis at the node related to the specified port. The value is based on current timer status, the attribute does not update the timer.

pa_input_pin_rise_transition

A read-only float attribute that returns the rise transition used in power analysis at the node related to the specified port. The value is based on current timer status, the attribute does not update the timer.

pa_output_pin_cap

A read-only float attribute that returns the capacitance used in power analysis on the specified port. The value is based on current timer status, the attribute does not update the timer. Value is returned for the current_corner.

pa_pin_voltage

A read-only float attribute that returns the voltage used in power analysis of the rail attached to the specified port.

pane_early

A read-only int attribute that returns the timing pane number used for early analysis on this port in current_corner.

p

pane_late

A read-only int attribute that returns the timing pane number used for early analysis on this port in current_corner.

parent_block

A read-only block collection attribute that returns the owner block object for this port.

parent_block_cell

A read-only cell collection attribute that returns the parent block cell object for this port.

parent_cell

A read-only cell collection attribute that returns the owner cell object for this port.

pattern_must_join

A settable boolean attribute that returns whether this is marked as pattern_must_join.

pattern_must_join_level

A settable int attribute that returns the level of pattern_must_join.

pg_function

A read-only string attribute that returns the PG pin. It is to model that internal PG pin is related to a source PG pin.

pg_type

A read-only string attribute (with allowed values: backup internal primary) that specifies the PG type of the port. It comes from the PG type of the net connected to this port.

physical_hierarchy_level

A read-only int attribute that specifies the depth of the physical hierarchy.

physical_status

A settable string attribute (with allowed values: fixed locked placed unplaced) that returns the physical placement status of the port. Valid values are unplaced, placed, fixed, and locked. If status is unplaced, the port has not been placed. If status is placed, the port has been placed and may be moved. If status is fixed, the port is fixed for automatic changes. If status is locked, the port is fixed for automatic and manual changes.

p

pin_capacitance

A read-only float attribute that returns the capacitance of this port. Value is returned for the `current_corner`.

pin_capacitance_max_fall

A read-only float attribute that returns the max condition falling edge capacitance modeled on an output port from its `set_load -pin_load` constraint. Value is returned for the `current_corner`.

pin_capacitance_max_rise

A read-only float attribute that returns the max condition rising edge capacitance modeled on an output port from its `set_load -pin_load` constraint. Value is returned for the `current_corner`.

pin_capacitance_min_fall

A read-only float attribute that returns the min condition falling edge capacitance modeled on an output port from its `set_load -pin_load` constraint. Value is returned for the `current_corner`.

pin_capacitance_min_rise

A read-only float attribute that returns the min condition rising edge capacitance modeled on an output port from its `set_load -pin_load` constraint. Value is returned for the `current_corner`.

pin_class

A read-only int attribute that returns the pin class of the `lib_pin`.

pin_direction

A read-only string attribute (with allowed values: `in`, `inout`, `internal`, `out`) that specifies the direction of a pin. Valid values are *in*, *out*, *inout*, or *unknown*. This is the same as "direction". This attribute is supported for compatibility with other tools. The "direction" attribute is supported for both ports and pins and therefore easier to use for heterogeneous collections of pins and ports.

pin_function_class

A read-only string attribute that returns the pin function class of the `lib_pin`. This is the canonical representation of the logic function of the `lib_pin`.

pin_number

A read-only int attribute that returns the pin number of the `lib_pin`.

p

port_direction

A read-only string attribute that returns the direction of a port. Possible values are *in*, *out*, *inout*, or *unknown*. This is the same as "direction". This attribute is supported for compatibility with other tools. The "direction" attribute is supported for both ports and pins and therefore easier to use for heterogeneous collections of pins and ports.

port_type

A settable string attribute (with allowed values: *analog_ground* *analog_power* *analog_signal* *clock* *deep_nwell* *deep_pwell* *ground* *nwell* *power* *pwell* *reset* *scan* *signal* *tie_high* *tie_low* *unset*) that returns the port type of a *lib_pin*.

power_domain

A read-only string attribute that returns the power domain name of the given port.

power_rail_voltage_max

A read-only float attribute that returns the voltage value on port or pin object for maximum condition.

power_rail_voltage_min

A read-only float attribute that returns the voltage value on port or pin object for minimum condition.

preferred_pin

A settable pin collection attribute that returns the pad pin which owns this port's shape. This exists only on a port which does not have any shapes of its own. Its terminals were automatically generated by tool and reference a pad pin shape in the lower lower. This attribute is used to indicate that one and only one of those pad pins is used as this port's shape. If this attribute is not set, this port uses all available pad pins connected to the port. See the terminal *is_owned_by_pad* attribute for more information.

process_label_early

A read-only string attribute that returns the process label constraint for early analysis for this port in the *current_corner*. Set by the *set_process_label* constraint.

process_label_early_file_line_info

A read-only string attribute that returns a Tcl list of two elements, the first showing the source file path and the second showing the source file line number, where the *set_process_label* constraint for early analysis was applied on this port.

p

process_label_early_source

A read-only string attribute that indicates the source scope of the constraint that gave this port its early analysis process label setting from the `set_process_label` command. The source could be from a constraint on the design, a hierarchical cell, a library cell, or a leaf cell.

process_label_late

A read-only string attribute that returns the process label constraint for late analysis for this port in the `current_corner`. Set by the `set_process_label` constraint.

process_label_late_file_line_info

A read-only string attribute that returns a Tcl list of two elements, the first showing the source file path and the second showing the source file line number, where the `set_process_label` constraint for late analysis was applied on this port.

process_label_late_source

A read-only string attribute that indicates the source scope of the constraint that gave this port its late analysis process label setting from the `set_process_label` command. The source could be from a constraint on the design, a hierarchical cell, a library cell, or a leaf cell.

process_number_early

A read-only float attribute that returns the process label constraint for early analysis for this port in the `current_corner`. Set by the `set_process_number` constraint.

process_number_early_file_line_info

A read-only string attribute that returns a Tcl list of two elements, the first showing the source file path and the second showing the source file line number, where the `set_process_number` constraint for early analysis was applied on this port.

process_number_early_source

A read-only string attribute that indicates the source scope of the constraint that gave this port its early analysis process number setting from the `set_process_number` command. The source could be from a constraint on the design, a hierarchical cell, a library cell, or a leaf cell.

p

process_number_late

A read-only float attribute that returns the process number constraint for late analysis for this port in the current_corner. Set by the set_process_number constraint.

process_number_late_file_line_info

A read-only string attribute that returns a Tcl list of two elements, the first showing the source file path and the second showing the source file line number, where the set_process_number constraint for late analysis was applied on this port.

process_number_late_source

A read-only string attribute that indicates the source scope of the constraint that gave this port its late analysis process number setting from the set_process_number command. The source could be from a constraint on the design, a hierarchical cell, a library cell, or a leaf cell.

propagated_clock

A read-only boolean attribute that returns true if clock edge times are delayed by propagating the values through the clock network. Affects all sequential cells in the transitive fanout of this port. If this attribute is not present, PrimeTime assumes ideal clocking. Set with set_propagated_clock command.

rail_name

A settable string attribute that returns the rail name of the lib pin.

related_bias_pin

A settable port collection attribute that defines all bias pins associated with a power or ground lib_pin.

related_bias_pin_name

A settable string attribute that returns all bias pins associated with a power or ground port.

related_clock

A read-only string attribute that returns the related ground pin of the lib_pin.

related_ground_pin

A settable port collection attribute that returns the related power pin of the lib_pin.

related_ground_pin_name

A settable string attribute that returns the related ground pin of the port.

related_power_pin

A settable port collection attribute that returns the related power pin of the lib_pin.

related_power_pin_name

A settable string attribute that returns the related power pin of the port.

related_supply_default_primary

A read-only boolean attribute that indicates the related supply of the port is assumed as primary by default.

scan_signals

A read-only scan_signal collection attribute that returns the scan_signals which contain this port.

setup_uncertainty

A read-only float attribute that returns the clock uncertainty (skew) of a clock used for setup (and other maximum delay) timing checks. Set with the set_clock_uncertainty command.

shadow_status

A settable string attribute (with allowed values: copied_down copied_up normal pulled_up pushed_down virtual_flat) that returns the shadow status of this pin.

signal_type

A read-only string attribute that returns the signal type of the test lib_pin.

skew_groups

A read-only collection attribute that captures the user defined skew group that the port belongs to.

skip_mv_check_on_port_for_dedicated_core_wrapper

A settable boolean attribute that, if set, indicates MV checks should not be performed for identifying legal UPF aware location for dedicated wrapper cell during DFT Scan Insertion step (compile/insert_dft).

slew_derate_from_library

A read-only float attribute that specifies how the transition times found in the library need to be derated to match the transition times between the characterization trip points.

p

slew_lower_threshold_pct_fall

A read-only float attribute that returns a value between 0.0 and 100.0. It specifies the lower trip point for measuring the transition (slew) time of a falling signal on this lib_pin. Note that this may be a library-level value.

slew_lower_threshold_pct_rise

A read-only float attribute that returns a value between 0.0 and 100.0. It specifies the lower trip point for measuring the transition (slew) time of a rising signal on this lib_pin. Note that this may be a library-level value.

slew_upper_threshold_pct_fall

A read-only float attribute that returns a value between 0.0 and 100.0. It specifies the upper trip point for measuring the transition (slew) time of a falling signal on this lib_pin. Note that this may be a library-level value.

slew_upper_threshold_pct_rise

A read-only float attribute that returns a value between 0.0 and 100.0. It specifies the upper trip point for measuring the transition (slew) time of a rising signal on this lib_pin. Note that this may be a library-level value.

static_probability

A read-only double attribute that returns the switching activity probability of a port. The value is based on current timer status, the attribute does not update the timer.

switch_function

A read-only string attribute that returns the switch function expression of the lib_pin.

temperature_early

A read-only float attribute that returns the temperature constraint for early analysis for this port in the current_corner. Set by the set_temperature constraint.

temperature_early_file_line_info

A read-only string attribute that returns a Tcl list of two elements, the first showing the source file path and the second showing the source file line number, where the set_temperature constraint for early analysis was applied on this port.

temperature_early_source

A read-only string attribute that indicates the source scope of the constraint that gave this port its early analysis temperature setting from the set_temperature

p

command. The source could be from a constraint on the design, a hierarchical cell, a library cell, or a leaf cell.

temperature_late

A read-only float attribute that returns the temperature constraint for late analysis for this port in the current_corner. Set by the set_temperature constraint.

temperature_late_file_line_info

A read-only string attribute that returns a Tcl list of two elements, the first showing the source file path and the second showing the source file line number, where the set_temperature constraint for late analysis was applied on this port.

temperature_late_source

A read-only string attribute that indicates the source scope of the constraint that gave this port its late analysis temperature setting from the set_temperature command. The source could be from a constraint on the design, a hierarchical cell, a library cell, or a leaf cell.

test_output_only

A read-only boolean attribute that indicates whether any output lib pin is described in statetable format. When set to true specify that a lib_pin is for test only.

three_state_function

A read-only string attribute that returns the output or inout lib_pins for the three_state logic function. The three_state logic function is defined in the library. If the three_state logic function evaluates to zero, then the lib_pin is enabled.

toggle_rate

A read-only double attribute that returns the switching activity toggle rate of a port. The value is based on current timer status, the attribute does not update the timer.

top_block

A read-only block collection attribute that returns the top block object for this port.

UPF_clamp_value

A read-only string attribute that specifies the clamp value to be used if the port has an isolation strategy applied to it.

UPF_driver_supply

A read-only collection attribute that specifies the driver supply of a macro cell output port, or the assumed driver supply of an external logic driving a primary input. Ignored if port is not on a terminal boundary or not top level

UPF_feedthrough

A read-only collection attribute that identifies a set of ports on the interface of a module or cell that are directly connected to a port inside the module or cell.

UPF_is_analog

A read-only boolean attribute that identifies a signal port on the interface of a module or cell that is an analog port.

UPF_literal_supply

A read-only collection attribute that identifies the supply set to be used to implement a literal constant value associated with an input port of an instance.

UPF_receiver_supply

A read-only collection attribute that specifies the receiver supply of a macro cell input port, or the assumed receiver supply of an external logic receiving a primary input. Ignored if port is not on a terminal boundary or not top level

UPF_related_supply_set

A read-only collection attribute that returns the supply set that is related to.

UPF_unconnected

A read-only boolean attribute that identifies a port on the interface of a module or cell that is not connected to either a source or a sink within the cell, and that is not connected to any other port on the interface of the module or cell.

user_case_value

A read-only string attribute that returns the case value set by the command *set_case_analysis*. This attribute is only defined on those ports where *set_case_analysis* has been directly set. Use the attribute "case_value" for both directly set case values and propagated case values.

user_clock_sense

A read-only string attribute that returns the sense set by the command *set_clock_sense*. This attribute is only defined on those ports where the *set_clock_sense* command was used directly.

via_ladder_min_layer_for_bottom

A read-only layer collection attribute that returns the tech layer used as the layer constraint for bottom layer connection to a via ladder.

voltage_early

A read-only float attribute that returns the voltage constraint for early analysis for this port in the current_corner. Set by the set_voltage constraint.

voltage_early_file_line_info

A read-only string attribute that returns a Tcl list of two elements, the first showing the source file path and the second showing the source file line number, where the set_voltage constraint for early analysis was applied on this port.

voltage_early_source

A read-only string attribute that indicates the source scope of the constraint that gave this port its early analysis voltage setting from the set_voltage command. The source could be from a constraint on the design, a hierarchical cell, a library cell, or a leaf cell.

voltage_late

A read-only float attribute that returns the voltage constraint for late analysis for this port in the current_corner. Set by the set_voltage constraint.

voltage_late_file_line_info

A read-only string attribute that returns a Tcl list of two elements, the first showing the source file path and the second showing the source file line number, where the set_voltage constraint for late analysis was applied on this port.

voltage_late_source

A read-only string attribute that indicates the source scope of the constraint that gave this port its late analysis voltage setting from the set_voltage command. The source could be from a constraint on the design, a hierarchical cell, a library cell, or a leaf cell.

wire_capacitance_max_fall

A read-only float attribute that returns the max condition falling edge capacitance modeled on an output port from its set_load -wire_load constraint. Value is returned for the current_corner.

p

wire_capacitance_max_rise

A read-only float attribute that returns the max condition rising edge capacitance modeled on an output port from its set_load -wire_load constraint. Value is returned for the current_corner.

wire_capacitance_min_fall

A read-only float attribute that returns the min condition falling edge capacitance modeled on an output port from its set_load -wire_load constraint. Value is returned for the current_corner.

wire_capacitance_min_rise

A read-only float attribute that returns the min condition rising edge capacitance modeled on an output port from its set_load -wire_load constraint. Value is returned for the current_corner.

worst_max_fall_slack

A read-only float attribute that returns the worst slack at a pin for falling maximum path delays. This attribute is valid only after timing has been updated.

worst_max_rise_slack

A read-only float attribute that returns the worst slack at a pin for rising maximum path delays. This attribute is valid only after timing has been updated.

worst_max_slack

A read-only float attribute that returns the worst slack value between the max_rise_slack and max_fall_slack attributes.

worst_min_fall_slack

A read-only float attribute that returns the worst slack at a pin for falling minimum path delays. This attribute is valid only after timing has been updated.

worst_min_rise_slack

A read-only float attribute that returns the worst slack at a pin for rising minimum path delays. This attribute is valid only after timing has been updated.

worst_min_slack

A read-only float attribute that returns the worst slack value between the min_rise_slack and min_fall_slack attributes.

See Also

- [get_attribute](#)
- [list_attributes](#)

port_bus_attributes

Description of the application defined port_bus attributes.

Description

Attributes are properties assigned to objects such as pins, cells, and nets. They can be read-only or settable. A read-only attribute is informational; you cannot set its value. To modify a settable attribute, use the `set_attribute` command.

To determine the value of an attribute, use the `get_attribute` command. To see a list of all port_bus attributes available, use the `list_attributes -application -class port_bus` command.

Attributes for port_bus

This section describes the port_bus attributes.

bit_order

A read-only string attribute (with allowed values: down up) that returns the bit order of a port bus. Possible values are *up* or *down*. It is *down* when *end* is less than *start* else it is *up*.

block

A read-only block collection attribute that is the block of a port bus.

color

A settable string attribute that returns the color of the port bus. This value must be a 24-bit RGB hexadecimal string prefaced by '#', for example: "#72c4f1" or "#ce8322".

end

A read-only int attribute that returns the end index of a port bus.

full_name

A read-only string attribute that returns the name of a port bus.

name

A read-only string attribute that returns the name of a port bus.

object_class

A read-only string attribute that returns the name of this object class. Returns 'port_bus' for port_bus objects.

ports

A read-only port collection attribute that returns the ports of a port bus.

p

start

A read-only int attribute that returns the start index of a port bus.

See Also

- [get_attribute](#)
- [list_attributes](#)

power_domain_attributes

Description of the application defined power_domain attributes.

Description

Attributes are properties assigned to objects such as pins, cells, and nets. They can be read-only or settable. A read-only attribute is informational; you cannot set its value. To modify a settable attribute, use the *set_attribute* command.

To determine the value of an attribute, use the *get_attribute* command. To see a list of all power_domain attributes available, use the *list_attributes -application -class power_domain* command.

Attributes for power_domain

This section describes the power_domain attributes.

associated_by_missing_va

A read-only boolean attribute that is true if the power domain is automatically assigned to voltage area by the tool due to mv.
upf.enable_missing_voltage_area.

available_supply_nets

A read-only collection attribute that returns the available supply net objects for this power domain object.

elements

A read-only collection attribute that returns the element objects for this power domain object.

full_name

A read-only string attribute that returns the full name of a power domain object.

name

A read-only string attribute that returns the name of a power domain object.

object_class

A read-only string attribute that returns the name of this object class. Returns 'power_domain' for power_domain objects.

scope

A read-only collection attribute that returns the scope object for this power domain object.

voltage_areas

A read-only collection attribute that returns the voltage area objects for this power domain object.

See Also

- [get_attribute](#)
- [list_attributes](#)
- [report_attributes](#)

power_strategy_attributes

Description of the application defined power_strategy attributes.

Description

Attributes are properties assigned to objects such as pins, cells, and nets. They can be read-only or settable. A read-only attribute is informational; you cannot set its value. To modify a settable attribute, use the *set_attribute* command.

To determine the value of an attribute, use the *get_attribute* command. To see a list of all power_strategy attributes available, use the *list_attributes -application -class power_strategy* command.

Attributes for power_strategy

This section describes the power_strategy attributes.

full_name

A read-only string attribute that returns the full name of a power strategy object.

name

A read-only string attribute that returns the name of a power strategy object.

p

object_class

A read-only string attribute that returns the name of this object class. Returns 'power_strategy' for power_strategy objects.

power_domain

A read-only collection attribute that returns the power domain object for this power strategy object.

type

A read-only string attribute that specifies the object type of a power strategy object, which can be ISO, LS, RETENTION, and SWITCH.

See Also

- [get_attribute](#)
- [list_attributes](#)
- [report_attributes](#)

pr_rule_attributes

Description of the application defined pr_rule attributes.

Description

Attributes are properties assigned to objects such as pins, cells, and nets. They can be read-only or settable. A read-only attribute is informational; you cannot set its value. To modify a settable attribute, use the *set_attribute* command.

To determine the value of an attribute, use the *get_attribute* command. To see a list of all pr_rule attributes available, use the *list_attributes -application -class pr_rule* command.

Attributes for pr_rule

This section describes the pr_rule attributes.

abut_table_bot_bot

A settable boolean attribute that defines whether or not the two bottom edges can be abutted when using double-back cell rows in your floorplan.

abut_table_top_bot

A settable boolean attribute that defines whether or not the top edge and the bottom edge can be abutted when not using double-back cell rows in your floorplan.

p

abut_table_top_top

A settable boolean attribute that defines whether or not the two top edges can be abutted when using double-back cell rows in your floorplan.

full_name

A read-only string attribute that returns the name of a pr rule.

name

A read-only string attribute that returns the name of a pr rule.

object_class

A read-only string attribute that returns the name of this object class. Returns 'pr_rule' for pr_rule objects.

row_spacing_bot_bot

A settable distance attribute that returns the spacing between the bottom edges when using double-back cell rows in your floorplan.

row_spacing_top_bot

A settable distance attribute that returns the spacing between the top edge and bottom edge when not using double-back cell rows in your floorplan.

row_spacing_top_top

A settable distance attribute that returns the spacing between the top edges when you are using double-back cell rows in your floorplan.

See Also

- [get_attribute](#)
- [list_attributes](#)

process_layer_attributes

Lists the predefined attributes for process layers.

Description

Attributes are properties assigned to objects such as pins, cells and nets. Definitions for process layers attributes are provided in the subsections that follow.

Attributes are read-only unless marked as settable. Read-only attributes are informational, and you cannot set its value. A settable attribute can be modified with the *set_attribute* command.

p

To determine the value of an attribute, use the *get_attribute* command. To see a list of all attributes available for a class of objects use the *list_attributes -application* command.

Process Layer Attributes

bottom_layers

A settable process_layer collection attribute for the process layers which are directly below this process layer.

full_name

A read-only string attribute for the full name of the process layer.

lib_name

A read-only string attribute representing the owner library name.

material

A settable material collection attribute for the material for this process layer.

material_list

A settable string attribute that contains the material names and their ratios for this process layer.

name

A read-only string attribute for the name of the process layer.

object_class

A read-only string attribute with the value of "process_layer".

tech_layer

A settable layer collection attribute for the tech_layer corresponding to this process layer.

thickness

A settable float attribute for the thickness of the process layer (in z-direction).

See Also

- [get_process_layers](#)
- [get_attribute](#)
- [list_attributes](#)
- [set_attribute](#)

pseudo_bump_attributes

Description of the application defined pseudo_bump attributes.

Description

Attributes are properties assigned to objects such as pins, cells, and nets. They can be read-only or settable. A read-only attribute is informational; you cannot set its value. To modify a settable attribute, use the `set_attribute` command.

To determine the value of an attribute, use the `get_attribute` command. To see a list of all pseudo_bump attributes available, use the `list_attributes -application -class pseudo_bump` command.

Attributes for pseudo_bump

This section describes the pseudo_bump attributes.

bbox

A read-only rect attribute that returns the bounding box of the boundary of the pseudo_bump.

boundary

A read-only polygon attribute that returns the boundary of the pseudo_bump.

effective_orientation

A read-only string attribute (with allowed values: R0 R90 R180 R270 MX MXR90 MY MYR90) that returns the effective orientation of the pseudo_bump, which is the pseudo_bump's orientation with respect to the current frame of reference.

This is the pseudo_bump's orientation added to its owning bump_region's orientation, as well as the orientations of its hierarchical owning cells.

full_name

A read-only string attribute that returns the full name of the pseudo_bump.

index

A read-only string attribute that returns the length two, specifying the index of the pseudo_bump, where the first integer is the column index, and the second the row index. The column and row values are 16-bit signed integers.

is_default

A read-only boolean attribute that returns whether all attributes of the pseudo_bump remain at their default values as specified by respective bump_region and bump_region_pattern. This attribute will become "false" if one or more attributes of the pseudo_bump have their values set directly by the user, overriding the default value. If the value is "false", it will become "true" if

p

all non-default valued attributes are reverted to their default values using the *remove_attribute* command.

is_destructible

A settable boolean attribute that indicates whether the *pseudo_bump* is a destructible bump. This attribute can be overridden from its default value using the *set_attribute* command, and can subsequently be reverted to the default value using the *remove_attribute* command.

is_dummy

A settable boolean attribute that indicates whether the *pseudo_bump* is a dummy bump. This attribute can be overridden from its default value using the *set_attribute* command, and can subsequently be reverted to the default value using the *remove_attribute* command.

is_hole

A settable boolean attribute that returns whether the *pseudo_bump* is hole, meaning the absence of a bump. This attribute can be overridden from its default value using the *set_attribute* command, and can subsequently be reverted to the default value using the *remove_attribute* command.

is_probe_pad

A settable boolean attribute that returns whether the *pseudo_bump* is a probe pad bump. This attribute can be overridden from its default value using the *set_attribute* command, and can subsequently be reverted to the default value using the *remove_attribute* command.

location

A read-only coord attribute that returns the location of the *pseudo_bump*. The location of a *pseudo_bump* is the center of its boundary. A *pseudo_bump* is formally a "in" its *bump_region* if its location is contained on or within the *bump_region*'s boundary.

name

A read-only string attribute that returns the name of the *pseudo_bump*. The name is auto-generated by the system, and incorporates its index.

net

A settable net collection attribute that returns the net of the *pseudo_bump*. The default value of this attribute is derived from the *pseudo_bump*'s owning *bump_region*'s *net_pattern* attribute. This attribute is synchronized with the *pseudo_bump*'s *net_name* attribute. This attribute can be overridden from its default value using the *set_attribute* command, and can subsequently be reverted to the default value using the *remove_attribute* command. If overridden,

p

then the `pseudo_bump`'s *port* attribute will be cleared, as a `pseudo_bump` may have only one object override, either net or port. To override the default net with an absence of a net, set the `pseudo_bump`'s *net_name* attribute to the empty string.

net_name

A settable string attribute that returns the name of the net of the `pseudo_bump`. The default value of this attribute is derived from the `pseudo_bump`'s owning `bump_region`'s *net_pattern* attribute. This attribute is synchronized with the `pseudo_bump`'s *net* attribute. This attribute can be overridden from its default value using the *set_attribute* command, and can subsequently be reverted to the default value using the *remove_attribute* command. If overridden, then the `pseudo_bump`'s *port* attribute will be cleared, as a `pseudo_bump` may have only one object override, either net or port.

object_class

A read-only string attribute that returns the name of this object class. Returns 'pseudo_bump' for `pseudo_bump` objects.

offset

A settable coord attribute that returns the offset of the `pseudo_bump` from its pattern-based default location.

orientation

A settable string attribute (with allowed values: R0 R90 R180 R270 MX MXR90 MY MYR90) that returns the orientation of the `pseudo_bump` with respect to its region. By default, this attribute is initialized to R0. This attribute can be overridden from its default value using the *set_attribute* command, and can subsequently be reverted to the default value using the *remove_attribute* command.

parent_block

A read-only block collection attribute that returns the owner block object for this `pseudo_bump`.

port

A settable port collection attribute that returns the port of the `pseudo_bump`. The default value of this attribute is derived from the `pseudo_bump`'s owning `bump_region`'s *port_pattern* attribute. This attribute is synchronized with the `pseudo_bump`'s *port_name* attribute. This attribute can be overridden from its default value using the *set_attribute* command, and can subsequently be reverted to the default value using the *remove_attribute* command. If overridden, then the `pseudo_bump`'s *net* attribute will be set to the port's net, if any, as a `pseudo_bump` may have only one object override, either port or net. To override

p

the default port with an absence of a port, set the `pseudo_bump`'s `port_name` attribute to the empty string.

port_name

A settable string attribute that returns the name of the port of the `pseudo_bump`. The default value of this attribute is derived from the `pseudo_bump`'s owning `bump_region`'s `port_pattern` attribute. This attribute is synchronized with the `pseudo_bump`'s `port` attribute. This attribute can be overridden from its default value using the `set_attribute` command, and can subsequently be reverted to the default value using the `remove_attribute` command. If overridden, then the `pseudo_bump`'s `port` attribute will be set to the port's net, if any, as a `pseudo_bump` may have only one object override, either port or net.

pseudo_tsv

A read-only `pseudo_tsv` collection attribute that returns the `pseudo_tsv` of the `pseudo_bump`. This attribute will be non-empty if and only if the `pseudo_bump`'s `bump_region`'s `has_pseudo_tsvs` attribute is "true".

radius

A settable distance attribute that returns the radius of the `pseudo_bump` shape. This attribute can be overridden from its default value using the `set_attribute` command, and can subsequently be reverted to the default value using the `remove_attribute` command.

region

A read-only `bump_region` collection attribute that returns the `bump_region` of the `pseudo_bump`.

shape

A settable string attribute (with allowed values: circle hexagon octagon pentagon septagon square triangle) that returns the shape of the `pseudo_bump`. This attribute can be overridden from its default value using the `set_attribute` command, and can subsequently be reverted to the default value using the `remove_attribute` command.

tsv_def_name

A settable string attribute that returns the name of the `pseudo_tsv_def` of the `pseudo_bump`'s `pseudo_tsv`. This attribute is synchronized with its `pseudo_tsv`'s `tsv_def_name` attribute.

tsv_offset

A settable coord attribute that returns the offset from the `pseudo_bump`'s location of the `pseudo_tsv` of the `pseudo_bump`. This attribute is synchronized with its `pseudo_tsv`'s `offset` attribute.

See Also

- [get_pseudo_bumps](#)
- [get_attribute](#)
- [list_attributes](#)
- [set_attribute](#)

pseudo_tsv_attributes

Description of the application defined pseudo_tsv attributes.

Description

Attributes are properties assigned to objects such as pins, cells, and nets. They can be read-only or settable. A read-only attribute is informational; you cannot set its value. To modify a settable attribute, use the *set_attribute* command.

To determine the value of an attribute, use the *get_attribute* command. To see a list of all pseudo_tsv attributes available, use the *list_attributes -application -class pseudo_tsv* command.

Attributes for pseudo_tsv

This section describes the pseudo_tsv attributes.

bbox

A read-only rect attribute that returns the bounding box of the pseudo_tsv.

effective_orientation

A read-only string attribute (with allowed values: R0 R90 R180 R270 MX MXR90 MY MYR90) that returns the effective orientation of the pseudo_tsv, which is the pseudo_tsv's orientation with respect to the current frame of reference. This is the pseudo_tsv's orientation added to its owning bump_region's orientation, as well as the orientations of its hierarchical owning cells.

full_name

A read-only string attribute that returns the full name of the pseudo_tsv.

index

A read-only int_pair attribute that returns the index of the pseudo_tsv, where the first integer is the column index, and the second the row index. The column and row values are 16-bit signed integers. The pseudo_tsv will have the same index attribute value as its pseudo_bump.

p

is_connected

A settable boolean attribute that specifies whether this tsv's net is the same as the associated pseudo_bump's net. This attribute can be overridden from its default value using the *set_attribute* command, and can subsequently be reverted to the default value using the *remove_attribute* command.

is_default

A read-only boolean attribute that indicates whether all attributes of the pseudo_tsv remain at their default values as specified by the bump_region's tsv_pattern attribute value. This attribute will become "false" if one or more attributes of the pseudo_tsv have their values set directly by the user, overriding the default value. If the value is "false", it will become "true" if all non-default valued attributes are reverted to their default values using the *remove_attribute* command.

is_hole

A settable boolean attribute that indicates whether the pseudo_tsv is a hole, meaning the absence of a tsv. This attribute can be overridden from its default value using the *set_attribute* command, and can subsequently be reverted to the default value using the *remove_attribute* command.

location

A read-only coord attribute that returns the location of the pseudo_tsv. The location of a pseudo_tsv is the location of its pseudo_bump, shifted by the *offset* attribute value.

name

A read-only string attribute that returns the name of the pseudo_tsv. The name is auto-generated by the system, and incorporates its index.

net

A settable net collection attribute that returns the net of the pseudo_tsv. The default value of this attribute is derived from the pseudo_tsv's owning bump_region's *net_pattern* attribute. If the pseudo_tsv has an associated pseudo_bump, this attribute is synchronized with the pseudo_bump's *net_name* attribute. This attribute can be overridden from its default value using the *set_attribute* command, and can subsequently be reverted to the default value using the *remove_attribute* command. To override the default net with an absence of a net, set the pseudo_tsv's *net_name* attribute to the empty string.

net_name

A settable string attribute that returns the name of the net of the pseudo_tsv. The default value of this attribute is derived from the pseudo_tsv's owning

p

bump_region's *net_pattern* attribute. This attribute can be overridden from its default value using the *set_attribute* command, and can subsequently be reverted to the default value using the *remove_attribute* command.

object_class

A read-only string attribute that returns the name of this object class. Returns 'pseudo_tsv' for pseudo_tsv objects.

offset

A settable coord attribute that returns the offset of the pseudo_tsv from its pseudo_bump. This attribute is synchronized with its pseudo_bump's *tsv_offset* attribute. This attribute can be overridden from its default value using the *set_attribute* command, and can subsequently be reverted to the default value using the *remove_attribute* command.

orientation

A settable string attribute (with allowed values: R0 R90 R180 R270 MX MXR90 MY MYR90) that returns the orientation of the pseudo_tsv with respect to its region. By default, this attribute is initialized to R0.

parent_block

A read-only block collection attribute that returns the owner block object for this pseudo_tsv.

pseudo_bump

A read-only pseudo_bump collection attribute that returns the value of the associated pseudo_bump. When the bump_region's *content_type* attribute is "bump_tsv", this value is never empty. When the bump_region's *content_type* attribute is "tsv", this value is always empty. In either case, the value doesn't change.

region

A read-only bump_region collection attribute that returns the bump_region of the pseudo_tsv.

tsv_def

A settable pseudo_tsv_def collection attribute that returns the pseudo_tsv_def this pseudo_tsv references. This attribute is synchronized with the pseudo_tsv's *tsv_def_name* attribute, as well as its pseudo_bump's *tsv_def_name* attribute. Cannot be empty if the pseudo_tsv is not a hole. This attribute can be overridden from its default value using the *set_attribute* command, and can subsequently be reverted to the default value using the *remove_attribute* command.

r

tsv_def_name

A settable string attribute that returns the name of the `pseudo_tsv_def` this `pseudo_tsv` references. This attribute is synchronized with the `pseudo_tsv`'s *tsv_def* attribute. Cannot be empty if the `pseudo_tsv` is not a hole. This attribute can be overridden from its default value using the *set_attribute* command, and can subsequently be reverted to the default value using the *remove_attribute* command.

See Also

- [get_pseudo_tsvs](#)
- [get_attribute](#)
- [list_attributes](#)
- [set_attribute](#)

r

routing_blockage_attributes

Description of the application defined `routing_blockage` attributes.

Description

Attributes are properties assigned to objects such as pins, cells, and nets. They can be read-only or settable. A read-only attribute is informational; you cannot set its value. To modify a settable attribute, use the *set_attribute* command.

To determine the value of an attribute, use the *get_attribute* command. To see a list of all `routing_blockage` attributes available, use the *list_attributes -application -class routing_blockage* command.

Attributes for routing_blockage

This section describes the `routing_blockage` attributes.

allow_via_ladder

A settable boolean attribute that indicates whether `via_ladders` may overlap the `routing_blockage`.

anchor

A read-only collection attribute that returns the anchor for which this object is the `dependent_object`. When non-empty, this object is the `dependent_object` of the

r

anchor, and its location is therefore constrained by the anchor's anchor_object. See the *create_anchor(2)* command man page for more details about anchors.

area

A read-only area attribute that returns the area of the routing_blockage.

bbox

A read-only rect attribute that returns the bounding-box of the routing_blockage. The format of a rectangle specification is {{llx lly} {urx ury}}, which specifies the lower-left and upper-right corners of the rectangle.

blockage_group_id

A settable int attribute that returns an integer value in the range from 0 to 255 for block boundary blockages. For all other routing blockages, this is an integer attribute in the range from 1 to 8 that represents the blockage group identification number assigned to this routing_blockage created by the create_routing_blockage command. Multiple blockages can be assigned the same blockage group ID number using multiple create_routing_blockage commands. All routing_blockages with the same blockage_group_id attribute belong to the same group. This attribute is 0 by default and doesn't apply for the net_type based association.

blockage_type

A read-only string attribute indicating the type of the routing_blockage. Valid values are boundary_external, boundary_internal, routing, routing_allow_fill_only, routing_for_design_rule, routing_for_top_level, routing_forbid_fill_only, routing_horizontal and routing_vertical.

boundary

A settable coord_list attribute that returns the boundary of the routing_blockage. The format of a coordinate_list specification is {{x1 y1} {x2 y2}...}.

bounding_box

A read-only bounding_box collection attribute that returns the bounding-box of the routing_blockage. The format of the collection specifies lower-left and upper-right corners of the rectangle.

for_floorplan_rules

A settable boolean attribute that returns true if the routing_blockage is floorplan_rule type.

full_name

A read-only string attribute that returns the name of the routing_blockage.

r

groups

A read-only group collection attribute that returns a collection of groups associated with a routing_blockage.

icc_route_guide

A read-only boolean attribute that returns true if the routing_blockage is converted from ICC.

icc_system_layer_blockage

A read-only boolean attribute that returns true if the routing_blockage is converted from ICC system layer blockage.

in_edit_group

A read-only boolean attribute that indicates whether the routing_blockage belongs to an edit_group.

is_allow_metal_fill_only

A settable boolean attribute that returns whether the routing_blockage is an 'allow metal fill only' blockage. Setting this attribute will reset is_zero_spacing to 'false'.

is_boundary_blockage

A read-only boolean attribute that indicates whether the routing_blockage is a block boundary blockage.

is_derived

A settable boolean attribute that returns whether the routing_blockage was automatically generated by the application. For more details, see *derive_placement_blockages* and *plan.place.auto_generate_blockages*.

is_design_rule_blockage

A settable boolean attribute that returns whether the routing_blockage is created for the purpose of design rule checking. This attribute is settable if the layer type blocked by this routing_blockage is VIA_CUT or LOCAL_VIA_CUT.

is_dummy_metal

A settable boolean attribute that returns whether a routing blockage is dummy metal or not.

is_external_boundary_blockage

A read-only boolean attribute that indicates whether the routing_blockage is an external block boundary blockage.

r

is_forbid_metal_fill_only

A read-only boolean attribute that indicates whether the routing_blockage is a 'forbid metal fill only' blockage.

is_internal_boundary_blockage

A read-only boolean attribute that indicates whether the routing_blockage is an internal block boundary blockage.

is_maskable

A read-only boolean attribute that indicates whether the routing_blockage will be streamed out.

is_shadow

A read-only boolean attribute that indicates whether the routing_blockage is pushed down from above.

is_zero_spacing

A settable boolean attribute that returns true if the routing blockage has zero min spacing.

layer

A settable layer collection attribute that specifies the layer which is blocked by this routing_blockage. If the layer type changes from VIA_CUT or LOCAL_VIA_CUT to other types as result of setting this attribute, the is_design_rule_blockage attribute will be re-set to false.

layer_name

A read-only string attribute that returns the name of the layer which is blocked by this routing_blockage.

mask_constraint

A settable string attribute (with allowed values: no_mask mask_one mask_two mask_three mask_four same_mask) that returns the multi-patterning mask constraint of this routing_blockage.

name

A settable string attribute that returns the name of the routing_blockage.

net_types

A settable string attribute that returns the net types blocked by the routing_blockage. If the string is empty, then the routing_blockage blocks all net types.

r

nets

A read-only net collection attribute that returns the specific nets associated with the `blockage_group_id` of this `routing_blockage`. Doesn't apply for the `net_type` based association.

number_of_nets

A read-only int attribute that returns the number of specific nets associated with the `blockage_group_id` of this `routing_blockage`. Doesn't apply for the `net_type` based association.

object_class

A read-only string attribute that returns the name of this object class. Returns 'routing_blockage' for `routing_blockage` objects.

parent_block

A read-only block collection attribute that returns the owner block object for this `routing_blockage`.

parent_block_cell

A read-only cell collection attribute that returns the parent block cell object for this `routing_blockage`.

parent_cell

A read-only cell collection attribute that returns the owner cell object for this `routing_blockage`.

physical_status

A settable string attribute (with allowed values: locked unrestricted) that returns the physical modification status of the `routing_blockage`. If status is unrestricted, the `routing_blockage` may be moved or rotated. If the status is locked, the `routing_blockage` may not be moved or rotated.

reserve_for_top_level_routing

A settable boolean attribute that indicates whether the `routing_blockage` reserves the area for top level routing. Routes in the top block may route through this `routing_blockage`, but routes in this block are blocked by this `routing_blockage`. Setting this attribute will reset `is_zero_spacing` to 'false'.

shadow_status

A settable string attribute (with allowed values: copied_down copied_up normal pulled_up pushed_down virtual_flat) that returns the shadow status of the `routing_blockage`.

r

top_block

A read-only block collection attribute that returns the top block object for this routing_blockage.

See Also

- [get_attribute](#)
- [list_attributes](#)

routing_corridor_attributes

Description of the application defined routing_corridor attributes.

Description

Attributes are properties assigned to objects such as pins, cells, and nets. They can be read-only or settable. A read-only attribute is informational; you cannot set its value. To modify a settable attribute, use the *set_attribute* command.

To determine the value of an attribute, use the *get_attribute* command. To see a list of all routing_corridor attributes available, use the *list_attributes -application -class routing_corridor* command.

Attributes for routing_corridor

This section describes the routing_corridor attributes.

bbox

A read-only rect attribute that returns the bbox of a routing corridor.

bounding_box

A read-only bounding_box collection attribute that returns the bounding-box of a routing_corridor. The format of the collection specifies lower-left and upper- right corners of the rectangle.

full_name

A read-only string attribute that returns the name of a routing_corridor.

groups

A read-only group collection attribute that returns the collection of groups associated with a routing corridor.

has_path

A read-only boolean attribute that indicates whether corridor has path shape.

r

has_tile

A read-only boolean attribute that indicates whether corridor has rectangle or rectilinear shape.

is_connected

A read-only boolean attribute that indicates whether all shapes of the routing_corridor are connected.

is_manhattan

A read-only boolean attribute that indicates whether all shapes of the routing_corridor are manhattan.

name

A settable string attribute that returns the name of a routing_corridor.

number_of_objects

A read-only int attribute that returns the number of objects associated with the routing_corridor.

object_class

A read-only string attribute that returns the name of this object class. Returns 'routing_corridor' for routing_corridor objects.

objects

A read-only net collection attribute that specifies the nets associated with the routing corridor. The data type of *objects* is collection.

parent_block

A read-only block collection attribute that is the owner block object for this routing_corridor.

parent_block_cell

A read-only cell collection attribute that returns the parent block cell object for this routing_corridor.

parent_cell

A read-only cell collection attribute that returns the owner cell object for this routing_corridor.

physical_status

A settable string attribute (with allowed values: locked unrestricted) that returns the physical modification status of the routing_corridor. If status is unrestricted,

r

the `routing_corridor` may be moved or rotated. If the status is locked, the `routing_corridor` may not be moved or rotated.

shapes

A read-only `routing_corridor_shape` collection attribute that returns the collection of shapes associated with the routing corridor.

supernets

A read-only supernet collection attribute that returns all associated supernets of this routing corridor.

top_block

A read-only block collection attribute that returns the top block object for this `routing_corridor`.

See Also

- [create_routing_corridor](#)
- [get_attribute](#)
- [list_attributes](#)
- [set_attribute](#)

routing_corridor_shape_attributes

Description of the application defined `routing_corridor_shape` attributes.

Description

Attributes are properties assigned to objects such as pins, cells, and nets. They can be read-only or settable. A read-only attribute is informational; you cannot set its value. To modify a settable attribute, use the `set_attribute` command.

To determine the value of an attribute, use the `get_attribute` command. To see a list of all `routing_corridor_shape` attributes available, use the `list_attributes -application -class routing_corridor_shape` command.

Attributes for routing_corridor_shape

This section describes the `routing_corridor_shape` attributes.

bbox

A read-only rect attribute that returns the bounding-box of a routing corridor shape. The format of a rectangle specification is `{llx lly} {urx ury}`, which

r

specifies the lower-left and upper-right corners of the rectangle. This attribute is settable only for rectangle routing_corridor_shape.

bounding_box

A read-only bounding_box collection attribute that returns the bounding-box of a routing_corridor_shape. The format of the collection specifies lower-left and upper-right corners of the rectangle.

end_endcap

A settable string attribute (with allowed values: flush full_width half_width) that returns the ending end alignment type of a path routing_corridor_shape.

full_name

A read-only string attribute that returns the name of a routing_corridor_shape object.

groups

A read-only group collection attribute that returns the groups of which this routing_corridor_shape is a member.

is_manhattan

A read-only boolean attribute that indicates whether the routing_corridor_shape is Manhattan.

max_routing_layer

A settable string attribute that returns the layer name of the max layer constraint for this routing_corridor_shape. The format of a coordinate specification is float number.

min_routing_layer

A settable string attribute that returns the layer name of the min layer constraint for this routing_corridor_shape.

name

A read-only string attribute that returns the name of a routing_corridor_shape object.

number_of_points

A read-only int attribute that returns the number of points of the routing_corridor_shape.

object_class

A read-only string attribute that returns the name of this object class. Returns 'routing_corridor_shape' for routing_corridor_shape objects.

r

parent_block

A read-only block collection attribute that returns the owner block object for this routing_corridor_shape.

parent_block_cell

A read-only cell collection attribute that returns the parent block cell object for this routing_corridor_shape.

parent_cell

A read-only cell collection attribute that returns the owner cell object for this routing_corridor_shape.

physical_status

A settable string attribute (with allowed values: locked unrestricted) that returns the physical modification status of the routing_corridor_shape. Valid values are unrestricted and locked. If status is unrestricted, the routing_corridor_shape may be freely modified. If status is locked, the routing_corridor_shape is fixed for automatic and manual changes.

points

A settable coord_list attribute that returns the points of the routing_corridor_shape. The format of coordinate list specification is {{x1 y1} {x2 y2} ...}, which specifies a list of points defining the boundary for region, or centerline for path.

routing_corridor

A read-only routing_corridor collection attribute that returns the routing corridor owning this routing_corridor_shape.

shape_type

A read-only string attribute (with allowed values: path polygon rectangle) that returns the routing_corridor_shape type.

start_endcap

A settable string attribute (with allowed values: flush full_width half_width) that returns the starting end alignment type of a path routing_corridor_shape.

top_block

A read-only block collection attribute that returns the top block object for this routing_corridor_shape.

r

width

A settable distance attribute that returns the width of a path
routing_corridor_shape.

See Also

- [create_routing_corridor_shape](#)
- [create_routing_corridor](#)
- [get_attribute](#)
- [report_attributes](#)
- [set_attribute](#)

routing_guide_attributes

Description of the application defined routing_guide attributes.

Description

Attributes are properties assigned to objects such as pins, cells, and nets. They can be read-only or settable. A read-only attribute is informational; you cannot set its value. To modify a settable attribute, use the *set_attribute* command.

To determine the value of an attribute, use the *get_attribute* command. To see a list of all routing_guide attributes available, use the *list_attributes -application -class routing_guide* command.

Attributes for routing_guide

This section describes the routing_guide attributes.

access_preference

A read-only string attribute that returns Access_preference data. Output is in format {"layer name" {wire/via_access_preference {rect} strength} ..}. Example {M1 {wire_access_preference {{20 20} {55 55}} .5} ...}

area

A read-only area attribute that returns the area of routing_guide.

bbox

A read-only rect attribute that returns the bounding box of routing_guide.

boundary

A settable coord_list attribute that returns the bounding box of routing_guide.

r

boundary_bounding_box

A settable bounding_box collection attribute that returns the bounding-box of the routing_guide. The format of the collection specifies lower-left and upper-right corners of the rectangle.

bounding_box

A read-only bounding_box collection attribute that returns the bounding-box of the routing_guide. The format of the collection specifies lower-left and upper-right corners of the rectangle.

design

A read-only design collection attribute that returns the design object for this routing_guide.

full_name

A read-only string attribute that returns the name.

groups

A read-only group collection attribute that returns the groups the routing_guide belongs to.

horizontal_track_utilization

A settable int attribute that returns the horizontal track utilization of routing_guide.

is_shadow

A read-only boolean attribute that returns true if shadow status is normal for routing_guide, false if not.

layer_names

A read-only string attribute that returns all the layer names of the routing_guide.

layers

A settable layer collection attribute that returns a collection of layers. The value is collection of layers of the routing_guide.

max_patterns

A read-only string attribute that returns Max_patterns data. Output is tcl list of sets. Each set consist of "layer name", "pattern type" and "constraint". Example { {M2 non_pref_dir_edge 3} {M4 non_pref_dir_edge 1} }

name

A settable string attribute that returns the name.

r

object_class

A read-only string attribute that returns the name of this object class. Returns 'routing_guide' for routing_guide objects.

parent_block

A read-only block collection attribute that returns the owner block object for this routing_guide.

parent_block_cell

A read-only block collection attribute that returns the parent block cell object for this routing_guide.

parent_cell

A read-only layer collection attribute that returns the owner cell object for this routing_guide.

physical_status

A settable string attribute (with allowed values: locked unrestricted) that returns the physical modification status of the site row. If status is unrestricted, the site_row may be moved or rotated. If the status is locked, the site_row may not be moved or rotated.

preferred_direction_only

A settable boolean attribute that returns true if preferred direction only is set on the routing_guide.

routing_guide_type

A read-only string attribute (with allowed values: access_preference area_double_via area_rule_region constrained_stack_via_region default design_boundary_blockage extra_detour_region forbid_obs_patch forbidden_marker forbidden_preferred_grid_extension max_patterns metal_cut_allowed min_area mixed_row_boundary_region over_icovl_cell_layers pin_access rdl_routing rectangle_only river_routing rule_group package_shape single_row_via_ladder_pattern_must_join_allowed standard_cell_region switched_direction_only constrained_region) that returns the type of routing guide.

rule_group

A settable string attribute that returns the name of the rule group if the routing guide corresponds to one. Otherwise, it is an empty string.

r

shadow_status

A settable string attribute (with allowed values: copied_down copied_up normal pulled_up pushed_down virtual_flat) that returns the shadow status of the routing_guide.

switch_preferred_direction

A settable boolean attribute that returns true if switch preferred direction is set on the routing_guide.

top_block

A read-only block collection attribute that returns the top block object for this routing_guide.

vertical_track_utilization

A settable int attribute that returns the vertical track utilization of routing_guide.

See Also

- [create_routing_guide](#)
- [get_attribute](#)
- [list_attributes](#)
- [set_attribute](#)

routing_rule_attributes

Description of the application defined routing_rule attributes.

Description

Attributes are properties assigned to objects such as pins, cells, and nets. They can be read-only or settable. A read-only attribute is informational; you cannot set its value. To modify a settable attribute, use the *set_attribute* command.

To determine the value of an attribute, use the *get_attribute* command. To see a list of all routing_rule attributes available, use the *list_attributes -application -class routing_rule* command.

Attributes for routing_rule

This section describes the routing_rule attributes.

r

cuts

A read-only string attribute that returns the cuts specification from which the routing_rule vias were derived. The string format is "{cut_layer_name {cut_name num_of_cuts} ...} ...". The attribute value exists if and only if the cuts specification was specified through the *create_routing_rule* -cuts option and the via list was not subsequently modified.

driver_taper_distance

A read-only distance attribute that returns the driver taper distance of the routing_rule.

driver_taper_over_pin_layers

A settable int attribute that returns the number of routing layers over the driver pin layer that the router may use for tapering.

driver_taper_under_pin_layers

A settable int attribute that returns the number of routing layers under the driver pin layer that the router may use for tapering.

full_name

A read-only string attribute that returns the name.

groups

A read-only group collection attribute that returns the groups in which this routing_rule is a member.

has_shield

A read-only boolean attribute that returns true if any shielding constraints exists, false otherwise.

ignore_spacing_to_blockage

A read-only boolean attribute that indicates whether or not the router should ignore this routing_rule's spacing constraints when considering the distance to blockages. If this value does not exist, the router uses the setting specified by the application option, route.common.ignore_var_spacing_to_blockage.

ignore_spacing_to_pg

A read-only boolean attribute that indicates whether or not the router should ignore this routing_rule's spacing constraints when considering the distance to power and ground nets. If this value does not exist, the router uses the setting specified by the application option, route.common.ignore_var_spacing_to_pg.

r

ignore_spacing_to_shield

A read-only boolean attribute that indicates whether or not the router should ignore this routing_rule's spacing constraints when considering the distance to shield nets. If this value does not exist, the router uses the setting specified by the application option, route.common.ignore_var_spacing_to_shield.

is_snap_to_track

A read-only boolean attribute that returns true if snap to track flag is set on layer, false otherwise.

mask_constraints

A settable string attribute that returns the mask constraint for each routing layer. The string format is "{layer_name mask_name [fixed]}...". The keyword, fixed, appears next to the mask_name if the mask is fixed on the layer. If does not appear if the mask is not fixed on the layer. A layer is listed if and only if it has a mask constraint.

name

A read-only string attribute that returns the name.

object_class

A read-only string attribute that returns the name of this object class. Returns 'routing_rule' for routing_rule objects.

parallel_wire

A read-only layer collection attribute that returns the layers on which to create fat wires for parallel wire purpose.

rdl_taper_distances

A read-only string attribute that returns the maximum length for tapering wires on each redistribution layer. This is supported only by the flip-chip router. The string format is "layer_name distance ...".

rdl_taper_widths

A read-only string attribute that returns the tapering width for each redistribution layer. This is supported only by the flip-chip router. The string format is "layer_name width ...".

shield_spacings

A read-only string attribute that returns the minimum shield spacing allowed between wires on each routing layer. The string format is "layer_name shield_spacing ...".

r

shield_widths

A read-only string attribute that returns the shield widths on each routing layer. The string format is "layer_name shield_width ...".

single_connection_to_pin

A read-only boolean attribute that returns whether or not the router must connect to a pin only once. If true, the router must connect to a pin only once. This applies to all pins connected to nets with this routing rule. If false, the router is allowed to connect to a pin any number of times. If this attribute is undefined, the default is false except for nets using a nondefault width routing rule. For those nets, the value is always true. This attribute takes precedence over the global app option "route.common.single_connection_to_pins".

spacing_length_thresholds

A read-only string attribute that returns the length thresholds for each spacing in this routing_rule's spacings attribute. The string format is "layer_name {spacing_length_threshold ...} ...".

spacing_multiplier

A read-only float attribute that returns the multiplier applied to the layer spacings in this routing_rule.

spacing_weight_levels

A read-only string attribute that returns the weight levels for each spacing in this routing_rule's spacings attribute. The string format is "layer_name {spacing_weight_level ...} ...".

spacings

A read-only string attribute that returns the minimum different-net spacing allowed between wires on each routing layer. The string format is "layer_name {spacing ...} ...".

taper_distance

A read-only distance attribute that returns the taper distance of the routing_rule.

taper_over_pin_layers

A read-only int attribute that returns the number of routing layers over the pin layer that the router may use for tapering.

taper_under_pin_layers

A read-only int attribute that returns the number of routing layers under the pin layer that the router may use for tapering.

r

via_spacings

A read-only string attribute that returns the via spacings between via layers. The string format is "{layer_name layer_name spacing} ...".

vias

A read-only string attribute that returns the vias which can be created during routing. The string format is "{via_definition_name site_spec rotation_spec} ...". The site_spec is formatted MxN in which M is the horizontal site count (i.e., the number of columns in the via) and N is the vertical site count (i.e., the number of rows in the via), separated by the x character. The rotation_spec is specified by r or nr, where r indicates a rotated via and nr indicates an unrotated via.

width_multiplier

A read-only float attribute that returns the multiplier applied to the layer widths in this routing_rule.

widths

A read-only string attribute that returns the wire widths on each routing layer. The string format is "layer_name width ...".

See Also

- [create_routing_rule](#)
- [get_attribute](#)
- [list_attributes](#)
- [set_attribute](#)

rp_blockage_attributes

Description of the application defined rp_blockage attributes.

Description

Attributes are properties assigned to objects such as pins, cells, and nets. They can be read-only or settable. A read-only attribute is informational; you cannot set its value. To modify a settable attribute, use the *set_attribute* command.

To determine the value of an attribute, use the *get_attribute* command. To see a list of all rp_blockage attributes available, use the *list_attributes -application -class rp_blockage* command.

Attributes for rp_blockage

This section describes the rp_blockage attributes.

r

allow_overlap

A settable boolean attribute that indicates whether the relative placement blockage can be overlapped with fixed objects.

bbox

A read-only rect attribute that returns the bounding-box of a `rp_blockage`. The format of a rectangle specification is `{{llx lly} {urx ury}}`, which specifies the lower-left and upper-right corners of the rectangle.

bounding_box

A read-only `bounding_box` collection attribute that returns the bounding-box of a `rp_blockage`. The format of the collection specifies lower-left and upper-right corners of the rectangle.

column

A read-only int attribute that returns the column position of `rp_blockage`.

full_name

A read-only string attribute that returns the name of `rp_blockage`.

height

A read-only int attribute that returns the height of `rp_blockage`.

name

A read-only string attribute that returns the name of `rp_blockage`.

object_class

A read-only string attribute that returns the name of this object class. Returns '`rp_blockage`' for `rp_blockage` objects.

override_alignment

A settable string attribute (with allowed values: `left right`) that returns the alignment to be used to override the alignment of the relative placement blockage in the column of its parent `rp_group`.

parent_block

A read-only block collection attribute that returns the owner block object for `rp_blockage`.

parent_block_cell

A read-only cell collection attribute that returns the parent block cell object for `rp_blockage`.

r

parent_cell

A read-only cell collection attribute that returns the owner cell object for `rp_blockage`.

parent_rp_group

A read-only string attribute that returns the parent `rp_group` name of `rp_blockage`.

physical_status

A read-only string attribute (with allowed values: `placed` `unplaced`) that returns the physical placement status of the relative placement blockage. The valid values are `unplaced` and `placed`. If status is `unplaced`, the relative placement blockage has not been placed. If status is `placed`, the relative placement blockage has been placed.

row

A read-only int attribute that returns the row position of `rp_blockage`.

top_block

A read-only block collection attribute that returns the top block object for `rp_blockage`.

top_rp_group

A read-only string attribute that returns the top `rp_group` name of `rp_blockage`.

width

A read-only int attribute that returns the column position of `rp_blockage`.

See Also

- [add_to_rp_group](#)
- [get_attribute](#)
- [list_attributes](#)
- [set_attribute](#)

rp_group_attributes

Description of the application defined `rp_group` attributes.

r

Description

Attributes are properties assigned to objects such as pins, cells, and nets. They can be read-only or settable. A read-only attribute is informational; you cannot set its value. To modify a settable attribute, use the *set_attribute* command.

To determine the value of an attribute, use the *get_attribute* command. To see a list of all *rp_group* attributes available, use the *list_attributes -application -class rp_group* command.

Attributes for *rp_group*

This section describes the *rp_group* attributes.

alignment

A read-only string attribute (with allowed values: left pin right) that returns the alignment to be used to align objects in a column of *rp_group*.

allow_non_rp_cells

A read-only boolean attribute that returns whether to allow the non relative placement cells in the unused areas of *rp_group*.

allow_non_rp_cells_on_blockages

A read-only boolean attribute that returns whether to allow the overlap of non relative placement cells with *rp_group* blockages and unused areas of *rp_group*.

anchor_column

A read-only int attribute that returns the column of relative placement group for *rp_location* anchor corner.

anchor_corner

A read-only string attribute (with allowed values: bottom_left bottom_right *rp_location* top_left top_right) that returns the corner for anchor point specified using attributes *x_offset* and *y_offset*. The possible values are bottom_left, bottom_right, top_left, top_right and *rp_location*.

anchor_row

A read-only int attribute that returns the row of relative placement group for *rp_location* anchor corner.

bbox

A read-only rect attribute that returns the bounding-box of a *rp_group*. The format of a rectangle specification is *{{llx lly} {urx ury}}*, which specifies the lower-left and upper-right corners of the rectangle.

r

bounding_box

A read-only `bounding_box` collection attribute that returns the bounding-box of a `rp_group`. The format of the collection specifies lower-left and upper-right corners of the rectangle.

column

A read-only int attribute that returns the column position of `rp_group`.

columns

A read-only int attribute that returns the number of columns in `rp_group`.

full_name

A read-only string attribute that returns the name of `rp_group`.

group_orient

A read-only string attribute (with allowed values: R0 R180 MX MY) that returns the orientation of a `rp_group`.

move_effort

A read-only string attribute (with allowed values: high low medium) that controls the movement of `rp_group`.

name

A read-only string attribute that returns the name of `rp_group`.

object_class

A read-only string attribute that returns the name of this object class. Returns 'rp_group' for `rp_group` objects.

optimization_restriction

A read-only string attribute (with allowed values: all_opt footprint_only no_opt size_in_place size_only) that controls optimization of relative placement cells.

override_alignment

A settable string attribute (with allowed values: left right) that sets the alignment to be used to override the alignment of the relative placement group in the column of its parent `rp_group`.

parent_block

A read-only block collection attribute that returns the owner block object for `rp_group`.

r

parent_block_cell

A read-only cell collection attribute that returns the parent block cell object for `rp_group`.

parent_cell

A read-only cell collection attribute that returns the owner cell object for `rp_group`.

parent_rp_group

A read-only string attribute that returns the parent `rp_group` name of `rp_group`.

physical_status

A read-only string attribute (with allowed values: fixed locked placed unplaced) that returns the physical placement status of the relative placement group. The valid values are unplaced and placed. If status is unplaced, the relative placement group has not been placed. If relative placement group is failed then also status will be unplaced. If status is placed, the relative placement group has been placed.

pin_name

A read-only string attribute that returns the name of pin on which the cells of a column of `rp_group` will be aligned.

row

A read-only int attribute that returns the row position of `rp_group`.

rows

A read-only int attribute that returns the number of rows in `rp_group`.

rp_only_keepout_margin

A read-only `margin_list` attribute that returns the relative placement only keepout margin values in the following order: left, bottom, right, and top. The format of a `margin_list` specification is `{l b r t}`.

site_row

A read-only string attribute (with allowed values: any even odd) that returns the type of site row for the starting location of the `rp_group`.

tiling_type

A read-only string attribute (with allowed values: `bit_slice` `horizontal_compression` `vertical_compression`) that returns the tiling pattern of cells of `rp_group`.

r

top_block

A read-only block collection attribute that is the top block object for `rp_group`.

top_rp_group

A read-only string attribute that returns the top `rp_group` name of `rp_group`.

utilization

A read-only double attribute that returns the utilization percentage for placement of relative placement group.

x_offset

A read-only distance attribute that returns the X-coordinate for `anchor_corner` of `rp_group`.

y_offset

A read-only distance attribute that returns the Y-coordinate for `anchor_corner` of `rp_group`.

See Also

- [create_rp_group](#)
- [get_attribute](#)
- [list_attributes](#)
- [set_attribute](#)

rtl_cell_attributes

Description of the application defined `rtl_cell` attributes.

Description

Attributes are properties assigned to objects such as pins, cells, and nets. They can be read-only or settable. A read-only attribute is informational; you cannot set its value. To modify a settable attribute, use the *set_attribute* command.

To determine the value of an attribute, use the *get_attribute* command. To see a list of all `rtl_cell` attributes available, use the *list_attributes -application -class rtl_cell* command.

Attributes for rtl_cell

This section describes the `rtl_cell` attributes.

r

full_name

A read-only string attribute that returns the full name of this object.

glitch_power

A read-only double attribute that returns the glitch power.

glitch_power_local

A read-only double attribute that returns the glitch power.

hdl_file

A read-only string attribute that returns the hdl file.

hdl_line

A read-only int attribute that returns the hdl line.

internal_power

A read-only double attribute that returns the internal power.

internal_power_local

A read-only double attribute that returns the internal power.

leakage_power

A read-only double attribute that returns the leakage power.

leakage_power_local

A read-only double attribute that returns the leakage power.

metrics_cong_cell_padding

A read-only double attribute that returns the cell padding.

metrics_cong_cell_padding_local

A read-only double attribute that returns the cell padding.

metrics_cong_logic_cong

A read-only int attribute that returns the number of logic congestion cells.

metrics_cong_logic_cong_local

A read-only int attribute that returns the number of logic congestion cells.

metrics_cong_number_cells

A read-only int attribute that returns the number of cells.

r

metrics_cong_number_cells_in_cong_area

A read-only int attribute that returns the number of congested cells.

metrics_cong_number_cells_in_cong_area_local

A read-only int attribute that returns the number of congested cells.

metrics_cong_number_cells_in_cong_channel

A read-only int attribute that returns the number of congested cells in channel.

metrics_cong_number_cells_in_cong_channel_local

A read-only int attribute that returns the number of congested cells in channel.

metrics_cong_number_cells_local

A read-only int attribute that returns the number of cells.

metrics_cong_overflow

A read-only double attribute that returns the congestion overflow.

metrics_cong_overflow_local

A read-only double attribute that returns the congestion overflow.

metrics_cong_percent_cells_in_cong_area

A read-only double attribute that returns the percent of cells in congestion area.

metrics_cong_percent_cells_in_cong_area_local

A read-only double attribute that returns the percent of cells in congestion area.

metrics_cong_percent_of_cong_area

A read-only double attribute that returns the congested cells percentage vs. total congested cells

metrics_cong_percent_of_cong_area_local

A read-only double attribute that returns the congested cells percentage vs. total congested cells

metrics_drc_estimate

A read-only double attribute that returns the DRC estimate.

metrics_drc_estimate_local

A read-only double attribute that returns the DRC estimate.

metrics_tim_bottleneck_count

A read-only int attribute that returns the bottleneck cells.

r

metrics_tim_bottleneck_count_local

A read-only int attribute that returns the bottleneck cells.

metrics_tim_logic_levels_viol

A read-only int attribute that returns the logic level.

metrics_tim_logic_levels_viol_local

A read-only int attribute that returns the logic level.

metrics_tim_max_logic_levels_viol

A read-only int attribute that returns the max logic level.

metrics_tim_max_logic_levels_viol_local

A read-only int attribute that returns the max logic level.

metrics_tim_net_length_viol

A read-only int attribute that returns the net length violation.

metrics_tim_net_length_viol_local

A read-only int attribute that returns the net length violation.

metrics_tim_nvp_io

A read-only int attribute that returns the NVP IO.

metrics_tim_nvp_io_local

A read-only int attribute that returns the NVP IO.

metrics_tim_nvp_r2r

A read-only int attribute that returns the NVP R2R.

metrics_tim_nvp_r2r_local

A read-only int attribute that returns the NVP R2R.

metrics_tim_nvp_total

A read-only int attribute that returns the NVP.

metrics_tim_nvp_total_local

A read-only int attribute that returns the NVP.

metrics_tim_optimization_effort

A read-only float attribute that returns the optimization effort.

r

metrics_tim_optimization_effort_local

A read-only float attribute that returns the optimization effort.

metrics_tim_path_length_viol

A read-only int attribute that returns the path length violation.

metrics_tim_path_length_viol_local

A read-only int attribute that returns the path length violation.

metrics_tim_repeater_levels_viol

A read-only int attribute that returns the repeater level violation.

metrics_tim_repeater_levels_viol_local

A read-only int attribute that returns the repeater level violation.

metrics_tim_tns_io

A read-only double attribute that returns the TNS IO.

metrics_tim_tns_io_local

A read-only double attribute that returns the TNS IO.

metrics_tim_tns_r2r

A read-only double attribute that returns the TNS R2R.

metrics_tim_tns_r2r_local

A read-only double attribute that returns the TNS R2R.

metrics_tim_tns_total

A read-only double attribute that returns the TNS.

metrics_tim_tns_total_local

A read-only double attribute that returns the TNS.

metrics_tim_wns_io

A read-only double attribute that returns the WNS IO.

metrics_tim_wns_io_local

A read-only double attribute that returns WNS IO.

metrics_tim_wns_r2r

A read-only double attribute that returns WNS R2R.

r

metrics_tim_wns_r2r_local

A read-only double attribute that returns WNS R2R.

metrics_tim_wns_total

A read-only double attribute that returns WNS.

metrics_tim_wns_total_local

A read-only double attribute that returns WNS.

multi_hier_cell

A read-only int attribute that returns the number of leaf cells belonging to multiple rtl_cell.

name

A read-only string attribute that returns the name of this object.

ndm_cell

A read-only collection attribute that returns the corresponding NDM cell (if exists).

object_class

A read-only string attribute that returns the name of this object class. Returns 'rtl_cell' for rtl_cell objects.

parent_cell

A read-only collection attribute that returns the parent cell.

rtl_module

A read-only collection attribute that returns the rtl module of the instance.

rtl_original_module

A read-only collection attribute that returns the original rtl module of the instance.

shape

A settable collection attribute that returns the corresponding shape (if exists).

short_name

A settable string attribute that returns the short name (if exists).

sim_scope_name

A read-only string attribute that returns the full path name used by simulation/verification.

r

switching_power

A read-only double attribute that returns the switching power.

switching_power_local

A read-only double attribute that returns the switching power.

total_power

A read-only double attribute that returns the total power.

total_power_local

A read-only double attribute that returns the total power.

See Also

- [get_attribute](#)
- [list_attributes](#)
- [set_attribute](#)

rtl_cell_construct_attributes

Description of the application defined rtl_cell_construct attributes.

Description

Attributes are properties assigned to objects such as pins, cells, and nets. They can be read-only or settable. A read-only attribute is informational; you cannot set its value. To modify a settable attribute, use the *set_attribute* command.

To determine the value of an attribute, use the *get_attribute* command. To see a list of all rtl_cell_construct attributes available, use the *list_attributes -application -class rtl_cell_construct* command.

Attributes for rtl_cell_construct

This section describes the rtl_cell_construct attributes.

end_line

A read-only int attribute that returns the end line of the construct.

file_name

A read-only string attribute that returns the RTL file.

full_name

A read-only string attribute that returns the full name of this object.

r

glitch_power

A read-only double attribute that returns the glitch power.

glitch_power_local

A read-only double attribute that returns the glitch power.

hdl_file

A read-only string attribute that returns the hdl file.

hdl_line

A read-only int attribute that returns the hdl line of the construct.

internal_power

A read-only double attribute that returns the internal power.

internal_power_local

A read-only double attribute that returns the internal power.

leakage_power

A read-only double attribute that returns the leakage power.

leakage_power_local

A read-only double attribute that returns the leakage power.

metrics_cong_cell_padding

A read-only double attribute that returns cell padding.

metrics_cong_cell_padding_local

A read-only double attribute that returns cell padding.

metrics_cong_logic_cong

A read-only int attribute that returns the number of logic congestion cells.

metrics_cong_logic_cong_local

A read-only int attribute that returns the number of logic congestion cells.

metrics_cong_number_cells

A read-only int attribute that returns the number of cells.

metrics_cong_number_cells_in_cong_area

A read-only int attribute that returns the number of congested cells.

r

metrics_cong_number_cells_in_cong_area_local

A read-only int attribute that returns the number of congested cells.

metrics_cong_number_cells_in_cong_channel

A read-only int attribute that returns the number of congested cells in a channel.

metrics_cong_number_cells_in_cong_channel_local

A read-only int attribute that returns the number of congested cells in a channel.

metrics_cong_number_cells_local

A read-only int attribute that returns the number of cells.

metrics_cong_overflow

A read-only double attribute that returns congestion overflow.

metrics_cong_overflow_local

A read-only double attribute that returns congestion overflow.

metrics_cong_percent_cells_in_cong_area

A read-only double attribute that returns the percent of cells in congestion area.

metrics_cong_percent_cells_in_cong_area_local

A read-only double attribute that returns the percent of cells in congestion area.

metrics_cong_percent_of_cong_area

A read-only double attribute that returns the congested cells percentage vs. total congested cells

metrics_cong_percent_of_cong_area_local

A read-only double attribute that returns the congested cells percentage vs. total congested cells

metrics_drc_estimate

A read-only double attribute that returns the DRC estimate.

metrics_drc_estimate_local

A read-only double attribute that returns the DRC estimate.

metrics_tim_bottleneck_count

A read-only int attribute that returns the bottleneck cells.

metrics_tim_bottleneck_count_local

A read-only int attribute that returns the bottleneck cells.

r

metrics_tim_logic_levels_viol

A read-only int attribute that returns the logic level.

metrics_tim_logic_levels_viol_local

A read-only int attribute that returns the logic level.

metrics_tim_max_logic_levels_viol

A read-only int attribute that returns the max logic level.

metrics_tim_max_logic_levels_viol_local

A read-only int attribute that returns the max logic level.

metrics_tim_net_length_viol

A read-only int attribute that returns the net length violation.

metrics_tim_net_length_viol_local

A read-only int attribute that returns the net length violation.

metrics_tim_nvp_io

A read-only int attribute that returns the NVP IO.

metrics_tim_nvp_io_local

A read-only int attribute that returns the NVP IO.

metrics_tim_nvp_r2r

A read-only int attribute that returns the NVP R2R.

metrics_tim_nvp_r2r_local

A read-only int attribute that returns the NVP R2R.

metrics_tim_nvp_total

A read-only int attribute that returns NVP.

metrics_tim_nvp_total_local

A read-only int attribute that returns NVP.

metrics_tim_optimization_effort

A read-only float attribute that returns optimization_effort.

metrics_tim_optimization_effort_local

A read-only float attribute that returns optimization_effort.

r

metrics_tim_path_length_viol

A read-only int attribute that returns path length violation.

metrics_tim_path_length_viol_local

A read-only int attribute that returns path length violation.

metrics_tim_repeater_levels_viol

A read-only int attribute that returns repeater level violation.

metrics_tim_repeater_levels_viol_local

A read-only int attribute that returns repeater level violation.

metrics_tim_tns_io

A read-only double attribute that returns TNS IO.

metrics_tim_tns_io_local

A read-only double attribute that returns TNS IO.

metrics_tim_tns_r2r

A read-only double attribute that is TNS R2R.

metrics_tim_tns_r2r_local

A read-only double attribute that is TNS R2R.

metrics_tim_tns_total

A read-only double attribute that is TNS.

metrics_tim_tns_total_local

A read-only double attribute that is TNS.

metrics_tim_wns_io

A read-only double attribute that is WNS IO.

metrics_tim_wns_io_local

A read-only double attribute that is WNS IO.

metrics_tim_wns_r2r

A read-only double attribute that is WNS R2R.

metrics_tim_wns_r2r_local

A read-only double attribute that is WNS R2R.

r

metrics_tim_wns_total

A read-only double attribute that is WNS.

metrics_tim_wns_total_local

A read-only double attribute that is WNS.

name

A read-only string attribute that returns the name of this object.

object_class

A read-only string attribute that returns the name of this object class. Returns 'rtl_cell_construct' for rtl_cell_construct objects.

parent_construct

A read-only collection attribute that returns the parent cell construct.

rtl_cell

A read-only collection attribute that returns the RTL cell.

start_line

A read-only int attribute that returns the start line of the construct.

switching_power

A read-only double attribute that returns the switching power.

switching_power_local

A read-only double attribute that returns the switching power.

total_power

A read-only double attribute that returns the total power.

total_power_local

A read-only double attribute that returns the total power.

type

A read-only string attribute that returns the type of the construct.

See Also

- [get_attribute](#)
- [list_attributes](#)
- [set_attribute](#)

r

rtl_line_attributes

Description of the application defined rtl_line attributes.

Description

Attributes are properties assigned to objects such as pins, cells, and nets. They can be read-only or settable. A read-only attribute is informational; you cannot set its value. To modify a settable attribute, use the *set_attribute* command.

To determine the value of an attribute, use the *get_attribute* command. To see a list of all rtl_line attributes available, use the *list_attributes -application -class rtl_line* command.

Attributes for rtl_line

This section describes the rtl_line attributes.

file_name

A read-only string attribute that returns the RTL file.

full_name

A read-only string attribute that returns the full name of this object.

glitch_power_local

A read-only double attribute that returns the glitch power.

internal_power_local

A read-only double attribute that returns the internal power.

leakage_power_local

A read-only double attribute that returns the leakage power.

line

A read-only int attribute that returns the RTL line.

metrics_cong_cell_padding_local

A read-only double attribute that returns the cell padding.

metrics_cong_logic_cong_local

A read-only int attribute that returns the number of logic congestion cells.

metrics_cong_number_cells_in_cong_area_local

A read-only int attribute that returns the number of congested cells.

r

metrics_cong_number_cells_in_cong_channel_local

A read-only int attribute that returns the number of congested cells in the channel.

metrics_cong_number_cells_local

A read-only int attribute that returns the number of cells.

metrics_cong_overflow_local

A read-only double attribute that returns the congestion overflow.

metrics_cong_percent_cells_in_cong_area_local

A read-only double attribute that returns the percent of cells in congestion area.

metrics_cong_percent_of_cong_area_local

A read-only double attribute that returns the congested cells percentage vs. total congested cells

metrics_drc_estimate_local

A read-only double attribute that returns the DRC estimate.

metrics_tim_bottleneck_count_local

A read-only int attribute that returns the bottleneck cells.

metrics_tim_logic_levels_viol_local

A read-only int attribute that returns the logic level.

metrics_tim_max_logic_levels_viol_local

A read-only int attribute that returns the max logic level.

metrics_tim_net_length_viol_local

A read-only int attribute that returns the net length violation.

metrics_tim_nvp_io_local

A read-only int attribute that returns the NVP IO.

metrics_tim_nvp_r2r_local

A read-only int attribute that returns NVP R2R.

metrics_tim_nvp_total_local

A read-only int attribute that returns NVP.

metrics_tim_optimization_effort_local

A read-only float attribute that returns optimization_effort.

r

metrics_tim_path_length_viol_local

A read-only int attribute that returns path length violation.

metrics_tim_repeater_levels_viol_local

A read-only int attribute that returns repeater level violation.

metrics_tim_tns_io_local

A read-only double attribute that returns TNS IO.

metrics_tim_tns_r2r_local

A read-only double attribute that returns TNS R2R.

metrics_tim_tns_total_local

A read-only double attribute that returns TNS.

metrics_tim_wns_io_local

A read-only double attribute that returns WNS IO.

metrics_tim_wns_r2r_local

A read-only double attribute that returns WNS R2R.

metrics_tim_wns_total_local

A read-only double attribute that returns WNS.

name

A read-only string attribute that returns the name of this object.

object_class

A read-only string attribute that returns the name of this object class. Returns 'rtl_line' for rtl_line objects.

parent

A read-only collection attribute that returns the rtl parent object.

parent_construct

A read-only collection attribute that returns the rtl parent construct object.

switching_power_local

A read-only double attribute that returns the switching power.

total_power_local

A read-only double attribute that returns the total power.

type

A read-only string attribute that returns the line type (parent object type).

See Also

- [get_attribute](#)
- [list_attributes](#)
- [set_attribute](#)

rtl_module_attributes

Description of the application defined rtl_module attributes.

Description

Attributes are properties assigned to objects such as pins, cells, and nets. They can be read-only or settable. A read-only attribute is informational; you cannot set its value. To modify a settable attribute, use the *set_attribute* command.

To determine the value of an attribute, use the *get_attribute* command. To see a list of all rtl_module attributes available, use the *list_attributes -application -class rtl_module* command.

Attributes for rtl_module

This section describes the rtl_module attributes.

end_line

A read-only int attribute that returns the end line of the module.

file_name

A read-only string attribute that returns the RTL file.

full_name

A read-only string attribute that returns the full name of this object.

glitch_power

A read-only double attribute that returns the glitch power.

glitch_power_local

A read-only double attribute that returns the glitch power.

hdl_file

A read-only string attribute that returns the hdl file.

r

hdl_line

A read-only int attribute that returns the hdl line.

internal_power

A read-only double attribute that returns the internal power.

internal_power_local

A read-only double attribute that returns the internal power.

leakage_power

A read-only double attribute that returns the leakage power.

leakage_power_local

A read-only double attribute that returns the leakage power.

metrics_cong_cell_padding

A read-only double attribute that returns the cell padding.

metrics_cong_cell_padding_local

A read-only double attribute that returns cell padding.

metrics_cong_logic_cong

A read-only int attribute that returns the number of logic congestion cells.

metrics_cong_logic_cong_local

A read-only int attribute that returns the number of logic congestion cells.

metrics_cong_number_cells

A read-only int attribute that returns the number of cells.

metrics_cong_number_cells_in_cong_area

A read-only int attribute that returns the number of congested cells.

metrics_cong_number_cells_in_cong_area_local

A read-only int attribute that returns the number of congested cells.

metrics_cong_number_cells_in_cong_channel

A read-only int attribute that returns the number of congested cells in a channel.

metrics_cong_number_cells_in_cong_channel_local

A read-only int attribute that returns the number of congested cells in a channel.

r

metrics_cong_number_cells_local

A read-only int attribute that returns the number of cells.

metrics_cong_overflow

A read-only double attribute that returns congestion overflow.

metrics_cong_overflow_local

A read-only double attribute that returns the congestion overflow.

metrics_cong_percent_cells_in_cong_area

A read-only double attribute that returns the percent of cells in congestion area.

metrics_cong_percent_cells_in_cong_area_local

A read-only double attribute that returns the percent of cells in congestion area.

metrics_cong_percent_of_cong_area

A read-only double attribute that returns the congested cells percentage vs. total congested cells

metrics_cong_percent_of_cong_area_local

A read-only double attribute that returns the congested cells percentage vs. total congested cells

metrics_drc_estimate

A read-only double attribute that returns the DRC estimate.

metrics_drc_estimate_local

A read-only double attribute that returns the DRC estimate.

metrics_tim_bottleneck_count

A read-only int attribute that returns the bottleneck cells.

metrics_tim_bottleneck_count_local

A read-only int attribute that returns the bottleneck cells.

metrics_tim_logic_levels_viol

A read-only int attribute that returns the logic level.

metrics_tim_logic_levels_viol_local

A read-only int attribute that returns the logic level.

metrics_tim_max_logic_levels_viol

A read-only int attribute that returns the max logic level.

r

metrics_tim_max_logic_levels_viol_local

A read-only int attribute that returns the max logic level.

metrics_tim_net_length_viol

A read-only int attribute that returns the net length violation.

metrics_tim_net_length_viol_local

A read-only int attribute that returns the net length violation.

metrics_tim_nvp_io

A read-only int attribute that returns the NVP IO.

metrics_tim_nvp_io_local

A read-only int attribute that returns the NVP IO.

metrics_tim_nvp_r2r

A read-only int attribute that returns the NVP R2R.

metrics_tim_nvp_r2r_local

A read-only int attribute that returns the NVP R2R.

metrics_tim_nvp_total

A read-only int attribute that returns the NVP.

metrics_tim_nvp_total_local

A read-only int attribute that returns NVP.

metrics_tim_optimization_effort

A read-only float attribute that returns optimization_effort.

metrics_tim_optimization_effort_local

A read-only float attribute that returns optimization_effort.

metrics_tim_path_length_viol

A read-only int attribute that returns path length violation.

metrics_tim_path_length_viol_local

A read-only int attribute that returns path length violation.

metrics_tim_repeater_levels_viol

A read-only int attribute that returns repeater level violation.

r

metrics_tim_repeater_levels_viol_local

A read-only int attribute that returns repeater level violation.

metrics_tim_tns_io

A read-only double attribute that returns TNS IO.

metrics_tim_tns_io_local

A read-only double attribute that returns TNS IO.

metrics_tim_tns_r2r

A read-only double attribute that returns TNS R2R.

metrics_tim_tns_r2r_local

A read-only double attribute that returns TNS R2R.

metrics_tim_tns_total

A read-only double attribute that returns TNS.

metrics_tim_tns_total_local

A read-only double attribute that returns TNS.

metrics_tim_wns_io

A read-only double attribute that returns WNS IO.

metrics_tim_wns_io_local

A read-only double attribute that returns WNS IO.

metrics_tim_wns_r2r

A read-only double attribute that returns WNS R2R.

metrics_tim_wns_r2r_local

A read-only double attribute that returns WNS R2R.

metrics_tim_wns_total

A read-only double attribute that returns WNS.

metrics_tim_wns_total_local

A read-only double attribute that returns WNS.

name

A read-only string attribute that returns the name of this object.

r

object_class

A read-only string attribute that returns the name of this object class. Returns 'rtl_module' for rtl_module objects.

rtl_cells

A read-only collection attribute that contains rtl cells instantiated from module.

rtl_original_module

A read-only collection attribute that returns the original rtl module of the module.

start_line

A read-only int attribute that returns the start line of the module.

switching_power

A read-only double attribute that returns the switching power.

switching_power_local

A read-only double attribute that returns the switching power.

total_power

A read-only double attribute that returns the total power.

total_power_local

A read-only double attribute that returns the total power.

See Also

- [get_attribute](#)
- [list_attributes](#)
- [set_attribute](#)

rtl_module_construct_attributes

Description of the application defined rtl_module_construct attributes.

Description

Attributes are properties assigned to objects such as pins, cells, and nets. They can be read-only or settable. A read-only attribute is informational; you cannot set its value. To modify a settable attribute, use the *set_attribute* command.

r

To determine the value of an attribute, use the *get_attribute* command. To see a list of all *rtl_module_construct* attributes available, use the *list_attributes -application -class rtl_module_construct* command.

Attributes for *rtl_module_construct*

This section describes the *rtl_module_construct* attributes.

end_line

A read-only int attribute that returns the end line of the construct.

file_name

A read-only string attribute that returns the RTL file.

full_name

A read-only string attribute that returns the full name of this object.

glitch_power

A read-only double attribute that returns the glitch power.

glitch_power_local

A read-only double attribute that returns the glitch power.

hdl_file

A read-only string attribute that returns the hdl file.

hdl_line

A read-only int attribute that returns the hdl line.

internal_power

A read-only double attribute that returns the internal power.

internal_power_local

A read-only double attribute that returns the internal power.

leakage_power

A read-only double attribute that returns the leakage power.

leakage_power_local

A read-only double attribute that returns the leakage power.

metrics_cong_cell_padding

A read-only double attribute that returns the cell padding.

r

metrics_cong_cell_padding_local

A read-only double attribute that returns the cell padding.

metrics_cong_logic_cong

A read-only int attribute that returns the number of logic congestion cells.

metrics_cong_logic_cong_local

A read-only int attribute that returns the number of logic congestion cells.

metrics_cong_number_cells

A read-only int attribute that returns the number of cells.

metrics_cong_number_cells_in_cong_area

A read-only int attribute that returns the number of congested cells.

metrics_cong_number_cells_in_cong_area_local

A read-only int attribute that returns the number of congested cells.

metrics_cong_number_cells_in_cong_channel

A read-only int attribute that returns the number of congested cells in a channel.

metrics_cong_number_cells_in_cong_channel_local

A read-only int attribute that returns the number of congested cells in a channel.

metrics_cong_number_cells_local

A read-only int attribute that returns the number of cells.

metrics_cong_overflow

A read-only double attribute that returns the congestion overflow.

metrics_cong_overflow_local

A read-only double attribute that returns congestion overflow.

metrics_cong_percent_cells_in_cong_area

A read-only double attribute that returns the percent of cells in congestion area.

metrics_cong_percent_cells_in_cong_area_local

A read-only double attribute that returns the percent of cells in congestion area.

metrics_cong_percent_of_cong_area

A read-only double attribute that returns the congested cells percentage vs. total congested cells

r

metrics_cong_percent_of_cong_area_local

A read-only double attribute that returns the congested cells percentage vs. total congested cells

metrics_drc_estimate

A read-only double attribute that returns the DRC estimate.

metrics_drc_estimate_local

A read-only double attribute that returns the DRC estimate.

metrics_tim_bottleneck_count

A read-only int attribute that returns the bottleneck cells.

metrics_tim_bottleneck_count_local

A read-only int attribute that returns the bottleneck cells.

metrics_tim_logic_levels_viol

A read-only int attribute that returns the logic level.

metrics_tim_logic_levels_viol_local

A read-only int attribute that returns the logic level.

metrics_tim_max_logic_levels_viol

A read-only int attribute that returns the max logic level.

metrics_tim_max_logic_levels_viol_local

A read-only int attribute that returns the max logic level.

metrics_tim_net_length_viol

A read-only int attribute that returns the net length violation.

metrics_tim_net_length_viol_local

A read-only int attribute that returns the net length violation.

metrics_tim_nvp_io

A read-only int attribute that returns the NVP IO.

metrics_tim_nvp_io_local

A read-only int attribute that returns the NVP IO.

metrics_tim_nvp_r2r

A read-only int attribute that returns the NVP R2R.

r

metrics_tim_nvp_r2r_local

A read-only int attribute that returns the NVP R2R.

metrics_tim_nvp_total

A read-only int attribute that returns the NVP.

metrics_tim_nvp_total_local

A read-only int attribute that returns NVP.

metrics_tim_optimization_effort

A read-only float attribute that returns optimization_effort.

metrics_tim_optimization_effort_local

A read-only float attribute that returns optimization_effort.

metrics_tim_path_length_viol

A read-only int attribute that returns path length violation.

metrics_tim_path_length_viol_local

A read-only int attribute that returns path length violation.

metrics_tim_repeater_levels_viol

A read-only int attribute that returns repeater level violation.

metrics_tim_repeater_levels_viol_local

A read-only int attribute that returns repeater level violation.

metrics_tim_tns_io

A read-only double attribute that returns TNS IO.

metrics_tim_tns_io_local

A read-only double attribute that returns TNS IO.

metrics_tim_tns_r2r

A read-only double attribute that returns TNS R2R.

metrics_tim_tns_r2r_local

A read-only double attribute that returns TNS R2R.

metrics_tim_tns_total

A read-only double attribute that returns TNS.

r

metrics_tim_tns_total_local

A read-only double attribute that returns TNS.

metrics_tim_wns_io

A read-only double attribute that returns WNS IO.

metrics_tim_wns_io_local

A read-only double attribute that returns WNS IO.

metrics_tim_wns_r2r

A read-only double attribute that returns WNS R2R.

metrics_tim_wns_r2r_local

A read-only double attribute that returns WNS R2R.

metrics_tim_wns_total

A read-only double attribute that returns WNS.

metrics_tim_wns_total_local

A read-only double attribute that returns WNS.

name

A read-only string attribute that returns the name of this object.

object_class

A read-only string attribute that returns the name of this object class. Returns 'rtl_module_construct' for rtl_module_construct objects.

parent_construct

A read-only collection attribute that is the parent module construct.

rtl_module

A read-only collection attribute that is the rtl module.

start_line

A read-only int attribute that returns the start line of the construct.

switching_power

A read-only double attribute that returns the switching power.

switching_power_local

A read-only double attribute that returns the switching power.

s

total_power

A read-only double attribute that returns the total power.

total_power_local

A read-only double attribute that returns the total power.

type

A read-only string attribute that returns the construct type.

See Also

- [get_attribute](#)
- [list_attributes](#)
- [set_attribute](#)

S

safety_core_group_attributes

Lists the predefined attributes for safety core groups.

Description

Attributes are properties assigned to objects such as pins, cells and nets. Definitions for safety core group attributes are provided in the subsections that follow.

Attributes are read-only unless marked as settable. Read-only attributes are informational, and you cannot set its value. A settable attribute can be modified with the *set_attribute* command.

To determine the value of an attribute, use the *get_attribute* command. To see a list of all attributes available for a class of objects use the *list_attributes -application* command.

Safety Core Group Attributes*control_groups*

A read-only name list attribute of the control groups for this safety core group.

correction_signal

A read-only pin or port collection attribute to which all the correction outputs are connected to.

delay_cells

A settable cell collection attribute of instances or cells which have been introduced with delay.

error_signal

A settable pin or port collection attribute by which compare error are connected.

full_name

A read-only string attribute for the full name of the safety core group.

group_safety_cores

A read-only group collection attribute of group of cell groups that constitute the safety core group.

name

A settable string attribute for the name of the safety core group.

object_class

A read-only string attribute with the value of "safety_core_group".

requirement_id

A read-only string attribute for the tracking id of the safety core group in requirement management system.

rule

A settable safety_core_rule collection attribute based on which this safety core group has been created.

safety_cores

A settable cell collection attribute that constitute the safety core group.

split_pins

A settable pin collection attribute, which should be split to be a part of different branches of high-fanout tree.

taps

A settable cell collection attribute of tap cells for the safety cores in this safety core group.

voting_cells

A settable cell collection attribute of logic cells, which form a part of the voting logic circuit.

See Also

- [get_safety_core_groups](#)
- [get_attribute](#)
- [list_attributes](#)
- [set_attribute](#)

safety_core_rule_attributes

Lists the predefined attributes for safety core rules.

Description

Attributes are properties assigned to objects such as pins, cells and nets. Definitions for safety core rule attributes are provided in the subsections that follow.

Attributes are read-only unless marked as settable. Read-only attributes are informational, and you cannot set its value. A settable attribute can be modified with the *set_attribute* command.

To determine the value of an attribute, use the *get_attribute* command. To see a list of all attributes available for a class of objects use the *list_attributes -application* command.

Safety Core Rule Attributes

distance_type

A read-only string attribute (with allowed values: xy radial) to indicate whether the separation distance was specified in the form of X & Y distances, or radial distance.

full_name

A read-only string attribute for the full name of the safety core rule.

isolation

A settable boolean attribute for indicating whether the safety core group associated with this rule needs isolation or not.

logic_mapping_refs

A read-only name list attribute of the library cells that can be instantiated for implementing the comparison logic.

logic_module

A read-only module collection attribute for the module containing the combinational logic which are driven by safety cores output.

name

A settable string attribute for the name of the safety core rule.

number_of_cores

A read-only unsigned integer attribute for the number of copies of core that need to be created.

object_class

A read-only string attribute with the value of "safety_core_rule".

r_distance

A settable distance attribute for the radial separation distance of the safety core rule. Valid only if "distance_type" is "radial".

routing_guardband

A settable distance attribute for the required internal net radial routing separation between the cores.

routing_guardband_xy

A settable distance pair attribute for the required internal net separation distance of the safety core rule.

routing_separation

A settable boolean attribute to indicate if the routing separation is required or not.

safety_cores

A settable cell collection attribute for the safety cores on which this safety core rule is applied.

split_pins

A settable pin collection attribute for the split pins for this safety core rule.

tap_mapping

A settable name list attribute of library cells, out of which the appropriate one is used for instantiating tap cells for the safety cores.

xy_distance

A settable distance pair attribute for the separation distance of the safety core rule. Valid only if "distance_type" is "xy".

See Also

- [get_safety_core_rules](#)
- [get_attribute](#)
- [list_attributes](#)
- [set_attribute](#)

safety_error_code_group_attributes

Lists the predefined attributes for safety error code groups.

Description

Attributes are properties assigned to objects such as pins, cells and nets. Definitions for safety error code group attributes are provided in the subsections that follow.

Attributes are read-only unless marked as settable. Read-only attributes are informational, and you cannot set its value. A settable attribute can be modified with the *set_attribute* command.

To determine the value of an attribute, use the *get_attribute* command. To see a list of all attributes available for a class of objects use the *list_attributes -application* command.

Safety Error Code Group Attributes

checkbits

A read-only pin or port collection attribute, where the checkbits from the encoder output are connected for this *safety_error_code_group*.

correction_signal

A read-only pin or port collection attribute to which the decoder correction output is connected to. This can be specified for rules of encoding type *ecc* and when the mode is either *decode* or *encode_decode*.

data

A read-only pin or port collection attribute of data signal to be encoded or decoded for this *safety_error_code_group*.

decoder

A read-only cell collection attribute of the hierarchical or leaf-level cell which is the decoder logic for this *safety_error_code_group*.

encoder

A read-only cell collection attribute of the hierarchical or leaf-level cell which is the encoder logic for this `safety_error_code_group`.

error_signal

A read-only pin or port collection attribute of the design to which the decoder error output is connected to.

full_name

A read-only string attribute for the full name of the safety error code group.

name

A settable string attribute for the name of the safety error code group.

object_class

A read-only string attribute with the value of "safety_error_code_group".

requirement_id

A read-only string attribute for the tracking id of the safety error code group in requirement management system.

rule

A read-only `safety_error_code_rule` collection attribute based on which this `safety_error_code_group` has been created.

split_pins

A settable pin collection attribute, which should be split to be a part of different branches of high-fanout tree.

taps

A settable cell collection attribute of tap cells for this safety error code group.

See Also

- [get_safety_error_code_groups](#)
- [get_attribute](#)
- [list_attributes](#)
- [set_attribute](#)

safety_error_code_rule_attributes

Lists the predefined attributes for safety error code rules.

Description

Attributes are properties assigned to objects such as pins, cells and nets. Definitions for safety error code rule attributes are provided in the subsections that follow.

Attributes are read-only unless marked as settable. Read-only attributes are informational, and you cannot set its value. A settable attribute can be modified with the *set_attribute* command.

To determine the value of an attribute, use the *get_attribute* command. To see a list of all attributes available for a class of objects use the *list_attributes -application* command.

Safety Error Code Rule Attributes

distance_type

A read-only string attribute (with allowed values: xy radial) that indicates whether the separation distance was specified in the form of X & Y distances, or radial distance.

full_name

A read-only string attribute for the full name of the safety error code rule.

isolation

A settable boolean attribute for indicating whether the safety error code group associated with this rule needs isolation or not.

name

A settable string attribute for the name of the safety error code rule.

object_class

A read-only string attribute with the value of "safety_error_code_rule".

r_distance

A settable distance attribute for the radial separation distance of the safety error code rule. Valid only if "distance_type" is "radial".

sequential

A read-only boolean attribute for indicating whether the rule's encoding type is sequential or not.

slice_size

A read-only unsigned integer attribute for defining the number of bits of data to be encoded as a group.

split_pin_types

A settable string list attribute (with allowed values: clock reset scan) for the pin types to be considered for splitting.

tap_mapping

A settable name list attribute of library cells, out of which the appropriate one is used for instantiating tap cells for the safety error code group associated with this rule.

type

A read-only string attribute (with allowed values: even_parity odd_parity ecc edc) for the encoding type of the safety error code rule.

xy_distance

A settable distance pair attribute for the separation distance of the safety error code rule. Valid only if “distance_type” is “xy”.

See Also

- [get_safety_error_code_rules](#)
- [get_attribute](#)
- [list_attributes](#)
- [set_attribute](#)

safety_register_group_attributes

Lists the predefined attributes for safety register groups.

Description

Attributes are properties assigned to objects such as pins, cells and nets. Definitions for safety register group attributes are provided in the subsections that follow.

Attributes are read-only unless marked as settable. Read-only attributes are informational, and you cannot set its value. A settable attribute can be modified with the *set_attribute* command.

To determine the value of an attribute, use the *get_attribute* command. To see a list of all attributes available for a class of objects use the *list_attributes -application* command.

Safety Register Group Attributes

correction_signal

A read-only pin or port collection attribute to which all the correction outputs are connected to.

full_name

A read-only string attribute for the full name of the safety register redundancy group.

grouped_register_pins

A settable pin collection attribute of register output pins, which are part of this register redundancy group.

grouped_registers

A settable cell collection attribute of register cells, which are part of this register redundancy group.

logic

A settable cell collection attribute of logic cells, which form a part of the voting logic circuit. These must be combinational cells.

name

A settable string attribute for the name of the safety register redundancy group.

object_class

A read-only string attribute with the value of "safety_register_group".

requirement_id

A read-only string attribute for the tracking id of the safety register redundancy group in requirement management system.

rule

A settable safety_register_rule collection attribute based on which this safety register redundancy group has been created.

split_pins

A settable pin collection attribute, which should be split to be a part of different branches of high-fanout tree.

tap_cells

A settable cell collection attribute of tap cells for the register in this safety register redundancy group.

See Also

- [get_safety_register_groups](#)
- [get_attribute](#)
- [list_attributes](#)
- [set_attribute](#)

safety_register_rule_attributes

Lists the predefined attributes for safety register rules.

Description

Attributes are properties assigned to objects such as pins, cells and nets. Definitions for safety register rule attributes are provided in the subsections that follow.

Attributes are read-only unless marked as settable. Read-only attributes are informational, and you cannot set its value. A settable attribute can be modified with the *set_attribute* command.

To determine the value of an attribute, use the *get_attribute* command. To see a list of all attributes available for a class of objects use the *list_attributes -application* command.

Safety Register Rule Attributes

distance_type

A read-only string attribute (with allowed values: xy radial) that indicates whether the separation distance was specified in the form of X & Y distances, or radial distance.

fit_rate_threshold

A read-only float attribute for the fit rate threshold for the safety register rule.

full_name

A read-only string attribute for the full name of the safety register rule.

has_isolation

A settable boolean attribute for indicating if the safety register group associated with this rule needs isolation or not.

logic_ref_name

A settable string attribute for the module name containing the combinational logic, which follows the duplicated or triplicated registers after the safety register group is created based on the given rule.

map_ref_names

A settable name list attribute of the one or more combinational library cells that should be considered for implementing the voting or error logic.

name

A settable string attribute for the name of the safety register rule. This attribute is settable.

object_class

A read-only string attribute with the value of "safety_register_rule".

r_distance

A settable distance attribute for the radial separation distance of the safety register rule. Valid only if "distance_type" is "radial".

redundancy_mode

A read-only integer attribute for the redundancy mode of the safety register rule.

register_type

A settable string attribute (with allowed values: dmr dual_mode fault_tolerant ft tmr triple_mode) for indicating whether the register should be mapped to another fault-tolerant variant, or should be replaced by a group of redundant flip-flops (redundancy group).

split_pin_types

A settable string list attribute (with allowed values: clock reset scan) for the pin types to be considered for splitting so as to be a part of different branches of clock tree or high-fanout net trees.

tap_ref_names

A settable name list attribute of library cells, out of which the appropriate one is used for instantiating tap cells for the safety registers associated with this rule.

xy_distance

A settable distance pair attribute for the separation distance of the safety register rule. Valid only if "distance_type" is "xy".

See Also

- [get_safety_register_rules](#)
- [get_attribute](#)

- [list_attributes](#)
- [set_attribute](#)

scenario_attributes

Description of the application defined scenario attributes.

Description

Attributes are properties assigned to objects such as pins, cells, and nets. They can be read-only or settable. A read-only attribute is informational; you cannot set its value. To modify a settable attribute, use the *set_attribute* command.

To determine the value of an attribute, use the *get_attribute* command. To see a list of all scenario attributes available, use the *list_attributes -application -class scenario* command.

Attributes for scenario

This section describes the scenario attributes.

active

A read-only boolean attribute that returns True if the scenario is active. If true, the configured analysis types (if any) is done. If false, no analysis is done.

auto_named

A read-only boolean attribute that returns True if the scenario name was automatically generated.

cell_em

A read-only boolean attribute that returns whether cell electromigration (EM) analysis is enabled for this scenario.

corner

A read-only collection attribute that returns the corner of the scenario.

dynamic_power

A read-only boolean attribute that returns whether dynamic power analysis is enabled for this scenario.

full_name

A read-only string attribute that returns the scenario's full name (the same as its name).

glitch_power

Specifies a Boolean attribute that is true if glitch power analysis is enabled for this scenario.

hold

A read-only boolean attribute that returns True if the scenario is configured for hold analysis.

leakage_power

Specifies a Boolean attribute that is true if leakage power analysis is enabled for this scenario.

max_cap

A read-only boolean attribute that returns True if the scenario is configured for max_capacitance DRC analysis.

max_tran

A read-only boolean attribute that returns True if the scenario is configured for max_transition DRC analysis.

min_cap

A read-only boolean attribute that returns True if the scenario is configured for min_capacitance DRC analysis.

mode

A read-only collection attribute that returns the scenario's mode.

name

A read-only string attribute that returns the scenario's name (either user- named or auto generated).

object_class

A read-only string attribute that returns the name of this object class. Returns 'scenario' for scenario objects.

power

A read-only boolean attribute that returns True, if the scenario is configured for power analysis.

setup

A read-only boolean attribute that returns True if the scenario is configured for setup analysis. All scenario attributes are read-only, and they always have valid values.

signal_em

A read-only boolean attribute that returns whether signal electromigration (EM) analysis is enabled for this scenario.

See Also

- [get_attribute](#)
- [list_attributes](#)
- [create_scenario](#)
- [set_scenario_status](#)

shape_attributes

Description of the application defined shape attributes.

Description

Attributes are properties assigned to objects such as pins, cells, and nets. They can be read-only or settable. A read-only attribute is informational; you cannot set its value. To modify a settable attribute, use the *set_attribute* command.

To determine the value of an attribute, use the *get_attribute* command. To see a list of all shape attributes available, use the *list_attributes -application -class shape* command.

Attributes for shape

This section describes the shape attributes.

access_direction

A read-only *access_dir* attribute that returns the accessible directions on the shape's pin.

anchor

A read-only collection attribute that returns the anchor for which this object is the *dependent_object*. When non-empty, this object is the *dependent_object* of the anchor, and its location is therefore constrained by the anchor's *anchor_object*. See the *create_anchor(2)* command man page for more details about anchors.

bbox

A settable *rect* attribute that returns the bounding-box of a shape. The format of a rectangle specification is *{{llx lly} {urx ury}}*, which specifies the lower-left and upper-right corners of the rectangle. This attribute is settable for rectangle shape and curved rectangle shape.

bottom_left_radius

A settable distance attribute that returns the radius of bottom left corner of the curved rectangle shape.

bottom_right_radius

A settable distance attribute that returns the radius of the bottom right corner of the curved rectangle shape.

boundary

A settable coord_list attribute that returns the boundary of the shape. The format of coordinate list specification is $\{\{x1\ y1\} \{x2\ y2\} \dots\}$, which specifies a list of points defining the boundary of the shape.

bounding_box

A settable bounding_box collection attribute that returns the bounding-box of a shape. The format of the collection specifies lower-left and upper-right corners of the rectangle.

center

A settable coord attribute that returns the origin of the curved shape. The nonlinear shape can either be circle, donut or ellipse shape.

curved_polyrect

A settable curved_poly_rect attribute that returns the outside boundary of the curved polygon or island shape. The format of a curved_poly_rect specification is $\{\{x1\ y1\} \{x2\ y2\} \{xx2\ yy2\} \text{convex}\} \{x3\ y3\} \{x4\ y4\} \{xx4\ yy4\} \text{convex}\} \dots\}$, which specifies $\{X1\ Y1\}$, $\{X2\ Y2\}$ are the coordinates of the start point of straight line segments Where $\{x2\ y2\} \{xx2\ yy2\} \text{convex}\}$, $\{x4\ y4\} \{xx4\ yy4\} \text{convex}\}$ are the start point + center + convex flag of the curved segments.

custom_route_flow

A settable string attribute that returns the value applied by custom router to this shape while saving design.

custom_shield_net1_associated

A settable string attribute that returns the stored signal net name associated with the custom router dynamic shield shape.

custom_shield_net2_associated

A settable string attribute that returns the stored signal net name associated with the custom router dynamic shield shape.

design_name

A read-only string attribute that returns the name of the design which owns the shape.

end_endcap

A settable string attribute (with allowed values: flush full_width half_width octagon round variable variable_octagon) that returns the style of the end endcap of the shape. This attribute only applies to shapes with shape_type "path".

end_extension

A settable distance attribute that returns the distance the shape extends past its end endpoint. This attribute only applies to shapes with shape_type "path".

full_name

A read-only string attribute that returns the name of the shape.

groups

A read-only group collection attribute that contains the groups in which this shape is a member.

height

A settable distance attribute that returns the height of the shape. This attribute only applies to shapes with shape_type "text".

holes

A settable curved_polygon_list attribute that contains the holes of the curved polygon or island shape. The format of a curved_poly_rect specification is { {x1 y1} {x2 y2 xx2 yy2 convex} {x3 y3} {x4 y4 xx4 yy4 convex} ...}, which specifies {X1 Y1}, {X2 Y2} are the coordinates of the start point of straight line segments Where {x2 y2 xx2 yy2 convex}, {x4 y4 xx4 yy4 convex} are the start point + center + convex flag of the curved segments.

ignore_ndr_spacing

A settable boolean attribute that indicates whether the secondary power router should ignore NDR spacing for the shape.

in_edit_group

A read-only boolean attribute that indicates whether the shape belongs to an edit group.

inner_radius

A settable distance attribute that returns the inner radius of the curved donut shape.

is_45

A read-only boolean attribute that indicates whether the shape has one or more edges at a 45 degree angle from an axis, but no edges that aren't at horizontal, vertical, or 45 degree angles.

is_drc_shape

A read-only boolean attribute that indicates whether a shape is a drc shape or not. Value can be true for line and path types only and is set by routing.

is_dummy_metal

A settable boolean attribute that returns whether a shape is dummy metal or not.

is_fixed

A read-only boolean attribute that returns True if the physical status of the shape is either fixed or locked.

is_horizontal

A read-only boolean attribute that returns a boolean value, only applicable to 2 point paths. The value is true if the two point path is parallel to the x-axis and false otherwise.

is_manhattan

A read-only boolean attribute that returns whether the shape has edges that are exclusively at horizontal or vertical angles.

is_mask_fixed

A settable boolean attribute that returns whether the mask of the shape is fixed.

is_parallel_wire

A read-only boolean attribute that returns whether this shape was created as a fat wire for parallel wire purpose. This attribute only exists on rect and path shapes.

is_pattern_must_join

A read-only boolean attribute that returns whether a shape is part of the pattern_must_join connection.

is_probe

A settable boolean attribute that returns whether this shape is a probe.

is_shadow

A read-only boolean attribute that returns whether the shape is pushed down from above.

is_vertical

A read-only boolean attribute that returns a boolean value, only applicable to 2 point paths. The value is true if the two point path is parallel to the y-axis and false otherwise.

is_via_ladder

A read-only boolean attribute that indicates whether shape is part of a via ladder.

justification

A settable string attribute (with allowed values: C CB CT LB LC LT RB RC RT) that returns the justification of the text string corresponding to the shape. This attribute only applies to shapes with shape_type "text".

layer

A read-only layer collection attribute that returns the layer on which the shape exists.

layer_name

A read-only string attribute that returns the name of the layer on which the shape exists.

layer_number

A read-only int attribute that returns the number of the layer on which the shape exists.

length

A read-only distance attribute that returns the length of the shape. This attribute only applies to shapes with shape_type "path".

mask_constraint

A settable string attribute (with allowed values: no_mask mask_one mask_two mask_three mask_four same_mask) that returns the multi-patterning mask_constraint of the shape.

name

A read-only string attribute that returns the name of the shape.

net

A settable net collection attribute that returns a collection representing the owner net of this shape.

net_type

A read-only string attribute (with allowed values: analog_ground analog_power analog_signal clock deep_nwell deep_pwell ground nwell power pwell reset scan signal tie_high tie_low unset) that returns the net type of the net which owns the shape.

number_of_points

A read-only int attribute that returns the number of points of the shape.

object_class

A read-only string attribute that returns the name of this object class. Returns 'shape' for shape objects.

object_id

A read-only int attribute that returns the object_id of the shape.

orientation

A settable string attribute (with allowed values: R0 R90 R180 R270 MX MXR90 MY MYR90) that returns the orientation of the shape.

origin

A settable coord attribute that returns the origin of the shape.

outer_radius

A settable distance attribute that returns the outer radius of the curved donut shape.

owner

A settable terminal, metal_area, or net collection attribute that returns the terminal or net which owns the shape.

parent_block

A read-only block collection attribute that returns the owner block object for this shape.

parent_block_cell

A read-only cell collection attribute that returns the parent block cell object for this shape.

parent_cell

A read-only cell collection attribute that returns the owner cell object for this shape.

physical_status

A settable string attribute (with allowed values: application_fixed fixed locked minor_change unrestricted) that returns the physical modification status of the shape. If status is unrestricted, the shape may be freely modified. If the status is application_fixed, the shape is fixed by the application for automatic changes (the application may change this status). If status is fixed, the shape is fixed by the user for automatic changes. If status is locked, the shape is fixed for automatic and manual changes.

pnet_ignore

A settable boolean attribute that marks PG shapes to be ignored by routing engines.

points

A settable coord_list attribute that returns the points of the shape. The format of coordinate list specification is {{x1 y1} {x2 y2} ...}, which specifies a list of points defining the boundary of the shape, or the centerline of the shape in the case where shape_type is "path".

purpose_number

A read-only int attribute that returns the purpose number of the shape.

radius

A settable distance attribute that returns the radius of the curved circle shape.

rule_type

A settable string attribute (with allowed values: default net_non_default none shape_non_default) that returns the rule type used to create the shape. If the value is default, it was created based on the default rule. If the value is net_non_default, it was created based on the net nondefault routing rule. If the value is none, it was created based on no rule. If the value is shape_non_default, it was created based on a shape specific routing rule. This is a settable attribute.

shadow_status

A settable string attribute (with allowed values: copied_down copied_up normal pulled_up pushed_down virtual_flat) that returns the shadow status of the shape.

shape_type

A read-only string attribute that returns the type of the shape, for example "rect", "polygon", "path", "text", etc.

shape_use

A settable string attribute (with allowed values: area_fill core_wire detail_route follow_pin global_route lib_cell_pin_connect macro_pin_connect opc partial_parallel_route pg_augmentation rdl ring shield_route stripe taper trunk user_route zero_skew) that returns the shape use.

start_endcap

A settable string attribute (with allowed values: flush full_width half_width octagon round variable variable_octagon) that returns the style of the start endcap of the shape. This attribute only applies to shapes with shape_type "path".

start_extension

A settable distance attribute that returns the distance the shape extends past its start endpoint. This attribute only applies to shapes with shape_type "path".

tag

A settable string attribute that returns the tag name of this shape associated with a P/G wire.

text

A settable string attribute that returns the text string corresponding to the shape. This attribute only applies to shapes with shape_type "text".

top_block

A read-only block collection attribute that returns the top block object for this shape.

top_left_radius

A settable distance attribute that returns the radius of the top left corner of the curved rectangle shape.

top_right_radius

A settable distance attribute that returns the radius of the top right corner of the curved rectangle shape.

width

A settable distance attribute that returns the width of the shape. This attribute only applies to shapes with shape_type "path".

x_axis_radius

A settable distance attribute that returns the radius of the x axis of the curved donut shape.

y_axis_radius

A settable distance attribute that returns the radius of the y axis of the curved donut shape.

angle

A read-only attribute that returns the angle in degree between 2-point path w.r.t. x-axis. The value range is [0,360).

See Also

- [create_shape](#)
- [get_attribute](#)
- [list_attributes](#)
- [set_attribute](#)

shape_pattern_attributes

Description of the application defined shape_pattern attributes.

Description

Attributes are properties assigned to objects such as pins, cells, and nets. They can be read-only or settable. A read-only attribute is informational; you cannot set its value. To modify a settable attribute, use the *set_attribute* command.

To determine the value of an attribute, use the *get_attribute* command. To see a list of all shape_pattern attributes available, use the *list_attributes -application -class shape_pattern* command.

Attributes for shape_pattern

This section describes the shape_pattern attributes.

alternating_mask_constraints

A settable mask_list attribute that returns the list of masks to apply the rule-based color inference for individual shapes in the shape_pattern. This attribute applies only when the mask_pattern is *alternating* or *alternate_m_dimensions* or *alternate_n_dimensions*. The value is a mDimension major ordered list of maskValue in string format. i.e. { mask_two mask_three mask_four ...} to be used for *alternating* or *alternate_m_dimensions* or *alternate_n_dimensions*

`mask_pattern`. The `maskValue` can be one of `no_mask`, `mask_one`, `mask_two`, `mask_three`, `same_mask`.

`base_bbox`

A settable `rect` attribute that returns the bounding-box of the base shape in this `shape_pattern`. The format of a rectangle specification is `{{llx lly} {urx ury}}`, which specifies the lower-left and upper-right corners of the rectangle.

`base_boundary`

A settable `coord_list` attribute that returns the boundary of the base shape in this `shape_pattern`. The format of coordinate list specification is `{{x1 y1} {x2 y2} ...}`, which specifies a list of points defining the boundary of the base shape.

`base_bounding_box`

A settable `bounding_box` collection attribute that returns the bounding-box of the base shape in this `shape_pattern`. The format of the collection specifies lower-left and upper-right corners of the rectangle.

`base_end_endcap`

A settable string attribute (with allowed values: `flush` `full_width` `half_width` `octagon` `round` `variable` `variable_octagon`) that returns the style of the end endcap of the base shape in this `shape_pattern`. This attribute only applies to `shape_patterns` with `shape_pattern_type` `"path_pattern"`.

`base_end_extension`

A settable distance attribute that returns the distance the base shape in this `shape_pattern` extends past its end endpoint. This attribute only applies to `shape_patterns` with `shape_pattern_type` `"path_pattern"`.

`base_is_45`

A read-only boolean attribute that indicates whether the base shape in this `shape_pattern` has one or more edges at a 45 degree angle from an axis, but no edges that aren't at horizontal, vertical, or 45 degree angles.

`base_is_horizontal`

A read-only boolean attribute that only applies to base shape in this `shape_pattern` of type 2 point path. The value is true if the two point path base shape is parallel to the x-axis and false otherwise.

`base_is_manhattan`

A read-only boolean attribute that indicates whether the base shape in this `shape_pattern` has edges that are exclusively at horizontal or vertical angles.

base_is_parallel_wire

A read-only boolean attribute that indicates whether the base shape in this shape_pattern was created as a fat wire for parallel wire purpose. This attribute only exists on shape_patterns with shape_pattern_type "rect_pattern" or "path_pattern".

base_is_vertical

A read-only boolean attribute that only applies to base shape in this shape_pattern of type 2 point paths. The value is true if the two point path base shape is parallel to the y-axis and false otherwise.

base_length

A read-only distance attribute that returns the length of the base shape in this shape_pattern. This attribute only applies to shape_patterns with shape_pattern_type "path_pattern".

base_points

A settable coord_list attribute that returns the points of the base shape in this shape_pattern. The format of coordinate list specification is {{x1 y1} {x2 y2} ...}, which specifies a list of points defining the boundary of the base shape, or the centerline of the base shape in the case where shape_pattern_type is "path_pattern".

base_start_endcap

A settable string attribute (with allowed values: flush full_width half_width octagon round variable variable_octagon) that returns the style of the start endcap of the base shape in this shape_pattern. This attribute only applies to shape_patterns with shape_pattern_type "path_pattern".

base_start_extension

A settable distance attribute that returns the distance the base shape in this shape_pattern extends past its start endpoint. This attribute only applies to shape_patterns with shape_pattern_type "path_pattern".

base_width

A settable distance attribute that returns the width of the base shape in this shape_pattern. This attribute only applies to shape_patterns with shape_pattern_type "path_pattern".

bbox

A read-only rect attribute that returns the bounding box of the shape_pattern.

bounding_box

A read-only `bounding_box` collection attribute that returns the bounding-box of the `shape_pattern`. The format of the collection specifies lower-left and upper-right corners of the rectangle.

design_name

A read-only string attribute that returns the name of the design which owns the `shape_pattern`.

displacements

A settable `delta_list` attribute that returns the arbitrary offsets for a single dimension of this `shape_pattern`. The format of delta list specification is `{ { distance:direction | {x-distance:x-direction y-distance:y-direction } } ... }`. Exactly `{m_dimension - 1}` or `{n_dimension - 1}` offsets must be specified for the single dimension `{M x 1}` or `{1 x N}` `shape_pattern`.

full_name

A read-only string attribute that returns the name of the `shape_pattern`.

grid_size

A settable int attribute that returns the offset multiplier. The value must be greater than zero.

groups

A read-only group collection attribute that contains the groups in which this `shape_pattern` is a member.

has_net

A read-only boolean attribute that indicates whether the `shape_pattern` associates with a net.

ignore_ndr_spacing

A settable boolean attribute that determines whether the secondary power router should ignore NDR spacing for the `shape_pattern`.

in_edit_group

A read-only boolean attribute that indicates whether the `shape_pattern` belongs to an edit group.

index_mask_constraint

A settable string attribute that returns the mask constraint at a given index in the `shape_pattern`. This attribute is subscripted. The subscript value should be `<mDimensionIndex>,<nDimensionIndex>`. The specified `mDimensionIndex` and

s

nDimensionIndex must be less than the m_dimension and n_dimension of the shape_pattern. Using this attribute, one may set the mask_constraint_override by specifying the mask-color and is_mask_fixed for a specific shape in the shape_pattern. For example to set mask as fixed at mDimensionIndex 3 nDimensionIndex 2:

```
prompt> set_attribute $shapePattern index_mask_constraint(3,2)
{mask_two fixed}
```

The value is a string in format of {maskValue [fixed]} e.g. {mask_two fixed} which means set the mask of shape at (3,2) position as mask_two and fixed. The fixed can be absent, which means the mask constraint isn't fixed. The maskValue can be one of no_mask, mask_one, mask_two, mask_three, same_mask.

index_occupied

A settable boolean attribute that indicates whether a given index in the shape_pattern is currently occupied or empty (hole). This attribute is subscripted. The subscript value should be <mDimensionIndex>,<nDimensionIndex>. The specified mDimensionIndex and nDimensionIndex must be less than the m_dimension and n_dimension of the shape_pattern. Using this attribute, one may remove a specific shape in the shape_pattern. For example to punch a hole at mDimensionIndex 3 nDimensionIndex 2:

```
prompt> set_attribute $shapePattern index_occupied(3,2) false
```

is_fixed

A read-only boolean attribute that returns true if the physical status of the shape_pattern is either fixed or locked.

is_shadow

A read-only boolean attribute that indicates whether the shape_pattern is pushed down from above.

is_uniform_mask_fixed

A settable boolean attribute that determines whether the uniform-mask of the shape_pattern is fixed.

layer

A read-only layer collection attribute that returns the layer on which the shape_pattern exists.

layer_name

A read-only string attribute that returns the name of the layer on which the shape_pattern exists.

layer_number

A read-only int attribute that returns the number of the layer on which the *shape_pattern* exists.

m_dimension

A settable int attribute that returns the number of repetitions in the first dimension.

m_displacement

A settable delta attribute that returns the uniform offset in the first dimension.

mask_constraint_overrides

A read-only string attribute that returns the multi-patterning override mask_constraint of the *shape_pattern*. The value is a string in format of {m-dimension n-dimension maskValue [fixed]} i.e. {0 1 mask_two fixed} which means the mask of 0x1 shape is mask_two and fixed. The fixed can be absent, which means the mask constraint isn't fixed. maskValue can be one of no_mask, mask_one, mask_two, mask_three, same_mask. This attribute reports the sparse override mask-constraint applied on top of rule-based mask_pattern to fix coloring or DRC issues for a select few individual shapes of the composite shape_pattern using the subscripted index_mask_constraint attribute in the recommended manual edit flow. This attribute is not settable but may be used to clear override mask-constraints for all shapes of the composite shape_pattern using the *remove_attribute* command as shown in the example below.

```
prompt> remove_attribute $shapePattern mask_constraint_overrides
```

The reported value of this attribute may be truncated if the resulting string is very large (>1024 items).

mask_pattern

A settable string attribute (with allowed values: *alternate_m_dimensions*, *alternate_n_dimensions*, *alternating*, *uniform*) that returns the mask pattern of the *shape_pattern*. in a multiple-patterning technology. The *uniform* mask_pattern means all shapes in this shape_pattern have the same mask values. The *alternating*, *alternate_m_dimensions* or *alternate_n_dimensions* mask_pattern means the *alternating_mask_constraints* would be used to apply the rule-based mask inference. The *alternating* mask_pattern means the mask value of shapes in this shape_patten would alternate so that each shape has a different mask from its adjacent shape. The *alternate_m_dimensions* or *alternate_n_dimensions* means the mask value of shapes in this shape_pattern alternates in the respective dimension and all shapes at the same dimension-index would have same mask.

n_dimension

A settable int attribute that returns the number of repetitions in the second dimension.

n_displacement

A settable delta attribute that returns the uniform offset in the second dimension.

name

A read-only string attribute that returns the name of the shape_pattern.

net

A read-only net collection attribute that represents the owner net of this shape_pattern.

net_type

A read-only string attribute (with allowed values: analog_ground analog_power analog_signal clock deep_nwell deep_pwell ground nwell power pwell reset scan signal tie_high tie_low unset) that returns the net type of the net which owns the shape_pattern.

object_class

A read-only string attribute that returns the name of this object class. Returns 'shape_pattern' for shape_pattern objects.

object_id

A read-only int attribute that returns the object_id of the shape_pattern.

occupancy_pattern

A read-only string attribute that indicates the existence of shapes in this shape_pattern. The occupancy_pattern defines a MxN bit pattern of base shape existence. The value is mDimension major ordered list of substrings that specifies the occupancy matrix of the shape_pattern. If the occupancy_pattern is smaller than the shape_pattern size, it is replicated in the mDimension- major order. Excess bits in the occupancy_pattern are ignored. The occupancy_pattern is formatted as a space delimited string of equal length substrings (e.g., "1010 0101 1011" is a 3x4 pattern). Each substring must consist of only 1s and 0s, representing a mDimension in the MxN shape_pattern, starting with the first mDimension in the shape_pattern. A 1 means the base shape is included. A 0 means the base shape is omitted. If not specified, all the base shapes are included. This attribute is not settable but may be used to clear all unoccupied (hole) shapes in the composite shape_pattern using the *remove_attribute* command.

```
prompt> remove_attribute $shapePattern occupancy_pattern
```

The reported value of this attribute may be truncated if the resulting string is very large (>1024 characters).

occupancy_pattern_size

A read-only string attribute that returns the size of the occupancy_pattern in this shape_pattern. The string lists the number of rows and number of columns in the pattern (e.g., "3 4" indicates the occupancy_pattern has 3 rows and 4 columns).

owner

A read-only net collection attribute that represents the terminal or net which owns the shape_pattern.

parent_block

A read-only block collection attribute that returns the owner block object for this shape_pattern.

parent_block_cell

A read-only cell collection attribute that returns the parent block cell object for this shape_pattern.

parent_cell

A read-only cell collection attribute that returns the owner cell object for this shape_pattern.

physical_status

A settable string attribute (with allowed values: application_fixed fixed locked minor_change unrestricted) that returns the physical modification status of the shape_pattern. If status is unrestricted, the shape_pattern may be freely modified. If the status is application_fixed, the shape_pattern is fixed by the application for automatic changes (the application may change this status). If status is fixed, the shape_pattern is fixed by the user for automatic changes. If status is locked, the shape_pattern is fixed for automatic and manual changes.

purpose_number

A read-only int attribute that returns the purpose number of the shape_pattern.

rule_type

A settable string attribute (with allowed values: default net_non_default none shape_non_default) that returns the rule type used to create the shape_pattern. If the value is default, it was created based on the default rule. If the value is net_non_default, it was created based on the net nondefault routing rule. If the value is none, it was created based on no rule. If the value is

`shape_non_default`, it was created based on a `shape_pattern` specific routing rule. This is a settable attribute.

shadow_status

A settable string attribute (with allowed values: `copied_down` `copied_up` `normal` `pulled_up` `pushed_down` `virtual_flat`) that returns the shadow status of the `shape_pattern`.

shape_pattern_type

A read-only string attribute that returns the type of the `shape_pattern`, for example `"rect_pattern"`, `"polygon_pattern"`, `"path_pattern"`.

shape_use

A settable string attribute (with allowed values: `area_fill` `core_wire` `detail_route` `follow_pin` `global_route` `lib_cell_pin_connect` `macro_pin_connect` `opc` `partial_parallel_route` `pg_augmentation` `rdl` `ring` `shield_route` `stripe` `taper` `trunk` `user_route` `zero_skew`) that returns the shape use.

size

A read-only string attribute that returns the size of the `shape_pattern`, listing the number of repetitions in both dimensions of the `shape_pattern` (e.g.,...). g., `"3 4"` indicates the `shape_pattern` has 3 repetitions in `m_dimension` and 4 repetitions in `n_dimension`).

tag

A settable string attribute that returns the tag name of this `shape_pattern` associated with a P/G wire.

top_block

A read-only block collection attribute that returns the top block object for this `shape_pattern`.

uniform_mask_constraint

A settable string attribute that returns the uniform mask constraint of the `shape_pattern`. This attribute is non-subscripted and applies only when the `mask_pattern` is *uniform*. Using this attribute, one may set the `mask_constraint` by specifying the mask-color and `is_mask_fixed` for all shapes in the `shape_pattern`. For example, to set `uniform_mask` of the `shape_pattern` with `mask_pattern uniform`:

```
prompt> set_attribute $shapePattern uniform_mask_constraint  
{mask_two fixed}
```

The value is a string in format of `{maskValue [fixed]}` e.g. `{mask_two fixed}` which means set the mask of 3x2 shape as `mask_two` and `fixed`. The `fixed` can be

absent, which means the mask constraint isn't fixed. The maskValue can be one of no_mask, mask_one, mask_two, mask_three, same_mask.

See Also

- [create_shape_pattern](#)
- [get_attribute](#)
- [list_attributes](#)
- [set_attribute](#)

shaping_blockage_attributes

Description of the application defined shaping_blockage attributes.

Description

Attributes are properties assigned to objects such as pins, cells, and nets. They can be read-only or settable. A read-only attribute is informational; you cannot set its value. To modify a settable attribute, use the *set_attribute* command.

To determine the value of an attribute, use the *get_attribute* command. To see a list of all shaping_blockage attributes available, use the *list_attributes -application -class shaping_blockage* command.

Attributes for shaping_blockage

This section describes the shaping_blockage attributes.

area

A read-only area attribute that returns the area of the shaping_blockage.

bbox

A read-only rect attribute that returns the bounding-box of the shaping_blockage. The format of a rectangle specification is {{llx lly} {urx ury}}, which specifies the lower-left and upper-right corners of the rectangle.

blockage_purpose

A settable string attribute (with allowed values: system user) that returns the purpose of the placement_blockage. Valid values are system and user.

boundary

A settable coord_list attribute that returns the boundary of the shaping_blockage. The format of a coordinate_list specification is {{x1 y1} {x2 y2}...}.

bounding_box

A read-only `bounding_box` collection attribute that returns the bounding-box of the `shaping_blockage`. The format of the collection specifies lower-left and upper-right corners of the rectangle.

full_name

A read-only string attribute that returns the name of the `shaping_blockage`.

groups

A read-only group collection attribute that returns the groups in which this `shaping_blockage` is a member.

is_shadow

A read-only boolean attribute that indicates whether the `shaping_blockage` is pushed down from above.

name

A settable string attribute that returns the name of the `shaping_blockage`.

object_class

A read-only string attribute that returns the name of this object class. Returns 'shaping_blockage' for `shaping_blockage` objects.

parent_block

A read-only block collection attribute that returns the owner block object for this `shaping_blockage`.

parent_block_cell

A read-only cell collection attribute that returns the parent block cell object for this `shaping_blockage`.

parent_cell

A read-only cell collection attribute that returns the owner cell object for this `shaping_blockage`.

physical_status

A settable string attribute (with allowed values: `locked` `unrestricted`) that returns the physical modification status of the `shaping_blockage`. If status is `unrestricted`, the `shaping_blockage` may be moved or rotated. If the status is `locked`, the `shaping_blockage` may not be moved or rotated.

shadow_status

A settable string attribute (with allowed values: copied_down copied_up normal pulled_up pushed_down virtual_flat) that returns the shadow status of the shaping_blockage. Valid values are copied_down, copied_up, normal, pulled_up, pushed_down, and virtual_flat.

top_block

A read-only block collection attribute that returns the top block object for this shaping_blockage.

See Also

- [get_attribute](#)
- [list_attributes](#)

shaping_channel_attributes

Description of the application defined shaping_channel attributes.

Description

Attributes are properties assigned to objects such as pins, cells, and nets. They can be read-only or settable. A read-only attribute is informational; you cannot set its value. To modify a settable attribute, use the *set_attribute* command.

To determine the value of an attribute, use the *get_attribute* command. To see a list of all shaping_channel attributes available, use the *list_attributes -application -class shaping_channel* command.

The channel object represents a shaping channel on one of the external_channels, child_default_channels, or boundary_channels shaping constraints. The shaping channel defines the channel spacing along the boundary of the constrained object. The spacing may either apply generally to the object's boundary or with respect to another (neighbor) object. Channels may exist independent of any shaping_constraint object, and are assigned to a shaping_constraint object with the create_shaping_constraint command.

Attributes for shaping_channel

This section describes the shaping_channel attributes.

channel_bottom

A settable double attribute that returns a distance-valued attribute with subscripted values min and max. Defines the channel spacing along the bottom of a boundary.

channel_left

A settable double attribute that returns a distance-valued attribute with subscripted values min and max. Defines the channel spacing along the left side of a boundary.

channel_right

A settable double attribute that returns a distance-valued attribute with subscripted values min and max. Defines the channel spacing along the right side of a boundary.

channel_top

A settable double attribute that returns a distance-valued attribute with subscripted values min and max. Defines the channel spacing along the top of a boundary.

full_name

A read-only string attribute that gives the full name of the object, which consists of the owning shaping_constraint name (if it exists) followed by the channel object name.

name

A settable string attribute that gives the name of the object.

neighbor

A read-only collection attribute that gives the neighbor object to which the channel spacing applies.

object_class

A read-only string attribute that returns the name of this object class. Returns 'shaping_channel' for shaping_channel objects.

owner

A read-only shaping_constraint collection attribute that gives the owning shaping_constraint object for this channel if it is assigned.

parent_block

A read-only block collection attribute that gives the block in which the channel exists.

See Also

- [create_shaping_channel](#)
- [remove_shaping_channels](#)
- [get_shaping_channels](#)

shaping_constraint_attributes

Description of the application defined shaping_constraint attributes.

Description

Attributes are properties assigned to objects such as pins, cells, and nets. They can be read-only or settable. A read-only attribute is informational; you cannot set its value. To modify a settable attribute, use the *set_attribute* command.

To determine the value of an attribute, use the *get_attribute* command. To see a list of all shaping_constraint attributes available, use the *list_attributes -application -class shaping_constraint* command.

The shaping_constraint object represents one of several types of shaping constraints that the user may define for the purpose of guiding the Design Planning block shaping process towards a more desirable solution.

Shaping constraints apply to constrainable objects, which consist of voltage areas, move bounds, cells, shaping groups, and (non lib-cell) blocks. Each type of constrainable object has a subset of shaping constraints that are valid for it. A breakdown of the valid constraints for each type of constrainable object can be found below. A given constrainable object may have exactly one shaping_constraint object for each of its valid constraint types. Removing a constrainable object removes each of its defined shaping_constraint objects.

Attributes for shaping_constraint

This section describes the shaping_constraint attributes.

allowable_orientation

A settable orientations attribute that defines the allowable orientation for the constrained object.

array_layout

A settable string attribute (with allowed values: east north south west) that defines the layout of object within the constrained object.

boundary_status

A read-only string attribute (with allowed values: fixed rigid shapeable) that describes the status of the constrained object's boundary. The value is read-only and derived from the `physical_status` and `is_rigid_boundary` values for the object.

full_name

A read-only string attribute that gives the full name of the object, which consists of the constrained object's name followed by the object's name.

is_rigid_boundary

A settable boolean attribute that describes the rigidity of the object's boundary.

max_area

A settable area attribute that defines the maximum allowed value for the constrained object's boundary area.

max_ratio

A settable float attribute that defines the maximum allowed ratio value for aspect ratio shaping constraints.

max_util

A settable float attribute that defines the maximum allowed utilization value for utilization valued shaping constraints.

min_area

A settable area attribute that defines the minimum allowed value for the constrained object's boundary area.

min_ratio

A settable float attribute that defines the minimum allowed ratio value for aspect ratio shaping constraints.

min_util

A settable float attribute that defines the minimum allowed utilization value for the utilization valued shaping constraints.

name

A read-only string attribute that gives the object's name.

neighbor_channels

A read-only `shaping_channel` collection attribute that is a collection valued attribute. Gives the neighbor channel objects associated with the channel valued shaping constraints.

object_channel

A read-only `shaping_channel` collection attribute that is a collection valued attribute. Gives the generic object channel associated with the channel valued shaping constraints.

object_class

A read-only string attribute that returns the name of this object class. Returns 'shaping_constraint' for `shaping_constraint` objects.

owner

A read-only collection attribute that gives the object to which this shaping constraint applies.

parent_block

A read-only block collection attribute that gives the block in which this shaping constraint exists.

reference_object

A settable collection attribute that returns the reference object for object valued shaping constraints.

relative_x

A settable float attribute that returns a value within the allowed range of 0-1. Defines the x coordinate for relative location shaping constraints.

relative_y

A settable float attribute that returns a value within the allowed range of 0-1. Defines the y coordinate for relative location shaping constraints.

target_area

A settable area attribute that returns an area-valued constraint. Defines the desired target value for the constrained object's boundary area.

target_ratio

A settable float attribute that returns a float-valued constraint. Defines the desired target ratio value for the aspect ratio shaping constraints.

target_util

A settable float attribute that returns a float-valued constraint. Defines the desired target utilization value for the utilization valued shaping constraints.

See Also

- [create_shaping_constraint](#)
- [remove_shaping_constraints](#)
- [get_shaping_constraints](#)

site_array_attributes

Description of the application defined site_array attributes.

Description

Attributes are properties assigned to objects such as pins, cells, and nets. They can be read-only or settable. A read-only attribute is informational; you cannot set its value. To modify a settable attribute, use the *set_attribute* command.

To determine the value of an attribute, use the *get_attribute* command. To see a list of all site_array attributes available, use the *list_attributes -application -class site_array* command.

Attributes for site_array

This section describes the site_array attributes.

boundary

A settable polygon attribute that returns the boundary of this site_array. The format of a boundary specification is {{{llx1 lly1} {urx1 ury1}} {{llx2 lly2} {urx2 ury2} ...} ...}. This is the boundary of the draw area specified by you, when creating the site array. Changing its value can cause changes in other site arrays in the stack.

effective_shapes

A read-only poly_list attribute that returns the actual regions of site rows created as part of the site array considering all margins, stacking orders and transparencies.

full_name

A read-only string attribute that returns the name of the site_array.

groups

A read-only group collection attribute that returns the groups containing the site array.

inner_margin

A settable margin_list attribute that returns the margin by which a site array's boundary is shrunk and resultant effective region is considered for its site rows.

is_aligned

A read-only boolean attribute that returns whether the site array is aligned.

is_alternate_rows_flipped

A settable boolean attribute that returns whether alternate rows of the site array are flipped.

is_default

A read-only boolean attribute that returns whether the site array is a default site array.

is_first_row_flipped

A settable boolean attribute that returns whether the first row of the site array is flipped.

is_horizontal

A settable boolean attribute that returns whether the site array is horizontal or vertical.

is_row_offset_align_to_site_array

A settable boolean attribute that returns whether row_offsets need to be aligned with the site array bounding box.

is_shadow

A read-only boolean attribute that indicates whether this site array is pushed down from above.

is_transparent

A settable boolean attribute that returns whether site array is transparent or opaque. Changing its value might cause changes in other site arrays in the stack.

name

A settable string attribute that returns the name of the site_array.

object_class

A read-only string attribute that returns the name of this object class. Returns 'site_array' for site_array objects.

parent_block

A read-only block collection attribute that returns the owner block object for this site array.

parent_block_cell

A read-only cell collection attribute that returns the parent block cell object for this site array.

parent_cell

A read-only cell collection attribute that returns the owner cell object for this site array.

physical_status

A settable string attribute (with allowed values: locked unrestricted) that returns the physical modification status of the site array. If status is unrestricted, the site array might be moved or rotated. If the status is locked, the site array might not be moved or rotated.

row_cycles

A settable unsigned int_list attribute that returns the cycle for site rows of site array.

row_offsets

A settable unsigned int_list attribute that returns the offset for site rows of site array.

site_def

A read-only site_def collection attribute that returns the site definition used for creation of the site array.

site_name

A read-only name_list attribute that returns the name of the site that is used to create the site array.

stacking_order

A read-only int attribute that returns the stacking order of the site array among all site arrays, zero being the bottom most site array.

top_block

A read-only block collection attribute that returns the top block object for this site array.

x_core_offset

A read-only distance attribute that returns the x-offset of sites grid from block's origin. By default the site rows created within a site array align with a grid passing through block's origin.

x_margin

A settable distance attribute that returns the x-offset of sites within a row of the site array.

y_core_offset

A read-only distance attribute that returns the y-offset of sites grid from block's origin. By default the site rows created within a site array align with a grid passing through block's origin.

y_margin

A settable distance_list attribute that returns the y-offset of site rows of the site array.

See Also

- [create_site_array](#)
- [get_attribute](#)
- [list_attributes](#)
- [set_attribute](#)

site_def_attributes

Description of the application defined site_def attributes.

Description

Attributes are properties assigned to objects such as pins, cells, and nets. They can be read-only or settable. A read-only attribute is informational; you cannot set its value. To modify a settable attribute, use the *set_attribute* command.

To determine the value of an attribute, use the *get_attribute* command. To see a list of all site_def attributes available, use the *list_attributes -application -class site_def* command.

Attributes for site_def

This section describes the site_def attributes.

full_name

A settable string attribute that returns the name of the site_def.

height

A settable distance attribute that returns the height of the site_def.

is_default

A settable boolean attribute that returns whether the site_def is the default of its tech.

legal_orientations

A read-only orientations attribute that specifies the allowed orientations of the site_def.

name

A settable string attribute that returns the name of the site_def.

object_class

A read-only string attribute that returns the name of this object class. Returns 'site_def' for site_def objects.

symmetry

A settable string attribute that returns whether the site_def is symmetrical about the X and/or Y and/or 90-degree rotation. Valid values is a set composed with the strings "X", "Y" and "R90". Example: { X } or { Y R90 } or { X Y R90 } and so on.

tech

A read-only tech collection attribute that returns the owning tech of the site_def.

type

A settable string attribute (with allowed values: core pad) that returns the type of the site_def as either core or pad.

width

A settable distance attribute that returns the width of the site_def.

See Also

- [create_site_def](#)
- [get_attribute](#)
- [list_attributes](#)
- [set_attribute](#)

site_row_attributes

Description of the application defined site_row attributes.

Description

Attributes are properties assigned to objects such as pins, cells, and nets. They can be read-only or settable. A read-only attribute is informational; you cannot set its value. To modify a settable attribute, use the *set_attribute* command.

To determine the value of an attribute, use the *get_attribute* command. To see a list of all site_row attributes available, use the *list_attributes -application -class site_row* command.

Attributes for site_row

This section describes the site_row attributes.

bbox

A read-only rect attribute that returns the bounding-box of a site row. The format of a rectangle specification is `{{llx lly} {urx ury}}`, which specifies the lower-left and upper-right corners of the rectangle.

bounding_box

A read-only bounding_box collection attribute that returns the bounding-box of a site row. The format of the collection specifies lower-left and upper-right corners of the rectangle.

cycle

A settable unsigned attribute that returns the cycle of site row. This is a settable attribute for site rows not belonging to a site array.

design_name

A read-only string attribute that returns the name of the design owning the site row.

full_name

A read-only string attribute that returns the name of the site_row object.

groups

A read-only group collection attribute that returns the groups in which this site row is a member.

in_edit_group

A read-only boolean attribute that returns whether a site row belongs to an edit_group.

is_derived

A read-only boolean attribute that returns whether a site row belongs to a site array.

is_shadow

A read-only boolean attribute that returns whether a site row is pushed down from above.

is_site_r90symmetrical

A read-only boolean attribute that returns whether the site definition used for the site row is symmetrical with 90-degree orientation.

is_site_xsymmetrical

A read-only boolean attribute that returns whether the site definition used for the site row is symmetrical about x-axis.

is_site_ysymmetrical

A read-only boolean attribute that returns whether the site definition used for the site row is symmetrical about y-axis.

legal_orientations

A read-only orientations attribute that returns the allowed orientations of the site definition used for the site row.

name

A settable string attribute that returns the name of the site_row object. This is a settable attribute for site rows not belonging to a site array.

object_class

A read-only string attribute that returns the name of this object class. Returns 'site_row' for site_row objects.

object_id

A read-only unsigned attribute that returns the object_id of the site_row.

offset

A settable unsigned attribute that returns the offset of site row. This is a settable attribute for site rows not belonging to a site array.

orientation

A settable string attribute that returns the orientation of the site row. This is a settable attribute for site rows not belonging to a site array.

origin

A settable coord attribute that returns the origin of the site row. This is a settable attribute for site rows not belonging to a site array.

parent_block

A read-only block collection attribute that returns the owner block object for this site row.

parent_block_cell

A read-only cell collection attribute that returns the parent block cell object for this site row.

parent_cell

A read-only cell collection attribute that returns the owner cell object for this site row.

physical_status

A settable string attribute (with allowed values: locked unrestricted) that returns the physical modification status of the site row. If status is unrestricted, the site_row might be moved or rotated. If the status is locked, the site_row might not be moved or rotated.

shadow_status

A settable string attribute (with allowed values: copied_down copied_up normal pulled_up pushed_down virtual_flat) that returns the shadow status of a site row.

site_array

A read-only site_array collection attribute that returns the owning site array of the site row, if the site row belongs to any site array.

site_count

A settable int attribute that returns the number of sites in the site row. This is a settable attribute for site rows not belonging to a site array.

site_def

A read-only `site_def` collection attribute that returns the site definition used for creation of the site row.

site_height

A read-only distance attribute that returns the height of the site definition used for the `site_row` object.

site_name

A read-only string attribute that returns the name of the site definition that is used to create the site row.

site_orientation

A settable string attribute that returns the orientation of sites within the site row. This is relative to the site row orientation. This is a settable attribute for site rows not belonging to a site array.

site_space

A settable distance attribute that returns the spacing between sites inside the site row. This is a settable attribute for site rows not belonging to a site array.

site_width

A read-only distance attribute that returns the width of the site definition used for the site row.

top_block

A read-only block collection attribute that returns the top block object for this site row.

See Also

- [create_site_row](#)
- [get_attribute](#)
- [list_attributes](#)
- [set_attribute](#)

skew_group_attributes

Description of the application defined `skew_group` attributes.

Description

Attributes are properties assigned to objects such as pins, cells, and nets. They can be read-only or settable. A read-only attribute is informational; you cannot set its value. To modify a settable attribute, use the *set_attribute* command.

To determine the value of an attribute, use the *get_attribute* command. To see a list of all skew_group attributes available, use the *list_attributes -application -class skew_group* command.

Attributes for skew_group

This section describes the skew_group attributes.

command_text

A read-only string attribute that returns the command used to create this skew_group.

file_line_info

A read-only string attribute that returns the Tcl or SDC source file name and line number pair of the command used to create this object, if it exists.

full_name

A read-only string attribute that returns a unique name for the skew_group. This is the name specified by you or a tool generated name, if you do not specify name.

mode

A read-only collection attribute that specifies the mode in which this skew group has been created.

object_class

A read-only string attribute that returns the name of this object class. Returns 'skew_group' for skew_group objects.

objects

A read-only collection attribute that specifies the clock pins grouped together with this skew group constraint.

See Also

- [get_attribute](#)
- [list_attributes](#)

stub_chain_attributes

Description of the application defined stub_chain attributes.

Description

Attributes are properties assigned to objects such as pins, cells, and nets. They can be read-only or settable. A read-only attribute is informational; you cannot set its value. To modify a settable attribute, use the *set_attribute* command.

To determine the value of an attribute, use the *get_attribute* command. To see a list of all stub_chain attributes available, use the *list_attributes -application -class stub_chain* command.

Attributes for stub_chain

This section describes the stub_chain attributes.

elements

A read-only list attribute that returns floating as well as ordered element lists.

endpoint

A settable pin, or port collection attribute that indicates the ending port or pin of the stub chain. The port or pin must be physical.

floating_elements

A read-only list attribute that returns zero or more floating element lists. The scan-in and scan-out pin of each triplet are from the scan cell. List format:
{scan_in_pin_name scan_out_pin_name bit_length}...}

full_name

A read-only string attribute that returns the name of the stub_chain.

max_bits

A settable int attribute that returns the maximum bit length of the stub_chain object.

name

A settable string attribute that returns the name of the stub_chain object.

object_class

A read-only string attribute that returns the name of this object class. Returns 'stub_chain' for stub_chain objects.

ordered_elements

A read-only list attribute that returns zero or more ordered element lists. The scan-in and scan-out pin of each triplet are from the scan cell. List format: {{{scan_in_pin_name scan_out_pin_name bit_length}...}...}

parent_block

A read-only block collection attribute that returns the owner block object for the stub_chain object.

parent_block_cell

A read-only cell collection attribute that returns the parent block cell object for this stub_chain object.

parent_cell

A read-only cell collection attribute that returns the owner cell object for this stub_chain object.

reorderable_net_wire_length

A read-only double attribute that returns the manhattan wire length of the reorderable nets in this stub chain. This is equivalent to the *total_net_wire_length* attribute value minus the wire length between ordered elements in the same ordered element list.

startpoint

A settable pin, or port collection attribute that indicates the starting port or pin of the stub chain. The port or pin must be physical.

top_block

A read-only block collection attribute that returns the top block object for the stub_chain object.

total_bit_length

A read-only int attribute that returns the total bit length of all the elements in this stub chain.

total_net_wire_length

A read-only double attribute that returns the manhattan wire length of all nets in this stub chain. This is the same length value as reported by the *report_stub_chains -length* command.

validity_status

A read-only string attribute (with allowed values: failed unknown validated) that returns the netlist consistency status of this stub_chain.

See Also

- [create_stub_chain](#)
- [get_attribute](#)
- [list_attributes](#)
- [set_attribute](#)

supernet_attributes

Description of the application defined supernet attributes.

Description

Attributes are properties assigned to objects such as pins, cells, and nets. They can be read-only or settable. A read-only attribute is informational; you cannot set its value. To modify a settable attribute, use the *set_attribute* command.

To determine the value of an attribute, use the *get_attribute* command. To see a list of all supernet attributes available, use the *list_attributes -application -class supernet* command.

Attributes for supernet

This section describes the supernet attributes.

anchor

A read-only string attribute that specifies the pin or port to which the supernet is anchored.

boundary_terms

A read-only pin, or port collection attribute that specifies all pins and ports connected to the supernet, which are on the boundary, that is the opposite term (if it exists) is not part of the supernet. These terms are either pins on non transparent cells or top-level ports.

bundles

A read-only bundle collection attribute that returns the bundles that contain the supernet. A supernet can be in multiple bundles, a single bundle, or no bundle.

drivers

A read-only pin, or port collection attribute that specifies the boundary terms, which are net drivers. These include bidirectional terms.

full_name

A read-only string attribute that returns the full name of a supernet.

groups

A read-only group collection attribute that returns the groups the supernet is a part of.

is_master

A read-only boolean attribute that specifies as true if this supernet is the primary, false otherwise.

loads

A read-only pin, or port collection attribute that specifies the boundary terms, which are net loads. These include bidirectional terms.

master

A read-only supernet collection attribute that specifies the supernet, which is primary.

name

A read-only string attribute that specifies the name of the supernet.

object_class

A read-only string attribute that returns the name of this object class. Returns 'supernet' for supernet objects.

routing_corridor

A read-only routing_corridor collection attribute that returns a routing corridor the supernet is associated to.

transparent_cells

A read-only cell collection attribute that specifies the transparent cells on this supernet.

See Also

- [get_attribute](#)
- [list_attributes](#)
- [get_supernets](#)
- [remove_supernets](#)

supply_net_attributes

Description of the application defined supply_net attributes.

Description

Attributes are properties assigned to objects such as pins, cells, and nets. They can be read-only or settable. A read-only attribute is informational; you cannot set its value. To modify a settable attribute, use the *set_attribute* command.

To determine the value of an attribute, use the *get_attribute* command. To see a list of all *supply_net* attributes available, use the *list_attributes -application -class supply_net* command.

Attributes for *supply_net*

This section describes the *supply_net* attributes.

full_name

A read-only string attribute that returns the full name of a supply net object.

name

A read-only string attribute that returns the name of a supply net object.

object_class

A read-only string attribute that returns the name of this object class. Returns 'supply_net' for *supply_net* objects.

reference_only

A read-only string attribute that indicates if the supply net is reference only. It can specify if supply net is internal.

resolve_type

A read-only string attribute that returns the resolution for this supply net object.

scope

A read-only collection attribute that returns the scope object for this supply net object.

supply_ports

A read-only collection attribute that returns all supply ports connected to this supply net object.

user_voltage_early

A read-only float attribute that returns the early voltage (if any) that has been directly set on this supply net object by the *set_voltage* command.

user_voltage_late

A read-only float attribute that returns the late voltage (if any) that has been directly set on this supply net object by the *set_voltage* command.

voltage_areas

A read-only collection attribute that returns the voltage area objects for this supply net object.

voltage_early

A read-only float attribute that returns the early voltage of this supply net object, for the current corner. This voltage might have been set directly on the supply net, or have been propagated from the supply net's connected network.

voltage_late

A read-only float attribute that returns the late voltage of this supply net object, for the current corner. This voltage might have been set directly on the supply net, or have been propagated from the supply net's connected network.

voltage_max

A read-only float attribute that returns an alias for the *voltage_late* attribute. Note that the value of the *voltage_max* attribute might be less than the value of the *voltage_min* attribute, since the speed of CMOS circuits typically improves with increased voltage.

voltage_min

A read-only float attribute that specifies an alias for the *voltage_early* attribute. Note that the value of the *voltage_min* attribute might be greater than the value of the *voltage_max* attribute, since the speed of CMOS circuits typically improves with increased voltage.

See Also

- [get_attribute](#)
- [list_attributes](#)
- [report_attributes](#)

supply_port_attributes

Description of the application defined supply_port attributes.

Description

Attributes are properties assigned to objects such as pins, cells, and nets. They can be read-only or settable. A read-only attribute is informational; you cannot set its value. To modify a settable attribute, use the *set_attribute* command.

To determine the value of an attribute, use the *get_attribute* command. To see a list of all *supply_port* attributes available, use the *list_attributes -application -class supply_port* command.

Attributes for *supply_port*

This section describes the *supply_port* attributes.

direction

A read-only string attribute (with allowed values: in inout internal out) that returns the direction of a supply port object.

full_name

A read-only string attribute that returns the full name of a supply port object.

name

A read-only string attribute that returns the name of a supply port object.

object_class

A read-only string attribute that returns the name of this object class. Returns 'supply_port' for *supply_port* objects.

scope

A read-only collection attribute that returns the scope object for this supply port object.

supply_nets

A read-only collection attribute that returns all supply nets connected to this supply port object.

user_voltage_early

A read-only float attribute that returns the early voltage (if any) that has been directly set on this supply port object by the *set_voltage* command.

user_voltage_late

A read-only float attribute that returns the late voltage (if any) that has been directly set on this supply port object by the *set_voltage* command.

voltage_early

A read-only float attribute that returns the early voltage of this supply port object, for the current corner. This voltage might have been set directly on the supply port, or have been propagated from the supply port's connected network.

voltage_late

A read-only float attribute that returns the late voltage of this supply port object, for the current corner. This voltage might have been set directly on the supply port, or have been propagated from the supply port's connected network.

voltage_max

A read-only float attribute that specifies an alias for the *voltage_late* attribute. Note that the value of the *voltage_max* attribute might be less than the value of the *voltage_min* attribute, since the speed of CMOS circuits typically improves with increased voltage.

voltage_min

A read-only float attribute that specifies an alias for the *voltage_early* attribute. Note that the value of the *voltage_min* attribute might be greater than the value of the *voltage_max* attribute, since the speed of CMOS circuits typically improves with increased voltage.

See Also

- [get_attribute](#)
- [list_attributes](#)
- [report_attributes](#)

supply_set_attributes

Description of the application defined *supply_set* attributes.

Description

Attributes are properties assigned to objects such as pins, cells, and nets. They can be read-only or settable. A read-only attribute is informational; you cannot set its value. To modify a settable attribute, use the *set_attribute* command.

To determine the value of an attribute, use the *get_attribute* command. To see a list of all *supply_set* attributes available, use the *list_attributes -application -class supply_set* command.

Attributes for `supply_set`

This section describes the `supply_set` attributes.

full_name

A read-only string attribute that returns the full name of a supply set object.

function

A read-only string attribute that returns the power and ground functions. If supply set functions are not resolved this attribute is empty.

ground_supply_net

A read-only collection attribute that returns the supply net used in resolving the ground function for this supply set object. If not resolved, this attribute is not reported.

name

A read-only string attribute that returns the name of a supply set object.

object_class

A read-only string attribute that returns the name of this object class. Returns 'supply_set' for `supply_set` objects.

power_supply_net

A read-only collection attribute that returns the supply net used in resolving the power function for this supply set object. If not resolved, this attribute is not reported.

scope

A read-only collection attribute that returns the scope object for this supply set object.

See Also

- [get_attribute](#)
- [list_attributes](#)
- [report_attributes](#)

t

t

tech_attributes

Description of the application defined tech attributes.

Description

Attributes are properties assigned to objects such as pins, cells, and nets. They can be read-only or settable. A read-only attribute is informational; you cannot set its value. To modify a settable attribute, use the *set_attribute* command.

To determine the value of an attribute, use the *get_attribute* command. To see a list of all tech attributes available, use the *list_attributes -application -class tech* command.

Attributes for tech

This section describes the tech attributes.

antenna_cut_to_metal

A settable boolean attribute that indicates whether antenna is cut to metal.

antenna_diode_distance

A settable distance attribute that specifies the antenna diode distance.

antenna_max_area

A settable area attribute that returns the antenna maximum area.

antenna_metal_to_cut

A settable boolean attribute that indicates whether antenna is metal to cut.

capacitance_precision

A settable int attribute that returns the capacitance precision.

capacitance_unit

A settable string attribute that returns the capacitance unit.

corner_spacing_mode

A settable int attribute that returns the corner spacing mode.

current_precision

A settable int attribute that returns the current precision.

t

current_unit

A settable string attribute that returns the current unit.

date

A read-only string attribute that returns the date.

dielectric

A settable double attribute that returns the dielectric.

fat_table_min_enclosed_area_mode

A settable int attribute that returns the fat table minimum enclosed area mode.

fat_table_spacing_mode

A settable int attribute that returns the fat table spacing mode.

fat_wire_extension_mode

A settable int attribute that returns the fat wire extension mode.

full_name

A settable string attribute that returns the technology name.

grid_resolution

A settable int attribute that returns the grid resolution. Grid resolution must be greater than zero.

inductance_precision

A settable int attribute that returns the inductance precision.

inductance_unit

A settable string attribute that returns the induction unit.

layers

A read-only layer collection attribute that returns the layers in this technology.

length_precision

A settable int attribute that returns the length precision.

lib

A read-only lib collection attribute that is the library of this technology.

lib_name

A read-only string attribute that returns the library name of this technology.

max_baseline_temperature

A settable double attribute that returns the maximum baseline temperature.

max_stack_level_mode

A settable int attribute that returns the maximum stack level mode.

min_baseline_temperature

A settable double attribute that returns the minimum baseline temperature.

min_edge_mode

A settable int attribute that returns the minimum edge mode.

min_length_mode

A settable int attribute that returns the minimum length mode.

name

A settable string attribute that returns the technology name.

nom_baseline_temperature

A settable double attribute that returns the nominal baseline temperature.

object_class

A read-only string attribute that returns the name of this object class. Returns 'tech' for tech objects.

parallel_length_mode

A settable int attribute that returns the parallel length mode.

power_precision

A settable int attribute that returns the power precision.

power_unit

A settable string attribute that returns the power unit.

purposes

A read-only tech_purpose collection attribute that contains the tech_purposes in this technology.

resistance_precision

A settable int attribute that returns the resistance precision.

resistance_unit

A settable string attribute that returns the resistance unit.

t

time_precision

A settable int attribute that returns the time precision.

time_unit

A settable string attribute that returns the time unit.

unit_length

A settable string attribute that returns the unit length.

voltage_precision

A settable int attribute that returns the voltage precision.

voltage_unit

A settable string attribute that returns the voltage unit.

See Also

- [get_attribute](#)
- [list_attributes](#)

tech_purpose_attributes

Description of the application defined tech_purpose attributes.

Description

Attributes are properties assigned to objects such as pins, cells, and nets. They can be read-only or settable. A read-only attribute is informational; you cannot set its value. To modify a settable attribute, use the *set_attribute* command.

To determine the value of an attribute, use the *get_attribute* command. To see a list of all tech_purpose attributes available, use the *list_attributes -application -class tech_purpose* command.

Attributes for tech_purpose

This section describes the tech_purpose attributes.

full_name

A read-only string attribute that specifies the name of the tech purpose.

name

A read-only string attribute that specifies the name of the tech purpose.

object_class

A read-only string attribute that returns the name of this object class. Returns 'tech_purpose' for tech_purpose objects.

purpose_number

A read-only int attribute that specifies the purpose number of the tech purpose.

See Also

- [get_attribute](#)
- [list_attributes](#)

terminal_attributes

Description of the application defined terminal attributes.

Description

Attributes are properties assigned to objects such as pins, cells, and nets. They can be read-only or settable. A read-only attribute is informational; you cannot set its value. To modify a settable attribute, use the *set_attribute* command.

To determine the value of an attribute, use the *get_attribute* command. To see a list of all terminal attributes available, use the *list_attributes -application -class terminal* command.

Attributes for terminal

This section describes the terminal attributes.

access_direction

A settable access_dir attribute that returns the access direction for the terminal.

bbox

A read-only rect attribute that returns the bounding-box of a design. The format of a rectangle specification is {{llx lly} {urx ury}}, which specifies the lower-left and upper-right corners of the rectangle.

boundary

A settable coord_list attribute that returns the boundary of the terminal's shape. The format of coordinate list specification is {{x1 y1} {x2 y2} ...}, which specifies a list of points defining the boundary of the shape for the terminal.

t

bounding_box

A read-only *bounding_box* collection attribute that returns the bounding-box of a design. The format of the collection specifies lower-left and upper-right corners of the rectangle.

class

A settable string attribute (with allowed values: bump core none) that returns the class of the terminal.

direction

A read-only string attribute (with allowed values: in inout internal out) that returns the direction of the terminal.

direction_constraint

An attribute only settable in Library Manager which controls connection direction by limiting links to upper layers, to lower layers, or leaving the terminal unconstrained.

eeq_class

A settable int attribute that returns the electrical equivalency class of the terminal.

full_name

A read-only string attribute that returns the full name of a terminal.

groups

A read-only group collection attribute that returns the groups in which this terminal is a member.

esd_sources

A read-only string attribute that contains the list of Electro Static Source (ESD) name and worst case voltage value assigned to this terminal.

is_bond_pad

A settable boolean attribute that indicates whether this terminal is a bond pad.

is_connectable

A settable boolean attribute that indicates whether router can use this terminal for routing purpose.

is_def_duplicate

A settable boolean attribute that indicates whether this terminal shape is duplicated in the DEF top level port. This attribute only exists on terminals

t

with shapes which are really owned in a lower level by a pad cell. See `is_owned_by_pad`.

is_destructible

A settable boolean attribute that indicates whether a `probe_pad` terminal is destructible.

is_direct_probe

A settable boolean attribute that indicates whether this terminal is a direct probe pad.

is_fixed

A read-only boolean attribute that returns true if the physical status of the terminal is either fixed or locked.

is_owned_by_pad

A read-only boolean attribute that indicates whether this terminal shape is really owned in a lower level by a pad cell. This value is true if and only if this terminal was automatically generated by the tool on shapeless ports when connected to the pad cell pin.

is_rectangle_only_rule_waived

A settable boolean attribute that returns true if rectangle only rule need to be waived.

layer

A read-only layer collection attribute that represents the layer on which the terminal exists.

matching_type

A read-only `matching_type` collection attribute that represents the `matching_type` for the terminal.

must_join_group

A settable string attribute that returns the `must_join_group` for the terminal.

name

A settable string attribute that returns the name of a terminal.

object_class

A read-only string attribute that returns the name of this object class. Returns 'terminal' for terminal objects.

t

parent_block

A read-only block collection attribute that returns the owner block object for this terminal.

parent_block_cell

A read-only cell collection attribute that returns the parent block cell object for this terminal.

parent_cell

A read-only cell collection attribute that returns the owner cell object for this terminal.

physical_status

A settable string attribute (with allowed values: application_fixed fixed locked unrestricted) that returns the physical modification status of the terminal. If status is unrestricted, the terminal may be freely modified. If the status is application_fixed, the terminal is fixed by the application for automatic changes (the application may change this status). If status is fixed, the terminal is fixed by the user for automatic changes. If status is locked, the terminal is fixed for automatic and manual changes.

port

A settable lib_pin, or port collection attribute that represents the port for the terminal. This attribute is settable and removable only on bond pad terminals. When setting the port of a bond pad terminal, the terminal is automatically renamed to match the port name. If a terminal with that name already exists, the name is uniquified by appending an '_' plus an integer. When removing a bond pad terminal from its port, the terminal is automatically renamed to "BOND_PAD_PIN_" plus an integer like all other bond pad terminals which do not belong to any port.

route_connection

A settable string attribute (with allowed values: connect skip) that determines whether the secondary power router should connect the terminal or skip the terminal.

shape

A settable shape collection attribute that represents the shape for the terminal.

top_block

A read-only block collection attribute that represents the block for the terminal.

See Also

- [get_terminals](#)
- [get_attribute](#)
- [list_attributes](#)
- [set_attribute](#)

timing_arc_attributes

Description of the application defined timing_arc attributes.

Description

Attributes are properties assigned to objects such as pins, cells, and nets. They can be read-only or settable. A read-only attribute is informational; you cannot set its value. To modify a settable attribute, use the *set_attribute* command.

To determine the value of an attribute, use the *get_attribute* command. To see a list of all timing_arc attributes available, use the *list_attributes -application -class timing_arc* command.

Attributes for timing_arc

This section describes the timing_arc attributes.

from_pin

A read-only collection attribute that returns the from pin for a timing_arc. If this timing_arc is for a timing check such as setup or hold, the from pin is the clock pin of the check.

invalid_reason

A read-only string attribute that returns the reason why the timing arc is not an active arc. See the description of is_disable and is_invalid attributes. The possible invalid_reason strings are: .nf set_disable_timing auto_loop_breaking inherited_loop_breaking user_case_0 user_case_1 netlist_tie_0 netlist_tie_1 case_disabled case_0 case_1 conditional_arc_disabled default_arc_disabled mode_values_disabled pin_disabled lib_arc_disabled preset_clear_disabled .fi

is_cellarc

A read-only boolean attribute that returns True if this timing arc is for a cell. If this attribute is true both the from_pin and to_pin will be on the same cell.

t

is_db_inherited_disabled

A read-only boolean attribute that returns True if the `timing_arc` is disabled because of a property from another tool.

is_disabled

A read-only boolean attribute that indicates whether this timing arc is disabled. This attribute has the same definition as it does in PrimeTime. It is true only if the arc has been directly disabled by `set_disable_timing`, case propagation, loop breaking, and arc modes or conditions disabled. This attribute is true for a subset of the timing arcs that `is_invalid` attribute is true for. It does not include timing_arcs that are disabled indirectly. Examples of timing_arcs that are disabled indirectly are timing arcs where the `lib_timing_arc` are disable or those with either the from or to pin disabled.

is_invalid

A read-only boolean attribute that indicates whether this timing arc is not active. This attribute is true for a superset of the timing arcs that `is_disable` attribute is true for. See the description of the attribute "invalid_reason" for the possible reasons a timing_arc might be invalid.

is_user_disabled

A read-only boolean attribute that indicates whether this timing arc has been disabled by the user.

mode

A read-only string attribute that returns the name of the mode this timing arc is active for. The attribute will is only defined for moded arcs.

object_class

A read-only string attribute that returns the name of this object class. Returns 'timing_arc' for timing_arc objects.

sdf_cond

A read-only string attribute that returns the SDF condition of the timing arc.

sense

A read-only string attribute that returns the sense of a timing_arc. Possible values are:

<code>rising_edge</code>	<code>rising_to_rise</code>
<code>falling_to_rise</code>	<code>falling_edge</code>
<code>rising_to_fall</code>	<code>falling_to_fall</code>
<code>rise_to_rise</code>	<code>fall_to_rise</code>
<code>rise_to_fall</code>	<code>fall_to_fall</code>
<code>positive_unate</code>	<code>negative_unate</code>

t

```

non_unate
clear_low
preset_high
preset_both
disable_low
disable_high_rise
disable_both_rise
disable_low_fall
enable_high
enable_both
enable_low_rise
enable_high_fall
enable_both_fall
retain_rise_to_rise
retain_falling_edge
retain_fall_to_fall
retain_negative_unate
max_clock_tree_positive_unate
max_clock_tree_fall_to_fall
max_clock_tree_rise_to_fall
max_clock_tree_non_unate
min_clock_tree_rise_to_rise
min_clock_tree_negative_unate
min_clock_tree_fall_to_rise
min_clock_tree_non_unate
nonseq_setup_clk_rise
nonseq_setup_fall_clk_rise
nonseq_setup_rise_clk_fall
nonseq_hold_clk_rise
nonseq_hold_fall_clk_rise
nonseq_hold_rise_clk_fall
clock_gating_setup_clk_rise
clock_gating_setup_fall_clk_rise
clock_gating_setup_rise_clk_fall
clock_gating_setup_fall_clk_fall
clock_gating_hold_clk_rise
clear_high
clear_both
preset_low
disable_high
disable_both
disable_low_rise
disable_high_fall
disable_both_fall
enable_low
enable_high_rise
enable_both_rise
enable_low_fall
retain_rising_edge
retain_fall_to_rise
retain_rise_to_fall
retain_positive_unate
retain
max_clock_tree_rise_to_rise
max_clock_tree_negative_unate
max_clock_tree_fall_to_rise
min_clock_tree_positive_unate
min_clock_tree_fall_to_fall
min_clock_tree_rise_to_fall
min_clock_tree_non_unate
nonseq_setup_rise_clk_rise
nonseq_setup_clk_fall
nonseq_setup_fall_clk_fall
nonseq_hold_rise_clk_rise
nonseq_hold_clk_fall
nonseq_hold_fall_clk_fall
clock_gating_setup_rise_clk_rise
clock_gating_setup_clk_fall
clock_gating_hold_rise_clk_rise

```

to_pin

A read-only collection attribute that returns the to pin for a timing_arc. If this timing_arc is for a timing check such as setup or hold, the from pin is the data pin of the check.

user_clock_sense

A read-only string attribute that returns the sense set by the command **set_clock_sense**. This attribute is only define on those pins where the **set_clock_sense** command was used directly. The possible values are: "positive", "negative", "rise_triggered_low_pulse", "rise_triggered_high_pulse", "fall_triggered_low_pulse", "fall_triggered_high_pulse".

t

when

A read-only string attribute that returns the when value for the timing_arc.

See Also

- [get_attribute](#)
- [list_attributes](#)

timing_exception_attributes

Lists the predefined attributes for timing_exception objects.

Description

Attributes are properties assigned to objects such as pins, cells and nets. Definitions for timing_exception attributes are provided in the subsections that follow.

Attributes are read-only unless marked as settable. Read-only attributes are informational, and you cannot set its value. A settable attribute can be modified with the *set_attribute* command.

To determine the value of an attribute, use the *get_attribute* command. To see a list of all attributes available for a class of objects use the *list_attributes -application* command.

command_text

A string attribute. The value is the equivalent command(s) that can re-create the object.

exception_type

A string attribute indicating the type of exception. Valid values are "false_path", "multicycle_path", and "max_min_delay".

mode

A collection attribute returning the mode that contains the exception.

file_line_info

A string attribute. If it exists, the value contains one or more Tcl or SDC source file name and line number pairs of the commands which were used to create this object.

full_name

A string attribute returning a unique name for the exception.

t

hold_fall_end

A Boolean attribute; true if the exception is multicycle and the hold fall multiplier is relative to the end clock.

hold_fall_multiplier

An int attribute for multicycle_path exceptions, specifying the multiplier value for hold fall.

hold_rise_end

A Boolean attribute; true if the exception is multicycle and the hold rise multiplier is relative to the end clock.

hold_rise_multiplier

An int attribute for multicycle_path exceptions, specifying the multiplier value for hold rise.

is_hold_fall_false

A Boolean attribute; true if the exception is false_path, and the hold fall timing is set to be false.

is_hold_rise_false

A Boolean attribute; true if the exception is false_path, and the hold rise timing is set to be false.

is_setup_fall_false

A Boolean attribute; true if the exception is false_path, and the setup fall timing is set to be false.

is_setup_rise_false

A Boolean attribute; true if the exception is false_path, and the setup rise timing is set to be false.

max_fall_delay

A float attribute for max_min_delay exceptions. Specifies the delay value for max_fall.

max_rise_delay

A float attribute for max_min_delay exceptions. Specifies the delay value for max_rise.

min_fall_delay

A float attribute for max_min_delay exceptions. Specifies the delay value for min_fall.

t

min_rise_delay

A float attribute for max_min_delay exceptions. Specifies the delay value for min_rise.

object_class

A string attribute, with the value "timing_exception".

setup_fall_multiplier

An int attribute for multicycle_path exceptions, specifying the multiplier value for setup fall.

setup_fall_start

A Boolean attribute; true if the exception is multicycle and the setup fall multiplier is relative to the start clock.

setup_rise_multiplier

An int attribute for multicycle_path exceptions, specifying the multiplier value for setup rise.

setup_rise_start

A Boolean attribute; true if the exception is multicycle and the setup rise multiplier is relative to the start clock.

through_group_count

An int attribute specifying the number of -through, -rise_through, and/or -fall_through options that were specified with the exception command. For example, if the exception was specified as "-through {a1 a2 a3} -rise_through {b1 b2 b3}", the *through_group_count* attribute value will be 2.

See Also

- [get_exceptions](#)
- [get_attribute](#)
- [list_attributes](#)
- [report_exceptions](#)
- [set_false_path](#)
- [set_max_delay](#)
- [set_min_delay](#)
- [set_multicycle_path](#)

t

timing_path_attributes

Description of the application defined timing_path attributes.

Description

Attributes are properties assigned to objects such as pins, cells, and nets. They can be read-only or settable. A read-only attribute is informational; you cannot set its value. To modify a settable attribute, use the *set_attribute* command.

To determine the value of an attribute, use the *get_attribute* command. To see a list of all timing_path attributes available, use the *list_attributes -application -class timing_path* command.

All timing_path attributes are read-only.

All time-valued attributes will be based on the user output units in effect when the attribute value is gotten, not when the timing path was originally created.

The object-valued attributes (such as startpoint_clock, mode, corner, etc.) are not guaranteed to have valid values if the design has been modified since the timing_path was created. However, the corresponding "name" attributes (for example, startpoint_clock_name) will always be valid, even if the object itself no longer exists.

Attributes for timing_path

This section describes the timing_path attributes.

all_blocks

A read-only string attribute that returns an ordered Tcl list of the "coarse grain" element that a path travels through, including elements such as the full names of hard or soft macros "_Top_" (represents the top block) and "_port_" (represents a design port).

arrival

A read-only time attribute that returns the timing_path's endpoint arrival time.

avg_cell_delay

A read-only float attribute that returns the average timing path cell delay.

capture_clock_paths

A read-only collection attribute that returns the timing_path's capture clock path, if it exists.

check_value

A read-only time attribute that returns the timing_path's check value: the setup or hold time for a latch, or the output delay for a port.

t

clock_cycles

A read-only int attribute that returns the clock cycles.

clock_uncertainty

A read-only time attribute that returns the timing_path's clock uncertainty time.

common_path_pessimism

A read-only time attribute that returns the timing_path's CRPR common_path pessimism time.

corner

A read-only collection attribute that returns the timing_path's corner.

corner_name

A read-only string attribute that returns the name of timing_path's corner.

crpr_common_point

A read-only collection attribute that returns the timing_path's CRPR common point.

delay_type

A read-only string attribute that returns whether the path is max or min based on the path type.

design_name

A read-only string attribute that returns the name of the timing_path's design.

driving_cell_adjustment

A read-only time attribute that returns the timing_path's driving_cell adjustment time, for paths starting at input ports.

end_block

A read-only string attribute that returns the last (end) element that appears in the Tcl list value in the blocks.

endpoint

A read-only collection attribute that returns the timing_path's endpoint.

endpoint_clk_skew

A read-only float attribute that returns the endpoint clock skew.

endpoint_clock

A read-only collection attribute that returns the timing_path's endpoint clock.

t

endpoint_clock_arrival_close_edge_type

A read-only string attribute that returns the timing_path's capture clock edge, as measured at the endpoint for close edge. The value is either "rise" or "fall".

endpoint_clock_arrival_open_edge_type

A read-only string attribute that returns the timing_path's capture clock edge, as measured at the endpoint for open edge. The value is either "rise" or "fall".

endpoint_clock_close_edge_arrival

A read-only time attribute that returns the arrival time of the close edge of the capture clock, at the endpoint.

endpoint_clock_close_edge_type

A read-only string attribute that returns the timing_path's capture clock edge, as measured at the clock source. The value is either "rise" or "fall".

endpoint_clock_close_edge_value

A read-only time attribute that returns the time of the close edge of the capture clock, measured at the clock source.

endpoint_clock_is_generated

A read-only boolean attribute that returns true if the endpoint clock is generated.

endpoint_clock_is_inverted

A read-only boolean attribute that returns true if the endpoint clock is inverted.

endpoint_clock_is_propagated

A read-only boolean attribute that returns true if the endpoint clock is a propagated clock, false if it is an ideal clock. You can set a clock as propagated using the set_propagated_clock command.

endpoint_clock_latency

A read-only time attribute that returns the capture clock arrival of the timing path, excluding the startpoint clock edge time (endpoint_clock_close_edge_value attribute).

endpoint_clock_name

A read-only string attribute that returns the name of the timing_path's endpoint clock.

t

endpoint_clock_open_edge_arrival

A read-only time attribute that returns the arrival time of the open edge of the capture clock, at the endpoint latch. If the endpoint is edge-triggered, the open and close edges are the same.

endpoint_clock_open_edge_type

A read-only string attribute that returns the type of clock edge (rise or fall) that opens the endpoint latch. Measured at clock source. The value will be either "rise" or "fall". If the endpoint is edge-triggered, the open and close edges are the same.

endpoint_clock_open_edge_value

A read-only time attribute that returns the time of the open edge of the capture clock, measured at the clock source.

endpoint_clock_period

A read-only time attribute that returns the period of the timing_path's endpoint clock.

endpoint_name

A read-only string attribute that returns the name of the timing_path's endpoint.

endpoint_recovery_time

A read-only float attribute that returns the endpoint recovery time.

endpoint_removal_time

A read-only float attribute that returns the endpoint removal time.

full_name

A read-only string attribute that returns the timing_path's name, which is synthesized from some of its other attributes. The name is not guaranteed to be unique.

input_delay

A read-only time attribute that returns the timing_path's startpoint input delay, if the startpoint has a set_input_delay constraint.

is_crossclock

A read-only boolean attribute that indicates if the timing path crosses clocks.

is_infeasible

A read-only boolean attribute that indicates if timing paths are infeasible.

t

is_multicycle

A read-only boolean attribute that indicates if a timing path is multicycle.

is_recovered

A read-only boolean attribute that returns the path recovered from missed borrow window.

launch_clock_paths

A read-only collection attribute that returns the timing_path's launch clock path, if it exists.

lib_name

A read-only string attribute that returns the name of the library containing the timing_path's design.

logic_level_threshold

A read-only float attribute that returns the logic level threshold.

logic_levels

A read-only int attribute that returns the number of logic levels.

max_cell_delay

A read-only float attribute that returns the maximum timing path cell delay.

max_cell_delay_point

A read-only collection attribute that returns the maximum timing path cell delay timing point.

max_fanout

A read-only int attribute that returns the maximum timing path fanout.

max_net_delay

A read-only float attribute that returns the maximum timing path net delay.

max_net_delay_point

A read-only collection attribute that returns the maximum timing path net delay point.

mode

A read-only collection attribute that returns the timing_path's mode.

mode_name

A read-only string attribute that returns the name of timing_path's mode.

t

name

A read-only string attribute that returns the `timing_path`'s name, which is synthesized from some of its other attributes. The name is not guaranteed to be unique.

network_latency

A read-only time attribute that returns the `timing_path`'s startpoint network latency.

num_logic_gates

A read-only int attribute that returns the number of logic gates in the path.

object_class

A read-only string attribute that returns the name of this object class. Returns 'timing_path' for `timing_path` objects.

path_group

A read-only collection attribute that returns the `timing_path`'s pathgroup.

path_group_name

A read-only string attribute that returns the name of the `timing_path`'s pathgroup.

path_type

A read-only string attribute that returns the `timing_path`'s analysis type, either "min" or "max".

percent_cell_delay

A read-only float attribute that returns the percentage of timing path cell delay.

percent_net_delay

A read-only float attribute that returns the percentage of timing path net delay.

points

A read-only collection attribute that returns the `timing_path`'s points.

propagation_type

A read-only string attribute that returns a value equivalent to the `path_type` attribute, with a value of 'min' or 'max'.

required

A read-only time attribute that returns the `timing_path`'s required time. The required attribute of an unconstrained path will have a value of "INFINITY".

t

scenario

A read-only collection attribute that returns the timing_path's scenario.

scenario_name

A read-only string attribute that returns the name of the timing_path's scenario.

slack

A read-only time attribute that returns the timing_path's slack. The slack attribute of an unconstrained path has a value of "INFINITY".

source_latency

A read-only time attribute that returns the timing_path's startpoint source latency.

start_block

A read-only string attribute that returns the first (start) element name that appears in the Tcl list value of the blocks attribute.

start_end_blocks

A read-only string attribute that returns only the first (start) and the last (end) element names from the list value of the blocks attribute (in Tcl list format).

start_end_blocks_sorted

A read-only string attribute that returns reordered element names based on "Synopsys name sorting rules", except for the first (start) and the last (end) element names.

startpoint

A read-only collection attribute that returns the timing_path's startpoint.

startpoint_clock

A read-only collection attribute that returns the timing_path's launch clock.

startpoint_clock_arrival_open_edge_type

A read-only string attribute that returns the timing_path's launch clock edge, as measured at the startpoint. The value will be either "rise" or "fall".

startpoint_clock_is_generated

A read-only boolean attribute that returns true if the startpoint clock is generated.

startpoint_clock_is_inverted

A read-only boolean attribute that returns true if the startpoint clock is inverted.

t

startpoint_clock_is_propagated

A read-only boolean attribute that returns true if the startpoint clock is a propagated clock, false if it is an ideal clock. You can set a clock as propagated using the `set_propagated_clock` command.

startpoint_clock_latency

A read-only time attribute that returns the launch clock arrival of the timing path, excluding the startpoint clock edge time (`startpoint_clock_open_edge_value` attribute).

startpoint_clock_name

A read-only string attribute that returns the name of the timing_path's launch clock.

startpoint_clock_open_edge_arrival

A read-only time attribute that returns the arrival time of the open edge of the launch clock, at the startpoint.

startpoint_clock_open_edge_type

A read-only string attribute that returns the timing_path's launch clock edge, as measured at the clock source. The value will be either "rise" or "fall".

startpoint_clock_open_edge_value

A read-only time attribute that returns the time of the open edge of the launch clock, measured at the clock source.

startpoint_clock_period

A read-only time attribute that returns the timing_path's launch clock period.

startpoint_name

A read-only string attribute that returns the name of the timing_path's startpoint.

through_blocks

A read-only string attribute that returns a Tcl list value similar to 'blocks', except that the first (start) and the last (end) element names are omitted.

total_cell_delay

A read-only float attribute that returns the total timing path cell delay.

total_delay

A read-only float attribute that returns the total timing path delay.

t

total_fanout

A read-only int attribute that returns the total timing path fanout.

total_net_delay

A read-only float attribute that returns the total timing path net delay.

variation_arrival

A read-only collection attribute that returns the arrival time variation of the path.

variation_common_path_pessimism

A read-only collection attribute that returns the variation of the clock reconvergence common path pessimism.

variation_endpoint_clock_latency

A read-only collection attribute that returns the capture-clock arrival time variation.

variation_endpoint_hold_time_value

A read-only collection attribute that returns the variation of the register hold time at the timing endpoint.

variation_endpoint_recovery_time_value

A read-only collection attribute that returns the variation of the recovery time at the timing endpoint.

variation_endpoint_removal_time_value

A read-only collection attribute that returns the variation of the removal time at the timing endpoint.

variation_endpoint_setup_time_value

A read-only collection attribute that returns the variation of the register setup time at the timing endpoint.

variation_required

A read-only collection attribute that returns the required time variation of the path.

variation_slack

A read-only collection attribute that returns the slack variation of the path.

variation_startpoint_clock_latency

A read-only collection attribute that returns the launch clock arrival time variation.

See Also

- [get_attribute](#)
- [list_attributes](#)
- [get_timing_paths](#)
- [report_timing](#)
- [set_user_units](#)
- [report_user_units](#)

timing_point_attributes

Description of the application defined timing_point attributes.

Description

Attributes are properties assigned to objects such as pins, cells, and nets. They can be read-only or settable. A read-only attribute is informational; you cannot set its value. To modify a settable attribute, use the *set_attribute* command.

To determine the value of an attribute, use the *get_attribute* command. To see a list of all timing_point attributes available, use the *list_attributes -application -class timing_point* command.

All timing_point attributes are read-only.

All time-valued or capacitance-valued attributes will be based on the user output units in effect when the attribute value is gotten, not when the timing_point was originally created.

The object-valued attributes (such as object) are not guaranteed to have valid values if the design has been modified since the timing_point was created. However, the corresponding "name" attributes (for example, object_name) will always be valid, even if the object itself no longer exists.

Attributes for timing_point

This section describes the timing_point attributes.

arrival

A read-only time attribute that returns the timing_point's arrival time.

capacitance

A read-only capacitance attribute that returns the timing_point's net capacitance.

cell_delay

A read-only float attribute that returns the timing point cell delay.

delay

A read-only time attribute that returns the timing_point's delay, relative to the previous point in the path.

delta_delay

A read-only time attribute that returns the timing_point's SI delta-delay time. This will not be valid unless SI analysis is turned on.

full_name

A read-only string attribute that returns the timing_point's name. This will be synthesized based on the object name.

name

A read-only string attribute that returns the timing_point's name. This will be synthesized based on the object name.

net_delay

A read-only float attribute that returns the timing point net delay.

num_fanout

A read-only int attribute that returns the timing point fanout number.

object

A read-only collection attribute that returns the timing_point's object: either a pin or a port.

object_class

A read-only string attribute that returns the name of this object class. Returns 'timing_point' for timing_point objects.

object_name

A read-only string attribute that returns the name of the timing_point's object.

rise_fall

A read-only string attribute that returns the timing_point's r/f value: either 'rise' or 'fall'. .

transition

A read-only time attribute that returns the timing_point's transition time.

variation_arrival

A read-only collection attribute that returns the arrival time variation of the timing point.

variation_increment

A read-only collection attribute that returns the incremental variation of the timing point.

variation_slack

A read-only collection attribute that returns the slack time variation of the timing point.

variation_transition

A read-only collection attribute that returns the transition time variation of the timing point.

See Also

- [get_attribute](#)
- [list_attributes](#)
- [get_timing_paths](#)
- [report_timing](#)
- [set_user_units](#)
- [report_user_units](#)

topological_constraint_attributes

Description of the predefined topological pin feedthrough constraint attributes.

Description

Attributes are properties assigned to objects such as pins, cells and nets. Definitions for the topological constraint attributes are provided in the subsections that follow.

Attributes are read-only unless marked as settable. Read-only attributes are informational, and you cannot set its value. A settable attribute can be modified with the *set_attribute* command.

To determine the value of an attribute, use the *get_attribute* command. To see a list of all attributes available for the class of objects use the *list_attributes -application* command.

topological_constraint Attributes

full_name

A string attribute for the full name of a topological pin feedthrough constraint.

name

A string attribute for the name of a topological pin feedthrough constraint.

object_class

owner

start_object

end_object

start_sides

end_sides

start_offset

end_offset

start_offset_range

end_offset_range

start_layers

end_layers

See Also

- [create_topological_constraint](#)
- [get_topological_constraints](#)
- [remove_topological_constraints](#)
- [report_topological_constraints](#)

topology_edge_attributes

Description of the application defined topology_edge attributes.

t

Description

Attributes are properties assigned to objects such as pins, cells, and nets. They can be read-only or settable. A read-only attribute is informational; you cannot set its value. To modify a settable attribute, use the *set_attribute* command.

To determine the value of an attribute, use the *get_attribute* command. To see a list of all topology_edge attributes available, use the *list_attributes -application -class topology_edge* command.

Attributes for topology_edge

This section describes the topology_edge attributes.

allow_feedthrough

A settable string attribute (with allowed values: all none select_off select_on) that returns a string specifying the allow_feedthrough mode of the topology_edge. "none" indicates that no feedthrough is allowed for this edge. "select_on" indicates that feedthrough is only allowed through the objects specified in the allow_feedthrough_select attribute. "select_off" indicates that feedthrough is allowed through all objects except those specified in the allow_feedthrough_select attribute. "all" indicates that feedthrough is allowed through all objects.

allow_feedthrough_select

A settable collection attribute that represents the objects for which the allow_feedthrough attribute setting is applied.

allow_feedthrough_type

A settable string attribute (with allowed values: any mixed pure) that returns a string specifying the allow_feedthrough type of the topology_edge.

allow_flyover

A settable string attribute (with allowed values: all none select_off select_on) that returns a string specifying the allow_flyover mode of the topology_edge. "none" indicates that no flyover is allowed for this edge. "select_on" indicates that flyover is only allowed over the objects specified in the allow_flyover_select attribute. "select_off" indicates that flyover is allowed over all objects except those specified in the allow_flyover_select attribute. "all" indicates that flyover is allowed over all objects.

allow_flyover_select

A settable collection attribute that represents the objects for which the allow_flyover attribute setting is applied.

t

constraint_groups

A settable `constraint_group` collection attribute that returns a collection representing the `constraint_groups` of which the `topology_edge` is a member.

end_node

A settable `topology_node` collection attribute that returns a collection representing the `topology_node` to which the `topology_edge`'s ending endpoint is connected. An empty collection indicates the endpoint is disconnected.

full_name

A read-only string attribute that returns the full name of the `topology_edge`.

group_master

A settable `topology_edge` collection attribute that returns a collection representing the `topology_edge` which is the group master of this `topology_edge`. A `topology_edge` group is a group of `topology_edges` sharing the same (or reversed direction) endpoints and coordinate points, as well as selected other attributes. The "master" edge of the group is the edge that governs the attributes of the other edges in the group (the "members").

group_members

A read-only `topology_edge` collection attribute that returns a collection representing the `topology_edges` that are members of the `topology_edge` group of which this edge is the master. A `topology_edge` group is a group of `topology_edges` sharing the same (or reversed direction) endpoints and coordinate points, as well as selected other attributes. The "master" edge of the group is the edge that governs the attributes of the other edges in the group (the "members").

is_group_master

A read-only boolean attribute that indicates whether this edge is the master of a `topology_edge` group. A `topology_edge` group is a group of `topology_edges` sharing the same (or reversed direction) endpoints and coordinate points, as well as selected other attributes. The "master" edge of the group is the edge that governs the attributes of the other edges in the group (the "members").

length

A read-only distance attribute that returns the total length of the `topology edge`.

name

A settable string attribute that returns the name of the `topology_edge`.

t

number_of_signals

A settable int attribute that returns an unsigned integer specifying the number of signals annotated on the topology_edge.

object_class

A read-only string attribute that returns the name of this object class. Returns 'topology_edge' for topology_edge objects.

objects

A settable net, supernet, or bundle collection attribute that represents the nets, supernets, and bundles associated with the topology_edge.

parent_block

A read-only block collection attribute that returns the owner block object for this topology_edge.

physical_status

A settable string attribute (with allowed values: locked minor_change unrestricted) that returns a string specifying the physical modification status of the topology_edge.

plan

A read-only topology_plan collection attribute that returns a collection representing the topology_plan that owns the topology_edge.

points

A read-only coord_list attribute that represents the geometric vertices of the topology_edge. The points are defined by the edge's endpoint origins as well as its *shape_layers* attribute.

repeaters

A read-only topology_repeater collection attribute that returns the set of topology_repeaters owned by the topology_edge.

shadow_status

A settable string attribute (with allowed values: copied_down copied_up normal pulled_up pushed_down virtual_flat) that returns the shadow_status of the topology_edge.

shape_layers

A settable shape_layer_list attribute that contains a list of triple elements of the form {{layer [H|V]length_or_percentage[/angle] width} {layer [H|V]length_or_percentage[/angle] width}. ..}. See the *-shape_layers* argument

t

section of the *create_topology_edge(2)* man page for a fuller description and usage examples of the *shape_layers* format.

start_node

A settable *topology_node* collection attribute that returns the *topology_node* to which the *topology_edge*'s starting endpoint is connected. An empty collection indicates the endpoint is disconnected.

See Also

- [get_topology_edges](#)
- [get_attribute](#)
- [list_attributes](#)
- [set_attribute](#)

topology_group_attributes

Description of the application defined *topology_group* attributes.

Description

Attributes are properties assigned to objects such as pins, cells, and nets. They can be read-only or settable. A read-only attribute is informational; you cannot set its value. To modify a settable attribute, use the *set_attribute* command.

To determine the value of an attribute, use the *get_attribute* command. To see a list of all *topology_group* attributes available, use the *list_attributes -application -class topology_group* command.

Attributes for topology_group

This section describes the *topology_group* attributes.

comment

A settable string attribute that can be used to document the *topology_group*.

full_name

A read-only string attribute that returns the full name of the *topology_group*.

is_default

A read-only boolean attribute that indicates whether the *topology_group* is the default *topology_group*.

t

name

A settable string attribute that returns the name of the topology_group.

number_of_plans

A read-only unsigned attribute that returns the number of topology_plans in the topology_group's "plans" collection attribute.

object_class

A read-only string attribute that returns the name of this object class. Returns 'topology_group' for topology_group objects.

parent_block

A read-only block collection attribute that returns the owner block object for this topology_group.

plans

A read-only topology_plan collection attribute that returns a collection representing the topology_plans of the topology_group.

See Also

- [get_topology_groups](#)
- [get_attribute](#)
- [list_attributes](#)
- [set_attribute](#)

topology_node_attributes

Description of the application defined topology_node attributes.

Description

Attributes are properties assigned to objects such as pins, cells, and nets. They can be read-only or settable. A read-only attribute is informational; you cannot set its value. To modify a settable attribute, use the *set_attribute* command.

To determine the value of an attribute, use the *get_attribute* command. To see a list of all topology_node attributes available, use the *list_attributes -application -class topology_node* command.

Attributes for topology_node

This section describes the topology_node attributes.

t

alignment

A settable string attribute (with allowed values: left right) that returns a string specifying the topology_node's constraint alignment.

bbox_percentage

A settable float_single_or_pair attribute that returns a list of one or two float values, specifying the bbox_percentage of the topology_node. For linear regions, specify a single value as the percentage. For 2D regions, specify two float values, where the first element of the list is the percentage to apply in the x-direction, and the second element is the percentage to apply in the y-direction.

constrained_objects

A settable collection attribute that returns a collection representing the constrained objects associated with the topology_node. Valid object types are ports and pins.

constraint_source

A settable string attribute (with allowed values: application user) that returns a string specifying the constraint_source of the topology_plan.

constraint_type

A settable string attribute (with allowed values: bbox edge origin side tap) that returns a string specifying the constraint_type of the topology_node.

corner_type

A settable string attribute (with allowed values: auto cross default river) that returns a string specifying the constraint corner type of the topology_node.

edge

A settable int_list attribute that returns a list of integers specifying the constraint edge numbers of the topology_node.

edges

A read-only topology_edge collection attribute that represents the set of topology_edges that have the topology_node as an endpoint.

end

A settable distance_or_percent attribute that returns the constraint end value of the topology_node. If the attribute value has a percentage symbol on its right hand side, then it represents a percentage value, otherwise it is a distance value.

t

full_name

A read-only string attribute that returns the full name of the topology_node.

halo

A settable distance attribute that returns the halo of the topology_node.

has_edges

A read-only boolean attribute that returns true if the topology_node's edges attribute is non-empty.

intrinsic_delay

A settable list_of_distance_or_percent attribute that returns one or two elements, depending on whether the topology node is an intermediate topology node, or a start or end topology node. If it is a start or end topology node, this option expects two values - one for the buffer intrinsic delay and the other for the register intrinsic delay. Each list element might be expressed either as a distance or a percentage value. If the list element has a percentage symbol on its right hand side, then it represents a percentage value, otherwise it is a distance value. For example, *-intrinsic_delay { 100 200 }*, where 100 um is the buffer intrinsic delay, 200 um is the register intrinsic delay. Or, *-intrinsic_delay { 25% 50% }*, where the values are: 25 percentage of the buffer intrinsic delay budget, 50 percentage of the register intrinsic delay budget. More examples: - Start and end topology nodes always have 2 values: { 100 50% } { 25% 50% } - Intermediate topology node may contains 1 or 2 values: { 100 200 } { 0 200 } { 0 0 } { 50% }

is_fixed

A read-only boolean attribute that returns whether the topology_node's physical_status attribute is locked.

is_reference

A read-only boolean attribute that returns whether the topology_node is in reference mode. When in reference mode, many of the topology_node's attribute values reflect those of the ref_node, and setting the topology_node's settable attributes actually set the attributes on the ref_node.

is_valid

A read-only boolean attribute that returns whether the topology_node's origin attribute has been correctly computed based on the node's other constraint-related attributes. If false, then the value of the *origin* attribute remains the last correctly computed value.

t

name

A settable string attribute that returns the name of the topology_node.

num_driving_edges

A read-only int attribute that returns the number of topology_edges in the topology_node's edges attribute's collection for which the topology_node is the end_node.

num_edges

A read-only int attribute that returns the number of topology_edges in the topology_node's edges attribute's collection.

object_class

A read-only string attribute that returns the name of this object class. Returns 'topology_node' for topology_node objects.

objects

A settable collection attribute that represents the objects associated with the topology_node.

offset

A settable distance attribute that returns the topology_node's constraint offset value.

origin

A settable coord attribute that represents the location of the topology_node.

parent_block

A read-only block collection attribute that returns the owner block object for this topology_node.

physical_status

A settable string attribute (with allowed values: locked minor_change unrestricted) that returns the physical modification status of the topology_node.

plan

A read-only topology_plan collection attribute that returns a collection representing the topology_plan that owns the topology_node.

range

A settable distance_or_percent attribute that returns an attribute that has either a distance or a percentage value, which specifies the constraint range value of the topology_node. If the attribute value has a percentage symbol on its right

t

hand side, then it represents a percentage value, otherwise it is a distance value.

ref_node

A settable topology_node collection attribute that returns a collection representing the ref_node of the topology_node. This will usually be populated with a topology_node (the "ref node") when the is_reference attribute is "true", unless the ref node has changed or disappeared before the design was loaded into memory. If is_reference is "true" but the ref_node attribute is empty, then you can refer to the ref_node_name and ref_node_plan_name attribute values to investigate the problem.

ref_node_name

A read-only string attribute that returns a string specifying the name of the ref_node of the topology_node. This attribute has a value when the is_reference attribute is "true".

ref_node_plan_name

A read-only string attribute that returns a string specifying the name of the topology_plan that owns the ref_node of the topology_node. This attribute has a value when the is_reference attribute is "true".

shadow_status

A settable string attribute (with allowed values: copied_down copied_up normal pulled_up pushed_down virtual_flat) that returns a string specifying the shadow_status of the topology_node.

side

A settable side_list attribute that returns a string list specifying the topology_node's constraint sides. Valid values are a single instance or list of: N, S, E, and W.

start

A settable distance_or_percent attribute that returns an attribute that has either a distance or a percentage value, which specifies the constraint start value of the topology_node. If the attribute value has a percentage symbol on its right hand side, then it represents a percentage value, otherwise it is a distance value.

See Also

- [get_topology_nodes](#)
- [get_attribute](#)

- [list_attributes](#)
- [set_attribute](#)

topology_plan_attributes

Description of the application defined topology_plan attributes.

Description

Attributes are properties assigned to objects such as pins, cells, and nets. They can be read-only or settable. A read-only attribute is informational; you cannot set its value. To modify a settable attribute, use the *set_attribute* command.

To determine the value of an attribute, use the *get_attribute* command. To see a list of all topology_plan attributes available, use the *list_attributes -application -class topology_plan* command.

Attributes for topology_plan

This section describes the topology_plan attributes.

allow_feedthrough

A settable string attribute (with allowed values: all none select_off select_on) that returns a string specifying the allow_feedthrough mode of the topology_plan. "none" indicates that no feedthrough is allowed for edges owned by this plan. "select_on" indicates that feedthrough is only allowed through the objects specified in the allow_feedthrough_select attribute. "select_off" indicates that feedthrough is allowed through all objects except those specified in the allow_feedthrough_select attribute. "all" indicates that feedthrough is allowed through all objects.

allow_feedthrough_select

A settable collection attribute that returns a collection representing the objects for which the allow_feedthrough attribute setting is applied.

allow_feedthrough_type

A settable string attribute (with allowed values: any mixed pure) that returns a string specifying the allow_feedthrough type of the topology_plan.

allow_flyover

A settable string attribute (with allowed values: all none select_off select_on) that returns a string specifying the allow_flyover mode of the topology_plan. "none" indicates that no flyover is allowed for edges owned by this plan. "select_on" indicates that flyover is only allowed over the objects specified in the allow_flyover_select attribute. "select_off" indicates that flyover is allowed

over all objects except those specified in the `allow_flyover_select` attribute. "all" indicates that flyover is allowed over all objects.

allow_flyover_select

A settable collection attribute that returns a collection representing the objects for which the `allow_flyover` attribute setting is applied.

balance_seq_repeaters

A settable boolean attribute that indicates whether balancing of the sequential topology repeaters is required in the branching topology plan.

block

A read-only block collection attribute that returns a collection representing the `topology_plan`'s block.

capture_budget

A settable float attribute that returns the capture budget of the topology plan.

color

A settable string attribute that returns the color of the `topology_plan`. This value must be a 24-bit RGB hexadecimal string prefaced by '#', for example: "#72c4f1" or "#ce8322".

comment

A settable string attribute that can be used to document the `topology_plan`.

edges

A read-only `topology_edge` collection attribute that returns a collection representing the `topology_edges` of the `topology_plan`.

full_name

A read-only string attribute that returns the full name of the `topology_plan`.

group

A settable `topology_group` collection attribute that returns a collection representing the `topology_group` that owns the `topology_plan`. A `topology_plan` is always owned by exactly one `topology_group`.

has_ref_nodes

A read-only boolean attribute that indicates whether the `topology_plan` has one or more nodes in reference mode.

launch_budget

A settable float attribute that returns the launch budget of the topology plan.

match_num_repeater

A settable boolean attribute that indicates whether the number of topology repeaters should exact match the number of pre-existing repeaters in netlist or the number of repeaters in the repeater constraints.

name

A settable string attribute that returns the name of the topology_plan.

net_estimation_rule

A settable name_list attribute that specifies the default names of the net_estimation_rules associated with the topology_plan.

nodes

A read-only topology_node collection attribute that represents the topology_nodes of the topology_plan.

number_of_signals

A settable int attribute that returns an unsigned integer specifying the number of signals annotated on the topology_plan.

object_class

A read-only string attribute that returns the name of this object class. Returns 'topology_plan' for topology_plan objects.

objects

A settable net, supernet, or bundle collection attribute that represents the nets, supernets, and bundles associated with the topology_plan.

parent_block

A read-only block collection attribute that returns the owner block object for this topology_plan.

physical_status

A settable string attribute (with allowed values: locked unrestricted) that specifies the physical modification status of the topology_plan.

pins

A read-only pin collection attribute that represents the pins in the objects collections of the topology_plan's nodes.

t

ports

A read-only port collection attribute that represents the ports in the objects collections of the topology_plan's nodes.

register_percentage

A settable string attribute that returns a string specifying a list of float numbers with desirable percentage-based locations of the sequential repeaters for the topology plan. These values must be between 0 and 1, for example "{0.3 0.6 0.9}".

register_req_number

A settable string attribute that returns a string specifying a list of blocks and the number of required (equal or more) sequential topology repeaters in these blocks. This is a collection of two-token strings of the form "{<blockName> <number>}" where <blockName> is a block name (allow pattern) and <number> is integer (-1, 0 or any positive value). These plan settings will override the corresponding block settings.

sequential_repeaters

A settable string attribute that returns a string specifying the sequential repeaters of the topology_plan. This is a two-token string of the form "<operator> <operand>", where "<operator>" is a relational operator that can be one of the following values: <, <=, =, >=, >, less, less_or_equal, equal, greater_or_equal, or greater. "<operand>" can be a topology_plan name, which may include wildcards, or it can be a non-negative integer. The topology_plan specified as the operand need not exist at the time this is set.

shield_nets

A settable net, supernet, or bundle collection attribute that represents the shielding nets, supernets, and bundles associated with the topology_plan.

shield_placement

A settable string attribute (with allowed values: double_interleave half_interleave interleave outside) that returns a string specifying the shield placement of the topology_plan.

stagger_pattern

A settable string attribute (with allowed values: even_bits_first none odd_bits_first) that returns the stagger pattern required for buffer/inverter implementation in a bus. Value defines which group of bits (even or odd) start first.

See Also

- [get_topology_plans](#)
- [get_attribute](#)
- [list_attributes](#)
- [set_attribute](#)

topology_repeater_attributes

Description of the application defined topology_repeater attributes.

Description

Attributes are properties assigned to objects such as pins, cells, and nets. They can be read-only or settable. A read-only attribute is informational; you cannot set its value. To modify a settable attribute, use the *set_attribute* command.

To determine the value of an attribute, use the *get_attribute* command. To see a list of all topology_repeater attributes available, use the *list_attributes -application -class topology_repeater* command.

Attributes for topology_repeater

This section describes the topology_repeater attributes.

edge

A read-only topology_edge collection attribute that represents the topology_edge that owns the topology_repeater.

full_name

A read-only string attribute that returns the full name of the topology_repeater.

name

A settable string attribute that returns the name of the topology_repeater.

object_class

A read-only string attribute that returns the name of this object class. Returns 'topology_repeater' for topology_repeater objects.

offset

A settable distance attribute that specifies the offset of the topology_repeater.

parent_block

A read-only block collection attribute that returns the owner block object for this topology_repeater.

physical_status

A settable string attribute (with allowed values: locked unrestricted) that specifies the physical modification status of the topology_repeater.

plan

A read-only topology_plan collection attribute that returns a collection representing the topology_plan of the topology_edge that owns the topology_repeater.

shadow_status

A settable string attribute (with allowed values: copied_down copied_up normal pulled_up pushed_down virtual_flat) that specifies the shadow_status of the topology_repeater.

type

A settable string attribute (with allowed values: repeater sequential) that specifies the type of the topology_repeater.

See Also

- [get_topology_repeater](#)
- [get_attribute](#)
- [list_attributes](#)
- [set_attribute](#)

track_attributes

Description of the application defined track attributes.

Description

Attributes are properties assigned to objects such as pins, cells, and nets. They can be read-only or settable. A read-only attribute is informational; you cannot set its value. To modify a settable attribute, use the *set_attribute* command.

To determine the value of an attribute, use the *get_attribute* command. To see a list of all track attributes available, use the *list_attributes -application -class track* command.

t

Attributes for track

This section describes the track attributes.

bbox

A read-only rect attribute that returns the bounding-box of the track. The format of a rectangle specification is `{{llx lly} {urx ury}}`, which specifies the lower-left and upper-right corners of the rectangle.

bounding_box

A read-only bounding_box collection attribute that returns the bounding-box of the track. The format of the collection specifies lower-left and upper-right corners of the rectangle.

design_name

A read-only string attribute that returns the name of the design owning the track.

direction

A read-only string attribute that returns the routing direction of the track. Valid values are horizontal and vertical.

end_grid_high_offset

A settable distance attribute that returns the offset from the left or bottom edge to the ending point of the first grid rectangle.

end_grid_high_offset_relative_to_core

A read-only distance attribute that returns the offset from the left or bottom edge of core area to the ending point of the first grid rectangle.

end_grid_high_steps

A settable distance_list attribute that returns the steps to use when creating subsequent grid points. Each step in the list is the distance between the previous ending grid point and the current ending grid point.

end_grid_low_offset

A settable distance attribute that returns the offset from the left or bottom edge to the starting point of the first grid rectangle.

end_grid_low_offset_relative_to_core

A read-only distance attribute that returns the offset from the left or bottom edge of core area to the starting point of the first grid rectangle.

t

end_grid_low_steps

A settable distance_list attribute that returns the steps to use when creating subsequent grid points. Each step in the list is the distance between the previous starting grid point and the current starting grid point.

full_name

A read-only string attribute that returns the name of the track.

groups

A read-only group collection attribute that returns the groups in which this track is a member.

high

A read-only distance attribute that returns the highest x-coordinate for a track with horizontal tracks or the highest y-coordinate for a track with vertical tracks.

high_coordinate

A read-only coord attribute that returns the highest point of the track.

in_edit_group

A read-only boolean attribute that indicates whether the track belongs to an edit_group.

is_default

A read-only boolean attribute that indicates whether the track is a default track. A track may be either default, non-reserved, or reserved. A default track may be used by any net and shape of any width.

is_derived

A read-only boolean attribute that indicates whether a track belongs to a site array.

is_non_reserved

A read-only boolean attribute that indicates whether the track is a non-reserved track. A track may be either default, non-reserved, or reserved. A non-reserved track is assigned a shape width constraint. It may be used only by nets without a nondefault rule and shapes with widths matching the track's assigned width constraint.

is_reserved

A read-only boolean attribute that indicates whether the track is a reserved track. A track may be either default, non-reserved, or reserved. A reserved track is assigned a shape width constraint. It may be used only by nets with

t

a nondefault rule and shapes with widths matching the track's assigned width constraint.

is_shadow

A read-only boolean attribute that indicates whether the track is pushed down from above.

layer

A read-only layer collection attribute that returns the layer on which the track exists.

layer_name

A read-only string attribute that returns the name of the layer on which the track exists.

layer_number

A read-only int attribute that returns the number of the layer on which the track exists.

low

A read-only distance attribute that returns the lowest x-coordinate for a track with horizontal tracks or the lowest y-coordinate for a track with vertical tracks.

low_coordinate

A read-only coord attribute that returns the lowest point of the track.

lower_layer_cut

A string attribute that specifies the lower via layer's cut name constraint on the track. The cut name is defined in the cutNameTbl of the lower via layer definition in the tech file.

mask_pattern

A settable string attribute that returns the list of colors used as a repeating pattern to define the colors for the tracks. Each track (from low to high within the track coordinates) has a color based on its numerical track value modulo the number of track colors, used as an index into the track color collection. Note that the default for each track is "no_mask", indicating that no specific mask is assigned to the track. Also note that the set of color masks is limited today to two values, "mask_one" and "mask_two". Note that the pattern can contain a maximum of five values that are repeated for all the tracks in the track specification. Lastly, to remove a track color specification, set the attribute using an empty list of values.

t

name

A read-only string attribute that returns the name of the track.

number_of_tracks

A read-only int attribute that returns the number of tracks in the track.

object_class

A read-only string attribute that returns the name of this object class. Returns 'track' for track objects.

object_id

A read-only int attribute that returns the object_id of the track.

parent_block

A read-only block collection attribute that returns the owner block object for this track.

parent_block_cell

A read-only cell collection attribute that returns the parent block cell object for this track.

parent_cell

A read-only cell collection attribute that returns the owner cell object for this track.

physical_status

A settable string attribute (with allowed values: locked unrestricted) that returns the physical placement status of the track. If the status is unrestricted, the track may be moved. If the status is locked, the track is fixed for automatic and manual changes.

shadow_status

A settable string attribute (with allowed values: copied_down copied_up normal pulled_up pushed_down virtual_flat) that returns the shadow status of the track.

space

A read-only distance attribute that returns the space between tracks.

start

A read-only distance attribute that returns the starting y-coordinate for a track with horizontal tracks or the starting x-coordinate for a track with vertical tracks.

u

top_block

A read-only block collection attribute that returns the top block object for this track.

track_line_starts

A read-only distance_list attribute that returns a distance list of y-coordinates for every track line of a horizontal track and list of x-coordinates for vertical tracks.

upper_layer_cut

A string attribute that specifies the upper via layer's cut name constraint on the track. The cut name is defined in the cutNameTbl of the upper via layer definition in the tech file.

via_shift_high

A distance attribute that specifies the amount to shift vias upward if the preferred routing direction is horizontal and rightward if the preferred routing direction is vertical.

via_shift_low

A distance attribute that specifies the amount to shift vias downward if the preferred routing direction is horizontal and leftward if the preferred routing direction is vertical.

width

A read-only distance attribute that returns the shape width constraint for non-reserved and reserved tracks.

See Also

- [get_attribute](#)
- [list_attributes](#)

u

utilization_config_attributes

Description of the application defined utilization_config attributes.

u

Description

Attributes are properties assigned to objects such as pins, cells, and nets. They can be read-only or settable. A read-only attribute is informational; you cannot set its value. To modify a settable attribute, use the *set_attribute* command.

To determine the value of an attribute, use the *get_attribute* command. To see a list of all utilization_config attributes available, use the *list_attributes -application -class utilization_config* command.

Attributes for utilization_config

This section describes the utilization_config attributes.

as_user_default

A read-only boolean attribute that indicates if a utilization_config is the user defined default configuration for utilization reporting.

capacity

A settable string attribute (with allowed values: boundary core_area site_array site_row) that returns a string attribute based on which the utilization of the block will be reported.

exclusions

A settable exclusions attribute that states the object types to be excluded from demand and capacity calculation. This attribute accepts multiple values, in which case the value should be a list of strings. The exclusion and inclusion object type categories must be distinct. Any object type specified as *exclusions* attribute can not be repeated as *inclusions* attribute of the same utilization_config.

inclusions

A settable exclusions attribute that states the object types to be included into demand and capacity calculation. This attribute accepts multiple values, in which case the value should be a list of strings. The inclusion and exclusion object type categories must be distinct. Any object type specified as *inclusions* attribute can not be repeated as *exclusions* attribute of the same utilization_config.

name

A read-only string attribute that returns the name of a utilization_config.

object_class

A read-only string attribute that returns the name of this object class. Returns 'utilization_config' for utilization_config objects.

v

scope

A read-only string attribute (with allowed values: block lib tech) that returns the scope of the `utilization_config`.

See Also

- [create_utilization_configuration](#)
- [remove_utilization_configurations](#)
- [get_utilization_configurations](#)
- [report_utilization](#)
- [get_attribute](#)
- [list_attributes](#)
- [set_attribute](#)

v

variation_attributes

Description of the application defined variation attributes.

Description

Attributes are properties assigned to objects such as pins, cells, and nets. They can be read-only or settable. A read-only attribute is informational; you cannot set its value. To modify a settable attribute, use the `set_attribute` command.

To determine the value of an attribute, use the `get_attribute` command. To see a list of all variation attributes available, use the `list_attributes -application -class variation` command.

Attributes for variation

This section describes the variation attributes.

mean

A read-only float attribute that returns the mean of the distribution of the variation.

mean_shift

A read-only float attribute that returns the mean shift of the distribution of the variation.

v

nominal

A read-only float attribute that returns the nominal of the distribution of the variation.

object_class

A read-only string attribute that returns the name of this object class. Returns 'variation' for variation objects.

skewness

A read-only float attribute that returns the skewness of the distribution of the variation.

std_dev

A read-only float attribute that returns the standard deviation of the distribution of the variation.

variance

A read-only float attribute that returns the variance of the distribution of the variation.

See Also

- [get_attribute](#)
- [list_attributes](#)
- [set_attribute](#)

via_attributes

Description of the application defined via attributes.

Description

Attributes are properties assigned to objects such as pins, cells, and nets. They can be read-only or settable. A read-only attribute is informational; you cannot set its value. To modify a settable attribute, use the *set_attribute* command.

To determine the value of an attribute, use the *get_attribute* command. To see a list of all via attributes available, use the *list_attributes -application -class via* command.

Attributes for via

This section describes the via attributes.

v

array_size

A settable string attribute that returns the size of the via. The string lists the number of rows and number of columns in the via (e.g., "3 4" indicates the via has 3 rows and 4 columns).

bbox

A read-only rect attribute that returns the bounding-box of a design. The format of a rectangle specification is `{{llx lly} {urx ury}}`, which specifies the lower-left and upper-right corners of the rectangle.

bounding_box

A read-only bounding_box collection attribute that returns the bounding-box of a design. The format of the collection specifies lower-left and upper-right corners of the rectangle.

cut_layer_names

A read-only string attribute that returns the cut layer names of the via.

cut_layers

A read-only layer collection attribute that returns the cut layers of the via.

cut_mask_constraint

A settable string attribute that returns the multi-patterning mask constraint of the via cuts. Valid values are `no_mask`, `mask_one`, `mask_two`, `mask_three`, `mask_four`, `mask_five`, `mask_six`, `mask_seven`, `mask_eight`, `mask_nine`, `mask_ten`, `mask_eleven`, `mask_twelve`, `mask_thirteen`, `mask_fourteen`, `mask_fifteen` and `same_mask`. For a custom via, `inherit_mask` is also a valid value to set on this attribute and indicates that the mask is inherited from the `via_def`. Note that when querying this attribute, the effective mask constraint is returned (i.e., `inherit_mask` is never returned as the value).

cut_pattern

A settable string attribute that returns the cut pattern of a simple array via. The pattern defines a MxN pattern of cuts. If the pattern is smaller than the array via size, the pattern is replicated left-to-right, bottom-to-top in the array via. Excess bits in the pattern are ignored. The pattern is formatted as a space delimited string of equal length substrings (e.g., "1010 0101 1011" is a 3x4 pattern). Each substring must consist of only 1s and 0s, representing a row in the MxN pattern, starting with the bottom row in the pattern. A 1 means the cut is included. A 0 means the cut is omitted. If the cut pattern does not exist, the via has a full array of cuts (i.e., no cuts are omitted). This attribute is settable on simple array vias. If set, a via specific cut pattern is assigned to this via which overrides the `via_def` cut pattern. If set to an empty string or removed through

v

the *remove_attributes* command, the via specific cut pattern is removed and this via will inherit the *via_def* cut pattern.

cut_pattern_size

A read-only string attribute that returns the size of the cut pattern of a simple array via. This attribute exists only on simple array vias. The string lists the number of rows and number of columns in the pattern (e.g., "3 4" indicates the cut pattern has 3 rows and 4 columns).

design_name

A read-only string attribute that returns the *design_name* of the via.

full_name

A read-only string attribute that returns the full name of a via.

groups

A read-only group collection attribute that returns the groups in which this via is a member.

has_net

A read-only boolean attribute that returns true if this via is connected to a net.

height

A read-only distance attribute that returns the height of the via.

ignore_ndr_spacing

A settable boolean attribute that determines whether the secondary power router should ignore NDR spacing for the via.

in_edit_group

A read-only boolean attribute that returns true if this via is in any *edit_group*.

is_cut_mask_fixed

A settable boolean attribute that determines if the cut mask is a hard type. e. cannot be changed by router.

is_default_via

A read-only boolean attribute that returns true if this via is a default via for its upper and lower metal layers.

is_fixed

A read-only boolean attribute that returns True if the physical status of the via is either fixed or locked.

v

is_lower_mask_fixed

A settable boolean attribute that returns whether the metal layer lower mask attribute is a hard type if the value is "true". e. cannot be changed by router.

is_parallel_wire

A read-only boolean attribute that returns whether this via was created as a fat wire for parallel wire purpose. This attribute only exists on simple vias, simple array vias, and custom vias.

is_pattern_must_join

A read-only boolean attribute that returns whether the via is part of the pattern_must_join connection.

is_shadow

A read-only boolean attribute that returns whether this via is a shadow via.

is_upper_mask_fixed

A settable boolean attribute that returns whether the metal layer upper mask attribute is a hard type if the value is "true". e. cannot be changed by router.

is_via_ladder

A read-only boolean attribute that returns whether the via is part of a via ladder. This is not a settable attribute.

is_via_staple

A settable boolean attribute that returns whether the via is a via staple.

lower_layer

A read-only layer collection attribute that returns the lower layer for the via.

lower_layer_bounding_box

A read-only bounding_box collection attribute that returns the bounding-box of the lower layer for the via. The format of a rectangle specification is {{llx lly} {urx ury}}, which specifies the lower-left and upper-right corners of the rectangle.

lower_layer_name

A read-only string attribute that returns the name of the lower layer for the via.

lower_mask_constraint

A settable string attribute that returns the multi-patterning mask constraint of the via lower enclosure. Valid values are no_mask, mask_one, mask_two, mask_three, mask_four, and same_mask. For a custom via, inherit_mask is also a valid value to set on this attribute and indicates that the mask is inherited from

v

the `via_def`. Note that when querying this attribute, the effective mask constraint is returned (i.e., `inherit_mask` is never returned as the value).

name

A read-only string attribute that returns the name of a via.

net

A settable net collection attribute that represents the owner net of this via.

net_type

A read-only string attribute (with allowed values: `analog_ground` `analog_power` `analog_signal` `clock` `deep_nwell` `deep_pwell` `ground` `nwell` `power` `pwell` `reset` `scan` `signal` `tie_high` `tie_low` `unset`) that returns the net type of net connected to the via.

number_of_columns

A settable int attribute that returns the number of columns if this is an array via.

number_of_rows

A settable int attribute that returns the number of rows if this is an array via.

object_class

A read-only string attribute that returns the name of this object class. Returns 'via' for via objects.

object_id

A read-only int attribute that returns the `object_id` of the via.

offset

A settable coord attribute that returns the offset of via from block boundary.

orientation

A settable string attribute (with allowed values: `R0` `R90` `R180` `R270` `MX` `MXR90` `MY` `MYR90`) that returns the orientation of the via.

origin

A settable coord attribute that returns the location of the via's center.

owner

A settable port, `lib_pin`, terminal, or net collection attribute that returns the net which owns the via.

v

parent_block

A read-only block collection attribute that returns the owner block object for this via.

parent_block_cell

A read-only cell collection attribute that returns the parent block cell object for this via.

parent_cell

A read-only cell collection attribute that returns the owner cell object for this via.

physical_status

A settable string attribute (with allowed values: application_fixed fixed locked minor_change unrestricted) that returns the physical modification status of the via. If status is unrestricted, the via may be freely modified. If the status is application_fixed, the via is fixed by the application for automatic changes (the application may change this status). If status is fixed, the via is fixed by the user for automatic changes. If status is locked, the via is fixed for automatic and manual changes.

pnet_ignore

A settable boolean attribute that indicates whether PG vias are to be ignored by routing engines.

rotation

A settable float attribute that returns the all-angle rotation value of the via in degrees. Allowed to be set only for vias instantiated within designs of type pcb, rdl_fanout or package.

rule_type

A settable string attribute (with allowed values: default net_non_default none shape_non_default) that returns the rule type used to create the via. If the value is default, it was created based on the default rule. If the value is net_non_default, it was created based on the net nondefault routing rule. If the value is none, it was created based on no rule. If the value is shape_non_default, it was created based on a via specific routing rule.

shadow_status

A settable string attribute (with allowed values: copied_down copied_up normal pulled_up pushed_down virtual_flat) that returns the shadow status of the via.

shape_use

A settable string attribute (with allowed values: `area_fill` `core_wire` `detail_route` `follow_pin` `global_route` `lib_cell_pin_connect` `macro_pin_connect` `opc` `partial_parallel_route` `pg_augmentation` `rdl` `ring` `shield_route` `stripe` `taper` `trunk` `user_route` `zero_skew`) that returns the shape use.

shapes

A read-only `poly_rect` collection attribute that describes the physical geometry of the via.

top_block

A read-only block collection attribute that returns the top block object for this via.

upper_layer

A read-only layer collection attribute that represents the upper layer for the via.

upper_layer_bounding_box

A read-only `bounding_box` collection attribute that represents the bounding-box of the upper layer for the via. The format of a rectangle specification is `{{llx lly} {urx ury}}`, which specifies the lower-left and upper-right corners of the rectangle.

upper_layer_name

A read-only string attribute that returns the name of the upper layer for the via.

upper_mask_constraint

A settable string attribute that returns the multi-patterning mask constraint of the via upper enclosure. Valid values are `no_mask`, `mask_one`, `mask_two`, `mask_three`, `mask_four`, and `same_mask`. For a custom via, `inherit_mask` is also a valid value to set on this attribute and indicates that the mask is inherited from the `via_def`. Note that when querying this attribute, the effective mask constraint is returned (i.e., `inherit_mask` is never returned as the value).

via_def

A settable `via_def` collection attribute that represents the via def for the via.

via_def_name

A read-only string attribute that returns the via def name for the via.

via_type

A read-only string attribute that returns the via type for the via.

width

A read-only distance attribute that returns the width of the via.

v

x_pitch

A settable distance attribute that returns the horizontal pitch if this is an array via.

y_pitch

A settable distance attribute that returns the vertical pitch if this is an array via.

See Also

- [get_vias](#)
- [get_attribute](#)
- [list_attributes](#)
- [set_attribute](#)

via_def_attributes

Description of the application defined via_def attributes.

Description

Attributes are properties assigned to objects such as pins, cells, and nets. They can be read-only or settable. A read-only attribute is informational; you cannot set its value. To modify a settable attribute, use the *set_attribute* command.

To determine the value of an attribute, use the *get_attribute* command. To see a list of all via_def attributes available, use the *list_attributes -application -class via_def* command.

Attributes for via_def

This section describes the via_def attributes.

bottom_via_def

A read-only via_def collection attribute that returns 0 or 1 via_def. It is an optional simple or custom via_def that sits on top of the through silicon via_def.

cut_height

A settable distance attribute that returns the height of the cut_layer shape of a simple via_def.

cut_layer_names

A settable string attribute that returns the list of cut layer names for the via_def. A simple via_def has a single cut_layer. A custom via_def may have more than one.

v

cut_layers

A settable layer collection attribute that represents the cut layers of the `via_def`. A simple `via_def` has a single cut layer. A custom `via_def` may have more than one.

cut_pattern

A settable string attribute that returns the default cut pattern on array vias of this simple `via_def`. The pattern defines a MxN pattern of cuts. If the pattern is smaller than the array via size, the pattern is replicated left-to-right, bottom-to-top in the array via. Excess bits in the pattern are ignored. The pattern is formatted as a space delimited string of equal length substrings (e.g., "1010 0101 1011" is a 3x4 pattern). Each substring must consist of only 1s and 0s, representing a row in the MxN pattern, starting with the bottom row in the pattern. A 1 means the cut is included. A 0 means the cut is omitted. If the cut pattern does not exist, the pattern is a full array of cuts (i.e., no cuts are omitted). This attribute is settable on simple `via_defs`. If set to an empty string or removed through the *remove_attributes* command, the cut pattern is removed and the pattern will default to a full array of cuts.

cut_pattern_size

A read-only string attribute that returns the size of the cut pattern. This attribute exists only on simple `via_defs`. The string lists the number of rows and number of columns in the pattern (e.g., "3 4" indicates the cut pattern has 3 rows and 4 columns).

cut_width

A settable distance attribute that returns the width of the `cut_layer` shape of a simple `via_def`.

full_name

A settable string attribute that returns the full name of a `via_def`.

is_compound_tsv

A read-only boolean attribute that returns true if this `via_def` represents a compound TSV.

is_default

A settable boolean attribute that returns true if this `via_def` is a default `via_def` for its upper and lower metal layers. Only a simple `via_def` may be a default `via_def`.

v

is_excluded_for_signal_route

A settable boolean attribute that returns true if this `via_def` represents vias which are excluded for use in signal (non-power) routing.

is_fat_wire_via_only

A settable boolean attribute that indicates whether this `via_def` is used only for fat wiring. This attribute is set from the `isFatWireViaOnly` field in the tech file's `ContactCode` definition. This attribute is settable directly by the user if and only if the `via_def` has `min_cut_x_spacing` and `min_cut_y_spacing` attributes, which are defined respectively by the `minCutXSpacing` and `minCutYSpacing` fields in the tech file's `ContactCode` definition.

is_redundant_via_insertion_only

A settable boolean attribute that returns true if this `via_def` represents vias that can only be used for redundant via insertion during routing.

lower_enclosure_height

A settable distance attribute that returns the height of the lower metal shape of a simple `via_def`.

lower_enclosure_width

A settable distance attribute that returns the width of the lower metal shape of a simple `via_def`.

lower_layer

A read-only layer collection attribute that represents the lower metal layer of a `via_def`.

lower_layer_name

A read-only string attribute that returns the name of the lower metal layer of the `via_def`.

lower_mask_pattern

A settable string attribute (with allowed values: `alternate_columns`, `alternate_rows`, `alternating`, `uniform`) that returns the lower mask pattern of the `via_def`.

mask_pattern

A settable string attribute (with allowed values: `alternate_columns`, `alternate_rows`, `alternating`, `uniform`) that returns the multi pattern of the via cuts.

v

min_columns

A settable int attribute that returns the minimum number of columns of cuts on a via of this array simple via_def.

min_cut_spacing

A settable distance attribute that returns the minimum spacing between cuts of an array via instantiated from this simple via_def.

min_cut_x_spacing

A read-only distance attribute that returns the minimum horizontal spacing between cuts of an array via instantiated from this simple via_def. This attribute exists if and only if the "minCutXSpacing" attribute was specified in the tech file.

min_cut_y_spacing

A read-only distance attribute that returns the minimum vertical spacing between cuts of an array via instantiated from this simple via_def. This attribute exists if and only if the "minCutYSpacing" attribute was specified in the tech file.

min_rows

A settable int attribute that returns the minimum number of rows of cuts on a via of this array simple via_def.

name

A settable string attribute that returns the name of a via_def.

object_class

A read-only string attribute that returns the name of this object class. Returns 'via_def' for via_def objects.

owner

A read-only block, or tech collection attribute that returns a collection representing the owner container of a via_def. The owner may be a block or a tech.

shapes

A settable poly_rect collection attribute representing the shapes of a via_def. A custom via_def is defined by its shapes. The shapes of a simple via_def are derived from its parameters.

source_type

A read-only string attribute (with allowed values: generated generated_for_special_nets multiple_cut multiple_cut_fixed single_cut single_cut_fixed) that is the source type of a via_def.

v

top_via_def

A read-only `via_def` collection attribute that is an optional simple or custom `via_def` that sits at bottom of the through silicon `via_def`.

tsv_keepout_spacing

A settable distance attribute that returns the through-silicon via ("TSV") keepout spacing for this `via_def`. Note that a `via_def` will have this attribute value if and only if it is of type `through_silicon_via_def`.

upper_enclosure_height

A settable distance attribute that returns the height of the upper metal shape of a simple `via_def`.

upper_enclosure_width

A settable distance attribute that returns the width of the upper metal shape of a simple `via_def`.

upper_layer

A read-only layer collection attribute representing the upper metal layer of a `via_def`.

upper_layer_name

A read-only string attribute that returns the name of the upper metal layer of a `via_def`.

upper_mask_pattern

A settable string attribute (with allowed values: `alternate_columns`, `alternate_rows`, `alternating`, `uniform`) that returns the upper mask pattern of the `via_def`.

via_def_type

A read-only string attribute that returns the type of a `via_def`. The value is one of: `simple_via_def`, `custom_via_def`, or `through_silicon_via_def`.

See Also

- [get_via_defs](#)
- [report_via_defs](#)
- [get_attribute](#)
- [list_attributes](#)
- [set_attribute](#)

via_ladder_attributes

Description of the application defined via_ladder attributes.

Description

Attributes are properties assigned to objects such as pins, cells, and nets. They can be read-only or settable. A read-only attribute is informational; you cannot set its value. To modify a settable attribute, use the *set_attribute* command.

To determine the value of an attribute, use the *get_attribute* command. To see a list of all via_ladder attributes available, use the *list_attributes -application -class via_ladder* command.

Attributes for via_ladder

This section describes the via_ladder attributes.

bbox

A read-only rect attribute that returns the bounding-box of this via_ladder. The format of a rectangle specification is *{{l1x l1y} {urx ury}}*, which specifies the lower-left and upper-right corners of the rectangle.

full_name

A read-only string attribute that returns the full name of the via_ladder.

is_custom

A settable boolean attribute that indicates whether the via_ladder is considered as a custom via_ladder. This is usually set on via_ladders created manually, for example, using the command *create_trunk_via_ladder*. The default value is false.

is_electromigration

A read-only boolean attribute that returns the type of via_ladder. This attribute is set to true, if the via_ladder is of type 'Electromigration'. The default value is false.

is_high_performance

A read-only boolean attribute that returns the type of via_ladder. This attribute is set to true, if the via_ladder is of type 'High Performance'. The default value is false.

is_pattern_must_join

A read-only boolean attribute that returns the type of via_ladder. This attribute is set to true, if the via_ladder is of type 'Pattern Must Join'. The default value is false.

v

is_stack_via

A read-only boolean attribute that returns the type of `via_ladder`. This attribute is set to true, if the `via_ladder` is of type 'Stack Via'. The default value is false.

name

A read-only string attribute that returns the name of a `via_ladder`.

net

A read-only net collection attribute that returns exactly one net on which the `via_ladder` is connected.

object_class

A read-only string attribute that returns the name of this object class. Returns 'via_ladder' for `via_ladder` objects.

object_id

A read-only int attribute that returns the `object_id` of the `via_ladder`.

parent_block

A read-only block collection attribute that returns the owner block object for this `via_ladder`.

parent_block_cell

A read-only cell collection attribute that returns the parent block cell object for this `via_ladder`.

parent_cell

A read-only cell collection attribute that returns the owner cell object for this `via_ladder`.

physical_status

A read-only string attribute (with allowed values: `application_fixed` `fixed` `locked` `minor_change` `unrestricted`) that returns the physical modification status of the `via_ladder`. If status is `unrestricted`, the `via_ladder` may be freely modified. If the status is `application_fixed`, the `via_ladder` is fixed by the application for automatic changes (the application may change this status). If status is `fixed`, the `via_ladder` is fixed by the user for automatic changes. If status is `locked`, the `via_ladder` is fixed for automatic and manual changes.

pin

A read-only pin collection attribute that returns the pin associated with the `via_ladder`.

v

shapes

A read-only shape, or via collection attribute that returns the shapes (layer shapes and vias) which collectively constitute the `via_ladder`.

via_rule_name

A read-only string attribute that returns the via rule name of the via ladder.

See Also

- [get_attribute](#)
- [list_attributes](#)
- [set_attribute](#)

via_matrix_attributes

Description of the application defined `via_matrix` attributes.

Description

Attributes are properties assigned to objects such as pins, cells, and nets. They can be read-only or settable. A read-only attribute is informational; you cannot set its value. To modify a settable attribute, use the `set_attribute` command.

To determine the value of an attribute, use the `get_attribute` command. To see a list of all `via_matrix` attributes available, use the `list_attributes -application -class via_matrix` command.

Attributes for via_matrix

This section describes the `via_matrix` attributes.

base_array_size

A settable string attribute that returns the size of a base via in the `via_matrix`. The string lists the number of rows and number of columns in the base via (e.g., "3 4" indicates the base via has cuts of 3 rows and 4 columns).

base_number_of_columns

A settable int attribute that returns the number of cut columns in the base via of this `via_matrix`.

base_number_of_rows

A settable int attribute that returns the number of cut rows in the base via of this `via_matrix`.

v

base_x_pitch

A settable distance attribute that returns the horizontal pitch in the base via of this `via_matrix`.

base_y_pitch

A settable distance attribute that returns the vertical pitch in the base via of this `via_matrix`.

bbox

A read-only rect attribute that returns the bounding-box of this `via_matrix`. The format of a rectangle specification is `{{llx lly} {urx ury}}`, which specifies the lower-left and upper-right corners of the rectangle.

cut_layer_names

A read-only string attribute that returns the cut layer names of the `via_matrix`.

cut_layers

A read-only layer collection attribute that returns the cut layers of the `via_matrix`.

cut_mask_constraint

A settable string attribute that returns the multi-patterning mask constraint of the `via_matrix` cuts. Only the ones at lower left cuts of specified base vias are to be reported. The specified base vias can be the lower left base vias and the base vias whose cut masks are different from the cut mask of the lower left base via. All other base vias take the cut mask of the lower left base via as theirs. The value is a string in format `{row column maskValue [fixed]}` i.e. `{0 1 mask_two fixed}` which means set the cut mask of 0X1 base via as `mask_two` and fixed. The fixed can be absent, which means the mask constraint isn't fixed. `maskValue` can be one of `no_mask`, `mask_one`, `mask_two`, `mask_three`, `mask_four`, `mask_five`, `mask_six`, `mask_seven`, `mask_eight`, `mask_nine`, `mask_ten`, `mask_eleven`, `mask_twelve`, `mask_thirteen`, `mask_fourteen`, `mask_fifteen`, `same_mask` Note: this attribute result only displays mask values for 100 base vias.

cut_mask_pattern

A settable string attribute that returns the cut layer mask pattern of the `via_matrix` in a multiple-patterning technology. Valid values are `arbitrary`, `uniform`, `alternate_columns` and `alternate_rows`. Default is `uniform`. The `uniform` means cut mask values in all base vias of the `via_matrix` are the same. When it is `alternate_columns` or `alternate_rows`, cut masks of the lower left cuts in the base vias alternates in columns or rows accordingly if the `via_def cut_layer number_of_masks` attribute is not less than 2. When it is `arbitrary`, cut masks of the lower left cuts in the base vias are considered independently. The cut

v

mask pattern in a base via is inherited from the `via_def` of the `via_matrix`. With mask pattern change, please remove the `cut_mask_constraint` then set the `cut_mask_constraint` to make sure the `cut_mask_constraint` is correct.

cut_pattern

A settable string attribute that returns the cut pattern in a base simple array via of the `via_matrix`. The pattern defines a MxN pattern of cuts. If the pattern is smaller than the base via size, the pattern is replicated left-to-right, bottom-to-top in the base via. Excess bits in the pattern are ignored. The pattern is formatted as a space delimited string of equal length substrings (e.g., "1010 0101 1011" is a 3x4 pattern). Each substring must consist of only 1s and 0s, representing a row in the MxN pattern, starting with the bottom row in the pattern. A 1 means the cut is included. A 0 means the cut is omitted. If the cut pattern does not exist, the base via has a full array of cuts (i.e., no cuts are omitted). This attribute is settable on simple vias. If set, a base via specific cut pattern is assigned to this `via_matrix` which overrides the `via_def` cut pattern. If set to an empty string or removed through the `remove_attributes` command, the base via specific cut pattern is removed and this base via will inherit the `via_def` cut pattern.

cut_pattern_size

A read-only string attribute that returns the size of the cut pattern in a base simple array via of this `via_matrix`. This attribute is valid only on a base simple array via. The string lists the number of rows and number of columns in the pattern (e.g., "3 4" indicates the cut pattern has 3 rows and 4 columns).

design_name

A read-only string attribute that returns the `design_name` of the `via_matrix`.

full_name

A read-only string attribute that returns the full name of the `via_matrix`.

has_net

A read-only boolean attribute that returns true if this `via_matrix` is connected to a net.

ignore_ndr_spacing

A settable boolean attribute that returns true if this `via_matrix` is to ignore ndr spacing.

in_edit_group

A read-only boolean attribute that returns true if this `via_matrix` is in any `edit_group`.

v

index_cut_mask_constraint

A settable string attribute that returns the multi-patterning mask constraint of the via_matrix cut. This attribute is subscripted. The subscript value is <row>,<column>. The row and column must be less than the values returned by the pattern_size attribute. Using this attribute you may specify a specific mask constraint in the via_matrix. maskValue can be one of no_mask, mask_one, mask_two, mask_three, mask_four, mask_five, mask_six, mask_seven, mask_eight, mask_nine, mask_ten, mask_eleven, mask_twelve, mask_thirteen, mask_fourteen, mask_fifteen, same_mask. For example to specify a maskValue at row 3 column 2:

```
prompt> set_attribute $vm index_cut_mask_constraint(3,2) mask_two
```

index_is_cut_mask_fixed

A settable boolean attribute that indicates whether a given-index cut mask in the via_matrix is true or not. This attribute is subscripted. The subscript value is <row>,<column>. The row and column must be less than the values returned by the pattern_size attribute. Using this attribute you may remove a specific entry in the via_matrix. For example, to set a fix at row 3 column 2:

```
prompt> set_attribute $vm index_is_cut_mask_fixed(3,2) false
```

index_is_lower_mask_fixed

A settable boolean attribute that indicates whether a given-index lower mask in the via_matrix is true or not. This attribute is subscripted. The subscript value is <row>,<column>. The row and column must be less than the values returned by the pattern_size attribute. Using this attribute you may remove a specific entry in the via_matrix. For example, to set a fix at row 3 column 2:

```
prompt> set_attribute $vm index_is_lower_mask_fixed(3,2) false
```

index_is_upper_mask_fixed

A settable boolean attribute that indicates whether a given-index upper mask in the via_matrix is true or not. This attribute is subscripted. The subscript value is <row>,<column>. The row and column must be less than the values returned by the pattern_size attribute. Using this attribute you may remove a specific entry in the via_matrix. For example, to set a fix at row 3 column 2:

```
prompt> set_attribute $vm index_is_upper_mask_fixed(3,2) false
```

index_lower_mask_constraint

A settable string attribute that returns the multi-patterning mask constraint of the via_matrix upper enclosure. This attribute is subscripted. The subscript value is <row>,<column>. The row and column must be less than the values returned by the pattern_size attribute. Using this attribute you may specify a specific mask

v

constraint in the `via_matrix`. `maskValue` can be one of `no_mask`, `mask_one`, `mask_two`, `mask_three`, `same_mask`. For example, to specify a `maskValue` at row 3 column 2:

```
prompt> set_attribute $vm index_lower_mask_constraint(3,2)
mask_two
```

index_occupied

A settable boolean attribute that indicates if a given index in the `via_matrix` is currently occupied or empty. This attribute is subscripted. The subscript value is `<row>,<column>`. The row and column must be less than the values returned by the `pattern_size` attribute. Using this attribute you may remove a specific entry in the `via_matrix`. For example, to punch a hole at row 3 column 2:

```
prompt> set_attribute $vm index_occupied(3,2) false
```

index_upper_mask_constraint

A settable string attribute that returns the multi-patterning mask constraint of the `via_matrix` upper enclosure. This attribute is subscripted. The subscript value is `<row>,<column>`. The row and column must be less than the values returned by the `pattern_size` attribute. Using this attribute you may specify a specific mask constraint in the `via_matrix`. `maskValue` can be one of `no_mask`, `mask_one`, `mask_two`, `mask_three`, `same_mask`. For example, to specify a `maskValue` at row 3 column 2:

```
prompt> set_attribute $vm index_upper_mask_constraint(3,2)
mask_two
```

is_default_via_matrix

A read-only boolean attribute that returns true if this `via_matrix` is a default `via_matrix` for its upper and lower metal layers.

is_shadow

A read-only boolean attribute that returns true if this `via_matrix` is a shadow `via_matrix`.

is_via_staple

A settable boolean attribute that returns true if these base vias in the `via_matrix` are staple vias.

lower_layer

A read-only layer collection attribute that returns a collection representing the lower layer for the `via_matrix`.

v

lower_layer_name

A read-only string attribute that returns the name of the lower layer for the `via_matrix`.

lower_mask_constraint

A settable string attribute that returns the multi-patterning mask constraint of the `via_matrix` lower enclosure. Only the ones of specified base vias are to be reported. The specified base vias can be the lower left base vias and the base vias whose lower masks are different from the lower mask of the lower left base via. All other base vias take the lower mask of the lower left base via as theirs. The value is a string in format {row column maskValue [fixed]} i.e. {0 1 mask_two fixed} which means set the upper mask of 0X1 base via as mask_two and fixed. The fixed can be absent, which means the mask constraint isn't fixed. maskValue can be one of no_mask, mask_one, mask_two, mask_three, same_mask. Note: this attribute result only displays mask values for 100 base vias.

lower_mask_pattern

A settable string attribute that returns the lower layer mask pattern of the `via_matrix` in a multiple-patterning technology. Valid values are arbitrary and uniform. The uniform means lower mask values in all base vias of the `via_matrix` are the same. When it is alternate_columns or alternate_rows, lower masks of the lower-layer in the base vias alternates in columns or rows accordingly if the `via_def lower_layer number_of_masks` attribute is not less than 2. When it is arbitrary, lower layer masks in the base vias are considered independently. With mask pattern change, please remove the `lower_mask_constraint` then set the `lower_mask_constraint` to make sure the `lower_mask_constraint` is correct.

name

A read-only string attribute that returns the name of a `via_matrix`.

net_type

A read-only string attribute (with allowed values: analog_ground analog_power analog_signal clock deep_nwell deep_pwell ground nwell power pwell reset scan signal tie_high tie_low unset) that returns the net type of net connected to the `via_matrix`.

number_of_columns

A settable int attribute that returns the number of columns in this `via_matrix`.

number_of_rows

A settable int attribute that returns the number of rows in this `via_matrix`.

v

object_class

A read-only string attribute that returns the name of this object class. Returns 'via_matrix' for via_matrix objects.

object_id

A read-only int attribute that returns the object_id of the via_matrix.

offset

A settable coord attribute that returns the offset of via_matrix from block boundary.

orientation

A settable string attribute (with allowed values: R0 R90 R180 R270 MX MXR90 MY MYR90) that returns the orientation of the via_matrix.

origin

A settable coord attribute that returns the origin of the via_matrix.

owner

A read-only net collection attribute that returns the net which owns the via_matrix.

parent_block

A read-only block collection attribute that returns the owner block object for this via_matrix.

parent_block_cell

A read-only cell collection attribute that returns the parent block cell object for this via_matrix.

parent_cell

A read-only cell collection attribute that returns the owner cell object for this via_matrix.

pattern

A settable string attribute that returns the pattern in a base simple array via of the via_matrix. The pattern defines a MxN pattern of base vias. If the pattern is smaller than the base via size, the pattern is replicated left-to-right, bottom-to-top in the base via. Excess bits in the pattern are ignored. The pattern is formatted as a space delimited string of equal length substrings (e.g., "1010 0101 1011" is a 3x4 pattern). Each substring must consist of only 1s and 0s, representing a row in the MxN pattern, starting with the bottom row in the pattern. A 1 means the cut is included. A 0 means the base via is omitted. If the

v

pattern does not exist, the base via has a full array of cuts (i.e., no base vias are omitted). This attribute is settable on base simple vias. If set, a base via specific pattern is assigned to this `via_matrix` which overrides the original base via pattern. If set to an empty string or removed through the *remove_attributes* command, the base via specific pattern is removed and this means all base vias are included. The value of this attribute may be truncated if the resulting string is very large (>1000 characters)

pattern_size

A read-only string attribute that returns the size of the pattern in a base simple array via of this `via_matrix`. The string lists the number of rows and number of columns in the pattern (e.g., "3 4" indicates the base-via pattern has 3 rows and 4 columns).

physical_status

A settable string attribute (with allowed values: `application_fixed` `fixed` `locked` `minor_change` `unrestricted`) that returns the physical modification status of the `via_matrix`. If status is `unrestricted`, the `via_matrix` may be freely modified. If the status is `application_fixed`, the `via_matrix` is fixed by the application for automatic changes (the application may change this status). If status is `fixed`, the `via_matrix` is fixed by the user for automatic changes. If status is `locked`, the `via_matrix` is fixed for automatic and manual changes.

shadow_status

A settable string attribute (with allowed values: `copied_down` `copied_up` `normal` `pulled_up` `pushed_down` `virtual_flat`) that returns the shadow status of the `via_matrix`.

shape_use

A settable string attribute (with allowed values: `area_fill` `core_wire` `detail_route` `follow_pin` `global_route` `lib_cell_pin_connect` `macro_pin_connect` `opc` `partial_parallel_route` `pg_augmentation` `rdl` `ring` `shield_route` `stripe` `taper` `trunk` `user_route` `zero_skew`) that returns the shape use.

top_block

A read-only block collection attribute that returns the top block object for this `via_matrix`.

upper_layer

A read-only layer collection attribute that represents the upper layer for the `via_matrix`.

v

upper_layer_name

A read-only string attribute that returns the name of the upper layer for the `via_matrix`.

upper_mask_constraint

A settable string attribute that returns the multi-patterning mask constraint of the `via_matrix` upper enclosure. Only the ones of specified base vias are to be reported. The specified base vias can be the lower left base vias and the base vias whose upper masks are different from the upper mask of the lower left base via. All other base vias take the upper mask of the lower left base via as theirs. The value is a string in format of {row column maskValue [fixed]} i.e. {0 1 mask_two fixed} which means set the upper mask of 0X1 base via as mask_two and fixed. The fixed can be absent, which means the mask constraint isn't fixed. maskValue can be one of no_mask, mask_one, mask_two, mask_three, same_mask. Note: this attribute result only displays mask values for 100 base vias.

upper_mask_pattern

A settable string attribute that returns the upper layer mask pattern of the `via_matrix` in a multiple-patterning technology. Valid values are arbitrary and uniform, alternate_columns and alternate_rows. Default is uniform. The uniform means upper mask values in all base vias of the `via_matrix` are the same. When it is alternate_columns or alternate_rows, upper masks of the upper-layer in the base vias alternates in columns or rows accordingly if the `via_def` upper_layer number_of_masks attribute is not less than 2. When it is arbitrary, upper layer mask in the base vias are considered independently. With mask pattern change, please remove the `upper_mask_constraint` then set the `upper_mask_constraint` to make sure the `upper_mask_constraint` is correct.

via_def

A settable `via_def` collection attribute that represents the via def for the `via_matrix`.

via_def_name

A read-only string attribute that returns the via def name for the `via_matrix`.

via_matrix_type

A read-only string attribute that returns the via type for the `via_matrix`.

via_orientation

A settable string attribute (with allowed values: R0 R90 R180 R270 MX MXR90 MY MYR90) that returns the orientation of base vias within the `via_matrix`.

v

x_pitch

A settable distance attribute that returns the horizontal pitch between base vias in this `via_matrix`.

y_pitch

A settable distance attribute that returns the vertical pitch between base vias in this `via_matrix`.

See Also

- [get_via_matrixes](#)
- [get_attribute](#)
- [list_attributes](#)
- [set_attribute](#)

via_region_attributes

Description of the application defined `via_region` attributes.

Description

Attributes are properties assigned to objects such as pins, cells, and nets. They can be read-only or settable. A read-only attribute is informational; you cannot set its value. To modify a settable attribute, use the `set_attribute` command.

To determine the value of an attribute, use the `get_attribute` command. To see a list of all `via_region` attributes available, use the `list_attributes -application -class via_region` command.

Attributes for via_region

This section describes the `via_region` attributes.

area

A read-only area attribute that returns the area value derived from the `bbox` of the `via_region`.

bbox

A read-only rect attribute that returns the via region, meaning the valid area for the center of a target via. The format of a rectangle specification is `{{llx lly} {urx ury}}`, which specifies the lower-left and upper-right corners of the rectangle.

v

bounding_box

A read-only *bounding_box* collection attribute that returns the bounding-box of a via region. The format of the collection specifies lower-left and upper-right corners of the rectangle.

design_name

A read-only string attribute that returns the design of the via region.

full_name

A read-only string attribute that returns the full name of a via region.

groups

A read-only group collection attribute that returns the groups in which this via region is a member.

is_override

A read-only boolean attribute that returns True if this via region is a design library specific override.

is_rectangle

A read-only boolean attribute that returns whether this via region is a rectangle or not.

is_rotate

A settable boolean attribute that returns whether this via region is for rotated target via or not. If it is true, it means the target via will be rotated in the region.

name

A read-only string attribute that returns the name of a via region.

object_class

A read-only string attribute that returns the name of this object class. Returns 'via_region' for via_region objects.

owner

A read-only terminal collection attribute that returns the terminal which the via region belongs to.

parent_block

A read-only block collection attribute that returns the owner block object for the via region.

v

parent_block_cell

A read-only cell collection attribute that represents the parent block for the via region.

parent_cell

A read-only cell collection attribute that returns the owner cell object for the via region.

points

A settable `coord_list` attribute that returns the points of the via region. The format of coordinate list specification is `{{x1 y1} {x2 y2} ...}`, which specifies a list of points defining the boundary of the via region,

type

A read-only string attribute that returns the type for the terminal.

via_def

A settable `via_def` collection attribute that returns the valid via definition for the via region.

See Also

- [get_via_regions](#)
- [get_attribute](#)
- [list_attributes](#)
- [set_attribute](#)

via_rule_attributes

Description of the application defined `via_rule` attributes.

Description

Attributes are properties assigned to objects such as pins, cells, and nets. They can be read-only or settable. A read-only attribute is informational; you cannot set its value. To modify a settable attribute, use the `set_attribute` command.

To determine the value of an attribute, use the `get_attribute` command. To see a list of all `via_rule` attributes available, use the `list_attributes -application -class via_rule` command.

Attributes for via_rule

This section describes the `via_rule` attributes.

v

cut_layer_name_table

A settable name_list attribute that returns the cut layer's names. This is must for definining a via_ladder.

cut_layer_name_table_size

A settable int attribute that returns the size of the cut_layer_name_table. This is must for defining a via_ladder.

cut_max_spacing_in_lower_metal_direction_table

A settable distance_list attribute that returns the cut max spacing in lower metal direction.

cut_name_table

A settable name_list attribute that returns the cut names. This is must for definining a via_ladder.

cut_to_lower_metal_end_of_line_max_spacing_table

A settable distance_list attribute that returns the cut to lower metal end of line max spacing.

cut_to_lower_metal_end_of_line_min_spacing_table

A settable distance_list attribute that returns the cut to lower metal end of line min spacing.

cut_x_min_spacing_table

A settable distance_list attribute that returns the cut x_min spacing.

cut_y_min_spacing_table

A settable distance_list attribute that returns the cut y_min spacing.

em_factor

A settable float attribute that returns the EM factor of via_ladder rule. The value must be positive. This applies only to via_ladder.

for_electro_migration

A settable boolean attribute that indicates if the via_ladder is for electro migration.

for_high_performance

A settable boolean attribute that indicates if the via_ladder is for high performance.

for_stack_via

A settable boolean attribute that indicates if the via_ladder is for stack via.

forbidden_on_pin_extension

A settable boolean attribute that indicates if the via_ladder constructed for this template definition is allowed to use pin metal extension. The default value is false to allow usage of pin metal extension by the constructed via_ladder.

full_name

A read-only string attribute that returns the name of a via rule.

max_num_stagger_tracks_for_top_level_ndr_track_use

A settable int attribute that returns an integer representing the maximum number of stagger tracks for use of top level tracks of via_ladder rule. This is an integer type and its value must be positive. This attribute may be specified only for via_ladder. .ss lower_layer_uses_auto_generated_tracks_table A boolean list attribute for the uses of auto generated tracks for defining legal position of via ladder with respect to lower level layer .ss upper_layer_uses_auto_generated_tracks_table A boolean list attribute for the uses of auto generated tracks or defining legal position of via ladder with respect to upper level layer

max_num_stagger_tracks_table

A settable int_list attribute that returns the maximum number of stagger tracks. Value must be between 0 and 9.

name

A read-only string attribute that returns the name of a via rule.

ndr_reserved_track_mode

A settable string attribute that defines ndr reserved track mode. Values accepted are "top", "all", "any" or "off".

num_cut_rows_table

A settable int_list attribute that returns the number of cut rows. This is must for defining a via_ladder.

num_cuts_per_row_table

A settable int_list attribute that returns the number of cuts per row. This is must for defining a via_ladder.

v

object_class

A read-only string attribute that returns the name of this object class. Returns 'via_rule' for via_rule objects.

owner

A read-only tech, or block collection attribute that returns a collection representing the owner container of a via_rule. The owner may be a block or a tech.

same_color_via_grid_map

A settable int_list attribute that returns the same color via grid map.

same_color_via_grid_map_cols

A settable int attribute that returns the same color via grid map columns.

same_color_via_grid_map_layer_name

A settable string attribute that returns the same color via grid map layer name.

same_color_via_grid_map_rows

A settable int attribute that returns the same color via grid map rows.

self_aligned_via_array_max_num_cuts_table

A settable int_list attribute that returns the self aligned via array maximum num of cuts.

self_aligned_via_array_max_spacing_threshold_table

A settable distance_list attribute that returns the self aligned via array max spacing threshold.

self_aligned_via_cluster_cut_edge_x_upsize_table

A settable distance_list attribute that returns the self aligned via cluster cut edge x_upsize.

self_aligned_via_cluster_cut_edge_y_upsize_table

A settable distance_list attribute that returns the self aligned via cluster cut edge y_upsize.

self_aligned_via_cluster_edge_x_unit_length_table

A settable distance_list attribute that returns the self aligned via cluster edge x_unit length.

v

self_aligned_via_cluster_edge_y_unit_length_table

A settable distance_list attribute that returns the self aligned via cluster edge y_unit length.

self_aligned_via_cluster_max_num_cuts_table

A settable int_list attribute that returns the self aligned via cluster maximum nums of cuts.

self_aligned_via_cluster_non_rect_max_length_table

A settable distance_list attribute that returns the self aligned via cluster non rectangle max length.

self_aligned_via_cluster_non_rect_min_length_table

A settable distance_list attribute that returns the self aligned via cluster non rectangle min length.

self_aligned_via_cluster_ortho_edge_upsize_table

A settable distance_list attribute that returns the self aligned via cluster ortho edge upsize.

self_aligned_via_cluster_rect_min_length_table

A settable distance_list attribute that returns the self aligned via cluster rectangle min length.

self_aligned_via_cluster_rect_width_table

A settable distance_list attribute that returns the self aligned via cluster rectangle width.

self_aligned_via_cluster_sav_edge_upsize_table

A settable distance_list attribute that returns the self aligned via cluster sav edge upsize.

self_aligned_via_cluster_upsize_table

A settable distance_list attribute that returns the self aligned via cluster upsize.

self_aligned_via_cluster_ushape_allowed_table

A settable bool_list attribute that indicates whether the self aligned via cluster ushapes are allowed or not.

self_aligned_via_diag_ortho_edge_upsize_table

A settable distance_list attribute that returns the self aligned via diagonal ortho edge upsize.

v

self_aligned_via_diag_sav_edge_upsize_table

A settable distance_list attribute that returns the self aligned via diagonal sav edge upsize.

self_aligned_via_one_edge_not_allowed

A settable boolean attribute that indicates whether the self aligned via one edge is allowed or not.

self_aligned_via_upper_layer_enc_table

A settable distance_list attribute that returns the self aligned via upper layer enclosure.

self_aligned_via_upper_layer_enc_table_size

A settable int attribute that returns the size of self_aligned_via_upper_layer_enc_table.

self_aligned_via_zigzag_ortho_edge_upsize_table

A settable distance_list attribute that returns the self aligned via zigzag ortho edge upsize.

self_aligned_via_zigzag_sav_edge_upsize_table

A settable distance_list attribute that returns the self aligned via zigzag sav edge upsize.

upper_metal_min_length_table

A settable distance_list attribute that returns the upper metal minimum length.

upsized_via_upper_metal_width_table

A settable distance_list attribute that returns the upsized via upper metal width.

upsized_via_x_edge_table

A settable distance_list attribute that returns the upsized via x_edge.

upsized_via_y_edge_table

A settable distance_list attribute that returns the upsized via y_edge.

See Also

- [get_attribute](#)
- [list_attributes](#)

voltage_area_attributes

Description of the application defined voltage_area attributes.

Description

Attributes are properties assigned to objects such as pins, cells, and nets. They can be read-only or settable. A read-only attribute is informational; you cannot set its value. To modify a settable attribute, use the *set_attribute* command.

To determine the value of an attribute, use the *get_attribute* command. To see a list of all voltage_area attributes available, use the *list_attributes -application -class voltage_area* command.

Attributes for voltage_area

This section describes the voltage_area attributes.

bbox

A read-only rect attribute that returns the bounding-box of a voltage_area. The format of a rectangle specification is {{llx lly} {urx ury}}, which specifies the lower-left and upper-right corners of the rectangle. This is the bounding box of the draw area specified by users when creating the voltage_area.

bounding_box

A read-only bounding_box collection attribute that returns the bounding-box of a voltage_area. The format of the collection specifies lower-left and upper-right corners of the rectangle. This is the bounding box of the draw area specified by users when creating the voltage_area.

cells

A read-only cell collection attribute that returns the leaf cells explicitly or implicitly associated with this voltage_area or its voltage_area_shapes.

color

A read-only int attribute that returns the color index of a voltage_area.

effective_guard_band_boundaries

A read-only poly_list attribute that returns the effective guard_band boundaries of a voltage_area. The effective guard_band boundary is calculated in the same way as in effective_shapes. The only difference is that the region is expanded by the guard_band margins.

effective_rectangles

A read-only poly_list attribute that returns the effective shapes of a voltage_area broken down into rectangles. The format of a rectangle poly_list specification is

v

{ {{llx1 lly1} {urx1 ury1}} {{llx2 lly2} {urx2 ury2}}...}, which specifies a list of lower-left and upper-right corners of rectangles.

effective_shapes

A read-only poly_list attribute that returns the effective shapes of a voltage_area. The effective_shape is calculated by comparing the stacking_order of the voltage_area_shapes of a voltage_area with all other voltage_area_shapes in the design and subtracting the overlapping region from the regions with lower stacking_order, and merging abutting shapes within the same voltage_area. The format of poly_list specification is {{x11 y11} {x12 y12} ...} {{x21 y21} {x22 y22} ...} ..., which specifies a list of points of rectilinear polygons.

full_name

A read-only string attribute that returns the name of a voltage_area.

ground_net

A read-only supply_net collection attribute that returns a single power ground net object.

groups

A read-only group collection attribute that returns the groups the voltage area shape object belongs to.

guard_band_boundaries

A read-only poly_list attribute that returns the region of this voltage_area. The format of a boundary specification is {{{x11 y11} {x12 y12} ...} {{x21 y21} {x22 y22} ...} ...}. This is the boundary of the guard band area covered by the voltage area.

guard_bands

A read-only guard_band_list attribute that returns the list of guard-bands of a voltage_area. The format of a guard_band_list is {{guard_band_x1 guard_band_y1} {guard_band_x2 guard_band_y2}...}, which specify a list of horizontal and vertical spacings from the edges of each the voltage_area_shape in the voltage_area. The guard band is the spacing along the boundary of a voltage_area where cells cannot be placed because of the lack of power supply rails.

guard_bands_overall

A read-only guard_band_list attribute that returns the list of guard-bands of a voltage_area. The format of a guard_band_list is {{guard_band_left guard_band_right guard_band_top guard_band_bottom}...}, which specify a list of horizontal and vertical spacings from the edges of each the

`voltage_area_shape` in the `voltage_area`. The guard band is the spacing along the boundary of a `voltage_area` where cells cannot be placed because of the lack of power supply rails.

in_edit_group

A read-only boolean attribute that indicates whether a `voltage_area` belongs to an `edit_group`.

interface_connectivity_planning

A settable string attribute (with allowed values: `false` `hard` `soft` `unset`) that returns whether to leave some space along edges of voltage area shapes free of macros during automatic macro placement. It works together with the app option `plan_macro_cross_connectivity_planning`; see that option's man page for more details. The default is `unset`.

is_fixed

A settable boolean attribute that indicates whether a `voltage_area` is in a fixed location. It can only be set to `true` if the `voltage_area` contains at least one shape.

is_fully_specified

A read-only boolean attribute that indicates whether the flat `voltage_area` is fully specified. The flat voltage area is not fully specified if the set of instances is associated with multiple set of regions of multiple sub-blocks. Otherwise, if the association is simple, it is `true`.

is_legal

A read-only boolean attribute that indicates whether a `voltage_area` is legal. A legal `voltage_area` is one that is associated with one or more power domain and whose power and group supply, if defined, match the primary power and group supplies of the associated power domains.

is_primary

A read-only boolean attribute that indicates whether this is the primary `voltage_area` of the associated power_domains.

is_shadow

A read-only boolean attribute that indicates whether a `voltage_area` is pushed down from above.

local_cells

A read-only cell collection attribute that returns the leaf cells explicitly associated with this `voltage_area`.

v

name

A read-only string attribute that returns the name of a voltage_area.

object_class

A read-only string attribute that returns the name of this object class. Returns 'voltage_area' for voltage_area objects.

parent_block

A read-only block collection attribute that returns a collection for the parent block of the voltage area.

parent_block_cell

A read-only cell collection attribute that returns a collection for the instance corresponding to the parent block of the voltage area.

parent_cell

A read-only cell collection attribute that returns a collection for the instance corresponding to the root hierarchy of the voltage area if it exists.

physical_status

A settable string attribute (with allowed values: locked unrestricted) that returns the physical modification status of the voltage_area. If status is unrestricted, the voltage_area may be freely modified. If status is locked, the voltage_area is fixed for automatic and manual changes.

power_domains

A read-only power_domain collection attribute that returns the power_domain objects for this voltage_area.

power_net

A read-only supply_net collection attribute that returns a single power supply net object.

region

A settable poly_list attribute that returns the region of this voltage_area shape. The format of a boundary specification is {{{x11 y11} {x12 y12} ...} {{x21 y21} {x22 y22} ...} ...}. This is the boundary of the draw area specified by users when creating the voltage_area. It is a settable attribute and changing its value can cause changes in other voltage_areas and voltage_area_shapes in the block.

v

shadow_status

A settable string attribute (with allowed values: copied_down copied_up normal pulled_up pushed_down virtual_flat) that returns the shadow status of a voltage_area.

shape_count

A read-only int attribute that returns the number of shapes in a voltage_area.

shapes

A read-only voltage_area_shape collection attribute that contains the voltage_area_shape objects in the voltage_area.

shaping_constraints

A read-only shaping_constraint collection attribute that returns the set of shaping_constraint objects defined for this voltage area.

signature

A settable string attribute that pairs up a voltage area with its associated power domain when the voltage area's name is different than its power domain.

target_utilization

A settable float attribute that returns the utilization of the voltage_area. Valid values are between 0.0 to 1.0.

top_block

A read-only block collection attribute that returns a collection of the top block for the corresponding voltage area.

use_power_domain_cells

A settable boolean attribute that indicates whether a voltage_area gets its cell membership implicitly from its associated power domains.

voltage_area_rules

A read-only voltage_area_rule collection attribute that returns the voltage_area_rules applying to this voltage_area. This collection contains the default rule if it exists and if this voltage_area is not explicitly assigned any rules.

See Also

- [create_voltage_area](#)
- [get_attribute](#)

v

- [list_attributes](#)
- [set_attribute](#)

voltage_area_rule_attributes

Description of the application defined voltage_area_rule attributes.

Description

Attributes are properties assigned to objects such as pins, cells, and nets. They can be read-only or settable. A read-only attribute is informational; you cannot set its value. To modify a settable attribute, use the *set_attribute* command.

To determine the value of an attribute, use the *get_attribute* command. To see a list of all voltage_area_rule attributes available, use the *list_attributes -application -class voltage_area_rule* command.

Attributes for voltage_area_rule

This section describes the voltage_area_rule attributes.

excluded_logical_feedthrough_hierarchy

A settable cell collection attribute that contains the cells explicitly excluded from logical feedthrough buffer insertion.

Note that only *included_logical_feedthrough_hierarchy* or *excluded_logical_feedthrough_hierarchy* may be set.

full_name

A read-only string attribute that returns the name of the voltage_area_rule.

included_logical_feedthrough_hierarchy

A settable cell collection attribute that contains the cells explicitly included in logical feedthrough buffer insertion.

Note that only *included_logical_feedthrough_hierarchy* or *excluded_logical_feedthrough_hierarchy* may be set.

is_buffering_allowed

A settable boolean attribute that indicates whether or not buffering is allowed through the voltage areas.

is_default_rule

A read-only boolean attribute that returns whether or not this rule is the default rule. The default rule applies to all voltage areas which are not assigned explicitly to a rule.

v

is_logical_feedthrough_allowed

A settable boolean attribute that indicates whether or not logical feedthrough buffering is allowed through the voltage areas.

is_new_cell_allowed

A settable boolean attribute that indicates whether or not new cells are allowed in the voltage areas. This attribute is settable, except on the default rule. This attribute must be true if buffering is allowed through the voltage areas. If this attribute is set to false, all buffering will be disallowed through the voltage areas.

is_pass_through_allowed

A settable boolean attribute that indicates whether or not nets may pass through the voltage areas.

is_physical_feedthrough_allowed

A settable boolean attribute that indicates whether or not physical feedthrough buffering is allowed through the voltage areas.

is_top_logical_feedthrough_excluded

A settable boolean attribute that indicates whether the top hierarchy is explicitly excluded from logical feedthrough buffer insertion. This attribute is settable. Note that only *is_top_logical_feedthrough_included* or *is_top_logical_feedthrough_excluded* may be set.

is_top_logical_feedthrough_included

A settable boolean attribute that indicates whether the top hierarchy is explicitly included in logical feedthrough buffer insertion. This attribute is settable. Note that only *is_top_logical_feedthrough_included* or *is_top_logical_feedthrough_excluded* may be set.

name

A settable string attribute that returns the name of the voltage_area_rule.

object_class

A read-only string attribute that returns the name of this object class. Returns 'voltage_area_rule' for voltage_area_rule objects.

voltage_areas

A read-only voltage_area collection attribute that returns the voltage_areas which are explicitly assigned to this rule.

See Also

- [create_voltage_area_rule](#)
- [get_attribute](#)
- [list_attributes](#)
- [set_attribute](#)

voltage_area_shape_attributes

Description of the application defined voltage_area_shape attributes.

Description

Attributes are properties assigned to objects such as pins, cells, and nets. They can be read-only or settable. A read-only attribute is informational; you cannot set its value. To modify a settable attribute, use the *set_attribute* command.

To determine the value of an attribute, use the *get_attribute* command. To see a list of all voltage_area_shape attributes available, use the *list_attributes -application -class voltage_area_shape* command.

Attributes for voltage_area_shape

This section describes the voltage_area_shape attributes.

bbox

A read-only rect attribute that returns the bounding box of the voltage area shape, including its guard band. The format of a rectangle specification is `{{llx lly} {urx ury}}`, which specifies the lower-left and upper-right corners of the rectangle.

boundary

A settable polygon attribute that returns the user-specified coordinates of the voltage area shape. Unlike the *bbox* attribute, this attribute does not include the guard band of the voltage area shape. The format of a boundary specification is `{{{x11 y11} {x12 y12} ...} {{x21 y21} {x22 y22} ...} ...}`, which specifies a list of coordinates of rectilinear polygons. Note that changing this attribute value can cause changes to other voltage areas and voltage area shapes in the block.

bounding_box

A read-only bounding_box collection attribute that returns the bounding box of the voltage area shape, including its guard band. The format of the collection specifies the lower-left and upper-right corners of the rectangle.

v

cells

A read-only cell collection attribute that returns the leaf cells explicitly associated with the voltage area shape.

effective_guard_band_boundaries

A read-only poly_list attribute that returns the effective boundary of the voltage area shape including its guard band. The effective guard band boundary is calculated in the same way as for the *effective_shapes* attribute. The only difference is that the shape is expanded by the x- and y-guard bands during the calculation.

effective_rectangles

A read-only poly_list attribute that returns the effective boundary of the voltage area shape as a list of rectangles. The format of the specification is { {{llx1 lly1} {urx1 ury1}} {{llx2 lly2} {urx2 ury2}}...}, which specifies a list of lower-left and upper-right corners of rectangles.

effective_shapes

A read-only poly_list attribute that returns the effective boundary of the voltage area shape. The effective_shape is calculated by comparing the stacking_order of the voltage area shapes of a voltage area with all other voltage area shapes in the block, subtracting the overlapping region from the regions with lower stacking order, and merging abutting shapes within the same voltage area. The format of the specification is {{{x11 y11} {x12 y12} ...} {{x21 y21} {x22 y22} ...} ...}, which specifies a list of coordinates of rectilinear polygons.

full_name

A read-only string attribute that returns the name of the voltage area shape.

groups

A read-only group collection attribute that returns the groups the voltage area shape object belongs to.

guard_band

A settable guard_band attribute that returns the horizontal and vertical guard bands of the voltage area shape. The format of a guard band specification is {guard band_x guard band_y}, which specifies the horizontal and vertical margins from the edges of the voltage area shape.

guard_band_boundary

A read-only polygon attribute that represents the boundary of the voltage area shape including its guard-band margins. The format of the boundary

v

specification is $\{\{x_{11} y_{11}\} \{x_{12} y_{12}\} \dots\} \{\{x_{21} y_{21}\} \{x_{22} y_{22}\} \dots\} \dots\}$, which specifies a list of coordinates of rectilinear polygons.

guard_band_overall

A settable guard_band attribute that returns the left, right, top, and bottom guard bands of the voltage area shape. The format of a guard band specification is {guard band_x guard band_y} or {gb_left, gb_right, gb_top, gb_bottom} which specifies the horizontal and vertical or all four side margins from the edges of the voltage area shape.

is_exclusive

A settable boolean attribute that indicates whether the voltage area shape should be treated as an exclusive area with respect to cells not associated with this shape (but are associated with its voltage area).

is_legal

A read-only boolean attribute that returns whether the voltage area shape is legal.

is_secondary_pg_region

A read-only boolean attribute that returns whether the voltage area shape is a secondary pg region.

name

A read-only string attribute that returns the name of the voltage area shape.

object_class

A read-only string attribute that returns the name of this object class. Returns 'voltage_area_shape' for voltage_area_shape objects.

parent_block

A read-only block collection attribute that returns the owner block for the voltage area shape.

parent_block_cell

A read-only cell collection attribute that returns the parent block cell for the voltage area shape.

parent_cell

A read-only cell collection attribute that returns the owner cell for the voltage area shape.

v

secondary_pg_nets

A read-only `supply_net` collection attribute that returns the secondary supply nets available on the voltage area shape.

stacking_order

A read-only int attribute that returns the number of voltage area shapes above the bottom of the voltage area shape position stack.

switch_cell_placement_type

A read-only string attribute (with allowed values: `array` `hybrid` `invalid` `pattern` `ring`) that returns the placement type of the switch cells in this voltage area shape. Valid values are *array*, *hybrid*, *invalid*, *pattern*, and *ring*.

target_utilization

A settable float attribute that returns the utilization of the voltage area shape. Valid values are between 0.0 and 1.0, inclusive.

top_block

A read-only block collection attribute that returns the top-level block for the voltage area shape.

voltage_area

A read-only `voltage_area` collection attribute that returns the voltage area associated with the voltage area shape.

voltage_area_name

A read-only string attribute that returns the name of the voltage area associated with the voltage area shape.

See Also

- [create_voltage_area_shape](#)
- [get_attribute](#)
- [list_attributes](#)
- [set_attribute](#)